
SPEKTRUM LEHRBUCH

T. Ottmann / P. Widmayer

Algorithmen und Datenstrukturen



3. Auflage

Spektrum
www.spektrum.de

SPEKTRUM LEHRBUCH

T. Ottmann / P. Widmayer

Algorithmen und Datenstrukturen



3. Auflage

Spektrum
AKADEMISCHER VERLAG

Inhaltsübersicht

Grundlagen	1
Sortieren	63
Suchen	147
Hashverfahren	169
Bäume	235
Manipulation von Mengen	377
Geometrische Algorithmen	419
Graphenalgorithmen	535
Ausgewählte Themen	617

Vorwort zur elektronischen Version

Die 3. Auflage unseres Lehrbuchs über Algorithmen und Datenstrukturen liegt nun auch in einer elektronischen Version vor. Bei der Konvertierung standen zwei Ziele im Vordergrund: Zum einen sollte der Buchtext layouterhaltend umgesetzt werden; zum anderen war uns die leichte Orientierung und Navigierbarkeit im elektronischen Dokument wichtig. Verschiedene andere Gründe führten zu der Entscheidung, PDF als Zielformat auszuwählen. Da keine Änderungen im Manuskript erfolgt sind, entspricht die elektronische Version genau der gebundenen Version. Wir hoffen, daß hiermit eine gelungene Ergänzung zum Lehrbuch entstanden ist.

Die vorliegende elektronische Version wird vorerst über dem FIZ-Server gegen entsprechende Lizenzgebühren bereitgestellt. Zusätzlich sind wir bemüht, diese Version ständig zu verbessern und — zumindest für die Benutzer in Freiburg — durch multimediale Anteile zu erweitern. Sie sollen jedoch zu einem späteren Zeitpunkt auch über den Server des Verlags zugänglich gemacht werden. Für Anregungen, Korrekturen und andere Hinweise jeder Art möchten wir Sie, liebe Leser, auch an dieser Stelle ausdrücklich bitten, damit wir diese sowohl für die elektronische Fassung als auch für die nächste Auflage der gebundenen Version dieses Lehrbuchs berücksichtigen können.

Schließlich möchten wir uns bei M. Will bedanken, der die Konvertierung nach PDF durchgeführt hat.

Vorwort zur 3. Auflage

In dieser nun vorliegenden 3. Auflage unseres Lehrbuches über Algorithmen und Datenstrukturen haben wir alle Hinweise auf Fehler und zahlreiche Verbesserungsvorschläge unserer Leser berücksichtigt. Dafür möchten wir uns ausdrücklich bedanken bei A. Brinkmann, S. Hanke, R. Hipke, W. Kuhn, R. Ostermann, D. Saupe, K. Simon, R. Typke, F. Widmer. Größere inhaltliche Änderungen wurden in den Abschnitten 5.6 und in den Abschnitten 6.1.1, 6.2.1, 8.5.1 und 8.6 vorgenommen. Die neugeschriebenen Teile des Manuskripts wurden von E. Patschke erfaßt. St. Schrödl hat die mühevollen Aufgabe übernommen, das Layout an das für den Spektrum Verlag typische Format anzupassen und hat auch die Herstellung der Druckvorlage überwacht.

Geändert hat sich seit der letzten Auflage nicht nur der Verlag unseres Lehrbuches, sondern das gesamte Umfeld für die Publikation von Büchern. Wir haben daher damit begonnen, multimediale Ergänzungen zu unserem Lehrbuch zu sammeln und auf dem Server des Verlags abzulegen. Wir möchten also Sie, liebe Leser, nicht nur weiterhin um Ihre Wünsche, Anregungen und Hinweise zur hier vorliegenden papiergebundenen Version unseres Lehrbuches bitten, sondern ausdrücklich auch um Anregungen für multimediale Ergänzungen aller Art.

Freiburg und Zürich
im September 1996

Thomas Ottmann
Peter Widmayer

Vorwort zur zweiten Auflage

In den gut zwei Jahren, die seit dem Erscheinen unseres Lehrbuches über Algorithmen und Datenstrukturen vergangen sind, haben wir von vielen Lesern Hinweise auf Fehler im Text und Wünsche für Verbesserungen und Ergänzungen erhalten. Wir haben in der nun vorliegenden zweiten Auflage alle bekanntgewordenen Fehler korrigiert, einige Abschnitte überarbeitet und den behandelten Stoff um einige aktuelle Themen ergänzt.

In dieser wie auch schon in der vorigen Auflage wurden alle Bäume mit Hilfe des Makropaketes `TreeTeX` von Anne Brüggemann-Klein und Derick Wood erstellt. Korrekturhinweise kamen von Bruno Becker, Stephan Gschwind, Ralf Hartmut Güting, Andreas Hutflesz, Brigitte Kröll, Thomas Lengauer und Mitarbeitern, Otto Nurmi, Klaus Simon, Ulrike Stege und Alexander Wolff. Das Manuskript für die neuen Abschnitte wurde von Frau Christine Kury erfaßt und in `LATEX` gesetzt. Frau Dr. Gabriele Reich hat das Manuskript noch einmal durchgesehen, inhaltliche und formale Fehler beseitigt und die Herstellung der Druckvorlage überwacht. Ihnen gebührt unser besonderer Dank.

Wir sind uns bewußt, daß auch diese zweite Auflage noch in vieler Hinsicht verbessert werden kann. Wir richten also an Sie, unsere Leser, die Bitte, uns auch weiter Wünsche, Anregungen, Hinweise und entdeckte Fehler mitzuteilen.

Vorwort

Im Zentrum des Interesses der Informatik hat sich in den letzten Jahren das Gebiet *Algorithmen und Datenstrukturen* beachtlich entwickelt. Dabei geht es sowohl um den *Entwurf* effizienter Algorithmen und Datenstrukturen als auch um die *Analyse* ihres Verhaltens. Die erzielten Fortschritte und die behandelten Probleme lassen erwarten, daß Algorithmen und Datenstrukturen noch lange Zeit Gegenstand intensiver Forschung bleiben werden.

Mit diesem Buch wenden wir uns in erster Linie an Studenten im Grundstudium. Wir haben uns bemüht, alle zum Grundwissen über Algorithmen und Datenstrukturen gehörenden Themen präzise, aber nicht allzu formal zu behandeln. Die Kenntnis einer Programmiersprache, etwa Pascal, und elementare mathematische Fertigkeiten sollten als Voraussetzungen zum Verständnis des Stoffs genügen. Die gestellten Übungsaufgaben dienen fast ausschließlich der Festigung erworbener Kenntnisse; offene Forschungsprobleme sind nicht aufgeführt.

An vielen Stellen und zu einigen Themen haben wir exemplarisch, unseren Neigungen folgend, deutlich mehr als nur Grundkonzepte dargestellt. Dabei ist es aber nicht unser Ziel, den aktuellen Stand des Gebiets erschöpfend abzuhandeln.

Man kann das Gebiet *Algorithmen und Datenstrukturen* auf verschiedene Arten gliedern: nach den *Algorithmen* oder Problembereichen, nach den *Datenstrukturen* oder Werkzeugen und nach den *Entwurfsprinzipien* oder Methoden. Wir haben die Gliederung des Stoffs nach einem einzelnen dieser drei Kriterien nicht erzwungen; stattdessen haben wir eine Mischung der Kriterien verwendet, weil uns dies natürlicher erscheint.

Dieses Buch ist hervorgegangen aus Vorlesungen, die wir über viele Jahre an den Universitäten Karlsruhe und Freiburg gehalten haben. Gleichzeitig mit diesem Buch haben wir Computerkurse über *Algorithmen und Datenstrukturen* angefertigt und in der universitären Lehre eingesetzt; die dafür notwendige Beschäftigung mit Fragen der Didaktik hat dieses Buch sicherlich beeinflusst.

Eine große Zahl von Personen, insbesondere Studenten, Mitarbeiter und Kollegen, hat Anteil am Zustandekommen dieses Buches; ihnen allen gebührt unser Dank.

Brunhilde Beck, Trudi Halboth, Christine Krause, Ma Li-Hong und Margit Stanzel haben das Manuskript hergestellt; die Fertigstellung der Druckvorlage besorgte Christine Krause. Insbesondere — aber nicht nur — das Einbinden von Abbildungen hat dabei so große $\text{\TeX}$ nische Schwierigkeiten bereitet, daß wir öfter die Expertise von Anne Brüggemann-Klein, Gabriele Reich und Sven Schuierer in Anspruch nehmen muß-

ten. Bruno Becker, Alois Heinz, Thomas Ohler, Rainer Schielin und Jörg Winckler haben dafür gesorgt, daß die verschiedenen elektronischen Versionen des Manuskripts in einem heterogenen Rechnernetz stets verfügbar waren. Die Universitäten Freiburg, Karlsruhe und Waterloo haben die technische Infrastruktur bereitgestellt.

Gabriele Reich hat das gesamte Manuskript und Anne Brüggemann-Klein, Christiaan Icking, Ursula Schmidt, Eljas Soisalon-Soininen und Lutz Wegner haben Teile des Manuskripts gelesen und kommentiert. Natürlich gehen alle verbliebenen Fehler ganz zu unseren Lasten. Weitere Hinweise und Anregungen stammen von Joachim Geidel, Andreas Hutflesz, Rolf Klein, Tilman Kühn, Hermann Maurer, Ulf Metzler, Heinrich Müller, Jörg Sack, Anno Schneider, Sven Schuierer, Hans-Werner Six, Stephan Voit und Derick Wood.

Dem B.I.-Wissenschaftsverlag danken wir für die große Geduld, die er uns entgegenbrachte. An Sie, unsere Leser, richten wir schließlich die Bitte, uns Wünsche, Anregungen, Hinweise oder einfach entdeckte Fehler mitzuteilen.

Inhaltsverzeichnis

1	Grundlagen	1
1.1	Algorithmen und ihre formalen Eigenschaften	1
1.2	Zwei Beispiele arithmetischer Algorithmen	5
1.2.1	Ein Multiplikationsverfahren	5
1.2.2	Polynomprodukt	8
1.3	Verschiedene Algorithmen für dasselbe Problem	12
1.4	Die richtige Wahl einer Datenstruktur	15
1.5	Lineare Listen	21
1.5.1	Sequentielle Speicherung linearer Listen	22
1.5.2	Verkettete Speicherung linearer Listen	25
1.5.3	Stapel und Schlangen	33
1.6	Ausblick auf weitere Datenstrukturen	40
1.7	Skip-Listen	42
1.7.1	Perfekte und randomisierte Skip-Listen	43
1.7.2	Analyse	49
1.8	Aufgaben	52
2	Sortieren	63
2.1	Elementare Sortierverfahren	66
2.1.1	Sortieren durch Auswahl	66
2.1.2	Sortieren durch Einfügen	69
2.1.3	Shellsort	71
2.1.4	Bubblesort	73
2.2	Quicksort	76
2.2.1	Quicksort: Sortieren durch rekursives Teilen	77
2.2.2	Quicksort-Varianten	85
2.3	Heapsort	89
2.4	Mergesort	96
2.4.1	2-Wege-Mergesort	96
2.4.2	Reines 2-Wege-Mergesort	100
2.4.3	Natürliches 2-Wege-Mergesort	102
2.5	Radixsort	105
2.5.1	Radix-exchange-sort	105

2.5.2	Sortieren durch Fachverteilung	107
2.6	Sortieren vorsortierter Daten	111
2.6.1	Maße für Vorsortierung	112
2.6.2	A-sort	116
2.6.3	Sortieren durch lokales Einfügen und natürliches Verschmelzen	121
2.7	Externes Sortieren	126
2.7.1	Das Magnetband als Externspeichermedium	126
2.7.2	Ausgeglichenes 2-Wege-Mergesort	128
2.7.3	Ausgeglichenes Mehr-Wege-Mergesort	132
2.7.4	Mehrphasen-Mergesort	135
2.8	Untere Schranken	138
2.9	Aufgaben	143
3	Suchen	147
3.1	Das Auswahlproblem	148
3.2	Suchen in sequentiell gespeicherten linearen Listen	153
3.2.1	Sequentielle Suche	153
3.2.2	Binäre Suche	154
3.2.3	Fibonacci-Suche	156
3.2.4	Exponentielle Suche	159
3.2.5	Interpolationssuche	160
3.3	Selbstanordnende lineare Listen	160
3.4	Aufgaben	166
4	Hashverfahren	169
4.1	Zur Wahl der Hashfunktion	171
4.1.1	Die Divisions-Rest-Methode	171
4.1.2	Die multiplikative Methode	172
4.1.3	Perfektes und universelles Hashing	173
4.2	Hashverfahren mit Verkettung der Überläufer	176
4.3	Offene Hashverfahren	181
4.3.1	Lineares Sondieren	184
4.3.2	Quadratisches Sondieren	186
4.3.3	Uniformes und zufälliges Sondieren	187
4.3.4	Double Hashing	190
4.3.5	Ordered Hashing	194
4.3.6	Robin-Hood-Hashing	199
4.3.7	Coalesced Hashing	200
4.4	Dynamische Hashverfahren	204
4.4.1	Lineares Hashing	206
4.4.2	Virtuelles Hashing	211
4.4.3	Erweiterbares Hashing	215
4.5	Das Gridfile	219
4.6	Aufgaben	229

5	Bäume	235
5.1	Natürliche Bäume	239
5.1.1	Suchen, Einfügen und Entfernen von Schlüsseln	242
5.1.2	Durchlaufordnungen in Binärbäumen	248
5.1.3	Analytische Betrachtungen	251
5.2	Balancierte Binärbäume	260
5.2.1	AVL-Bäume	260
5.2.2	Bruder-Bäume	273
5.2.3	Gewichtsbalancierte Bäume	289
5.3	Randomisierte Suchbäume	296
5.3.1	Treaps	296
5.3.2	Treaps mit zufälligen Prioritäten	300
5.4	Selbstanordnende Binärbäume	304
5.4.1	Splay-Bäume	305
5.4.2	Amortisierte Worst-case-Analyse	310
5.5	B-Bäume	317
5.5.1	Suchen, Einfügen und Entfernen in B-Bäumen	322
5.6	Weitere Klassen	327
5.6.1	Übersicht	327
5.6.2	Konstante Umstrukturierungskosten und relaxiertes Balancieren	333
5.6.3	Eindeutig repräsentierte Wörterbücher	350
5.7	Optimale Suchbäume	356
5.8	Alphabetische und mehrdimensionale Suchbäume	362
5.8.1	Tries	363
5.8.2	Quadranten- und 2d-Bäume	364
5.9	Aufgaben	368
6	Manipulation von Mengen	377
6.1	Vorrangwarteschlangen	378
6.1.1	Dijkstras Algorithmus zur Berechnung kürzester Wege	379
6.1.2	Implementation von Priority Queues mit verketteten Listen und balancierten Bäumen	382
6.1.3	Linksbäume	384
6.1.4	Binomial Queues	387
6.1.5	Fibonacci-Heaps	394
6.2	Union-Find-Strukturen	402
6.2.1	Kruskals Verfahren zur Berechnung minimaler spannender Bäume	403
6.2.2	Vereinigung nach Größe und Höhe	406
6.2.3	Methoden der Pfadverkürzung	410
6.3	Allgemeiner Rahmen	413
6.4	Aufgaben	417

7	Geometrische Algorithmen	419
7.1	Einleitung	419
7.2	Das Scan-line-Prinzip	420
7.2.1	Sichtbarkeitsproblem	422
7.2.2	Das Schnittproblem für iso-orientierte Liniensegmente	425
7.2.3	Das allgemeine Liniensegment-Schnittproblem	428
7.3	Geometrisches Divide-and-conquer	435
7.3.1	Segmentschnitt mittels Divide-and-conquer	435
7.3.2	Inklusions- und Schnittprobleme für Rechtecke	441
7.4	Geometrische Datenstrukturen	444
7.4.1	Reduktion des Rechteckschnittproblems	445
7.4.2	Segment-Bäume	448
7.4.3	Intervall-Bäume	454
7.4.4	Prioritäts-Suchbäume	457
7.5	Das Zickzack-Paradigma	471
7.6	Anwendungen geometrischer Datenstrukturen	485
7.6.1	Ein Spezialfall des HLE-Problems	485
7.6.2	Dynamische Bereichssuche mit einem festen Fenster	492
7.7	Distanzprobleme und ihre Lösung	496
7.7.1	Distanzprobleme	497
7.7.2	Das Voronoi-Diagramm	501
7.7.3	Die Speicherung des Voronoi-Diagramms	505
7.7.4	Die Konstruktion des Voronoi-Diagramms	507
7.7.5	Lösungen für Distanzprobleme	516
7.8	Aufgaben	525
8	Graphenalgorithmen	535
8.1	Topologische Sortierung	543
8.2	Transitive Hülle	546
8.2.1	Transitive Hülle allgemein	547
8.2.2	Transitive Hülle für azyklische Digraphen	548
8.3	Durchlaufen von Graphen	551
8.3.1	Einfache Zusammenhangskomponenten	553
8.3.2	Strukturinformation durch Tiefensuche	554
8.4	Zusammenhangskomponenten	557
8.4.1	Zweifache Zusammenhangskomponenten	559
8.4.2	Starke Zusammenhangskomponenten	561
8.5	Kürzeste Wege	566
8.5.1	Kürzeste Wege in Distanzgraphen	567
8.5.2	Kürzeste Wege in beliebig bewerteten Graphen	572
8.5.3	Alle kürzesten Wege	576
8.6	Minimale spannende Bäume	578
8.7	Flüsse in Netzwerken	584
8.8	Zuordnungsprobleme	596
8.8.1	Maximale Zuordnungen in bipartiten Graphen	598
8.8.2	Maximale Zuordnungen im allgemeinen Fall	601
8.8.3	Maximale gewichtete Zuordnungen	609

Inhaltsverzeichnis	XVII
8.9 Aufgaben	610
9 Ausgewählte Themen	617
9.1 Suchen in Texten	617
9.1.1 Das naive Verfahren zur Textsuche	618
9.1.2 Das Verfahren von Knuth-Morris-Pratt	619
9.1.3 Das Verfahren von Boyer-Moore	624
9.1.4 Signaturen	630
9.1.5 Approximative Zeichenkettensuche	631
9.2 Parallele Algorithmen	643
9.2.1 Einfache Beispiele paralleler Algorithmen	645
9.2.2 Paralleles Mischen und Sortieren	651
9.2.3 Systolische Algorithmen	662
9.3 Aufgaben	666
Literaturverzeichnis	671
Index	683

Kapitel 1

Grundlagen

1.1 Algorithmen und ihre formalen Eigenschaften

In der Informatik unterscheidet man üblicherweise zwischen Verfahren zur Lösung von Problemen und ihrer Implementation in einer bestimmten Programmiersprache auf bestimmten Rechnern. Man nennt die Verfahren *Algorithmen*. Sie sind *das* zentrale Thema der Informatik. Die Entwicklung und Untersuchung von Algorithmen zur Lösung vielfältiger Probleme gehört zu den wichtigsten Aufgaben der Informatik. Die meisten Algorithmen erfordern jeweils geeignete Methoden zur Strukturierung der von den Algorithmen manipulierten Daten. Algorithmen und Datenstrukturen gehören also zusammen. Die richtige Wahl von Algorithmen und Datenstrukturen ist ein wichtiger Schritt zur Lösung eines Problems mit Hilfe von Computern. Thema dieses Buches ist das systematische Studium von Algorithmen und Datenstrukturen aus vielen Anwendungsbereichen. Bevor wir damit beginnen, wollen wir einige grundsätzliche Überlegungen zum Algorithmenbegriff vorausschicken.

Was ist ein Algorithmus?

Dies ist eine eher philosophische Frage, auf die wir in diesem Buch keine präzise Antwort geben werden. Das ist glücklicherweise auch nicht nötig. Wir werden nämlich in diesem Buch (nahezu) ausschließlich positive Aussagen über die Existenz von Algorithmen durch explizite Angabe solcher Algorithmen machen. Dazu genügt ein intuitives Verständnis des Algorithmenbegriffs und die Einsicht, daß sich die konkret angegebenen Algorithmen etwa in einer höheren Programmiersprache wie Pascal formulieren lassen. Erst wenn man eine Aussage der Art „Es gibt *keinen* Algorithmus, der dieses Problem löst“ beweisen will, benötigt man eine präzise formale Fassung des Algorithmenbegriffs. Sie hat ihren Niederschlag in der bereits 1936 aufgestellten Church'schen These gefunden, in der Algorithmen mit den auf bestimmten Maschinen, zum Beispiel auf sogenannten Turing-Maschinen, ausführbaren Programmen identifiziert werden. Das Studium des formalisierten Algorithmenbegriffs ist aber nicht das Thema dieses Buches.

Wie teilt man Algorithmen mit?

Das ist die Frage nach der sprachlichen Formulierung von Algorithmen. Wir legen Wert darauf, die Mitteilung oder Formulierung von Algorithmen deutlich von ihrer Realisierung durch ein Programm, durch einen Schaltkreis, eine mechanische Vorrichtung usw. zu trennen. Algorithmen haben eine davon unabhängige Existenz und können durchaus auf sehr verschiedene Arten mitgeteilt werden. Wir werden meistens die deutsche Umgangssprache und eine um umgangssprachliche Mittel erweiterte Pascal-ähnliche Programmiersprache benutzen.

Obwohl wir uns stets bemühen, dem Prinzip der Entwicklung von Algorithmen durch schrittweise Verfeinerung zu folgen und gut strukturierte und dokumentierte Programme, d. h. Formulierungen von Algorithmen, anzugeben, ist die Programmiermethodik, mit der man das erreicht, ebenfalls nicht Gegenstand dieses Buches.

Welche formalen Eigenschaften von Algorithmen werden studiert?

Die wichtigste formale Eigenschaft eines Algorithmus ist zweifellos dessen Korrektheit. Dazu muß gezeigt werden, daß der Algorithmus die jeweils gestellte Aufgabe richtig löst. Man kann die Korrektheit eines Algorithmus im allgemeinen nicht durch Testen an ausgewählten Beispielen nachweisen. Denn — dies hat E. Dijkstra in einem berühmt gewordenen Satz bemerkt — man kann durch Testen zwar die Anwesenheit, nicht aber die Abwesenheit von Fehlern, also die Korrektheit eines Programmes, zeigen. Präzise oder gar voll formalisierte Korrektheitsbeweise verlangen, daß auch das durch einen Algorithmus zu lösende Problem vollständig und präzise spezifiziert ist. Da wir uns in der Regel mit einer recht informellen, inhaltlichen Problembeschreibung begnügen, verzichten wir auch auf umfangreiche, formale Korrektheitsbeweise. Wo aber die Korrektheit eines Algorithmus nicht unmittelbar offensichtlich ist, geben wir durch Kommentare, Angabe von Schleifen- und Prozedurinvarianten und andere Hinweise im Text ausreichende Hilfen, auf die der Leser selbst formalisierte Korrektheitsbeweise gründen kann.

Die zweite wichtige formale Eigenschaft eines Algorithmus, für die wir uns in diesem Buch interessieren, ist seine Effizienz. Die weitaus wichtigsten Maße für die Effizienz sind der zur Ausführung des Algorithmus benötigte Speicherplatz und die benötigte Rechenzeit. Man könnte beides durch Implementation des Algorithmus in einer konkreten Programmiersprache auf einem konkreten Rechner für eine Menge repräsentativ gewählter Eingaben messen. Solche experimentell ermittelten Meßergebnisse lassen sich aber nicht oder nur schwer auf andere Implementationen und andere Rechner übertragen.

Aus dieser Schwierigkeit bieten sich zwei mögliche Auswege an. Erstens kann man einen idealisierten Modellrechner als Referenzmaschine benutzen und die auf diesem Rechner zur Ausführung des Algorithmus benötigte Zeit und den benötigten Speicherplatz messen. Ein in der Literatur zu diesem Zweck sehr häufig benutztes Maschinenmodell ist das der (real) RAM (Random-Access-Maschine, gegebenenfalls mit Real-Zahl-Arithmetik). Eine solche Maschine verfügt über einige Register und eine abzählbar unendliche Menge einzeln adressierbarer Speicherzellen. Die Register und Speicherzellen können je eine im Prinzip unbeschränkt große ganze (oder gar reelle) Zahl aufnehmen. Das Befehlsrepertoire für eine RAM ähnelt einfachen, herkömmlichen Assemblersprachen. Neben Transportbefehlen zum Laden von und Speichern in direkt

und indirekt adressierten Speicherzellen gibt es arithmetische Befehle zur Verknüpfung zweier Registerinhalte mit den üblichen für ganze (oder reelle) Zahlen erklärten Operationen sowie bedingte und unbedingte Sprungbefehle. Die Kostenmaße Speicherplatz und Laufzeit erhalten dann folgende Bedeutung: Der von einem Algorithmus benötigte Platz ist die Anzahl der zur Ausführung benötigten RAM-Speicherzellen; die benötigte Zeit ist die Zahl der ausgeführten RAM-Befehle.

Natürlich ist die Annahme, daß Register und Speicherzellen eine im Prinzip unbeschränkt große ganze oder gar reelle Zahl enthalten können, eine idealisierte Annahme, über deren Berechtigung man in jedem Einzelfall erneut nachdenken sollte. Sofern die in einem Problem auftretenden Daten, wie etwa die zu sortierenden ganzzahligen Schlüssel im Falle des Sortierproblems, in einigen Speicherwörtern realer Rechner Platz haben, ist die Annahme wohl gerechtfertigt. Kann man die Größe der Daten aber nicht von vornherein beschränken, ist es besser, ein anderes Kostenmaß zu nehmen und die Länge der Daten explizit zu berücksichtigen. Man spricht im ersten Fall vom Einheitskostenmaß und im letzten Fall vom logarithmischen Kostenmaß. Wir werden in diesem Buch durchweg das Einheitskostenmaß verwenden. Wir messen also etwa die Größe eines Sortierproblems in der Anzahl der zu sortierenden ganzen Zahlen, nicht aber in der Summe ihrer Längen in dezimaler oder dualer Darstellung.

Wir werden in diesem Buch Algorithmen nicht als RAM-Programme formulieren und dennoch versuchen, stets die Laufzeit abzuschätzen, die sich bei Formulierung des Algorithmus als RAM-Programm (oder in der Assemblersprache eines realen Rechners) ergeben würde. Dabei geht es uns in der Regel um das *Wachstum* der Laufzeit bei wachsender Problemgröße und nicht um den genauen Wert der Laufzeit. Da es dabei auf einen konstanten Faktor nicht ankommt, ist das keineswegs so schwierig, wie es auf den ersten Blick scheinen mag.

Eine zweite Möglichkeit zur Messung der Komplexität, d.h. insbesondere der Laufzeit eines Algorithmus, besteht darin, einige die Effizienz des Algorithmus besonders charakterisierende Parameter genau zu ermitteln. So ist es beispielsweise üblich, die Laufzeit eines Verfahrens zum Sortieren einer Folge von Schlüsseln durch die Anzahl der dabei ausgeführten Vergleichsoperationen zwischen Schlüsseln und die Anzahl der ausgeführten Bewegungen von Datensätzen zu messen. Bei arithmetischen Algorithmen interessiert beispielsweise die Anzahl der ausgeführten Additionen oder Multiplikationen.

Laufzeit und Speicherbedarf eines Algorithmus hängen in der Regel von der Größe der Eingabe ab, die im Einheitskostenmaß oder logarithmischen Kostenmaß gemessen wird. Man unterscheidet zwischen dem Verhalten im besten Fall (englisch: best case), dem Verhalten im Mittel (average case) und dem Verhalten im schlechtesten Fall (worst case).

Wir können uns beispielsweise für die bei Ausführung eines Algorithmus für ein Problem der Größe N im besten bzw. im schlechtesten Fall erforderliche Laufzeit interessieren. Dazu betrachtet man sämtliche Probleme der Größe N , bestimmt die Laufzeit des Algorithmus für alle diese Probleme und nimmt dann davon das Minimum bzw. Maximum. Auf den ersten Blick scheint es viel sinnvoller zu sein, die durchschnittliche Laufzeit des Algorithmus für ein Problem der Größe N zu bestimmen, also eine Average-case-Analyse durchzuführen. Es ist aber in vielen Fällen gar nicht klar, worüber man denn den Durchschnitt bilden soll, und insbesondere die Annahme, daß etwa jedes Problem der Größe N gleichwahrscheinlich ist, ist in der Praxis oft nicht

gerechtfertigt. Hinzu kommt, daß eine Average-case-Analyse häufig technisch schwieriger durchzuführen ist als etwa eine Worst-case-Analyse. Wir werden daher in den meisten Fällen eine Worst-case-Analyse für Algorithmen durchführen. Dabei kommt es uns auf einen konstanten Faktor bei der Ermittlung der Laufzeit und auch des Speicherplatzes in Abhängigkeit von der Problemgröße N in der Regel nicht an. Wir versuchen lediglich, die Größenordnung der Laufzeit- und Speicherplatzfunktionen in Abhängigkeit von der Größe der Eingabe zu bestimmen. Um solche Größenordnungen, also Wachstumsordnungen von Funktionen auszudrücken und zu bestimmen hat sich eine besondere Notation eingebürgert, die sogenannte *Groß-Oh-* und *Groß-Omega-Notation*.

Statt zu sagen, „für die Laufzeit $T(N)$ eines Algorithmus in Abhängigkeit von der Problemgröße N gilt für alle N : $T(N) \leq c_1 \cdot N + c_2$ mit zwei Konstanten c_1 und c_2 “, sagt man „ $T(N)$ ist von der Größenordnung N “ (oder: „ $T(N)$ ist $O(N)$ “, oder: „ $T(N)$ ist in $O(N)$ “) und schreibt: $T(N) = O(N)$ oder $T(N) \in O(N)$. Genauer definiert man für eine Funktion f die Klasse der Funktionen $O(f)$ wie folgt:

$$O(f) = \{g \mid \exists c_1 > 0 : \exists c_2 > 0 : \forall N \in \mathbb{Z}^+ : g(N) \leq c_1 \cdot f(N) + c_2\}$$

Dabei werden nur Funktionen mit nichtnegativen Werten betrachtet, weil negative Laufzeiten und Speicherplatzanforderungen keinen Sinn machen.

Die üblicherweise gewählten Schreibweisen $O(N)$, $O(N^2)$, $O(N \log N)$, usw. sind insofern formal nicht ganz korrekt, als die Variable N eigentlich als gebundene Variable gekennzeichnet werden müßte. D.h. man müßte statt $O(N^2)$ beispielsweise folgendes schreiben:

$$O(f), \quad \text{mit} \quad f(N) = N^2,$$

oder unter Verwendung der λ -Notation für Funktionen

$$O(\lambda N.N^2).$$

Beispiel: Die Funktion $g(N) = 3N^2 + 6N + 7$ ist in $O(N^2)$. Denn es gilt beispielsweise mit $c_1 = 9$ und $c_2 = 7$ für alle nichtnegativen, ganzzahligen N $g(N) \leq c_1 N^2 + c_2$, also $g(N) = O(N^2)$. Man kann ganz allgemein leicht zeigen, daß das Wachstum eines Polynoms vom Grade k von der Größenordnung $O(N^k)$ ist.

Das im Zusammenhang mit der Groß-Oh-Notation benutzte Gleichheitszeichen hat *nicht* die für die Gleichheitsrelation üblicherweise geltenden Eigenschaften. So folgt beispielsweise aus $f(N) = O(N^2)$ auch $f(N) = O(N^3)$; aber natürlich ist $O(N^2) \neq O(N^3)$.

Mit Hilfe der Groß-Oh-Notation kann man also eine Abschätzung des Wachstums von Funktionen nach oben beschreiben. Zur Angabe von unteren Schranken für die Laufzeit und den Speicherbedarf von Algorithmen muß man das Wachstum von Funktionen nach unten abschätzen können. Dazu benutzt man die *Groß-Omega-Notation* und schreibt $f \in \Omega(g)$ oder $f = \Omega(g)$, um auszudrücken, daß f mindestens so stark wächst wie g . D.E. Knuth schlägt in [90] vor, die Groß-Omega-Notation präzise wie folgt zu definieren.



$$\Omega(g) = \{h \mid \exists c > 0 : \exists n_0 > 0 : \forall n > n_0 : h(n) \geq c \cdot g(n)\}$$

Es ist also $f \in \Omega(g)$ genau dann, wenn $g \in O(f)$ ist. Uns scheint diese Forderung zu scharf. Denn ist etwa $f(N)$ eine Funktion, die für alle geraden N den Wert 1 und für

alle ungeraden N den Wert N^2 hat, so könnte man nur $f \in \Omega(1)$ schließen, obwohl für unendlich viele N gilt $f(N) = N^2$. Man wird einem Algorithmus intuitiv einen großen Zeitbedarf zuordnen, wenn er für beliebig große Probleme diesen Bedarf hat. Um die Effizienz von Algorithmen nach unten abzuschätzen, definieren wir daher

$$\Omega(g) = \{h \mid \exists c > 0 : \exists \text{ unendlich viele } n : h(n) \geq c \cdot g(n)\}.$$

Gilt für eine Funktion f sowohl $f \in O(g)$ als auch $f \in \Omega(g)$, so schreiben wir $f = \Theta(g)$.

Die weitaus häufigsten und wichtigsten Funktionen zur Messung der Effizienz von Algorithmen in Abhängigkeit von der Problemgröße N sind folgende:

- logarithmisches Wachstum: $\log N$
- lineares Wachstum: N
- N -log N -Wachstum: $N \cdot \log N$
- quadratisches, kubisches, ... Wachstum: $N^2, N^3, \dots$
- exponentielles Wachstum: $2^N, 3^N, \dots$

Da es uns in der Regel auf einen konstanten Faktor nicht ankommt, ist es nicht erforderlich, die Basis von Logarithmen in diesen Funktionen anzugeben. Wenn nichts anderes gesagt ist, setzen wir immer voraus, daß alle Logarithmen zur Basis 2 gewählt sind.

Es ist heute allgemeine Überzeugung, daß höchstens solche Algorithmen praktikabel sind, deren Laufzeit durch ein Polynom in der Problemgröße beschränkt bleibt. Algorithmen, die exponentielle Schrittzahl erfordern, sind schon für relativ kleine Problemgrößen nicht mehr ausführbar.

1.2 Zwei Beispiele arithmetischer Algorithmen

Wir wollen jetzt das Führen eines Korrektheitsnachweises und das Analysieren von Laufzeit und Speicherbedarf an zwei Algorithmen erläutern, die wohlbekannte arithmetische Probleme lösen. Wir behandeln zunächst ein Verfahren zur Berechnung des Produkts zweier nichtnegativer ganzer Zahlen und dann ein rekursives Verfahren zur Berechnung des Produkts zweier Polynome mit ganzzahligen Koeffizienten.

1.2.1 Ein Multiplikationsverfahren

Wendet man das aus der Schule für Zahlen in Dezimaldarstellung bekannte Verfahren zur Multiplikation auf zwei in Dualdarstellung gegebene Zahlen an, so erhält man beispielsweise für die zwei Zahlen 1101 und 101 folgendes Schema.

$$\begin{array}{r} 1101 \cdot 101 \\ \hline 1101 \\ 0000 \\ 1101 \\ \hline 1000001 \end{array}$$

Der Multiplikand 1101 wird der Reihe nach von rechts nach links mit den Ziffern des Multiplikators 101 multipliziert, wobei man das Gewicht der Ziffern durch entsprechendes Herausrücken nach links berücksichtigt. Am Schluß werden die Teilsummen aufaddiert. Das Herausrücken um eine Position nach links entspricht einem Verdoppungsschritt für in Dualdarstellung gegebene Zahlen. Statt alle Zwischenergebnisse auf einmal am Schluß aufzuaddieren, kann man sie natürlich Schritt für Schritt akkumulieren.

Nehmen wir an, daß a und b die zwei zu multiplizierenden ganzen Zahlen sind, und daß x , y und z Variablen vom Typ *integer* sind, so kann ein dieser Idee folgendes Multiplikationsverfahren durch folgendes Programmstück beschrieben werden:

```
x := a;
y := b;
z := 0;
while y > 0 do
{*}  if not odd(y)
      then
        begin
          y := y div 2;
          x := x + x
        end
      else
        begin
          y := y - 1;
          z := z + x
        end;
      {jetzt ist z = a · b}
```

Wir verfolgen die Wirkung dieses Programmstücks am selben Beispiel, d h. für die Anfangswerte 1101 für x und 101 für y . Wir notieren in Tabelle 1.1 die Werte der Variablen in Dualdarstellung zu Beginn eines jeden Durchlaufs durch die **while**-Schleife, d h. jedesmal, wenn die die Schleife kontrollierende Bedingung $y > 0$ überprüft wird.

x	y	z	Anzahl Schleifeniterationen
1101	101	0	0
1101	100	1101	1
11010	10	1101	2
110100	1	1101	3
110100	0	1000001	4

Tabelle 1.1

Es ist nicht schwer, in Tabelle 1.1 die gleichen Rechenschritte wiederzuerkennen, die vorher beim aus der Schule bekannten Verfahren zur Multiplikation dieser zwei Zahlen ausgeführt wurden. Ein Beweis für die Korrektheit des Verfahrens ist diese Beobachtung jedoch nicht. Dazu müssen wir vielmehr zeigen, daß für zwei beliebige nichtnegative ganze Zahlen a und b gilt, daß das Programmstück für diese Zahlen terminiert und es das Produkt der Zahlen a und b als Wert der Variablen z liefert.

Um das zu zeigen, benutzen wir eine sogenannte *Schleifeninvariante*; das ist eine den Zustand der Rechnung charakterisierende, von den Variablen abhängende Bedingung. In unserem Fall nehmen wir die Bedingung

$$P: y \geq 0 \quad \text{und} \quad z + x \cdot y = a \cdot b$$

und zeigen, daß die folgenden drei Behauptungen gelten.

Behauptung 1: P ist vor Ausführung der **while**-Schleife richtig, d.h. vor erstmaliger Ausführung der **if**-Anweisung $\{*\}$.

Behauptung 2: P bleibt bei einmaliger Ausführung der in der **while**-Schleife zu iterierenden Anweisung richtig. D h. genauer, gelten die die **while**-Schleife kontrollierende Bedingung und die Bedingung P vor Ausführung der Anweisung $\{*\}$, so gilt nach Ausführung der Anweisung $\{*\}$ ebenfalls P .

Behauptung 3: Die in der **while**-Schleife zu iterierende **if**-Anweisung wird nur endlich oft ausgeführt. Man sagt statt dessen auch kurz, daß die **while**-Schleife terminiert.

Nehmen wir einmal an, diese drei Behauptungen seien bereits bewiesen. Dann erhalten wir die gewünschte Aussage, daß das Programmstück terminiert und am Ende $z = a \cdot b$ ist, mit den folgenden Überlegungen. Daß das Programmstück für beliebige Zahlen a und b terminiert, folgt sofort aus Behauptung 3. Wegen Behauptung 1 und Behauptung 2 muß nach der letzten Ausführung der in der **while**-Schleife zu iterierenden Anweisung $\{*\}$ P gelten und die die **while**-Schleife kontrollierende Bedingung $y > 0$ natürlich falsch sein. D.h. wir haben

$$(y \geq 0 \quad \text{und} \quad z + x \cdot y = a \cdot b) \quad \text{und} \quad (y \leq 0),$$

also $y = 0$ und damit $z = a \cdot b$ wie gewünscht.

Die Gültigkeit von Behauptung 1 ist offensichtlich. Zum Nachweis von Behauptung 2 nehmen wir an, es gelte vor Ausführung der **if**-Anweisung $\{*\}$

$$(y \geq 0 \quad \text{und} \quad z + x \cdot y = a \cdot b) \quad \text{und} \quad (y > 0).$$

Fall 1: [y ist gerade] Dann wird y halbiert und x verdoppelt. Es gilt also nach Ausführung der **if**-Anweisung $\{*\}$ immer noch $(y \geq 0 \text{ und } z + x \cdot y = a \cdot b)$.

Fall 2: [y ist ungerade] Dann wird y um 1 verringert und z um x erhöht und daher gilt ebenfalls nach Ausführung der **if**-Anweisung wieder $(y \geq 0 \text{ und } z + x \cdot y = a \cdot b)$.

Zum Nachweis der Behauptung 3 genügt es zu bemerken, daß bei jeder Ausführung der in der **while**-Schleife zu iterierenden **if**-Anweisung der Wert von y um mindestens 1 abnimmt. Nach höchstens y Iterationen muß also $y \leq 0$ werden und damit die Schleife terminieren. Damit ist insgesamt die Korrektheit dieses Multiplikationsalgorithmus bewiesen.

Wie effizient ist das angegebene Multiplikationsverfahren? Zunächst ist klar, daß das Verfahren nur konstanten Speicherplatz benötigt, wenn man das Einheitskostenmaß zugrundelegt, denn es werden nur drei Variablen zur Aufnahme beliebig großer ganzer Zahlen verwendet. Legt man das logarithmische Kostenmaß zugrunde, ist der Speicherplatzbedarf linear von der Summe der Längen der zu multiplizierenden Zahlen abhängig. Es macht wenig Sinn, zur Messung der Laufzeit das Einheitskostenmaß zugrunde zu legen. Denn in diesem Maß gemessen ist die Problemgröße konstant gleich 2. Wir interessieren uns daher für die Anzahl der ausgeführten arithmetischen Operationen in Abhängigkeit von der Länge und damit der Größe der zwei zu multiplizierenden Zahlen a und b .

Beim Korrektheitsbeweis haben wir uns insbesondere davon überzeugt, daß die in der **while**-Schleife iterierte **if**-Anweisung höchstens y -mal ausgeführt werden kann, mit $y = b$. Das ist eine sehr grobe Abschätzung, die allerdings sofort zu einer Schranke in $O(b)$ für die zur Berechnung des Produkts $a \cdot b$ ausgeführten Divisionen durch 2, Additionen und Subtraktionen führt. Eine genaue Analyse zeigt, daß die Zahl y , die anfangs den Wert b hat, genau einmal weniger halbiert wird, als ihre Länge angibt; eine Verringerung um 1 erfolgt gerade so oft, wie die Anzahl der Einsen in der Dualdarstellung von y angibt.

Nehmen wir an, daß alle Zahlen in Dualdarstellung vorliegen, so ist die Division durch 2 nichts anderes als eine Verschiebe- oder Shift-Operation um eine Position nach rechts, wobei die am weitesten rechts stehende Ziffer (eine 0) verlorengeht; die Verdoppelung von x entspricht einer Shift-Operation um eine Position nach links, wobei eine 0 als neue am weitesten rechts stehende Ziffer nachgezogen wird. Das Verkleinern der ungeraden Zahl y um 1 bedeutet, die Endziffer 1 in eine 0 zu verwandeln. Es ist realistisch anzunehmen, daß alle diese Operationen in konstanter Zeit ausführbar sind. Nimmt man an, daß auch die Anweisung $z := z + x$ in konstanter Zeit ausgeführt werden kann, ergibt sich eine Gesamtlaufzeit des Verfahrens von der Größenordnung $O(\text{Länge}(b))$. Legt man die vielleicht realistischere Annahme zugrunde, daß die Berechnung der Summe $z + x$ in der Zeit $O(\text{Länge}(z) + \text{Länge}(x))$ ausführbar ist, ergibt sich eine Gesamtlaufzeit von der Größenordnung $O(\text{Länge}(b) \cdot (\text{Länge}(a) + \text{Länge}(b)))$.

1.2.2 Polynomprodukt

Ein ganzzahliges Polynom vom Grade $N - 1$ kann man sich gegeben denken durch die N ganzzahligen Koeffizienten $a_0, \dots, a_{N-1}$. Es hat die Form

$$p(x) = a_0 + a_1x^1 + \dots + a_{N-1}x^{N-1}.$$

Wir lassen zunächst offen, wie ein solches Polynom programmtechnisch realisiert wird. Seien nun zwei Polynome $p(x)$ und $q(x)$ vom Grade $N - 1$ gegeben; $p(x)$ wie oben angegeben und

$$q(x) = b_0 + b_1x^1 + \dots + b_{N-1}x^{N-1}.$$

Wie kann man das Produkt der beiden Polynome $r(x) = p(x) \cdot q(x)$ berechnen? Bereits in der Schule lernt man, daß das Produktpolynom ein Polynom vom Grade $2N - 2$ ist, das man erhält, wenn man jeden Term $a_i x^i$ des Polynoms p mit jedem Term $b_j x^j$ des

Polynoms q multipliziert und dann die Terme mit gleichem Exponenten sammelt. Es ist leicht, eine Implementation dieses sogenannten naiven Verfahrens anzugeben, wenn man voraussetzt, daß die Polynome durch Arrays realisiert werden, die die Koeffizienten enthalten. Setzen wir die Deklarationen

```
var
   $p, q$ : array  $[0 \dots N - 1]$  of integer;
   $r$ : array  $[0 \dots 2N - 2]$  of integer
```

voraus, so kann das Produktpolynom durch eine doppelt geschachtelte **for**-Schleife wie folgt berechnet werden.

```
for  $i := 0$  to  $2N - 2$  do  $r[i] := 0$ ;
for  $i := 0$  to  $N - 1$  do
  for  $j := 0$  to  $N - 1$  do  $r[i + j] := r[i + j] + p[i] * q[j]$ 
```

Diese Darstellung zeigt unmittelbar, daß zur Berechnung der Koeffizienten des Produktpolynoms genau N^2 Koeffizientenprodukte berechnet werden.

Wir wollen jetzt ein anderes Verfahren zur Berechnung des Produktpolynoms angeben, das mit weniger als N^2 Koeffizientenproduktberechnungen auskommt. Das Verfahren folgt der Divide-and-conquer-Strategie, die ein sehr allgemeines und mächtiges Prinzip zur algorithmischen Lösung von Problemen darstellt. Es kann als Problemlösungsschema wie folgt formuliert werden.

Divide-and-conquer-Verfahren zur Lösung eines Problems der Größe N

1. *Divide:* Teile das Problem der Größe N in (wenigstens) zwei annähernd gleich große Teilprobleme, wenn $N > 1$ ist; sonst löse das Problem der Größe 1 direkt.
2. *Conquer:* Löse die Teilprobleme auf dieselbe Art (rekursiv).
3. *Merge:* Füge die Teillösungen zur Gesamtlösung zusammen.

Um dieses Prinzip auf das Problem, das Produkt zweier Polynome zu berechnen, einfach anwenden zu können, nehmen wir an, daß die Koeffizientenzahl N beider Polynome p und q eine Potenz von 2 ist. Dann kann man schreiben

$$p(x) = p_l(x) + x^{\frac{N}{2}} p_r(x)$$

mit

$$\begin{aligned} p_l(x) &= a_0 + a_1 x^1 + \dots + a_{\frac{N}{2}-1} x^{\frac{N}{2}-1} \\ p_r(x) &= a_{\frac{N}{2}} + a_{\frac{N}{2}+1} x^1 + \dots + a_{N-1} x^{\frac{N}{2}-1}. \end{aligned}$$

Ebenso kann man auch schreiben

$$q(x) = q_l(x) + x^{\frac{N}{2}} q_r(x)$$

mit zwei analog definierten Polynomen $q_l(x)$ und $q_r(x)$ vom Grade $\frac{N}{2} - 1$.

Dann ist

$$\begin{aligned} r(x) &= p(x)q(x) \\ &= p_l(x)q_l(x) + (p_l(x)q_r(x) + p_r(x)q_l(x))x^{\frac{N}{2}} + p_r(x)q_r(x)x^N. \end{aligned}$$

Wir haben also das Problem, das Produkt zweier Polynome (vom Grade $N - 1$) mit jeweils N Koeffizienten zu berechnen, zerlegt in das Problem, vier Produkte von Polynomen mit jeweils $N/2$ Koeffizienten zu berechnen: Das sind die Produkte $p_l q_l$, $p_l q_r$, $p_r q_l$, $p_r q_r$. Wie man daraus die Koeffizienten des Produktpolynoms $r(x)$ erhält, ohne daß weitere Koeffizientenprodukte berechnet werden müssen, ist ebenfalls aus der oben angegebenen Gleichung für $r(x)$ abzulesen. Wir wollen die Anzahl der Multiplikationen von Koeffizienten, die ausgeführt werden, wenn man zwei Polynome vom Grade $N - 1$ mit N Koeffizienten miteinander multipliziert, mit $M(N)$ bezeichnen. Ein dem Divide-and-conquer-Prinzip folgender Algorithmus zur Berechnung des Produktpolynoms, der auf der oben angegebenen Zerlegung des Problems in vier Teilprobleme halber Größe beruht, führt also eine Anzahl von Koeffizientenproduktberechnungen durch, die durch folgende Rekursionsformel beschrieben werden kann.

$$M(N) = 4 \cdot M\left(\frac{N}{2}\right)$$

Natürlich ist

$$M(1) = 1.$$

Weil wir angenommen hatten, daß N eine Potenz von 2 ist, also $N = 2^k$ für ein $k \geq 0$, erhält man als Lösung dieser Rekursionsgleichung sofort

$$M(N) = 4^k = (2^2)^k = (2^k)^2 = N^2.$$

Ein auf der oben angegebenen Zerlegung gegründetes Divide-and-conquer-Verfahren liefert also keine Verbesserung gegenüber dem naiven Verfahren. Es ist aber nicht schwer, eine andere Zerlegung des Problems anzugeben, so daß ein auf dieser Zerlegung gegründetes Divide-and-conquer-Verfahren mit weniger Koeffizientenproduktberechnungen auskommt. Wir setzen

$$\begin{aligned} z_l(x) &= p_l(x)q_l(x), \\ z_r(x) &= p_r(x)q_r(x), \end{aligned}$$

und

$$z_m(x) = (p_l(x) + p_r(x))(q_l(x) + q_r(x)).$$

Dann ist

$$p(x)q(x) = z_l(x) + (z_m(x) - z_l(x) - z_r(x))x^{\frac{N}{2}} + z_r(x)x^N.$$

Ein auf dieser Zerlegung gegründetes Divide-and-conquer-Verfahren zur Berechnung des Produkts zweier Polynome mit N Koeffizienten kann also wie folgt formuliert werden.

Verfahren zur Multiplikation zweier Polynome $p(x)$ und $q(x)$ mit N Koeffizienten:

Falls $N = 1$ ist, berechne das Produkt der beiden Koeffizienten; sonst:

1. *Divide: Zerlege die Polynome $p(x)$ und $q(x)$ in der Form $p(x) = p_l(x) + p_r(x) \cdot x^{\frac{N}{2}}$ und $q(x) = q_l(x) + q_r(x) \cdot x^{\frac{N}{2}}$, setze $p_m(x) = p_l(x) + p_r(x)$ und $q_m(x) = q_l(x) + q_r(x)$.*
2. *Conquer: Wende das Verfahren (rekursiv) an, um die folgenden Polynomprodukte zu berechnen:*

$$z_l(x) = p_l(x)q_l(x), \quad z_m(x) = p_m(x)q_m(x), \quad z_r(x) = p_r(x)q_r(x)$$

3. *Merge: Setze $p(x)q(x) = z_l(x) + (z_m(x) - z_l(x) - z_r(x))x^{\frac{N}{2}} + z_r(x)x^N$.*

Offenbar sind p_l , p_m , p_r und q_l , q_m , q_r Polynome mit $N/2$ Koeffizienten. Da außer im Fall $N = 1$ keinerlei Koeffizientenprodukte berechnet werden müssen, erhält man für die Anzahl der nach diesem Verfahren berechneten Koeffizientenprodukte die Rekursionsformel

$$M(1) = 1$$

und

$$M(N) = 3 \cdot M\left(\frac{N}{2}\right).$$

Für $N = 2^k$ hat diese Formel die Lösung

$$M(N) = 3^k = 2^{(\log 3)k} = 2^{k(\log 3)} = N^{\log 3} = N^{1.58\dots}.$$

Wir haben also eine Verbesserung gegenüber dem naiven Verfahren erreicht.

Das angegebene Verfahren ist nicht das beste bekannte Verfahren zur Berechnung des Produkts zweier Polynome in dem Sinne, daß die Anzahl der ausgeführten Koeffizientenproduktberechnungen möglichst klein wird. Es zeigt aber die Anwendbarkeit des Divide-and-conquer-Prinzips sehr schön und ebenso, wie man nach dieser Strategie entworfene Algorithmen analysiert, nämlich durch Aufstellen und Lösen einer Rekursionsgleichung. Wir werden in diesem Buch noch zahlreiche weitere Beispiele für dieses Prinzip bringen.

Wir nennen an dieser Stelle nur einige weitere Probleme, die auf diese Weise gelöst und analysiert werden können, ohne daß wir dabei auf irgendwelche Details eingehen. Es sind die Multiplikation langer ganzer Zahlen, die Multiplikation zweier $N \times N$ -Matrizen nach der Methode von Strassen [Str78], binäres Suchen (vgl. dazu Kapitel 3), die Sortierverfahren Quicksort, Heapsort, Mergesort (vgl. dazu Kapitel 2) und Verfahren aus der Geometrie, zum Beispiel zur Berechnung aller Schnitte von Liniensegmenten in der Ebene (vgl. dazu Kapitel 7).

1.3 Verschiedene Algorithmen für dasselbe Problem

Wie am Beispiel des Polynomprodukts im vorigen Abschnitt bereits gezeigt wurde, kann man dasselbe Problem durchaus mit verschiedenen Algorithmen lösen. Das Ziel ist natürlich, den für ein Problem besten Algorithmus zu finden und zu implementieren. Das verlangt insbesondere eine möglichst optimale Nutzung der Ressourcen Speicherplatz und Rechenzeit. Wie wichtig die richtige Algorithmenwahl zur Lösung eines Problems sein kann, zeigt ein von Jon Bentley behandeltes Problem  das wir in diesem Abschnitt genauer diskutieren wollen. Es handelt sich um das *Maximum-Subarray-Problem*.

Gegeben sei eine Folge X von N ganzen Zahlen in einem Array. Gesucht ist die maximale Summe aller Elemente in einer zusammenhängenden Teilfolge. Sie wird als maximale Teilsumme bezeichnet, jede solche Folge als maximale Teilfolge.

Für die Eingabefolge $X[1 \dots 10]$

31, -41, 59, 26, -53, 58, 97, -93, -23, 84

ist die Summe der Teilfolge $X[3 \dots 7]$ mit Wert 187 die Lösung des Problems. Eine Variante dieses Problems wurde übrigens im Rahmen des 4. Bundeswettbewerbs Informatik 1985 als Aufgabe gestellt (Aktienkurs-Analyse , vgl. Aufgabe 1.5).

Ein sofort einsichtiges, *naïves* Verfahren zur Lösung des Problems benutzt drei ineinandergeschachtelte **for**-Schleifen, um die maximale Teilsumme als Wert der Variablen *maxtsumme* zu berechnen.

```

maxtsumme := 0;
for u := 1 to N do
  for o := u to N do
    begin
      {bestimme die Summe der Elemente in der Teilfolge X[u .. o]}
      Summe := 0;
      for i := u to o do Summe := Summe + X[i];
      {bestimme den größeren der beiden Werte Summe
       und maxtsumme}
      maxtsumme := max(Summe, maxtsumme)
    end

```

Die Lösung ist einfach, aber ineffizient, denn sie benötigt für eine Folge der Länge N offenbar

$$\sum_{u=1}^N \sum_{o=u}^N \sum_{i=u}^o 1 = \Theta(N^3)$$

Schritte, d. h. genauer Zuweisungen, Additionen und Maximumbildungen.

Jetzt folgen wir dem Divide-and-conquer-Prinzip zur Lösung des Maximum-Subarray-Problems. Die Anwendbarkeit dieses Prinzips ergibt sich aus folgender Überlegung.

Wird eine gegebene Folge in der Mitte geteilt, so liegt die maximale Teilfolge entweder ganz in einem der beiden Teile oder sie umfaßt die Trennstelle, liegt also teils im linken und teils im rechten Teil. Im letzteren Fall gilt für das in einem Teil liegende Stück der maximalen Teilfolge: Die Summe der Elemente ist maximal unter allen zusammenhängenden Teilfolgen in diesem Teil, die das Randelement an der Trennstelle enthalten.

Wir wollen die maximale Summe von Elementen, die das linke bzw. das rechte Randelement einer Folge von Elementen enthält, kurz das linke bzw. rechte Randmaximum nennen. Das linke Randmaximum $lmax$ für eine Folge $X[l], \dots, X[r]$ ganzer Zahlen kann man in $\Theta(r-l)$ Schritten wie folgt bestimmen.

```

 $lmax := 0;$ 
 $summe := 0;$ 
for  $i := l$  to  $r$  do
  begin
     $summe := summe + X[i];$ 
     $lmax := \max(lmax, summe)$ 
  end

```

Entsprechend kann man auch das rechte Randmaximum $rmax$ für eine Folge ganzer Zahlen in einer Anzahl von Schritten bestimmen, die linear mit der Anzahl der Folgeelemente wächst. Das dem Divide-and-conquer-Prinzip folgende Verfahren zur Berechnung der maximalen Teilsumme in einer Folge X ganzer Zahlen kann nun wie folgt formuliert werden.

```

Algorithmus  $maxtsum(X);$ 
{liefert eine maximale Teilsumme der Folge  $X$  ganzer Zahlen}
begin
  if  $X$  enthält nur ein Element  $a$ 
    then
      if  $a > 0$ 
        then  $maxtsum := a$ 
        else  $maxtsum := 0$ 
      else
        begin
          {Divide;}
          teile  $X$  in eine linke und eine rechte Teilfolge  $A$  und  $B$ 
            annähernd gleicher Größe;
          {Conquer;}
           $maxtinA := maxtsum(A);$ 
           $maxtinB := maxtsum(B);$ 
          bestimme das rechte Randmaximum  $rmax(A)$  der
            linken Teilfolge  $A$ ;
          bestimme das linke Randmaximum  $lmax(B)$  der
            rechten Teilfolge  $B$ ;
          {Merge;}
           $maxtsum := \max(maxtinA, maxtinB, rmax(A) + lmax(B))$ 
        end
      end  $\{maxtsum\}$ 

```

Bezeichnet nun $T(N)$ die Anzahl der Schritte, die erforderlich ist, um den Algorithmus *maxtsum* für eine Folge der Länge N auszuführen, so gilt offenbar folgende Rekursionsformel:

$$T(N) = 2 \cdot T\left(\frac{N}{2}\right) + \text{Const} \cdot N$$

Da natürlich $T(1)$ konstant ist, erhält man als Lösung dieser Gleichung und damit als asymptotische Laufzeit des Verfahrens

$$T(N) = \Theta(N \log N).$$

Das ist schon viel besser als die Laufzeit des naiven Verfahrens. Aber ist es bereits das bestmögliche Verfahren? Nein — denn die Anwendung eines weiteren algorithmischen Lösungsprinzips, des Scan-line-Prinzips, liefert uns ein noch besseres Verfahren. Wir haben eine aufsteigend sortierte, lineare Folge von Inspektionsstellen (oder: Ereignispunkten), die Positionen $1, \dots, N$ der Eingabefolge. Wir durchlaufen die Eingabe in der durch die Inspektionsstellen vorgegebenen Reihenfolge und führen zugleich eine vom jeweiligen Problem abhängige, dynamisch veränderliche, d.h. an jeder Inspektionsstelle gegebenenfalls zu korrigierende Information mit. In unserem Fall ist das die maximale Summe *bisMax* einer Teilfolge im gesamten bisher inspizierten Anfangsstück und das an der Inspektionsstelle endende rechte Randmaximum *ScanMax* des bisher inspizierten Anfangsstücks. Das ist in Abbildung 1.1 dargestellt.

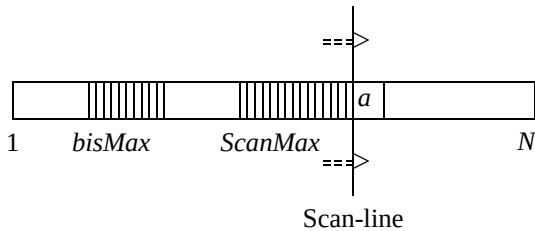


Abbildung 1.1: Scan-Line

Nehmen wir nun an, daß wir bereits ein Anfangsstück der Länge l der gegebenen Folge inspiziert haben und die maximale Teilsumme *bisMax* sowie das rechte Randmaximum *ScanMax* in diesem Anfangsstück kennen. Was ist die maximale Teilsumme, wenn man das $(l + 1)$ -te Element, sagen wir a , hinzunimmt? Die maximale Teilfolge des neuen Anfangsstücks der Länge $l + 1$ liegt entweder bereits im Anfangsstück der Länge l , oder sie enthält das neu hinzugenommene Element a , reicht also bis zum rechten Rand. Das rechte Randmaximum der neuen Folge mit $l + 1$ Elementen erhält man nun aus dem rechten Randmaximum der Folge durch Hinzunahme von a , also aus dem alten Wert von *ScanMax*, indem man a hinzuaddiert, vorausgesetzt, daß dieser Wert insgesamt positiv bleibt. Ist das nicht der Fall, so ist die maximale Summe von Elementen, die das rechte Randelement enthält, die Summe der Elemente der leeren Folge, also 0. Damit erhält man folgendes Verfahren, das hier etwas allgemeiner beschrieben ist, als es für das behandelte Problem nötig wäre.

```

Q := Folge der Inspektionsstellen von links nach rechts;
      { = Folge der Positionen  $1, \dots, N$  }
{Initialisiere}
ScanMax := 0;
bisMax := 0;
while Q noch nicht erschöpft do
  begin
    q := nächstes Element von Q;
    a := das Element an Position q;
    {update ScanMax und bisMax}
    if ScanMax + a > 0
      then ScanMax := ScanMax + a
      else ScanMax := 0;
    bisMax := max(bisMax, ScanMax)
  end

```

Am Ende enthält dann *bisMax* den gewünschten Wert.

Dies ist ein Algorithmus, der in linearer Zeit ausführbar ist. Denn an jeder der N Inspektionsstellen müssen nur konstant viele Schritte (u.a. zum Update von *ScanMax* und *bisMax*) und damit insgesamt nur $\Theta(N)$ Schritte ausgeführt werden. Das ist asymptotisch optimal. Es gibt keinen Algorithmus zur Bestimmung der maximalen Teilsumme einer Folge von N Elementen, der für beliebig viele N mit weniger als $c \cdot N$ Schritten, für eine positive Konstante c , auskommt. Der Grund ist, daß zur Bestimmung der maximalen Teilfolge offensichtlich alle Folgeelemente wenigstens einmal betrachtet werden müssen. Das sind aber bereits N Schritte.

1.4 Die richtige Wahl einer Datenstruktur

Die beiden ersten der im vorigen Abschnitt angegebenen drei verschiedenen Algorithmen zur Lösung des Maximum-Subarray-Problems haben vorausgesetzt, daß die Folge der ganzen Zahlen, für die die maximale Teilsumme ermittelt werden sollte, in einem Array gegeben ist. Wir haben die gleiche Datenstruktur zur Implementation verschiedener Verfahren benutzt.

Bereits im täglichen Leben machen wir aber die Erfahrung, daß die richtige Organisationsform für eine Menge von Daten und damit die richtige Datenstrukturwahl ganz erheblichen Einfluß darauf hat, wie effizient sich bestimmte Operationen für die Daten ausführen lassen. Denken wir etwa an ein Telefonbuch: Es ist leicht, zu einem gegebenen Namen die zugehörige Telefonnummer zu finden; für die umgekehrte Aufgabe ist aber das bei Telefonbüchern übliche Gliederungsprinzip, zunächst nach Orten und innerhalb eines Ortes nach Namen alphabetisch sortiert, wenig geeignet. Da der normale Telefonbenutzer aber höchst selten den zu einer Telefonnummer gehörigen Namen sucht, lohnt es sich nicht, etwa nach Nummern aufsteigend sortierte Telefonbücher an die Telefonkunden auszugeben.

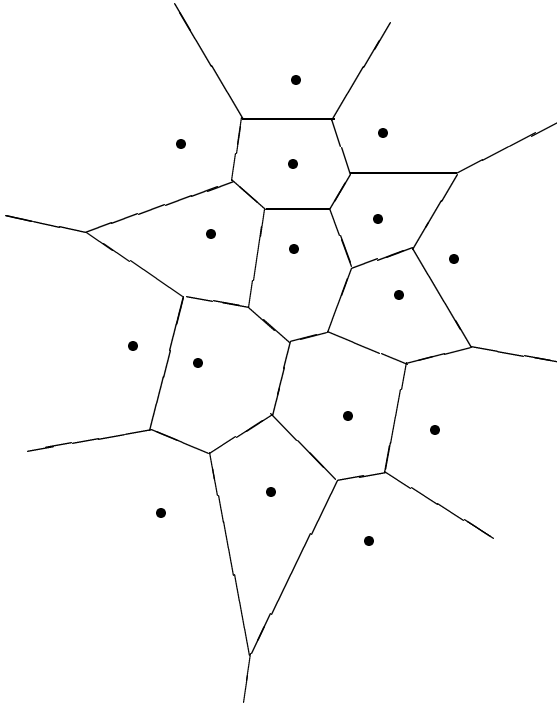
Nicht immer ist die richtige Wahl einer Datenstruktur so einfach. Es gibt viele Fälle, in denen es keineswegs auf der Hand liegt, welche Organisationsform für eine Menge von Daten zu wählen ist, um bestimmte Operationen auf der Datenmenge effizient ausführen zu können. Wir geben ein Beispiel, das als *Post-office-Problem* bekannt ist: Für eine gegebene, als fest vorausgesetzte Menge M von Orten (mit Postämtern) und für einen beliebig gegebenen Ort p , der in der Regel nicht zu M gehört (also kein Postamt hat), soll festgestellt werden, welches der dem Ort p nächstgelegene Ort aus M ist. Wie kann man die Menge M strukturieren, um derartige Anfragen, sogenannte *Nearest-neighbor-queries*, möglichst effizient ausführen zu können? Eine alphabetische Reihenfolge der Orte hilft offenbar wenig. Auf den ersten Blick scheint nichts anderes übrig zu bleiben, als für einen gegebenen Ort p wie folgt vorzugehen. Man betrachtet der Reihe nach jeden Ort $q \in M$ und berechnet die Distanz $d(p, q)$ zwischen p und q . Schließlich stellt man fest, für welches q die Distanz $d(p, q)$ minimalen Wert hat. Es ist offensichtlich, daß der Aufwand zur Beantwortung einer derartigen Nachbarschaftsanfrage wenigstens linear mit der Anzahl der Orte in M wächst, wenn man so vorgeht. Kann man es besser machen? Die Idee liegt nahe, die Orte aus M zunächst in eine Landkarte einzutragen und dann für einen gegebenen Ort p nachzusehen, welchem Ort aus M p am nächsten liegt. Die Landkarte mit den darin eingetragenen Orten aus M ist also eine Datenstruktur für M , die Anfragen nach nächsten Nachbarn besser unterstützt. Wir idealisieren und präzisieren diese Idee noch weiter und nehmen an, daß der Abstand zwischen je zwei Orten die gewöhnliche, euklidische Distanz ist. Sind p und q Punkte mit reellwertigen Koordinaten $p = (p_x, p_y)$ und $q = (q_x, q_y)$ in einem kartesischen Koordinatensystem, so sei also die Distanz $d(p, q)$ zwischen p und q definiert durch

$$d(p, q) = \sqrt{(p_x - q_x)^2 + (p_y - q_y)^2}.$$

Dann kann man die euklidische Ebene für eine gegebene Menge M von Punkten in Gebiete gleicher nächster Nachbarn einteilen. Jedem Punkt $p \in M$ ordnet man ein Gebiet $VR(p)$ der Ebene zu, das genau alle Punkte enthält, deren Distanz zu p geringer ist als zu allen anderen Punkten aus M . Auf diese Weise erhält man für jede (feste) Menge M von Punkten eine vollständige Aufteilung der Ebene in disjunkte Gebiete, die sich höchstens an den Rändern berühren. Abbildung 1.2 zeigt ein Beispiel einer derartigen Struktur für eine Menge von 16 Punkten.

Man nennt eine solche Einteilung der Ebene das zur Menge M gehörende *Voronoi-Diagramm* $VD(M)$ und die einem Punkt $p \in M$ zugeordnete Region $VR(p)$ die *Voronoi-Region* von p . Für eine genaue Definition von $VD(M)$, für Algorithmen zur Konstruktion von $VD(M)$ und für die Möglichkeit zur Speicherung von $VD(M)$ verweisen wir auf Kapitel 7. Es ist bereits jetzt klar, wie man zu einem gegebenen Punkt p den nächsten Nachbarn von q in M finden kann: Man bestimmt die Voronoi-Region, in die p fällt. Ist $p \in VR(q)$, so ist q nächster Nachbar von p . Man kann zeigen, daß die Region $VR(q)$, in die p fällt, in $O(\log N)$ Schritten bestimmt werden kann, wenn N die Gesamtzahl der Punkte in der gegebenen Menge M ist. Das Voronoi-Diagramm, auf Papier gezeichnet oder mit den Mitteln einer Programmiersprache beschrieben und im Rechner geeignet gespeichert, ist also eine Datenstruktur, die Nearest-neighbor-queries gut unterstützt.

Die Frage nach der richtigen Datenstruktur kann man also genauer so formulieren: Gegeben sei eine Menge von Daten und eine Folge von Operationen mit diesen Daten; man finde eine Speicherungsform für die Daten und Algorithmen für die auszufüh-

**Abbildung 1.2**

renden Operationen so, daß die Operationen der gegebenen Folge möglichst effizient ausführbar sind. Auch in dieser Formulierung sind noch viele für die richtige Wahl wesentliche Parameter offengelassen: Ist die Folge der Operationen vorher bekannt? Wenn nicht, kennt man dann wenigstens die (relativen) Häufigkeiten der verschiedenen in der Folge auftretenden Operationen? Kommt es bei der Effizienz in erster Linie auf die Ausführungszeit, auf den Speicherbedarf, auf die leichte Programmierbarkeit, usw. an? Auf jeden Fall dürfte klar sein, daß man die richtige Speicherungsform für eine Menge von Daten nicht unabhängig davon wählen kann, welche Operationen mit welcher Häufigkeit mit den Daten ausgeführt werden.

Daten und Operationen mit den Daten gehören also zusammen. Es ist heute üblich geworden, sie als Einheit aufzufassen und von *abstrakten Datentypen* (ADT) zu sprechen: Ein ADT besteht aus einer oder mehreren Mengen von Objekten und darauf definierten Operationen, die mit in der Mathematik üblichen Methoden spezifiziert werden können. Wir geben einige Beispiele an, zuerst den ADT *Polynom*. Die Menge der Objekte ist die Menge der Polynome mit ganzzahligen Koeffizienten. Die Menge der Operationen enthält genau die Addition und Multiplikation zweier Polynome. Nimmt man zur Menge der Operationen weitere hinzu, z.B. die erste Ableitung eines Polynoms (die etwa für ein Polynom $p(x) = 3x^3 + 6x - 7$ das Polynom $p'(x) = 9x^2 + 6$ liefert), so erhält man

einen anderen als den oben angegebenen ADT. In beiden Fällen hat man nur eine Sorte von Objekten. Das ist eher die Ausnahme.

Meistens hat man mehrere verschiedene Mengen von Objekten, und die Operationen sind nicht nur auf Objekte einer Sorte beschränkt, wie in dem oben schon diskutierten Beispiel einer Menge von Punkten, für die Nearest-neighbor-queries beantwortet werden sollen. Als ADT *Punktmenge* kann man dieses Beispiel folgendermaßen beschreiben. Eine erste Menge von Objekten ist die Klasse aller endlichen Mengen von Punkten in der Ebene; eine weitere Menge von Objekten ist die Menge aller Punkte der Ebene. Die Operation *nächster Nachbar* ordnet einer Menge M von Punkten und einem Punkt p einen Punkt aus M zu. Weitere Beispiele für Operationen auf diesen Objektmengen sind die Operation des Einfügens eines Punktes in eine Menge und das Entfernen eines Punktes aus einer Menge. Sie liefern als Ergebnis wieder eine Menge von Punkten. Will man auch noch zu je zwei Punkten die euklidische Distanz ermitteln können, muß man die Menge der reellen Zahlen als weitere Objektmenge und die oben definierte Distanzfunktion als weitere Operation hinzunehmen.

Die zur Definition eines ADT benutzten Objektmengen und Operationen werden, wie in der Mathematik üblich, ohne Rücksicht auf ihre programmtechnische Realisierung spezifiziert. Die zwei wichtigsten Methoden sind die *konstruktive* und die *axiomatische* Methode.

Bei der *konstruktiven Methode* geht man von bekannten mathematischen Modellen aus und konstruiert daraus neue; die jeweils benötigten Operationen werden explizit oder implizit mit Hilfe schon bekannter definiert. So kann man beispielsweise Punkte in der euklidischen Ebene als Paare reeller Zahlen auffassen und die Operation „nächster Nachbar“ auf bekannte Operationen für reelle Zahlen zurückführen. Gemeint sind hier natürlich die reellen Zahlen als Objekte der Mathematik und nicht ihre Realisierung als Daten vom Typ *real* in einer konkreten Programmiersprache auf einem konkreten Rechner.

Bei der *axiomatischen Methode* werden die Objektmengen nur implizit durch die Angabe von Axiomen für die mit den Objekten auszuführenden Operationen festgelegt. Das geschieht ganz analog etwa zur üblichen Definition einer Gruppe in der Mathematik: Eine Menge G zusammen mit einer auf G definierten Verknüpfungsoperation heißt Gruppe, wenn für die Elemente von G und die Verknüpfungsoperation die üblichen Gruppenaxiome gelten.

Es ist möglich, Algorithmen so zu formulieren, daß man nur auf Objekte und Operationen abstrakter Datentypen zurückgreift. Wir geben dafür ein Beispiel und formulieren einen Algorithmus, der zu einer gegebenen, endlichen Menge M von Punkten ein Paar (p, q) von zwei verschiedenen Punkten aus M liefert, dessen (euklidische) Distanz minimal unter allen Distanzen von Punkten aus M ist. Dabei setzen wir einen ADT „Punktmenge“ voraus, für den insbesondere die Operationen „nächster Nachbar“ und „Distanz zweier Punkte“ definiert sind.

Algorithmus *Nearest-neighbors* (M);

{liefert ein Paar (p_0, q_0) von Punkten aus M mit minimaler euklidischer Distanz}

Fall 1: [$M = \emptyset$ oder M enthält nur einen Punkt] Dann ist das Paar nächster Nachbarn nicht definiert.

Fall 2: [M enthält wenigstens zwei verschiedene Punkte p und q]

- (a) Wähle zwei verschiedene Punkte p_0 und q_0 aus M und berechne ihre Distanz dist .
- (b) Bestimme für jeden Punkt p aus M den nächsten Nachbarn q von p in $M \setminus \{p\}$; berechne die Distanz $d(p, q)$ der Punkte p und q ; falls $d(p, q) < \text{dist}$, setze $p_0 := p$, $q_0 := q$, $\text{dist} := d(p, q)$.

Man sieht in dieser Formulierung, daß auch einige weitere Operationen für eine Punktmenge M (nicht nur „nächster Nachbar“ und „Distanz zweier Punkte“) ausführbar sein müssen: Es muß möglich sein, festzustellen, ob $M = \emptyset$ ist oder ob M nur einen Punkt enthält; ferner muß es möglich sein, einen Punkt aus M auszuwählen und aus M zu entfernen. Es werden jedoch keinerlei implementationsabhängige Details für Punktmenge benötigt.

Soll der Algorithmus in einer konkreten Programmiersprache implementiert werden, ist es nötig, den ADT Punktmenge durch Angabe von Datenstrukturen für die Objektmenge und Algorithmen für die benutzten Operationen zu realisieren. Dazu müssen wir sie auf die in der jeweils benutzten Sprache vorhandenen Datentypen und Grundoperationen zurückführen. Denn die Programmiersprache besitzt in der Regel keine Datentypen und Operationen für Variablen des entsprechenden Typs, die man direkt zur Implementation des abstrakten Datentyps nehmen könnte. Ist die Programmiersprache beispielsweise die Sprache Pascal, so kann man Punktmenge ausgehend vom Grundtyp *real* und unter Benutzung der in Pascal vorhandenen Möglichkeiten zur Definition strukturierter Datentypen definieren. Da der **set**-Typ in Pascal nur die Zusammenfassung einer Menge von Objekten eines einfachen Typs außer *real* erlaubt, kann man diese Strukturierungsmethode nicht nehmen, um Punktmenge in Pascal zu realisieren. Eine Möglichkeit ist beispielsweise, die Punkte einer Menge als Elemente eines Arrays passender maximaler Größe zu vereinbaren.

const

$\text{maxZahl} = \{\text{passend gewählte Zahl}\};$

type

Punkt = **record**

$\text{xcoord, ycoord: real}$

end;

Punktmenge = **record**

$\text{elementzahl: integer};$

$\text{element: array [1 .. maxZahl] of Punkt}$

end

Eine Punktmenge M ist dann nichts anderes als eine Variable vom oben vereinbarten Typ. Es ist nicht schwer, alle zur Formulierung des Algorithmus *Nearest-neighbors* benutzten Operationen als Funktionen und Prozeduren zu formulieren, die diese Datenstruktur benutzen. Wir geben ein einfaches Beispiel.

```
function empty ( $M$ : Punktmenge) : boolean;
{liefert true genau dann, wenn  $M$  die leere Menge ist}
begin
  empty := (0 =  $M$ .elementzahl)
end
```

Eine Realisierung des ADT Punktmenge als Voronoi-Diagramm ist nicht so offensichtlich, weil nicht klar ist, wie diese Struktur mit den Mitteln einer Programmiersprache, wie z.B. Pascal, beschrieben werden kann (vgl. hierzu Kapitel 7).

Wir unterscheiden also zwischen Datentypen, abstrakten Datentypen und Datenstrukturen. *Datentypen* sind die in Programmiersprachen üblicherweise vorhandenen Grundtypen, wie *integer*, *real*, *boolean*, *character*, und die daraus mit den jeweils vorhandenen Strukturierungsmethoden, wie **record**, **array**, **set**, **file**, gebildeten zusammengesetzten Typen. Ein Datentyp legt die Menge der möglichen Werte und die zulässigen Operationen mit Variablen dieses Typs fest.

Ein *abstrakter Datentyp* ist das Analogon zu einer mathematischen Theorie. Er besteht aus einer oder mehreren, mit üblichen mathematischen Methoden festgelegten Mengen von Objekten und darauf definierten Operationen.

Eine *Datenstruktur* ist eine Realisierung der Objektmengen eines ADT mit den Mitteln einer Programmiersprache, z.B. als Kollektion von Variablen verschiedener Datentypen. Man geht häufig nicht ganz bis auf die programmiersprachliche Ebene hinunter und beschreibt eine Datenstruktur nur soweit, daß die endgültige Festlegung mit Mitteln einer Programmiersprache nicht mehr schwierig ist. Man kann eine Datenstruktur auch als Speicherstruktur auffassen, nämlich als Abbild der im mathematischen Sinne idealen Objektmengen eines ADT im Speicher eines realen Rechners. Zur Realisierung oder, wie man auch sagt, zur *Implementierung* eines ADT gehört aber nicht nur die Wahl einer Datenstruktur, sondern auch die Angabe von Algorithmen (Prozeduren und Funktionen) für die Operationen des ADT.

Die begriffliche Unterscheidung zwischen ADT und Datenstruktur wird in diesem Buch nicht immer streng durchgehalten. Wir sprechen manchmal von der *Implementierung einer Datenstruktur* und meinen damit eigentlich die Implementation eines ADT durch eine Datenstruktur. Wir möchten aber ausdrücklich betonen, daß der Begriff Datenstruktur sich stets auf Objekte der realen Welt und Operationen mit ihnen, nicht auf ideale Objekte der Mathematik bezieht.

Wir werden im folgenden die wichtigsten elementaren ADT (lineare Listen, Stapel, Schlangen, Bäume, Mengen) und mögliche Implementationen besprechen.

1.5 Lineare Listen

Lineare Listen basieren auf dem in der Mathematik wohlbekannten Konzept einer endlichen Folge von Elementen eines bestimmten Grundtyps. Man denke etwa an eine endliche Folge ganzer oder reeller Zahlen. Für eine endliche Folge von Zahlen spricht man üblicherweise vom ersten, zweiten und allgemein vom i -ten Element und bezeichnet sie mit a_1, a_2 und a_i . Man kann an eine Folge ein Element anhängen, ein Element an einer bestimmten Stelle einfügen oder entfernen und aus zwei Folgen durch „Hintereinanderhängen“ (Verketten) eine neue Folge machen. Es ist ferner üblich, auch die leere Folge explizit zuzulassen.

Entsprechend kann man den ADT „(lineare) Liste“ wie folgt definieren. Die Menge der Objekte ist die Menge aller endlichen Folgen von Elementen eines gegebenen Grundtyps. Strenggenommen müßte man für jeden Grundtyp einen eigenen ADT angeben. Wir werden das nicht tun, sondern uns den jeweiligen Grundtyp beliebig, aber fest gegeben denken. Wir setzen allerdings meistens voraus, daß der Grundtyp wenigstens zwei Komponenten hat, eine ganzzahlige Schlüsselkomponente und eine Komponente, die die „eigentliche“ Information enthält. Das heißt, mögliche Grundtypen sind wie folgt vereinbart:

```

type
  Grundtyp = record
    key : integer;
    info : {infotype}
    {eventuell weitere Komponenten}
  end

```

Wir beschreiben eine lineare Liste L mit $N \geq 1$ Elementen durch $L = \langle a_1, \dots, a_n \rangle$; $\langle \rangle$ bezeichnet die leere Liste. Folgende Operationen mit linearen Listen werden betrachtet.

Einfügen(x, p, L): Das Einfügen eines neuen Elementes x (vom jeweiligen Grundtyp) in die Liste L an der Position p ; alle Elemente ab Position p rücken dabei um eine Position nach hinten (man sagt auch: nach rechts). Diese Operation verändert die Liste L zur Liste L' wie folgt. Ist $L = \langle a_1, \dots, a_n \rangle$ und $1 \leq p \leq n$, so ist das Ergebnis die Liste $L' = \langle a_1, \dots, a_{p-1}, x, a_p, \dots, a_n \rangle$; ist $L = \langle \rangle$ und $p = 1$, so ist $L' = \langle x \rangle$ das Ergebnis der Einfügeoperation. Ist $p = n + 1$, so ist $L' = \langle a_1, \dots, a_n, x \rangle$ das Ergebnis. In allen anderen Fällen ist das Ergebnis undefiniert.

Entfernen(p, L): Das Entfernen eines Elementes an der Position p macht aus der Liste $L = \langle a_1, \dots, a_{p-1}, a_p, a_{p+1}, \dots, a_n \rangle$ die Liste $L' = \langle a_1, \dots, a_{p-1}, a_{p+1}, \dots, a_n \rangle$, falls $1 \leq p \leq n$. Sonst ist die Operation *Entfernen* undefiniert.

Suchen(x, L): Diese Operation liefert die Position des Elementes x in der Liste L , falls x in L vorkommt, und 0 sonst. Kommt x mehr als einmal in L vor, wird die von links oder von rechts her erste Position geliefert, an der x vorkommt.

Zugriff(p, L): Diese Operation liefert das Element a_p an der p -ten Position in $L = \langle a_1, \dots, a_n \rangle$, falls $1 \leq p \leq n$. Sonst ist die Operation undefiniert.

Wir wollen uns zunächst mit diesen Operationen begnügen. Je nach Anwendungsfall kann es aber sinnvoll sein, weitere Operationen mit linearen Listen vorzusehen. Das können beispielsweise sein: Eine Funktion, die prüft, ob eine Liste L leer ist oder nicht, die Operation des Hintereinanderhängens (Verkettens) zweier linearer Listen, das Bilden von Teillisten oder das Ausgeben (Drucken) aller Elemente einer linearen Liste nach aufsteigenden Positionen.

Sehr oft möchte man auch statt der oben angegebenen Einfüge- und Entferne-Operationen Elemente nicht an einer bestimmten, explizit gegebenen Position, sondern nur abhängig vom Wert (der Schlüsselkomponente) des Elementes einfügen oder entfernen können. Wir bezeichnen diese Operationen mit

$$\text{Einfügen}(x, L) \quad \text{und} \quad \text{Entfernen}(x, L).$$

Alle bisher genannten Operationen operieren auf bereits bestehenden linearen Listen. Es ist meistens üblich, wenigstens eine Operation explizit vorzusehen, die eine lineare Liste erzeugt, die Initialisierung einer linearen Liste als leere Liste. Natürlich könnte man auch die Initialisierung nichtleerer linearer Listen zu einer gegebenen, nichtleeren Menge von Elementen des Grundtyps explizit vorsehen. Andererseits lassen sich solche Listen aber offensichtlich aus der anfangs leeren Liste durch iteriertes Einfügen sämtlicher Elemente erzeugen.

Wir geben jetzt mögliche Implementationen linearer Listen an. Dabei kommt es uns nicht so sehr darauf an, sämtliche für lineare Listen interessanten Operationen programmtechnisch zu realisieren, als vielmehr darauf, die Auswirkungen einer bestimmten Datenstrukturwahl auf die Komplexität der Operationen exemplarisch zu zeigen. Man kann die zahlreichen möglichen Implementation linearer Listen in zwei Klassen einteilen.

1. *Sequentiell gespeicherte lineare Listen*: Hier sind die Listenelemente in einem zusammenhängenden Speicherbereich so abgelegt, daß man — wie bei Arrays — auf das i -te Element über eine Adreßrechnung zugreifen kann.

2. *Verkettet gespeicherte lineare Listen*: Hier sind die Listenelemente in Speicherzellen abgelegt, deren Zusammenhang durch Zeiger hergestellt wird.

Wir behandeln beide Speicherungsformen getrennt.

1.5.1 Sequentielle Speicherung linearer Listen

Wir wählen als Datenstruktur zur Implementation sequentiell gespeicherter linearer Listen ein Array von Elementen des Grundtyps.

```

const
    maxelzahl = {genügend groß gewählte Konstante};
type
    Liste = record
        element: array [0 .. maxelzahl] of Grundtyp;
        elzahl: integer
    end
```

Eine lineare Liste ist dann gegeben durch eine Variable

var *L*: *Liste*

L.elzahl ist die Anzahl der Listenelemente. Falls diese Zahl nicht 0 und kleiner oder gleich der maximalen Elementzahl *maxelzahl* ist, sind

$$L.element[1], \dots, L.element[elzahl]$$

die Listenelemente an den Positionen $1, \dots, elzahl$. Wir haben im Array der Elemente des Grundtyps eine 0-te Position als uneigentliche Listenposition vorgesehen, weil wir so die Suchoperationen besonders bequem implementieren können. Vor Beginn der Suche nach *x* schreiben wir das gesuchte Element *x* an diese Position. Damit wirkt *x* als sogenannter *Stopper* im Falle einer erfolglosen Suche.

```
function Suchen (x: Grundtyp; L: Liste) : integer;
{ liefert die von rechts her erste Position, an der x in L
  vorkommt, und den Wert 0, falls x in L nicht vorkommt }
var
  pos: integer;
begin
  L.element[0] := x;
  pos := L.elzahl;
  while L.element[pos] ≠ x do
    pos := pos - 1;
  Suchen := pos
end {Suchen}
```

Wird ein Element durch seinen Schlüssel eindeutig identifiziert, genügt es natürlich

$$L.element[0].key := x.key \quad \text{statt} \quad L.element[0] := x$$

und

$$L.element[pos].key \neq x.key \quad \text{statt} \quad L.element[pos] \neq x$$

zu schreiben.

Wir geben noch die Prozeduren zum Einfügen und Entfernen eines Elementes für den Fall an, daß die Position, an der ein Element eingefügt bzw. entfernt werden soll, gegeben ist. Sie zeigen, daß es im allgemeinen nötig ist, für ein neu einzufügendes Element zunächst Platz zu schaffen und eine durch Entfernen eines Elementes entstehende Lücke durch Verschieben von Elementen wieder zu schließen.

```
procedure Einfügen (x: Grundtyp; p: integer; var L: Liste);
{ liefert die durch Einfügen von x an Position p in L entstehende
  Liste, wenn p eine gültige Position innerhalb L oder die Position
  unmittelbar nach Listenende ist, und eine Fehlermeldung sonst }
var
  pos: integer;
begin
  if L.elzahl = maxelzahl
  then Fehler ('Liste voll')
```

```

else
  if ( $p > L.elzahl+1$ ) or ( $p < 1$ )
    then Fehler ('ungültige Position')
  else
    begin
      for  $pos := L.elzahl$  downto  $p$  do {verschieben}
         $L.element[pos + 1] := L.element[pos]$ ;
       $L.element[p] := x$ ;
       $L.elzahl := L.elzahl + 1$ 
    end
  end {Einfügen}

```

```

procedure Entfernen ( $p$ : integer; var  $L$ : Liste);
{entfernt das Element an Position  $p$  aus der Liste  $L$ , falls  $p$  eine
 gültige Position innerhalb  $L$  ist, und liefert eine Fehlermeldung
 sonst}
var
   $pos$ : integer;
begin
  if  $L.elzahl = 0$ 
    then Fehler ('Liste ist leer')
  else
    if ( $p > L.elzahl$ ) or ( $p < 1$ )
      then Fehler ('ungültige Position')
    else
      begin
         $L.elzahl := L.elzahl - 1$ ;
        for  $pos := p$  to  $L.elzahl$  do {verschieben}
           $L.element[pos] := L.element[pos + 1]$ 
        end
      end
    end {Entfernen}

```

Um in eine sequentiell gespeicherte Liste der Länge N ein neues Element einzufügen oder ein Element zu entfernen, müssen offenbar im ungünstigsten Fall $\Omega(N)$ Elemente verschoben werden. Der günstigste Fall liegt vor, wenn nur am Ende eingefügt und entfernt wird; dann sind keine Verschiebungen notwendig. Wenn man annimmt, daß jede der N möglichen Positionen gleichwahrscheinlich ist, kann man erwarten, daß im Mittel etwa die Hälfte der Elemente verschoben werden muß. Das Einfügen und Entfernen eines Elementes erfordert bei sequentieller Speicherung einer linearen Liste also sowohl im Mittel wie im schlechtesten Fall $\Omega(N)$ Schritte. Dabei spielt es keine Rolle, ob die Einfüge- bzw. Entferne-Position explizit gegeben ist oder mit Hilfe der Suchoperation zunächst gefunden werden muß. Denn ist der Schlüssel eines Elementes gegeben, muß man ebenfalls im Mittel und im schlechtesten Fall $\Theta(N)$ Schritte ausführen, um das Element mit diesem Schlüssel in einer Liste der Länge N zu finden bzw. festzustellen, daß es kein Element mit diesem Schlüssel in der Liste gibt.

Sind jedoch die Elemente einer sequentiell gespeicherten linearen Liste nach auf- oder absteigenden Schlüsselwerten sortiert, gibt es effizientere Verfahren zum Suchen eines Elementes. Verfahren zum Sortieren werden in Kapitel 2, Verfahren zum Suchen in sequentiell gespeicherten linearen Listen in Kapitel 3 genauer diskutiert. Wir halten hier nur fest, daß das Einfügen und Entfernen in sequentiell gespeicherten linearen Listen „teuer“ ist, das Suchen aber jedenfalls dann sehr effizient möglich ist, wenn die Liste sortiert ist.

1.5.2 Verkettete Speicherung linearer Listen

Statt die Listenelemente so in einem zusammenhängenden Speicherbereich abzulegen, daß man den Speicherplatz des i -ten Listenelementes durch eine Adreßrechnung leicht bestimmen kann, gehen wir jetzt so vor: Wir speichern zusammen mit jedem Listenelement einen Verweis auf das jeweils nächste Element ab. Die Listenelemente können also beliebig über den Speicher verstreut sein; insbesondere ist es nicht mehr erforderlich, vorab einen Bereich hinreichender Größe zur Aufnahme aller Listenelemente zu reservieren. Der belegte Speicherplatz paßt sich vielmehr dynamisch der jeweiligen aktuellen Größe der Liste an. Man benötigt allerdings nicht nur für die Listenelemente selbst, sondern auch für die Zeiger Speicherplatz.

Eine lineare Liste kann implementiert werden als eine Folge von Knoten; jeder Knoten enthält ein Listenelement des jeweiligen Grundtyps und einen Zeiger auf das jeweils nächste Listenelement. Die Knoten haben also folgenden Typ.

```

type
  Zeiger =  $\uparrow$ Knoten;
  Knoten = record
    dat: Grundtyp;
    next: Zeiger
  end
    
```

Eine Liste $L = \langle a_1, \dots, a_n \rangle$ von n Elementen des jeweiligen Grundtyps kann man wie in Abbildung 1.3 veranschaulichen.

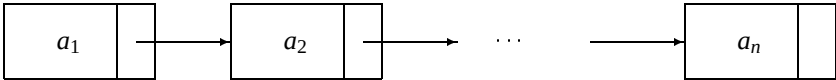


Abbildung 1.3

Wir müssen aber noch festlegen, wie wir den Listenanfang, das Listenende und die leere Liste kennzeichnen. Hier gibt es zahlreiche Möglichkeiten, die alle verschiedene Vor- und Nachteile haben, d h. insbesondere Auswirkungen auf die Implementation der

für Listen auszuführenden Operationen. Wir geben im folgenden einige Möglichkeiten an und diskutieren ausgewählte Listenoperationen exemplarisch.

Eine erste Möglichkeit ist die, eine Liste durch einen Zeiger auf den Listenanfang zu realisieren und das Listenende durch einen **nil**-Zeiger zu markieren. Eine lineare Liste ist also vollständig beschrieben durch eine Variable L vom Typ **Zeiger**.

var L : *Zeiger*

L ist leer genau dann, wenn L den Wert **nil** hat. Eine Position in einer verkettet gespeicherten Liste wird also nicht, wie bei sequentiell gespeicherten Listen, durch eine laufende Nummer, sondern durch einen Zeiger auf ein Listenelement angegeben. Um in der Liste L nach einem Element x des Grundtyps zu suchen, muß man nicht nur den Fall gesondert betrachten, daß L leer ist, sondern auch jedesmal prüfen, ob beim Inspizieren des jeweils nächsten Listenelements nicht schon das Listenende erreicht ist, das durch einen **nil**-Zeiger (graphisch: durch einen Punkt, wie in Abbildung 1.4 zu sehen) markiert ist.

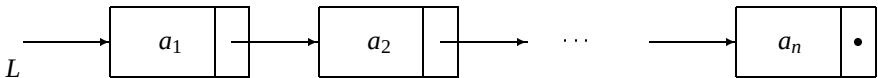


Abbildung 1.4

```

function Suchen ( $x$ : Grundtyp;  $L$ : Zeiger) : Zeiger;
  {liefert einen Zeiger auf das von links her erste Vorkommen des
   Elementes  $x$ , falls  $x$  in  $L$  vorkommt, und den Wert nil sonst}
  var
    pos: Zeiger;
  begin
    if  $L = \text{nil}$ 
    then Suchen := nil
    else
      begin
        pos :=  $L$ ;
        while ( $\text{pos} \uparrow . \text{dat} \neq x$ ) and ( $\text{pos} \uparrow . \text{next} \neq \text{nil}$ ) do
          pos :=  $\text{pos} \uparrow . \text{next}$ ;
        {jetzt ist  $\text{pos} \uparrow . \text{dat} = x$  oder  $\text{pos} \uparrow . \text{next} = \text{nil}$ }
        if  $\text{pos} \uparrow . \text{dat} = x$ 
        then { $x$  gefunden} Suchen := pos
        else { $x$  kommt nicht vor} Suchen := nil
      end
    end {Suchen}

```

Man beachte, daß wir die Position eines Elementes durch einen Zeiger auf einen Knoten realisiert haben, dessen *dat*-Komponente das Element ist.

Diese Implementation einer linearen Liste hat offensichtlich mehrere Schönheitsfehler. Man muß den Fall der leeren Liste und die explizite Abfrage auf das Listenende nicht nur beim Suchen gesondert behandeln. Auch beim Einfügen eines Elementes und beim Entfernen treten zahlreiche Sonderfälle auf.

Alle diese Schwierigkeiten entfallen bei der folgenden Implementation. Eine lineare Liste ist gegeben durch einen Kopfzeiger *head* und einen Schwanzzeiger *tail*, die jeweils auf zwei uneigentliche, sogenannte Dummy-Elemente zeigen; die eigentlichen Listenelemente befinden sich zwischen diesen beiden Dummy-Elementen (vgl. Abbildung 1.5).

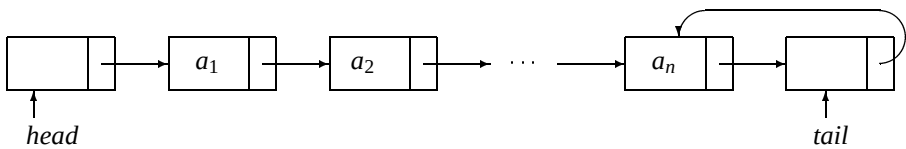


Abbildung 1.5

Die Liste ist durch den Kopf- und Schwanzzeiger gegeben.

var *head, tail*: Zeiger

Wie vorher wird die i -te Position realisiert durch einen Zeiger auf den Knoten, der das i -te Listenelement enthält. Der Schwanzzeiger *tail* markiert also die Position $n + 1$, d. h. die Position nach dem Listenende.

Wir setzen (willkürlich) fest, daß die *next*-Komponente des das Listenende markierenden Dummy-Elementes auf das vorangehende Element zurückverweist. Das erleichtert das Hintereinanderhängen zweier Listen, wie wir weiter unten zeigen werden. Die leere Liste hat also die in Abbildung 1.6 gezeigte Form. Sie wird durch die Prozedur *Initialisiere* erzeugt.

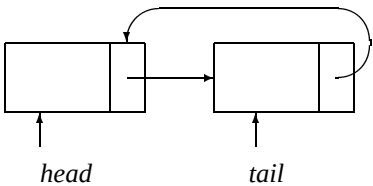


Abbildung 1.6


```

procedure Initialisiere (var head, tail: Zeiger);
begin
    new(head);
    new(tail);
    head↑.next := tail;
    tail↑.next := head
end {Initialisiere}

```

Zum Suchen eines Elementes x vom Grundtyp kann man die schon bei der sequentiellen Speicherung linearer Listen benutzte Stopper-Technik anwenden und das gesuchte Element vor Beginn der Suche in das Dummy-Element am Listeneende schreiben.

```

function Suchen ( $x$ : Grundtyp; head, tail: Zeiger) : Zeiger;
    {liefert einen Zeiger auf das von links her erste Vorkommen
     des Elementes  $x$ , falls  $x$  in der Liste mit Kopfzeiger head und
     Schwanzzeiger tail vorkommt, und den Wert tail sonst}
var
    pos: Zeiger;
begin
    tail↑.dat :=  $x$ ; {Stopper}
    pos := head;
    repeat pos := pos↑.next
    until pos↑.dat =  $x$ ;
    Suchen := pos
end {Suchen}

```

Beim Einfügen und Entfernen eines Elementes an einer gegebenen Position p ist es notwendig, den *next*-Zeiger des Vorgängers des p -ten Knotens der Liste umzulegen; auf diesen Zeiger kann man aber nicht mehr ohne weiteres (in konstanter Zeit) zugreifen, wenn man Position p wie bisher als einen Zeiger auf den Knoten auffaßt, dessen Datenkomponente das p -te Listenelement ist.

Nehmen wir beispielsweise an, daß ein neues Element x an Position p eingefügt werden soll. Die Situation vor dem Einfügen kann graphisch wie in Abbildung 1.7 dargestellt werden.

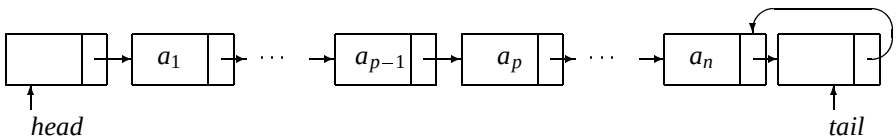


Abbildung 1.7

Nach dem Einfügen wird daraus die Situation von Abbildung 1.8.

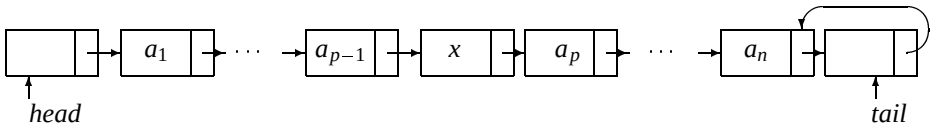


Abbildung 1.8

Die gewünschte Situation kann im Falle des Einfügens durch einen Kunstgriff erreicht werden. Man ersetzt das p -te Element a_p durch x und fügt a_p an der $(p + 1)$ -ten Position ein.

```

procedure Einfügen ( $x$  : Grundtyp;  $p$ ,  $head$  : Zeiger; var  $tail$  : Zeiger);
  {liefert die Liste mit Kopfzeiger  $head$  und Schwanzzeiger  $tail$ , die
   durch Einfügen von  $x$  an der Stelle, auf die  $p$  zeigt, entsteht}
var
  hilf : Zeiger;
begin
  if  $p = tail$ 
    then  $hilf := tail$ 
    else  $hilf := p \uparrow .next$ ;
  new( $p \uparrow .next$ );
   $p \uparrow .next \uparrow .next := hilf$ ;
   $p \uparrow .next \uparrow .dat := p \uparrow .dat$ ;
   $p \uparrow .dat := x$ ;
  if  $p = tail$  {eingefügt an letzter Position}
    then  $tail := tail \uparrow .next$ ;
  if  $hilf = tail$  {eingefügt an vorletzter Position}
    then  $tail \uparrow .next := p \uparrow .next$ 
end {Einfügen}
  
```

Man beachte, daß diese Prozedur das Einfügen eines neuen Elementes x auch dann korrekt bewerkstelligt, wenn p die Position des letzten Elementes oder die Position unmittelbar nach Listeneende (also die Position $tail$) ist.

Das Entfernen eines Elementes an einer gegebenen Position p läßt sich so im allgemeinen nicht durchführen, weil beim Entfernen des letzten Elementes der Zeiger $tail \uparrow .next$ nicht korrekt adjustiert werden kann, wenn man auf den Vorgänger von p in der Liste keinen Zugriff hat. Man muß die Liste vom Anfang an durchlaufen, um den dem Element an Position p vorangehenden Knoten in der Liste zu bestimmen, damit man dessen $next$ -Komponente über p hinweg auf den nächstfolgenden Knoten zeigen lassen kann. Diese Schwierigkeit entfällt, wenn man eine andere Implementation des Positionsbegriffs vornimmt. Statt zu sagen: „Die Position p innerhalb der Liste ist gegeben durch einen Zeiger auf den Knoten mit dat -Komponente a_p , der das p -te Listenelement enthält“, kann man auch sagen: „Die Position p ist gegeben durch einen Zeiger auf den Knoten, dessen $next$ -Komponente einen Zeiger auf den Knoten mit dat -

Komponente a_p enthält.“ Die Position 1 ist also gegeben durch den Zeiger *head* auf das Dummy-Element am Listenkopf usw.

Man „hängt“ also gewissermaßen mit dem Zeiger einen Knoten „zurück“ und schaut auf den nächstfolgenden voraus, um das gegebenenfalls notwendige Umlegen von Zeigern zu erleichtern. Wir verzichten darauf, Prozeduren zum Einfügen, Entfernen usw. für lineare Listen anzugeben, wenn der Positionsbegriff wie zuletzt beschrieben implementiert wird. Vielmehr begnügen wir uns damit zu zeigen, wie man ein Listenelement mit gegebenem Wert x (dessen Position also zunächst bestimmt werden muß) nach dieser Technik des *Zurückhängens mit Vorausschauen* entfernt.

```
procedure Entfernen ( $x$  : Grundtyp; head, tail : Zeiger);
{entfernt den von links her ersten Knoten mit Datenkomponente  $x$ 
 aus einer Liste mit Kopfzeiger head und Schwanzzeiger tail, falls  $x$ 
 in der Liste vorkommt; sonst wird eine Fehlermeldung ausgegeben}
var
  pos : Zeiger;
begin
  pos := head;
  tail↑.dat :=  $x$ ; {Stopper}
  while pos↑.next↑.dat ≠  $x$  do pos := pos↑.next;
  if pos↑.next ≠ tail
    then pos↑.next := pos↑.next↑.next
    else Fehler (' $x$  kommt nicht vor');
  if pos↑.next = tail
    {letztes Element wurde entfernt}
    then tail↑.next := pos
end {Entfernen}
```

Dabei soll die Prozedur *Fehler* das Programm nach der entsprechenden Fehlermeldung beenden. Wir haben hier, wie auch im Falle der anderen Listenoperationen, besonders darauf achten müssen, den *next*-Zeiger des Dummy-Elementes am Listende auf den vorangehenden Knoten zeigen zu lassen. Das macht es möglich, das Hintereinanderhängen (Verketten) zweier Listen in konstanter Schrittzahl auszuführen.

```
procedure Verketten (head1, head2, tail1, tail2 : Zeiger;
  var head, tail : Zeiger);
{liefert zu zwei Listen mit Kopf- und Schwanzzeiger head1, head2, tail1, tail2 eine neue Liste mit Kopfzeiger head und Schwanzzeiger tail, die durch Anhängen der zweiten Liste an das Ende der ersten entsteht}
begin
  head := head1;
  tail1↑.next↑.next := head2↑.next;
  tail := tail2;
  if tail2↑.next = head2 {leere Liste 2}
    then tail↑.next = tail1↑.next
end {Verketten}
```

Um das Einfügen und Entfernen von Listenelementen bei gegebener Position möglichst einfach ausführen zu können, kann man zu jedem Listenelement nicht nur einen Zeiger auf das nächstfolgende, sondern auch auf das jeweils vorangehende Listenelement abspeichern. Man spricht in diesem Fall von *doppelt verketteter Speicherung* einer linearen Liste; entsprechend nennt man die bisher besprochene Form der Speicherung auch *einfach verkettete Speicherung*. In einer doppelt verketteten linearen Liste haben die Knoten also folgendes Format.

```

type
  Zeiger = ↑Knoten;
  Knoten = record
    dat : Grundtyp;
    vor, nach : Zeiger
  end

```

Nehmen wir beispielsweise an, es soll das Element an Position p im Innern der Liste entfernt werden; die Position p sei durch einen Zeiger auf einen Knoten mit Datenkomponente a_p realisiert, wie in Abbildung 1.9 zu sehen.

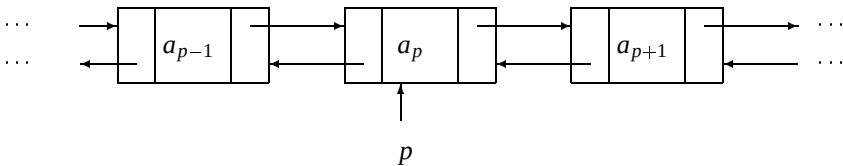


Abbildung 1.9

Das Entfernen wird erreicht durch folgende Zuweisung:

```

 $p↑.vor↑.nach := p↑.nach;$ 
 $p↑.nach↑.vor := p↑.vor;$ 

```

Natürlich kann man auch doppelt verkettete lineare Listen mit und ohne Kopf- und Schwanzzeiger bzw. mit und ohne ein Dummy-Element am Listenanfang oder -ende implementieren.

Eine abschließende, allgemeine Bemerkung zum Entfernen von Listenelementen: Wir haben die nach dem Entfernen nicht mehr benötigten Knoten nicht zur neuen und eventuell anderen Verwendung explizit freigegeben, sondern sie nur aus der die jeweilige Liste realisierenden verketteten Struktur durch Umlegen von Zeigern entfernt. Man kann diese Knoten bei manchen Pascal-Implementationen durch einen Aufruf der Standardprozedur *dispose* explizit freigeben. Man kann sie aber auch in einer eigenen Freiliste sammeln und jedesmal zunächst dort nachsehen, ob man nicht von dieser Freiliste einen Knoten nehmen kann, bevor man einen neuen durch einen Aufruf der Standardprozedur *new* schafft.

Wir fassen einige Varianten verketteter gespeicherter linearer Listen noch einmal stichwortartig zusammen.

Implementation 1: Einfach verkettete Liste; gegeben durch Zeiger auf Listenanfang; Listenende durch **nil**-Zeiger markiert, kein Schwanzzeiger; Position p durch Zeiger auf Knoten mit Datenkomponente a_p realisiert.

Implementation 2: Einfach verkettete Liste; gegeben durch einen Kopf- und einen Schwanzzeiger, die jeweils auf ein Dummy-Element zeigen; Position p durch Zeiger auf Knoten mit Datenkomponente a_p realisiert.

Implementation 3: Wie Implementation 2, aber Position p durch Zeiger auf Knoten realisiert, dessen *next*-Komponente einen Zeiger auf Knoten mit Datenkomponente a_p enthält.

Implementation 4: (Vgl. Abbildung 1.10) Doppelt verkettete lineare Liste, mit Kopfzeiger *head* und Schwanzzeiger *tail*, die auf das erste bzw. letzte Listenelement zeigen; die *vor*-Komponente des ersten und die *nach*-Komponente des letzten Listenelementes haben den Wert **nil**; die Position p ist durch einen Zeiger auf das Listenelement mit Datenkomponente a_p realisiert.

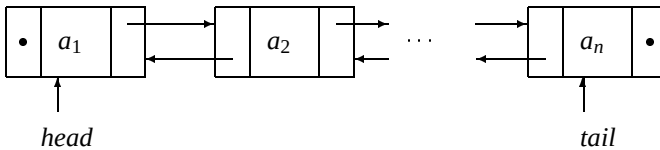


Abbildung 1.10

Wir stellen für diese vier Implementationen die im schlechtesten Fall zur Ausführung ausgewählter Listenoperationen benötigten Schrittzahlen für Listen der Länge N in Tabelle 1.2 zusammen.

Im Gegensatz zur sequentiellen Speicherung bringt es kaum Vorteile, die Elemente einer verketteten gespeicherten linearen Liste etwa nach aufsteigenden Schlüsselwerten in den Knoten zu speichern. Lediglich die erfolglose Suche kann unter Umständen etwas verkürzt werden, weil beim Durchlaufen der Liste vom Anfang her die Suche bereits abgebrochen werden kann, sobald man auf ein Listenelement gestoßen ist, dessen Wert größer als der des gesuchten ist.

Es kann jedoch sinnvoll sein, eine lineare Liste etwa nach abnehmenden Suchhäufigkeiten zu ordnen, wenn diese vorher bekannt sind. Kennt man die (relativen) Suchhäufigkeiten nicht, so kann man verschiedene Strategien implementieren, die mit der Zeit eine für das Suchen günstige Anordnung (nach abnehmenden Suchhäufigkeiten) entstehen lassen. Wir gehen auf diese Strategien in Kapitel 3 genauer ein.

Wenn wir offenlassen wollen, wie eine lineare Liste implementiert wird, schreiben wir:

```
type
  Grundtyp = {der jeweilige Grundtyp};
  Liste = list of Grundtyp
```

	Implementation			
	1	2	3	4
Einfügen eines neuen Elementes am Listenanfang	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
Einfügen eines Elementes an gegebener Position	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
Entfernen eines Elementes an gegebener Position	$\Theta(N)$	$\Theta(N)$	$\Theta(1)$	$\Theta(1)$
Suchen eines Elementes mit gegebenem Wert	$\Theta(N)$	$\Theta(N)$	$\Theta(N)$	$\Theta(N)$
Hintereinanderhängen zweier Listen	$\Theta(N)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$

Tabelle 1.2

Dann können wir eine Liste L einfach als Variable vom Typ *Liste* vereinbaren und diesen Typ auch in den jeweils benötigten Funktionen und Prozeduren zur Manipulation von Listen verwenden.

1.5.3 Stapel und Schlangen

Statt das Einfügen und Entfernen von Elementen an einer beliebigen Position innerhalb einer linearen Liste zuzulassen, genügt es für viele Anwendungen, wenn diese Operationen am Anfang oder am Ende einer Liste ausgeführt werden können. Wir führen für diese Operationen eigene Bezeichnungen ein.

$pushhead(L, x)$: Fügt das Element x am Anfang der Liste L ein.

Wir nehmen also an, daß man vom Anfang der Liste L sprechen kann. Dies kann man auch explizit machen und eine Funktion top mit folgender Bedeutung definieren.

$top(L)$: Liefert den Wert des ersten („obersten“) Elementes der Liste L .

$top(L)$ ist natürlich nur dann definiert, wenn die Liste L nicht leer ist. Sei $leer$ eine Funktion, die für eine Liste L den Wert *true* liefert, wenn L leer ist, und *false* sonst. Dann ist $top(L)$ nicht definiert, falls $leer(L)$ gilt. Es ist jedoch stets, also für jedes L und x ,

$$top(pushhead(L, x)) = x.$$

Entsprechend definiert man eine Funktion $pushtail$ wie folgt:

$pushtail(L, x)$: Fügt das Element x am Ende der Liste L ein.

Operationen zum Entfernen von Elementen am Anfang bzw. Ende von L werden so definiert:

$pophead(L, x)$: Entfernt das erste Element (am Anfang) von L und weist es der Variablen x vom Grundtyp zu; falls $leer(L)$, ist $pophead(L, x)$ nicht definiert.

$poptail(L, x)$: Entfernt das letzte Element (am Ende) von L und weist es der Variablen x vom Grundtyp zu; falls $leer(L)$, ist $poptail(L, x)$ nicht definiert.

Es ist ferner möglich, auch eine Funktion *bottom* (oder: *rear*) zu definieren, die den Wert des letzten (untersten) Elementes einer Liste L liefert.

Werden für lineare Listen nur die Operationen bzw. Funktionen *Initialisieren*, *leer*, *top*, *bottom*, *pushhead*, *pophead*, *pushtail*, *poptail* benötigt, hat man Listen mit kontrollierten Zugriffspunkten. Sie können leicht so implementiert werden, daß alle Operationen in konstanter Schrittzahl ausführbar sind, und zwar gilt das sowohl bei sequentieller als auch bei geketteter Speicherung der Liste L .

Zwei Spezialfälle haben eine besondere Bedeutung und auch einen eigenen Namen erhalten. *Stapel*: Hier sind *Initialisieren*, *leer*, *top*, *pushhead* und *pophead* die einzigen zugelassenen Operationen. *Schlange*: Hier sind *Initialisieren*, *leer*, *top*, *pushtail* und *pophead* die einzigen zugelassenen Operationen.

Im Stapel lassen sich also Elemente nach dem sogenannten LIFO-Prinzip (last in first out) und in Schlangen nach dem FIFO-Prinzip (first in first out) speichern. Die Operationen *pushhead* und *pophead* bei Stapeln werden meistens einfach *push* und *pop* genannt. Ferner nimmt man meistens an, daß die *pop*-Operation nur das oberste Element vom Stapel entfernt, ohne es zugleich einer Variablen vom Grundtyp zuzuweisen. Denn man kann ja, falls nötig, das oberste Element von S mit Hilfe von $top(S)$ zunächst einer Variablen vom Grundtyp zuweisen, bevor man $pop(S)$ ausführt. Bei Schlangen spricht man statt von *pophead* und *pushtail* auch von *dequeue* und *enqueue*.

Wir überlassen es dem Leser, sich eine geeignete Implementation für eine lineare Liste mit kontrollierten Zugriffspunkten, insbesondere also für Stapel und Schlangen, genau zu überlegen. Dabei ist darauf zu achten, daß die jeweiligen Operationen in konstanter Schrittzahl ausführbar sind. Daher ist es beispielsweise nicht ohne weiteres möglich, für einen sequentiell gespeicherten Stapel einfach die Implementation aus Abschnitt 1.5.1 zu übernehmen und dabei das Element an Position 1 als oberstes Element des Stapels anzusehen. Abbildung 1.11 zeigt, wie ein sequentiell gespeicherter Stapel implementiert werden kann.

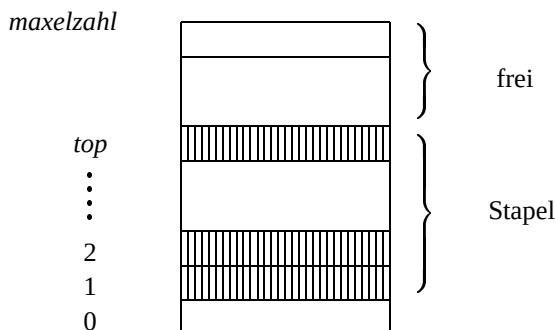


Abbildung 1.11

Werden in einer Schlange etwa ebenso häufig neue Elemente hinten angehängt wie vorne entfernt werden, bleibt die Länge der Schlange nahezu unverändert. Übernimmt man einfach die Implementation aus Abschnitt 1.5.1 für eine sequentiell gespeicherte Schlange, so „kriecht“ die Schlange offenbar im anfangs reservierten Speicherbereich maximaler Länge an das Ende dieses Bereichs, wenn man das vordere Element der Schlange im Array zunächst an Index 1 ablegt. Um zu verhindern, daß man keine Elemente am Ende mehr anfügen kann, wenn die Schlange am Ende angestoßen ist, obwohl vorne noch viel Platz ist, ist es sinnvoll, sich den reservierten Speicherbereich zyklisch geschlossen vorzustellen: Stößt die Schlange am rechten Ende des reservierten Bereichs an, beginnt man, am Anfang dieses Bereichs weitere Elemente einzufügen. Abbildung 1.12 veranschaulicht dies.

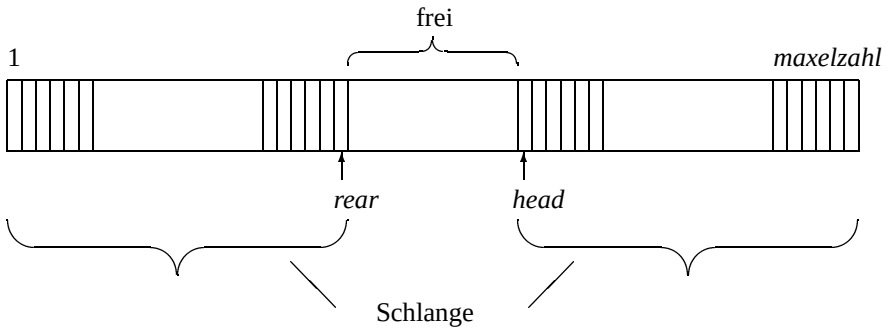


Abbildung 1.12

Wir überlassen es dem Leser, sich genau zu überlegen, wie die Operationen *pophead* und *pushtail* implementiert werden können.

Ein wichtiger Anwendungsfall für Schlangen sind *Warteschlangen* aller Art, z.B. Kunden vor Kassen, Akten vor Sachbearbeitern, Druckaufträge vor Druckern usw. Häufig ordnet man den in eine (Warte-)Schlange einzureihenden Elementen des jeweiligen Grundtyps Prioritäten zu und erwartet, daß Elemente mit höherer Priorität Vorrang vor solchen mit niedrigerer Priorität haben; d h. sie müssen entsprechend eher aus der Schlange entfernt werden. Man spricht in diesem Fall von *Vorrangwarteschlangen* (englisch: *priority queues*). Sie werden in Kapitel 6 genauer behandelt.

Wichtige Anwendungen für Stapel findet man im Zusammenhang mit dem Erkennen und Auswerten wohlgeformter Klammerausdrücke, bei der Realisierung von Unterprogrammaufrufen und der Auflösung rekursiver Funktionen und Prozeduren in iterative. Wir bringen dazu zwei einfache Beispiele.

Beispiel 1: Erkennen wohlgeformter Klammerausdrücke

Wir wollen Zeichenreihen, die aus öffnenden und schließenden Klammern bestehen, daraufhin überprüfen, ob sie wohlgeformt sind, d h. ob sie aus passenden Paaren öffnender und schließender Klammern aufgebaut sind. $((()))$ ist ein wohlgeformter Klam-

merausdruck; ((() ist keiner. Die Menge der wohlgeformten Klammerausdrücke kann man wie folgt induktiv definieren.

- (0) $()$ ist ein wohlgeformter Klammerausdruck.
- (1) Sind w_1 und w_2 wohlgeformte Klammerausdrücke, so ist auch der durch Hintereinanderschreiben von w_1 und w_2 entstehende Ausdruck $w_1 w_2$ ein wohlgeformter Klammerausdruck.
- (2) Mit w ist auch (w) ein wohlgeformter Klammerausdruck.
- (3) Nur die nach (0) bis (2) gebildeten Zeichenreihen sind wohlgeformte Klammerausdrücke.

Wie kann man durch einmaliges, zeichenweises Lesen von links nach rechts feststellen, ob eine nur aus den Zeichen „(“ und „)“ gebildete Zeichenreihe ein wohlgeformter Klammerausdruck ist? Es ist nicht schwer, sich davon zu überzeugen, daß man das feststellen kann, wenn man nach folgender Methode verfährt. Wir benutzen einen Stapel zur Speicherung öffnender Klammern. Immer wenn wir beim Lesen von links nach rechts auf eine öffnende Klammer stoßen, legen wir sie auf dem Stapel ab. Treffen wir auf eine schließende Klammer, sehen wir im Stapel nach, ob dort noch eine öffnende Klammer steht; wenn ja, entfernen wir sie. Wenn nein, gibt es mehr schließende als öffnende Klammern. Im letzten Fall ist die Zeichenreihe kein wohlgeformter Klammerausdruck. Ist am Ende der Stapel leer, ist die gelesene Zeichenreihe ein wohlgeformter Klammerausdruck, sonst nicht.

Wir geben eine genauere Formulierung dieses Verfahrens an, ohne daß wir dabei auf eine spezielle Implementation von Stapeln zurückgreifen wollen. Daher nehmen wir an, daß wir einen Stapel als Liste des gewünschten Grundtyps wie folgt vereinbart haben.

```

type
    Stapel = list of Klammerauf;
var
    S : Stapel
  
```

Wir verwenden nur die für Stapel zugelassenen Operationen *Initialisieren*, *push*, *pop* und *leer* und eine Prozedur zum Lesen des jeweils nächsten Zeichens:

```

Initialisiere S als leeren Stapel;
while noch nicht alle Zeichen gelesen do
    begin lies nächstes Zeichen x;
        if  $x = '('$ 
            then push(S, x)
            else  $\{x = ')'\}$ 
                {hole zugehörige '(' vom Stapel}
                if leer(S)
                    then {kein wohlgeformter Klammerausdruck}
                    else pop(S)
            end; {while}
    if not leer(S)
        then {kein wohlgeformter Klammerausdruck}
  
```

Ein solcher Stapel, der nur gleiche Elemente speichert, kann natürlich auch einfach durch einen Zähler modelliert werden, der die Anzahl der Elemente auf dem Stapel angibt. Wir haben dieses Beispiel gewählt, weil man auf ähnliche Art auch das Erkennen und Auswerten arithmetischer Ausdrücke erledigen kann. Das Verfahren wird allerdings komplizierter, wenn man die üblichen Vorrangsregeln (Punktrechnung geht vor Strichrechnung) beim Auswerten arithmetischer Ausdrücke beachten muß.

Beispiel 2: *Iterative Auswertung einer rekursiv definierten Funktion oder Prozedur*

Wir nehmen den Binomialkoeffizienten als Beispiel einer rekursiv definierten Funktion. Für zwei natürliche Zahlen n und k , mit $0 \leq k \leq n$, ist $\binom{n}{k}$ wie folgt definiert.

$$\binom{n}{k} = \begin{cases} 1, & \text{falls } k = 0 \text{ oder } k = n \\ \binom{n-1}{k-1} + \binom{n-1}{k}, & \text{falls } 0 < k < n \end{cases}$$

$\binom{n}{k}$ ist die Anzahl der verschiedenen Möglichkeiten, k Elemente aus einer Menge von n Elementen auszuwählen. Man kann diese Definition unmittelbar in eine Funktionsdeklaration übersetzen.

```
function bin ( $n, k$ : integer) : integer;
{berechnet die Anzahl der Möglichkeiten,  $k$  aus  $n$  Elementen
 zu wählen, unter der Annahme, daß  $0 \leq k \leq n$  ist}
begin
  if ( $k = 0$ ) or ( $k = n$ )
  then bin := 1
  else bin := bin( $n - 1, k - 1$ ) + bin( $n - 1, k$ )
end {bin}
```

Um dieses Programm abzuarbeiten, muß offenbar einer der zwei rekursiven Funktionsaufrufe zunächst zurückgestellt werden und der andere (auf dieselbe Art) soweit abgearbeitet werden, bis man schließlich bei einem Funktionsaufruf angelangt ist, der unmittelbar den Wert 1 liefert. Erst dann können die vorher zurückgestellten Funktionsaufrufe weiter bearbeitet werden. Entscheiden wir uns (willkürlich) dafür, den ersten Funktionsaufruf zunächst zurückzustellen und den zweiten weiterzubearbeiten, ergibt sich beispielsweise das Berechnungsschema von Tabelle 1.3 bei der Berechnung von $\binom{4}{2}$.

Man erhält also einen Stapel noch nicht erledigter Teilprobleme. Anfangs enthält der Stapel das zu lösende Anfangsproblem, das ist die Berechnung von $\binom{n}{k}$ bzw. die Aufforderung zur Auswertung von $\text{bin}(n, k)$. Ein Problem ist in diesem Fall durch die beiden Argumente n und k vollständig beschrieben. Dann schaut man jeweils nach, ob auf dem Stapel noch unerledigte Probleme liegen. Ist das oberste Problem von der Form $\binom{n}{k}$ mit $0 < k < n$, so ersetzt man es durch zwei (Teil-)Probleme: $\binom{n-1}{k-1}$ wird das zweitoberste und $\binom{n-1}{k}$ das neue oberste Element. Ist das oberste Problem von der Form $\binom{n}{k}$ mit $k = 0$ oder $k = n$, entfernt man es und erhöht das anfangs mit 0 initialisierte Zwischenergebnis um 1. Das wird solange durchgeführt, bis der Problemstapel leer ist.

noch zu berechnen (Problemstapel)	bisheriges Zwischen- ergebnis z
$\binom{4}{2}$	$z = 0$
$\binom{3}{1} + \binom{3}{2}$	
$\binom{3}{1} + \binom{2}{1} + \binom{2}{2}$	
$\binom{3}{1} + \binom{2}{1}$	$z = 1$
$\binom{3}{1} + \binom{1}{0} + \binom{1}{1}$	
$\binom{3}{1} + \binom{1}{0}$	$z = 2$
$\binom{3}{1}$	$z = 3$
$\binom{2}{0} + \binom{2}{1}$	
$\binom{2}{0} + \binom{1}{0} + \binom{1}{1}$	
$\binom{2}{0} + \binom{1}{0}$	$z = 4$
$\binom{2}{0}$	$z = 5$
	$z = 6$

Tabelle 1.3

Als Grundtyp für den Problemstapel können wir in diesem Fall wählen

```

type
  problem = record
    o, u : integer
  end

```

Wir setzen voraus, daß folgende Vereinbarungen getroffen sind:

```

type
  stack = list of problem;
var
  S : stack;
  p, q, x : problem

```

Dann kann die Berechnung der Binomialkoeffizienten $\binom{n}{k}$, d.h. das Abarbeiten der rekursiv deklarierten Funktion $\text{bin}(n, k)$ mit Hilfe des Stapels S und der für Stapel zugelassenen Operationen folgendermaßen ausgeführt werden.

```

Initialisiere  $S$  { mit dem Anfangsproblem  $p$ , für das  $p.o = n$ 
    und  $p.u = k$  gilt };
 $z := 0$ ; { Zwischenergebnis initialisiert }
repeat
     $x := \text{top}(S)$ ;
     $\text{pop}(S)$ ;
    if  $(x.u = 0)$  or  $(x.u = x.o)$ 
        then  $z := z + 1$ 
        else
            begin
                 $q.o := x.o - 1$ ;
                 $q.u := x.u - 1$ ;
                 $\text{push}(q, S)$ ;
                 $q.u := x.u$ ;
                 $\text{push}(q, S)$ 
            end
until  $\text{leer}(S)$ 

```

Dies ist ein einfaches Beispiel für ein allgemeines Prinzip, nach dem sich rekursive Funktionen und Prozeduren mit Hilfe eines Stapel von (Teil-)Problemen abarbeiten lassen. Es kann als *Schema zur Rekursionselimination* wie folgt formuliert werden.

1. *Initialisiere den Problemstapel S mit dem zu lösenden Anfangsproblem.*
2. **repeat**
bearbeite das oberste Problem p von S ;
müssen (bei der Bearbeitung von p) Teilprobleme $p_1, p_2, \dots$ zurückgestellt werden, staple sie auf S
until *Stapel S leer.*

Nach diesem Schema erhält man dann ein gleichwertiges nichtrekursives, also iteratives Programm. Der Leser wird in den folgenden Kapiteln zahlreiche Beispiele finden, auf die dieses Schema zur Rekursionselimination anwendbar ist. So einfach, wie es hier scheint, ist die Anwendung des Schemas in den meisten Fällen allerdings nicht. Es ist oft nicht klar, wie ein Problem so vollständig beschrieben werden kann, daß es zurückgestellt und auf einem Problemstapel abgelegt werden kann. Es ist ferner häufig nicht möglich, das jeweils oberste Problem vollständig zu bearbeiten, weil in die Bearbeitung durchaus Ergebnisse von zunächst zurückgestellten Teilproblemen eingehen können. Bevor man dann mit der Bearbeitung eines Teilproblems beginnt, muß man sich unter Umständen den gesamten, bis zur Zurückstellung erreichten Zwischenzustand der Rechnung genau merken, zunächst das Teilproblem lösen, und dann die (Haupt-) Rechnung fortsetzen. Das oben formulierte Schema zur Rekursionsauflösung ist also eher als ein sehr grober Rahmen, aber keinesfalls als eine mechanisch anwendbare Regel zu verstehen.

1.6 Ausblick auf weitere Datenstrukturen

In diesem Abschnitt wollen wir eine kurze Vorschau auf weitere abstrakte Datentypen und Datenstrukturen geben, die in späteren Kapiteln ausführlich behandelt werden. Dazu gehören insbesondere Mengen.

Mengen unterscheiden sich von (linearen) Listen vor allem dadurch, daß man den Elementen einer Menge üblicherweise keine Ordnungsnummer zuordnet, also nicht vom ersten, zweiten, dritten, ... Element einer Menge spricht. Das mathematische Mengenkonzept geht davon aus, daß man alle Objekte eines gegebenen Universums, die eine bestimmte Eigenschaft haben, zu einer neuen Gesamtheit zusammenfassen kann — zur Menge aller Elemente mit dieser Eigenschaft. Dieses Prinzip zur Bildung von Mengen wird *Komprehensionsschema* genannt. Es ist ein sehr mächtiges Mittel zur Mengenbildung, das allerdings mit gehöriger Vorsicht benutzt werden muß, um widersprüchliche Aussagen über Mengen zu vermeiden. (Ein berühmtes Beispiel ist die Menge U aller Mengen, die sich nicht selbst als Element enthalten. Für U gilt: U enthält sich selbst als Element genau dann, wenn U sich nicht selbst als Element enthält.)

Mathematiker lernen den sinnvollen Gebrauch des Komprehensionsschemas zur Mengenbildung in der Regel durch Erfahrung. Daneben gibt es eine axiomatisierte Mengenlehre als mathematische Theorie. Sie ist auf der Elementbeziehung $\in$ als einzigem Grundbegriff aufgebaut. Dementsprechend könnte man sich einen abstrakten Datentyp Menge gegeben denken durch den Bereich aller Mengen im mathematischen Sinne, zusammen mit einer einzigen, zweistelligen Relation **in**: x **in** S ist wahr genau dann, wenn x ein Element der Menge S ist.

Das Komprehensionsschema als Operation zur Bildung von Mengen ist als Operation eines abstrakten Datentyps zu allgemein; die Elementbeziehung als einzige zugelassene Operation ist in vielen Fällen nicht ausreichend. Als Bausteine in Algorithmen treten durchweg nur endliche Mengen, aber neben der Elementbeziehung zahlreiche weitere Operationen auf. Je nach dem Spektrum der jeweils zugelassenen Operationen werden eigene abstrakte Datentypen mit besonderen Implementationen eingeführt. Wir behandeln einige wichtige Fälle im Kapitel 6 unter dem Stichwort *Mengenmanipulationsprobleme* und begnügen uns hier mit einer groben Übersicht.

Der Datentyp **set**: In Pascal kann man eine variable Anzahl von Elementen desselben Grundtyps zu einem **set** zusammenfassen und Variablen vom **set**-Typ verwenden. Als Grundtypen sind nur einfache Typen, aber nicht der Typ *real* zugelassen. Die Menge der durch den **set**-Typ beschriebenen Werte ist — idealerweise — die Menge aller Teilmengen der Menge der Werte des Grundtyps. In Wirklichkeit sind aber durch die Implementation der Sprache starke Beschränkungen in der Anzahl der zugelassenen Elemente gegeben. Somit ist dieser in die Programmiersprache eingebaute Datentyp von sehr eingeschränktem Wert für die Anwendungen. Wir werden ihn in diesem Buch nicht verwenden.

Wörterbücher (Dictionaries): Als Wörterbuch wird eine Menge von Elementen eines gegebenen Grundtyps bezeichnet, auf der man die Operationen Suchen, Einfügen und Entfernen von Elementen ausführen kann. Darüberhinaus wird stillschweigend vorausgesetzt, daß es eine Operation zur Initialisierung des leeren Wörterbuches gibt. Man nimmt — wie bei linearen Listen — meistens an, daß alle Elemente über einen in der

Regel ganzzahligen Schlüssel identifizierbar sind. Es ist üblich, die Such-, Einfüge- und Entferne-Operation nur vom jeweiligen Schlüssel abhängig zu machen, so daß man zur weiteren Vereinfachung häufig annimmt, daß ein Wörterbuch eine Menge S ganzzahliger Schlüssel ist, auf der folgende Operationen ausgeführt werden.

Suchen(x): Liefert den Wert *true* genau dann, wenn x in S vorkommt, und *false* sonst.

Wenn x in S vorkommt und x Schlüssel eines Elementes mit vielleicht umfangreicher Datenkomponente ist, so soll als Ergebnis der Suchoperation natürlich auch der Zugriff auf die jeweilige Datenkomponente möglich sein.

Einfügen(x): Ersetze S durch $S \cup \{x\}$.
Entfernen(x): Ersetze S durch $S \setminus \{x\}$.

Das Problem, eine geeignete Implementation für Wörterbücher zu finden, also eine Datenstruktur zusammen mit möglichst effizienten Algorithmen zum Suchen, Einfügen und Entfernen von Schlüsseln, nennt man das *Wörterbuchproblem*.

Es ist offensichtlich, daß sequentiell oder verkettet gespeicherte lineare Listen eine mögliche Implementation von Wörterbüchern (also eine Lösung des Wörterbuchproblems) darstellen. Hashverfahren (vgl. Kapitel 4) und Bäume aller Art (vgl. hierzu Kapitel 5) liefern weitere Implementationsmöglichkeiten.

Kollektionen paarweise disjunkter Mengen: In einer Reihe von Anwendungen treten Kollektionen von paarweise disjunkten Mengen auf, für die einige oder alle der folgenden Operationen ausgeführt werden können.

Einfügen(S, x): Fügt das Element x in Menge S ein.
Entfernen(S, x): Entfernt das Element x aus Menge S .
Suchen(S, x): Liefert *true*, wenn Element x in Menge S vorkommt, und *false* sonst.
Find(x): Liefert den Namen derjenigen Menge, die Element x enthält, wenn es eine solche Menge in der Kollektion gibt; sonst ist der Wert undefiniert.

Diese Operationen verändern wohl einzelne Mengen der Kollektion, aber nicht die Kollektion selbst. Die beiden folgenden Operationen dagegen verändern die Kollektion.

Union(A, B, C): Vereinigt die Mengen A und B zur Menge C .

Es wird hier also angenommen, daß die Mengen A und B aus der Kollektion entfernt werden und dafür $C = A \cup B$ neu aufgenommen wird. Für vollständig geordnete Mengen von Schlüsseln kann man in offensichtlicher Weise auch eine Operation *Split* zum Zerteilen einer Menge nach einem bestimmten Schlüssel definieren.

Split(S, x): Zerteilt die Menge S in zwei Mengen A und B mit:
 $A = \{y \mid y \in S \text{ und } y \leq x\}$ und
 $B = \{y \mid y \in S \text{ und } y > x\}$.

Es wird also S aus der Kollektion entfernt und dafür A und B neu aufgenommen. Man nimmt in der Regel an, daß alle Mengen der Kollektion einen eindeutigen Namen besitzen. Ferner wird stillschweigend vorausgesetzt, daß man eine Menge (z.B. als leere oder einelementige Menge) initialisieren kann. Das impliziert insbesondere die Vergabe eines die Menge eindeutig identifizierenden Namens. Das Problem, eine geeignete Implementation für eine Kollektion von Mengen zu finden, so daß sich jede der hier genannten Operationen effizient ausführen läßt, nennen wir das *allgemeine Mengenmanipulationsproblem*. Es wird in Kapitel 6 behandelt.

Ein besonders wichtiger Spezialfall ist der, daß man mit einer Kollektion von lauter einelementigen Mengen startet und dann eine Reihe von Union- und Find-Operationen ausführt. Die Aufgabe, für diesen Fall eine effiziente Implementation zu finden, ist als *Union-Find-Problem* bekannt und jede dazu geeignete Datenstruktur als Union-Find-Struktur. Dieses Problem wird ebenfalls in Kapitel 6 behandelt.

1.7 Skip-Listen

In diesem Abschnitt wird eine mögliche Implementation von Wörterbüchern durch verkettete gespeicherte lineare Listen vorgestellt, die es — anders als die im Abschnitt 1.5.2 diskutierten Varianten — erlaubt, alle drei Wörterbuchoperationen Suchen, Einfügen und Entfernen von Schlüsseln für eine Liste von N Elementen mit hoher Wahrscheinlichkeit in Zeit $O(\log N)$ auszuführen. Diese von W. Pugh [Pug83] vorgeschlagene Datenstruktur mit dem Namen *Skip-Liste* und die zugehörigen Algorithmen zum Suchen, Einfügen und Entfernen sind ein erstes Beispiel für eine *randomisierte* Datenstruktur. Ein weiteres Beispiel bringt Abschnitt 5.3. Der Algorithmus zum Einfügen von Elementen in eine Skip-Liste verwendet einen Zufallsgenerator (Münzwurf). Die Struktur der durch iteriertes Einfügen einer Folge von Schlüsseln in die anfangs leere Liste entstehenden Skip-Liste hängt vom Ausgang zufälliger Münzwürfe ab. Dadurch kann zwar nicht verhindert werden, daß wie im Fall gewöhnlicher, sortierter, linearer Listen, vgl. Abschnitt 1.5.2, Strukturen zur Speicherung von N Schlüsseln entstehen, für die das Ausführen einer einzelnen Wörterbuchoperation Zeit $\Omega(N)$ kostet; dieser Fall ist jedoch sehr unwahrscheinlich. Man kann erwarten, daß eine Skip-Liste entsteht, die es erlaubt, Suchen, Einfügen und Entfernen von Schlüsseln in Zeit $O(\log N)$ auszuführen. Wendet man das durch den Zufall (Münzwurf) gesteuerte Einfügeverfahren mehrfach auf dieselbe Schlüsselfolge, jedesmal beginnend mit der anfangs leeren Liste, iteriert an, so ist der Erwartungswert (gemittelt über alle zufälligen Folgen von Münzwürfen) für die zur Ausführung einer Such-, Einfüge- und Entferneoperation erforderliche Zeit in einer Skip-Liste mit N Elementen von der Größenordnung $O(\log N)$.

Wir stellen im folgenden Abschnitt die Struktur und die zugehörigen Algorithmen für die Wörterbuchoperationen vor und analysieren anschließend ihr Laufzeitverhalten.

1.7.1 Perfekte und randomisierte Skip-Listen

Wir nehmen ohne Einschränkung an, daß die Menge der als Wörterbuch zu organisierenden Daten eine Menge ganzzahliger Schlüssel ist. Die durch den jeweiligen Schlüssel identifizierbare „eigentliche“ Information wird also zur Vereinfachung der Darstellung unterdrückt. Um in einer „gewöhnlichen“ sortierten, verketteten gespeicherten linearen Liste einen Schlüssel x zu suchen, muß man die Liste unter Umständen vom Anfang bis zum Ende vollständig durchlaufen, um x zu finden oder festzustellen, daß x in der Liste nicht vorkommt. Die Suche geht offensichtlich schneller, wenn man Elemente überspringen (englisch: skip) kann. Nehmen wir beispielsweise an, daß die Listenelemente die Schlüssel der Reihe nach in aufsteigender Reihenfolge speichern und es nicht nur von jedem Listenelement einen Zeiger auf das nächste, sondern darüberhinaus auch von jedem zweiten Listenelement einen Zeiger auf das übernächste Element gibt. Abbildung 1.13 (a) zeigt eine solche Liste, die die Schlüssel $\{2, 4, 8, 15, 17, 20, 43, 47\}$ speichert.

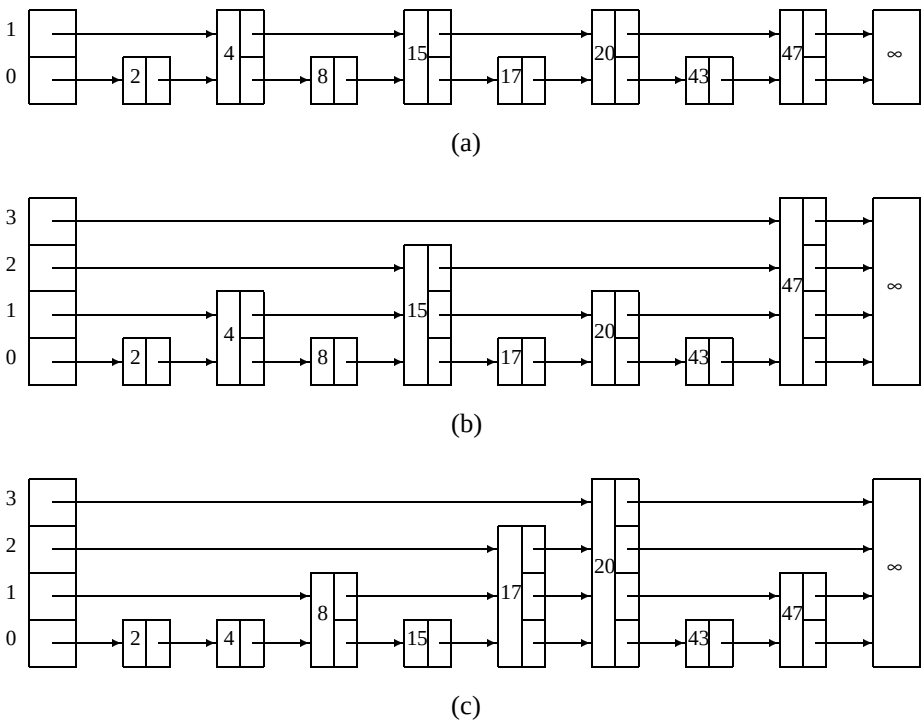


Abbildung 1.13

Jedes Listenelement ist durch einen Zeiger auf Niveau 0 mit dem nächstfolgenden Listenelement verbunden. Ferner ist jedes zweite Listenelement durch einen zusätzli-

chen Zeiger auf Niveau 1 mit dem übernächsten Element verbunden. Am Anfang der Liste befindet sich ein Kopfelement (ohne Schlüssel), das Anfangszeiger auf die Listen der auf Niveau 0 und 1 miteinander verketteten Listenelemente enthält. Am Ende befindet sich ein Endelement mit Schlüssel ∞ , der größer als alle in der Liste auftretenden Schlüssel ist. (Es spielt die Rolle eines Stoppers für die Suche.) Um nach einem Schlüssel x zu suchen, folgt man zunächst den Zeigern auf Niveau 1 bis ein Element angetroffen wird, dessen Schlüssel größer als x ist. Dann wechselt man von dem diesem Element in der Niveau-1-Liste unmittelbar vorangehenden Element auf das Niveau 0 und findet dort entweder x oder stellt fest, daß x in der Liste nicht vorkommt. Bei der Suche nach dem Schlüssel 17 werden also in der Liste von Abbildung 1.13 (a), beginnend mit dem Kopfelement, der Reihe nach die Elemente mit den Schlüssel 4, 15, 20, 17 inspiziert. Man inspiziert also im ungünstigsten Fall nur etwa die Hälfte der Listenelemente. Durch Einführung zusätzlicher Zeiger konnte die Suchzeit in einer verketteten gespeicherten linearen Liste verkürzt werden.

Eine Verallgemeinerung dieser Beobachtung führt zu folgender Definition: Eine *perfekte Skip-Liste* ist eine sortierte, verkettete lineare Liste mit Kopf- und Endelement, für die gilt: Jedes 2^i -te (eigentliche) Element hat einen Zeiger auf das 2^i Positionen weiter rechts stehende Element, für jedes $i = 0, \dots, \lfloor \log N \rfloor$. Dabei ist N die Anzahl der (eigentlichen) Listenelemente. D.h. jedes Element hat einen Zeiger auf Niveau 0 auf das nächstfolgende; die Elemente an den Positionen 2, 4, 6... sind zusätzlich durch Zeiger auf Niveau 1 miteinander verkettet; die Elemente an den Positionen 4, 8, 12... sind zusätzlich durch Zeiger auf Niveau 2 miteinander verkettet usw. Das Kopfelement enthält Anfangszeiger auf die (aufsteigend sortierten) Niveau- i -Listen, für jedes $i = 0, \dots, \lfloor \log N \rfloor$; das Endelement hat einen Schlüssel ∞ , der größer ist als alle in der Liste gespeicherten Schlüssel. Jedes (eigentliche) Listenelement hat also einen Niveau-0-Zeiger, die Hälfte der Elemente hat zusätzlich einen Niveau-1-Zeiger, ein Viertel zusätzlich einen Niveau-2-Zeiger usw. Nehmen wir zur Vereinfachung einmal an, daß N eine Potenz von 2 ist und zählen wir die vom Kopfelement ausgehenden Zeiger nicht mit, so ist die Gesamtzahl der Zeiger einer perfekten Skip-Liste also

$$N + \frac{N}{2} + \frac{N}{4} + \dots + 1 = \sum_{i=0}^{\lfloor \log N \rfloor} \frac{N}{2^i} \leq 2N,$$

d.h. nur doppelt so groß wie in einer „gewöhnlichen“ verketteten gespeicherten linearen Liste. Abbildung 1.13 (b) zeigt ein Beispiel für eine perfekte Skip-Liste mit acht Schlüssel.

Ist N die Anzahl der gespeicherten Schlüssel, so hat jedes Element höchstens $\lfloor \log N \rfloor + 1$ Zeiger. Hat ein Element $p \uparrow i + 1$ Zeiger auf den Niveaus $0, \dots, i$, so sagen wir: $p \uparrow$ ist ein Element mit *Höhe* i . Wir bezeichnen die Höhe von $p \uparrow$ mit $p \uparrow .höhe$.

Für jedes i mit $0 \leq i \leq p \uparrow .höhe$ sei $p \uparrow .next[i]$ der Zeiger von $p \uparrow$ auf das 2^i Positionen weiter rechts stehende Element oder das Endelement, wenn es 2^i Positionen rechts von $p \uparrow$ kein Element mehr gibt. Die maximale Höhe eines Elementes in einer (perfekten) Skip-Liste wird *Listenhöhe* genannt. Dies ist zugleich die Höhe des Kopfelements. Sie hat für eine perfekte Skip-Liste mit N Elementen den Wert $\lfloor \log N \rfloor$. Ist die perfekte Skip-Liste L durch einen Zeiger $L.kopf$ auf das Kopfelement gegeben und hat L die Listenhöhe $L.höhe$, so kann die Suche nach einem Schlüssel x wie folgt beschrieben werden:

```

function Suchen ( $x$  : integer;  $L$  : liste) : Zeiger;
  {liefert einen Zeiger auf das Element mit Schlüssel  $x$ , falls  $x$  in der
   Skip-Liste  $L$  mit Zeiger  $L.kopf$  auf das Kopfelement und
   Listenhöhe  $L.höhe$  vorkommt, und nil sonst}
  var
     $p$  : Zeiger;
     $i$  : integer;
  begin
     $p := L.kopf$ ;
    for  $i := L.höhe$  downto 0 do
      {folge Niveau- $i$ -Zeigern}
      (*) while  $p \uparrow.next[i] \uparrow.key < x$  do
         $p := p \uparrow.next[i]$ ;
        {jetzt ist ( $p = L.kopf$  und  $x \leq p \uparrow.next[0] \uparrow.key$ ) oder
         ( $p \neq L.kopf$  und  $p \uparrow.key < x \leq p \uparrow.next[0] \uparrow.key$ )}
         $p := p \uparrow.next[0]$ ;
      (**) if  $p \uparrow.key = x$ 
        then { $x$  kommt an Position  $p$  in  $L$  vor}
          Suchen :=  $p$ 
        else { $x$  kommt nicht in  $L$  vor}
          Suchen := nil
    end {Suchen}

```

Verfolgen wir beispielsweise die Suche nach dem Schlüssel $x = 17$ in der perfekten Skip-Liste von Abbildung 1.13 (b), so wird der Schlüssel x der Reihe nach mit den folgenden Schlüsseln verglichen (in den mit (*) und (**) markierten Programmzeilen): 47, 15, 47, 20, 17.

Folgt man also Zeigern der perfekten Skip-Liste nach abnehmenden Niveaus, so muß man einem Zeiger auf Niveau i höchstens einmal folgen. Dabei trifft man dann auf ein Element der Höhe i . D h. die Anweisung $p := p \uparrow.next[i]$ wird für festes i höchstens einmal ausgeführt; lediglich die die **while**-Schleife (*) kontrollierende Bedingung kann zweimal geprüft werden, ist aber beim zweiten Mal garantiert nicht erfüllt. Man hätte daher statt der **while**-Schleife auch eine **if**-Anweisung nehmen können, um zu erreichen, daß an der Stelle (*) in der Funktion *Suchen* auf Niveau i vorgerückt wird, bis $x \leq p \uparrow.next[i] \uparrow.key$ ist. Die hier angegebene Realisierung des Suchverfahrens für perfekte Skip-Listen kann jedoch unverändert auch für die später erklärten randomisierten Skip-Listen benutzt werden.

Aus der Beschränkung für die Höhe einer perfekten Skip-Liste folgt natürlich sofort, daß das Suchen stets in $O(\log N)$ Schritten ausgeführt werden kann. Einfügen oder Entfernen eines Schlüssels x würde jedoch eine vollständige Reorganisation der perfekten Skip-Liste erfordern und daher $\Omega(N)$ Schritte benötigen. Will man beispielsweise in die perfekte Skip-Liste von Abbildung 1.13 (b) ein neues kleinstes Element mit Schlüssel 1 einfügen, so müssen sämtliche bisherigen Elemente ihre Höhen ändern, um wieder eine perfekte Skip-Liste zu ergeben. Man verzichtet daher auf die Forderung, daß die Höhen aufeinanderfolgender Elemente dem starren Schema perfekter Skip-Listen unterliegen und sorgt vielmehr dafür, daß Elemente mit verschiedenen Höhen etwa im gleichen Ver-

hältnis wie bei perfekten Skip-Listen auftreten, ihre Verteilung innerhalb der Liste aber zufällig erfolgt. Abbildung 1.13 (c) zeigt ein Beispiel einer Skip-Liste, die für jedes i , $0 \leq i \leq 3$, die gleiche Zahl von Elementen mit Höhe i hat wie die perfekte Skip-Liste von Abbildung 1.13 (b), aber in anderer Weise über die Liste verteilt. Solange Elemente mit großen Höhen relativ selten und solche mit niedrigen Höhen dafür häufiger auftreten, kann man erwarten, daß die Suche nach einem Schlüssel x nach dem für perfekte Skip-Listen angegebenen Verfahren nicht nur weiterhin unverändert durchgeführt werden kann, sondern auch effizient bleibt. Das werden wir im Abschnitt 1.7.2 genauer analysieren. Statt also eine perfekte Skip-Liste zu erzeugen, sorgt man lediglich dafür, daß Elemente mit jeweils verschiedenen Höhen im selben Verhältnis auftreten wie in perfekten Skip-Listen, diese aber gleichmäßig und zufällig über die Liste verteilt werden. Dieser Effekt wird dadurch erreicht, daß man beim Einfügen eines Schlüssels x die Höhe $p \uparrow .höhe$ des Elementes p , das x speichert, unabhängig von allen anderen Elementen zufällig wählt im Bereich $[0, maxhöhe]$ und zwar so, daß die Wahrscheinlichkeit dafür, daß $p \uparrow .höhe = i$ ist, gleich $1/2^{i+1}$ ist:

$$\text{prob}(p \uparrow .höhe = i) = \frac{1}{2^{i+1}}, \quad 0 \leq i \leq maxhöhe.$$

Dabei ist *maxhöhe* eine (global festgesetzte) obere Schranke für die Listenhöhe und damit auch für die Höhe jedes einzelnen Elementes. Der Wert von *maxhöhe* wird orientiert an der Listenhöhe einer perfekten Skip-Liste für N Elemente, wobei N groß genug gewählt wird, um alle je auftretenden Elemente in Skip-Listen mit höchstens N Elementen unterbringen zu können. Man wählt also $maxhöhe = \lfloor \log N \rfloor$ für genügend groß gewähltes N . Um die nachfolgende Analyse zu vereinfachen, ignorieren wir allerdings die Höhenbeschränkung und tun so, als könne die Höhe eines Listenelementes beliebig groß werden. Denn die Wahrscheinlichkeit dafür, daß ein Listenelement eine Höhe hat, die $\lfloor \log N \rfloor$ übersteigt, ist so gering, daß wir sie vernachlässigen können.

Die auf diese Weise durch iteriertes Einfügen in die anfangs leere Liste entstehenden *randomisierten* Skip-Listen heißen (entsprechend dem Vorschlag von Pugh [Pug88]) einfach *Skip-Listen*. 

Nehmen wir also an, wir hätten eine parameterlose Funktion *randomhöhe()*, die bei ihrem Aufruf eine Höhe mit den genannten Eigenschaften liefert. Dann können wir das *Einfügen* eines neuen Schlüssels x in eine Skip-Liste L mit Kopfzeiger $L.kopf$ und Höhe $L.höhe$ wie folgt beschreiben. Wir suchen zunächst nach x . Da wir annehmen, daß x in der Skip-Liste noch nicht vorkommt, endet die Suche erfolglos beim Element mit dem größten Schlüssel, der kleiner oder gleich x ist. (Falls x kleiner als alle Schlüssel in der Liste L ist, endet die Suche schon beim Kopfelement.) Hinter dieses Element wird ein neues Element $p \uparrow$ mit Schlüssel x und zufällig gewählter Höhe $p \uparrow .höhe$ eingeschoben. Es müssen dazu alle über $p \uparrow$ hinwegführenden Niveau- i -Zeiger, mit $0 \leq i \leq p \uparrow .höhe$ verändert werden. Damit das möglich ist, sammelt man während der Suche nach x die Quellen aller dieser Zeiger in einem Zeiger-Array *update*: Für jedes i enthält *update*[i] einen Zeiger auf das am weitesten rechts liegende Listenelement mit Höhe i links von der Einfügestelle. Ist die $p \uparrow$ zufällig zugewiesene Höhe $p \uparrow .höhe$ größer als die bisherige Listenhöhe $L.höhe$, müssen das Kopfelement durch zusätzliche Zeiger auf $p \uparrow$ und $L.höhe$ entsprechend verändert werden. Genauer kann das Verfahren wie folgt beschrieben werden:

```

procedure Einfügen ( $x : \text{integer}$ ; var  $L : \text{Liste}$ );
{fügt Schlüssel  $x$  in Skip-Liste  $L$  mit Zeiger  $L.kopf$  auf das
Anfangselement und Listenhöhe  $L.höhe$  ein}
var
   $update : \text{array } [0 \dots maxhöhe] \text{ of Zeiger}$ ;
   $p : \text{Zeiger}$ ;
   $i : \text{integer}$ ;
   $neuehöhe : 0 \dots maxhöhe$ ;
begin
   $p := L.kopf$ 
  for  $i := L.höhe$  downto 0 do
    begin
      while  $p \uparrow .next[i] \uparrow .key < x$  do
         $p := p \uparrow .next[i]$ ;
         $update[i] := p$ 
      end {for};
       $p := p \uparrow .next[0]$ ;
    if  $p \uparrow .key = x$ 
      then {Schlüssel  $x$  kommt schon vor}
      else {einfügen}
        begin
           $neuehöhe := randomhöhe()$ ;
          if  $neuehöhe > L.höhe$ 
            then
              begin
                {neues Element direkt mit Kopfelement verknüpfen
                und Listenhöhe adjustieren}
                for  $i := L.höhe + 1$  to  $neuehöhe$  do
                   $update[i] := L.kopf$ ;
                   $L.höhe := neuehöhe$ 
                end;
                {schaffe neues Element mit Höhe  $neuehöhe$  und Schlüssel  $x$ }
                 $new(p)$ ;
                 $p \uparrow .höhe := neuehöhe$ ;  $p \uparrow .key := x$ ;
                for  $i := 0$  to  $neuehöhe$  do
                  {schiebe  $p \uparrow$  in die Niveau- $i$ -Listen jeweils unmittelbar nach
                  dem Element  $update[i] \uparrow$  ein}
                  begin
                     $p \uparrow .next[i] := update[i] \uparrow .next[i]$ ;
                     $update[i] \uparrow .next[i] := p$ 
                  end
                end
              end {Einfügen}

```

Das Entfernen eines Elementes mit Schlüssel x aus einer Skip-Liste L erfolgt völlig analog: Zunächst sucht man nach x . Dabei benutzt man wieder ein Array $update$ und merkt sich für jedes i in $update[i]$ einen Zeiger auf das rechteste Element in L links von

x mit Höhe i . Dann kann man das Element $p \uparrow$ mit Schlüssel x aus allen Niveau- i -Listen, $0 \leq i \leq p \uparrow .höhe$, entfernen. Falls nach Entfernen von $p \uparrow$ die Listenhöhe gesunken ist, muß man sie entsprechend adjustieren. Um festzustellen, ob dieser Fall vorliegt, muß man dem Zeiger des Kopfelementes auf dem höchsten Niveau folgen und nachsehen, ob er noch auf ein eigentliches Listenelement oder auf das Endelement mit Schlüssel ∞ zeigt. D.h. die Listenhöhe kann um 1 verringert werden, wenn gilt

$$L.kopf \uparrow .next[L.höhe] \uparrow .key = \infty.$$

Genauer kann das Verfahren zum Entfernen eines Schlüssels x aus einer Skip-Liste L wie folgt beschrieben werden:

```

procedure Entfernen ( $x : integer$ ; var  $L : Liste$ );
var
     $update : array [0 .. maxhöhe] \text{ of Zeiger}$ ;
     $p : Zeiger$ ;
     $i : integer$ ;
begin
     $p := L.kopf$ ;
    for  $i := L.höhe$  downto 0 do
        begin
            while  $p \uparrow .next[i] \uparrow .key < x$  do
                 $p := p \uparrow .next[i]$ ;
             $update[i] := p$ 
            end; {for}
         $p := p \uparrow .next[0]$ ;
        if  $p \uparrow .key = x$ 

            then
                {Element  $p \uparrow$  entfernen und ggfs. Listenhöhe adjustieren}
            begin
                for  $i := 0$  to  $p \uparrow .höhe$  do
                    {entferne  $p \uparrow$  aus Niveau- $i$ -Liste}
                     $update[i] \uparrow .next[i] := p \uparrow .next[i]$ ;
                    while ( $L.höhe \geq 1$ ) and ( $L.kopf \uparrow .next[L.höhe] \uparrow .key = \infty$ ) do
                         $L.höhe := L.höhe - 1$ 
                    end
                end {Entfernen}
            end

```

Die Verfahren zum Einfügen und Entfernen von Elementen in Skip-Listen haben eine Eigenschaft, die sie von den entsprechenden Verfahren für die im Kapitel 5 ausführlich behandelten Suchbäume, insbesondere von den Verfahren für natürliche Bäume, ganz wesentlich unterscheidet: Eine Skip-Liste, aus der ein Element entfernt wurde, hat dieselbe Struktur, als wäre das Element niemals dagewesen. Daher bleibt auch nach einer längeren Folge von Updates die „Zufälligkeit“ der Struktur erhalten. In diesem Sinne sind Skip-Listen unabhängig von der Erzeugungshistorie. Anders als etwa natürliche Suchbäume können Skip-Listen durch iteriertes Einfügen und Entfernen von Elementen nicht „degenerieren“.

1.7.2 Analyse

Das Einfügeverfahren für Skip-Listen benutzt eine Funktion *randomhöhe()*, die eine zufällige Höhe erzeugt und zwar so, daß gilt: Die Wahrscheinlichkeit dafür, daß die Höhe 0 erzeugt wird, ist $1/2$ und für jedes $i \geq 0$ ist die Wahrscheinlichkeit dafür, daß die Höhe $i + 1$ erzeugt wird, halb so groß wie die, daß die Höhe i erzeugt wird. Also ist die Wahrscheinlichkeit dafür, daß genau die Höhe i erzeugt wird, gleich $1/2^{i+1}$, und die Wahrscheinlichkeit dafür, daß eine Höhe $\geq i$ erzeugt wird, gleich $1/2^i$, für jedes $i \geq 0$. Bei der Implementation des Einfügeverfahrens haben wir zur Vereinfachung zusätzlich vorausgesetzt, daß die von *randomhöhe()* gelieferte Höhe stets kleiner oder gleich einer global festgesetzten maximalen Höhe *maxhöhe* bleibt. Weil die Wahrscheinlichkeit, ein Element mit einer Höhe von etwa 15 zu erzeugen, schon „praktisch“ gleich Null ist, werden wir in der Analyse von dieser globalen Höhenbeschränkung der Einfachheit halber zunächst absehen. Zur Realisierung von *randomhöhe()* setzen wir eine parameterlose Funktion *random()* voraus, die unabhängige und gleichverteilte Zufallszahlen im Bereich $(0, 1)$ liefert. Dann erzeugt die folgende Funktion *randomhöhe()* Höhen im Bereich $[0, \text{maxhöhe}]$ mit exponentiell (mit dem Faktor $1/2$) abnehmenden Wahrscheinlichkeiten.

```
function randomhöhe() : integer;
var
    höhe : integer;
begin
    höhe := 0;
    while (random() <  $\frac{1}{2}$ ) and (höhe < maxhöhe) do
        höhe := höhe + 1;
    randomhöhe := höhe
end
```

Erzeugt man die Höhen mit dieser Funktion *randomhöhe()*, so ist der Erwartungswert für die Anzahl der Elemente mit Höhe i oder größer in einer Liste mit N Elementen gleich $N/2^i$, für jedes $i \geq 0$. Die Höhenverteilung der Elemente stimmt also mit der von perfekten Skip-Listen überein.

Wir schätzen nun die *Suchkosten* in einer Skip-Liste ab. Nach dem in Abschnitt 1.7.1 angegebenen Verfahren beginnen wir die Suche beim Kopfelement der Liste und führen dann jeweils einen der folgenden beiden Schritte aus: Entweder folgen wir einem Zeiger auf dem gerade aktuellen Niveau von einem Element zum nächstfolgenden oder aber wir gehen innerhalb eines Elementes von einem Niveau zum nächstniedrigeren über. Tritt der erste Fall ein, d.h. folgen wir einem Zeiger auf Niveau i , so hat das Element, auf das dieser Zeiger zeigt, die Höhe i , für jedes $i \geq 0$. Natürlich gibt es auch Zeiger auf Höhe i , die auf Elemente mit Höhe $> i$ zeigen, aber denen *folgt* unser Algorithmus nicht (er *prüft* sie höchstens), weil für solche Zeiger ein Zeiger auf dasselbe Element mit größerer Höhe ebenfalls bereits *geprüft* wurde. Das Entlanglaufen von Niveau- i -Zeigern zu Elementen mit Höhe i , $i \geq 0$, und das Herabsetzen des aktuellen Niveaus wird solange durchgeführt, bis das Niveau 0 erreicht ist und dort der gesuchte Schlüssel gefunden wird oder aber festgestellt wird, daß der gesuchte Schlüssel in der Skip-Liste nicht vorkommt. Abbildung 1.14 zeigt ein Beispiel eines solchen Suchpfades nach dem Schlüssel 16 in der Skip-Liste von Abbildung 1.13 (c).

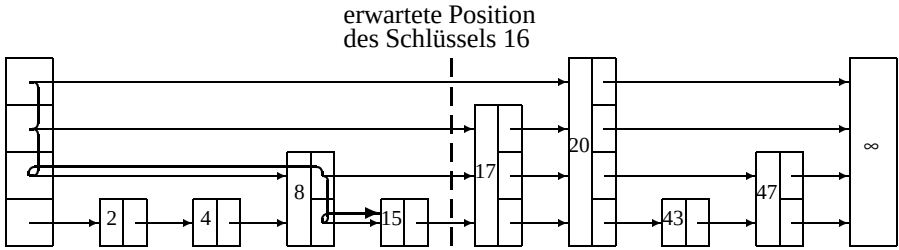


Abbildung 1.14

Um den Erwartungswert für die Länge des Suchpfades zu berechnen, verfolgen wir den Suchpfad rückwärts, beginnend beim Niveau-0-Zeiger auf das Element, das den gesuchten Schlüssel enthält oder das, falls der gesuchte Schlüssel nicht vorkommt, den kleinsten Schlüssel enthält, der größer als der gesuchte ist. Dazu nehmen wir allgemeiner an, daß wir uns innerhalb eines Elementes $p \uparrow$ auf dem Niveau i befinden, $i \geq 0$, und fragen uns, was der Erwartungswert $EC(k)$ für die Länge eines Suchpfades ist, der vom Niveau i in $p \uparrow$ aus gerechnet nach links zurückverfolgt k Niveaus hinaufsteigt. $EC(k)$ ist also die Anzahl der Schritte, die man vom Niveau i in $p \uparrow$ aus beim Zurückverfolgen des Suchpfades benötigt, um erstmals auf ein k Niveaus über Niveau i liegendes Niveau zu gelangen. Als Schritt zählen wir dabei jeweils das Heraufklettern um ein Niveau und das Zurücklaufen eines Niveau- i -Zeigers von einem Element mit Höhe i zu seinem Ursprung. Wir machen keine Annahmen über die Höhe von $p \uparrow$ oder die Höhen der Elemente links von $p \uparrow$. Wir setzen allerdings voraus, daß $p \uparrow$ nicht das Kopfelement der Skip-Liste ist. (Diese letzte Annahme ist gleichbedeutend mit der Annahme, daß die Liste nach links unbegrenzt ist.)

Wir haben angenommen, daß wir uns in $p \uparrow$ auf Niveau i befinden. Also ist $p \uparrow \text{.höhe} \geq i$ und mit Wahrscheinlichkeit von jeweils $1/2$ ist $p \uparrow \text{.höhe} = i$ und $p \uparrow \text{.höhe} > i$ aufgrund unseres Verfahrens zur Erzeugung einer zufälligen Höhe. D.h. sind wir mit unserem durch zufällige Münzwürfe gesteuerten Heraufsetzen der Höhe bereits bis zur Höhe i gekommen, so ist die Wahrscheinlichkeit dafür, daß wir aufhören oder fortfahren, die Höhe hinaufzusetzen, jeweils $1/2$.

Fall 1: $i = p \uparrow \text{.höhe}$. Das impliziert, daß der zurückverfolgte Suchpfad vom Element $p \uparrow$ zu einem Element mit wenigstens Höhe i geht und von diesem Element noch immer k Niveaus hinaufklettern muß.

Fall 2: $i < p \uparrow \text{.höhe}$. Das impliziert, daß der zurückverfolgte Suchpfad $p \uparrow$ wenigstens ein Niveau hinaufklettert und nicht einen Niveau- i -Zeiger zurückläuft. Also muß der Suchpfad von diesem neuen Niveau $i + 1$ aus gerechnet noch $k - 1$ Niveaus hinaufsteigen, um beim Zurückverfolgen insgesamt k Niveaus hinaufzusteigen.

Damit erhalten wir die folgende Rekursionsgleichung für $EC(k)$:

$$\begin{aligned}
 EC(k) &= \frac{1}{2} \cdot ((\text{Kosten, um einen Niveau-}i\text{-Zeiger zurückzulaufen}) + EC(k)) \\
 &+ \frac{1}{2} \cdot ((\text{Kosten, um vom Niveau } i \text{ zu Niveau } i + 1 \text{ hinaufzusteigen}))
 \end{aligned}$$

$$\begin{aligned}
 &+ EC(k-1)) \\
 = &\frac{1}{2} \cdot (1 + EC(k)) + \frac{1}{2} (1 + EC(k-1))
 \end{aligned}$$

Es gilt also

$$EC(k) = 2 + EC(k-1).$$

Da die Länge eines kürzesten Pfades, der beim Zurückverfolgen 0 Niveaus hinaufklettert, natürlich 0 ist, gilt

$$EC(0) = 0.$$

Die Rekursionsformel hat die Lösung

$$EC(k) = 2k.$$

Wir verwenden dieses Ergebnis, um den Erwartungswert für die Länge eines Suchpfades in einer Skip-Liste mit Länge N zu berechnen. Dazu zerlegen wir den (zurückverfolgten) Suchpfad in drei Teile.

Teil 1: Zuerst betrachten wir den Teil des Suchpfades, den man ausgehend vom Niveau 0 im Element mit dem gesuchten Schlüssel zurücklaufen muß, um $\log_2 N - 1$ Niveaus hinaufzusteigen. Den Erwartungswert für die Länge dieses Teils des Suchpfades haben wir gerade berechnet. Er ist $EC(\log_2 N - 1) = 2(\log_2 N - 1)$.

Teil 2: Dann schätzen wir ab, wieviele Knoten mit Höhe wenigstens $\log_2 N - 1$ es in der Skip-Liste höchstens gibt. Denn sind wir beim Zurückverfolgen des Suchpfades bereits auf Niveau $\log_2 N - 1$ angekommen, so wird man im weiteren Verlauf sicher noch höchstens so viele Zeiger zurückverfolgen müssen, wie es insgesamt Knoten mit Höhe mindestens $\log_2 N - 1$ in der Skip-Liste gibt. Offenbar ist der Erwartungswert der Anzahl der Elemente mit Höhe mindestens $\log_2 N - 1$ gleich dem Produkt aus der Anzahl N der Listenelemente und der Wahrscheinlichkeit dafür, daß ein Listenelement die Höhe mindestens $\log_2 N - 1$ hat, also höchstens

$$N \cdot \left(\frac{1}{2}\right)^{\log_2 N - 1} = N \left(\frac{1}{2}\right)^{\log_2 N} \cdot 2 = 2.$$

Teil 3: Schließlich schätzen wir ab, wieviele Niveaus man noch vom Niveau $\log_2 N - 1$ bis zur Listenhöhe, also bis zur Höhe des Kopfelementes, hinaufsteigen muß. Wir haben die Listenhöhe willkürlich global beschränkt durch *maxhöhe*, also eine nicht allzu weit oberhalb von $\log_2 N - 1$ liegende Konstante. Man kann allerdings auch ohne diese Beschränkung argumentieren und (durch einen nicht ganz einfachen Beweis) zeigen, daß der Erwartungswert für die Höhe einer Skip-Liste mit N Elementen gleich $\log_2 N + 1$ ist, wenn man die Höhenbeschränkung fallen läßt. Nehmen wir an, daß der Erwartungswert für die Differenz zwischen der Listenhöhe und $\log_2 N - 1$ gleich 2 ist, dann ergibt sich insgesamt als obere Schranke für die Suchkosten der Wert

$$2(\log N - 1) + 2 + 2 = O(\log N).$$

Es ist klar, daß auch die Kosten für das *Einfügen* und *Entfernen* von Elementen in Skip-Listen von derselben Größenordnung sind.

Wir haben hier nur eine obere Schranke für die Kosten der drei Wörterbuchoperationen Suchen, Einfügen und Entfernen hergeleitet. In [10] ist der Erwartungswert für die Kosten exakt berechnet worden. Das Ergebnis zeigt, daß die oben angegebene Abschätzung recht scharf ist. Kürzlich haben Munro und Papadakis [11] eine determinierte Variante von Skip-Listen vorgestellt, die es erlaubt, alle drei Wörterbuchoperationen stets, also auch im schlechtesten Fall, in Zeit $O(\log N)$ auszuführen. Diese Struktur hat Ähnlichkeiten mit den im Abschnitt 5.2 eingeführten balancierten Bäumen. Anders als die in diesem Abschnitt eingeführten Skip-Listen sind die determinierten Skip-Listen aber nicht mehr unabhängig von der Entstehungshistorie.

1.8 Aufgaben

Aufgabe 1.1

Für welche der folgenden Paare von Funktionen f und g gilt $f(n) = O(g(n))$, $f(n) = \Omega(g(n))$ bzw. $f(n) = \Theta(g(n))$ für natürliches Argument n ? Dabei soll stets $[x]$ den ganzzahligen Anteil von x bezeichnen.

- | | | |
|-------|--|-------------------------------------|
| (i) | $f(n) = \lfloor \sqrt{n} \rfloor$; | $g(n) = 1000n$ |
| (ii) | $f(n) = \lfloor \log_{10} n \rfloor$; | $g(n) = \lfloor \log_2 n \rfloor$ |
| (iii) | $f(n) = \lfloor \sqrt[3]{n} \rfloor$; | $g(n) = \lfloor \sqrt{n} \rfloor$ |
| (iv) | $f(n) = n^2$; | $g(n) = \lfloor n \log n \rfloor$ |
| (v) | $f(n) = 176n^2 - 36n + 17$; | $g(n) = n^2$ |
| (vi) | $f(n) = \lfloor n \log n \rfloor + \lfloor \sqrt{n} \rfloor$; | $g(n) = \lfloor n \log^2 n \rfloor$ |

Aufgabe 1.2

Ein Polynom vom Grade N läßt sich auch schreiben in der Form

$$p(x) = r_0(x - r_1)(x - r_2) \dots (x - r_N).$$

Leiten Sie aus dieser Schreibweise eine mögliche Form zur Repräsentation von Polynomen ab und beschreiben Sie, wie zwei Polynome bei der gewählten Repräsentation miteinander multipliziert werden. Wie würden Sie zwei Polynome bei dieser Repräsentation addieren?

Aufgabe 1.3

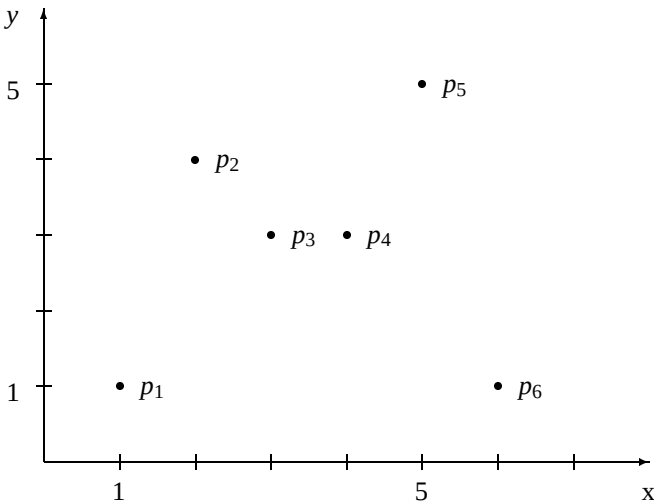
Seien u und v zwei Dualzahlen der Länge N , wobei $N = 2^n$ eine Zweierpotenz sei. Überzeugen Sie sich davon, daß das „Schulverfahren“ zur Multiplikation $O(N^2)$ Schritte benötigt. Entwerfen Sie dann ein Divide-and-conquer-Verfahren zur Berechnung des Produkts analog zum Verfahren zur Berechnung des Produkts zweier Polynome und analysieren Sie die Laufzeit des Verfahrens.

Aufgabe 1.4

Im $\mathbb{N} \times \mathbb{N}$ -Gitter sei eine Menge M von Punkten mit paarweise verschiedenen x -Werten gegeben. Die Dominanzzahl $dz(p, M)$ eines Punktes $p \in M$ bezüglich M ist die Anzahl aller Punkte aus M , die links unterhalb von p liegen, also

$$dz(p, M) = \#\{q = (x_q, y_q) \in M \mid x_q < x_p \wedge y_q \leq y_p\} \text{ für } p = (x_p, y_p).$$

Beispiel:



Für $M = \{p_1, \dots, p_6\}$ ist

$$\begin{aligned} dz(p_1, M) &= 0, \quad dz(p_2, M) = 1, \quad dz(p_3, M) = 1, \quad dz(p_4, M) = 2, \\ dz(p_5, M) &= 4, \quad dz(p_6, M) = 1. \end{aligned}$$

- a) Eine Möglichkeit zur Bestimmung der Dominanzzahlen aller Punkte einer Menge M ist das folgende, der Divide-and-conquer-Strategie folgende Verfahren:

DZ-Bestimmung (M : Punktmenge)

Besteht M nur aus einem einzigen Element p , dann ist $dz(p, M) = 0$, sonst:

1. {Divide} Wähle einen x -Wert x_0 so, daß die vertikale Gerade $x = x_0$ die Menge M in zwei nahezu gleichgroße Teilmengen M_1 und M_2 aufteilt. M_1 sei dabei die Menge mit den kleineren x -Werten.
2. {Conquer}
 - 2.1 Bestimme die Dominanzzahlen aller Punkte in M_1 bezüglich M_1 durch *DZ-Bestimmung (M_1)*.
 - 2.2 Bestimme die Dominanzzahlen aller Punkte in M_2 bezüglich M_2 durch *DZ-Bestimmung (M_2)*.
3. {Merge} Sortiere die Elemente in $M = M_1 \cup M_2$ nach aufsteigenden y -Werten zu einer Folge $p_1, \dots, p_n$ von Punkten. Bei gleichen y -Werten ordne die Elemente aus M_1 vor denen aus M_2 ein. Setze $M1Count := 0$, und durchlaufe die Punkte gemäß der Sortierung wie folgt:

```

for  $i := 1$  to  $n$  do
  if  $p_i \in M_1$ 
    then

```

```

begin
   $M1Count := M1Count + 1;$ 
   $dz(p_i, M) := dz(p_i, M_1);$ 
end
else  $\{p_i \in M_2\}$ 
   $dz(p_i, M) := dz(p_i, M_2) + M1Count$ 

```

Stellen Sie eine Rekursionsformel auf für

$T(N)$ = Anzahl der Schritte, die zur Bestimmung der Dominanzzahlen einer Menge mit N Elementen nach dem Verfahren *DZ-Bestimmung* benötigt wird.

Begründen Sie diese und geben Sie eine Lösung der Rekursionsformel an. Nehmen Sie dabei an, daß N Zahlen in $N \log N$ Schritten sortiert werden können, und beachten Sie b).

b) Zeigen Sie, daß die Rekursionsformel

$$T(N) = 2 \cdot T\left(\frac{N}{2}\right) + c_0 \cdot N \cdot \log^k N + c_1 N$$

die Lösung

$$T(N) = O(N \cdot \log^{k+1} N)$$

hat.

c) Geben Sie ein anderes als das in a) angegebene Verfahren an, das zu gegebener Menge M und gegebenem Punkt $p \in M$ die Zahl $dz(p, M)$ berechnet. Schätzen Sie die Laufzeit ab, wenn mit diesem Verfahren die Dominanzzahlen aller Punkte p aus M bezüglich M berechnet werden.

Aufgabe 1.5

(Bundeswettbewerb Informatik 1985 )

Ein großes Wirtschaftsmagazin will seinen Lesern eine Analyse der Börsenentwicklung der letzten fünf Jahre präsentieren. Dazu sollen unter anderem die Kurse der wichtigsten Aktien in diesem Zeitraum untersucht werden. Für jede Aktie soll nachträglich ein bester Einkaufstag festgestellt werden.

Dabei wird angenommen, daß ein Kapitalanleger jede Aktie höchstens einmal eingekauft hätte, und zwar in einer beliebigen Stückzahl, und daß er zum Ende des betrachteten Zeitraums alle Stücke wieder verkauft hat. Der beste Einkaufstag für eine Aktie wäre dann derjenige gewesen, der zu einem eingesetzten Betrag den höchsten Gewinn geliefert hätte (Steuern, Gebühren und alternative Anlagemöglichkeiten sollen außer Betracht bleiben).

Das Wirtschaftsmagazin hat von einem Börsendienst Informationen über die Notierungen jeder Aktie für alle Börsentage der letzten fünf Jahre gekauft. Für jede Aktie erhält es eine Zahlenfolge. Die erste Zahl ist der Kurs der Aktie am ersten Börsentag und jede folgende Zahl gibt die absolute Kursveränderung gegenüber dem Vortag an, in der Reihenfolge der Börsentage. Der Kurs, der sich für einen gewissen Tag ergibt, gilt für alle Käufe und Verkäufe dieses Tages.

Unterstützen Sie die Kursanalyse durch Schreiben eines Programms, das für eine Aktie aus der gegebenen Zahlenfolge nachträglich einen besten Einkaufstag, einen besten Verkaufstag und den dabei höchsten erzielbaren Gewinn (in Prozent vom eingesetzten Betrag) ermittelt. Da das Programm sehr lange Zahlenfolgen bearbeiten muß, ist es außerordentlich wichtig, daß die Laufzeit bei zunehmender Zahlenfolgenlänge nicht stärker als nötig wächst.

Beispiel: Die Eingabe

„127.5 -0.5 2 -1 1 3.5 -13 7 -2 -6 -9 -21 -17 -5 0.5 4 -7 -12 2.5 -3 2“

liefert die Ausgabe:

„Ein bester Einkaufstag wäre der 14. Börsentag gewesen, ein dazugehöriger Verkaufstag der 16. Börsentag. Der so realisierbare Gewinn wäre 6.7669 % vom eingesetzten Betrag gewesen.“

Aufgabe 1.6

Gegeben sei ein lineares Feld positiver reeller Zahlen, in Pascal beschrieben durch die folgenden Vereinbarungen:

```
const n = {irgendeine positive Zahl, z.B.} 500;
type feld = array [1 .. n] of real;
var a : feld
```

Gegeben seien außerdem eine Funktion $g : \mathbb{R} \rightarrow \{0, 1\}$, die als Werte 0 oder 1 liefert, und die folgende Funktion *gtest*:

```
function gtest (li, re: integer) : integer;
var m : integer;
begin
  if li > re
  then gtest := 0
  else
    begin
      m := (li + re) div 2;
      gtest := gtest(li, m - 1) + g(a[m]) + gtest(m + 1, re)
    end
  end
```

- Beschreiben Sie, welches Resultat die Funktion beim Aufruf *gtest*(1, n) für ein gegebenes Feld *a* liefert.
- Ermitteln Sie größenordnungsmäßig die Anzahl der Additionen bei der Ausführung eines Aufrufs von *gtest*(li, re) im schlimmsten Fall, in Abhängigkeit von $|re - li|$, mit Hilfe einer Rekursionsformel.
- Geben Sie in Pascal ein alternatives (iteratives) Verfahren zur Ermittlung des Funktionswertes *gtest* an; verwenden Sie denselben Funktionskopf.

Aufgabe 1.7

Das maximale Element in einem linearen Feld kann auf folgende Weise bestimmt werden.

```

program Maximum (input, output);
const  $N = \{\text{eine feste Zahl}\}$ ;
type feld =  $\text{array}[1 \dots N]$  of integer;
var a : feld;
procedure max (var a : feld; i, j : integer; var m : integer);
  {bestimmt das maximale Element im Bereich  $a[i], \dots, a[j]$ 
   und weist es m zu}
var  $m_1, m_2, \text{mitte}$  : integer;
begin
  if  $i = j$ 
  then  $m := a[i]$ 
  else
    if  $i = j - 1$ 
    then
      begin
        if  $a[i] < a[j]$ 
        then  $m := a[j]$ 
        else  $m := a[i]$ 
      end
    else
      begin
         $\text{mitte} := (i + j) \text{ div } 2$ ;
         $\text{max}(a, i, \text{mitte}, m_1)$ ;
         $\text{max}(a, \text{mitte} + 1, j, m_2)$ ;
        if  $m_1 < m_2$ 
        then  $m := m_2$ 
        else  $m := m_1$ 
      end
    end
  end; {max}

begin {Maximum}
  {Eingabe der Werte von  $a[1], \dots, a[N]$ }
   $\text{max}(a, 1, N, m)$ 
end {Maximum}.

```

- Berechnen Sie die Anzahl der Vergleichsoperationen, die zwischen Elementen des Feldes ausgeführt werden, durch Aufstellen und Lösen einer Rekursionsgleichung.
- Vergleichen Sie das angegebene Verfahren mit dem „naiven“ Verfahren zur Bestimmung des Maximums.
- Ändern Sie das Verfahren so ab, daß zugleich das maximale und das minimale Element des linearen Feldes bestimmt wird und ermitteln Sie ebenfalls die Anzahl der ausgeführten Vergleichsoperationen zwischen Feldelementen.

Aufgabe 1.8

Gegeben sei eine sortierte Liste L voneinander verschiedener ganzer Zahlen in sequentieller Speicherung. Gegeben sei außerdem eine Zahl x . Gesucht ist das größte Element in L , das $\leq x$ ist. Die Länge der Liste L sei N .

- Geben Sie in verbaler Beschreibung einen Algorithmus an, der diese Aufgabe in logarithmischer Schrittzahl löst.
- Folgende Pascal-Vereinbarungen seien gegeben:

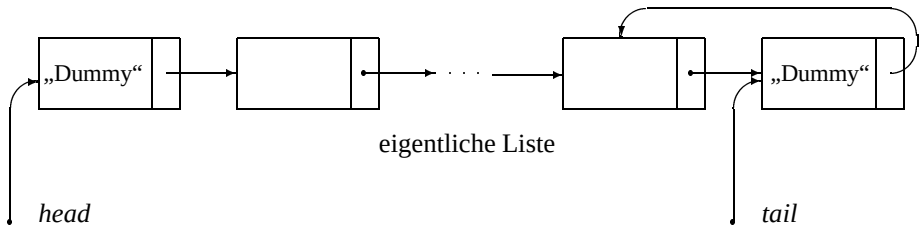
```
const  $N = \{z.B.\} 500;$ 
type  $feld = \text{array}[1..N]$  of  $integer$ 
```

Schreiben Sie eine Funktion zu dem in a) entwickelten Algorithmus.

```
function  $suche$  (var  $liste: feld; x, li, re: integer$ ) :  $integer$ ;
{sucht das größte Element  $\leq x$  im Bereich  $liste[li..re]$ }
```

Aufgabe 1.9

Gegeben sei eine nichtleere verkettete lineare Liste L ganzer Zahlen mit ungerader Elementzahl. Die Liste beginnt und endet mit je einem bedeutungslosen Dummy-Element, das einen beliebigen Wert haben kann. Zwischen den beiden Dummy-Elementen befinden sich die eigentlichen Listenelemente.



Die Struktur der Knoten der Liste in Pascal wie folgt gegeben.

```
type  $Zeiger = \uparrow Knoten;$ 
       $Knoten = \text{record}$ 
         $key : integer;$ 
         $next : Zeiger$ 
      end
```

- Schreiben Sie eine Prozedur

```
procedure  $mitte$  ( $head, tail : Zeiger$ );
```

die das Element an der mittleren Position der Liste entfernt.

- Schreiben Sie eine Prozedur

procedure teilen (**var** *head*, *headeven*, *headodd* : Zeiger);

die die Liste *L* mit Anfangszeiger *head* aufteilt in zwei (anfangs leere, also durch je zwei Dummy-Elemente gegebene) Listen mit Anfangszeiger *headeven* bzw. *headodd*. In die eine Liste sollen die Elemente aus *L* mit geradzahligem Eintrag gehängt werden, in die andere die Elemente mit ungeradzahligem Eintrag. Aufrufe von *new* sind dabei nicht erlaubt.

c) Schreiben Sie eine Prozedur

procedure umdrehen (*head*, *tail* : Zeiger);

die die Reihenfolge der Elemente, d.h. die Zeiger, in der Liste umdreht, ohne eine zweite Liste zu verwenden (d.h. Aufrufe von *new* sind nicht erlaubt).

Aufgabe 1.10

Schreiben Sie ein Programm, das ein Element mit gegebenem Schlüssel aus einer geketteten gespeicherten linearen Liste mit Dummy-Elementen am Anfang und Ende entfernt. Verwenden Sie nicht die Technik des „Zurückhängens mit Vorausschauen“, sondern verfahren Sie wie folgt. Durchsuchen Sie die Liste nach dem zu entfernenden Element. Ersetzen Sie das zu entfernende Element durch dessen Nachfolger in der Liste und entfernen Sie diesen. Achten Sie auf mögliche Sonderfälle wie Entfernen des ersten oder letzten Elementes, und schätzen Sie den Aufwand ab.

Aufgabe 1.11

Schreiben Sie ein Programm, das ein Element mit gegebenem Schlüssel aus einer aufsteigend sortierten linearen Liste, die verkettet gespeichert ist, entfernt. Schätzen Sie den Aufwand ab.

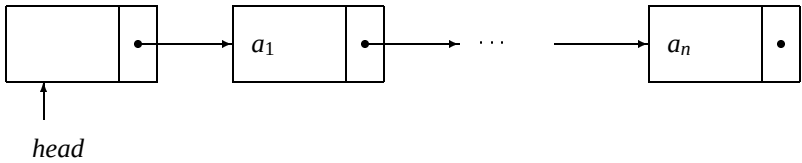
Aufgabe 1.12

Gegeben sei eine verkettet gespeicherte lineare Liste mit Anfangszeiger *head* und Listenelementen des in Aufgabe 1.9 vereinbarten Typs. Die *key*-Komponenten seien entlang der Verkettung aufsteigend sortiert. Das Ende der Liste ist durch ein Listenelement gekennzeichnet, dessen *next*-Komponente den Wert **nil** hat. Schreiben Sie eine Prozedur *Teile* mit folgenden Eigenschaften. Aus der *head*-Liste werden alle Listenelemente, deren *key*-Komponente kleiner als ein gegebener Schlüssel *k* ist — und nur diese — an die anfangs leere *kleiner*-Liste übergeben und dabei aus der *head*-Liste entfernt. Es kann vorausgesetzt werden, daß die *head*-Liste zuvor sowohl Elemente mit *key*-Komponente $< k$ als auch solche mit *key*-Komponente $\geq k$ enthält. Verwenden Sie folgenden Prozedurkopf:

procedure Teile (*k* : integer; **var** *head*, *kleiner* : Zeiger);

Aufgabe 1.13

Gegeben sei eine verkettet gespeicherte Liste durch einen Zeiger auf das Kopfelement *head*, der Typ der Elemente sei wie in Aufgabe 1.9 vereinbart.



Position i sei implementiert als ein Zeiger auf das Element, dessen $next$ -Zeiger auf ein Element mit Schlüssel a_i zeigt. Schreiben Sie eine Prozedur, die die Elemente an den Positionen p und $p \uparrow .next$ miteinander vertauscht, wenn $p \uparrow .next \neq \text{nil}$ ist.

Aufgabe 1.14

Erstellen Sie zwei Pascal-Programme zur iterativen Berechnung der folgenden verallgemeinerten Binomialkoeffizienten.

				1				
				1		2		
			1		3		4	
		1		4		7		8
	1		5		11		15	16

- Das allgemeine Bildungsgesetz lautet $\binom{d}{0} = 1$, $\binom{d}{d} = 2^d$ und $\binom{d}{h} = \binom{d-1}{h} + \binom{d-1}{h-1}$.
- a) Das erste Pascal-Programm verwende einen Stapel, der verkettet gespeichert, also über Zeiger realisiert wird.
 - b) Das zweite Pascal-Programm verwende ein Berechnungsschema, das mehrfache Berechnung von gleichen Teilresultaten vermeidet.

Aufgabe 1.15

Einfache, vollständig geklammerte, arithmetische Ausdrücke können wie folgt definiert werden.

- (1) Jede Variable $a, b, c, \dots$ ist ein einfacher, vollständig geklammerter, arithmetischer Ausdruck.
- (2) Mit α und β sind auch $(\alpha + \beta)$, $(\alpha - \beta)$, $(\alpha * \beta)$, (α / β) einfache, vollständig geklammerte, arithmetische Ausdrücke.
- (3) Sonst nichts.

Geben Sie ein Verfahren zur Auswertung von Ausdrücken mit Hilfe eines Stapels an, das mit Hilfe eines Stapels Variablen, Operatoren und Zwischenergebnisse speichert. Das Verfahren soll nur auf die für Stapel üblichen Operationen, aber nicht auf eine konkrete Implementation Bezug nehmen.

Beispiel: Die Auswertung des Ausdrucks

$$(c + ((a + b) * a))$$

erzeugt bei Auswertung (d. h. Lesen von links nach rechts) folgende Stapelbelegungen (jede Zeile ist eine Stapelbelegung, das oberste Element steht jeweils rechts).

c
 $c, +$
 $c, +, a$
 $c, +, a, +$
 $c, +, a, +, b$
 $c, +, (a + b)$
 $c, +, (a + b), *$
 $c, +, (a + b), *, a$
 $c, +, ((a + b) * a)$
 $(c + ((a + b) * a))$

Aufgabe 1.16

Ein einfacher, vollständig geklammerter, arithmetischer Ausdruck ist wie in Aufgabe 1.15 definiert. Ein solcher Ausdruck heißt auch arithmetischer Ausdruck in Infixnotation. Der äquivalente arithmetische Ausdruck in Postfixnotation ist analog wie folgt definiert.

- (1) Jede Variable $a, b, c, \dots$ ist ein Ausdruck in Postfixnotation.
- (2) Sind $(\alpha + \beta)$, $(\alpha - \beta)$, $(\alpha * \beta)$, (α / β) Ausdrücke in Infixnotation, so sind $\alpha\beta+$, $\alpha\beta-$, $\alpha\beta*$, $\alpha\beta/$ die äquivalenten arithmetischen Ausdrücke in Postfixnotation.
- (3) Sonst nichts.

Beispiel: Der zu $((a + b) * a) + c$ äquivalente arithmetische Ausdruck in Postfixnotation ist $ab + a * c +$.

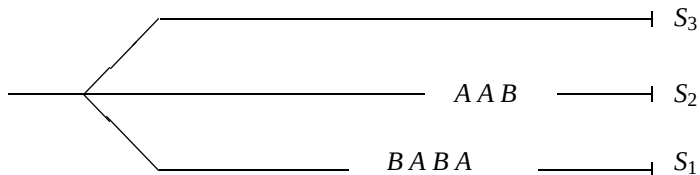
- a) Geben Sie ein Verfahren an, das einen Ausdruck in Infixnotation in den äquivalenten arithmetischen Ausdruck in Postfixnotation mit Hilfe zweier Stapel umwandelt (einen Stapel für den arithmetischen Ausdruck, den zweiten als Hilfsstapel für Operationen). Das Verfahren soll nur auf die für Stapel üblichen Operationen, aber nicht auf eine konkrete Implementation Bezug nehmen.
- b) Geben Sie eine mögliche Implementation des Verfahrens in Pascal an.
- c) Geben Sie ein Verfahren an, das mit Hilfe eines Stapels den Wert eines in Postfixnotation gegebenen Ausdrucks errechnet.

Aufgabe 1.17

Geben Sie ein Verfahren an, das mit Hilfe eines Stapels eine Folge von Buchstaben einliest und in umgekehrter Reihenfolge wieder ausgibt. Die Länge der Folge ist unbekannt, das Ende der Eingabe durch einen Punkt markiert. Das Verfahren soll nur auf die für Stapel üblichen Operationen, aber nicht auf eine konkrete Implementation Bezug nehmen.

Aufgabe 1.18

In einem Sackbahnhof mit drei Gleisen befinden sich in den Gleisen S_1 und S_2 zwei Züge mit Waggons für Zielbahnhof A bzw. B . Gleis S_3 sei leer (vgl. Abbildung).



Betrachten Sie S_1 , S_2 , S_3 als Stapel und erstellen Sie ein Programmstück in Pseudo-Pascal unter Verwendung der unten angegebenen Funktionen bzw. Prozedur, das zwei beliebige aus Waggons für A und B bestehende Züge so umordnet, daß anschließend in S_1 alle Waggons für A und in S_2 alle Waggons für B stehen.

```
procedure push(var S: Stapel; X: Waggon);
    {push stellt X auf S ab}
function pop(var S: Stapel) : Waggon;
    {pop liefert vordersten Waggon von S und entfernt ihn von S,
     wenn S nicht leer ist; Fehler sonst};
function top(S: Stapel) : Zielbahnhof;
    {top liefert den Zielbahnhof des 1. Waggons in S, ohne ihn
     zu entfernen};
function leer(S: Stapel) : boolean;
    {leer liefert true, wenn S leer; false sonst}.
```

Literaturliste zu Kapitel 1: Grundlagen

Seite 4

[90] D. E. Knuth. Big omicron and big omega and big theta. SIGACT News, 8(2):18-24, 1976.

Seite 11

[177] V. Strassen. Gaussian elimination is not optimal. Numer. Math., 13:354-356, 1969.

Seite 12

[16] J. L. Bentley. Programming pearls. Comm. ACM, 27:865-871, 1984.

[77] P. Heyderhoff, Hrsg. Bundeswettbewerb Informatik: Aufgaben und Lösungen, Band 1. Ernst Klett Schulbuchverlag, 1989.

Seite 42

[152] W. Pugh. Skip lists: A probabilistic alternative to balanced trees. Comm. ACM, 33(6):668-676, 1990. (Erste Fassung in [151]).

[151] W. Pugh. Skip lists: A probabilistic alternative to balanced trees. In Proc. Workshop of Algorithms and Data Structures, S. 437-449, 1989. Lecture Notes in Computer Science 382.

Seite 46

[151] W. Pugh. Skip lists: A probabilistic alternative to balanced trees. In Proc. Workshop of Algorithms and Data Structures, S. 437-449, 1989. Lecture Notes in Computer Science 382.

Seite 52

[143] T. Papadakis, J. I. Munro und P. V. Poblete. Analysis of the expected search cost in skip lists. In Proc. 2nd Scandinavian Workshop on Algorithm Theory, S. 160- 172. Lecture Notes in Computer Science 447, Springer, 1990.

[127] J. I. Munro und X. Papadakis. Deterministic skip lists. In Proc. 3rd Annual Symposium On Discrete Algorithms (SODA), S. 367-375, 1992.

Seite 54

[77] P. Heyderhoff, Hrsg. Bundeswettbewerb Informatik: Aufgaben und Lösungen, Band 1. Ernst Klett Schulbuchverlag, 1989.

Kapitel 2

Sortieren

Untersuchungen von Computerherstellern und -nutzern zeigen seit vielen Jahren, daß mehr als ein Viertel der kommerziell verbrauchten Rechenzeit auf Sortiervorgänge entfällt. Es ist daher nicht erstaunlich, daß große Anstrengungen unternommen wurden, möglichst effiziente Verfahren zum Sortieren von Daten mit Hilfe von Computern zu entwickeln. Das gesammelte Wissen über Sortierverfahren füllt inzwischen Bände. Noch immer erscheinen neue Erkenntnisse über das Sortieren in wissenschaftlichen Fachzeitschriften (vgl. z.B. [1]), und zahlreiche theoretisch und praktisch wichtige Probleme im Zusammenhang mit dem Problem, eine Menge von Daten zu sortieren, sind ungelöst. Zunächst wollen wir das Sortierproblem genauer fixieren: Wir nehmen an, es sei eine Menge von Sätzen gegeben; jeder Satz besitzt einen *Schlüssel*. Zwischen Schlüsselns ist eine Ordnungsrelation „<“ oder „≤“ erklärt. Außer der Schlüsselkomponente können Sätze weitere Komponenten als „eigentliche“ Information enthalten. Wenn nichts anderes gesagt ist, werden wir annehmen, daß die Schlüssel ganzzahlig sind. Das ist auch ein in der Praxis oft vorliegender Fall. Man denke etwa an Artikelnummern, Hausnummern, Personalnummern, Scheckkartennummern usw. (Es kommen aber auch andere Schlüssel vor, z.B. alphabetisch und insbesondere lexikographisch geordnete Namen!)

Das *Sortierproblem* kann präziser wie folgt formuliert werden. Gegeben ist eine Folge von Sätzen (englisch: items) $s_1, \dots, s_N$; jeder Satz s_i hat einen Schlüssel k_i . Man finde eine Permutation π der Zahlen von 1 bis N derart, daß die Umordnung der Sätze gemäß π die Schlüssel in aufsteigende Reihenfolge bringt:

$$k_{\pi(1)} \leq k_{\pi(2)} \leq \dots \leq k_{\pi(N)}.$$

In dieser Problemformulierung sind viele Details offengelassen, die für die Lösung durchaus wichtig sind. Was heißt es, daß eine Folge von Sätzen „gegeben“ ist? Ist damit gemeint, daß sie in schriftlicher Form oder auf Lochkarten, Magnetband, Platte oder Diskette vorliegen? Ist die Anzahl der Sätze bekannt? Ist das Spektrum der vorkommenden Schlüssel bekannt? Welche Operationen sind erlaubt, um die Permutation π zu bestimmen? Wie geschieht die „Umordnung“? Sollen die Datensätze physisch bewegt werden oder genügt es, eine Information zu berechnen, die das Durchlaufen, insbesondere die Ausgabe der Datensätze in aufsteigender Reihenfolge erlaubt?

Wir können unmöglich alle in der Realität auftretenden Parameter des Sortierproblems berücksichtigen. Vielmehr wollen wir uns auf einige prinzipielle Aspekte des Problems beschränken und dazu weitere Annahmen machen. Zunächst nehmen wir an, daß die Menge der zu sortierenden Datensätze vollständig im Hauptspeicher Platz findet. Sortierverfahren, die von dieser Annahme ausgehen, heißen auch *interne* Sortierverfahren. Im Abschnitt 2.7 dieses Kapitels gehen wir dann auch auf *externe* Sortierverfahren ein, die eine auf einem externen Speichermedium (Diskette, Platte, Band) vorliegende Folge von Datensätzen voraussetzen, die nicht gänzlich im Hauptspeicher Platz findet.

Die Lösung des Sortierproblems verlangt in jedem Fall die Lösung zweier Teilprobleme, nämlich

- (a) eines Informationsbeschaffungsproblems und
- (b) eines Datentransportproblems.

Zu (a): Meistens nimmt man an, daß als einzige verwendbare Information zur Lösung des Sortierproblems, also zur Bestimmung der Permutation π , das Ergebnis von Vergleichen zwischen je zwei Schlüsseln zugelassen ist. Wir können also feststellen, ob für zwei Schlüssel k und k' gilt

$$k = k', k < k' \text{ oder } k > k'.$$

Wir gehen von dieser Annahme in den Abschnitten 2.1 bis 2.4 und 2.6 bis 2.8 aus. Für ganzzahlige Schlüssel ist es aber natürlich auch sinnvoll, andere Operationen wie Addition, Subtraktion, Multiplikation und ganzzahlige Division zuzulassen. Wir gehen darauf im Abschnitt 2.5 ein.

Zu (b): In der Regel verlangt man, daß als Lösung des Sortierproblems die zu sortierenden Datensätze in einem zusammenhängenden Speicherbereich nach aufsteigenden Schlüsseln geordnet vorliegen sollen. Dazu müssen sie bewegt werden, wenn sie nicht schon von vornherein so vorlagen. Als „Bewegungen“ läßt man unter anderem zu: Das Vertauschen zweier benachbarter oder zweier beliebiger Sätze; das Verschieben einzelner Sätze oder ganzer Blöcke um eine feste oder beliebige Distanz; das Plazieren an eine direkt oder indirekt gegebene Speicheradresse.

Eine Folge von N im Hauptspeicher, also intern, vorliegenden Sätzen mit ganzzahligen Schlüsseln kann man programmtechnisch einfach als Array der Länge N realisieren.

```

type
  item = record
    key : integer;
    info : {infotype}
  end;
  sequence = array [1 .. N] of item;
var
  a : sequence
```

Eine Lösung des Sortierproblems (für eine intern gegebene Folge von Datensätzen) besteht dann in der Angabe einer Prozedur, die das als Eingabe- und Ausgabeparameter übergebene Array a verändert. Es soll erreicht werden, daß nach Aufruf der Prozedur die Elemente von a nach aufsteigenden Schlüsseln sortiert sind. D h. für alle i , $1 \leq i < N$, gilt

$$a[i].key \leq a[i+1].key.$$

Wir setzen also für alle internen Sortierverfahren folgenden einheitlichen Rahmen voraus (außer bei einigen rekursiv formulierten Sortierverfahren, wo die Parameterliste anders angegeben ist).

```

program Rahmen_für_Sortierverfahren (input, output);
const
     $N = \{\text{Anzahl der zu sortierenden Sätze}\};$ 
type
    item = record
        key: integer;
        info: {infotype}
    end;
    sequence = array [0 .. N] of item;
var
    a : sequence;
procedure XYZ-sort (var a : sequence);
    {hier folgt die jeweilige Sortierprozedur}
begin
    {Eingabe: Lies  $a[1], \dots, a[N]$ };
    XYZ-sort(a);
    {Ausgabe: Schreibe  $a[1], \dots, a[N]$ }
end.
```

Daß der Array-Typ *sequence* hier mit Index 0 beginnend indiziert ist, hat lediglich programmtechnische Gründe. Die zu sortierenden Elemente stehen nach wie vor an den Positionen 1 bis N .

Wir behandeln im Abschnitt 2.1 *elementare Sortierverfahren* (Sortieren durch Auswahl, Sortieren durch Einfügen, Shellsort und Bubblesort). Für diese Verfahren ist typisch, daß im ungünstigsten Fall $\Theta(N^2)$ Vergleichsoperationen zwischen Schlüsseln ausgeführt werden müssen, um N Schlüssel zu sortieren.

Das im Abschnitt 2.2 behandelte Verfahren *Quicksort* benötigt im Mittel nur $O(N \log N)$ Vergleichsoperationen; es ist das interne Sortierverfahren mit der besten mittleren Laufzeit, weil es im Detail sehr effizient implementiert werden kann.

Zwei Verfahren, die eine Menge von N Schlüsseln stets mit nur $O(N \log N)$ Vergleichsoperationen zu sortieren erlauben, sind die Verfahren *Heapsort* und *Mergesort*, die in den Abschnitten 2.3 und 2.4 diskutiert werden. Im Abschnitt 2.8 zeigen wir, daß $\Omega(N \log N)$ Vergleichsoperationen auch tatsächlich notwendig sein können. Im Abschnitt 2.5 lassen wir die Voraussetzung fallen, daß nur Vergleichsoperationen zwischen Schlüsseln zugelassen werden. Wir geben ein Verfahren (*Radixsort*) an, das die arithmetischen Eigenschaften der zu sortierenden Schlüssel ausnutzt.

2.1 Elementare Sortierverfahren

Wir gehen in diesem Abschnitt davon aus, daß die zu sortierenden N Datensätze Elemente eines global vereinbarten Feldes a sind. Es wird jeweils eine Sortierprozedur mit a als Eingabe- und Ausgabeparameter angegeben, die bewirkt, daß nach Ausführung der Prozedur die ersten N Elemente von a so angeordnet sind, daß die Schlüsselkomponenten aufsteigend sortiert sind:

$$a[1].key \leq a[2].key \leq \dots \leq a[N].key$$

Wir erläutern vier verschiedene Verfahren. *Sortieren durch Auswahl* folgt der naheliegenden Idee, die Sortierung durch Bestimmung des Elements mit kleinstem, zweitkleinstem, drittkleinstem ... usw. Schlüssel zu erreichen. Die jedem Skatspieler geläufige Methode des Einfügens des jeweils nächsten Elements an die richtige Stelle liegt dem Verfahren *Sortieren durch Einfügen* zugrunde. Das von D.L. Shell vorgeschlagene Verfahren *Shellsort*, vgl. [3], kann als Verbesserung des Sortierens durch Einfügen angesehen werden. *Bubblesort* ist ein Sortierverfahren, das solange zwei jeweils benachbarte, nicht in der richtigen Reihenfolge stehende Elemente vertauscht, bis keine Vertauschungen mehr nötig sind, das Feld a also sortiert ist. Wir geben in jedem Fall zunächst eine verbale Beschreibung des Sortierverfahrens an und bringen dann eine mögliche Implementation in Pascal.

Zur Messung der Laufzeit der Verfahren verwenden wir zwei naheliegende Größen. Dies ist zum einen die Anzahl der zum Sortieren von N Sätzen ausgeführten *Schlüsselvergleiche* und zum anderen die Anzahl der ausgeführten *Bewegungen* von Datensätzen. Für beide Parameter interessieren uns die im günstigsten Fall (best case), die im schlechtesten Fall (worst case) und die im Mittel (average case) erforderlichen Anzahlen. Wir bezeichnen die jeweiligen Größen mit

$$C_{min}(N), C_{max}(N) \text{ und } C_{mit}(N)$$

für die Anzahlen der Schlüsselvergleiche (englisch: comparisons) und mit

$$M_{min}(N), M_{max}(N) \text{ und } M_{mit}(N)$$

für die Anzahlen der Bewegungen (englisch: movements). Die Mittelwerte $C_{mit}(N)$ und $M_{mit}(N)$ werden dabei üblicherweise auf die Menge aller $N!$ möglichen Ausgangsanordnungen von N zu sortierenden Datensätzen bezogen.

2.1.1 Sortieren durch Auswahl

Method: Man bestimme diejenige Position j_1 , an der das Element mit minimalem Schlüssel unter $a[1], \dots, a[N]$ auftritt, und vertausche $a[1]$ mit $a[j_1]$. Dann bestimme man diejenige Position j_2 , an der das Element mit minimalem Schlüssel unter $a[2], \dots, a[N]$ auftritt (das ist das Element mit zweitkleinstem Schlüssel unter allen N Elementen), und vertausche $a[2]$ mit $a[j_2]$ usw., bis alle Elemente an ihrem richtigen Platz stehen.

Wir bestimmen also der Reihe nach das *i*-kleinste Element, $i = 1, \dots, N - 1$, und setzen es an die richtige Position. Genauer gesagt bestimmen wir natürlich die Position, an der das Element mit dem *i*-kleinsten Schlüssel steht. D.h. für jedes $i = 1, \dots, N - 1$ gehen wir der Reihe nach wie folgt vor. Wir können voraussetzen, daß $a[1], \dots, a[i - 1]$ bereits die $i - 1$ kleinsten Schlüssel in aufsteigender Reihenfolge enthält. Wir suchen unter den verbleibenden $N - i + 1$ Elementen das mit kleinstem Schlüssel, sagen wir $a[\min]$, und vertauschen $a[i]$ und $a[\min]$.

Beispiel: Gegeben sei ein Feld mit sieben Schlüsseln:

j :	1	2	3	4	5	6	7
$a[j].key$:	15	2	43	17	4	8	47

Das kleinste Element steht an Position 2; Vertauschen von $a[1]$ und $a[2]$ ergibt:

$a[j].key$:	2	15	43	17	4	8	47
--------------	---	----	----	----	---	---	----

Das zweitkleinste Element steht jetzt an Position 5; Vertauschen von $a[2]$ und $a[5]$ ergibt:

$a[j].key$:	2	4	43	17	15	8	47
--------------	---	---	----	----	----	---	----

Die weiteren vier Schritte (Bestimmung des *i*-kleinsten Elements und jeweils Vertauschen mit $a[i]$) kann man kurz wie folgt zusammenfassen:

$i = 3$:	2	4	8	17	15	43	47
$i = 4$:	2	4	8	15	17	43	47

Ab jetzt ($i = 5, 6$) verändert sich das Feld a nicht mehr.

Man sieht an diesem Beispiel, daß keine Vertauschung nötig ist, wenn das *i*-kleinste Element bereits an Position i steht. Die folgende Pascal-Version des Verfahrens nutzt diese Möglichkeit nicht aus.

procedure Auswahlsort (**var** a : sequence);

var

$i, j, \min$: integer;

t : item; {Hilfsspeicher}

begin

for $i := 1$ **to** $N - 1$ **do**

begin

{bestimme Position $\min$ des kleinsten unter
den Elementen $a[i], \dots, a[N]$ }


```

    min := i;
    for j := i + 1 to N do
{ * }      if a[j].key < a[min].key
            then min := j;
            {vertausche Elemente an Position i und Position min}
{ ** }      t := a[min];
            a[min] := a[i];
            a[i] := t
    end
end

```

Analysis: Man sieht, daß zur Bestimmung des i -kleinsten Elements, $i = 1, \dots, N - 1$, jeweils die Position des Minimums in der Restfolge $a[i], \dots, a[N]$ bestimmt wird. Die dabei ausgeführte Anzahl von Schlüsselvergleichen (in Programmzeile $\{*\}$) ist unabhängig von der Ausgangsanordnung jeweils $(N - i)$. Damit ist

$$C_{\min}(N) = C_{\max}(N) = C_{\text{mit}}(N) = \sum_{i=1}^{N-1} (N - i) = \sum_{i=1}^{N-1} i = \frac{N(N-1)}{2} = \Theta(N^2).$$

Zählt man nur die Bewegungen von Datensätzen, die in den drei Programmzeilen ab $\{**\}$ ausgeführt werden, so werden, wieder unabhängig von der Ausgangsanordnung, genau

$$M_{\min}(N) = M_{\max}(N) = M_{\text{mit}}(N) = 3(N - 1)$$

Bewegungen durchgeführt. Die Abschätzung der in der auf $\{*\}$ folgenden Programmzeile zur Adjustierung des Wertes von min ausgeführten Zuweisungen hängt natürlich vom Ergebnis des vorangehenden Schlüsselvergleiches und damit von der Ausgangsanordnung ab. Im günstigsten Fall wird diese Zuweisung nie, im ungünstigsten Fall jedesmal und damit insgesamt wieder $\Theta(N^2)$ Mal durchgeführt. Wir haben einfache Zuweisungen deswegen getrennt von Schlüsselvergleichen und Bewegungen von Datensätzen betrachtet, weil sie weniger aufwendig sind und in allen unseren Beispielen ohnehin von den Schlüsselvergleichen dominiert werden. Die Abschätzung der mittleren Anzahl von Zuweisungen, die in der auf $\{*\}$ folgenden Zeile ausgeführt werden, ist schwieriger und wird hier übergangen.

Kann man das Verfahren effizienter machen, etwa dadurch, daß man eine „bessere“ Methode zur Bestimmung des jeweiligen Minimums (in der Restfolge) verwendet? Der folgende Satz zeigt, daß dies jedenfalls dann nicht möglich ist, wenn als einzige Operation Vergleiche zwischen Schlüsseln zugelassen sind.

Satz 2.1 *Jeder Algorithmus zur Bestimmung des Minimums von N Schlüsseln, der allein auf Schlüsselvergleichen basiert, muß wenigstens $N - 1$ Schlüsselvergleiche ausführen.*

Beweis: Jeder Algorithmus zur Bestimmung des Minimums von N Schlüsseln $k_1, \dots, k_N$ läßt sich als ein nach dem K.-o.-System ausgeführter Wettkampf auffassen: Von zwei Teilnehmern k_i und k_j , $1 \leq i, j \leq N$, $i \neq j$, scheidet der größere aus. Der Sieger des Wettkampfs steht erst dann fest, wenn alle anderen Teilnehmer ausgeschieden sind. Weil bei jedem Wettkampf genau ein Teilnehmer ausscheidet, benötigt man also $N - 1$ Wettkämpfe zur Ermittlung des Siegers. $\square$

Obwohl es Sortierverfahren gibt, die mit weniger als $\Theta(N^2)$ Vergleichsoperationen zwischen Schlüsseln auskommen, um N Datensätze zu sortieren, ist das Verfahren Sortieren durch Auswahl unter Umständen das bessere. Sind Bewegungen von Datensätzen besonders teuer, aber Vergleichsoperationen zwischen Schlüsseln billig, so ist Sortieren durch Auswahl gut, weil es nur linear viele Bewegungen von Datensätzen ausführt. Dieser Fall kann z.B. für Datensätze mit (kleinem) ganzzahligem Schlüssel, aber umfangreichem und kompliziert strukturiertem Datenteil vorliegen.

2.1.2 Sortieren durch Einfügen

Methode: Die N zu sortierenden Elemente werden nacheinander betrachtet und in die jeweils bereits sortierte, anfangs leere Teilfolge an der richtigen Stelle eingefügt.

Nehmen wir also an, $a[1], \dots, a[i-1]$ seien bereits sortiert, d.h. $a[1].key \leq \dots \leq a[i-1].key$. Dann wird das i -te Element $a[i]$ an der richtigen Stelle in die Folge $a[1], \dots, a[i-1]$ eingefügt. Das geschieht so, daß man $a[i].key$ der Reihe nach mit $a[i-1].key, a[i-2].key, \dots$ vergleicht und das Element $a[j]$ dabei jeweils um eine Position nach rechts verschiebt, für $j = i-1, i-2, \dots$, wenn $a[j].key > a[i].key$ ist. Sobald man erstmals an eine Position j gekommen ist, so daß $a[j].key \leq a[i].key$ ist, hat man die richtige Stelle gefunden, an der das Element $a[i]$ eingefügt werden kann, nämlich die Position $j+1$.

Das Einfügen des i -ten Elementes $a[i]$ an der richtigen Stelle in der Folge der Elemente $a[1], \dots, a[i-1]$ verlangt also im allgemeinen ein Verschieben von j Elementen um eine Position nach rechts, wobei j zwischen 0 und $i-1$ liegen kann. Zur Bestimmung der Einfügestelle wird stets eine Vergleichsoperation mehr als die Anzahl der Verschiebungen durchgeführt.

Beispiel: Betrachten wir wieder das Feld a mit den sieben Schlüsseln 15, 2, 43, 17, 4, 8, 47. Die aus nur einem einzigen Element bestehende Teilfolge $a[1]$ ist natürlich bereits sortiert. Einfügen von $a[2]$ in diese Folge verlangt die Verschiebung von $a[1]$ um eine Position nach rechts und liefert:

j :	1	2	3	4	5	6	7
$a[j].key$:	2	15	43	17	4	8	47

Wir haben hier das Ende des bereits sortierten Anfangsstücks des Feldes a durch einen Doppelstrich markiert. Einfügen von $a[3]$ erfordert keine Verschiebung. Einfügen von $a[4]$ bewirkt die Verschiebung von $a[3]$ um eine Position nach rechts und liefert:

$a[j].key$:	2	15	17	43	4	8	47
--------------	---	----	----	----	---	---	----

Die weiteren Schritte lassen sich kurz wie folgt angeben:

$a[j].key$:	2	4	15	17	43	8	47
--------------	---	---	----	----	----	---	----

3 Verschiebungen

$$a[j].key :$$

2	4	8	15	17	43	47
---	---	---	----	----	----	----

3 Verschiebungen

$$a[j].key :$$

2	4	8	15	17	43	47
---	---	---	----	----	----	----

0 Verschiebungen

Es liegt nahe, das Verfahren wie folgt in Pascal zu implementieren:

```

procedure Einfügesort (var a : sequence);
var
    i, j, k : integer;
    t : item; {Hilfsspeicher}
begin
    for i := 2 to N do
        begin
            {füge a[i] an der richtigen Stelle in a[1], ..., a[i-1] ein}
            j := i;
            {**} t := a[i];
            k := t.key;
            {*} while a[j-1].key > k do
                begin
                    {nach rechts verschieben}
                    {**} a[j] := a[j-1];
                    j := j - 1
                end;
            {**} a[j] := t
        end
    end

```

Eine genaue Betrachtung des Programms zeigt, daß die **while**-Schleife nicht korrekt terminiert. Ist der Schlüssel k des nächsten einzufügenden Elements $a[i]$ kleiner als die Schlüssel aller Elemente $a[1], \dots, a[i-1]$, so wird die Bedingung $a[j-1].key > k$ nie falsch für $j = i, \dots, 2$. Eine ganz einfache Möglichkeit, ein korrektes Terminieren der Schleife zu sichern, besteht darin, am linken Ende des Feldes einen Stopper für die lineare Suche abzulegen. Setzt man $a[0].key := k$ direkt vor der **while**-Schleife, wird die Schleife korrekt beendet, ohne daß in Programmzeile $\{*\}$ jedesmal geprüft werden muß, ob j noch im zulässigen Bereich liegt. Wir gehen im folgenden von dieser Annahme aus.

Analyse: Zum Einfügen des i -ten Elementes werden offenbar mindestens ein und höchstens i Schlüsselvergleiche in Programmzeile $\{*\}$ und zwei oder höchstens $i+1$ Bewegungen von Datensätzen in Programmzeilen $\{**\}$ ausgeführt. Daraus ergibt sich sofort

$$C_{\min}(N) = N - 1; \quad C_{\max}(N) = \sum_{i=2}^N i = \Theta(N^2);$$

$$M_{\min}(N) = 2(N-1); \quad M_{\max}(N) = \sum_{i=2}^N (i+1) = \Theta(N^2).$$

Die im Mittel zum Einfügen des i -ten Elementes ausgeführte Anzahl der Schlüsselvergleiche und Bewegungen von Datensätzen hängt offenbar eng zusammen mit der erwarteten Anzahl von Elementen, die im Anfangsstück $a[1], \dots, a[i-1]$ in der falschen Reihenfolge bezüglich des i -ten Elements stehen. Man nennt diese Zahl die erwartete Anzahl von *Inversionen* (Fehlstellungen), an denen das i -te Element beteiligt ist.

Genauer: Ist $k_1, \dots, k_N$ eine gegebene Permutation π von N verschiedenen Zahlen, so heißt ein Paar (k_i, k_j) eine Inversion, wenn $i < j$, aber $k_i > k_j$ ist. Die Gesamtzahl der Inversionen einer Permutation π heißt *Inversionszahl* von π . Sie kann als Maß für die „Vorsortiertheit“ von π verwendet werden. Sie ist offenbar 0 für die aufsteigend sortierte Folge und $\sum_{i=1}^N (N-i) = \Theta(N^2)$ für die absteigend sortierte Folge. Im Mittel kann man erwarten, daß die Hälfte der dem i -ten Element k_i vorangehenden Elemente größer als k_i ist. Die mittlere Anzahl von Inversionen und damit die mittlere Anzahl von Schlüsselvergleichen und Bewegungen von Datensätzen, die die Prozedur *Einfügesort* ausführt, ist damit von der Größenordnung

$$\sum_{i=1}^N \frac{i}{2} = \Theta(N^2).$$

Wir werden später (im Abschnitt 2.6) Sortierverfahren kennenlernen, die die mit der Inversionszahl gemessene Vorsortierung in einem noch zu präzisierenden Sinne optimal nutzen.

Es ist naheliegend zu versuchen, das Sortieren durch Einfügen dadurch zu verbessern, daß man ein besseres Suchverfahren zur Bestimmung der Einfügestelle für das i -te Element verwendet. Das hilft aber nur wenig. Nimmt man beispielsweise das im Kapitel 3 besprochene binäre Suchen an Stelle des in der angegebenen Prozedur benutzten linearen Suchens, so kann man zwar die Einfügestelle mit $\log i$ Schlüsselvergleichen bestimmen, muß aber immer noch im schlechtesten Fall i und im Mittel $i/2$ Bewegungen von Datensätzen ausführen, um für das i -te Element Platz zu machen. Ein wirklich besseres Verfahren zum Sortieren von N Datensätzen, das auf der dem Sortieren durch Einfügen zugrunde liegenden Idee basiert, erhält man dann, wenn man die zu sortierenden Sätze in einer ganz anderen Datenstruktur (nicht als Array) speichert. Es gibt Strukturen, die das Einfügen eines Elementes mit einer Gesamtschrittzahl (das ist mindestens die Anzahl von Schlüsselvergleichen und Bewegungen) erlauben, die proportional zu $\log d$ ist; dabei ist d der Abstand der aktuellen Einfügestelle von der jeweils vorangehenden. Wir besprechen ein darauf gegründetes Sortierverfahren (Sortieren durch *lokales Einfügen*) im Abschnitt 2.6.

2.1.3 Shellsort

Method: Anstelle des beim Sortieren durch Einfügen benutzten wiederholten Verschiebens um eine Position nach rechts versuchen wir, die Elemente in größeren Sprüngen schneller an ihre endgültige Position zu bringen. Zunächst wählen wir eine abnehmende und mit 1 endende Folge von sogenannten Inkrementen $h_t, h_{t-1}, \dots, h_1$. Das ist

eine Folge positiv ganzzahliger Sprungweiten, z.B. die Folge 5, 3, 1. Dann betrachten wir der Reihe nach für jedes der abnehmenden Inkremente h_i , $t \geq i \geq 1$, alle Elemente im Abstand h_i voneinander. Man kann also die gegebene Folge auffassen als eine Menge von (höchstens) h_i verzahnt gespeicherten, logisch aber unabhängigen Folgen f_j , $1 \leq j \leq h_i$. Die zur Folge f_j gehörenden Elemente stehen im Feld an Positionen $j, j + h_i, j + 2h_i, \dots$, also an Positionen $j + m \cdot h_i$, $0 \leq m \leq \lfloor \frac{N-j}{h_i} \rfloor$. Anfangs haben wir h_t solcher Folgen, später, bei $h_1 = 1$, gerade eine einzige Folge. Für jedes h_i , $t \geq i \geq 1$, sortieren wir jede Folge f_j , $1 \leq j \leq h_i$, mittels Einfügesort. Weil das in f_j zu einem Folgeelement benachbarte Element um h_i Positionen versetzt im Feld gespeichert ist, bewirkt dieses Sortiervorgehen, daß ein Element bei einer Bewegung um h_i Positionen nach rechts wandert. Der letzte dieser t Durchgänge ist identisch mit dem gewöhnlichen Einfügesort; nur müssen die Elemente jetzt nicht mehr so weit nach rechts verschoben werden, da sie vorher schon ein gutes Stück gewandert sind.

Diese auf D.L. Shell zurückgehende Methode nennt man auch *Sortieren mit abnehmenden Inkrementen*. Man nennt eine Folge $k_1, \dots, k_N$ von Schlüsseln *h-sortiert*, wenn für alle i , $1 \leq i \leq N - h$, gilt: $k_i \leq k_{i+h}$. Für eine abnehmende Folge $h_t, \dots, h_1 = 1$ von Inkrementen wird also mit Hilfe von Sortieren durch Einfügen eine h_t -sortierte, dann eine h_{t-1} -sortierte usw. und schließlich eine 1-sortierte, also eine sortierte Folge hergestellt.

Beispiel: Betrachten wir das Feld a mit den sieben Schlüsseln 15, 2, 43, 17, 4, 8, 47 und die Folge 5, 3, 1 von Inkrementen. Wir betrachten zunächst die Elemente im Abstand 5 voneinander.

15 2 43 17 4 8 47

Um daraus mittels Sortieren durch Einfügen eine 5-sortierte Folge zu machen, wird das Element mit Schlüssel 15 mit dem Element mit Schlüssel 8 vertauscht und somit um fünf Positionen nach rechts verschoben. Außerdem wird 2 mit 47 verglichen, aber nicht vertauscht. Wir erhalten:

8 2 43 17 4 15 47

Jetzt betrachten wir alle Folgen von Elementen mit Abstand 3 und erhalten nach Sortieren:

8 2 15 17 4 43 47

Diese 3-sortierte Folge wird jetzt — wie beim Sortieren durch Einfügen — endgültig 1-sortiert. Dazu müssen jetzt nur noch insgesamt vier Verschiebungen um je eine Position nach rechts durchgeführt werden. Insgesamt wurden sechs Bewegungen (ein 5-er Sprung, ein 3-er Sprung und vier 1-er Sprünge) ausgeführt. Sortieren der Ausgangsfolge mit Einfügesort erfordert dagegen acht Bewegungen.

procedure Shellsort (**var** a : sequence);

var

i, j, k : integer;

t : item; {Hilfsspeicher}

continue : boolean; {für Schleifenabbruch}

begin

for each h {einer endlichen, abnehmenden, mit 1 endenden

```

    Folge von Inkrementen} do
    {stelle h-sortierte Folge her}
    for  $i := h + 1$  to  $N$  do
        begin
             $j := i$ ;
             $t := a[i]$ ;
             $k := t.key$ ;
             $continue := true$ ;
            while  $(a[j - h].key > k)$  and  $continue$  do
                begin
                    {h-Sprung nach rechts}
                     $a[j] := a[j - h]$ ;
                     $j := j - h$ ;
                     $continue := (j > h)$ 
                end;
             $a[j] := t$ 
        end
    end

```

Die wichtigste Frage im Zusammenhang mit dem oben angegebenen Verfahren Shellsort ist, welche Folge von abnehmenden Inkrementen man wählen soll, um die Gesamtzahl der Bewegungen möglichst gering zu halten. Auf diese Frage hat man inzwischen eine ganze Reihe überraschender, aber insgesamt doch nur unvollständiger Antworten erhalten. Beispielsweise kann man zeigen, daß die Laufzeit des Verfahrens $O(N \log^2 N)$ ist, wenn als Inkremente alle Zahlen der Form $2^p 3^q$ gewählt werden, die kleiner als N sind (vgl. [1]). Ein weiteres bemerkenswertes Resultat ist, daß das Herstellen einer h -sortierten Folge aus einer bereits k -sortierten Folge (wie im Verfahren Shellsort für abnehmende Inkremente k und h) die k -Sortiertheit der Folge nicht zerstört.

2.1.4 Bubblesort

Methode: Läßt man als Bewegungen nur das wiederholte Vertauschen benachbarter Datensätze zu, so kann eine nach aufsteigenden Schlüsseln sortierte Folge von Datensätzen offensichtlich wie folgt hergestellt werden. Man durchläuft die Liste $a[1], \dots, a[N]$ der Datensätze und betrachtet dabei je zwei benachbarte Elemente $a[i]$ und $a[i + 1]$, $1 \leq i < N$. Ist $a[i].key > a[i + 1].key$, so vertauscht man $a[i]$ und $a[i + 1]$. Nach diesem ersten Durchlauf ist das größte Element an seinem richtigen Platz am rechten Ende angelangt. Dann geht man die Folge erneut durch und vertauscht, falls nötig, wiederum je zwei benachbarte Elemente. Dieses Durchlaufen wird solange wiederholt, bis keine Vertauschungen mehr aufgetreten sind; d.h. alle Paare benachbarter Sätze stehen in der richtigen Reihenfolge. Damit ist das Feld a nach aufsteigenden Schlüsseln sortiert. Größere Elemente haben also die Tendenz, wie Luftblasen im Wasser langsam nach oben aufzusteigen. Diese Analogie hat dem Verfahren den Namen Bubblesort eingebracht.

Beispiel: Betrachten wir wieder das Feld a mit den sieben Schlüsseln 15, 2, 43, 17, 4, 8, 47. Beim ersten Durchlauf werden folgende Vertauschungen benachbarter Elemente durchgeführt.

```

15,  2
2,  15, 43, 17
      17, 43,  4
            4, 43,  8
                  8, 43, 47

```

Nach dem ersten Durchlauf ist die Reihenfolge der Schlüssel des Feldes a also

2, 15, 17, 4, 8, 43, 47.

Der zweite Durchlauf liefert die Reihenfolge

2, 15, 4, 8, 17, 43, 47.

Der dritte Durchlauf liefert schließlich

2, 4, 8, 15, 17, 43, 47,

also die aufsteigend sortierte Folge von Sätzen. Beim Durchlaufen dieser Folge müssen keine benachbarten Elemente mehr vertauscht werden. Das zeigt den erfolgreichen Abschluß des Sortierverfahrens. Man erhält damit das folgende naheliegende Programmgerüst des Verfahrens:

```

procedure bubblesort (var  $a$  : sequence);
  var
     $i$  : integer;
  begin
    repeat
      for  $i := 1$  to  $(N - 1)$  do
        if  $a[i].key > a[i + 1].key$ 
          then {vertausche  $a[i]$  und  $a[i + 1]$ }
        until {keine Vertauschung mehr aufgetreten}
    end

```

Bei Verwenden einer booleschen Variablen für den Test, ob Vertauschungen auftraten, kann das Verfahren wie folgt als Pascal-Prozedur geschrieben werden:

```

procedure bubblesort (var  $a$  : sequence);
  var
     $i$  : integer;
    nichtvertauscht : boolean;
     $t$  : item; {Hilfsspeicher}
  begin
    repeat
      nichtvertauscht := true;

```

```

for  $i := 1$  to  $(N - 1)$  do
  { *}   if  $a[i].key > a[i + 1].key$ 
    then
      begin
        { ** }    $t := a[i];$ 
        { ** }    $a[i] := a[i + 1];$ 
        { ** }    $a[i + 1] := t;$ 
                 $nichtvertauscht := false$ 
      end
    until  $nichtvertauscht$ 
  end

```

An dieser Stelle wollen wir noch auf eine kleine Effizienzverbesserung mit Hilfe eines Programmiertricks hinweisen. Sind alle Schlüssel verschieden, so kann man die Prüfung, ob bei einem Durchlauf noch eine Vertauschung auftrat, ohne eine boolesche Variable wie folgt testen. Man besetzt zu Beginn der **repeat**-Schleife die für die Vertauschung vorgesehene Hilfsspeichervariable t mit dem Wert von $a[1]$. Tritt beim Durchlaufen wenigstens eine Vertauschung zweier Elemente $a[i]$ und $a[i + 1]$ mit $i > 1$ auf, hat t am Ende des Durchlaufs nicht mehr denselben Wert wie zu Beginn. Der Wert von t kann am Ende des Durchlaufs höchstens dann noch den ursprünglichen Wert $a[1]$ haben, wenn entweder keine Vertauschung aufgetreten ist oder die einzige beim Durchlauf vorgenommene Vertauschung die der Elemente $a[1]$ und $a[2]$ war. In beiden Fällen ist das Feld am Ende des Durchlaufs sortiert.

Die eben skizzierte Möglichkeit zur Implementation des Verfahrens Bubblesort ist in ganz seltenen Fällen besser als die von uns angegebene, da unter Umständen ein „unnötiges“ Durchlaufen eines bereits nach aufsteigenden Schlüsseln sortierten Feldes vermieden wird.

Wir haben stets das ganze Feld durchlaufen, obwohl das natürlich nicht nötig ist. Denn nach dem i -ten Durchlauf befinden sich die i größten Elemente bereits am rechten Ende. Man erhält also eine Effizienzverbesserung auch dadurch, daß man im i -ten Durchlauf nur die Elemente an den Positionen $1, \dots, (N - i) + 1$ inspiziert. Wir überlassen es dem Leser, sich zu überlegen, wie man das implementieren kann.

Analyse: Die Abschätzung der im günstigsten und schlechtesten Fall ausgeführten Anzahlen von Schlüsselvergleichen (in Programmzeile { * }) und Bewegungen von Datensätzen (in Programmzeilen { ** }) ist einfach. Ist das Feld a bereits nach aufsteigenden Schlüsseln sortiert, so wird die **for**-Schleife des oben angegebenen Programms genau einmal durchlaufen und dabei keine Vertauschung vorgenommen. Also ist

$$C_{min}(N) = N - 1, \quad M_{min}(N) = 0.$$

Der ungünstigste Fall für das Verfahren Bubblesort liegt vor, wenn das Feld a anfangs nach absteigenden Schlüsseln sortiert ist. Dann rückt das Element mit minimalem Schlüssel bei jedem Durchlauf der **repeat**-Schleife um eine Position nach vorn. Es sind dann N Durchläufe nötig, bis es ganz vorn angelangt ist und festgestellt wird, daß keine Vertauschung mehr aufgetreten ist (wendet man den erwähnten Programmiertrick an, so sind es nur $N - 1$ Durchläufe). Es ist nicht schwer zu sehen, daß in diesem Fall beim i -ten Durchlauf, $1 \leq i < N$, $(N - i)$ Vertauschungen benachbarter Elemente, also $3(N - i)$ Bewegungen, und natürlich jedesmal $N - 1$ Schlüsselvergleiche ausgeführt

werden. Damit ist:

$$\begin{aligned} C_{\max} &= N(N-1) = \Theta(N^2) \\ M_{\max} &= \sum_{i=1}^{N-1} 3(N-i) = \Theta(N^2) \end{aligned}$$

Man kann zeigen, daß auch

$$C_{\text{mit}}(N) = M_{\text{mit}}(N) = \Theta(N^2)$$

gilt (vgl. ).

Wir verzichten hier auf diesen Nachweis, denn Bubblesort ist ein zwar durchaus populäres, aber im Grunde schlechtes elementares Sortierverfahren. Nur für den Fall, daß ein bereits nahezu vollständig sortiertes Feld (für das die Inversionszahl der Schlüsselfolge klein ist) vorliegt, werden wenige Vergleichsoperationen und Bewegungen von Datensätzen ausgeführt. Das Verfahren ist stark asymmetrisch bezüglich der Durchlaufrichtung. Ist z.B. die Ausgangsfolge schon „fast sortiert“, d. h. gilt für $k_1, \dots, k_N$

$$k_i \leq k_{i+1}, \quad 1 \leq i < N-1,$$

und ist k_N das minimale Element, so sind $N-1$ Durchläufe nötig, um es an den Anfang zu schaffen. Man hat versucht, diese Schwäche dadurch zu beheben, daß man das Feld a abwechselnd von links nach rechts und umgekehrt durchläuft. Diese (geringfügig bessere) Variante ist als *Shakersort* bekannt. Außer einem schönen Namen hat das Verfahren aber keine Vorzüge, wenn man es etwa mit dem Verfahren Sortieren durch Einfügen vergleicht.

2.2 Quicksort

Wir stellen in diesem Abschnitt ein Sortierverfahren vor, das 1962 von C.A.R. Hoare  veröffentlicht wurde und den Namen Quicksort erhielt, weil es erfahrungsgemäß eines der schnellsten, wenn nicht *das* schnellste interne Sortierverfahren ist. Das Verfahren folgt der Divide-and-conquer-Strategie zur Lösung des Sortierproblems. Es benötigt zwar, wie die elementaren Sortierverfahren, $\Omega(N^2)$ viele Vergleichsoperationen zwischen Schlüsseln im schlechtesten Fall, im Mittel werden jedoch nur $O(N \log N)$ viele Vergleichsoperationen ausgeführt. Quicksort operiert auf den Elementen eines Feldes a von Datensätzen mit Schlüsseln, die wir ohne Einschränkung als ganzzahlig annehmen. Es ist ein sogenanntes In-situ-Sortierverfahren. Das bedeutet, daß zur (Zwischen-) Speicherung für die Datensätze kein zusätzlicher Speicher benötigt wird, außer einer konstanten Anzahl von Hilfsspeicherplätzen für Tauschoperationen. Nur für die Verwaltung der Information über noch nicht vollständig abgearbeitete und durch rekursive Anwendung der Divide-and-conquer-Strategie generierte Teilprobleme wird zusätzlicher Speicherplatz benötigt. Quicksort kann auf viele verschiedene Arten implementiert werden. Wir geben im Abschnitt 2.2.1 eine naheliegende Version an und analysieren das Verhalten im schlechtesten Fall, im besten Fall und im Mittel. Im Abschnitt 2.2.2 besprechen

wir einige Varianten des Verfahrens, die unter bestimmten Voraussetzungen ein besseres Verhalten liefern. Als Beispiel behandeln wir insbesondere den Fall, daß das zu sortierende Feld viele Sätze mit gleichen Schlüsseln hat — ein in der Praxis durchaus nicht seltener Fall.

Quicksort ist sehr empfindlich gegen minimale Programmänderungen. Jede Version einer Implementation muß sorgfältig daraufhin überprüft werden, ob sie auch wirklich in allen Fällen das korrekte Ergebnis liefert.

2.2.1 Quicksort: Sortieren durch rekursives Teilen

Methode: Um eine Folge $F = k_1, \dots, k_N$ von N Schlüsseln nach aufsteigenden Werten zu sortieren, wählen wir ein beliebiges Element $k \in \{k_1, \dots, k_N\}$ und benutzen es als Angelpunkt, genannt *Pivotelement*, für eine Aufteilung der Folge ohne k in zwei Teilfolgen F_1 und F_2 . F_1 besteht nur aus Elementen von F , die kleiner oder gleich k sind, F_2 nur aus Elementen von F , die größer oder gleich k sind. Ist F_1 eine Folge mit $i - 1$ Elementen und F_2 eine Folge mit $N - i$ Elementen, so ist i die endgültige Position des Pivotelements k . Also kann man das Sortierproblem dadurch lösen, daß man F_1 und F_2 rekursiv auf dieselbe Weise sortiert und die Ergebnisse in offensichtlicher Weise zusammensetzt. Zuerst kommt die durch Sortieren von F_1 entstandene Folge, dann das Pivotelement k (an Position i) und dann die durch Sortieren von F_2 entstandene Folge.

Läßt man alle Implementationsdetails zunächst weg, so kann die Struktur des Verfahrens auf einer hohen sprachlichen Ebene wie folgt beschrieben werden.

Algorithmus Quicksort (F : Folge);
 {sortiert die Folge F nach aufsteigenden Werten}
Falls F die leere Folge ist oder F nur aus einem einzigen Element besteht, bleibt F unverändert; sonst:
Divide: Wähle ein Pivotelement k von F (z.B. das letzte) und teile F ohne k in Teilfolgen F_1 und F_2 bzgl. k :
 F_1 enthält nur Elemente von F ohne k , die $\leq k$ sind,
 F_2 enthält nur Elemente von F ohne k , die $\geq k$ sind;
Conquer: Quicksort(F_1); Quicksort(F_2);
 {nach Ausführung dieser beiden Aufrufe sind F_1 und F_2 sortiert}
Merge: Bilde die Ergebnisfolge F durch Hintereinanderhängen von F_1 , k , F_2 in dieser Reihenfolge.

Der für die Implementation des Verfahrens wesentliche Schritt ist der Aufteilungsschritt. Die Aufteilung bzgl. eines gewählten Pivotelementes soll in situ, d.h. am Ort, an dem die ursprünglichen Sätze abgelegt sind, ohne zusätzlichen, von der Anzahl der zu sortierenden Folgeelemente abhängigen Speicherbedarf erfolgen. Die als Ergebnis der Aufteilung entstehenden Teilfolgen F_1 und F_2 könnte man programmtechnisch als Arrays geringerer Länge zu realisieren versuchen. Das würde bedeuten, eine rekursive Prozedur *quicksort* zu schreiben mit einem Array variabler Länge als Ein- und Ausgabeparameter. Das ist in der Programmiersprache Pascal nicht möglich — und glücklicherweise auch nicht nötig. Wir schreiben eine Prozedur, die das als Ein- und Ausgabeparameter gegebene Feld a der Datensätze verändert. Die zwischendurch durch

Aufteilen entstandenen Teilfolgen werden durch ein Paar von Zeigern (Indizes) auf das Array realisiert. Diese Zeiger werden Eingabeparameter der Prozedur *quicksort*.

procedure *quicksort* (**var** *a* : *sequence*; *l, r* : *integer*);
 {sortiert die Elemente $a[l], \dots, a[r]$ des Feldes *a*
 nach aufsteigenden Schlüsseln}

Ein Aufruf der Prozedur *quicksort*(*a*, 1, *N*) sortiert also die gegebene Folge von Datensätzen.

Als Pivotelement *v* wollen wir den Schlüssel des Elements $a[r]$ am rechten Ende des zu sortierenden Teilfeldes wählen. Eine In-situ-Aufteilung des Bereiches $a[l], \dots, a[r]$ bzgl. *v* kann man nun wie folgt erreichen. Man wandert mit einem Positionszeiger *i* vom linken Ende des aufzuteilenden Bereiches nach rechts über alle Elemente hinweg, deren Schlüssel kleiner ist als *v*, bis man schließlich ein Element mit $a[i].key \geq v$ trifft. Symmetrisch dazu wandert man mit Zeiger *j* vom rechten Ende des aufzuteilenden Bereiches nach links über alle Elemente hinweg, deren Schlüssel größer ist als *v*, bis man schließlich ein Element mit $a[j].key \leq v$ trifft. Dann vertauscht man $a[i]$ mit $a[j]$, wodurch beide bezüglich *v* in der richtigen Teilfolge stehen.

Das wird solange wiederholt, bis die Teilfolge $a[l], \dots, a[r]$ vollständig inspiziert ist. Das kann man daran feststellen, daß die Zeiger *i* und *j* übereinander hinweg gelaufen sind. Wenn dieser Fall eintritt, hat man zugleich auch die endgültige Position des Pivotelementes gefunden. Wir wollen jetzt dieses Vorgehen als Pascal-Prozedur realisieren. Dazu ist es bequem, die sprachlichen Möglichkeiten von Pascal um ein allgemeineres Schleifenkonstrukt zu erweitern. Wir verwenden eine Schleife der Form

begin-loopS
 <statement>
if <condition>
 then exit-loop;
 <statement>
end-loop

mit offensichtlicher Bedeutung.

procedure *quicksort* (**var** *a* : *sequence*; *l, r* : *integer*);
var
 v, i, j : *integer*;
 t : *item*; {Hilfsspeicher}
begin
 if *r* > *l*
 then
 begin
 i := *l* - 1;
 j := *r*;
 v := $a[r].key$; {Pivotelement}
 begin-loop

```
{*}      repeat i := i + 1 until a[i].key ≥ v;
{*}      repeat j := j - 1 until a[j].key ≤ v;
        if i ≥ j
            then {i ist Pivotposition}
            exit-loop;
        {**} t := a[i];
        {**} a[i] := a[j];
        {**} a[j] := t
        end-loop;
        {**} t := a[i];
        {**} a[i] := a[r];
        {**} a[r] := t;
        quicksort(a, l, i - 1);
        quicksort(a, i + 1, r)
    end
end
```

Wir erläutern den Aufteilungsschritt am Beispiel eines Aufrufs von *quicksort*(*a*, 4, 9) für den Fall, daß die Folge der Schlüssel im Bereich *a*[4]...*a*[9] die Schlüssel 5, 7, 3, 1, 6, 4 sind, vgl. Tabelle 2.1.

Array-Position	3	4	5	6	7	8	9	10	...
Schlüssel	...	5	7	3	1	6	4	...	4 ist Pivot-Element
		$\uparrow_i$					$\uparrow_j$		
			$\uparrow_i$		$\uparrow_j$				1. Halt der Zeiger <i>i</i> , <i>j</i>
	...	1	7	3	5	6	4	...	
			$\uparrow_i$	$\uparrow_j$					2. Halt der Zeiger <i>i</i> , <i>j</i>
	...	1	3	7	5	6	4	...	
			$\uparrow_j$	$\uparrow_i$					letzter Halt der Zeiger <i>i</i> , <i>j</i>
	...	1	3	4	5	6	7	...	
			$\uparrow_j$	$\uparrow_i$					

Tabelle 2.1

Da wir als Pivotelement *v* das Element am rechten Ende des aufzuteilenden Bereichs gewählt haben, ist klar, daß es im Bereich, den der Zeiger *i* beim Wandern nach rechts überstreicht, stets ein Element mit Schlüssel $\geq v$ gibt. Die erste der beiden **repeat**-Schleifen terminiert also sicher. Die zweite **repeat**-Schleife terminiert aber dann nicht korrekt, wenn das Pivotelement der minimale Schlüssel unter allen Schlüssel_{en} im Bereich *a*[*l*]...*a*[*r*] ist. Dann gibt es nämlich kein $j \in \{r - 1, r - 2, \dots, l\}$ mit *a*[*j*].key ≤ *v*. Die zweite **repeat**-Schleife terminiert sicher dann, wenn an Position *l* - 1 ein Element steht, für das gilt *a*[*l* - 1].key ≤ *a*[*j*].key für alle j mit $l \leq j \leq r$. Das kann man für den

ersten Aufruf $quicksort(a, 1, N)$ sichern durch Abspeichern eines Stoppers an Position 0 mit $a[0].key \leq \min_i \{a[i].key\}$. Bei allen rekursiven Aufrufen ist die entsprechende Bedingung von selbst gesichert. Das zeigt folgende Überlegung. Unter der Annahme, daß es vor einem Aufruf von $quicksort(a, l, r)$ ein Element an Position $l - 1$ gibt mit $a[l - 1].key \leq a[j].key$ für alle j mit $l \leq j \leq r$, gilt die entsprechende Bedingung auch, bevor die rekursiven Aufrufe $quicksort(a, l, i - 1)$ und $quicksort(a, i + 1, r)$ ausgeführt werden. Das ist trivial für den erstgenannten Aufruf. Ferner hat die vorangehende Aufteilung bewirkt, daß an Position i ein Element steht mit $a[i].key \leq a[j].key$ für alle j mit $i + 1 \leq j \leq r$.

Wir haben die Abbruchbedingungen für die **repeat**-Schleifen übrigens so gewählt, daß das Verfahren auch auf Felder anwendbar ist, in denen dieselben Schlüssel mehrfach auftreten. Es werden in diesem Fall allerdings unnötige Vertauschungen vorgenommen und die Elemente mit gleichem Schlüssel können ihre relative Position ändern. Wir zeigen im Abschnitt 2.2.2 eine Möglichkeit, das zu vermeiden.

Analyse: Wir schätzen zunächst die im *schlechtesten Fall* bei einem Aufruf von $quicksort(a, 1, N)$ auszuführende Anzahl von Schlüsselvergleichen (in den Programmzeilen $\{*\}$) sowie die Anzahl der Bewegungen (in Programmzeilen $\{**\}$) ab. Zur Aufteilung eines Feldes der Länge N werden die Schlüssel aller Elemente im aufzuteilenden Bereich mit dem Pivotelement verglichen. In der Regel werden zwei Schlüssel je zweimal mit dem Pivotelement verglichen. Es sind immer $N + 1$ Vergleiche insgesamt, wenn das Pivotelement in den $N - 1$ Restelementen nicht vorkommt, sonst N Vergleiche. Im ungünstigsten Fall wechseln dabei alle Elemente je einmal ihren Platz. Die im ungünstigsten Fall auszuführende Anzahl von Schlüsselvergleichen und Bewegungen hängt damit stark von der Anzahl der Aufteilungsschritte und damit von der Zahl der initiierten rekursiven Aufrufe ab. Ist das Pivotelement das Element mit kleinstem oder größtem Schlüssel, ist eine der beiden durch Aufteilung entstehenden Folgen jeweils leer und die andere hat jeweils ein Element (nämlich das Pivotelement) weniger als die Ausgangsfolge. Dieser Fall tritt z.B. für eine bereits aufsteigend sortierte Folge von Schlüsseln ein. Die in diesem Fall initiierte Folge von rekursiven Aufrufen kann man durch den Baum in Abbildung 2.1 veranschaulichen.

Damit ist klar, daß zum Sortieren von N Elementen mit Quicksort für die maximale Anzahl $C_{max}(N)$ von auszuführenden Schlüsselvergleichen gilt

$$C_{max}(N) \geq \sum_{i=2}^N (i + 1) = \Omega(N^2).$$

Genauso gilt auch für die maximale Anzahl von Bewegungen

$$M_{max}(N) = \Omega(N^2).$$

Quicksort benötigt im schlechtesten Fall also quadratische Schrittzahl.

Im günstigsten Fall haben die durch Aufteilung entstehenden Teilfolgen stets etwa gleiche Länge. Dann hat der Baum der initiierten rekursiven Aufrufe die minimale Höhe (ungefähr $\log N$) wie im Beispiel von Abbildung 2.2 mit 15 Schlüsseln.

Zur Aufteilung aller Folgen auf einem Niveau werden $\Theta(N)$ Schlüsselvergleiche durchgeführt. Da der Aufrufbaum im günstigsten Fall nur die Höhe $\log N$ hat, folgt unmittelbar

$$C_{min}(N) = \Theta(N \log N).$$

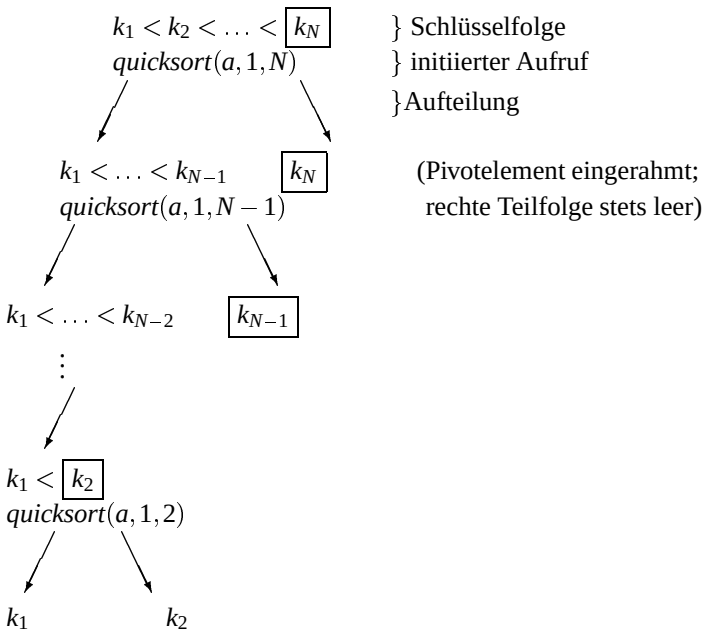


Abbildung 2.1

Es ist nicht schwer zu sehen, daß die Gesamtlaufzeit von Quicksort im günstigsten Fall durch $\Theta(N \log N)$ abgeschätzt werden kann.

Wir wollen jetzt zeigen, daß die mittlere Laufzeit von Quicksort nicht viel schlechter ist als die Laufzeit im günstigsten Fall. Um das zu zeigen, gehen wir von folgenden Annahmen aus. Erstens nehmen wir an, daß alle N Schlüssel paarweise verschieden voneinander sind. Wir können daher für Quicksort ohne Einschränkung voraussetzen, daß die Schlüssel die Zahlen $1, \dots, N$ sind. Zweitens betrachten wir jede der $N!$ möglichen Anordnungen von N Schlüsseln als gleichwahrscheinlich.

Wird Quicksort für eine Folge $k_1, \dots, k_N$ von Schlüsseln aufgerufen, so folgt aus den Annahmen, daß jede Zahl k , $1 \leq k \leq N$, mit gleicher Wahrscheinlichkeit $1/N$ an Position N auftritt und damit als Pivotelement gewählt wird. Wird k Pivotelement, so werden durch Aufteilung zwei Folgen mit Längen $k-1$ und $(N-k)$ erzeugt, für die *quicksort* rekursiv aufgerufen wird. Man kann nun zeigen, daß die durch Aufteilung entstehenden Teilfolgen wieder „zufällig“ sind, wenn die Ausgangsfolge „zufällig“ war. Durch Aufteilung sämtlicher Folgen $k_1, \dots, k_N$ mit $k_N = k$ erhält man wieder sämtliche Folgen von $k-1$ und $(N-k)$ Elementen. Das erlaubt es, unmittelbar eine Rekursionsformel für die mittlere Laufzeit $T(N)$ des Verfahrens Quicksort zum Sortieren von N Schlüsseln aufzustellen. Offensichtlich ist $T(1) = a$, für eine Konstante a . Falls $N \geq 2$ ist, gilt mit einer Konstanten b

$$T(N) \leq \frac{1}{N} \cdot \sum_{k=1}^N (T(k-1) + T(N-k)) + bN.$$

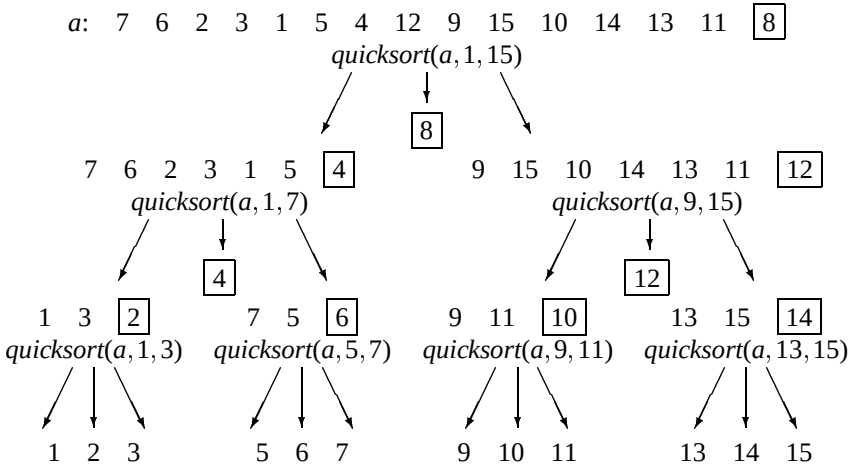


Abbildung 2.2

In dieser Formel gibt der Term bN den Aufteilungsaufwand für eine Folge der Länge N an.

Es folgt für $N \geq 2$, da $T(0) = 0$ ist,

$$T(N) \leq \frac{2}{N} \cdot \sum_{k=1}^{N-1} T(k) + bN.$$

Wir zeigen per Induktion, daß hieraus

$$T(N) \leq c \cdot N \log N$$

für $N \geq 2$ mit einer hinreichend groß gewählten Konstanten c folgt (dabei nehmen wir an, daß N gerade ist; der Fall, daß N ungerade ist, läßt sich analog behandeln).

Der Induktionsanfang ist klar. Sei nun $N \geq 3$, und setzen wir für alle $i < N$ voraus, daß bereits $T(i) \leq c \cdot i \cdot \log i$ gilt. Dann folgt:

$$\begin{aligned}
 T(N) &\leq \frac{2}{N} \left[\sum_{k=1}^{N-1} T(k) \right] + bN \\
 &\leq \frac{2c}{N} \left[\sum_{k=1}^{N-1} k \cdot \log k \right] + bN \\
 &= \frac{2c}{N} \left[\sum_{k=1}^{\frac{N}{2}} k \cdot \underbrace{\log k}_{\leq \log N - 1} + \sum_{k=1}^{\frac{N}{2}-1} \left(\frac{N}{2} + k \right) \underbrace{\log \left(\frac{N}{2} + k \right)}_{\leq \log N} \right] + bN \\
 &\leq \frac{2c}{N} \left[\frac{N}{4} \left(\frac{N}{2} + 1 \right) \log N - \frac{N^2}{8} - \frac{N}{4} + \left(\frac{3N^2}{8} - \frac{3N}{4} \right) \log N \right] + bN
 \end{aligned}$$

$$\begin{aligned}
&= \frac{2c}{N} \left[\left(\frac{N^2}{2} - \frac{N}{2} \right) \log N - \frac{N^2}{8} - \frac{N}{4} \right] + bN \\
&= c \cdot N \log N - \underbrace{c \cdot \log N}_{\geq 0} - \frac{cN}{4} - \frac{c}{2} + bN \\
&\leq c \cdot N \log N - \frac{cN}{4} - \frac{c}{2} + bN
\end{aligned}$$

Haben wir jetzt $c \geq 4b$ gewählt, so folgt unmittelbar

$$T(N) \leq c \cdot N \log N.$$

Damit ist bewiesen, daß Quicksort im Mittel $O(N \log N)$ Zeit benötigt. Wir haben die gesamte mittlere Laufzeit von Quicksort abgeschätzt. Eine entsprechende Rekursionsgleichung gilt natürlich auch für die mittlere Anzahl von Schlüsselvergleichen, die damit ebenfalls im Mittel $O(N \log N)$ ist.

Quicksort benötigt außer einer einzigen Hilfsspeicherstelle beim Aufteilen eines Feldes keinen zusätzlichen Speicher zur Zwischenspeicherung von Datensätzen. Wie bei jeder rekursiven Prozedur muß aber Buch geführt werden über die begonnenen, aber noch nicht abgeschlossenen rekursiven Aufrufe der Prozedur *quicksort*. Das können bis zu $\Omega(N)$ viele Aufrufe sein. Eine Möglichkeit, den zusätzlich benötigten Speicherplatz in der Größenordnung von $O(\log N)$ zu halten, besteht darin, jeweils das kleinere der beiden durch Aufteilung erhaltenen Teilprobleme zuerst (rekursiv) zu lösen. Das größere Teilproblem kann dann nicht auch rekursiv gelöst werden, weil sonst die Schachtelungstiefe der rekursiven Aufrufe weiterhin linear in der Anzahl der Folgelemente sein könnte, wie etwa im Falle einer vorsortierten Folge. Man behilft sich, indem man die sich ergebenden größeren Teilprobleme durch eine Iteration löst. Damit ergibt sich folgender Block der Prozedur *quicksort*.

```

begin {quicksort mit logarithmisch beschränkter Rekursionstiefe}
  while  $r > l$  do
    begin
      {wähle Pivot-Element und teile Folge auf wie bisher}
      {statt zweier rekursiver Aufrufe verfare wie folgt:}
      if  $(i - 1 - l) \leq (r - i - 1)$ 
        then
          begin
            {rekursiver Aufruf für  $a[l] \dots a[i - 1]$ }
            quicksort( $a, l, i - 1$ );
            {Iteration für  $a[i + 1] \dots a[r]$ }
             $l := i + 1$ 
          end
        else
          begin
            {rekursiver Aufruf für  $a[i + 1] \dots a[r]$ }
            quicksort( $a, i + 1, r$ );
            {Iteration für  $a[l] \dots a[i - 1]$ }

```



```

        r := i - 1
    end
end
end {Quicksort}

```

Natürlich kann man Quicksort auch gänzlich iterativ programmieren, indem man sich die Indizes für das linke und das rechte Ende der noch zu sortierenden Teilfolgen merkt (z.B. mit Hilfe eines Stapels, vgl. Kapitel 1). Sortiert man die jeweils kleinere Teilfolge zuerst, so muß man sich nie mehr als $O(\log N)$ Indizes merken.

Nach einem Vorschlag von B. Durian  kann man Quicksort auch mit nur konstantem zusätzlichem Speicherplatz realisieren, ein wenig zu Lasten der Laufzeit. Hier merkt man sich die noch zu sortierenden Teilfolgen nicht explizit, sondern sucht sie in der Gesamtfolge auf. Nach einer Aufteilung wird zuerst die linke, dann die rechte Teilfolge sortiert. Betrachten wir einen Ausschnitt aus dem Ablauf des Sortierprozesses, wie ihn Abbildung 2.3 zeigt.

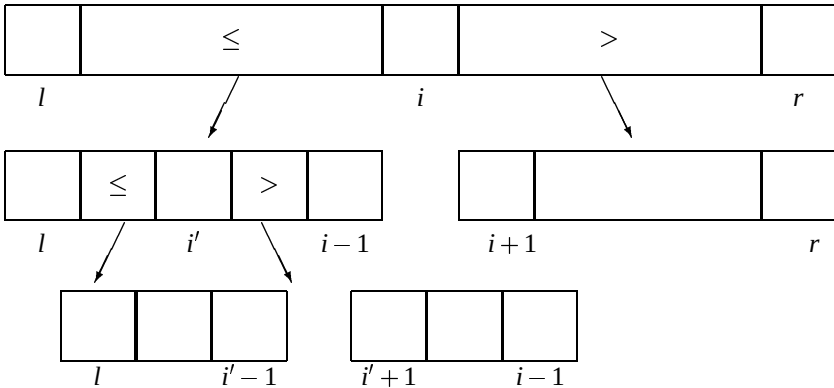


Abbildung 2.3

Von den gezeigten Teilfolgen wird also zuerst $a[l] \dots a[i' - 1]$ sortiert, dann $a[i' + 1] \dots a[i - 1]$ und schließlich $a[i + 1] \dots a[r]$. Das Problem ist nun, daß man zum Sortieren der Teilfolge $a[i' + 1] \dots a[i - 1]$ die rechte Grenze, also den Index $i - 1$, kennen muß. Bisher haben wir uns dies implizit in der Rekursion oder explizit im Stapel gemerkt. Jetzt nutzen wir die Kenntnis aus, daß alle Schlüssel in der Teilfolge $a[i' + 1] \dots a[i - 1]$ höchstens so groß wie das Pivotelement $a[i]$ sein können; alle Schlüssel in $a[i + 1] \dots a[r]$ müssen größer sein. Wir können also, ausgehend von $a[i']$, den Index $i - 1$ finden, wenn wir $a[i].key$ kennen, etwa so:

```

{Sei  $v := a[i].key$ , der Schlüssel des Pivotelements}
  m := i';
  while  $a[m].key \leq v$  do m := m + 1;
  {jetzt ist  $m = i + 1$ }
  m := m - 2;
  {jetzt ist  $m = i - 1$ , der gewünschte Index}

```

Nun muß man natürlich noch $a[i].key$ kennen, ohne den Index i gespeichert zu haben. Das ist aber leicht möglich, wenn wir vor dem Sortieren der Elemente $a[l] \dots a[i' - 1]$ das Element $a[i]$ mit dem Element $a[i' + 1]$ tauschen. Dann ergibt sich

$$v := a[i' + 1].key$$

vor Beginn des gerade angegebenen Programmstücks; das Ausfüllen des Rests des Blockes der Prozedur *quicksort* überlassen wir dem interessierten Leser.

Das asymptotische Laufzeitverhalten von Quicksort ändert sich durch die zusätzlichen Vergleichs- und Bewegungsoperationen nicht, da ja bereits der Aufteilungsschritt lineare Zeit kostet. Verwendet man statt sequentieller Suche binäre Suche nach Position $i - 1$, so ergibt sich eine nur wenig höhere Laufzeit als bei rekursivem Quicksort.

2.2.2 Quicksort-Varianten

Das im vorigen Abschnitt angegebene Verfahren Quicksort benötigt für bereits sortierte oder fast sortierte Eingabefolgen quadratische Schrittzahl. Der Grund dafür ist, daß in diesen Fällen die Wahl des Pivotelementes am rechten Ende des aufzuteilenden Bereichs keine gute Aufteilung des Feldes in zwei nahezu gleich große Teilfelder liefert. Es gibt mehrere Strategien für eine bessere Wahl des Pivotelementes. Die bekanntesten sind die *3-Median-* und die *Zufalls-Strategie*.

Im Falle der *3-Median-Strategie* wird als Pivotelement der Median (d.h. das mittlere) von drei Elementen im aufzuteilenden Bereich gewählt. Wählt man die drei Elemente vom linken und rechten Ende und aus der Mitte, so besteht eine gute Chance dafür, daß das mittlere dieser drei Elemente eine Aufteilung in annähernd gleiche Teile liefert. Um das mittlere von drei Elementen a, b, c zu bestimmen, die nicht paarweise verschieden sein müssen, kann man wie folgt vorgehen.

```

if  $a > b$  then vertausche ( $a, b$ );
 $\{a = \min(a, b)\}$ 
if  $a > c$  then vertausche ( $a, c$ );
 $\{a = \min(a, b, c)\}$ 
if  $b > c$  then vertausche ( $b, c$ );
 $\{a, b, c \text{ sind jetzt aufsteigend sortiert; also ist } b \text{ das mittlere}$ 
   $\text{der drei Elemente } a, b, c\}$ 

```

Setzt man das Element mit dem mittleren der drei Schlüssel $a = a[l].key$, $b = a[r].key$ und $c = a[m].key$ mit $m = (l + r) \text{ div } 2$ vor Beginn der Aufteilung des Bereichs $a[l] \dots a[r]$ an das rechte Ende des aufzuteilenden Bereichs, kann die Aufteilung wie bisher erfolgen. Insgesamt erhalten wir folgende Prozedur:

```

procedure median_of_three_quicksort (var  $a$  : sequence;  $l, r$  : integer);
var
     $v, m, i, j$  : integer;
     $t$  : item; {Hilfsspeicher}
begin
    if  $r > l$ 
    then
        begin
             $m := (r + l) \text{ div } 2$ ;
            if  $a[l].key > a[r].key$ 
            then
                begin
                     $t := a[l]$ ;
                     $a[l] := a[r]$ ;
                     $a[r] := t$ 
                end;
            if  $a[l].key > a[m].key$ 
            then
                begin
                     $t := a[l]$ ;
                     $a[l] := a[m]$ ;
                     $a[m] := t$ 
                end;
            if  $a[r].key > a[m].key$ 
            then
                begin
                     $t := a[r]$ ;
                     $a[r] := a[m]$ ;
                     $a[m] := t$ 
                end;
            {jetzt steht Median von  $a[l]$ ,  $a[m]$  und  $a[r]$  an Position  $r$ ;
             weiter wie bisher ...}
        end
    end
end

```

Im Falle der *Zufalls-Strategie* wählt man das Pivotelement zufällig unter den Schlüsseln im aufzuteilenden Bereich $a[l] \dots a[r]$. Statt einfach $v := a[r].key$ zu setzen, wählt man zunächst k zufällig und gleichverteilt aus dem Bereich der Indizes $l, \dots, r$ und vertauscht $a[k]$ mit $a[r]$, bevor mit der Aufteilung des Bereichs $a[l] \dots a[r]$ begonnen wird. Der Effekt dieser Änderung von Quicksort ist drastisch. Es gibt keine „schlechten“ Eingabefolgen mehr! Das auf diese Weise *randomisierte* (zufällig gemachte) Quicksort behandelt alle Eingabefolgen (annähernd) gleich. Natürlich kann man auch so nicht vermeiden, daß ein schlechterster Fall auftritt, in dem das Verfahren quadratische Schrittzahl benötigt. Man kann aber leicht zeigen (vgl. z.B. [E]), daß der Erwartungswert für die zum Sortieren einer beliebigen, aber festen Eingabefolge mit randomisiertem Quicksort erforderliche Anzahl von Schlüsselvergleichen gleich $O(N \log N)$ ist.

Ob die Implementation und Verwendung von randomisiertem Quicksort zweckmäßig ist, hängt vom jeweiligen Anwendungsfall ab. Im Falle stark vorsortierter Eingabefolgen kann es unter Umständen ausreichen, die Eingabefolgen zunächst einmal „zufällig“ zu permutieren und darauf das normale Quicksort-Verfahren anzuwenden.

Die von uns im Abschnitt 2.2.1 angegebene Version des Verfahrens Quicksort läßt gleiche Schlüssel in der Eingabefolge zu. Nicht selten treten in Anwendungen Folgen mit vielen Wiederholungen auf. Man denke etwa an eine Datei mit offenstehenden Kundenrechnungen. Für einen Kunden kann es mehrere Rechnungen geben; dann haben alle dieselbe Kundennummer. Das von uns angegebene Sortierverfahren kann diesen Fall (Sortieren nach aufsteigenden Kundennummern) durchaus erledigen, zieht aber aus der Tatsache möglicher Wiederholungen keinen Nutzen.

Man nennt ein Sortierverfahren *glatt* (englisch: *smooth*), wenn es N verschiedene Schlüssel im Mittel in $O(N \log N)$ und N gleiche Schlüssel in $O(N)$ Schritten zu sortieren vermag mit einem „glatten“ Übergang zwischen diesen Werten. Wir ersparen uns eine präzise Definition dieses Begriffs und geben statt dessen ein Beispiel für ein solches Verfahren an.

Die wesentliche Idee besteht darin, bei der Aufteilung eines Bereiches $a[l] \dots a[r]$ nach dem Schlüssel des rechten Elementes $a[r]$ alle Elemente im aufzuteilenden Bereich, deren Schlüssel gleich dem Schlüssel des Pivotelementes sind, in der Mitte zu sammeln. An Stelle einer Zerlegung in zwei Folgen F_1 und F_2 mit dem Pivotelement dazwischen wird also eine Zerlegung in drei Folgen F_l , F_m und F_r angestrebt, so daß gilt

F_l enthält alle Elemente mit Schlüssel $< v$;

F_m enthält alle Elemente mit Schlüssel $= v$;

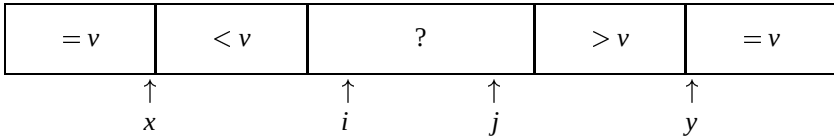
F_r enthält alle Elemente mit Schlüssel $> v$.

Hier bezeichnet $v = a[r].key$ das Pivotelement.

Da wir natürlich eine In-situ-Aufteilung des Bereichs $a[l] \dots a[r]$ haben wollen, und da wir außerdem nicht im vorhinein wissen, wo die endgültige Position des Pivotelementes ist, ergibt sich folgendes Problem: Wo soll man die Elemente zwischenspeichern, deren Schlüssel mit dem des Pivotelementes übereinstimmen?

Zur Lösung dieses Problems gibt es (wenigstens) vier verschiedene Möglichkeiten (vgl. [1]). Erstens kann man die Elemente am Anfang und Ende des aufzuteilenden Bereichs sammeln und sie dann später in die Mitte befördern. Zweitens kann man die Elemente nur am Anfang oder nur am Ende sammeln. Drittens kann man sie als wachsenden Block durch das Array wandern lassen, bis sie schließlich ihre richtige Position erreicht haben. Schließlich kann man sie in der ersten oder zweiten Hälfte an vielen Stellen verstreut ablegen und in einem zweiten Durchgang sammeln.

Wir diskutieren nur die erste Möglichkeit genauer. Außer den zwei Zeigern i und j , mit denen wir über das Array $a[l] \dots a[r]$ hinweg wandern, verwenden wir zwei weitere Zeiger x und y , die das jeweilige Ende des Anfangs- und Endstücks von $a[l] \dots a[r]$ markieren, in dem die Elemente mit Schlüssel gleich dem des Pivotelements gesammelt werden.



Anfangs ist $i = l - 1$, $j = r$, $x = l - 1$, $y = r$. Die Schleife zur Aufteilung des Bereichs $a[l] \dots a[r]$ nach dem Pivotelement $v = a[r].key$ bekommt jetzt folgende Gestalt:

begin-loop

```

repeat  $i := i + 1$  until  $a[i].key \geq v$ ;
repeat  $j := j - 1$  until  $a[j].key \leq v$ ;
if  $i \geq j$  then exit-loop;
if  $(a[i].key > v)$  and  $(a[j].key < v)$ 
  then {vertausche  $a[i]$  und  $a[j]$ }
    begin
       $t := a[i]$ ;
       $a[i] := a[j]$ ;
       $a[j] := t$ 
    end;
if  $(a[i].key > v)$  and  $(a[j].key = v)$ 
  then {hänge  $a[j]$  an das linke Endstück an}
    begin
       $t := a[j]$ ;
       $a[j] := a[i]$ ;
       $a[i] := a[x + 1]$ ;
       $a[x + 1] := t$ ;
       $x := x + 1$ 
    end;
if  $(a[i].key = v)$  and  $(a[j].key < v)$ 
  then {hänge  $a[i]$  an das rechte Endstück an}
    begin
       $t := a[i]$ ;
       $a[i] := a[j]$ ;
       $a[j] := a[y - 1]$ ;
       $a[y - 1] := t$ ;
       $y := y - 1$ 
    end;
if  $(a[i].key = v)$  and  $(a[j].key = v)$ 
  then
    begin
      {hänge  $a[i]$  an das linke Endstück an}
       $t := a[i]$ ;
       $a[i] := a[x + 1]$ ;
       $a[x + 1] := t$ ;
       $x := x + 1$ ;
      {hänge  $a[j]$  an das rechte Endstück an}
       $t := a[j]$ ;

```

```

    a[j] := a[y - 1];
    a[y - 1] := t;
    y := y - 1
end
end-loop

```

Am Ende der Aufteilung steht der Zeiger i auf dem ersten Element des Teilstücks mit Schlüssel größer oder gleich v . Dies ist die erste Position, an die das rechte Endstück mit Schlüsseln gleich dem Pivotelement getauscht werden muß; das linke Endstück muß links neben dieser Position zu liegen kommen. Da die Längen aller beteiligten Teilstücke bekannt sind, kann man dies leicht mit zwei Schleifen (ohne weitere Schlüsselvergleiche auszuführen) programmieren. Wir überlassen die Einzelheiten dem Leser.

Nach der Aufteilung in die drei Teilfolgen F_l , F_m und F_r müssen natürlich nur F_l und F_r rekursiv auf dieselbe Art sortiert werden. Für eine Datei, in der keine Schlüssel mehrfach auftreten, bedeutet das keine Ersparnis. Sind — das ist das andere Extrem — alle Schlüssel identisch, ist überhaupt kein rekursiver Aufruf nötig. L. Wegner  zeigt, daß unter geeigneten Annahmen über die Verteilung der Schlüssel gilt, daß das oben skizzierte, auf einem Drei-Wege-Split beruhende Quicksort im Mittel $O(N \log n + N)$ Zeit benötigt, wobei n die Anzahl der verschiedenen Schlüssel unter den N Schlüsseln der Eingabefolge ist.

2.3 Heapsort

Alle in den Abschnitten 2.1 und 2.2 behandelten Sortierverfahren benötigen im schlimmsten Fall eine Laufzeit von $\Theta(N^2)$ für das Sortieren von N Schlüsseln. Im Abschnitt 2.8 wird gezeigt, daß zum Sortieren von N Schlüsseln mindestens $\Omega(N \log N)$ Schritte benötigt werden, wenn Information über die Ordnung der Schlüssel nur durch Schlüsselvergleiche gewonnen werden kann. Solche Sortierverfahren heißen *allgemeine* Sortierverfahren, weil außer der Existenz einer Ordnung keine speziellen Bedingungen an die Schlüssel geknüpft sind. Man kann sich nun fragen: Gibt es überhaupt Sortierverfahren, die mit $O(N \log N)$ Operationen auskommen, selbst im schlimmsten Fall? Wir werden sehen, daß solche Verfahren tatsächlich existieren; Heapsort ist eines von ihnen.

Heapsort (Sortieren mit einer Halde) folgt dem Prinzip des Sortierens durch Auswahl (vgl. Abschnitt 2.1.1), wobei aber die Auswahl geschickt organisiert ist. Dazu wird eine Datenstruktur verwendet, der *Heap* (die *Halde*), in der die Bestimmung des Maximums einer Menge von N Schlüsseln in einem Schritt möglich ist. Eine Folge $F = k_1, k_2, \dots, k_N$ von Schlüsseln nennen wir einen Heap, wenn $k_i \leq k_{\lfloor \frac{i}{2} \rfloor}$ für $2 \leq i \leq N$ gilt. Anders ausgedrückt: $k_i \geq k_{2i}$ und $k_i \geq k_{2i+1}$, sofern $2i \leq N$ bzw. $2i + 1 \leq N$.

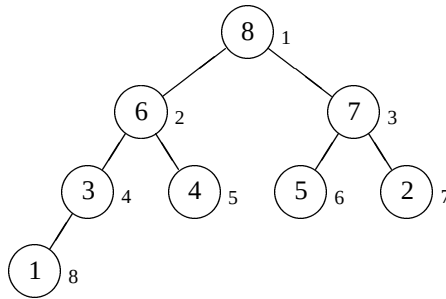


Abbildung 2.4

Beispiel: Die Folge $F = 8, 6, 7, 3, 4, 5, 2, 1$ genügt der Heap-Bedingung, weil gilt: $8 \geq 6$, $8 \geq 7$, $6 \geq 3$, $6 \geq 4$, $7 \geq 5$, $7 \geq 2$, $3 \geq 1$. Diese Beziehung kann man graphisch wie in Abbildung 2.4 veranschaulichen.

Beim Eintrag k_i ist der Index i mit angegeben, um den Bezug zwischen F und dem Schaubild zu erleichtern. In die oberste Zeile kommt der Schlüssel k_1 ; in die nächste Zeile kommen die Schlüssel k_2 und k_3 . Die Beziehungen $k_1 \geq k_2$ und $k_1 \geq k_3$ werden durch zwei Verbindungslinien (Kanten) dargestellt. In Zeile j kommen Schlüssel k_{2j-1} bis k_{2j} , von links nach rechts. Außerdem werden Kanten zu den entsprechenden Schlüssel der vorangehenden Zeile gezeichnet. Das so definierte Schaubild repräsentiert den Heap als Binärbaum (vgl. hierzu auch Kapitel 5). Jedem Schlüssel entspricht ein Knoten des Baumes, und zwischen den Knoten für Schlüssel k_i und k_{2i} bzw. k_i und k_{2i+1} gibt es eine Kante. Schlüssel k_1 steht an der Wurzel des Baumes. Schlüssel k_{2i} ist der linke, k_{2i+1} der rechte Sohn von Schlüssel k_i ; k_i ist der Vater von k_{2i} und k_{2i+1} . Interpretiert man den Heap als Binärbaum, so kann man die Heap-Bedingung auch wie folgt formulieren. Ein Binärbaum ist ein Heap, wenn der Schlüssel jedes Knotens mindestens so groß ist wie die Schlüssel seiner beiden Söhne (falls es diese gibt).

Wir gehen im folgenden immer davon aus, daß die Schlüssel in einem Array gespeichert sind, auch wenn wir manchmal in Erklärungen auf die Baumstruktur Bezug nehmen. Stellen wir uns einmal vor, eine Folge von Schlüssel sei als Heap gegeben (also etwa Folge F im obigen Beispiel), und wir sollen die Schlüssel in absteigender Reihenfolge ausgeben. Das ist für den ersten Schlüssel ganz leicht, denn k_1 ist ja das Maximum aller Schlüssel. Wie bestimmen wir aber jetzt den nächstkleineren Schlüssel? Eine offensichtliche Methode ist doch die, den gerade ausgegebenen Schlüssel aus der Folge zu entfernen und die restliche Folge wieder zu einem Heap zu machen. Dann steht nämlich der nächstkleinere Schlüssel wieder an der Wurzel, und wir können nach demselben Verfahren fortfahren.

Das ergibt für das absteigende Sortieren der Schlüssel eines Heaps folgende **Methode**:

{Anfangs besteht der Heap aus Schlüssel $k_1, \dots, k_N$ }
Solange der Heap nicht leer ist, wiederhole:
gib k_1 aus; {das ist der nächstgrößere Schlüssel}

*entferne k_1 aus dem Heap;
stelle die Heap-Bedingung für die restlichen Schlüssel her,
so daß die neue Wurzel an Position 1 steht.*

Der schwierigste Teil ist hier das Wiederherstellen der Heap-Bedingung. Wir nutzen die Tatsache aus, daß nach dem Entfernen der Wurzel ja noch zwei Teil-Heaps vorliegen. Im obigen Beispiel der Folge $F = 8, 6, 7, 3, 4, 5, 2, 1$ gibt es nach Entfernen von $k_1 = 8$ zwei Teil-Heaps, die Abbildung 2.5 zeigt.

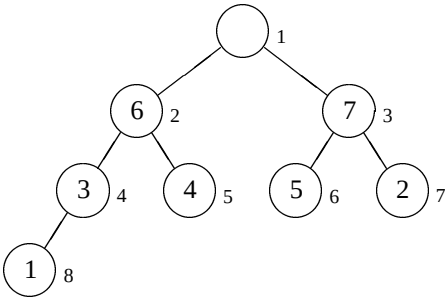


Abbildung 2.5

Wir machen daraus einen Heap, indem wir zunächst den Schlüssel mit höchstem Index an die Wurzel schreiben, wobei aber im allgemeinen die Heap-Bedingung verletzt wird. Dies zeigt Abbildung 2.6.

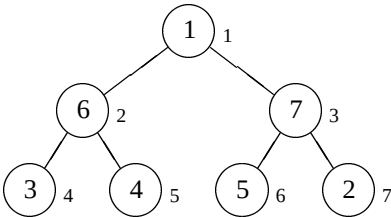


Abbildung 2.6

Dann lassen wir den (neuen) Schlüssel k_1 im Heap nach unten *versickern* (*sift down*), indem wir ihn solange immer wieder mit dem größeren seiner beiden Söhne vertauschen, bis beide Söhne kleiner sind oder der Schlüssel unten angekommen ist, vgl. Abbildung 2.7.

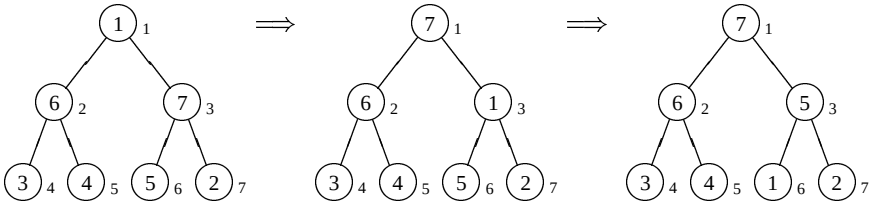


Abbildung 2.7

Damit ist die Heap-Bedingung für die Schlüsselfolge erfüllt. Für das Entfernen des Maximums und das Herstellen der Heap-Bedingung für die Folge der Schlüssel $k_1, \dots, k_m$ eignet sich also die folgende Methode:

*{entferne Maximum aus Heap $k_1, \dots, k_m$, und mache
restliche Schlüsselfolge wieder zu einem Heap}
übertrage k_m nach k_1 ;
versickere k_1 im Bereich k_1 bis k_{m-1} .*

Das Versickern eines Schlüssels geschieht wie folgt:

*{versickere k_i im Bereich k_i bis k_m }
Solange k_i einen linken Sohn k_j hat, wiederhole:
falls k_i einen rechten Sohn hat,
so sei k_j derjenige Sohn von k_i mit größerem Schlüssel;
falls $k_i < k_j$,
so vertausche k_i mit k_j und setze $i := j$,
sonst halte an {die Heap-Bedingung gilt}.*

Tabelle 2.2 zeigt am Beispiel der Folge $F = 8, 6, 7, 3, 4, 5, 2, 1$, wie die Schlüssel von F absteigend sortiert werden.

Statt die Schlüssel in absteigender Reihenfolge auszugeben, können wir sie mit dem angegebenen Verfahren auch in aufsteigender Reihenfolge sortieren, wenn wir das jeweils aus dem Heap entfernte Maximum nicht ausgeben, sondern an die Stelle desjenigen Schlüssels schreiben, der nach dem Entfernen des Maximums nach k_1 übertragen wird. Dann läßt sich das Sortieren eines Heaps wie folgt beschreiben:

{sortiere Heap a : sequence im Bereich von 1 bis r : integer}
var
 i : integer;
 t : item;
begin
 for $i := r$ **downto** 2 **do**
 begin {tausche $a[1]$ mit $a[i]$, versickere $a[1]$ }
 {M1} $t := a[i]$;
 {M1} $a[i] := a[1]$;
 {M1} $a[1] := t$;

Kommentar	Ausgabe	k_1	k_2	k_3	k_4	k_5	k_6	k_7	k_8
Anfangsheap		8	6	7	3	4	5	2	1
gib k_1 aus	8								
übertrage k_8 nach k_1		1	6	7	3	4	5	2	
versickere k_1		7		1					
				5			1		
gib k_1 aus	7								
übertrage k_7 nach k_1		2	6	5	3	4	1		
versickere k_1		6	2						
			4			2			
gib k_1 aus, übertrage k_6 ,	6	5	4	1	3	2			
versickere etc.	5	4	3	1	2				
	4	3	2	1					
	3	2	1						
	2	1							
	1	leer							

Tabelle 2.2

```
versickere(a, 1, i - 1)
end
end
```

Dabei ist *versickere* wie folgt erklärt:

```
procedure versickere (var a : sequence; i, m: integer);
{versickere a[i] bis höchstens nach a[m]}
var
  j : integer;
  t : item;
begin
  while 2 * i ≤ m do {a[i] hat linken Sohn}
    begin
      j := 2 * i; {a[j] ist linker Sohn von a[i]}
      if j < m
        then {a[i] hat rechten Sohn}
      {C1}   if a[j].key < a[j + 1].key then j := j + 1;
            {jetzt ist a[j].key größer}
      {C2}   if a[i].key < a[j].key
        then {tausche a[i] mit a[j]}
            begin
              {M2}   t := a[i];
              {M2}   a[i] := a[j];
              {M2}   a[j] := t;
```

```

        i := j {versickere weiter}
    end
    else i := m {halte an, Heap-Bedingung erfüllt}
end
end
end

```

Analyse: Außerhalb der Prozedur *versickere* werden beim Sortieren eines Heaps, der aus N Schlüsseln besteht, gerade $\Theta(N)$ Bewegungen von Datensätzen ausgeführt (vgl. Programmzeilen $\{M1\}$). Außerdem werden beim Versickern Datensätze bewegt (vgl. Programmzeilen $\{M2\}$). Beim Versickern wird ein Schlüssel wiederholt mit einem seiner Söhne vertauscht. Im Schaubild, das den Heap als Binärbaum zeigt, wandert der Schlüssel bei jeder Vertauschung eine Zeile — man sagt: eine *Stufe* oder ein *Niveau* (*level*) — tiefer. Die Anzahl der Schlüssel verdoppelt sich von Stufe zu Stufe; lediglich auf der letzten Stufe können einige Schlüssel fehlen. Ein Heap mit j Stufen speichert also zwischen 2^{j-1} und $2^j - 1$ Schlüssel. Ein Heap für N Schlüssel, mit $2^{j-1} \leq N \leq 2^j - 1$, hat also $j = \lceil \log(N+1) \rceil$ Stufen. Daher kann die **while**-Schleife der Prozedur *versickere* bei einem Prozeduraufruf höchstens $\lceil \log(N+1) \rceil - 1$ Mal durchlaufen werden. Da die Prozedur *versickere* genau $N - 1$ Mal aufgerufen wird, ergibt sich eine obere Schranke von $O(N \log N)$ Ausführungen jeder der Zeilen $\{C1\}$, $\{C2\}$ und $\{M2\}$. Damit gilt:

$$C_{\max}(N) = O(N \log N), \quad M_{\max}(N) = O(N \log N).$$

Das Verfahren, einen Heap zu sortieren, können wir erst dann zum Sortieren einer beliebigen Schlüsselfolge verwenden, wenn wir diese in einen Heap umgewandelt haben. Der Erfinder von Heapsort, J.W.J. Williams (vgl. [E]), hat dafür eine Methode angegeben, die in $O(N \log N)$ Schritten einen Heap konstruiert. Ein schnelleres Verfahren, das wir im folgenden erläutern, stammt von R.W. Floyd (vgl. [F]).

Die Grundidee besteht darin, in einer Schlüsselfolge von k_N nach vorne Teil-Heaps zu erzeugen. Nehmen wir an, die Heap-Bedingung sei für alle Schlüssel der Folge ab einem gewissen k_l erfüllt, d.h., es gelte $k_{\lfloor \frac{i}{2} \rfloor} \geq k_i$ für $\lfloor \frac{i}{2} \rfloor \geq l$. Das ist anfangs, in der unsortierten Folge, gesichert für $l = \lfloor \frac{N}{2} \rfloor + 1$. Dann können wir die Heap-Bedingung für alle Schlüssel ab k_{l-1} herstellen, indem wir k_{l-1} in der Folge $k_{l-1}, \dots, k_N$ versickern. Die Voraussetzung für das Versickern eines Schlüssels, nämlich, daß die beiden Söhne des Schlüssels Wurzeln von Teil-Heaps sind, ist gesichert, weil die Heap-Bedingung für alle Schlüssel mit höherem Index erfüllt ist. Zunächst lassen wir also $k_{\lfloor \frac{N}{2} \rfloor}$ versickern, dann $k_{\lfloor \frac{N}{2} \rfloor - 1}$ usw., bis schließlich k_1 versickert. Die erhaltene Folge ist ein Heap, weil die Heap-Bedingung ab Schlüssel k_1 , also für alle Schlüssel, erfüllt ist.

Methode: Eine gegebene Folge $F = k_1, k_2, \dots, k_N$ von N Schlüsseln wird in einen Heap umgewandelt, indem die Schlüssel $k_{\lfloor \frac{N}{2} \rfloor}, k_{\lfloor \frac{N}{2} \rfloor - 1}, \dots, k_1$ (in dieser Reihenfolge) in F versickern.

Beispiel: Betrachten wir die Folge $F = 2, 1, 5, 3, 4, 8, 7, 6$ und die Veränderungen, die sich beim Versickern der Schlüssel ergeben:

Versickere Schlüssel	Folge
anfangs	2, 1, 5, 3, 4, 8, 7, 6
$k_4 = 3$	2, 1, 5, 6, 4, 8, 7, 3
$k_3 = 5$	2, 1, 8, 6, 4, 5, 7, 3
$k_2 = 1$	2, 6, 8, 3, 4, 5, 7, 1
$k_1 = 2$	8, 6, 7, 3, 4, 5, 2, 1

Die erhaltene Folge ist ein Heap.

Das Sortierverfahren Heapsort für eine Folge F von Schlüsseln besteht nun darin, F zunächst in einen Heap umzuwandeln und den Heap dann zu sortieren. Das ergibt folgende Sortierprozedur.

```

procedure heapsort (var  $a$  : sequence);
{sortiert die Elemente  $a[1]$  bis  $a[N]$ }
var
   $i$  : integer;
   $t$  : item;
begin
  {wandle  $a[1]$  bis  $a[N]$  in einen Heap um}
  for  $i := N \text{ div } 2$  downto 1 do versickere( $a, i, N$ );
  {sortiere den Heap}
  for  $i := N$  downto 2 do
    begin {tausche  $a[1]$  mit  $a[i]$ , versickere  $a[1]$ }
       $t := a[i]$ ;
       $a[i] := a[1]$ ;
       $a[1] := t$ ;
      versickere( $a, 1, i - 1$ )
    end
  end

```

Analyse: Sei $2^{j-1} < N \leq 2^j - 1$, also j die Anzahl der Stufen des Heaps für N Schlüssel. Numerieren wir die Stufen von oben nach unten von 1 bis j . Dann gibt es auf Stufe k höchstens 2^{k-1} Schlüssel. Die Anzahl der Bewege- und Vergleichsoperationen zum Versickern eines Elementes der Stufe k ist proportional zu $j - k$. Insgesamt ergibt sich für die Anzahl der Operationen zum Umwandeln einer unsortierten Folge in einen Heap:

$$\sum_{k=1}^{j-1} 2^{k-1} (j - k) = \sum_{k=1}^{j-1} k \cdot 2^{j-k-1} = 2^{j-1} \sum_{k=1}^{j-1} \frac{k}{2^k} \leq N \cdot 2 = O(N)$$

Das Aufbauen eines Heaps aus einer unsortierten Folge ist also in linearer Zeit möglich. Damit ergibt sich die Zeitschranke für Heapsort aus dem Sortieren des Heaps zu

$$C_{\max}(N) = O(N \log N), \quad M_{\max}(N) = O(N \log N).$$

Experimente zeigen, daß dies auch die mittlere Anzahl von Bewegungen und Vergleichsoperationen für Heapsort ist. Heapsort ist also das erste von uns behandelte Sortierverfahren, das asymptotisch optimale Laufzeit im schlechtesten Fall hat. Diese Laufzeit variiert für verschiedene Eingabefolgen nur geringfügig; insbesondere nützt oder schadet Vorsortierung bei Heapsort praktisch nichts. Man kann Heapsort jedoch so modifizieren, daß es Vorsortierung ausnützt. Eine solche Heapsort-Variante, *Smoothsort* , benötigt $O(N)$ Zeit für eine vorsortierte Folge und $O(N \log N)$ Zeit im schlimmsten Fall. Heapsort ist kein *stabiles* Verfahren, d h., die relative Position gleicher Schlüssel kann sich beim Sortieren ändern. Im Gegensatz zu den gängigen Varianten von Quicksort, das im Durchschnitt schneller ist als Heapsort, benötigt Heapsort nur konstant viel zusätzlichen Speicherplatz; es ist also ein echtes In-situ-Sortierverfahren.

2.4 Mergesort

Das Verfahren *Mergesort* (*Sortieren durch Verschmelzen*) ist eines der ältesten und bestuntersuchten Verfahren zum Sortieren mit Hilfe von Computern. John von Neumann hat es bereits 1945 vorgeschlagen. Es folgt — ähnlich wie Quicksort — der Strategie, eine Folge durch rekursives Aufteilen zu sortieren. Im Unterschied zu Quicksort wird aber hier die Folge in gleich große Teilfolgen aufgeteilt. Die (rekursiv) sortierten Teilfolgen werden dann verschmolzen. Dazu verwendet man linear viel zusätzlichen Speicherplatz. Als Ausgleich dafür kann die Laufzeit von Mergesort für eine Folge von N Sätzen $O(N \log N)$ nicht übersteigen.

In Abschnitt 2.4.1 beschreiben und analysieren wir eine einfache Realisierung von Mergesort, das rekursive Aufteilen in zwei Teilfolgen (*2-Wege-Mergesort*). Man kann Mergesort auch leicht ohne Rekursion als das Verschmelzen immer größerer Teilfolgen formulieren; dieses *reine 2-Wege-Mergesort* (*straight 2-way merge sort*) beschreiben wir im Abschnitt 2.4.2. Nützt man die in der zu sortierenden Folge bereits vorhandenen sortierten Teilfolgen aus, so erhält man das *natürliche 2-Wege-Mergesort* (*natural 2-way merge sort*); dieses Verfahren beschreiben wir im Abschnitt 2.4.3. Schließlich eignet sich Mergesort ganz besonders gut für das Sortieren von Daten auf Sekundärspeichern, das *externe Sortieren* (vgl. Abschnitt 2.7).

2.4.1 2-Wege-Mergesort

Methode: Eine Folge $F = k_1, \dots, k_N$ von N Schlüsseln wird sortiert, indem sie zunächst in zwei möglichst gleich große Teilfolgen $F_1 = k_1, \dots, k_{\lfloor \frac{N}{2} \rfloor}$ und $F_2 = k_{\lfloor \frac{N}{2} \rfloor + 1}, \dots, k_N$ aufgeteilt wird. Dann wird jede dieser Teilfolgen mittels Mergesort sortiert. Die sortierte Folge ergibt sich durch Verschmelzen der beiden sortierten Teilfolgen. Mergesort folgt also, ähnlich wie Quicksort, dem allgemeinen Prinzip des Divide-and-conquer. Dabei ist wichtig, daß das Verschmelzen sortierter Folgen einfacher ist als das Sortieren. Zwei sortierte Folgen werden verschmolzen, indem man je einen Positionszeiger

(Index) durch die beiden Folgen so wandern läßt, daß die Elemente beider Folgen insgesamt in sortierter Reihenfolge angetroffen werden. Beginnt man mit beiden Zeigern am jeweils linken Ende der beiden Folgen, so bewegt man in einem Schritt denjenigen der beiden Zeiger um eine Position nach rechts (in der betreffenden Folge), der auf den kleineren Schlüssel zeigt. Man übernimmt einen Schlüssel immer dann in die Resultatfolge, wenn ein Zeiger bisher auf diesen Schlüssel gezeigt hat und im aktuellen Schritt weiterwandert. Sobald eine der Folgen erschöpft ist, übernimmt man den Rest der anderen Folge in die Resultatfolge.

Beispiel: Die beiden sortierten Folgen $F_1 = 1, 2, 3, 5, 9$ und $F_2 = 4, 6, 7, 8, 10$ sollen verschmolzen werden. Zunächst zeigen zwei Positionszeiger i und j auf die Anfangselemente beider Folgen, also 1 und 4, wie in Abbildung 2.8 dargestellt.

	F_1	F_2	Resultatfolge
anfangs:	1, 2, 3, 5, 9 $\uparrow i$	4, 6, 7, 8, 10 $\uparrow j$	$\emptyset$

Abbildung 2.8

Da $k_i < k_j$ gilt, wandert Zeiger i in Folge F_1 , und k_i wird in die Resultatfolge übernommen (Abbildung 2.9).

$1 < 4:$	1, 2, 3, 5, 9 $\uparrow i$	4, 6, 7, 8, 10 $\uparrow j$	1
----------	-------------------------------	--------------------------------	---

Abbildung 2.9

Im nächsten Schritt ist wieder $k_i = 2 < 4 = k_j$, also wandert wieder Zeiger i in Folge F_1 . Wir zeigen in Tabelle 2.3 den Prozeß des Verschmelzens bis zum Ende.

Die Struktur des Verfahrens Mergesort kann man, ohne Berücksichtigung von Implementationsdetails, wie folgt beschreiben:

Algorithmus Mergesort (F : Folge);
{sortiert Schlüsselfolge F nach aufsteigenden Werten}
Falls F die leere Folge ist oder nur aus einem einzigen Schlüssel besteht,
bleibt F unverändert; sonst:

Divide: Teile F in zwei etwa gleich große Teilfolgen, F_1 und F_2 ;
Conquer: Mergesort(F_1); Mergesort(F_2);
 {jetzt sind beide Teilfolgen F_1 und F_2 sortiert}
Merge: Bilde die Resultatfolge durch Verschmelzen von F_1 und F_2 .

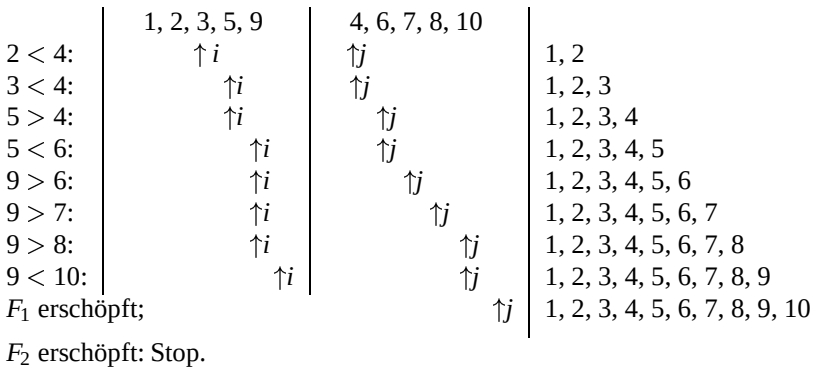


Tabelle 2.3

Betrachten wir als Beispiel die Anwendung des Verfahrens Mergesort auf die Folge $F = 2, 1, 3, 9, 5, 6, 7, 4, 8, 10$. Zunächst wird F aufgeteilt in die beiden Teilfolgen $F_1 = 2, 1, 3, 9, 5$ und $F_2 = 6, 7, 4, 8, 10$. Dann werden beide Teilfolgen mittels Mergesort sortiert; das ergibt $F_1 = 1, 2, 3, 5, 9$ und $F_2 = 4, 6, 7, 8, 10$. Diese beiden Folgen werden, wie im vorangegangenen Beispiel gezeigt, zur Folge $F = 1, 2, 3, 4, 5, 6, 7, 8, 9, 10$ verschmolzen.

Für die programmtechnische Realisierung von Mergesort nehmen wir an, daß die Folge der zu sortierenden Datensätze in einem Feld a an den Positionen 1 bis N gespeichert ist. Weil *mergesort* rekursiv für Teilfolgen aufgerufen wird, verwenden wir zwei Feldindizes für das erste und das letzte Element der zu sortierenden Teilfolge.

```

procedure mergesort (var  $a$  : sequence;  $l, r$  : integer);
{sortiert  $a[l]$  bis  $a[r]$  nach aufsteigenden Schlüssel $n$ }
var
     $m$  : integer;
begin
    if  $l < r$  {sonst : leere oder einelementige Folge}
    then
        begin
             $m := (l + r) \text{ div } 2$ ; {das ist die Mitte der Folge}
            mergesort( $a, l, m$ );
            mergesort( $a, m + 1, r$ );
            { $a[l] \dots a[m]$  und  $a[m + 1] \dots a[r]$  sind sortiert}
            merge( $a, l, m, r$ ) {Verschmelzen}
        end
    end

```

Das Verschmelzen zweier Teilfolgen, die im Feld a an benachbarten Feldpositionen stehen, wird durch die Prozedur *merge* erreicht. Wir verwenden dazu ein zusätzliches Feld b , das zunächst die Resultatfolge aufnimmt. Anschließend wird die Resultatfolge von b nach a zurückkopiert.

```

procedure merge (var  $a$  : sequence;  $l, m, r$  : integer);
  {verschmilzt die beiden sortierten Teilfolgen  $a[l] \dots a[m]$ 
   und  $a[m+1] \dots a[r]$  und speichert sie in  $a[l] \dots a[r]$ }
var
   $b$  : sequence; {Hilfsfeld zum Verschmelzen}
   $h, i, j, k$  : integer;
begin
   $i := l$ ; {inspiziere noch  $a[i]$  bis  $a[m]$  der ersten Teilfolge}
   $j := m + 1$ ; {inspiziere noch  $a[j]$  bis  $a[r]$  der zweiten Teilfolge}
   $k := l$ ; {das nächste Element der Resultatfolge ist  $b[k]$ }
  while ( $i \leq m$ ) and ( $j \leq r$ ) do
    begin {beide Teilfolgen sind noch nicht erschöpft}
      {C}      if  $a[i].key \leq a[j].key$ 
                then {übernimm  $a[i]$  nach  $b[k]$ }
                  begin
                    {M1}       $b[k] := a[i]$ ;
                               $i := i + 1$ 
                  end
                else {übernimm  $a[j]$  nach  $b[k]$ }
                  begin
                    {M1}       $b[k] := a[j]$ ;
                               $j := j + 1$ 
                  end;
                 $k := k + 1$ 
            end;
    if  $i > m$ 
      then {erste Teilfolge ist erschöpft; übernimm zweite}
        {M2}      for  $h := j$  to  $r$  do  $b[k + h - j] := a[h]$ 
      else {zweite Teilfolge ist erschöpft; übernimm erste}
        {M2}      for  $h := i$  to  $m$  do  $b[k + h - i] := a[h]$ ;
      {speichere sortierte Folge von  $b$  zurück nach  $a$ }
      {M3}      for  $h := l$  to  $r$  do  $a[h] := b[h]$ 
    end

```

Man erkennt, daß die für beide Teilfolgen erforderlichen Aktionen völlig gleichartig sind; wie man diese Aktionen parametrisiert, beschreiben wir beim Verschmelzen mehrerer Teilfolgen in Abschnitt 2.7.

Analyse: Schlüsselvergleiche werden nur in der Prozedur *merge* in der mit {C} markierten Zeile ausgeführt. Nach jedem Schlüsselvergleich wird einer der beiden Positionszeiger weiterbewegt. Sobald eine Teilfolge erschöpft ist, werden keine weiteren Schlüsselvergleiche mehr ausgeführt. Für zwei Teilfolgen der Länge n_1 bzw. n_2 ergeben sich also mindestens $\min(n_1, n_2)$ und höchstens $n_1 + n_2 - 1$ Schlüsselvergleiche. Zum Verschmelzen zweier etwa gleich langer Teilfolgen der Gesamtlänge N benötigen wir also $\Theta(N)$ Schlüsselvergleiche; das ist der ungünstigste Fall. Damit ergibt sich für

die Anzahl $C(N)$ der zum Sortieren von N Schlüsseln benötigten Vergleichsoperationen

$$C(N) = \underbrace{C\left(\left\lceil \frac{N}{2} \right\rceil\right) + C\left(\left\lfloor \frac{N}{2} \right\rfloor\right)}_{\text{Schlüsselvergleiche zum Sortieren der beiden Teilfolgen}} + \underbrace{\Theta(N)}_{\text{Verschmelzen}} = \Theta(N \log N).$$

Das gilt für den besten ebenso wie für den schlechtesten (und damit auch für den mittleren) Fall gleichermaßen:

$$C_{\min}(N) = C_{\max}(N) = C_{\text{mit}}(N) = \Theta(N \log N).$$

Mergesort ist also ein Sortierverfahren, das größenordnungsmäßig nicht mehr Schlüsselvergleiche benötigt, als im schlimmsten Fall auch tatsächlich erforderlich sind (vgl. Abschnitt 2.8); es ist worst-case-optimal. Damit hat es sich ausgezahlt, die rekursive Aufteilung möglichst ausgeglichen vorzunehmen. Da bei jedem Aufteilungsschritt die Folgenlänge etwa halbiert wird, ergeben sich nach $\lceil \log N \rceil$ Aufteilungsschritten stets Teilfolgen der Länge 1, die nicht weiter rekursiv behandelt werden müssen. Die Rekursionstiefe ist also — etwa im Gegensatz zu Quicksort — logarithmisch beschränkt.

An den mit $\{M \dots\}$ markierten Zeilen der Prozedur *merge* läßt sich ablesen, daß viel mehr Bewegungen von Datensätzen ausgeführt werden als Schlüsselvergleiche. Für jeden Schlüsselvergleich wird auch eine Bewegung eines Datensatzes (Zeilen $\{M1\}$) ausgeführt. Zusätzlich werden die restlichen Elemente einer Teilfolge nach b übernommen (Zeilen $\{M2\}$), wenn die andere Teilfolge erschöpft ist. Schließlich wird noch die gesamte Resultatfolge von b nach a zurückkopiert (Zeile $\{M3\}$). Beim Verschmelzen zweier Teilfolgen der Gesamtlänge N werden also gerade $2N$ Bewegungen ausgeführt. Damit ergibt sich auch hier

$$M_{\min}(N) = M_{\max}(N) = M_{\text{mit}}(N) = \Theta(N \log N).$$

Viele der Bewegungen kann man vermeiden, wenn man den Bereich, in dem sortierte Teilfolgen gespeichert sind (also a oder b), parametrisiert (vgl. Abschnitt 2.7). Am asymptotischen Aufwand ändert sich dabei nichts. Weil bei Mergesort Teilfolgen immer nur sequentiell inspiziert werden, kann man dieses Verfahren auch für Datensätze in verketteten linearen Listen verwenden. Dann entfallen Bewegungen von Datensätzen komplett; stattdessen werden lediglich Listenzeiger geändert. Allerdings benötigt auch diese Mergesort-Variante linear viel zusätzlichen Speicherplatz, nämlich für die Listenzeiger.

2.4.2 Reines 2-Wege-Mergesort

Im rekursiven 2-Wege-Mergesort, wie in Abschnitt 2.4.1 beschrieben, dient die Prozedur *mergesort* lediglich zur Organisation der Verschmelzungen von Teilfolgen. Das eigentliche Sortieren ist dann erst das Verschmelzen von Teilfolgen der Länge 1, später der Länge 2 usw., bis zum Verschmelzen zweier Teilfolgen der Länge $N/2$. Beim *reinen 2-Wege-Mergesort* (*straight 2-way merge sort*) werden die Verschmelzungen von Teilfolgen genauso organisiert, und zwar ohne Rekursion und ohne Aufteilung.

Methode: Eine Folge $F = k_1, \dots, k_N$ von Schlüsseln wird sortiert, indem sortierte Teilfolgen zu immer längeren Teilfolgen verschmolzen werden. Anfangs ist jeder Schlüssel k_i , $1 \leq i \leq N$, eine sortierte Teilfolge. In einem Durchgang (von links nach rechts durch die Folge) werden jeweils zwei benachbarte Teilfolgen zu einer Folge verschmolzen. Beim ersten Durchgang wird also k_1 mit k_2 verschmolzen, k_3 mit k_4 usw. Dabei kann es vorkommen, daß am Ende eines Durchganges eine Teilfolge übrig bleibt, die nicht weiter verschmolzen wird. Bei jedem Durchgang verdoppelt sich also die Länge der sortierten Teilfolgen, außer eventuell am rechten Rand. Die gesamte Folge ist sortiert, sobald in einem Durchgang nur noch zwei Teilfolgen verschmolzen worden sind.

Beispiel: Betrachten wir die Folge $F = 2, 1, 3, 9, 5, 6, 7, 4, 8, 10$ aus Abschnitt 2.4.1. Teilfolgen sind voneinander durch einen senkrechten Strich getrennt. Anfangs haben alle Teilfolgen die Länge 1.

2 | 1 | 3 | 9 | 5 | 6 | 7 | 4 | 8 | 10

Nach einem Durchgang sind je zwei benachbarte Teilfolgen verschmolzen.

1, 2 | 3, 9 | 5, 6 | 4, 7 | 8, 10

Wir geben die Teilfolgen nach jedem weiteren Durchgang an.

1, 2, 3, 9 | 4, 5, 6, 7 | 8, 10
 1, 2, 3, 4, 5, 6, 7, 9 | 8, 10
 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Die folgende Prozedur *straightmergesort* realisiert das reine 2-Wege-Mergesort.

```
procedure straightmergesort (var a : sequence; l, r : integer);
  {sortiert a[l] ... a[r] nach aufsteigenden Schlüsselwerten;
   l und r werden nicht für rekursive Aufrufe benötigt und sind
   nur wegen der Analogie zu mergesort hier angegeben}
var
  size, ll, mm, rr : integer;
begin
  size := 1; {Länge der bereits sortierten Teilfolgen}
  while size < r - l + 1 do
    begin {verschmilz Teilfolgen der Länge size}
      rr := l - 1; {Elemente bis inklusive a[rr] sind bearbeitet}
      while rr + size < r do
        begin {es gibt noch mindestens zwei Teilfolgen}
          ll := rr + 1; {linker Rand der ersten Teilfolge}
          mm := ll + size - 1; {rechter Rand}
          if mm + size ≤ r
            then {r noch nicht überschritten}
              rr := mm + size
            else {zweite Teilfolge ist kürzer}
              rr := r;
          merge(a, ll, mm, rr)
        end
      end
    end
```

```

    end;
    {ein Durchlauf ist beendet; sortierte Teilfolgen haben jetzt
     die Länge 2 * size}
    size := 2 * size
  end
  {a ist sortiert}
end

```

Analyse: Schlüsselvergleiche und Bewegungen finden auch hier nur innerhalb der Prozedur *merge* statt. Bei jedem Durchgang durch die Folge werden insgesamt N Datensätze mittels *merge* verschmolzen. Weil sich bei jedem Durchgang die Länge der sortierten Teilfolgen verdoppelt, erhält man nach $\lceil \log N \rceil$ Durchgängen eine sortierte Folge. Damit gilt wie erwartet

$$C_{\min}(N) = C_{\max}(N) = C_{\text{mit}}(N) = \Theta(N \log N)$$

und

$$M_{\min}(N) = M_{\max}(N) = M_{\text{mit}}(N) = \Theta(N \log N).$$

2.4.3 Natürliches 2-Wege-Mergesort

Ausgehend vom reinen 2-Wege-Mergesort liegt es nahe, den Verschmelze-Prozeß nicht mit einelementigen Teilfolgen zu beginnen, sondern bereits anfangs möglichst lange sortierte Teilfolgen zu verwenden. Auf diese Weise versucht man, eine natürliche, in der gegebenen Folge bereits enthaltene Vorsortierung auszunutzen. Betrachten wir noch einmal die Folge $F = 2, 1, 3, 9, 5, 6, 7, 4, 8, 10$. In F findet man vier längstmögliche, bereits sortierte Teilfolgen benachbarter Folgeelemente.

$$2 \mid 1, 3, 9 \mid 5, 6, 7 \mid 4, 8, 10$$

Verschmilzt man nun, wie beim reinen 2-Wege-Mergesort, in jedem Durchgang benachbarte Teilfolgen, so erhält man nach zwei Durchgängen eine sortierte Folge.

$$\begin{array}{ccccccccc} 1, & 2, & 3, & 9 & | & 4, & 5, & 6, & 7, & 8, & 10 \\ 1, & 2, & 3, & 4, & 5, & 6, & 7, & 8, & 9, & 10 \end{array}$$

Methode: Eine Folge $F = k_1, \dots, k_N$ von Schlüsseln wird sortiert, indem sortierte Teilfolgen zu immer längeren sortierten Teilfolgen verschmolzen werden. Anfangs wird F in längstmögliche sortierte Teilfolgen benachbarter Schlüssel (die sogenannten *Runs*) geteilt. Dann werden wiederholt benachbarte Teilfolgen verschmolzen (wie beim reinen 2-Wege-Mergesort), bis schließlich eine sortierte Folge entstanden ist.

Bei der programmtechnischen Realisierung ist der einzige Unterschied zum reinen 2-Wege-Mergesort das Finden der sortierten Teilfolgen. Eine sortierte Teilfolge ist zu Ende, wenn ein kleinerer Schlüssel auf einen größeren folgt. Damit ergibt sich die Prozedur *naturalmergesort*:

```

procedure naturalmergesort (var a : sequence; l, r : integer);
  {sortiert  $a[l] \dots a[r]$  nach aufsteigenden Schlüsselwerten}
var
  ll, mm, rr : integer;
begin
  repeat
    rr := l - 1; {Elemente bis inklusive  $a[rr]$  sind bearbeitet}
    while rr < r do
      begin {finde und verschmilz die nächsten Runs}
        ll := rr + 1; {linker Rand}
        mm := ll; { $a[ll] \dots a[mm]$  ist sortiert}
        {C1} while (mm < r) and ( $a[mm + 1].key \geq a[mm].key$ ) do
          mm := mm + 1;
          {jetzt ist mm das letzte Element des ersten Runs}
          if mm < r
            then {es ist noch ein zweiter Run vorhanden}
              begin
                rr := mm + 1; {rechter Rand}
                {C1} while (rr < r) and ( $a[rr + 1].key \geq a[rr].key$ ) do
                  rr := rr + 1;
                  merge(a, ll, mm, rr)
                end
              else {kein zweiter Run vorhanden: fertig}
                rr := mm
              end
            until ll = l {dann ist  $a[l] \dots a[r]$  ein Run, also sortiert}
          end
        until ll = l {dann ist  $a[l] \dots a[r]$  ein Run, also sortiert}
      end
    end
  end

```

Die angegebene Prozedur *naturalmergesort* ist so noch nicht ganz korrekt. Die beiden kombinierten Bedingungen

$$(\textit{mm} < \textit{r}) \textbf{ and } (a[\textit{mm} + 1].key \geq a[\textit{mm}].key)$$

und

$$(\textit{rr} < \textit{r}) \textbf{ and } (a[\textit{rr} + 1].key \geq a[\textit{rr}].key)$$

führen zu einem Fehler, wenn das Feldelement $a[r + 1]$ nicht existiert. Der Grund liegt darin, daß in Pascal keine Annahmen über das Auswerten von Teilen zusammengesetzter Bedingungen gemacht werden. Das bedeutet, daß möglicherweise der Teil $(a[mm + 1].key \geq a[mm].key)$ der ersten Bedingung auch dann noch ausgewertet wird, wenn $(mm < r)$ bereits den Wert *false* liefert. Dann ist aber $mm = r$, und damit erfolgt ein Zugriff auf $a[r + 1]$. Der Wert des Feldelements beeinflusst den Wahrheitswert der die Schleife kontrollierenden Bedingung natürlich nicht. Es genügt also, ein um ein Feldelement größeres Feld zu vereinbaren und das Feldelement mit dem höchsten Index unbenutzt zu lassen.

Analyse: Zur Ermittlung der Runs werden gegenüber reinem 2-Wege-Mergesort zusätzliche Schlüsselvergleiche ausgeführt, und zwar linear viele in jedem Durchgang. Bei $\lceil \log N \rceil$ Durchgängen im schlimmsten Fall ergeben sich damit zusätzlich $O(N \log N)$ Schlüsselvergleiche. Damit ist, wie auch schon beim reinen 2-Wege-Mergesort,

$$C_{\max}(N) = \Theta(N \log N).$$

Der Vorzug des natürlichen 2-Wege-Mergesort liegt aber gerade in der Ausnutzung einer Vorsortierung. Im besten Fall ist die gegebene Folge bereits komplett sortiert, besteht also aus nur einem einzigen Run. Einmaliges Durchlaufen der Folge genügt, um dies festzustellen und das Sortieren zu beenden. Also gilt

$$C_{\min}(N) = \Theta(N).$$

Um die mittlere Anzahl $C_{\text{mit}}(N)$ von Schlüsselvergleichen zu bestimmen, überlegen wir uns, wieviele natürliche Runs eine zufällig gewählte Permutation von N Schlüsseln im Mittel enthält. Betrachten wir zwei benachbarte Schlüssel k_i und k_{i+1} der zufällig gewählten Permutation. Dann ist die Wahrscheinlichkeit, daß $k_i < k_{i+1}$ ist, gleich der Wahrscheinlichkeit, daß $k_i > k_{i+1}$ ist, also gerade $1/2$ (unter der Annahme, daß alle Schlüssel verschieden sind). Falls $k_i > k_{i+1}$, dann ist bei k_i ein Run zu Ende. Die Anzahl der Stellen, an denen ein Run zu Ende ist, ist also etwa $N/2$; damit ergeben sich im Mittel etwa $N/2$ Runs. Beim reinen 2-Wege-Mergesort erhalten wir bereits nach einem Durchlauf gerade $N/2$ Runs; daher sparen wir beim natürlichen 2-Wege-Mergesort lediglich einen Durchlauf im Mittel, also lediglich etwa N Schlüsselvergleiche. Somit ergibt sich

$$C_{\text{mit}}(N) = \Theta(N \log N).$$

Anders ausgedrückt heißt das, daß im Mittel eine zufällig gewählte Schlüsselfolge nicht besonders gut vorsortiert ist, wenn man die Anzahl der Runs als Maß für die Vorsortierung wählt (vgl. dazu Abschnitt 2.6).

Die Anzahl der Bewegungen von Datensätzen läßt sich aufgrund dieser Überlegungen unmittelbar angeben:

$$M_{\min}(N) = 0$$

und

$$M_{\max}(N) = M_{\text{mit}}(N) = \Theta(N \log N).$$

Wenn Bewegungen von Datensätzen unerwünscht sind (z.B. wenn Datensätze groß sind), kann es vorteilhaft sein, verkettete lineare Listen von Datensätzen zu sortieren. Dann genügt es nämlich, die Zeiger von Listenelementen zu ändern; Bewegungen von Datensätzen erübrigen sich. Verkettete Listen sind deswegen für Mergesort-Varianten besonders geeignet, weil stets alle Teilfolgen nur sequentiell inspiziert werden; die Möglichkeit des Zugriffs auf beliebige Feldelemente haben wir nie in Anspruch genommen. Aus diesem Grund ist Mergesort auch ein gutes externes Sortierverfahren (vgl. Abschnitt 2.7).

Wir haben in allen Mergesort-Varianten die Prozedur *merge* für das Verschmelzen von zwei Teilfolgen verwendet. Dabei wurde linear viel zusätzlicher Speicherplatz benötigt. Es gibt auch Verfahren, die das Verschmelzen in situ, mit nur konstant viel zusätzlichem Speicherplatz bewerkstelligen, und die trotzdem nur linear viele Schlüsselvergleiche ausführen (siehe z.B.  oder .

2.5 Radixsort

In allen bisher behandelten Sortierverfahren waren Schlüsselvergleiche die einzige Informationsquelle, um die richtige Anordnung der Datensätze zu ermöglichen. Wir haben zwar zur Vereinfachung stets vorausgesetzt, daß die Schlüssel ganzzahlig sind. Die bisher besprochenen Sortierverfahren haben aber keine arithmetischen Eigenschaften der Schlüssel benutzt. Vielmehr wurde immer nur vorausgesetzt, daß das Universum der Schlüssel angeordnet ist und die relative Anordnung zweier Schlüssel durch einen in konstanter Zeit ausführbaren Schlüsselvergleich festgestellt werden kann.

Wir lassen diese Annahme jetzt fallen und nehmen an, daß die Schlüssel Wörter über einem aus m Elementen bestehenden Alphabet sind. Beispiele sind:

$m = 10$ und die Schlüssel sind Dezimalzahlen;

$m = 2$ und die Schlüssel sind Dualzahlen;

$m = 26$ und die Schlüssel sind Wörter über dem Alphabet $\{a, \dots, z\}$.

Man kann die Schlüssel also als m -adische Zahlen auffassen. Daher nennt man m auch die Wurzel (lateinisch: *radix*) der Darstellung.

Für die in diesem Abschnitt besprochenen Sortierverfahren machen wir folgende, vereinfachende Annahme: Die Schlüssel der N zu sortierenden Datensätze sind m -adische Zahlen gleicher Länge. Wenn alle Schlüssel verschieden sind, muß folglich die Länge wenigstens $\log_m N$ betragen. In Abschnitt 2.5.1 setzen wir sogar $m = 2$ voraus. Radix-Sortierverfahren inspizieren die einzelnen Ziffern der m -adischen Schlüssel. Wir setzen daher voraus, daß wir eine in konstanter Zeit ausführbare Funktion $z_m(i, k)$ haben, die für einen Schlüssel k die Ziffer mit Gewicht m^i in der m -adischen Darstellung von k , also die i -te Ziffer von rechts liefert, wenn man Ziffernpositionen ab 0 zu zählen beginnt. Es ist also z.B.:

$$z_{10}(0, 517) = 7;$$

$$z_{10}(1, 517) = 1;$$

$$z_{10}(2, 517) = 5.$$

In Abschnitt 2.5.1 geben wir ein Radix-Sortierverfahren an, das eine rekursive Aufteilung des zu sortierenden Feldes analog zu Quicksort vornimmt. Dieses Verfahren hat in der Literatur den Namen *Radix-exchange-sort*. Das in Abschnitt 2.5.2 besprochene Radix-Sortierverfahren heißt Binsort, Bucketsort oder auch *Sortieren durch Fachverteilung*, weil es die zu sortierenden Datensätze wiederholt in Fächern (Bins, Buckets) ablegt, bis schließlich eine sortierte Reihenfolge vorliegt.

2.5.1 Radix-exchange-sort

Method: Wir teilen das gegebene, nach aufsteigenden Binärschlüsseln gleicher Länge zu sortierende Feld $a[1] \dots a[N]$ von Datensätzen in Abhängigkeit vom führenden Bit der binären Sortierschlüssel in zwei Teile. Alle Elemente, deren Schlüssel eine führende 0 haben, kommen in die linke Teilfolge, und alle Elemente, deren Schlüssel eine

führende 1 haben, kommen in die rechte Teilfolge. Die Aufteilung wird ähnlich wie bei Quicksort in situ durch Vertauschen von Elementen des Feldes erreicht. Die Teilfolgen werden rekursiv auf dieselbe Weise sortiert, wobei natürlich jetzt das zweite Bit von links die Rolle des führenden Bits übernimmt.

Die Aufteilung eines Bereichs nach einer bestimmten Bitposition der Schlüssel des Bereichs erfolgt wie bei Quicksort. Man wandert mit zwei Zeigern vom linken und rechten Ende über den aufzuteilenden Bereich. Immer wenn der linke Zeiger auf ein Element stößt, das an der für die Aufteilung maßgeblichen Bitposition eine 1 hat, und der rechte auf ein Element stößt, das dort eine 0 hat, werden die beiden Elemente vertauscht. Die Aufteilung ist beendet, wenn die Zeiger übereinandergelaufen sind.

Tabelle 2.4 zeigt am Beispiel von sieben Binär-Schlüsseln der Länge 4 das Ergebnis der einzelnen Aufteilungsschritte.

Für die Aufteilung maßgebliches Bit:							
1011	0010	1101	1001	0011	0101	1010	3
0101	0010	0011 ;	1001	1101	1011	1010	2
0011	0010 ;	0101 ;	1001	1010	1011 ;	1101	1
0011	0010 ;	0101 ;	1001 ;	1010	1011 ;	1101	0
0010 ;	0011 ;	0101 ;	1001 ;	1010 ;	1011 ;	1101	

Tabelle 2.4

In dieser Tabelle ist das Ende der jeweils durch Aufteilung entstandenen Folgen durch „;“ markiert. Im Unterschied zu Quicksort kann man nicht direkt einen Stopper verwenden, um das Wandern der Zeiger im Aufteilungsschritt zu beenden. Wir nehmen daher in die das Wandern der Zeiger kontrollierende Schleifenbedingung explizit den Test auf, ob ein Zeiger das Ende des jeweils aufzuteilenden Bereichs bereits erreicht hat.

```

procedure radixexchangesort (var  $a$  : sequence;  $l, r, b$  : integer);
{sortiert die Elemente  $a[l] \dots a[r]$  nach aufsteigenden Werten
 der Endstücke von Schlüsseln, die aus Bits
 an den Positionen  $0, \dots, b$  bestehen}
var
   $i, j$  : integer; {Zeiger}
   $t$  : item; {Hilfsspeicher}
begin
  if  $r > l$ 
  then
    begin
      {teile Bereich  $a[l] \dots a[r]$  abhängig vom Bit an Position  $b$ 
 der Schlüssel auf}
       $i := l - 1$ ;
       $j := r + 1$ ;
    
```

```

begin-loop
  repeat  $i := i + 1$  until ( $z_2(b, a[i].key) = 1$ ) or ( $i \geq j$ );
  repeat  $j := j - 1$  until ( $z_2(b, a[j].key) = 0$ ) or ( $i \geq j$ );
  if  $i \geq j$ 
    then exit-loop;
     $t := a[i];$ 
     $a[i] := a[j];$ 
     $a[j] := t$ 
  end-loop;
  {alle Elemente mit einer 0 an Bit-Position  $b$  stehen jetzt
   links von allen Elementen mit einer 1 an dieser Position;
    $i$  zeigt auf den Beginn dieser rechten Teilfolge.
   Gibt es keine Elemente in  $a[l] \dots a[r]$  mit einer 1
   an Bitposition  $b$ , so ist  $i = r + 1$ }
  if  $b > 0$  {das 0-te Bit ist noch nicht inspiziert}
    then
      begin
         $radixexchangesort(a, l, i - 1, b - 1);$ 
         $radixexchangesort(a, i, r, b - 1)$ 
      end
    end
  end

```

Ein Aufruf von $radixexchangesort(a, 1, N, \text{Schlüssellänge})$ sortiert dann das gegebene Feld.

Der Aufteilungsschritt für einen Bereich $a[l] \dots a[r]$, abhängig vom Schlüsselbit an Position b , benötigt wie bei Quicksort $c \cdot (r - l)$ Schritte mit einer Konstanten c . Zwar kann die Prozedur $radixexchangesort$ für Schlüssel der Länge $b + 1$ insgesamt $(2^{b+1} - 1)$ -mal rekursiv aufgerufen werden, aber die maximale Rekursionstiefe ist höchstens b . Alle Aufteilungen auf derselben Rekursionsstufe, also alle von derselben Bitposition abhängigen Aufteilungen, können insgesamt in linearer Zeit ausgeführt werden. Daraus folgt, daß die Laufzeit des Verfahrens — unabhängig von der Eingabefolge — stets durch $O(N \cdot b)$ abgeschätzt werden kann. Ist $b = \log N$, dann ist Radix-exchange-sort eine echte Alternative zu Quicksort. Hat man aber wenige lange Schlüssel, ist Radix-exchange-sort schlecht.

2.5.2 Sortieren durch Fachverteilung

Mancher Leser erinnert sich vielleicht noch ungefähr daran, wie das Sortieren eines Lochkartenstapels durch ein mechanisches Sortiergerät abläuft. Der zu sortierende Kartenstapel wird (in Abhängigkeit von der Lochung an einer bestimmten Position) auf verschiedene Fächer verteilt. Die in den Fächern abgelegten Teilstapel werden dann in einer bestimmten, festen Reihenfolge eingesammelt und erneut, abhängig von der nächsten Lochkartenposition, verteilt usw., bis schließlich ein sortierter Stapel entstanden ist. Charakteristisch für dieses Sortiervorgehen ist also der Wechsel zwischen einer

Verteilungsphase und einer *Sammelphase*. Wir beschreiben beide Phasen nun genauer und setzen dazu voraus, daß das Feld der zu sortierenden Datensätze $a[1] \dots a[N]$ m -adische Schlüssel gleicher Länge l hat. Verteilungs- und Sammelphase werden insgesamt l -mal durchgeführt. Denn die Verteilungsphase hängt ab von der jeweils gerade betrachteten Ziffer an Position t der m -adischen Schlüssel, wobei t die Positionen von 0 bis $l - 1$ durchläuft, also von der niedrigstwertigen zur höchstwertigen Ziffernposition.

In der *Verteilungsphase* werden die Datensätze auf m Fächer verteilt. Das i -te Fach F_i nimmt alle Datensätze auf, deren Schlüssel an Position t die Ziffer i haben. Der jeweils nächste Satz wird stets „oben“ auf die in seinem Fach bereits vorhandenen Sätze gelegt.

In der *Sammelphase* werden die Sätze in den Fächern $F_0, \dots, F_{m-1}$ so eingesammelt, daß die Sätze im Fach F_{i+1} als Ganzes „oben“ auf die Sätze im Fach F_i gelegt werden (für $0 \leq i < m - 1$). Die relative Anordnung der Sätze innerhalb eines jeden Fachs bleibt unverändert.

In der auf eine Sammelphase folgenden Verteilungsphase müssen die Datensätze von „unten“ nach „oben“ verteilt werden, also zuerst wird der „unterste“ Satz in sein Fach gelegt, dann der „zweitunterste“ usw., bis schließlich der „oberste“ Satz auf ein Fach verteilt ist. Am Ende der letzten Sammelphase sind dann die Datensätze von „unten“ nach „oben“ sortiert.

Wir illustrieren das Verfahren am Beispiel einer Menge von 12 Datensätzen mit zweistelligen Dezimalschlüsseln. (D h. wir haben $N = 12$, $m = 10$, $l = 2$.) Gegeben sei die unsortierte Schlüsselfolge

40, 13, 22, 54, 15, 28, 76, 04, 77, 38, 16, 18.

Während der ersten Verteilungsphase werden die Schlüssel in Abhängigkeit von der am weitesten rechts stehenden Dezimalziffer (an Position $t = 0$) wie folgt auf zehn Fächer verteilt.

								18	
				04		16		38	
<u>40</u>	<u>13</u>	<u>22</u>	<u>54</u>	<u>15</u>	<u>28</u>	<u>76</u>	<u>04</u>	<u>77</u>	<u>38</u>
F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9

Nach der ersten Sammelphase ergibt sich die Schlüsselfolge

40, 22, 13, 54, 04, 15, 76, 16, 77, 28, 38, 18.

Erneute Verteilung nach der Ziffer an Position $t = 1$ ergibt:

									18
									16
				15	28				77
<u>04</u>	<u>13</u>	<u>22</u>	<u>38</u>	<u>40</u>	<u>54</u>	<u>16</u>	<u>76</u>	<u>18</u>	<u>28</u>
F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9

Sammeln der Schlüssel in den Fächern ergibt die sortierte Schlüsselfolge

04, 13, 15, 16, 18, 22, 28, 38, 40, 54, 76, 77.

Für den Nachweis der Korrektheit des Verfahrens ist die folgende Beobachtung wichtig: Die von der Ziffer an Position t abhängige Verteilungs- und Sammelphase liefert eine aufsteigende Sortierung der Sätze nach dieser Schlüsselziffer; dabei bleibt die relative Anordnung der Schlüssel innerhalb der Fächer erhalten. Genauer: War die Zahlenfolge vor Beginn eines Durchgangs (bestehend aus Verteilungs- und Sammelphase) nach aufsteigenden Werten sortiert, wenn man nur die Ziffernpositionen $0, \dots, t-1$ betrachtet (bzw. unsortiert, wenn $t=0$ ist), so folgt: Nach Ende des Durchgangs sind die Schlüssel bzgl. der Ziffernpositionen $0, \dots, t$ aufsteigend sortiert. l Durchgänge stellen also eine insgesamt sortierte Reihenfolge her, wenn dabei der Reihe nach die Ziffernpositionen $0, \dots, l-1$ betrachtet werden.

Eine naive Implementation des Verfahrens erfordert die programmtechnische Realisierung von m Fächern als Felder der Größe N ; denn jedes Fach kann ja im ungünstigsten Fall alle N Datensätze aufnehmen müssen. Das ist schon für $m=10$ kein gangbarer Weg, weil dann zur Sortierung von N Datensätzen $m \cdot N$ zusätzliche Speicherplätze reserviert werden müssen. Es gibt zwei naheliegende Möglichkeiten, diese enorme Speicherplatzverschwendung zu vermeiden. Eine erste Möglichkeit ist, zu Beginn eines jeden Durchlaufs (dessen Verteilungsphase von der t -ten Ziffer abhängt) zu zählen, wieviele Sätze in jedes Fach fallen werden. Da insgesamt nicht mehr als N Datensätze verteilt werden müssen, genügt es dann, insgesamt N zusätzliche Speicherplätze zu vereinbaren. Man kann in diesem Bereich alle Fächer unterbringen und die jeweiligen Bereiche der einzelnen Fächer aus den zuvor ermittelten Anzahlen leicht bestimmen. Wir nennen die Indizes der Grenzen dieser Bereiche die *Verteilungszahlen*. Eine andere Möglichkeit ist, die m Fächer als m verkettete lineare Listen mit variabler Länge zu realisieren. In der Verteilungsphase werden die Datensätze stets an das Ende der jeweiligen Liste angehängt. In der Sammelphase werden die Listen der Reihe nach von vorn nach hinten durchlaufen.

Für beide Varianten werden Prozeduren angegeben, die eine Sortierung des Feldes a vom Typ *sequence* mit ganzzahligen Schlüsselkomponenten $a[i].key$ vornehmen. Wir nehmen an, daß die Schlüssel m -adische Zahlen der Länge l sind, wobei m und l außerhalb der Prozeduren festgelegte Konstanten sind. Wir erinnern daran, daß für jede Zahl t , mit $0 \leq t < m$, $z_m(t, a[i].key)$ die t -te Ziffer des m -adischen Schlüssels des i -ten Satzes ist.

```

procedure radixsort_1 (var  $a$  : sequence);
  var
     $b$  : sequence; {Speicher zur Aufnahme der Fächer}
     $c$  : array  $[0 \dots m]$  of integer; {Verteilungszahlen}
     $i, j, t$  : integer;
  begin
    for  $t := 0$  to  $l-1$  do
      begin {Durchlauf}
        {Verteilungsphase: Verteilungszahlen bestimmen}
        for  $i := 0$  to  $m-1$  do  $c[i] := 0$ ;
        for  $i := 1$  to  $N$  do
          begin
             $j := z_m(t, a[i].key)$ ;
             $c[j] := c[j] + 1$ 

```

```

    end;
     $c[m-1] := N + 1 - c[m-1];$ 
    for  $i := 2$  to  $m$  do  $c[m-i] := c[m-i+1] - c[m-i];$ 
    { $c[i]$  ist Index des Anfangs von Fach  $F_i$  im Feld  $b$ }
    {verteilen}
    for  $i := 1$  to  $N$  do
        begin
             $j := z_m(t, a[i].key);$ 
             $b[c[j]] := a[i];$ 
             $c[j] := c[j] + 1$ 
        end;
    {Sammelphase}
    for  $i := 1$  to  $N$  do  $a[i] := b[i]$ 
end {Durchlauf}
end

```

Für die zweite Radixsort-Variante machen wir Gebrauch von dem in Kapitel 1 beschriebenen Datentyp *Liste*, der die Menge aller Listen von Objekten eines gegebenen Grundtyps einschließlich der leeren Liste bezeichnet und mit **list of** vereinbart wird. Wir erinnern daran, daß für eine Variable L vom Typ *Liste*, also

var L : **list of** Grundtyp

und eine Variable x des Grundtyps, also

var x : Grundtyp

die folgenden Prozeduren und Funktionen für L und x erklärt und in konstanter Zeit ausführbar sind:

$pushtail(L, x)$: hängt x an das Ende von L an; das Resultat ist L ;
 $pophead(L, x)$: entfernt das erste Element aus L ; die entstehende Liste ist L ; das entfernte Element ist x ;
 $empty(L)$: liefert den Wert *true* genau dann, wenn L die leere Liste ist, und den Wert *false* sonst;
 $init(L)$: liefert für L die leere Liste.

Dann läßt sich die Prozedur *radixsort_2* wie folgt angeben:

```

procedure radixsort_2 (var  $a$  : sequence);
var
     $L$  : array  $[0 \dots (m-1)]$  of list of item;
     $i, j, t$  : integer;
begin
    for  $j := 0$  to  $(m-1)$  do  $init(L[j]);$  {Fächer leeren}
    for  $t := 0$  to  $l-1$  do
        begin {Durchlauf}
            {Verteilungsphase}
            for  $i := 1$  to  $N$  do {verteilen}

```

```

begin
   $j := z_m(t, a[i].key);$ 
   $pushtail(L[j], a[i])$ 
end;
{Sammelphase}
 $i := 1;$ 
for  $j := 0$  to  $m - 1$  do { $L[j]$  einsammeln}
  while not  $empty(L[j])$  do
    begin
       $pophead(L[j], a[i]);$ 
       $i := i + 1$ 
    end {while}
  end {Durchlauf}
end

```

Aus den angegebenen Programmstücken kann man unmittelbar ablesen, daß beide Radixsort-Versionen in $O(l(m+N))$ Schritten ausführbar sind. Der Speicherbedarf liegt in beiden Fällen in der Größenordnung $O(N+m)$.

Wir diskutieren einige Spezialfälle genauer. Sollen m verschiedene Schlüssel im Bereich $0, \dots, m-1$ sortiert werden, so ist also $m = N$ und $l = 1$. In diesem Fall liefert Radixsort eine sortierte Folge in linearer Zeit und mit linearem Platz. Dieser sehr spezielle Fall kann natürlich viel einfacher wie folgt gelöst werden. Man vereinbart ein Feld b vom Typ

array[$0 \dots (m-1)$] **of** *item*

und erreicht durch die Anweisung

for $i := 1$ **to** N **do** $b[a[i].key] := a[i],$

daß die Sätze des gegebenen Feldes a in b nach aufsteigenden Schlüsseln sortiert vorkommen. Die Sortierung ist hier also trivial erreichbar, weil jedes „Fach“ genau einen Satz aufnimmt.

Haben die gegebenen N Datensätze Schlüssel fester Länge l im Bereich $0, \dots, m^l - 1$, ist also l konstant und $m < N$, so ist Radixsort in linearer Zeit ausführbar. Es ist klar, daß $l \geq \lceil \log_m N \rceil$ sein muß, wenn alle N Schlüssel verschieden sind. Solange die Schlüssel „kurze“ m -adische Zahlen sind, also $l = c \cdot \lceil \log_m N \rceil$ mit einer „kleinen“ Konstanten c , bleibt Radixsort ein praktisch brauchbares Verfahren mit einer Gesamtlaufzeit von $O(N \log N)$ im schlechtesten Fall.

2.6 Sortieren vorsortierter Daten

Nicht selten sind zu sortierende Datenbestände bereits teilweise vorsortiert. Sie können etwa aus einigen bereits sortierten Teilen zusammengesetzt sein, oder an ein größeres, sortiertes File werden am Ende einige wenige Sätze in beliebiger Reihenfolge angehängt. Viele Sortierv Verfahren ziehen aber aus einer Vorsortierung keinerlei Nutzen.

Schlimmer noch: Manche Sortierv Verfahren, wie etwa Quicksort, sind für vorsortierte Daten sogar besonders schlecht. Damit stellt sich die Frage: Gibt es Sortierv Verfahren, die die in einem Datenbestand bereits vorhandene Vorsortierung optimal nutzen?

In dieser Form ist die Frage natürlich viel zu unpräzise formuliert, um eine klare ja/nein Antwort zu erlauben. Wir werden daher in Abschnitt 2.6.1 zunächst einige gebräuchliche Maße zur Messung der Vorsortierung einer Folge von Schlüsseln vorstellen und präzise definieren, was es heißt, daß ein Sortierv Verfahren von einer so gemessenen Vorsortierung optimalen Gebrauch macht. In Abschnitt 2.6.2 stellen wir ein erstes „adaptives“ Sortierv Verfahren vor, das die Vorsortierung einer Folge optimal nutzt, wenn man sie mit der Inversionszahl mißt. Das in Abschnitt 2.6.3 besprochene *Sortieren durch lokales Einfügen* ist sogar für drei verschiedene Vorsortierungsmaße optimal.

2.6.1 Maße für Vorsortierung

Betrachten wir einige verschiedene Folgen von 9 Schlüsseln, die aufsteigend sortiert werden sollen:

$$\begin{array}{lllllllll} F_a : & 2 & 1 & 4 & 3 & 6 & 5 & 8 & 7 & 9 \\ F_b : & 6 & 7 & 8 & 9 & 1 & 2 & 3 & 4 & 5 \\ F_c : & 5 & 1 & 7 & 4 & 9 & 2 & 8 & 3 & 6 \end{array}$$

Intuitiv würde man sagen: Die Folge F_c ist weniger vorsortiert als F_a und F_b . F_a ist global schon ganz gut sortiert, denn kleine Schlüssel stehen eher am Anfang, große Schlüssel eher am Ende der Folge. Die Unordnung in F_a ist also lokaler Natur. Tatsächlich sind gegenüber der sortierten Folge einfach Paare benachbarter Schlüssel vertauscht. Das umgekehrte gilt für Folge F_b . Lokal ist F_b ganz gut sortiert, denn die meisten Paare benachbarter Schlüssel in F_b stehen in der richtigen Reihenfolge, aber global ist F_b ziemlich ungeordnet, denn große Schlüssel stehen am Anfang, kleine am Ende der Folge. Wie läßt sich das quantitativ messen? Wir machen dazu jetzt drei verschiedene Vorschläge und diskutieren ihre Vor- und Nachteile.

Die erste Möglichkeit besteht darin, die Anzahl der *Inversionen* (oder: Fehlstellungen) zu messen. Das ist die Anzahl von Paaren von Schlüsseln, die in der falschen Reihenfolge stehen. In der Beispielfolge F_a sind dies die vier Paare (2,1), (4,3), (6,5), (8,7), und in der Beispielfolge F_b die 20 Paare

$$\begin{array}{l} (6,1), (6,2), (6,3), (6,4), (6,5), \\ (7,1), (7,2), (7,3), (7,4), (7,5), \\ (8,1), (8,2), (8,3), (8,4), (8,5), \\ (9,1), (9,2), (9,3), (9,4), (9,5). \end{array}$$

Allgemein: Sei $F = \langle k_1, \dots, k_N \rangle$ eine Folge von Schlüsseln, die aufsteigend sortiert werden soll. Wir setzen voraus, daß alle Schlüssel k_i verschiedene (ganze) Zahlen sind. Dann heißt die Anzahl der Paare in falscher Reihenfolge

$$\text{inv}(F) = \left| \{ (i, j) \mid 1 \leq i < j \leq N \text{ und } k_i > k_j \} \right|$$

die *Inversionszahl* von F . Falls F bereits aufsteigend sortiert ist, so ist offenbar $inv(F) = 0$. Im ungünstigsten Fall kann jedes Element k_i in einer Folge F vor Elementen $k_{i+1}, \dots, k_N$ stehen, die sämtlich kleiner als k_i sind. Dies ist der Fall für eine absteigend sortierte Folge F , für deren Inversionszahl offenbar gilt:

$$inv(F) = (N - 1) + (N - 2) + \dots + 2 + 1 = \frac{N(N - 1)}{2}.$$

Die Inversionszahl mißt die globale Vorsortierung. Die oben angegebenen Beispielfolgen zeigen das sehr deutlich: Die Folge F_a hat eine kleine, die Folge F_b hat eine große Inversionszahl. Dennoch würde man auch Folge F_b als gut vorsortiert ansehen; sie läßt sich leicht und schnell etwa mit dem Verfahren Sortieren durch Verschmelzen in eine sortierte Reihenfolge bringen. Das nächste Maß berücksichtigt diese Art Vorsortierung besser. Man nimmt die Anzahl der bereits aufsteigend sortierten Teilfolgen einer gegebenen Folge. Diese Zahl heißt die *Run-Zahl* (englisch: run = Lauf). Sie ist für eine Folge $F = \langle k_1, \dots, k_N \rangle$ wie folgt definiert:

$$runs(F) = |\{i | 1 \leq i < N \text{ und } k_{i+1} < k_i\}| + 1.$$

Für die Folge F_b ist offenbar $runs(F_b) = 2$, weil in F_b nur eine Stelle vorkommt, an der ein größeres Element einem kleineren unmittelbar vorangeht. In der Folge F_a gibt es vier solcher Stellen, die wir mit einem „↑“ markiert haben.

$$\begin{array}{cccccccccc} 2 & 1 & 4 & 3 & 6 & 5 & 8 & 7 & 9 \\ & \uparrow & & \uparrow & & \uparrow & & \uparrow & \end{array}$$

Also ist $runs(F_a) = 4 + 1 = 5$. Für eine bereits aufsteigend sortierte Folge ist die Run-Zahl 1. Im ungünstigsten Fall kann an jeder Stelle zwischen je zwei benachbarten Elementen ein größeres einem kleineren Folgeelement unmittelbar vorangehen. Das ist der Fall für absteigend sortierte Folgen. Die Run-Zahl einer absteigend sortierten Folge der Länge N ist also N . Eine kleine Run-Zahl ist also ein Indiz für einen hohen Grad von Vorsortiertheit. Die Run-Zahl ist aber eher ein lokales Maß für die Vorsortierung. Denn eine intuitiv gut vorsortierte Folge mit nur lokaler Unordnung, wie die Folge $\langle 2, 1, 4, 3, \dots, N, N - 1 \rangle$ hat eine hohe Run-Zahl (von ungefähr $N/2$).

Ein die genannten Nachteile (vorwiegend lokal oder vorwiegend global orientiert) vermeidendes Maß für die Vorsortierung beruht auf der Messung der *längsten aufsteigenden Teilfolge las* (longest ascending subsequence) einer Folge $F = \langle k_1, \dots, k_N \rangle$:

$$las(F) = \max\{t \mid \exists i(1), \dots, i(t) \text{ so daß } 1 \leq i(1) < \dots < i(t) \leq N \text{ und } k_{i(1)} < \dots < k_{i(t)}\}.$$

Offensichtlich gilt für eine Folge F der Länge N stets $1 \leq las(F) \leq N$. Für eine gut vorsortierte Folge F ist $las(F)$ groß, und für eine schlecht vorsortierte, wie die absteigend sortierte Folge F , ist $las(F)$ klein. $las(F)$ wächst also gerade umgekehrt wie die beiden vorher eingeführten Maße inv und $runs$. Man benutzt daher besser die Differenz $N - las(F)$ zur Messung der Vorsortierung. Sie gibt an, wieviele Elemente von F wenigstens entfernt werden müssen (englisch: remove), um eine aufsteigend sortierte Folge zu hinterlassen:

$$rem(F) = N - las(F).$$

Die oben angegebene Beispielfolge F_a hat mehrere längste aufsteigende Teilfolgen mit Länge fünf. F_b hat eine längste aufsteigende Teilfolge ebenfalls mit Länge fünf. Also gilt für beide Folgen

$$\text{rem}(F_a) = \text{rem}(F_b) = 9 - 5 = 4.$$

Das Maß rem ist weniger intuitiv als die Maße inv und runs . Die Berechnung von $\text{rem}(F)$ für eine gegebene Folge F ist ein durchaus nichttriviales algorithmisches Problem. Wir verweisen hier nur auf die Arbeit von H. Mannila [10].

Es gibt in der Literatur noch weitere Vorschläge zur Messung der Vorsortiertheit von Schlüsselfolgen und auch den Versuch einer allgemeinen Theorie durch axiomatische Fassung der ein Maß für Vorsortierung charakterisierenden Eigenschaften. Da es uns nur auf einige grundsätzliche Aspekte des Problems, vorsortierte Folgen möglichst effizient zu sortieren, ankommt, wollen wir uns mit der Betrachtung der drei oben angegebenen Maße begnügen.

Wir wollen jetzt präzisieren, was es heißt, daß ein Sortierverfahren Vorsortierung optimal nutzt, wenn man ein Maß m zur Messung der Vorsortierung wählt. Erinnern wir uns daran, daß jeder Algorithmus zur Lösung des Sortierproblems zwei Teilprobleme löst, und zwar ein Informationsbeschaffungsproblem und ein Datentransportproblem. Die sortierte Schlüsselfolge muß zunächst unter allen anderen Folgen (gleicher Länge) identifiziert werden. Im einfachsten Fall geschieht dies durch Ausführen von Vergleichsoperationen zwischen je zwei Schlüsseln. Algorithmen, die nur auf diese Weise Informationen über die (relative) Anordnung der zu sortierenden Schlüssel gewinnen, heißen *allgemeine* Sortierverfahren. Sämtliche bisher besprochenen Verfahren mit Ausnahme von Radixsort gehören zu dieser Klasse. Zweitens ist ein Datentransportproblem zu lösen. Die zu sortierende Folge muß durch Bewegen von Datensätzen (oder Zeigern) in die richtige Reihenfolge gebracht werden.

Für die Definition eines *m-optimalen Sortierverfahrens*, wobei m ein Maß für die Vorsortierung ist, beschränken wir uns auf die Klasse der allgemeinen Sortierverfahren. Dann kann man als untere Schranke für die Laufzeit eines solchen Verfahrens die zum Sortieren ausgeführte Zahl von Schlüsselvergleichen nehmen.

Wann ist ein Sortierverfahren *m-optimal*? Intuitiv doch dann, wenn das Verfahren zum Sortieren einer Folge F auch nur die für Folgen dieses Vorsortierungsgrades minimal nötige Schrittzahl tatsächlich benötigt. Folgen mit kleinem $m(F)$ -Wert sollen schnell, Folgen mit größerem $m(F)$ -Wert entsprechend langsamer sortiert werden. Versuchen wir zunächst, die mindestens erforderliche Anzahl von Schlüsselvergleichen abzuschätzen für ein gegebenes Sortierverfahren. Wir nehmen an, daß das Verfahren keine überflüssigen Schlüsselvergleiche ausführt.

Wir können das Verfahren wie jedes allgemeine Sortierverfahren durch einen sogenannten *Entscheidungsbaum* mit genau $N!$ Blättern repräsentieren (vgl. auch Abschnitt 2.8). Der Entscheidungsbaum ist ein Mittel zur statischen Veranschaulichung aller Schlüsselvergleiche, die zum Sortieren aller $N!$ Folgen der Länge N mit Hilfe eines gegebenen Verfahrens ausgeführt werden. Jedem Pfad von der Wurzel zu einem Blatt in diesem Baum entspricht die zur Identifizierung (und Sortierung) einer bestimmten Schlüsselfolge ausgeführte Folge von Vergleichsoperationen.

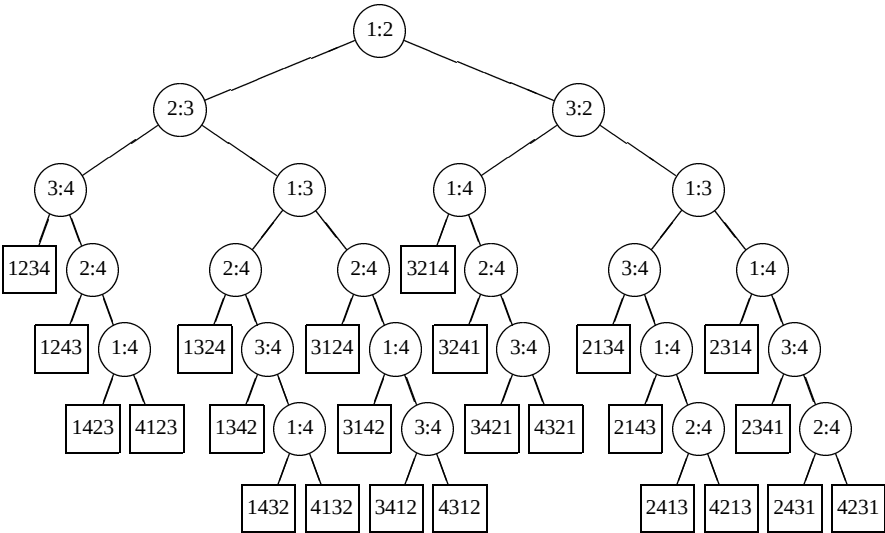


Abbildung 2.10

Wir betrachten als Beispiel in Abbildung 2.10 einen Entscheidungsbaum für vier verschiedene Schlüssel.

Er modelliert das Verhalten eines bestimmten allgemeinen Sortierverfahrens für Schlüsselfolgen der Länge 4. Ist eine Folge $F = \langle k_1, k_2, k_3, k_4 \rangle$ gegeben, so werden zunächst die Schlüssel k_1 und k_2 miteinander verglichen. Ist $k_1 < k_2$, werden als nächste die Schlüssel k_2 und k_3 miteinander verglichen; das entspricht dem Hinabsteigen von dem mit 1 : 2 beschrifteten Wurzelknoten im Entscheidungsbaum zu dessen linkem Sohn. Falls der Vergleich zwischen k_1 und k_2 ergeben hätte, daß $k_1 > k_2$, würde dann anschließend k_3 mit k_2 verglichen und dann k_1 mit k_4 bzw. k_1 mit k_3 , je nachdem, ob $k_3 < k_2$ war oder nicht.

Jedem Pfad von der Wurzel des Entscheidungsbaumes zu einem Blatt entspricht also diejenige Folge von Schlüsselvergleichen, die zur Identifizierung und Sortierung der durch das Blatt repräsentierten Eingabefolge ausgeführt wird. Beispielsweise repräsentiert das Blatt 3 2 1 4 die aufsteigend sortierte Schlüsselfolge, für die $k_3 < k_2 < k_1 < k_4$ gilt. Um sie zu identifizieren, werden nacheinander k_1 mit k_2 , k_3 mit k_2 und k_1 mit k_4 verglichen, d.h. nacheinander die Knoten 1 : 2, 3 : 2 und 1 : 4 betrachtet. Zur Identifizierung dieser Folge werden also nur wenige Schlüsselvergleiche ausgeführt.

Ein allgemeines Sortierverfahren A nutzt die mit einem Maß m gemessene Vorsortierung voll aus, wenn alle bezüglich m maximal vorsortierten Folgen im A entsprechenden Entscheidungsbaum durch Blätter repräsentiert werden, die „so nah wie möglich“ bei der Wurzel sind. Man betrachtet für eine gegebene Folge F also die Menge aller Folgen F' gleicher Länge, die genau so gut wie F oder besser vorsortiert sind. Sei r ihre Anzahl, also:

$$r = |\{F' \mid m(F') \leq m(F)\}|$$

Weil Entscheidungsbäume Binärbäume sind, die auf jedem Niveau i höchstens 2^i Blätter haben können, folgt: Es gibt wenigstens eine Folge $F_0 \in \{F' \mid m(F') \leq m(F)\}$, deren Abstand von der Wurzel des Entscheidungsbaumes wenigstens $\lceil \log r \rceil$ ist. Darüberhinaus muß auch der mittlere Abstand aller Blätter, die Folgen in der Menge $\{F' \mid m(F') \leq m(F)\}$ entsprechen, in $\Omega(\log r)$ sein. (Für einen Beweis dieser Tatsache vgl. Abschnitt 2.8.)

Anders formuliert: Jeder Algorithmus muß zur Identifizierung (und Sortierung) einer Menge von Folgen gleicher Länge und mit gegebenem Vorsortierungsgrad g sowohl im Mittel wie im schlechtesten Fall wenigstens

$$\Omega(\log(|\{F' \mid m(F') \leq g\}|))$$

Vergleichsoperationen zwischen Schlüsseln ausführen.

Man wird ein Verfahren sicher dann *m-optimal* nennen, wenn es diese untere Schranke bis auf einen konstanten Faktor erreicht. Diese Forderung ist allerdings etwas zu scharf, weil die Menge aller Folgen der Länge N mit einem gegebenen Vorsortierungsgrad weniger als 2^N Elemente haben kann. Ein konstantes Vielfaches der Mindestschrittzahl würde dann eine sublineare Laufzeit verlangen. Man erwartet aber andererseits, daß jedes Sortierverfahren jedes Element wenigstens einmal betrachten muß und damit wenigstens Zeitbedarf $\Omega(N)$ hat.

Damit haben wir die endgültige Definition eines *m-optimalen Sortierverfahrens* A . A heißt *m-optimal*, wenn es eine Konstante c gibt, so daß für alle N und alle Folgen F mit Länge N gilt: A sortiert F in Zeit

$$T_A(F, m) \leq c \cdot (N + \log(|\{F' \mid m(F') \leq m(F)\}|)).$$

2.6.2 A-sort

Wir wollen jetzt Sortierverfahren angeben, die Vorsortierung im zuvor präzisierten Sinne optimal nutzen. Wir beginnen mit dem Vorsortierungsmaß *inv* und fragen uns, nach welcher Strategie ein Sortierverfahren arbeiten muß, das die mit der Inversionszahl gemessene Vorsortierung optimal ausnutzt.

Für eine Folge $F = \langle k_1, \dots, k_N \rangle$ von Schlüsseln ist offenbar

$$\text{inv}(F) = |\{(i, j) \mid 1 \leq i < j \leq N \text{ und } k_i > k_j\}| = \sum_{j=1}^N h_j$$

mit

$$h_j = |\{i \mid 1 \leq i < j \leq N \text{ und } k_i > k_j\}|.$$

Für jedes j ist h_j die Anzahl der dem j -ten Element k_j in der gegebenen Folge vorangehenden Elemente, die bei aufsteigender Sortierung k_j nachfolgen müssen. Die Größen h_j lassen sich also auch so deuten: Fügt man $k_1, k_2, \dots$ der Reihe nach in die anfangs leere und sonst stets aufsteigend sortierte Liste ein, so gibt h_j den Abstand des jeweils nächsten einzufügenden Elementes k_j vom Ende der bisher erhaltenen Liste an, die bereits die Elemente $k_1, \dots, k_{j-1}$ enthält.

Betrachten wir ein Beispiel (Folge F_c aus Abschnitt 2.6.1):

$$F = \langle k_1, \dots, k_9 \rangle = \langle 5, 1, 7, 4, 9, 2, 8, 3, 6 \rangle$$

Für diese Folge ergeben sich die in Tabelle 2.5 gezeigten Einzelschritte.

nächstes einzufügendes Element k_i	h_i = Abstand der Einfügestelle vom Ende der bisherigen Liste	nach Einfügen erhaltene Liste mit Markierung der Einfügestelle
$k_1 = 5$	$h_1 = 0$	<u>5</u>
$k_2 = 1$	$h_2 = 1$	<u>1</u> , 5
$k_3 = 7$	$h_3 = 0$	1, 5, <u>7</u>
$k_4 = 4$	$h_4 = 2$	1, <u>4</u> , 5, 7
$k_5 = 9$	$h_5 = 0$	1, 4, 5, 7, <u>9</u>
$k_6 = 2$	$h_6 = 4$	1, <u>2</u> , 4, 5, 7, 9
$k_7 = 8$	$h_7 = 1$	1, 2, 4, 5, 7, <u>8</u> , 9
$k_8 = 3$	$h_8 = 5$	1, 2, <u>3</u> , 4, 5, 7, 8, 9
$k_9 = 6$	$h_9 = 3$	1, 2, 3, 4, 5, <u>6</u> , 7, 8, 9

Tabelle 2.5

Ist die $\sum h_j$, also die Inversionszahl, klein, so müssen auch die h_j (im Durchschnitt) klein sein; d h. das jeweils nächste Element wird nah am rechten Ende der bisher erzeugten Liste eingefügt. Im Extremfall einer bereits aufsteigend sortierten Folge mit Inversionszahl 0 und $h_1 = \dots = h_N = 0$ wird jedes Element ganz am rechten Ende eingefügt. Um Folgen mit kleiner Inversionszahl schnell zu sortieren, sollte man also der *Strategie des Sortierens durch iteriertes Einfügen* folgen und dabei eine Struktur „dynamische, sortierte Liste“ verwenden, die das Einfügen in der Nähe des Listenendes effizient erlaubt. Ein Array läßt sich dafür nicht nehmen. Denn man kann unter Umständen zwar die Einfügestelle für das nächste Element schnell finden (etwa mit binärer oder exponentieller Suche, vgl. hierzu das Kapitel 3), muß aber viele Elemente verschieben, um das nächste Element an der richtigen Stelle unterzubringen. Eine verkettete lineare, aufsteigend sortierte Liste mit einem Zeiger auf das Listenende, die vom Ende her durchsucht (und vom Anfang an ausgegeben) werden kann, ist ebenfalls nicht besonders gut. Zwar kann man in eine derartige Struktur ein neues Element in konstanter Schrittzahl einfügen, sobald man die Einfügestelle gefunden hat. Zum Finden der Einfügestelle benötigt man aber h Schritte, wenn sie den Abstand h vom Listenende hat. Was man brauchen könnte, ist eine Struktur, die beide Vorteile — schnelles Finden der Einfügestelle nahe dem Listenende und schnelles Einfügen — miteinander verbindet.

Strukturen zur Speicherung sortierter Folgen, die das Suchen und Einfügen eines neuen Elementes im Abstand h vom Ende der Folge in $O(\log h)$ Schritten erlauben, gibt es in der Tat. Geeignet gewählte Varianten balancierter, blattorientierter Suchbäume haben die verlangten angenehmen Eigenschaften, wenn man Suchen und Einfügen richtig

implementiert. Wir skizzieren hier grob die der Lösung zugrundeliegende Idee und verweisen auf das Kapitel über Bäume für die Details.

Die sortierte Folge wird in einer aufsteigend sortierten, verketteten Liste gespeichert. Ein üblicherweise *Finger* genannter Zeiger weist auf das Listenende mit dem jeweils größten Element. Über dieser Liste befindet sich ein (binärer) balancierter Suchbaum. Die Listenelemente sind zugleich die Blätter dieses Baumes. Der Suchbaum erlaubt nicht nur eine normale, bei der Wurzel beginnende Suche von oben nach unten. Man kann mit einer Suche auch an den Blättern bei der Position beginnen, auf die der Finger zeigt. Von dort läuft man das rechte Rückgrat des Baumes hinauf solange, bis man erstmals bei einem Knoten angekommen ist, der die Wurzel eines Teilbaumes mit der gesuchten Stelle ist. Von diesem Knoten aus wird wie üblich abwärts gesucht, bis man die gesuchte Stelle (unter den Blättern) gefunden hat. Fügt man jetzt ein neues Element in die Liste ein, muß man unter Umständen die darüberbefindliche Suchstruktur rebalancieren. Das kann zu einem erneuten Hinaufwandern von der Einfügestelle bis (schlimmstenfalls) zur Wurzel führen.

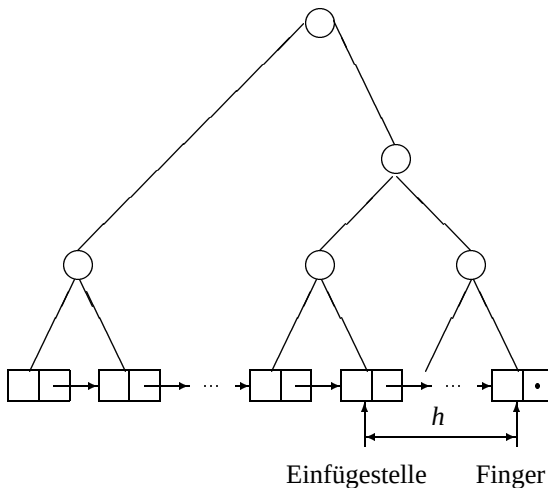


Abbildung 2.11

Falls man die richtigen „Wegweiser“ an den inneren Knoten des Suchbaumes postiert, kann somit die Suche nach einer h Elemente von der Position des Fingers am Ende entfernten Einfügestelle stets in $O(\log(h + 1))$ Schritten ausgeführt werden. Die Suche geht damit immer schnell. Zwar kann das Einfügen an der richtigen Stelle in der Liste in $O(1)$ Schritten ausgeführt werden; das anschließende Rebalancieren des Suchbaums kann aber $\Omega(\log N)$ Schritte im ungünstigsten Fall kosten.

Glücklicherweise tritt dieser ungünstige Fall für die meisten Typen balancierter Bäume nicht allzu häufig ein. Genauer gilt etwa für AVL-Bäume und 1-2-Bruder-Bäume: Die über eine Folge von N iterierten Einfügungen in den anfangs leeren Baum amor-

tisierten Rebalancierungskosten (ohne Suchkosten) sind im schlechtesten Fall $O(N)$. D.h. als Folge einer einzelnen Einfügeoperation kann zwar Zeit $\Omega(\log N)$ erforderlich sein, um den Baum zu rebalancieren, der mittlere Rebalancierungsaufwand pro Einfügeoperation, gemittelt über eine beliebige Folge von Einfügeoperationen in den anfangs leeren Baum, ist aber konstant.

Das Sortierverfahren verläuft daher so: Die N Elemente der gegebenen Folge $F = \langle k_1, \dots, k_N \rangle$ werden der Reihe nach in die anfangs leere Struktur der oben beschriebenen Art eingefügt. Wir nennen das Verfahren *A-sort* für adaptives Sortieren oder AVL-Sortieren (vgl. [2]).

Bezeichnet wieder h_j den Abstand des Folgeelementes k_j vom jeweiligen Listenden, also von der Fingerposition, so gilt für die Gesamtlaufzeit von A-sort offensichtlich:

$$T(F) = \underbrace{O(N)}_{\text{Umstrukturierungsaufwand}} + \underbrace{O\left(\sum_{j=1}^N \log(h_j + 1)\right)}_{\text{gesamter Suchaufwand}}$$

Um die Zeit $T(F)$ zum Sortieren einer Folge F mit der Anzahl der Inversionen von F in Verbindung bringen zu können, beachten wir, daß $\text{inv}(F) = \sum_{j=1}^N h_j$ gilt und:

$$\begin{aligned} \sum_{j=1}^N \log(h_j + 1) &= \log\left(\prod_{j=1}^N (h_j + 1)\right) \\ &= N \log\left(\prod_{j=1}^N (h_j + 1)^{\frac{1}{N}}\right) \\ &\leq N \log\left(\sum_{j=1}^N \frac{(h_j + 1)}{N}\right) \quad \{*\} \\ &= N \log\left(1 + \frac{\sum_{j=1}^N h_j}{N}\right) \\ &= N \log\left(1 + \frac{\text{inv}(F)}{N}\right) \end{aligned}$$

In $\{*\}$ wurde die Tatsache benutzt, daß das arithmetische Mittel nie kleiner sein kann als das geometrische Mittel der Größen $(h_j + 1)$, $j = 1, \dots, N$.

Damit folgt insgesamt, daß das Verfahren A-sort jede Folge F der Länge N in Zeit

$$T(F) = O\left(N + N \log\left(1 + \frac{\text{inv}(F)}{N}\right)\right)$$

sortiert. Die Laufzeit ist also linear, solange $\text{inv}(F) \in O(N)$ bleibt, und für die maximale Inversionszahl $N(N-1)/2$ wie zu erwarten $O(N \log N)$.

Ist das Verfahren *inv*-optimal? Nach der im Abschnitt 2.6.1 gegebenen Definition genügt es dazu zu zeigen, daß

$$\log(|\{F' \mid \text{inv}(F') \leq \text{inv}(F)\}|) \in \Omega\left(N \cdot \log\left(1 + \frac{\text{inv}(F)}{N}\right)\right)$$

ist. Man muß also zeigen, daß der Logarithmus der Anzahl der Permutationen einer Folge von N Elementen mit höchstens $I(N)$ Inversionen von der Größenordnung $\Omega(N \log(1 + \frac{I(N)}{N}))$ ist. Dies ist tatsächlich der Fall; wir verweisen dazu auf [1].

Wie verhält sich A-sort, wenn man ein anderes Maß für die Vorsortierung wählt? Wir zeigen, daß A-sort nicht *runs*-optimal ist, indem wir nachweisen:

- (a) Falls A-sort *runs*-optimal ist, muß A-sort alle Folgen mit nur zwei Runs in linearer Zeit sortieren.
- (b) Es gibt eine Folge mit nur zwei Runs, für die das Verfahren A-sort $\Omega(N \log N)$ Zeit benötigt.

Zum Beweis von (a) schätzen wir die Anzahl der Permutationen von N Zahlen mit höchstens zwei Runs ab. Offenbar kann man jede solche Permutation mit höchstens zwei Runs erzeugen, indem man eine der 2^N möglichen Teilmengen der N Zahlen nimmt, sie aufsteigend sortiert und daraus einen Run bildet. Der zweite Run besteht aus der aufsteigend sortierten Folge der übriggebliebenen Elemente. Also folgt, daß es höchstens $O(2^N)$ Permutationen von N Elementen mit nur zwei Runs geben kann. Nach der Definition in 2.6.1 muß ein *runs*-optimaler Algorithmus A zumindest jede Folge F mit höchstens zwei Runs sortieren in Zeit

$$\begin{aligned} T_A(F, \text{runs}) &\leq c \cdot (N + \log(|\{F' \mid \text{runs}(F') \leq 2\}|)) \\ &= c \cdot (N + \log 2^N) = O(N). \end{aligned}$$

Zum Nachweis von (b) betrachten wir die Folge

$$F = \left\langle \underbrace{\frac{N}{2} + 1, \frac{N}{2} + 2, \dots, N}_{\frac{N}{2}}, \underbrace{1, 2, \dots, \frac{N}{2}}_{\frac{N}{2}} \right\rangle$$

(Hier nehmen wir ohne Einschränkung an, daß N gerade ist.) Es ist $\text{runs}(F) = 2$, aber $\text{inv}(F) = \Theta(N^2)$. Genauer gilt für die Größen h_j , die die Laufzeit des Verfahrens A-sort bestimmen, in diesem Fall:

$$h_j = 0, \text{ für } 1 \leq j \leq \frac{N}{2}, \text{ und } h_j = \frac{N}{2}, \text{ für } \frac{N}{2} < j \leq N$$

Die Laufzeit von A-sort für diese Folge ist wenigstens gleich dem gesamten Suchaufwand und Umstrukturierungsaufwand, also:

$$\begin{aligned}
\sum_{j=1}^N c(1 + \log(h_j + 1)) &= cN + c \cdot \log \prod_{j=\frac{N}{2}+1}^N \left(1 + \frac{N}{2}\right) \\
&= cN + c \cdot \log \left(1 + \frac{N}{2}\right)^{\frac{N}{2}} \\
&= \Omega(N \log N)
\end{aligned}$$

□

Es ist intuitiv klar, warum A-sort für diese Folge F soviel Zeit benötigt. Alle Zahlen $1, 2, \dots, \frac{N}{2}$ müssen relativ weit von der festen Position des Fingers am rechten Ende der jeweiligen Liste eingefügt werden. Es wäre daher sicher besser, wenn man die Suche nach der jeweils nächsten Einfügestelle nicht immer wieder dort starten würde, sondern den Finger mitbewegen würde.

2.6.3 Sortieren durch lokales Einfügen und natürliches Verschmelzen

Wir haben bereits am Ende des vorigen Abschnitts gesehen, daß die Implementation des Sortierens durch Einfügen mit Hilfe dynamischer, sortierter Listen mit einem festgehaltenen Finger am rechten Ende nicht immer zu einem m -optimalen Verfahren führt. Es ist naheliegend, die Einfügestrategie wie folgt zu verändern: Man fügt die Elemente der gegebenen, zu sortierenden Folge der Reihe nach in die anfangs leere und stets aufsteigend sortierte Liste ein; zur Bestimmung der Einfügestelle für das jeweils nächste Element startet man eine Suche aber nicht jedesmal von derselben, festen Position am Listenende, sondern von der Position, an der das letzte Element eingefügt wurde. Man benötigt also eine Struktur mit einem *beweglichen Finger*. Der Finger zeigt stets auf die Position des jeweils zuletzt eingefügten Elementes. Das ist der Ausgangspunkt für die Suche nach der richtigen Einfügestelle für das jeweils nächste einzufügende Element.

Der Aufwand für die Suche nach der jeweils nächsten Einfügestelle hängt also entscheidend ab von der Distanz zweier nacheinander einzufügender Elemente der Ausgangsfolge. Für eine Folge $F = \langle k_1, \dots, k_n \rangle$ ist die Distanz d_j des Elementes k_j vom vorangehenden offenbar:

$$d_j = \left| \{i \mid 1 \leq i < j \text{ und } (k_{j-1} < k_i < k_j \text{ oder } k_j < k_i < k_{j-1})\} \right|$$

Beispiel: Für die Folge $F = \langle 5, 1, 7, 4, 9, 2, 8, 3, 6 \rangle$ (F_C aus 2.6.1) hat das siebente Element $k_7 = 8$ die Distanz $d_7 = 3$ vom vorangehenden Element $k_6 = 2$, da links von k_7 drei Elemente der Folge F stehen, deren Platz bei aufsteigender Sortierung zwischen k_6 und k_7 ist.

Für gut vorsortierte Folgen wird man erwarten, daß die Distanzen d_j klein sind. Ein Vorsortierung berücksichtigendes Sortierverfahren kann sich das zunutze machen, wenn es der allgemeinen Strategie des Sortierens durch Einfügen folgt und dies Verfahren durch eine Struktur mit einem beweglichen Finger implementiert ist, der auf das jeweils zuletzt eingefügte Element zeigt.

Man möchte natürlich erreichen, daß die Suche nach der nächsten Einfügestelle für ein Element mit Distanz d von der Finger-Position möglichst in $O(\log d)$ Schritten ausführbar ist. Das kann eine zu der im Abschnitt 2.6.2 beschriebenen analoge Hybridstruktur leisten, die aus einer dynamischen, verketteten sortierten Liste mit darübergestülptem Suchbaum besteht, wenn der Suchpfad immer wie in Abbildung 2.12 aussieht.

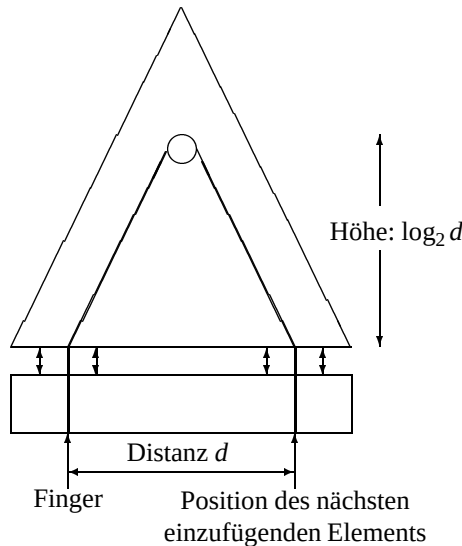


Abbildung 2.12

Leider kann es aber vorkommen, daß zwei ganz nah benachbarte Blätter eines (binären) Suchbaumes nur durch einen Pfad Länge $\Omega(\log N)$, wobei N die gesamte Anzahl aller Blätter ist, miteinander verbunden sind, wie Abbildung 2.13 zeigt. Man wird also, anders als im Fall eines festen Fingers am rechten Ende, nicht immer erwarten können, daß man zur Bestimmung einer Einfügestelle mit Distanz d nur $O(\log d)$ Niveaus im Suchbaum hinauf- und auch wieder hinabsteigen muß, wenn man nicht zusätzliche Verbindungen der Knoten untereinander vorsieht. Mit einer niveauweisen Verkettung benachbarter Knoten im Baum kann man das Problem allerdings lösen. Es gibt in der Literatur zahlreiche Vorschläge für solche Strukturen.

Wir verweisen auf das Kapitel über Bäume und auf [1] und begnügen uns mit der (hoffentlich plausiblen) Feststellung, daß es eine Struktur mit einem beweglichen Fin-

ger gibt, die zum *Sortieren durch lokales Einfügen* wie folgt verwendet werden kann: Die zu sortierenden Elemente werden der Reihe nach in die anfangs leere Struktur eingefügt. Die bereits eingefügten Elemente sind aufsteigend sortiert und miteinander verkettet. Ein Finger zeigt auf das jeweils zuletzt eingefügte Element. Die Stelle, an der das jeweils nächste Element mit Distanz d in die verkettete, aufsteigend sortierte Liste der bereits betrachteten Elemente eingefügt werden muß, kann in Zeit $O(\log d)$ gefunden werden. Die zur Rebalancierung der Suchstruktur nach einer Einfügung erforderliche Schrittzahl ist im Durchschnitt (genommen über eine Folge von Einfügungen in die anfangs leere Struktur) konstant. Damit folgt, daß das Verfahren Sortieren durch lokales Einfügen eine Folge $F = \langle k_1, \dots, k_N \rangle$ mit Distanzen $d_j, 1 \leq j \leq N$, stets in Zeit

$$T(F) = O(N) + O\left(\sum_{j=1}^N \log(1 + d_j)\right)$$

Schritten zu sortieren erlaubt.

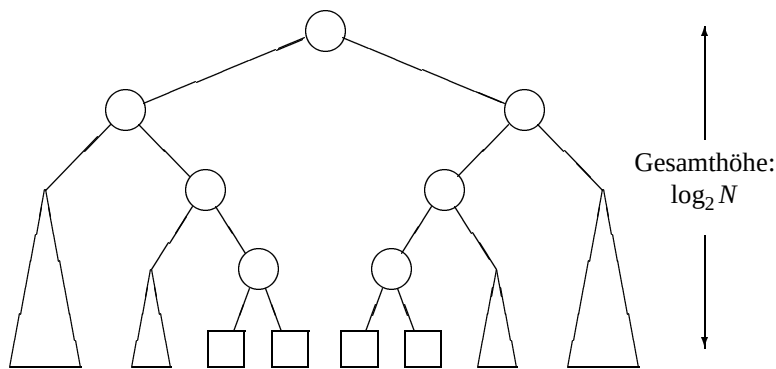


Abbildung 2.13

Es ist intuitiv sofort plausibel, daß Sortieren durch lokales Einfügen für Folgen mit kleiner Inversionszahl eher besser sein muß als das im Abschnitt 2.6.2 vorgestellte Verfahren A-sort. Denn für jede Folge $F = \langle k_1, \dots, k_N \rangle$ gilt: Die Distanz d_j eines Elementes vom vorangehenden kann nicht größer sein als das Maximum der Abstände von k_j und k_{j-1} vom rechten Ende. Genauer: Die Anzahl der links von k_j in F stehenden Elemente, die in der sortierten Folge zwischen k_{j-1} und k_j gehören, ist kleiner als die größere der Anzahlen h_j und h_{j-1} von Elementen, die links von k_j bzw. links von k_{j-1} in F stehen und größer als k_j bzw. k_{j-1} sind. Daher folgt:

$$d_j + 1 \leq \max(h_{j-1}, h_j) + 1 \leq (h_j + 1)(h_{j-1} + 1)$$

und

$$\begin{aligned} \sum_{j=1}^N \log(1 + d_j) &\leq \sum_{j=1}^N \log((h_j + 1)(h_{j-1} + 1)) \\ &\leq 2 \sum_{j=1}^N \log(h_j + 1). \end{aligned}$$

Es ergibt sich also wie im Falle von A-sort

$$T(F) = O\left(N + N \log\left(1 + \frac{\text{inv}(F)}{N}\right)\right)$$

als Abschätzung für die Laufzeit des Sortierens durch lokales Einfügen. Das Verfahren Sortieren durch lokales Einfügen ist damit ebenfalls *inv*-optimal.

H. Mannila [10] hat gezeigt, daß Sortieren durch lokales Einfügen auch *runs*-optimal und *rem*-optimal ist. In beiden Fällen verläuft der Beweis analog zur Argumentation im Falle der *inv*-Optimalität in zwei Schritten. Zuerst wird die Laufzeit des Verfahrens für alle Folgen mit gegebenem Vorsortierungsgrad, gemessen mit den Maßen *runs* und *rem*, abgeschätzt (Worst-case-Zeitschranke). Dann werden untere Schranken für die Mindestanzahl von Permutationen von N Elementen und gegebenem Vorsortierungsgrad angegeben. Genauer wird gezeigt:

Satz 2.2 Jede Folge F von N Schlüsseln kann nach dem Verfahren Sortieren durch lokales Einfügen sortiert werden in

- (a) $O(N(1 + \log(\text{runs}(F))))$
und
(b) $O\left(N + \sum_{j=0}^{\text{rem}(F)-1} \log(N - j)\right)$ Schritten.

Satz 2.3

- (a) Es gibt Konstanten c und d derart, daß gilt:

$$\log(|\{F \mid \text{runs}(F) \leq t\}|) \geq c \cdot N \cdot \log t - d \cdot t;$$

- (b) $\log(|\{F \mid \text{rem}(F) \leq s\}|) \geq \sum_{j=0}^{s-1} \log(N - j).$

Für die nicht einfachen Beweise von Satz 2.2 (a), (b) und Satz 2.3 (a) verweisen wir auf die Arbeit von H. Mannila [10]. Wir begnügen uns damit, die einfach nachzuweisende untere Schranke in Satz 2.3 (b) herzuleiten.

Wieviele Permutationen F von N Schlüsseln mit $\text{rem}(F) \leq s$, also mit einer wenigstens $N - s$ langen aufsteigenden Teilfolge, gibt es mindestens? Wir können Folgen mit $\text{rem}(F) \leq s$ bilden, indem wir zunächst $N - s$ Elemente aus den N Elementen auswählen, diese Elemente in aufsteigende Reihenfolge bringen und die verbleibenden s Elemente in beliebiger Reihenfolge dahinterschreiben. Also gibt es wenigstens

$$\binom{N}{N-s} s! = \frac{N!}{(N-s)!} = \prod_{j=0}^{s-1} (N - j)$$

derartige Folgen. Also ist

$$\log(|\{F \mid \text{rem}(F) \leq s\}|) \geq \log \prod_{j=0}^{s-1} (N-j) = \sum_{j=0}^{s-1} \log(N-j)$$

und Satz 2.3 (b) ist bewiesen. Aus den beiden Sätzen folgt sofort, daß Sortieren durch lokales Einfügen *runs*-optimal und *rem*-optimal im Sinne der Definition aus Abschnitt 2.6.1 ist.

Daß das geringe Distanzen zwischen aufeinanderfolgenden Folgeelementen ausnutzende Verfahren Sortieren durch lokales Einfügen *runs*-optimal ist, ist durchaus überraschend (und nicht leicht zu zeigen). Es gibt ein Verfahren, für das man wesentlich eher vermuten würde, daß es *runs*-optimal ist. Das ist das *Sortieren durch natürliches Verschmelzen* (vgl. Abschnitt 2.4.3). Es sortiert eine Folge F nach folgender Methode: Man verschmilzt je zwei der ursprünglich in F vorhandenen „natürlichen“ Runs zu je einem neuen Run. Das ergibt eine Folge mit nur noch etwa halb so vielen Runs wie in der ursprünglich gegebenen Folge. Auf die neue Folge wendet man dasselbe Verfahren an usw., bis man schließlich eine nur noch aus einem einzigen Run bestehende und damit sortierte Folge erhalten hat.

Betrachten wir als Beispiel die Folge $F = F_c$ aus Abschnitt 2.6.1:

$$5 \mid 1 \ 7 \mid 4 \ 9 \mid 2 \ 8 \mid 3 \ 6 \mid$$

Die Folge hat fünf „natürliche“ Runs, die wir durch vertikale Striche voneinander getrennt haben. Verschmelzen des ersten und zweiten sowie des dritten und vierten Runs ergibt:

$$1 \ 5 \ 7 \mid 2 \ 4 \ 8 \ 9 \mid 3 \ 6$$

Verschmelzen der ersten beiden Runs ergibt:

$$1 \ 2 \ 4 \ 5 \ 7 \ 8 \ 9 \mid 3 \ 6$$

Abermaliges Verschmelzen der verbliebenen zwei Runs liefert die aufsteigend sortierte Folge.

Bezeichnet man das Herstellen einer neuen Folge durch paarweises Verschmelzen je zweier benachbarter Runs als einen *Durchgang*, so ist klar, daß jeder Durchgang in linearer Zeit ausführbar ist und höchstens $O(\log t)$ Durchgänge zum Sortieren ausgeführt werden müssen, wenn t die Anzahl der ursprünglich vorhandenen Runs ist. Damit erhält man als Laufzeit des Verfahrens für beliebige Folgen der Länge N unmittelbar

$$T(F) = O(N(1 + \log(\text{runs}(F)))).$$

Zusammen mit Satz 2.3 (a) folgt, daß Sortieren durch natürliches Verschmelzen *runs*-optimal ist.

Abschließend noch eine Bemerkung zum Speicherbedarf aller in diesem Abschnitt besprochenen Sortierv Verfahren. A-sort, Sortieren durch lokales Einfügen und natürliches Verschmelzen sortieren nicht „am Ort“. Sie benötigen $\Theta(N)$ zusätzlichen Speicherplatz zum Sortieren von Folgen der Länge N . Im Falle der grob skizzierten Hybridstrukturen kann der in $\Theta(N)$ versteckte Faktor beträchtlich sein.

2.7 Externes Sortieren

In den bisherigen Abschnitten über das Sortieren sind wir stets davon ausgegangen, daß die zu sortierenden Daten und alle Zwischenergebnisse im Hauptspeicher des verwendeten Rechners Platz finden. Programmtechnisch haben wir diese Situation durch die Datenstruktur des Arrays modelliert. Der Zugriff auf einen Datensatz hat stets konstante Zeit beansprucht, unabhängig davon, um welchen Datensatz es ging. Entsprechend haben wir unsere Sortieralgorithmen nicht darauf abgestellt, die Datensätze in einer gewissen Reihenfolge zu betrachten.

Die Situation ist beim Sortieren von Datensätzen auf Externspeichern (Sekundärspeichern) grundlegend anders. Gängige Externspeicher heutiger Rechner sind vor allem Magnetplatten, aber auch Magnetbänder und Magnettrommeln. Wir wollen in diesem Abschnitt lediglich das Sortieren mit Magnetbändern betrachten, weil hier die Restriktionen am stärksten sind. Die vorgestellten Verfahren können auch für Platten oder Trommeln benutzt werden, obwohl sie die dort verfügbaren Operationen nicht voll ausschöpfen. Im wesentlichen kann man die Datensätze auf ein Band sequentiell schreiben oder von dort sequentiell lesen. Der Zugriff auf einen Datensatz nahe am Bandende ist, wenn man gerade auf einen Datensatz am Bandanfang zugegriffen hat, sehr aufwendig. Es muß auf sämtliche Datensätze zwischen dem aktuellen und dem gewünschten ebenfalls zugegriffen werden. Dieses Modell präzisieren wir im Abschnitt 2.7.1.

Man wird versuchen, Sortieralgorithmen an diese Situation anzupassen, indem man Datensätze in derjenigen Reihenfolge bearbeitet, in der sie schon auf dem Magnetband stehen. Man kennt heute viele verschiedene Verfahren, die dieser Idee folgen. Weil in der Anfangszeit der elektronischen Rechner Internspeicher noch knapper und teurer war als heute, hat man sich bereits sehr früh intensiv mit Methoden des externen Sortierens beschäftigt. D. Knuth  widmet diesem Thema weit über hundert Seiten und berichtet über erste externe Sortierverfahren aus dem Jahr 1945 (von J.P. Eckert und J.W. Mauchly), unter anderem über das *ausgeglichene 2-Wege-Mergesort* für Magnetbänder, das wir in Abschnitt 2.7.2 vorstellen. Das *ausgeglichene Mehr-Wege-Mergesort*, das es erlaubt, mehrere Bänder in den Sortierprozeß einzubeziehen, wird im Abschnitt 2.7.3 erläutert. Schließlich präsentieren wir im Abschnitt 2.7.4 das *Mehrphasen-Mergesort*. All diese Verfahren sind Varianten des Sortierens durch Verschmelzen. Grundsätzlich eignen sich auch andere Methoden, wie etwa Radixsort oder Quicksort, zum externen Sortieren (vgl.  und ); wir wollen uns hier aber auf die gebräuchlicheren Mergesort-Varianten beschränken.

2.7.1 Das Magnetband als Externspeichermedium

Das Speichermedium Magnetband ähnelt in vieler Hinsicht den Magnetbändern in Audio-Cassetten; an viele Mikrorechner kann man ja heute einfache Cassettenrecorder als Externspeichergeräte anschließen. Datensätze werden auf einem Magnetband streng sequentiell gespeichert. Mit einem Magnetband können folgende Operationen ausgeführt werden:

Zurückspulen und Lese- oder Schreibzustand wählen: Das Band wird an den Anfang zurückgespult. Beim sequentiellen Bearbeiten des Bandes können entweder nur Einträge vom Band gelesen oder nur Einträge auf das Band geschrieben werden. Daher wird, zusammen mit dem Zurückspulen, entweder der Lese- oder der Schreibzustand gewählt. (Nach einer weiteren Rückspuloperation kann dann für dasselbe Band ein anderer Zustand gewählt werden.) Wir wählen folgende Bezeichnungen für diese Operationen:

reset(t) : Rückspulen des Bandes t (englisch: tape) mit Wahl des Lesezustands;

rewrite(t) : Rückspulen des Bandes t mit Wahl des Schreibzustandes.

Lesen oder Schreiben: Das Lesen ist das Übertragen des nächsten Datensatzes vom Band in den Internspeicher des Rechners. Das Band muß sich im Lesezustand befinden. Entsprechend ist das Schreiben das Übertragen eines Datensatzes vom Internspeicher an die nächste Stelle auf dem Band, wobei das Band im Schreibzustand sein muß. Der Internspeicherbereich, in den gelesen bzw. von dem geschrieben wird, wird ebenfalls spezifiziert, und zwar einfach durch Angabe einer Variablen für den Datensatz:

read(t, d) : Lesen des nächsten Datensatzes vom Band t und Zuweisen an die Variable d ;

write(t, d) : Schreiben des Werts der Variablen d als nächsten Datensatz auf Band t .

Magnetbänder haben in der Realität nur eine endliche Länge; wir wollen hier von dieser Restriktion abstrahieren und Bänder als einseitig unendliche Datenspeicher ansehen. Wir müssen also nicht befürchten, beim Lesen oder Beschreiben eines Bandes das Bandende zu erreichen. Beim Lesen werden wir aber sehr wohl das Ende der Datensätze erreichen, die dort auf dem Band gespeichert sind. Eine Funktion liefert uns die nötige Information.

Feststellen, ob das Ende der Datensätze erreicht ist: Diese Funktion verwenden wir nur im Lesezustand. Sobald das Ende erreicht ist, sind alle Datensätze gelesen worden. Das weitere Lesen eines Datensatzes ist dann nicht mehr sinnvoll. Wir bezeichnen diese Funktion mit *eof* (englisch: *end of file*):

eof(t) : liefert den Wert *true* genau dann, wenn das Datenende für Band t erreicht ist, und sonst den Wert *false*.

Wir haben die Namen für Operationen und Funktionen bewußt gewählt wie die Datei-Bearbeitungs-Prozeduren und -Funktionen in Pascal, weil das Datei-Konzept in Pascal gerade Magnetbänder modelliert. Moderne Magnetbänder erlauben über die genannten Operationen hinaus etwa das Rückwärts-Lesen, das Rückwärts-Schreiben, Lesen mit Schreiben kombiniert und andere mehr, die natürlich von passenden Sortierverfahren durchaus genutzt werden können. Wir lassen dieses aber hier unberücksichtigt, damit der Grundgedanke externen Sortierens möglichst deutlich zutage tritt.

Typischerweise sind Ein-/Ausgabe-Operationen (Externzugriffe) mit Sekundärspeichern erheblich langsamer als interne Operationen, sei es das Umspeichern von Daten im Hauptspeicher oder das Anstellen von Berechnungen. Schnelle externe Sortierverfahren müssen daher in erster Linie bemüht sein, die Anzahl der Externzugriffe so gering wie möglich zu halten. Das Komplexitätsmaß, das wir für externe Sortierverfahren mit Bändern verwenden, basiert daher auf der Anzahl der Externzugriffe. Weil bei Bändern Externzugriffe nur rein sequentiell möglich sind, zählen wir, wie oft ein ganzes Band gelesen oder geschrieben wurde. Die Kosten für das Lesen oder Schreiben eines Bandes sind proportional zur Anzahl der Datensätze auf diesem Band. Sind alle

Datensätze auf mehrere Bänder verteilt, so ergeben sich beim Lesen all dieser Bänder Gesamtkosten, die proportional zur Anzahl der Datensätze insgesamt sind. Das Lesen/Schreiben aller Datensätze nennen wir einen *Durchgang* (englisch: *pass*). Als Komplexitätsmaß wählen wir die Anzahl der Durchgänge, die zum Sortieren von N Datensätzen benötigt werden; wir bezeichnen die minimale, mittlere und maximale Anzahl mit $P_{\min}(N)$, $P_{\text{mit}}(N)$ und $P_{\max}(N)$.

2.7.2 Ausgeglichenes 2-Wege-Mergesort

Die zu sortierenden N Datensätze stehen auf einem Magnetband, dem Eingabe-Band t_1 . Die Anzahl N der Datensätze ist so groß, daß nicht alle Datensätze gleichzeitig im Hauptspeicher Platz finden. Den externen Mergesort-Varianten liegt nun folgende naheliegende Idee zugrunde. Man teilt die N Datensätze gedanklich in $\lceil \frac{N}{I} \rceil$ Teilfolgen von jeweils höchstens I Datensätzen, wobei I die Anzahl der Datensätze ist, die höchstens im Internspeicher Platz finden. Dann betrachtet man der Reihe nach jede dieser Teilfolgen separat. Man liest die Teilfolge ganz in den Internspeicher ein, sortiert sie mit einem der bereits bekannten Verfahren und schreibt die sortierte Teilfolge auf den Externspeicher.

Schließlich verschmilzt man die sortierten Teilfolgen. Das Verschmelzen kann effizient mit Hilfe des Externspeichers geschehen, weil sowohl die zu verschmelzenden Teilfolgen als auch die entstehende Resultatfolge rein sequentiell gelesen bzw. geschrieben werden. Die zu verschmelzenden Teilfolgen müssen dazu auf verschiedenen Bändern stehen. Anfangs muß man also die Datensätze auf dem Eingabeband auf mehrere Bänder aufteilen; danach verschmilzt man die sortierten Teilfolgen. Die entstandene Folge muß wieder aufgeteilt werden usw., bis man schließlich eine vollständig sortierte Folge erzeugt hat. Dieser Wechsel zwischen *Aufteilungs-* und *Verschmelzungsphase* ist charakteristisch für externe Mergesort-Verfahren. Wohl das einfachste solche Verfahren ist das *ausgeglichene 2-Wege-Mergesort* (*balanced 2-way-mergesort*), das wir jetzt vorstellen.

Methode: Wir verwenden vier Bänder, t_1, t_2, t_3, t_4 . Das Eingabeband ist t_1 . Es werden wiederholt I Datensätze von t_1 gelesen, intern sortiert, und abwechselnd solange auf t_3 und t_4 geschrieben, bis t_1 erschöpft ist. Dann stehen also $\lceil \frac{N}{(2 \cdot I)} \rceil$ sortierte Teilfolgen (Runs) der Länge I auf t_3 , und $\lfloor \frac{N}{(2 \cdot I)} \rfloor$ Runs der Länge I auf t_4 . Jetzt werden die Runs von t_3 und t_4 verschmolzen. Dabei entstehen Runs der Länge $2 \cdot I$. Diese werden abwechselnd auf t_1 und t_2 verteilt. Dann stehen also etwa $\frac{N}{(4 \cdot I)}$ Runs der Länge $2 \cdot I$ auf jedem der Bänder t_1, t_2 . Nach jeder Aufteilungs- und Verschmelzungsphase hat sich die Run-Länge verdoppelt und die Anzahl der Runs etwa halbiert. Wir fahren fort, die Runs von zweien der Bänder zu verschmelzen und abwechselnd auf die anderen beiden zu verteilen, dann die Bänder (logisch) zu vertauschen, bis schließlich nur noch ein Run auf einem der Bänder übrig bleibt.

Das Verschmelzen zweier externer Runs kann mit Hilfe zweier Variablen für die beiden Bänder geschehen, also mit ganz wenig internem Speicherplatz. Beim internen Sortieren zu Beginn wird der gesamte interne Speicherplatz genutzt, um möglichst lange Runs zu erzielen. Betrachten wir ein Beispiel.

Beispiel: Der Hauptspeicher des Rechners fasse drei Datensätze ($I = 3$), und die Folge der zu sortierenden Schlüssel sei

$$F = 12, 5, 2, 15, 13, 6, 14, 1, 4, 9, 10, 3, 11, 7, 8.$$

Anfangs stehen die Datensätze, hier repräsentiert durch ihre Schlüssel, auf Band t_1 ; alle anderen Bänder sind leer:

t_1 : 12, 5, 2, 15, 13, 6, 14, 1, 4, 9, 10, 3, 11, 7, 8.

t_2 :

t_3 :

t_4 :

Zunächst werden jeweils I Datensätze von t_1 gelesen, sortiert, und abwechselnd auf t_3 und t_4 aufgeteilt. Nach den ersten drei gelesenen Datensätzen ergibt sich folgendes Bild:

t_1 : 12, 5, 2, 15, 13, 6, 14, 1, 4, 9, 10, 3, 11, 7, 8.

t_2 : __

t_3 : 2, 5, 12 __

t_4 : __

Die Stelle des nächsten Datensatzes jedes Bandes ist unterstrichen. Nachdem alle Datensätze auf t_3 und t_4 verteilt sind, haben wir folgende Situation (Runs sind durch ; getrennt):

t_1 : 12, 5, 2, 15, 13, 6, 14, 1, 4, 9, 10, 3, 11, 7, 8. __

t_2 : __

t_3 : 2, 5, 12; 1, 4, 14; 7, 8, 11. __

t_4 : 6, 13, 15; 3, 9, 10. __

Alle Bänder werden zurückgespult, t_3 und t_4 werden gelesen, t_1 und t_2 beschrieben. Der ursprüngliche Inhalt von t_1 ist damit verloren. Wir zeigen die Situationen, die sich jeweils nach einer Verschmelzungs- und Verteilungsphase ergeben:

t_1 : 2, 5, 6, 12, 13, 15; 7, 8, 11. __

t_2 : 1, 3, 4, 9, 10, 14. __

t_3 : __

t_4 : __

t_1 : __

t_2 : __

t_3 : 1, 2, 3, 4, 5, 6, 9, 10, 12, 13, 14, 15. __

t_4 : 7, 8, 11. __

t_1 : 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15. __

t_2 : __

t_3 : __

t_4 : __

Daß die sortierte Folge auf Band t_1 entstanden ist, ist hierbei Zufall. Im allgemeinen kann sie auf Band t_1 oder t_3 entstehen, wenn von den beiden Bändern t_1 und t_2 (bzw. t_3 und t_4) entstehende Runs zuerst auf t_1 und danach auf t_2 (bzw. zuerst auf t_3 und danach auf t_4) geschrieben werden.

Zur programmtechnischen Realisierung dieses Verfahrens setzen wir die folgenden Definitionen voraus:

```
const
     $k = 4$ ; {Anzahl der Bänder}
type
    tape = file of item;
    tapes = array [1 ..  $k$ ] of tape
```

Dann läßt sich die Sortierprozedur wie folgt angeben:

```
procedure balanced_2_way_mergesort (var t : tapes; var i : integer);
    {sortiert die Datensätze von Band t[1] auf eines der Bänder t[i],
     und liefert Band-Nummer i zurück}
var
    ein1, ein2, aus1, aus2, aus: integer;
begin
    anfangsverteilung({von:} t[1], {nach:} t[3], t[4]);
    {Runs der Länge 1 sind auf t[3] und t[4] verteilt}
    {wähle 2 Ein- und 2 Ausgabebänder:}
    ein1 := 3;
    ein2 := 4;
    aus1 := 1;
    aus2 := 2;
    {falls ein2 leer ist, steht die sortierte Folge auf ein1}
    reset(t[ein2]);
    while not eof(t[ein2]) do
        begin
            aus := aus2; {zuletzt benutztes Ausgabeband}
            reset(t[ein1]);
            rewrite(t[aus1]);
            rewrite(t[aus2]);
            while not eof(t[ein2]) do
                begin {ein2 wird zuerst erschöpft}
                    naechstes(aus); {welches Ausgabeband}
                    mergeruns(t[ein1], t[ein2], t[aus])
                    {verschmilz je einen Run aus Band ein1
                     und Band ein2 und schreibe ihn auf Band aus}
                end;
            copyrest(t[ein1], t[aus]); {falls noch ein Run auf ein1}
            {ein1 und ein2 sind erschöpft; wechse Ein-/Ausgabebänder}
```

```

    tausche(aus1, ein1);
    tausche(aus2, ein2);
    reset(t[ein2])
  end;
  {jetzt steht die sortierte Folge auf t[ein1]}
  i := ein1
end

```

Wir geben noch kurz an, was die bisher nicht erklärten Prozeduren leisten, ohne sie aber im Detail auszufüllen; das überlassen wir dem interessierten Leser.

procedure *anfangsverteilung* (**var** t_1, t_3, t_4 : *tape*);
 {sortiert Teilfolgen der Länge I , die aus t_1 stammen, und
 speichert sie abwechselnd nach t_3 und t_4 , beginnend mit t_3 ;
 dort werden also Runs der Länge I gespeichert}

procedure *naechstes* (**var** aus : *integer*);
 {wechselt das Ausgabeband, also $1 \rightarrow 2, 2 \rightarrow 1, 3 \rightarrow 4, 4 \rightarrow 3$ }

procedure *mergeruns* (**var** t_1, t_2, t : *tape*);
 {verschmilzt je einen Run aus t_1 und t_2 und schreibt
 den doppelt langen verschmolzenen Run auf t ;
 beim Verschmelzen werden gleichzeitig lediglich zwei
 Datensätze intern gespeichert, je einer aus t_1 und t_2 }

procedure *copyrest* (**var** t_1, t : *tape*);
 {überträgt den Rest der Datensätze von t_1 nach t }

procedure *tausche* (**var** i, j : *integer*);
 {vertauscht die Werte von i und j }

Analyse: Nach jeder Verschmelzungs- und Verteilungsphase hat sich die Anzahl der Runs (etwa) halbiert. In der Anfangsverteilung haben wir aus N Datensätzen unter Zuhilfenahme des Internspeichers der Größe I in einem Durchgang $\lceil \frac{N}{I} \rceil$ Runs hergestellt. Damit ergibt sich nach $\lceil \log(\frac{N}{I}) \rceil$ Durchgängen ein einziger Run; also ist

$$P_{\min}(N) = P_{\text{mit}}(N) = P_{\max}(N) = \left\lceil \log \left(\frac{N}{I} \right) \right\rceil$$

bei vier Bändern und Internspeichergroße I .

2.7.3 Ausgeglichenes Mehr-Wege-Mergesort

Das im Abschnitt 2.7.2 beschriebene Verfahren des 2-Wege-Mergesort läßt sich leicht auf ein Mehr-Wege-Verschmelzen verallgemeinern.

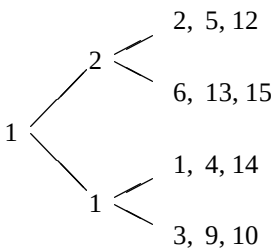
Methode: Wir verwenden $2k$ Bänder, $t_1, \dots, t_{2k}$. Das Eingabeband ist t_1 . Es werden wiederholt I Datensätze von t_1 gelesen, intern sortiert und abwechselnd auf $t_{k+1}, t_{k+2}, \dots, t_{2k}$ geschrieben solange, bis t_1 erschöpft ist. Dann stehen etwa $\frac{N}{(k \cdot I)}$ Runs der Länge I auf t_i , $k+1 \leq i \leq 2k$. Die k Bänder $t_{k+1}, \dots, t_{2k}$ sind jetzt die Eingabebänder für ein k -Wege-Verschmelzen, die k Bänder $t_1, \dots, t_k$ sind die Ausgabebänder. Nun werden die ersten Runs der Eingabebänder zu einem Run der Länge $k \cdot I$ verschmolzen und auf das Ausgabeband t_1 geschrieben. Dann werden die nächsten k Runs der Eingabebänder verschmolzen und nach t_2 geschrieben. So werden der Reihe nach Runs der Länge $k \cdot I$ auf die Ausgabebänder geschrieben, bis die Eingabebänder erschöpft sind. Nach dieser Verschmelzungs- und Aufteilungsphase tauschen die Eingabe- und Ausgabebänder ihre Rollen. Das k -Wege-Verschmelzen und k -Wege-Aufteilen wird solange fortgesetzt, bis die gesamte Folge der Datensätze als ein Run auf einem der Bänder steht.

Beim k -Wege-Verschmelzen von Runs auf k Bändern wird zunächst der erste Datensatz jedes Runs in den Internspeicher gelesen. Von den k intern gespeicherten Datensätzen wird derjenige mit kleinstem Schlüssel für die Ausgabe ausgewählt und auf das Ausgabeband geschrieben. Der ausgewählte Datensatz wird dann im Internspeicher ersetzt durch den nächsten Datensatz im zugehörigen Eingabe-Run; dieser wird vom entsprechenden Eingabeband gelesen. Dies wird solange fortgesetzt, bis alle k Eingabe-Runs erschöpft sind.

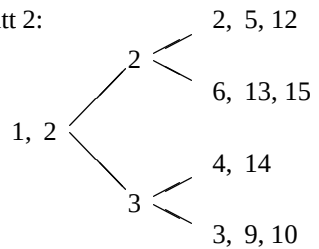
Damit das Verschmelzen von k Datensätzen im Internspeicher geschehen kann, muß $k \leq I$ gelten. Für kleine Werte von k mag es vernünftig sein, unter den gerade betrachteten k Datensätzen denjenigen mit minimalem Schlüssel durch lineares Durchsehen aller k Schlüssel zu bestimmen; für größere Werte von k ist es vorteilhaft, das k -Wege-Verschmelzen mittels einer Halde (Heap) oder eines *Auswahlbaumes* (selection tree) zu unterstützen. In einer solchen Datenstruktur kostet das Entfernen des Minimums und Hinzufügen eines neuen Schlüssels nur $O(\log k)$ Schritte (interne Vergleichs- und Rechenoperationen), wenn k Schlüssel gespeichert sind (vgl. Abschnitt 2.3 und Kapitel 6).

Betrachten wir als Beispiel das 4-Wege-Verschmelzen mit einem 2-stufigen Auswahlbaum:

Schritt 1:

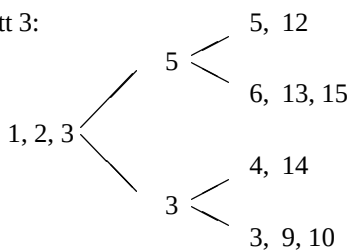


Schritt 2:

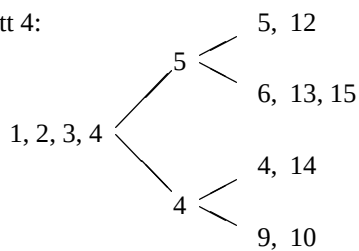


Ausgabeband
Internspeicher
Eingabebänder

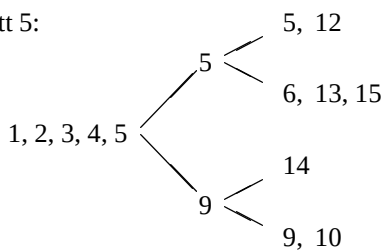
Schritt 3:



Schritt 4:

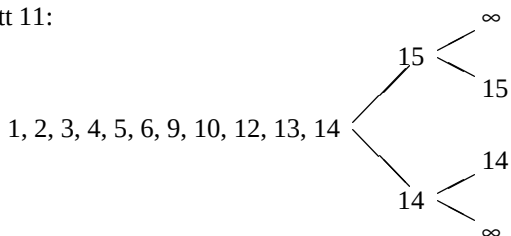


Schritt 5:

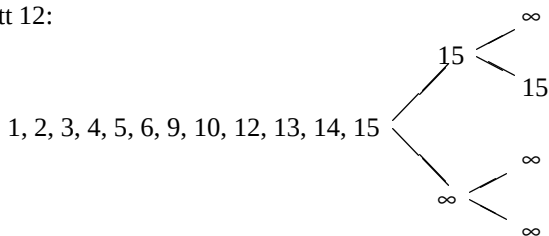


⋮

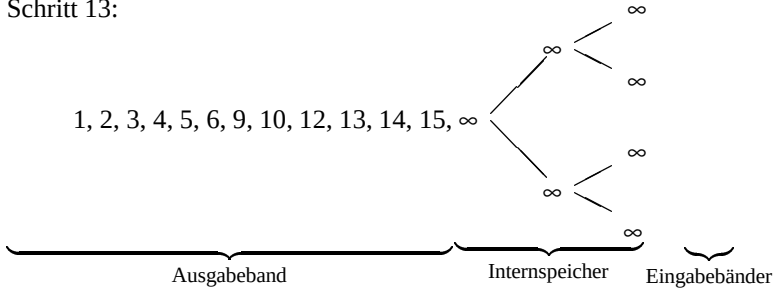
Schritt 11:



Schritt 12:



Schritt 13:



Diese Methode, genannt *Auswahl und Ersetzen (replacement selection)*, läßt sich vorteilhaft auch schon für die Anfangsverteilung verwenden, indem man die Eingabefolge mit sich selbst *k*-Wege-verschmilzt.

Auswahl und Ersetzen für eine unsortierte Eingabefolge: Die Datensätze stehen unsortiert auf dem Eingabeband. Der Internspeicher faßt *I* Datensätze. Zunächst werden die ersten *I* Datensätze vom Eingabeband gelesen. Von den *I* intern gespeicherten Datensätzen wird derjenige mit kleinstem Schlüssel für die Ausgabe ausgewählt und auf das Ausgabeband geschrieben. Der ausgewählte Datensatz wird im Internspeicher ersetzt durch den nächsten Datensatz auf dem Eingabeband. Ist der neue Schlüssel nicht kleiner als der des soeben ausgegebenen Datensatzes, dann wird der neue Datensatz noch zum gleichen Run gehören wie der soeben ausgegebene. Ist er jedoch kleiner, so gehört er zum nächsten Run; er wird erst ausgegeben, wenn der aktuelle Run beendet ist. Als nächster wird der kleinste Schlüssel ausgewählt, der nicht kleiner ist als der soeben ausgegebene; er kann noch zum aktuellen Run hinzugefügt werden. Dies wird solange wiederholt, bis alle Schlüssel im Internspeicher kleiner sind als der zuletzt ausgegebene; dann muß ein neuer Run angefangen werden.

Beispiel: Betrachten wir die Folge $F = 12, 5, 2, 15, 13, 6, 14, 1, 4, 9, 10, 3, 11, 7, 8$ aus Abschnitt 2.7.2, für $I = 3$:

Anfangs: $\underbrace{\hspace{1cm}}$ | $\underbrace{12, 5, 2}_{\text{Internspeicher}}$ | $\underbrace{15, 13, 6, 14, 1, 4, 9, 10, 3, 11, 7, 8}_{\text{Eingabeband}}$

Schritt 1: 2 | 12, 5, 15 | 13, 6, 14, 1, 4, 9, 10, 3, 11, 7, 8
 Schritt 2: 2, 5 | 12, 15, 13 | 6, 14, 1, 4, 9, 10, 3, 11, 7, 8
 Schritt 3: 2, 5, 12 | 15, 13, 6 | 14, 1, 4, 9, 10, 3, 11, 7, 8
 Schritt 4: 2, 5, 12, 13 | 15, 6, 14 | 1, 4, 9, 10, 3, 11, 7, 8
 Schritt 5: 2, 5, 12, 13, 14 | 15, 6, 1 | 4, 9, 10, 3, 11, 7, 8
 Schritt 6: 2, 5, 12, 13, 14, 15 | 6, 1, 4 | 9, 10, 3, 11, 7, 8

Alle Schlüssel im Internspeicher sind kleiner als der zuletzt ausgegebene; daher wird ein neuer Run angefangen.

Schritt 7: 2, 5, 12, 13, 14, 15; 1 | 6, 4, 9 | 10, 3, 11, 7, 8
 Schritt 8: 2, 5, 12, 13, 14, 15; 1, 4 | 6, 9, 10 | 3, 11, 7, 8

⋮

Schritt 15: 2, 5, 12, 13, 14, 15; 1, 4, 6, 9, 10, 11; 3, 7, 8

Am Beispiel erkennt man, daß die Länge von Runs I übersteigen kann. Man kann zeigen, daß beim Verfahren Auswahl und Ersetzen die durchschnittliche Länge von Runs $2 \cdot I$ ist, Runs also im Mittel doppelt so lang sind wie beim internen Sortieren nach Abschnitt 2.7.2 (vgl. ). Außerdem werden Runs, die schon in der Eingabefolge vorhanden sind, noch v  angert, also eine Vorsortierung berücksichtigt.

Analyse: Anfangs werden mittels Auswahl und Ersetzen mindestens ein Run, höchstens $\lceil \frac{N}{I} \rceil$ Runs und im Mittel $\lceil \frac{N}{(2 \cdot I)} \rceil$ Runs hergestellt. Nach jeder Verschmelzungs- und Verteilungsphase hat sich die Anzahl der Runs auf das $1/k$ -fache verringert. Damit ist

$$P_{min}(N) = 1; \quad P_{mit}(N) = \left\lceil \log_k \left(\frac{N}{(2 \cdot I)} \right) \right\rceil; \quad P_{max}(N) = \left\lceil \log_k \left(\frac{N}{I} \right) \right\rceil$$

bei $2k$ Bändern und Internspeichergröße I .

2.7.4 Mehrphasen-Mergesort

Während beim ausgeglichenen k -Wege-Mergesort alle k Eingabebänder benutzt werden, wird nur auf eines der Ausgabebänder geschrieben. Die anderen Ausgabebänder sind temporär nutzlos. Da normalerweise k viel kleiner als I ist, wäre es wünschenswert, diese Bänder mit als Eingabebänder heranzuziehen. Man will aber sicherlich nicht alle Runs auf ein einziges Ausgabeband speichern, weil man sonst vor der nachfolgenden Verschmelze-Phase noch eine zusätzliche Verteilungsphase einschieben müßte.

Methode: Beim *Mehrphasen-Mergesort* (*polyphase mergesort*) arbeitet man mit $k + 1$ Bändern, von denen zu jedem Zeitpunkt k Eingabebänder sind und eines das Ausgabeband ist. Man schreibt solange alle entstehenden Runs auf das Ausgabeband, bis eines der Eingabebänder erschöpft ist (das ist eine *Phase*). Dann wird das leergewordene Eingabeband zum Ausgabeband, und das bisherige Ausgabeband wird zurückgespult und dient als ein Eingabeband. Damit hat man wieder k Eingabebänder und ein (anderes) Ausgabeband. Der Sortiervorgang ist beendet, wenn alle Eingabebänder erschöpft sind.

Dieses Verfahren funktioniert nur dann wie beabsichtigt, wenn zu jedem Zeitpunkt (außer am Schluß) nur ein Eingabeband leer wird. Betrachten wir ein Beispiel für drei Bänder. In der Anfangsverteilung seien die 13 Runs $r_1, r_2, \dots, r_{13}$ auf die beiden Bänder t_1 und t_2 verteilt wie folgt (t_3 ist leer):

t_1	t_2	t_3
$r_1 r_2 \dots r_8$	$r_9 r_{10} \dots r_{13}$	leer

Dann werden die jeweils nächsten Runs von t_1 und t_2 verschmolzen und auf das Ausgabeband t_3 geschrieben, bis t_2 erschöpft ist (das ist die erste Phase).

$$r_6 r_7 r_8 \mid \text{leer} \mid r_{1,9} r_{2,10} r_{3,11} r_{4,12} r_{5,13}$$

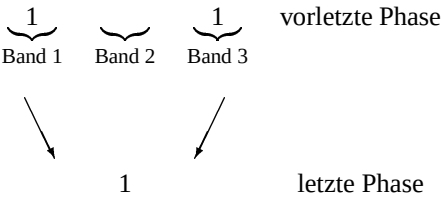
Jetzt wird t_2 zum Ausgabeband; die jeweils nächsten Runs von t_1 und t_3 werden verschmolzen, bis t_1 leer ist.

$$\text{leer} \mid r_{6,1,9}r_{7,2,10}r_{8,3,11} \mid r_{4,12}r_{5,13}$$

Wir zeigen die Situation nach jeder Phase, bis zum Ende:

$$\begin{array}{ccc|ccc|ccc} r_{6,1,9,4,12}r_{7,2,10,5,13} & & & r_{8,3,11} & & & \text{leer} & & \\ r_{7,2,10,5,13} & & & \text{leer} & & & r_{6,1,9,4,12,8,3,11} & & \\ \text{leer} & & & r_{7,2,10,5,13,6,1,9,4,12,8,3,11} & & & \text{leer} & & \end{array}$$

Der Aufwand für eine Phase ist dabei im allgemeinen (außer für die Anfangsverteilung und für die letzte Phase) niedriger als für einen Durchgang beim Mehrwege-Mergesort, weil in einer Phase nicht alle Datensätze bearbeitet werden. Überlegen wir uns anhand der obigen Illustration, wie denn die Runs anfangs auf die beiden Eingabebänder verteilt sein müssen, damit nach jeder Phase (außer der letzten) nur ein Band erschöpft ist. Am Ende muß gerade ein Run auf einem Band stehen. Dieser Run muß entstehen aus zwei Runs, die auf zwei Bändern stehen.



Einer der beiden Runs der vorletzten Phase, sagen wir ohne Einschränkung der Allgemeinheit der Run auf Band 1, muß entstanden sein aus einem Run auf Band 2, das leer wurde, und einem Run auf Band 3:

Band			Phase
1	2	3	
	1		i
1		1	$i - 1$
	1	2	$i - 2$

Entsprechend können wir Situationen in vorangehenden Phasen konstruieren, die zum gewünschten Ergebnis führen.

$$\begin{array}{ccc|ccc} & & & 1 & & \\ & & & \vdots & & \\ & & & F_n & & \\ F_{n+1} = F_{n-1} + F_n & & & F_{n-1} & & F_n \end{array}$$

Damit zeigt sich, daß die Run-Zahlen auf den beiden Anfangsbändern von der Form F_{n-1} und F_n sein müssen, mit

$$F_0 = 0, F_1 = 1 \text{ und } F_n = F_{n-1} + F_{n-2}, n \geq 2.$$

Das sind gerade die Fibonacci-Zahlen. Man kann eine Anfangsverteilung auf zwei der drei Bänder so, daß die Run-Zahlen gerade zwei aufeinanderfolgende Fibonacci-Zahlen sind, erreichen, indem man fiktive Runs nach Bedarf zu den durch die Eingabe erhaltenen Runs hinzunimmt (natürlich ohne sie wirklich abzuspeichern).

Analyse: Man kann zeigen (vgl. ) , daß Mehrphasen-Mergesort bei drei Bändern im Mittel

$$P_{mit}(N) = 1.04 \cdot \log S + 0.99$$

Durchgänge benötigt, wenn man anfangs aus den N Datensätzen S Runs erzeugt. Damit kann man mit Mehrphasen-Mergesort und drei Bändern etwa gleich schnell sortieren wie mit ausgeglichenem Mehrwege-Mergesort und vier Bändern.

Die Strategie des Mehrphasen-Mergesort läßt sich auch auf mehr als drei Bänder anwenden. Die Verteilung der Runs auf die k Eingabebänder muß dann den Fibonacci-Zahlen höherer Ordnung folgen:

$$\begin{aligned} F_n^k &= F_{n-1}^k + F_{n-2}^k + \dots + F_{n-k}^k \quad \text{für } n \geq k, \quad \text{und} \\ F_n^k &= 0 \quad \text{für } 0 \leq n \leq k-2, \quad \text{und} \\ F_{k-1}^k &= 1. \end{aligned}$$

Die Anzahl der zum Sortieren von N Datensätzen mit $k+1$ Bändern benötigten Durchgänge ist dann für größere Werte von k (ab $k=6$) etwa

$$P_{mit}(N) = 0.5 \log S,$$

wenn man anfangs S Runs erzeugt hat.

Weder haben wir in diesem Abschnitt alle grundlegenden Strategien zum externen Sortieren mit Bändern behandelt, noch sind externe Mergesort-Verfahren erschöpfend betrachtet worden. Der interessierte Leser findet in [89] noch eine Fülle von Verfahren und Überlegungen, auch praktischer Natur. Beispielsweise kann man versuchen, auch die Rückspulzeit des Ausgabebandes beim Phasenende des Mehrphasen-Mergesort zum Sortieren zu nutzen; diesem Gedanken folgt das *kaskadierende Mergesort* (*cascade mergesort*). Wenn das Magnetband auch rückwärts gelesen werden kann und zwischen Lesen und Schreiben vorwärts und rückwärts umgeschaltet werden kann, so läßt sich das *oszillierende Mergesort* (*oscillating mergesort*) einsetzen. Als Zusammenfassung vieler Überlegungen formuliert D. Knuth einen Satz .

Theorem *It is difficult to decide which merge pattern is best in a given situation.*

Dieser Satz bedarf sicherlich keines Beweises.

2.8 Untere Schranken

Die große Zahl und unterschiedliche Laufzeit der von uns diskutierten Sortierverfahren legt die Frage nahe, wieviele Schritte zum Sortieren von N Datensätzen denn mindestens benötigt werden. In dieser Form ist die Frage natürlich zu vage formuliert, um eine präzise Antwort zu ermöglichen. Der zum Sortieren von N Datensätzen mindestens erforderliche Aufwand hängt ja unter anderem davon ab, was wir über die zu sortierenden Datensätze wissen und welche Operationen wir im Sortierverfahren zulassen. Wir wollen daher zunächst nur die Klasse der allgemeinen Sortierverfahren betrachten. Das sind Verfahren, die zur Lösung des Sortierproblems nur Vergleichsoperationen zwischen Schlüsseln benutzen. Alle von uns vorgestellten allgemeinen Sortierverfahren haben jedenfalls im schlechtesten Fall wenigstens $\Omega(N \log_2 N)$ Vergleichsoperationen zwischen Schlüsseln benötigt. Diese Zahl von Vergleichsoperationen war auch im Mittel erforderlich, wenn man über alle $N!$ möglichen Anordnungen von N Schlüsseln mittelt und jede Anordnung als gleichwahrscheinlich ansieht. Kann man mit weniger Vergleichsoperationen auskommen? Die Antwort ist „ja“, wenn man etwa weiß, daß die Datensätze gut vorsortiert sind, vgl. hierzu Abschnitt 2.6, aber „nein“ im schlechtesten Fall und im Mittel. Wir zeigen dazu den folgenden Satz.

Satz 2.4 *Jedes allgemeine Sortierverfahren benötigt zum Sortieren von N verschiedenen Schlüsseln sowohl im schlechtesten Fall als auch im Mittel wenigstens $\Omega(N \log N)$ Schlüsselvergleiche.*

Zum Beweis dieses Satzes müssen wir uns eine Übersicht über alle möglichen allgemeinen Sortierverfahren verschaffen. Wir haben das bereits in Abschnitt 2.6 mit Hilfe von Entscheidungsbaum gemacht. Im Entscheidungsbaum kann man die von einem allgemeinen Sortierverfahren ausgeführten Schlüsselvergleiche festhalten. Jeder innere Knoten enthält ein Paar (i, j) von Indizes und repräsentiert einen Vergleich zwischen den Schlüsseln k_i und k_j , den ein Sortierverfahren A für N Schlüssel $k_1, \dots, k_N$ ausführt. Am Anfang hat A keine Information über die „richtige“ Anordnung von $k_1, \dots, k_N$, d. h. alle $N!$ Permutationen sind möglich. Nach einem Vergleich der Schlüssel k_i und k_j kann A jeweils alle diejenigen Permutationen ausschließen, in denen k_i vor bzw. hinter k_j auftritt. Das wird im Entscheidungsbaum durch einen mit „ $i : j$ “ beschrifteten Knoten mit zwei Ausgängen modelliert, dessen linker Ausgang die Bedingung „ $k_i \leq k_j$ “ und dessen rechter Ausgang die Bedingung „ $k_i > k_j$ “ repräsentiert. Jedes Blatt ist mit derjenigen Permutation der Schlüssel $k_1, \dots, k_N$ markiert, die alle Bedingungen erfüllt, die auf dem Weg von der Wurzel zu diesem Blatt auftreten. (Ein Beispiel für einen Entscheidungsbaum mit vier Schlüsseln zeigt Abbildung 2.10). Die zum Sortieren von $k_1, \dots, k_N$ durch A ausgeführte Anzahl von Schlüsselvergleichen ist die Zahl der Knoten auf dem Pfad von der Wurzel des Entscheidungsbaumes zum mit $(k_1, \dots, k_N)$ markierten Blatt. Man nennt diese Zahl die *Tiefe* des Blattes. (Vgl. hierzu auch das Kapitel 5.) Der Algorithmus A könnte natürlich überflüssige Vergleiche durchführen, um die Permutation $k_1, \dots, k_N$ zu identifizieren. Weil wir aber an der Mindestanzahl von Vergleichsoperationen interessiert sind, können wir annehmen, daß keine überflüssigen Vergleiche auftreten. Weil alle $N!$ Anordnungen der N Schlüssel möglich sind, muß der das Verfahren A modellierende Entscheidungsbaum (mindestens) $N!$ Blätter haben. Zum Nachweis der unteren Schranke genügt es nun, die folgende Behauptung zu zeigen:

Behauptung Die maximale und die mittlere Tiefe eines Blattes in einem Binärbaum mit k Blättern ist wenigstens $\log_2 k$.

Der erste Teil der Behauptung ist trivial, weil ein Binärbaum, dessen sämtliche Blätter höchstens die Tiefe t haben, auch nur höchstens 2^t Blätter haben kann. Zum Nachweis des zweiten Teils nehmen wir an, die Behauptung sei falsch. Sei T der kleinste Binärbaum, für den die Behauptung nicht gilt. T habe k Blätter. Dann muß $k \geq 2$ sein und T einen linken Teilbaum T_1 mit k_1 Blättern und einen rechten Teilbaum T_2 mit k_2 Blättern haben, mit $k_1 < k$, $k_2 < k$ und $k_1 + k_2 = k$ (siehe Abbildung 2.14). Da T_1 und T_2 kleiner sind als T , muß gelten:

$$\text{mittlere Tiefe } (T_1) \geq \log_2 k_1$$

$$\text{mittlere Tiefe } (T_2) \geq \log_2 k_2.$$

Offenbar ist für jedes Blatt von T die Tiefe, bezogen auf die Wurzel von T , um genau eins größer als die Tiefe des Blattes in T_1 bzw. T_2 . Daher gilt:

$$\begin{aligned} \text{mittlere Tiefe } (T) &= \frac{k_1}{k} (\text{mittlere Tiefe } (T_1) + 1) + \frac{k_2}{k} (\text{mittlere Tiefe } (T_2) + 1) \\ &\geq \frac{k_1}{k} (\log_2 k_1 + 1) + \frac{k_2}{k} (\log_2 k_2 + 1) \\ &= \frac{1}{k} (k_1 \log_2 (2k_1) + k_2 \log_2 (2k_2)) = f(k_1, k_2) \end{aligned}$$

Diese Funktion $f(k_1, k_2)$ nimmt ihr Minimum unter der Nebenbedingung $k_1 + k_2 = k$ für $k_1 = k_2 = \frac{k}{2}$ an; also ist

$$\text{mittlere Tiefe } (T) \geq \frac{1}{k} \left(\frac{k}{2} \log_2 k + \frac{k}{2} \log_2 k \right) = \log_2 k.$$

Die Behauptung gilt also doch für T im Widerspruch zur Annahme.

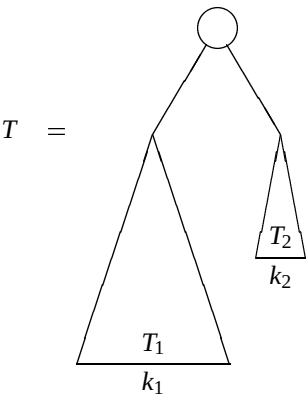


Abbildung 2.14

Aus der soeben bewiesenen Behauptung kann man nun die untere Schranke für die maximale und mittlere Zahl von Vergleichsoperationen zum Sortieren von N Schlüsseln mit Hilfe eines allgemeinen Sortierverfahrens leicht herleiten. Da der zugehörige Entscheidungsbaum wenigstens $N!$ Blätter haben muß, ist die maximale und mittlere Tiefe eines Blattes wenigstens $\log(N!) \in \Omega(N \log N)$. Denn es ist $N! \geq (\frac{N}{2})^{\frac{N}{2}}$ und damit $\log(N!) \geq \frac{N}{2} \log(\frac{N}{2}) = \Omega(N \log N)$. $\square$

Kann man N Zahlen im Mittel und im schlechtesten Fall schneller als in größenordnungsmäßig $N \log N$ Schritten sortieren, wenn man andere Operationen, also nicht nur Vergleichsoperationen zwischen Schlüsseln zuläßt? Ein bemerkenswertes Ergebnis dieser Richtung wurde 1978 von W. Dobosiewicz  erzielt. Er hat ein Sortierverfahren angegeben, das neben arithmetischen Operationen auch noch die sogenannte Floor-Funktion „ $\lfloor \cdot \rfloor$ “ benutzt, die einer reellen Zahl x die größte ganze Zahl $i \leq x$ zuordnet, und gezeigt: Das Verfahren erlaubt es, N reelle Zahlen stets in $O(N \log N)$ Schritten zu sortieren; der Erwartungswert der Laufzeit des Verfahrens für N gleichverteilte Schlüssel ist sogar $O(N)$.

Wir zeigen jetzt, daß es nichts nützt, wenn man nur die arithmetischen Operationen $+$, $-$, $*$, $/$ zuläßt. Wir lassen also die Annahme fallen, daß Vergleichsoperationen die einzigen zugelassenen Operationen zwischen Schlüsseln sind. Stattdessen nehmen wir an, die zu sortierenden Schlüssel seien reelle Zahlen, die addiert, subtrahiert, multipliziert und dividiert werden dürfen. Wir verallgemeinern das Konzept von Entscheidungsbaum wie folgt zum Konzept von rationalen Entscheidungsbaum.

Ein *rationaler Entscheidungsbaum* für N reellwertige Eingabevariablen ist ein Binärbaum, dessen innere Knoten Entscheidungen abhängig vom Wert rationaler Funktionen für die Eingabevariablen repräsentieren. Jedem inneren Knoten i ist eine Funktion $B_i(x_1, \dots, x_N)$ zugeordnet; bei i wird nach links verzweigt, falls $B_i(x_1, \dots, x_N) > 0$ (oder: $B_i(x_1, \dots, x_N) \geq 0$) ist, und nach rechts sonst. Ferner wird jedem Blatt j eine rationale Funktion $A_j(x_1, \dots, x_N)$ zugeordnet.

Ein rationaler Entscheidungsbaum berechnet in offensichtlicher Weise eine auf einer Menge $W \subseteq \mathbb{R}^N$ definierte, reellwertige Funktion. Betrachten wir dazu das in Abbildung 2.15 gezeigte Beispiel.

Dieser rationale Entscheidungsbaum berechnet folgende, reellwertige Funktion:

$$f(x_1, x_2) = \begin{cases} x_1 + x_2; & \text{falls } x_1 + x_2^2 \geq 0 \text{ und } (x_1^3 - x_2)/(x_1 x_2) > 0 \\ x_1/x_2; & \text{falls } x_1 + x_2^2 \geq 0 \text{ und } (x_1^3 - x_2)/(x_1 x_2) < 0 \\ (x_1 + x_2)/x_1; & \text{falls } x_1 + x_2^2 < 0 \end{cases}$$

Diese Funktion f ist auf dem Gebiet $W \subseteq \mathbb{R}^2$ definiert, dessen Gestalt und Eigenschaften uns hier nicht interessieren.

Allgemein kann man die von einem rationalen Entscheidungsbaum berechnete Funktion f von N reellwertigen Variablen $X = x_1, \dots, x_N$ schreiben in der Form:

$$f(X) = \begin{cases} A_1(X); & \text{falls } X \in M_1 \\ \vdots \\ A_m(X); & \text{falls } X \in M_m \end{cases}$$

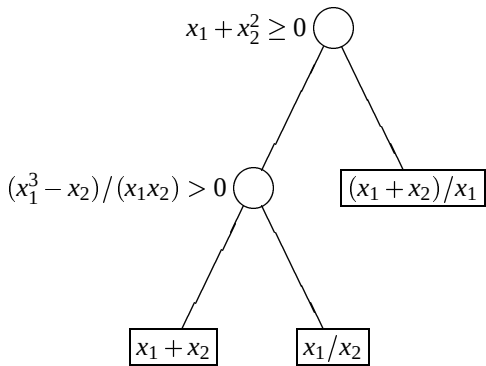


Abbildung 2.15

Dabei sind die $A_j(X)$ die an den Blättern des Entscheidungsbaumes stehenden Funktionen, und M_j bezeichnet die Konjunktion der Bedingungen, die auf dem Pfad von der Wurzel bis zum mit A_j beschrifteten Blatt gültig sind.

Wir stellen nun eine Verbindung her zwischen Sortierverfahren, die die Operationen $\{<, +, -, *, /\}$ benutzen dürfen, und rationalen Entscheidungsbaumen zur Berechnung von Funktionen. Wir betrachten dazu die sogenannte *Sortierindexfunktion* von N Argumenten $f(x_1, \dots, x_N) = x_1^{r_1} + \dots + x_N^{r_N}$, mit $r_i = \text{Rang}$ von x_i in der Folge der Argumente bei aufsteigender Sortierung. Für $N = 2$ gilt also beispielsweise:

$$f(x_1, x_2) = \begin{cases} x_1^1 + x_2^2; & \text{falls } x_1 < x_2 \\ x_2^1 + x_1^2; & \text{falls } x_1 \geq x_2 \end{cases}$$

Kann man nun das Sortierproblem für N reelle Zahlen $x_1, \dots, x_N$ mit k Vergleichsoperationen des Typs $B_i(x_1, \dots, x_N) > 0$ oder $B_i(x_1, \dots, x_N) \geq 0$ für rationale Funktionen B_i lösen, so kann man auch die Sortierindexfunktion von N Argumenten mit k derartigen Vergleichsoperationen berechnen. Denn der Wert der Sortierindexfunktion kann ohne weitere Vergleichsoperation berechnet werden, sobald die Anordnung der Argumente bekannt ist. Damit liefert eine untere Schranke für die Berechnung der Sortierindexfunktion auch eine untere Schranke für das Sortierproblem. Eine untere Schranke für die Berechnung der Sortierindexfunktion kann man aber — wie im Falle „gewöhnlicher“ Entscheidungsbaume — sofort aus einer unteren Schranke für die Blattzahl eines rationalen Entscheidungsbaums zur Berechnung der Sortierindexfunktion ableiten. Um eine solche Schranke herzuleiten, zeigen wir ganz allgemein, daß jeder rationale Entscheidungsbaum zur Berechnung einer Funktion $f : \mathbb{R}^N \rightarrow \mathbb{R}$, die in wenigstens q verschiedene Teile zerfällt, wenigstens q Blätter haben muß. Genauer zeigen wir (vgl.



Satz 2.5 Sei $f : \mathbb{R}^N \rightarrow \mathbb{R}$ eine auf $W \subseteq \mathbb{R}^N$ definierte Funktion, seien $X_1, \dots, X_q \in W$ paarweise verschiedene Punkte und $Q_1, \dots, Q_q$ paarweise verschiedene rationale Funktionen, die auf Kreisen mit Radius $e > 0$ um $X_1, \dots, X_q$ definiert sind und dort mit f übereinstimmen, also:

$$f(X) = Q_i(X), \quad \text{für alle } X \in U(X_i, e) = \{X : |X_i - X| < e\} \subseteq W.$$

Dann muß jeder Entscheidungsbaum zur Berechnung von f wenigstens q Blätter haben.

Zum Beweis dieses Satzes benutzt man wohlbekannte Fakten aus der algebraischen Geometrie. Wir skizzieren die Argumentationskette und verweisen auf [1] für weitere Einzelheiten.

Den durch einen rationalen Entscheidungsbaum zur Berechnung von f gegebenen Algorithmus A kann man schreiben in der Form:

$$A(X) = \begin{cases} A_1(X); & \text{falls } X \in M_1 \\ \vdots \\ A_m(X); & \text{falls } X \in M_m \end{cases}$$

Dabei ist m die Anzahl der Blätter des Entscheidungsbaumes. Weil A f berechnet, muß für jedes i gelten:

$$U(X_i, e) = (U(X_i, e) \cap M_1) \cup \dots \cup (U(X_i, e) \cap M_m)$$

Auf der rechten Seite dieser Gleichung steht die endliche Vereinigung von Mengen, die durch rationale Bedingungen definiert sind. Weil die linke Seite eine Menge ist, die alle Punkte einer ganzen Kreisscheibe mit positivem Radius enthält, muß auch wenigstens eine der Mengen $(U(X_i, e) \cap M_j)$ diese Eigenschaft haben! (Das ist das erste Ergebnis aus der algebraischen Geometrie, das wir benötigen.) Es gibt also ein j , so daß für alle Punkte einer ganzen Kreisscheibe in M_j die Funktionen Q_j und A_j dort übereinstimmen. Weil Q_j und A_j rationale Funktionen sind, müssen sie dann überhaupt identisch sein. (Das ist das zweite benötigte Ergebnis aus der algebraischen Geometrie; man kann es als eine Verallgemeinerung des bekannten Nullstellensatzes für Polynome auffassen.) Also muß es wenigstens so viele verschiedene A_j wie Q_j geben, d. h. $m \geq q$. $\square$

Kehren wir nun zur Sortierindexfunktion zurück und wenden wir Satz 2.5 darauf an. Diese Funktion ist für die $q = N!$ verschiedenen Punkte $X_\pi = (\pi(1), \dots, \pi(N))$, π Permutation von $\{1, \dots, N\}$, definiert und stimmt jeweils auf einem Kreis mit Radius $e > 0$, $e < \frac{1}{2}$, um diese Punkte mit einer rationalen Funktion, der Funktion $Q_\pi(X_1, \dots, X_N) = X_1^{\pi(1)} + \dots + X_N^{\pi(N)}$, überein. Daher muß jeder rationale Entscheidungsbaum zur Berechnung der Sortierindexfunktion wenigstens $N!$ Blätter haben. Die maximale und mittlere Tiefe eines Blattes ist damit in $\Omega(N \log N)$.

In den letzten Jahren ist es gelungen, wesentlich stärkere Sätze dieser Art zum Nachweis unterer Schranken zu beweisen. Dazu wurden sogenannte *algebraische Entscheidungsbäume* definiert und Sätze analog zu Satz 2.5 für solche Bäume bewiesen. Zum Nachweis dieser Sätze werden aber mächtige Hilfsmittel aus der algebraischen Geometrie benötigt, die weit über den Rahmen dieses Buches hinausgehen. Den interessierten Leser verweisen wir auf [1] und auf die Originalarbeit [2].

2.9 Aufgaben

Aufgabe 2.1

Sortieren Sie die unten angegebene, in einem Feld a gespeicherte Schlüsselfolge mit dem jeweils angegebenen Verfahren und geben Sie jede neue Belegung des Feldes nach einem Schlüsseltausch an.

- a) 40–15–31–8–26–22; Auswahlsort
- b) 35–22–10–51–48; Bubblesort.

Aufgabe 2.2

Schreiben Sie eine Pascal-Prozedur für Sortieren durch Auswahl, wobei die zu sortierende Zahlenfolge nicht in einem Array, sondern in Form einer linearen Liste vorliegt. Die Listenelemente seien durch folgende Typvereinbarung gegeben:

```

type listenzeiger =  $\uparrow$ listenelement;
      listenelement = record
                        key : integer;
                        next : listenzeiger
                      end;
```

Der Beginn der Liste werde durch ein Dummy-Element, das keinen Eintrag enthält und auf das ein Zeiger *head* zeigt, markiert. Das Listenende wird durch ein Listenelement gekennzeichnet, dessen *next*-Komponente den Wert **nil** hat. Die Liste enthalte mindestens zwei Elemente außer dem Dummy-Element.

Aufgabe 2.3

Geben Sie für die unten angegebenen Zahlenfolgen jeweils mit Begründung die Laufzeit der Sortierv Verfahren Auswahlsort, Einfügesort und Bubblesort in Groß-Oh-Notation an.

- a) $1, \frac{N}{2} + 1, 2, \frac{N}{2} + 2, \dots, \frac{N}{2}, N$ (N gerade)
- b) $N, 1, N - 1, 2, N - 2, 3, \dots, N - \frac{N}{2} + 1, \frac{N}{2}$ (N gerade)
- c) $N, 1, 2, 3, \dots, N - 1$

d) $2, 3, 4, \dots, N, 1$

Aufgabe 2.4

Sei N eine gerade Zahl. Wie groß ist der Aufwand zum Sortieren der Folge

$$\frac{N}{2}, \frac{N}{2} + 1, \dots, N, 1, 2, \dots, \frac{N}{2} - 1$$

bei den Sortierv Verfahren: 2-Wege-Mergesort, reines 2-Wege-Mergesort, natürliches 2-Wege-Mergesort?

Aufgabe 2.5

Gegeben sei das Array a von neun Elementen mit den Schlüsseln

41 62 13 84 35 96 57 28 79.

Geben Sie alle Aufrufe der Prozedur *quicksort* und die Reihenfolge ihrer Abarbeitung an, die als Folge eines Aufrufs von *quicksort*($a, 1, 9$) im Hauptprogramm für obiges Array auftreten.

Aufgabe 2.6

Schreiben Sie in Pascal eine iterative Quicksort-Prozedur. D.h. die Quicksort-Prozedur wird nicht rekursiv für die neu entstandenen Teilfolgen aufgerufen, sondern man merkt sich die Grenzen der Teilfolgen, z.B. mit Hilfe eines Stapels. Sie dürfen die für Stapel üblichen Operationen verwenden, ohne sie näher auszuführen. Achten Sie darauf, daß möglichst wenig zusätzlicher Speicherplatz benötigt wird.

Aufgabe 2.7

- Gegeben sei die Schlüsselfolge 85, 20, 63, 18, 51, 37, 90, 33. Erzeugen Sie für diese Folge einen Heap und stellen Sie ihn als Binärbaum dar.
- Sortieren Sie die Schlüsselfolge 40, 15, 31, 8, 2, 6, 22 (aufsteigend) mit Heapsort. Stellen Sie zunächst einen Heap her und geben Sie dann jede neue Belegung nach dem Schlüsseltausch an.
- Geben Sie größenordnungsmäßig die Komplexität der folgenden Operationen auf einem Heap mit N Elementen im schlimmsten Fall an. Am Ende einer jeden Operation soll stets ein Heap zurückbleiben.

Einfügen eines beliebigen Elementes, Suchen des Maximums, Suchen eines beliebigen Elementes, Entfernen eines beliebigen Elementes, Suchen des Minimums, Entfernen des Maximums.

Aufgabe 2.8

Gegeben sei die in einem Array a der Länge 10 abgelegte Schlüsselfolge

	1	2	3	4	5	6	7	8	9	10
$a :$	20	14	15	8	10	12	9	5	3	6

Sortieren Sie diese Folge in aufsteigender Reihenfolge mit dem Verfahren Heapsort und geben Sie als Zwischenschritte die Belegung von a an, die vorliegt, bevor das Element $a[1]$ an seine endgültige Position i getauscht wird.

	1	2	3	4	5	6	7	8	9	10
$i = 10$										
9										
8										
7										
6										
5										
4										
3										
2										

Aufgabe 2.9

Überprüfen Sie, ob die folgenden Sortierverfahren stabil sind (d.h. die Reihenfolge von Elementen mit gleichem Sortierschlüssel wird während des Sortierverfahrens nicht vertauscht): Auswahlort, Einfügesort, Shellsort, Mergesort, Radixsort, Bubblesort, Quicksort.

Aufgabe 2.10

Zeigen Sie: Läßt man als einzige Operation zwischen Schlüsseln Vergleichsoperationen zu, so benötigt man wenigstens $N - 1$ Vergleiche im schlechtesten Fall, um zwei sortierte Folgen

$$x_1 \leq x_2 \leq \dots \leq x_{\frac{N}{2}} \quad \text{und} \quad y_1 \leq y_2 \leq \dots \leq y_{\frac{N}{2}}$$

zu einer einzigen sortierten Folge $z_1 \leq z_2 \leq \dots \leq z_N$ zu verschmelzen.

Aufgabe 2.11

Sortieren Sie die angegebene Zahlenfolge durch Fachverteilung. Geben Sie dabei die Belegung der einzelnen Fächer nach jeder Verteilphase an und jeweils die Folge, die nach einer Sammelphase entstanden ist.

1234, 2479, 7321, 4128, 5111, 4009, 6088, 9999, 7899, 6123,
3130, 4142, 7000, 0318, 8732, 3038, 5259, 4300, 8748, 6200

Wenn Sie die folgenden Zahlen zu sortieren hätten und dabei entweder Sortieren durch Fachverteilung oder ein Verfahren, das nur mit Schlüsselvergleichen arbeitet, verwenden könnten, welche Überlegungen würden Sie anstellen?

12345678, 43482809, 91929394, 91929390

Aufgabe 2.12

Berechnen Sie für die Schlüsselreihe F_1 die Vorsortierungsmaße $inv(F_1)$, $runs(F_1)$, $rem(F_1)$:

$$F_1 : 2, 5, 6, 1, 4, 3, 8, 9, 6$$

Berechnen Sie für die Schlüssel Folge F_i ($i = 2, 3, 4$) ebenfalls $\text{inv}(F_i)$, $\text{runs}(F_i)$, $\text{rem}(F_i)$, jeweils in Abhängigkeit von N (N gerade).

$$F_2 : 1, \frac{N}{2} + 1, 2, \frac{N}{2} + 2, \dots, \frac{N}{2}$$

$$F_3 : N, \frac{N}{2}, N - 1, \frac{N}{2} - 1, \dots, \frac{N}{2} + 1, 1$$

$$F_4 : 1, N, 2, N - 1, 3, \dots, \frac{N}{2}, \frac{N}{2} + 1$$

Aufgabe 2.13

- Geben Sie eine Folge F von sieben Schlüsseln an, für die $\text{inv}(F) < \text{runs}(F)$ gilt.
- Geben Sie eine Folge F von sieben Schlüsseln an, für die $\text{runs}(F) < \text{inv}(F)$ gilt.
- Geben Sie eine Folge von sieben Schlüsseln an, für die das natürliche 2-Wege-Mergesort möglichst wenige und das Verfahren A-sort möglichst viele Schlüsselvergleiche benötigt.
- Geben Sie eine Folge F von sieben Schlüsseln mit $\text{runs}(F) \leq 3$ an, für die das Verfahren A-sort möglichst viele Schlüsselvergleiche ausführt.

Aufgabe 2.14

Als weiteres Vorsortierungsmaß findet man in der Literatur $\text{exc}(F)$; das ist die kleinste Anzahl von Vertauschungen, die nötig sind, um eine Folge F in aufsteigende Ordnung zu bringen.

- Geben Sie jeweils eine Folge F mit N Schlüsseln an, für die
 - $\text{exc}(F)$ maximal wird;
 - $\text{exc}(F) = \lfloor \frac{N}{2} \rfloor$ ist.
- Welche Beziehung gilt zwischen $\text{exc}(F)$ und $\text{inv}(F)$ für alle Folgen F ?

Aufgabe 2.15

Geben Sie allgemeine Bedingungen an, die für jedes Vorsortierungsmaß m gelten sollten.

Literaturliste zu Kapitel 2: Sortieren

Seite 63

[109] E. E. Lindstrom, J. S. Vitter und C. K. Wong, Hrsg. IEEE Transactions on Computers, Special Issue on Sorting, C-34. 1985.

Seite 66

[168] D. L. Shell. A high-speed sorting procedure. Comm. ACM, 2:30-32, 1959.

Seite 73

[89] D.E. Knuth. The Art of Computer Programming, Vol.3: Sorting and Searching. Addison-Wesley, Reading, Massachusetts, 1973.

Seite 76

[89] D. E. Knuth. The Art of Computer Programming, Vol.3: Sorting and Searching. Addison-Wesley, Reading, Massachusetts, 1973.

[80] C. A. R. Hoare. Quicksort. Computer Journal, 5:10-15, 1962. [81] R.N. Hoorspool. Practical fast searching in strings. Software-Practice and Experience, 10:501-506, 1980.

Seite 84

[40] B. Durian. Quicksort without a stack. In J. Gruska, B. Rován und J. Wiederman, Hrsg., Proc. Math. Foundations of Computer Science, Prag, S. 283-289. Lecture Notes in Computer Science 233, Springer, 1986.

Seite 86

[123] K. Mehlhorn. Datenstrukturen und effiziente Algorithmen, Band 1, Sortieren und Suchen. Teubner, Stuttgart, 1986.

Seiten 87, 89

[192] L. Wegner. Quicksort for equal keys. IEEE Transactions on Computers, C 34:362-366, 1985.

Seite 94

[194] J. W. J. Williams. Algorithm 232. Comm. ACM, 7:347-348, 1964.

[55] R. W. Floyd. Algorithm 245, treesort 3. Comm. ACM, 7:701, 1964.

Seite 96

[36] E. W. Dijkstra. Smoothsort, an alternative for sorting in situ. Science of Computer Programming, 1:223-233, 1982. Vgl. auch: Errata, Science of Computer Programming 2:85, 1985.

Seite 104

[184] L. Trabb Pardo. Stable sorting and merging with optimal space and time bounds. SIAM J. Comput., 6:351-372, 1977.

[95] M. A. Kronrod. An optimal ordering algorithm without a field of operation. Dokladi Akademii Nauk SSSR, 186:1256-1258, 1969.

Seite 114

[117] H. Mannila. Measures of presortedness and optimal sorting algorithms. IEEE Transactions on Computers, C-34:318-325, 1985.

Seite 119

[123] K. Mehlhorn. Datenstrukturen und effiziente Algorithmen, Band 1, Sortieren und Suchen. Teubner, Stuttgart, 1986.

Seite 120

[117] H. Mannila. Measures of presortedness and optimal sorting algorithms. IEEE Transactions on Computers, C-34:318-325, 1985.

[123] K. Mehlhorn. Datenstrukturen und effiziente Algorithmen, Band 1, Sortieren und Suchen. Teubner, Stuttgart, 1986.

Seite 124

[117] H. Mannila. Measures of presortedness and optimal sorting algorithms. IEEE Transactions on Computers, C-34:318-325, 1985.

Seite 126

[89] D. E. Knuth. The Art of Computer Programming, Vol.3: Sorting and Searching. Addison-Wesley, Reading, Massachusetts, 1973.

[170] H.-W. Six und L. Wegner. EXQUISIT: Applying quicksort to external files. In Proc. 19th Annual Allerton Conference on Communication, Control and Computing, S. 348-354, 1981.

Seiten 135, 137

[89] D.E. Knuth. The Art of Computer Programming, Vol.3: Sorting and Searching. Addison-Wesley, Reading, Massachusetts, 1973.

Seite 140

[38] W. Dobosiewicz. Sorting by distributive partitioning. Information Processing Letters, 7(1):1-6, 1978.

Seiten 141, 142

[164] A. Schmitt. On the number of relational operators necessary to compute certain functions of real variables. Acta Informatica, 19:297-304, 1983.

Seite 143

[123] K. Mehlhorn. Datenstrukturen und effiziente Algorithmen, Band 1, Sortieren und Suchen. Teubner, Stuttgart, 1986.

[14] M. BenOr. Lower bounds for algebraic computation trees. In Proc. 15th ACM Annual Symposium on Theory of Computing, S. 80-86, 1983.

Kapitel 3

Suchen

Das Suchen in Datenmengen ist eine der wichtigsten und grundlegendsten Operationen, die man mit Computern ausführen können möchte. Man denke an das Suchen nach einem Stichwort in einem Wörterbuch oder einer Enzyklopädie, die Suche nach einer Telefonnummer in einem Telefonverzeichnis, nach einem Namen in einer Symboltabelle, nach einer Kontonummer, Personalnummer, usw. Wir setzen in diesem Kapitel durchweg voraus, daß die Information, nach der wir suchen, durch einen Schlüssel eindeutig identifizierbar ist. Meistens nehmen wir der Einfachheit halber an, daß die Schlüssel positive ganze Zahlen sind, wie etwa Kontonummern, Personalnummern, Auftragsnummern. In der Praxis treten allerdings alphabetische Schlüssel, also Namen über einem endlichen Alphabet, ebenfalls häufig auf. Wir lassen bei der Diskussion verschiedener Suchverfahren die über den Schlüssel identifizierbare „eigentliche“ Information meistens unberücksichtigt. Als Argument für eine Suchoperation benutzen wir in der Regel also den Suchschlüssel k . Die Suche nach k in der Menge der gespeicherten Daten kann entweder *erfolgreich* enden bei einem Datum mit Schlüssel k oder aber *erfolglos*, falls es kein Datum mit Schlüssel k in der betrachteten Menge gibt. Im Falle einer erfolgreichen Suche nehmen wir natürlich an, daß wir über den Schlüssel auch Zugriff auf die „eigentliche“ Information haben und diese Information etwa lesen, ausgeben, verändern können.

Wir wollen in diesem Kapitel nur elementare Suchverfahren behandeln; das sind Verfahren, die, wie allgemeine Sortiervverfahren, nur Vergleichsoperationen zwischen Schlüsseln ausführen. Arithmetische Operationen, die es erlauben, aus dem Suchschlüssel direkt die Speicheradresse zu berechnen, werden in diesem Kapitel ebenso wenig behandelt wie besondere Datenstrukturen, die das Suchen und Wiederfinden gespeicherter Daten besonders unterstützen. Wir beschränken uns in diesem Kapitel im wesentlichen auf die Diskussion der wichtigsten elementaren Verfahren zum Suchen in linearen Listen, die sequentiell oder verkettet gespeichert sind. Arithmetische Verfahren zur direkten Bestimmung der Speicheradresse aus einem gegebenen Schlüssel werden ausführlich im Kapitel 4 über Hashing behandelt. Ebenso werden die wichtigsten die Suche unterstützenden Baumstrukturen in einem eigenen Kapitel 5 behandelt. Daß eine vorhandene Sortierung beim Suchen hilft, weiß jeder aus alltäglicher Erfahrung. Wir werden den möglichen Gewinn auch quantitativ abschätzen.

Wir haben etwa beim Sortieren durch Auswahl und bei Heapsort gesehen, daß manche Sortierverfahren auf der Suche nach dem kleinsten Schlüssel oder allgemeiner auf Verfahren zum Suchen nach dem i -kleinsten Schlüssel aufbauen. Diese Suchoperation wird im Unterschied zur Suche nach einem gegebenen Schlüssel üblicherweise als *Auswahl* (*Selection*) bezeichnet. Wir diskutieren das Auswahlproblem ebenfalls in diesem Kapitel und zwar im Abschnitt 3.1. Abschnitt 3.2 enthält die üblichen elementaren Suchverfahren für sequentiell gespeicherte Schlüssel. Im Abschnitt 3.3 diskutieren wir die wichtigsten Verfahren zur Selbstanordnung von linearen Listen. Das sind Verfahren, die nicht nur eine Suchoperation ausführen, sondern auch eventuell eine Umordnung der Liste vornehmen, um künftige Suchoperationen schneller durchführen zu können.

3.1 Das Auswahlproblem

Um das Element mit kleinstem oder größtem Schlüssel in einer Liste von N Elementen zu finden, genügt es, jedes Element der Liste einmal zu inspizieren. Sortieren ist nicht erforderlich. Sucht man das Element mit zweitkleinstem oder zweitgrößtem Schlüssel, kann man zunächst das Element mit kleinstem bzw. größtem Schlüssel bestimmen und aus der Liste entfernen und dann aus der Restliste von $N - 1$ Elementen wiederum das Element mit dem kleinsten bzw. größten Schlüssel bestimmen. Auf analoge Weise fortfahrend kann man also das Element mit i -kleinstem Schlüssel, für jedes i mit $1 \leq i \leq N$, durch i -malige Bestimmung des Elements mit jeweils kleinstem Schlüssel aus Listen mit $N, N - 1, \dots, N - i + 1$ Elementen in insgesamt $\Theta(i \cdot N)$ Schritten bestimmen. Für $i = N/2$ liefert dies insbesondere ein Verfahren zur Bestimmung des *mittleren* Elements aus einer Folge von N Schlüsseln, das Laufzeit $\Theta(N^2)$ hat. Das ist nicht sehr effizient, denn in Kapitel 2 haben wir gesehen, daß eine Folge von N Schlüsseln in $O(N \log N)$ Zeit sortiert werden kann. Da in einer sortierten Schlüsselfolge der i -kleinste Schlüssel in $O(N)$ Zeit bestimmt werden kann, läßt sich das mittlere Element einer Folge von N Schlüsseln offenbar mit Laufzeit $O(N \log N)$ bestimmen.

Gibt es Verfahren, die den i -kleinsten von N Schlüsseln und insbesondere den mittleren Schlüssel, den sogenannten *Median*, schneller zu bestimmen erlauben? Die zwei im Kapitel 2 behandelten Sortierverfahren Heapsort und Quicksort geben einen Hinweis darauf, wie eine Verbesserung des naiven Verfahrens erreicht werden könnte. Wir können versuchen, durch eine geeignete Datenstruktur die i -malige Bestimmung des jeweils kleinsten Schlüssels aus immer kleineren Listen zu beschleunigen; das führt zur Verwendung von Heaps analog zu Heapsort. Andererseits können wir eine Divide-and-conquer-Strategie analog zu Quicksort verfolgen, um den i -kleinsten von N Schlüsseln zu bestimmen. Wir diskutieren beide Möglichkeiten genauer.

Zur Bestimmung des i -kleinsten von N Schlüsseln können wir aus den gegebenen N Schlüsseln zunächst einen Heap herstellen. Anders als beim Verfahren Heapsort bauen wir aber einen sogenannten *Min-Heap* auf, also einen Heap, bei dem die Schlüssel der Väter stets *kleiner* (oder gleich) den Schlüsseln der Söhne sind. Das ist aber auch schon der einzige Unterschied zu den im Abschnitt 2.3 verwendeten Heaps. Man speichert also die Elemente in einem Array und kann das Array in $O(N)$ Schritten in einen Min-

Heap verwandeln. Das Element mit kleinstem Schlüssel steht an der Wurzel (d.h. an Position 1 im Array). Nun entfernt man i -mal nacheinander das jeweils kleinste Element aus dem Min-Heap, macht das letzte Element zur Wurzel und läßt es versickern (wie bei Heapsort). Das kostet jedesmal $O(\log N)$ Schritte. Auf diese Weise kann man das i -kleinste Element in insgesamt $O(N + i \log N)$ Schritten bestimmen. Insbesondere kann man so den Median in $O(N \log N)$ Schritten finden. Das ist schon besser als das naive Verfahren, aber keineswegs optimal. Denn die folgende Überlegung wird zeigen, daß man den i -kleinsten von N Schlüsseln stets in linearer Zeit, also in $O(N)$ Schritten bestimmen kann.

Um das Element mit i -kleinstem Schlüssel zu bestimmen, teilen wir die gegebene Folge von N Elementen wie bei Quicksort bezüglich eines geeignet gewählten Pivotelements in zwei Teilfolgen auf. Nach der Aufteilung wird aber (im Unterschied zu Quicksort) nur eine der durch Aufteilung entstandenen Teilfolgen weiter betrachtet.

Zum Aufteilen eines Bereichs $a[l], \dots, a[r]$ von Elementen verwenden wir folgende Funktion:

```
function teile( $l, r$  : integer; pivot : keytype) : integer;
{teilt den Bereich  $a[l], \dots, a[r]$  in zwei Gruppen:
 $a[l], \dots, a[m-1]$  sind Elemente mit Schlüssel  $\leq$  pivot,
 $a[m], \dots, a[r]$  sind Elemente mit Schlüssel  $\geq$  pivot;
die Funktion liefert als Wert den Beginn  $m$  der zweiten Gruppe}
```

Eine mögliche Implementation für die Funktion *teile* kann man leicht aus der im Abschnitt 2.2 angegebenen Prozedur *quicksort* ablesen. Das gibt uns folgenden Algorithmus, um das Element mit i -kleinstem Schlüssel unter den Elementen $a[l], \dots, a[r]$ zu bestimmen (anfangs ist $l = 1$ und $r = N$): Wir wählen ein Pivotelement v und teilen den Bereich mit Hilfe der Funktion *teile* auf. Falls $i \leq m - l$ ist, dann kommt das i -kleinste Element in der ersten durch Aufteilung entstandenen Gruppe vor. Falls $i > m - l$ ist, dann ist das Element mit i -kleinstem Schlüssel in $a[l], \dots, a[r]$ das Element mit $i - (m - l)$ -kleinstem Schlüssel in der zweiten durch Aufteilung entstandenen Gruppe $a[m], \dots, a[r]$. Wenn schließlich $l = r$ (und $i = 1$) geworden ist, haben wir das gesuchte Element gefunden.

Man erhält das folgende naheliegende Programmgerüst für das Verfahren:

```
procedure auswahl ( $l, r, i$  : integer);
{liefert das Element mit  $i$ -kleinstem Schlüssel unter den Elementen
 $a[l], \dots, a[r]$ ; es wird  $r \geq l$  und  $1 \leq i \leq (r - l) + 1$  vorausgesetzt}
var  $m, v$  : integer;
begin
  if  $r > l$  then {aufteilen}
    begin
      wähle Pivotelement  $v$ ;
       $m :=$  teile( $l, r, v$ );
      if  $(i \leq m - l)$ 
        then auswahl( $l, m - 1, i$ )
        else auswahl( $m, r, i - m + l$ )
    end
  else { $r = l$ }
    {jetzt muß  $i = 1$  sein, also ist  $a[l]$  das gesuchte Element}
  end
```

Wenn wir das Pivotelement so ungünstig wählen, daß eine der durch Aufteilung entstehenden Folgen stets nur ein Element enthält und wir rekursiv jedesmal die andere Folge weiter betrachten müssen, benötigt dieses Verfahren zur Bestimmung des Elementes mit i -kleinstem Schlüssel in einer gegebenen Folge von N Elementen natürlich $\Omega(N^2)$ Schritte.

Man kann gegenüber diesem ungünstigsten möglichen Fall also nur dann etwas gewinnen, wenn es gelingt, das Pivotelement so zu wählen, daß man bei jedem Aufteilungsschritt mit Sicherheit einen bestimmten Bruchteil der noch zu betrachtenden Elemente ausschließen kann. Nehmen wir einfach einmal an, das Pivotelement könne stets so bestimmt werden, daß eine Aufteilung eines Bereichs von N Elementen nach diesem Pivotelement zwei Gruppen liefert, die jeweils $q \cdot N$ und $(1 - q) \cdot N$ Elemente haben, mit einem festen Faktor q , $0 < q < 1$. Wir können ohne Einschränkung annehmen, daß q die größere der beiden Zahlen q und $(1 - q)$ ist.

Nach der Aufteilung eines Bereichs der Länge N muß die Prozedur *auswahl* zur Bestimmung des i -kleinsten Elements dann höchstens für einen Bereich mit Länge $q \cdot N$ aufgerufen werden. Die Aufteilung selbst (mit Hilfe der Funktion *teile*) kann in $O(N)$ Schritten durchgeführt werden.

Bezeichnen wir mit $T(N)$ die Anzahl der Schritte, die erforderlich ist, um das Element mit i -kleinstem Schlüssel unter N Elementen mit Hilfe der Prozedur *auswahl* zu bestimmen, so gilt offenbar folgende Rekursionsgleichung:

$$\begin{aligned} T(N) &= T(q \cdot N) + c \cdot N, \text{ mit einer Konstanten } c \\ &\leq c \cdot N \cdot \sum_{i=0}^{\infty} q^i = c \cdot N \cdot \frac{1}{1 - q} = O(N). \end{aligned}$$

Das Verfahren zur Bestimmung des i -kleinsten Elementes ist also in linearer Zeit ausführbar, wenn das Pivotelement stets richtig gewählt werden kann. Hier hilft die folgende von Blum u.a. vorgeschlagene *Median-of-median-Strategie* [20].

Um in einer Folge von N Elementen mit paarweise verschiedenen Schlüsseln das i -kleinste Element zu finden, benutze das folgende Verfahren *Auswahl*:

0. {Rekursionsabbruch}

Falls $N < \text{Konstante}$, berechne i -kleinstes Element direkt und Stop. Sonst:

1. Teile die N Elemente in $\lceil \frac{N}{5} \rceil$ Gruppen zu je fünf Elementen und höchstens eine Gruppe mit höchstens vier Elementen auf.
2. Sortiere jede dieser $\lceil \frac{N}{5} \rceil$ Gruppen (in konstanter Zeit) und bestimme in jeder Gruppe das mittlere Element. Es ist für alle Fünfergruppen eindeutig bestimmt und auch für die letzte Gruppe mit weniger als fünf Elementen, wenn diese Gruppe eine ungerade Zahl von Elementen hat. Hat die letzte Gruppe eine gerade Zahl von Elementen, so wähle das größere der beiden mittleren Elemente. Man erhält so insgesamt $\lceil \frac{N}{5} \rceil$ Mediane in Zeit $O(N)$.

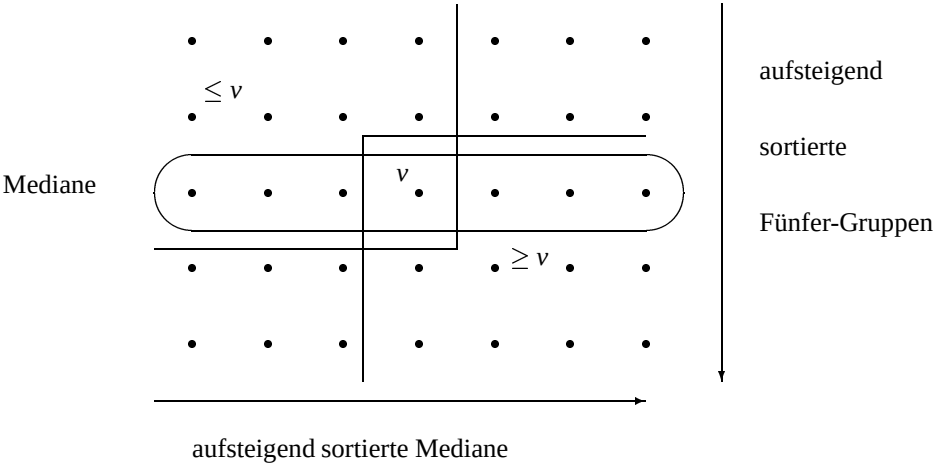


Abbildung 3.1

3. Wende das Verfahren *Auswahl* rekursiv auf die $\lceil \frac{N}{5} \rceil$ Mediane an, um das mittlere Element v dieser Mediane zu finden. (Falls $\lceil \frac{N}{5} \rceil$ gerade ist, ist v das größere der beiden mittleren Elemente.) v heißt Median der Mediane. Abbildung 3.1 zeigt ein Beispiel für den Fall $N = 35$.

Jetzt wählt man den Median der Mediane als Pivotelement für die Aufteilung aller Elemente und fährt wie bekannt fort:

- 4. Teile die N Elemente bezüglich v auf in zwei Gruppen: Gruppe 1 enthält die k Elemente, die kleiner als v sind und Gruppe 2 enthält die $N - k - 1$ Elemente, die größer als v sind. Die Aufteilung kann in Zeit $O(N)$ durchgeführt werden.
- 5. Falls $i \leq k$ ist, bestimmt man mit Hilfe des Verfahrens *Auswahl* rekursiv das i -kleinste Element unter den k Elementen der Gruppe 1. Ist $i > k + 1$, so bestimmt man mit Hilfe des Verfahrens *Auswahl* rekursiv das $i - (k + 1)$ -te Element in der Gruppe 2. Falls $i = k + 1$ ist, hat man das i -te Element in der Ausgangsfolge gefunden (nämlich v).

Die Wahl von v als Median von Medianen sichert, daß mit Ausnahme der Gruppe, in der v selbst vorkommt, und der möglicherweise vorkommenden einzigen Gruppe mit weniger als fünf Elementen, jede Fünfergruppe mit mittlerem Element kleiner als v wenigstens drei Elemente enthält, die kleiner als v sind. Genauso enthält jede Fünfergruppe mit mittlerem Element größer als v wenigstens drei Elemente, die größer als v sind. (Abbildung 3.1 veranschaulicht die Situation für den Fall $N = 35$.) Also gibt es in der Ausgangsfolge wenigstens

$$3 \cdot (\lceil \frac{1}{2} \lceil \frac{N}{5} \rceil \rceil - 2) \geq \frac{3N}{10} - 6$$

Elemente, die kleiner als v sind, und ebenso viele Elemente, die größer als v sind.

Daraus folgt sofort, daß das Verfahren *Auswahl* im Schritt 5 in jedem Fall für höchstens $\lceil 7N/10 + 6 \rceil$ Elemente rekursiv aufgerufen werden muß. Die Median-of-median-Strategie sichert also, daß man nach der Aufteilung stets einen festen Bruchteil der noch zu betrachtenden Elemente ausschließen kann. Dennoch folgt die Linearität der Laufzeit des angegebenen Verfahrens zur Auswahl des i -kleinsten Elements noch nicht unmittelbar, da wir das Verfahren nicht nur im Schritt 5, sondern auch im Schritt 3 zur Bestimmung des Medians von $\lceil \frac{N}{5} \rceil$ Elementen rekursiv aufgerufen haben. Wir rufen das Verfahren also nicht nur einmal, sondern zweimal für einen jeweils unterschiedlichen Bruchteil der ursprünglich gegebenen Elemente auf. Daß die Laufzeit dennoch linear in N bleibt, sieht man folgendermaßen ein.

Sei wieder $T(N)$ die Anzahl der Schritte, die erforderlich ist, um das Element mit i -kleinstem Schlüssel unter N Elementen mit Hilfe des angegebenen Verfahrens zu finden. Dann liest man aus der Verfahrensbeschreibung sofort die folgende Rekursionsformel ab:

$$(*) \quad T(N) \leq T\left(\frac{N}{5}\right) + T\left(\frac{7}{10}N + 6\right) + a \cdot N,$$

mit einer Konstanten a .

Aus dieser Rekursionsformel läßt sich wie folgt eine obere Schranke für $T(N)$ ableiten. Wähle eine Konstante c so, daß $c \geq 80a$ und $c \geq T(N)/N$ für alle $N \leq 91$ gilt. Wir zeigen durch Induktion, daß $T(N) \leq cN$ gilt; dabei ist es ausreichend, den Induktionsschritt für $N > 91$ zu betrachten. In diesem Fall kann $T(N)$ abgeschätzt werden nach

$$\begin{aligned} T(N) &\leq c \cdot \left\lceil \frac{N}{5} \right\rceil + c \cdot \frac{7}{10}N + 6 + aN && \text{(nach Induktionsvoraussetzung, da} \\ &&& \frac{N}{5} \text{ und } \frac{7}{10}N + 6 \text{ für } N > 91 \\ &&& \text{echt kleiner als } N \text{ sind)} \\ &\leq c \cdot \frac{1}{5}N + c + c \cdot \frac{7}{10}N + 7c + aN \\ &= c \cdot \frac{9}{10}N + 8c + aN \\ &\leq c \cdot \frac{9}{10}N + 8c + \frac{1}{80}cN && \text{(nach Wahl von } c) \\ &= c \cdot \frac{73}{80}N + 8 \\ &\leq c \cdot N && \text{(wegen } N > 91) \end{aligned}$$

Wir halten fest:

Satz 3.1 *Das i -te Element in einer Folge von N Elementen kann in höchstens $O(N)$ Schritten gefunden werden.*

Dieser Satz ist von erheblichem prinzipiellem Interesse. Das zu seinem Beweis von uns angegebene *Auswahl*-Verfahren ist jedoch kaum von praktischem Wert, weil viele Sonderfälle berücksichtigt werden müssen, die das Verfahren für kleine N kompliziert machen; die asymptotisch geringe Laufzeit macht sich erst für sehr große N bemerkbar.

3.2 Suchen in sequentiell gespeicherten linearen Listen

Wir nehmen an, daß die Elemente der zu durchsuchenden Liste Komponenten eines wie folgt vereinbarten Arrays sind:

var a : **array** $[0 \dots \max N]$ **of** $item$;

Die gegebenen N Elemente sollen an den Positionen $1, \dots, N$ stehen, $N \leq \max N$; jedes Element $a[i]$ hat eine Schlüsselkomponente $a[i].key$.

3.2.1 Sequentielle Suche

Das einfachste Suchverfahren, das keinerlei weitere Voraussetzungen verlangt, ist die *sequentielle* oder *lineare* Suche. Wird ein Element mit gegebenem Schlüssel k gesucht, so durchlaufen wir alle Elemente des Arrays von vorn nach hinten oder umgekehrt und vergleichen den Schlüssel jedes Elements mit dem Suchschlüssel. Die Suche kann erfolgreich abgeschlossen werden, sobald ein Element mit diesem Schlüssel k gefunden wurde. Um nicht immer prüfen zu müssen, ob bereits alle Listenelemente inspiziert wurden, verwendet man üblicherweise einen Stopper an Position 0, der dafür sorgt, daß eine am Listenende beginnende Suche auf jeden Fall erfolgreich endet. Das führt zur folgenden programmtechnischen Realisierung des Verfahrens:

```
procedure sequentialsearch ( $k$  : integer);
{durchsucht  $a[1], \dots, a[N]$  nach Element mit Schlüssel  $k$ }
var  $i$  : integer;
begin
   $a[0].key := k$ ; {Stopper}
   $i := N + 1$ ;
  repeat
     $i := i - 1$ 
  until  $a[i].key = k$ ;
  if  $i \neq 0$ 
    then { $a[i]$  ist gesuchtes Element}
    else {es gibt kein Element mit Schlüssel  $k$ }
  end {sequentialsearch}
```

Es ist offensichtlich, daß das Verfahren im schlechtesten Fall $N + 1$ Schlüsselvergleiche für eine erfolglose Suche benötigt. Wenn man annimmt, daß jede Anordnung der N Schlüssel gleichwahrscheinlich ist, wird man erwarten können, daß eine erfolgreiche Suche im Mittel

$$\frac{1}{N} \sum_{i=1}^N i = \frac{N+1}{2}$$

Schlüsselvergleiche ausführt.

Natürlich könnte man dieses Suchverfahren leicht auch für verkettet gespeicherte lineare Listen entsprechend implementieren. Das Verfahren macht nämlich von der Möglichkeit des direkten Zugriffs auf ein Element über seine Position innerhalb der Liste keinen Gebrauch. Das ist bei allen folgenden Verfahren anders.

Darüberhinaus setzen wir für den Rest des Abschnitts 3.2 voraus, daß die Listenelemente nach aufsteigenden Schlüsselwerten sortiert vorliegen. Es gilt also

$$a[1].key \leq a[2].key \leq \dots \leq a[N].key.$$

3.2.2 Binäre Suche

Das *binäre Suchen* folgt der Divide-and-conquer-Strategie und kann am einfachsten rekursiv beschrieben werden:

Verfahren *binäres_Suchen* (L : Liste; k : Schlüssel);
 {sucht in der Liste L mit aufsteigend sortierten Schlüsseln nach Element mit Schlüssel k }

1. Falls L leer ist, endet die Suche erfolglos; sonst betrachte das Element $a[m]$ an der mittleren Position m in L .
2. Falls $k < a[m].key$, durchsuche die linke Teilliste $a[1], \dots, a[m-1]$ nach demselben Verfahren.
3. Falls $k > a[m].key$, durchsuche die rechte Teilliste $a[m+1], \dots, a[N]$ nach demselben Verfahren.
4. Sonst ist $k = a[m].key$ und das gesuchte Element gefunden.

Zur programmtechnischen Realisierung dieses Verfahrens ist es bequem, die Grenzen des zu durchsuchenden Bereichs explizit als Parameter einer rekursiven Prozedur mitzuführen.

```
procedure binsearch ( $l, r, k$  : integer);
{durchsucht  $a[l], \dots, a[r]$  nach einem Element mit Schlüssel  $k$ }
var  $m$  : integer;
begin
   $m := (l + r) \text{ div } 2$ ;
  if  $l > r$ 
  then {Liste leer, Suche endet erfolglos}
  else
    begin
      if  $k < a[m].key$ 
      then binsearch( $l, m - 1, k$ )
      else if  $k > a[m].key$ 
      then binsearch( $m + 1, r, k$ )
      else { $a[m].key = k$ ; Suche endet erfolgreich}
    end
  end
end
```

Ein Aufruf im Hauptprogramm der Form *binsearch* $(1, N, k)$ liefert dann das gewünschte Ergebnis.

Die angegebene programmtechnische Realisierung des binären Suchens ist natürlich nur eine von vielen Möglichkeiten. Wir geben als zweite Variante noch eine iterative Version an.

```
function binsearch (k : integer) : integer;
  {liefert den Index eines Elementes mit Schlüssel k im Bereich  $a[1], \dots,$ 
    $a[N]$ , falls es ein Element mit diesem Schlüssel gibt, und 0 sonst}
  var m, l, r : integer;
begin
  l := 1; r := N;
  repeat
    m := (l + r) div 2;
    if k < a[m].key
      then r := m - 1
      else l := m + 1
  until (k = a[m].key) or (l > r);
  if k = a[m].key
    then binsearch := m
    else binsearch := 0
end
```

Weil wir nach jedem Vergleich des Suchschlüssels *k* mit dem Schlüssel des mittleren Elementes des zu durchsuchenden Bereichs die Hälfte der noch zu betrachtenden Elemente ausschließen können, folgt unmittelbar, daß bei binärer Suche für erfolgreiche und erfolglose Suche in einem Array mit *N* Elementen niemals mehr als $\lceil \log_2(N+1) \rceil$ Schlüssel miteinander verglichen werden.

Zur Abschätzung des mittleren Suchaufwands belastet man üblicherweise das Inspizieren eines Schlüssels und die gegebenenfalls notwendige Entscheidung, in der linken oder rechten Hälfte weiterzusuchen, mit den Kosten 1. Die zum Wiederfinden eines Elements erforderlichen Kosten sind dann gleich der Zahl der ausgeführten Schlüsselvergleiche nur unter der Annahme, daß das Verfahren binäre Suche in einer Programmiersprache implementiert wird, die einen Vergleichsoperator mit drei möglichen Ausgängen besitzt. Man nimmt also an, daß man in einem Schritt feststellen kann, ob ein gesuchter Schlüssel gleich, kleiner oder größer als ein inspizierter Schlüssel ist. Man beachte, daß beispielsweise die iterative Pascal Version des Verfahrens binäre Suche jeweils zwei Schlüsselvergleiche für diese Feststellung benötigt.

Um den mittleren Suchaufwand des Verfahrens binäre Suche abschätzen zu können, nehmen wir an, daß $N = 2^n - 1$ ist, für passendes *n*. Dann erfordert das Wiederfinden des Elementes an der mittleren Position genau eine Kosteneinheit, das Wiederfinden der Elemente an der mittleren Position in der jeweils linken und rechten Hälfte genau zwei Kosteneinheiten, usw. Es werden also genau $(i+1)$ Kosteneinheiten benötigt, um eins von 2^i Elementen wiederzufinden, $i = 0, \dots, n-1$. Für $n = 3$, also $N = 2^3 - 1 = 7$, kann man diesen Zusammenhang durch Abbildung 3.2 veranschaulichen.

Damit ergibt sich für den mittleren Suchaufwand des binären Suchens:

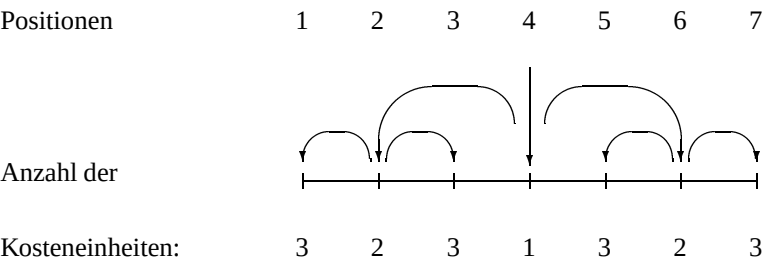


Abbildung 3.2

$$\begin{aligned} C_{mit}(N) &= \frac{1}{N}(\text{Gesamtkosten}) \\ &= \frac{1}{N} \sum_{i=0}^{n-1} (i+1) \cdot 2^i \\ &= \frac{1}{N} ((n-1) \cdot 2^n + 1) = \frac{1}{N} ((N+1) \log_2(N+1) - N) \\ &\approx \log_2(N+1) - 1, \text{ f\"ur gro\ss e } N. \end{aligned}$$

Im Mittel verursacht binäres Suchen also etwa eine Kosteneinheit weniger als im schlechtesten Fall.

3.2.3 Fibonacci-Suche

Ein dem binären Suchen analoges Suchverfahren ist die *Fibonacci-Suche*. Dieses Verfahren führt keine (ganzzahlige) Division zur Bestimmung der jeweils mittleren Position des Suchbereichs durch, sondern kommt mit Additionen und Subtraktionen aus. Anstatt den Suchbereich wie beim binären Suchen jeweils strikt in der Mitte zu teilen, nimmt man bei der Fibonacci-Suche eine Teilung entsprechend der Folge der Fibonacci-Zahlen vor, die wie folgt definiert sind:

$$F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2} \quad \text{für } (n \geq 2).$$

Nehmen wir nun der Einfachheit halber an, daß das zu durchsuchende Feld die Länge $F_n - 1$ hat, es sei also $N = F_n - 1$. Dann teilen wir den zu durchsuchenden Bereich entsprechend dem Paar der vorangehenden Fibonacci-Zahlen auf (vgl. Abbildung 3.3).

Die Fibonacci-Suche kann also folgendermaßen beschrieben werden: Vergleiche den Schlüssel des Elements an der Position $i = F_{n-2}$ mit dem gesuchten Schlüssel k . Falls $a[i].key > k$ ist, durchsuchen wir den linken (unteren) Bereich mit $F_{n-2} - 1$ Elementen auf dieselbe Weise. Falls $a[i].key < k$ ist, durchsuchen wir den rechten (oberen) Bereich

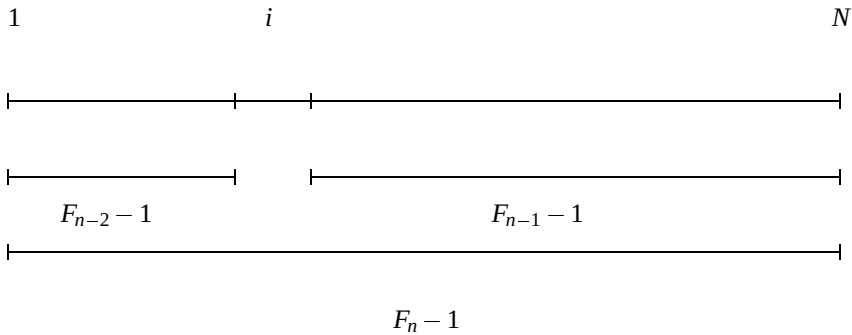


Abbildung 3.3

mit $F_{n-1} - 1$ Elementen auf dieselbe Weise. Ist $a[i].key = k$, so endet die Suche erfolgreich. Die Suche endet erfolglos, wenn der im Anschluß an einen Schlüsselvergleich zu durchsuchende Rest des Feldes leer geworden ist.

Um für einen Suchbereich mit Länge $F_j - 1$ die Position des nächsten zu betrachtenden Elements leicht finden zu können, merken wir uns zu jedem Suchbereich jeweils ein Paar von Fibonacci-Zahlen. Zu einem Bereich mit Länge $F_j - 1$ merken wir uns das Paar $(f_1, f_2) = (F_{j-3}, F_{j-2})$.

Nehmen wir also an, der zu durchsuchende Bereich habe die Länge $F_j - 1$ und wir kennen $(f_1, f_2) = (F_{j-3}, F_{j-2})$. Dann wird der Suchschlüssel k als nächstes mit dem Schlüssel des Elements an Position $i = f_2$ verglichen.

Falls $k > a[i].key$ ist, müssen wir rechts weitersuchen. Der zu durchsuchende Bereich hat dann die Länge $F_{j-1} - 1$. Er ist leer, falls $F_{j-1} = 1$ ist, was wir am Wert von f_1 leicht ablesen können. Sobald $f_1 = F_{j-3} = 0$ (und $F_{j-2} = 1$) geworden ist, ist $F_{j-1} = F_{j-3} + F_{j-2} = 1$. Als neues Paar von Fibonacci-Zahlen müssen wir uns bei nichtleerem Bereich das Paar

$$(f'_1, f'_2) = (F_{j-4}, F_{j-3})$$

merken, das man aus dem alten Paar (f_1, f_2) leicht wie folgt erhält:

$$(f'_1, f'_2) = (f_2 - f_1, f_1)$$

Falls $k < a[i].key$ ist, müssen wir links weitersuchen. Der zu durchsuchende Bereich hat dann die Länge $F_{j-2} - 1$. Er ist leer, falls $F_{j-2} = 1$, also $f_2 = 1$ geworden ist. Als neues Paar von Fibonacci-Zahlen müssen wir uns bei nichtleerem Bereich das Paar

$$(f'_1, f'_2) = (F_{j-5}, F_{j-4})$$

merken, das ebenfalls aus dem alten Paar (f_1, f_2) leicht berechnet werden kann.

In der folgenden programmtechnischen Realisierung des Verfahrens Fibonacci-Suche gehen wir davon aus, daß die Fibonacci-Zahlen F_n, F_{n-2}, F_{n-3} explizit gegeben sind, also etwa als Konstanten im Rahmenprogramm der Suchprozedur vereinbart wurden.

```

procedure fibsearch (k : integer);
var i, f1, f2, aux : integer;
    gefunden, nichtgefunden : boolean;
begin
    gefunden := false; nichtgefunden := false;
    f1 := Fn-3; f2 := Fn-2; i := f2;
    repeat
        if k > a[i].key {oberen Bereich durchsuchen}
        then
            if f1 = 0 {Suche beendet}
            then nichtgefunden := true
            else
                begin
                    i := i + f1;
                    aux := f1;
                    f1 := f2 - f1;
                    f2 := aux
                end
            else
                if k < a[i].key {unteren Bereich durchsuchen}
                then
                    if f2 = 1 {Suche beendet}
                    then nichtgefunden := true
                    else
                        begin
                            i := i - f1;
                            f2 := f2 - f1;
                            f1 := f1 - f2
                        end
                    else {k = a[i].key}
                        gefunden := true
                    until gefunden or nichtgefunden;
                    {Ausgabe von i, falls gefunden, sonst Fehlermeldung}
                end
            end
    end

```

Wieviele Schlüsselvergleiche werden bei der Suche nach einem Schlüssel k maximal ausgeführt? Ausgehend von einem Suchbereich mit Länge $F_j - 1$ ist die Länge des nächsten zu durchsuchenden Bereichs höchstens $F_{j-1} - 1$. Daher sind zum Durchsuchen eines Bereichs mit Anfangslänge $F_n - 1$ mit Hilfe von Fibonacci-Suche schlimmstenfalls n Schlüsselvergleiche erforderlich. Nun ist

$$\begin{aligned}
 F_n &= \frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2} \right)^n - \frac{1-\sqrt{5}}{2} \left(\frac{1-\sqrt{5}}{2} \right)^n = \frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2} \right)^n \\
 &\approx c \cdot 1.618^n, \text{ mit einer Konstanten } c.
 \end{aligned}$$

Für $N + 1 = c \cdot 1.618^n$ Elemente benötigt man also $O(n)$ Schlüsselvergleiche im schlechtesten Fall; d.h. die maximal erforderliche Anzahl von Schlüsselvergleichen ist

$$C_{\max}(N) = O(\log_{1.618}(N + 1)) = O(\log_2 N),$$

also von derselben Größenordnung wie beim binären Suchen. Man kann zeigen, daß auch die im Mittel ausgeführte Anzahl von Schlüsselvergleichen von dieser Größenordnung ist, vgl. dazu z.B. 

3.2.4 Exponentielle Suche

Binäre Suche und Fibonacci-Suche setzen voraus, daß man die Länge des zu untersuchenden Bereichs vor Beginn der Suche kennt. Es kann aber Fälle geben, in denen der Suchbereich zwar endlich, aber „praktisch“ unbegrenzt groß ist. In einem solchen Fall ist es vernünftig, zu einem gegebenen Suchschlüssel k zunächst eine obere Grenze für den zu durchsuchenden Bereich zu bestimmen, in dem ein Element mit Schlüssel k liegen muß, wenn es überhaupt ein solches Element gibt. Dieser Idee folgt die *exponentielle Suche*.

Um in einer Liste $a[1], \dots, a[N]$ mit sehr großem N ein Element mit Schlüssel k zu finden, bestimmen wir zunächst in exponentiell wachsenden Schritten einen Bereich, in dem ein solches Element liegen muß, wie folgt:

```
i := 1;
while k > a[i].key do i := i + i;
```

Für das auf diese Weise bestimmte i gilt dann

$$a[i/2].key < k \leq a[i].key.$$

(Dabei wird $a[0] = 0$ angenommen.) Es genügt also, diesen Bereich nach einem Element mit Schlüssel k zu durchsuchen. Weil wir vorausgesetzt haben, daß die Elemente aufsteigend sortierte, verschiedene positive ganzzahlige Schlüssel haben, wachsen die Schlüssel mindestens so stark wie die Indizes der Elemente. Daher wird i in der oben angegebenen **while**-Schleife maximal $\log_2 k$ mal, beginnend beim Anfangswert 1, verdoppelt. Das gesuchte i läßt sich also mit $\log_2 k$ Schlüsselvergleichen bestimmen. Ebenso ist klar, daß der Suchbereich

$$a[i/2], a[i/2 + 1], \dots, a[i]$$

maximal k Elemente enthalten kann. Durchsucht man diesen Bereich nun mit Hilfe des Verfahrens binäres Suchen oder Fibonacci-Suche, so werden nochmals $O(\log k)$ Schlüsselvergleiche ausgeführt. Exponentielle Suche erlaubt es also, in einer Folge von N Elementen mit aufsteigend sortierten Schlüsseln nach einem Element mit Schlüssel k stets in $O(\log k)$ Schritten erfolgreich oder erfolglos zu suchen. Das ist immer dann ein sinnvolles Verfahren, wenn k sehr klein im Vergleich zu N ist.

3.2.5 Interpolationssuche

Bei binärer Suche und Fibonacci-Suche hängt die Position des jeweils nächsten inspierten Elements nur von der Länge des Suchbereichs, nicht aber von den Werten der Schlüssel im Suchbereich ab. Aus dem täglichen Leben weiß man, daß das in manchen Fällen nicht sinnvoll ist. Man denke etwa daran, wie wir üblicherweise nach einem Namen in einem dicken Telefonbuch einer großen Stadt suchen. Suchen wir etwa nach dem Namen „Bayer“, werden wir das Buch weit vorne, suchen wir den Namen „Zimmermann“, werden wir es weit hinten aufschlagen. Wir schätzen also intuitiv die Position des Namens (des Suchschlüssels) aus dem Wert. Diese Idee führt zu einem Suchverfahren, das als *Interpolationssuche* bekannt ist. Man kann es am einfachsten als eine Variante des binären Suchens erklären. Beim binären Suchen haben wir als nächstes zu inspizierendes Element das Element mit Index m gewählt, wobei

$$m = l + \frac{1}{2}(r - l)$$

ist und l und r die linke und rechte Grenze des Suchbereichs bezeichnen. Bei der Interpolationssuche ersetzt man nun den Faktor $\frac{1}{2}$ durch eine geeignete Schätzung für die wahrscheinliche (oder erwartete) Position des Suchschlüssels k :

$$m = l + \frac{k - a[l].key}{a[r].key - a[l].key}(r - l)$$

Natürlich muß man m noch zur nächstkleineren oder -größeren Zahl runden. Es ist sofort klar, daß dies nur dann eine gute Schätzung für die Position des Suchschlüssels im Bereich $a[l], \dots, a[r]$ ist, wenn die Schlüsselwerte in diesem Bereich einigermaßen gleichverteilt sind. Man kann zeigen (vgl. z.B. [10]), daß Interpolationssuche im Mittel $\log_2 \log_2 N + 1$ Schlüsselvergleiche ausführt, wenn die N Schlüssel unabhängig und gleichverteilte Zufallszahlen sind. Man beachte aber, daß dieser Vorteil der geringen Anzahl von Schlüsselvergleichen durch die größere Komplexität der auszuführenden arithmetischen Operationen leicht wieder verloren geht. Außerdem benötigt Interpolationssuche im schlimmsten Fall linear viele Schlüsselvergleiche, im Unterschied zu allen anderen in Abschnitt 3.2 vorgestellten Suchverfahren, die Sortierung ausnutzen.

3.3 Selbstanordnende lineare Listen

Sind die Zugriffshäufigkeiten für die Elemente linearer Listen sehr unterschiedlich, kann es ratsam sein, die Elemente, auf die häufig zugegriffen wird, möglichst weit vorn und die Elemente, auf die selten zugegriffen wird, am Ende der Liste zu plazieren, und die Liste dann stets linear von vorn nach hinten zu durchsuchen. Leider kennt man aber oft die (relativen) Zugriffshäufigkeiten nicht im voraus, so daß man sie auch bei der Organisation von Listen nicht berücksichtigen kann. Man kann aber versuchen, nach jedem Zugriff auf ein Element die Liste so zu verändern, daß eine künftige Suche nach diesem Element schneller geht. Wir diskutieren in diesem Abschnitt die wichtigsten

Strategien zur Selbstanordnung von Listen, die dieses Ziel verfolgen. Die betrachteten Listen sind im allgemeinen nicht nach Schlüsselwerten sortiert und können sequentiell oder verkettet gespeichert vorliegen.

Folgende drei Strategien sind in der Literatur besonders ausführlich untersucht worden:

MF-Regel (Move-to-front): Mache ein Element zum ersten Element der Liste, nachdem auf das Element (als Ergebnis einer erfolgreichen Suche) zugegriffen wurde. Die relative Anordnung der übrigen Elemente bleibt unverändert.

T-Regel (Transpose): Vertausche ein Element mit dem unmittelbar vorangehenden, nachdem auf das Element zugegriffen wurde.

FC-Regel (Frequency Count): Ordne jedem Element einen Häufigkeitszähler zu, der anfangs 0 ist und die Anzahl der Zugriffe auf das Element speichert. Nach jedem Zugriff auf ein Element wird dessen Häufigkeitszähler um 1 erhöht. Ferner wird die Liste nach jedem Zugriff neu geordnet und zwar so, daß die Häufigkeitszähler der Elemente in absteigender Reihenfolge sind.

Die Wirkung dieser Regeln wird dann besonders klar, wenn die Zugriffshäufigkeiten der Elemente sehr unterschiedlich sind oder die Suchargumente in der Zugriffsfolge stark gebündelt auftreten. Zur Verdeutlichung betrachten wir folgendes Beispiel.

Gegeben sei die aufsteigend sortierte Liste von sieben Schlüsseln 1, 2, 3, 4, 5, 6, 7. Die erste Zugriffsfolge greift auf die Elemente in der Liste zehnmal nacheinander in der Reihenfolge 1, ..., 7 zu. Die zweite Zugriffsfolge greift zunächst zehnmal auf 1, dann zehnmal auf 2, usw. und schließlich zehnmal auf 7 zu.

In beiden Zugriffsfolgen wird auf jedes Element der gegebenen Liste zehnmal zugegriffen. Was sind die Kosten, wenn man auf die Elemente etwa nach der MF-Regel zugreift? Es ist üblich, als Kosten (oder Schrittzahl) für den Zugriff auf ein Element, das sich an Position i in der Liste befindet, i anzusetzen. Dann kann man die Kosten für beide Zugriffsfolgen leicht angeben.

Die ersten sieben Zugriffe der ersten Folge benötigen $\sum_{i=1}^7 i = \frac{7 \cdot 8}{2}$ Schritte. Danach befinden sich die sieben Schlüssel in der Anordnung 7, 6, 5, ..., 1. Jeder weitere Zugriff der ersten Folge benötigt jetzt genau sieben Schritte, weil das jeweils nächste gesuchte Element ganz am Listenende steht. Als durchschnittliche Kosten pro Zugriff der ersten Zugriffsfolge erhält man also:

$$\frac{\frac{7 \cdot 8}{2} + 7 \cdot 9 \cdot 7}{10 \cdot 7} = 6.7$$

In der zweiten Zugriffsfolge benötigt der $(10 \cdot i + 1)$ -te Zugriff jeweils $(i + 1)$ Schritte ($0 \leq i < 7$). Alle anderen Zugriffe benötigen nur einen Schritt, da sich das Element, auf das nach der MF-Regel zugegriffen wird, bereits am Listenanfang befindet. Es ergeben sich in diesem Fall also als durchschnittliche Kosten:

$$\frac{\sum_{i=1}^7 i + 9 \cdot 7 \cdot 1}{10 \cdot 7} = 1.3$$

Die relative Zugriffshäufigkeit ist in beiden Fällen für alle Schlüssel gleich. Vorabsortierung und statische Anordnung nach abnehmenden relativen Zugriffshäufigkeiten kann also nichts bringen. Die Liste kann irgendwie, muß aber fest angeordnet werden. Dann sind die durchschnittlichen Zugriffskosten $(10 \cdot \sum_{i=1}^7 i) / 70 = 4$.

Das zeigt, daß die MF-Regel zu geringeren durchschnittlichen Kosten führen kann als die „beste“ statische Anordnung. Dies ist insbesondere dann der Fall, wenn die Suchschlüssel in der Zugriffsfolge stark gebündelt auftreten.

Das Vorziehen eines Elements an den Listenanfang nach der MF-Regel ist natürlich eine sehr drastische Veränderung, die erst allmählich korrigiert wird, wenn ein „seltenes“ Element „irrtümlich“ an den Listenanfang gesetzt wurde und auf das Element dann lange nicht mehr zugegriffen wird. Die T-Regel ist in diesem Punkte vorsichtiger und macht entsprechend geringere Fehler; die häufig gesuchten Elemente wandern erst ganz allmählich an den Listenanfang. Man kann aber leicht Zugriffsfolgen angeben, so daß Zugriffe nach der T-Regel praktisch überhaupt nichts nützen: Man betrachte etwa eine Folge von Zugriffen, in der man immer wieder auf die letzten beiden Elemente N , $N - 1$, N , $N - 1$, ... der Liste $1, \dots, N$ zugreift. Jeder Zugriff verursacht die maximalen Kosten N .

Die FC-Regel sorgt dafür, daß nach jedem Zugriff die Listenelemente nach abnehmender Zugriffshäufigkeit geordnet sind. Diese Regel hat gegenüber den beiden anderen den schwerwiegenden Nachteil, daß man zusätzlichen Speicherplatz zur Aufnahme der Häufigkeitszähler bereitstellen muß. Falls man die Zugriffshäufigkeiten nicht ohnehin aus anderen Gründen mitführt (etwa, um eine Benutzerstatistik aufzustellen), lohnt die Verwendung der FC-Regel also nicht.

In der Literatur sind neben den genannten noch zahlreiche weitere Permutationsregeln zur Selbstanordnung von Listen vorgeschlagen worden. Eine gute Übersicht gibt

 Was ist die optimale Strategie? Offenbar ist diese Frage schon deshalb nicht leicht zu beantworten, weil eine allgemein akzeptierte, präzise Fassung des Optimalitätsbegriffs schwierig ist. Der Optimalitätsbegriff muß ja nicht nur unterschiedliche Zugriffshäufigkeiten, sondern auch Clusterungen von Zugriffsfolgen, die sogenannte *Lokalität*, berücksichtigen können. Daher findet man in der Literatur meistens nur asymptotische Aussagen über das erwartete Verhalten der Strategien zur Selbstanordnung für Zugriffsfolgen, die bestimmten Wahrscheinlichkeitsverteilungen genügen, Lokalität in Zugriffsfolgen bleibt unberücksichtigt. Besonders die MF-Regel ist in dieser Richtung intensiv untersucht worden. Es gibt ferner eine Reihe experimentell ermittelter Meßergebnisse für reale Daten. So berichten Bentley und McGeoch  Die T-Regel ist schlechter als die FC-Regel; die MF-Regel und die FC-Regel sind vergleichbar gut, die MF-Regel ist allerdings in manchen Fällen besser.

Man versucht also, die verschiedenen Strategien zur Selbstanordnung von Listen relativ zueinander zu beurteilen. Ein bemerkenswertes theoretisches Ergebnis in dieser Richtung, das das sehr gute, beobachtete Verhalten der MF-Regel untermauert, gelang Sleator und Tarjan . Zur Formulierung ihrer Aussage führen wir zunächst einige Bezeichnungen ein.

Wir denken uns eine Liste von N Elementen gegeben, auf der wir eine Folge s von m Zugriffsoperationen ausführen wollen. Verfahren A sei eine Strategie zur Selbstanordnung, also etwa die MF- oder T-Regel oder irgendeine andere. Mit $C_A(s)$ bezeichnen wir die gesamte Schrittzahl zur Ausführung aller Zugriffsoperationen der Folge s , beginnend mit der anfangs gegebenen Liste. Dabei nehmen wir an, daß der Zugriff auf ein Listenelement an Position i genau i Schritte benötigt; Vorziehen eines Elements, auf das zugegriffen wurde, an eine näher am Listenanfang befindliche Position kostet nichts. Die dazu erforderlichen Vertauschungen benachbarter Elemente nennen wir *ko-*

stenfreie Vertauschungen. Jede andere Vertauschung benachbarter Elemente heißt eine *zahlungspflichtige Vertauschung*; sie wird mit den Kosten 1 belastet. Wir betrachten also nur solche Algorithmen zur Selbstanordnung, die nach dem Zugriff auf ein Element dieses an eine andere Stelle bewegen und sonst alles fest lassen. Die Vertauschung des Elementes mit einem linken Nachbarn ist frei; jede Vertauschung mit einem rechten Nachbarn kostet eine Einheit.

$C_A(s)$ ist also die Gesamtzahl der Schritte zur Ausführung von s ohne zahlungspflichtige Vertauschungen. $F_A(s)$ bezeichne die Anzahl der kostenfreien und $X_A(s)$ die Anzahl der kostenpflichtigen Vertauschungen bei Ausführung von s mit Verfahren A . Für die MF-, T- und FC-Regel gilt natürlich:

$$X_{MF}(s) = X_T(s) = X_{FC}(s) = 0$$

Greift man auf ein Element an Position i zu, so kann man das Element anschließend maximal mit allen $(i - 1)$ vorangehenden Elementen kostenfrei vertauschen. Daher muß für jede Strategie A gelten: $F_A(s) \leq C_A(s) - m$. Nun gilt :

Satz 3.2 Für jeden Algorithmus A zur Selbstanordnung von Listen und für jede Folge s von m Zugriffsoperationen gilt

$$C_{MF}(s) \leq 2 \cdot C_A(s) + X_A(s) - F_A(s) - m.$$

Dieser Satz besagt grob, daß die MF-Regel höchstens doppelt so schlecht ist wie *jeder* andere Algorithmus zur Selbstanordnung von Listen. Die MF-Regel ist damit nicht wesentlich schlechter als die beste überhaupt denkbare Strategie. Selbst Vorkenntnisse über die Zugriffsverteilung können nicht viel nützen. Sleator und Tarjan  beweisen sogar ein noch etwas stärkeres Resultat, da sie auch Einfüge- und Streichoperationen in der Operationsfolge s zulassen.

Der Beweis des Satzes benutzt eine Technik, die als *Bankkonto-Paradigma* bekannt geworden ist. Es dient dazu, die durchschnittlichen Kosten pro Operation für eine beliebige Folge von Operationen nach oben hin abzuschätzen. Eine solche Abschätzung nennt man eine *amortisierte Worst-case-Analyse*. Würde man jede Einzeloperation einer beliebig gewählten Operationsfolge einfach durch die schlechtestenfalls mögliche Schrittzahl abschätzen, würde man im allgemeinen eine unrealistisch schlechte Abschätzung der für eine Folge von Operationen erforderlichen Schrittzahl erhalten. Denn in vielen Fällen benötigen nur sehr wenige Operationen einer ganzen Folge von Operationen den für eine Einzeloperation möglichen Maximalaufwand. Das Bankkonto-Paradigma ist eine Methode zur Ermittlung und Verteilung der anfallenden Gesamtkosten.

Wir ordnen daher jedem bei der Abarbeitung der Zugriffsfolge auftretenden Bearbeitungszustand einen Kontostand zu. Eine Einheit auf dem Konto repräsentiert gewissermaßen eine Kosteneinheit bei der Abschätzung der Gesamtkosten. Genauer: Seien eine Liste L , eine Folge s von m Zugriffsoperationen und ein Algorithmus A zur Ausführung gegeben. Wir wollen den Aufwand bei der Abarbeitung von s nach der MF-Regel $C_{MF}(s)$ mit dem Aufwand $C_A(s)$ bei Abarbeitung von s mit Hilfe von A vergleichen. Dazu lassen wir A und MF die Operationsfolge s gleichzeitig, parallel abarbeiten. Anfangs starten A und MF mit derselben Liste. Nach Ausführung jeder weiteren Operation sind die von A und MF erzeugten Listen im allgemeinen verschieden; dieses Paar von

Listen charakterisiert den bis dahin erreichten Bearbeitungszustand. Wir werden ihm einen Kontostand ϕ zuordnen.

Nun definieren wir als die *amortisierte Zeit* a_l zur Ausführung der l -ten Operation der Folge s die *wirkliche* Schrittzahl (Zeit) t_l zur Ausführung dieser Operation plus die Differenz $\phi_l - \phi_{l-1}$ der Kontostände. Dabei bedeutet ϕ_l den Kontostand nach Ausführung der l -ten Operation und ϕ_{l-1} den Kontostand vor Ausführung der l -ten Operation, also nach Ausführung der $(l-1)$ -ten Operation der Folge s . D.h. es ist

$$a_l = t_l + \phi_l - \phi_{l-1}, \text{ für } 1 \leq l \leq m.$$

ϕ_0 ist der Kontostand zu Beginn, d.h. vor Ausführung der Operationsfolge s . Damit gilt:

$$\begin{aligned} \sum_{l=1}^m a_l &= \sum_{l=1}^m t_l + \phi_m - \phi_0, \text{ also} \\ \sum_{l=1}^m t_l &= \sum_{l=1}^m a_l + \phi_0 - \phi_m \end{aligned}$$

Wir können also die gesamte (wirkliche) Schrittzahl zur Ausführung der m Operationen der Folge s nach oben abschätzen, wenn es uns gelingt, die amortisierten Kosten a_l für jedes l nach oben abzuschätzen, und wenn wir ϕ_0 und ϕ_m kennen.

Als ersten Schritt müssen wir also jedem Bearbeitungszustand einen Kontostand zuordnen. Bearbeitungszustände sind durch das Paar von Listen, also die erreichte Permutation von Elementen der Liste, charakterisiert, auf der die nächste Zugriffsooperation nach dem Verfahren A bzw. nach der MF-Regel operiert. Wir ordnen daher ganz allgemein zwei Listen L_1 und L_2 , die dieselben Elemente in unterschiedlicher Anordnung enthalten, einen *Kontostand* $bal(L_1, L_2)$ wie folgt zu:

$$bal(L_1, L_2) = \text{Anzahl der Inversionen von Elementen in } L_2 \text{ bzgl. } L_1$$

Dabei heißt ein Paar i, j von Elementen eine Inversion in L_2 bzgl. L_1 , wenn i in L_2 vor j und i in L_1 nach j auftritt.

Beispiel: Gegeben seien die zwei Listen

$$\begin{aligned} L_1 &: 4, 3, 5, 1, 7, 2, 6 \\ L_2 &: 3, 6, 2, 5, 1, 4, 7 \end{aligned}$$

Dann gilt in L_2 : 3 vor 4, 6 vor 2, 6 vor 5, 6 vor 1, 6 vor 4, 6 vor 7, 2 vor 5, 2 vor 1, 2 vor 4, 2 vor 7, 5 vor 4, 1 vor 4, aber in L_1 der Reihe nach jeweils die umgekehrte Relation; alle anderen Paare stehen in L_2 und L_1 in derselben Anordnung. Es ist also $bal(L_1, L_2) = 12$.

Wenn (i, j) eine Inversion von L_2 bzgl. L_1 ist, so ist (j, i) eine Inversion in L_1 bzgl. L_2 . Daher ist $bal(L_1, L_2) = bal(L_2, L_1)$, obwohl die Definition des Kontostandes für ein Paar von Listen asymmetrisch formuliert ist. Der Kontostand $bal(L_1, L_2)$ mißt, wieviele Elemente in L_2 „falsch“ stehen, wenn man die Reihenfolge der Elemente in L_1 als die „richtige“ ansieht. Deshalb kann man die Elemente in L_1 und L_2 auch so umnummerieren und umbenennen, daß in L_1 gerade 1, 2, 3, ..., N in dieser Reihenfolge auftreten und die Inversionszahl unverändert bleibt.

Wir erläutern dies für die beiden oben angegebenen Listen mit sieben Elementen. Das erste Element 4 in L_1 kommt an Position 6 in L_2 vor; das zweite Element 3 in L_1 kommt an Position 1 in L_2 vor, usw. Statt die Listen L_1 und L_2 zu betrachten, können wir also auch die folgenden nehmen:

$$\begin{aligned} L_1' &: 1, 2, 3, 4, 5, 6, 7 \\ L_2' &: 2, 7, 6, 3, 4, 1, 5 \end{aligned}$$

Es ist $bal(L_1, L_2) = bal(L_1', L_2')$, wie man leicht nachprüft.

Nun wollen wir die amortisierten Kosten a_l der l -ten Zugriffsoperation nach der MF-Regel durch die Zugriffskosten auf dasselbe Element nach der A-Regel abschätzen. Der Bearbeitungszustand vor Ausführung der Zugriffsoperation sei charakterisiert durch das Paar L_A und L_{MF} von Listen. Wir können annehmen, daß L_A die Liste $1, 2, \dots, N$ ist und auf das i -te Element i in L_A zugegriffen wird. Der Kontostand vor Ausführung der Zugriffsoptionen ist $bal(L_A, L_{MF})$. Sei k die Position, an der das Element i in der Liste L_{MF} auftritt. Diese Situation wird in Abbildung 3.4 veranschaulicht.

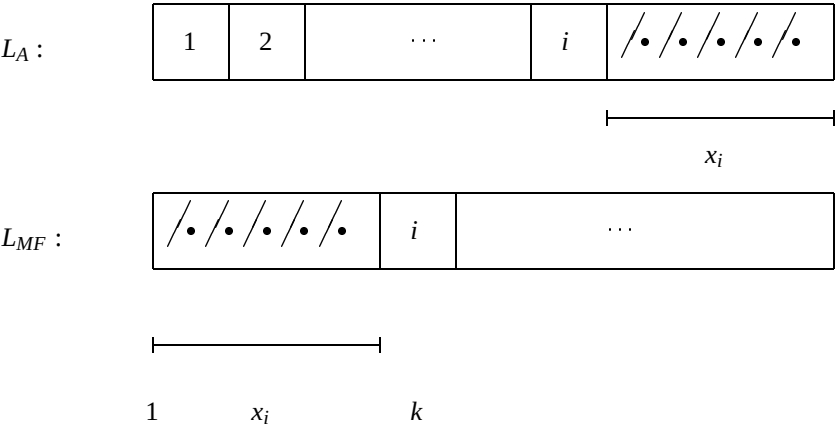


Abbildung 3.4

Sei x_i die Anzahl der Elemente, die i in der Liste L_{MF} vorangehen, aber i in der Liste L_A folgen. (Jedes dieser Elemente ist an einer Inversion in L_{MF} bzgl. L_A beteiligt.) Der Zugriff auf i nach der MF-Regel kostet $t_l = k$ Schritte; durch Vorziehen von i an den Listenanfang entsteht eine neue Liste L_{MF}' . Die Zahl der Inversionen in L_{MF}' bzgl. L_A nimmt offenbar um x_i ab und um genau $k - 1 - x_i$ Inversionen zu. Ein Zugriff auf i in L_A ohne Vertauschung kostet i Schritte. Jede kostenfreie Vertauschung von i mit einem i in L_A vorangehenden Element verringert die Anzahl der Inversionen in L_{MF}' bzgl. der veränderten Liste L_A um 1; mit anderen Worten, jedes Vorziehen von i in L_A um eine Position nach vorn bewirkt, daß es in L_{MF}' ein Element j weniger gibt, für das gilt: i geht in L_{MF}' j voran, aber i folgt in der veränderten L_A -Liste auf j . Genauso folgt, daß

jede kostenpflichtige Vertauschung in L_A , also jedes Nach-hinten-Schieben von i in L_A um eine Position eine weitere Inversion in L_{MF}' erzeugt. Führt die A-Regel also nach dem Zugriff auf i $F_A(i)$ kostenfreie oder $X_A(i)$ kostenpflichtige Vertauschungen durch, so entsteht eine neue Liste L_A' , und es gilt für den neuen Kontostand

$$bal(L_A', L_{MF}') = bal(L_A, L_{MF}) - x_i + (k - 1 - x_i) - F_A(i) + X_A(i).$$

Da die wirkliche Zeit t_i , um auf das Element i nach der MF-Regel zuzugreifen und es an den Listenanfang vorzuziehen, nach Annahme gleich k ist, erhält man als amortisierte Kosten a_i des Zugriffs auf i nach der MF-Regel:

$$\begin{aligned} a_i &= t_i + bal(L_A', L_{MF}') - bal(L_A, L_{MF}) \\ &= k - x_i + (k - 1 - x_i) - F_A(i) + X_A(i) \\ &= 2(k - x_i) - 1 - F_A(i) + X_A(i). \end{aligned}$$

Weil x_i die Anzahl der Elemente ist, die i in L_{MF} vorangehen, aber i in L_A folgen, ist $k - 1 - x_i$ die Anzahl der Elemente, die i in L_{MF} und in L_A vorangehen. Das können aber höchstens $i - 1$ sein. Daher ist $k - x_i \leq i$, und es folgt:

$$a_i \leq 2i - 1 - F_A(i) + X_A(i).$$

Da i die Zugriffskosten (ohne Vertauschungen) nach dem Verfahren A sind, folgt:

$$\sum_{l=1}^m a_l \leq 2C_A(s) - m - F_A(s) + X_A(s).$$

Weil das Bankkonto anfangs Null ist, $bal(L, L) = 0$, und das Bankkonto für die nach Ausführung aller m Operationen der Folge s entstehenden Listen L', L'' nicht negativ sein kann, folgt aus der letzten Abschätzung sofort die Behauptung des Satzes:

$$\begin{aligned} C_{MF}(s) &\leq \sum_{l=1}^m a_l + bal(L, L) - bal(L', L'') \\ &\leq 2C_A(s) + X_A(s) - F_A(s) - m \end{aligned}$$

□

Sleator und Tarjan zeigen, daß der Beweis dieses Satzes auf jede Heuristik zur Selbstanordnung von linearen Listen ausgedehnt werden kann, die verlangt, daß ein Element an Position k , auf das zugegriffen wurde, nach dem Zugriff um einen festen Bruchteil k/d an den Listenanfang gezogen wird.

3.4 Aufgaben

Aufgabe 3.1

Gegeben sei die Liste $L = 1, 2, 3, 4, 5, 6, 7$ und die Zugriffsfolge s mit 21 Zugriffen:

7, 2, 7, 3, 3, 7, 4, 4, 4, 7, 5, 5, 5, 5, 7, 6, 6, 6, 6, 6, 7

Vergleichen Sie das Verhalten der MF- und der T-Regel für diese Zugriffsfolge s , indem Sie das folgende Schema ergänzen:

nächstes Element von s	L_{MF}	Zugriffs- kosten nach MF-Regel	L_T	Zugriffs- kosten nach T-Regel	Konto- stand $bal(L_{MF}, L_T)$
—	1,2,3,4,5,6,7	—	1,2,3,4,5,6,7	—	0
7	7,1,2,3,4,5,6	7	1,2,3,4,5,7,6	7	5
.	.	.	.	.	.
.	.	.	.	.	.
.	.	.	.	.	.
	Gesamtkosten:		Gesamtkosten:		

Aufgabe 3.2

Zeigen Sie, daß der im Abschnitt 3.3 bewiesene Satz richtig bleibt, wenn die Operationsfolge s nicht nur Zugriffsoperationen, sondern auch Einfügungen und Streichungen von Elementen in Listen enthält. Um ein Element in eine Liste einzufügen, durchsucht man die ganze Liste vom Anfang bis zum Ende und fügt das Element als neues letztes Element in die Liste ein, wenn es in der Liste nicht schon vorkommt. Die Kosten, ein Element in eine Liste mit Länge i einzufügen, betragen also $i + 1$. Entfernen eines Elementes an Position i kostet i Schritte. Unmittelbar nach einer Einfüge- oder Zugriffsoperation können kostenfreie oder kostenpflichtige Vertauschungen vorgenommen werden.

Aufgabe 3.3

Gegeben sei das Feld a mit der folgenden Schlüssel-Belegung:

	1	2	3	4	5	6	7	8
$a :$	1	2	4	8	16	32	64	128

Man beschreibe die Suche nach dem Schlüssel 34 im obigen Feld a durch Angabe der Folge der ausgeführten Schlüsselvergleiche, wenn als Suchstrategie exponentielle Suche zur Eingrenzung des Suchbereichs mit anschließender linearer Suche angewandt wird.

Aufgabe 3.4

Gegeben sei eine sortierte Liste von 20 Elementen, die in einem Array mit Länge 20 sequentiell abgespeichert sei. Man gebe für jeden beliebigen Suchschlüssel k an, in welcher Reihenfolge die Schlüssel der Listenelemente mit k verglichen werden, wenn die Fibonacci-Suche als Suchverfahren verwendet wird. Dazu stelle man den der Fibonacci-Suche entsprechenden Suchbaum für eine Liste mit Länge 20 dar. Schließlich berechne man explizit die im Mittel beim Durchsuchen der Liste mit 20 Elementen mittels Fibonacci-Suche erforderliche Anzahl von Schlüsselvergleichen, wobei vorausgesetzt wird, daß die relative Zugriffshäufigkeit für alle Elemente gleich groß ist.

Aufgabe 3.5

Geben Sie für ein Paar V_1, V_2 von Suchverfahren aus Abschnitt 3.2 (sequentielle Suche, binäre Suche, Fibonacci-Suche, exponentielle Suche, Interpolationssuche) einen Suchschlüssel k und zwei Zahlenfolgen A_1 und A_2 an, so daß im schlimmsten Fall in A_1 die Suche nach k mit V_1 größenordnungsmäßig schneller ist als mit V_2 (in A_2 mit V_2 schneller ist als mit V_1), falls dies überhaupt möglich ist.

Literaturliste zu Kapitel 3: Suchen

Seite 159

[89] D. E. Knuth. The Art of Computer Programming, Vol.3: Sorting and Searching. Addison-Wesley, Reading, Massachusetts, 1973.

Seite 160

[199] A. C. Yao und F. F. Yao. The complexity of searching an ordered random table. In Proc. 17th Annual Symposium on Foundations of Computer Science, S. 173-177, 1976.

Seite 162

[76] J. H. Hester und D. S. Hirschberg. Self-organizing linear search. ACM Computing Surveys, 17:295-311, 1985.

[17] J. L. Bentley und C. McGeoch. Amortized analyses of self-organizing sequential search heuristics. Comm. ACM, 28:404-411, 1985.

[172] D. D. Sleator und R. E. Tarjan. Amortized efficiency of list update and paging rules. Comm. ACM, 28:202-208, 1985.

Seite 163

[172] D. D. Sleator und R. E. Tarjan. Amortized efficiency of list update and paging rules. Comm. ACM, 28:202-208, 1985.

Kapitel 4

Hashverfahren

In den Kapiteln 1 und 3 haben wir einige Methoden kennengelernt, die es erlauben, eine Menge von Datensätzen so zu speichern, daß die Operationen Suchen, Einfügen und Entfernen unterstützt werden. Jeder Datensatz ist dabei gekennzeichnet durch einen eindeutigen Schlüssel. Zu jedem Zeitpunkt ist lediglich eine (kleine) Teilmenge K aller möglichen Schlüssel K (englisch: keys) gespeichert. Statt nun bei der Suche nach einem Datensatz mit Schlüssel k mehrere Schlüsselvergleiche mit Schlüsseln aus K auszuführen, wird bei Hash-Verfahren versucht, durch eine *Berechnung* festzustellen, wo der Datensatz mit Schlüssel k gespeichert ist. Die Datensätze werden in einem linearen Feld mit Indizes $0, \dots, m-1$ gespeichert; dieses Feld nennt man die *Hashtabelle*, m ist die *Größe* der Hashtabelle. Eine Abbildung, die *Hashfunktion* $h : K \rightarrow \{0, \dots, m-1\}$ ordnet jedem Schlüssel k einen Index $h(k)$ mit $0 \leq h(k) \leq m-1$ zu, die *Hashadresse*. Im allgemeinen ist K eine sehr kleine Teilmenge von K ; so treten etwa in einem Pascal-Programm nur wenige der $\approx 26 \cdot 36^{79}$ zulässigen Namen auf (ein Name beginnt mit einem der 26 Buchstaben, danach kommen bis zu 79 weitere Buchstaben oder Ziffern, wenn eine Programmzeile bis zu 80 Zeichen lang ist). Die Hashfunktion kann also im allgemeinen nicht injektiv sein, sondern muß verschiedene Schlüssel auf dieselbe Hashadresse abbilden. Zwei Schlüssel k, k' mit $h(k) = h(k')$ heißen *Synonyme*; befinden sich beide Schlüssel in der aktuellen Schlüsselmenge K , so ergibt sich eine *Adreßkollision*. Treten in K keine Synonyme auf, so kann jeder Datensatz in der Hashtabelle an der seiner Hashadresse entsprechenden Stelle gespeichert werden. Bei Adreßkollisionen hingegen muß eine Sonderbehandlung vorgenommen werden.

Ein Hashverfahren muß also zwei Forderungen genügen. Erstens sollen möglichst wenige Kollisionen auftreten. Dies kann angestrebt werden durch die Wahl einer „guten“ Hashfunktion. Zweitens sollen Adreßkollisionen möglichst effizient aufgelöst werden.

Die Wahl der Hashfunktion werden wir im Abschnitt 4.1 diskutieren. Die Abschnitte 4.2 und 4.3 sind ganz der Diskussion von Strategien zur Kollisionsauflösung unter verschiedenen Annahmen gewidmet. Weil auch die beste Hashfunktion Kollisionen nicht ganz vermeiden kann, sind Hashverfahren im schlimmsten Fall sehr ineffiziente Realisierungen der Operationen Suchen, Einfügen und Entfernen; im Durchschnitt sind sie aber weitaus effizienter als Verfahren, die auf Schlüsselvergleichen basieren. So ist etwa die Zeit zum Suchen eines Schlüssels nicht abhängig von der Anzahl der gespei-

cherten Schlüssel, vorausgesetzt, daß genügend viel Speicherplatz zur Verfügung steht. Für eine Hashtabelle der Größe m , die gerade n Schlüssel speichert, nennen wir den Quotienten aus n und m , $\alpha = n/m$, den *Belegungsfaktor* der Tabelle. Die Anzahl der zum Suchen, Einfügen oder Entfernen eines Schlüssels benötigten Schritte hängt im wesentlichen vom Belegungsfaktor α ab. Dabei muß man bei manchen Hashverfahren annehmen, daß nur wenige Entferne-Operationen durchgeführt worden sind, weil diese auch im Mittel die Effizienz nachhaltig beeinträchtigen. Hashverfahren sind also gerade dann besonders effizient, wenn nach vielen anfänglichen Einfügeoperationen fast nur noch gesucht und fast nicht entfernt wird.

Im folgenden legen wir der Beschreibung der Hash-Verfahren die Definition der Hashtabelle als Feld von Datensätzen zugrunde:

```

const
     $m = \{\text{eine geeignete positive ganze Zahl}\};$ 
type
    datensatz = record
         $k : \text{key};$ 
         $\text{item} : \text{itemtype}$ 
    end;
     $\text{hashadresse} = 0 \dots m - 1;$ 
     $\text{hashtabelle} = \text{array} [\text{hashadresse}] \text{ of } \text{datensatz};$ 
var
     $t : \text{hashtabelle};$ 

```

Wegen der fest gewählten Größe der Hashtabelle sind die meisten der von uns präsentierten Verfahren nur halbdynamisch. Es können nie mehr als m Datensätze (kollisionsfrei, falls zusätzlicher Speicherplatz verwendet wird) gespeichert sein. Hiervon unterscheiden sich *dynamische Hashverfahren*, bei denen die Größe der Adreßtabelle (in Sprüngen) variabel ist; wir werden solche Verfahren in Abschnitt 4.4 behandeln. Im Abschnitt 4.5 schließlich präsentieren wir ein populäres Hashverfahren für mehrdimensionale Schlüssel, das ebenfalls dynamisch ist und sich für eine Realisierung auf Externspeichermedien mit Direktzugriff eignet, das *Gridfile*.

Bei der Analyse der Effizienz von Hashverfahren geht es uns in erster Linie um die durchschnittliche Laufzeit für die Operationen Suchen, Einfügen und Entfernen. Im schlimmsten Fall sind diese Operationen extrem langsam; dieser Fall ist leicht direkt aus der Beschreibung der Verfahren ableitbar. Wir werden stets zwei Erwartungswerte C_n und C'_n angeben, bezogen auf feste Tabellengröße m . Dabei ist C_n der Erwartungswert für die Anzahl der betrachteten Einträge der Hashtabelle bei *erfolgreicher* Suche, C'_n der Erwartungswert für die Anzahl der betrachteten Einträge der Hashtabelle bei *erfolgloser* Suche. Welche Wahrscheinlichkeitsverteilung unserer Rechnung zugrundeliegt, werden wir jeweils an Ort und Stelle erläutern.

Dem Entfernen eines Datensatzes muß stets eine erfolgreiche Suche vorausgehen; entsprechend ist der Aufwand für das Entfernen gerade C_n , wenn der betreffende Eintrag lediglich als entfernt markiert wird. Dem Einfügen eines Datensatzes muß stets eine erfolglose Suche vorausgehen; entsprechend ist der Einfüge-Aufwand gerade C'_n , wenn der betreffende Datensatz einfach an der ersten gefundenen freien Stelle eingetragen wird. Betrachten wir jedoch zunächst mögliche Hashfunktionen etwas genauer.

4.1 Zur Wahl der Hashfunktion

Eine gute Hashfunktion sollte möglichst leicht und schnell berechenbar sein und die zu speichernden Datensätze möglichst gleichmäßig auf den Speicherbereich verteilen, um Adreßkollisionen zu vermeiden.

Die von der Hashfunktion zu gegebenen Schlüsseln gelieferten Hashadressen sollten also über dem Adreßbereich gleichverteilt sein, und zwar selbst dann, wenn die Schlüssel aus K alles andere als gleichverteilt sind (etwa bei der Vorliebe von Programmierern für Namen wie $x, x1, x2, y1, y2, z1, z2$). Daß dennoch Adreßkollisionen selbst bei einer optimal gewählten Hashfunktion wahrscheinlich sind, zeigt das *Birthday Paradox*, vgl.  Wenn 23 Personen oder mehr in einem Raum sind, haben wahrscheinlich zwei davon am gleichen Tag des Jahres Geburtstag. Allgemeiner gilt: Wenn eine Hashfunktion $\sqrt{\pi m/2}$ Schlüssel auf eine Hashtabelle der Größe m abbildet, dann gibt es wahrscheinlich eine Adreßkollision (für $m = 365$ ist $\lfloor \sqrt{\pi m/2} \rfloor = 23$).

Wir werden im folgenden von nichtnegativen ganzzahligen Schlüsseln ausgehen, also $K \subseteq \mathbb{N}_0$ annehmen. Wenn Schlüssel zunächst als Zeichenfolgen gegeben sind (wie im Beispiel der Namen in Pascal-Programmen), so interpretieren wir die ihnen entsprechenden Bitfolgen einfach als positive ganze Zahlen, etwa im Dualsystem. Dann ist klar, daß eine Hashfunktion nicht nur gleichverteilte Schlüssel möglichst gleichmäßig auf den Adreßbereich streuen muß, sondern auch Häufungen (englisch: cluster) fast gleicher Schlüssel aufbrechen muß.

4.1.1 Die Divisions-Rest-Methode

Ein naheliegendes Verfahren zur Erzeugung einer Hashadresse $h(k)$, $0 \leq h(k) \leq m - 1$, zu gegebenem Schlüssel $k \in \mathbb{N}_0$ ist es, den Rest von k bei ganzzahliger Division durch m zu nehmen:

$$h(k) = k \bmod m$$

Dann ist allerdings eine gute Wahl von m entscheidend. Ist etwa m eine gerade Zahl, so ist $h(k)$ gerade, wenn k gerade ist; ist k ungerade, so ist auch $h(k)$ ungerade. Das ist für viele Schlüssel schlecht, z.B. dann, wenn die letzte Dualziffer einen Sachverhalt repräsentiert (0 = männlich, 1 = weiblich). Ebenfalls schlecht wäre die Wahl von m als Potenz der Basis des Zahlensystems, in dem Schlüssel dargestellt sind. So liefert etwa $m = 2^i$ die letzten i Bits der Dualdarstellung von k für $h(k)$; die restlichen Bits gehen überhaupt nicht in die Betrachtung ein. Ähnliche Argumente zeigen, daß m keine der Zahlen $r^i \pm j$, i und j kleine nichtnegative ganze Zahlen, teilen sollte, wobei r die Basis des Zahlensystems der Schlüssel ist. Eine gute Wahl ist die, m als Primzahl zu wählen, die keine solche Zahl $r^i \pm j$ teilt. Diese Wahl hat sich in praktisch allen Fällen ausgezeichnet bewährt (vgl. .

4.1.2 Die multiplikative Methode

Der gegebene Schlüssel wird mit einer irrationalen Zahl multipliziert; der ganzzahlige Anteil des Resultats wird abgeschnitten. Auf diese Weise erhält man für verschiedene Schlüssel verschiedene Werte zwischen 0 und 1; für Schlüssel $1, 2, 3, \dots, n$ sind diese Werte ziemlich gleichmäßig im Intervall $[0, 1)$ verstreut, wie ein Satz von Vera Turán Sós [10] (vgl. auch [11]) zeigt:

Sei Θ eine irrationale Zahl. Plaziert man die Punkte $\Theta - \lfloor \Theta \rfloor, 2\Theta - \lfloor 2\Theta \rfloor, 3\Theta - \lfloor 3\Theta \rfloor, \dots, n\Theta - \lfloor n\Theta \rfloor$ in das Intervall $[0, 1]$, dann haben die $n + 1$ Intervallteile höchstens drei verschiedene Längen. Außerdem fällt der nächste Punkt, $(n + 1)\Theta - \lfloor (n + 1)\Theta \rfloor$, in einen der größten Intervallteile.

Von allen Zahlen Θ , $0 \leq \Theta \leq 1$, führt der *goldene Schnitt*

$$\phi^{-1} = \frac{\sqrt{5} - 1}{2} \approx 0.6180339887$$

zur gleichmäßigsten Verteilung. Damit erhalten wir folgende Hashfunktion:

$$h(k) = \lfloor m(k\phi^{-1} - \lfloor k\phi^{-1} \rfloor) \rfloor$$

Insbesondere bilden die Werte $h(1), h(2), \dots, h(10)$ für $m = 10$ gerade eine Permutation der Zahlen $0, 1, \dots, 9$, nämlich $6, 2, 8, 4, 0, 7, 3, 9, 5, 1$. Der Leser kann sich selbst davon überzeugen, daß jede dieser Hashadressen, in der gegebenen Reihenfolge betrachtet, in ein größtes Intervall zwischen zwei bereits betrachteten Hashadressen fällt und dieses Intervall gemäß dem goldenen Schnitt teilt.

Man kann die Berechnung von $h(k)$ noch beschleunigen, wenn man ganze Zahlen im Rechner als Bruchzahlen mit Dezimalpunkt vor der höchstwertigen Ziffer ansieht, und wenn man für m eine Zweierpotenz wählt; dann läßt sich die Berechnung von $h(k)$ mit einer ganzzahligen Multiplikation und einer (oder zwei) Shift-Operation(en) vornehmen. Wir wollen dies hier nicht im einzelnen erläutern; der interessierte Leser sei verwiesen auf [12] oder [13].

Neben diesen beiden Methoden gibt es noch zahlreiche andere, die z.B. nach einer Transformation des Schlüssels (in ein anderes Zahlensystem oder durch Quadrieren oder durch Falten auf kurze Länge mit Verknüpfen von Teilstücken) einzelne Ziffernpositionen auswählen. Lum, Yuen und Dodd [14] haben das Verhalten einer Reihe verschiedener Hashfunktionstypen studiert. Dazu gehören das Divisions-Rest-Verfahren, die multiplikative Methode, die Ziffernanalyse, die Mid-square-Methode, die Faltung und die algebraische Verschlüsselung. Sie haben festgestellt, daß das Divisions-Rest-Verfahren im Durchschnitt die besten Resultate lieferte. Wir werden daher ab Abschnitt 4.2 stets eine nach dem Divisions-Rest-Verfahren arbeitende Hashfunktion verwenden, wenn wir Hashverfahren und damit Strategien zur Kollisionsauflösung beschreiben.

4.1.3 Perfektes und universelles Hashing

Ist die Anzahl der zu speichernden Schlüssel nicht größer als die Anzahl der zur Verfügung stehenden Speicherplätze, gilt also für die Teilmenge K der Menge K aller möglichen Schlüssel $|K| \leq m$, so ist eine kollisionsfreie Speicherung von K immer möglich. Wenn wir K kennen und K fest bleibt, können wir leicht eine injektive Abbildung $h : K \rightarrow \{0, \dots, m-1\}$ z.B. wie folgt berechnen: Wir ordnen die Schlüssel in K lexikographisch und bilden jeden Schlüssel auf seine Ordnungsnummer ab. Wir haben damit eine *perfekte* Hashfunktion, die Kollisionen gänzlich vermeidet. Eine solche Situation (K fest und vorher bekannt) liegt z.B. dann vor, wenn den Schlüsselworten einer Programmiersprache feste Plätze in einer Symboltabelle zugeordnet werden sollen. Dieser Fall ist aber eher die Ausnahme als die Regel. Im allgemeinen kennen wir $K \subseteq K$ nicht und können selbst dann, wenn $|K| \leq m$ bleibt, nicht sicher sein, daß Kollisionen vermieden werden.

Bleiben wir beim Beispiel der Verwaltung von Tabellen durch Compiler. Es könnte z.B. sein, daß eine vom Compiler fest gewählte Zuordnung von benutzerdefinierten Namen zu Plätzen in einer Symboltabelle auf besondere Vorlieben eines Programmierers für die Wahl von Namen keine Rücksicht nimmt und daher jedesmal zu vielen Kollisionen führt. Denn sobald die Hashfunktion fest gewählt ist, kann man stets viele Schlüssel finden, die sämtlich auf dieselbe Hashadresse abgebildet werden. Die einzige Möglichkeit, diese sehr unerwünschte Situation zu vermeiden, ist, die Hashfunktion zufällig aus einer sorgfältig gewählten Menge von Hashfunktionen auszuwählen. Statt anzunehmen, daß die aktuelle Schlüsselmenge K zufällig und gleichverteilt im Universum K aller möglichen Schlüssel gewählt wird, machen wir also eine wesentlich weniger kritische Annahme über das Hashverfahren: Wir nehmen an, daß die vom Verfahren benutzte Hashfunktion h zufällig und gleichverteilt aus einer Menge H möglicher Hashfunktionen gewählt wird. Die Auswahl von h ist Teil des Verfahrens und unterliegt, ganz anders als die Auswahl von $K \subseteq K$, nicht einer möglicherweise sehr einseitigen Vorliebe des Benutzers. Diese Art der Randomisierung garantiert daher (ganz ähnlich wie bei randomisiertem Quicksort), daß eine schlecht gewählte Schlüsselmenge K nicht jedesmal zu vielen Kollisionen führt: Zwar kann eine einzelne Funktion $h \in H$ noch immer viele Schlüssel aus K auf dieselbe Adresse abbilden. Gemittelt über alle Funktionen aus H ist das aber nicht mehr möglich.

Sei also H eine endliche Kollektion von Hashfunktionen, so daß jede Funktion aus H jeden Schlüssel im Universum K aller möglichen Schlüssel auf eine Hashadresse aus $\{0, \dots, m-1\}$ abbildet. H heißt *universell*, wenn für je zwei verschiedene Schlüssel $x, y \in K$ gilt:

$$\frac{|\{h \in H : h(x) = h(y)\}|}{|H|} \leq \frac{1}{m}$$

Mit anderen Worten, H ist universell, wenn für jedes Paar von zwei verschiedenen Schlüsseln höchstens der m -te Teil aller Funktionen der Klasse zu einer Adreßkollision für die Schlüssel des Paares führen.

Betrachten wir also ein beliebiges, festes Paar von zwei verschiedenen Schlüsseln x und y . Dann ist die Wahrscheinlichkeit dafür, daß x und y von einer zufällig aus H gewählten Funktion h auf dieselbe Hashadresse abgebildet werden, höchstens $1/m$. Denn höchstens $1/m$ der Funktionen aus H führen zu einer Adreßkollision bei x und y .

Wir definieren eine Funktion δ , die für zwei Schlüssel x und y aus K und eine Hashfunktion $h \in H$ anzeigt, ob eine Kollision vorliegt:

$$\delta(x, y, h) = \begin{cases} 1 & \text{falls } h(x) = h(y) \text{ und } x \neq y \\ 0 & \text{sonst} \end{cases}$$

Man kann δ wie folgt auf Mengen $Y \subseteq K$ von Schlüsseln und auf ganz H ausdehnen:

$$\begin{aligned} \delta(x, Y, h) &= \sum_{y \in Y} \delta(x, y, h) \\ \delta(x, y, H) &= \sum_{h \in H} \delta(x, y, h) \end{aligned}$$

Offenbar ist H universell, wenn für je zwei beliebige $x, y \in K$ mit $x \neq y$ gilt: $\delta(x, y, H) \leq |H|/m$.

Wir überlegen uns zunächst, welche Vorteile es hat, mit einer universellen Klasse H von Hashfunktionen zu arbeiten, bevor wir die Existenz solcher Klassen nachweisen. Nehmen wir an, wir wollen eine (vorher nicht bekannte) Folge von Schlüsseln aus dem Universum K aller möglichen Schlüssel in die Hashtabelle der Größe m , also auf eine der Adressen $\{0, \dots, m-1\}$ abbilden. Sei H eine universelle Klasse von Hashfunktionen $h: K \rightarrow \{0, \dots, m-1\}$. Dann wählen wir eine Funktion $h \in H$ zufällig aus und bilden mit ihr die Schlüssel der Reihe nach auf die Hashadressen ab. Die Hashfunktion bleibt also bei der ganzen Folge von Einfügungen fest. Soll ein Schlüssel x an der Stelle $h(x)$ gespeichert werden, so kann es natürlich sein, daß dieser Platz bereits besetzt ist. Nehmen wir an, daß zum Zeitpunkt des Einfügens von x in der Hashtabelle bereits die Menge S von Schlüsseln gespeichert ist und jeweils alle Schlüssel mit derselben Hashadresse in je einer linearen Liste zusammengefaßt werden. Es ist vernünftig, als Maß für den Aufwand zum Einfügen von x in die Hashtabelle die Anzahl der Elemente aus S zu nehmen, mit denen x kollidiert (Das wird im folgenden Abschnitt 4.2 genauer erläutert). Um diesen Aufwand abzuschätzen, berechnen wir den Erwartungswert $E[\delta(x, S, h)]$:

$$\begin{aligned} E[\delta(x, S, h)] &= \sum_{h \in H} \delta(x, S, h) / |H| \\ &= \frac{1}{|H|} \sum_{h \in H} \sum_{y \in S} \delta(x, y, h) \\ &= \frac{1}{|H|} \sum_{y \in S} \sum_{h \in H} \delta(x, y, h) \\ &= \frac{1}{|H|} \sum_{y \in S} \delta(x, y, H) \\ &\leq \frac{1}{|H|} \sum_{y \in S} |H|/m \\ &= |S|/m \end{aligned}$$

Man kann also erwarten, daß eine aus einer universellen Klasse H von Hashfunktionen zufällig gewählte Funktion h eine beliebige, noch so „einseitig“ gewählte Folge von Schlüsseln des Universums K so gleichmäßig wie nur möglich über die zur Verfügung stehenden Adressen verteilt.

Wir wollen jetzt zeigen, daß universelle Klassen von Hashfunktionen existieren und sogar relativ leicht konstruiert werden können. Dazu nehmen wir an, daß alle Schlüssel nichtnegative ganze Zahlen sind und $|K| = p$ eine Primzahl ist, d. h. wir setzen zur Vereinfachung $K = \{0, \dots, p-1\}$ voraus.

Für zwei beliebige Zahlen $a \in \{1, \dots, p-1\}$ und $b \in \{0, \dots, p-1\}$ sei die Funktion $h_{a,b} : K \rightarrow \{0, \dots, m-1\}$ wie folgt definiert:

$$h_{a,b}(x) = ((ax + b) \bmod p) \bmod m.$$

Dann gilt:

Satz 4.1 Die Klasse $H = \{h_{a,b} \mid 1 \leq a < p \text{ und } 0 \leq b < p\}$ ist eine universelle Klasse von Hashfunktionen.

Zum Beweis überlegen wir uns zunächst, daß für festes x und y , $0 \leq x, y < p$, $x \neq y$, die Zahlenpaare (r, q) mit $r = (ax + b) \bmod p$ und $q = (ay + b) \bmod p$ den gesamten möglichen Bereich aller Paare (r, q) mit $0 \leq r, q < p$ und $r \neq q$ durchlaufen, wenn a und b im gesamten Bereich $1 \leq a < p$ und $0 \leq b < p$ gewählt werden. Denn erstens ist für jedes zulässige a und b das Paar (r, q) mit $r = (ax + b) \bmod p$ und $q = (ay + b) \bmod p$ ein zulässiges Paar (r, q) mit $0 \leq r, q < p$ und $r \neq q$, da nach Voraussetzung $x \neq y$ und p prim ist. Zweitens gilt auch für jedes Paar (r, q) mit $0 \leq r, q < p$ und $r \neq q$, daß sich r und q schreiben lassen in der Form $r = (ax + b) \bmod p$ und $q = (ay + b) \bmod p$ mit geeignet gewählten a und b im zulässigen Bereich $1 \leq a < p$ und $0 \leq b < p$. Denn weil p eine Primzahl ist, kann im Ring der Restklassen von p das System von zwei linearen Gleichungen $r \equiv (ax + b) \bmod p$ und $q \equiv (ay + b) \bmod p$ eindeutig nach a und b aufgelöst werden.

Nun gilt für eine Funktion $h_{a,b} \in H$, daß sie x und y auf dieselbe Hashadresse abbildet, also $h_{a,b}(x) = h_{a,b}(y)$, genau dann, wenn $(ax + b) \equiv (ay + b) \bmod m$ ist. Um abzuschätzen, wieviele Funktionen aus H x und y auf dieselbe Adresse abbilden, genügt es also abzuschätzen, für wieviele Paare (q, r) mit $0 \leq q, r < p$ und $q \neq r$ die Zahlen q und r in dieselbe Restklasse modulo m fallen. Für festes q , $0 \leq q < p$, kann es offenbar höchstens $(p-1)/m$ Zahlen $r \neq q$ geben mit $q \equiv r \bmod m$. Damit gibt es unter den $p \cdot (p-1)$ Zahlenpaaren (q, r) mit $q \equiv (ax + b) \bmod p$ und $r \equiv (ay + b) \bmod p$, $1 \leq a < p$, $0 \leq b < p$, höchstens $p(p-1)/m$ viele, für die q und r in dieselbe Restklasse modulo m fallen. Also ist

$$|\{h \in H : h(x) = h(y)\}| \leq p \cdot (p-1)/m = |H|/m$$

und damit H universell. □

Der gerade bewiesene Satz legt die folgende Strategie zur Wahl einer Hashfunktion nahe. Nehmen wir an, wir wissen, wieviele Schlüssel auf einen gegebenen Bereich von m Adressen abgebildet werden. Dann wählen wir eine Primzahl p , die größer oder gleich der Zahl der Schlüssel ist, und wählen zwei Zahlen a und b zufällig im Bereich $1 \leq a < p$ und $0 \leq b < p$. Dann ist $h_{a,b}$ eine „gute“ Hashfunktion.

Man beachte, daß wir keinerlei Voraussetzungen über die Größe des Adreßbereichs gemacht haben. Es schadet also nicht, m beispielsweise als Zweierpotenz zu wählen. Klassen universeller Hashfunktionen wurden erstmals von Carter und Wegman in [EW78] vorgestellt. Eine Verallgemeinerung des Begriffs der universellen Klasse von Hashfunktionen und weitere Eigenschaften solcher Klassen findet man z.B. in [Weg85]. Dort werden auch Verfahren zur Konstruktion perfekter Hashfunktionen diskutiert, die effizienter sind als das von uns zu Anfang dieses Abschnitts angegebene naive Verfahren.

4.2 Hashverfahren mit Verkettung der Überläufer

Soll in eine Hashtabelle t , die bereits den Schlüssel k enthält, ein Synonym k' von k eingefügt werden, so ergibt sich eine Adreßkollision. Der Platz $h(k) = h(k')$ ist bereits besetzt, und k' , ein *Überläufer*, muß anderswo gespeichert werden. Eine einfache Art, Überläufer zu speichern, ist die, sie außerhalb der Hashtabelle abzulegen, und zwar in dynamisch veränderbaren Strukturen. So kann man etwa die Überläufer zu jeder Hashadresse in einer linearen Liste verketteten; diese Liste wird an den Hashtabelleneintrag angehängt, der sich durch Anwendung der Hashfunktion auf die Schlüssel ergibt.

Beispiel:

Größe der Hashtabelle $m = 7$;

$K = \{0, 1, \dots, 500\}$;

$h(k) = k \bmod m$;

wir zeigen hier nur die zu den Datensätzen gehörenden Schlüssel (nicht die ganzen Datensätze).

Nach Einfügen der Schlüssel 12, 53, 5, 15, 2, 19, 43 in dieser Reihenfolge in die anfangs leere Hashtabelle ergibt sich die in Abbildung 4.1 gezeigte Situation. Dabei haben wir Überläufer jeweils am Ende der aktuellen Überlaufkette angefügt.

Methoden: Separate Verkettung der Überläufer

Jedes Element der Hashtabelle t ist Anfangselement einer Überlaufkette (verkettete lineare Liste).

Suchen nach Schlüssel k : Beginne bei $t[h(k)]$ und folge den Verweisen der Überlaufkette, bis entweder k gefunden wurde (erfolgreiche Suche) oder das Ende der Überlaufkette erreicht ist (erfolglose Suche).

Einfügen eines Schlüssels k : Suche nach k ; die Suche verläuft erfolglos (sonst wird k nicht eingefügt) und endet am Ende einer Überlaufkette oder bei $t[h(k)]$. Im letzteren Fall trage k in $t[h(k)]$ ein; sonst erzeuge ein neues Listenelement und hänge es ans Ende der Überlaufkette an.

Entfernen eines Schlüssels k : Suche nach k ; die Suche verläuft erfolgreich (sonst kann k nicht entfernt werden). Steht k in der Hashtabelle, so streiche k dort; falls eine Überlaufkette bei $t[h(k)]$ beginnt, so übertrage das erste Element der Überlaufkette nach

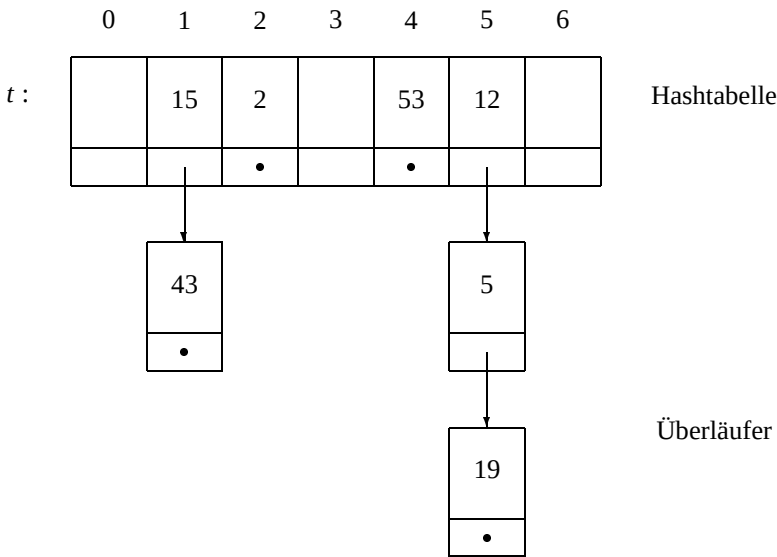


Abbildung 4.1

$t[h(k)]$ und entferne es aus der Überlaufkette. Steht k in einem Element der Überlaufkette, so entferne dieses Element aus der Überlaufkette.

Mit wenigen Modifikationen der zu Beginn dieses Kapitels angegebenen Definition der Hashtabelle lässt sich diese Methode leicht in Pascal-ähnlicher Notation beschreiben. Allerdings fällt auf, daß die unterschiedlichen Fälle (Schlüssel in Hashtabelle oder in Überlaufkette, gegebenenfalls Nachziehen des ersten Überläufers in die Hashtabelle beim Entfernen, usw.) einige Abfragen erfordern, die die Laufzeit der Operationen spürbar beeinträchtigen. Wenn man bereit ist, unter Umständen etwas Speicherplatz zu opfern, so kann man auch einfach *alle* Datensätze in den Überlaufketten speichern; in der Hashtabelle benötigt man dann nur Zeiger auf den Listenanfang.

Das obige Beispiel ist dann wie in Abbildung 4.2 darstellbar. Diese Methode ist als *direkte Verkettung* der Überläufer bekannt. Im Unterschied zur separaten Verkettung der Überläufer wird man hier weniger Speicherplatz benötigen, wenn Datensätze ziemlich groß sind, weil man bei direkter Verkettung bei leeren Hashtabellenplätzen nur wenig Speicherplatz ungenutzt läßt. Sind jedoch die Datensätze klein und über die Hashtabelle gleichmäßig verteilt, etwa ein Datensatz pro Hashadresse, so benötigt man bei direkter Verkettung der Überläufer natürlich mehr Speicherplatz, und zwar für die Anfangszeiger auf Überlaufketten.

Method: *Direkte Verkettung der Überläufer*

Jedes Element der Hashtaele ist ein Zeiger auf eine (Überlauf-) Kette.

Suchen nach Schlüssel k : Beginne bei $t[h(k)] \uparrow$ und folge den Verweisen der Überlaufkette, bis entweder k gefunden wurde (erfolgreiche Suche) oder das Ende der Überlaufkette erreicht ist (erfolglose Suche).

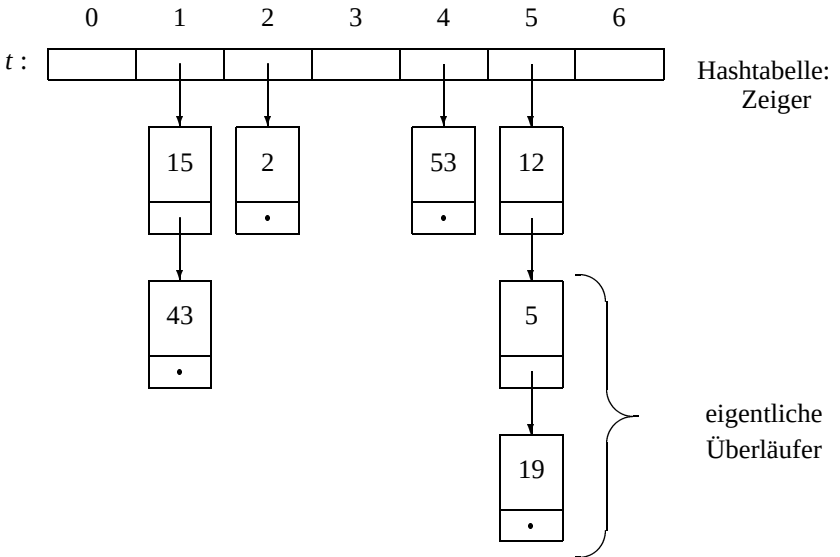


Abbildung 4.2

Einfügen eines Schlüssels k : Suche nach k ; die Suche endet erfolglos am Ende einer Überlaufkette (sonst wird k nicht eingefügt). Schaffe ein neues Listenelement und hänge es ans Ende der Überlaufkette an.

Entfernen eines Schlüssels k : Suche nach k ; die Suche verläuft erfolgreich (sonst kann k nicht entfernt werden) und endet bei einem Element der Überlaufkette. Entferne dieses Element aus der Überlaufkette.

Im wesentlichen handelt es sich hierbei also stets um Operationen in linearen verketteten Listen. Wir ergänzen die zu Beginn dieses Kapitels angegebenen Definitionen, damit wir die beschriebenen Prozeduren genauer angeben können.

type

$zeiger = \uparrow listenelement$;

$listenelement = \mathbf{record}$

$k : \text{key}$;

$item : \text{itemtype}$;

$next : zeiger$

end;

$hashtabledirekt = \mathbf{array} [\text{hashadresse}] \text{ of } zeiger$;

$listenoperation = (\text{suche}, \text{einfüge}, \text{entferne})$;

procedure operation ($op : listenoperation$; **var** $z : zeiger$;

var $ds : \text{datensatz}$);

{führt in Liste ab Zeiger z Operation op mit Datensatz ds aus}

begin

if $z = \text{nil}$

```

then {Listenende}
  case op of
    suche : write('Wert tritt nicht auf');
    entferne : write('Wert nicht in der Liste');
    einfüge : begin
      new(z);
       $z \uparrow .k := ds.k$ ;
       $z \uparrow .item := ds.item$ ;
       $z \uparrow .next := \text{nil}$ 
    end
  end
else
  if  $z \uparrow .k = ds.k$ 
    then
      case op of
        suche :  $ds.item := z \uparrow .item$ ; {liefere item in ds ab}
        entferne :  $z := z \uparrow .next$ ; {ändere Zeiger in der Liste}
        einfüge : write('ist bereits vorhanden')
      end
    else operation(op,  $z \uparrow .next$ , ds)
  end {operation}

```

Die Verwendung dieser Prozedur ist klar: Soll beispielsweise nach einem Datensatz mit Schlüssel k gesucht werden, so wird für eine Variable

var ds : datensatz

nach der Zuweisung

$ds.k := k$ {Suchschlüssel}

die Prozedur

operation (suche, $t[h(ds.k)]$, ds)

aufgerufen. Wenn ein Datensatz mit Schlüssel k gefunden wurde, so enthält $ds.item$ die entsprechende Information.

Analyse: Betrachten wir zunächst die Methode der direkten Verkettung der Überläufer. Wir nehmen an, daß die Hashfunktion alle Hashadressen mit gleicher Wahrscheinlichkeit (Gleichverteilung) und von Operation zu Operation unabhängig liefert; d h. die Wahrscheinlichkeit, daß bei der j -ten Operation die Adresse j' ausgewählt wird ($0 \leq j' \leq m - 1$), ist unabhängig von j stets gleich $1/m$, für alle j' .

Bei einer erfolglosen Suche nach k betrachten wir alle Einträge der bei $t[h(k)]$ beginnenden Überlaufkette. Die durchschnittliche Anzahl der Einträge in einer Kette ist gerade n/m , wenn n Einträge auf m Ketten verteilt sind. Da dies auch der Belegungsfaktor α ist, erhalten wir:

$$C'_n = \alpha$$

Ist bei einer erfolgreichen Suche k um i Listenelemente vom Listenanfang $t[h(k)]$ entfernt, so betrachten wir gerade diese i Einträge. Sehen wir uns die Schlüssel einmal in der Reihenfolge an, in der sie eingefügt worden sind. Beim Einfügen des j -ten Schlüssels ist die durchschnittliche Listenlänge gerade $(j-1)/m$. Also betrachten wir bei einer späteren Suche nach dem j -ten Schlüssel gerade $1 + (j-1)/m$ Einträge im Durchschnitt, wenn stets am Listenende eingefügt wird und kein Datensatz entfernt wurde. Im Mittel ist die Anzahl der bei der erfolgreichen Suche nach einem Schlüssel betrachteten Einträge also

$$C_n = \frac{1}{n} \sum_{j=1}^n (1 + (j-1)/m) = 1 + \frac{n-1}{2m} \approx 1 + \frac{\alpha}{2},$$

wenn nach jedem Schlüssel mit gleicher Wahrscheinlichkeit gesucht wird. Man beachte, daß diese und die folgenden Analysen das Entfernen von Schlüsseln nicht berücksichtigen.

Die Analyse der Effizienz der erfolglosen und erfolgreichen Suche bei separater Verkettung ist etwas komplizierter; sie kann nachgelesen werden bei  Wir geben hier nur das Resultat wieder:

$$C'_n \approx \alpha + e^{-\alpha}; \quad C_n \approx 1 + \frac{\alpha}{2}$$

Nach den angegebenen Methoden der direkten und separaten Verkettung der Überläufer ist klar, daß die Effizienz der erfolgreichen Suche auch gleichzeitig die Effizienz der Entferne-Operation ist, und daß das Einfügen gerade so effizient ist wie die erfolglose Suche.

Die Effizienz der erfolglosen Suche läßt sich verbessern, wenn man die Überlaufketten sortiert hält. Dann muß man beim erfolglosen Suchen nicht stets bis zum Listenende suchen, sondern kann im Mittel schon in der Mitte der Liste der Überläufer aufhören (man beachte, daß die erfolglose Suche für $\alpha \leq 1$ im Mittel schneller ist als die erfolgreiche). Diese Modifikation lohnt sich besonders bei häufiger erfolgloser Suche, also etwa am Anfang des Aufbaus der Hashtabelle und der Überlaufketten durch fortgesetztes Einfügen. In Fällen mit sehr begrenzter Dynamik, etwa in dem für die Anwendung von Hashverfahren typischen Fall vieler Einfügungen in einer Initialisierungsphase und vieler Suchanfragen danach, kann es attraktiver sein, die Effizienz der erfolgreichen Suche zu steigern, z.B. durch Verwendung selbstanordnender Listen (vgl. Kapitel 3).

Die direkte oder separate Verkettung der Überläufer weist einige wesentliche Vorzüge gegenüber anderen Hashverfahren auf, die wir im folgenden noch erläutern werden (vgl. Abschnitt 4.3). Der Erwartungswert (s.o.) und die Varianz für die Anzahl der betrachteten Einträge sind niedrig. Ein Belegungsfaktor von mehr als 1 ist möglich; d.h., selbst wenn die zu verwaltende Datenmenge mehr als vorgesehen wächst, so arbeitet das Verfahren noch korrekt. Echte Entfernungen von Einträgen sind möglich; eine Belastung von Speicher und Laufzeit durch als gelöscht markierte Einträge wird vermieden. Die direkte Verkettung der Überläufer eignet sich für den Einsatz mit Externspeichern; so

könnte man etwa die Hashtabelle im Internspeicher halten und Datenseiten verketten. Tabelle 4.1 vermittelt einen Eindruck von der Effizienz der Verkettung der Überläufer; die angegebenen Werte errechnen sich durch Einsetzen von α in die jeweilige Formel.

Anzahl bei der Suche betrachteter Einträge	separate Verkettung		direkte Verkettung	
	erfolgreich	erfolglos	erfolgreich	erfolglos
$\alpha = 0.50$	1.250	1.110	1.250	0.50
0.90	1.450	1.307	1.450	0.90
0.95	1.475	1.337	1.475	0.95
1.00	1.500	1.368	1.500	1.00

Tabelle 4.1

Die entscheidenden Nachteile der Methoden der Verkettung der Überläufer sind der Speicherplatzbedarf für die Zeiger und die Tatsache, daß selbst dann Platz für Überläufer außerhalb der Hashtabelle benötigt wird, wenn in der Hashtabelle noch viele Plätze frei sind. Andere Hashverfahren, die ohne zusätzlichen Speicherplatz auskommen, werden wir im nächsten Abschnitt präsentieren.

4.3 Offene Hashverfahren

Im Unterschied zur Verkettung der Überläufer *außerhalb* der Hashtabelle versucht man bei offenen Hashverfahren, Überläufer *in* der Hashtabelle unterzubringen. Wenn also beim Versuch, den Schlüssel k in die Hashtabelle an Position $h(k)$ einzutragen, festgestellt wird, daß $t[h(k)]$ bereits belegt ist, so muß man nach einer festen Regel einen anderen, nicht belegten Platz (eine *offene* Stelle) finden, an dem man k unterbringen kann. Da man von vornherein nicht wissen kann, welche Plätze belegt sein werden und welche nicht, definiert man für jeden Schlüssel eine Reihenfolge, in der alle Speicherplätze, und zwar einer nach dem anderen, betrachtet werden. Sobald ein betrachteter Platz frei ist, wird der Schlüssel dort gespeichert. Die Folge der zu betrachtenden Speicherplätze für einen Schlüssel nennt man die *Sondierungsfolge* zu diesem Schlüssel. Methoden, die diesem Schema folgen, hat W.W. Peterson 1957 [1] als *offene Hashverfahren* genannt. Von den zahlreichen Varianten offener Hashverfahren werden wir nur einige der wichtigsten erläutern; dabei geht es fast immer um die Wahl einer geeigneten Sondierungsfolge.

Natürlich ist das *Entfernen* von Schlüsseln bei all diesen Verfahren problematisch. Ein bereits in der Hashtabelle vorhandener Schlüssel k versperrt ja einem neu einzufügenden Schlüssel k' im allgemeinen einen Platz, den k' gemäß seiner Sondierungsfolge betrachtet. Der neue Schlüssel k' weicht also auf einen anderen Platz (später in der Sondierungsfolge) aus. Wird nun k entfernt, so kann k' nicht wiedergefunden werden, weil der leergewordene Platz von k in der Sondierungsfolge von k' vor dem aktuellen Platz von k' auftritt. In diesem Fall wird k bei den meisten Verfahren dann auch nicht wirklich entfernt, sondern lediglich als entfernt markiert. Wird ein neuer Schlüssel eingefügt, so wird der Platz von k als frei angesehen; wird ein Schlüssel gesucht, so wird der Platz von k als belegt angesehen. Der Effizienz von Hashverfahren ist diese Vorgehensweise nicht besonders zuträglich; für offene Hashverfahren gilt daher in besonderem Maße die Annahme, daß fast nur eingefügt und gesucht und fast nie entfernt wird.

Wir definieren nun ein Schema für offene Hashverfahren, das sich für die meisten (aber nicht alle) der offenen Hashverfahren als Grundlage eignet.

Method: *Offene Hashverfahren*

Sei $s(j, k)$ eine Funktion von j und k so, daß $(h(k) - s(j, k)) \bmod m$ für $j = 0, 1, \dots, m - 1$, eine Sondierungsfolge bildet, d.h. eine Permutation aller Hashadressen. Es sei stets noch mindestens ein Platz in der Hashtabelle frei.

Suchen nach Schlüssel k : Beginne mit Hashadresse $i = h(k)$. Solange k nicht in $t[i]$ gespeichert ist und $t[i]$ nicht frei ist, suche weiter bei $i = (h(k) - s(j, k)) \bmod m$, für aufsteigende Werte von j . Falls $t[i]$ belegt ist, wurde k gefunden; sonst war die Suche erfolglos.

Einfügen eines Schlüssels k : Wir nehmen an, daß k nicht schon in t vorkommt (das kann durch eine Suche festgestellt werden). Beginne mit Hashadresse $i = h(k)$. Solange $t[i]$ belegt ist, mache weiter bei $i = (h(k) - s(j, k)) \bmod m$, für steigende Werte von j . Trage k bei $t[i]$ ein.

Entfernen eines Schlüssels k : Suche nach Schlüssel k . Verläuft die Suche erfolgreich und ist i die Adresse, an der k gefunden wird, dann markiere $t[i]$ als entfernt; sonst kommt k nicht in t vor und kann auch nicht entfernt werden.

Es ist leicht, dieses Schema in ein Programmstück zu übersetzen, wenn wir voraussetzen, daß wir etwa über eine entsprechende Markierung feststellen können, ob ein Platz $t[i]$ frei, belegt oder als entfernt markiert ist. Wir verwenden dazu die Definitionen

type

zustand = (frei, belegt, entfernt);

markentabelle = **array** [*hashadresse*] **of** *zustand*;

var

marke : *markentabelle*

Anfangs sind alle Plätze frei. Der Wert von *marke*[i] gibt den Zustand des Platzes $t[i]$ an. Wir nehmen an, daß der **mod**-Operator zur Berechnung von Adressen gemäß der Sondierungsfolge wie beschrieben verwendet werden kann, daß also

$(h(ds.k) - s(j, ds.k)) \bmod m$ zyklisch bezüglich des Bereichs $0 \dots m - 1$ von $h(ds.k)$ aus um $s(j, ds.k)$ Positionen links liegt.

```
procedure Suchen (var ds: datensatz; var t: hashtabelle);
{sucht in der Hashtabelle t nach Datensatz mit Schlüssel ds.k und liefert
 ds.item oder eine Meldung über die Erfolglosigkeit der Suche}
var
  i, j : hashadresse;
begin
  j := 0; {Anzahl inspizierter Einträge}
  repeat
    i := (h(ds.k) - s(j, ds.k)) mod m;
    j := j + 1
  until (t[i].k = ds.k) or (marke[i] = frei);
  if marke[i] = belegt
    then ds.item := t[i].item
    else write('Suche erfolglos beendet')
end
```

```
procedure Einfügen (ds: datensatz; var t: hashtabelle);
{fügt Datensatz ds in Hashtabelle t ein}
var
  i, j : hashadresse;
begin
  j := 0; {Anzahl inspizierter Einträge}
  repeat
    i := (h(ds.k) - s(j, ds.k)) mod m;
    j := j + 1
  until marke[i] <> belegt;
  t[i] := ds;
  marke[i] := belegt
end
```

```
procedure Entfernen (k: key; var t: hashtabelle);
{entfernt Datensatz mit Schlüssel k aus der Hashtabelle t}
var
  i, j : hashadresse;
begin
  j := 0; {Anzahl inspizierter Einträge}
  repeat
    i := (h(k) - s(j, k)) mod m;
    j := j + 1
  until (t[i].k = k) or (marke[i] = frei);
  if marke[i] = belegt
    then marke[i] := entfernt {entferne}
    else write('Schlüssel nicht vorhanden')
end
```

Natürlich hätten auch die drei angegebenen Prozeduren wegen ihrer großen Ähnlichkeit zu einer einzigen vereinigt und entsprechend parametrisiert werden können; wir überlassen dies dem interessierten Leser. Um die Grundidee nicht zu verschleiern, haben wir bei den angegebenen Prozeduren keine Vorkehrungen gegen mehrfaches Einfügen des gleichen Schlüssels getroffen und nicht sichergestellt, daß wirklich immer ein freier Platz in der Hashtabelle existiert.

4.3.1 Lineares Sondieren

Beim linearen Sondieren ergibt sich für den Schlüssel k die Sondierungsfolge

$$h(k), h(k) - 1, h(k) - 2, \dots, 0, m - 1, \dots, h(k) + 1,$$

also die Sondierungsfunktion

$$s(j, k) = j.$$

Beispiel:

Größe der Hashtabelle $m = 7$;
 $K = \{0, 1, \dots, 500\}$;
 $h(k) = k \bmod m$;
 $s(j, k) = j$ (lineares Sondieren).

Dann führen die Schlüssel 12, 53, 5, 15, 2, 19, in dieser Reihenfolge in die leere Hashtabelle eingefügt, zu folgenden Situationen. Nach Einfügen von 12, 53:

	0	1	2	3	4	5	6
$t :$					53	12	

Nach Einfügen von 5: $h(5) = 5 \bmod 7 = 5$ ist belegt; der nächste Index der Sondierungsfolge ist 4, ebenfalls belegt; der nächste Index ist 3, nicht belegt:

	0	1	2	3	4	5	6
$t :$				5	53	12	

Nach Einfügen von 15, 2, 19 (Sondierungsfolge 5–4–3–2–1–0):

	0	1	2	3	4	5	6
$t :$	19	15	2	5	53	12	

Das lineare Sondieren (englisch: linear probing) ist zwar ein sehr einfaches Verfahren, hat aber auch einige Nachteile. Im gezeigten Beispiel etwa ist nach Einfügen von 12, 53, 5 die Wahrscheinlichkeit für einen neu einzufügenden Schlüssel, in der Hashtabelle an einer gewissen Hashadresse gespeichert zu werden, für die verschiedenen Hashadressen drastisch verschieden. Im Eintrag $t[2]$ werden alle Schlüssel k mit $h(k) = 2$ oder $h(k) = 3$ oder $h(k) = 4$ oder $h(k) = 5$ gespeichert, im Eintrag $t[1]$ dagegen nur alle Schlüssel k mit $h(k) = 1$. Bei einer uniformen Hashfunktion, die die Schlüssel mit gleicher Wahrscheinlichkeit auf jede der Hashadressen abbildet, hat in der beschriebenen Situation $t[2]$ die Chance $4/7$, mit dem nächsten Schlüssel belegt zu werden, während diese Chance für $t[1]$ nur $1/7$ beträgt. Lange belegte Teilstücke der Hashtabelle haben also eine stärkere Tendenz zu wachsen als kurze. Dieser Effekt wird noch verstärkt, weil lange belegte Teilstücke zu größeren zusammenwachsen (englisch: to coalesce), wenn die Lücken zwischen ihnen geschlossen werden. Als Folge dieses Phänomens der *primären Häufung* (englisch: *primary clustering*) verschlechtert sich die Effizienz des linearen Sondierens drastisch, sobald sich der Belegungsfaktor α dem Wert 1 nähert.

Analyse: Eine Analyse der Effizienz des linearen Sondierens  zeigt, daß für die durchschnittliche Anzahl der bei erfolgloser bzw. erfolgreicher Suche betrachteten Einträge C'_n bzw. C_n gilt:

$$C'_n \approx \frac{1}{2} \left(1 + \frac{1}{(1 - \alpha)^2} \right)$$
$$C_n \approx \frac{1}{2} \left(1 + \frac{1}{1 - \alpha} \right)$$

Tabelle 4.2 vermittelt durch Einsetzen einiger Werte für α in diese beiden Formeln einen Eindruck von der Effizienz des linearen Sondierens.

Anzahl betrachteter Einträge	lineares Sondieren	
	erfolgreich	erfolglos
$\alpha = 0.50$	1.5	2.5
0.90	5.5	50.5
0.95	10.5	200.5
1.00	—	—

Tabelle 4.2

4.3.2 Quadratisches Sondieren

Um die primäre Häufung des linearen Sondierens zu vermeiden, wird beim quadratischen Sondieren für Schlüssel k um $h(k)$ herum mit quadratisch wachsendem Abstand nach einem freien Platz gesucht. Die Sondierungsfolge für Schlüssel k ist

$$h(k), h(k) + 1, h(k) - 1, h(k) + 4, h(k) - 4, \dots$$

Die Sondierungsfunktion ist für die Sondierungsfolge $h(k) - s(j, k)$ definiert als

$$s(j, k) = (\lceil j/2 \rceil)^2 (-1)^j$$

Wenn m eine Primzahl der Form $4i + 3$ ist, dann ist garantiert, daß die Sondierungsfolge eine Permutation der Hashadressen 0 bis $m - 1$ ist, vgl. [1].

Beispiel: Füge 12, 53, 5, 15, 2, 19 in die anfangs leere Hashtabelle ein. Das ergibt nach Einfügen von 12, 53, 5 (Sondierungsfolge $h(5)$, $h(5) + 1$), 15, 2:

	0	1	2	3	4	5	6
$t :$		15	2		53	12	5

Nach Einfügen von 19 (Sondierungsfolge $h(19) = 5$, $5 + 1$, $5 - 1$, $(5 + 4) \bmod 7 = 2$, $5 - 4 = 1$, $(5 + 9) \bmod 7 = 0$):

	0	1	2	3	4	5	6
$t :$	19	15	2		53	12	5

Zwar ist hier die primäre Häufung vermieden, aber ein anderes Phänomen, die *sekundäre Häufung* (englisch: *secondary clustering*), beeinträchtigt die Effizienz: Zwei Synonyme k und k' durchlaufen stets dieselbe Sondierungsfolge, behindern sich also auf Ausweichplätzen. Das gilt natürlich ebenfalls für das lineare Sondieren. Im angegebenen Beispiel war das der Fall für die Schlüssel 5 und 19. Würde man ein weiteres Synonym von 5 (etwa 26) einfügen, so würden sowohl Schlüssel 5 als auch 19 den neu einzufügenden Schlüssel behindern.

Analyse: Eine Analyse der Effizienz des quadratischen Sondierens zeigt (vgl. [1]), daß für die durchschnittliche Anzahl der bei erfolgreicher bzw. erfolgloser Suche betrachteten Einträge C_n bzw. C'_n gilt:

$$C_n \approx 1 + \ln \left(\frac{1}{1 - \alpha} \right) - \frac{\alpha}{2}$$

$$C'_n \approx \frac{1}{1 - \alpha} - \alpha + \ln \left(\frac{1}{1 - \alpha} \right)$$

Tabelle 4.3 vermittelt durch einige in diese Formeln eingesetzte Werte für α einen Eindruck von der Effizienz des quadratischen Sondierens.

Anzahl betrachteter Einträge	quadratisches Sondieren	
	erfolgreich	erfolglos
$\alpha = 0.50$	1.44	2.19
0.90	2.85	11.40
0.95	3.52	22.05
1.00	—	—

Tabelle 4.3

4.3.3 Uniformes und zufälliges Sondieren

Die beim linearen und beim quadratischen Sondieren auftretenden Probleme der primären und sekundären Häufung liegen in der Unabhängigkeit der Sondierungsfunktion von k begründet. Die Sondierungsfolge ist für alle Synonyme die gleiche. Schlüssel werden sich natürlich weniger behindern, wenn die Sondierungsfolge auch für Synonyme variiert. Im Idealfall landen Schlüssel in rein zufällig gewählten Plätzen der Hashtabelle mit gleicher Wahrscheinlichkeit für jeden Platz. Diesen Fall realisiert das *uniforme Sondieren* (englisch: *uniform probing*). Hier ist die Folge $s(j, k)$ für $j = 0, 1, \dots, m - 1$ eine Permutation der Hashadressen, die nur von k abhängt, und zwar so, daß jede der $m!$ möglichen Permutationen mit gleicher Wahrscheinlichkeit verwendet wird. Man vermutet [1], daß uniformes Sondieren die Anzahl der Kollisionen beim Einfügen minimiert, also bezüglich des Einfügens optimal ist; die asymptotische Optimalität ist bereits bekannt [2]. Es ist jedoch sehr aufwendig, uniformes Sondieren praktisch zu realisieren; das *zufällige Sondieren* (englisch: *random probing*) bietet sich als Alternative mit fast gleicher Effizienz an. Hierbei wählt man, abhängig von k , eine zufällige Hashadresse für $s(j, k)$. Im Gegensatz zum uniformen Sondieren kann es also vorkommen, daß ein bereits für $s(j, k)$ gewählter Wert für $s(j', k)$, $j' > j$, nochmals gewählt wird, bevor alle Plätze der Hashtabelle betrachtet wurden. Wir werden noch sehen (vgl. Abschnitt 4.3.4), daß auch ein anderes, weniger aufwendig zu realisierendes Verfahren fast die gleiche Effizienz bietet wie das uniforme (und das zufällige) Sondieren.

Analyse: Wir betrachten hier nur die Effizienz des uniformen Sondierens; die des zufälligen Sondierens ist geringfügig schlechter. Jeder Schlüssel wird an einem zufällig gewählten Platz in der Hashtabelle abgespeichert, also ist jede der $\binom{m}{n}$ möglichen Belegungen der m Plätze durch n Schlüssel mit $m - n$ freien Plätzen gleichwahrscheinlich. Die Wahrscheinlichkeit (englisch: probability) p_i , daß genau i Plätze inspiziert werden müssen, um den $(n + 1)$ -ten Schlüssel einzufügen, ist die Anzahl der Situationen, in denen $i - 1$ bestimmte Plätze belegt sind und ein bestimmter Platz frei ist, bezogen auf die Anzahl aller Situationen mit n Schlüsseln, also

$$p_i = \binom{m - i}{n - i + 1} / \binom{m}{n},$$

da außer den $i - 1$ fest vorgegebenen noch $n - (i - 1)$ Schlüssel auf $m - i$ Plätze verteilt sind. Die durchschnittliche Anzahl betrachteter Plätze beim Einfügen, d h. bei erfolgloser Suche, ist dann

$$C'_n = \sum_{i=1}^m i \cdot p_i$$

$$\left\{ \text{mit } \sum_{i=1}^m p_i = 1 \right\}$$

$$\begin{aligned} &= m + 1 - \sum_{i=1}^m (m + 1) p_i - \sum_{i=1}^m (-i \cdot p_i) \\ &= m + 1 - \sum_{i=1}^m (m + 1 - i) p_i \end{aligned}$$

$$\left\{ \text{mit } \binom{a}{b} = \binom{a}{a-b} \right\}$$

$$= m + 1 - \sum_{i=1}^m (m + 1 - i) \binom{m-i}{m-n-1} / \binom{m}{n}$$

$$\left\{ \text{mit } \binom{a}{b} = \frac{b+1}{a+1} \binom{a+1}{b+1} \right\}$$

$$\begin{aligned} &= m + 1 - \sum_{i=1}^m (m + 1 - i) \frac{(m-n)}{(m-i+1)} \binom{m-i+1}{m-n} / \binom{m}{n} \\ &= m + 1 - (m-n) \sum_{i=1}^m \binom{m-i+1}{m-n} / \binom{m}{n} \end{aligned}$$

$$\left\{ \text{mit } \sum_{c=1}^a \binom{c}{b} = \binom{a+1}{b+1} \right\}$$

$$\begin{aligned} &= m + 1 - (m-n) \binom{m+1}{m-n+1} / \binom{m}{n} \\ &= m + 1 - (m-n) \frac{(m+1)}{(m-n+1)} \\ &= \frac{(m+1)}{(m-n+1)} \\ &\approx \frac{m}{m-n} = \frac{1}{1-\alpha} = 1 + \alpha + \alpha^2 + \alpha^3 + \dots \end{aligned}$$

Die Potenzreihe für C'_n kann intuitiv interpretiert werden: Mit Wahrscheinlichkeit 1 muß mindestens ein Platz inspiziert werden, mit Wahrscheinlichkeit α muß mehr als ein Platz inspiziert werden, mit Wahrscheinlichkeit α^2 müssen mehr als zwei Plätze inspiziert werden, usw.

Die durchschnittliche Anzahl der inspizierten Plätze bei erfolgreicher Suche ist bei offenen Hashverfahren

$$C_n = \frac{1}{n} \sum_{i=0}^{n-1} C'_i,$$

also

$$\begin{aligned} C_n &= \frac{1}{n} \sum_{i=0}^{n-1} \frac{m+1}{m-i+1} \\ &= \frac{m+1}{n} \left(\frac{1}{m+1} + \frac{1}{m} + \dots + \frac{1}{m-n+2} \right) \end{aligned}$$

$$\left\{ \text{mit } H_n = \sum_{i=1}^n \frac{1}{i}, \text{ die } n\text{-te harmonische Zahl} \right\}$$

$$= \frac{m+1}{n} (H_{m+1} - H_{m-n+1})$$

$$\left\{ \text{mit } H_n \approx \ln n + \gamma \left(+\frac{1}{2n} - \frac{1}{12n^2} + \dots \right), \text{ wobei } \gamma = 0.57\dots \right\}$$

$$\approx \frac{1}{\alpha} (\ln(m+1) + \gamma - \ln(m-n+1) - \gamma)$$

$$= \frac{1}{\alpha} \ln \left(\frac{m+1}{m-n+1} \right)$$

$$\approx \frac{1}{\alpha} \ln \left(\frac{1}{1-\alpha} \right)$$

Zusammenfassend gilt also für uniformes und näherungsweise auch für zufälliges Sondieren:

$$C'_n \approx \frac{1}{1-\alpha}$$

$$C_n \approx \frac{1}{\alpha} \ln \left(\frac{1}{1-\alpha} \right)$$

Tabelle 4.4 vermittelt durch einige in diese Formeln eingesetzten Werte für α einen Eindruck von der Effizienz des uniformen Sondierens.

Anzahl betrachteter Einträge	uniformes Sondieren	
	erfolgreich	erfolglos
$\alpha = 0.50$	1.39	2
0.90	2.56	10
0.95	3.15	20
1.00	—	—

Tabelle 4.4

4.3.4 Double Hashing

Die Effizienz des uniformen Sondierens wird bereits annähernd erreicht, wenn man statt einer zufälligen Permutation für die Sondierungsfolge eine zweite Hashfunktion verwendet; die gewählte Sondierungsfolge für Schlüssel k ist

$$h(k), h(k) - h'(k), h(k) - 2 \cdot h'(k), \dots, h(k) - (m - 1)h'(k),$$

jeweils modulo m , wenn $h'(k)$ die zweite Hashfunktion bezeichnet. Für die Sondierungsfunktion ergibt sich

$$s(j, k) = j \cdot h'(k).$$

Dabei muß $h'(k)$ so gewählt werden, daß für alle Schlüssel k die Sondierungsfolge eine Permutation der Hashadressen bildet. Das bedeutet, daß $h'(k) \neq 0$ sein muß und m nicht teilen darf. Wählen wir m als Primzahl, dann gilt dies sicher für jedes $h'(k)$ und für alle k ; diese Wahl von m ist ja auch günstig für die Divisions-Rest-Methode bei der Hashfunktion h . Wählt man $h'(k)$ abhängig von $h(k)$, so werden manche (oder gar alle) Synonyme die gleiche Sondierungsfolge haben; eine gewisse sekundäre Häufung ist die Folge. Das kann man vermeiden, wenn man $h'(k)$ von $h(k)$ unabhängig wählt, wenn also für zwei verschiedene Schlüssel k und k' gilt:

$$p[h(k) = h(k') \text{ und } h'(k) = h'(k')] = p[h(k) = h(k')] \cdot p[h'(k) = h'(k')],$$

wobei $p[\text{Bedingung}]$ die Wahrscheinlichkeit dafür ist, daß die angegebene Bedingung gilt. Anders ausgedrückt heißt das: Sind zwei Schlüssel Synonyme bezüglich h , so sind sie mit Wahrscheinlichkeit $1/m'$ Synonyme bezüglich h' (m' ist die Anzahl der Werte, die die Funktion h' annehmen kann); also sind zwei Schlüssel mit Wahrscheinlichkeit $1/(m \cdot m')$ Synonyme bezüglich h und h' gleichzeitig.

Ist m eine Primzahl und $h(k) = k \bmod m$, so erfüllt $h'(k) = 1 + k \bmod (m - 2)$ die obigen Anforderungen (das ist besser als $1 + k \bmod (m - 1)$, weil $m - 1$ gerade ist).

Beispiel: Wir betrachten wieder den Fall, daß die Schlüssel 12, 53, 5, 15, 2, 19 in die anfangs leere Hashtabelle der Größe 7 eingefügt werden sollen. Wir wählen also $h(k) = k \bmod 7$ und $h'(k) = 1 + k \bmod 5$. Die Sondierungsfolge für k ist $h(k), h(k) - h'(k), h(k) - 2h'(k)$, jeweils modulo m . Das ergibt nach Einfügen von 12, 53:

	0	1	2	3	4	5	6
t :					53	12	

Nach Einfügen von 5 (Sondierungsfolge ist $h(5) = 5 \bmod 7 = 5$, $5 - (1 + 5 \bmod 5) = 4$, $5 - 2 = 3$), 15, 2:

	0	1	2	3	4	5	6
t :		15	2	5	53	12	

Nach Einfügen von 19 (Sondierungsfolge ist $h(19) = 19 \bmod 7 = 5$, $5 - (1 + 19 \bmod 5) = 0$):

	0	1	2	3	4	5	6
t :	19	15	2	5	53	12	

Beim Einfügen des Schlüssels 19 müssen hier also lediglich zwei Plätze (nämlich $t[5]$ und $t[0]$) inspiziert werden, während es beim linearen und beim quadratischen Sondieren jeweils sechs Plätze waren.

Double Hashing ist genauso effizient wie uniformes Sondieren; der theoretische Unterschied ist minimal, wenn $h'(k)$ unabhängig von $h(k)$ gewählt wird. Da Double Hashing ein leicht implementierbares Verfahren ist, bietet es sich als praktisch einsetzbares offenes Hashverfahren an. Entsprechend ist es die Grundlage für zwei Methoden, bei denen versucht wird, auf Kosten der Einfügezeit die Effizienz der erfolgreichen Suche zu verbessern.

Verbesserung der erfolgreichen Suche

Methoden zur Verbesserung der erfolgreichen Suche basieren auf der Erkenntnis, daß die durchschnittliche Suchzeit für erfolgreiche Suche bei Hashverfahren ohne Häufung mit unterschiedlicher Reihenfolge des Einfügens der Schlüssel variiert. So ist etwa die durchschnittliche Suchzeit im gerade betrachteten Beispiel für die erfolgreiche Suche $(\text{Suchzeit}(12) + \text{Suchzeit}(53) + \text{Suchzeit}(5) + \text{Suchzeit}(15) + \text{Suchzeit}(2) + \text{Suchzeit}(19))/6 = (1 + 1 + 3 + 1 + 1 + 2)/6 = 1.5$. Fügt man die Schlüssel jedoch in der Reihenfolge 53, 5, 15, 2, 19, 12 ein, so ergibt sich die Situation

	0	1	2	3	4	5	6
t :	19	15	2		53	5	12

und damit eine durchschnittliche erfolgreiche Suchzeit von $8/6 = 1.33 \dots$. Für die Einfügereihenfolge 12, 5, 19, 53, 2, 15 ergibt sich die durchschnittliche erfolgreiche Suchzeit zu $10/6 = 1.66 \dots$.

In Fällen, in denen wesentlich häufiger erfolgreich gesucht wird als eingefügt, kann es daher lohnend sein, die Schlüssel beim Einfügen eines neuen Schlüssels so zu reorganisieren, daß die Suchzeit verkürzt wird. So berichten etwa Bell und Kaman , daß ein COBOL-Compiler beim Übersetzen 735 Einträge in eine Symboltabelle vorgenommen und diese Tabelle 10988 Mal angesprochen hat; das sind etwa 14 Suchoperationen pro Einfügung.

Brents Algorithmus

Betrachten wir in unserem Beispiel des Einfügens der Schlüssel 12, 53, 5, 15, 2, 19 die Operation des Einfügens von Schlüssel 5. In der Situation

	0	1	2	3	4	5	6
$t :$					53	12	

wird Schlüssel 5 nach Inspektion der Plätze 5, 4, 3 bei $t[3]$ eingetragen:

	0	1	2	3	4	5	6
$t :$				5	53	12	

Die durchschnittliche Suchzeit ist damit $(1 + 1 + 3)/3 = 5/3 = 1.66 \dots$. Die Adreßkollision von Schlüssel 5 mit Schlüssel 12 in $t[5]$ hätte man aber auch anders lösen können. Statt Schlüssel 12 in $t[5]$ zu belassen, hätte man Schlüssel 5 in $t[5]$ eintragen können und Schlüssel 12 weiter sondieren lassen können. Die Sondierungsfolge für Schlüssel 12 wäre dann $12 \bmod 7 = 5$, $5 - (1 + 12 \bmod 5) = 2$ und die Situation damit

	0	1	2	3	4	5	6
$t :$			12		53	5	

mit einer durchschnittlichen Suchzeit für die erfolgreiche Suche von $(1 + 1 + 2)/3 = 4/3 = 1.33 \dots$. Die entstandene Situation entspricht gerade derjenigen, die sich beim Einfügen von 5, 53, 12 ergibt. Es ist also eine für die erfolgreiche Suche günstige Einfügereihenfolge simuliert worden.

Methode: Brents Algorithmus

Einfügen eines Schlüssels k : Beginne mit Hashadresse $i = h(k)$. Solange $t[i]$ belegt ist, betrachte die beiden Hashadressen $b = (i - h'(k)) \bmod m$ und $b' = (i - h'(k')) \bmod m$ mit $k' = t[i]$. Ist $t[b]$ frei oder $t[b']$ belegt, fahre fort mit $i = b$; andernfalls trage k an Hashadresse i ein und fahre fort mit $k = k'$ und $i = b'$. Jetzt ist $t[i]$ frei; trage k bei $t[i]$ ein.


```

procedure BrentEinfügen (ds: datensatz; var t: hashtabelle);
{fügt Datensatz ds in Hashtabelle t ein}
var
    i, b, bb : hashadresse;
begin
    i := h(ds.k);
    while marke[i] = belegt do
        begin
            b := (i - h'(ds.k)) mod m;
            bb := (i - h'(t[i].k)) mod m;
            if (marke[b] = frei) or (marke[bb] = belegt)
                then i := b
            else begin
                vertausche (ds, t[i]);
                i := bb;
            end;
        end;
    t[i] := ds;
    marke[i] := belegt
end

```

Brents Analyse  zeigt, daß die Zeit für erfolglose Suche unverändert bleibt, aber die Zeit für erfolgreiche Suche auch bei voller Hashtabelle stets unter durchschnittlich 2.5 inspizierten Einträgen liegt:

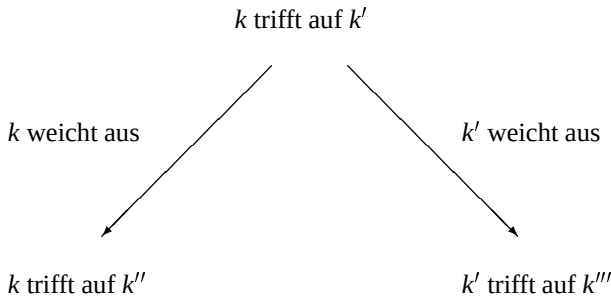
$$C'_n \approx \frac{1}{1 - \alpha}$$

$$C_n \approx 1 + \frac{\alpha}{2} + \frac{\alpha^3}{4} + \frac{\alpha^4}{15} - \frac{\alpha^5}{18} + \dots < 2.5$$

Binärbaum-Sondieren

Das Binärbaum-Sondieren (englisch: binary tree hashing) kann als eine Fortführung von Brents Idee angesehen werden ( ). Wenn Schlüssel *k* an Hashadresse *h(k)* nicht in die Hashtabelle eingetragen werden kann, weil sich dort schon ein Schlüssel *k'* befindet, so gibt es zwei Möglichkeiten. Entweder bleibt *k'* an seinem Platz, und für *k* wird gemäß der Sondierungsfolge für *k* ein anderer Platz gesucht, oder *k'* wird in der Hashtabelle durch *k* ersetzt, und für *k'* wird gemäß der Sondierungsfolge für *k'* ein anderer Platz gesucht.

Wenn sich auf eine der beiden Arten sogleich ein leerer Platz findet, so wird der entsprechende Schlüssel dort eingetragen, und das Einfügen ist beendet. Andernfalls werden beide Alternativen analog weiterverfolgt. Hier liegt der Unterschied zu Brents Algorithmus, bei dem nur die erste Alternative weiterverfolgt wird. Nach jeder Inspektion eines belegten Platzes ergibt sich eine weitere mögliche Sondierungsfolge, nämlich die zu dem in der Hashtabelle gespeicherten Schlüssel. Insgesamt hat die Abfolge der Inspektionen einzelner Plätze die Gestalt eines Binärbaumes:



Dieser Binärbaum wird niveauweise inspiziert; sobald ein freier Platz angetroffen wird, wird der ausweichende Schlüssel dort eingetragen. Die Analyse von Gonnet, Munro  ergibt für die erfolgreiche Suche:

$$C_n \approx 1 + \frac{\alpha}{2} + \frac{\alpha^3}{4} + \frac{\alpha^4}{15} + \dots < 2.2$$

Allerdings sind die Kosten für das Einfügen, insbesondere bei fast voller Hashtabelle, relativ hoch; in der beispielhaft betrachteten Anwendung des COBOL-Compilers spielt das aber kaum eine Rolle.

4.3.5 Ordered Hashing

Erinnern wir uns an das Beschleunigen der erfolglosen Suche beim Hashing mit Verkettung der Überläufer. Dort haben wir die Überlaufketten sortiert mit Schlüsseln belegt. Statt beim Einfügen einen Schlüssel hinten an die Überlaufkette anzuhängen, haben wir den Schlüssel in der Überlaufkette an der Position, die sich durch die Sortierung ergab, eingefügt. Das kann man, ebenso wie die Beschleunigung der erfolgreichen Suche beim Double Hashing, als die Simulation einer günstigeren Einfügereihenfolge der Schlüssel mit unsortierten Überlaufketten ansehen. Dieses Prinzip wollen wir nun auch auf offene Hashverfahren anwenden. Die Synonyme des gesuchten Schlüssels k sollen also in der Hashtabelle so abgespeichert sein, daß wir sie gemäß der Sondierungsfolge für k in sortierter Reihenfolge antreffen; wir wollen so den Fall simulieren, daß diese Schlüssel in sortierter Reihenfolge eingefügt worden sind. Wir legen uns hier (willkürlich) auf eine aufsteigende Sortierung fest.

Betrachten wir ein weiteres Mal das bekannte Beispiel. Haben wir die Schlüssel 12, 53, 5, 15, 2, 19 in die anfangs leere Hashtabelle nach dem Prinzip des Double Hashing eingefügt, so ergibt sich folgende Situation:

	0	1	2	3	4	5	6
$t :$	19	15	2	5	53	12	

Die Synonyme 12, 5, 19 sind in dieser Reihenfolge eingefügt worden; entsprechend steht an der Hashadresse $h(12) = h(5) = h(19) = 5$ der Schlüssel 12. Bei einer Suche nach Schlüssel 5 finden wir also zunächst in $h(5)$ die 12. Die Suche muß also fortgesetzt werden. Nehmen wir nun an, die Schlüssel seien in aufsteigend sortierter Reihenfolge eingefügt worden; dann könnten wir aus $h(5) = 12 > 5$ bereits schließen, daß Schlüssel 5 nicht in der Hashtabelle vorkommt; die erfolglose Suche könnte also früher abgebrochen werden.

Soweit gleicht das *Ordered Hashing* dem Sortieren der Überlaufketten beim Verketteten der Überläufer. Es gibt aber zwei wesentliche Unterschiede, die durch die Unterlegung eines offenen Hashverfahrens bedingt sind. Erstens kann ein neu einzufügender Schlüssel nicht einfach in die Sondierungskette eingefügt werden; Schlüssel, die in der Sondierungskette folgen, müssen eventuell in dieser Kette nach hinten rücken (vgl. Einfügen in ein Array, Kapitel 1). Zweitens kann ein Schlüssel, der in der Sondierungskette nach hinten rückt, auf einen bereits belegten Platz treffen; das kann vielen Schlüsseln in der Sondierungskette passieren. Der Platz, auf den ein solcher Schlüssel trifft, ist im allgemeinen nicht mit einem Synonym belegt.

Der erste Unterschied ist nicht besonders problematisch, denn beim herkömmlichen Einfügen eines neuen Schlüssels werden ja ohnedies alle besetzten Plätze in der Sondierungsfolge inspiziert. Der zweite Unterschied ist schon eher beunruhigend, denn er bedeutet doch, daß die Verschiebung in einer Sondierungskette auf andere übergreifen kann.

Fügen wir als Beispiel die Schlüssel 2, 12, 15, 53, 5 in die anfangs leere Hashtabelle mit Double Hashing ein (vgl. Abschnitt 4.3.4). Nach Einfügen von 2, 12, 15, 53 ergibt sich:

	0	1	2	3	4	5	6
$t :$		15	2		53	12	

Nun soll Schlüssel 5 eingefügt werden, nach dem Prinzip des Ordered Hashing. Da in $h(5) = 5$ ein größerer Schlüssel, nämlich 12, gespeichert ist, wird 12 von 5 verdrängt, und 5 wird in $t[5]$ gespeichert. Für Schlüssel 12 muß nun gemäß seiner Sondierungsfolge ein neuer Platz gesucht werden; in unserem Beispiel ist das wegen $h'(12) = 3$ der Platz $t[2]$. Platz $t[2]$ ist aber bereits belegt mit Schlüssel 2, und 2 ist kein Synonym von Schlüssel 12. Nun gibt es zwei Möglichkeiten: Entweder Schlüssel 2 bleibt in $t[2]$, und Schlüssel 12 sucht weiter, oder Schlüssel 12 wird in $t[2]$ gespeichert, und Schlüssel 2 sucht einen anderen Platz. Die zweite Möglichkeit scheidet aus, denn, würden wir Schlüssel 12 in $t[2]$ speichern und dann nach Schlüssel 2 suchen, so würde die Suche bereits bei $t[2]$ erfolglos abgebrochen werden müssen, obwohl Schlüssel 2 in der Hashtabelle gespeichert ist. Wir folgen also auch hier der Regel, daß der kleinere zweier konkurrierender Schlüssel den umkämpften Platz besetzt; der größere sucht weiter. Der nächste für Schlüssel 12 zu inspizierende Platz ist somit $t[6]$; er ist frei und Schlüssel 12 wird eingefügt:

	0	1	2	3	4	5	6
$t :$		15	2		53	5	12

Nach Einfügen der Schlüssel 19 und 43 ergibt sich schließlich mit $h(19) = 5$, $h'(19) = 5$, $h(43) = 1$, $h'(43) = 4$ und $h'(53) = 4$:

	0	1	2	3	4	5	6
t :	19	15	2	53	43	5	12

Methode: *Ordered Hashing für offene Hashverfahren*

Sei $s(j, k)$ eine Funktion von j und k so, daß $(h(k) - s(j, k)) \bmod m$ für $j = 0, 1, \dots, m - 1$ eine Sondierungsfolge, d. h. eine Permutation aller Hashadressen bildet. Es sei stets noch mindestens ein Platz in der Hashtabelle frei (das ist im obigen Beispiel nicht der Fall, macht aber die algorithmische Beschreibung einfacher).

Suchen nach Schlüssel k : Beginne mit Hashadresse $i = h(k)$. Solange k nicht in $t[i]$ gespeichert ist, $t[i]$ nicht frei und $t[i].k < k$ ist, suche weiter bei $i = (h(k) - s(j, k)) \bmod m$, für aufsteigende Werte von j . Falls $t[i]$ belegt und $t[i].k = k$ ist, so wurde k gefunden; sonst war die Suche erfolglos.

Einfügen eines Schlüssel k : Wir nehmen an, daß k nicht schon in t vorkommt. Beginne mit Hashadresse $i = h(k)$. Solange $t[i]$ belegt ist, vertausche $t[i].k$ mit k , falls $k < t[i].k$, und mache weiter beim nächsten zu k gehörigen i . Trage k bei $t[i]$ ein.

Entfernen eines Schlüssels k : wie bisher.

procedure *orderedSuchen* (**var** ds : datensatz; **var** t : hashtabelle);
 {sucht in der Hashtabelle t gemäß Ordered Hashing nach dem Datensatz mit Schlüssel $ds.k$ und liefert $ds.item$ oder eine Meldung über die Erfolglosigkeit der Suche}

var

i, j : hashadresse;

begin

$j := 0$; {Anzahl inspizierter Einträge}

repeat

$i := (h(ds.k) - s(j, k)) \bmod m$;

$j := j + 1$

until $(t[i].k = ds.k)$ **or** $(marke[i] = frei)$

{*} **or** $((marke[i] = belegt)$ **and** $(t[i].k > ds.k))$;

{*} **if** $(marke[i] = belegt)$ **and** $(t[i].k = ds.k)$
 then $ds.item := t[i].item$ {Suche erfolgreich}
 else write('Suche erfolglos beendet')

end

procedure *orderedEinfügen* (ds : datensatz; **var** t : hashtabelle);
 {fügt Datensatz ds gemäß Ordered Hashing in Hashtabelle t ein}

var

i : hashadresse;

begin

{*} $i := h(ds.k)$;

while $marke[i] <> frei$ **do**

```
begin
{*}      if  $ds.k < t[i].k$ 
{*}      then vertausche( $ds, t[i]$ );
{*}       $i := (i - s(1, ds.k)) \bmod m$ 
end;
 $t[i] := ds$ ;
marke[i] := belegt
end
```

Zu Beginn des Abschnitts 4.3 haben wir die entsprechende Beschreibung der Methoden und Prozeduren für offene Hashverfahren ohne Ordnung angegeben; die dort gemachten Bemerkungen gelten hier ebenfalls. In den Prozeduren *orderedSuchen* und *orderedEinfügen* haben wir die Programmzeilen mit einem $\{*\}$ kenntlich gemacht, die neu hinzugekommen oder geändert worden sind. Überdies haben wir beim *orderedEinfügen* die Struktur der Schleife ein wenig geändert. Bemerkenswert ist, daß wir beim Einfügen die neue Hashadresse nicht in der allgemeinen Form $i = (h(ds.k) - s(j, ds.k)) \bmod m$ berechnen können, weil ja für einen Schlüssel einer anderen Sondierungsfolge (kein Synonym für k) dessen Position j in dessen Sondierungsfolge unbekannt ist. Soll in unserem Beispiel in der Situation

	0	1	2	3	4	5	6
$t :$		15	2		53	12	

mit Double Hashing der Schlüssel 5 eingefügt werden, so wird Schlüssel 12 auf Platz $t[2]$ verdrängt, wo sich bereits ein Schlüssel befindet. Sofern dieser Schlüssel von Schlüssel 12 verdrängt wird (etwa wenn statt 2 dort 72 stünde), muß die neue Hashadresse für den verdrängten Schlüssel berechnet werden können ohne Kenntnis darüber, wie oft dieser Schlüssel bereits ausgewichen ist. Das geht bei den bisher von uns betrachteten Verfahren nur in den Fällen, in denen für jeden Schlüssel in der Sondierungsfolge aufeinanderfolgende Plätze um einen festen Betrag versetzt sind. Das heißt, daß das Ordered Hashing nicht auf das quadratische Sondieren, das zufällige Sondieren und das pseudozufällige Sondieren anwendbar ist; es ist anwendbar für lineares Sondieren und Double Hashing. In beiden Fällen läßt sich die Sondierungsfolge statt durch $(h(k) - s(j, k)) \bmod m$ für $j = 0, \dots, m - 1$ auch durch $i = h(k)$ im ersten Schritt und $i = i - s(1, k)$ danach berechnen, weil $s(j, k) - s(j - 1, k) = s(1, k)$ für alle $j, 1 \leq j \leq m - 1$ gilt.

Überlegen wir uns nun, wann denn das Prinzip des Ordered Hashing korrekt ist. Sei $p_0(k), p_1(k), \dots, p_{m-1}(k)$ die Sondierungsfolge für Schlüssel k , also $p_j(k) = (h(k) - s(j, k)) \bmod m$. Der Suchalgorithmus liefert stets das richtige Resultat, wenn für jeden Schlüssel k auf Platz $p_j(k)$ gilt, daß alle Schlüssel auf Plätzen $p_i(k), i < j$, kleiner sind als k . Dann nämlich werden beim Suchen nach k die Plätze $p_i(k), i < j$, und schließlich $p_j(k)$ inspiziert. Man beachte, daß diese Forderung natürlich nichts über die Schlüssel auf Plätzen $p_{i'}, i' > j$, impliziert. Anfangs, also bei leerer Hashtabelle, ist die Forderung für alle gespeicherten Schlüssel trivialerweise erfüllt. Wir überlegen uns nun, daß diese Forderung nach dem Einfügen eines Schlüssels erfüllt bleibt, wenn sie davor erfüllt war. Das ist aber offensichtlich, denn wenn beim Einfügen an einer Stelle p ein Schlüssel

eingetragen wird, dann war entweder $t[p]$ frei, oder in $t[p]$ war ein größerer Schlüssel gespeichert. Ein Schlüsselwert auf einem Platz $p_i(k)$, wobei k auf Platz $p_j(k)$, $j > i$, gespeichert ist, kann also nur verringert werden.

Jetzt ist auch klar, daß als *entfernt* markierte Plätze nicht einfach wieder belegt werden können. Ein solcher Platz kann aber natürlich mit einem kleineren Schlüssel wieder belegt werden. In der Prozedur *orderedEinfügen* könnte man also die Zeilen

```
{*}   if  $ds.k < t[i].k$ 
{*}   then vertausche( $ds, t[i]$ );
```

ersetzen durch

```
{*}   if  $ds.k < t[i].k$ 
{*}   then
{*}       if  $marke[i] = \text{entfernt}$ 
{*}       then exit while-loop
{*}       else vertausche( $ds, t[i]$ );
```

und damit manche der als *entfernt* markierten Plätze wiederverwenden.

Amble und Knuth  haben gezeigt, daß eine Menge von Schlüsseln unabhängig von der Reihenfolge ihres Einfügens mit Ordered Hashing immer gleich auf die Plätze einer Hashtabelle verteilt wird; also ergibt sich stets dieselbe Situation, als hätte man die Schlüssel sortiert eingefügt. Um dies einzusehen, nehmen wir an, daß es zwei verschiedene Situationen (Belegungen der Hashtabelle) für dieselbe Schlüsselmenge gibt; mindestens ein Schlüssel befindet sich demnach in beiden Situationen nicht am gleichen Platz. Betrachten wir jetzt den kleinsten Schlüssel k , der sich in beiden Situationen nicht am selben Platz befindet. Einmal landet er am Platz $p_i(k)$, das andere Mal am Platz $p_j(k)$, $i \neq j$. Sei nun $i < j$ (sonst vertausche i mit j). Befindet sich k am Platz $p_j(k)$, so befindet sich ein kleinerer Schlüssel $k' < k$ am Platz $p_i(k)$; in der anderen Situation befindet sich k' jedoch nicht am Platz $p_i(k)$, denn dort befindet sich ja k . Also befindet sich k' in beiden Situationen an verschiedenen Plätzen, und $k' < k$; ein Widerspruch zur Annahme. Damit ist klar, daß es nur eine Anordnung einer Menge von Schlüsseln mit Ordered Hashing in einer Hashtabelle gibt.

Analyse: Die Effizienz der erfolgreichen Suche ändert sich durch Anwendung des Prinzips des Ordered Hashing im Durchschnitt nicht (gegenüber dem zugrundeliegenden Verfahren), wohl aber die der erfolglosen Suche. Bei Ordered Hashing ist eine erfolglose Suche genauso teuer wie die erfolgreiche Suche wäre, wenn sich der gesuchte Schlüssel außer den tatsächlich eingetragenen Schlüsseln in der Hashtabelle befände:

$$\begin{aligned} \text{ordered}C'_n &= C_{n+1} \approx C_n \\ \text{ordered}C_n &= C_n \end{aligned}$$

Die Anzahl der beim Einfügen inspizierten Einträge ist nur geringfügig höher als beim zugrundeliegenden Verfahren. Mit Ordered Hashing ist es also gelungen, die Kosten für die erfolglose Suche auf die Kosten für die erfolgreiche Suche zu reduzieren, um den Preis etwas erhöhter Einfügekosten.

4.3.6 Robin-Hood-Hashing

Wir haben gesehen, wie man die Effizienz von Double Hashing für die erfolgreiche Suche durch Brents Algorithmus oder durch Binärbaum-Sondieren und für die erfolglose Suche durch Ordered Hashing verbessern kann. Dies gelang durch geeignetes Umordnen von Schlüsseln anlässlich einer Einfügeoperation. Bei Brents Variation des Double Hashing dient das Umordnen von Schlüsseln dazu, die durchschnittliche Effizienz der erfolgreichen Suche zu verbessern, also den Erwartungswert der Länge von Sondierungsfolgen zu verringern; Binärbaum-Sondieren ist eine natürliche Verallgemeinerung mit demselben Ziel. *Robin-Hood-Hashing* () ordnet ebenfalls Schlüssel beim Einfügen um, aber mit dem Ziel der Verringerung der Länge der *längsten* Sondierungsfolge.

Methoden: Robin-Hood-Hashing

Einfügen eines Schlüssels k : Beginne mit Hashadresse $i = h(k)$. Solange $t[i]$ belegt ist, vergleiche die relative Position j der Adresse i in der Sondierungsfolge von k mit der relativen Position j' der Adresse i in der Sondierungsfolge von $k' = t[i].k$: Ist $j' \geq j$, so fahre fort mit $i = (i - h'(k)) \bmod m$, sonst trage k bei $t[i]$ ein und fahre fort mit $k = k'$ und $i = (i - h'(k')) \bmod m$. Jetzt ist $t[i]$ frei; trage k bei $t[i]$ ein.

Robin-Hood-Hashing ändert also nichts an der durchschnittlichen Länge von Sondierungsfolgen, sondern gleicht nur die Längen der verschiedenen Sondierungsfolgen einander an — wie Robin Hood den Bestand an Gütern nicht geändert hat, sondern nur deren Verteilung. Erstaunlicherweise sinkt mit Robin-Hood-Hashing die Varianz der Länge von Sondierungsfolgen von einem Wert von fast $2m$ für Double Hashing auf einen Wert von weniger als 2, also eine sehr kleine Konstante. Die Varianz bleibt sogar konstant, wenn die Hashtabelle voll ist. Der Erwartungswert für die Länge der längsten Sondierungsfolge ist bei n gespeicherten Schlüsseln höchstens um $\lceil \log_2 n \rceil$ höher als der Erwartungswert aller Längen. Für eine volle Hashtabelle ergibt sich als Erwartungswert für die Länge der längsten Sondierungsfolge $\Theta(\ln m)$. Diesen Wert kann man nur um einen konstanten Faktor verbessern, wenn man die Schlüssel in der Hashtabelle so umbringt, daß die Länge der längsten Sondierungsfolge minimiert wird (); um dies tun zu können, muß man aber das entsprechende Zuordnungsproblem () lösen, das selbst $O(n^2 \log n)$ Zeit () kosten kann.

Kennt man zu einer Hashtabelle die Länge l der längsten auftretenden Sondierungsfolge, so kann man dies für eine Beschleunigung der erfolglosen Suche ausnutzen (): Jede Suche, auch eine erfolglose, wird nach dem Betrachten von l Hashtabelleneinträgen abgebrochen. Diese Länge l kann man bei Robin-Hood-Hashing (ohne Entferne-Operationen) ohne Zusatzaufwand mitführen, weil man beim Einfügen eines (verdrängten) Schlüssels dessen relative Position in seiner Sondierungsfolge ohnehin kennen muß. Trifft man beim Einfügen eines Schlüssels k auf einen mit k' belegten

Platz, so berechnet man die aktuelle Position von k' in seiner Sondierungsfolge durch eine Suche nach k' ; die entsprechende Information für k kennt man bereits. Bei dieser Realisierung ist das Einfügen eines Schlüssels bei Robin-Hood-Hashing ineffizienter als etwa bei Double Hashing, weil ja die durchschnittliche Länge von Sondierungsfolgen nicht verkürzt worden ist und beim Einfügen für jeden betrachteten, belegten Platz eine erfolgreiche Suche durchgeführt werden muß. Mit einem schlaun Algorithmus für die Suche (*smart searching* ) läßt sich sowohl die Effizienz der Suche als auch die des Einfügens deutlich verbessern. Dabei benutzen wir die Kenntnis des auf die nächstgelegene ganze Zahl gerundeten Erwartungswerts s der Länge von Sondierungsfolgen und beginnen bei der Suche nach einem Schlüssel k nicht an Position 1 seiner Sondierungsfolge, sondern an Position s . Die zu s gehörende Adresse für Schlüssel k kann leicht berechnet werden; sie ist bei Double Hashing $h(k) - (s - 1)h'(k)$. Finden wir Schlüssel k nicht an dem zu s gehörenden Platz, so sondieren wir der Reihe nach die Plätze zu $s + 1, s - 1, s + 2, s - 2, \dots$ nach unten bis 1 und nach oben bis l , falls s durch Abrunden entstand, und sonst $s - 1, s + 1, s - 2, s + 2, \dots$. Wenn k dabei nicht gefunden wird, endet die Suche erfolglos. Die Effizienz der erfolglosen Suche verbessert sich bei diesem Verfahren natürlich nicht, aber der Erwartungswert für die erfolgreiche Suche ist eine Konstante. Selbst bei einer vollen Hashtabelle werden stets weniger als 2.8 Einträge inspiziert. Die höchste Effizienz bei der Suche erzielt man, wenn man Hashtabelleinträge in genau derjenigen Reihenfolge betrachtet, die sich durch die Anordnung aller in der Sondierungsfolge zum gesuchten Schlüssel vorkommenden Plätze nach absteigenden Erfolgswahrscheinlichkeiten ergibt. In diesem Fall inspiziert man bei einer Suche stets weniger als 2.6 Einträge. Damit ist nicht nur die Effizienz der Suche, sondern auch die Effizienz des Einfügens fast dieselbe wie bei Double Hashing (bis auf einen kleinen konstanten Faktor). Außerdem kann man mit Robin-Hood-Hashing auch eine Folge von Entferne- und Einfügeoperationen durchführen, ohne daß die Suchzeit degeneriert. Experimente hierzu und zum Vergleich von Robin-Hood-Hashing mit anderen offenen Hashverfahren sind in  ausführlich beschrieben.

4.3.7 Coalesced Hashing

Vergleichen wir rückblickend die Effizienz aller bisher betrachteten Verfahren, so zeigt sich, daß sowohl die erfolglose als auch die erfolgreiche Suche bei Verkettung der Überläufer am schnellsten ist. Das ist auch intuitiv plausibel, denn bei offenen Hashverfahren war es ja stets möglich, daß wir beim Inspizieren der Plätze gemäß der Sondierungsfolge für einen Schlüssel k andere Schlüssel k' angetroffen haben, die keine Synonyme von k waren. Andererseits haben die Verfahren der Verkettung der Überläufer (vgl. Abschnitt 4.2) den Nachteil, daß selbst dann neuer Speicherplatz außerhalb der Hashtabelle dynamisch bereitgestellt und belegt werden muß, wenn in der Hashtabelle noch Plätze frei sind. Das Verfahren des *Coalesced Hashing* (englisch: to coalesce = verschmelzen) verbindet das Prinzip des offenen Hashing mit dem der Verkettung der Überläufer. Alle Überläufer befinden sich in einer Überlaufkette, die in der Hashtabelle abgespeichert ist. Jeder Eintrag der Hashtabelle besteht aus dem Schlüssel samt dem zugehörigen Datensatz und einem Zeiger (realisiert als Hashadresse) auf den nächsten Eintrag in der Überlaufkette. Ein einzufügender Schlüssel wird ans Ende der Überlaufkette angehängt.

Betrachten wir das Beispiel, das uns bisher begleitet hat. Die Hashtabellengröße sei 7, die Hashfunktion sei $k \bmod 7$ für Schlüssel k , und die Schlüssel 12, 53, 5, 15, 19, 43 seien in dieser Reihenfolge in die anfangs leere Hashtabelle einzufügen. Nach Einfügen von 12, 53 erhalten wir folgende Situation:

	0	1	2	3	4	5	6	
$t :$					53	12		Schlüssel
								Verweise

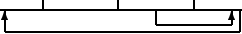
Beim Einfügen von Schlüssel 5 stellen wir fest, daß $h(5) = 5$ bereits mit Schlüssel 12 belegt ist; es gibt aber noch keine Überläufer, Schlüssel 12 ist das Ende der Überlaufkette. Statt nun dynamisch einen neuen Speicherplatz zu allokalieren (wie beim Verketteten der Überläufer außerhalb der Hashtabelle), müssen wir uns hier für einen freien Platz in der Hashtabelle entscheiden, den wir mit Schlüssel 5 belegen wollen. Wir legen uns darauf fest, den von rechts her ersten freien Platz in der Hashtabelle zu nehmen (also denjenigen mit höchster Hashadresse). Schlüssel 5 wird also bei $t[6]$ eingetragen und mit $t[5]$ verkettet:

	0	1	2	3	4	5	6	
$t :$					53	12	5	
						6		



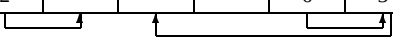
Nach Einfügen von 15 und 19 ergibt sich

	0	1	2	3	4	5	6	
$t :$		15		19	53	12	5	
						6	3	



und nach Einfügen von 43 schließlich

	0	1	2	3	4	5	6	
$t :$		15	43	19	53	12	5	
		2				6	3	



Method: *Coalesced Hashing*

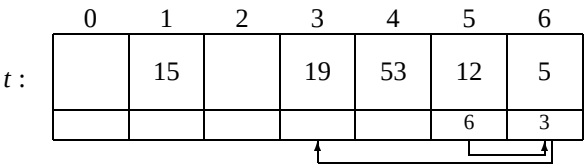
Jeder Eintrag der Hashtabelle besteht aus dem Datensatz mit Schlüssel und einem Verweis (Hashadresse) auf den Nachfolger in der Überlaufkette.

Suchen nach dem Schlüssel k : Beginne bei $t[h(k)]$ und folge den Verweisen der Überlaufkette, bis entweder k gefunden wurde (erfolgreiche Suche) oder das Ende der Überlaufkette erreicht ist (erfolglose Suche).

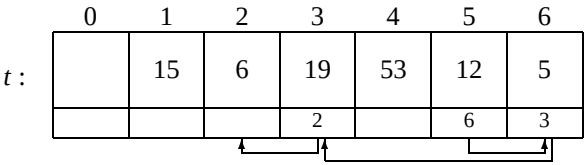
Einfügen eines Schlüssels k : Suche nach k ; die Suche verläuft erfolglos (sonst wird k nicht eingefügt) und endet am Ende einer Überlaufkette oder bei $t[h(k)]$. Im letzteren Fall trage k in $t[h(k)]$ ein; sonst wähle das freie Hashtablenelement mit größter Hashadresse, hänge es an die Überlaufkette an und trage k dort ein.

Entfernen eines Schlüssels k : Suche nach k ; die Suche verläuft erfolgreich (sonst kann k nicht entfernt werden). Steht k in $t[h(k)]$, so lösche k dort; verweise $t[h(k)]$ auf ein Element der Überlaufkette, so übertrage dieses nach $t[h(k)]$. Lösche k an seiner alten Position und adjustiere den Verweis auf das nächste Element in der Überlaufkette, falls k in der Überlaufkette auftritt.

Bis auf das Auswählen eines freien Eintrags in der Hashtabelle gleicht also diese Methode völlig dem Hashing mit separater Verkettung der Überläufer. Es gibt aber einen wichtigen Unterschied bei den entstehenden Situationen. Fügen wir gemäß obiger Regel in der nach Einfügen von 12, 53, 5, 15, 19 entstandenen Situation



statt des Schlüssel 43 (wie im obigen Beispiel) jetzt den Schlüssel 6 ein, so stellen wir fest, daß $t[h(6)] = t[6]$ bereits belegt ist, und hängen Schlüssel 6 an das Ende der Überlaufkette ab $t[6]$ an:



Die Überlaufkette ab $t[5]$ enthält somit auch den Schlüssel 6, obwohl 6 kein Synonym von 5 ist; entsprechend enthält die Überlaufkette ab $t[6]$ auch die Schlüssel 5 und 19, obwohl beide keine Synonyme von Schlüssel 6 sind. Die beiden Überlaufketten von Schlüssel 5 und 6 sind verschmolzen. Die Korrektheit der angegebenen Methode wird hiervon nicht beeinträchtigt, wohl aber die Effizienz. Da Überlaufketten etwas länger werden als beim separaten Verketteten, dauert die Suche etwas länger; im Durchschnitt ist das aber sehr wenig, wie eine Analyse zeigt:

$$C'_n \approx 1 + \frac{1}{4} (e^{2\alpha} - 1 - 2\alpha)$$

$$C_n \approx 1 + \frac{1}{8\alpha} (e^{2\alpha} - 1 - 2\alpha) + \frac{1}{4}\alpha$$

Tabelle 4.5 vermittelt durch einige in diese Formeln eingesetzte Werte von α einen Eindruck von der Effizienz des Coalesced Hashing.

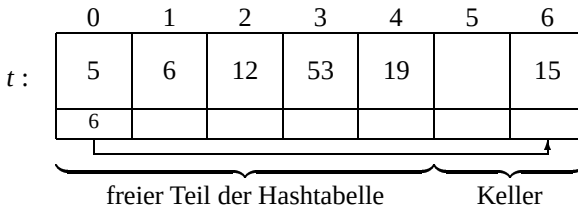
Anzahl betrachteter Einträge	Coalesced Hashing	
	erfolgreich	erfolglos
$\alpha = 0.50$	1.30	1.18
0.90	1.68	1.81
0.95	1.74	1.95
1.00	1.80	2.10

Tabelle 4.5

Diese beachtliche Effizienz der Suche beim Coalesced Hashing wird erzielt um den Preis eines etwas höheren Speicherplatzbedarfs für die Verweise und eines etwas höheren Zeitbedarfs für das Einfügen, da ja nach einem freien Platz in der Hashtabelle gesucht werden muß. Verzichtet man auf das Wiederbelegen der Plätze als *entfernt* markierter Einträge, so kann man einen einzigen Verweis (Hashadresse), ausgehend von Hashadresse $m - 1$, schrittweise durch die Hashtabelle bewegen (lineares Suchen, vgl. Kapitel 3) und an einem gefundenen freien Platz bis zur nächsten Einfügeoperation ruhen lassen; alle Plätze mit höherer Hashadresse sind ja schon belegt. Dann durchläuft dieser Verweis höchstens soviele Plätze, wie Schlüssel in die Hashtabelle eingefügt worden sind. Im Durchschnitt benötigt eine Einfüge-Operation also gerade einen Versuch, um einen leeren Platz in der Tabelle zu finden. Man kann zeigen, daß für eine zufällige Einfügung etwa $\alpha \cdot e^\alpha$ Plätze auf der Suche nach einem freien Platz inspiziert werden müssen.

Das Coalesced Hashing in der beschriebenen Form geht zurück auf Williams [1]. Inzwischen sind viele Varianten des Verfahrens untersucht worden. Eine Wesentliche sperrt einen Teil der Hashtabelle, den *Keller*, für die normale Benutzung und weist diesem Teil nur Überläufer zu. Sobald der Keller voll ist, wird auch der Platz im Rest der Hashtabelle verwendet.

In unserem Beispiel einer Hashtabelle der Größe 7 wählen wir $t[0]$ bis $t[4]$ als frei verfügbaren Teil der Hashtabelle; $t[5]$ bis $t[6]$ ist der Keller. Die Hashfunktion ändert sich damit zu $h(k) = k \bmod 5$, die Algorithmen für das Suchen, Einfügen und Entfernen bleiben unverändert. Fügen wir nun die Schlüssel 12, 53, 5, 15, 19, 6 in die Hashtabelle ein, so entsteht folgende Situation:



Man erwartet, daß sich durch die Verwendung des Kellers die Verschmelzung von Überlaufketten reduziert; solange Überläufer im Keller abgelegt werden, gibt es keine Verschmelzung. Im Beispiel ist dies der Fall. Die Verschmelzung von Überlaufketten ist also umso geringer, je größer der Keller ist. Andererseits reduziert sich, wenn ein Speicherplatz fester Größe insgesamt zur Verfügung steht, bei großem Keller der freie Teil der Hashtabelle; dadurch werden Kollisionen wahrscheinlicher, und damit gibt es mehr Überläufer. Im Extremfall ist nur $t[0]$ frei, und $t[1]$ bis $t[m - 1]$ bilden den Keller; dann sind alle Schlüssel in einer einzigen Überlaufkette gespeichert. Die Anzahl der Überläufer sinkt also mit kleinerem Keller. Nennen wir m_h die Anzahl der frei verfügbaren Plätze der Hashtabelle und m_k die Anzahl der Plätze im Keller ($m_h + m_k = m$), dann ist das Verhältnis m_h/m für die Effizienz der Suche entscheidend. In einer vollen Hashtabelle ist der Erwartungswert für die erfolgreiche Suche bei $m_h/m = 0.853 \dots$ minimal, für die erfolglose Suche bei $m_h/m = 0.782 \dots$ der Wert $m_h/m = 0.86$ scheint ein guter Kompromiß für beide Fälle und einen großen Bereich von Belegungsfaktoren zu sein.

4.4 Dynamische Hashverfahren

Wenngleich alle der von uns bisher vorgestellten Hashverfahren die Operationen Einfügen und Entfernen unterstützen, so ist die tatsächlich realisierte Dynamik der Verfahren nur begrenzt. Bei offenen Hashverfahren ist das Einfügen von Schlüsseln über den vorgesehenen Speicherplatz hinaus unmöglich; bei Hashverfahren mit Verkettung der Überläufer ist es prinzipiell zwar möglich, beeinträchtigt aber die Effizienz der Verfahren erheblich. Im Extremfall degeneriert nach unvorhergesehen vielen Einfügeoperationen ein Hashverfahren mit Verkettung der Überläufer zur Verwaltung relativ weniger, sehr langer verketteter linearer Listen. Im anderen Extremfall wird eine sehr große Hashtabelle für nur wenige Einträge freigehalten. Wir wollen in diesem Abschnitt vier Hashverfahren für stark wachsende oder schrumpfende Datenbestände vorstellen. Solche *dynamischen Hashverfahren* (vgl. Übersichtsartikel) sind insbesondere für Daten von Bedeutung, die auf Externspeichern verwaltet werden, wie etwa die Datensätze einer Datenbank (man kann sie aber auch als Hauptspeicherstrukturen einsetzen). Die kleinste Einheit des Zugriffs auf den Externspeicher ist der *Datenblock*; eine Lese- bzw. Schreiboperation auf den Externspeicher überträgt einen Block vom Externspeicher in den Hauptspeicher des Rechners bzw. vom Hauptspeicher auf den Externspeicher. Bei den meisten Rechnern haben Blöcke eine feste Größe, typi-

scherweise 512 Byte oder ein Vielfaches davon. Damit können in der Regel mehrere Datensätze in einem Block gespeichert werden. Sei b (Blockkapazität) die Anzahl der Datensätze, die neben einigen Verwaltungsinformationen in einem Block gespeichert werden. Dann können wir einen Block wie folgt beschreiben:

```

const
     $b = 30$ ; {Beispiel für Blockkapazität}
type
    block = record
        verwaltung : {z.B. Anzahl belegter Einträge, etc.};
        eintrag : array[1 ..  $b$ ] of datensatz
    end

```

Anstelle einer Hashtabelle verwenden wir dann eine Datei, bestehend aus Blöcken:

```

type
    hashdatei = file of block

```

Wir setzen voraus, daß ein Block in der Datei durch seine relative Adresse, beginnend bei Adresse 0, direkt angesprochen werden kann. Eine Datei von m Blöcken mit Adressen 0 bis $m - 1$ wächst durch Anhängen des Blocks mit Adresse m und schrumpft durch Abhängen des Blocks mit Adresse $m - 1$. Eine Hashfunktion ordnet einem Schlüssel k die relative Adresse $h(k)$ mit $0 \leq h(k) \leq m - 1$ des Blocks zu, in dem der Datensatz mit Schlüssel k zu speichern ist. Adreßkollisionen sind also hier kein Problem, solange es nicht mehr als b Synonyme gibt, denn diese können ja gemeinsam in einem Datenblock gespeichert werden. Wir wollen uns nicht darum kümmern, wie Schlüssel *innerhalb* eines Datenblocks eingefügt, entfernt und wiedergefunden werden können, weil dies wegen der kleinen Größe von b nur geringen Rechenaufwand bedeutet. Um ein Vielfaches teurer sind dagegen Blockzugriffe, also das Lesen oder Schreiben eines ganzen Blocks. Als Maß für die Effizienz von externen, dynamischen Hashverfahren verwendet man daher üblicherweise die Anzahl erforderlicher Blockzugriffe.

Die besondere Problematik dynamischer Hashverfahren liegt nun darin, daß man nicht einfach ein und dieselbe Hashfunktion bei sich änderndem m verwenden kann, weil man sonst gespeicherte Schlüssel nicht unbedingt wiederfindet, und daß man eine globale Reorganisation, also das Umspeichern sämtlicher Datensätze gemäß einem geänderten m , aus Effizienzgründen vermeiden möchte. Man kann beide Probleme lösen, indem man nur Teilbereiche des gesamten Speichers — meist einzelne Datenblöcke — reorganisiert und sich merkt, für welche Teilbereiche eine Reorganisation erfolgt ist und welche neue Hashfunktion dabei verwendet wurde. Bei den Vereinbarungen

```

var
     $hd$  : hashdatei;
     $m$  {aktuelle Anzahl der Blöcke in  $hd$ },
     $n$  {aktuelle Anzahl in  $hd$  gespeicherter Datensätze} : integer

```

kann für viele dynamische Hashverfahren ein Rahmen für das Einfügen eines Datensatzes ds in eine Hashdatei hd mit m Blöcken und n aktuell gespeicherten Datensätzen wie folgt beschrieben werden:

```

while hd mit m Blöcken und n Datensätzen ist für ds zu klein do
  begin {erweitere hd um einen Block}
    füge neuen, leeren Block mit Adresse m an hd an;
    wähle Blockadresse i im Bereich 0 bis  $m - 1$ ;
    adaptiere Hashfunktion h;
    verteile Datensätze aus Block i gemäß h auf Blöcke i und m;
     $m := m + 1$ 
  end;
  trage ds in Block mit Adresse  $h(ds.k)$  in hd ein

```

Die Suche nach einem Datensatz mit Schlüssel k besteht dann einfach im Prüfen des Inhalts des Blocks mit Adresse $h(k)$. Beim Entfernen von Einträgen wird — analog zum Einfügen — überprüft, ob die Hashdatei zu groß ist; sie wird gegebenenfalls um einen Block verkleinert. Innerhalb des gegebenen Rahmens unterscheiden sich dynamische Hashverfahren im Kriterium für das Erweitern oder Schrumpfen der Hashdatei um einen Block und in der Wahl der adaptierten Hashfunktion und ihrer Speicherung und damit der Wahl des Blocks der zu verteilenden Datensätze.

Im nächsten Abschnitt werden wir ein dynamisches Hashverfahren, das *lineare Hashing*, mit einer sehr einfachen Hashfunktion vorstellen; genauer an die zu speichernden Daten angepaßte Verfahren, bei denen das Speichern der Hashfunktion selbst zu einem Datenverwaltungsproblem wird, präsentieren wir dann in den folgenden Abschnitten.

4.4.1 Lineares Hashing

Bei linearem Hashing  besteht die Hashfunktion h zu jedem Zeitpunkt aus höchstens zwei einfachen Hashfunktionen h_1 und h_2 , die jeweils die gesamte Hashdatei adressieren. Für eine anfängliche Dateigröße von m_0 Blöcken der Hashdatei und eine aktuelle Größe von m Blöcken, wobei $m_0 \cdot 2^l \leq m < m_0 \cdot 2^{l+1}$ für eine natürliche Zahl l gilt, adressiert h_1 den Adreßbereich $0 \dots m_0 \cdot 2^l - 1$ und h_2 den Adreßbereich $0 \dots m_0 \cdot 2^{l+1} - 1$. Dabei ist je nach Schlüssel k entweder $h_2(k) = h_1(k)$ oder $h_2(k) = h_1(k) + m_0 \cdot 2^l$. Der Dateilevel l gibt dabei die Anzahl der kompletten Dateiverdoppelungen an. Weil also $h_2(k)$ die Datensätze des Blocks $h_1(k)$ verteilt, ergibt sich nach Einfügen eines leeren Blocks mit Adresse m die Adresse i des Blocks, der die zu verteilenden Datensätze enthält, als $i = m - m_0 \cdot 2^l$. Damit durchläuft i der Reihe nach die Adressen 0 bis $m_0 \cdot 2^l - 1$; anfangs ist $i = 0$ und $l = 0$. Hashfunktion h_1 ist dann für diejenigen Schlüssel k anzuwenden, für die $i \leq h_1(k) \leq m_0 \cdot 2^l - 1$ gilt. Für alle anderen Schlüssel k , das sind diejenigen mit $0 \leq h_1(k) < i$, ist h_2 anzuwenden. Der gesamte dynamische Hashdateizustand ist also durch den Dateilevel l und die Adresse i der nächsten Seite mit zu verteilenden Datensätzen (der nächsten zu *splittenden* Seite) charakterisiert, wenn h_1 und h_2 festliegen. Eine geeignete Wahl für h_1 und h_2 ergibt sich beispielsweise durch Anwendung der Divisions-Rest-Methode, mit $h_1(k) = k \bmod (m_0 \cdot 2^l)$ und $h_2(k) = k \bmod (m_0 \cdot 2^{l+1})$.

Weil die nächste zu splittende Seite unabhängig von einem einzufügenden Datensatz festliegt, kann bei linearem Hashing keine Rücksicht darauf genommen werden, ob ein Datensatz noch in dem ihm zugeordneten Datenblock Platz findet. Man entscheidet sich hier dafür, gemäß der aktuellen Hashfunktion h bei mehr als b Synonymen Ketten von Blöcken für diese Synonyme zu bilden, ganz ähnlich wie bei Hashverfahren mit Verkettung der Überläufer. Überlaufblöcke werden dabei in einem eigenen Speicherbereich untergebracht. Wir werden der Einfachheit halber einen durch die Hashfunktion adressierten Block (*Primärblock*) und evtl. ihm zugeordnete Überlaufblöcke (*Sekundärblöcke*) nicht unterscheiden und logisch wie einen Block behandeln. Es sollte aber klar sein, daß die Verwendung von Überlaufblöcken zusätzliche Externspeicherzugriffe und damit Effizienzeinbußen nach sich zieht.

Das Kriterium für das Erweitern der Hashdatei um einen Block ist bei linearem Hashing üblicherweise der Belegungsfaktor $n/(b \cdot m)$ der Hashdatei. Würde er als Folge einer Einfügeoperation einen festgesetzten Schwellenwert überschreiten, so wird die Hashdatei erweitert; würde er als Folge einer Entferne-Operation einen (anderen) Schwellenwert unterschreiten, so wird die Hashdatei um einen Block verkleinert. Das Verkleinern erfolgt hier völlig symmetrisch zum Erweitern der Datei. Die Einträge im Block mit Adresse m und im Block mit Adresse $m - m_0 \cdot 2^l$ werden zusammengefaßt und im Block mit Adresse $m - m_0 \cdot 2^l$ abgelegt; i und l werden wiederum entsprechend angepaßt.

Aber auch andere Hashfunktionen als nach der Divisions-Rest-Methode sind vorstellbar. Für manche Operationen ist es wünschenswert, in einem Datenblock möglichst Datensätze mit nahe beieinander liegenden Schlüsseln zu speichern — etwa beim Finden des einem Suchschlüssel nächstgelegenen Schlüssels (best match query, nearest neighbour query) oder beim Finden aller Schlüssel in einem gewissen Bereich (range query). Die Divisions-Rest-Methode leistet diese *ordnungserhaltende* Abbildung von Schlüsseln auf Adressen offenbar nicht. So erhält man beispielsweise eine ordnungserhaltende Abbildung ganzzahliger Schlüssel, indem man diese durch Bitstrings fester Länge darstellt, und die von links her ersten (also in der Zahl höchstwertigen) l Bits eines jeden Schlüssels in umgekehrter Reihenfolge, also von rechts nach links, als Dualzahl liest und als Hashadresse im Bereich von 0 bis 2^l ansieht [10]. Um Häufungen von Schlüsseln zu vermeiden, betrachtet man manchmal auch Bitstrings, die sich aus Schlüsseln durch Anwenden einer Hashfunktion und entsprechende Interpretation des Hashfunktionswertes ergeben, sogenannte *Pseudoschlüssel*. Wir wollen dies hier nicht mehr explizit berücksichtigen, weil dieser Unterschied keinen Effekt auf die vorgestellten Verfahren hat, und stattdessen stets Schlüssel direkt als Bitstrings ansehen.

Beispiel: Betrachten wir die mit linearem Hashing und der beschriebenen ordnungserhaltenden Hashfunktion organisierte Hashdatei, die sich durch Einfügen der Schlüssel 12, 53, 5, 15, 2, 19, 43 in dieser Reihenfolge in die Hashdatei ergibt, die anfangs aus einem leeren Datenblock besteht ($m_0 = 1$). In jedem Datenblock können bis zu zwei Datensätze gespeichert werden; wir zeigen im folgenden nur deren Schlüssel. Wählen wir 0.9 als Schwellenwert des Belegungsfaktors zum Erweitern der Datei und die feste Darstellungslänge von 6 Bits für jeden Schlüssel, so ergibt sich bei der in Abbildung 4.3 gezeigten Ausgangssituation vor dem Einfügen des zweiten Schlüssels ein

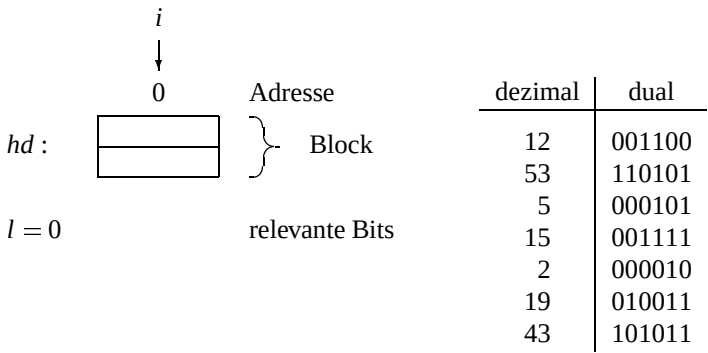


Abbildung 4.3

Split des Blocks 0 in Blöcke 0 und 1, und nach dem Eintragen dieses Schlüssels die in Abbildung 4.4 gezeigte Situation.

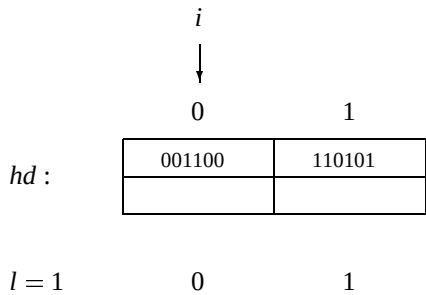


Abbildung 4.4

Schlüssel 5 kann auf dem freien Platz in Block 0 gespeichert werden; der Schwellenwert für den Belegungsfaktor wird nicht überschritten. Dies geschieht erst bei der Einfügung von Schlüssel 15. Hierbei wird ein neuer Block, nämlich mit Adresse 2, an die Hashdatei angehängt. Die in Block 0 gespeicherten Schlüssel werden gemäß ihrem zweiten Bit auf Blöcke 0 und 2 verteilt: Schlüssel mit führenden Bits 00 bleiben im Block 0, Schlüssel mit führenden Bits 01 (solche treten bisher nicht auf) werden in Block 2 gespeichert (01 rückwärts gelesen ergibt 10, also die duale Darstellung der Hashadresse 2). Dann wird der einzufügende Schlüssel 15 gemäß seiner beiden führenden Bits in Block 0 eingetragen. Hierbei muß für Block 0 ein Überlaufblock angelegt werden. Die Adresse des Überlaufblocks entstammt einem anderen Adreßbereich und sei hier nicht von Bedeutung. Damit ergibt sich die in Abbildung 4.5 dargestellte Situation.

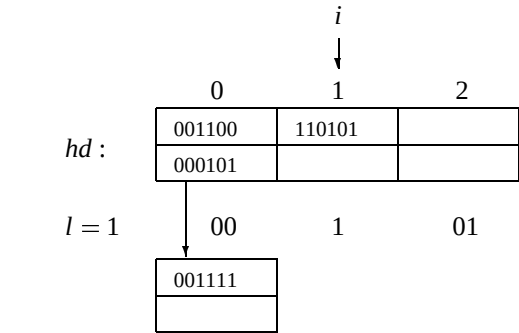


Abbildung 4.5

Schlüssel 2 kann ohne weitere Reorganisation der Datei in Block 0 (genauer: dessen Überlaufblock) eingefügt werden. Erst Schlüssel 19 führt wieder zu einem Überschreiten des Schwellenwerts des Belegungsfaktors und damit zum Anhängen eines neuen Datenblocks an die Hashdatei. Damit ist eine weitere Dateiverdoppelung beendet, und wir erhalten die in Abbildung 4.6 gezeigte Situation.

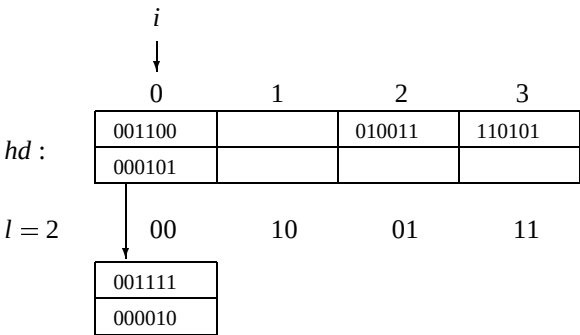


Abbildung 4.6

Schließlich kann Schlüssel 43 in Datenblock 1 eingetragen werden und die Folge der Einfügungen ist beendet.

Beziehen wir die Anzahl l bereits erfolgter Dateiverdoppelungen in die Hashfunktion ein, so adressiert bei aktueller Dateigröße m und Anfangsgröße m_0 mit nächstem zu splittendem Datenblock i offenbar die Hashfunktion h_l die Datenblöcke mit Adressen i bis $m_0 \cdot 2^l - 1$, und h_{l+1} adressiert Blöcke 0 bis $i - 1$ und $m_0 \cdot 2^l$ bis m , wie in Abbildung 4.7 gezeigt.

Da bei linearem Hashing nach dem Erweitern der Datei um einen Block das Kriterium für das Erweitern der Datei sicher nicht mehr erfüllt ist, realisiert schon die folgende Spezialisierung des allgemeinen Prinzips dynamischer Hashverfahren die Einfügeoperation:

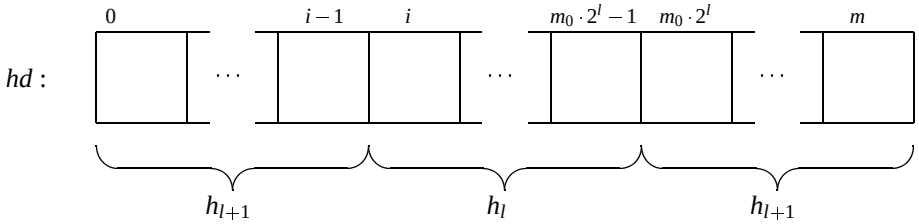


Abbildung 4.7

```

procedure Einfügen (ds: datensatz; var hd: hashdatei;
    var m, n, i, l : integer; schwelle: real);
{fügt Datensatz ds in Hashdatei hd mit Dateilevel l ein}
begin
    if  $(n + 1) / (b \cdot m) > \text{schwelle}$ 
    then {erweitere hd um einen Block}
        begin
            reserviere Block mit Adresse m für hd;
            verteile Datensätze aus Block i gemäß  $h_{l+1}$  auf Blöcke i und m;
             $m := m + 1$ ;
            if  $i < m_0 \cdot 2^l - 1$ 
            then  $i := i + 1$ 
            else {Dateiverdoppelung ist erfolgt}
                begin
                     $i := 0$ ;
                     $l := l + 1$ 
                end
            end;
        end;
     $n := n + 1$ ;
    {bestimme den ds.k zugeordneten Block}
    if  $(i \leq h_l(ds.k))$  and  $(h_l(ds.k) \leq m_0 \cdot 2^l - 1)$ 
    then trage ds im Block  $h_l(ds.k)$  ein
    else trage ds im Block  $h_{l+1}(ds.k)$  ein
end

```

Wir sparen uns die genaue algorithmische Beschreibung der Operationen Suchen und Entfernen, weil das Suchen nach einem Datensatz mit Schlüssel k lediglich das Bestimmen des k zugeordneten Blocks (wie am Ende der Einfügeprozedur) und das Inspizieren dieses Blocks ist, und weil das Entfernen mit einem eventuellen Verschmelzen von Blöcken völlig symmetrisch zum Einfügen operiert. Das bedeutet auch, daß die Entferneoperation ebenso wie die Einfügeoperation beim Reorganisieren von Teilen der Hashdatei keinerlei Rücksicht auf die aktuelle Verteilung der Datensätze nimmt. So wurde etwa in unserem Beispiel ein neuer Block (derjenige mit Adresse 2) angelegt, ohne daß er Datensätze des übergelaufenen Blocks 0 aufnahm. Bei Schlüsseln, die über

dem Universum K aller möglichen Schlüssel einigermaßen gleichverteilt sind, ist dies nicht unbedingt ein gravierender Nachteil. Man kann zeigen, daß bei Gleichverteilung der Datensätze die erwartete Speicherplatzausnutzung in einem dynamischen Hashverfahren, das mit rekursiver Halbierung (Verteilung von Datensätzen aus einem Block auf zwei Blöcke mit gleich großem Hashadreibereich) arbeitet, ohne Berücksichtigung von Überläufern bei $\ln 2$, also etwa 69 %, liegt . Strebt man jedoch einen konstant hohen Belegungsfaktor an, so ergibt sich zwischen zwei aufeinander folgenden Dateiverdoppelungen (man sagt auch: während einer *Expansion*) eine gewisse Diskontinuität bei der erwarteten Länge von Überlaufketten. Zu Beginn der Expansion werden alle Überlaufketten etwa gleich lang sein, aber gegen Ende der Expansion werden die Überlaufketten bereits gesplitteter Blöcke wesentlich kürzer sein als diejenigen noch nicht gesplitteter Blöcke. Dieser spürbare Effekt läßt sich mit Hilfe *partieller Expansionen* abschwächen . Dazu verteilt man etwa in einer ersten partiellen Expansion den Inhalt von jeweils zwei Datenblöcken auf drei Datenblöcke, von denen einer die Datei vergrößert, und in einer zweiten partiellen Expansion entsprechend von drei Datenblöcken auf vier Datenblöcke.

Trotzdem wird häufig die Speicherplatzausnutzung der Überlaufblöcke deutlich hinter derjenigen der Primärblöcke zurückbleiben, insbesondere dann, wenn die Kapazität der Überlaufblöcke groß ist. Man kann nun versuchen, mehreren Primärblöcken gemeinsam wenige Überlaufblöcke zuzuordnen (*overflow bucket sharing*). Dann muß man sich fragen, wie diese verwaltet werden sollen. Da eine statische Struktur für Überlaufdatensätze der Dynamik des linearen Hashing entgegensteht, kann man das Problem der Verwaltung von Überlaufdatensätzen als das ursprüngliche Problem ansehen, beschränkt auf eine kleinere Anzahl von Datensätzen. Es ist demnach natürlich, diese rekursiv mittels linearem Hashing zu verwalten . So erhält man mehrere Rekursionsebenen von linearem Hashing, bis schließlich keine Überläufer mehr auftreten. Die resultierende bessere Speicherplatzausnutzung erkauft man sich dabei durch Operationen, die sich über die rekursiven Ebenen der Daten fortsetzen können.

Es ist klar, daß bei stark ungleich verteilten Schlüsseln lineares Hashing degenerieren kann, im Extremfall zur Verwaltung einer einzigen linearen Kette von Überlaufblöcken. Eine Garantie für die Anzahl der zur Suche benötigten Externzugriffe läßt sich also nicht geben. Wir wollen in den nächsten Abschnitten andere dynamische Hashverfahren vorstellen, bei denen solch eine Garantie gegeben werden kann.

4.4.2 Virtuelles Hashing

Bei virtuellem Hashing  werden — im Unterschied zu linearem Hashing — Überlaufblöcke vollständig vermieden. Anstatt — wie bei linearem Hashing — die Hashdatei nur um jeweils einen Block zu vergrößern, verdoppelt man die Größe der Hashdatei bei virtuellem Hashing in einem Schritt, wenn eine Einfügeoperation in einen bereits vollen Datenblock durchgeführt werden soll und nicht schon beide Blöcke, auf welche die Datensätze verteilt werden müssen, zur Hashdatei gehören (Verfahren VH1 in  und ). Natürlich sollen nach einer Dateiverdoppelung nur die Sätze des überlaufenden Blocks verteilt werden, und nicht etwa die Sätze anderer Blöcke. Das kann dazu führen, daß Sätze nicht gemäß der Hashfunktion gespeichert sind, die der

aktuellen Hashdateigröße entspricht, sondern gemäß einer für eine kleinere Hashdateigröße verwendeten Hashfunktion. Es genügt also nicht, zu jedem Zeitpunkt mit nur zwei Hashfunktionen alle Datenseiten zu adressieren. Wir müssen vielmehr für alle im Zeitablauf eingesetzten Hashfunktionen vermerken, für welche Datenblöcke sie aktuell relevant sind. Bei virtuellem Hashing geschieht dies mit je einer Bittabelle für jede erfolgte Dateiverdoppelung und damit für jede im Zeitablauf verwendete Hashfunktion, außer der letzten. Sei wieder l die Anzahl der erfolgten Dateiverdoppelungen, und m_0 die Anfangsgröße der Hashdatei. Dann speichert für $0 \leq j \leq l - 1$ die j -te Bittabelle bit_j gerade $m_0 \cdot 2^j$ Bits, eines für jeden der Datenblöcke 0 bis $m_0 \cdot 2^j - 1$. Ein Bit hat genau dann den Wert 1, wenn die Hashfunktion h_j nicht ausreicht, um die gemäß der aktuellen Hashfunktion h auf diesen Block gehörenden Datensätze in der aktuellen Hashdatei zu adressieren. Dann muß also eine der Hashfunktionen $h_{j+1}, h_{j+2}, \dots$ verwendet werden; h_j ist für diesen Block veraltet.

Beispiel: Betrachten wir wieder das Einfügen der Schlüssel 12, 53, 5, 15, 2, 19, 43 in dieser Reihenfolge in die Hashdatei, die anfangs aus einem leeren Datenblock besteht ($m_0 = 1$). In jedem Datenblock finden zwei Datensätze Platz; wieder sei $h_j(k)$ mit $0 \leq j \leq l$ der Wert der Dualzahl der ersten j Bits von k , rückwärts gelesen. Schlüssel 12 und 53 (vgl. deren Dualdarstellung in Abbildung 4.3) werden mit $h_0(k) \equiv 0$ in Datenblock 0 eingefügt. Das Einfügen von Schlüssel 5 bringt Block 0 zum Überlaufen; er muß gesplittet werden. Die Hashdateigröße wird von einem auf zwei Blöcke verdoppelt, und in Bittabelle bit_0 wird vermerkt, daß h_0 zur Adressierung nicht ausreicht, wie in Abbildung 4.8 dargestellt.

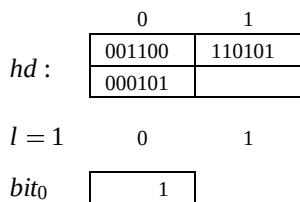


Abbildung 4.8

Für 15, den nächsten einzufügenden Schlüssel, wird nun die aktuelle Hashadresse berechnet. Hierfür wird zunächst h_0 auf Schlüssel 15 angewandt, mit Resultat 0. Dann wird $bit_0[0]$ überprüft; weil dieses Bit den Wert 1 hat, wird nun $h_1(15)$ berechnet, wieder mit Resultat 0. Weil erst eine Dateiverdoppelung erfolgt ist, gibt es keine weitere Bittabelle, und h_1 ist die auf Schlüssel 15 anzuwendende Hashfunktion. Also läuft Block 0 erneut über. Da h_1 die auf den einzufügenden Schlüssel anzuwendende Hashfunktion war, muß Block 0 mittels h_2 gesplittet werden, also seinen Inhalt und den einzufügenden Schlüssel auf Blöcke 0 und 2 verteilen. Nachdem aber Block 2 nicht schon zur Hashdatei gehört, ist eine Dateiverdoppelung erforderlich. Sie führt zu der in Abbildung 4.9 gezeigten Situation.

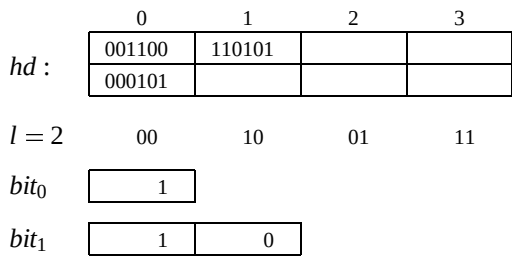


Abbildung 4.9

Auch hier ist Schlüssel 15 in den vollen Block $h_2(15)$ einzufügen, und wieder ist eine Dateiverdoppelung erforderlich. Allgemein ist eine Dateiverdoppelung dann erforderlich, wenn ein Datensatz in einen vollen Block eingefügt werden soll, dessen höchstes vermerktes Bit, also bit_{l-1} , eine 1 ist. Wir erhalten somit die in Abbildung 4.10 gezeigte Situation, in der die Datensätze aus Block 0 und der einzufügende Datensatz auf Blöcke 0 und 4 verteilt sind. Mit Ausnahme von Block 0 ist bisher kein Block gesplittet worden.

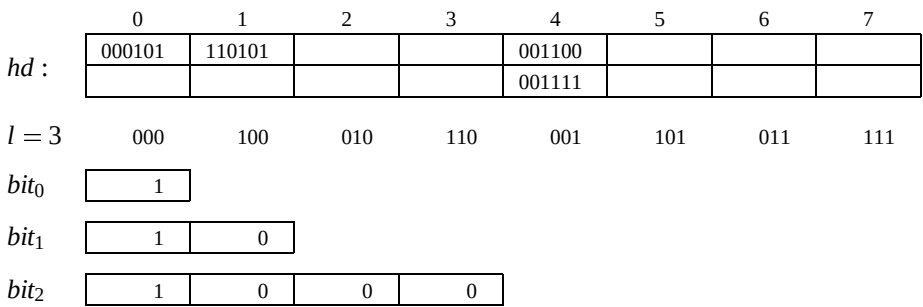


Abbildung 4.10

Das Einfügen der restlichen drei Schlüssel verläuft ohne weitere Dateiverdoppelungen. Für Schlüssel 2 erhalten wir $bit_0[h_0(2)] = 1$, $bit_1[h_1(2)] = 1$, $bit_2[h_2(2)] = 1$, und wegen $l = 3$ wird Schlüssel 2 schließlich gemäß $h_3(2)$ in Block 0 eingefügt. Entsprechend landet Schlüssel 19 mit $bit_0[h_0(19)] = 1$, $bit_1[h_1(19)] = 1$ und $bit_2[h_2(19)] = 0$ gemäß $h_2(19)$ im Block 2. Schließlich wird Schlüssel 43 gemäß $h_1(43)$ in Block 1 eingefügt.

Allgemein läßt sich also für eine Hashdatei der Anfangsgröße m_0 mit l erfolgten Verdoppelungen (Dateilevel l) mit Hilfe von l Bittabellen bit_j , $0 \leq j \leq l - 1$, der Typen

```
type bit_j = array [0 .. m_0 · 2^j - 1] of bit
```

die aktuelle Hashadresse $h(k)$ eines Schlüssels k wie folgt ermitteln:

```

j := 0;
while (j < l) and (bitj[hj(k)] = 1) do j := j + 1;
h(k) := hj(k)

```

Adressiert Hashfunktion h_j die Sätze eines Datenblocks, so bezeichnen wir j als den *Level* dieses Blocks. Wie wir am Beispiel gesehen haben, bewirkt das Einfügen eines Datensatzes in einen vollen Block mit Level l eine Dateiverdoppelung. Der Dateilevel ändert sich auf $l + 1$, und eine neue Bittabelle bit_l mit einer 1 für den betroffenen Datenblock und sonst lauter Nullen wird angelegt. Außerdem werden die Datensätze des betroffenen Blocks verteilt. Beim Einfügen in einen vollen Block mit kleinerem Level entfallen die Dateiverdoppelung und das Anlegen einer Bittabelle; es müssen lediglich ein vorhandener Bittabelleneintrag von 0 auf 1 verändert und die Datensätze verteilt werden. Damit kann das Einfügen ohne Rücksicht auf Implementierungsdetails wie folgt beschrieben werden:

```

procedure Einfügen(ds: datensatz; var hd: hashdatei; var l: integer;
var bit: sequence of bittabelle);
{fügt Datensatz ds in Hashdatei hd mit Dateilevel l und Bittabellen
bit0 bis bitl-1 ein}
var j : integer;
begin
    ermittle Hashadresse hj(ds.k) und Level j des Blocks, in den k
    einzufügen ist;
    while Block hj(ds.k) ist voll do
        begin
            if j = l
            then {Block hat Dateilevel l}
                begin
                    verdopple hd;
                    l := l + 1;
                    kreierte bitl = (0, 0, ..., 0)
                end;
                bitj[hj(ds.k)] := 1;
                verteile Sätze von Block hj(ds.k) auf Blöcke hj(ds.k) und
                    hj+1(ds.k) gemäß hj+1;
                ermittle erneut hj(ds.k) und Level j;
            end;
        trage ds in Block hj(ds.k) ein
    end

```

Nimmt man hierbei an, daß alle Bittabellen im Hauptspeicher gehalten werden können, so genügt für das Wiederfinden eines Datensatzes bei gegebenem Schlüssel offenbar ein Externzugriff. Diesen garantiert extrem schnellen Zugriff erkaufte man sich aber mit einer sehr schlechten Speicherplatzausnutzung. Für n Datensätze in der Datei und b Sätze pro Datenseite kann man zeigen, daß die Speicherplatzausnutzung von

der Größenordnung $O(n^{-(1/b)})$ ist, also mit wachsender Dateigröße abnimmt. Außerdem ist klar, daß die Speicherplatzausnutzung stark schwankt: Unmittelbar nach einer Dateiverdoppelung sinkt sie schlagartig auf die Hälfte. Diesen Effekt kann man vermeiden, wenn man die Hashfunktion nur zur Adressierung *virtueller* und nicht tatsächlicher Datenblöcke verwendet. Zu diesem Zweck übernimmt eine Adreßtabelle die Rolle der Hashdatei: Die jeweils aktuelle Hashfunktion adressiert einen Eintrag der Adreßtabelle. Ein Adreßtabelleneintrag ist dann lediglich die Adresse eines Blocks der Hashdatei. Statt einer Dateiverdoppelung findet also hier eine Adreßtabellenverdoppelung statt; die Datei wächst nur um einzelne Blöcke (Verfahren VH0 in [10, 11]). Für das in Abbildung 4.10 gezeigte Beispiel ergibt sich dann die in Abbildung 4.11 dargestellte Situation.

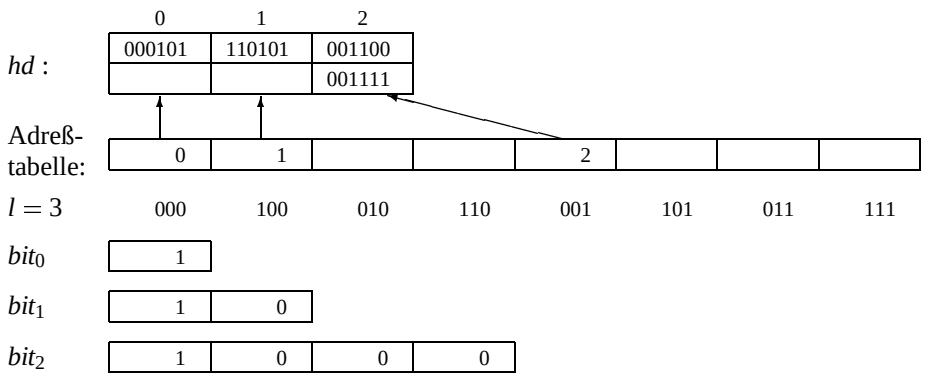


Abbildung 4.11

Man kann zeigen, daß die mittlere Speicherplatzausnutzung der Hashdatei hier um den Mittelwert $\ln 2 \approx 0.69$ pendelt. Soll auch hier noch mit einem einzigen Externspeicherzugriff ein Datensatz wiedergefunden werden können, so muß die Adreßtabelle neben den Bittabellen im Hauptspeicher Platz finden. Weil die Adreßtabelle Platz für mehr Einträge vorsieht als alle Bittabellen zusammen, und weil ein Eintrag der Adreßtabelle mehr Platz benötigt als ein Bit, kann dies unter Umständen eine unrealistische Annahme sein. Möglicherweise muß man dann die Adreßtabelle (und vielleicht sogar die Bittabellen) auf dem Externspeicher verwalten; dann können für das Wiederfinden eines Datensatzes zwei oder mehr Externzugriffe nötig werden. Im nächsten Abschnitt werden wir ein Verfahren vorstellen, bei dem man einen Schlüssel stets mit höchstens zwei Externzugriffen wiederfindet.

4.4.3 Erweiterbares Hashing

Erweiterbares Hashing (vorgestellt in [10] mit Ordnungserhaltung in [11]) hat eine starke Ähnlichkeit mit virtuellem Hashing mit Adreßtabelle. Wie dort wird bei erweiterbarem Hashing die Adreßtabelle bei Bedarf verdoppelt. Dieser Bedarf tritt ein, wenn

durch das Einfügen eines Datensatzes ein Datenblock geteilt werden muß und die beiden Adressen der beiden resultierenden Datenblöcke nicht in der bereits vorhandenen Adreßtabelle zu speichern sind. Während bei virtuellem Hashing mit Adreßtabelle die Adresse eines Datenblocks nur einmal in der Adreßtabelle auftritt und die unbenutzten Adreßtabellenfelder über die Bittabellen erkennbar sind, wird bei erweiterbarem Hashing jedes Adreßtabellenfeld benutzt. Damit spart man sich die Bittabellen; die bisher nicht benutzten Adreßtabelleneinträge müssen jetzt sinnvoll angegeben werden. Das ist aber leicht möglich, weil es wenigstens einen Adreßtabelleneintrag für jeden Datenblock gibt und damit die Adreßtabelle Schlüssel nach den ersten l Bits wenigstens so fein unterscheidet wie für die Verteilung auf Datenblöcke erforderlich. So gibt es beispielsweise in der in Abbildung 4.11 dargestellten Situation nur einen mit Bit 1 beginnenden Schlüssel (nämlich 110101), aber vier mit Bit 1 beginnende Nummern von Adreßtabelleneinträgen (nämlich 100, 110, 101 und 111). Wir können also einfach mit allen vier Adreßtabelleneinträgen auf denselben Datenblock verweisen, wie in Abbildung 4.12 gezeigt. Eintrag * in der Adreßtabelle repräsentiert eine fiktive Adresse, nämlich die eines leeren Datenblocks, den wir nicht explizit speichern.

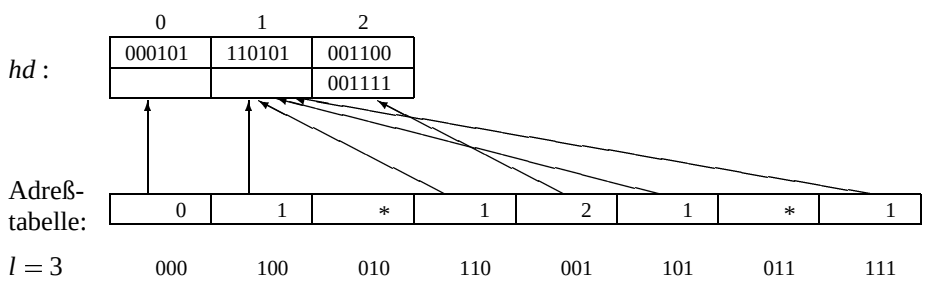


Abbildung 4.12

Betrachten wir zum genaueren Verständnis wieder das sukzessive Einfügen der Schlüssel 12, 53, 5 und 15, so ergibt sich nach Einfügen der ersten drei dieser Schlüssel die in Abbildung 4.13 gezeigte Situation.

Das Einfügen von Schlüssel 15 führt zu einem Split des Datenblocks 0. Dazu wird zunächst die Adreßtabelle verdoppelt und der Adreßtabellenlevel, also die Anzahl der zur Bestimmung der Nummer eines Adreßtabelleneintrags herangezogenen Bits, um 1 erhöht, wie in Abbildung 4.14 gezeigt.

Die Verdoppelung der Adreßtabelle ist das Anhängen einer identischen Kopie der bisherigen Adreßtabelle an sich selbst. Dann wird der überlaufende Datenblock gesplittet, indem ein neuer Block kreiert wird und der Inhalt des überlaufenden Blocks und der einzufügende Eintrag verteilt werden. In der gezeigten Situation ist jedoch ein Split des Datenblocks 0 nicht erfolgreich: Beide gespeicherten Einträge und der einzufügende Eintrag beginnen mit Bits 00, lassen sich also in den ersten $l = 2$ Bits nicht unterscheiden. Der neu angelegte Block, auf den der Adreßtabelleneintrag 01 verweist, bleibt leer. In diesem Fall wollen wir uns, in einer kleinen Modifikation des Vorschlags in  das

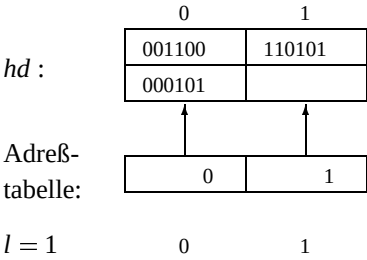


Abbildung 4.13

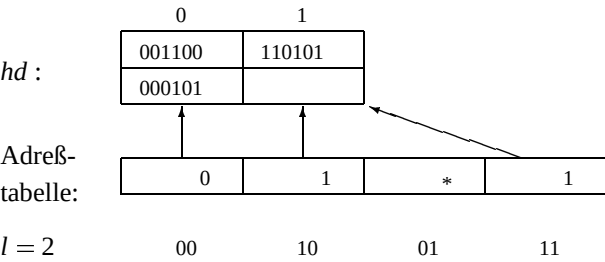


Abbildung 4.14

explizite Speichern eines leeren Blocks sparen und stattdessen den Adreßverweis als Verweis auf einen leeren Block kenntlich machen. Die Adreßverweise für verschiedene leere Blöcke werden verschieden gewählt. Dann wird eine weitere Adreßtabellenverdoppelung durchgeführt, die mit der in Abbildung 4.15 gezeigten Situation endet.

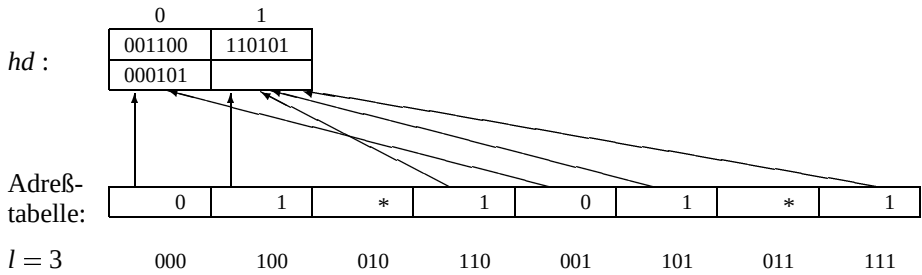


Abbildung 4.15

Weil die drei fraglichen Schlüssel nicht auch noch im dritten Bit übereinstimmen, ist ein Split des Datenblocks 0 jetzt erfolgreich. Block 0 wird in Blöcke 0 und 2 (die

nächste freie Datenblockadresse) aufgeteilt. Die beiden vor der Aufteilung auf Block 0 verweisenden Adreßtabelleneinträge werden gemäß dem dritten Bit angepaßt, wie in Abbildung 4.12 gezeigt.

Unter der Annahme, daß nicht nur die Datenblöcke, sondern auch die Adreßtabelle auf Externspeicher verwaltet werden, kommt man bei der Suche nach einem Datensatz mit gegebenem Schlüssel bei erweiterbarem Hashing stets mit höchstens zwei Externzugriffen aus (das *Zwei-Zugriffs-Prinzip* des erweiterbaren Hashing): Für Level l der Adreßtabelle wird zunächst gemäß den ersten l Bits des Schlüssels auf einen Adreßtabelleneintrag zugegriffen. Wird dort auf einen leeren Block verwiesen, so endet die Suche erfolglos; sonst wird der dort referenzierte Datenblock gelesen und inspiziert. Um beim Versuch des Einfügens in einen vollen Datenblock ohne weitere Externzugriffe entscheiden zu können, ob die Adreßtabelle verdoppelt werden muß, merken wir uns neben dem Level der Adreßtabelle (auch *globale Tiefe* genannt) für jeden Datenblock einen Level, die *lokale Tiefe*. Die lokale Tiefe eines Datenblocks i ist die Länge des kürzesten Anfangsstücks eines Schlüssels, das die Schlüssel im Block i von allen anderen unterscheidet. Beispielsweise haben in der in Abbildung 4.15 gezeigten Situation beide Datenblöcke die lokale Tiefe 1; in der Situation in Abbildung 4.12 dagegen haben Blöcke 0 und 2 die lokale Tiefe 3. Die Schlüssel aller Sätze in einem Block mit lokaler Tiefe t stimmen also mindestens in den ersten t Bits überein, und alle Sätze mit solchen Schlüsseln befinden sich in diesem Block. Auf einen Block mit lokaler Tiefe t verweisen bei globaler Tiefe l genau 2^{l-t} Einträge der Adreßtabelle. Das sind natürlich genau diejenigen Einträge, deren Hashadressen (relative Nummern in der Adreßtabelle) in den ersten t Bits mit den Schlüsseln im Block übereinstimmen.

Beim Einfügen eines Satzes in einen Block wird zunächst durch eine Suche der Block identifiziert, in den der Satz einzufügen ist. Verweist der durch die ersten l Bits des Schlüssels identifizierte Adreßtabelleneintrag auf einen leeren Block, so wird ein Block erzeugt und der einzufügende Schlüssel dort eingetragen. Verweist dagegen der Adreßtabelleneintrag auf einen nicht leeren und nicht vollen Datenblock, so wird der einzufügende Schlüssel dort eingetragen. Interessant ist also der Fall, daß ein Datensatz in einen bereits vollen Block eingefügt werden müßte. In diesem Fall wird der betreffende Block zunächst gesplittet. Damit dies gelingen kann, muß wenigstens ein weiteres Bit der Schlüssel als Unterscheidungsmerkmal verwendet werden. Aus dem Block mit lokaler Tiefe t vor dem Split werden zwei Blöcke mit jeweils lokaler Tiefe $t + 1$. Falls vor dem Split bereits $t = l$ gilt, muß zunächst die globale Tiefe l erhöht werden. Hierzu wird die Größe der Adreßtabelle verdoppelt. Wie wir bereits in unserem Beispiel gesehen haben, muß der Split eines Blocks nicht notwendigerweise dazu führen, daß der einzufügende Schlüssel auch gespeichert werden kann. In diesem Fall unterscheiden sich die Schlüssel der $b + 1$ zu speichernden Sätze in den ersten $t + 1$ Bits nicht. Dann werden Blocksplit und womöglich sogar Adreßtabellenverdoppelung wiederholt durchgeführt, bis schließlich eine Aufteilung gelingt.

Das Verfahren zum Entfernen eines Datensatzes ist auch bei erweiterbarem Hashing gerade die Umkehrung des Einfügens. Zunächst wird der zu entfernende Datensatz aus dem entsprechenden Block gelöscht. Dann wird überprüft, ob ein Blocksplit rückgängig gemacht werden kann, indem zwei Blöcke zu einem verschmolzen werden. Zwei Blöcke können dann verschmolzen werden, wenn die dort gespeicherten Sätze gemeinsam in einen Block passen und die Blöcke durch einen Split aus einem Block entstehen können — solche Blöcke heißen auch *Brüder*. Zwei Blöcke sind Brüder, wenn sie die

gleiche lokale Tiefe t haben und die Schlüssel aller in beiden Blöcken gespeicherten Datensätze in den ersten $t - 1$ Bits übereinstimmen. Dann stimmen die Hashadressen von Verweisen auf diese Blöcke in der Adreßtabelle ebenfalls in den ersten $t - 1$ Bits überein. Das Verschmelzen geschieht dann durch Zusammenlegen der Sätze auf einen der beiden Brüder und das Anpassen der Adreßtabelleneinträge. Wenn nach einer Verschmelzung für jeden Block die lokale Tiefe echt kleiner ist als die globale Tiefe der Adreßtabelle, so verweisen auf jeden Datenblock mindestens zwei Einträge der Adreßtabelle, und die Adreßtabelle kann halbiert werden. Diese Operation ist völlig symmetrisch zur Verdoppelung der Adreßtabelle.

Ein wichtiges Argument für den Einsatz von erweiterbarem Hashing für die Organisation externer Dateien ist neben der garantierten Effizienz der Suchoperation und der im Mittel akzeptablen Effizienz des Einfügens und Entfernens eine gute Speicherplatzausnutzung. Bei gleichverteilten Schlüsseln ergibt sich nach dem zufälligen Einfügen von n Datensätzen in die anfangs leere Hashdatei eine mittlere Anzahl von $(n/b) \ln 2$ Blöcken. Damit sind Blöcke durchschnittlich zu etwa 69 % belegt, wie dies auch für viele andere Strukturen gilt, die mit rekursivem Halbieren arbeiten. Im Unterschied dazu wächst die Größe der Adreßtabelle überlinear in n , mit $O((1/b)n^{1+1/b})$. Die Blockkapazität b spielt offensichtlich auch hier eine gewichtige Rolle. Eine genauere, aber kompliziertere Analyse der Größe der Adreßtabelle findet man in

4.5 Das Gridfile

In den vorangehenden Abschnitten haben wir das Problem des Speicherns und Wiederfindens von Schlüsseln mit genau einer Komponente, sogenannte *eindimensionale* Schlüssel, betrachtet. Bei vielen Datenverwaltungsproblemen hat man es aber mit *mehrdimensionalen* Schlüsseln, also Schlüsseln mit mehreren Komponenten, zu tun. So kann man beispielsweise einen Eintrag in einem Telefonbuch als aus zwei Komponenten bestehend ansehen: Die erste Komponente ist der Teilnehmername samt Adresse, die zweite Komponente die Telefonnummer. Organisiert man nun die Einträge in einer Datenstruktur gemäß der ersten Komponente, so wird die Suche nach Datensätzen mit gegebener zweiter Komponente im allgemeinen nicht unterstützt. Beispielsweise ist es nicht leicht, im Telefonbuch einen Teilnehmer mit gegebener Telefonnummer zu finden. Mehrdimensionale Hashverfahren versuchen hier, Abhilfe zu schaffen, indem mehrdimensionale Schlüssel so verwaltet werden, daß die Suche nach Datensätzen mit einigen vorgegebenen Schlüsselkomponentenwerten für alle Komponenten gleich gut unterstützt wird. Außerdem sollen natürlich das Einfügen und Entfernen von Datensätzen effizient möglich sein. Da es sich bei mehrdimensionalen Schlüsseln manchmal um geometrische Daten handelt, wie etwa Koordinaten von Punkten in der Ebene, ist es darüber hinaus wünschenswert, räumlich orientierte Anfragen zu unterstützen. So möchte man beispielsweise einen rechteckigen Ausschnitt aus einer Landkarte (mit Städten als Punkten) auf dem Bildschirm anzeigen. Um diese Punkte zu finden, führt man eine Bereichsanfrage aus. Bei der *Bereichsanfrage* fragt man nach allen Schlüsseln, deren sämtliche Komponenten in einen jeweils vorgegebenen Bereich (ein Schlüsselintervall)

fallen. Ist ein Schlüssel ein Paar kartesischer Koordinaten der Ebene, so ist der Bereich einer Bereichsanfrage ein achsenparalleles Rechteck. Auch im eindimensionalen Fall spielt die räumliche Nähe von Schlüsseln bereits eine gewisse Rolle, nämlich bei ordnungserhaltenden dynamischen Hashverfahren. Die mehrdimensionale Bereichsanfrage kann man als Verallgemeinerung der Suche nach einem Schlüssel mit anschließendem sequentiellen Inspizieren der benachbarten Schlüssel ansehen, wie sie durch Ordnungserhaltung unterstützt wird.

Sei d die Dimension der Schlüssel, und sei K_i das Universum der i -ten Schlüsselkomponente, $1 \leq i \leq d$. Dann ist $K = K_1 \times K_2 \times \dots \times K_d$ das Universum aller möglichen d -dimensionalen Schlüssel. Für die Menge der Dimensionen $D = \{1, \dots, d\}$ und $I \subseteq D$ betrachten wir genauer die folgenden Operationen:

- *Suchen* nach Schlüssel $k = (k_1, \dots, k_d) \in K$ mit vorgegebenem k_i für $i \in I$;
- *Bereichsanfrage* nach allen Schlüssel $k = (k_1, \dots, k_d) \in K$ mit $k_i^u \leq k_i \leq k_i^o$ für alle $i \in I$, wobei k_i^u die untere und k_i^o die obere Bereichsgrenze in Dimension i ist;
- *Einfügen* eines Schlüssels k ;
- *Entfernen* eines Schlüssels k .

Falls $I \subset D$, so sprechen wir von *partieller Suche* (*partial match query*) und *partieller Bereichsanfrage* (*partial range query*).

Bei mehrdimensionalen Hashverfahren versucht man nun, in Verallgemeinerung von eindimensionalen Verfahren den mehrdimensionalen Raum in mehrdimensionale Rechtecke einzuteilen, die gerade das Produkt eindimensionaler Intervalle sind. Für die einzelnen Dimensionen versucht man dann, übliche eindimensionale dynamische Hashverfahren zu verwenden. Jedem Teilraum des Datenraums wird genau ein Datenblock zugeordnet, wie wir dies schon von dynamischen Hashverfahren kennen; derselbe Block kann mehreren Teilräumen zugeordnet sein. Zur klaren Unterscheidung nennen wir einen einzelnen Teilraum *Gitterzelle*; die Vereinigung der Gitterzellen, denen derselbe Block zugeordnet ist, heißt *Blockregion*. Wir beschränken uns in diesem Abschnitt wegen der einfacheren Darstellung auf zweidimensionale Daten; die Verallgemeinerung für höhere Dimensionen sollte klar sein. Mit der Einteilung des Datenraums in rechteckige Gitterzellen erreicht man, daß alle in einem Block gespeicherten Punkte räumlich dicht beieinander liegen, eine günstige Voraussetzung für Bereichsanfragen.

Betrachten wir zunächst das in Abbildung 4.16 gezeigte Beispiel mit zweidimensionalen Schlüsseln, die als Punkte in der Ebene gezeichnet sind und mit zweidimensionalem erweiterbarem Hashing mit ordnungserhaltender Hashfunktion verwaltet werden. Wegen der besseren geometrischen Zuordnung geben wir die Hashadressen in Abbildung 4.16 in jeder Dimension in aufsteigender Sortierung an; die Speicherung der Adreßtabelle bleibt davon unberührt.

Wir verwalten also eine zweidimensionale Adreßtabelle, deren Spalten mit der einen und deren Zeilen mit der anderen eindimensionalen Hashfunktion adressiert werden, gemäß erweiterbarem Hashing (EXCELL in ). Ein Eintrag in der Adreßtabelle ist die Adresse desjenigen Datenblocks, in dem die Punkte der entsprechenden Gitterzelle gespeichert sind. Bei einer Blockkapazität b von zwei Datensätzen können wir die in

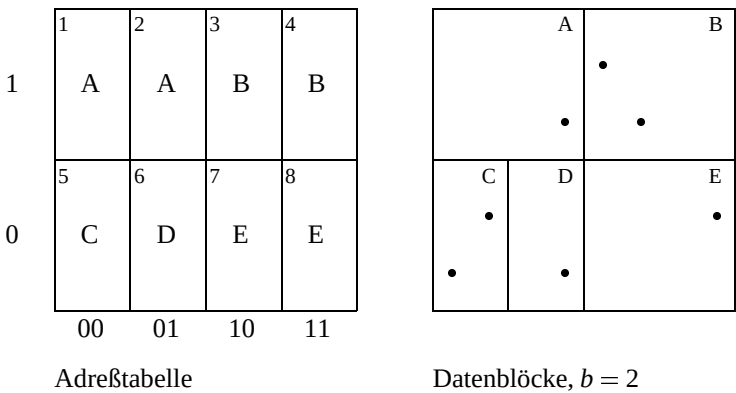


Abbildung 4.16

Abbildung 4.16 zeigte Aufteilung der Daten auf fünf Blöcke wählen. Als Folge der feinen Unterteilung des Datenraums in Teilräume durch die bei erweiterbarem Hashing gewählte Adreßtabelle gibt es auch in unserem Beispiel Blockregionen, die durch Vereinigung mehrerer Gitterzellen entstehen. In diesen Fällen gibt es mehrere Verweise von der Adreßtabelle auf den entsprechenden Datenblock. In unserem Beispiel ist dies so für die Datenblöcke A, B und E. Der Nachteil mehrerer Verweise auf Datenblöcke ist im mehrdimensionalen Fall aber leichter behebbar als im eindimensionalen: Schon wenige Hashadressen genügen, um viele Adreßtableneinträge zu verwalten, weil die Anzahl der Adreßtableneinträge das Produkt der Anzahlen der Hashadressen in den verschiedenen Dimensionen ist, und nicht — wie es im eindimensionalen der Fall wäre — deren Summe. Damit wird es attraktiv, die Hashadressen in allen Dimensionen explizit zu verwalten; für realistische Anwendungsfälle kann dies leicht im Hauptspeicher geschehen. Somit entfällt die Notwendigkeit zur Adreßtabellenverdoppelung, und damit läßt sich die Adreßtabelle zur Situation von Abbildung 4.16 wie in Abbildung 4.17 gezeigt angeben. In findet man eine Analyse der Größe der Adreßtabelle für beide Verfahren.

Ein sehr bekanntes und bewährtes mehrdimensionales Hashverfahren, das man als mehrdimensionales erweiterbares Hashing mit den angegebenen Modifikationen ansehen kann, ist das *Gridfile* , das wir im folgenden genauer erläutern werden. Die Einteilung des Datenraums jede Dimension geben wir hierbei in Koordinatenwerten statt in führenden Bits von Schlüsseln an, weil damit der geometrische Bezug einfacher erkennbar ist. Die Einteilung des Datenraums für jede Dimension heißt *Scale*; die Adreßtabelle heißt *Directory*. Die Scales werden im Hauptspeicher verwaltet, die Directory-Matrix wird dagegen extern gespeichert. Dies geschieht mit dem Ziel der Zwei-Zugriffs-Garantie für die exakte Suche, wie bei erweiterbarem Hashing. Überdies erreicht man beim Gridfile, daß die partielle Suche für jede spezifizierte Schlüsselkomponente gleichermaßen effizient ist.

Wir werden ein Gridfile im folgenden kompakter graphisch darstellen, indem wir die Aufteilung des Datenraums in Gitterzellen gemäß der Adreßtabelle und die Aufteilung

1	1 A	2 A	3 B
	4 C	5 D	6 E
0	00	01	1

Abbildung 4.17

des Datenraums in Blockregionen übereinander zeichnen. Gestrichelte Linien trennen dabei Gitterzellen, die zur selben Blockregion gehören; durchgezogene Linien trennen Regionen. Außerdem vermerken wir die Scales in jeder Dimension, die Directoryadressen und die Datenblockadressen (siehe Abbildung 4.18).

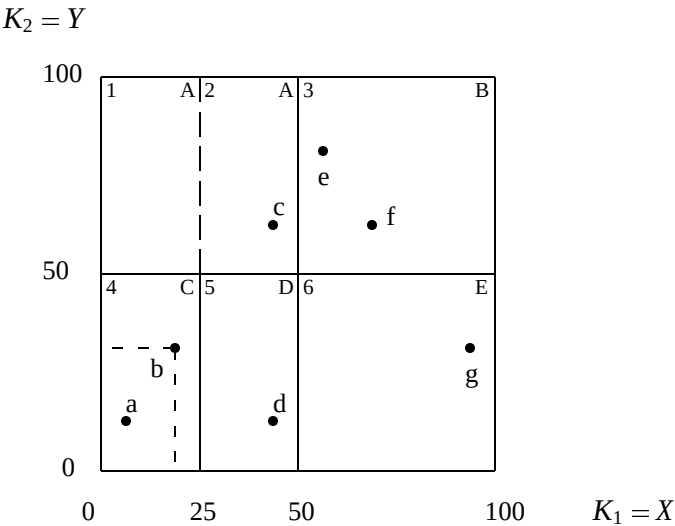


Abbildung 4.18

Bezeichnen wir für $d = 2$ K_1 mit X , K_2 mit Y , k_1 mit x und k_2 mit y , so kann die *exakte Suche* nach $k = (x,y)$ wie folgt durchgeführt werden:

1. Bestimme anhand der X -Scales die Spalte s der Directory-Matrix, in die x fällt; bestimme anhand der Y -Scales die Zeile z der Directory-Matrix, in die y fällt.
2. Berechne die Externspeicheradresse a_1 des Directory-Elements in Zeile z und Spalte s .
3. Lies den Directory-Block *dir* mit Adresse a_1 in den Hauptspeicher.
4. Bestimme die Externspeicheradresse a_2 des Datenblocks zu derjenigen Gitterzelle in *dir*, in die (x,y) fällt.
5. Lies den Datenblock *dat* mit Adresse a_2 in den Hauptspeicher.
6. Durchsuche *dat* nach (x,y) und berichte das Ergebnis.

In dem in Abbildung 4.18 gezeigten Beispiel führt die Suche nach Punkt $b = (20, 38)$ zur Bestimmung der zweiten Zeile (von oben) und der ersten Spalte (von links) der Directory-Matrix und damit zum Directory-Element mit Adresse 4. Dieses enthält den Verweis auf Datenblock C , in dem die Punkte a und b gespeichert sind. Die Suche nach b endet also erfolgreich.

Lediglich in Schritten 3 und 5 des Algorithmus zur exakten Suche findet je ein Externzugriff statt; die exakte Suche benötigt also stets genau zwei Zugriffe, wenn die Scales im Hauptspeicher verwaltet werden — das *Zwei-Zugriffs-Prinzip* des Gridfiles.

Damit sollte auch klar sein, wie die partielle Suche und die Bereichsanfrage beantwortet werden können. Bei der Bereichsanfrage etwa sucht man zunächst nach dem linken unteren Punkt des rechteckigen Anfragebereichs und überprüft für alle Punkte im gefundenen Datenblock, ob sie im Anfragebereich liegen. Dann setzt man die Suche nach rechts und nach oben über benachbarte Zeilen und Spalten und daraus berechenbare Directoryelemente fort. Das bedeutet, daß auch auf der Directory-Matrix eine Bereichsanfrage durchgeführt wird: Gesucht sind alle Gitterzellen, die den Anfragebereich schneiden. Als Folge davon muß die Directory-Matrix, die ja wegen ihrer Größe im allgemeinen auch auf Externspeicher verwaltet wird, dieselben Operationen unterstützen wie die Datenstruktur für die ursprünglich gegebenen Datenpunkte. Es ist also vernünftig, Gitterzellen ebenso wie Datenpunkte in einem Gridfile zu organisieren. Dies führt zum Mehr-Ebenen-Gridfile , das für die meisten realen Anwendungsfälle mit nur zwei Ebenen auskommt, wenn ein großes Wurzel-Directory im Hauptspeicher gehalten werden kann .

Nehmen wir für das Beispiel der in Abbildung 4.16 gezeigten Datenpunkte an, daß jeder Datenblock $b = 2$ Punkte, jeder Directory-Block $b' = 2$ Adressen von Datenblöcken und das Wurzel-Directory $b'' = 4$ Adressen von Directoryblöcken speichern kann. Dann ergibt sich für die gezeigten Datenblöcke A, B, C, D und E das in Abbildung 4.19 gezeigte 2-Ebenen-Directory mit Directoryblöcken A', B' und C' und einem Wurzeldirectory. Eine Bereichsanfrage mit dem Anfragebereich $[40 \dots 60] \times [40 \dots 60]$ führt in der gezeigten Situation im Wurzeldirectory auf die Directoryblockadressen A', B' und C' , und für diese Directoryblöcke auf die Datenblockadressen A, B, D, E . Die Effizienz einer Bereichsanfrage ist also nach unten beschränkt durch die Effizienz der exakten

Suche; mit größer werdenden Anfragebereichen steigt in der Tendenz auch die Anzahl der als Antwort gefundenen Datensätze und die Anzahl der benötigten Externzugriffe. Die Effizienz von Bereichsanfragen mit großen Anfragebereichen ist eng an die Speicherplatzausnutzung gekoppelt, weil Externzugriffe, die wenig zur Antwort beitragen, nur für Gitterzellen und Datenblockregionen am Rand des Anfragebereiches ausgeführt werden müssen. Eine genaue Analyse ergibt, daß im Mittel $O(n^{1-|I|/d})$ Externzugriffe für die partielle Suche nach $|I|$ von d Schlüsseln in einem Gridfile mit n Datensätzen ausreichen. Diese Effizienz wird für optimal gehalten. Dabei ist es natürlich stets wichtig, daß sich das Gridfile an dynamisch veränderliche Datenmengen anpaßt. Wir werden im folgenden genauer betrachten, wie dies bei Einfüge- und Entferneoperationen geschieht.

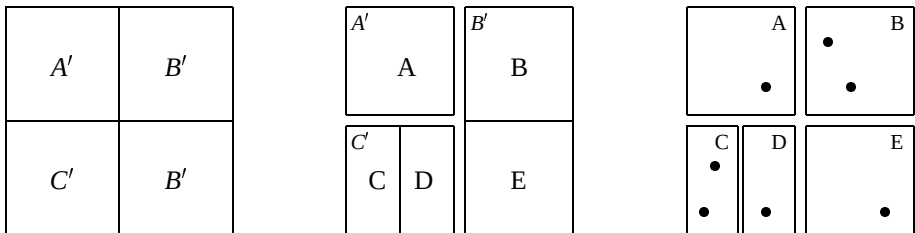


Abbildung 4.19

Beim *Einfügen* eines Datensatzes wird zunächst durch eine exakte Suche der Datenblock ermittelt, in den der Datensatz einzufügen ist. Sofern der Datensatz in diesem Block noch Platz findet, wird er dort eingefügt, der Block auf den Externspeicher zurückschrieben, und die Einfügeoperation ist beendet. Andernfalls muß ein neuer Datenblock kreiert werden. Zu diesem Zweck wird der fragliche Block in zwei Blöcke geteilt, indem seine Region entlang einer Koordinatenachse in der Mitte *zerschnitten* (*gesplittet*, englisch: *split*) wird — ein *Datenblocksplit*. Die Datensätze werden gemäß der beiden neuen Datenblockregionen auf die beiden neuen Datenblöcke aufgeteilt. Die neue Situation muß im Directory vermerkt werden. Weil das Directory (als Ganzes beim Ein-Ebenen-Directory und als lokaler Directoryblock im Mehr-Ebenen-Directory) als Matrix organisiert bleiben muß, durchtrennt die Splitlinie in allen von der Splitdimension verschiedenen Dimensionen den gesamten (zum Directoryblock lokalen) Datenraum. Wie schon bei erweiterbarem Hashing kann hier natürlich der Fall auftreten, daß ein Blocksplit nicht zum wirklichen Verteilen von Datensätzen führt, daß einer der neu geschaffenen Blöcke also leer bleibt; in diesem Fall wird der Blocksplit rekursiv für den noch immer übergroßen Block fortgesetzt. Somit ist nur noch die Wahl der Splitdimension bei einem Blocksplit offen. Betrachten wir dazu das in Abbildung 4.18 gezeigte Beispiel und nehmen wir an, daß ein Directoryblock $b' = 6$ Datenblockadressen verwalten kann; Abbildung 4.18 zeigt gerade einen Directoryblock mit Verweisen auf die Datenblöcke A, B, C, D und E. Im folgenden geben wir drei Regeln an, von denen die

erste in dieser Reihenfolge angewendet wird, die eine eindeutige Splitentscheidung liefert:

- (1) Teile die längste Seite einer Datenblockregion.
Soll im Beispiel der Abbildung 4.18 Datenblockregion *C* geteilt werden, so findet ein *waagerechter Split* statt, also eine Aufteilung der Region $[0 \dots 25] \times [0 \dots 50]$ in Regionen $[0 \dots 25] \times [0 \dots 25]$ und $[0 \dots 25] \times [25 \dots 50]$. Wegen der Matrixeigenschaft des Directoryblocks sind damit zwei Verweise auf Datenblock *D* und zwei Verweise auf Datenblock *E* erforderlich; die Anzahl der Verweise kann sich durch einen Split also mehr als eigentlich nötig erhöhen.
- (2) Teile eine Datenblockregion gemäß einer vorhandenen Einteilung in Gitterzellen.
Soll im Beispiel der Abbildung 4.18 die Datenblockregion *A* geteilt werden, so erfolgt ein vertikaler Split, weil Regel 1 keine eindeutige Entscheidung liefert und gemäß Regel 2 die bereits vorhandene vertikale Splitlinie verwendet werden muß. Im Directory wird lediglich ein Teil der Verweise geändert; die Gitterzeleinteilung ändert sich nicht.
- (3) Teile eine Datenblockregion in derjenigen Dimension, in der die kleinste Anzahl von Teilungen vermerkt ist.
Im Beispiel der Abbildung 4.18 hat demnach ein Split der Datenblockregion *B* in waagerechter Richtung zu erfolgen.

Liefert keine dieser Regeln eine eindeutige Entscheidung, so wird die Blockregion entlang einer beliebigen Dimension geteilt, etwa abwechselnd nach *X* und *Y*. Die vorgestellte Splitstrategie präferiert keine der Dimensionen vor einer anderen, führt also in der Tendenz zu Blockregionen, deren Verhältnis von Länge zu Breite möglichst nahe bei 1 liegt. Im Unterschied zu directorylosen Strukturen ist es beim Gridfile (wie schon bei erweiterbarem Hashing) nicht erforderlich, leere Datenblöcke explizit zu speichern. Statt dessen genügt es, entsprechend markierte Verweise im Directory zu verwalten.

Die Teilung eines Datenblocks führt im entsprechenden Directoryblock im allgemeinen zur Erhöhung der Anzahl der zu verwaltenden Verweise. Läuft der Directoryblock über, so wird auch dieser geteilt. Man kann hier im wesentlichen dieselben Regeln verwenden wie beim Teilen einer Datenblockregion. Beim Teilen einer Directoryblockregion muß die Einteilung der beiden resultierenden Blöcke in Gitterzellen überprüft werden, weil diese als Folge der Teilung günstiger werden kann. Betrachten wir dazu als Beispiel Abbildung 4.18 mit einer Directoryblockkapazität von $b' = 5$ und Datenblockkapazität $b = 2$ und nehmen wir an, daß der gezeigte Directoryblock soeben durch Einfügen des Punktes *b* und damit durch Einziehen der Splitlinie $x = 25$ mit der Verfeinerung der Einteilung von vier auf sechs Gitterzellen übergeworden ist. Teilen wir nun die Directoryblockregion (willkürlich) waagerecht, so entfällt die Notwendigkeit, Datenblockregion *A* in zwei Gitterzellen aufzuteilen; wir kommen also mit fünf Verweisen auf die fünf Datenblöcke aus, die allerdings nicht in einem Directoryblock untergebracht werden können (vgl. Abbildung 4.20). Denselben Effekt können wir bereits in Abbildung 4.19 gegenüber Abbildung 4.18 beobachten.

Das *Löschen* eines Datensatzes aus einem Gridfile wird realisiert durch eine exakte Suche nach dem zu löschenden Datensatz, gefolgt vom anschließenden Entfernen des Datensatzes im entsprechenden Datenblock und Zurückschreiben dieses Blocks. Im

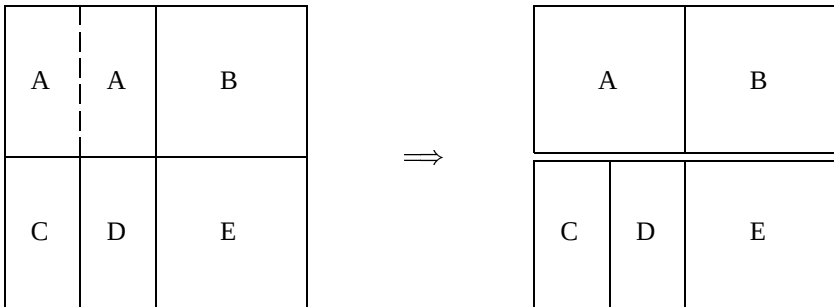


Abbildung 4.20

Unterschied zum Einfügen sind nach dem Löschen keine weiteren Aktionen zwingend erforderlich; im Interesse einer guten Speicherplatzausnutzung, die ja auch für die Effizienz von Anfragen wichtig ist, sind solche Aktionen dennoch geboten. Symmetrisch zum Aufteilen (Split) einer Region bei einem Blocküberlauf nach einer Einfügeoperation kann man nach einer Entferneoperation zwei Blöcke *verschmelzen* (englisch: *merge*), um die Speicherplatzausnutzung nicht unter ein gewisses Mindestmaß absinken zu lassen. Damit sich in einem dynamischen Anwendungsfall, mit weiteren noch zu erwartenden Einfüge- und Entferneoperationen, nicht ständig Teile- und Verschmelzeoperationen abwechseln, wird eine Verschmelzeoperation nur nach schrittweiser Überprüfung zweier Bedingungen durchgeführt. Zunächst muß die Speicherplatzausnutzung für den Datenblock, aus dem ein Datensatz soeben gelöscht wurde, eine vorgegebene Schranke unterschreiten, damit eine Verschmelzeoperation überhaupt erwogen und die dafür notwendigen Externzugriffe ausgeführt werden. Liegt eine solche *Schranke für die Überprüfung der Verschmelzung* etwa bei 30 %, so ist einerseits sichergestellt, daß Verschmelzeoperationen nicht allzu häufig unternommen werden, und andererseits sind die Aussichten auf einen genügend schwach gefüllten Partnerblock für die Verschmelzung nicht allzu schlecht. Liegt die Füllung eines Datenblocks unterhalb dieser Schranke, so wird unter allen gemäß der Gitterzelleneinteilung und der Verschmelzestrategie möglichen Partnern für eine Verschmelzung derjenige mit der schwächsten Füllung ermittelt. Eine obere *Schranke für das Durchführen der Verschmelzung* — typischerweise bei etwa 70 % — gibt die höchste nach der Verschmelzung beider Blöcke akzeptable Speicherplatzausnutzung an, bei der die Verschmelzung noch durchgeführt wird.

Die *Verschmelzestrategie* legt fest, welche Regionen überhaupt als Partner für eine Verschmelzung in Frage kommen. Dabei wird stets gefordert, daß die durch die Verschmelzung entstehende Blockregion rechteckig ist. In dem in Abbildung 4.18 gezeigten Beispiel ist damit ein Verschmelzen der Blockregionen A und C nicht zulässig. Die *Nachbarstrategie* läßt nun alle Verschmelzungen zu, bei denen ein rechteckiger Bereich entsteht. So können etwa gemäß der Nachbarstrategie die Regionen D und E in Abbildung 4.18 verschmolzen werden; bei Datenblockkapazität $b = 2$ passen auch tatsächlich die Inhalte beider Blöcke zusammen in einen Block. Im Hinblick auf eine hohe Speicherplatzausnutzung scheint diese am wenigsten restriktive Verschmelzestrategie

ganz besonders günstig zu sein. Daß dies nicht unbedingt so ist, zeigt das Beispiel in Abbildung 4.21. Dort sieht man, daß nach der gezeigten und gemäß Nachbarstrategie zulässigen Verschmelzung von Block A mit Block E, B mit C und D, F mit G, H mit L und I mit J und K keine weitere Verschmelzung mehr möglich ist, selbst wenn fast alle Datensätze entfernt werden — eine *Verklemmung (deadlock)*. Die Speicherplatzausnutzung kann also hier beliebig absinken. Da man dies auf alle Fälle vermeiden möchte, muß man bei Anwendung der Nachbarstrategie Verklemmungen durch entsprechende Prüfung beim Verschmelzen verhindern. Es sollte klar sein, daß dies nicht immer ganz einfach und effizient möglich ist.

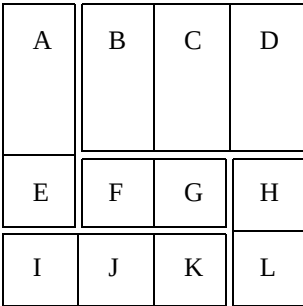


Abbildung 4.21

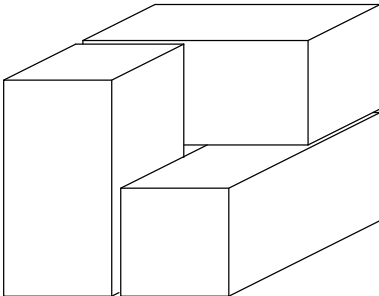


Abbildung 4.22

Die *Bruderstrategie (buddy merge)* erlaubt nur das Verschmelzen solcher Blöcke, die durch eine Teilung aus einem gemeinsamen Block hervorgegangen sein können. In diesem Fall macht eine Verschmelzung gerade eine Teilung rückgängig. Während eine Region höchstens einen Bruder in jeder Dimension hat, besitzt sie in jeder Dimension bis zu zwei Nachbarn; im zweidimensionalen Fall kann man also bei der Nachbarstrategie unter bis zu vier Partnern wählen, bei der Bruderstrategie aber höchstens unter

zweiten. In dem in Abbildung 4.21 gezeigten Beispiel etwa hat Region G die beiden Brüder H und K und zusätzlich die beiden Nachbarn F und C . F ist kein Bruder von G , weil F und G nicht durch einen Split aus einer Region hervorgegangen sein können; die Gitterzellengrenze, die F von G trennt, muß zeitlich vor einer anderen G begrenzenden Linie eingeführt worden sein. Dagegen ist G entweder durch Abtrennen von H oder durch Abtrennen von K entstanden; jede dieser beiden Regionen kann als Partner beim Verschmelzen dienen. Die Bruderstrategie stellt im zweidimensionalen Fall sicher, daß keine Verklemmung auftritt; bereits im dreidimensionalen sind aber Verklemmungen möglich, wie Abbildung 4.22 zeigt.

Weil eine Region in jeder Dimension einen Bruder haben kann, ist es manchmal sinnvoll, unmittelbar nach der Aufteilung einer Region in zwei neue Regionen die Möglichkeit der Verschmelzung, gewissermaßen mit dem anderen Bruder, zu überprüfen. So führt etwa in dem in Abbildung 4.23 gezeigten Beispiel bei einer Datenblockkapazität von $b = 3$ Datensätzen das Einfügen des Datensatzes k zunächst zu einem Aufteilen des Blocks A auf die Blöcke A und D ; bei einer oberen Schranke von 35 % für das Überprüfen und von 70 % für das Durchführen der Verschmelzung kann aber dann D mit C verschmolzen und somit die Speicherplatzausnutzung verbessert werden.

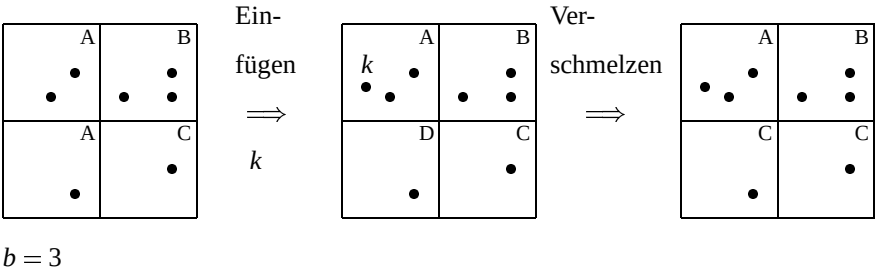


Abbildung 4.23

Das Verschmelzen von Directoryblöcken unterscheidet sich vom Verschmelzen von Datenblöcken durch die Notwendigkeit der Anpassung der Gitterzelleneinteilungen der beiden zu verschmelzenden Blöcke. Während beim Teilen von Directoryblöcken Splitlinien entfallen können, kann das Verschmelzen eine Verfeinerung der Einteilung bewirken. Zur Illustration dieses Phänomens können wir Abbildung 4.20 von rechts nach links lesen. Nehmen wir an, daß Blockregion B durch Verschmelzen zweier Blockregionen nach dem Entfernen eines Datensatzes entstanden ist, und daß die Schranken für das Prüfen und Durchführen einer Verschmelzung vorschreiben, die beiden rechts in Abbildung 4.20 dargestellten Directoryblöcke zu verschmelzen. Das Resultat der Verschmelzung ist der links in Abbildung 4.20 dargestellte Directoryblock, der aber nicht nur fünf, sondern sechs Regionen verwalten muß. Dieser Effekt muß vor der Durchführung der Verschmelzung zweier Directoryblöcke bedacht werden, weil sonst im Extremfall der resultierende Directoryblock bereits wieder übertoll sein kann (in unserem Beispiel wäre dies der Fall für Directoryblockkapazität $b' = 5$).

Eine Analyse des durchschnittlichen Verhaltens des Gridfiles hat sich als schwierig herausgestellt. Simulationen haben gezeigt, daß die durchschnittliche Auslastung von Datenblöcken in vielen Situationen bei etwa 70 % (ungefähr $\ln 2$) liegt, ein Wert, der sich für viele Strukturen ergibt, die mit rekursivem Halbieren arbeiten (z.B. B+-Bäume). Analytische Überlegungen zum Verhalten von Gridfiles findet man in [1] und [2]. Bei Datenblockkapazität b wächst das Directory des Gridfiles bei n gleichverteilten Datensätzen mit $O(n^{(1+1/b)})$, wie dies auch schon bei erweiterbarem Hashing der Fall war. Bei einer ungünstigen Verteilung der Datensätze, im zweidimensionalen Fall etwa entlang einer Diagonalen, wächst das Directory sogar mit $O(n^d)$ für ein d -dimensionales Gridfile. Trotz dieses relativ schlechten schlimmsten Falles ist das Gridfile eine für viele Anwendungen geeignete mehrdimensionale Datenstruktur.

4.6 Aufgaben

Aufgabe 4.1

Wieviele Schritte werden im schlechtesten Fall benötigt, um in eine anfangs leere Hashtabelle n Schlüssel einzufügen, wenn zur Überlaufbehandlung die Methode der separaten Verkettung mit unsortierten bzw. sortierten Listen verwendet wird? Wieviele Schritte benötigt man in diesen beiden Fällen, um nach jedem der n eingefügten Schlüssel einmal zu suchen?

Aufgabe 4.2

Zeigen Sie, daß die mittlere Anzahl von Hashtabellenplätzen, die bei einer erfolgreichen Suche (mit gleicher Wahrscheinlichkeit für alle Schlüssel) inspiziert werden, bei Hashing mit linearem Sondieren nicht von der Reihenfolge abhängt, in der die Schlüssel in die anfangs leere Hashtabelle eingefügt worden sind.

Gilt die entsprechende Aussage auch für quadratisches Sondieren?

Aufgabe 4.3

Geben Sie die Belegung einer Hashtabelle der Größe 13 an, wenn die Schlüssel

5, 1, 19, 23, 14, 17, 32, 30, 2

in die anfangs leere Tabelle eingefügt werden und offenes Hashing mit Hashfunktion $h(k) = k \bmod 13$ und

- linearem Sondieren;
- linearem Sondieren mit Sondierungsfunktion $s(j, k) = -j$;
- quadratischem Sondieren

verwendet wird.

Vergleichen Sie die Anzahlen der beim Einfügen betrachteten Hashtabellenplätze für diese drei Sondierungsverfahren. Welche Kosten sind für eine erfolgreiche Suche zu erwarten, wenn nach jedem vorhandenen Schlüssel mit gleicher Wahrscheinlichkeit gesucht wird?

Aufgabe 4.4

Gegeben seien eine Hashtabelle der Größe 7 mit der Belegung

	0	1	2	3	4	5	6
$t :$	1	164	8	21	73	22	89

und die Hashfunktion $h(k) = (\text{Quersumme}(k)) \bmod 7$. Als Kollisionsstrategie wird quadratisches Sondieren angewandt.

- Geben Sie alle Reihenfolgen an, in denen die Schlüssel in die anfangs leere Hash-tabelle eingefügt worden sein können.
- Gibt es eine andere Reihenfolge, die zu einer geringeren durchschnittlichen Anzahl zu inspizierender Hashtabellenplätze bei der erfolgreichen Suche führt, wenn die Suche nach jedem Schlüssel gleich wahrscheinlich ist?

Aufgabe 4.5

Gegeben sei eine anfangs leere Hashtabelle mit 13 Elementen, in die der Reihe nach die Schlüssel 14, 21, 27, 28, 8, 18, 15, 36, 5, 2 mit Double Hashing eingefügt werden sollen. Die zu verwendenden Hashfunktionen seien $h(k) = k \bmod 13$ und $h'(k) = 1 + k \bmod 11$. Geben Sie die Belegung der Hashtabelle an, wenn die Schlüssel

- in der gegebenen Reihenfolge;
- in sortierter Reihenfolge;
- in der gegebenen Reihenfolge mit Brents Algorithmus;
- in sortierter Reihenfolge mit Brents Algorithmus;
- in der gegebenen Reihenfolge mit Binärbaum-Sondieren;
- in sortierter Reihenfolge mit Binärbaum-Sondieren;
- in der gegebenen Reihenfolge mit Ordered Hashing;
- in sortierter Reihenfolge mit Ordered Hashing

eingefügt werden.

Wieviele Hashtabellenplätze müssen beim Einfügen eines der Schlüssel, bei der erfolgreichen und bei der erfolglosen Suche jeweils höchstens inspiziert werden?

Aufgabe 4.6

- Sind die beiden bei Double Hashing verwendeten Hashfunktionen $h(k) = k \bmod 7$ und $h'(k) = 1 + k \bmod 5$ unabhängig?
- Ist $h'(k) = k^2 \bmod 7$ eine für h geeignete zweite Hashfunktion?

Aufgabe 4.7

Lösen Sie Aufgabe 4.5 für Robin-Hood-Hashing. Vergleichen Sie die erwartete Anzahl inspizierter Hashtabelleneinträge für die erfolgreiche Suche (bei gleicher Suchwahrscheinlichkeit für jeden Schlüssel) bei den Fällen a) bis h) der Aufgabe 4.5 mit Robin-Hood-Hashing mit dem Standard-Suchalgorithmus und mit smart searching. Dabei soll für smart searching als Erwartungswert der Länge von Sondierungsfolgen gerade deren Mittelwert für die gespeicherten Schlüssel verwendet werden.

Aufgabe 4.8

Lösen Sie Aufgabe 4.5 für Coalesced Hashing ohne Keller. Vergleichen Sie auch die Effizienz der erfolgreichen Suche (vgl. Aufgabe 4.7). Bei welcher Kellergröße ist im Beispiel die erfolgreiche Suche am schnellsten, wenn die Hashfunktion weiterhin nach der Divisions-Rest-Methode gewählt wird? Wie lang ist dann die längste Überlaufkette? Bei welcher Kellergröße ist im Beispiel die längste Überlaufkette am kürzesten, und wie schnell ist dann die erfolgreiche Suche?

Aufgabe 4.9

Verfolgen Sie die Entwicklung einer nach linearem Hashing organisierten Hashdatei mit Datenblockkapazität $b = 2$, wenn in die anfangs aus drei leeren Blöcken bestehende Datei die Schlüssel 5, 12, 43, 16, 19, 1990, 53 in dieser Reihenfolge eingefügt werden. Verwenden Sie dazu Hashfunktionen nach der Divisions-Rest-Methode und den Schwellenwert 0.8 für den Belegungsfaktor als Auslöser einer Block-Split-Operation.

- Wieviele Blöcke werden für die ersten vier, wieviele für die ersten fünf und wieviele für alle sieben Schlüssel verwendet?
- Kommt es im Verlauf des Einfügens vor, daß sich die Anzahl der im Mittel für die erfolgreiche Suche benötigten Externzugriffe verringert, obwohl sich die Anzahl gespeicherter Schlüssel erhöht hat? Welches ist der beste Wert, welches der schlechteste?
- Gelangt man für die gegebene Schlüsselfolge zu einer besseren Speicherplatzausnutzung oder einer besseren mittleren Anzahl von Externzugriffen für die erfolgreiche Suche, wenn man mit einer anderen anfänglichen Dateigröße beginnt oder einen anderen Schwellenwert für den Belegungsfaktor wählt? Welches sind die besten Werte?
- Wie ändert sich die Situation bei Verwendung einer ordnungserhaltenden Hashfunktion?

Aufgabe 4.10

Geben Sie eine genaue algorithmische Beschreibung für das Entfernen eines Datensatzes einschließlich des Verschmelzens von Blöcken an. Betrachten Sie die in der Aufgabenstellung der Aufgabe 4.9 beschriebene Situation, und verfolgen Sie die Entwicklung der Hashdatei, wenn alle Schlüssel in *derselben* Reihenfolge wieder entfernt werden, in der sie eingefügt wurden. Beantworten Sie die Fragen a) bis d) von Aufgabe 4.9 entsprechend.

Aufgabe 4.11

Betrachten Sie eine anfangs aus einem leeren Block bestehende Hashdatei mit Blockkapazität $b = 2$, die mit linearem Hashing organisiert ist, wobei die Hashfunktionen nach der Divisions-Rest-Methode gebildet werden und ein Block-Split stattfindet, wenn der Belegungsfaktor den Wert 1 erreicht. Geben Sie je eine Folge von n Schlüsseln an, die in der gegebenen Reihenfolge in die leere Hashdatei eingefügt werden, so daß

- die mittlere Anzahl von Externzugriffen für die erfolgreiche Suche linear von n abhängt und sich nach jeder Dateiverdopplung für jeden Schlüssel die Hashadresse ändert;
- zu keinem Zeitpunkt Überlaufblöcke erforderlich sind;
- unter den während einer Expansion noch nicht gesplitteten Blöcken stets soviele Blöcke überlaufen, wie in dieser Expansion bereits gesplittet worden sind (aber höchstens alle noch nicht gesplitteten Blöcke).

Aufgabe 4.12

Geben Sie für virtuelles Hashing ohne und mit Adreßtabelle eine genaue algorithmische Beschreibung für das Entfernen eines Datensatzes einschließlich dem Verschmelzen von Blöcken an.

Aufgabe 4.13

Geben Sie für erweiterbares Hashing genaue algorithmische Beschreibungen an für Suchen, Einfügen und Entfernen von Datensätzen einschließlich Aufteilen und Verschmelzen von Blöcken und Verdoppeln und Halbieren der Adreßtabelle. Ein leerer Block kann explizit gespeichert, durch einen ihm eigenen Verweis dargestellt, oder durch einen für alle Blöcke gleichen Verweis dargestellt werden; wie unterscheiden sich die Algorithmen?

Aufgabe 4.14

In einem zweidimensionalen Gridfile reicht das Universum der ganzzahligen Schlüssel beider Dimensionen von 0 bis 20. Ein Datenblock kann höchstens vier Punkte, ein Directory-Block höchstens vier Verweise speichern. Beim Split wird eine Region im Zweifel senkrecht geteilt. Fügen Sie in das anfangs leere Gridfile die Punkte (4,6), (8,10), (18,4), (3,16), (14,18), (16,13), (11,2), (18,8), (12,9), (13,7), (20,7) und (16,2) ein.

- Wieviele Externzugriffe verursacht die teuerste der Einfügeoperationen, wenn von einer Operation zur nächsten kein Block im Hauptspeicher gepuffert wird? Wie lautet die Antwort, wenn ein Directory-Block jeder Ebene und ein Datenblock gepuffert werden?
- Wie hoch ist die Speicherplatzausnutzung von Datenblöcken, wie hoch die von Directory-Blöcken, in der nach dem Einfügen aller Punkte entstandenen Situation im Mittel und für den am besten und den am schlechtesten ausgenutzten Block?

- c) Geben Sie eine Bereichsanfrage an, bei der die Anzahl gelesener Punkte, die nicht zur Antwort gehören, am höchsten ist. Wieviele Blöcke können dabei höchstens gelesen werden?
- d) Geben Sie eine erfolgreiche und eine erfolglose partielle Suchanfrage an, bei der die Anzahl gelesener Blöcke am höchsten ist. Wieviele Punkte werden dabei höchstens gelesen, wieviele mindestens?

Aufgabe 4.15

Entwerfen Sie einen Algorithmus zur Beantwortung einer Anfrage nach einem nächsten Nachbarn (nearest neighbor, best match) eines gegebenen Anfragepunktes in einem zweidimensionalen Gridfile. Der nächste Nachbar eines Anfragepunktes ist derjenige Punkt in der betrachteten Menge, der zum Anfragepunkt die geringste Distanz hat.

Beziehen Sie neben der euklidischen Metrik (L_2) auch die Manhattan-Metrik (L_1) und die Maximums-Metrik (L_∞) in Ihre Überlegungen ein. Zur Erinnerung: Die Distanz d_i in Metrik L_i zwischen zwei Punkten (x, y) und (x', y') ist definiert als $d_i((x, y), (x', y')) = (|x - x'|^i + |y - y'|^i)^{1/i}$.

Aufgabe 4.16

Entwerfen Sie einen Algorithmus, der für ein zweidimensionales Gridfile mit Nachbar-Verschmelze-Strategie das Entstehen von Verklebungen verhindert.

Literaturliste zu Kapitel 4: Hashverfahren

Seite 171

- [52] W. Feller. An Introduction to Probability Theory and its Applications, Volume I. John Wiley & Sons, New York, 1968.
- [89] D. E. Knuth. The Art of Computer Programming, Vol.3: Sorting and Searching. Addison-Wesley, Reading, Massachusetts, 1973.

Seite 172

- [185] V. Turan So's. On the theory of diophantine approximations. Acta Math. Acad. Sci. Hung., 8:461-472, 1957.
- [89] D. E. Knuth. The Art of Computer Programming, Vol.3: Sorting and Searching. Addison-Wesley, Reading, Massachusetts, 1973.
- [175] T. A. Standish. Data Structure Techniques. Addison-Wesley, Reading, Massachusetts, 1980.
- [113] V. Y. Lum, P. S. T. Yuen und M. Dodd. Key-to-address transform techniques: a fundamental performance study on large existing formatted files. Comm. ACM, 14:228-235, 1971.

Seite 176

- [25] J. L. Carter und M. N. Wegman. Universal classes of hash functions. Journal of Computer and System Sciences, 18:143-154, 1979.
- [123] K. Mehlhorn. Datenstrukturen und effiziente Algorithmen, Band 1, Sortieren und Suchen. Teubner, Stuttgart, 1986.

Seite 180

- [89] D. E. Knuth. The Art of Computer Programming, Vol.3: Sorting and Searching. Addison-Wesley, Reading, Massachusetts, 1973.

Seite 181

- [146] W. W. Peterson. Addressing for random-access storage. IBM J. Research and Development, 1:130-146, 1957.

Seite 185

- [89] D. E. Knuth. The Art of Computer Programming, Vol.3: Sorting and Searching. Addison-Wesley, Reading, Massachusetts, 1973.

Seite 186

- [155] C. E. Radtke. The use of quadratic residue search. Comm. ACM, 13:103-105, 1970.
- [89] D. E. Knuth. The Art of Computer Programming, Vol.3: Sorting and Searching. Addison-Wesley, Reading, Massachusetts, 1973.

Seite 187

- [188] J. D. Ullman. A note on the efficiency of hash functions. J. Assoc. Comput. Mach.,
- [198] A. C. Yao. Uniform hashing is optimal. J. Assoc. Comput. Mach., 32(3):687-693,1985.

Seite 192

- [13] J .R. Bell und C. H. Kaman. The linear quotient hash code. Comm. ACM, 13:675-677, 1970.

Seite 193

- [23] R. P. Brent. Reducing the retrieval time of scatter storage techniques. Comm. ACM, 16:105-109, 1973.
- [69] G. H. Gonnet und I. Munro. Efficient ordering of hash tables. SIAM J. Comput., 8(3):463-478, 1979.
- [116] E. G. Mallach. Scatter storage techniques: A unifying viewpoint and a method for reducing retrieval times. The Computer Journal, 20(2):137-140, 1977.

Seite 194

[69] G. H. Gonnet und I. Munro. Efficient ordering of hash tables. SIAM J. Comput., 8(3):463-478, 1979.

Seite 198

[8] O. Amble und D. E. Knuth. Ordered hash tables. Computer Journal, 17:135-142, 1974.

Seite 199

[26] P. Celis. Robin Hood Hashing. Ph.D. dissertation, Technical Report CS-86-14, Waterloo, Ontario, Canada, 1986.

[27] P. Celis, P.-A. Larson und J. I. Munro. Robin Hood hashing. In Proc. 26th Annual Symposium on Foundations of Computer Science, S. 281-288. Computer Society Press of the IEEE, 1985.

[161] R. L. Rivest. Optimal arrangement of keys in a hash table. J. Assoc. Comput. Mach., 25(2):200-209, 1978.

[69] G. H. Gonnet und I. Munro. Efficient ordering of hash tables. SIAM J. Comput., 8(3):463-478, 1979.

[148] G. Poonan. Optimal Placement of Entries in Hash Tables. ACM Computer Science Conference, 25, 1976.

[92] D. König. Graphok e's matrixok. Matematikai e's Fizikai Lapok, 38:116-119, 1931.

[60] M. L. Fredman und R. E. Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. J. Assoc. Comput. Mach., 34:596-615, 1987.

[114] G. E. Lyon. Packed scatter tables. Comm. ACM, 21(10):857-865, 1978.

Seite 200

[27] P. Celis, P.-A. Larson und J. I. Munro. Robin Hood hashing. In Proc. 26th Annual Symposium on Foundations of Computer Science, S. 281-288. Computer Society Press of the IEEE, 1985.

[26] P. Celis. Robin Hood Hashing. Ph.D. dissertation, Technical Report CS-86-14, Waterloo, Ontario, Canada, 1986.

Seite 203

[89] D. E. Knuth. The Art of Computer Programming, Vol.3: Sorting and Searching. Addison-Wesley, Reading, Massachusetts, 1973.

[193] F. A. Williams. Handling identifiers as internal symbols in language processors. Comm. ACM, 2(6):21-24, 1959.

Seite 204

[67] G. H. Gonnet. Handbook of Algorithms and Data Structures. Addison-Wesley, 1984.

[102] P. A. Larson. Dynamische Hashverfahren. Informatik-Spektrum, 6(1):7-19, 1983.

[47] R. J. Enbody und H. C. Du. Dynamic hashing schemes. ACM Computing Surveys, 20(2):85-113, 1988.

[103] P. A. Larson. Dynamic Hash Tables. Comm. ACM, 31(4):446-457, 1988. [104] E. L. Lawler. Combinatorial optimization: Networks and matroids. Holt, Rinehart, and Winston, New York, 1976.

Seite 206

[112] W. Litwin. Linear hashing: A new tool for file and table addressing. In Proc. 6th Conference on Very Large Data Bases, S. 212-223, Montreal, 1980.

[111] W. Litwin. Hachage Virtuel: une nouvelle technique d'adressage de memoires. Ph.D. thesis, Univ. Paris VI, 1979. The`se de Doctorat d'Etat.

Seite 207

[136] J. A. Orenstein. A dynamic hash file for random and sequential accessing. In Proc. 9th Conference on Very Large Data Bases, S. 132-141, Florenz, 1983.

Seite 211

- [100] P. A. Larson. Dynamic hashing. BIT, 18:184-201, 1978.
- [51] R. Fagin, J. Nievergelt, N. Pippenger und H. R. Strong. Extendible hashing - a fast access method for dynamic files. ACM Trans. Database Systems, 4(3):315- 344, 1979.
- [124] H. Mendelson. Analysis of extendible hashing. IEEE Trans. Softw. Eng., SE 8(6):611-619, 1982.
- [101] P. A. Larson. Linear hashing with partial expansions. In Proc. 6th Conference on Very Large Data Bases, S. 224-232, Montreal, 1980.
- [157] K. Ramamohanarao und R. Sacks-Davis. Recursive linear hashing. ACM Trans. Database Systems, 9(3):369-391, 1984.
- [111] W. Litwin. Hachage Virtuel: une nouvelle technique d'adressage de memoires. Ph.D. thesis, Univ. Paris VI, 1979. The`se de Doctorat d'Etat.
- [110] W. Litwin. Virtual hashing: a dynamically changing hashing. In Proc. 4th Conference on Very Large Data Bases, S. 517-523, 1978.

Seite 215

- [110] W. Litwin. Virtual hashing: a dynamically changing hashing. In Proc. 4th Conference on Very Large Data Bases, S. 517-523, 1978.
- [111] W. Litwin. Hachage Virtuel: une nouvelle technique d'adressage de memoires. Ph.D. thesis, Univ. Paris VI, 1979. The`se de Doctorat d'Etat.
- [178] M. Tamminen. Order preserving extendible hashing and bucket tries. BIT, 21(4):419-435, 1981.
- [51] R. Fagin, J. Nievergelt, N. Pippenger und H. R. Strong. Extendible hashing - a fast access method for dynamic files. ACM Trans. Database Systems, 4(3):315- 344, 1979.

Seite 216

- [51] R. Fagin, J. Nievergelt, N. Pippenger und H. R. Strong. Extendible hashing - a fast access method for dynamic files. ACM Trans. Database Systems, 4(3):315- 344, 1979.

Seite 219

- [100] P. A. Larson. Dynamic hashing. BIT, 18:184-201, 1978.
- [124] H. Mendelson. Analysis of extendible hashing. IEEE Trans. Softw. Eng., SE 8(6):611-619, 1982.
- [51] R. Fagin, J. Nievergelt, N. Pippenger und H. R. Strong. Extendible hashing - a fast access method for dynamic files. ACM Trans. Database Systems, 4(3):315- 344, 1979.
- [197] A. C. Yao. A note on the analysis of extendible hashing. Information Processing Letters, 11:84-86, 1980.
- [53] P. Flajolet. On the performance evaluation of extendible hashing and trie searching. Acta Informatica, 20:345-369, 1983.

Seite 220

- [179] M. Tamminen. The extendible cell method for closest point problems. BIT, 22:27-41, 1982.

Seite 221

- [158] M. Regnier. Analysis of grid file algorithms. BIT, 25(2):335-357, 1985.
- [129] J. Nievergelt, H. Hinterberger und K. C. Sevcik. The grid file: An adaptable, symmetric multikey file structure. ACM Trans. Database Systems, 9(1):38-71, 1984.

Seite 223

- [94] R. Krishnamurthy und K.-Y. Whang. Multilevel Grid Files. IBM Research Report, Yorktown Heights, 1985.
- [78] K. Hinrichs. The Grid File System: Implementation and case studies of applications. Ph.D. dissertation, Institut für Informatik, ETH Zürich, Schweiz, 1985.

Seite 224

- [54] P. Flajolet und C. Puech. Partial match retrieval of multidimensional data. J. Assoc. Comput. Mach., 33(2):371-407, 1986.
- [160] R. L. Rivest. Partial-match retrieval algorithms. SIAM J. Comput., 5(1):19-50, 1976.

Seite 229

- [158] M. Regnier. Analysis of grid file algorithms. BIT, 25(2):335-357, 1985.
- [51] R. Fagin, J. Nievergelt, N. Pippenger und H. R. Strong. Extendible hashing - a fast access method for dynamic files. ACM Trans. Database Systems, 4(3):315- 344, 1979.
- [53] P. Flajolet. On the performance evaluation of extendible hashing and trie searching. Acta Informatica, 20:345-369, 1983.
- [124] H. Mendelson. Analysis of extendible hashing. IEEE Trans. Softw. Eng., SE 8(6):611-619, 1982.

Kapitel 5

Bäume

Bäume gehören zu den wichtigsten in der Informatik auftretenden Datenstrukturen. Entscheidungsbäume, Syntaxbäume, Ableitungsbäume, Codebäume, spannende Bäume, baumartig strukturierte Suchräume, Suchbäume und viele andere belegen die Allgegenwart von Bäumen. Wir haben in den vorangehenden Kapiteln bereits mehrfach Bäume als intuitives Konzept benutzt, so z.B. zur Erläuterung des Sortierverfahrens Heapsort in Abschnitt 2.3, beim Nachweis unterer Schranken für das Sortierproblem in Abschnitt 2.8 und beim Binärbaum-Sondieren in Abschnitt 4.3.4. Wir wollen jetzt eine systematische Behandlung von Begriffen im Zusammenhang mit Bäumen vornehmen und Algorithmen für Bäume behandeln.

Bäume sind verallgemeinerte Listenstrukturen. Ein Element — üblicherweise spricht man von *Knoten* — hat nicht, wie im Falle linearer Listen, nur einen Nachfolger, sondern eine endliche, begrenzte Anzahl von sogenannten *Söhnen*. In der Regel ist einer der Knoten als *Wurzel* des Baumes ausgezeichnet. Das ist zugleich der einzige Knoten ohne *Vorgänger*. Jeder andere Knoten hat einen (unmittelbaren) Vorgänger, der auch *Vater* des Knotens genannt wird. Eine Folge $p_0, \dots, p_k$ von Knoten eines Baumes, die die Bedingung erfüllt, daß p_{i+1} Sohn von p_i ist für $0 \leq i < k$, heißt *Pfad* mit Länge k , der p_0 mit p_k verbindet. Jeder von der Wurzel verschiedene Knoten eines Baumes ist durch genau einen Pfad mit der Wurzel verbunden. Man kann Bäume als spezielle planare, zyklenfreie Graphen auffassen. Die Knoten des Baumes sind die Knoten des Graphen; je zwei Knoten p und q sind durch eine Kante miteinander verbunden, wenn q Sohn von p (und damit p Vater von q) ist. Ist unter den Söhnen eines jeden Knotens eines Baumes eine Anordnung definiert, so daß man vom ersten, zweiten, dritten usw. Sohn eines Knotens sprechen kann, so nennt man den Baum *geordnet*. Dies darf man nicht mit der *Ordnung eines Baumes* verwechseln. Darunter versteht man nämlich die maximale Anzahl von Söhnen eines Knotens. Besonders wichtig sind geordnete Bäume der Ordnung 2; sie heißen auch binäre Bäume oder *Binärbäume*. Statt vom ersten und zweiten Sohn spricht man bei Binärbäumen vom *linken* und *rechten* Sohn eines Knotens. Wir werden in diesem Kapitel nur geordnete Bäume betrachten.

Da die Menge der Knoten eines Baumes stets als endlich vorausgesetzt wird, muß es Knoten geben, die keine Söhne haben. Diese Knoten werden üblicherweise als *Blätter* bezeichnet; alle anderen Knoten nennt man *innere Knoten*. Die Menge aller Bäume der

Ordnung d , $d \geq 1$, kann man äquivalent auch rekursiv definieren und entlang dieser Definition auf natürliche Art veranschaulichen:

- (1) Der aus einem einzigen Knoten bestehende Baum ist ein Baum der Ordnung d . Wir veranschaulichen ihn graphisch durch:



- (2) Sind $t_1, \dots, t_d$ beliebige Bäume der Ordnung d , so erhält man einen (weiteren) Baum der Ordnung d , indem man die Wurzeln von $t_1, \dots, t_d$ zu Söhnen einer neugeschaffenen Wurzel w macht. t_i ($1 \leq i \leq d$) heißt *i-ter Teilbaum* der Wurzel w . Wir veranschaulichen den neuen Baum graphisch wie in Abbildung 5.1.

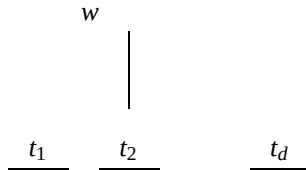


Abbildung 5.1

(In der Informatik wachsen die Bäume also in anderer Richtung als in der Natur: die Wurzel oben, die Blätter unten!)

Wir haben in dieser rekursiven Definition verlangt, daß jeder Knoten eines Baumes der Ordnung d entweder keinen oder genau d Söhne hat. Demzufolge sind die in der Abbildung 5.2 (a) und (b) gezeigten Bäume der Definition entsprechend gültige Binärbäume, der Baum aus Beispiel (c) aber nicht.

Die Anzahl der Söhne eines Knotens p nennt man häufig auch den *Rang* von p . Manchmal bezeichnet man den durch  veranschaulichten Baum auch als *leeren* Baum und fordert sogar explizit an Stelle der Bedingung (1), daß der aus *keinem* Knoten bestehende leere Baum ein Baum der Ordnung d ist. Dann besagt die Bedingung (2) zwar, daß jeder Knoten eines Baumes der Ordnung d genau d Söhne haben muß; von denen können aber einige oder gar alle leer sein, d.h. es handelt sich um gar nicht existierende Söhne. Das ist eine andere Möglichkeit, um auszudrücken, daß ein Knoten in einem Baum der Ordnung d auch weniger als d Söhne haben kann. Man findet in der Literatur beide Varianten, und wir werden in diesem Kapitel auch beide Varianten benötigen.

Bäume der Ordnung $d > 2$ nennt man auch *Vielwegbäume*. Wir bringen eine wichtige Klasse derartiger Bäume im Abschnitt 5.5, die Klasse der B-Bäume. Sie sind ein typischer Vertreter einer Klasse von Bäumen, für die man üblicherweise fordert, daß die Anzahl der Söhne jedes Knotens zwischen einer festen Unter- und Obergrenze liegen muß. Für Binärbäume werden wir jedoch durchweg verlangen, daß jeder Knoten genau zwei oder keinen Sohn haben soll. Die einzige Ausnahme bilden die im Abschnitt 5.2 behandelten Bruder-Bäume.

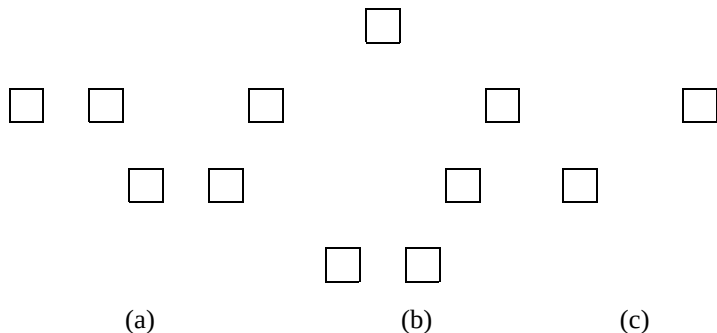


Abbildung 5.2

Wir haben bisher nur strukturelle Eigenschaften und Begriffe im Zusammenhang mit Bäumen besprochen. Dazu gehören auch noch die Begriffe *Höhe* eines Baumes und *Tiefe* eines Knotens. Die Höhe h eines Baumes ist der maximale Abstand eines Blattes von der Wurzel; sie kann auf naheliegende Weise rekursiv definiert werden, siehe Abbildung 5.3.

$$h \left(\begin{array}{c} \square \end{array} \right) = 0$$

$$h \left(\begin{array}{c} \square \\ \begin{array}{cc} \underbrace{\hspace{1cm}}_{t_1} & \underbrace{\hspace{1cm}}_{t_d} \end{array} \end{array} \right) = \max h(t_1), \dots, h(t_d) + 1$$

Abbildung 5.3

Der Binärbaum aus Abbildung 5.2 (a) hat also die Höhe 3 und der Binärbaum aus Abbildung 5.2 (b) die Höhe 4. Die *Tiefe* eines Knotens ist sein Abstand zur Wurzel, d h. die Anzahl der Kanten auf dem Pfad von diesem Knoten zur Wurzel. Man faßt die Knoten eines Baumes gleicher Tiefe zu *Niveaus* zusammen. Die Knoten auf dem Niveau i sind alle Knoten mit Tiefe i .

Ein Baum heißt *vollständig*, wenn er auf jedem Niveau die maximal mögliche Knotenzahl hat und sämtliche Blätter dieselbe Tiefe haben.

Obwohl es eine ganze Reihe interessanter und tiefliegender Sätze über die strukturellen Eigenschaften von Bäumen gibt, ist der eigentliche Grund für die Bedeutung von Bäumen ein anderer. Bäume sind eine Struktur zur Speicherung von Schlüsseln. Wir werden der Einfachheit halber annehmen, daß die Schlüssel stets ganzzahlig sind, wenn nicht ausdrücklich etwas anderes gesagt ist. Die Schlüssel werden dabei so gespeichert, daß sie sich nach einem einfachen und effizienten Verfahren wiederfinden lassen. Das *Suchen* nach einem in einem Baum gespeicherten Schlüssel ist aber nur eine der üblicherweise für Bäume erklärten Operationen. Weitere sind das *Einfügen* eines neuen Knotens (mit gegebenem Schlüssel), das *Entfernen* eines Knotens (mit gegebenem Schlüssel), das *Durchlaufen* aller Knoten eines Baumes in bestimmter Reihenfolge, das *Aufspalten* eines Baumes in mehrere, das *Zusammenfügen* mehrerer Bäume zu einem neuen und das *Konstruieren* eines Baumes mit bestimmten Eigenschaften.

Die drei wichtigsten Operationen sind das Suchen, Einfügen und Entfernen. Man nennt diese drei Operationen auch die *Wörterbuchoperationen* und eine Struktur, die es erlaubt, eine Menge von Schlüsseln zu speichern, zusammen mit Algorithmen für diese Struktur für die Wörterbuchoperationen auch eine *Implementation eines Wörterbuches* (englisch: *dictionary*), vgl. dazu auch Abschnitt 1.6.

In manchen Anwendungen treten praktisch keine Einfügungen und Entfernungen von Knoten auf. Das Universum der in einem Suchbaum abzuspeichernden Schlüssel ist fest und das Suchen die bei weitem überwiegende Operation. Dann kann man einen statischen Suchbaum konstruieren und dabei gegebenenfalls unterschiedliche Suchhäufigkeiten für verschiedene Schlüssel berücksichtigen. Je nachdem, ob die Suchhäufigkeiten fest und vorher bekannt sind oder sich im Laufe der Zeit ändern können, hat man das Ziel, statisch optimale oder dynamisch optimale oder fast optimale Suchbäume zu erzeugen. Wir behandeln nur den statischen Fall genauer in den Abschnitten 5.6 und 5.7.

Das andere Extrem ist der Fall, daß Bäume durch fortgesetztes, iteriertes Einfügen aus dem anfangs leeren Baum erzeugt werden. Wir zeigen im Abschnitt 5.1 über natürliche Bäume, wie man auf einfache Weise zu einer gegebenen Folge von Schlüsseln einen binären Suchbaum so aufbauen kann, daß auch die meisten anderen Operationen einfach ausführbar sind. Es wird sich herausstellen, daß die Reihenfolge, in der die Schlüssel in den anfangs leeren Baum nach und nach eingefügt werden, die Struktur des entstehenden Baumes stark beeinflusst. Es können sowohl zu linearen Listen degenerierte als auch nahezu vollständig ausgeglichene Binärbäume erzeugt werden. Daher kann man nicht ohne weiteres garantieren, daß die drei wichtigsten Basisoperationen für Bäume, das Suchen, Einfügen und Entfernen von Schlüsseln, sämtlich in einer Anzahl von Schritten ausführbar sind, die logarithmisch mit der Anzahl der im Baum gespeicherten Schlüssel wächst.

Es gibt jedoch Techniken, die es erlauben, einen Baum, der nach einer Einfüge- oder Entferne-Operation in Gefahr gerät, *aus der Balance* zu geraten, also zu degenerieren, wieder so zu rebalancieren, daß alle drei Basisoperationen in logarithmischer Schrittzahl ausführbar sind. Einige solcher Rebalancierungstechniken besprechen wir im Abschnitt 5.2 über *balancierte Binärbäume*.

5.1 Natürliche Bäume

In diesem Abschnitt wollen wir zeigen, wie Binärbäume zur Speicherung von Schlüsseln eingesetzt werden können und zwar so, daß man die im Baum gespeicherten Schlüssel auf einfache Weise wiederfinden kann bzw. feststellen kann, daß ein Schlüssel nicht im Baum vorkommt. Wir nehmen an, daß sämtliche Schlüssel paarweise verschieden sind.

Wir können zwei prinzipiell verschiedene Speicherungsformen unterscheiden. Sind die Schlüssel nur in den inneren Knoten gespeichert und haben die Blätter keine Schlüssel, so spricht man von *Suchbäumen*. Sind die Schlüssel in den Blättern gespeichert, spricht man von *Blattsuchbäumen*.

Suchbäume lassen sich folgendermaßen charakterisieren. Für jeden Knoten p gilt: Die Schlüssel im linken Teilbaum von p sind sämtlich kleiner als der Schlüssel von p , und dieser ist wiederum kleiner als sämtliche Schlüssel im rechten Teilbaum von p .

Die Blätter repräsentieren die Intervalle zwischen den in den inneren Knoten gespeicherten Schlüsseln.

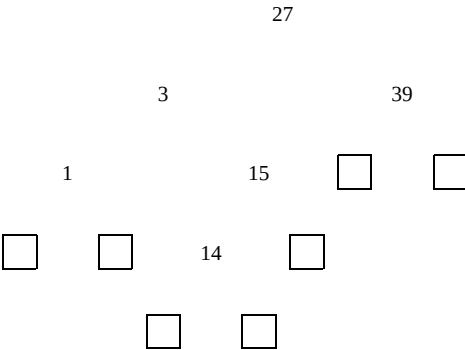


Abbildung 5.4

Abbildung 5.4 zeigt einen binären Suchbaum, der die Schlüsselmenge $\{1, 3, 14, 15, 27, 39\}$ speichert. Diese 6 Schlüssel sind die Schlüssel der inneren Knoten. Die 7 Blätter repräsentieren von links nach rechts die Intervalle $(-\infty, 1), (1, 3), (3, 14), (14, 15), (15, 27), (27, 39), (39, \infty)$.

Der Name *Suchbaum* und auch die Bemerkung, daß die Blätter Schlüsselintervalle repräsentieren, wird erst klar, wenn wir uns überlegen, wie man in einem solchen Baum nach einem Schlüssel x sucht. Wir beginnen bei der Wurzel p und vergleichen x mit dem bei p gespeicherten Schlüssel; ist x kleiner als der Schlüssel von p , setzen wir die Suche beim linken Sohn von p fort. Ist x größer als der Schlüssel von p , setzen wir die Suche beim rechten Sohn von p fort. Genauer verfahren wir nach folgender Methode:

Suche(p, x);

sucht im Baum mit Wurzel p nach einem Schlüssel x

Fall 1 [p ist innerer Knoten mit linkem Sohn p_l und rechtem Sohn p_r]

if $x < \text{Schlüssel}(p)$

then *Suche*(p_l, x)

else

if $x > \text{Schlüssel}(p)$

then *Suche*(p_r, x)

else $x = \text{Schlüssel}(p)$,

d.h. gesuchter Schlüssel p gefunden

Fall 2 [p ist Blatt]

gesuchter Schlüssel kommt im Baum nicht vor

Es ist offensichtlich, daß die Suche nach einem Schlüssel entweder beim Knoten endet, der x speichert, falls x im Baum vorkommt, oder aber an einem Blatt, und zwar an einem Blatt, das ein Intervall repräsentiert, das den gesuchten Schlüssel enthält.

Im Falle von *Blattsuchbäumen* speichern die Blätter die eigentlichen Schlüssel; die inneren Knoten speichern ebenfalls Werte. Die an den inneren Knoten gespeicherten Werte dienen aber lediglich als Wegweiser zu den an den Blättern gespeicherten Schlüsseln. Es gibt viele Möglichkeiten für die Wahl der an den inneren Knoten abzulegenden *Wegweiser*. Jeder zwischen dem maximalen Schlüssel im linken Teilbaum eines Knotens p und dem minimalen Schlüssel im rechten Teilbaum von p liegende Wert ist ein möglicher Kandidat, weil er es erlaubt, eine bei der Wurzel beginnende Suche nach einem an den Blättern gespeicherten Schlüssel bei p richtig zu dirigieren. Eine besonders einfache und übliche Wahl ist es, an jedem inneren Knoten stets den maximalen Schlüssel im linken Teilbaum abzulegen.

Ein Beispiel eines nach diesem Schema aufgebauten Blattsuchbaumes für die Menge 1, 3, 14, 15, 27, 39 ist in Abbildung 5.5 dargestellt.

Das Verfahren zum Suchen eines Schlüssel x kann dann offenbar wie folgt beschrieben werden:

Suche(p, x);

sucht im Baum mit Wurzel p nach einem Blatt mit Wert x

Fall 1 [p ist innerer Knoten mit linkem Sohn p_l und rechtem Sohn p_r]

if $x \leq \text{Schlüssel}(p)$

then *Suche*(p_l, x)

else *Suche*(p_r, x)

Fall 2 [p ist Blatt]

if $x = \text{Schlüssel}(p)$

then *Schlüssel bei p gefunden*

else *Schlüssel kommt im Baum nicht vor*

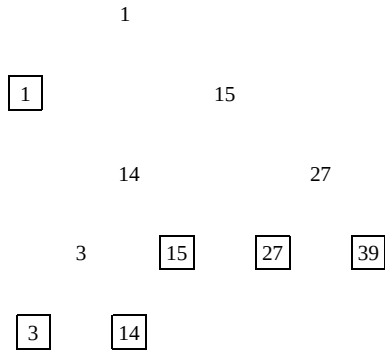


Abbildung 5.5

Wir beschränken uns im folgenden darauf, Algorithmen und Programme für die erste Variante (Suchbäume) anzugeben. Es sollte dem Leser nicht schwerfallen, entsprechende Algorithmen und Programme auch für die zweite Variante (Blattsuchbäume) zu entwickeln.

Es gibt grundsätzlich zwei verschiedene Möglichkeiten, Bäume programmtechnisch zu realisieren, die *Array*- und die *Zeiger*-Realisierung. Bei der *Array-Realisierung* werden die Knoten eines Baumes als Elemente eines Arrays vereinbart. Die Position der Söhne eines Knotens an Position i kann durch eine „Adreßrechnung“ aus i ermittelt werden. (Diese Art der Realisierung von Bäumen wurde für Heaps im Verfahren Heapsort benutzt.) Bei der *Zeiger-Realisierung* wird die Beziehung zwischen einem Knoten und seinen Söhnen über Zeiger hergestellt. Man vereinbart die Knoten also etwa wie folgt:

```
type
  Knotenzeiger = Knoten;
  Knoten = record
    leftson, rightson : Knotenzeiger;
    key : integer;
    info : infotype
  end
```

Ein Baum ist dann gegeben durch einen Zeiger auf die Wurzel:

```
var root : Knotenzeiger
```

Da die Blätter eines Suchbaumes keine Schlüssel (oder andere Informationen) speichern, müssen sie auch nicht explizit als Knoten des oben angegebenen Typs repräsentiert werden. Man kann sie vielmehr einfach durch **nil**-Zeiger in den jeweiligen Vätern repräsentieren. Der in Abbildung 5.4 angegebene Suchbaum zur Speicherung der Schlüsselmenge 1, 3, 14, 15, 27, 39 kann dann etwas genauer wie in Abbildung 5.6

graphisch veranschaulicht werden. Die die Blätter repräsentierenden **nil**-Zeiger sind durch Punkte angedeutet.

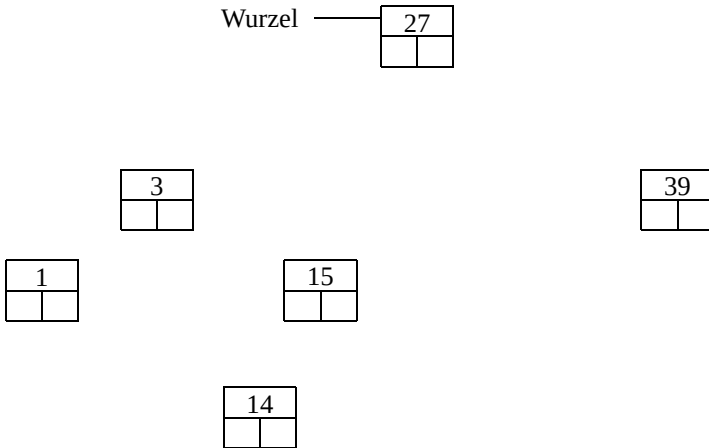


Abbildung 5.6

5.1.1 Suchen, Einfügen und Entfernen von Schlüsseln

Das angegebene Verfahren zum Suchen eines Schlüssels im Baum mit Wurzel p kann leicht in eine Pascal-Prozedur übersetzt werden.

```

procedure Suchen ( $p$  : Knotenzeiger;  $x$  : integer);
  sucht im Baum mit Wurzel  $p$  nach Schlüssel  $x$ 
begin
  if  $p = \text{nil}$ 
  then write('Es gibt keinen Knoten im Baum mit Schlüssel  $x$ ')
  else
    if  $x < p.key$ 
    then Suchen( $p.leftson, x$ )
    else
      if  $x > p.key$ 
      then Suchen( $p.rightson, x$ )
      else  $p.key = x$ 
        write('Knoten mit Schlüssel',  $x$ , 'gefunden')
  end Suchen
  
```

Statt einer rekursiven hätte man natürlich auch leicht eine iterative Suchprozedur angeben können. Das angegebene Suchverfahren und seine Implementation hat allerdings zwei „Schönheitsfehler“. Erstens wird an jedem Knoten zunächst geprüft, ob der Knoten ein Blatt ist oder nicht. Diese Abfrage ist für alle Knoten mit Ausnahme höchstens des letzten auf jedem Suchpfad negativ zu beantworten. Zweitens kann man auf den Knoten mit Schlüssel x nicht wirklich zugreifen, sondern erhält lediglich eine Meldung, daß der Schlüssel x gefunden wurde.

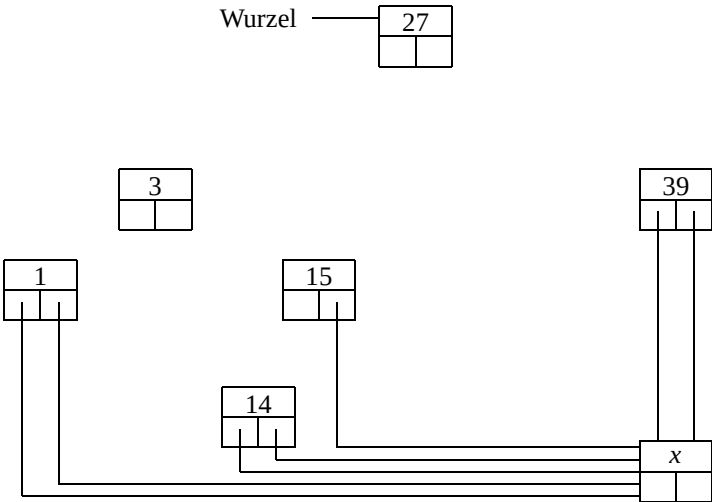


Abbildung 5.7

Den ersten Schönheitsfehler kann man mit einer von linearen Listen bekannten und bewährten Methode beheben. Man verwendet einen fiktiven *Dummy*-Knoten als Stopper, in dem man den gesuchten Schlüssel vor Beginn der Suche ablegt. Wenn sämtliche **nil**-Zeiger durch Zeiger auf diesen Knoten ersetzt werden, endet die Suche auf jeden Fall erfolgreich, nämlich spätestens beim Stopper. Man kann also auf die Abfrage $p = \text{nil}$ verzichten und kann stattdessen am Ende der Suche prüfen, ob der Schlüssel x im Stopper-Knoten gefunden wurde oder nicht. Abbildung 5.7 veranschaulicht diese Implementationsmöglichkeit.

Den zweiten Schönheitsfehler kann man dadurch beheben, daß man an Stelle eines Value-Parameters einen Variable-Parameter verwendet. Man kann aber auch eine Funktion deklarieren, die einen Zeiger auf den gesuchten Knoten abliefern, wenn der gesuchte Knoten im Baum vorkommt, und sonst den Wert **nil**. Wir überlassen die Ausführung der Details dem Leser.

Um einen Schlüssel in einen Suchbaum *einzufügen*, suchen wir zunächst nach dem einzufügenden Schlüssel im gegebenen Baum. Falls der einzufügende Schlüssel nicht schon im Baum vorkommt, endet die Suche erfolglos in einem Blatt, also je nach Implementation bei einem **nil**-Zeiger oder beim Stopper. Wir fügen dann den gesuchten

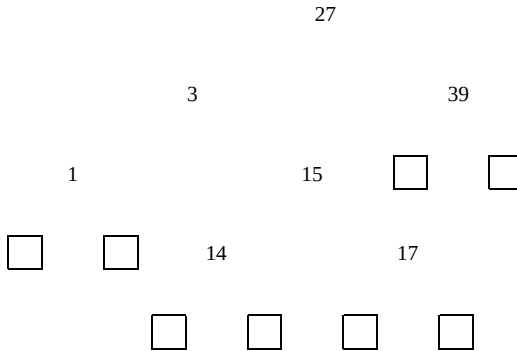


Abbildung 5.8

Schlüssel an der erwarteten Position unter den Blättern ein; d.h. wir ersetzen das Blatt durch einen inneren Knoten mit dem einzufügenden Schlüssel als Wert und zwei Blättern als Söhnen. Auf diese Weise erreicht man offensichtlich, daß der entstehende Baum wieder ein Suchbaum ist.

Fügt man beispielsweise in den eingangs dieses Abschnitts (Abbildung 5.4) angegebenen Suchbaum den Schlüssel 17 ein, so entsteht der Suchbaum in Abbildung 5.8.

Das folgende Programmstück liest eine Folge von paarweise verschiedenen Schlüssel und fügt sie der Reihe nach in den anfangs leeren Baum ein. Der entstehende Baum ist ein Baum, dessen Blätter durch **nil**-Zeiger repräsentiert werden.

```

program Baumaufbau (input, output);
type
  Knotenzeiger = Knoten;
  Knoten = record
    leftson, rightson : Knotenzeiger;
    key : integer;
    info : infotype
  end;
var
  wurzel : Knotenzeiger;
  k : integer;

procedure Einfügen (var p : Knotenzeiger; k : integer);
begin
  if p = nil
  then neuen Knoten mit Schlüssel k einfügen
  begin
    new(p);
    p .leftson := nil;
    p .rightson := nil;
  end

```

```

    p .key := k
  end
else
  if k < p .key
    then Einfügen(p .leftson, k)
  else
    if k > p .key
      then Einfügen(p .rightson, k)
    else write('Schlüssel kam schon vor')
  end; Einfügen
begin Baumaufbau
  wurzel := nil;
  while not eof(input) do
    begin
      read(k);
      Einfügen(wurzel, k)
    end
  end. Baumaufbau

```

Der auf diese Weise entstehende Suchbaum für eine Menge von Schlüsseln hängt sehr stark davon ab, in welcher Reihenfolge die Schlüssel in den anfangs leeren Baum eingefügt werden. Es können sowohl zu Listen degenerierte Suchbäume der Höhe N entstehen, wenn man N Schlüssel etwa in aufsteigend sortierter Reihenfolge einfügt. Es können aber auch niedrige, nahezu vollständige Suchbäume mit minimal möglicher Höhe $\log_2 N$ entstehen, bei denen sämtliche Blätter auf höchstens zwei verschiedenen Niveaus auftreten.

Abbildung 5.9 zeigt als Beispiel für diese beiden Extremfälle zwei Suchbäume für die Menge $1, 3, 14, 15, 27, 39$, die entstehen, wenn man die Schlüssel in der Reihenfolge $15, 39, 3, 27, 1, 14$ bzw. in der Reihenfolge $1, 3, 14, 15, 27, 39$ in den anfangs leeren Baum einfügt.

Ein auf diese Weise durch iteriertes Einfügen in den anfangs leeren Baum zu einer Schlüsselfolge entstehender binärer Suchbaum heißt *natürlicher Baum*.

Eine wichtige Frage ist, ob die gut ausgeglichenen niedrigen Bäume oder die hohen, zu Listen degenerierten Bäume häufiger auftreten, wenn man alle den $N!$ möglichen Anordnungen von N Schlüsseln entsprechenden natürlichen Bäume erzeugt. Wir werden diese Frage in Abschnitt 5.1.3 beantworten.

Zunächst überlegen wir uns, wie man einen Schlüssel aus einem Suchbaum *entfernen* kann, so daß der entstehende Baum wieder ein Suchbaum ist. Man sucht zunächst nach dem zu entfernenden Schlüssel x . Kommt x im Baum nicht vor, ist nichts zu tun. Ist x der Schlüssel eines Knotens, der keinen oder nur einen inneren Knoten als Sohn hat, ist das Entfernen einfach. Man entfernt den Knoten mit Schlüssel x und ersetzt ihn gegebenenfalls durch seinen einzigen Sohn. Schwieriger ist das Entfernen von x , wenn x Schlüssel eines Knotens ist, dessen beide Söhne innere Knoten sind, die Schlüssel gespeichert haben. Wir reduzieren in diesem Fall das Problem, den Schlüssel x zu entfernen, folgendermaßen auf einen der beiden einfacheren Fälle. Sei x der Schlüssel des Knotens p . Dann suchen wir im rechten Teilbaum von p den Knoten q mit dem kleinsten Schlüssel y , der größer als x ist. Der Knoten q (und y) heißt der *symmetrische*

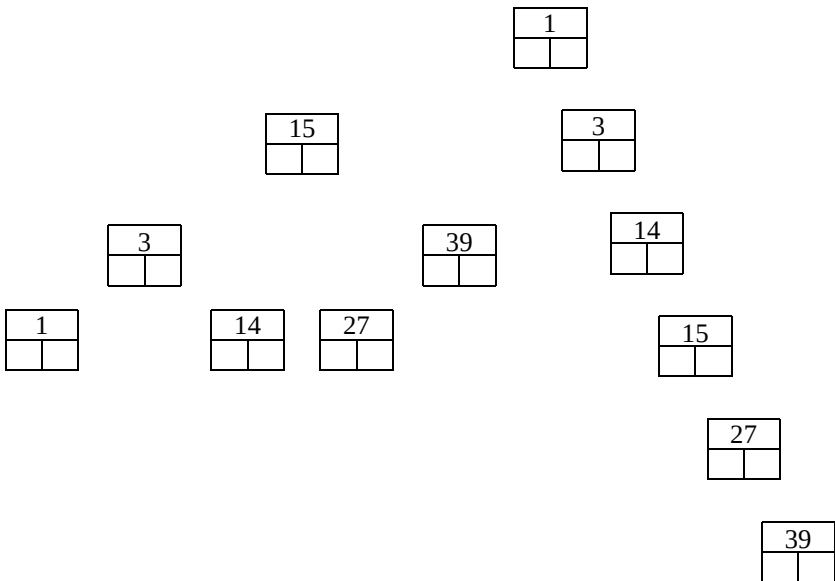


Abbildung 5.9

Nachfolger von p (und x) (vgl. hierzu auch Abschnitt 5.1.2). Der Knoten q ist der am weitesten links stehende innere Knoten im rechten Teilbaum von p und kann daher höchstens einen inneren Knoten als rechten Sohn haben. Man ersetzt nun den Schlüssel x des Knotens p durch den Schlüssel y und entfernt den Knoten q (mit seinem Schlüssel y). Abbildung 5.10 veranschaulicht dies.

Die im folgenden angegebene Prozedur *Entfernen* unter Verwendung der Funktion *vatersymnach* ist eine mögliche Implementation des Verfahrens.

```

function vatersymnach ( $p$  : Knotenzeiger) : Knotenzeiger;
  liefert für einen Knotenzeiger  $p$  mit  $p$  .rightson = nil einen Zeiger
  auf den Vater des symmetrischen Nachfolgers von  $p$ 
begin
  if  $p$  .rightson .leftson = nil
  then sonst ist  $p$  das Ergebnis
  begin
     $p := p$  .rightson;
    while  $p$  .leftson .leftson = nil do
       $p := p$  .leftson
    end;
    vatersymnach :=  $p$ 
  end
end vatersymnach
  
```

```

procedure Entfernen (var  $p$  : Knotenzeiger;  $k$  : integer);
  entfernt einen Knoten mit Schlüssel  $k$  aus dem Baum mit Wurzel  $p$ 
  
```

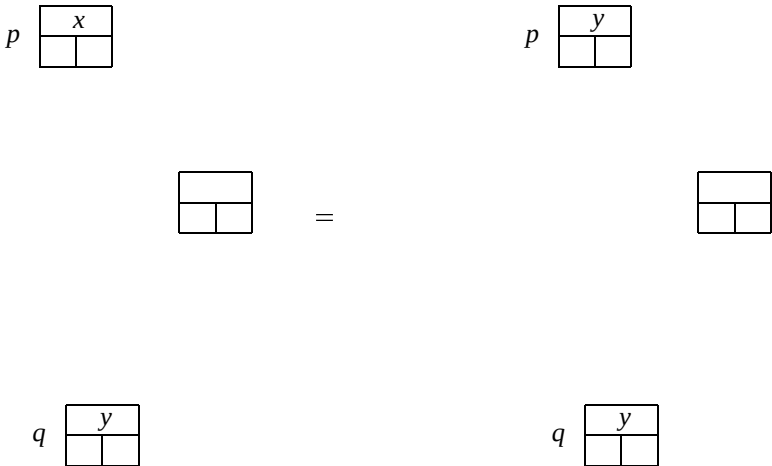


Abbildung 5.10

```
var
   $q$  : Knotenzeiger;
begin
  if  $p = \text{nil}$ 
  then Schlüssel  $k$  nicht im Baum
  else
    if  $k < p.\text{key}$ 
    then Entfernen( $p.\text{leftson}, k$ )
    else
      if  $k > p.\text{key}$ 
      then Entfernen( $p.\text{rightson}, k$ )
      else  $p.\text{key} = k$ 
      if  $p.\text{leftson} = \text{nil}$ 
      then  $p := p.\text{rightson}$ 
      else
        if  $p.\text{rightson} = \text{nil}$ 
        then  $p := p.\text{leftson}$ 
        else  $p.\text{leftson} = \text{nil}$  and  $p.\text{rightson} = \text{nil}$ 
        begin
           $q := \text{vatersymnach}(p)$ ;
          if  $q = p$ 
```

```

then rechter Sohn von q ist
      symmetrischer Nachfolger
begin
  p .key := q .rightson .key;
  q .rightson := q .rightson .rightson
end
else linker Sohn von q ist
      symmetrischer Nachfolger
begin
  p .key := q .leftson .key;
  q .leftson := q .leftson .rightson
end
end

```

end *Entfernen*

Wir haben das Entfernen eines Schlüssels eines Knotens p mit zwei inneren Knoten als Söhnen willkürlich auf das Entfernen des symmetrischen Nachfolgers reduziert. Stattdessen hätte man ebenso gut den *symmetrischen Vorgänger* von p , d. h. den am weitesten rechts stehenden Knoten im linken Teilbaum von p nehmen können. Man kann auch Strategien implementieren, die mal die eine, mal die andere Möglichkeit wählen. Das hat durchaus Einfluß auf die Struktur der durch iteriertes Entfernen entstehenden Bäume. Wir kommen auf diesen Punkt im Abschnitt 5.1.3 wieder zurück.

5.1.2 Durchlaufordnungen in Binärbäumen

Das Inspizieren aller Knoten eines Graphen im allgemeinen und eines Baumes im besonderen ist häufig nötig, um bestimmte Eigenschaften von Knoten, der in den Knoten gespeicherten Schlüssel und der Struktur des Graphen bzw. Baumes zu ermitteln. Algorithmen zum Durchlaufen aller Knoten eines Baumes in einer bestimmten Reihenfolge bilden das weitgehend problemunabhängige Gerüst für spezifische Aufgaben. Solche Aufgaben sind beispielsweise das Ausdrucken, Markieren, Kopieren usw. aller in einem binären Suchbaum auftretenden Knoten oder Schlüssel in bestimmter Reihenfolge, die Berechnung der Summe, des Durchschnitts, der Anzahl usw. aller in einem Baum gespeicherten Schlüssel, die Ermittlung der Höhe eines Baumes oder der Tiefe eines Knotens, die Prüfung, ob alle Blätter eines Baumes auf demselben Niveau liegen, usw.

Die drei wichtigsten Reihenfolgen, in denen man sämtliche Knoten eines Binärbaumes durchlaufen kann, sind die *Hauptreihenfolge* (oder: *Preorder*), die *Nebenreihenfolge* (oder: *Postorder*) und die *symmetrische Reihenfolge* (oder: *Inorder*). Diese Reihenfolgen lassen sich sehr einfach rekursiv formulieren, das Verfahren zum Durchlaufen aller Knoten eines Baumes in Hauptreihenfolge beispielsweise so:

Durchlaufen aller Knoten eines Binärbaumes mit Wurzel p in Hauptreihenfolge:

1. *Besuche die Wurzel p ;*
2. *durchlaufe den linken Teilbaum von p in Hauptreihenfolge;*
3. *durchlaufe den rechten Teilbaum von p in Hauptreihenfolge.*

Grob vereinfacht kann man die Hauptreihenfolge so charakterisieren:
Hauptreihenfolge: Wurzel, linker Teilbaum, rechter Teilbaum.
Entsprechend lauten die übrigen zwei Reihenfolgen:
Nebenreihenfolge: linker Teilbaum, rechter Teilbaum, Wurzel.
Symmetrische Reihenfolge: linker Teilbaum, Wurzel, rechter Teilbaum.
Eine mögliche Implementation etwa der symmetrischen Reihenfolge als rekursive Prozedur ist:

```
procedure symtraverse (p : Knotenzeiger);  
  durchläuft sämtliche Knoten des Baumes mit Wurzel p in  
  symmetrischer Reihenfolge  
begin  
  if p = nil  
  then  
    begin  
      symtraverse(p .leftson);  
      besuche die Wurzel; d.h. gib z.B. den Schlüssel p .key  
      aus durch write(p .key)  
      symtraverse(p .rightson)  
    end  
  end symtraverse
```

Schreibt man an Stelle des Kommentars in dieser Prozedur wirklich die Anweisung *write(p .key)* und ruft die Prozedur *symtraverse* für die Wurzel eines binären Suchbaums auf, so werden die im Baum gespeicherten Schlüssel in aufsteigend sortierter Reihenfolge ausgegeben.

Abbildung 5.11 zeigt einen Suchbaum mit sechs Schlüsseln und die Folge der Schlüssel in Haupt-, Neben- und symmetrischer Reihenfolge.

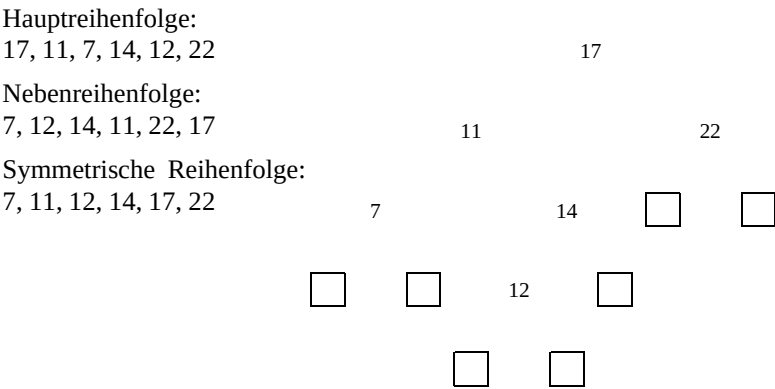


Abbildung 5.11

Die Bezeichnungen Haupt-, Neben- und symmetrische Reihenfolge bzw. Preorder, Postorder, Inorder sollen deutlich machen, wann die Wurzel eines Baumes betrachtet

wird: Vor, nach oder zwischen den Teilbäumen. Natürlich gibt es zu den von uns angegebenen Links-vor-rechts-Varianten auch die umgekehrten, in denen jeweils die rechten Teilbäume vor den linken betrachtet werden.

Da man bekanntlich jede rekursive Prozedur unter Zuhilfenahme eines Stapels in eine äquivalente iterative umwandeln kann, gilt dies natürlich insbesondere für die oben angegebenen Prozeduren zum Durchlaufen der Knoten in Haupt-, Neben- und symmetrischer Reihenfolge.

Eine Möglichkeit, Rekursion und Stapel beim Durchlaufen von Bäumen gänzlich zu vermeiden, besteht in der Einführung zusätzlicher Zeiger. Von jedem Knoten gibt es einen Zeiger auf dessen Nachfolger in der Haupt-, Neben- oder symmetrischen Reihenfolge; diese Zeiger müssen unter Umständen zusätzlich zu den schon bestehenden, von den Vätern auf die jeweiligen Söhne zeigenden Verweisen vorgesehen werden. Das ist im Falle der symmetrischen Reihenfolge jedoch nicht nötig. Der symmetrische Nachfolger eines inneren Knoten p ist nämlich entweder der linkeste Knoten im rechten Teilbaum, falls p überhaupt einen rechten Teilbaum hat, oder aber, falls p keinen rechten Teilbaum hat, ein weiter oben im Baum vorkommender Knoten. Im letzten Fall kann man an Stelle des **nil**-Zeigers, der andeutet, daß p keinen rechten Sohn hat, einen Zeiger auf den symmetrischen Nachfolger von p als Wert von p *.rightson* abspeichern.

Entsprechend kann man auch für die Knoten ohne linken Sohn an Stelle des **nil**-Zeigers einen Zeiger auf den symmetrischen Vorgänger in p *.leftson* ablegen. Dann treten je ein **nil**-Zeiger nur noch beim linkensten und rechtensten Knoten auf. Bäume mit dieser Zeigerstruktur heißen üblicherweise *gefädelt* Bäume. Ein Beispiel zeigt Abbildung 5.12.

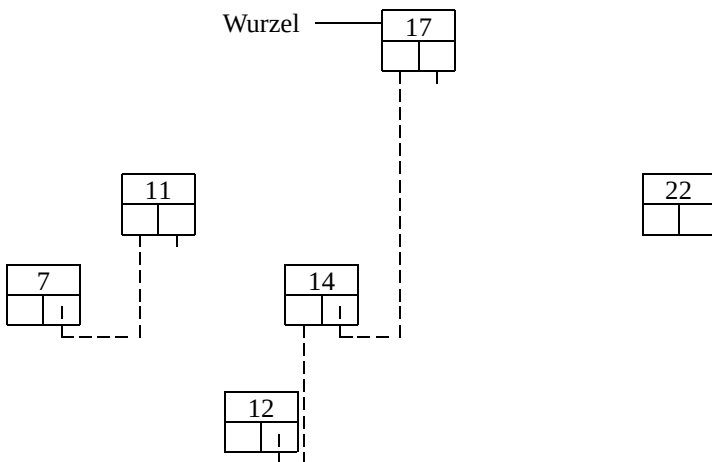


Abbildung 5.12

Natürlich muß man jetzt die *Fädelungszeiger* von den *echten* Zeigern unterscheiden können, die von den Vätern auf die jeweiligen Söhne zeigen. Setzen wir das einmal voraus, so kann man beispielsweise den symmetrischen Nachfolger eines Knotens wie folgt bestimmen:

```
Algorithmus symnach ( $p$  : Knotenzeiger) : Knotenzeiger;
Fall 1 [ $p$  .rightson = nil]
  Dann hat  $p$  keinen symmetrischen Nachfolger.
Fall 2 [ $p$  .rightson ≠ nil]
  [Fall 2.1] [ $p$  .rightson ist Fädelungszeiger]
     $\text{symnach} := p$  .rightson;
  [Fall 2.2] [ $p$  .rightson ist kein Fädelungszeiger]
     $q := p$  .rightson;
    while  $q$  .leftson =  $p$  do  $q := q$  .leftson;
     $\text{symnach} := q$ .
```

Um die Knoten in symmetrischer Reihenfolge zu durchlaufen, genügt es dann, den linken Knoten im Baum zu bestimmen und von dort aus mit Hilfe von *symnach* solange den symmetrischen Nachfolger des jeweils betrachteten Knotens zu besuchen, bis der rechteste Knoten r im Baum erreicht ist, der offenbar durch die Bedingung r .rightson = **nil** charakterisiert ist.

Man kann binäre Suchbäume von vornherein in dieser Form als gefädelte Bäume aufbauen. Dazu müssen natürlich beim Einfügen und Entfernen von Schlüsseln die Fädelungszeiger gegebenenfalls neu adjustiert werden. Wir überlassen es dem Leser, sich die Implementationsdetails zu überlegen.

5.1.3 Analytische Betrachtungen

Ein Binärbaum mit N inneren Knoten hat $N + 1$ Blätter. Seine Höhe kann maximal N sein und muß mindestens $\log_2(N + 1)$ sein. Der Aufwand zum Ausführen der drei wichtigsten Operationen für binäre Suchbäume, das Suchen, Einfügen und Entfernen von Schlüsseln, hängt unmittelbar von der Höhe des jeweiligen Baumes ab. In jedem Fall muß man ungünstigstenfalls einem Pfad von der Wurzel zu einem Blatt folgen, um die Operation auszuführen. Der im schlechtesten Fall erforderliche Aufwand zum Suchen, Einfügen und Entfernen eines Schlüssels in einem binären Suchbaum mit Höhe h ist damit von der Größenordnung $O(h)$. Dabei kann h zwischen $\log_2(N + 1)$ und N liegen, wenn der Baum vor Ausführen der Operation N Schlüssel hatte. Im schlechtesten Fall sind Suchbäume und die wichtigsten für sie typischen Operationen nicht besser als verkettete gespeicherte lineare Listen. Wir wollen jetzt zeigen, daß das Verhalten im Mittel wesentlich besser ist. Um dieser Aussage einen präzisen Sinn zu geben, muß zunächst genau gesagt werden, worüber denn gemittelt wird. Dafür gibt es zwei grundsätzlich verschiedene Möglichkeiten.

(a) *Random-tree-Analyse:* Wir nehmen an, daß jede der $N!$ möglichen Anordnungen von N Schlüsseln gleichwahrscheinlich ist und betrachten den Suchbaum, der zu einer zufällig gewählten Folge von N Schlüsseln durch iteriertes Einfügen in den anfangs leeren Baum entsteht. Gemittelt wird hier also über die den $N!$ möglichen Schlüsselfolgen zugeordneten natürlichen Bäume.

(b) *Gestalts-Analyse:* Wir betrachten die Menge aller strukturell verschiedenen binären Suchbäume mit N Schlüsseln und bilden das Mittel über diese Menge.

Nehmen wir als Beispiel die Menge aller möglichen Anordnungen der drei Schlüssel 1, 2, 3 und die Menge der strukturell verschiedenen Suchbäume zur Speicherung dieser drei Schlüssel: Fügt man die Schlüssel der Reihe nach in den anfangs leeren Baum ein, so werden gut ausgeglichene, niedrige Bäume und zu linearen Listen degenerierten, hohen Bäume mit jeweils unterschiedlicher Häufigkeit erzeugt. Die Übersicht in Abbildung 5.13 zeigt alle strukturell verschiedenen Suchbäume mit drei Schlüsseln und die Permutationen, die sie jeweils erzeugen.

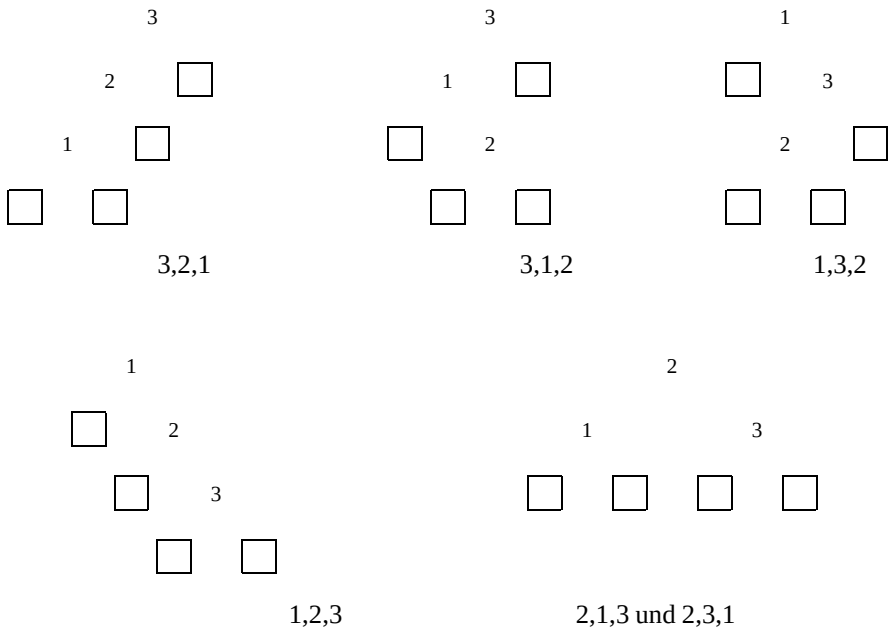


Abbildung 5.13

Der vollständige Binärbaum mit Höhe 2 wird von zwei, jeder der vier verschiedenen Bäume mit Höhe 3 nur von je einer Permutation erzeugt.

Als ein Maß für die Güte eines binären Suchbaumes führen wir die *interne Pfadlänge* und die *durchschnittliche Suchpfadlänge* ein. Die interne Pfadlänge $I(t)$ eines Baumes t ist die Summe aller Abstände der inneren Knoten zur Wurzel. Man kann die interne Pfadlänge rekursiv wie folgt definieren:

- (0) Ist $t = \square$, so ist $I(t) = 0$.
- (1) Ist t ein Baum mit linkem Teilbaum mit Wurzel t_l und rechtem Teilbaum mit Wurzel t_r , so ist

$$I(t) = I(t_l) + I(t_r) + \text{Zahl der inneren Knoten von } t.$$

Denn von der Wurzel von t aus gesehen haben alle inneren Knoten von t_l und t_r einen um 1 größeren Abstand zur Wurzel von t als zur jeweiligen Wurzel von t_l bzw. t_r . Die Wurzel von t hat den Abstand 1 zur Wurzel von t . Die interne Pfadlänge mißt also die gesamten *Besuchskosten* für die inneren Knoten des Baumes. Es ist leicht zu sehen, daß gilt:

$$I(t) = \sum_{\substack{p \text{ innerer} \\ \text{Knoten von } t}} (\text{Tiefe}(p) + 1)$$

Ein Beispiel ist in Abbildung 5.14 dargestellt.

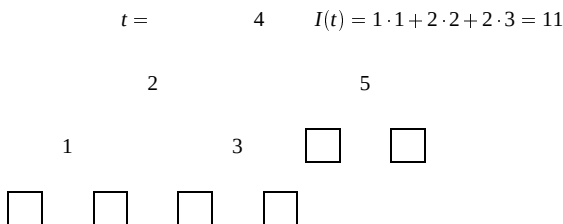


Abbildung 5.14

Bezeichnen wir die Anzahl der inneren Knoten eines Baumes t mit t , so ist die *durchschnittliche Suchpfadlänge*

$$\bar{I}(t) = \frac{I(t)}{t}.$$

Die durchschnittliche Suchpfadlänge mißt also, wieviele Knoten bei erfolgreicher Suche nach einem im Baum t gespeicherten Schlüssel im Mittel (über alle Schlüssel) zu besuchen sind.

Wir berechnen jetzt die Erwartungswerte von $I(t)$ und $\bar{I}(t)$ für einen zufällig erzeugten bzw. für einen der strukturell möglichen Bäume mit N inneren Knoten.

Random trees

Die Berechnung der internen Pfadlänge eines zufällig erzeugten, binären Suchbaumes kann sehr ähnlich erfolgen wie die Berechnung der mittleren Laufzeit des Sortierverfahrens Quicksort. Wir können ohne Einschränkung annehmen, daß die Menge der N iteriert in den anfangs leeren Baum einzufügenden Schlüssel die Menge $1, \dots, N$ ist. Ist dann $s_1, \dots, s_N$ eine zufällige Permutation dieser N Schlüssel, so ist die erste Zahl $s_1 = k$ mit Wahrscheinlichkeit $1/N$ für jedes k zwischen 1 und N . Wird k Schlüssel der Wurzel, so hat der linke Teilbaum der Wurzel, der alle Schlüssel enthält, die kleiner als k sind, $k - 1$ Elemente und der rechte Teilbaum der Wurzel entsprechend $N - k$ Elemente.

Bezeichnen wir mit $EI(N)$ den Erwartungswert für die interne Pfadlänge eines zufällig erzeugten binären Suchbaumes mit N inneren Knoten, so erhält man aus der bereits angegebenen Rekursionsformel zur Berechnung der internen Pfadlänge unmittelbar:

$$\begin{aligned} EI(0) &= 0, \quad EI(1) = 1, \\ EI(N) &= \frac{1}{N} \sum_{k=1}^N (EI(k-1) + EI(N-k) + N) \\ &= N + \frac{1}{N} \left(\sum_{k=1}^N EI(k-1) + \sum_{k=1}^N EI(N-k) \right) \\ &= N + \frac{2}{N} \left(\sum_{k=0}^{N-1} EI(k) \right) \end{aligned}$$

Also ist

$$EI(N+1) = (N+1) + \frac{2}{N+1} \sum_{k=0}^N EI(k),$$

und daher

$$\begin{aligned} (N+1) EI(N+1) &= (N+1)^2 + 2 \sum_{k=0}^N EI(k) \\ N EI(N) &= N^2 + 2 \sum_{k=0}^{N-1} EI(k). \end{aligned}$$

Aus den beiden letzten Gleichungen folgt

$$\begin{aligned} (N+1)EI(N+1) - N EI(N) &= 2N + 1 + 2 EI(N) \\ (N+1)EI(N+1) &= (N+2)EI(N) + 2N + 1 \\ EI(N+1) &= \frac{2N+1}{N+1} + \frac{N+2}{N+1} EI(N). \end{aligned}$$

Nun zeigt man leicht durch vollständige Induktion über N , daß für alle $N \geq 1$ gilt:

$$EI(N) = 2(N+1)H_N - 3N$$

Dabei bezeichnet $H_N = 1 + \frac{1}{2} + \dots + \frac{1}{N}$ die N -te harmonische Zahl, die wie folgt abgeschätzt werden kann:

$$H_N = \ln N + \gamma + \frac{1}{2N} + O\left(\frac{1}{N^2}\right)$$

Dabei ist $\gamma = 0.5772\dots$ die sogenannte Eulersche Konstante. Damit ist

$$EI(N) = 2N \ln N - (3 - 2\gamma) N + 2 \ln N + 1 + 2\gamma + O\left(\frac{1}{N}\right)$$

und daher

$$\begin{aligned} \frac{EI(N)}{N} &= 2 \ln N - (3 - 2\gamma) + \frac{2 \ln N}{N} + \dots \\ &= \frac{2}{\log_2 e} \log_2 N - (3 - 2\gamma) + \frac{2 \ln N}{N} + \dots \\ &= \frac{2 \log_{10} 2}{\log_{10} e} \log_2 N - (3 - 2\gamma) + \frac{2 \ln N}{N} + \dots \\ &= 1.386 \log_2 N - (3 - 2\gamma) + \frac{2 \ln N}{N} + \dots \end{aligned}$$

Wir vergleichen diesen Wert für den mittleren Abstand zur Wurzel eines Knotens in einem zufällig erzeugten Baum mit dem mittleren Abstand eines Knotens in einem vollständigen Binärbaum mit $N = 2^h - 1$ inneren Knoten. In einem vollständigen Binärbaum mit Höhe h hat jeder innere Knoten zwei innere Knoten oder zwei Blätter als Söhne, und alle Blätter haben dieselbe Tiefe. Für einen solchen Baum ist die durchschnittliche Suchpfadlänge minimal unter allen Bäumen mit derselben Knotenzahl. Sie ist offenbar:

$$\bar{I}_{min}(N) = \frac{1}{N} \sum_{i=0}^{h-1} (i+1) 2^i = \frac{1}{2^h - 1} [(h-1) 2^h + 1]$$

Wegen $h = \log_2(N+1)$ ist also:

$$\bar{I}_{min}(N) = \frac{1}{2^h - 1} [(h-1)(2^h - 1) + h] = \log_2(N+1) + \frac{\log_2(N+1)}{N} - 1$$

Vergleicht man dies mit der zuvor ermittelten durchschnittlichen Suchpfadlänge $\frac{EI(N)}{N}$ eines zufällig erzeugten Baumes, so ergibt sich das bemerkenswerte Ergebnis, daß der Wert für einen zufällig erzeugten Baum nur etwa 40% über dem minimal möglichen liegt.

Erzeugt man also einen binären Suchbaum aus dem anfangs leeren Baum durch iteriertes Einfügen von N Schlüsseln in zufällig gewählter Reihenfolge, so entsteht ein Suchbaum, für den die Suchoperation nur etwa 40% teurer ist als für einen vollständigen binären Suchbaum. Auch eine einzelne weitere Einfüge- und Entferne-Operation in einem solchen Baum kann durchschnittlich in $1.386 \log_2 N$ Schritten ausgeführt werden. Führt man jedoch weitere Einfüge- und Entferne-Operationen aus, bleibt das nicht mehr so. Der Grund dafür ist, daß wir das Entfernen eines Schlüssels eines inneren Knotens mit zwei nichtleeren Teilbäumen auf das Entfernen des symmetrischen Nachfolgers reduziert haben. Es leuchtet ein, daß durch diese Vorschrift eher größere Schlüssel zu Schlüsseln der Wurzel werden, also nach vielen Einfügungen und Entfernungen

Bäume entstehen, die „linkslastig“ sind. Denn immer wenn die Wurzel eines (Teil-) Baumes entfernt wird, wird sie durch einen größeren Schlüssel ersetzt, wenn ihr rechter Teilbaum nicht leer war. Eine genaue quantitative Analyse dieses Sachverhaltes gelang J. Culberson [1]. Er hat den Fall analysiert, daß nach N zufälligen Einfügungen in den anfangs leeren Baum jeweils abwechselnd je ein zufällig gewählter Schlüssel entfernt und eingefügt wird. Nennt man ein Paar von Entferne- und Einfüge-Operationen eine *Update-Operation*, so gilt: Führt man in einem zufällig erzeugten Suchbaum mit N Schlüsseln wenigstens N^2 *Update-Operationen* aus, so ist der Erwartungswert für die durchschnittliche Suchpfadlänge $\Theta(\sqrt{N})$ für hinreichend große N . Den nicht einfachen Beweis dieses Sachverhaltes findet man in [2]. Es ist klar, daß ein entsprechendes Ergebnis gilt, wenn man das Entfernen eines Schlüssels statt auf den symmetrischen Nachfolger stets auf den symmetrischen Vorgänger reduziert. Daher liegt es nahe, bei jeder Entfernung zufällig zwischen symmetrischen Vorgängern und symmetrischen Nachfolgern zu wählen. Experimente zeigen, daß dann auch nach einer großen Zahl von Updates besser balancierte Bäume entstehen. Der analytische Nachweis dafür ist bisher nicht gelungen.

Gestaltsanalyse

Wir wollen jetzt die mittlere (gesamte) Pfadlänge eines Baumes mit N inneren Knoten berechnen, wobei über alle strukturell möglichen Bäume gemittelt wird. Es wird sich herausstellen, daß die mittlere Pfadlänge eines Baumes mit N inneren Knoten gleich $N \sqrt{\pi} + O(N)$ ist; jeder Knoten hat also im Mittel einen Abstand $O(\sqrt{N})$ von der Wurzel.

Dieser Nachweis gelingt mit Hilfe sogenannter *erzeugender Funktionen*. Das sind formale Potenzreihen, die zur Analyse struktureller Eigenschaften von rekursiv definierten Strukturen — zu denen ja auch Binärbäume gehören — herangezogen werden können. Wir demonstrieren die Verwendung formaler Potenzreihen zunächst an einem sehr einfachen Beispiel und berechnen die Anzahl der strukturell verschiedenen Binärbäume mit N inneren Knoten. Um sämtliche strukturell möglichen Bäume mit N inneren Knoten zu erzeugen, kann man doch offenbar folgendermaßen vorgehen. Man macht einen Knoten zur Wurzel und wählt unabhängig voneinander alle strukturell möglichen linken und rechten Teilbäume, aber natürlich so, daß insgesamt ein Baum mit N inneren Knoten entsteht. Genauer: Bezeichnen wir mit B_N die Anzahl der strukturell möglichen Binärbäume mit N inneren Knoten, so erhält man alle strukturell möglichen Binärbäume, deren linker Teilbaum genau i innere Knoten enthält (für ein festes i , $0 \leq i \leq N-1$) wie folgt. Man wählt unabhängig voneinander alle strukturell möglichen Binärbäume mit i inneren Knoten als linke und mit $(N-i-1)$ inneren Knoten als rechte Teilbäume und verbindet sie zu einem neuen Binärbaum mit N inneren Knoten; dafür gibt es $B_i \cdot B_{N-i-1}$ Möglichkeiten, vgl. Abbildung 5.15.

Weil i beliebig zwischen 0 und $N-1$ liegen kann, muß also gelten:

$$B_N = B_0 \cdot B_{N-1} + B_1 \cdot B_{N-2} + \dots + B_{N-1} \cdot B_0$$

Dieser Ausdruck hat eine formale Ähnlichkeit mit den bei der Multiplikation zweier Polynome auftretenden Koeffizienten, die man ausnutzen kann. Wir definieren eine formale Potenzreihe

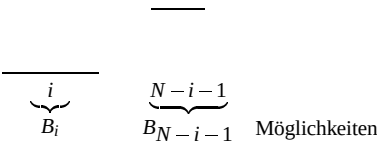


Abbildung 5.15

$$B(z) = \sum_{N \geq 0} B_N \, z^N \tag{5.1}$$

und interpretieren die Koeffizienten B_N wie oben angegeben. Dann gilt nach den Rechenregeln für das Multiplizieren formaler Potenzreihen:

$$\begin{aligned} B(z) \, B(z) &= (B_0 + B_1 z^1 + B_2 z^2 + \dots)(B_0 + B_1 z^1 + B_2 z^2 + \dots) \\ &= \underbrace{(B_0 B_0)}_{=B_1} + \underbrace{(B_0 B_1 + B_1 B_0)}_{=B_2} z^1 + \\ &\quad + \underbrace{(B_0 B_2 + B_1 B_1 + B_2 B_0)}_{=B_3} z^2 + \dots \end{aligned}$$

Weil natürlich $B_0 = 1$ ist, erhält man also:

$$1 + z \, B(z) \, B(z) = B(z)$$

Das ist eine quadratische Gleichung für $B(z)$, die leicht formal aufgelöst werden kann und als eine mögliche Lösung liefert:

$$B(z) = \frac{1 - \sqrt{1 - 4z}}{2z} = \frac{1}{2z} (1 - (1 - 4z)^{\frac{1}{2}}) \tag{5.2}$$

(Die andere Lösung der quadratischen Gleichung für $B(z)$ kommt nicht in Frage, denn die Gleichung soll ja für beliebige z und damit insbesondere für $z = 0$ gelten, d.h. es muß $B(0) = 1$ sein. Das ist aber nur für die hier angegebene Lösung möglich.)

Bekanntlich gilt für beliebige x mit $|x| < 1$ und r :

$$(1 + x)^r = \sum_{k \geq 0} \binom{r}{k} x^k$$

Wendet man das auf Gleichung (5.2) an und setzt $|z| < 1$ voraus, so ergibt sich:

$$\begin{aligned}
B(z) &= \frac{1}{2z} \left(1 - \sum_{k \geq 0} \binom{\frac{1}{2}}{k} (-4z)^k \right) \\
&= \frac{1}{2z} \left(1 + \sum_{N+1 \geq 0} \binom{\frac{1}{2}}{N+1} (-1)^N (4z)^{N+1} \right) \\
&= \frac{1}{2z} + \frac{1}{2z} \sum_{N+1 \geq 0} \binom{\frac{1}{2}}{N+1} (-1)^N 2^{2N+2} z^{N+1} \\
&= \frac{1}{2z} + \sum_{N+1 \geq 0} \binom{\frac{1}{2}}{N+1} (-1)^N 2^{2N+1} z^N \\
&= \frac{1}{2z} + \binom{\frac{1}{2}}{0} (-1)^{-1} 2^{-1} z^{-1} + \sum_{N \geq 0} \binom{\frac{1}{2}}{N+1} (-1)^N 2^{2N+1} z^N \\
&= \sum_{N \geq 0} \binom{\frac{1}{2}}{N+1} (-1)^N 2^{2N+1} z^N
\end{aligned}$$

Ein Koeffizientenvergleich dieser Darstellung mit der ursprünglich definierten Reihe (5.1) ergibt:

$$B_N = \binom{\frac{1}{2}}{N+1} (-1)^N 2^{2N+1} \quad (5.3)$$

Wir haben damit unser Ziel erreicht und einen expliziten Ausdruck für die Anzahl B_N der strukturell möglichen Bäume mit N inneren Knoten gefunden. Die Zahlenfolge (5.3) ist eine in der Zahlentheorie wohlbekannte Folge, nämlich die Folge der Catalanischen Zahlen. Man kann den in (5.3) angegebenen Ausdruck etwas anders schreiben und zeigen, daß gilt:

$$B_N = \frac{1}{N+1} \binom{2N}{N} = \frac{4^N}{N \pi N} + O\left(\frac{4^N}{N^5}\right)$$

Auf ähnliche Weise können wir auch die *gesamte interne Pfadlänge* I_N aller strukturell möglichen Bäume mit N inneren Knoten berechnen.

Die gesuchte durchschnittliche Länge eines Suchpfades eines Baumes mit N inneren Knoten ist dann I_N/B_N .

Zur Berechnung von I_N nutzt man die bereits bekannte Möglichkeit zur rekursiven Berechnung der internen Pfadlänge eines Baumes mit N inneren Knoten aus. Ist t ein Baum mit N inneren Knoten und linkem Teilbaum t_l und rechtem Teilbaum t_r , so ist seine interne Pfadlänge $I(t)$ mit:

$$I(t) = \begin{cases} 0, & \text{falls } t = \square; \\ I(t_l) + I(t_r) + t, & \text{sonst.} \end{cases}$$

Fragen wir also zunächst: Was ist die gesamte interne Pfadlänge aller strukturell möglichen Binärbäume mit N inneren Knoten, deren linker Teilbaum genau i innere Knoten enthält für ein festes i mit $0 \leq i < N$? Es ist nicht schwer zu sehen, daß wegen der oben angegebenen, für jeden einzelnen Baum geltenden Rekursionsformel gilt:

$$I_i B_{N-i-1} + B_i I_{N-i-1} = \text{Gesamtgröße aller Bäume mit } N \text{ inneren Knoten, deren linker Teilbaum } i \text{ innere Knoten hat.}$$

Definiert man also S_N als Summe aller Knotenzahlen aller strukturell möglichen Bäume mit N inneren Knoten, und führt man zwei weitere formale Potenzreihen

$$S(z) = \sum_{N \geq 0} S_N z^N, \quad I(z) = \sum_{N \geq 0} I_N z^N$$

ein, so folgt offenbar:

$$\begin{aligned} I(z) &= z I(z) B(z) + z B(z) I(z) + S(z) \\ &= 2 z I(z) B(z) + S(z) \end{aligned} \quad (5.4)$$

Nun ist nach Definition von $S(z)$ und $B(z)$ natürlich

$$S(z) = \sum_{N \geq 0} N B_N z^N$$

und damit

$$S(z) = z \sum_{N \geq 0} N B_N z^{N-1} = z B'(z). \quad (5.5)$$

Dabei bezeichnet $B'(z)$ die (formale) Ableitung der Potenzreihe $B(z)$, d.h. $B'(z) = \frac{d}{dz}(B(z))$,

$$B'(z) = B_1 + 2B_2z^1 + 3B_3z^2 + \dots$$

Wie im vorigen Fall kann man nun eine explizite Darstellung der gesuchten Koeffizienten I_N herleiten. Aus den Gleichungen (5.4) und (5.5) folgt:

$$\begin{aligned} I(z)(1 - 2zB(z)) &= z B'(z) \\ I(z) &= \frac{1}{1 - 2zB(z)} z \frac{d}{dz}(B(z)) \\ &= \frac{1}{1 - 4z} - \frac{1}{2z} \frac{1}{1 - 4z} + \frac{1}{2z} \end{aligned}$$

Entwickelt man dies wie vorher in eine unendliche Reihe, erhält man nach einer längeren Rechnung:

$$I(z) = \sum_{N \geq 0} (4^N - (2N+1)B_N)z^N$$

Ein Koeffizientenvergleich ergibt also:

$$I_N = (4^N - (2N + 1)B_N)$$

Damit ergibt sich für die mittlere interne Pfadlänge eines Baumes mit N inneren Knoten (gemittelt über alle strukturell möglichen Bäume mit N inneren Knoten):

$$\frac{I_N}{B_N} = N - \frac{1}{\pi N} + O(N)$$

Der mittlere Abstand eines Knotens von der Wurzel eines Binärbaumes mit N inneren Knoten ist also ungefähr $\frac{1}{\pi} \sqrt{N}$ und nicht $O(\log_2 N)$!

5.2 Balancierte Binärbäume

Das Suchen, Einfügen und Entfernen eines Schlüssels in einem zufällig erzeugten binären Suchbaum mit N Schlüsseln ist zwar im Mittel in $O(\log_2 N)$ Schritten ausführbar. Im schlechtesten Fall kann jedoch ein Aufwand von der Ordnung $\Omega(N)$ zur Ausführung dieser Operationen erforderlich sein, weil der gegebene Baum mit N Schlüsseln zu einer linearen Liste degeneriert ist. Es ist daher natürlich, durch zusätzliche Bedingungen an die Struktur der Bäume ein Degenerieren zu verhindern. Die Operationen zum Einfügen und Entfernen von Schlüsseln werden dann allerdings komplizierter als für die im Abschnitt 5.1 behandelten natürlichen Bäume. Man findet in der Literatur eine große Vielfalt von Bedingungen an die Struktur von Bäumen, die sichern, daß ein Baum mit N Knoten eine Höhe $O(\log N)$ hat und daß Suchen, Einfügen und Entfernen von Schlüsseln in logarithmischer Zeit möglich ist. Der historisch erste Vorschlag aus dem Jahr 1962 sind die AVL-Bäume, die auf Adelson-Velskij und Landis zurückgehen

 Hier wird ein Degenerieren von Suchbäumen verhindert durch eine Forderung an die Höhendifferenz der beiden Teilbäume eines jeden Knotens. Diese Bäume heißen daher auch höhenbalancierte Bäume. Wir behandeln AVL-Bäume im Abschnitt 5.2.1. Eng verwandt mit den höhenbalancierten Bäumen sind die in 5.2.2 behandelten Bruder-Bäume. Für sie wird die eine logarithmische Höhe garantierende Dichte erzwungen durch die Forderung, daß alle Blätter denselben Abstand zur Wurzel haben müssen, und durch eine Bedingung an den Verzweigungsgrad von Knoten. In Abschnitt 5.2.3 werden gewichtsbalancierte Bäume betrachtet. Das sind Binärbäume mit der Eigenschaft, daß für jeden Knoten die Gewichte der Teilbäume, das ist die Anzahl ihrer Knoten bzw. Blätter, in einem bestimmten Verhältnis zueinander stehen.

5.2.1 AVL-Bäume

Ein binärer Suchbaum ist *AVL-ausgeglichen* oder *höhenbalanciert*, kurz: ein AVL-Baum, wenn für jeden Knoten p des Baumes gilt, daß sich die Höhe des linken Teilbaumes von der Höhe des rechten Teilbaumes von p höchstens um 1 unterscheidet.

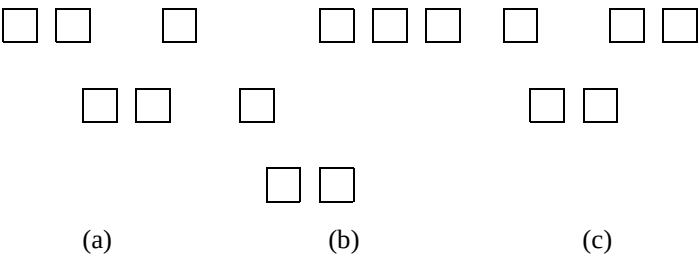


Abbildung 5.16

Die Bäume in Abbildung 5.16 (a) und (c) sind Beispiele für AVL-Bäume. Der Baum in Abbildung 5.16 (b) ist kein AVL-Baum. Da es uns nur auf die Struktur der Bäume ankommt, haben wir die Schlüssel in den Knoten weggelassen.

Wir wollen uns zunächst überlegen, daß AVL-Bäume nicht zu linearen Listen degenerieren können. Die Höhenbedingung sichert vielmehr, daß AVL-Bäume mit N inneren Knoten und $N + 1$ Blättern eine Höhe von $O(\log N)$ haben. Dazu überlegen wir uns, was die minimale Blatt- und Knotenzahl eines AVL-Baumes gegebener Höhe h ist.

Offenbar gilt: Ein AVL-Baum der Höhe 1 hat 2 Blätter und ein AVL-Baum der Höhe 2 mit minimaler Blattzahl hat 3 Blätter (vgl. Abbildung 5.17). Einen AVL-Baum der Höhe $h + 2$ mit minimaler Blattzahl erhält man, wenn man je einen AVL-Baum mit Höhe $h + 1$ und h mit minimaler Blattzahl wie in Abbildung 5.18 zu einem Baum der Höhe $h + 2$ zusammenfügt.

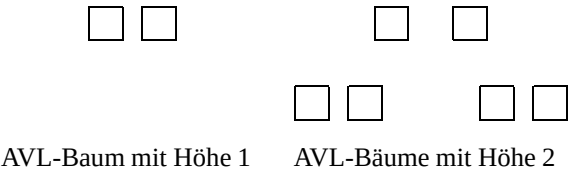


Abbildung 5.17



Abbildung 5.18

Bezeichnet nun F_i die i -te Fibonacci-Zahl, also $F_0 = 0$, $F_1 = 1$, $F_{i+2} = F_i + F_{i+1}$, so folgt unmittelbar aus den obigen Überlegungen: Ein AVL-Baum mit Höhe h hat wenigstens F_{h+2} Blätter.

Es gilt

$$F_h = \frac{1}{\sqrt{5}} \left(\left(\frac{1 + \sqrt{5}}{2} \right)^{h+1} - \left(\frac{1 - \sqrt{5}}{2} \right)^{h+1} \right),$$

wie man leicht durch vollständige Induktion beweist. Der negative Term $\left(\frac{1 - \sqrt{5}}{2} \right)^{h+1}$ ist dem Betrag nach kleiner als 1 und wird daher mit wachsendem h rasch kleiner. Daher gilt (vgl. auch Abschnitt 3.2.3):

$$F_h \approx \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^{h+1} = 0.7236 \dots \cdot 1.618 \dots^h$$

(Genauer: F_h ist die $\frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^{h+1}$ nächstgelegene ganze Zahl.)

Die Anzahl der Blätter eines AVL-Baumes wächst also exponentiell mit der Höhe. Daraus folgt umgekehrt, daß ein AVL-Baum mit N Blättern (und $N - 1$ inneren Knoten) eine Höhe $h \approx 1.44 \dots \log_2 N$ hat. Denn sei ein AVL-Baum mit N Blättern gegeben und sei h seine Höhe. Dann muß gelten:

$$N \geq F_{h+2} \approx 1.894 \cdot 1.618^h,$$

also

$$h \leq \frac{1}{\log_2 1.618 \dots} \log_2 N \approx \frac{\log_2 1.894 \dots}{\log_2 1.618 \dots} \cdot 1.44 \dots \log_2 N + 1.$$

Suchen, Einfügen und Entfernen von Schlüsseln

Da AVL-Bäume insbesondere binäre Suchbäume sind, kann man in ihnen nach einem Schlüssel genauso suchen wie in einem natürlichen Baum. Dazu folgt man im schlechtesten Fall einem Pfad von der Wurzel zu einem Blatt. Weil die Höhe logarithmisch beschränkt bleibt, ist klar, daß man in einem AVL-Baum mit N Schlüsseln in höchstens $O(\log N)$ Schritten einen Schlüssel wiederfinden kann bzw. feststellen kann, daß ein Schlüssel im Baum nicht vorkommt.

Um einen Schlüssel in einen AVL-Baum *einzu*fügen, sucht man zunächst nach dem Schlüssel im Baum. Wenn der einzufügende Schlüssel noch nicht im Baum vorkommt, endet die Suche in einem Blatt, das die erwartete Position des Schlüssels repräsentiert. Man fügt den Schlüssel dort ein, wie im Falle natürlicher Bäume. Im Unterschied zu natürlichen Bäumen kann aber nunmehr ein Suchbaum vorliegen, der kein AVL-Baum mehr ist.

Betrachten wir als Beispiel den Baum in Abbildung 5.19.

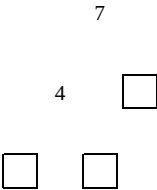


Abbildung 5.19

Fügen wir in diesen Baum den Schlüssel 5 ein, entsteht der Baum in Abbildung 5.20.

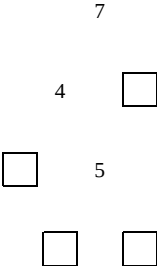


Abbildung 5.20

Das ist kein AVL-Baum mehr, weil für die Wurzel dieses Baumes sich die Höhen des rechten und linken Teilbaumes um mehr als 1 unterscheiden. Man muß also die AVL-

Ausgeglichenheit wiederherstellen. Dazu läuft man von der Einfügestelle den Suchpfad entlang zur Wurzel zurück und prüft an jedem Knoten, ob die Höhendifferenz zwischen linkem und rechtem Teilbaum noch innerhalb der vorgeschriebenen Grenzen liegt. Ist das nicht der Fall, führt man eine sogenannte Rotation oder eine Doppelrotation durch, die die Sortierung der Schlüssel nicht beeinflußt, aber die Höhendifferenzen in den richtigen Bereich bringt.

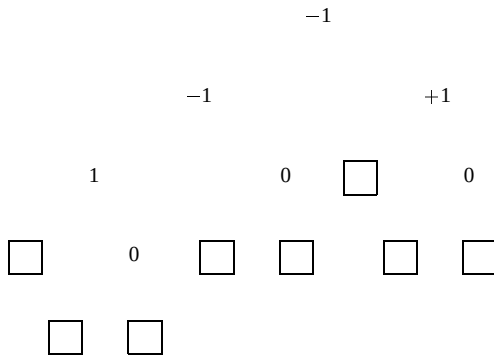
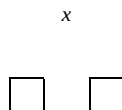


Abbildung 5.21

Man könnte vermuten, daß man zur Prüfung der Höhenbedingung an einem Knoten im Baum die Höhen der Teilbäume des Knotens kennen muß. Das ist jedoch glücklicherweise nicht der Fall. Es genügt, an jedem inneren Knoten p den sogenannten *Balancefaktor* $bal(p)$ mitzuführen, der wie folgt definiert ist:

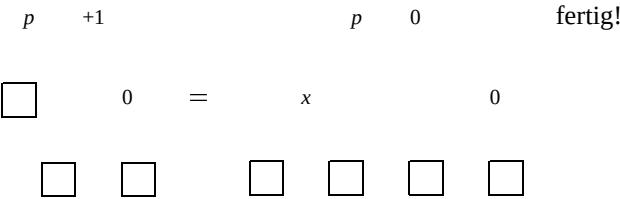
$$bal(p) = \frac{\text{Höhe des rechten Teilbaumes von } p}{\text{Höhe des linken Teilbaumes von } p}.$$

AVL-Bäume sind offenbar gerade dadurch charakterisiert, daß für jeden inneren Knoten p gilt: $bal(p) \in \{-1, 0, +1\}$. Abbildung 5.21 zeigt einen AVL-Baum mit rechts an die Knoten geschriebenen Balancefaktoren. Da es uns hier nur auf die Struktur des Baumes ankommt, haben wir keine Schlüssel in den Knoten angegeben. Wir geben das Verfahren zum Einfügen eines Schlüssels jetzt genauer an. Wird der Schlüssel x in den leeren Baum eingefügt, erhält man den Baum

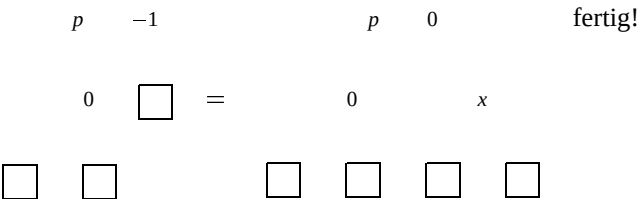


und ist fertig. Sonst sei p der Vater des Blattes, bei dem die Suche endet. Drei Fälle sind möglich:

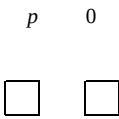
Fall 1 [$bal(p) = +1$]



Fall 2 [$bal(p) = -1$]

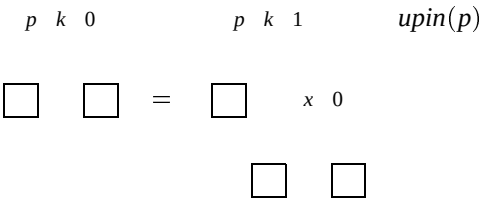


Fall 3 [$bal(p) = 0$]



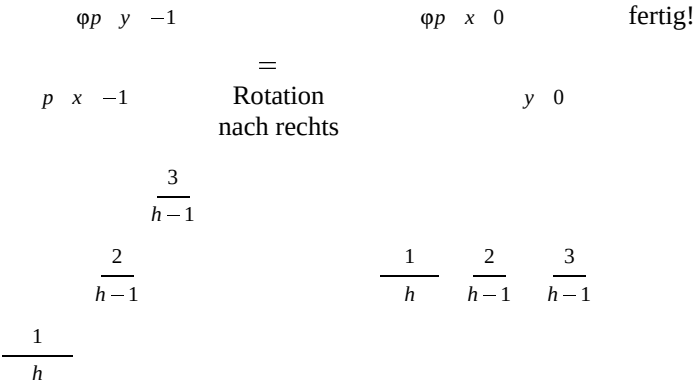
Durch Einfügen eines neuen Knotens als rechten oder linken Sohn von p wird p ein Knoten mit Balancefaktor -1 oder $+1$, und die Höhe des Teilbaumes mit Wurzel p wächst um 1. Wir rufen daher eine Prozedur $upin(p)$ für den Knoten p auf, die den Suchpfad zurückläuft, die Balancefaktoren prüft, gegebenenfalls adjustiert und Umstrukturierungen (sogenannte Rotationen oder Doppelrotationen) vornimmt, die sicherstellen, daß für alle Knoten die Höhendifferenzen der jeweils zugehörigen Teilbäume wieder höchstens 1 sind. Also:

Fall 3.1 [$bal(p) = 0$ und einzufügender Schlüssel $x >$ Schlüssel k von p]



Die Invariante sagt, daß der Teilbaum mit Wurzel p in der Höhe um 1 gewachsen ist. Aus der Voraussetzung $bal(\phi p) = -1$ kann man in diesem Fall schließen, daß bereits vor dem Einfügen des neuen Schlüssels in den linken Teilbaum von ϕp mit Wurzel p dieser Teilbaum eine um 1 größere Höhe hatte als der rechte Teilbaum von ϕp . Da der Teilbaum mit Wurzel p in der Höhe noch um 1 gewachsen ist, ist die AVL-Ausgeglichenheit bei ϕp verletzt. Wir müssen also umstrukturieren und unterscheiden dazu zwei Fälle, je nachdem, ob $bal(p) = +1$ oder $bal(p) = -1$ ist. (Wegen der Invariante ist $bal(p) = 0$ nicht möglich!)

Fall 1.3.1 [$bal(p) = -1$]



$\frac{3}{h-1}$

$\frac{2}{h-1}$

$\frac{1}{h}$

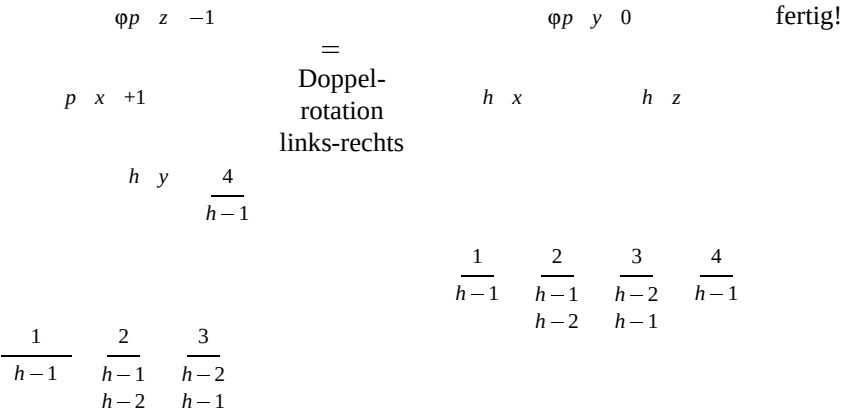
$\frac{1}{h}$

$\frac{2}{h-1}$

$\frac{3}{h-1}$

Man beachte: Nach Voraussetzung ist die Höhe des Teilbaumes mit Wurzel p um 1 gewachsen und der linke Teilbaum von p um 1 höher als der rechte. Eine *Rotation nach rechts* bringt den Baum bei ϕp wieder in die Balance. Es ist keine weitere Umstrukturierung nötig, weil der durch Rotation entstehende Teilbaum mit Wurzel ϕp in der Höhe nicht mehr gewachsen ist. Wir haben unter die drei Teilbäume die Höhen geschrieben, um so zu zeigen, daß der entstehende Baum nach der Umstrukturierung wieder ausgeglichen ist. Die Höhen sind aber selbstverständlich nicht explizit gespeichert und werden nicht benötigt, um festzustellen, daß die angegebene Umstrukturierung ausgeführt werden soll.

Fall 1.3.2 [$bal(p) = +1$]



$\frac{4}{h-1}$

$\frac{1}{h-1}$

$\frac{2}{h-1}$

$\frac{3}{h-2}$

$\frac{4}{h-1}$

$\frac{1}{h-1}$

$\frac{2}{h-1}$

$\frac{3}{h-2}$

$\frac{4}{h-1}$

Man beachte: Entweder sind die Teilbäume 2 und 3 beide leer oder die einzig möglichen Höhenkombinationen für die Teilbäume 2 und 3 sind (h_1, h_2) und (h_2, h_1) . Falls nicht beide Teilbäume leer sind, können sie nicht gleiche Höhe haben. Denn auf Grund der Invarianten ist der Teilbaum mit Wurzel p in der Höhe um 1 gewachsen und wegen der Annahme von Fall 1.3.2 ist der rechte Teilbaum von p um 1 höher als sein linker. Eine *Doppelrotation*, d.h. zunächst eine Rotation nach links bei p und dann eine Rotation nach rechts bei ϕp , stellt die AVL-Ausgeglichenheit bei ϕp wieder her. Eine weitere Umstrukturierung ist nicht nötig, da der Teilbaum mit Wurzel ϕp in der Höhe nicht wächst.

Fall 2 [p ist rechter Sohn seines Vaters ϕp]
 In diesem Fall geht man völlig analog vor und gleicht den Baum, wenn nötig, durch eine Rotation nach links bzw. eine Doppelrotation rechts-links bei ϕp wieder aus. Zur Veranschaulichung der Rotation nach links liest man die im Fall 1.3.1 gezeigte Abbildung von rechts nach links. Die Doppelrotation rechts-links erhält man aus der im Fall 1.3.2 gezeigten Figur durch Vertauschen der linken und rechten Teilbäume von p und ϕp .

Wir zeigen die Umstrukturierung noch einmal an einem Beispiel und beginnen mit dem Baum in Abbildung 5.22. Dieser Baum ist ein AVL-Baum. Wir fügen den Schlüssel 9 ein und erhalten Abbildung 5.23.

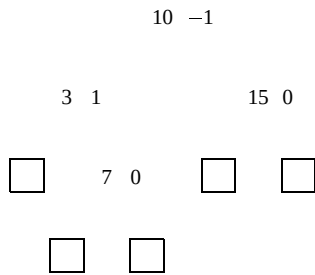


Abbildung 5.22

Das ist kein AVL-Baum mehr; eine Rotation nach links bei p stellt die AVL-Ausgeglichenheit wieder her (siehe Abbildung 5.24). Einfügen von 8 und anschließende Doppelrotation liefert Abbildung 5.25.

Ein Aufruf der Prozedur *upin* kann schlimmstenfalls für alle Knoten auf dem Suchpfad von der Einfügestelle zurück zur Wurzel erforderlich sein. In jedem Fall wird aber höchstens eine Rotation oder Doppelrotation durchgeführt. Denn nach Ausführung einer Rotation oder Doppelrotation in den Fällen 1.3.1 und 1.3.2 und den dazu symmetrischen Fällen wird die Prozedur *upin* nicht mehr aufgerufen. Die Umstrukturierung einschließlich der Adjustierung der Balancefaktoren ist also beendet und die AVL-Ausgeglichenheit wiederhergestellt. Damit ist klar, daß das Einfügen eines neuen Schlüssels in einen AVL-Baum mit N Schlüssel in $O(\log N)$ Schritten ausführbar ist.

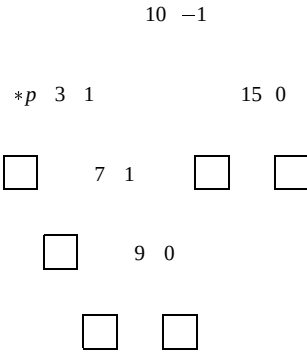


Abbildung 5.23

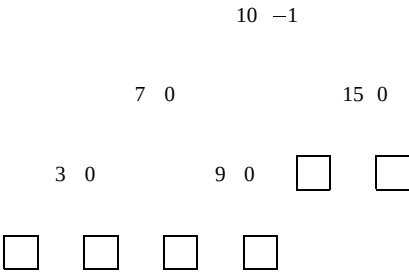


Abbildung 5.24

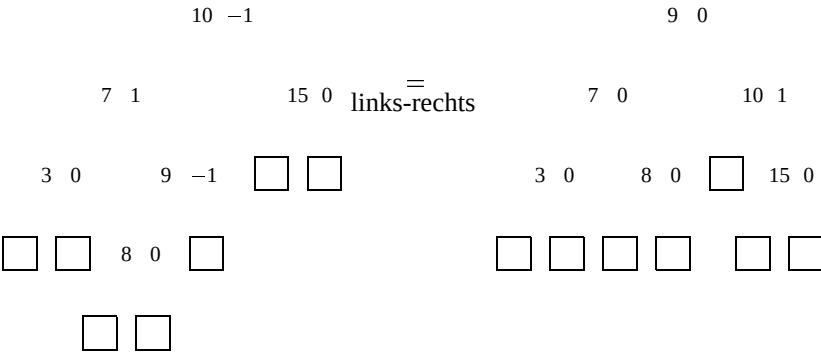


Abbildung 5.25

Das Entfernen eines Schlüssels aus einem AVL-Baum

Zunächst geht man genauso vor wie bei natürlichen Suchbäumen. Man sucht nach dem zu entfernenden Schlüssel. Findet man ihn nicht, ist das Entfernen bereits beendet. Sonst liegt einer der folgenden drei Fälle vor.

Fall 1: Der zu entfernende Schlüssel ist der Schlüssel eines Knotens, dessen beide Söhne Blätter sind. Dann entfernt man den Knoten und ersetzt ihn durch ein Blatt. Falls der Baum nunmehr nicht der leere Baum geworden ist, bezeichne p den Vater des neuen Blattes. Weil der Teilbaum von p , der durch das Blatt ersetzt wurde, die Höhe 1 hatte, muß der andere Teilbaum von p mit Wurzel q die Höhe 0, 1 oder 2 haben. Hat er die Höhe 1, so ändert man einfach die Balance von p von 0 auf +1 oder -1 und ist fertig. Hat der Teilbaum mit Wurzel q die Höhe 0, so ändert man die Balance p von +1 oder -1 auf 0. In diesem Fall ist die Höhe des Teilbaums mit Wurzel p um 1 gefallen. Damit können sich auch für alle Knoten auf dem Suchpfad nach p die Balancefaktoren und die Höhen der Teilbäume verändert haben. Wir rufen daher eine Prozedur $upout(p)$ auf, die die AVL-Ausgeglichenheit wiederherstellt. Hatte schließlich der Teilbaum mit Wurzel q die Höhe 2, d.h. war $bal(p) = -1$ und q kein Blatt, so führt man zunächst eine Rotation oder Doppelrotation aus, um den Baum mit Wurzel p wieder auszugleichen. Dabei kann ein anderer Knoten r an die Wurzel dieses Teilbaumes gelangen. Wenn die Wurzelbalance dieses Teilbaumes auf 0 gesetzt wird, ist seine Höhe um 1 gesunken, so daß wieder $upout(r)$ aufgerufen wird, um die AVL-Ausgeglichenheit wiederherzustellen.

(Bemerkung: Die im letzten Fall erforderlichen Umstrukturierungen werden auch ausgeführt, wenn man die weiter unten beschriebene Prozedur $upout$ einfach für das Blatt aufruft, das den entfernten Knoten ersetzt.)

Fall 2: Der zu entfernende Schlüssel ist der Schlüssel eines Knotens p , der nur einen inneren Knoten q als Sohn hat. Dann müssen beide Söhne von q Blätter sein. Man ersetzt also den Schlüssel von p durch den Schlüssel von q und ersetzt q durch ein Blatt. Damit ist nunmehr p ein Knoten mit $bal(p) = 0$ und die Höhe des Teilbaums mit Wurzel p um 1 gesunken (von 2 auf 1). Auch in diesem Fall rufen wir $upout(p)$ auf, um die AVL-Ausgeglichenheit wiederherzustellen.

Fall 3: Der zu entfernende Schlüssel ist der Schlüssel eines Knotens p , dessen beide Söhne innere Knoten sind. Dann geht man wie im Falle natürlicher Suchbäume vor und ersetzt den Schlüssel durch den Schlüssel des symmetrischen Nachfolgers (oder Vorgängers) und entfernt den symmetrischen Nachfolger (oder Vorgänger). Das muß dann ein Knoten sein, dessen Schlüssel wie im Fall 1 und 2 beschrieben entfernt wird.

In jedem Fall haben wir das Entfernen reduziert auf die Ausführung der Prozedur $upout(p)$ für einen Knoten p mit $bal(p) = 0$, dessen Teilbaum in der Höhe um 1 gefallen ist.

Wir geben diese Prozedur $upout$ nun genauer an. Sie kann längs des Suchpfades rekursiv aufgerufen werden, adjustiert die Höhenbalancen und führt gegebenenfalls Rotationen oder Doppelrotationen durch, um den Baum wieder auszugleichen. Wenn $upout(p)$ aufgerufen wird, gilt: $bal(p) = 0$ und der Teilbaum mit Wurzel p ist in der Höhe um 1 gefallen. Wir müssen darauf achten, daß diese Invariante vor jedem rekursiven Aufruf von $upout$ gilt. Wir unterscheiden wieder zwei Fälle, je nachdem ob p linker oder rechter Sohn seines Vaters $\wp$ ist.

Fall 1 [p ist linker Sohn seines Vaters ϕp]
Fall 1.1 [$bal(\phi p) = -1$]

$$\phi p \quad -1 \qquad \phi p \quad 0 \qquad upout(\phi p)$$

$$p \quad 0 \qquad = \qquad 0$$

Man beachte, daß vor dem rekursiven Aufruf von *upout* die Invariante für ϕp gilt.
Fall 1.2 [$bal(\phi p) = 0$]

$$\phi p \quad 0 \qquad \phi p \quad 1 \qquad \text{fertig!}$$

$$p \quad 0 \qquad = \qquad p \quad 0$$

Fall 1.3 [$bal(\phi p) = +1$]

$$\phi p \quad +1$$

$$p \quad 0 \qquad q$$

Der rechte Teilbaum von ϕp mit Wurzel q ist also höher als der linke mit Wurzel p , der darüber hinaus noch in der Höhe um 1 gefallen ist. Wir machen eine Fallunterscheidung nach dem Balancefaktor von q .
Fall 1.3.1 [$bal(q) = 0$]

$$\phi p \quad v + 1$$
$$p \quad u \quad 0 \qquad q \quad w \quad 0$$

$$=$$

Rotation
nach links

$$p \quad u \quad 0$$

$$w \quad -1$$
$$v \quad +1$$

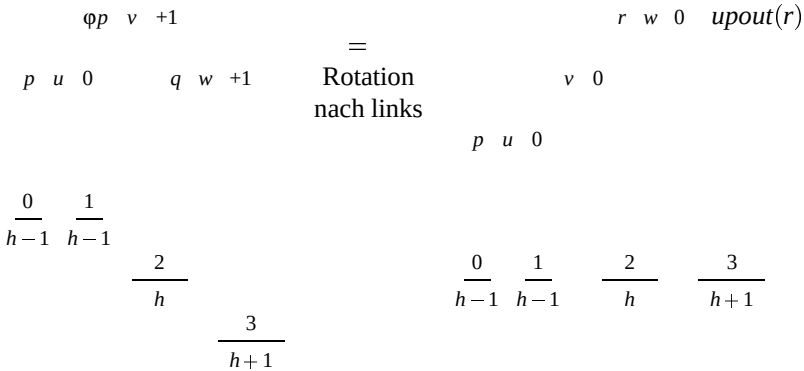
$$\text{fertig!}$$

$$\frac{0}{h-1} \quad \frac{1}{h-1}$$
$$\frac{2}{h+1} \quad \frac{3}{h+1}$$

$$\frac{0}{h-1} \quad \frac{1}{h-1} \quad \frac{2}{h+1}$$

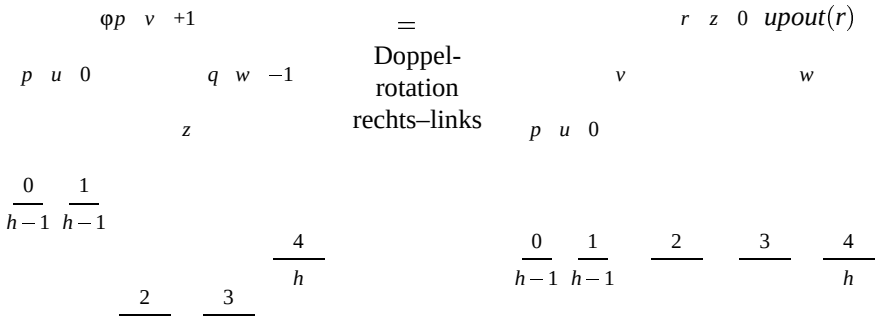
$$\frac{3}{h+1}$$

Fall 1.3.2 $[bal(q) = +1]$



Man beachte, daß vor dem rekursiven Aufruf von *upout* die Invariante für *r* gilt!

Fall 1.3.3 $[bal(q) = -1]$



Weil der Teilbaum mit Wurzel *p* in der Höhe um 1 gefallen ist und der rechte Teilbaum von ϕp vor dem Entfernen eines Schlüssels aus dem Teilbaum mit Wurzel *p* um 1 höher war als der linke, folgt, daß der Teilbaum mit Wurzel *q* die Höhe *h* + 2 haben muß. Wegen $bal(q) = -1$ hat der linke Teilbaum von *q* mit Wurzel *z* die Höhe *h* + 1 und der rechte die Höhe *h*. Die Teilbäume von *z* können entweder beide die Höhe *h* oder höchstens einer von ihnen die Höhe *h* - 1 haben. In jedem Fall gleicht die angegebene Umstrukturierung den Baum wieder aus. Dabei hängen die Balancefaktoren der Knoten *v* und *w* vom Balancefaktor von *z* ab. Auf jeden Fall hat der Teilbaum mit Wurzel *r* den Balancefaktor 0 und seine Höhe ist um 1 gefallen. Es gilt also die Invariante für den Aufruf von *upout*.

Der Fall 2 [*p* ist rechter Sohn seines Vaters ϕp] ist völlig symmetrisch zum Fall 1 und wird daher nicht näher behandelt.

Anders als im Falle der Prozedur *upin* kann es vorkommen, daß auch nach einer Rotation oder Doppelrotation die Prozedur *upout* erneut aufgerufen werden muß. Daher reicht im allgemeinen eine einzige Rotation oder Doppelrotation nicht aus, um den Baum nach Entfernen eines Schlüssels wieder AVL-ausgeglichen zu machen. Es ist nicht schwer, Beispiele zu finden, in denen an allen Knoten auf dem Suchpfad von der Entfernerstelle zurück zur Wurzel eine Rotation oder Doppelrotation ausgeführt werden muß. Da jedoch der Aufwand zum Ausführen einer einzelnen Rotation oder Doppelrotation konstant ist, und da die Höhe *h* von AVL-Bäumen mit *N* Schlüsseln durch

1.44... $\log_2 N$ beschränkt ist, folgt unmittelbar: Das Entfernen eines Schlüssels aus einem AVL-Baum mit N Schlüsseln ist in $O(\log N)$ Schritten ausführbar. Damit sind alle drei Wörterbuchoperationen Suchen, Einfügen und Entfernen auch im schlechtesten Fall in $O(\log N)$ Schritten ausführbar.

AVL-Bäume sind also eine *worst-case-effiziente* Implementation von Wörterbüchern im Gegensatz zu natürlichen Bäumen, die im *Average-case* zwar genauso effizient sind, im Worst-case aber $\Omega(N)$ Schritte zum Ausführen der Wörterbuchoperationen benötigen.

Eine interessante Frage für jede Klasse balancierter Bäume ist, was der mittlere Aufwand zur Ausführung der Wörterbuchoperationen ist, wenn man über eine Folge derartiger Operationen mittelt. Man kann für AVL-Bäume, die im nächsten Abschnitt 5.2.2 behandelten Bruder-Bäume und auch für die im Abschnitt 5.2.3 behandelten gewichtsbalancierten Bäume zeigen, daß der Aufwand pro Einfüge-Operation gemittelt über eine Folge von Einfüge-Operationen *konstant* ist. Dieser Nachweis ist am einfachsten für die Klasse der Bruder-Bäume zu führen. Darüberhinaus können auch weitere, das mittlere Verhalten des Einfügeverfahrens charakterisierende Parameter für die Klasse der Bruder-Bäume besonders leicht hergeleitet werden mit Hilfe einer Technik, die als *Fringe-Analyse* bekannt ist und in Abschnitt 5.2.2 genauer behandelt wird.

5.2.2 Bruder-Bäume

Bruder-Bäume kann man in einem präzisierbaren Sinn als expandierte AVL-Bäume auffassen . Durch Einfügen unärer Knoten an den richtigen Stellen erhält man einen Baum, dessen sämtliche Blätter dieselbe Tiefe haben; und umgekehrt entsteht aus einem Bruder-Baum ein höhenbalancierter Baum, wenn man die unären Knoten mit ihren einzigen Söhnen verschmilzt. Man könnte diesen Zusammenhang dazu benutzen, Such-, Einfüge- und Entferne-Operationen für Bruder-Bäume zu gewinnen, indem man sie von den AVL-Bäumen herüberzieht. Wenn man das macht, erhält man aber Algorithmen, die sich von den im folgenden angegebenen unterscheiden, weniger leicht erklärbar und insbesondere nicht so einfach zu analysieren sind. Unsere Algorithmen folgen einer Strategie, die sich stark an die im Abschnitt 5.5 behandelten Verfahren für B-Bäume anlehnt.

Zunächst jedoch zur genauen Definition der Bruder-Bäume: Im Unterschied zu allen anderen Binärbäumen erlauben wir, daß ein innerer Knoten auch nur einen Sohn haben kann. Natürlich dürfen nicht zu viele unäre Knoten vorkommen, weil man dann offensichtlich entartete Bäume mit großer Höhe und wenigen Blättern erhalten könnte. Man erzwingt daher eine Mindestdichte durch eine Bedingung an die *Brüder* unärer Knoten. Dabei heißen zwei Knoten Brüder, wenn sie denselben Vater haben. Genauer definieren wir: Ein binärer Baum heißt ein *Bruder-Baum*, wenn jeder innere Knoten einen oder zwei Söhne hat, jeder unäre Knoten einen binären Bruder hat und alle Blätter dieselbe Tiefe haben. Abbildung 5.26 enthält einige Beispiele.

Als unmittelbare Folgerung aus der Definition erhält man: Ist ein Knoten p der einzige Sohn seines Vaters, so ist p ein Blatt oder binär. Von zwei Söhnen eines binären Knotens kann höchstens einer unär sein.

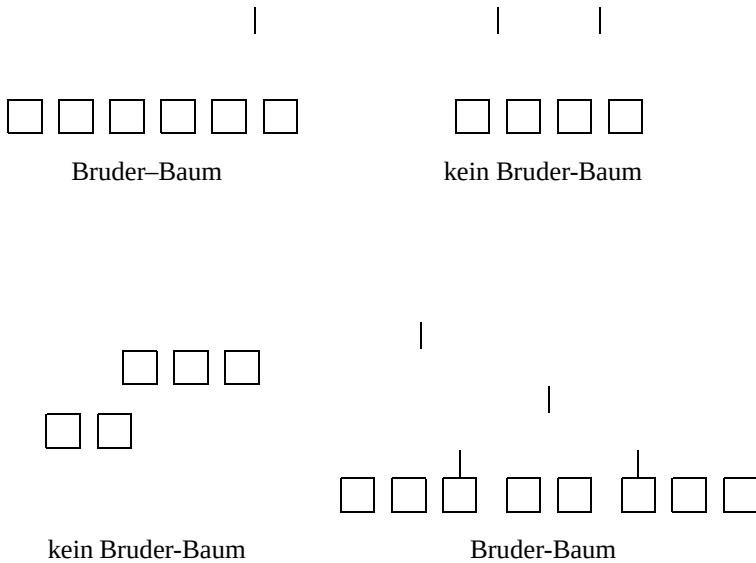


Abbildung 5.26

Offensichtlich ist die Anzahl der Blätter eines Bruder-Baumes stets um 1 größer als die Anzahl der binären (inneren) Knoten.

Betrachten wir die Folge der Bruder-Bäume mit einer gegebenen Höhe und minimaler Blattzahl in Tabelle 5.1.

Wie für AVL-Bäume folgt auch hier: Ein Bruder-Baum mit Höhe h hat wenigstens F_{h+2} Blätter. (F_i ist die i -te Fibonacci-Zahl.) Also umgekehrt: Ein Bruder-Baum mit N Blättern und $(N - 1)$ inneren Knoten hat eine Höhe $h = 1.44 \dots \log_2 N$.

Wir haben bislang offengelassen, wie Schlüssel in Bruder-Bäumen gespeichert werden können. Dazu gibt es, wie bei binären Suchbäumen, bei denen jeder innere Knoten zwei Söhne hat, auch zwei Möglichkeiten. Erstens kann man Bruder-Bäume als Blattsuchbäume organisieren. Dann sind die Schlüssel die Werte der Blätter, z.B. von links nach rechts aufsteigend sortiert; innere Knoten enthalten Wegweiser zum Auffinden der Schlüssel an den Blättern. Natürlich genügt es, Wegweiser an den binären Knoten aufzustellen.

Die andere Möglichkeit besteht darin, die Schlüssel in den binären Knoten zu speichern und, wie für binäre Suchbäume, zu verlangen, daß für jeden binären Knoten p gilt: Die Schlüssel im linken Teilbaum von p sind sämtlich kleiner als der Schlüssel von p , und dieser ist wiederum kleiner als sämtliche Schlüssel im rechten Teilbaum von p . Die unären Knoten und die Blätter speichern natürlich keine Schlüssel. Wir wollen im folgenden nur noch diese Variante betrachten und sprechen von *1-2-Bruder-Bäumen*. Diese Bezeichnung hat ihren Ursprung in einer für Vielwegbäume üblichen

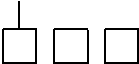
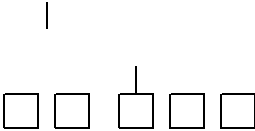
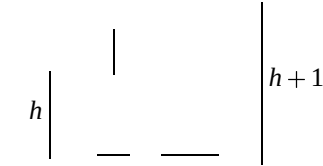
Höhe	Bruder-Bäume mit minimaler Blattzahl	Blattzahl
1		2
2		3
3		5
$\vdots$		$\vdots$
$h + 2$		F_{h+4}
jeweils Bäume minimaler Blattzahl		

Tabelle 5.1

Sprechweise: Man spricht dort von *a-b-Bäumen*, wobei *a* und *b* zwei natürliche Zahlen mit $b \geq a$ sind, also z.B. von 2-3-Bäumen, 2-4-Bäumen oder $m/2$ -*m*-Bäumen für ein $m \geq 2$. Das sind Bäume mit der Eigenschaft, daß jeder innere Knoten mindestens *a* und höchstens *b* Söhne hat. Man fordert weiter, daß alle Blätter gleiche Tiefe haben müssen und jeder Knoten mit *i* Söhnen genau $(i - 1)$ Schlüssel gespeichert hat. 1-2-Bruder-Bäume sind damit spezielle 1-2-Bäume. Die im Abschnitt 5.5 behandelten B-Bäume sind $m/2$ -*m*-Bäume.

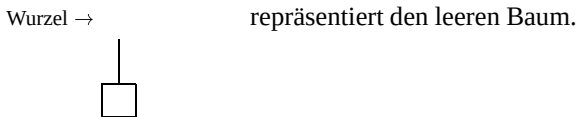
Suchen, Einfügen und Entfernen von Schlüsseln

Bevor wir die Algorithmen zum Suchen, Einfügen und Entfernen von Schlüsseln in 1-2-Bruder-Bäumen angeben, wollen wir noch eine Vorbemerkung zur möglichen Implementation machen. Es ist natürlich, die Knoten eines 1-2-Bruder-Baumes als Record mit Varianten zu definieren. Blätter werden implizit durch **nil**-Zeiger in ihren Vätern repräsentiert. Alle anderen Knoten sind von folgendem Typ:

```

type
  arity = (unary, binary);
  Knotenzeiger = Knoten;
  Knoten = record
    case tag : arity of
      unary : (son : Knotenzeiger);
      binary : (leftson, rightson : Knotenzeiger;
                key : integer;
                info : infotype )
    end
  
```

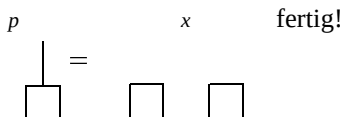
Obwohl üblicherweise der leere Baum durch den Wert **nil** einer Variablen *wurzel* vom Typ Knotenzeiger repräsentiert wird, wollen wir hier eine für unsere Zwecke bequemere Form wählen:



Das *Suchen* in einem 1-2-Bruder-Baum nach einem gegebenen Schlüssel *x* unterscheidet sich nur unwesentlich vom Suchen in binären Suchbäumen. Man muß lediglich einen weiteren Fall vorsehen. Trifft man bei der Suche nach einem Schlüssel *x* auf einen unären Knoten, so setzt man die Suche bei dessen Sohn fort.

Zum *Einfügen* eines neuen Schlüssels *x* in einen 1-2-Bruder-Baum sucht man zunächst im Baum nach *x*. Wenn der Schlüssel *x* im Baum noch nicht vorkommt, endet die Suche erfolglos in einem Blatt. Sei *p* der Vater dieses Blattes.

Fall 1 [*p* hat nur einen Sohn]



Fall 2 [p hat bereits zwei Söhne und damit einen Schlüssel $p.key$]
 Wir können ohne Einschränkung annehmen, daß $x < p.key$ ist. (Sonst vertausche man x und $p.key$.) In diesem Fall kann man den Schlüssel x nicht mehr im Knoten p unterbringen. Man versucht daher, den Schlüssel x bzw. einen anderen Schlüssel, um Platz für x zu schaffen, beim Bruder von p oder beim Vater von p unterzubringen. Findet man in der unmittelbaren Verwandtschaft des Knotens p keinen Knoten, der noch Platz hat, also unär war und binär gemacht werden könnte, so verschiebt man das Einfügeproblem rekursiv um ein Niveau nach oben, bis man gegebenenfalls bei der Wurzel angelangt ist. Wenn dieser letzte Fall eintritt, wird der Baum durch Schaffen einer neuen Wurzel um ein Niveau aufgestockt. (Bruder-Bäume wachsen also an der Wurzel und nicht an den Blättern wie die AVL-Bäume!) Man teilt oder spaltet also einen unären bzw. binären Knoten in einen binären bzw. einen unären und einen binären Knoten. Diese intuitive Idee führt zu folgender Prozedur up , die in der in Abbildung 5.27 dargestellten Anfangssituation aufgerufen wird.

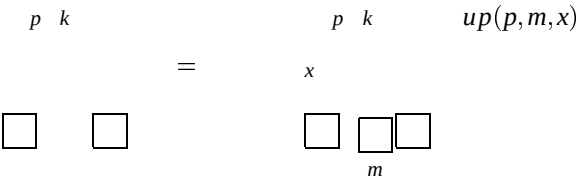
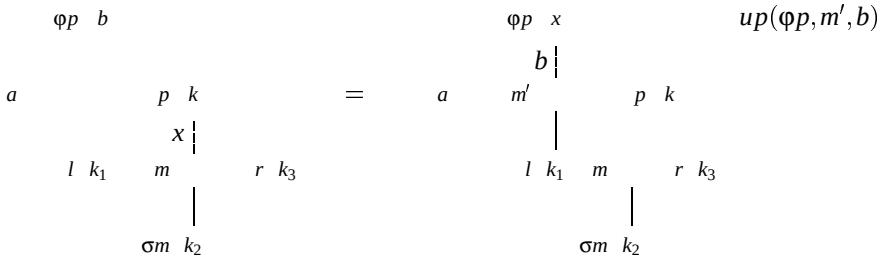


Abbildung 5.27

Vor dem ersten Aufruf der Prozedur up und vor jedem späteren rekursiven Aufruf gilt die folgende Invariante. Wenn $up(p, m, x)$ aufgerufen wird, gelten (1), (2) und (3):

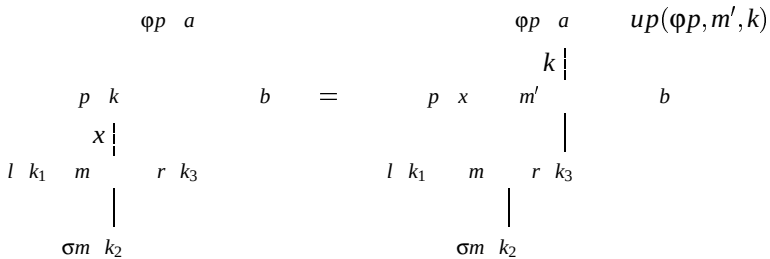
- (1) p hat zwei Söhne p_l und p_r , die beide Wurzeln von 1-2-Bruder-Bäumen sind.
- (2) Der Knoten m ist entweder ein Blatt oder hat einen einzigen Sohn, der Wurzel eines 1-2-Bruder-Baumes ist.
- (3) Schlüssel im linken Teilbaum von $p < x$
 $<$ Schlüssel im Teilbaum von m
 $<$ Schlüssel von p
 $<$ Schlüssel im rechten Teilbaum von p

Fall 1 [p hat einen linken Bruder mit zwei Söhnen]

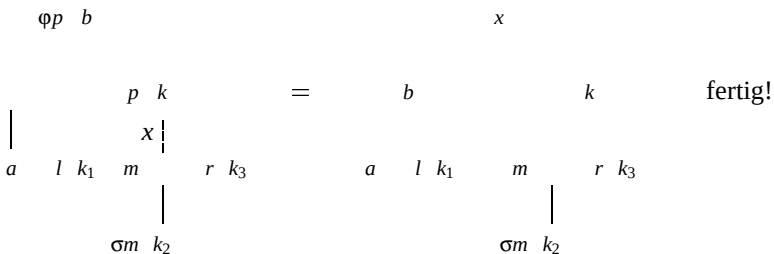


Falls l, m, r Blätter sind, wenn also die Prozedur $up(p, \cdot, \cdot)$ erstmals aufgerufen wird, existiert σm nicht. In diesem Fall muß man natürlich auch die Schlüssel k_1, k_2, k_3 weglassen. Ähnliche Annahmen muß man auch in den folgenden Figuren machen, um den Blattfall abzudecken.

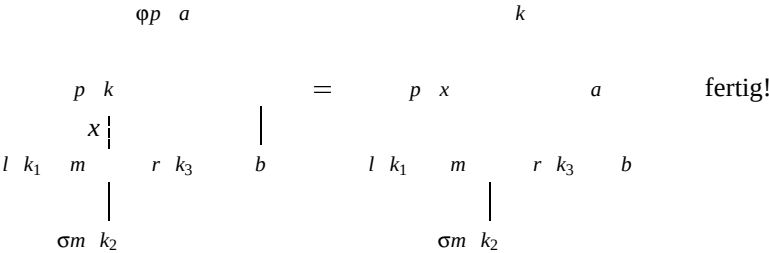
Fall 2 [p hat einen rechten Bruder mit zwei Söhnen]



Fall 3 [p hat einen linken Bruder mit nur einem Sohn]

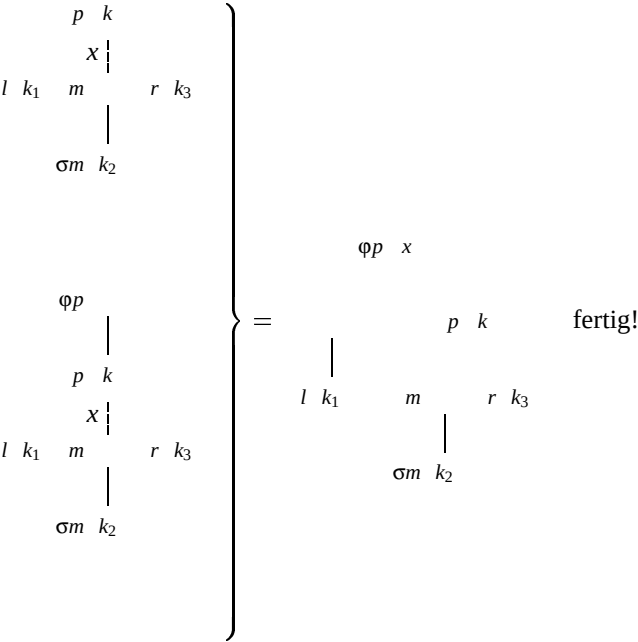


Fall 4 [p hat einen rechten Bruder mit nur einem Sohn]



fertig!

Fall 5 [p hat keinen Bruder] Dann ist p entweder die Wurzel oder einziger Sohn seines Vaters.

 $=$

φp

x

l

k_1

m

r

k_3

σm

k_2

fertig!

Wir betrachten als Beispiel die Folge der 1-2-Bruder-Bäume, die sich durch iteriertes Einfügen der Schlüssel 1, 2, 3, 4, 5 in aufsteigender Reihenfolge in den anfangs leeren Baum ergibt, vgl. Abbildung 5.28. Weiteres Einfügen der Schlüssel 6 und 7 liefert den vollständigen Binärbaum mit Höhe 3. Durch einen nicht ganz einfachen Induktionsbeweis $\Rightarrow$ läßt sich zeigen, daß iteriertes Einfügen von $2^k - 1$ Schlüsseln in aufsteigend sortierter Reihenfolge den vollständigen Binärbaum mit Höhe h liefert. Bruder-Bäume verhalten sich damit gerade entgegengesetzt zu den im Abschnitt 5.5 behandelten B-Bäumen. Iteriertes Einfügen in auf- oder absteigend sortierter Reihenfolge liefert besonders niedrige Bruder-Bäume, aber besonders hohe B-Bäume. In keinem

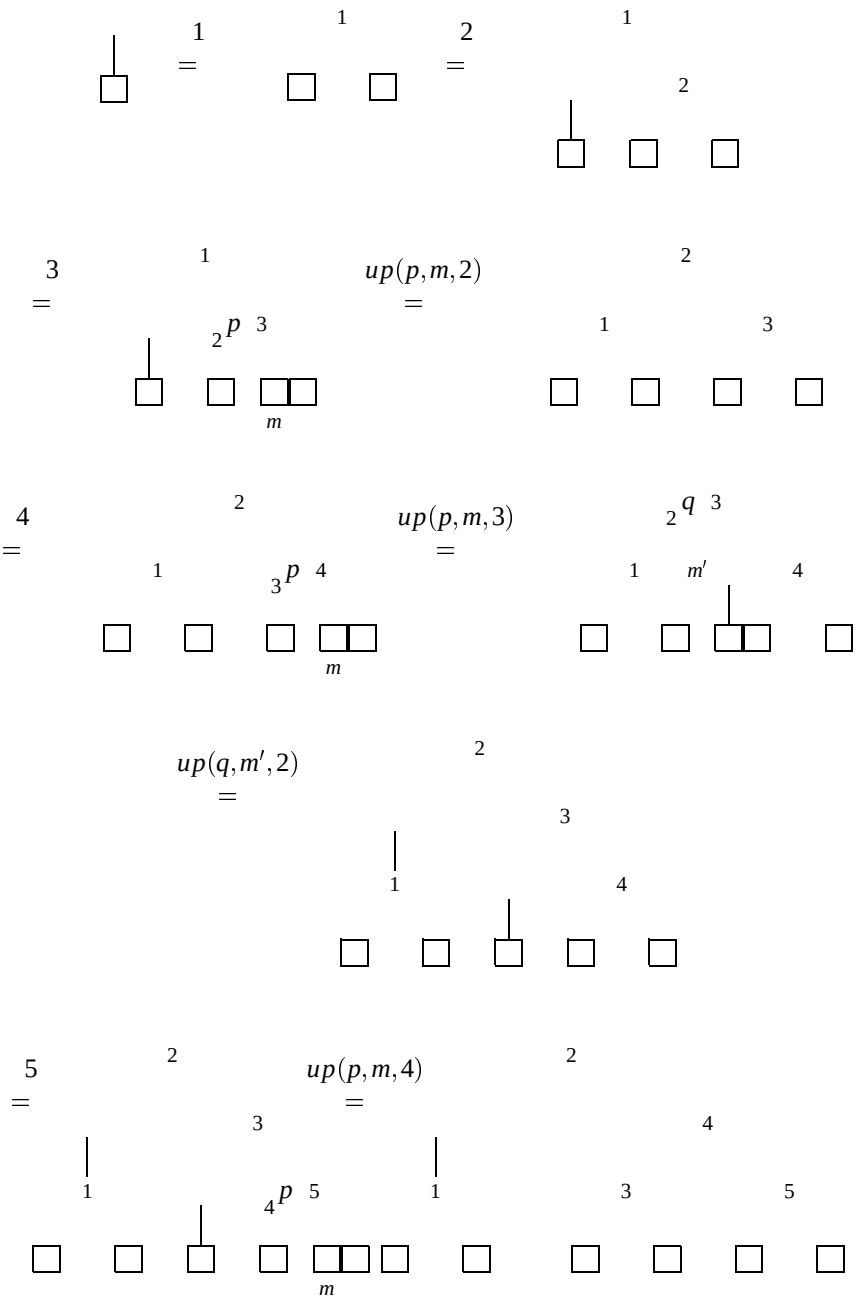


Abbildung 5.28

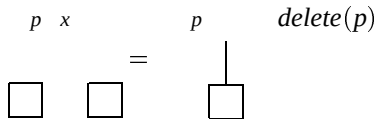
Fall kann die Höhe eines 1-2-Bruder-Baumes, der durch iteriertes Einfügen von $N - 1$ Schlüsseln in den anfangs leeren Baum entsteht, größer sein als $1.44 \dots \log_2 N$. Welche Höhe wird man im Mittel erwarten können, wenn man über alle möglichen Anordnungen von Schlüsseln und die ihnen durch iteriertes Einfügen in den anfangs leeren Baum zugeordneten 1-2-Bruder-Bäume mittelt?

Eine Antwort auf diese Frage und andere das mittlere Verhalten des Einfügeverfahrens charakterisierende Eigenschaften werden wir mit Hilfe der Fringe-Analyse-Technik erhalten, die wir am Ende dieses Abschnitts besprechen.

Zunächst sieht man der Prozedur *up* unmittelbar an, daß sie im schlechtesten Fall längs des Suchpfades von der Einfügestelle zurück zur Wurzel aufgerufen werden kann. Damit gilt: Das Einfügen eines neuen Schlüssels in einen 1-2-Bruder-Baum mit N Schlüsseln ist in $O(\log N)$ Schritten ausführbar.

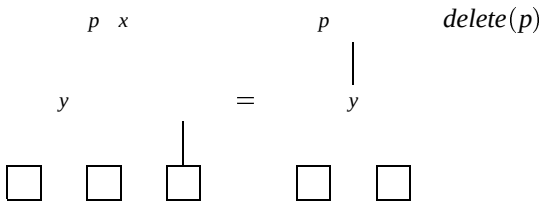
Um einen Schlüssel x aus einem 1-2-Bruder-Baum zu *entfernen*, sucht man zunächst nach x im Baum. Wenn es keinen (binären) Knoten mit Wert x im Baum gibt, ist man bereits fertig. Sonst ist der zu entfernende Schlüssel x der Schlüssel eines binären Knotens p . Wie im Fall binärer Suchbäume oder im Falle von AVL-Bäumen muß man auch hier unter Umständen das Entfernen des Schlüssels von p auf das Entfernen des symmetrischen Nachfolgers reduzieren. Dann kann man ohne Einschränkung annehmen, daß einer der folgenden Fälle vorliegt:

Fall 1 [Die Söhne von p sind Blätter]



Man macht p unär, entfernt den Schlüssel x von p und ruft die weiter unten erklärte Prozedur *delete(p)* auf.

Fall 2 [Der rechte (oder linke) Sohn von p ist unär und hat ein Blatt als einzigen Sohn]



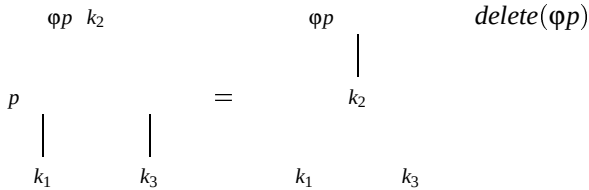
Da ein vorher binärer Knoten p unär gemacht worden ist, kann in der Verwandtschaft von p eine der Bruder-Bäume charakterisierenden Bedingungen verletzt sein. Ein unärer Knoten hat möglicherweise keinen binären Bruder mehr. Die Prozedur *delete* sorgt dafür, daß diese Bedingung wiederhergestellt wird, indem zwei unäre Knoten zu einem binären verschmolzen werden.

Wenn *delete(p)* aufgerufen wird, gilt die folgende Invariante: Der Knoten p ist unär und der einzige Sohn von p ist die Wurzel eines 1-2-Bruder-Baumes. p hat seinen Schlüssel verloren, außer für p und den Bruder von p , falls es ihn gibt, gilt die Bedingung, daß unäre Knoten binäre Brüder haben.

Fall 1 [p hat einen Bruder mit zwei Söhnen]

Dann ist nichts zu tun.

Fall 2 [p hat einen Bruder mit nur einem Sohn]



Der Fall, daß p rechter Sohn seines Vaters ist, wird natürlich genauso behandelt.

Fall 3 [p hat keinen Bruder]

Fall 3.1 [p ist die Wurzel]

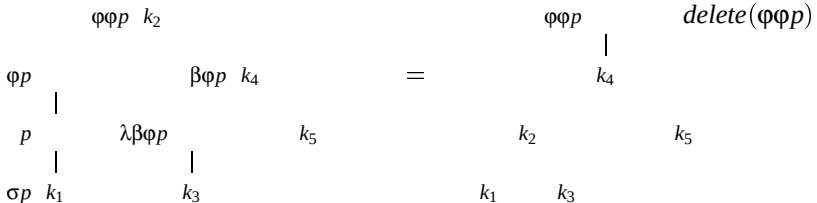
Dann entfernt man p , macht den einzigen Sohn von p zur neuen Wurzel und ist fertig.

Fall 3.2 [p ist einziger Sohn seines Vaters φp]

Aufgrund der Invarianten muß φp einen binären Bruder $\beta \varphi p$ haben. Wir machen eine Fallunterscheidung je nachdem, ob $\beta \varphi p$ drei oder vier Enkel hat:

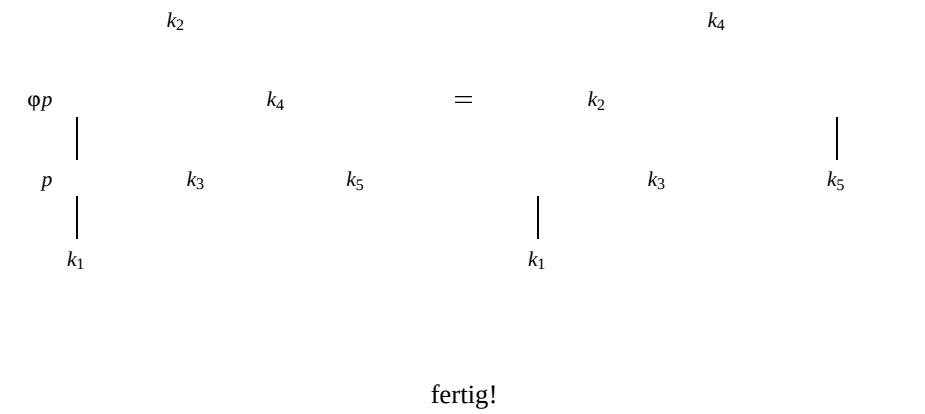
Fall 3.2.1 [Der linke oder der rechte Sohn von $\beta \varphi p$ hat nur einen Sohn]

Wir nehmen an, daß φp der linke Sohn seines Vaters ist, und daß der linke Sohn von $\beta \varphi p$ nur einen Sohn hat. Die übrigen, zu diesem Fall symmetrischen Fälle werden analog behandelt.



Fall 3.2.2 [Beide Söhne von $\beta \varphi p$ haben zwei Söhne]

Wir behandeln nur den Fall, daß φp linker Sohn seines Vaters ist, und überlassen den symmetrischen Fall dem Leser.



Man sieht der Prozedur *delete* unmittelbar an, daß sie schlechtestenfalls längs eines Pfades von den Blättern zurück zur Wurzel aufgerufen wird. Damit gilt: Das Entfernen eines Schlüssel x aus einem 1-2-Bruder-Baum mit N Schlüssel ist in $O(\log N)$ Schritten ausführbar.

Wir haben also insgesamt eine weitere Implementationsmöglichkeit für Wörterbücher, die es erlaubt, jede der Operationen Suchen, Einfügen und Entfernen eines Schlüssels auch im schlechtesten Fall in $O(\log N)$ Schritten auszuführen.

Analytische Betrachtungen

1-2-Bruder-Bäume enthalten im allgemeinen unäre Knoten, die keine Schlüssel speichern. Wieviele können das sein? Wir diskutieren diese Frage zunächst im statischen Fall: D.h. wir betrachten einen beliebigen 1-2-Bruder-Baum und setzen nichts über seine Entstehungsgeschichte voraus. Dann untersuchen wir dieselbe Frage im dynamischen Fall: D.h. wir schätzen die Anzahl der unären Knoten in einem 1-2-Bruder-Baum ab, der aus dem anfangs leeren Baum durch eine Folge von N zufälligen Einfügungen entsteht.

Die Analyse des statischen Falls ist einfach. Wir betrachten zwei beliebige benachbarte Niveaus im Baum und sehen, daß nur die Knotenkonfigurationen aus Abbildung 5.29 möglich sind.

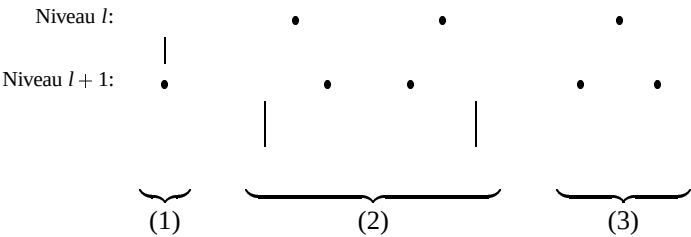


Abbildung 5.29

Für jeden unären Knoten auf Niveau l muß es einen binären Bruder auf demselben Niveau geben. Daher gilt für das Verhältnis

$$U = \frac{\text{Anzahl binäre Knoten auf Niveau } l \text{ und } l+1}{\text{Anzahl Knoten insgesamt auf Niveau } l \text{ und } l+1} :$$

Konfiguration	U
(2)	$\frac{2}{3}$
(3)	$\frac{3}{3}$
(1) und eine Konfig. aus (2)	$\frac{3}{5}$
(1) und (3)	$\frac{4}{5}$

Folglich ist $\frac{3}{5} U \leq 1$.

Was für je zwei beliebige benachbarte Niveaus gilt, muß auch für einen 1-2-Bruder-Baum insgesamt gelten. Damit gilt: Wenigstens $3/5$ der inneren Knoten eines 1-2-Bruder-Baumes müssen binär sein und speichern also einen Schlüssel. Ein 1-2-Bruder-Baum mit N Schlüsseln hat daher höchstens $\frac{5}{3}N$ innere (unäre und binäre) Knoten.

Aus dieser einfachen Beobachtung kann man bereits eine wichtige Folgerung für den über eine Folge iterierter Einfügungen gemittelten *mittleren* Aufwand zum Einfügen eines Schlüssels ziehen. Eine Inspektion der aufwärts umstrukturierenden Prozedur *up* zeigt, daß jeder Aufruf dieser Prozedur zur Schaffung eines oder höchstens zweier Knoten führt. Beim ersten Aufruf wird ein zusätzliches Blatt erzeugt. Jeder weitere Aufruf für einen Knoten, der verschieden von der Wurzel ist, erzeugt genau einen weiteren (unären) Knoten. Ein Aufruf von *up* für die Wurzel erzeugt einen unären und einen binären Knoten. Das sind auch bereits alle Möglichkeiten, wie neue Knoten erzeugt werden können. Sonst werden höchstens vorher unäre Knoten binär gemacht, und die Umstrukturierung mit Hilfe von *up* endet. Fügt man also N Schlüssel in den anfangs leeren Baum ein, so kann man aus der insgesamt erzeugten Knotenzahl auf die insgesamt ausgeführten Aufrufe von *up* schließen. Da höchstens $\frac{5}{3}N$ innere Knoten und ebensoviele Blätter insgesamt erzeugt worden sind, ist die durchschnittliche Anzahl der Aufrufe von *up* pro Einfügung konstant. Zählt man den Suchaufwand zum Finden der jeweiligen Einfügestelle nicht mit, so folgt: Der durchschnittliche Aufwand zum Einfügen eines Schlüssels in einen 1-2-Bruder-Baum ist konstant, wenn man den Durchschnitt über eine Folge von Einfügungen in den anfangs leeren Baum nimmt.

Eine entsprechende Aussage ist für AVL-Bäume übrigens bei weitem nicht so leicht herzuleiten. Denn es ist zwar richtig, daß für einen AVL-Baum nach dem Einfügen eines neuen Schlüssels höchstens eine einzige Rotation oder Doppelrotation ausgeführt werden muß; zu den Umstrukturierungen muß man aber auch das Adjustieren der Balancefaktoren hinzurechnen, das an jedem Knoten längs des Suchpfades erforderlich sein kann.

Wir kommen jetzt zum dynamischen Fall und wollen den Erwartungswert für die Anzahl der unären und binären Knoten ausrechnen, wenn man eine zufällig gewählte Folge von N Schlüsseln in den anfangs leeren 1-2-Bruder-Baum einfügt. Genau werden wir diese Werte nur für den *Rand* (englisch: *fringe*), d h. für die Knoten auf den blattnahen Niveaus ausrechnen. Die dafür von A. Yao [Yao77] entwickelte Methode heißt daher auch *Fringe-Analyse*. Sie ist nicht nur auf 1-2-Bruder-Bäume, sondern auch auf viele andere Baumklassen anwendbar.

Wir begnügen uns damit, die Anzahl der binären Knoten auf den zwei untersten, den Blättern nächsten Niveaus innerer Knoten zu berechnen, für einen 1-2-Bruder-Baum, der durch eine Folge von N zufälligen Einfügungen in den anfangs leeren 1-2-Bruder-Baum entsteht. Dazu schauen wir uns zunächst einmal an, welche Teilbäume mit niedriger Höhe 1 oder 2 am Rand eines 1-2-Bruder-Baumes auftreten können. Es gibt offenbar die in Abbildung 5.30 dargestellten Möglichkeiten.

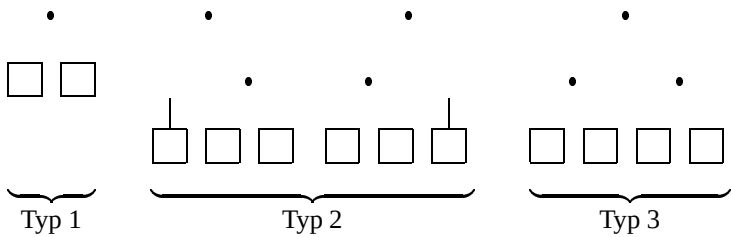


Abbildung 5.30

Sei T ein 1-2-Bruder-Baum. Wir sagen: T gehört zur Klasse (x_1, x_2, x_3) , wenn T x_i Teilbäume vom Typ i hat ($1 \leq i \leq 3$). Dabei darf kein Teilbaum doppelt gezählt werden, d h. die Anzahl der Blätter von T muß gleich $2x_1 + 3x_2 + 4x_3$ sein. Derselbe 1-2-Bruder-Baum kann aber durchaus zu mehreren Klassen gehören.

Sei nun ein 1-2-Bruder-Baum mit $N - 1$ Schlüsseln und N Blättern gegeben. Dann sagen wir: Das Einfügen des N -ten Schlüssels x erfolgt zufällig, wenn die Wahrscheinlichkeit dafür, daß x in eines der durch die bereits vorhandenen Schlüssel bestimmten N Schlüsselintervalle fällt, für jedes dieser Intervalle gleich groß ist, nämlich $1/N$. Die Wahrscheinlichkeit dafür, daß x in einen Teilbaum vom Typ i fällt, ist damit gleich dem Anteil, den die Blätter von Teilbäumen vom Typ i zur gesamten Blattzahl beisteuern; sie ist also $(i + 1) \frac{x_i}{N}$ für jedes $i, 1 \leq i \leq 3$.

Beispiel: Der 1-2-Bruder-Baum aus Abbildung 5.31 gehört zur Klasse $(2, 1, 0)$ und $(0, 1, 1)$.

Sei nun $A_i(N)$ der Erwartungswert für die Anzahl von Teilbäumen des Typs i nach N zufälligen Einfügungen in den anfangs leeren Baum. Für kleine Werte von N kann man $A_i(N)$ leicht explizit ausrechnen, weil es nicht schwer ist, sich eine vollständige Übersicht über alle durch iteriertes Einfügen entstehenden 1-2-Bruder-Bäume zu verschaffen. Beispielsweise entsteht nach vier Einfügungen stets, d h. mit Wahrscheinlich-

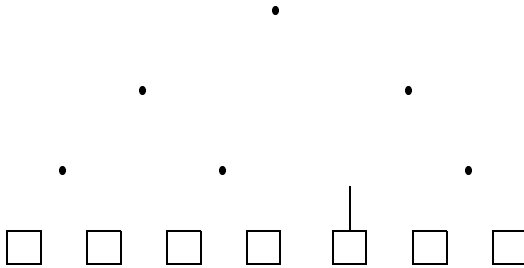


Abbildung 5.31

keit 1, der Baum in Abbildung 5.32. Tabelle 5.2 enthält mögliche Werte von $A_i(N)$ für $N = 1, \dots, 6$.

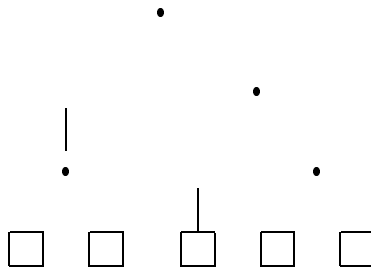


Abbildung 5.32

Zur Berechnung von $A_i(N)$ für beliebige N benutzen wir die folgenden Hilfssätze:

Lemma 5.1 *Sei T ein 1-2-Bruder-Baum. Wird ein neuer Schlüssel in einen Teilbaum des Typs 1 (bzw. des Typs 2) von T eingefügt, so erhöht sich die Zahl der Teilbäume vom Typ 2 (bzw. 3) um 1 und die Zahl der Teilbäume vom Typ 1 (bzw. 2) erniedrigt sich um 1.*

Beweis: Wir beschränken uns auf die erste Aussage: Die Wurzel eines Teilbaumes vom Typ 1 ist entweder einziger Sohn eines unären Vaters oder hat einen binären Bruder. Damit folgt die Behauptung aus der Definition des Einfügeverfahrens. $\square$

Genauso einfach zeigt man:

Lemma 5.2 *Sei T ein 1-2-Bruder-Baum. Wird ein neuer Schlüssel in einen Teilbaum vom Typ 3 von T eingefügt, so erhöht sich die Zahl der Teilbäume vom Typ 1 und 2 jeweils um 1 und die Zahl der Teilbäume vom Typ 3 erniedrigt sich um 1.*

N	$A_1(N)$	$A_2(N)$	$A_3(N)$
1	1	0	0
2	0	1	0
3	0	0	1
4	1	1	0
5	$\frac{3}{5}$	$\frac{4}{5}$	$\frac{2}{5}$
6	$\left\{ \begin{matrix} 0 \\ \frac{4}{5} \end{matrix} \right.$	$\left\{ \begin{matrix} 1 \\ 1 \end{matrix} \right.$	$\left\{ \begin{matrix} 1 \\ \frac{3}{5} \end{matrix} \right.$

Tabelle 5.2

Ist also T ein 1-2-Bruder-Baum mit $N - 1$ Schlüsseln der Klasse (x_1, x_2, x_3) , so wird aus T mit Wahrscheinlichkeit p ein Teilbaum der Klasse (x'_1, x'_2, x'_3) mit folgenden Werten für x'_i und p :

x'_1	x'_2	x'_3	p	} $\Sigma = 1$
$x_1 - 1$	$x_2 + 1$	x_3	$2 \frac{x_1}{N}$	
x_1	$x_2 - 1$	$x_3 + 1$	$3 \frac{x_2}{N}$	
$x_1 + 1$	$x_2 + 1$	$x_3 - 1$	$4 \frac{x_3}{N}$	

$A_1(N - 1)$ nimmt also mit Wahrscheinlichkeit $2 \frac{A_1(N-1)}{N}$ um 1 ab und nimmt mit Wahrscheinlichkeit $4 \frac{A_3(N-1)}{N}$ um 1 zu, d h. es gilt:

$$A_1(N) = A_1(N - 1) - \frac{2}{N}A_1(N - 1) + \frac{4}{N}A_3(N - 1)$$

Analog gilt:

$$\begin{aligned} A_2(N) &= A_2(N - 1) - \frac{3}{N}A_2(N - 1) + (1 - \frac{3}{N})A_2(N - 1) \\ &= (1 - \frac{6}{N})A_2(N - 1) + 1 \\ A_3(N) &= A_3(N - 1) + \frac{3}{N}A_2(N - 1) - \frac{4}{N}A_3(N - 1) \\ &= (1 - \frac{4}{N})A_3(N - 1) + \frac{3}{N}A_2(N - 1) \end{aligned}$$

Durch vollständige Induktion zeigt man leicht, daß dieses System von Rekursionsgleichungen mit den oben angegebenen Anfangsbedingungen folgende Lösung hat:

$$\left. \begin{aligned} A_1(N) &= \frac{4}{7.5}(N+1) \\ A_2(N) &= \frac{1}{7}(N+1) \\ A_3(N) &= \frac{3}{7.5}(N+1) \end{aligned} \right\} \quad \text{für } N \geq 6.$$

Wir nennen einen 1-2-Bruder-Baum *zufällig*, wenn er durch eine Folge zufälliger Einfügungen in den anfangs leeren Baum entsteht.

Als *untere Schranke* für die Anzahl der Schlüssel auf den zwei untersten Niveaus innerer Knoten in zufälligen 1-2-Bruder-Bäumen mit N Schlüsseln erhalten wir:

$$1 \cdot A_1(N) + 2 \cdot A_2(N) + 3 \cdot A_3(N) = \frac{23}{35}(N+1) = 0.657 \dots (N+1)$$

Da ungünstigstenfalls jeder Typ-1-Teilbaum einen unären Vater hat, erhalten wir als *obere Schranke* für die Gesamtzahl der inneren Knoten auf den zwei untersten Niveaus:

$$2A_1(N) + 3(A_2(N) + A_3(N)) = \frac{32}{35}(N+1)$$

Für die zwei untersten Niveaus eines zufällig erzeugten 1-2-Bruder-Baumes ist also das Verhältnis der Anzahl der binären Knoten zur Gesamtzahl der Knoten auf diesen Niveaus wenigstens $\frac{23}{32} = 0.71875$. Wir können demnach erwarten, daß wenigstens 23 von 32 Knoten binär sind und nicht nur 3 von 5, wie unsere statische Abschätzung ergeben hat.

Eine genauere Abschätzung für das Verhältnis der Zahl der binären zur Gesamtzahl von Knoten auf den zwei untersten Niveaus ist nur eine mögliche Folgerung, die man aus der Berechnung der Erwartungswerte $A_i(N)$ für die Anzahl der Teilbäume vom Typ i in einem zufällig erzeugten 1-2-Bruder-Baum ziehen kann.

Da in einem Binärbaum etwa die Hälfte der inneren Knoten unmittelbar oberhalb der Blätter vorkommt, kann man über die Erwartungswerte für die Anzahl der binären und unären Knoten auf den zwei untersten Niveaus auch bessere Schranken für die entsprechenden Anzahlen im gesamten Baum erhalten. Man schätzt diese Zahl auf den untersten Niveaus wie oben angegeben ab und benutzt oberhalb die aus der statischen Betrachtung gewonnene Abschätzung.

Weiter liefern die Erwartungswerte $A_i(N)$, für $i = 1, 2, 3$, auch eine Aussage darüber, wie groß die Wahrscheinlichkeit dafür wenigstens ist, daß eine weitere Einfügung in einen zufällig erzeugten 1-2-Bruder-Baum zu höchstens einem bzw. mindestens zwei (rekursiven) Aufrufen der Prozedur *up* führt. Fällt nämlich der nächste einzufügende Schlüssel in einen Teilbaum des Typs 2, so wird *up* genau einmal, fällt sie in einen Teilbaum des Typs 3, so wird *up* wenigstens zweimal aufgerufen.

Das sind einige Beispiele für Aussagen, die mit Hilfe der Fringe-Analyse-Methode hergeleitet werden können. Die Methode führt im allgemeinen nicht zu so einfach elementar lösbaren Rekursionsgleichungen wie für die Erwartungswerte $A_i(N)$ im Falle von 1-2-Bruder-Bäumen. Man muß vielmehr im allgemeinen stärkere mathematische Hilfsmittel heranziehen, um die Erwartungswerte für Teilbäume, die im Rand zufällig erzeugter Bäume auftreten, zu berechnen. Das ist z.B. erforderlich, wenn man die im Abschnitt 5.5 behandelten B-Bäume mit dieser Methode analysiert.

5.2.3 Gewichtsbalancierte Bäume

Balancierte Binärbäume sind ganz grob dadurch charakterisiert, daß für jeden Knoten p der linke und rechte Teilbaum von p nicht zu unterschiedliche Größe haben dürfen. Die Größe kann dabei, wie im Falle der AVL-Bäume, durch die Höhe oder — und das ist der in diesem Abschnitt diskutierte Fall — über die Anzahl der Knoten bzw. Blätter bestimmt sein. Bei gewichtsbalancierten Bäumen wird gefordert, daß die Anzahl der Knoten bzw. Blätter im linken und rechten Teilbaum eines jeden Knotens sich nicht zu stark unterscheiden dürfen [1, 2]. Wir wissen bereits, daß für jeden Binärbaum die Anzahl der Blätter stets um 1 größer ist als die Anzahl der binären inneren Knoten.

Wir wollen für einen Knoten p eines Binärbaumes, der Wurzel eines Teilbaumes T_p ist, mit $W(p)$ und $W(T_p)$ die Anzahl der Blätter des Teilbaumes T_p bezeichnen; $W(p)$ und $W(T_p)$ nennt man üblicherweise auch das Gewicht (englisch: weight) von p bzw. von T_p .

Ist T ein Baum mit $W(T)$ Blättern, dessen linker Teilbaum T_l $W(T_l)$ Blätter hat, so nennt man den Quotienten

$$\rho(T) = \frac{W(T_l)}{W(T)}$$

die *Wurzelbalance* von T . Man fordert nun, daß die Wurzelbalance für jeden Teilbaum innerhalb bestimmter Grenzen liegen muß. Ist α eine Zahl mit $0 < \alpha \leq \frac{1}{2}$, so heißt ein binärer Suchbaum T von *beschränkter Balance* α oder *gewichtsbalanciert mit Balance* α oder kurz ein $\text{BB}[\alpha]$ -Baum, wenn für jeden Teilbaum T' von T gilt:

$$\alpha \leq \rho(T') \leq 1 - \alpha \quad (*)$$

Durch diese Forderung ist natürlich nicht nur das Verhältnis der Knotenzahlen im linken Teilbaum eines jeden Knotens zur gesamten Knotenzahl im Teilbaum dieses Knotens festgelegt. Denn ist p ein Knoten mit linkem Sohn p_l und rechtem Sohn p_r , so ist natürlich

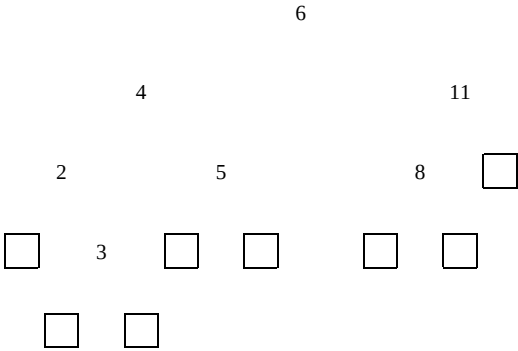
$$W(p_r) = W(p) - W(p_l)$$

und daher gilt mit (*) nicht nur, daß für jeden Knoten p eines $\text{BB}[\alpha]$ -Baumes

$$\alpha \leq \frac{W(p_l)}{W(p)} \leq 1 - \alpha$$

ist, sondern auch

$$\alpha \leq 1 - \frac{W(p_r)}{W(p)} \leq 1 - \alpha. \quad (**)$$



Wurzelbalancen:	
Knoten mit	Balance
Schlüssel	
6	$\frac{5}{8}$
4	$\frac{3}{5}$
11	$\frac{2}{3}$
2	$\frac{1}{3}$
5	$\frac{1}{2}$
3	$\frac{1}{2}$
8	$\frac{1}{2}$

Abbildung 5.33

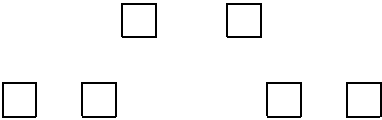


Abbildung 5.34

Als Beispiel betrachte man Abbildung 5.33. Offenbar gilt für $\alpha = \frac{1}{4}$, daß alle Wurzelbalancen zwischen $\frac{1}{4}$ und $\frac{3}{4}$ liegen. Der Baum ist damit ein $BB[\frac{1}{4}]$ -Baum.

Über den Parameter α läßt sich die Güte der Ausgeglichenheit steuern. Je näher α bei 0 liegt, um so weniger restriktiv ist die Forderung der Gewichtsbalanciertheit; je näher α bei $\frac{1}{2}$ liegt, um so besser ausgeglichen müssen die Bäume in $BB[\alpha]$ sein. Man kann aber α nicht gleich $\frac{1}{2}$ setzen oder auch nur beliebig nahe an den Wert $\frac{1}{2}$ herankommen lassen, weil dann die Forderung () so restriktiv ist, daß nicht mehr für jede Knotenzahl N ein Baum existiert, der in $BB[\alpha]$ liegt. So gibt es beispielsweise nur zwei Suchbäume mit zwei inneren Knoten, wie in Abbildung 5.34 dargestellt wird. Die Wurzelbalance des linken Baumes ist $\frac{2}{3}$ und die des rechten ist $\frac{1}{3}$. Beide Bäume liegen in $BB[\frac{1}{3}]$, aber $BB[\frac{1}{2}]$ enthält keinen Baum mit 2 inneren Knoten.

Wir setzen im folgenden voraus, daß α stets so gewählt ist, daß in $BB[\alpha]$ wenigstens ein Baum mit N Knoten für jedes N liegt. (Wählt man $\alpha \in [\frac{1}{4}, 1 - \frac{\sqrt{2}}{2}]$, so gilt die Bedingung; vgl. hierzu [] oder [].)

Der Aufwand zur Ausführung der für Suchbäume typischen Operationen Suchen, Einfügen und Entfernen hängt unmittelbar von der Höhe der jeweils betrachteten Bäume ab. Wir wollen uns daher zunächst überlegen, daß die über die Knotengewichte definierte Balancebedingung impliziert, daß gewichtsbalancierte Bäume eine Höhe haben, die logarithmisch von der Anzahl der Knoten abhängt. Gewichtsbalancierte Bäume sind dadurch charakterisiert, daß man beim Hinabsteigen von einem Knoten p zu einem seiner Söhne stets einen Mindestbruchteil der Blätter verliert, der durch den Balancefaktor α bestimmt ist. Genauer: Ist p ein Knoten mit linkem Sohn p_l und rechtem Sohn p_r , so folgt aus () (und ()):

- (i) $W(p_l) \geq (1 - \alpha)W(p)$
- (ii) $W(p_r) \geq (1 - \alpha)W(p)$

Bemerkung: Eine analoge Bedingung gilt weder für höhenbalancierte Bäume noch für Bruder-Bäume. Wenn man beispielsweise einen Bruder-Baum T betrachtet, dessen Wurzel als linken Teilbaum T_l einen "Fibonacci-Baum" mit Höhe h und F_{h+1} Blättern hat und als rechten Teilbaum T_r einen vollständigen Binärbaum mit derselben Höhe, so gilt:

$$W(T_l) = F_{h+1} = c \cdot 1.618 \dots^h$$

mit einer Konstanten c und

$$W(T) = c \cdot 1.618 \dots^h + 2^h.$$

Nehmen wir nun an, es gibt ein α , $0 < \alpha < 1$, so daß $W(T_r) \geq (1 - \alpha)W(T)$. Dann folgt aus (ii)

$$2^h \geq (1 - \alpha)(c \cdot 1.618 \dots^h + 2^h)$$

und damit

$$\frac{1}{1 - \alpha} \geq (1 + c \cdot (\frac{1.618}{2})^h).$$

Weil $\alpha < 1$ ist, muß $1/(1 - \alpha) > 1$ sein. Man erhält also einen Widerspruch, da $(1.618 / 2)^h$ mit wachsendem h gegen 0 geht. □

Sei nun ein gewichtsbalancierter Baum T aus $BB[\alpha]$ mit Höhe h gegeben. Wir betrachten einen Pfad maximaler Länge von der Wurzel zu einem Blatt. Seien $p_1, p_2, \dots, p_h$ die (inneren) Knoten auf diesem Pfad. Der Knoten p_1 ist also die Wurzel und p_h ist ein Knoten, dessen beide Söhne Blätter sind. Daher ist

$$W(T) = W(p_1) \quad \text{und} \quad W(p_h) = 2.$$

Wegen (i) und (ii) gilt:

$$\begin{aligned} W(p_2) &= (1 - \alpha)W(p_1) \\ W(p_3) &= (1 - \alpha)W(p_2) \\ &\vdots \\ W(p_h) &= (1 - \alpha)W(p_{h-1}) \end{aligned}$$

Also

$$2 - (1 - \alpha)^{h-1} W(p_1) = (1 - \alpha)^{h-1} N,$$

wenn $N = W(p_1)$ die Anzahl der Blätter des Baumes T bezeichnet. Durch Logarithmieren dieser Ungleichung erhält man

$$1 - (h - 1) \log_2(1 - \alpha) + \log_2 N,$$

also

$$h - 1 - \frac{\log_2 N - 1}{\log_2(1 - \alpha)} = O(\log N).$$

Die Höhe h eines Baumes aus $BB[\alpha]$ ist also logarithmisch in der Anzahl der Blätter oder Knoten beschränkt.

Suchen, Einfügen und Entfernen von Schlüsseln

Da gewichtsbalancierte Bäume insbesondere binäre Suchbäume sind, kann man in ihnen genauso suchen wie in natürlichen Bäumen. Weil die Höhe eines Baumes aus $BB[\alpha]$ mit N Knoten von der Größenordnung $O(\log N)$ ist, kann man die Operation *Suchen* ebenfalls stets in $O(\log N)$ Schritten ausführen.

Um einen Schlüssel in einen Baum T aus $BB[\alpha]$ *einzufügen*, sucht man zunächst nach dem einzufügenden Schlüssel im Baum. Wenn der Schlüssel in T noch nicht vorkommt, endet die Suche erfolglos in einem Blatt, das die erwartete Position des einzufügenden Schlüssel repräsentiert. Wie bei natürlichen Bäumen ersetzt man dieses Blatt durch einen inneren Knoten, der den neu einzufügenden Schlüssel aufnimmt, und gibt ihm zwei Blätter als Söhne. Der resultierende Baum ist damit zwar wieder ein Suchbaum, aber möglicherweise kein gewichtsbalancierter Baum aus $BB[\alpha]$ mehr. Denn man hat ja durch Schaffen eines weiteren inneren Knotens und eines neuen Blattes die Gewichte aller Knoten auf dem Pfad von der Wurzel zur Einfügestelle verändert. Beim *Entfernen* eines Schlüssels tritt eine ähnliche Situation ein. Man entfernt einen Schlüssel zunächst genauso, wie man es von natürlichen Bäumen kennt. Man reduziert das Entfernen also gegebenenfalls auf das Entfernen des symmetrischen Nachfolgers oder Vorgängers eines Knotens und kann daher ohne Einschränkung annehmen, daß man den Schlüssel

eines Knotens entfernt, dessen beide Söhne Blätter sind. Ersetzt man nun diesen Knoten durch ein Blatt, so haben sich wieder die Gewichte aller Knoten auf dem Pfad von der Wurzel bis zur Entfernestelle verändert. Man muß also unter Umständen den Baum umstrukturieren, um wieder einen $BB[\alpha]$ -Baum zu erhalten. Dazu geht man ähnlich vor wie bei AVL-Bäumen. Man läuft den Suchpfad zurück und prüft an jedem Knoten, ob die Wurzelbalance an diesem Knoten noch im Bereich $[\alpha, 1 - \alpha]$ liegt. Ist das nicht der Fall, führt man eine Rotation oder Doppelrotation durch, um die Wurzelbalance an dieser Stelle wieder in den vorgeschriebenen Bereich zurückzubringen.

Hier stellt sich natürlich zunächst die Frage, wie man denn überhaupt erkennen kann, ob an einem bestimmten Knoten die Wurzelbalance noch im vorgeschriebenen Bereich liegt. Darüber hinaus muß man natürlich zeigen, daß Rotationen und Doppelrotationen wirklich geeignete Maßnahmen sind, um die Wurzelbalance an einem bestimmten Knoten in den verlangten Bereich zurückzuführen.

Wir führen an jedem Knoten dessen *Gewicht* (weight) als zusätzliches Attribut mit. Die Knotengewichte kann man bei jeder Einfüge- und Entferne-Operation leicht ändern; notwendige Änderungen bleiben auf den Suchpfad beschränkt. Aus den Gewichten kann man die benötigten Wurzelbalancen leicht berechnen. Das Knotenformat von $BB[\alpha]$ -Bäumen kann man in Pascal etwa wie folgt vereinbaren:

```

type
  Knotenzeiger = Knoten;
  Knoten = record
    key : integer;
    leftson, rightson : Knotenzeiger;
    weight : integer;
    info : infotype
  end

```

Gegenüber AVL-Bäumen und Bruder-Bäumen muß man also im Falle gewichtsbalanzierter Bäume an jedem Knoten eine im Prinzip unbeschränkt große Information mitführen, die zur Überprüfung und Sicherung der Ausgeglichenheit herangezogen wird. Das ist natürlich ein Nachteil, wenn es auf eine besonders Speicherplatz sparende Implementation einer Klasse balancierter Bäume ankommt.

Nach dem Einfügen oder Entfernen eines Schlüssels läuft man nun auf dem Suchpfad zur Wurzel zurück und überprüft an jedem Knoten die Wurzelbalance des zugehörigen Teilbaumes.

Liegt die Wurzelbalance $\rho(T_p)$ des Teilbaumes mit Wurzel p außerhalb des Bereiches $[\alpha, 1 - \alpha]$, sind zwei Fälle möglich:

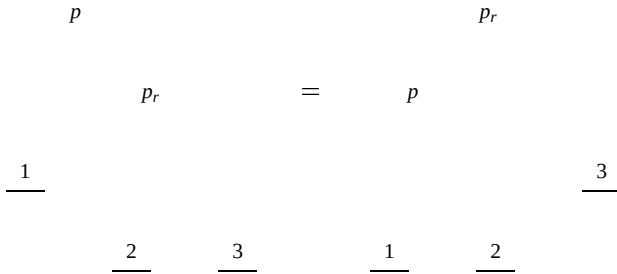
Fall 1: $\rho(T_p) < \alpha$

Fall 2: $\rho(T_p) > (1 - \alpha)$

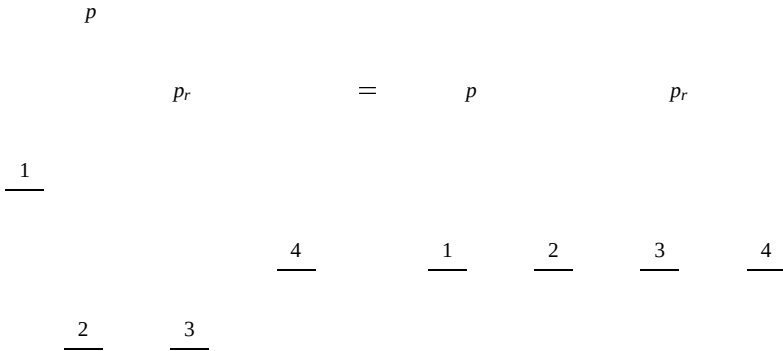
Betrachten wir zunächst den Fall 1 etwas genauer. Die Bedingung $\rho(T_p) < \alpha$ bedeutet, daß der rechte Teilbaum gegenüber dem linken zu schwer geworden ist, und zwar entweder, weil im rechten Teilbaum ein Knoten (und ein Blatt) eingefügt wurde oder weil im linken Teilbaum ein Knoten entfernt wurde. Um die Wurzelbalance bei p wieder in den Bereich $[\alpha, 1 - \alpha]$ zurückzubringen, müssen wir den rechten Sohn p_r von p leichter machen. Wie im Falle von AVL-Bäumen versuchen wir das mit Hilfe

einer Rotation nach links oder einer Doppelrotation rechts-links. Welche dieser Operationen gewählt werden muß, hängt ab vom Balancefaktor α und vom Wert der Wurzelbalance von p_r . Man kann zeigen (vgl. z.B. [5]), daß es eine von α abhängige Zahl $d \in [\alpha, 1 - \alpha]$ gibt, derart, daß eine Umstrukturierung entsprechend der folgenden Fallunterscheidung auf jeden Fall die Wurzelbalance in den Bereich $[\alpha, 1 - \alpha]$ zurückführt, wenn α im Bereich $[\frac{1}{4}, 1 - \frac{\sqrt{2}}{2}]$ liegt.

Fall 1.1 $[\rho(T_{p_r}) \leq d]$ Ausgleichen durch einfache Rotation nach links



Fall 1.2 $[\rho(T_{p_r}) > d]$ Ausgleichen durch Doppelrotation rechts-links



Wir betrachten als Beispiel den Baum mit den vier Schlüsseln $2, 5, 6, 8$ aus $BB[\frac{2}{7}]$ in Abbildung 5.35.

Eine Überprüfung der Wurzelbalancen nach Einfügen des Schlüssels 9 zeigt, daß die Wurzelbalance beim Knoten p nicht mehr im vorgeschriebenen Bereich $[\frac{2}{7}, \frac{5}{7}]$ liegt. Eine Rotation bei p genügt, um beim Knoten p die Wurzelbalance in den vorgeschriebenen Bereich zurückzuführen.

Fügen wir in den Baum aus $BB[\frac{1}{4}]$ in Abbildung 5.36 den Schlüssel 2 ein, so genügt eine einfache Rotation nach links an der Wurzel des neuen Baumes nicht mehr, um die Wurzelbalance dort in den Bereich $[\frac{1}{4}, \frac{3}{4}]$ zurückzuführen. Eine Doppelrotation leistet dies aber.

Bisher haben wir nur den Fall 1 betrachtet; er kann eintreten, wenn ein Knoten auf der rechten Seite von p eingefügt oder auf der linken Seite von p entfernt wurde. Der Fall 2, $\rho(T_p) > (1 - \alpha)$, kann eintreten, wenn in einem zuvor ausgeglichenen Baum entweder links ein Knoten eingefügt oder rechts einer entfernt wurde. Dann wird in Abhängigkeit

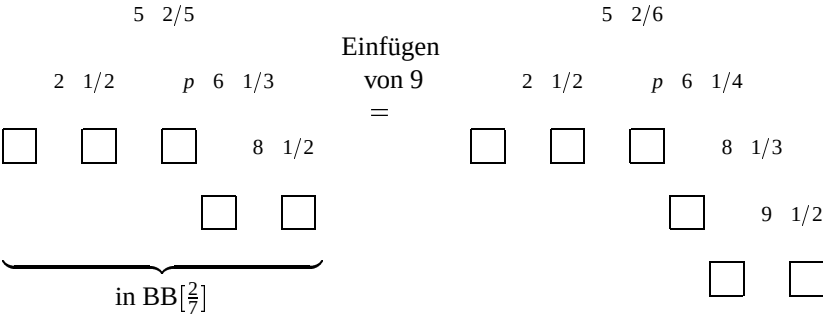


Abbildung 5.35

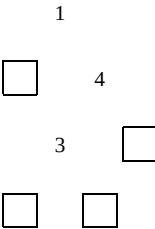


Abbildung 5.36

von einem geeignet gewählten Wert $d \in [\alpha, 1 - \alpha]$ eine Rotation nach rechts oder eine Doppelrotation links-rechts ausgeführt, die dafür sorgt, daß die Wurzelbalance bei p in den vorgeschriebenen Bereich zurückkehrt.

Der Nachweis, daß nach einer Rotation oder Doppelrotation die Wurzelbalance bei einem Knoten p wieder im vorgeschriebenen Bereich liegt, ist technisch umständlich, aber nicht schwierig. Er verläuft im Prinzip so: Man berechnet die Wurzelbalancen der transformierten Bäume aus den ursprünglichen Wurzelbalancen. Weil man weiß, daß die ursprünglichen Wurzelbalancen im Bereich $[\alpha, 1 - \alpha]$ lagen, erhält man automatisch Schranken für die Wurzelbalancen der transformierten Bäume; man muß sich dann nur noch davon überzeugen, daß die letzteren im vorgeschriebenen Bereich liegen. Dieser Nachweis gelingt allerdings nur, wenn $\alpha \in [\frac{1}{4}, 1 - \frac{\sqrt{2}}{2}]$ ist.

Wir verzichten auf die Ausführung der Details und fassen nur das Ergebnis noch einmal zusammen. Gewichtsbalancierte Bäume sind eine Möglichkeit zur Implementierung von Wörterbüchern, die es erlaubt, jede der Operationen Suchen, Einfügen und Entfernen von Schlüsseln auch im schlechtesten Fall in $O(\log N)$ Schritten auszuführen.

Die über eine Anzahl iterierter Einfüge- und Entferne-Operationen gemittelte Anzahl von Rotationen und Doppelrotationen, die erforderlich ist, um stets Bäume in $BB[\alpha]$ zu erhalten, ist konstant, obwohl im schlechtesten Fall eine einzelne Einfüge- oder Entferne-Operation durchaus längs sämtlicher Knoten des Suchpfades, also $\Omega(h)$, $h =$

Höhe des Baumes, viele Rotationen und Doppelrotationen auslösen kann. Auch dieses Ergebnis wollen wir hier nicht beweisen, sondern verweisen dazu auf [1].

5.3 Randomisierte Suchbäume

Fügt man N Schlüssel der Reihe nach in einen anfangs leeren binären Suchbaum ein, so kann, wie wir in Abschnitt 5.1 gesehen haben, ein natürlicher Suchbaum entstehen, dessen durchschnittliche Suchpfadlänge von der Größenordnung $N/2$ ist. Glücklicherweise treten solche zu linearen Listen „degenerierten“ binären Suchbäume unter den $N!$ möglichen Anordnungen von N Schlüsseln entsprechenden Suchbäumen nicht allzu häufig auf. Daher sind die Erwartungswerte für die durchschnittliche Suchpfadlänge und die Kosten zur Ausführung einer Einfüge- oder Entferne-Operation für einen zufällig erzeugten binären Suchbaum mit N Schlüsseln nur von der Größenordnung $O(\log N)$.

Wir wollen in diesem Abschnitt zeigen, wie eine einfache Randomisierungsstrategie helfen kann, „schlechte“ Eingabefolgen zu vermeiden. Durch geeignete Randomisierung der Verfahren zum Einfügen und Entfernen von Schlüsseln analog zu randomisiertem Quicksort, vgl. Abschnitt 2.2.2, wird gesichert, daß unabhängig von der Einfügereihenfolge für jede Menge von N Schlüsseln gilt: Der Erwartungswert für die Kosten einer einzelnen Such-, Einfüge- oder Entferne-Operation in einem randomisierten Suchbaum mit N Schlüsseln ist von der Größenordnung $O(\log N)$. Das wird auf folgende Weise erreicht: Man ordnet jedem Schlüssel eine zufällig gewählte „Zeitmarke“ als Priorität zu. Die Einfüge- und Entferne-Verfahren werden dann so verändert, daß gilt: Unabhängig von der tatsächlichen Reihenfolge, in der die Update-Operationen ausgeführt werden, die eine aktuelle Schlüsselmenge S liefern, wird immer derjenige natürliche Suchbaum zur Speicherung von S erzeugt, der entstanden wäre, wenn man die Elemente von S in der durch ihre Prioritäten gegebenen zeitlichen Reihenfolge in den anfangs leeren Baum der Reihe nach eingefügt hätte. Wir beschreiben nun diese Idee im folgenden genauer und analysieren die Verfahren anschließend.

Randomisierte Suchbäume wurden von Aragon und Seidel [2] erfunden. Unsere Analyse folgt der vereinfachten Darstellung in [3].

5.3.1 Treaps

Gegeben sei eine Menge S von Objekten mit der Eigenschaft, daß jedes Element $x \in S$ zwei Komponenten hat, eine Schlüsselkomponente $x.key$ und eine Prioritätskomponente $x.priority$. Wir nehmen an, daß die Schlüsselkomponenten einem vollständig geordneten Universum entstammen, also ohne Einschränkung ganzzahlig sind. Die Prioritäten sollen einem davon möglicherweise verschiedenen, ebenfalls vollständig geordneten Universum entstammen. Ein *Treap* zur Speicherung von S ist ein binärer Suchbaum für die Schlüsselkomponenten und ein Min-heap für die Prioritäten der Elemente von S . Ein Treap ist also eine Hybridstruktur, die die Eigenschaften von binären Suchbäumen

(trees) und Vorrangwarteschlangen (heaps), vgl. Abschnitt 2.3 und 6.1, miteinander verbindet. Im Abschnitt 7.4.4 werden wir eine Variante dieser Struktur zur Speicherung von Punkten in der Ebene diskutieren, die von McCreight [McCr83] vorgeschlagen und Prioritäts-Suchbaum genannt wurde.

Genauer gilt für jeden Knoten p eines Treaps: Speichert p das Element x , so gelten für p die folgende Suchbaum- und Heapbedingung.

Suchbaumbedingung: Für jedes Element y im linken Teilbaum von p ist $y.key < x.key$ und für jedes Element y im rechten Teilbaum von p ist $x.key < y.key$.

Heapbedingung: Für jedes in einem Sohn von p gespeicherte Element z gilt $x.priority > z.priority$.

Beispiel: Abbildung 5.37 zeigt einen Treap, der die Elemente der Menge $S = \{(1, 4), (2, 1), (3, 8), (4, 5), (5, 7), (6, 6), (8, 2), (9, 3)\}$ speichert. Dabei soll die erste Zahl jeweils den Schlüssel und die zweite die Priorität bezeichnen.

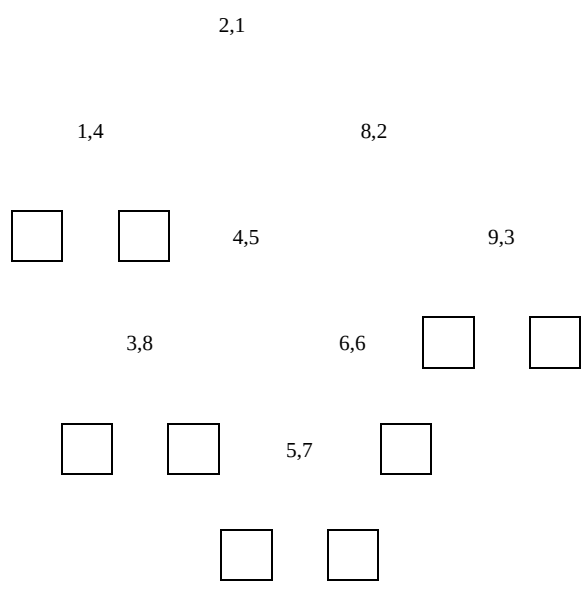


Abbildung 5.37

Wir überlegen uns zunächst, daß es für jede Menge S von Elementen mit paarweise verschiedenen Schlüsseln und Prioritäten genau einen Treap gibt, der S speichert. Ist nämlich x das eindeutig bestimmte Element von S mit minimaler Priorität, so muß x an der Wurzel des Treap gespeichert werden. Teilt man nun die restlichen Elemente von S in die zwei Mengen $S_1 = \{y \mid y.key < x.key\}$ und $S_2 = \{y \mid y.key > x.key\}$, so müssen auf

dieselbe Weise konstruierte Treaps jeweils linke und rechte Teil-Treaps der Wurzel (mit Element x) werden. Die Eindeutigkeit des S speichernden Treap folgt damit induktiv.

Suchen und Einfügen in Treaps

Sei nun ein Treap gegeben, der eine Menge S von Elementen speichert. Die *Suche* nach einem Element x kann wie bei normalen binären Suchbäumen nur unter Benutzung der Schlüsselkomponenten durchgeführt werden. Wie kann man ein neues Element x mit neuer Schlüssel- und Prioritätskomponente in einen Treap *einfügen*? Dazu geht man wie folgt vor: Zunächst wird das Blatt, bei dem die Suche nach $x.key$ (erfolglos) endet, durch einen inneren Knoten ersetzt, der x speichert. Der resultierende Baum ist ein Suchbaum für die Schlüsselkomponenten, aber im allgemeinen kein Treap, weil die Heapbedingung für die Prioritäten möglicherweise nicht gilt. Denn $x.priority$ kann kleiner sein als die Priorität des beim Vater von x gespeicherten Elements. Die uns schon bekannten Rotationsoperationen zur lokalen Umstrukturierung von binären Suchbäumen können dazu benutzt werden, die Heapbedingung wieder herzustellen. Abbildung 5.38 zeigt diese Operationen. Offenbar kann man durch Ausführen einer Rotation (nach links oder rechts) ein Element um ein Niveau heraufbewegen; gleichzeitig wird dadurch ein anderes herabbewegt. Dabei bleibt die Suchbaumstruktur erhalten.

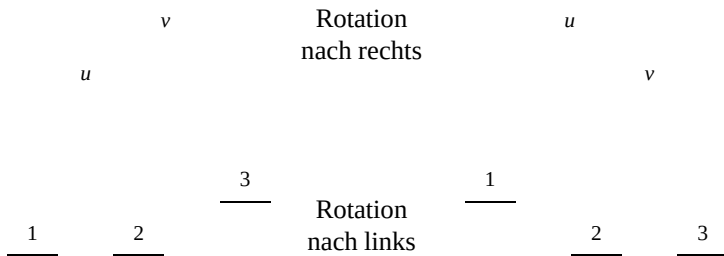


Abbildung 5.38

Falls also die Heapbedingung für x nicht gilt, wird x durch Rotationen nach links oder rechts solange nach oben bewegt, bis die Heapbedingung wieder gilt oder x bei der Wurzel angekommen ist. Abbildung 5.39 zeigt die zur Wiederherstellung der Heapbedingung nach Einfügen des Elements $(7, 0)$ in den Treap von Abbildung 5.37 erforderlichen Schritte. Darin sind die zwei Knoten, für die eine Rotation nach links oder rechts durchgeführt wird, jeweils durch einen „ “ gekennzeichnet.

Entfernen von Elementen aus Treaps

Zum *Entfernen* eines Elements verfährt man genau umgekehrt. Durch Rotationen nach links oder rechts bewegt man das zu entfernende Element x solange abwärts, bis beide Söhne des Knotens, der x speichert, Blätter sind. Dabei hängt die Entscheidung, ob x durch eine Rotation nach links oder rechts um ein Niveau nach unten bewegt wird,



Abbildung 5.39

jeweils davon ab, welcher der beiden Söhne des Knotens, der x gespeichert hat, das Element mit kleinerer Priorität gespeichert hat. Dies Element muß durch die Rotation um ein Niveau hochgezogen werden. Ist x bei einem Knoten angelangt, dessen beide Söhne Blätter sind, entfernt man diesen Knoten und ersetzt ihn durch ein Blatt. Abbildung 5.39 zeigt zugleich ein Beispiel für eine Entferne-Operation: Um aus dem letzten Treap das Element $(7, 0)$ zu entfernen, muß das Element $(7, 0)$ durch Ausführung der angegebenen Rotationen in umgekehrter Reihenfolge und Richtung nach unten bewegt werden, bis es entfernt werden kann.

5.3.2 Treaps mit zufälligen Prioritäten

Ein *randomisierter Suchbaum* für eine Menge S von Schlüsseln ist ein Treap für eine Menge von Elementen, deren Schlüssel genau die Schlüssel in S sind und deren Prioritäten unabhängig und gleichverteilt zufällig gewählt sind. Wir setzen also voraus, daß keine zwei Schlüssel die gleiche Priorität erhalten. Ferner soll die Zuweisung von Prioritäten so erfolgen, daß jede Permutation der Elemente von S gleich wahrscheinlich ist, wenn man die Elemente von S nach wachsenden Prioritäten ordnet. Um die Zufälligkeit auch nach einer Einfüge- oder Entferne-Operation sicherzustellen, muß der Mechanismus der Zuweisung von Prioritäten zu Schlüsseln, z.B. durch einen Zufallszahlengenerator, vor dem Benutzer verborgen bleiben. Denn sonst könnte er leicht durch „einseitige“ Wahl von Schlüsseln (und Prioritäten) dennoch degenerierte Bäume erzeugen. Fügen wir also in eine Menge S von N Schlüsseln einen weiteren Schlüssel x ein, so nehmen wir an, daß x eine Priorität zugewiesen wird, für die gilt: Die Wahrscheinlichkeit dafür, daß die x zugewiesene Priorität in eines der durch die den bisherigen Elementen zugewiesenen Prioritäten definierten Prioritätsintervalle fällt, ist für jedes Intervall gleich groß. Damit ist klar, daß die Struktur eines randomisierten Suchbaumes für eine Menge von N Schlüsseln mit der eines zufällig erzeugten Suchbaumes für diese Schlüssel identisch ist. Insbesondere ist damit der Erwartungswert für die durchschnittliche Suchpfadlänge von der Größenordnung $O(\log N)$, vgl. Abschnitt 5.1.3.

Wir berechnen jetzt die Erwartungswerte für die Kosten einer einzelnen Such-, Einfüge- und Entferne-Operation. Da eine Einfüge-Operation als invers ausgeführte Entferne-Operation aufgefaßt werden kann, genügt es, die Kosten der Such- und Entferne-Operation abzuschätzen. Die Kosten der Entferne-Operation setzen sich aus zwei Anteilen zusammen, den Kosten, um auf das zu entfernende Element x zuzugreifen (Suchkosten) und den Kosten, x zu den Blättern hinunter zu rotieren und dort zu entfernen (Entfernungskosten).

Suchkosten

Wir berechnen den Erwartungswert für die Kosten, um auf den m -ten Schlüssel in einem randomisierten Suchbaum mit N Schlüsseln zuzugreifen. Dazu nehmen wir ohne Einschränkung an, daß im Suchbaum die Schlüssel $1, \dots, N$ gespeichert sind und auf m , $1 \leq m \leq N$, zugegriffen wird. Um auf den Schlüssel m zuzugreifen, müssen wir dem Pfad von der Wurzel zu m im Treap folgen. Zur Berechnung der Kosten einer Suchoperation (für die erfolgreiche Suche nach m) genügt es also, den Erwartungswert für den Abstand eines Schlüssels m , $1 \leq m \leq N$, in einem zufällig erzeugten Baum zu

berechnen, der die Schlüssel $1, \dots, N$ speichert. Wir betrachten dazu sämtliche Permutationen der Schlüssel $1, \dots, N$ und für jede Permutation σ den natürlichen Baum, der sich ergibt, wenn man die Schlüssel in der durch σ bestimmten Reihenfolge in den anfangs leeren Baum einfügt. Dann berechnen wir den Abstand von m von der Wurzel dieses Baumes und mitteln über alle Permutationen. Anders formuliert: Wählen wir eine Permutation σ der Schlüssel $1, \dots, N$ zufällig und jede der $N!$ möglichen Permutationen mit derselben Wahrscheinlichkeit, so berechnen wir den Erwartungswert für den Abstand des m -ten Schlüssels von der Wurzel des zu σ gehörenden natürlichen Baumes. Jeden Pfad von der Wurzel eines natürlichen Baumes zum Schlüssel m kann man in zwei Teile zerlegen, in $P_{\leq}(m)$ und $P_{\geq}(m)$.

$P_{\leq}(m)$ enthält genau die Schlüssel, die auf dem Pfad von der Wurzel zu m liegen und kleiner oder gleich m sind.

$P_{\geq}(m)$ enthält genau die Schlüssel, die auf dem Pfad von der Wurzel zu m liegen und größer oder gleich m sind.

Aus Symmetriegründen genügt es, den Erwartungswert für $P_{\leq}(m)$ zu berechnen.

Ist eine Permutation $\sigma = (a_1, \dots, a_N)$, also $a_i = \sigma(i)$, $1 \leq i \leq N$, gegeben, so liegen genau die Schlüssel k im σ zugeordneten natürlichen Baum auf $P_{\leq}(m)$, für die gilt:

- (1) $k \leq m$
- (2) k kommt in σ links von m (einschließlich m) vor (d.h. k wurde vor m eingefügt).
- (3) k ist größer als alle in σ links von k auftretenden Elemente, die ebenfalls $\leq m$ sind.

Beispiel: Sei $\sigma = (7, 2, 8, 9, 1, 4, 6, 5, 3)$. Der σ entsprechende natürliche Baum ist der Baum mit derselben Struktur wie der letzte Treap aus Abbildung 5.39; er ist noch einmal in Abbildung 5.40 dargestellt. Dann ist $P_{\leq}(5) = (2, 4, 5)$, $P_{\leq}(3) = (2, 3)$, $P_{\leq}(9) = (7, 8, 9)$ und $P_{\geq}(5) = (7, 6, 5)$.

Betrachtet man in einer Permutation σ der Zahlen $1, \dots, N$ nur die Elemente, die kleiner oder gleich m sind, in derselben Reihenfolge, in der sie in σ auftreten, so erhält man aus allen Permutationen von $1, \dots, N$ alle Permutationen von $1, \dots, m$ und zwar jede Permutation mit gleicher Wahrscheinlichkeit, wenn man jede Permutation von $1, \dots, N$ mit gleicher Wahrscheinlichkeit wählt. Zur Berechnung des Erwartungswertes für $P_{\leq}(m)$ genügt es also, eine zufällige Permutation τ von $1, \dots, m$ zu betrachten und dafür den Erwartungswert EH_m für die Anzahl der Zahlen k zu bestimmen mit der Eigenschaft, daß k größer ist als alle links von k in τ auftretenden Schlüssel. Offenbar hat eine Zahl $k > 1$ diese Eigenschaft genau dann, wenn k sie auch in der Folge hat, die entsteht, wenn man 1 wegläßt. Der Erwartungswert für die Anzahl dieser Zahlen ist daher EH_{m-1} . Die Zahl 1 muß noch hinzugezählt werden, wenn 1 das erste Element in τ ist. Das ist mit Wahrscheinlichkeit $1/m$ der Fall. Damit erhält man die Rekursionsformel

$$EH_m = EH_{m-1} + \frac{1}{m}$$

mit der Lösung $EH_m = \sum_{k=1}^m \frac{1}{k} = O(\log m)$.

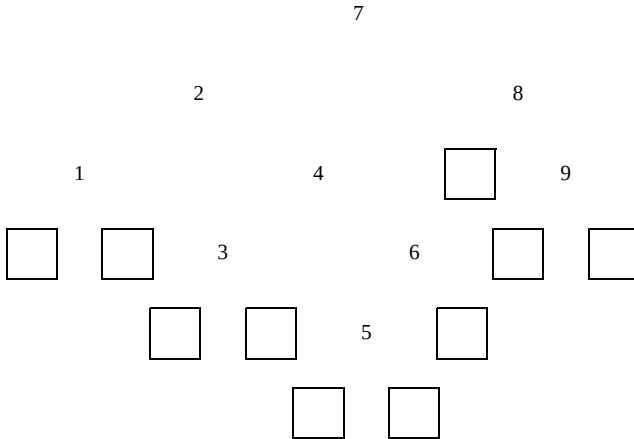


Abbildung 5.40

Man erhält also als Erwartungswert für $P_{\leq}(m)$ den Wert $O(\log m) = O(\log N)$, weil $m \leq N$ ist. Analog folgt, daß auch der Erwartungswert von $P_{\geq}(m)$ von der Größenordnung $O(\log N)$ ist. Die Suche ist daher in jedem Fall in $O(\log N)$ Schritten ausführbar.

Entfernungskosten

Um ein Element m aus einem Treap zu entfernen, muß man zunächst auf m zugreifen und m dann durch Rotationen solange abwärts bewegen, bis m bei den Blättern angekommen ist. Wir müssen also noch den Erwartungswert für die Anzahl der auszuführenden Rotationen berechnen. Zunächst zeigen die in Abbildung 5.38 erläuterten Rotationsoperationen folgendes: Wird ein Element durch eine Rotation nach rechts um ein Niveau abwärts bewegt (Element v in Abbildung 5.38), so nimmt dadurch die Länge des rechten Pfades im linken Teilbaum des Knotens, der das Element speichert, um 1 ab; die Länge des linken Pfades im rechten Teilbaum des Knotens, der das hinunterbewegte Element speichert, bleibt unverändert.

Analog gilt: Wird ein Element durch eine Rotation nach links um ein Niveau abwärts bewegt (Element u in Abbildung 5.38), so nimmt dadurch die Länge des linken Pfades im rechten Teilbaum des Knotens, der das Element speichert, um 1 ab; die Länge des linken Pfades im rechten Teilbaum des Knotens, der das hinterunterbewegte Element speichert, bleibt unverändert.

Aus diesen Beobachtungen folgt sofort, daß die Anzahl der Rotationen, um ein Element m von einem Knoten p bis zu den Blättern hinunterzubewegen, gleich der Summe der Länge des rechten Pfades im linken Teilbaum von p und der Länge des linken Pfades im rechten Teilbaum von p ist.

Beispiel: Für den Baum aus Abbildung 5.40 gilt: Die Knoten mit den Schlüsseln 2, 4, 6 bilden den rechten Pfad im linken Teilbaum des Knotens, der 7 speichert; und

der Knoten mit Schlüssel 8 ist der einzige Knoten auf dem linken Pfad im rechten Teilbaum des Knotens, der 7 speichert. Die Summe der Längen dieser Pfade ist 4. Vier Rotationen genügen also, um 7 von der Wurzel zu den Blättern zu bewegen. Das sind genau die in Abbildung 5.39 gezeigten Rotationen in umgekehrter Richtung und Reihenfolge.

Aus Symmetriegründen genügt es natürlich, den Erwartungswert EG_m für die Länge des rechten Pfades im linken Teilbaum des Knotens zu berechnen, der m gespeichert hat, wenn m ein Schlüssel in einem zufällig erzeugten binären Suchbaum für N Schlüssel $1, \dots, N$ und $1 \leq m \leq N$ ist. Natürlich können im linken Teilbaum des Knotens, der m gespeichert hat, nur Schlüssel $k < m$ auftreten. Betrachten wir also eine Permutation σ der Schlüssel $1, \dots, N$, so liegt ein Schlüssel k auf dem rechten Pfad im linken Teilbaum des Knotens, der m gespeichert hat im σ entsprechenden Baum, wenn folgendes gilt: k tritt rechts von m in σ auf (d.h. k wurde nach m eingefügt) und k ist größer als alle Schlüssel aus $1, \dots, m-1$, die k in σ vorangehen und links oder rechts von m auftreten.

Beispiel: Ist $\sigma = (7, 2, 8, 9, 1, 4, 6, 5, 3)$ und $m = 7$, so haben genau 2, 4, 6 die genannte Eigenschaft; ist $m = 4$, so nur $k = 3$.

Es genügt also, für eine zufällig gewählte Permutation τ von $1, \dots, m$ die Anzahl EG_m der Zahlen k zu bestimmen, für die gilt:

- (1) k tritt in τ rechts von m auf,
- (2) k ist größer als alle in τ k vorangehenden Elemente aus $1, \dots, m-1$, die rechts von m liegen.

Wenn wir die Bedingung (1) einfach weglassen und nur die Anzahl der Zahlen bestimmen wollen, die (2) erfüllen, können wir direkt das zuvor bei der Analyse der Suchkosten bereits hergeleitete Ergebnis übernehmen; der gesuchte Erwartungswert ist von der Größenordnung $O(\log m)$. Man kann aber mehr zeigen, nämlich, daß $EG_m < 1$ ist, und zwar wie folgt: In einer zufällig gewählten Permutation τ von $1, \dots, m$ erfüllt eine Zahl $k > 1$ die Bedingungen (1) und (2) genau dann, wenn sie die entsprechenden Bedingungen für die (ebenfalls zufällige) Permutation erfüllt, die man erhält, wenn man 1 wegläßt. Der Erwartungswert für die Anzahl der Zahlen $k > 1$, die (1) und (2) erfüllen, ist daher gleich EG_{m-1} . Die Zahl 1 erfüllt die Bedingungen (1) und (2) genau dann, wenn m die erste Zahl und 1 die zweite Zahl in der Permutation τ ist. Die Wahrscheinlichkeit dafür ist $1/(m-1)$. Also gilt für EG_m die folgende Rekursionsformel:

$$\begin{aligned} EG_m &= EG_{m-1} + \frac{1}{m(m-1)} \quad \text{und} \\ EG_1 &= 0. \end{aligned}$$

Diese Gleichung hat die Lösung $EG_m = (m-1)/m < 1$.

Insgesamt ergibt sich damit, daß der Erwartungswert für die Anzahl der Rotationen nach der Entfernung eines Schlüssels aus einem randomisierten Suchbaum kleiner als 2 ist. Dasselbe gilt natürlich auch für das Einfügen, weil Einfügen und Entfernen in randomisierten Suchbäumen invers zueinander sind.

Praktische Realisierung

Eine Implementation randomisierter Suchbäume erfordert es, Schlüsseln zufällige Prioritäten zuzuweisen und zwar so, daß nach jeder Update-Operation die Prioritäten der Schlüssel der jeweils vorliegenden Menge unabhängige und gleichverteilte Zufallsvariablen sind. Irgendwelche Annahmen über die Verteilung der Schlüssel selbst werden nicht gemacht. Aragon und Seidel [1] schlagen dazu vor, als Prioritäten zufällige und gleichverteilte reelle Zahlen aus dem Intervall $[0, 1)$ zu nehmen und sie wie folgt zu erzeugen: Man generiert die Dualdarstellung der den Schlüsseln zugewiesenen Prioritäten nach Bedarf bitweise Stück für Stück, indem man mit Hilfe eines 0-1-wertigen Zufallszahlengenerators immer gerade so viele Bits erzeugt, wie erforderlich sind, um eine eindeutige Anordnung der den Schlüsseln zugewiesenen Prioritäten zu ermöglichen. Wird also z.B. ein neuer Schlüssel x in einen randomisierten Suchbaum eingefügt, so fügt man x an der vom Suchverfahren erwarteten Position unter den Blättern ein. Ist p der Vater dieses Blattes und hat p einen Schlüssel y gespeichert, dem als Priorität durch n zufällig erzeugte Bits a_i bisher ein Wert $0.a_1 \dots a_n$ zugewiesen wurde, so erzeugt man so viele neue Bits b_j bis die Bitfolgen $0.a_1 a_2 \dots$ und $0.b_1 b_2 \dots$ erstmals eine eindeutige Anordnung ermöglichen; unter Umständen kann es erforderlich werden, auch die Bitfolge a_i zu verlängern. Meistens wird aber schon nach wenigen Bits klar sein, welche Bitfolge Anfangsstück der Dualdarstellung der reellen Zahl mit größerem oder kleinerem Wert ist. Dann weist man die so erhaltene Bitfolge x als Priorität zu. Wird nun x nach oben rotiert, so kann es erforderlich werden, die x zugewiesene Priorität mit den anderen Schlüsseln zugewiesenen Prioritäten zu vergleichen. Wenn die bisher erzeugten Bitfolgen keine eindeutige Entscheidung zur Anordnung der Prioritäten erlauben, werden in jedem Fall so viele weitere Bits zufällig erzeugt, bis erstmals eine eindeutige Entscheidung möglich ist. Man kann zeigen [2], daß der Erwartungswert für die zusätzliche Zahl von Bits, die nötig ist, um nach einer Update-Operation eine eindeutige Anordnung der Prioritäten zu ermöglichen, konstant ist (höchstens 12).

5.4 Selbstanordnende Binärbäume

Ganz ähnlich wie bei linearen Listen, vgl. Abschnitt 3.3, kann man auch für binäre Suchbäume Strategien zur Selbstanordnung entwickeln. Das Ziel ist dabei, möglichst ohne explizite Speicherung von Balance-Informationen oder Häufigkeitszählern eine Strukturanpassung an unterschiedliche Zugriffshäufigkeiten zu erreichen. Schlüssel, auf die relativ häufig zugegriffen wird, sollen näher zur Wurzel wandern. Dafür können andere, auf die seltener zugegriffen wird, zu den Blättern hinabwandern. Sind die Zugriffshäufigkeiten fest und vorher bekannt, so kann man Suchbäume konstruieren, die optimal in dem Sinne sind, daß sie die Suchkosten minimieren unter der Voraussetzung, daß sich die Struktur des Suchbaumes während der Folge der Suchoperationen nicht ändert. Verfahren zur Konstruktion optimaler Suchbäume werden im Abschnitt 5.7 vorgestellt. Wir behandeln in diesem Abschnitt den Fall, daß die Zugriffshäufigkeiten nicht bekannt und möglicherweise (über die Zeit) variabel sind.

Durch Ausführung von Rotationen kann der Abstand zur Wurzel eines in einem binären Suchbaum gespeicherten Schlüssels verändert werden, ohne daß die Suchbaumstruktur dadurch zerstört wird. Es ist daher naheliegend, diese Beobachtung für die Entwicklung von Heuristiken zur Selbstanordnung von binären Suchbäumen zu nutzen. So entspricht der *T-Regel* (Transpositionsregel) für lineare Listen die Strategie, nach Ausführung einer Suche das gefundene Element durch eine Rotation um ein Niveau hinaufzubewegen, falls es nicht schon an der Wurzel gefunden wird. Analog entspricht der *MF-Regel* (Move-to-front) für lineare Listen die folgende Move-to-root-Strategie für binäre Suchbäume: Nach jedem Zugriff auf einen Schlüssel wird er durch Rotationen solange hinauf bewegt, bis er bei der Wurzel angekommen ist. Leider haben diese beiden einfachen und naheliegenden Strategien die unangenehme Eigenschaft, daß es beliebig lange Zugriffsfolgen gibt, für die die pro Zugriff benötigte Zeit für einen Baum mit N Schlüsseln von der Größenordnung $\Theta(N)$ ist, vgl.  Wir werden im folgenden Abschnitt jedoch eine Variante der Move-to-root-Heuristik zur Selbstanordnung von binären Bäumen kennenlernen, die amortisierte logarithmische Kosten für alle drei Wörterbuchoperationen garantiert. D.h. die über eine beliebige Folge von Such-, Einfüge- und Entferne-Operationen gemittelten Kosten pro Operation sind von der Größenordnung $O(\log N)$. Natürlich kann eine einzelne Operation für einen nach dieser Strategie entstandenen sogenannten *Splay-Baum* mit N Schlüsseln durchaus $\Theta(N)$ Schritte kosten. Das ist aber nur möglich, wenn vorher genügend viele „billige“ Operationen vorgekommen sind, so daß die Durchschnittskosten über die gesamte Operationsfolge pro Operation $O(\log N)$ sind. Wir erhalten damit zwar nicht dasselbe Verhalten wie bei der Verwendung von balancierten Bäumen im schlechtesten Fall für jede einzelne Operation, aber ein gleich gutes Verhalten für die Operationenfolge im schlechtesten Fall und damit für jede einzelne Operation im Durchschnitt und sogar ein wesentlich besseres, wenn die Zugriffshäufigkeiten auf Schlüssel sehr stark unterschiedlich sind.

5.4.1 Splay-Bäume

Splay-Bäume sind reine binäre Suchbäume, d.h. ohne jede zusätzliche Information wie Balance-Faktoren oder Häufigkeitszähler o.ä., die sich durch eine Variante der Move-to-root-Strategie selbst anordnen. Die wichtigste Operation ist die *Splay-Operation*: Sie verbreitert (englisch: splay) den Suchbaum so, daß nicht nur jeder Schlüssel x , auf den zugegriffen wurde, durch Rotationen zur Wurzel bewegt wird; sondern durch geschickte Zusammenfassung der Rotationen zu Paaren wird darüberhinaus zugleich erreicht, daß sich die Längen sämtlicher Pfade zu Schlüsseln auf dem Suchpfad zu x etwa halbieren. Eine künftige Suche nach einem dieser Schlüssel wird also als Folge der Suche nach x schneller.

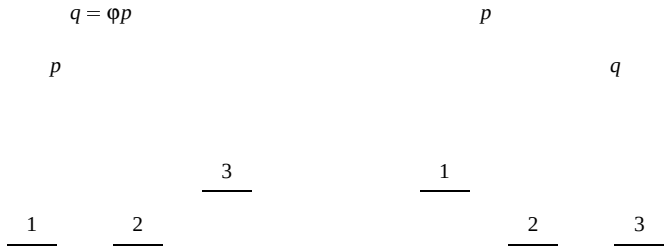
Wir erläutern jetzt zunächst die Splay-Operation, und dann, wie die Wörterbuchoperationen darauf zurückgeführt werden können.

Sei t ein binärer Suchbaum und x ein Schlüssel. Dann ist das Ergebnis der Operation $\text{Splay}(t, x)$ der binäre Suchbaum, den man wie folgt erhält.

Schritt 1: Suche nach x in t . Sei p der Knoten, bei dem die (erfolgreiche) Suche endet, falls x in t vorkommt, und sei p der Vater des Blattes, bei dem eine erfolglose Suche nach x in t endet, sonst.

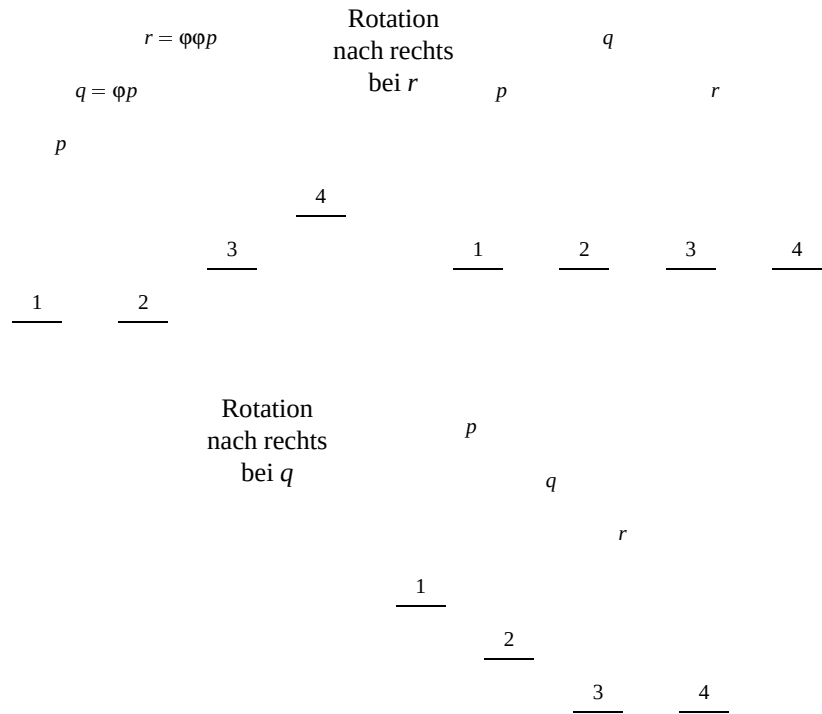
Schritt 2: Wiederhole die folgenden Operationen *zig*, *zig-zig* und *zig-zag* beginnend bei *p* solange, bis sie nicht mehr ausführbar sind, weil *p* Wurzel geworden ist.

Fall 1: [*p* hat Vater $\varnothing p$ und $\varnothing p$ ist die Wurzel]
Dann führe die Operation „*zig*“ aus, d.h. eine Rotation nach links oder rechts, die *p* zur Wurzel macht.



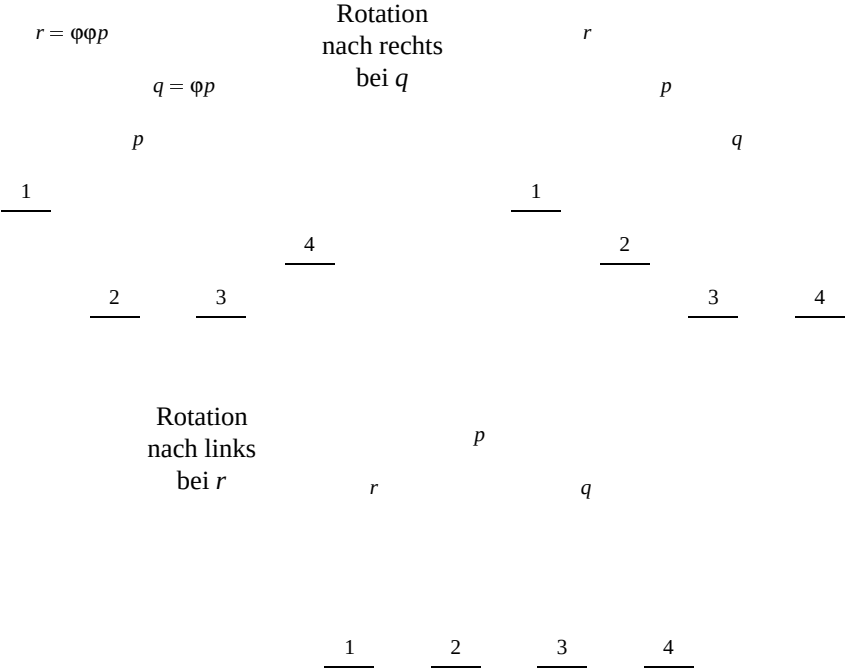
Fall 2: [*p* hat Vater $\varnothing p$ und Großvater $\varnothing \varnothing p$ und *p* und $\varnothing p$ sind beides *rechte* oder beides *linke* Söhne]

Dann führe die Operation „*zig-zig*“ aus, d.h. zwei aufeinanderfolgende Rotationen in dieselbe Richtung, die *p* zwei Niveaus hinaufbewegen.



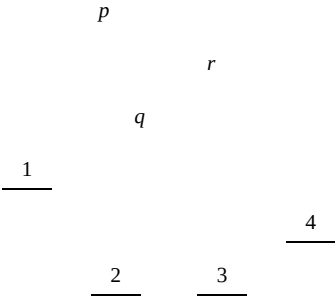
Fall 3: [*p* hat Vater $\varnothing p$ und Großvater $\varnothing \varnothing p$ und einer der beiden Knoten *p* und $\varnothing p$ ist linker und der andere rechter Sohn seines jeweiligen Vaters]

Dann führe die Operation „*zig-zag*“ aus, d.h. zwei Rotationen in entgegengesetzte Richtungen, die *p* zwei Niveaus hinaufbewegen.



In jedem dieser drei Fälle haben wir nur jeweils eine der möglichen symmetrischen Varianten veranschaulicht.

Die Splay-Operation kann als eine Variante der Move-to-root-Strategie aufgefaßt werden: Der Schlüssel, auf den zugegriffen wird, wird zur Wurzel rotiert. Während bei der Move-to-root-Strategie jedoch Rotationen strikt „von unten nach oben“ durchgeführt werden, werden bei der Splay-Operation Rotationen nicht immer (nämlich im zig-zig-Fall nicht) strikt in dieser Reihenfolge durchgeführt. Hier liegt der einzige Unterschied zur Move-to-root-Strategie; sie würde im zig-zig-Fall zunächst eine Rotation nach rechts bei q und dann eine Rotation nach rechts bei r durchführen. Als Ergebnis würde man statt des Baumes im Fall 2 erhalten:



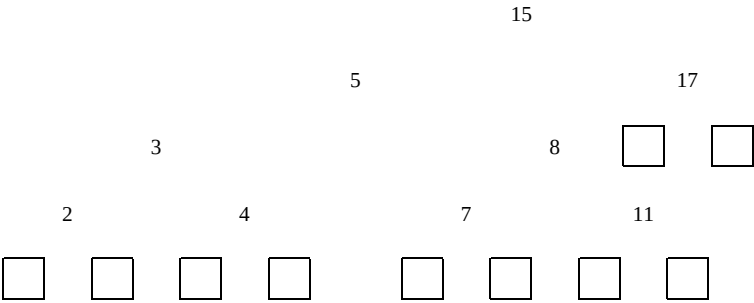


Abbildung 5.41

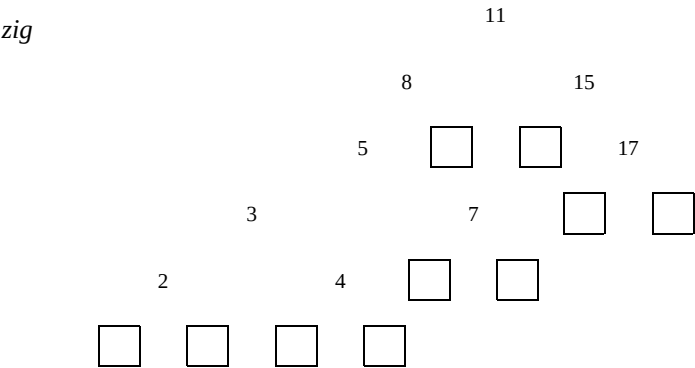
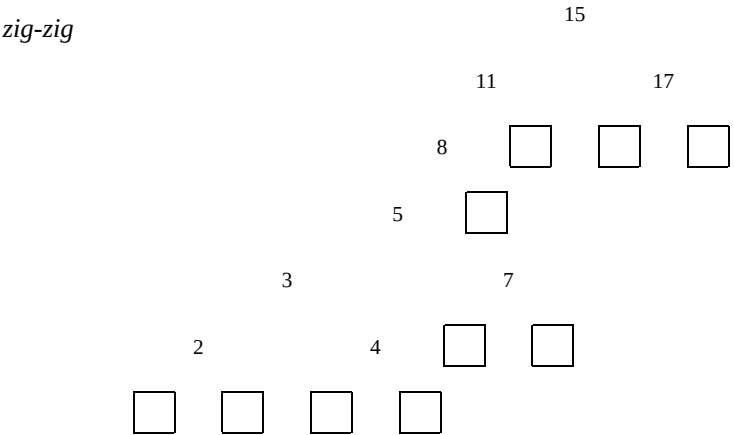
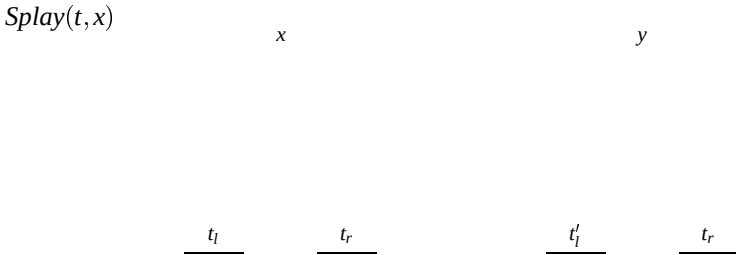


Abbildung 5.42

Zum *Entfernen* eines Schlüssels x aus einem Baum t führe zunächst wieder $Splay(t, x)$ aus. Falls x nicht Schlüssel der Wurzel ist, ist nichts zu tun; denn dann kam x in t gar nicht vor. Andernfalls hat der Baum den Schlüssel x an der Wurzel und einen linken Teilbaum t_l und einen rechten Teilbaum t_r . Dann führe $Splay(t_l, +\infty)$ aus, wobei $+\infty$ ein Schlüssel ist, der größer ist als alle Schlüssel in t_l . Dadurch entsteht ein Baum t'_l mit dem größten Schlüssel y von t_l an der Wurzel und einem leeren rechten Teilbaum. Ersetze diesen leeren Teilbaum durch t_r .



Man beachte, daß die Ausführung einer Operation $Splay(t, x)$ stets eine Suche nach x im Baum t einschließt. Dasselbe gilt daher auch für jede Wörterbuchoperation. Bei der Analyse der Kosten für die einzelnen Operationen kann man daher die Suchkosten unberücksichtigt lassen, da sie durch die Kosten der längs des Suchpfades auszuführenden Rotationen dominiert werden.

Offensichtlich kann jede Operation Splay, Suchen, Einfügen und Entfernen auf einen beliebigen binären Suchbaum angewandt werden.

Die Klasse aller Bäume, die man erhält, wenn man ausgehend vom anfangs leeren Baum eine beliebige Folge von Such-, Einfüge- und Entferne-Operationen ausführt mit den hier dafür angegebenen Verfahren, heißt die Klasse der *Splay-Bäume*.

5.4.2 Amortisierte Worst-case-Analyse

Zur Abschätzung der Kosten der drei Wörterbuchoperationen müssen wir die Kosten zur Ausführung einer Splay-Operation abschätzen. Denn alle Wörterbuchoperationen wurden auf die Splay-Operation zurückgeführt. Ähnlich wie im Fall selbstanordnender linearer Listen werden wir dazu das Bankkonto-Paradigma verwenden, um die amortisierten Kosten pro Operation zu berechnen. Eine Splay-Operation $Splay(t, x)$ für einen Baum t und einen Schlüssel x besteht darin, auf x zuzugreifen, den Suchpfad zurückzulaufen und entlang dieses Pfades eine Folge von *zig-zag*-, *zig-zig*- und *zig*-Operationen durchzuführen. Wir messen die Kosten durch die Anzahl der ausgeführten Rotationen (plus 1, falls keine Rotation ausgeführt wird). Darin sind die Suchkosten enthalten. Jede *zig*-Operation schlägt mit einer und jede *zig-zig*- oder *zig-zag*-Operation mit zwei Rotationen zu Buche. Manchmal muß man viele, ein anderes Mal wenige Rotationen ausführen. Betrachten wir z.B. den Fall, daß wir der Reihe nach die Schlüssel $1, 2, \dots, N$ in den anfangs leeren Baum nach dem im vorigen Abschnitt angegebenen Verfahren einfügen. Dann wird der jeweils nächste Schlüssel zur neuen Wurzel. Es entsteht also ein zu einer linearen Liste „degenerierter“ Baum. Führt man jetzt als nächstes eine Suchoperation nach dem Schlüssel 1 durch, so müssen nach dem Zugriff auf diesen Schlüssel

N Rotationen durchgeführt werden, um den Schlüssel 1 zur Wurzel zu befördern. Der entstandene Baum hat dann aber die Eigenschaft, daß die weitere Suche nach anderen Schlüsseln billiger wird. Abbildung 5.43 zeigt ein Beispiel für den Fall $N = 5$.

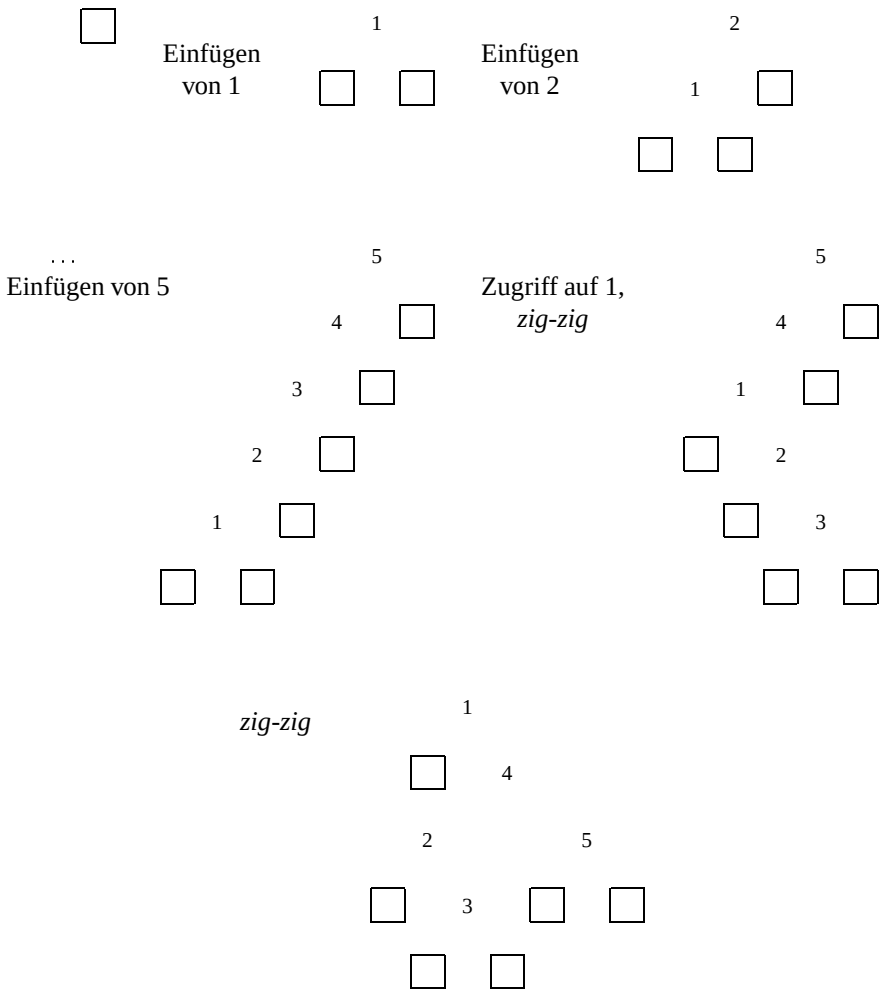


Abbildung 5.43

Manchmal muß man also zur Ausführung einer Splay-Operation viele, ein anderes Mal wenige Einzeloperationen (Rotationen) ausführen. Stellen wir uns daher vor, wir hätten einen festen, nur von der Größe der Struktur abhängigen Durchschnittsbetrag zur Verfügung, den wir für eine Splay-Operation insgesamt ausgeben dürfen. Führen wir dann eine „billige“ Splay-Operation durch, so sparen wir Geld, das wir einem Kon-

to gutschreiben. Dann können wir bei „teuren“ Operationen Geld vom Konto entnehmen, um den erforderlichen Mehraufwand zu bezahlen. Der Gesamtbetrag des für eine Operationsfolge ausgegebenen Geldes ist ein Maß für die Kosten.

Wir ordnen also jedem binären Suchbaum einen nur von seiner Größe abhängigen Kontostand zu. Nehmen wir an, daß niemals Strukturen mit mehr als N Knoten entstehen. Dann werden wir zeigen, daß jede Folge von m Operationen mit einer „Gesamtinvestition“ von $O(m \log N)$ Geldeinheiten, also im Durchschnitt mit Kosten $O(\log N)$ pro Operation, ausführbar ist.

Genauer sei ϕ_l der nach Ausführung der l -ten Operation vorliegende Kontostand. Dann sind die *amortisierten Kosten (Zeit)* a_l der l -ten Operation in der Folge der m Operationen die Summe der tatsächlichen Kosten (Zeit) t_l plus die Differenz der Kontostände:

$$a_l = t_l + \phi_l - \phi_{l-1}, \text{ für } 1 \leq l \leq m.$$

Dabei ist ϕ_0 der Kontostand am Anfang und ϕ_m der Kontostand der Struktur, die am Ende der Operationsfolge vorliegt. Ist $\phi_0 \leq \phi_m$, so ist die gesamte zur Ausführung der m Operationen verbrauchte amortisierte Zeit $\sum_{i=1}^m a_i$ eine obere Schranke für die wirklich verbrauchte Zeit $\sum_{i=1}^m t_i$. Denn es gilt dann

$$\sum_{i=1}^m t_i = \sum_{i=1}^m a_i + \phi_0 - \phi_m \leq \sum_{i=1}^m a_i.$$

Dazu müssen wir zunächst eine geeignete Funktion ϕ finden, die einem Baum einen Kontostand zuordnet.

Wir benutzen die von Sleator und Tarjan [10] vorgeschlagene Funktion ϕ . Sie erlaubt es nicht nur, die behauptete amortisierte Zeitschranke von $O(\log N)$ für jede Wörterbuchoperation herzuleiten, sondern auch weitere Eigenschaften von Splay-Bäumen.

Für jeden Schlüssel x sei $w(x)$ ein beliebiges, aber festes, positives *Gewicht* (englisch: *weight*). Für einen Knoten p sei $s(p)$, die *Größe* von p (englisch: *size*), die Summe aller Gewichte von Schlüssel n im Teilbaum mit Wurzel p . Schließlich sei $r(p)$, der *Rang* von p , definiert durch

$$r(p) = \log_2 s(p).$$

Für einen Baum t mit Wurzel p und für einen in p gespeicherten Schlüssel x sind $r(t)$ und $r(x)$ definiert als Rang $r(p)$.

Man beachte, daß verschiedene Schlüsselgewichte lediglich ein Parameter der Analyse, aber nicht der Algorithmen von Splay-Bäumen sind.

Wir werden später insbesondere den Fall $w(x) = 1$ für alle Schlüssel x betrachten.

Nun definieren wir den einem Splay-Baum t zugeordneten *Kontostand* $\phi(t)$ als die Summe aller Ränge von (inneren) Knoten von t .

Basis der Splay-Baum Analyse ist das folgende Lemma.

Lemma 5.3 (Zugriffs-Lemma) *Die amortisierte Zeit, um eine Operation $\text{Splay}(t, x)$ auszuführen, ist höchstens $3(r(t) - r(x)) + 1$.*

Zum Beweis betrachten wir zunächst den Fall, daß x bereits Schlüssel der Wurzel ist. Dann wird nur auf x zugegriffen und weiter keine Operation ausgeführt. Die tatsächliche Zeit stimmt also mit der amortisierten überein; beide haben den Wert 1 und das Zugriffs-Lemma gilt in diesem Fall, da sich $r(t)$ und $r(x)$ in diesem Fall nicht ändern. Wir können also annehmen, daß wenigstens eine Rotation ausgeführt wird. Für jede im Zuge der Ausführung von $Splay(t, x)$ durchgeführte *zig*-, *zig-zig*- und *zig-zag*-Operation, die einen Knoten p betrifft, betrachten wir die Größe $s(p)$ und den Rang $r(p)$ unmittelbar vor und die Größe $s'(p)$ und den Rang $r'(p)$ unmittelbar nach Ausführung einer dieser Operationen. Wir werden zeigen, daß jede *zig-zig*- oder *zig-zag*-Operation für p in amortisierter Zeit von höchstens $3(r'(p) - r(p))$ und jede *zig*-Operation in amortisierter Zeit höchstens $3(r'(p) - r(p)) + 1$ ausführbar ist. Nehmen wir einmal an, wir hätten das bereits bewiesen und sei $r^{(i)}(x)$ der Rang von x nach Ausführen der i -ten von insgesamt k *zig-zig*-, *zig-zag*- oder *zig*-Operationen. (Genau die letzte Operation ist eine *zig*-Operation.) Dann ergibt sich als amortisierte Zeit zur Ausführung von $Splay(t, x)$ insgesamt die folgende obere Schranke:

$$\begin{aligned} & 3(r^{(1)}(x) - r(x)) \\ + & 3(r^{(2)}(x) - r^{(1)}(x)) \\ & \vdots \\ + & 3(r^{(k)}(x) - r^{(k-1)}(x)) + 1 \\ = & 3(r^{(k)}(x) - r(x)) + 1. \end{aligned}$$

Weil x durch die k Operationen zur Wurzel gewandert ist, ist $r^{(k)}(x) = r(t)$ und damit das Zugriffs-Lemma bewiesen. Wir müssen daher nur noch die amortisierten Kosten jeder einzelnen Operation abschätzen. Dazu betrachten wir jeden der drei Fälle getrennt.

Fall 1 [*zig*] Dann ist $q = \phi p$ die Wurzel. Es wird eine Rotation ausgeführt. Die tatsächlichen Kosten der *zig*-Operation sind also 1. Es können durch die Rotation höchstens die Ränge von p und q geändert worden sein. Die amortisierten Kosten am_{zig} der *zig*-Operation sind daher:

$$\begin{aligned} am_{zig} &= 1 + (r'(p) + r'(q)) - (r(p) + r(q)) \\ &= 1 + r'(q) - r(p), \text{ da } r'(p) = r(q) \\ &\quad 1 + r'(p) - r(p), \text{ da } r'(p) = r'(q) \\ &\quad 1 + 3(r'(p) - r(p)), \text{ da } r'(p) = r(p) \end{aligned}$$

Bevor wir die nächsten beiden Fälle behandeln, formulieren wir einen Hilfssatz, den wir dabei verwenden.

Hilfssatz 5.1 Sind a und b positive Zahlen und gilt $a + b \leq c$, so folgt $\log_2 a + \log_2 b \leq 2 \log_2 c - 2$.

Zum Beweis des Hilfssatzes gehen wir aus von der bekannten Tatsache, daß das geometrische Mittel zweier positiver Zahlen niemals größer als das arithmetische ist:

$$\frac{\overline{ab}}{ab} = \frac{(a+b)/2}{\frac{c}{2}}, \text{ also nach Voraussetzung}$$

Quadrieren und Logarithmieren ergibt sofort die gewünschte Behauptung.

Kehren wir nun zum Beweis des Zugriffs-Lemmas zurück und behandeln die restlichen zwei Fälle.

Fall 2 [zig-zag] Sei $q = \phi p$ und $r = \phi \phi p$. Eine auf p ausgeführte zig-zag-Operation hat tatsächliche Kosten 2, weil zwei Rotationen ausgeführt werden. Es können sich höchstens die Ränge von p , q und r ändern. Ferner ist $r'(p) = r(r)$. Also gilt für die amortisierten Kosten

$$\begin{aligned} am_{\text{zig-zag}} &= 2 + (r'(p) + r'(q) + r'(r)) - (r(p) + r(q) + r(r)) \\ &= 2 + r'(q) + r'(r) - r(p) - r(q) \end{aligned}$$

Nun ist $r(q) = r(p)$, weil p vor Ausführung der zig-zag-Operation Sohn von q war. Daher folgt

$$am_{\text{zig-zag}} = 2 + r'(q) + r'(r) - 2r(p) \quad (*)$$

Um die Abschätzung für $r'(q) + r'(r)$ zu erhalten, betrachten wir noch einmal die Abbildung, in der die zig-zag-Operation veranschaulicht wird. Daraus entnehmen wir, daß gilt $s'(q) + s'(r) = s'(p)$. Die Definition des Ranges und der oben angegebene Hilfssatz liefern damit $r'(q) + r'(r) = 2r'(p) - 2$. Setzt man das in (*) ein, erhält man

$$\begin{aligned} am_{\text{zig-zag}} &= 2(r'(p) - r(p)) \\ &= 3(r'(p) - r(p)), \text{ da } r'(p) = r(p). \end{aligned}$$

Fall 3 [zig-zig] Sei wieder $q = \phi p$ und $r = \phi \phi p$. Eine auf p ausgeführte zig-zig-Operation hat tatsächliche Kosten 2, weil zwei Rotationen ausgeführt werden. Genau wie im vorigen Falle folgt zunächst:

$$am_{\text{zig-zig}} = 2 + r'(q) + r'(r) - r(p) - r(q)$$

Da vor Ausführung der zig-zig-Operationen p Sohn von q und nachher q Sohn von p ist, folgt $r(p) = r(q)$ und $r'(p) = r'(q)$. Daher gilt

$$am_{\text{zig-zig}} = 2 + r'(p) + r'(r) - 2r(p)$$

Diese letzte Summe ist kleiner oder gleich $3(r'(p) - r(p))$ genau dann, wenn

$$r(p) + r'(r) = 2r'(p) - 2 \quad (**)$$

ist. Zum Nachweis von (**) betrachten wir noch einmal die Abbildung, die die zig-zig-Operation veranschaulicht. Daraus entnimmt man, daß gilt $s(p) + s'(r) = s'(p)$. Mit Hilfe des oben angegebenen Hilfssatzes und der Definition der Ränge erhält man daraus sofort die gewünschte Ungleichung (**). Damit ist das Zugriffs-Lemma bewiesen.

□

Eine genaue Betrachtung der im Beweis des Zugriffs-Lemmas benutzten Argumentation zeigt folgendes: Nur im Fall 3 (der zig-zig-Operation) ist die Abschätzung der amortisierten Kosten scharf. Sie wird überhaupt erst dadurch möglich, daß hier die strikte „bottom-up-Rotations-Strategie“ der Move-to-root-Heuristik lokal durchbrochen wird.

Wir ziehen eine erste Folgerung aus dem Zugriffs-Lemma.

Satz 5.1 *Das Ausführen einer beliebigen Folge von m Wörterbuchoperationen, in der höchstens N mal die Operation Einfügen vorkommt und die mit dem anfangs leeren Splay-Baum beginnt, benötigt höchstens $O(m \log N)$ Zeit.*

Zum Beweis wählen wir sämtliche Gewichte gleich 1 und erhalten als amortisierte Kosten einer Splay-Operation $Splay(t, x)$ die Schranke $3 \cdot (r(t) - r(x)) + 1$. Weil in diesem Fall für jeden im Verlauf der Operationsfolge erzeugten Baum $s(t) \leq N$ gilt und jede Wörterbuchoperation höchstens ein konstantes Vielfaches der Kosten der Splay-Operation verursacht, folgt die Behauptung. $\square$

Wir haben bereits darauf hingewiesen, daß die Splay-Operation auf einen beliebigen binären Suchbaum anwendbar ist. Das Zugriffs-Lemma erlaubt es, die amortisierten Kosten einer Splay-Operation und damit auch die amortisierten Kosten einer Zugriffs-(Such-)Operation abzuschätzen. Wegen

$$t = a + (\phi_{\text{vorher}} - \phi_{\text{nachher}})$$

kann man die realen Kosten abschätzen, wenn man die durch die Operation bedingte Veränderung des Kontostandes kennt.

Eine auf einem beliebigen Baum mit N Knoten ausgeführte Splay- (oder Such-) Operation wird den Kontostand in der Regel verringern. Die maximal mögliche Abnahme des Kontostandes und der damit zur Ausführung der Operation neben den amortisierten Kosten maximal vom Konto zu entnehmende Geldbetrag kann leicht abgeschätzt werden. Ist $W = \sum_{i=1}^N w_i$ die Summe aller Gewichte der im Baum gespeicherten Schlüssel, so ändert sich durch die Splay-Operation für jeden einzelnen Schlüssel i mit Gewicht w_i der Rang $r(i)$ vor Ausführung und $r'(i)$ nach Ausführung der Splay-Operation höchstens um den Betrag

$$r(i) - r'(i) \leq \log W - \log w_i.$$

Also kann die Gesamtveränderung des Kontostandes wie folgt abgeschätzt werden:

$$\begin{aligned} \phi_{\text{vorher}} - \phi_{\text{nachher}} &= \sum_{i=1}^N (\log W - \log w_i) \\ &= \sum_{i=1}^N \log \frac{W}{w_i}. \end{aligned}$$

Dieselbe Überlegung gilt auch für eine Folge von m Zugriffs-Operationen: Die zur Ausführung der m Operationen erforderlichen wirklichen Kosten $\sum_{i=1}^m t_i$ ist die Summe der amortisierten Kosten $\sum_{i=1}^m a_i$ plus die Gesamtveränderung des Kontos $\phi_0 - \phi_m$ vor und nach Ausführung der Operationsfolge. Die Gesamtveränderung des Kontos kann wie oben gezeigt durch $\sum_{i=1}^N \log(W/w_i)$ abgeschätzt werden.

Wählt man nun wieder $w_i = 1$ für jedes i , so ergibt sich zunächst als amortisierte Zeit für jeden Zugriff auf einen Schlüssel x die Schranke $3(r(t) - r(x)) + 1 \leq 3 \log_2 N + 1$ aus dem Zugriffs-Lemma. Ferner ist die Gesamtveränderung des Kontos durch m Zugriffsooperationen höchstens $\sum_{i=1}^N \log(W/w_i) = N \log N$.

Damit erhält man sofort folgenden Satz.

Satz 5.2 *Führt man für einen beliebigen binären Suchbaum mit N Schlüsseln m -mal die Operation Suchen aus, so ist die dafür insgesamt benötigte Zeit von der Größenordnung $O((N + m) \log N + m)$.*

Man beachte, daß eine einzelne Such-Operation sehr wohl $\Omega(N)$ Schritte kosten kann, z.B. dann, wenn man mit einem zu einer linearen Liste „degenerierten“ Baum mit Höhe N startet und auf den Schlüssel mit größtem Abstand zur Wurzel zugreift. Aus Satz 5.2 folgt jedoch, daß für jede genügend lange Folge von Zugriffsooperationen, d.h. falls $m = \Omega(N)$, die pro Operation im Mittel über die Operationsfolge erforderliche Zeit durch $O(\log N)$ beschränkt bleibt. Das ist weniger als man für balancierte Bäume erreicht hat, aber mehr als für natürliche Suchbäume gilt. Erkauft wird dieses Verhalten dadurch, daß anders als für natürliche Suchbäume oder balancierte Bäume jede Zugriffs-Operation nach dem für Splay-Bäume definierten Verfahren die Struktur des Baumes verändert (falls nicht gerade auf die Wurzel zugegriffen wird): Jeder Zugriff „verbessert“ den Baum in dem Sinne, daß künftige Suchoperationen beschleunigt werden. Genauer kann das durch folgenden Satz ausgedrückt werden.

Satz 5.3 *Führt man für einen beliebigen binären Suchbaum mit N Schlüsseln insgesamt m -mal die Operation Suchen aus, so daß dabei auf Schlüssel i $q(i)$ -mal zugegriffen wird, so ist die dafür insgesamt benötigte Zeit von der Größenordnung*

$$O(m + \sum_{i=1}^N q(i) \log(\frac{m}{q(i)})).$$

Zum Beweis wählen wir als Gewicht des Schlüssels i den Wert $w_i = q(i)/m$ und damit $W = \sum_{i=1}^N w_i = 1$ und $\sum_{i=1}^N q(i) = m$. Dann folgt aus dem Zugriffs-Lemma für die amortisierten Kosten eines Zugriffs auf einen beliebigen Schlüssel i die obere Schranke

$$\begin{aligned} 3(r(t) - r(i)) + 1 &= 3(\log W - \log w_i) + 1 \\ &= 3(\log_2 1 - \log_2 \frac{q(i)}{m}) + 1 \\ &= 3 \log_2(\frac{m}{q(i)}) + 1. \end{aligned}$$

Die gesamten amortisierten Zugriffskosten sind also höchstens von der Größenordnung

$$\sum_{i=1}^N q(i) (3 \log_2(\frac{m}{q(i)}) + 1) = O(m + \sum_{i=1}^N q(i) \log(\frac{m}{q(i)})).$$

Da sich durch eine einzelne Zugriffsooperation auf Schlüssel i der Kontostand höchstens um $\log W - \log w_i$ verändern kann, ergibt sich als Gesamtveränderung nach m Operationen höchstens der Betrag

$$\sum_{i=1}^N q(i) \log\left(\frac{W}{w_i}\right) = \sum_{i=1}^N q(i) \log\left(\frac{m}{q(i)}\right).$$

Damit folgt die Behauptung des Satzes. □

Wir vergleichen das Ergebnis mit den Suchkosten eines optimalen Suchbaumes, also eines Suchbaumes, der die minimalen Suchkosten unter allen (statischen) Suchbäumen für N Schlüssel hat, so daß mit der Häufigkeit $q(i)$ auf Schlüssel i zugegriffen wird und $\sum_{i=1}^N q(i) = m$ ist. Die Suchkosten eines jeden Suchbaumes sind definiert durch

$$\sum_{i=1}^N q(i)(\text{Tiefe}(i) + 1) = \sum_{i=1}^N q(i) + \sum_{i=1}^N q(i)\text{Tiefe}(i).$$

Dabei ist $\text{Tiefe}(i)$ der Abstand des Schlüssels i von der Wurzel des Baumes.

Mit Hilfe von Argumenten aus der Informationstheorie kann man nun zeigen, daß in einem optimalen Suchbaum die Tiefe eines Schlüssels i , auf den mit der relativen Häufigkeit $q(i)/m$ -mal zugegriffen wird, wenigstens von der Größenordnung $\log(m/q(i))$ sein muß. D.h. es werden in einem solchen Baum zwar Schlüssel, auf die häufiger zugegriffen wird, näher bei der Wurzel sein können, als solche, auf die seltener zugegriffen wird. Dennoch müssen die Schlüssel aufgrund der Binärstruktur den angegebenen Mindestabstand zur Wurzel haben. Aus diesen Überlegungen folgt, daß Splay-Bäume sich „von selbst“ optimalen Suchbäumen anpassen: Obwohl die Zugriffshäufigkeiten nicht bekannt sind, sorgt das Splay-Verfahren dafür, daß durch Zugriffsoperationen Suchbäume entstehen, deren Suchkosten sich von denen entsprechender optimaler Suchbäume (für bekannte Zugriffshäufigkeiten) nur um einen konstanten Faktor unterscheiden. Damit haben Splay-Bäume eine Eigenschaft, die völlig analog ist zu selbstanordnenden linearen Listen, die nach der Move-to-front-Regel manipuliert werden, vgl. hierzu Abschnitt 3.3.

5.5 B-Bäume

Ohne es explizit zu sagen, sind wir in den Abschnitten 5.1 und 5.2 davon ausgegangen, daß die als natürliche oder balancierte Bäume strukturierten Datenmengen vollständig im Hauptspeicher Platz finden. Nicht selten hat man es aber mit Datenmengen zu tun, die nicht mehr im Hauptspeicher des jeweils vorhandenen Rechners gehalten werden können. Sie müssen dann auf sogenannten Hintergrundspeichern, wie Magnetbändern, Magnetplatten oder Disketten, abgelegt werden. Nur die jeweils aktuell etwa für eine Änderungsoperation benötigten Daten werden bei Bedarf vom Hintergrundspeicher in den Hauptspeicher geladen. Man spricht in diesem Fall üblicherweise von Dateien und faßt die Menge der Dienstprogramme zur Handhabung von Dateien zu einem Dateiverwaltungssystem zusammen. Wenn man eine Datei wie eine Internspeicherstruktur, also etwa als AVL-Baum, strukturiert und die Knoten dieses Baumes mehr oder weniger beliebig auf der Magnetplatte, der Diskette oder einem anderen Hintergrundspeicherme-

dium ablegt, so wird man im allgemeinen keineswegs ähnlich effizient suchen, einfügen und entfernen können wie bei interner Speicherung der Datei. Denn zwischen interner Speicherung und Speicherung auf Hintergrundspeichern bestehen grundlegende Unterschiede, die wir zunächst genauer erläutern wollen. Als Ergebnis unserer Überlegungen wird sich ergeben, daß eine spezielle Art von Vielwegbäumen, sogenannte B-Bäume, eine für auf Hintergrundspeichern abgelegte Dateien gut geeignete Organisationsform sind.

Eine Datei besteht aus einzelnen Datensätzen. Die Datei der Studenten an der Universität Freiburg besteht beispielsweise aus in einzelne Felder unterteilten Sätzen, die alle für die Universitätsverwaltung relevanten Daten über die jeweiligen Studenten enthalten. Jedes Feld hat eine bestimmte Bedeutung. Man nennt es daher auch *Attribut*.

Beispiel: Studentendatei

Felder	:	Feld 1	Feld 2	Feld 3	Feld 4
Attribute	:	Matr.Nr.	Name	Fach	Semester
Sätze	:	(4711,	Elvira Schön,	Chemie,	14)
		(007,	Hubert Stahl,	Mikrosystemtechnik,	3)
		(1010,	Monika Bit,	Informatik,	1)

Ein Satzfeld, das zur Identifizierung eines Satzes in einer Operation dient, wird auch *Satzschlüssel* genannt. Wir setzen (wie bisher stets) voraus, daß die Sätze über einen *ganzzahligen* Schlüssel identifiziert werden können. Im Beispiel der Studentendatei kann die Matrikelnummer als Schlüssel genommen werden. Da wir annehmen, daß die Datei auf einem Hintergrundspeicher abgelegt ist, stellt sich natürlich die Frage, woher das Dateiverwaltungssystem weiß, wo ein Satz mit gegebenem Schlüssel auf dem Hintergrundspeicher zu finden ist.

Wir setzen voraus, daß der zur Verfügung stehende Hintergrundspeicher ein Medium mit *direktem Zugriff* ist (z.B. eine Magnetplatte oder Diskette, aber kein Magnetband, das nur *sequentiellen Zugriff* erlaubt). Damit ist folgendes gemeint. Die Oberfläche der Magnetplatte oder Diskette ist durch konzentrische Kreise in *Spuren* und durch Kreis-ausschnitte in *Sektoren* geteilt. Hierdurch ist die Magnetplatte oder Diskette in direkt adressierbare *Blöcke* gegliedert. Die *Adresse eines Blocks* ist durch seine Spur- und Sektornummer gegeben. Wir nehmen an, daß in jedem Block ein oder mehrere Sätze der Datei gespeichert werden können. Der Dateiverwaltung steht nun permanent eine Tabelle im Hauptspeicher zur Verfügung, in der niedergelegt ist, unter welcher Blockadresse ein durch seinen Schlüssel identifizierter Satz zu finden ist. Diese Tabelle ist ein vollständiges Inhaltsverzeichnis der auf der Magnetplatte oder Diskette abgelegten Datei und wird als *Indextabelle* (kurz: *Index*) bezeichnet. Erhält die Dateiverwaltung etwa den Auftrag, einen Satz mit bestimmtem Schlüssel zu holen, durchsucht sie den Index, um die Blockadresse des Satzes mit diesem Schlüssel festzustellen; die Blockadresse wird dann zur Positionierung des Schreib-Lesekopfes benutzt und der Block in den Hauptspeicher geladen. Das Suchen im Index geht relativ schnell, da es im Hauptspeicher stattfindet und der Index beispielsweise als geordneter Binärbaum organisiert sein kann. Das Positionieren des Schreib-Lesekopfes auf eine bestimmte Blockadresse und das *Laden*, d. h. das Übertragen eines Blocks oder mehrerer aufeinanderfolgender Blöcke vom Hintergrundspeicher benötigt jedoch um Größenordnungen (bis zu 10000

mal) mehr Zeit als eine Suche nach einem Schlüssel im Hauptspeicher. Schwierig wird es nun, wenn der Index so groß ist, daß er nicht im Hauptspeicher Platz hat. Denn dann müssen offenbar Teile des Index wie die Datei selbst auf dem Hintergrundspeicher gehalten werden; nur ein Teil des Index ist im Hauptspeicher *resident*. Dann kann folgender Fall eintreten. Der Benutzer fordert den Zugriff auf einen Satz, dessen Schlüssel aber gerade nicht im residenten Teil des Index zu finden ist. Dann müssen Teile des auf dem Hintergrundspeicher befindlichen Index in den Hauptspeicher geholt werden. Dabei ist es natürlich wünschenswert, nur die richtigen Teile laden zu müssen. In jedem Fall sollte die Anzahl der erforderlichen Hintergrundspeicherzugriffe klein sein, weil sie erhebliche Zeit beanspruchen. Eine gute Möglichkeit zur Lösung dieser Probleme ist es, den Index hierarchisch als Baum, eben als B-Baum zu organisieren.

Dazu denkt man sich den gesamten Index in einzelne *Seiten* unterteilt. Jede Seite enthält eine bestimmte Anzahl von Indexelementen. Die Seiten sind zusammenhängend auf der Magnetplatte oder der Diskette gespeichert. Die Größe der Seiten ist so gewählt, daß mit einem Platten- (oder Disketten-) zugriff genau eine Seite in den Hauptspeicher geladen werden kann. So kann die Seitengröße beispielsweise der Blockgröße entsprechen. Dann kann der gesamte Index auch als Folge von Blöcken angesehen werden, in denen die Seiten des Index gespeichert sind. Jede Seite enthält aber nicht nur einen Teil des Index, sondern darüber hinaus Zusatzinformationen, aus denen das Dateiverwaltungssystem ermitteln kann, welche Seite neu in den Hauptspeicher zu laden ist, wenn der gesuchte Schlüssel nicht in dem gerade residenten Teil des Index zu finden ist. Diese Zusatzinformationen sind natürlich ebenfalls Blockadressen und damit *Zeiger* auf andere Teile des Index. Da in Abhängigkeit vom gesuchten, aber nicht gefundenen Schlüssel auf verschiedene Seiten verzweigt werden kann, ist es ganz natürlich, sich den Index hierarchisch aufgebaut als einen Vielwegbaum vorzustellen. Die Knoten entsprechen den Seiten; jeder Knoten enthält Schlüssel und Zeiger auf weitere Knoten. Durch zusätzliche Forderungen an die Struktur dieser Bäume sorgt man dafür, daß sich die typischen Wörterbuchoperationen, d.h. das Suchen, Einfügen und Entfernen von Schlüsseln (genauer: von durch ihre Schlüssel identifizierten Datensätzen) effizient ausführen lassen. Damit ist die den B-Bäumen zugrunde liegende Idee grob skizziert. Zur präzisen Definition sehen wir zunächst von der bei der Speicherung von Schlüsseln einzuhaltenden Anordnung der Schlüssel untereinander ab und beschreiben nur die B-Bäume charakterisierenden strukturellen Eigenschaften.

Ein *B-Baum der Ordnung m* ist ein Baum mit folgenden Eigenschaften:

- (1) Alle Blätter haben die gleiche Tiefe.
- (2) Jeder Knoten mit Ausnahme der Wurzel und der Blätter hat wenigstens $m/2$ Söhne.
- (3) Die Wurzel hat wenigstens 2 Söhne.
- (4) Jeder Knoten hat höchstens m Söhne.
- (5) Jeder Knoten mit i Söhnen hat $i - 1$ Schlüssel.

Bemerkung: Die Terminologie im Zusammenhang mit B-Bäumen ist in der Literatur nicht ganz einheitlich. Man spricht manchmal von B-Bäumen der Ordnung k und fordert statt der zweiten Bedingung, daß jeder innere Knoten außer der Wurzel wenigstens $k + 1$ Söhne haben muß, und statt der vierten Bedingung, daß jeder Knoten höchstens $2k + 1$ Söhne haben darf. Wir haben die Terminologie von D. Knuth übernommen, da sie zu dem zu Beginn dieses Kapitels eingeführten Begriff der *Ordnung* eines Baumes paßt.

B-Bäume der Ordnung 3 heißen auch 2-3-Bäume; ganz allgemein könnte man B-Bäume der Ordnung m in sinnvoller Weise auch $m/2$ - m -Bäume nennen, weil jeder innere Knoten mit Ausnahme der Wurzel mindestens $m/2$ und höchstens m Söhne hat.

Deuten wir einen Schlüssel einfach durch einen Punkt an, so zeigt Abbildung 5.44 das Beispiel eines 2-3-Baumes, also eines B-Baumes der Ordnung 3.

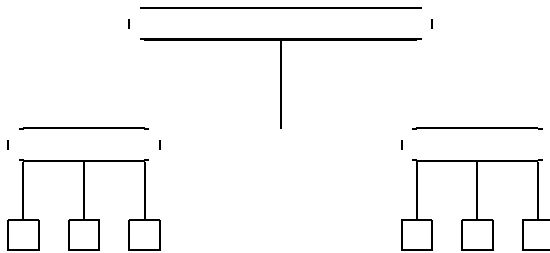


Abbildung 5.44

Dieser Baum hat sieben Schlüssel und acht Blätter. Die Anzahl der Blätter ist also um 1 größer als die Anzahl der Schlüssel. Das ist natürlich kein Zufall, sondern eine einfache Folgerung aus den die Struktur von B-Bäumen bestimmenden Bedingungen (1) – (5). Das beweist man durch Induktion über die Höhe von B-Bäumen. Hat der Baum die Höhe 1, so besteht er aus der Wurzel und k Blättern mit $2 \leq k \leq m$. Er muß dann wegen Bedingung 5. $k - 1$ Schlüssel haben. Sind $t_1, \dots, t_l$, $2 \leq l \leq m$, die l Teilbäume gleicher Höhe h eines B-Baumes mit Höhe $h + 1$ und jeweils $n_1, \dots, n_l$ Blättern und nach Induktionsvoraussetzung jeweils $(n_1 - 1), \dots, (n_l - 1)$ Schlüsseln, so muß die Wurzel wegen Bedingung 5. $l - 1$ Schlüssel haben. Der Baum hat damit wiederum insgesamt $\sum_{i=1}^l n_i$ Blätter und $\sum_{i=1}^l (n_i - 1) + l - 1 = \sum_{i=1}^l n_i - 1$ Schlüssel gespeichert.

Um die Anzahl der in einem B-Baum mit gegebener Höhe h gespeicherten Schlüssel abzuschätzen, genügt es also, die Anzahl seiner Blätter abzuschätzen. Ein B-Baum der Ordnung m mit gegebener Höhe h hat die minimale Blattzahl, wenn seine Wurzel nur 2 und jeder andere innere Knoten nur $m/2$ Söhne hat. Daher ist die minimale Blattzahl

$$N_{\min} = 2 \cdot \left(\frac{m}{2}\right)^{h-1}.$$

Die Blattzahl wird maximal, wenn jeder innere Knoten die maximal mögliche Anzahl m von Söhnen hat. Daher ist die maximale Blattzahl

$$N_{max} = m^h.$$

Ist umgekehrt ein B-Baum mit N Schlüsseln gegeben, so hat er $(N + 1)$ Blätter. Hat der Baum die Höhe h , so muß gelten:

$$N_{min} = 2^{\frac{m}{2}^{h-1}} \quad (N + 1) \leq m^h = N_{max}$$

Also:

$$h \geq 1 + \log_{\lceil \frac{m}{2} \rceil} \left(\frac{N + 1}{2} \right) \quad \text{und} \quad h \leq \log_m (N + 1).$$

Wir haben also wieder die für eine Klasse balancierter Bäume typische Eigenschaft, daß die Höhe eines B-Baumes logarithmisch in der Anzahl der gespeicherten Schlüssel beschränkt ist. Da die Ordnung m eines B-Baumes üblicherweise etwa bei 100 bis 200 liegt, sind B-Bäume besonders niedrig. Ist etwa $m = 199$, so haben B-Bäume mit bis zu 1999999 Schlüsseln höchstens die Höhe 4.

Wir haben bisher nichts über die Anordnung der Schlüssel in den Knoten eines B-Baumes vorausgesetzt. Für das Suchen, Einfügen und Entfernen von Schlüsseln ist sie natürlich von ausschlaggebender Bedeutung.

Ist p ein innerer Knoten eines B-Baumes der Ordnung m , so hat p l Schlüssel und $(l + 1)$ Söhne, $m/2 \leq l + 1 \leq m$. Es ist zweckmäßig, sich vorzustellen, daß die l Schlüssel $s_1, \dots, s_l$ und die $(l + 1)$ Zeiger $p_0, \dots, p_l$ auf die Söhne von p wie in Abbildung 5.45 innerhalb des Knotens p angeordnet sind.

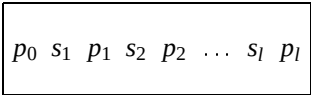


Abbildung 5.45

Dem Schlüssel s_i werden die Zeiger p_{i-1} und p_i zugeordnet, wobei p_{i-1} ein Zeiger auf den $(i - 1)$ -ten und p_i ein Zeiger auf den i -ten Sohn von p ist; der i -te Sohn von p (bzw. der $(i - 1)$ -te Sohn) ist die Wurzel des Teilbaums T_{p_i} (bzw. $T_{p_{i-1}}$).

Das Knotenformat eines B-Baumes der Ordnung m kann also in Pascal wie folgt vereinbart werden:

```
const
    m = Ordnung des B-Baumes ;
type
    Knotenzeiger = Knoten;
    Knoten = record
        Sohnzahl l : 0 .. m;
```

Schlüssel $s : \mathbf{array} [1 \dots m] \text{ of integer};$
Sohn $p : \mathbf{array} [0 \dots m] \text{ of Knotenzeiger}$
end;

Man verlangt nun zusätzlich zu den bereits angegebenen Bedingungen (1) – (5) die folgende Anordnung der Schlüssel:

- (6) Für jeden Knoten p mit l Schlüsseln $s_1, \dots, s_l$ und $(l + 1)$ Söhnen $p_0, \dots, p_l$ ($m/2 \leq l + 1 \leq m$) gilt: Für jedes i , $1 \leq i \leq l$, sind alle Schlüssel in $T_{p_{i-1}}$ kleiner als s_i , und s_i wiederum ist kleiner als alle Schlüssel in T_{p_i} .

Das ist die natürliche Erweiterung der von binären Suchbäumen wohlbekannten Ordnungsbeziehung auf Vielwegbäume. (Natürlich haben wir auch hier wieder stillschweigend vorausgesetzt, daß sämtliche Schlüssel paarweise verschieden sind.)

Das Beispiel in Abbildung 5.46 zeigt einen B-Baum der Ordnung 3, der die Schlüsselmenge $\{1, 3, 5, 6, 7, 12, 15\}$ speichert.

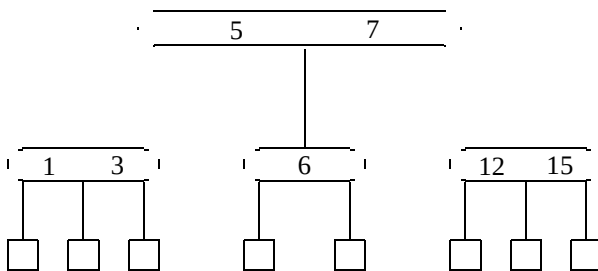


Abbildung 5.46

5.5.1 Suchen, Einfügen und Entfernen in B-Bäumen

Das *Suchen* nach einem Schlüssel x in einem B-Baum der Ordnung m kann als natürliche Verallgemeinerung des von binären Suchbäumen bekannten Verfahrens aufgefaßt werden. Man beginnt bei der Wurzel und stellt zunächst fest, ob der gesuchte Schlüssel x einer der im gerade betrachteten Knoten p gespeicherten Schlüssel $s_1, \dots, s_l$, $1 \leq l \leq m - 1$, ist. Ist das nicht der Fall, so bestimmt man das kleinste i , $1 \leq i \leq l$, für das $x < s_i$ ist, falls es ein solches i gibt; sonst ist $x > s_l$. Im ersten Fall setzt man die Suche bei dem Knoten fort, auf den der Zeiger p_{i-1} zeigt; im letzten Fall folgt man dem Zeiger p_l . Das wird solange fortgesetzt, bis man den gesuchten Schlüssel gefunden hat oder die Suche in einem Blatt erfolglos endet. Es ist klar, daß man im schlechtesten Fall höchstens alle Knoten auf einem Pfad von der Wurzel zu einem Blatt betrachten muß.

Wir lassen offen, wie die Suche nach x innerhalb eines Knotens p mit den Schlüsseln $s_1, \dots, s_l$ und den Zeigern $p_0, \dots, p_l$ erfolgt. Um dasjenige i zu finden, für das $x = s_i$ gilt, bzw. das kleinste i zu bestimmen, für das $x < s_i$ ist, bzw. festzustellen, daß $x > s_l$ ist, kann man beispielsweise sowohl lineares als auch binäres Suchen verwenden. Da diese Suche in jedem Fall im Internspeicher stattfindet, beeinflußt sie die Effizienz des gesamten Suchverfahrens weit weniger als die Anzahl der Knoten, die betrachtet werden müssen, die ja unmittelbar mit der Zahl der bei der Suche nach x erforderlichen Hintergrundspeicherzugriffe zusammenhängt.

Um einen neuen Schlüssel x in einen B-Baum *einzuügen*, sucht man zunächst im Baum nach x . Da x im Baum noch nicht vorkommt, endet die Suche erfolglos in einem Blatt, das die erwartete Position des Schlüssels x repräsentiert. Sei der Knoten p der Vater dieses Blattes. Der Knoten p habe die Schlüssel $s_1, \dots, s_l$ gespeichert, und die Suche nach x ende beim Blatt, auf das der Zeiger p_i zeigt, $0 \leq i \leq l$. Dann sind zwei Fälle möglich:

Fall 1: Der Knoten p hat noch nicht die maximal zulässige Anzahl $m - 1$ von Schlüsseln gespeichert. In diesem Fall fügt man x in p zwischen s_i und s_{i+1} ein (bzw. vor s_1 , falls $i = 0$, und nach s_l , falls $i = l$), schafft ein neues Blatt, und nimmt in p einen neuen Zeiger auf dieses Blatt auf. Der Einfügevorgang (vgl. Abbildung 5.47) ist damit beendet.

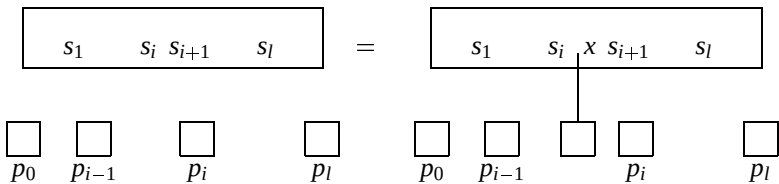
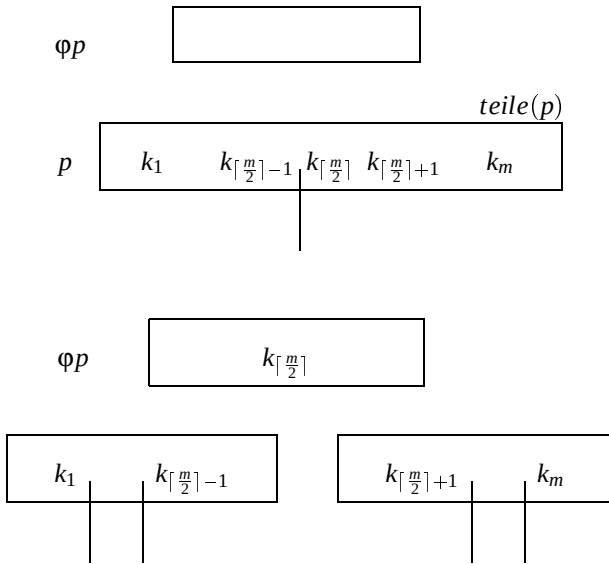


Abbildung 5.47

Fall 2: Der Knoten p hat bereits die maximal zulässige Anzahl $m - 1$ von Schlüsseln gespeichert. In diesem Fall ordnen wir den Schlüssel x seiner Größe entsprechend unter die $m - 1$ Schlüssel von p ein, schaffen, wie vorher im Fall 1, ein neues Blatt und teilen nun den zu großen Knoten mit m Schlüsseln und $m + 1$ Blättern als Söhne in der Mitte auf. D.h.: Sind $k_1, \dots, k_m$ die Schlüssel in aufsteigender Reihenfolge (also die in p zuvor bereits gespeicherten $m - 1$ Schlüssel und der neu eingefügte Schlüssel x), so bildet man zwei neue Knoten, die jeweils die Schlüssel $k_1, \dots, k_{\lfloor m/2 \rfloor - 1}$ und $k_{\lfloor m/2 \rfloor + 1}, \dots, k_m$ enthalten, und fügt den mittleren Schlüssel $k_{\lfloor m/2 \rfloor}$ auf dieselbe Weise in den Vater des Knotens p ein. Dieses *Teilen eines überlaufenden Knotens* wird solange rekursiv längs eines Pfades zurück von den Blättern zur Wurzel wiederholt, bis ein Knoten erreicht ist, der noch nicht die Maximalzahl von Schlüsseln gespeichert hat, oder bis die Wurzel erreicht ist. Muß die Wurzel geteilt werden, so schafft man eine neue Wurzel, die die durch Teilung entstehenden Knoten als Söhne und den vor der Teilung mittleren Schlüssel als einzigen Schlüssel hat. Der Vorgang des Teilens eines überlaufenden Knotens ist in Abbildung 5.48 dargestellt.



und $teile(\phi p)$, falls ϕp (nach Einfügen von $k_{\lceil \frac{m}{2} \rceil}$) m Schlüssel hat

Abbildung 5.48

Es ist klar, daß man im ungünstigsten Fall dem Suchpfad von den Blättern zurück zur Wurzel folgen und jeden Knoten auf diesem Pfad teilen muß. Daraus ergibt sich sofort, daß das Einfügen eines neuen Schlüssels in einen B-Baum der Ordnung m mit N Schlüsseln (und $N + 1$ Blättern) in $O(\log_{\lceil m/2 \rceil} (N + 1))$ Schritten ausführbar ist.

Wir verfolgen ein Beispiel und fügen in den in Abbildung 5.46 gezeigten B-Baum der Ordnung 3 den Schlüssel 14 ein. Dazu zeigen wir die Situation in den Abbildungen 5.49–5.51 jeweils unmittelbar vor der Teilung eines Knotens; ein überlaufender, also zu teilender Knoten ist jeweils durch einen ϕ markiert.

Zum *Entfernen* eines Schlüssels aus einem B-Baum der Ordnung m geht man umgekehrt vor. Man sucht den Schlüssel im Baum, entfernt ihn und verschmilzt gegebenenfalls einen Knoten mit einem Bruder, wenn er nach Entfernen eines Schlüssels *unterläuft*, also weniger als $\frac{m}{2} - 1$ Schlüssel gespeichert hat. Ein Unterlauf der Wurzel, die ja nur einen Schlüssel gespeichert haben muß, bedeutet natürlich, daß die Wurzel keinen Schlüssel mehr gespeichert und nur noch einen einzigen Sohn hat. Man kann dann die Wurzel entfernen und den einzigen Sohn zur neuen Wurzel machen. Wir überlassen die Ausführung der Details dem interessierten Leser und weisen lediglich auf die Ähnlichkeit zum Entfernen von Schlüsseln aus 1-2-Bruder-Bäumen hin. Wie dort muß man das Entfernen eines Schlüssels eines inneren Knotens aus einem B-Baum zunächst auf das Entfernen eines Schlüssels unmittelbar oberhalb der Blätter reduzieren. Dann wird man den Fall, daß man zum Auffüllen eines unterlaufenden Knotens einen Schlüssel von einem Bruder dieses Knotens borgen kann, anders behandeln als den Fall, daß ein unterlaufender Knoten nur (unmittelbare) Brüder hat, die die Minimalzahl von

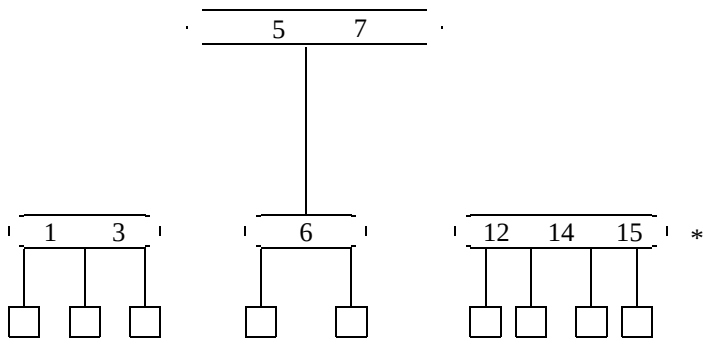


Abbildung 5.49

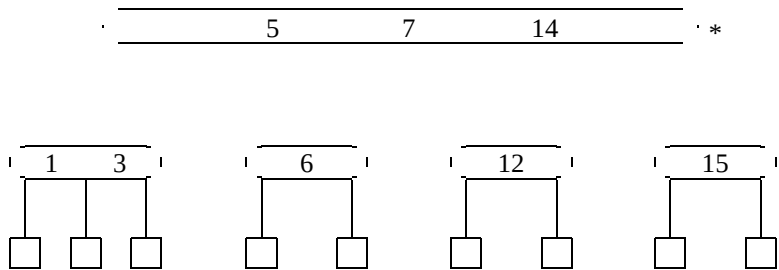


Abbildung 5.50

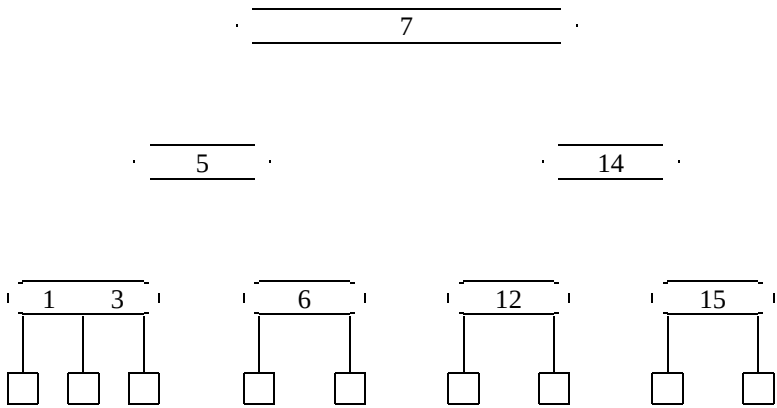


Abbildung 5.51

Schlüsseln gespeichert haben. In diesem Fall kann der Knoten mit einem Bruder verschmolzen werden. Es ist nicht schwer zu sehen, daß das Entfernen eines Schlüssels aus einem B-Baum der Ordnung m mit N Schlüsseln stets in $O(\log_{\lceil m/2 \rceil} (N + 1))$ Schritten ausführbar ist.

B-Bäume sind also eine weitere Möglichkeit zur Implementation von Wörterbüchern, die es gestattet, jede der drei Operationen Suchen, Einfügen und Entfernen von Schlüsseln in logarithmischer Zeit in der Anzahl der Schlüssel auszuführen. Das Verhalten im Mittel ist besser. Wie im Falle von 1-2-Bruder-Bäumen gilt auch hier, daß die Gesamtzahl der ausgeführten Knotenteilungen für eine Folge iterierter Einfügungen linear mit der Anzahl der insgesamt erzeugten Knoten zusammenhängt. Weil ein B-Baum der Ordnung m , der N Schlüssel gespeichert hat, höchstens

$$\frac{N}{\frac{m}{2}} \frac{1}{1} + 1$$

innere Knoten haben kann, ist die mittlere Anzahl von Teilungsoperationen konstant, wenn man über eine Folge von N Einfügeoperationen in den anfangs leeren Baum mittelt, obwohl natürlich eine einzelne Einfügeoperation $\Omega(\log_{\lceil m/2 \rceil} N)$ Knotenteilungen erfordern kann.

Erwartungswerte für die in einem Knoten gespeicherte Schlüsselzahl, also für die *Speicherplatzausnutzung* eines B-Baumes der Ordnung m und weitere das Einfügeverfahren charakterisierende Parameter kann man mit Hilfe der Fringe-Analysetechnik berechnen (vgl. [10]). Es ergibt sich, daß man (unabhängig von m) eine Speicherplatzausnutzung von $\ln 2 \approx 69\%$ erwarten kann, wenn man eine zufällig gewählte Folge von N Schlüsseln in den anfangs leeren B-Baum der Ordnung m einfügt, d h. die Knoten des entstehenden B-Baumes sind nur zu gut $2/3$ gefüllt.

Fügt man Schlüssel in auf- oder absteigend sortierter Reihenfolge in den anfangs leeren B-Baum ein, entstehen B-Bäume mit besonders schlechter Speicherplatzausnutzung. Die Knoten sind (in allen Fällen, in denen $N = 2^{\frac{m}{2}h}$ ist) *minimal* gefüllt, d.h. die Wurzel hat nur einen und jeder andere innere Knoten nur $\frac{m}{2} - 1$ Schlüssel. B-Bäume verhalten sich also gerade anders als 1-2-Bruder-Bäume: B-Bäume werden besonders dünn, 1-2-Bruder-Bäume aber besonders dicht, wenn man Schlüssel in auf- oder absteigend sortierter Reihenfolge einfügt.

Es gibt verschiedene Vorschläge, die schlechte Speicherplatzausnutzung von B-Bäumen zu verhindern. Man kann (wie bei 1-2-Bruder-Bäumen) zunächst die unmittelbaren oder gar alle Brüder eines überlaufenden Knotens daraufhin untersuchen, ob man ihnen nicht Schlüssel abgeben kann, bevor man den Knoten teilt und den mittleren Schlüssel und damit eventuell auch das Überlaufproblem auf das nächsthöhere Niveau verschiebt (vgl. hierzu ). Andere Vorschläge zielen darauf ab, für eine Folge bereits sortierter Schlüssel B-Bäume nicht durch iteriertes Einfügen in den anfangs leeren Baum zu erzeugen, sondern möglichst optimale Anfangsstrukturen zu erzeugen in der Hoffnung, daß nachfolgende Einfügungen oder Entfernungen von Schlüsseln den Baum höchstens allmählich, d.h. für eine große Zahl solcher Operationen, stark vom Optimum abweichen lassen.

5.6 Weitere Klassen

Neben den in 5.2 und 5.5 genannten Beispielen für Klassen balancierter Bäume findet man in der Literatur zahlreiche weitere Vorschläge. Allen Klassen gemeinsam ist die Eigenschaft, daß durch die jeweils geforderte Balance-Bedingung eine Klasse von Bäumen definiert wird, deren Höhe logarithmisch in der Knotenzahl bleibt. Sonst werden aber sehr unterschiedliche Ziele verfolgt. Wir geben zunächst eine grobe Übersicht und besprechen dann zwei Aspekte genauer.

5.6.1 Übersicht

Dichte Bäume

Wie wir bereits gesehen haben, besitzen Bruder-Bäume und B-Bäume im allgemeinen mehr Knoten als zur Speicherung einer Menge von Schlüsseln unbedingt notwendig ist. Man kann mit Hilfe der Technik der Fringe-Analyse zeigen, daß man in beiden Fällen eine Speicherplatzausnutzung von etwa 70% für „zufällig“ erzeugte Bäume erwarten kann. Verschiedene Vorschläge zielen darauf ab, *dichte* balancierte Bäume zu erhalten, die vollständigen Bäumen nahekommen. D.h. sie sollen geringe Höhe und keine „überflüssigen“ Knoten haben, aber natürlich soll das Einfügen und Entfernen von Schlüsseln immer noch in logarithmischer Schrittzahl ausführbar sein.

Es ist intuitiv klar, wie man das erreichen kann. Man bezieht in die Umstrukturierungen immer größere Umgebungen (Nachbarn von Knoten auf demselben Niveau, größere „Verwandtschaften“ von Knoten auch auf verschiedenen Niveaus) in die Betrachtungen ein. Das Einfüge- oder Entferne-Problem wird erst dann rekursiv — analog zu Bruder-Bäumen und B-Bäumen — auf das nächsthöhere Niveau verschoben, wenn es sich in der fixierten größeren Umgebung nicht lösen läßt. Die Arbeiten [10] [11] zeigen, daß man auf diese Weise vollständigen Bäumen beliebig nahekommen kann und asymptotisch eine Speicherplatzausnutzung von 100% erreicht. Natürlich hängt die Komplexität der zum Rebalancieren erforderlichen Umstrukturierungsalgorithmen von der Größe der jeweils betrachteten Umgebung ab. Je mehr Brüder oder Nachbarn eines Knotens man in die Betrachtung einbezieht, umso komplizierter werden die Einfüge- und Entferne-Verfahren. Andererseits werden aber die (durch iteriertes Einfügen) erzeugten Bäume auch immer dichter.

Reduktion der Balanceinformation

AVL-Bäume haben gegenüber gewichtsbalancierten Bäumen den großen Vorteil, daß die an jedem Knoten zur Sicherung der AVL-Ausgeglichenheit zu speichernde und zu überprüfende Balanceinformation sehr klein ist. Es genügt, sich einen von drei möglichen Werten 0, 1 oder -1 an jedem Knoten für die Höhendifferenz zwischen linkem und rechtem Teilbaum zu merken. An jedem Knoten eines gewichtsbalancierten Baumes muß man dagegen das Gewicht des gesamten Teilbaumes dieses Knotens, also eine prinzipiell nicht beschränkte Information mitführen. Es hat eine ganze Reihe von schließlich auch erfolgreichen Versuchen gegeben, „einseitig“ höhenbalancierte Bäume und Algorithmen mit logarithmischer Schrittzahl zum Einfügen und Entfernen von Schlüsseln für solche Bäume zu finden. Ein einseitig, z.B. linksseitig höhenbalancierter Binärbaum ist dabei charakterisiert durch die Eigenschaft, daß für jeden Knoten p des Baumes gilt: Die Höhen der beiden Teilbäume von p sind entweder gleich oder aber der linke Teilbaum von p ist um 1 höher als der rechte. Zur Speicherung der Höhendifferenz reicht also ein Bit an jedem Knoten aus. In [12] wurde ein in $O(\log^2 n)$ Schritten ausführbarer Einfügealgorithmus und in [13] ein in logarithmischer Schrittzahl, d.h. in $O(\log n)$ Schritten ausführbares Verfahren zum Entfernen von Schlüsseln für einseitig höhenbalancierte Bäume angegeben.

Man kann zu solchen Verfahren auch auf dem „Umweg“ über einseitige Bruderbäume kommen. Zunächst wird die Bedingung an die Verteilung der unären und binären Knoten in Bruderbäumen wie folgt verschärft. Wir verlangen, daß jeder unäre Knoten einen *rechten* Bruder haben soll mit zwei Söhnen. Für die so definierte Klasse von *Rechts-Bruder-Bäumen* kann man Verfahren zum Einfügen und Entfernen von Schlüsseln angeben, deren Laufzeit logarithmisch in der Knotenzahl ist (vgl. dazu [14]). Bruder-Bäume kann man als „expandierte“ höhenbalancierte Bäume und umgekehrt höhenbalancierte Bäume als durch Zusammenziehen unärer Knoten mit ihren jeweils einzigen Söhnen entstehende kontrahierte Bruder-Bäume auffassen (vgl. [15]). In [16] wird dieser Zusammenhang ausgenutzt und ein in logarithmischer Schrittzahl ausführbares Einfügeverfahren für einseitig höhenbalancierte Bäume angegeben. Auch in diesen Fällen kann man beobachten, daß eine Verschärfung der Balancebedingungen dazu führt, daß die Update-Verfahren komplizierter werden.

Wege zur Vereinheitlichung

Die große Vielfalt der in der Literatur zu findenden Klassen balancierter Bäume macht es schwer, die verschiedenen Klassen miteinander zu vergleichen. Man möchte ferner nicht für jede neue Variante einer Balancebedingung, also für jede neue Forderung an die statische Struktur von Bäumen, entsprechende Einfüge- und Entferne-Verfahren jedesmal neu erfinden. Es hat daher nicht an Versuchen gefehlt, möglichst viele Klassen balancierter Bäume in einem einheitlichen Rahmen zu behandeln. Zwei Vorschläge sind in diesem Zusammenhang bemerkenswert, die *Rot-schwarz-Bäume* von Guibas und Sedgwick [1] und das *Schichtenmodell* von van Leeuwen und Overmars [2]. Rot-schwarz-Bäume erlauben es, AVL-Bäume, B-Bäume und viele andere Klassen balancierter Bäume einheitlich zu repräsentieren und zu implementieren. Ein Rot-schwarz-Baum ist ein Binärbaum, dessen Kanten entweder rot oder schwarz sind. Die roten (auch: horizontalen) Kanten dienen dazu, Knoten mit mehr als zwei Nachfolgern, wie sie etwa in B-Bäumen vorkommen, binär zu repräsentieren; die schwarzen Kanten entsprechen den sonst üblichen Kanten zwischen Vätern und Söhnen. Knoten der Ordnung 3 und 4 kann man in diesem Rahmen wie in Abbildung 5.52 repräsentieren.

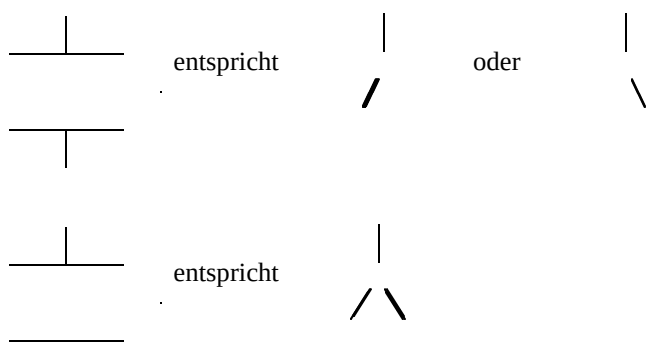


Abbildung 5.52: Rote Kanten sind dick, schwarze dünn gezeichnet.

Als Balancierungsbedingung wird dann verlangt, daß alle Pfade von der Wurzel zu einem Blatt dieselbe Anzahl von schwarzen Kanten haben — dabei werden nur die Kanten zwischen inneren Knoten gezählt. (Das entspricht offenbar der von B-Bäumen und Bruder-Bäumen bekannten Bedingung, daß alle Blätter denselben Abstand zur Wurzel haben müssen.) Weitere Balancebedingungen hängen davon ab, welche Baumklasse in diesem Rahmen repräsentiert werden soll. Will man etwa die Klasse der *2-3-4-Bäume* (das sind Bäume, bei denen jeder innere Knoten 2, 3 oder 4 Söhne hat) im Rahmen der Rot-schwarz-Bäume repräsentieren, so wird zusätzlich verlangt, daß kein Pfad von einem inneren Knoten zu einem Blatt zwei aufeinanderfolgende rote Kanten haben darf. Damit sind in einem 2-3-4-Baum nur die „roten“ Teilbäume aus Abbildung 5.53 möglich.

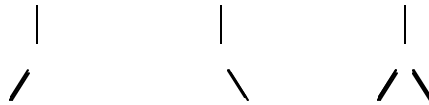
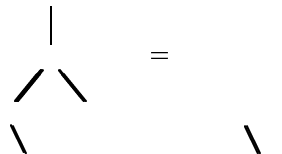


Abbildung 5.53

Ein neuer Knoten wird stets an der erwarteten Position unter den Blättern mit einer roten Kante angefügt. Dadurch kann es vorkommen, daß zwei rote Kanten aufeinanderfolgen. In einem solchen Fall wird eine *Rotation* oder ein *Farbwechsel* ausgeführt, ein Prozeß, der sich rekursiv bis zur Wurzel fortsetzen kann. Wir geben je ein Beispiel für diese Operationen an (siehe Abbildung 5.54); die nicht angegebenen symmetrischen Fälle sind analog zu behandeln.

Farbwechsel



(Doppel-)Rotation

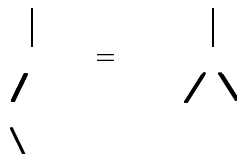
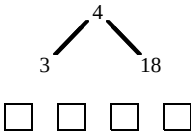


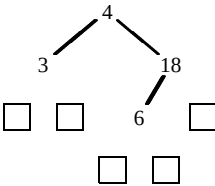
Abbildung 5.54

Wir zeigen am Beispiel der Schlüsselfolge 4, 3, 18, 6, 17, 10, 9, 11, wie mit Hilfe dieser Operationen 2-3-4-Bäumen entsprechende Rot-schwarz-Bäume erzeugt werden können.

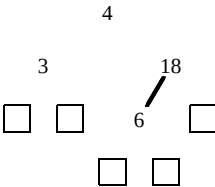
Nach Einfügen der Schlüssel 4, 3, 18 in den anfangs leeren Baum entsteht:



Einfügen des Schlüssels 6 an der erwarteten Position unter den Blättern ergibt zu-
nächst:

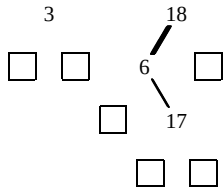


Ein Farbwechsel liefert den zulässigen Baum:

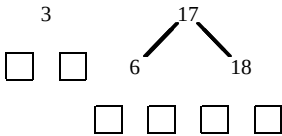


Wir geben die weitere Operationsfolge kurz an:

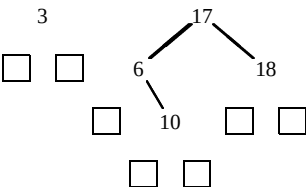
Einfügen von 17
4



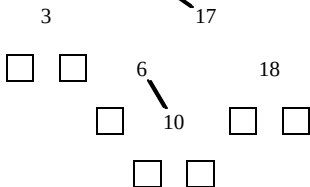
Rotation
4

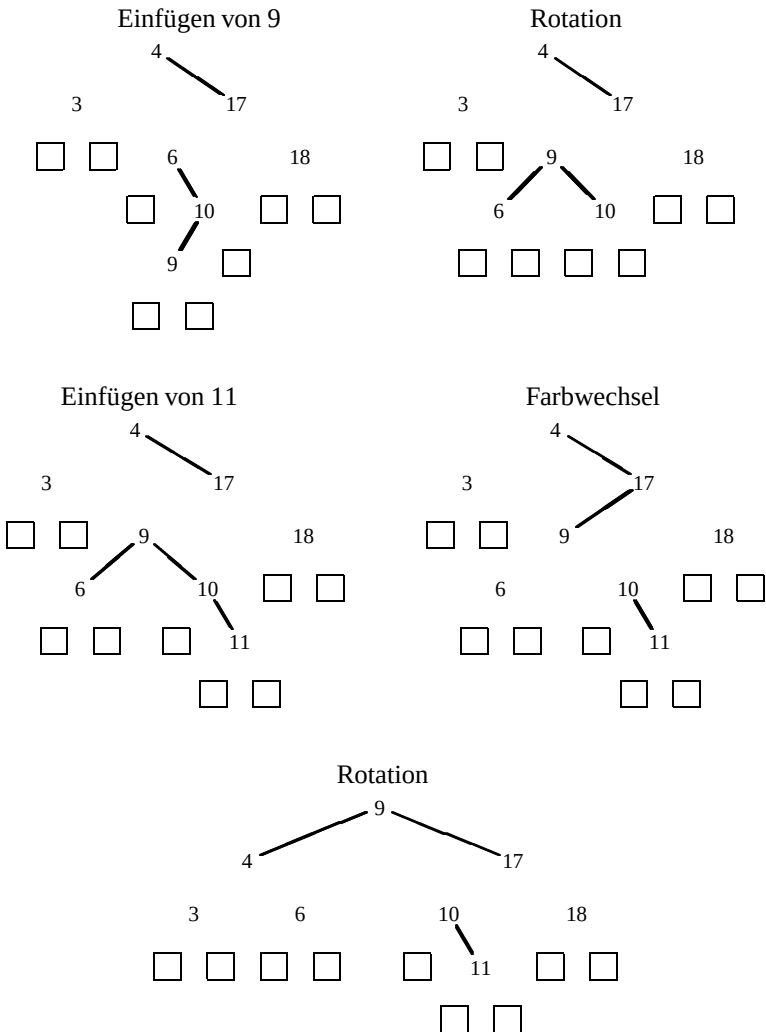


Einfügen von 10
4



Farbwechsel
4





Es ist nicht schwer zu sehen, daß die Operationen Farbwechsel und Rotation ausreichen, um aus einem gültigen, einem 2-3-4-Baum entsprechenden Rot-schwarz-Baum wieder einen solchen Baum zu machen, wenn man einen neuen Knoten wie beschrieben einfügt.

AVL-Bäume lassen sich als spezielle Bäume dieser Art auffassen, wenn man ihre Kanten richtig färbt. Definieren wir als Höhe eines Knotens die Länge des längsten Pfades von dem Knoten zu einem Blatt. Dann färbt man genau diejenigen Kanten rot, die von Knoten mit gerader Höhe zu Knoten mit ungerader Höhe führen. Es ist leicht zu zeigen, daß dadurch ein AVL-Baum zu einem speziellen gültigen 2-3-4-Baum in Rot-schwarz-Repräsentation wird.

Auch andere Klassen balancierter Bäume lassen sich in diesem Rahmen darstellen. Auf welche Weise eine Darstellung durch Rot-schwarz-Bäume möglich ist, muß man sich aber in jedem Fall gesondert überlegen.

Im nächsten Abschnitt stellen wir eine Variante des Schichtenmodells von van Leeuwen und Overmars [1] vor, das auf spezielle Bedürfnisse (konstante Zahl struktureller Änderungen pro Update und Entkopplung von Updates und Rebalancierung) zugeschnitten ist. Das Schichtenmodell ist ein Rahmen zur statischen Definition von Klassen balancierter Bäume. Man sorgt wie im Fall von höhen- oder gewichtsbalancierten Bäumen durch geeignete Strukturbedingungen dafür, daß Bäume mit N Blättern stets eine Höhe haben, die in $O(\log N)$ liegt.

Für die in [1] definierten Klassen balancierter Bäume ist leicht zu sehen, daß nicht jeder zur jeweiligen Klasse gehörender Baum durch iteriertes Einfügen von Schlüsseln in den anfangs leeren Baum erzeugt werden kann. Ob und gegebenenfalls welche Unterschiede zwischen einer statisch definierten Klasse von balancierten Bäumen und der Klasse aller Bäume bestehen, die durch Ausführen von Einfüge- oder Entferne-Operationen aus gegebenen Anfangsbäumen gewonnen werden können, ist für viele Klassen balancierter Bäume und zugehöriger Update-Verfahren noch offen (vgl. hierzu [1]).

5.6.2 Konstante Umstrukturierungskosten und relaxiertes Balancieren

Die bisher dargestellten Verfahren zum Ausgleichen von Bäumen nach dem Einfügen oder Entfernen eines Schlüssels in einem balancierten Suchbaum führen im schlechtesten Fall eine logarithmische Zahl struktureller Änderungen durch. Es kann vorkommen, daß man für sämtliche Knoten längs eines Pfades von den Blättern zur Wurzel Rotationen durchführen oder Knoten spalten bzw. verschmelzen muß. Wir stellen jetzt eine Klasse balancierter Bäume vor, die sich nach jeder Einfüge- oder Entferne-Operation durch endlich viele (höchstens drei) Rotationen wieder ausgleichen lassen. Eine Klasse von Bäumen dieser Art und Update-Verfahren für diese Klasse wurden erstmals von Olivé angegeben [2]. Einen anderen Vorschlag findet man in [3].

Außer dieser Eigenschaft, daß pro Update nur *konstante Umstrukturierungskosten* erforderlich sind, haben die in diesem Abschnitt definierten Bäume eine weitere, bemerkenswerte Eigenschaft: Sie eignen sich besonders gut für den Einsatz in *Mehrbenutzerumgebungen* oder Situationen, wo plötzlich eine sehr große Zahl von Updates erledigt werden muß, so daß möglicherweise nicht genug Zeit ist, um die erforderlichen Umstrukturierungen sogleich nach jeder einzelnen Update-Operation durchzuführen.

Ohne es explizit zu fordern, sind wir nämlich bisher stets stillschweigend davon ausgegangen, daß die drei Wörterbuchoperationen Suchen, Einfügen und Entfernen von Schlüsseln in Bäumen strikt nacheinander ausgeführt werden. Die jeweils nächste Operation darf erst begonnen werden, wenn die jeweils vorangehende vollständig abgeschlossen ist. Insbesondere dann, wenn mehrere Benutzer gleichzeitig konkurrierend auf eine als Baum strukturierte Menge von Daten zugreifen können, möchte man aber auch mehrere Such-, Einfüge- und Entferne-Prozesse gleichzeitig (englisch: *concurrent*) ausführen können. Solange nur Suchoperationen ausgeführt werden, gibt es dabei wenig Probleme. Denn so können durchaus mehrere Suchprozesse auf denselben Knoten zugreifen (Man muß die jeweils betrachteten Knoten nur für Schreibprozesse sperren). Man kann sich also eine Menge parallel ablaufender Suchprozesse in einem

Suchbaum vorstellen als einen Strom von voneinander unabhängigen, von oben (von der Wurzel) nach unten (zu den Blättern) verlaufenden Einzelprozessen, die sich nicht gegenseitig stören. Die nach dem Einfügen oder Entfernen von Schlüsseln insbesondere bei balancierten Bäumen durchgeführten Strukturänderungen können jedoch dazu führen, daß begonnene und noch nicht beendete Suchprozesse falsche Ergebnisse liefern. Es kann ferner vorkommen, daß parallel ablaufende strukturelle Änderungen nach einer Einfüge- oder Entferne-Operation sich gegenseitig stören. Wir erläutern dies an einem einfachen Beispiel. Nehmen wir an, in einem AVL-Baum wird eine Suche nach einem Schlüssel k begonnen, bevor eine vorangehende Einfüge- oder Entferne-Operation vollständig abgeschlossen wurde, die unter anderem eine Rotation bei einem Knoten q zur Wiederherstellung der AVL-Ausgeglichenheit ausführt, vgl. Abbildung 5.55.

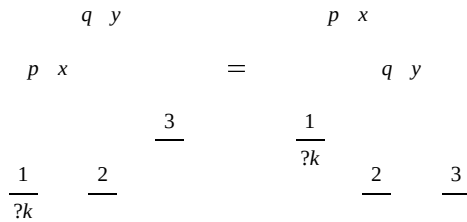


Abbildung 5.55

Nehmen wir an, der Prozeß des Suchens nach dem Schlüssel k sei auf dem Weg von der Wurzel abwärts beim Knoten q angelangt. Ein Schlüsselvergleich ergibt, daß nunmehr der linke Sohn von q betrachtet werden muß. Nehmen wir an, daß jetzt eine Rotation bei q ausgeführt wird, bevor der Suchprozeß fortgesetzt wird. Es folgt, daß die Suche nach k möglicherweise im falschen Teilbaum fortgesetzt wird.

Ähnliche Probleme treten bei nahezu allen Balancierungsverfahren auf. Es können sogar dann falsche Ergebnisse geliefert werden, wenn auf eine Balancierung verzichtet wird, wie bei natürlichen Bäumen. Entfernt man den Schlüssel eines inneren Knotens aus einem solchen Baum, muß er zunächst durch seinen symmetrischen Vorgänger oder Nachfolger ersetzt werden. „Überholt“ nun ein Such-Prozeß einen Entferne-Prozeß an einem solchen Knoten, bevor die Schlüssel ausgetauscht wurden, kann eine Suche falsch dirigiert werden. Wie wir weiter unten erläutern, kann man die beim Entfernen von Schlüsseln auftretenden Probleme aber dadurch umgehen, daß man Blattsuchbäume verwendet.

Es gibt verschiedene Vorschläge in der Literatur, ein reibungsloses, korrektes Miteinander verschiedener Such-, Einfüge- und Entferne-Prozesse sicherzustellen. Wir nennen einige Ansätze.

Sperrstrategien

Knoten, die von einer begonnenen, aber noch nicht abgeschlossenen Umstrukturierungsmaßnahme betroffen sein könnten, werden für nachfolgende Prozesse vorsorglich

gesperrt. Das Verfolgen einer naiven Sperrstrategie kann allerdings leicht dazu führen, daß etwa die Wurzel eines Baumes gesperrt werden muß und damit ein paralleles Abarbeiten mehrerer Prozesse praktisch unmöglich wird. Man findet jedoch in der Literatur eine große Zahl besserer, aber auch komplexerer Sperrstrategien.

Reine Top-down-Update-Verfahren

Es sind Update-Verfahren entwickelt worden, die wie Suchprozesse niemals bereits inspizierte und verlassene Knoten beeinflussen können. Statt also beispielsweise in einem B-Baum nach dem Einfügen eines Schlüssels einen Suchpfad von unten nach oben zurückzulaufen und dabei, falls nötig, überlaufende Knoten zu spalten, geht man so vor: Bereits bei der Suche nach einem neu einzufügenden Schlüssel werden „kritische“, d. h. die maximal mögliche Schlüsselzahl enthaltende Knoten vorsorglich gespalten. Man spart damit das Zurücklaufen längs des Suchpfades und kann gefahrlos mehrere Prozesse gleichzeitig ablaufen lassen. Es genügt, die jeweils gerade betrachteten oder zu spaltenden Knoten zu sperren, um eine konsistente Bearbeitung zu sichern.

Die Reduktion des Entfernens von Schlüsseln innerer Knoten auf das Entfernen des symmetrischen Nachfolgers oder Vorgängers kann es erfordern, Zeiger auf den Knoten weit oben im Baum stehenzulassen („dangling pointer“), die später erneut inspiziert werden müssen. Um ein reines Top-down-Vorgehen zu ermöglichen, betrachtet man daher Blattsuchbäume und wählt die „Wegweiser“ an den inneren Knoten so, daß sie auch nach dem Entfernen von Schlüsseln der Blätter stehenbleiben können, ohne daß nachfolgende Suchoperationen falsch geleitet werden.

Umstrukturierung als Hintergrundprozeß

Die nach dem Einfügen oder Entfernen von Schlüsseln in balancierten Suchbäumen unter Umständen erforderlichen Umstrukturierungen werden von den Update-Operationen abgekoppelt und als getrennte, im Hintergrund ablaufende, lokale, strukturelle Änderungsoperationen implementiert. Es wird also darauf verzichtet, nach jeder Einfüge- oder Entferne-Operation einen das jeweilige Balancierungskriterium erfüllenden Suchbaum wiederherzustellen. Vielmehr wird eine Anzahl von Umstrukturierungsprozessen generiert, die konkurrierend zu den eigentlichen Update-Operationen ausgeführt werden. Erst wenn alle diese Prozesse vollständig beendet sind, muß wieder ein balancierter Suchbaum vorliegen.

Man spricht in diesem Fall von *relaxiertem Balancieren*. Statt zu fordern, daß die Balance-Bedingung unmittelbar nach jeder Update-Operation wiederhergestellt wird, können die Umstrukturierungsoperationen nach Belieben zurückgestellt und nach Bedarf oder Möglichkeit mit den Such- und Update-Operationen verschränkt ausgeführt werden. In der Literatur findet man zahlreiche Vorschläge für relaxiertes Balancieren (vgl. z.B. [1], [2], [3], [4], [5]). Wir beschreiben jetzt eine besonders einfache und elegante Lösung aus [6].

Stratifizierte Bäume

Stratifizierte Bäume sind Blattsuchbäume, die aus verschiedenen *Schichten* (auch *Straßen* genannt) bestehen. Als Balancebedingung wird gefordert, daß alle Blätter denselben Abstand zur Wurzel haben müssen, wenn man nur die Anzahl der Straßen zählt. Sei nun Z die in Abbildung 5.56 gezeigte Menge von vier Binärbäumen mit den Höhen 1 und 2. Dann ist die Klasse der *Z-stratifizierten Bäume* die kleinste Klasse von Bäumen,

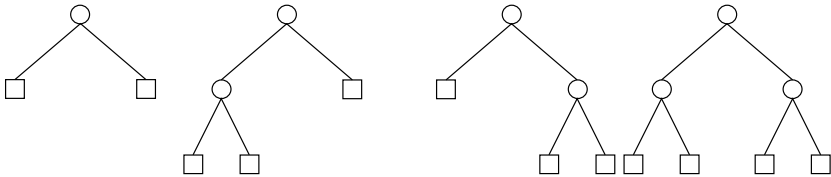


Abbildung 5.56: Menge Z von stratifizierten Bäumen

die man wie folgt erhält:

1. Jeder Baum aus Z ist Z -stratifiziert.
2. Sei ein Z -stratifizierter Baum gegeben. Ersetzt man jedes Blatt des Baumes durch einen Baum aus Z , so ist das Ergebnis wieder ein Z -stratifizierter Baum.

Z -stratifizierte Bäume können daher schematisch wie in Abbildung 5.57 dargestellt werden. Man beachte, daß die Zerlegung eines gegebenen Binärbaumes in Straßen, die zeigt, daß der Baum Z -stratifiziert ist, nicht eindeutig sein muß. Wir sehen also Bäume mit verschiedenen Zerlegungen als verschieden an und denken uns die Zerlegung stets explizit gegeben. Eine Möglichkeit zur Repräsentation der Straßengrenzen ist, die Knoten unterhalb und oberhalb einer Straßengrenze unterschiedlich einzufärben. Es ist nicht schwer zu sehen, daß die soeben definierte Klasse der Z -stratifizierten Bäume identisch ist mit der Klasse der symmetrischen binären B -Bäume [1] der Klasse der halb-balancierten Bäume von Olivié [2] und der Klasse der Rot-schwarz Bäume von Guibas und Sedgewick [3], wenn man die jeweiligen Update-Verfahren nicht berücksichtigt. Ferner ist klar, daß die Höhe eines Z -stratifizierten Baumes mit N Blättern (gemessen in der Anzahl der Kanten eines längsten Pfades von der Wurzel zu einem Blatt) von der Größenordnung $O(\log N)$ ist.

Wir beschreiben jetzt die Update-Verfahren, also das Einfügen und Entfernen von Schlüsseln für Z -stratifizierte Bäume. Den Umstrukturierungsoperationen, die nach einer Einfügung oder Entfernung eines Schlüssels ausgeführt werden müssen, liegt folgende Idee zugrunde:

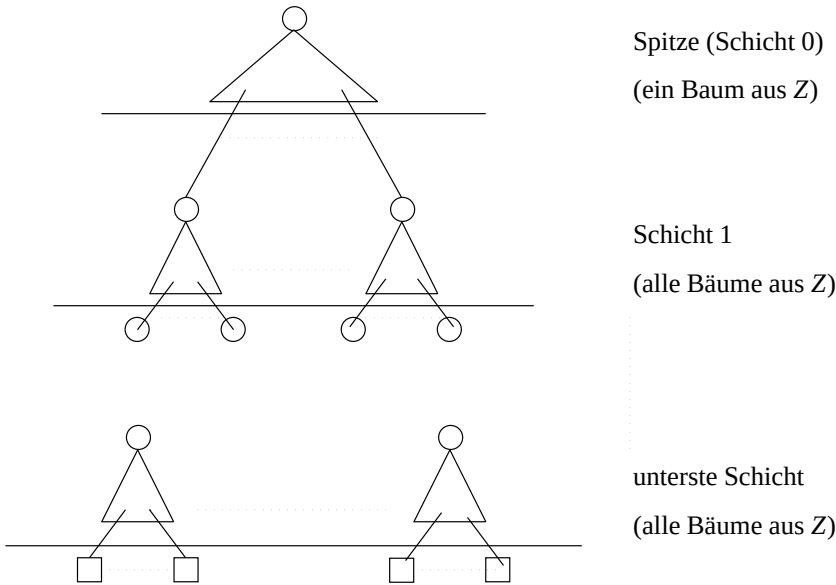


Abbildung 5.57: Struktur eines Z-stratifizierten Baumes

Es wird entweder eine auf die lokale Umgebung eines Knotens beschränkte strukturelle Änderung durchgeführt, oder das Umstrukturierungsproblem wird ohne jede Strukturänderung auf das nächst höhere Niveau, das heißt auf die nächste Straße verschoben. Unter Strukturveränderung verstehen wir dabei stets nur die Änderung von Zeigern; Farbänderungen, also jede lokale Verschiebung einer Straßengrenze, zählen nicht. Dieser Unterschied ist gerechtfertigt durch die bereits oben erläuterte Tatsache, daß es nicht erforderlich ist, Knoten in einer Mehrbenutzerumgebung zu sperren, wenn sich lediglich ihre Farbe ändert. Denn eine Farbänderung kann niemals eine Suchoperation in die falsche Richtung leiten.

Einfügen in Z-stratifizierte Bäume

Um einen neuen Schlüssel in einen Z-stratifizierten Suchbaum einzufügen, bestimmen wir zunächst seine Position unter den Blättern und ersetzen das Blatt, bei dem die erfolglose Suche endet, durch einen inneren Knoten mit zwei Blättern. Diese zwei Blätter speichern jetzt den alten Schlüssel, wo die Suche endete, und den neu eingefügten Schlüssel. Beachte, daß der so entstandene Baum jetzt kein Z-stratifizierter Suchbaum mehr ist, weil ein innerer Knoten unmittelbar unter der untersten Straßengrenze auftritt. Um das zu korrigieren und die Balancebedingung wiederherzustellen, versehen wir diesen Knoten mit einer Push-up-Marke (siehe Abbildung 5.58). Die Aufgabe, die wir für einen Knoten mit einer Push-up-Marke lösen müssen, ist, ihn über die Straßengrenze hinüber zu schieben, unterhalb der er auftritt. (Diese Aufgabe nennen wir auch eine Push-up-Forderung.) Dabei müssen wir darauf achten, daß die Z-stratifizierte Struktur des Baumes wiederhergestellt wird. Zugleich wollen wir erreichen, daß nur eine konstante Anzahl struktureller Änderungen ausgeführt wird. Daher gehen wir so vor, daß

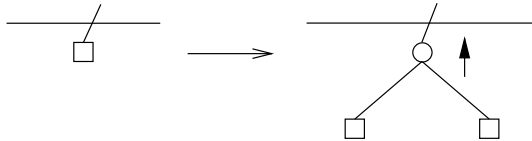


Abbildung 5.58: Einfügen eines neuen Schlüssels mit Setzen einer Push-up-Marke

das Beseitigen einer Push-up-Marke aus einer Bewegung des Knotens mit der Marke über die Straßengrenze hinweg besteht und

1. entweder zu einer strukturellen Änderung führt, die nur ein paar Knoten auf der gerade betrachteten Straße betrifft, und Halt oder
2. zu einer Push-up-Forderung führt für einen Knoten, der unmittelbar unterhalb der Grenze zur nächsthöheren Straße auftritt, aber zu keiner strukturellen Änderung.

Wir unterscheiden also zwei Fälle zur Behandlung von Knoten mit einer Push-up-Marke:

Fall 1 [Es gibt genug Platz in der nächsthöheren Schicht]

Dieser Fall liegt vor, wenn der Knoten mit der Push-up-Marke an einem Baum aus der Menge Z hängt, der nicht die maximale Anzahl von vier Blättern hat. In diesem Fall kann man durch Ausführen von höchstens zwei Rotationen (Einfach- oder Doppelrotation) den Baum aus Z durch einen anderen mit einem zusätzlichen Blatt ersetzen, alle Teilbäume in der gleichen Reihenfolge wieder anhängen und so die Balancebedingung wiederherstellen. Abbildung 5.59 zeigt die in diesem Fall erforderlichen Strukturänderungen. Dabei sind alle symmetrischen Fälle weggelassen.

Fall 2 [Es gibt nicht genug Platz auf der nächsthöheren Schicht]

Dieser Fall liegt vor, wenn der Knoten mit der Push-up-Marke ein Blatt eines vollständigen Binärbaumes der Höhe 2 ist. Denn nun kann man die Push-up-Forderung nicht durch eine lokale Strukturänderung auf der nächsthöheren Schicht erledigen. Also verschieben wir in diesem Fall die Push-up-Forderung rekursiv auf die nächsthöhere Schicht, indem wir die Marke einfach an die Wurzel dieses vollständigen Binärbaums der Höhe 2 auf der nächsthöheren Schicht heften und den Knoten, der vorher die Push-up-Marke hatte, über die Straßengrenze hinaufziehen, ohne eine Strukturänderung durchzuführen. Abbildung 5.60 zeigt eine der vier Möglichkeiten, wo der Knoten mit der Push-up-Marke vorkommen kann. Wir nehmen stillschweigend an, daß eine neue Schicht und eine neue Spitze eingefügt werden, sobald eine Push-up-Marke die Wurzel des ursprünglich gegebenen Z -stratifizierten Baumes erreicht hat. Z -stratifizierte Bäume wachsen also an der Wurzel durch Abspalten eines Knotens von einem Baum, der einen Knoten mehr hat als der Baum mit Höhe 2 und der maximalen Blattzahl 4.

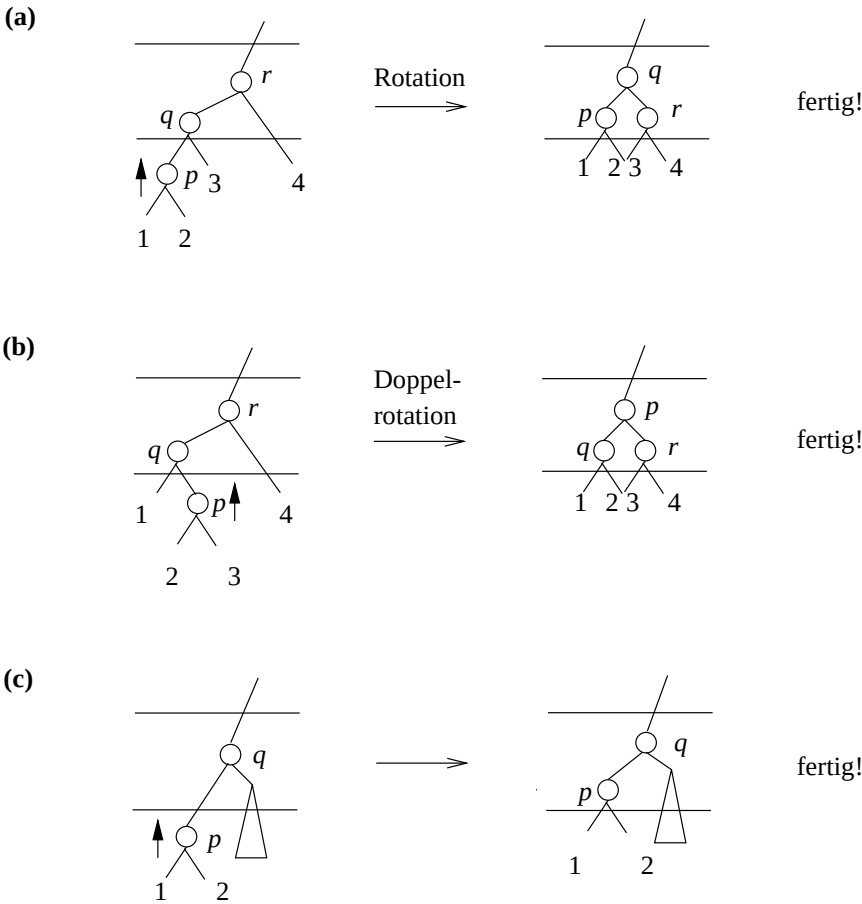


Abbildung 5.59: Lokale Umstrukturierungen bei einer Push-up-Forderung

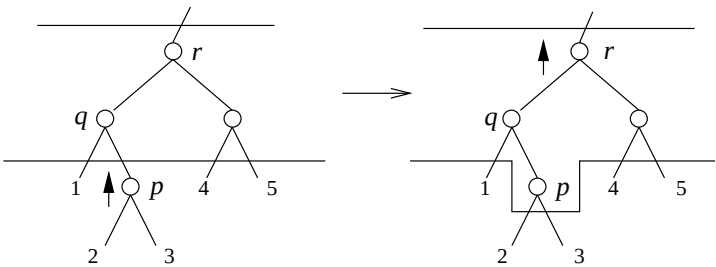


Abbildung 5.60: Rekursive Verschiebung einer Push-up-Forderung zum nächstshöheren Niveau

Wie wir gesehen haben, kann eine einzelne Einfügung zu einer Push-up-Forderung für einen Knoten führen, der unmittelbar unterhalb der untersten Straßengrenze auftritt. Das Erfüllen dieser Push-up-Forderung kann entweder zu einer Reihe weiterer Push-up-Forderungen für Knoten führen, die auf dem Suchpfad liegen und unmittelbar unterhalb der Grenzen zu nächsthöheren Schichten auftreten, ohne eine Strukturänderung durchführen zu müssen, oder aber zu einer lokalen Strukturänderung und Halt. Dabei besteht die Strukturänderung in dem Ersetzen eines Baumes aus der Menge Z von Straßenbäumen durch einen anderen. Sie wird realisiert durch eine Einfach- oder Doppelrotation. Damit dürfte klar sein, daß eine Push-up-Forderung stets durch eine konstante Zahl struktureller Änderungen erfüllt werden kann.

Wir beschreiben jetzt, wie eine *Folge von Einfügungen* behandelt wird, so daß es nicht erforderlich ist, den Baum nach jeder einzelnen Einfügung umzustrukturieren (Dabei lassen wir natürlich zu, daß der Baum zwischenzeitlich nicht mehr Z -stratifiziert ist). Zunächst beobachten wir, daß Push-up-Forderungen akkumuliert werden können und im Baum konkurrierend aufsteigen können so lange nur gesichert ist, daß keine zwei Push-up-Forderungen denselben Straßenbaum betreffen. Falls also mehrere Push-up-Marken an Knoten angebracht sind, die vom selben Straßenbaum über eine Straßengrenze herunterhängen, behandeln wir sie einfach nacheinander in beliebiger Reihenfolge wie oben beschrieben. Sobald eine Push-up-Forderung verschwunden ist (durch eine Strukturänderung oder durch rekursives Hinaufschieben auf die nächsthöhere Schicht), können wir bereits damit beginnen, die nächste Push-up-Forderung zu erfüllen. Abbildung 5.61 zeigt an einem Beispiel, wie hier vorzugehen ist.

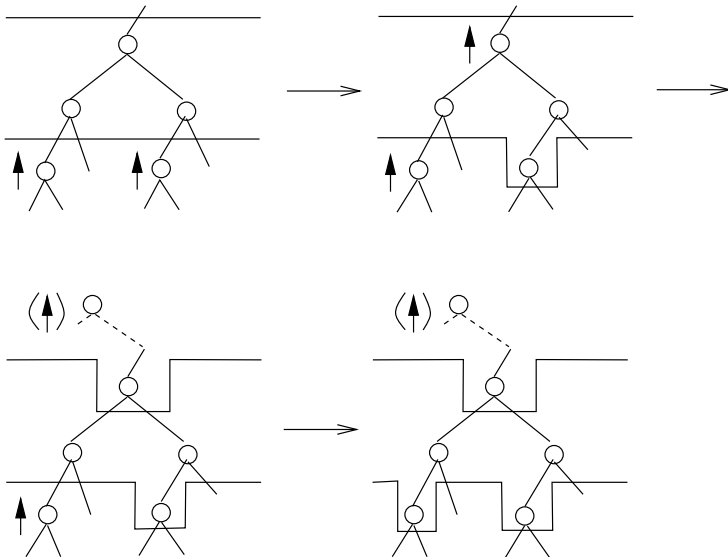


Abbildung 5.61

Dies löst das Problem, wie man Folgen von Einfügungen behandeln kann, die sämtlich verschiedene Blätter des ursprünglich gegebenen Z-stratifizierten Baumes betreffen. Wir fügen einfach an jeden neu erzeugten internen Knoten unterhalb der untersten Straßengrenze eine Push-up-Marke an. Nun sehen wir, daß wir dasselbe auch in dem Falle tun können, daß eine Einfügung in ein Blatt fällt, das nicht Blatt des ursprünglich gegebenen Z-stratifizierten Baumes ist, sondern ein Blatt, das durch eine frühere Einfügung erzeugt worden ist. Das heißt, wir können Push-up-Forderungen für Knoten, die unter der untersten Straßengrenze auftreten, einfach akkumulieren und wie vorher erledigen. Wir erfüllen sie in der Weise, daß wir stets Knoten unmittelbar unterhalb der untersten Straßengrenze des ursprünglich gegebenen Z-stratifizierten Baumes zuerst behandeln (Diese Bedingung gilt zum Beispiel, wenn wir die Push-up-Forderungen in derselben Reihenfolge erfüllen, in der wir die Knoten eingefügt haben.)

In dieser Weise kann also eine Folge von Einfügungen zu einem Wachstum des ursprünglich gegebenen Z-stratifizierten Baumes unterhalb der untersten Straßengrenze führen, das vergleichbar ist mit dem Wachstum eines natürlichen Suchbaumes. Jeder neu erzeugte Knoten hat eine Push-up-Marke. Die Push-up-Forderungen werden, wie oben beschrieben, von oben nach unten, aber sonst in beliebiger Reihenfolge erledigt. Sind alle Push-up-Forderungen erfüllt, ist der resultierende Baum wieder ein Z-stratifizierter Suchbaum. Abbildung 5.62 zeigt schematisch das Bild eines Z-stratifizierten Baumes nach einer Reihe von Einfügungen mit noch nicht erfüllten Push-up-Forderungen.

Entfernen aus Z-stratifizierten Bäumen

Um einen Schlüssel aus einem Z-stratifizierten Suchbaum zu entfernen, suchen wir ihn zunächst unter den Blättern und versehen das Blatt mit einer Löschmarke „ $\times$ “. Eine Löschmarke kann entweder unmittelbar beseitigt werden durch eine Strukturänderung in der Umgebung des betroffenen Blattes auf der untersten Schicht, oder aber sie führt dazu, daß der Bruder des entfernten Blattes mit einer Pull-down-Marke (Pull-down-Forderung) versehen wird. Denn eine an einem Blatt eines Baumes aus Z mit drei oder vier Blättern angebrachte Löschmarke kann leicht dadurch entfernt werden, daß man den Baum aus Z durch einen Baum ersetzt, der ein Blatt (und einen inneren Knoten) weniger hat. Hat allerdings ein Blatt eines Baumes aus Z mit nur zwei Blättern eine Löschmarke, so kann man nach Entfernen des Blattes die Balancebedingung nicht direkt wiederherstellen. Vielmehr führt das Beseitigen der Löschmarke zu einer Pull-down-Forderung „ $\times$ “. Das ist in Abbildung 5.63 erläutert, in der alle symmetrischen Fälle weggelassen wurden.

Hat ein Knoten (also anfangs der Bruder des entfernten Blattes) eine Pull-down-Marke, so befindet er sich selbst unmittelbar unter einer Straßengrenze und sein Vater zwei Straßen oberhalb der Straße, auf der er selbst auftritt. Das ist natürlich ein Verstoß gegen die Z-Stratifiziertheit des Baumes. Um diesen Verstoß zu beheben, müssen wir den Vater des Knotens mit der Pull-down-Marke eine Straße hinunterziehen und zugleich dafür sorgen, daß die Schichtenstruktur des Baumes durch eine konstante Anzahl struktureller Änderungen wiederhergestellt wird. Das Beseitigen einer Pull-down-Marke besteht also in einer Bewegung eines Knotens über eine Straßengrenze nach unten hinweg und

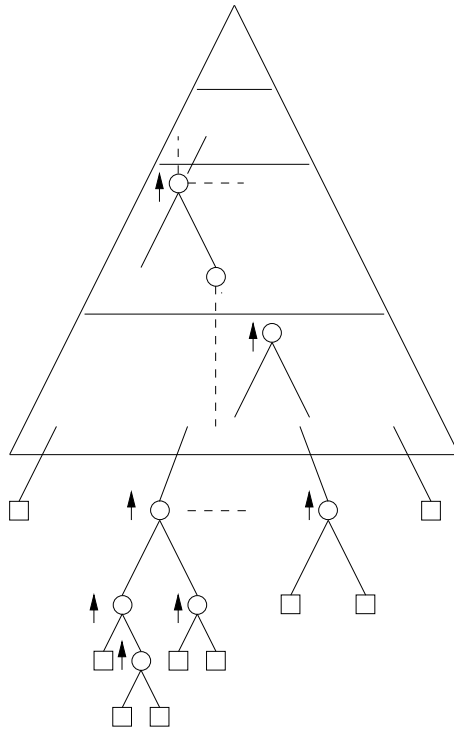


Abbildung 5.62: Z-stratifizierter Baum nach einer Reihe von Einfügungen mit noch nicht erfüllten Push-up-Forderungen

1. entweder einer lokalen strukturellen Änderung des Z-stratifizierten Baumes in der Schicht, in der der Vater des Knotens mit der Pull-down-Marke anschließend vorkommt und Halt
2. oder aber in einer rekursiven Verschiebung der Pull-down-Marke zum Vater des Knotens und keiner strukturellen Änderung im Baum.

Wir unterscheiden also wieder zwei Fälle je nachdem, wieviele Knoten in der unmittelbaren Verwandtschaft des Knotens v mit der Pull-down-Marke vorkommen.

Fall 1 [Es gibt genug Knoten in der Umgebung des Knotens v mit der Pull-down-Marke, vgl. Abbildung 5.64]

In diesem Fall kann die Pull-down-Forderung durch eine strukturelle Änderung, die nur wenige Knoten in der Umgebung des Knotens v betrifft, erledigt werden.

Um festzustellen, ob *Fall 1* vorliegt, inspizieren wir zunächst den Brüdern w von v . w kann auf derselben Schicht wie sein Vater p auftreten oder eine Schicht darunter. (Beachte, daß v genau zwei Schichten unterhalb von p liegt.)

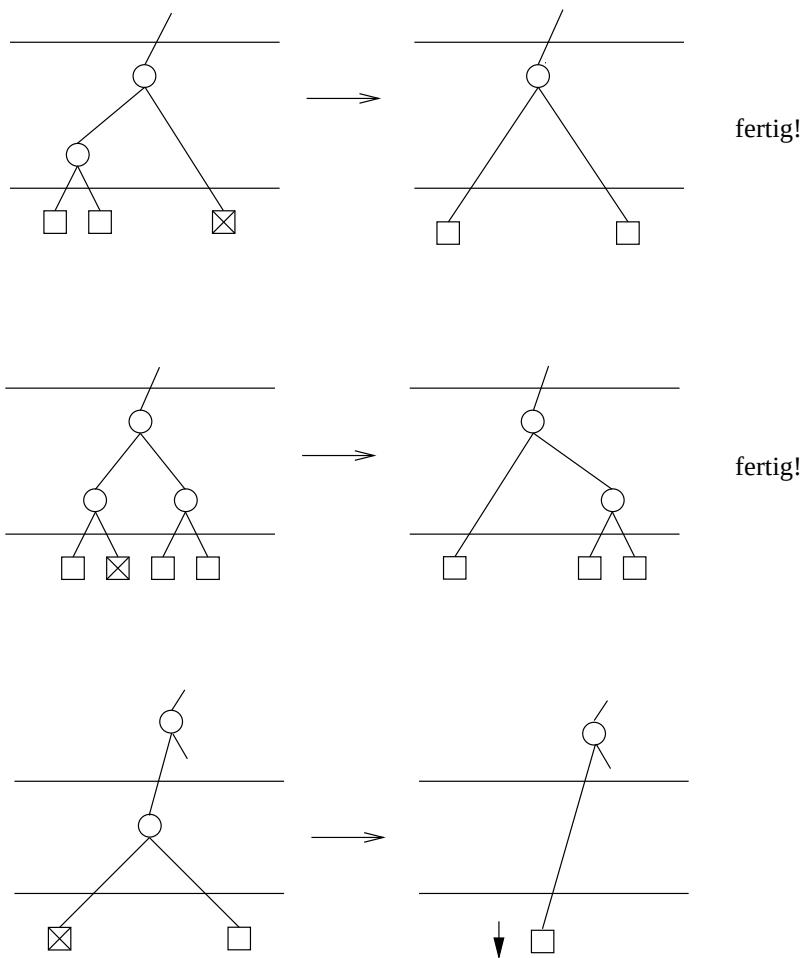


Abbildung 5.63: Löschen eines Schlüssels mit Setzen einer Pull-Down-Mark

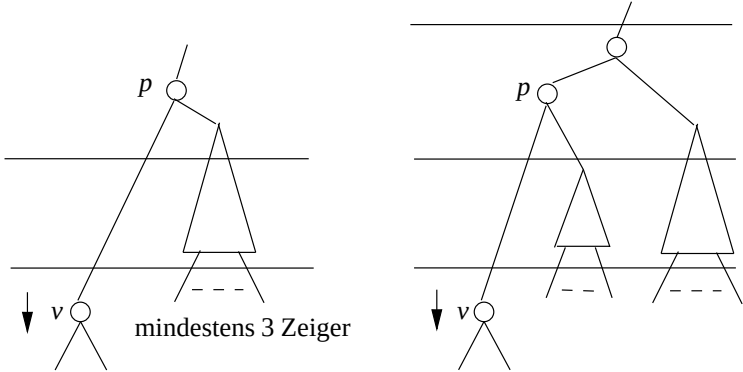


Abbildung 5.64: Der Knoten mit der Pull-down-Mark hat genug Knoten in seiner Umgebung

Wir betrachten zunächst den Fall, daß p und w in der gleichen Schicht liegen. Dann wissen wir, daß außer dem Zeiger, der p und v miteinander verbindet, wenigstens vier weitere Zeiger die Straßengrenze schneiden, unterhalb derer v liegt. Also ist es auf jeden Fall möglich, den Teilbaum mit Wurzel p durch einen neuen Straßenbaum aus Z zu ersetzen und die Teilbäume unterhalb von w so neu zu verteilen, daß v einen Vater auf der zwischen v und p liegenden Schicht erhält und die Z -stratifizierte Baumstruktur wiederhergestellt wird.

Um die Fallunterscheidung zu vereinfachen und die mehrfache Behandlung ähnlicher Fälle zu vermeiden, zeigen wir allerdings nicht explizit, wie in diesem Falle der Baum umzustrukturieren ist. Vielmehr führen wir die folgende Transformation durch, die den hier vorliegenden Fall auf einen anderen Fall reduziert, der ebenfalls unter *Fall 1* subsumierbar ist: Führe eine einfache Rotation bei p aus wie in Abbildung 5.65 (d) zu sehen, in der wieder alle symmetrischen Fälle weggelassen wurden. Man beachte, daß als Ergebnis dieser Rotation p einen Sohn auf der nächsten und den anderen Sohn v zwei Schichten unter seiner eigenen Schicht hat. Ferner treten p und der Vater von p auf der gleichen Schicht auf.

Wir können jetzt also annehmen, daß p und w auf verschiedenen Schichten auftreten. Das heißt, ein Sohn v von p ist zwei und der andere Sohn w von p eine Schicht unterhalb von p . Der Knoten w kann keinen, einen oder zwei Söhne auf derselben Schicht haben. Die letzteren beiden Fälle lassen sich unter *Fall 1* subsumieren und wie in Abbildung 5.65 (a) und (b) gezeigt behandeln, wobei wieder alle symmetrischen Fälle weggelassen wurden.

Im Falle, daß w keinen Sohn auf derselben Schicht hat, schauen wir nach oben zum Vater q von p . q kann auf derselben Schicht wie p auftreten. Dies ist ebenfalls eine Situation, die unter *Fall 1* subsumiert wird. Denn es ist in diesem Falle möglich, q den Sohn p wegzunehmen, so daß q dennoch Wurzel eines Straßenbaumes oberhalb von p bleibt, wie in Abbildung 5.65 (c) zu sehen.

Der einzige Fall, der nicht unter *Fall 1* subsumierbar ist, ist also eine Situation, in der der auf der Schicht unter der Schicht von p auftretende Knoten w keinen Sohn auf derselben Schicht wie w hat und in der p und der Vater q von p auf verschiedenen Schichten auftreten (p und w sind also jeweils Wurzeln von Bäumen aus Z mit der Höhe 1). Diese Situation bezeichnen wir als *Fall 2*:

Fall 2 [Es gibt nicht genügend Knoten in der Umgebung des Knotens v mit einer Pull-down-Marke]

In diesem Fall hat also der Knoten v mit der Pull-down-Marke die minimale Anzahl von Verwandten in seiner Umgebung. Wir können die Pull-down-Forderung daher nicht in der Umgebung von v erledigen. Also verschieben wir die Pull-down-Forderung von v auf den Vater p , indem wir einfach p unter die Straßengrenze oberhalb der p auftritt, hinunterziehen und die Pull-down-Marke bei p anbringen, vgl. Abbildung 5.66.

Man beachte, daß in diesem Fall keinerlei strukturelle Änderung (Änderung von Zeigern) ausgeführt wird. Ferner erfüllt der Knoten p offensichtlich die Invarianz-Bedingung, die wir oben für Knoten mit Pull-down-Marke formuliert haben, nämlich: p tritt unmittelbar unter einer Straßengrenze auf und der Vater von p liegt zwei Straßen oberhalb von p .

Wir nehmen übrigens stillschweigend an, daß eine Schicht an der Spitze des Z -stratifizierten Baumes verschwindet, wenn eine Pull-down-Marke den Sohn v der Wurzel p des Baumes erreicht hat und die Schicht zwischen dem Knoten v und seinem Vater

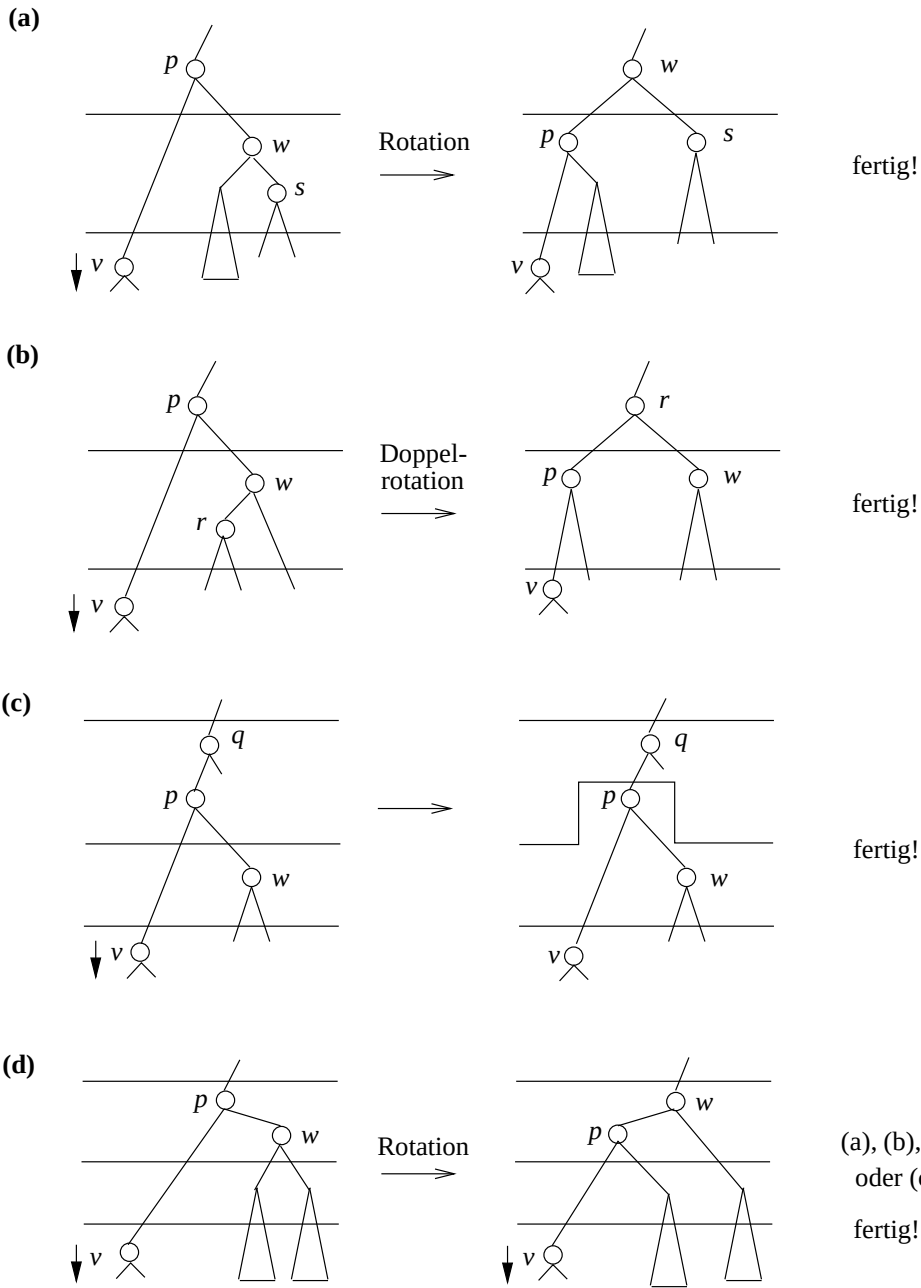


Abbildung 5.65: Lokale Umstrukturierungen bei einer Pull-down-Forderung

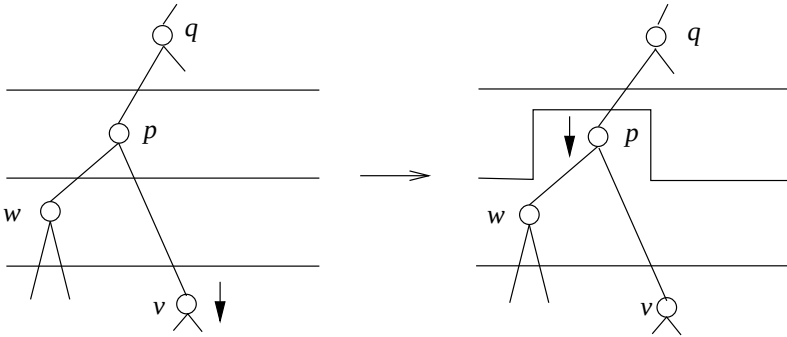


Abbildung 5.66: Rekursive Verschiebung einer Pull-down-Forderung zum nächsthöheren Niveau

p leer geworden ist. Denn in diesem Fall macht das Hinunterschieben des Knotens p unter die oberste Straßengrenze diese Grenze überflüssig.

Wie wir gesehen haben, führt eine einzelne Entfernung aus einem Z -stratifzierten Baum dazu, daß ein Blatt des Baumes mit einer Löschmarke versehen wird. Die Beseitigung dieser Löschmarke kann entweder unmittelbar durch eine auf die Umgebung dieses Blattes beschränkte strukturelle Änderung auf der untersten Schicht erfolgen, oder aber sie löst eine Pull-down-Forderung für den Bruder des entfernten Blattes aus. Pull-down-Forderungen (also Knoten mit Pull-down-Marken) können in dem Baum hochsteigen durch rekursive Verschiebung auf höhere Schichten, aber ohne strukturelle Änderungen, bis sie schließlich durch eine strukturelle Änderung beseitigt werden, die aber immer nur eine konstante Anzahl von Knoten und Zeigern betrifft.

Wir erläutern jetzt, wie eine *Folge von Entfernungen* in der Weise behandelt werden kann, so daß es nicht erforderlich ist, den Baum direkt nach jeder einzelnen Entfernung wieder umzustrukturieren. Zunächst beobachten wir, daß Entfernungen einfach dadurch akkumuliert werden können, daß man für jede Entfernung ein Blatt mit einer Löschmarke versieht und zunächst nichts weiter tut. Die Löschmarken können nun konkurrierend in beliebiger Reihenfolge beseitigt werden, wie oben beschrieben, so lange nur sichergestellt ist, daß die Beseitigung von mehreren Löschmarken niemals denselben Straßenbaum betrifft. Man muß sie nur nacheinander in beliebiger Reihenfolge behandeln durch die zuvor beschriebenen Rebalancierungsoperationen. Das impliziert insbesondere, daß die Beseitigung einer Löschmarke eines Knotens mit Pull-down-Marke (als Ergebnis einer vorher beseitigten Löschmarke), nicht erfolgen kann, bevor die Pull-down-Marke beseitigt oder im Baum weiter hochgestiegen ist. Beachtet man aber diese Bedingung, so ist gesichert, daß die Beseitigung zweier Löschmarken an den Blättern desselben Z -Straßenbaumes immer zu einem korrekten Ergebnis führt: Bevor die zweite Löschmarke beseitigt wird, hat eine Pull-down-Forderung den Vater des betroffenen Blattes eine Schicht hinuntergezogen, vgl. hierzu Abbildung 5.67 für eine graphische Erläuterung.

Kommen als Folge mehrerer beseitigter Löschmarken mehrere Pull-down-Marken an Knoten im Baum vor, so kann man sie stets konfliktfrei mit Hilfe der angegebenen Transformationen entweder beseitigen oder auf die nächsthöhere Schicht verschieben. Solange sie nicht denselben Baum aus Z betreffen, können sie sich nämlich nicht stören

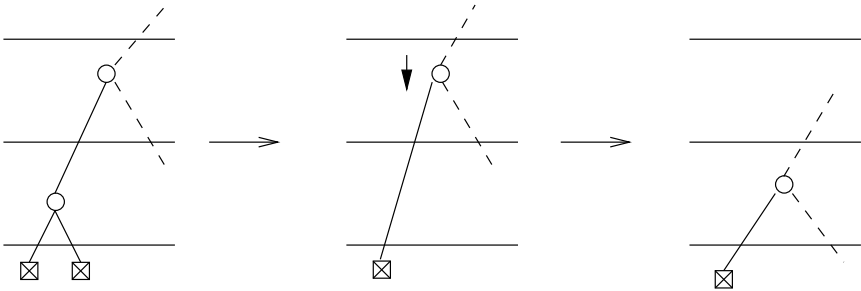


Abbildung 5.67: Beseitigung zweier Löschmarken an den Blättern desselben Z-Straßenbaumes

und man kann sie daher in beliebiger Reihenfolge behandeln. Kommt in der Umgebung eines Knotens mit Pull-down-Marke ein weiterer Knoten mit Pull-down-Marke vor, muß die weiter oben liegende Pull-down-Marke zuerst beseitigt werden. Dieses *Top-down-Vorgehen* zur Beseitigung mehrerer Pull-down-Marken ist immer möglich und korrekt mit Ausnahme eines einzigen Falls: Es kann als Ergebnis des rekursiven Verschiebens mehrerer Pull-down-Marken nach oben vorkommen, daß beide Söhne v und w eines Knotens p eine Pull-down-Marke haben und v und w zwei Schichten unter p liegen. Dann verschiebe man einfach p um eine Schicht nach unten, beseitige die Pull-down-Marken von v und w und bringe eine Pull-down-Marke bei p an, falls p keinen Vater auf seiner Schicht hat; sonst genügt bereits das Hinunterschieben von p , um beide Pull-down-Forderungen zu erfüllen. Das ist graphisch in Abbildung 5.68 gezeigt.

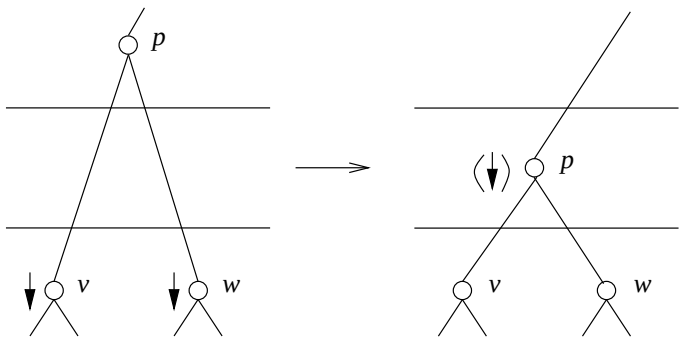


Abbildung 5.68: Gleichzeitiges Beseitigen von zwei Pull-down-Marken

Auf diese Weise wird sichergestellt, daß jede Folge von akkumulierten Entfernungen und die von Ihnen ausgelösten Umstrukturierungsprozesse beliebig verzahnt ausgeführt werden können, ganz genauso, als hätte man sie nacheinander (seriell) ausgeführt. Abbildung 5.69 zeigt schematisch einen nach einer Reihe von Entfernungen und strukturellen Änderungen entstandenen Z-stratifizierten Suchbaum.

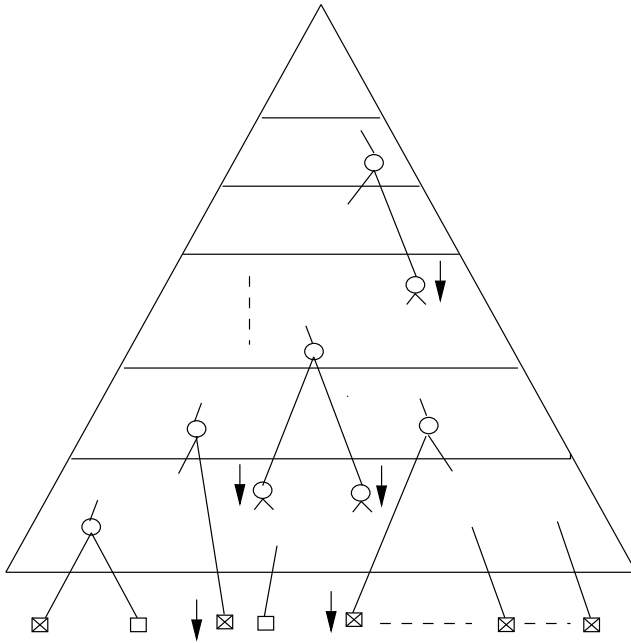


Abbildung 5.69: Z-stratifizierter Baum nach einer Reihe von Entfernungen mit noch nicht erfüllten Pull-down-Forderungen

Wie wir gesehen haben, wachsen und schrumpfen Z-stratifizierte Suchbäume also an der Wurzel. Neue Schlüssel wandern in den Baum von unten hinein, das heißt über die unterste Straßengrenze. Ebenso werden Schlüssel entfernt, indem man sie an der untersten Straßengrenze aus dem Baum herauszieht. Jetzt können wir erklären, wie beliebig verzahnte *Folgen von Einfügungen, Entfernungen und Umstrukturierungen* ausgeführt werden können.

Wenn eine Einfügung oder Entfernung in ein Blatt fällt, das unmittelbar unter der untersten Straßengrenze liegt, geschieht zunächst nichts Neues mit Ausnahme der Möglichkeit, daß jetzt eine Einfügung in ein Blatt fallen kann, das eine Löschrunde trägt. Es ist klar, wie man dann vorzugehen hat: Beseitige die Löschrunde und füge den Schlüssel an dieser Stelle wieder ein, siehe Abbildung 5.70.

Falls umgekehrt eine Entfernung in ein Blatt fällt, das durch eine frühere Einfügung entstanden und das noch nicht hinaufgewandert ist zur untersten Straßengrenze, kann man das Blatt und den zugehörigen inneren Knoten einfach entfernen und eine Pull-down-Marke beseitigen. Abbildung 5.71 zeigt ein Beispiel für dieses Ereignis.

Abgesehen von diesen geringfügigen Änderungen und Zusätzen ist nichts Neues erforderlich, um sicherzustellen, daß Einfügungen, Entfernungen und Rebalancierungsoperationen (das heißt also das Beseitigen von Push-up-, Löschrunde- und Pull-down-Marken) nebenläufig und beliebig verzahnt ausgeführt werden können. Man muß im Konfliktfall (wenn mehrere Push-up-, Pull-down- oder Löschrunden an Knoten in der-

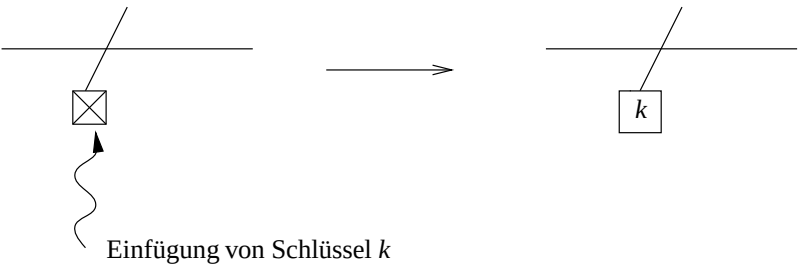


Abbildung 5.70: (Wieder-)Einfügung eines Schlüssels in ein Blatt mit Löschmarke

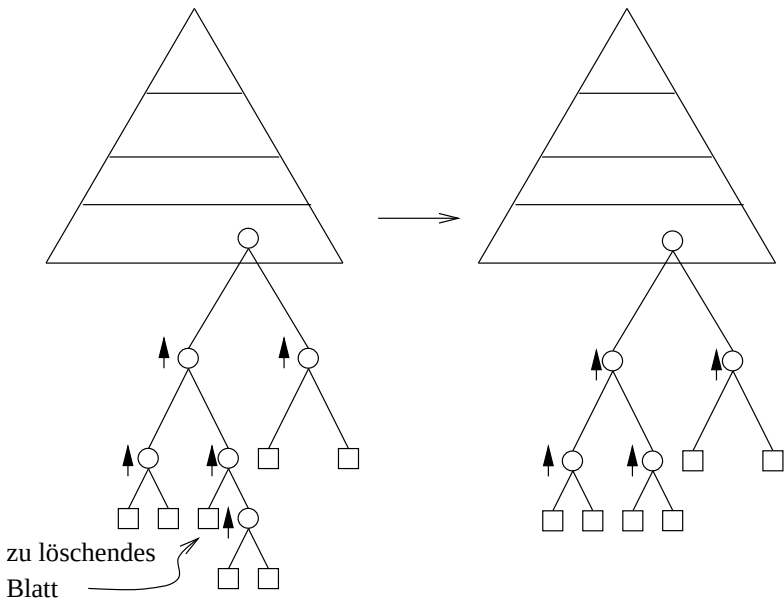


Abbildung 5.71: Entfernung eines durch Einfügung entstandenen Blattes

selben Umgebung vorkommen) nur darauf achten, der Top-down-Strategie zu folgen: Die jeweils weiter oben befindliche Marke muß ggfs. zuerst beseitigt werden. Das ist mit Hilfe der beschriebenen Transformationen immer möglich. Diese Überlegungen können im folgenden Satz zusammengefaßt werden:

Satz 5.4 *Sei T ein Z -stratifizierter Suchbaum, und sei eine beliebig verzahnte Folge von Einfügungen, Entfernungen und Transformationen zur Rebalancierung gegeben, die auf T angewandt wird. Dann ist die Anzahl der strukturellen Änderungen (Änderungen von Zeigern, die erforderlich sind, um die Balancebedingung für T wieder herzustellen, das heißt, um T wieder Z -stratifiziert zu machen) höchstens von der Größenordnung $O(i + d)$, wobei i die Anzahl der Einfügungen und d die Anzahl der Entfernungen ist.*

Wir sehen also, daß man mit derselben Anzahl struktureller Änderungen auskommt, die man auch aufzuwenden hätte, um einen gegebenen Baum jeweils unmittelbar nach einer Update-Operation wieder Z -stratifiziert zu machen.

Wir bemerken abschließend noch, daß keinerlei Umstrukturierungsoperationen erforderlich sind, wenn man zunächst eine Reihe von Einfügungen und dann eine Reihe von Entfernungen für einen gegebenen Baum ausführt und am Schluß der Baum wieder seine ursprüngliche Gestalt hat, ohne daß man zwischendurch irgendwelche Rebalancierungs-Operationen begonnen oder erledigt hat. Dies ist ein durchaus wichtiger Unterschied zu anderen in der Literatur vorgeschlagenen Verfahren zum relaxierten Balancieren.

5.6.3 Eindeutig repräsentierte Wörterbücher

Auch wenn eine Klasse von Bäumen durch eine statische Bedingung an die Struktur der Bäume festgelegt ist, kann es immer noch viele verschiedene Bäume in der Klasse geben, die sämtlich die gleiche Menge von Schlüsseln speichern. Wir können beginnend mit dem anfangs leeren Baum eine Reihe von Einfüge- und Entferne-Operationen ausführen, um schließlich einen Baum zu erhalten, der eine bestimmte Menge von Schlüsseln speichert. In der Regel hängt die Struktur dieses Baumes von seiner Entstehungsgeschichte, also von der Reihenfolge der Einfüge- und Entferne-Operationen ab.

Wir wollen jetzt Bäume als spezielle, durch Zeiger verbundene Graphen auffassen, die in ihren Knoten die Schlüssel speichern. Wir nennen ein Wörterbuch *mengen-eindeutig* repräsentiert, wenn jede Menge von Schlüsseln durch genau eine derartige Struktur repräsentiert ist. Bei Mengen-Eindeutigkeit kommt es also auf die Reihenfolge der Operationen, mit der man eine Struktur zur Speicherung einer gegebenen Schlüsselmenge erzeugt, nicht an. Es gibt genau einen Graphen, dessen Knoten die Schlüssel speichern.

Wir nennen ein Wörterbuch *größen-eindeutig* repräsentiert, wenn sogar jede Menge derselben Größe jeweils durch genau eine Struktur repräsentiert wird. Größen-Eindeutigkeit impliziert natürlich Mengen-Eindeutigkeit. Wir verlangen darüberhinaus stets, daß die Knoten des Graphen angeordnet sind und die Schlüssel der Größe nach in den Knoten mit aufsteigender Ordnungsnummer abgelegt sind. Wir bezeichnen diese Eigenschaft auch als *Ordnungs-Eindeutigkeit*.

Das Problem der eindeutigen Repräsentierung von Wörterbüchern besteht in der Suche nach möglichst effizienten Algorithmen zum Suchen, Einfügen und Entfernen von Schlüsseln für Wörterbücher, die mengen- oder größen-eindeutig repräsentiert sind.

Ein einfaches Beispiel für eine größen-eindeutige Repräsentierung von Wörterbüchern sind sortierte, verkettet gespeicherte lineare Listen. Die im Abschnitt 5.3.2 beschriebenen randomisierten Suchbäume sind ein Beispiel für eine mengen-eindeutige aber nicht größen-eindeutige Repräsentation von Wörterbüchern. (Dabei unterstellen wir, daß die zur Berechnung der Prioritäten benutzte Hash-Funktion beliebig, aber fest gewählt ist.)

Man kann nun zeigen, daß die Forderung nach mengen- oder größen-eindeutiger Repräsentierung von Wörterbüchern zur Folge hat, daß wenigstens eine der drei Wörterbuchoperationen Suchen, Einfügen und Entfernen von Schlüsseln mehr als $O(\log n)$ Zeit für Strukturen mit n Schlüsseln benötigt. Es wurde erstmals von Snyder in [Sny71] für eine große Klasse von Verfahren zum Suchen, Einfügen und Entfernen von Schlüsseln in Datenstrukturen gezeigt, daß die untere Grenze für den Aufwand zur Ausführung dieser Operationen bei eindeutig repräsentierten Datenstrukturen von der Größenordnung $\Omega(\bar{n})$ ist. Es ist also kein Zufall, daß AVL-Bäume, Bruder-Bäume, gewichtsbalancierte Bäume, B-Bäume und all die anderen zuvor genannten Klassen balancierter Bäume keine eindeutig repräsentierten Datenstrukturen sind. Der Wert dieser Aussage hängt natürlich stark von dem in diesem Zusammenhang benutzten Verfahrens- und Aufwandsbegriff ab. Natürlich sollten alle bekannten Verfahren zum Suchen, Einfügen und Entfernen von Schlüsseln in Listen, balancierte und unbalancierte Bäume aller Art darunter subsumierbar sein.

Snyder (vgl. [Sny71]) gibt auch eine von ihm „Qualle“ genannte größen-eindeutige Struktur zur Repräsentation von Wörterbüchern an, die es erlaubt, jede der drei Wörterbuchoperationen in Zeit $O(\bar{n})$ auszuführen. Die in den Beweisen für die obere und untere Schranke zugelassenen Operationen stimmen aber nicht überein. Wir werden jetzt eine größen- und ordnungseindeutige Repräsentation von Wörterbüchern angeben, die die von Snyder angegebene untere Schranke im gewissen Sinne unterbietet.

Dazu betrachten wir eine größen- und ordnungseindeutige Repräsentation von Wörterbüchern durch Graphen mit begrenztem Ausgangsgrad (jeder Knoten hat höchstens die Ordnung k , k fest) und nehmen an, daß es für jede Zahl n genau einen Graphen mit n -Knoten gibt. Ferner unterstellen wir, daß die Knoten eines jeden Graphen eine feste Ordnung haben. Die Elemente einer gegebenen Menge von Schlüsseln der Größe n sind dann in den Knoten des Graphen in der Weise gespeichert, daß der i -größte Schlüssel im Knoten mit der Ordnungsnummer i abgelegt ist, für jedes i .

Jede Suche startet bei einem bestimmten Knoten, den wir die *Wurzel* nennen und läuft dann Kanten des Graphen entlang, bis das gesuchte Element in einem Knoten gefunden ist oder die Suche erfolglos endet. Alle Elemente müssen also von der Wurzel aus erreichbar sein. Daraus folgt sofort, daß jeder Knoten mit Ausnahme der Wurzel wenigstens eine in den Knoten hineinführende Kante hat. Die Kosten der Suche sind die Anzahl der bei der Suche durchlaufenen Kanten plus eins.

Wenn man eine Update-Operation ausführt, also eine Einfügung oder Entfernung, darf der Graph durch eine der folgenden Operationen verändert werden: Schaffen oder Entfernen eines Knotens, das Ändern, Hinzufügen oder Entfernen einer den Knoten verlassenden Kante (Zeiger-Änderung), Austauschen von Elementen zwischen zwei Knoten.

Jede dieser Operationen verlangt Kosten der Größenordnung $\Theta(1)$. In diesem Kostenmodell kann man nun die folgende untere Schranke beweisen, vgl. [17].

Satz 5.5 Für jede größen- und ordnungseindeutige Repräsentation von Wörterbüchern durch Graphen benötigt wenigstens eine der drei Wörterbuchoperationen Zeit $\Omega(n^{1/3})$.

Wir verzichten auf einen Beweis dieses Satzes und zeigen vielmehr eine mit der im Satz behaupteten unteren Schranke übereinstimmende obere Schranke.

Halbdynamische c -Ebenen-Sprunglisten

Wir führen zunächst eine Variante der von Snyder in [17] eingeführten Struktur ein, die wir 2-Ebenen-Sprungliste nennen, für die dieselbe $O(n)$ Worst-case-Zeitschranke für alle drei Wörterbuchoperationen gilt. Um die Präsentation von 2-Ebenen-Sprunglisten zu vereinfachen, nehmen wir an, daß $i^2 \leq n < (i + 1)^2$ für ein festes i ist. Das heißt, wir nehmen an, daß die Größe n des Wörterbuches nicht beliebig infolge von Einfügungen und Entfernungen schwanken kann, sondern immer zwischen gegebenen Schranken $i^2 \leq n < (i + 1)^2$ für ein festes i bleibt. Eine 2-Ebenen-Sprungliste der Größe n besteht nun aus einer doppelt verketteten Liste von n Knoten $1, \dots, n$. Für jedes p , $1 \leq p < n$ sind also die Knoten p und $p + 1$ miteinander durch ein Paar von Zeigern auf Ebene 1 miteinander verknüpft. Wir nennen die Folge der durch Zeiger auf Ebene 1 miteinander verknüpften Knoten auch die 1-Ebenen-Liste. Ferner sind die Knoten $1, i + 1, 2i + 1, \dots, \lfloor n/i \rfloor \cdot i + 1$ miteinander zu einer 2-Ebenen-Liste verknüpft, die wir auch die oberste Ebenen-Liste nennen. Der letzte Knoten dieser Liste ist die Wurzel der 2-Ebenen-Sprungliste. Abbildung 5.72 zeigt die Struktur einer 2-Ebenen-Sprungliste.

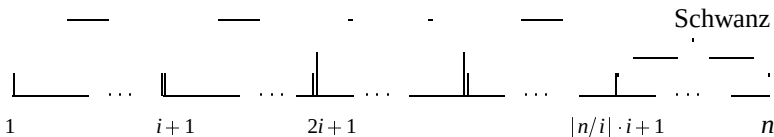


Abbildung 5.72: 2-Ebenen-Sprungliste der Größe n

Wir verlangen, daß die Elemente einer Menge mit n Schlüsseln in aufsteigender Ordnung in den Knoten $1, 2, \dots, n$ abgelegt sind. Damit haben wir also eine größen- und ordnungseindeutige Repräsentation von Wörterbüchern.

Nun sollte klar sein, wie man nach einem Schlüssel in einer solchen Struktur sucht und dabei höchstens $2i$ Schlüsselvergleiche ausführt: Man benutze ausgehend von der Wurzel die oberste Ebenen-Liste, um die Folge von höchstens i Knoten zu bestimmen, die den gesuchten Schlüssel enthalten kann und führe anschließend eine lineare Suche durch, indem man den Zeigern auf Ebene 1 folgt. Solange n im Bereich $i^2 \leq n < (i + 1)^2$ bleibt, können Updates ebenfalls in Zeit $O(i)$ ausgeführt werden: Bestimme zuerst die Einfüge- oder Entferneposition in der 1-Ebenen-Liste. Das benötigt $O(i)$ Schritte. Dann füge das Element in diese Liste ein oder entferne es daraus. Das ist eine in konstanter Zeit ausführbare Operation. Sie hat zur Folge, daß eine Folge von Knoten auf Ebene

1, die von einem Zeiger auf der obersten Ebene übersprungen wird, entweder zu lang geworden ist (nach einer Einfügung) oder zu kurz (nach einer Entfernung). Also müssen einige Zeiger auf der obersten Ebene um eine Position nach links oder um eine Position nach rechts verschoben werden.

Abbildung 5.73 zeigt ein Beispiel einer Einfügung von Schlüssel 9 in eine 2-Ebenen-Sprungliste der Größe 11, die die Schlüssel 2, 3, 5, 7, 8, 10, 11, 12, 14, 17, 19 speichert. Beachte, daß das Einfügen die Länge des Schwanzes der 2-Ebenen-Sprungliste um eins verlängert.

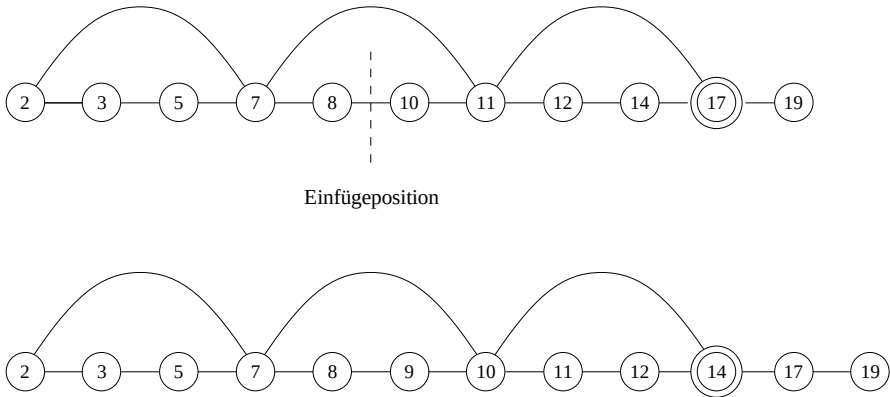


Abbildung 5.73: Einfügung von 9 in eine 2-Ebenen-Sprungliste

Folglich muß die oberste Ebenen-Liste um ein Element verlängert werden, sobald die Länge des Schwanzes i übersteigt. Analog kann eine Entfernung es erfordern, die oberste Ebenen-Liste um ein Element zu verkürzen. Das Adjustieren der obersten Ebenen-Liste nach einer Einfügung oder Entfernung ist aber in jedem Fall in $O(i)$ Schritten im schlechtesten Fall möglich.

So wie wir 2-Ebenen-Sprunglisten eingeführt haben, sind sie nur halbdynamisch, weil wir nicht erlaubt haben, daß ihre Größe n beliebig variieren darf. Es ist aber nicht allzuschwer, sich zu überlegen, daß man die Struktur auch volldynamisch machen kann, ohne daß man ihre wesentlichen Eigenschaften zerstört. Wir verzichten auf eine explizite Darstellung und verweisen dazu auf [3]. Statt dessen führen wir halbdynamische c -Ebenen-Sprunglisten für jedes $c \geq 3$ als natürliche Verallgemeinerung von 2-Ebenen-Sprunglisten ein. Wir nehmen also der Einfachheit halber wieder an, daß $i^c \leq n < (i+1)^c$ für ein festes i ist. Eine c -Ebenen-Sprungliste der Größe n besteht nun aus n Knoten $1, \dots, n$. Die Knoten sind miteinander verknüpft durch Zeiger, die auf verschiedenen Ebenen verlaufen, nämlich auf unteren Ebenen und auf oberen Ebenen.

Untere Ebenen. Für jedes $j, 1 \leq j \leq c/2$, und jedes $p, 1 \leq p \leq n - i^{j-1}$, sind die Knoten p und $p + i^{j-1}$ durch ein Paar von Zeigern auf Ebene j miteinander verknüpft.

Obere Ebenen. Für jedes $j, c/2 + 1 \leq j \leq c$, sind die Knoten $1, 1 + i^{j-1}, 2 + i^{j-1}, 3 + i^{j-1}, \dots$ miteinander verknüpft, wobei höchstens $i^{j-1} - 1$ Knoten in

einem Schwanz übrig bleiben. Der letzte Knoten dieser obersten Ebenen-Liste ist die Wurzel.

Die Knoten einer c -Ebenen-Sprungliste, die durch Zeiger auf Ebene j miteinander verknüpft sind, bilden die Folge der j -Ebenen-Liste. Eine j -Ebenen-Liste hat maximal die Länge $n/i^{j-1} = O(i^{c-j+1})$. Man beachte den Unterschied zwischen den unteren und oberen Ebenen. In den unteren Ebenen ist jeder Knoten Teil einer j -Ebenen-Liste, während die oberen Ebenen jeweils nur eine j -Ebenen-Liste enthalten, die jede nur einen Knoten einschließen.

Abbildung 5.74 zeigt die Struktur einer 3-Ebenen-Sprungliste der Größe 30 mit zwei unteren und einer obersten Ebenen-Liste. Man beachte, daß eine c -Ebenen-Sprungliste der Größe n einen Speicherbedarf von $O(c \cdot n)$ hat.

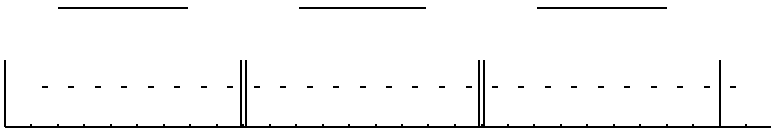


Abbildung 5.74: 3-Ebenen-Sprungliste der Größe 30

Wir verlangen wieder, daß die Schlüssel einer Menge von n Elementen in aufsteigender Reihenfolge in den Knoten $1, \dots, n$ einer c -Ebenen-Sprungliste der Größe n abgelegt sind. Das ergibt dann eine größen- und ordnungseindeutige Repräsentation von Wörterbüchern.

Um nach einem Schlüssel zu *suchen*, beginnen wir bei der Wurzel in der obersten Ebenen-Liste und bestimmen die Folge von höchsten i^{c-1} Knoten, die den gesuchten Schlüssel enthalten können. Dann folgen wir für jedes $j = c-1, c-2, \dots, 1$ einer Folge von Zeigern auf Ebene j , um die Position des gesuchten Schlüssels in der j -Ebenen-Liste zu bestimmen, bis wir den gesuchten Schlüssel gefunden haben oder j den Wert 1 bekommen hat und der gesuchte Schlüssel nicht an seiner erwarteten Position in der 1-Ebenen-Liste gefunden wurde. Beachte, daß für jedes $j, c-1 \geq j \geq 1$, die Suche beschränkt ist auf einen Teil der j -Ebenen-Liste mit Länge höchstens i . So folgt, daß eine erfolgreiche oder erfolglose Suche in Zeit $O(c \cdot i) = O(c \cdot n^{1/c})$ im schlechtesten Fall ausführbar ist.

In Abbildung 5.75 ist ein möglicher Suchpfad in der 3-Ebenen-Sprungliste von Abbildung 5.74 durch fettgedruckte Zeiger dargestellt.

Um einen Schlüssel in eine c -Ebenen-Sprungliste *einzufigen*, bestimmt man zunächst die erwartete Position des neuen Schlüssels durch eine Suche wie vorher erläutert. Dann fügt man das neue Element in *alle* unteren j -Ebenen-Listen ein, $1 \leq j \leq c/2$. Es werden alle Zeiger auf Ebene j , die über die Einfügeposition hinwegspringen, adjustiert; siehe Abbildung 5.76. Das heißt, eine Einfügeoperation kann aufgefaßt werden als ein gleichzeitiges Einfügen des neuen Elementes in i^{j-1} angeordnete, doppelt verkettete, lineare Listen für alle $j, 1 \leq j \leq c/2$. Das benötigt Zeit $O(1 + i + i^2 + \dots + i^{\lceil c/2 \rceil - 1}) = O(i^{\lceil c/2 \rceil - 1})$ insgesamt. Dann müssen die Zeiger aller Knoten in den Listen auf den

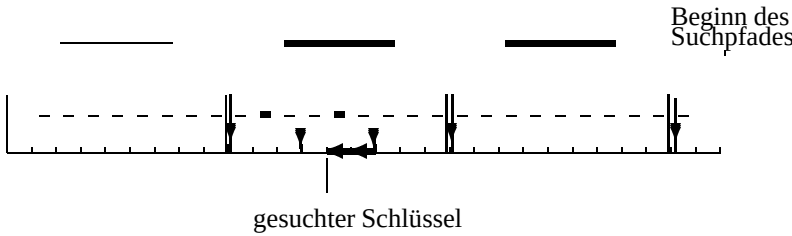


Abbildung 5.75: Beispiel eines möglichen Suchpfades

oberen Ebenen rechts von der Einfügeposition um eine Position nach links verschoben werden. Das benötigt Zeit $O(\sum_{j=\lceil c/2 \rceil + 1}^c n/i^{j-1}) = O(\sum_{j=\lceil c/2 \rceil + 1}^c i^{c-j+1}) = O(i^{\lfloor c/2 \rfloor})$ im schlechtesten Fall. Die Gesamtkosten sind also $O(i^{\lfloor c/2 \rfloor - 1} + i^{\lfloor c/2 \rfloor})$. Das führt zu zwei Fällen, je nachdem ob c gerade oder ungerade ist. Ist c gerade, benötigt das Einfügen Zeit $O(\sqrt[n]{n})$, ist c ungerade, benötigt das Einfügen eines neuen Elementes in eine c -Ebenen-Sprungliste der Größe n Zeit $O(n^{(c-1)/2c})$. In jedem Fall ist die resultierende c -Ebenen-Sprung-Liste eine Liste der Größe $n + 1$.

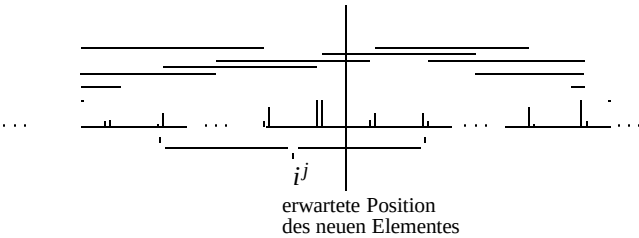


Abbildung 5.76: Auswirkungen durch eine Einfügung in eine j -Ebenen-Liste

Das Entfernen kann in völlig analoger Weise mit den gleichen asymptotischen Kosten durchgeführt werden. Man geht gerade umgekehrt wie beim Einfügen vor.

Auch hier kann man die Struktur volldynamisch machen, also die Beschränkung, daß n stets zwischen i^c und $(i + 1)^c$ bleiben muß, fallen lassen. Dazu gibt es allgemeine Techniken, die hier nicht weiter erläutert werden. Insgesamt erhalten wir folgendes Resultat:

Satz 5.6 Für jedes $c \geq 3$ sind c -Ebenen-Sprunglisten eine größen- und ordnungseindeutige Repräsentation von Wörterbüchern, die Platz $O(c \cdot n)$ beansprucht. Die Wörterbuchoperationen verlangen zu ihrer Ausführung höchstens die folgenden Kosten: Das Suchen ist ausführbar in der Zeit $O(c \cdot n^{1/c})$; Einfügen und Entfernen benötigen Zeit $O(\sqrt[n]{n})$, wenn c gerade ist, und Zeit $O(n^{(c-1)/2c})$, wenn c ungerade ist.

Wählt man in diesem Satz $c = 3$, erhält man das im Lichte von Snyder's Ergebnis [E] etwas überraschende Resultat, daß in 3-Ebenen-Sprunglisten jede der drei Wörterbuchoperationen in Zeit $O(n^{1/3})$ ausführbar ist.

5.7 Optimale Suchbäume

Suchbäume sind eine Datenstruktur zur Speicherung von Schlüsseln, so daß insbesondere die Such- (oder Zugriffs-)Operation effizient ausführbar ist. Wir haben bisher keinerlei Annahmen über die *Zugriffshäufigkeiten* gemacht und vielmehr darauf geachtet, daß auch die zwei anderen Wörterbuchoperationen, das Einfügen und Entfernen von Schlüsseln, effizient ausführbar sind. In diesem Abschnitt gehen wir davon aus, daß die Schlüsselmenge fest vorgegeben ist und die Zugriffshäufigkeiten sowohl für die Schlüssel, die im Baum vorkommen, als auch für die nicht vorhandenen Objekte im vorhinein bekannt sind. Es wird das Problem diskutiert, wie man unter diesen Annahmen einen „optimalen“, d.h. die Suchkosten minimierenden Suchbaum konstruieren kann. Dazu werden zunächst ein Kostenmaß zur Messung der Suchkosten und der Begriff des optimalen Suchbaumes präzise definiert. Dann werden ein Verfahren zur Konstruktion optimaler Suchbäume angegeben und dessen Laufzeit und Speicherbedarf analysiert.

Im allgemeinen hat man nicht nur Schlüssel k_i , nach denen mit Häufigkeit a_i (erfolgreich) gesucht wird, sondern man nimmt an, daß auch die Häufigkeiten b_j bekannt sind, mit denen nach „nicht vorhandenen“ Objekten im Intervall (k_j, k_{j+1}) erfolglos gesucht wird. Wir gehen also von folgender Situation aus:

$S = k_1, \dots, k_N$ Menge von N verschiedenen Schlüsseln, $k_1 < k_2 < \dots < k_N$.

$a_i =$ (absolute) Häufigkeit, mit der nach k_i gesucht wird, $1 \leq i \leq N$.

$I = (k_0, k_{N+1})$ offenes Intervall aller Schlüssel, nach denen — erfolgreich oder erfolglos — gesucht wird; es gilt $k_0 < k_1$ und $k_N < k_{N+1}$. Typische Werte sind $k_0 = -\infty$ und $k_{N+1} = +\infty$.

$b_j =$ (absolute) Häufigkeit, mit der nach einem $x \in (k_j, k_{j+1})$ gesucht wird, mit $0 \leq j \leq N$.

In einem Suchbaum für S bezüglich I sind die k_i die Werte der inneren Knoten. Die Intervalle zwischen den Schlüsseln werden durch die Blätter repräsentiert. Als Maß für die gesamten Suchkosten eines Baumes nimmt man üblicherweise die *gewichtete Pfadlänge*, die mit Hilfe des Gewichtes eines Baumes definiert ist:

$$W = \sum_i a_i + \sum_j b_j$$

heißt das *Gewicht* des Baumes, und

$$P = \sum_{i=1}^N (\text{Tiefe}(k_i) + 1) a_i + \sum_{j=0}^N \text{Tiefe}(\text{Blatt}(k_j, k_{j+1})) b_j$$

heißt *gewichtete Pfadlänge* des Baumes.

Beispiel: Gegeben sei eine Menge von vier Schlüsseln mit folgenden Zugriffshäufigkeiten für die Schlüssel und Intervalle:

$(-\infty, k_1)$	k_1	(k_1, k_2)	k_2	(k_2, k_3)	k_3	(k_3, k_4)	k_4	(k_4, ∞)
4	1	0	3	0	3	0	3	10

Ein möglicher Suchbaum für diese Menge ist in Abbildung 5.77 angegeben. Der Baum hat die gewichtete Pfadlänge 48.

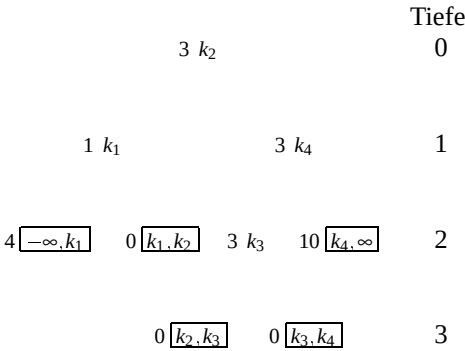


Abbildung 5.77

Die gewichtete Pfadlänge mißt, wieviele Schlüsselvergleiche für die erfolgreichen und erfolglosen Such-Operationen insgesamt ausgeführt werden. Jeden im Baum gespeicherten Schlüssel k_i findet man mit $\text{Tiefe}(k_i) + 1$ Schlüsselvergleichen wieder. Sucht man nach einem $x \in (k_j, k_{j+1})$, also nach einem Schlüssel, der im Baum nicht vorkommt, muß man bei der üblichen Implementation von Bäumen (Blätter werden durch **nil**-Zeiger in ihren Vätern repräsentiert) genau $\text{Tiefe}(k_j, k_{j+1})$ Schlüsselvergleiche ausführen, um festzustellen, daß x im Baum nicht vorkommt.

Bemerkung: Statt der absoluten Häufigkeiten verwendet man oft auch die relativen Suchhäufigkeiten $\alpha_i = a_i/W$ und $\beta_j = b_j/W$ und betrachtet statt P die *normierte gewichtete Pfadlänge* P/W .

Seien nun N Schlüssel $k_i, 1 \leq i \leq N$, mit Häufigkeiten $a_i, 1 \leq i \leq N$, ein Schlüsselintervall $I = (k_0, k_{N+1})$ mit $k_0 < k_1$ und $k_N < k_{N+1}$ und $b_j, 0 \leq j \leq N$, gegeben. Ein Suchbaum T für $S = \{k_1, \dots, k_N\}$ bezüglich I heißt *optimal*, wenn seine gewichtete Pfadlänge minimal ist unter allen Suchbäumen für S bezüglich I .

Wir wollen jetzt ein Verfahren zur Konstruktion optimaler Suchbäume angeben. Es beruht wesentlich auf der folgenden Beobachtung: Jeder Teilbaum eines optimalen Suchbaumes ist selbst ein optimaler Suchbaum.

Das folgt unmittelbar aus der folgenden, rekursiven Berechnungsmethode für die gewichtete Pfadlänge. Ist T ein Baum mit linkem Teilbaum T_l und rechtem Teilbaum T_r , so kann man die gewichtete Pfadlänge $P(T)$ des Baumes T wie folgt aus den gewichteten Pfadlängen $P(T_l)$ und $P(T_r)$, den Gewichten der Teilbäume und der Zugriffshäufigkeit für die Wurzel berechnen:

$$\begin{aligned} P(T) &= P(T_l) + \text{Gewicht}(T_l) \\ &\quad + \text{Zugriffshäufigkeit der Wurzel} \\ &\quad + P(T_r) + \text{Gewicht}(T_r) \\ &= P(T_l) + P(T_r) + \text{Gewicht}(T) \quad () \end{aligned}$$

Ist dabei für $S = k_1, \dots, k_N$ und $I = (k_0, k_{N+1})$ der Schlüssel an der Wurzel k_l , $1 \leq l \leq N$, so ergibt sich als Schlüsselmenge für den linken Teilbaum $S' = k_1, \dots, k_{l-1}$ und als Schlüsselintervall $I' = (k_0, k_l)$; entsprechend ergibt sich für den rechten Teilbaum $S'' = k_{l+1}, \dots, k_N$ und $I'' = (k_l, k_{N+1})$.

Falls T ein Blatt ist, gilt natürlich $P(T) = 0$.

Wir teilen nun den gesamten Suchraum $(-\infty, k_1) k_1 (k_1, k_2) k_2 \dots k_{N-1} (k_{N-1}, k_N) k_N (k_N, \infty)$ in immer größere, zusammenhängende Teile ein, für die wir jeweils einen optimalen Suchbaum konstruieren. D.h. wir berechnen größere optimale Teilbäume aus kleineren. Sei

$T(i, j)$ optimaler Suchbaum für $(k_i, k_{i+1}) k_{i+1} \dots k_j (k_j, k_{j+1})$,
 $W(i, j)$ das Gewicht von $T(i, j)$, also $W(i, j) = b_i + a_{i+1} + \dots + a_j + b_j$,
 $P(i, j)$ die gewichtete Pfadlänge von $T(i, j)$.

Wegen () kann man offenbar den optimalen Suchbaum $T(i, j)$ und seine gewichtete Pfadlänge $P(i, j)$ berechnen, sobald man den Index l der Wurzel von $T(i, j)$ kennt. Das zeigt Abbildung 5.78.

$T(i, j)$, $W(i, j)$, $P(i, j)$ sind definiert für alle $j \geq i$. Falls $j = i$ ist, besteht $T(i, j)$ nur aus dem Blatt (k_i, k_{i+1}) . Es gilt:

$$\begin{aligned} (i) \quad & \begin{cases} W(i, i) = b_i = \text{Häufigkeit, mit der nach } x \in (k_i, k_{i+1}) \\ \text{gesucht wird} & (0 \leq i \leq N) \\ W(i, j) = W(i, j-1) + a_j + b_j & (0 \leq i < j \leq N) \end{cases} \\ (ii) \quad & \begin{cases} P(i, i) = 0 & (0 \leq i \leq N) \\ P(i, j) = W(i, j) + \min_{i < l \leq j} P(i, l-1) + P(l, j) & (0 \leq i < j \leq N) \end{cases} \end{aligned}$$

Sei $r(i, j)$ diejenige Zahl, für die das Minimum angenommen wird, also der Index der Wurzel von $T(i, j)$. Gesucht ist $T(0, N)$; dieser Baum ist durch die Zahlen $r(i, j)$, $0 \leq i < j \leq N$, offenbar völlig bestimmt. Die beiden Gleichungen (i) und (ii) legen unmittelbar ein Verfahren zur (simultanen) Berechnung von $W(i, j)$, $P(i, j)$, $r(i, j)$ für alle i und j mit $0 \leq i < j \leq N$ nahe: Definieren wir die *Breite* h eines Baumes als die Anzahl der im Baum gespeicherten Schlüssel, so haben die Bäume $T(i, j)$ die Breite $h = j - i$ für alle i, j mit $0 \leq i < j \leq N$. Daher können wir $W(i, j)$, $P(i, j)$, $r(i, j)$ durch Induktion über $h = j - i$ wie folgt berechnen:

Fall 1 [$h = j - i = 0$]

Dann ist $j = i$, also $T(i, i) = \boxed{k_i, k_{i+1}}$, $0 \leq i \leq N$.

Setze $W(i, i) := b_i$, $P(i, i) := 0$, $r(i, i)$ undefiniert.

Fall 2 [$h = j - i = 1$]
Dann ist $j = i + 1$, und $T(i, i + 1)$ hat den Schlüssel k_{i+1} an der Wurzel, $0 \leq i < N$.
Abbildung 5.79 zeigt diese Situation.

Setze

$$\begin{aligned} W(i, i + 1) &:= W(i, i) + a_{i+1} + W(i + 1, i + 1), \\ P(i, i + 1) &:= W(i, i + 1), \\ r(i, i + 1) &:= i + 1. \end{aligned}$$

Fall 3 [$h = j - i \geq 2$]
Für jedes i, j mit $h = j - i \geq 2, 0 \leq i < j \leq N$, bestimmen wir dasjenige l (das größte, falls es mehrere gibt), $i < l \leq j$, für das $P(i, l - 1) + P(l, j)$ minimal wird. Wegen $(l - 1 - i) < h$ und $(j - l) < h$ können wir dabei voraussetzen, daß alle in Frage kommenden Werte $P(i, l - 1)$ und $P(l, j)$ bereits bekannt sind.

Setze

$$\begin{aligned} W(i, j) &:= W(i, l - 1) + W(l, j) + a_l, \\ P(i, j) &:= P(i, l - 1) + P(l, j) + W(i, j), \\ r(i, j) &:= l. \end{aligned}$$

Das beschriebene Verfahren benötigt drei Felder zur Speicherung der Werte von $W(i, j), P(i, j), r(i, j)$; es hat also Platzbedarf $\Theta(N^2)$.

Die Fälle $h = 0$ und $h = 1$ lassen sich in $O(N)$ Schritten erledigen. Zur Ausführung der im Fall $h \geq 2$ angegebenen Operationen reichen $O(N^3)$ Schritte. Um das einzusehen, können wir annehmen, daß alle Gewichte $W(i, j)$ für $0 \leq i \leq j \leq N$ bereits (in höchstens $O(N^2)$ Schritten) berechnet wurden. Dann beschreibt das folgende Programmstück die im Fall 3 auszuführenden Operationen:

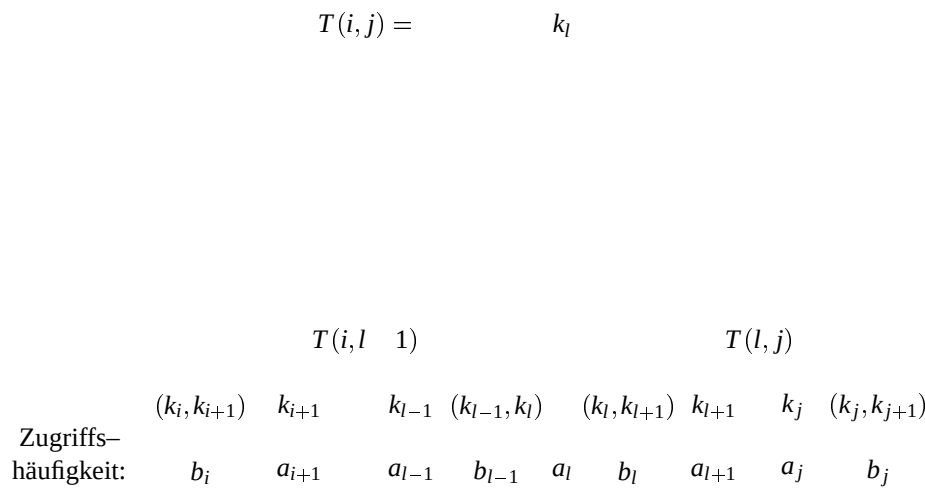


Abbildung 5.78

$$k_{i+1}$$

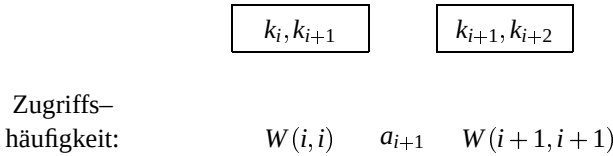


Abbildung 5.79

```

for  $h := 2$  to  $N$  do
  for  $i := 0$  to  $(N - h)$  do
    begin
       $j := i + h$ ;
      finde das (größte)  $l, i < l < j$ , für das
         $P(i, l - 1) + P(l, j)$  minimal wird;
       $P(i, j) := P(i, l - 1) + P(l, j) + W(i, j)$ ;
       $r(i, j) := l$ 
    end
  
```

In der inneren **for**-Schleife ist das Minimum von $h = j - i$ Elementen zu bestimmen; alle anderen Operationen sind jeweils in konstanter Schrittzahl ausführbar. Das ergibt folgende Abschätzung für die Gesamtschrittzahl:

$$\sum_{h=2}^N \sum_{i=0}^{N-h} O(h+1) = \sum_{h=2}^N O((N-h)(h+1)) = O(N^3)$$

Für große N ist das natürlich in vielen Fällen nicht effizient genug. Man erhält eine Verbesserung durch Ausnutzen des sogenannten *Monotonieprinzips*. Man kann zeigen, daß für alle i, j mit $0 \leq i < j \leq N$ gilt:

$$r(i, j - 1) \leq r(i, j) \leq r(i + 1, j)$$

(Einen Hinweis auf den Beweis findet man z.B. in )

Dann genügt es, in der inneren **for**-Schleife zum Auffinden desjenigen l , für das $P(i, l - 1) + P(l, j)$ minimal wird (bzw. des größten l mit dieser Eigenschaft, wenn es mehrere solche l gibt), l aus dem Bereich $r(i, j - 1) \leq l \leq r(i + 1, j)$ zu betrachten. Für festes h ist dann die innere **for**-Schleife in folgender Schrittzahl ausführbar:

$$\begin{aligned} &O\left(N-h+\sum_{i=0}^{N-h}(r(i+1,i+h)-r(i,i+h-1))+1\right) \\ &=O(N-h+r(1,h)-r(0,h-1)+1 \\ &\quad +r(2,h+1)-r(1,h)+1 \\ &\quad +r(3,h+2)-r(2,h+1)+1 \\ &\quad \vdots \\ &\quad +r(N-h+1,N)-r(N-h,N-1)+1) \\ &=O(N-h+N-h+1+\underbrace{(r(N-h+1,N)-r(0,h-1))}_{\leq N}) \\ &=O(N) \end{aligned}$$

Damit ist das Verfahren insgesamt in $O(N^2)$ Schritten ausführbar.

Beispiel (Fortsetzung): Wir geben die Belegung der Felder $W(i, j)$, $P(i, j)$, $r(i, j)$, mit $0 \leq i \leq j \leq 4$, für die am Anfang dieses Abschnitts und in Abbildung 5.77 angegebenen Schlüssel, Intervalle und Suchhäufigkeiten an. Zunächst berechnet man die Belegung des Feldes $W(i, j)$, vgl. Tabelle 5.3.

$W(i, j)$

$i \ j$	0	1	2	3	4
0	4	5	8	11	24
1		0	3	6	19
2			0	3	16
3				0	13
4					10

Tabelle 5.3

Nun berechnet man der Reihe nach für wachsendes $h := j - i$ die Werte von $P(i, j)$ und $r(i, j)$. Das Ergebnis zeigt Tabelle 5.4. Aus den Werten des Feldes $r(i, j)$ kann man den Suchbaum $T(0, 4)$ in Abbildung 5.80 ablesen. Dieser Baum $T(0, 4)$ hat die gewichtete Pfadlänge $P(0, 4) = 43$.

Leider sind die (absoluten oder relativen) Zugriffshäufigkeiten für eine konkrete Folge von Schlüsseln und Intervallen meistens nicht genau bekannt. Man ist dann auf Schätzungen angewiesen. Für sehr große N — man denke etwa an ein Lexikon mit vielen hunderttausend Einträgen — ist auch ein in $O(N^2)$ Schritten ausführbarer Algorithmus viel zu langsam. Statt einen optimalen Suchbaum nach der beschriebenen Methode

$P(i, j)$

$i \ j$	0	1	2	3	4
0	0	5	11	19	43
1		0	3	9	28
2			0	3	19
3				0	13
4					0

$r(i, j)$

$i \ j$	0	1	2	3	4
0	–	1	1	2	4
1		–	2	3	4
2			–	3	4
3				–	4
4					–

Tabelle 5.4

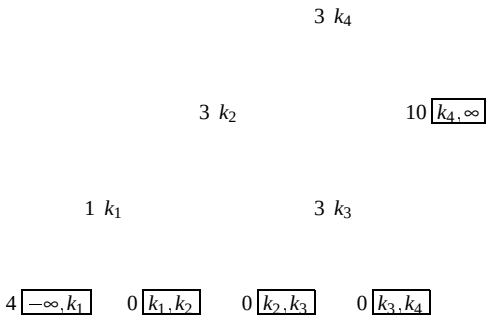


Abbildung 5.80

zu konstruieren, begnügt man sich daher damit, einen „fast optimalen“ Suchbaum möglichst effizient, d.h. möglichst in $O(N)$ Schritten zu konstruieren. Die bekannten Konstruktionsverfahren folgen meistens naheliegenden Strategien, wie z.B. der folgenden: Wähle die Wurzel eines Teilbaums stets so, daß die Summe der Zugriffshäufigkeiten für alle Schlüssel im linken und rechten Teilbaum sich möglichst wenig unterscheidet. Wir verzichten auf eine genauere Diskussion solcher Verfahren und eine präzise Definition des Begriffs „fast optimal“. Der interessierte Leser konsultiere z.B. [1].

5.8 Alphabetische und mehrdimensionale Suchbäume

Außer den bisher vorgestellten Varianten von Bäumen gibt es eine große Zahl weiterer. Wir diskutieren in diesem Abschnitt kurz sogenannte alphabetische Suchbäume (englisch: tries) und mehrdimensionale Suchbäume. In beiden Fällen wird nicht mehr

vorausgesetzt, daß die in einer Baumstruktur zu speichernden Daten durch je einen einzigen, als Einheit zu betrachtenden Schlüssel charakterisiert werden können. Alphabetische Suchbäume benutzen die Darstellung von Schlüsseln als Ziffern- oder Buchstabenfolgen; mehrdimensionale Suchbäume sind Strukturen zur Speicherung von Objekten, wie z.B. Punkten in der Ebene, die sich auf natürliche Weise durch zusammengesetzte Schlüssel, wie z.B. ein Paar kartesischer Koordinaten, charakterisieren lassen.

5.8.1 Tries

Das Wort „Trie“ wird üblicherweise wie das englische Wort „try“ gesprochen, um es vom Wort „tree“ unterscheiden zu können. Es hat seinen Ursprung im Wort *retrieval*, das auf die Verwendung von Tries als Suchstruktur hinweist. Wir erläutern die Tries zugrundeliegende Idee an einem Beispiel. Die Menge der Wörter *wer*, *weiß*, *wo*, *wir*, *sind* kann durch einen alphabetischen Suchbaum wie in Abbildung 5.81 repräsentiert werden.

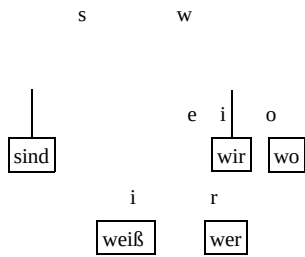


Abbildung 5.81

Wir fassen die Wörter also als Buchstabenfolgen auf, verzweigen genau dort, wo verschiedene Buchstaben unterschieden werden müssen, und erhalten so eine Struktur, in der wir sämtliche Wörter durch buchstabenweises Vergleichen wiederfinden können. Suchen wir etwa nach dem Wort „weiß“, folgen wir zunächst dem Verweis für den ersten Buchstaben, d.h. dem w-Zeiger, dann dem Verweis für den zweiten Buchstaben usw., bis wir bei dem gesuchten Wort angekommen sind. Das Suchen (und Einfügen) in alphabetischen Suchbäumen besteht also darin, im *i*-ten Schritt dem Zeiger für den *i*-ten Buchstaben zu folgen, und das solange zu wiederholen, bis man genügend Buchstaben inspiziert hat, um das gesuchte Wort von allen anderen im alphabetischen Suchbaum zu unterscheiden.

Das genannte Beispiel entspricht insofern nicht dem allgemeinen Fall, als kein Wort Präfix eines anderen ist. Beispielsweise ist es nicht ohne weiteres möglich, das Wort „Wort“ im obigen alphabetischen Suchbaum unterzubringen. Man macht daher zunächst alle Wörter künstlich, durch explizite Berücksichtigung des Leerzeichens, gleich

lang und kann dann *alle* Wörter einer gegebenen Länge h in einem alphabetischen Suchbaum der Höhe h mit n^h Blättern repräsentieren, wobei n die Alphabetgröße bezeichnet.

Soll, wie im angegebenen Beispiel, nur eine sehr kleine Teilmenge des riesigen Universums aller möglichen Schlüssel mit Länge h über dem Alphabet von n Buchstaben repräsentiert werden, wählt man natürlich eine möglichst speicherplatzsparende Implementation. Das bedeutet zweierlei. Erstens werden für großes n nicht alle n möglichen Verzweigungen explizit repräsentiert. Zweitens werden unäre Verweisketten (wie im Beispiel) soweit wie möglich verkürzt.

Besonderes Interesse haben binäre Tries, also alphabetische Suchbäume für Wörter über dem binären Alphabet $\{0, 1\}$ gefunden, weil sie eng mit binären Codes zusammenhängen, die zur Datenkomprimierung Verwendung finden. Offenbar kann man jeden binären Trie als binären Code-Baum für die Werte der Blätter auffassen, wie in folgendem Beispiel (vgl. Abbildung 5.82).

000 codiert „sind“, 10 codiert „wer“ usw. „Wer weiß wo wir sind“ wird also codiert durch 100010111000.

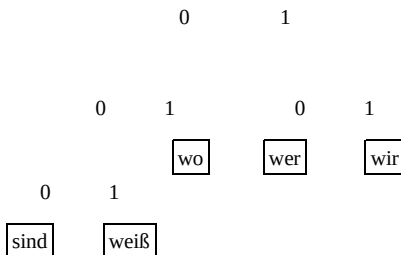


Abbildung 5.82

Ein Code ist dann besonders gut, wenn häufig vorkommende Wörter besonders kurze Codes haben. Es gibt eine Reihe von Verfahren, um für bekannte Häufigkeitsverteilungen in diesem Sinne „optimale“ Codes als binäre Tries zu finden.

5.8.2 Quadranten- und 2d-Bäume

Eine Menge angeordneter Schlüssel kann man auffassen als eine Menge von Punkten auf der Linie. Suchbäume sind also Strukturen zur Speicherung von Punkten im eindimensionalen Raum derart, daß sich die für Punkte typischen Operationen Suchen, Einfügen und Entfernen effizient ausführen lassen. Es gibt eine Reihe von Vorschlägen zur Verallgemeinerung von Suchbäumen. Sie haben zum Ziel, Punkte im 2-, 3-, — allgemein im k -dimensionalen Raum — so abzuspeichern, daß wieder die für Punkte typischen Operationen gut unterstützt werden. Natürlich kann man auch Punkte im k -dimensionalen Raum, $k \geq 2$, in gewöhnlichen Suchbäumen abspeichern. Man bildet

dazu einfach aus den k Koordinaten einen einzigen, den jeweiligen Punkt eindeutig charakterisierenden „Superschlüssel“ und benutzt diesen Schlüssel für die Operationen Suchen, Einfügen und Entfernen. Solange man also keine anderen Operationen ausführen will, besteht keine Notwendigkeit zur Verallgemeinerung von Suchbäumen. Typischerweise möchte man aber für Punkte im k -dimensionalen Raum, $k \geq 2$, auch andere Operationen ausführen. Beispiele für solche Operationen sind:

Bereichsanfrage (englisch: *range query*): Gegeben sei ein k -dimensionaler, rechteckiger Bereich (ein „Hyperrechteck“, wenn $k > 2$ ist). Die Aufgabe besteht darin, alle gespeicherten Punkte zu berichten, die in den Bereich fallen. Dabei wird üblicherweise angenommen, daß die Bereichsgrenzen parallel zu kartesischen Koordinaten sind.

Partielle Bereichssuche (englisch: *partial match query*): Gegeben sind i Koordinatenwerte, $i < k$. Gesucht sind alle gespeicherten Punkte, die für die gegebenen Koordinaten die gegebenen Werte und für die restlichen Koordinaten beliebige Werte haben.

Dies sind Beispiele für typisch geometrische Suchoperationen. Eine gut gewählte Suchstruktur sollte auf geometrische Nachbarschaftsbeziehungen möglichst Rücksicht nehmen, um solche geometrischen Operationen zu unterstützen. Wir besprechen zwei derartige Strukturen für den Fall $k = 2$. Die Verallgemeinerung für $k > 2$ ist offensichtlich. Wir erläutern, wie man eine Menge von Punkten in der Ebene der Reihe nach in einen anfangs leeren Quadranten-Baum bzw. 2d-Baum iteriert einfügt analog zum Einfügen in natürliche Bäume.

Quadranten-Bäume

Seien N Punkte $P_1, P_2, \dots, P_N$ in der Ebene gegeben. Die Punkte lassen sich wie folgt in einen Baum der Ordnung 4 einfügen. P_1 wird in der Wurzel gespeichert. Ein durch P_1 gelegtes Koordinatenkreuz zerlegt die Ebene in vier Quadranten (vgl. Abbildung 5.83).

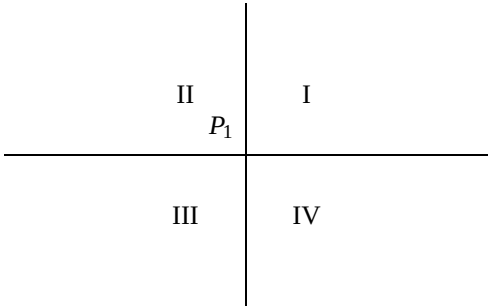


Abbildung 5.83

Die Wurzel erhält vier Zeiger auf Söhne, einen für jeden Quadranten. Der nächste Punkt P_2 wird i -ter Sohn von P_1 , wenn P_2 in den i -ten Quadranten bzgl. P_1 fällt. Entsprechend fährt man für die übrigen Punkte fort. D.h. der jeweils nächste Punkt wird i -ter Sohn seines Vaters, wenn er in den i -ten durch den Vater definierten Quadranten

fällt und der Vater nicht bereits einen i -ten Sohn besitzt. Hat der Vater schon einen i -ten Sohn, so wird das Einfügen bei diesem Sohn fortgesetzt.

Betrachten wir als Beispiel die sieben Punkte $A = (7, 9)$, $B = (15, 14)$, $C = (10, 5)$, $D = (3, 13)$, $E = (13, 6)$, $F = (17, 2)$, $G = (3, 2)$ in Abbildung 5.84.

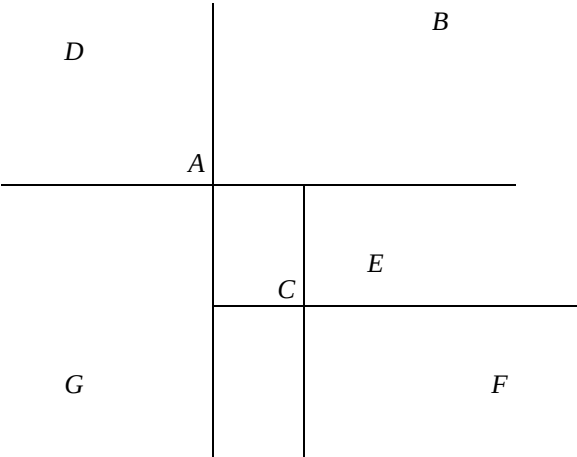


Abbildung 5.84

Fügt man diese Punkte der Reihe nach in den anfangs leeren Quadranten-Baum ein, erhält man Abbildung 5.85.

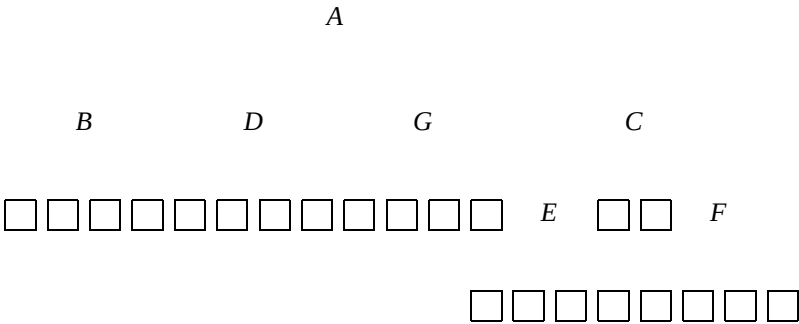


Abbildung 5.85

Es dürfte klar sein, wie man in einem Quadranten-Baum nach Punkten sucht oder weitere Punkte einfügt. (Das Entfernen von Punkten ist offenbar nicht so einfach, es sei denn, der zu entfernende Punkt hat nur Blätter als Söhne.) Zur Bestimmung aller Punkte in einem gegebenen, rechteckigen Bereich beginnt man bei der Wurzel und prüft, ob der dort gespeicherte Punkt im Bereich liegt. Dann setzt man die Bereichssuche bei all den Söhnen fort, deren zugehöriger Quadrant einen nichtleeren Durchschnitt mit dem gegebenen Bereich hat.

2d-Bäume

Wir bauen einen Binärbaum wie einen natürlichen Baum, wobei wir allerdings abwechselnd die x - und y -Koordinate der Punkte heranziehen, um die Position des jeweils nächsten Punktes im Baum zu bestimmen. Wir erläutern das Verfahren wieder am Beispiel derselben Menge von sieben Punkten in Abbildung 5.86.

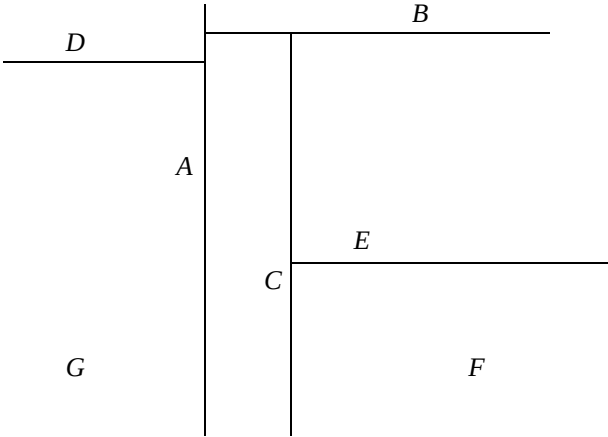


Abbildung 5.86

Beginnt man, zunächst nach x , dann nach y , dann wieder nach x usw. zu unterscheiden, ergibt sich durch iteriertes Einfügen der Punkte $A, \dots, G$ in den anfangs leeren Baum der 2d-Baum in Abbildung 5.87.

Wieder dürfte unmittelbar klar sein, wie man nach einem Punkt sucht bzw. wie man einen neuen Punkt in einen 2d-Baum einfügt. Das Entfernen von Punkten ist dagegen nicht so einfach. Bereichsanfragen werden offenbar dadurch unterstützt, daß eine Bereichssuche immer dann bei nur einem von zwei Söhnen fortgesetzt werden muß, wenn der Bereich ganz auf einer Seite der durch den Punkt definierten Trennlinie liegt.

Quadranten- und 2d-Bäume ebenso wie zahlreiche andere Strukturen zur mehrdimensionalen Suche sind intensiv studiert worden. Der interessierte Leser möge dazu etwa das Buch [] konsultieren.

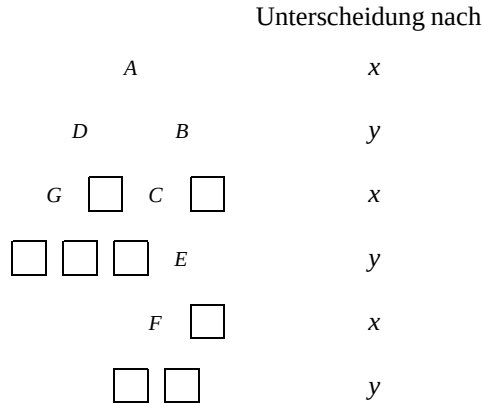


Abbildung 5.87

5.9 Aufgaben

Aufgabe 5.1

Gegeben sei die Folge F von acht Schlüsseln

$$F = 4, 8, 7, 2, 5, 3, 1, 6$$

- a) Geben Sie den zu F gehörenden natürlichen Baum an.
- b) Welcher Baum entsteht aus dem in a) erzeugten Baum, wenn man den Schlüssel 4 löscht?
- c) Geben Sie alle Folgen F' von acht Schlüsseln an, die die Eigenschaft haben, daß der zu F' gehörende natürliche Baum mit dem von F erzeugten übereinstimmt und F' wie folgt beginnt: $F' = 4, 2, 8, 7, \dots$

Aufgabe 5.2

- a) Geben Sie den natürlichen Baum an, der entsteht, wenn man der Reihe nach die Schlüssel 10, 5, 14, 9, 11, 12, 15, 6 in den anfangs leeren Baum einfügt.
- b) Ersetzen Sie in dem bei a) erhaltenen Baum die **nil**-Zeiger durch Verweise auf den symmetrischen Vorgänger (wenn der linke Sohn eines Knotens **nil** ist) bzw. Nachfolger (wenn der rechte Sohn eines Knotens **nil** ist), soweit diese existieren.
- c) Welcher Baum entsteht, wenn man Schlüssel 10 entfernt?

Aufgabe 5.3

Die Struktur eines Binärbaumes sei durch folgende Typvereinbarung festgelegt:

```

type Knotenzeiger = knoten;
      knoten = record
          key : integer;
          rechts, links : Knotenzeiger
      end;

```

Ein Baum sei durch einen Zeiger auf die Wurzel und der leere Baum sei durch einen **nil**-Verweis repräsentiert.

Schreiben Sie Funktionen, die die Anzahl der inneren Knoten, die gesamte Pfadlänge (das ist die Summe aller Abstände aller inneren Knoten von der Wurzel, gemessen in der Zahl der Kanten) und die Gesamtanzahl der Blätter berechnet.

Aufgabe 5.4

Binärbäume seien wie in Aufgabe 5.3 vereinbart; jedoch soll jeder Knoten zusätzlich eine Komponente *hoehe* besitzen. Wir nehmen an, daß jeder innere Knoten zwei Söhne besitzt. Beide Zeiger eines externen Knotens haben den Wert **nil**. Jeder Baum bestehe aus mindestens einem (externen) Knoten.

Ergänzen Sie die folgende Definition einer Funktion

```

function tiefstknoten(wurzel : Knotenzeiger) : Knotenzeiger;

```

in Pascal so, daß für das Argument *wurzel* als Funktionswert ein Zeiger auf einen externen Knoten mit maximaler Tiefe (Endpunkt eines Pfades maximaler Länge) in dem in *wurzel* wurzelnden Binärbaum berechnet wird.

```

function tiefstknoten(wurzel : Knotenzeiger) : Knotenzeiger;
var p : Knotenzeiger;
begin
    markhoehe(wurzel);
    p := wurzel;
    while      ...      do
        ...
    tiefstknoten := p
end

```

Ein Aufruf *markhoehe*(*wurzel*) bewirkt, daß der Komponente *hoehe* jedes Knotens *k* in dem Binärbaum mit Wurzel *wurzel* die Höhe des in *k* wurzelnden Teilbaums als Wert zugewiesen wird.

Aufgabe 5.5

Gegeben sei ein Binärbaum *B* mit ganzzahligen Schlüsseln. Gegeben sei außerdem ein Schlüssel *x*. Gesucht ist in *B* der größte Schlüssel $\leq x$.

- Geben Sie einen Algorithmus an, der diese Aufgabe in $O(h)$ Schritten löst, wenn *h* die Höhe von *B* ist.
- Setzen Sie die Vereinbarungen von Aufgabe 5.3 voraus und schreiben Sie in Pascal eine vollständige Funktion zu dem in a) entwickelten Algorithmus. Dabei können Sie davon ausgehen, daß für jedes *x* ein größter im Binärbaum gespeicherter Schlüssel mit Wert $\leq x$ stets vorkommt, da im Binärbaum ein „unechter“ Schlüssel mit Wert $-\infty$ gespeichert ist.

(Hinweis: Verwenden Sie einen Hilfszeiger, der stets am jeweils letzten Knoten stehenbleibt, an dem man beim Hinabsteigen im Baum rechts abgebogen ist.)

Aufgabe 5.6

Ein gefädelter Binärbaum sei durch einen Zeiger auf die Wurzel gegeben. Entwerfen Sie eine Pascal-Prozedur *feinfüge*, die beim Aufruf mit *feinfüge(wurzel, k)* den Schlüssel *k* unter Beibehaltung der Fädung in den Baum einfügt.

Aufgabe 5.7

Gegeben sei die Folge der Schlüssel eines sortierten Binärbaumes in Hauptreihenfolge:

20, 15, 5, 18, 17, 16, 25, 22

- Stellen Sie diesen Baum mit Vorgänger- und Nachfolger-Fädung graphisch dar.
- Geben die Reihenfolge der Schlüssel in Nebenreihenfolge an.

Aufgabe 5.8

Das Durchlaufen aller Knoten eines Baumes in „umgekehrter Hauptreihenfolge“ ist wie folgt definiert:

- Betrachte die Wurzel.
 - Durchlaufe den rechten Teilbaum der Wurzel in umgekehrter Hauptreihenfolge.
 - Durchlaufe den linken Teilbaum der Wurzel in umgekehrter Hauptreihenfolge.
- Gegeben sei der Binärbaum aus Abbildung 5.88 mit acht inneren Knoten (Blätter sind durch **nil**-Zeiger repräsentiert). Jeder innere Knoten hat ein unbesetztes Schlüsselfeld. Tragen Sie die Schlüssel 1, 2, ..., 8 so in diesen Baum ein, daß der Schlüssel die Knotennummer in umgekehrter Hauptreihenfolge ist.
 - Das Knotenformat eines Binärbaums sei wie in Aufgabe 5.3 vereinbart.
Ein nichtleerer binärer Baum mit einer festen Anzahl *N* von inneren Knoten sei gegeben durch einen Zeiger auf die Wurzel. Schreiben Sie eine Prozedur

procedure *numerierte* (**var** *wurzel* : *Knotenzeiger*);

die eine „Numerierung“ aller inneren Knoten (wie in a) beschrieben) in umgekehrter Hauptreihenfolge vornimmt.

- Wie kann man (eventuell durch Einführen zusätzlicher Zeiger anstelle von **nil**-Zeigern) die Speicherung von Bäumen analog zur Fädung für die symmetrische Reihenfolge so ändern, daß man einen Binärbaum in umgekehrter Hauptreihenfolge iterativ durchlaufen kann?

Aufgabe 5.9

Erstellen Sie eine rekursive Pascal-Prozedur *Pfad* (*p* : *Knotenzeiger*; *k* : *integer*), die für einen sortierten Binärbaum mit Zeiger *wurzel* auf die Wurzel beim Aufruf *Pfad(wurzel, k)* die Schlüsselwerte auf dem Pfad vom Knoten, der den Suchschlüssel *k* speichert, zur Wurzel in dieser Reihenfolge ausgibt. Es sei bei einem Aufruf *Pfad(wurzel, k)* garantiert, daß der Schlüssel *k* im Baum auftritt.

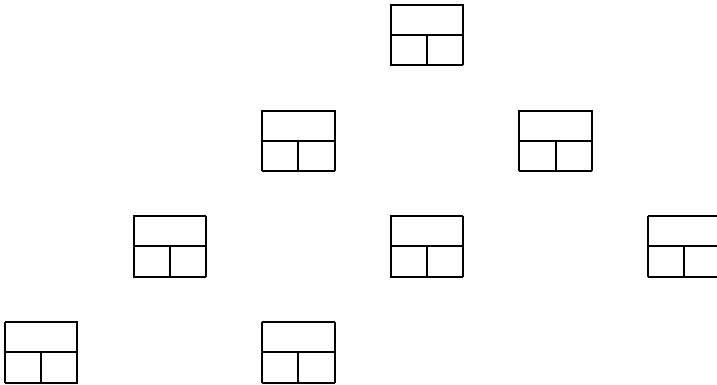


Abbildung 5.88

Aufgabe 5.10

a) Gegeben sei der in Abbildung 5.89 gezeigte Binärbaum mit vier inneren Knoten:

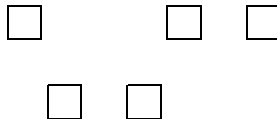


Abbildung 5.89

Geben Sie an, mit welcher Wahrscheinlichkeit dieser Baum durch sukzessives Einfügen der Schlüssel aus der Menge $1, 2, 3, 4$ in den anfangs leeren natürlichen Baum erzeugt wird, wenn jede Permutation der Schlüssel $1, \dots, 4$ als gleichwahrscheinlich vorausgesetzt wird.

- b) Mit welcher Wahrscheinlichkeit kommt der in a) angegebene Baum in der Menge aller strukturell verschiedenen Binärbäume mit vier inneren Knoten vor, wenn jeder sortierte Binärbaum mit Schlüsseln $1, \dots, 4$ als gleichwahrscheinlich vorausgesetzt wird?

Aufgabe 5.11

Gegeben sei der natürliche Baum aus Abbildung 5.90:

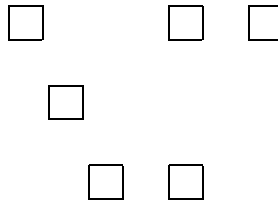


Abbildung 5.90

- a) Geben Sie alle Reihenfolgen von Schlüsseln an, die diesen natürlichen Baum erzeugen.
- b) Geben Sie alle übrigen strukturell möglichen Bäume mit gleicher Höhe und fünf inneren Knoten an.

Aufgabe 5.12

- a) Zeigen Sie, daß der vollständige natürliche Binärbaum mit sieben inneren Knoten von mindestens 49 Permutationen der Zahlen $1, \dots, 7$ erzeugt wird bei sukzessivem Einfügen der Schlüssel aus der Menge $1, 2, 3, \dots, 7$ in den anfangs leeren Baum.
- b) Geben Sie einen natürlichen Baum mit sieben inneren Knoten an, der nur genau einmal erzeugt wird.

Aufgabe 5.13

- a) Geben Sie alle natürlichen Bäume mit vier inneren Knoten an, die jeweils von genau einer Permutation der Zahlen $1, \dots, 4$ erzeugt werden.
- b) Geben Sie einen natürlichen Baum mit zehn inneren Knoten an, der von genau zwei Permutationen der Zahlen $1, \dots, 10$ erzeugt wird, und nennen Sie die Permutationen.

Aufgabe 5.14

Geben Sie den AVL-Baum an, der durch Einfügen der Schlüssel

10, 15, 11, 4, 8, 7, 3, 2, 13

in den anfangs leeren Baum entsteht.

Aufgabe 5.15

a) Ergänzen Sie die folgende Pascal-Funktionsdefinition so, daß als Funktionswert die Höhe des durch den Zeiger p auf die Wurzel gegebenen Baumes geliefert wird.

```
function hoehe ( $p$  : Knotenzeiger) : integer;  
var l, r : integer;
```

b) Ergänzen Sie die folgende Pascal-Funktionsdefinition so, daß der Wert *true* genau dann geliefert wird, wenn der durch den Zeiger p auf die Wurzel gegebene Baum höhenbalanciert ist. Die Funktion *hoehe* darf dabei verwendet werden.

```
function ausgeglichen ( $p$  : Knotenzeiger) : boolean;
```

Aufgabe 5.16

Gegeben sei der in Abbildung 5.91 gezeigte 1-2-Bruder-Baum:

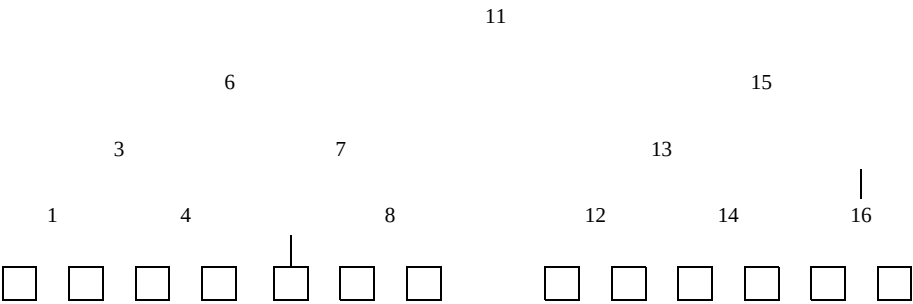


Abbildung 5.91

- a) Geben Sie den Baum an, der durch Einfügen des Schlüssels 2 entsteht (mit Zwischenschritten).
- b) Geben Sie den Baum an, der durch Entfernen des Schlüssel 11 aus dem ursprünglich gegebenen Baum entsteht.

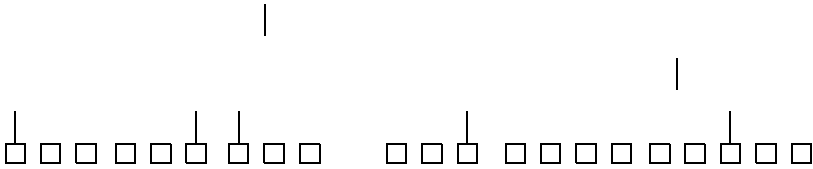


Abbildung 5.92

Aufgabe 5.17

- Gegeben sei der in Abbildung 5.92 gezeigte Bruder-Baum mit Höhe 5 und 21 Blättern.
Geben Sie eine Position unter den Blättern an, an der eine weitere Einfügung zu einer Umstrukturierung bis zur Wurzel hin und damit zu einem Wachstum der Höhe des Baumes um 1 führt.
- Welche Eigenschaft muß ein Bruder-Baum haben, so daß eine einzige weitere Einfügung zu einem Wachstum der Höhe führt?
- Wieviele Blätter muß ein Bruder-Baum mit Höhe h wenigstens haben, damit eine einzige weitere Einfügung an geeigneter Stelle zu einem Bruder-Baum mit Höhe $h + 1$ führen kann?
- Geben Sie für jede Höhe h einen Bruder-Baum mit Höhe h mit minimal möglicher Blattzahl und eine Position unter den Blättern an, so daß eine Einfügung an dieser Stelle zu einem Bruder-Baum mit Höhe $h + 1$ führt.

Aufgabe 5.18

- Geben Sie einen Bruder-Baum der Höhe 4 mit minimal möglicher Blattzahl an.
- Wieviele Schlüssel muß man mindestens einfügen, damit die Höhe des Baumes um 1 wächst? Wieviele Schlüssel kann man höchstens einfügen, ohne daß der Baum in der Höhe wächst?
- Geben Sie für den unter a) konstruierten Baum eine längstmögliche Folge von Schlüsseln an, derart, daß der durch ihr sukzessives Einfügen entstehende Baum nicht in der Höhe wächst. (Markieren Sie die Einfügestellen oder geben Sie explizit eine Schlüsselfolge an.)

Aufgabe 5.19

- a) Welche beiden Bruder-Bäume entstehen durch iteriertes Einfügen der Schlüssel $1, 2, \dots, 7$ und $1, 2, \dots, 15$ in den anfangs leeren Baum? Was kann man aufgrund dieser zwei Beispiele für eine aufsteigend sortierte Folge von $N = 2^k - 1$ ($k \geq 1$) Schlüsseln als Resultat der Einfügung mit Hilfe des Einfügeverfahrens für 1-2-Bruder-Bäume erwarten?
- b) Welche Folge von 1-2-Bruder-Bäumen wird erzeugt, wenn man der Reihe nach 7 Schlüssel in *absteigender* Reihenfolge in den anfangs leeren Baum einfügt? Geben Sie die Folge der 7 erzeugten Bäume an.
- c) Welche Änderung an dem in Abschnitt 5.2.2 angegebenen Verfahren zum Einfügen von Schlüsseln in 1-2-Bruder-Bäume bewirkt, daß beim iterierten Einfügen von Schlüsseln in absteigender Reihenfolge vollständige Binärbäume erzeugt werden?

Aufgabe 5.20

- a) Geben Sie an, welche 1-2-Bruder-Bäume mit fünf Schlüsseln (und sechs Blättern) durch Einfügen von fünf Schlüsseln in den anfangs leeren Baum entstehen können.
- b) Mit welcher Wahrscheinlichkeit treten die Bäume aus a) auf, wenn man eine zufällige Folge von fünf Schlüsseln in den anfangs leeren Baum iteriert einfügt? Es wird also angenommen, daß die dem jeweiligen Einfügeschritt vorangehende (erfolglose) Suche nach dem jeweils einzufügenden Schlüssel mit gleicher Wahrscheinlichkeit an jedem der Blätter des Baumes enden kann.

Aufgabe 5.21

Gegeben sei der in Abbildung 5.93 gezeigte 1-2-Bruder-Baum mit drei Schlüsseln (durch Punkte angedeutet) und Höhe 2.

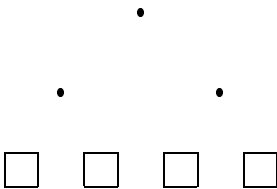


Abbildung 5.93

Geben Sie an, mit welcher Wahrscheinlichkeit daraus ein 1-2-Bruder-Baum mit sieben Schlüsseln und Höhe 4 durch Einfügen weiterer vier Schlüssel entsteht. Dabei wird vorausgesetzt, daß der jeweils nächste einzufügende Schlüssel mit derselben Wahrscheinlichkeit in jedes der Schlüsselintervalle des gegebenen Baumes fällt.

Aufgabe 5.22

Gegeben sei ein zufällig erzeugter 1-2-Bruder-Baum mit N Schlüsseln. Geben Sie die Wahrscheinlichkeit dafür an, daß

- die Umstrukturierung (mit Hilfe der Prozedur *up*) bereits nach dem ersten Schritt abbricht.
- die Umstrukturierung wenigstens noch Knoten auf dem zweituntersten Niveau innerer Knoten betrifft.

Aufgabe 5.23

Eine Folge $S = s_1, \dots, s_N$ von N Schlüsseln ist wie folgt auf Blöcke von je zwei oder drei Schlüsseln aufzuteilen: Man schafft im ersten Schritt den Block s_1, ∞ . Dabei ist ∞ ein „Pseudoschlüssel“, der größer als alle in S auftretenden Schlüssel ist. Hat man bereits die Blöcke $B_1, \dots, B_k$ erzeugt, so ist die in der Reihenfolge der Blöcke und innerhalb der Blöcke von links nach rechts vorkommende Folge von Schlüsseln aufsteigend sortiert. Der nächste Schlüssel s wird jeweils so in diese Folge eingefügt, daß man versucht, ihn in den von links her ersten Block einzufügen, der einen Schlüssel größer als s enthält. Hat dieser Block bereits drei Schlüssel, so zerlegt man ihn in zwei Blöcke mit je zwei Schlüsseln.

Beispiel: $S = 3, 2, 1, 5, 4$

$$\begin{aligned} \boxed{3, \infty} &= {}_2 \quad \boxed{2, 3, \infty} = {}_1 \quad \boxed{1, 2} \quad \boxed{3, \infty} \\ &= {}_5 \quad \boxed{1, 2} \quad \boxed{3, 5, \infty} = {}_4 \quad \boxed{1, 2} \quad \boxed{3, 4} \quad \boxed{5, \infty} \end{aligned}$$

Berechnen Sie die mittlere Anzahl von Blöcken der Größe 2 und 3 nach N Einfügungen unter der Annahme, daß jede der $N!$ möglichen Anordnungen von N Schlüsseln gleichwahrscheinlich ist.

Aufgabe 5.24

- Geben Sie die Struktur eines höhenbalancierten Baumes der Höhe 4 an, für den die Wurzelbalance (Verhältnis der Anzahl der Blätter des linken Teilbaums zur Gesamtblätterzahl) möglichst klein ist.
- Zeigen Sie: Es ist möglich, höhenbalancierte Bäume mit Höhe h anzugeben, für die die Wurzelbalance mit wachsender Höhe h beliebig klein wird.

Aufgabe 5.25

- Fügen Sie die Punkte 7, 19, 23, 4, 12, 17, 8, 11, 2, 9 und 13 in einen anfangs leeren B-Baum der Ordnung 3 ein.
- Entfernen Sie die Punkte 12 und 17.
- Welchen Aufwand benötigt man zum Entfernen eines Schlüssels im mittleren (schlechtesten) Fall?

Literaturliste zu Kapitel 5: Bäume

Seite 256

[32] J. Culberson. The effect of updates in binary search trees. In Proc. 17th ACM Annual Symposium on Theory of Computing, Providence, Rhode Island, S. 205-212, 1985.

Seite 260

[1] G. M. Adelson-Velskii und Y. M. Landis. An algorithm for the organization of information. Doklady Akademia Nauk SSSR, 146:263-266, 1962. English Translation: Soviet Math. 3, 1259-1263.

Seite 273

[138] Th. Ottmann, H.-W. Six und D. Wood. On the correspondence between AVL trees and brother trees. Computing, 23:43-54, 1979.

Seite 279

[33] K. Culik, Th. Ottmann und D. Wood. Dense multiway trees. ACM Trans. Database Systems, 6:486-512, 1981.

Seite 285

[196] A.C. Yao. On random 2-3 trees. Acta Informatica, 9:159-170, 1978.

Seite 289

[131] J. Nievergelt und E. M. Reingold. Binary search trees of bounded balance. SIAM Journal on Computing, 2:33-43, 1973.

[128] I. Nievergelt und C. K. Wong. On binary search trees. In Proc. IFIP Congress 71 North-Holland Publishing Co., Amsterdam, S. 91-98, 1972.

Seite 291

[123] K. Mehlhorn. Datenstrukturen und effiziente Algorithmen, Band 1, Sortieren und Suchen. Teubner, Stuttgart, 1986.

[131] J. Nievergelt und E. M. Reingold. Binary search trees of bounded balance. SIAM Journal on Computing, 2:33-43, 1973.

Seite 294

[123] K. Mehlhorn. Datenstrukturen und effiziente Algorithmen, Band 1, Sortieren und Suchen. Teubner, Stuttgart, 1986.

Seite 296

[123] K. Mehlhorn. Datenstrukturen und effiziente Algorithmen, Band 1, Sortieren und Suchen. Teubner, Stuttgart, 1986.

[10] C. R. Aragon und R. G. Seidel. Randomized search trees. In Proc. 30th IEEE Symposium on Foundations of Computer Science, S. 540-545, 1989.

[93] D. C. Kozen. The Design and Analysis of Algorithms. Springer, New York u.a., 1991. Texts and Monographs in Computer Science.

Seite 297

[119] E. M. McCreight. Efficient algorithms for enumerating intersecting intervals and rectangles. Technical Report PARC CSL-80-9, Xerox Palo Alto Res. Ctr., Palo Alto, CA, 1980.

Seite 304

[10] C. R. Aragon und R. G. Seidel. Randomized search trees. In Proc. 30th IEEE Symposium on Foundations of Computer Science, S. 540-545, 1989.

Seite 305

[7] B. Allen und J. I. Munro. Selforganizing search trees. J. Assoc. Comput. Mach., 25(4):526-535, 1978.

Seite 312

[173] D. D. Sleator und R. E. Tarjan. Self-adjusting binary search trees. Journal of the ACM, 32:652-686, 1985.

Seite 320

[89] D. E. Knuth. The Art of Computer Programming, Vol.3: Sorting and Searching. Addison-Wesley, Reading, Massachusetts, 1973.

Seite 326

[196] A. C. Yao. On random 2-3 trees. Acta Informatica, 9:159-170, 1978.

Seite 327

[33] K. Culik, Th. Ottmann und D. Wood. Dense multiway trees. ACM Trans. Database Systems, 6:486-512, 1981.

Seite 328

[33] K. Culik, Th. Ottmann und D. Wood. Dense multiway trees. ACM Trans. Database Systems, 6:486-512, 1981.

[118] H. A. Maurer, Th. Ottmann und H.-W. Six. Implementing dictionaries using binary trees of very small height. Information Processing Letters, 5(1):11-14, 1976.

[79] D. S. Hirschberg. An insertion technique for one-sided height-balanced trees. Comm. ACM, 19:471-473, 1976.

[200] S. H. Zweben und M. A. McDonald. An optimal method for deletion in one-sided height-balanced trees. Comm. ACM, 21:441-445, 1978.

[137] Th. Ottmann, H.-W. Six und D. Wood. Right brother trees. Comm. ACM, 21:769-776, 1978.

[156] K.R. Räihä und S. H. Zweben. An optimal insertion algorithm for one-sided height-balanced binary search trees. Comm. ACM, 22:508-512, 1979.

[138] Th. Ottmann, H.-W. Six und D. Wood. On the correspondence between AVL trees and brother trees. Computing, 23:43-54, 1979.

Seite 329

[70] L. J. Guibas und R. Sedgewick. A dichromatic framework for balanced trees. In Proc. 19th Annual Symposium on Foundations of Computer Science, Ann Arbor, Michigan, S. 8-21, 1978.

[106] J. van Leeuwen und H. M. Overmars. Stratified balanced search trees. Acta Informatica, 18:345-359, 1983.

Seite 333

[106] J. van Leeuwen und H. M. Overmars. Stratified balanced search trees. Acta Informatica, 18:345-359, 1983.

[141] Th. Ottmann und D. Wood. A comparison of iterative and defined classes of search trees. International Journal of Computer and Information Sciences, 11:155-178, 1982.

[135] H. Olivie'. A new class of balanced search trees: Half-balanced binary search trees. RAIRO Informatique The'orique, 16:51-71, 1982.

[181] R. E. Tarjan. Updating a balanced search tree in $O(1)$ rotations. Information Processing Letters, 16:253-257, 1983.

Seite 335

[86] J. L. W. Kessels. On-the-fly optimization of data structures. In Comm. ACM, 26, S. 895-901, 1983.

- [99] K. Larsen und R. Fagerberg. B-trees with relaxed balance. In Proc. 9th International Parallel Processing Symposium, IEEE Computer Society Press, S. 196-202, 1995.
- [98] K. Larsen. AVL trees with relaxed balance. In Proc. 8th International Parallel Processing Symposium, IEEE Computer Society Press, S. 888-893, 1994.
- [132] O. Nurmi und E. Soisalon Soininen. Uncoupling updating and rebalancing in chromatic binary trees. In Proc. 10th ACM Symposium on Principles of Database Systems, S. 192-198, 1991.
- [133] O. Nurmi, E. Soisalon Soininen und D. Wood. Concurrency control in database structures with relaxed balance. In Proc. 6th ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, San Diego, California, S. 170- 176, 1987.
- [74] S. Hanke, Th. Ottmann und E. Soisalon-Soininen. Relaxed Balancing Made Simple. Technical report, Institut für Informatik, Universität Freiburg, Germany and Laboratory of Information Processing Science, Helsinki University, Finland, 1996. (anonymous ftp from <ftp.informatik.uni-freiburg.de> in directory /documents/reports/report71/) (<http://hyperg.informatik.uni-freiburg.de/Report71>).

Seite 336

- [12] R. Bayer. Symmetric binary B-trees: Data structures and maintenance algorithms. Acta Informatica, 1:290-306, 1972.
- [134] H. Olivie'. A Study of Balanced Binary Trees and Balanced One-Two Trees. PhD thesis, University of Antwerpen, 1980.
- [70] L. J. Guibas und R. Sedgewick. A dichromatic framework for balanced trees. In Proc. 19th Annual Symposium on Foundations of Computer Science, Ann Arbor, Michigan, S. 8-21, 1978.

Seite 351

- [174] L. Snyder. On uniquely represented data structures. In Proc. 18th Annual Symposium on Foundations of Computer Science, Providence, Rhode Island, S. 142- 147, 1977.

Seite 352

- [9] A. Andersson und Th. Ottmann. New tight bounds on uniquely represented dictionaries. In SIAM Journal of Computing, volume 24, S. 1091-1103, October 1995.
- [174] L. Snyder. On uniquely represented data structures. In Proc. 18th Annual Symposium on Foundations of Computer Science, Providence, Rhode Island, S. 142- 147, 1977.

Seite 353

- [9] A. Andersson und Th. Ottmann. New tight bounds on uniquely represented dictionaries. In SIAM Journal of Computing, volume 24, S. 1091-1103, October 1995.

Seite 356

- [174] L. Snyder. On uniquely represented data structures. In Proc. 18th Annual Symposium on Foundations of Computer Science, Providence, Rhode Island, S. 142- 147, 1977.

Seite 360

- [89] D. E. Knuth. The Art of Computer Programming, Vol.3: Sorting and Searching. Addison-Wesley, Reading, Massachusetts, 1973.

Seite 362

- [123] K. Mehlhorn. Datenstrukturen und effiziente Algorithmen, Band 1, Sortieren und Suchen. Teubner, Stuttgart, 1986.

Seite 367

- [122] K. Mehlhorn. Data structures and algorithms, Vol. 3: Multidimensional searching and computational geometry. Springer, Berlin, 1984.

Kapitel 6

Manipulation von Mengen

Datenstrukturen zur Repräsentation einer Kollektion von Datenmengen, auf der gewisse Operationen ausgeführt werden sollen, wurden erstmals von Aho, Hopcroft und Ullman systematisch behandelt. Die abstrakte Behandlung solcher Mengenmanipulationsprobleme erleichtert in vielen Fällen den Entwurf und die Analyse von Algorithmen aus verschiedenen Anwendungsgebieten. Man formuliert Algorithmen zunächst auf hohem Niveau unter Rückgriff auf Strukturen und Operationen zur Manipulation von Mengen, die in herkömmlichen Programmiersprachen üblicherweise nicht vorkommen. In einem zweiten Schritt überlegt man sich dann, wie die Kollektion von Datenmengen und die benötigten Operationen implementiert, also programmtechnisch realisiert werden können. Besonders erfolgreich war dieser Ansatz bei der Verbesserung und Neuentwicklung von Algorithmen auf Graphen. Beispiele sind Verfahren zur Berechnung spannender Bäume, kürzester Pfade und maximaler Flüsse, vgl. hierzu auch das Kapitel 8 und die Monographie von Tarjan [1].

Einen wichtigen Spezialfall eines Mengenmanipulationsproblems, das sogenannte Wörterbuchproblem, haben wir im Kapitel 1 und besonders im Kapitel 5 bereits ausführlich behandelt. Dort ging es um die Frage, wie eine Menge von Schlüsseln abgespeichert werden soll, damit die Operationen Suchen (Zugriff), Einfügen und Entfernen von Schlüsseln möglichst effizient ausführbar sind. Wir werden sehen, daß die im Kapitel 5 zur Lösung des Wörterbuchproblems benutzten Bäume auch für viele andere Mengenmanipulationsprobleme benutzt werden können.

Wir gehen in diesem Kapitel davon aus, daß die Datenmengen stets Mengen ganzzahliger Schlüssel sind, obwohl die Schlüssel in den meisten Anwendungen lediglich zur eindeutigen Identifizierung der „eentlichen“ Information dienen.

Neben dem bereits genannten Wörterbuchproblem sind zwei Spezialfälle des Mengenmanipulationsproblems in der Literatur besonders ausführlich behandelt worden: Vorrangwarteschlangen (Priority Queues), die im Abschnitt 6.1 behandelt werden, und Union-Find-Strukturen, die im Abschnitt 6.2 diskutiert werden. Im Abschnitt 6.3 geben wir einen allgemeinen Rahmen zur Behandlung von Mengenmanipulationsproblemen an und zeigen Möglichkeiten zur Lösung solcher Probleme mit Hilfe verschiedener Klassen von Bäumen auf.

6.1 Vorrangwarteschlangen

Als *Vorrangwarteschlange* (englisch: *priority queue*) bezeichnet man eine Datenstruktur zur Speicherung einer Menge von Elementen, für die eine Ordnung (Prioritätsordnung) definiert ist, so daß folgende Operationen ausführbar sind: *Initialisieren* (der leeren Struktur), *Einfügen* eines Elementes (Insert), *Minimum Suchen* (Access Min), *Minimum Entfernen* (Delete Min). Wir nehmen der Einfachheit halber an, daß die Elemente ganzzahlige Schlüssel sind und die Prioritätsordnung die übliche Anordnung ganzer Zahlen ist.

Der Begriff Vorrangwarteschlange erinnert an offensichtliche Anwendungen für solche Strukturen. Man denke an Kunden, die vor Kassen warten, an Aufträge, die auf ihre Ausführung warten, an Akten, die im Bearbeitungsstapel eines Sachbearbeiters auf ihre Erledigung warten. Die Prioritätsordnung ist hier durch den Ankunftszeitpunkt oder die Dringlichkeit festgelegt; die zeitlich ersten (frühesten) oder dringendsten Ereignisse haben Vorrang vor anderen.

Der Begriff Priority Queue wurde von Knuth  eingeführt. Andere Autoren, z.B.  und , benutzen den Begriff *Heap* (Halde), den wir in Abschnitt 2.3 für eine spezielle Datenstruktur reserviert haben, die im Sortierverfahren Heapsort verwendet wurde. Selbstverständlich sind Heaps eine mögliche Implementation für Priority Queues. Ein Heap mit N Schlüsseln erlaubt das *Einfügen* eines neuen Elementes und das *Entfernen des Minimums* in $O(\log N)$ Schritten; da das Minimum stets am Anfang des Heaps steht, kann die Operation *Access Min* in konstanter Zeit ausgeführt werden. In Abschnitt 2.3 haben wir nicht das Minimum, sondern das Maximum aller Schlüssel am Anfang des Heaps gespeichert. Dies gibt einfach eine andere Prioritätsordnung über den Schlüsseln ($>$ statt $<$) wieder und kann offensichtlich ebenso leicht im Heap realisiert werden.

Neben den genannten Operationen wird häufig verlangt, daß für Priority Queues weitere Operationen ausführbar sind und zwar: das *Entfernen beliebiger Elemente*, also nicht nur des Minimums, und das *Herabsetzen* eines Schlüssels um einen vorgegebenen Wert (*Decrease-Key-Operation*). Hierbei wird allerdings in der Regel vorausgesetzt, daß die Position des Schlüssels, den man entfernen möchte oder dessen Wert man erniedrigen möchte, bekannt ist; d.h. den Aufwand, den betreffenden Schlüssel in der Priority Queue zu finden, läßt man bei diesen Operationen außer Betracht. Schließlich verlangt man häufig, daß das *Zusammenfügen* (*Verschmelzen*) zweier elementfremder Priority Queues (Operation *Meld* oder *Merge*) möglich ist.

Zwar lassen sich alle diese Operationen auch für die im Abschnitt 2.3 eingeführten Heaps ausführen. Weil Heaps aber eine sehr starre Struktur haben, ist es besonders schwierig, zwei Heaps schnell zu einem neuen zusammenzufügen. Zwei offensichtliche Möglichkeiten sind jedoch die folgenden. Erstens kann man sämtliche Elemente des kleineren Heaps in den größeren einfügen. Das ist in $O(k \log(N+k))$ Schritten ausführbar, wobei k die Anzahl der Elemente des kleineren Heaps und N die des größeren Heaps ist. Die zweite Möglichkeit ist, die vorhandenen Strukturen aufzulösen und einen neuen Heap mit allen $N+k$ Elementen aufzubauen. Der Aufbau ist in $O(N+k)$ Schritten durchführbar.

In Abschnitt 6.1.1 geben wir zunächst ein Beispiel für die Verwendung von Priority Queues an. In Abschnitt 6.1.2 zeigen wir dann, wie man Priority Queues mit Hilfe bereits bekannter Strukturen implementieren kann. Schließlich führen wir in den folgenden Abschnitten eine Reihe neuer Strukturen ein, die zeigen, daß Priority Queues sehr einfach und effizient implementiert werden können.

Man beachte, daß in keinem Fall die Operation des Suchens eines Schlüssels besonders unterstützt wird.

6.1.1 Dijkstras Algorithmus zur Berechnung kürzester Wege

Als einziges Beispiel eines Algorithmus, der mit Hilfe von Operationen für Priority Queues bequem formuliert werden kann, wollen wir ein Verfahren zur Berechnung kürzester Wege in gerichteten Graphen diskutieren. Eine ausführlichere Behandlung von Algorithmen für Graphen erfolgt im Kapitel 8.

Gegeben sei ein gerichteter Graph G mit Knotenmenge V und Kantenmenge E . Wir verzichten hier auf eine formale Definition von Graphen und von Begriffen im Zusammenhang mit Graphen; der interessierte Leser sei auf Kapitel 8 verwiesen. Man stelle sich einfach ein Netz von Einbahnstraßen zwischen Orten vor. Die Orte bilden die Knotenmenge V , und die Einbahnstraßen sind die gerichteten Kanten zwischen den Orten. Jede Kante e hat eine nichtnegative Länge $l(e)$.

Wir wollen ein Problem lösen, das in der Literatur unter dem Namen *Single-source-shortest-paths-Problem* (oder *one-to-all shortest paths*, vgl. [1]) bekannt ist. Gegeben sei ein Knoten (ein Startort) s . Die Aufgabe besteht darin, für jeden Knoten $v \in V$ des Graphen G den kürzesten Weg von s nach v zu bestimmen. Wir begnügen uns damit, nicht den Weg selbst, sondern nur seine Länge zu bestimmen. Wir setzen der Einfachheit halber voraus, daß jeder Knoten $v \in V$ auch durch wenigstens einen Weg von s aus erreichbar ist. Abbildung 6.1 zeigt ein Beispiel für einen solchen Graphen; die Länge $l(e)$ einer Kante e ist jeweils als Beschriftung der Kante e angegeben.

Seien also ein Graph G mit Knotenmenge V und ein Knoten $s \in V$ gegeben. Um zu jedem Knoten $v \in V$ die Länge eines kürzesten Weges von s nach v zu bestimmen, könnte man natürlich sämtliche Wege von s nach v betrachten und unter diesen den mit kürzester Länge auswählen. Das ist aber höchstens für Graphen mit sehr wenigen Knoten und Kanten noch praktikabel.

Bereits 1959 hat Dijkstra [2] ein wesentlich effizienteres Verfahren vorgeschlagen. Wir skizzieren das Verfahren jetzt, ohne auf alle Implementationsdetails einzugehen und ohne die Korrektheit des Verfahrens zu begründen.

Das Verfahren besteht darin, sukzessive für jeden Knoten $v \in V$ den kürzesten Weg von s nach v zu bestimmen. Dazu wird eine Menge S von Knoten betrachtet und schrittweise vergrößert, für die der kürzeste Weg von s aus bereits bekannt ist. Jedem Knoten $v \in V$ wird eine vorläufige *Distanz* $d(v)$ zugeordnet. Falls $v \in S$ ist, ist $d(v)$ auch bereits die Länge des kürzesten Weges von s nach v . Falls $v \notin S$ ist, so ist $d(v)$ die Länge eines kürzesten Weges der Form $s \dots ww$, mit $w \in S$, d.h. $d(v) = \min\{\{d(w) + l(wv); w \in S\}\}$ bzw. $d(v) = \infty$, falls ein solcher Weg nicht existiert.

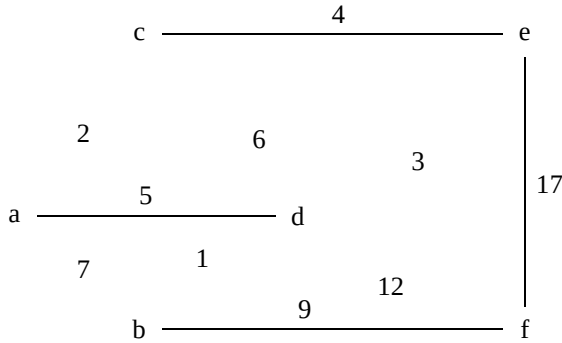


Abbildung 6.1

Anfangs ist $d(s) = 0$, und für alle von s verschiedenen Knoten $v \in V$ ist $d(v) = \infty$, und S ist leer. Dann wird S nach dem Prinzip „Knoten mit kürzester Distanz von s zuerst“ schrittweise wie folgt vergrößert, bis S alle Knoten V des Graphen enthält:

1. Wähle Knoten $v \in V \setminus S$ mit minimaler Distanz $d(v)$.
2. Nimm v zu S hinzu.
3. Für jede Kante vw von v zu einem Knoten $w \notin S$ ersetze $d(w)$ durch $\min(\{d(w), d(v) + l(vw)\})$.

Wir implementieren $V \setminus S$ als Priority Queue und wählen als Prioritäten (Schlüssel) die Distanzen $d(v)$. Wir denken uns ferner für jeden Knoten $v \in V$ die Menge aller Knoten $w \in V$ mit $vw \in E$ in einer *Nachfolgermenge* $N(v)$ zusammengefaßt. Dann kann man das soeben informell beschriebene Verfahren von Dijkstra etwas genauer wie folgt formulieren:

procedure *shortestpath* $((V, E) : \text{Graph}; s : \text{Knoten});$
 {berechnet zum Graphen mit Knotenmenge V und Kantenmenge E
 und zu $s \in V$ für jeden Knoten $v \in V$ die
 Länge $d(v)$ des kürzesten Weges von s nach v }
begin
 {Initialisierung}
for all $v \in V \setminus \{s\}$ **do** $d(v) := \infty;$
 $d(s) := 0;$ $S := \emptyset;$
 Initialisiere $V \setminus S := V$ als Priority Queue, geordnet nach Distanzen
 $d(v), v \in V;$
 {anfangs ist also s minimales Element in $V \setminus S$; es folgen alle
 übrigen Elemente von V in beliebiger Reihenfolge}
 {vergrößern von S nach dem Prinzip: Knoten mit kürzester Distanz
 von s zuerst}
while $V \setminus S \neq \emptyset$ **do**
 begin

```
(*)      v := min(V \ S); deletemin(V \ S);
        S := S ∪ {v};
        for all w ∈ N(v) \ S do
            if d(v) + l(vw) < d(w)
(**)      then decreasekey(w, d(v) + l(vw))
        end
    end
```

Wir verfolgen diesen Algorithmus am Beispiel des Graphen aus Abbildung 6.1 und Startknoten *a*. Dazu geben wir die Mengen *S* und *V \ S*, durch einen Strich getrennt, nach der Initialisierungsphase und nach jedem Durchlauf der **while**-Schleife an. *V \ S* ist von links nach rechts nach aufsteigenden Distanzen geordnet. Ferner geben wir jeweils für die Knoten der Nachfolgermenge *N(v)* des minimalen Elementes *v* ∈ *V \ S* die Distanzen von *s* an. Der Ablauf des Verfahrens ist in Tabelle 6.1 zusammengefaßt. Das Ergebnis kann man in der letzten Zeile ablesen.

$(v, d(v))$ für $v \in S$ $(v, d(v))$ für $v \in V \setminus S$	für $v = \min(V \setminus S)$ und $w \in N(v) \setminus (S \cup \{v\})$: $(w, l(vw))$ $\min(d(w), d(v) + l(vw))$
$\emptyset$ $(a, 0), (b, \infty), (c, \infty), (d, \infty), (e, \infty), (f, \infty)$	$(b, 7), (c, 2), (d, 5)$ $(b, 7), (c, 2), (d, 5)$
$(a, 0)$ $(c, 2), (d, 5), (b, 7), (e, \infty), (f, \infty)$	$(e, 4)$ $(e, 6)$
$(a, 0), (c, 2)$ $(d, 5), (e, 6), (b, 7), (f, \infty)$	$(b, 1), (f, 12)$ $(b, 6), (f, 17)$
$(a, 0), (c, 2), (d, 5)$ $(e, 6), (b, 6), (f, 17)$	$(f, 17)$ $(f, 17)$
$(a, 0), (c, 2), (d, 5), (e, 6)$ $(b, 6), (f, 17)$	$(f, 9)$ $(f, 15)$
$(a, 0), (c, 2), (d, 5), (e, 6), (b, 6)$ $(f, 15)$	<u>$\emptyset$</u>
$(a, 0), (c, 2), (d, 5), (e, 6), (b, 6), (f, 15)$ $\emptyset$	

Tabelle 6.1

Die Zeit, die das Verfahren für einen Graphen mit *n* Knoten und *m* Kanten benötigt, hängt offenbar ganz entscheidend davon ab, welche Zeit für die Bestimmung und das Entfernen des Minimums in Zeile (*) und das Herabsetzen eines Schlüssels in Zeile

(**) benötigt wird. Es ist klar, daß die Zeile (*) höchstens n -mal ausgeführt wird. Weil die Anzahl aller Elemente in allen Nachfolgermengen $N(v)$ von Knoten $v \in V$ natürlich genau gleich der Anzahl m der Kanten des gegebenen Graphen ist, wird die Zeile (**) insgesamt höchstens m -mal ausgeführt.

Bei geeigneter programmtechnischer Realisierung des Graphen ergibt sich dann als Laufzeit für den Algorithmus von Dijkstra die Größenordnung $O(t_{init} + n \cdot (t_m + t_{dm}) + m \cdot t_{dk})$. Dabei ist t_{init} die zur Initialisierung, also insbesondere zum Aufbau einer Priority Queue mit n Elementen erforderliche Schrittzahl; t_m , t_{dm} und t_{dk} bezeichnen jeweils die Zeit zur Ausführung einer Operation *Access Min*, *Delete Min* und *Decrease Key* auf der Priority Queue $V \setminus S$, die höchstens n Elemente enthält. Statt eine Abschätzung der Laufzeit über die im schlechtesten Fall zur Ausführung einer einzelnen Operation benötigten Zeit vorzunehmen, könnte man natürlich auch eine globalere, aber amortisierte Worst-case-Abschätzung für die Laufzeit des Verfahrens von Dijkstra vornehmen.

Nehmen wir einmal an, daß die Initialisierung auf jeden Fall in Zeit $O(n + m)$ möglich ist. Dann benötigt das Verfahren von Dijkstra höchstens soviele Schritte, wie erforderlich sind, um insgesamt n Operationen *Access Min* und *Delete Min* auszuführen, plus die Gesamtanzahl von Schritten zur Ausführung von m *Decrease Key* Operationen; die Operationen betreffen dabei jeweils Priority Queues mit höchstens n Elementen.

Verschiedene Implementationen von Priority Queues liefern damit unmittelbar verschiedene Implementationen für das Verfahren von Dijkstra zur Berechnung kürzester Wege.

6.1.2 Implementation von Priority Queues mit verketteten Listen und balancierten Bäumen

Verkettet gespeicherte, nicht sortierte Listen bilden eine erste offensichtliche Möglichkeit zur Implementation von Priority Queues. Wir können wie im Abschnitt 1.5 willkürlich voraussetzen, daß die Liste stets je ein unechtes Dummy-Element am Anfang und am Ende hat und der *next*-Zeiger des letzten Elements auf dieses selbst zurückweist. Die Liste ist durch den Zeiger *head* auf das Dummy-Element am Anfang gegeben. Die Verwendung von Dummy-Elementen ist ein Implementationstrick, der sichert, daß sich das Einfügen in die leere Liste algorithmisch nicht vom Einfügen in die nichtleere Liste unterscheidet. Ein neues Element wird immer nach dem Dummy-Element des Listenanfangs eingefügt. Daher ist das *Einfügen* in konstanter Zeit ausführbar. Um das *Minimum suchen* zu können, durchläuft man die Liste vom Anfang an mit zwei Hilfszeigern Z_1 und Z_2 . Während Z_1 die Liste durchläuft, markiert Z_2 das bis dahin kleinste gefundene Element. Offenbar benötigt diese Operation im schlechtesten Fall $\Omega(N)$ Schritte. Der minimale Schlüssel wird entfernt, indem der Zeiger des Vorgängers auf das dem Minimum nachfolgende Element gerichtet wird. Das *Entfernen* des bereits gefundenen minimalen Elements ist in konstanter Zeit durchführbar. Zwei unsortierte, verkettete Listen kann man einfach aneinanderhängen. Dazu durchläuft man eine der Listen mit einem Zeiger, um den Zeiger des letzten Elements auf das erste Element der anderen Liste zu richten. Dieser Vorgang benötigt im schlechtesten Fall $\Omega(N)$ Schritte, wenn N die Länge der durchsuchten Liste ist. Falls die Operation des *Zusammenfügens* häufig benötigt wird, ist es besser, Priority Queues als Listen mit Anfangs- und Endzeiger zu

implementieren. Dann muß man beim Zusammenfügen keine Liste mehr durchlaufen, und die Operation wird in konstanter Zeit ausführbar.

Natürlich kann man Priority Queues auch mit Hilfe verketteter gespeicherter, sortierter linearer Listen implementieren. Wir können wieder eine Implementation mit Dummy-Elementen verwenden. Wir achten aber darauf, daß die Schlüssel aufsteigend sortiert sind. Beim *Einfügen* ist jetzt darauf zu achten, daß das neue Element die aufsteigende Ordnung der Schlüssel nicht zerstört. Das Suchen der richtigen Einfügeposition und das anschließende Einfügen benötigen im schlechtesten Fall $\Omega(N)$ Schritte. Bei dieser Implementation steht der kleinste Schlüssel immer am Anfang der Liste (nach dem Dummy-Element). Daher ist die Operation *Minimum suchen* in konstanter Zeit ausführbar. Zum *Entfernen* des Minimums genügt es, den *next*-Zeiger des *head*-Elements umzuhängen. Um zwei Listen zusammenzufügen, durchläuft man mit einem Zeiger die eine Liste und fügt die Elemente in die andere Liste an der jeweils richtigen Stelle ein. Die dazu erforderliche Schrittzahl ist proportional zur Gesamtanzahl der Elemente in beiden Listen. Es ist nicht schwer, sich zu überlegen, wie die übrigen Operationen (Entfernen eines beliebigen Elements, Herabsetzen eines Schlüssels) bei dieser Implementation von Priority Queues mit linearen Listen ausgeführt werden können.

Um für alle Operationen Laufzeiten von der Größenordnung $O(\log N)$ zu erhalten, kann man Implementationen mit Baumstrukturen verwenden. Grundsätzlich eignet sich dazu jede Klasse balancierter Bäume. Wir skizzieren hier, wie eine Implementation mit Bruder-Bäumen, vgl. Abschnitt 5.2.2, aussehen könnte. Für die Implementation verwenden wir eine Variante von Bruder-Bäumen, bei der die Schlüssel in den Blättern der Bäume gespeichert werden; die inneren Knoten enthalten jeweils das Minimum im Teilbaum unterhalb des Knotens. Die in den Blättern gespeicherten Schlüssel müssen, anders als bei Suchbäumen, nicht sortiert vorliegen. Natürlich ist es überflüssig, in den unären Knoten eines Bruder-Baumes Schlüssel zu speichern; denn nach der gerade getroffenen Festlegung müssen die Werte unärer Knoten mit denen ihrer einzigen Söhne identisch sein.

Abbildung 6.2 zeigt eine als Bruder-Baum gespeicherte Priority Queue mit acht Schlüsseln.

Einen neuen Schlüssel kann man an beliebiger Stelle unter den Blättern *einfügen*, z.B. immer ganz rechts. Im allgemeinen ist es dann erforderlich, den Baum umzustrukturieren, damit nach dem Einfügen wieder ein Bruder-Baum entsteht. Auf die für Bruder-Bäume typischen Umstrukturierungsoperationen nach einer Einfügung sind wir in Abschnitt 5.2.2 eingegangen; im Unterschied zur dort angegebenen Beschreibung muß man in der Umstrukturierungsinvariante aber jetzt die Sortierung von Schlüsselwerten ignorieren. Wichtig ist hier außerdem, daß im ungünstigsten Fall der Baum längs eines Pfades vom neuen Blatt bis zur Wurzel umstrukturiert werden muß. Zugleich müssen die Minima längs dieses Pfades adjustiert werden. Man weiß, daß das Einfügen in einen Bruder-Baum in $O(\log N)$ Schritten ausführbar ist. Das Minimum kann stets an der Wurzel abgelesen werden. Die Operation *Access Min* ist deshalb in $O(1)$ Schritten ausführbar. Um das Minimum zu *entfernen*, muß man das Blatt mit dem minimalen Wert entfernen. Dazu folgt man auf dem Weg von der Wurzel zu den Blättern immer dem Knoten mit dem kleineren Wert. Nach dem Entfernen des Blattes, das das Minimum enthält, sind im allgemeinen Umstrukturierungen nötig, um wieder einen Bruder-Baum zu erhalten. Ferner müssen auch die Werte der inneren Knoten auf dem Pfad von dem Vater des entfernten Schlüssels bis zur Wurzel geändert werden. Das Entfernen eines

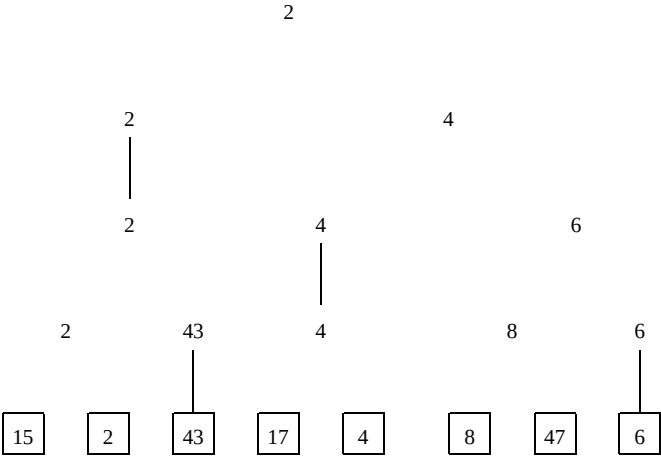


Abbildung 6.2

beliebigen Elementes ist ebenfalls in insgesamt $O(\log N)$ Schritten ausführbar, wenn man die Position des zu entfernenden Elements kennt. Dasselbe gilt für das Herabsetzen eines Schlüssels, das man als Entfernen des alten und Wiedereinfügen des neuen Schlüsselwertes realisieren kann.

Das *Zusammenfügen* zweier als balancierte Bäume implementierter Priority Queues läuft auf das Vereinigen zweier Bäume unterschiedlicher Höhe hinaus. Dazu suchen wir im Baum *A* den rechtesten Teilbaum mit gleicher Höhe wie Baum *B*. (Ohne Einschränkung nehmen wir an, daß *A* der höhere Baum ist.) Hat dieser Teilbaum einen unären Vater, können wir Baum *B* zum zweiten Teilbaum dieses Vaterknotens machen. Hat der Vater *p* des Teilbaums bereits zwei Söhne, so fügt man die Wurzel von *B* als dritten Sohn von *p* ein und strukturiert den Baum von *p* aufwärts so um, daß insgesamt wieder ein Bruder-Baum entsteht. Zum Schluß müssen noch die Werte innerer Knoten auf dem Pfad von der Stelle, an der der Baum eingefügt wurde, bis zur Wurzel überprüft und gegebenenfalls verändert werden. Diese Operation ist insgesamt in $O(\log N)$ Schritten ausführbar.

6.1.3 Linksbäume

Balancierte Bäume haben die Eigenschaft, daß *jeder* Pfad von der Wurzel zu einem Blatt des Baumes mit $N + 1$ Blättern und N inneren Knoten eine Länge der Größenordnung $O(\log N)$ hat. Für die Implementation von Priority Queues reicht eine wesentlich schwächere Forderung aus, um zu sichern, daß die für Priority Queues typischen Operationen Access Min in Zeit $O(1)$ und das Einfügen, Entfernen des Minimums und das Verschmelzen zweier Priority Queues sämtlich in Zeit $O(\log N)$ ausführbar sind. Es genügt, dafür zu sorgen, daß Bäume verwendet werden, die zwei Bedingungen erfüllen.

Die Schlüsselwerte der Söhne müssen (wie bei Heaps) stets größer sein als der Schlüssel des Vaters. Ferner muß es wenigstens einen Pfad von der Wurzel zu einem Blatt mit Länge $O(\log N)$ geben. Wenn man dann Einfüge- und Verschmelze-Vorgänge längs eines solchen kurzen Pfades vornimmt, kann man ein Verhalten garantieren, das genauso gut ist wie bei der Verwendung einer Klasse balancierter Bäume. Diese Idee führt zur Definition von Linksbäumen. Ein binärer Baum heißt ein *Linksbaum*, wenn gilt: Jeder innere Knoten enthält neben dem Schlüssel und den zwei Zeigern auf die beiden Söhne noch ein sogenanntes Distanzfeld, in dem die Entfernung des Knotens zum nächstgelegenen Blatt festgehalten wird. Blätter haben die Distanz 0. Das Knotenformat kann also durch folgende Typvereinbarung beschrieben werden.

```

type Linksbaum = ↑Knoten;
      Knoten = record
        Schlüssel : integer;
        Dist : integer;
        links, rechts : Linksbaum;
        info : {infotype}
      end

```

Für einen Linksbaum wird nun zunächst gefordert, daß für jeden inneren Knoten p gilt: $p.\text{Schlüssel} < p.\text{links}\uparrow.\text{Schlüssel}$ und $p.\text{Schlüssel} < p.\text{rechts}\uparrow.\text{Schlüssel}$. Ferner verlangt man, daß für jeden inneren Knoten p gilt: $p.\text{Dist} = 1 + \min(p.\text{links}\uparrow.\text{Dist}, p.\text{rechts}\uparrow.\text{Dist})$, $p.\text{links}\uparrow.\text{Dist} \geq p.\text{rechts}\uparrow.\text{Dist}$. Also muß stets $p.\text{Dist} = 1 + p.\text{rechts}\uparrow.\text{Dist}$ gelten.

Aufgrund der letzten Bedingung ist ein kürzester Pfad von der Wurzel zu einem Blatt immer der Pfad zum rechtesten Blatt. Die Abbildung 6.3 zeigt das Beispiel eines Linksbaumes, der die Schlüssel $\{2, 3, 4, 5, 7, 8, 9\}$ speichert; Schlüssel sind links oben, Distanzen jeweils rechts oben in den Knoten eingetragen. Blätter sind durch **nil**-Zeiger in den Vätern repräsentiert.

Wie bei Heaps kann man das Minimum in konstanter Zeit an der Wurzel ablesen. Weil offenbar jeder Teilbaum eines Linksbaumes wieder ein Linksbaum sein muß, kann man alle anderen Operationen an Linksbäumen auf das Verschmelzen zweier Linksbäume zu einem neuen zurückführen. Das *Einfügen* eines Schlüssels k in den Linksbaum A kann man auffassen als Verschmelzen von A mit dem Linksbaum B , der einen einzigen inneren Knoten mit Schlüssel k und Distanz 1 enthält. Das *Entfernen des Minimums* aus einem Linksbaum bedeutet das Entfernen der Wurzel und Verschmelzen der beiden Teilbäume der Wurzel. Das *Entfernen eines beliebigen* inneren Knotens p eines Linksbaumes kann man wie folgt durchführen. Der Linksbaum zerfällt beim Wegnehmen von p in einen oberhalb von p liegenden Teilbaum A , den linken Teilbaum B und den rechten Teilbaum C von p . In A ersetzt man p durch ein Blatt, das man gegebenenfalls mit seinem Bruder vertauscht, um links den Teilbaum mit größerer Distanz anzubringen. Dann adjustiert man die Distanz des Vaters von p . Dies setzt man für die Knoten von A auf dem Pfad zur Wurzel fort. Schließlich verschmilzt man die drei Linksbäume A , B und C zu einem neuen. Schließlich kann man das *Herabsetzen* eines Schlüssels als Entfernen des Schlüssels und anschließendes Wiedereinfügen des neuen, herabgesetzten Schlüsselwertes auffassen. Man beachte aber, daß das Adjustieren der Distanzen und gegebenenfalls das erforderliche Vertauschen von Teilbäumen auf dem Pfad von p

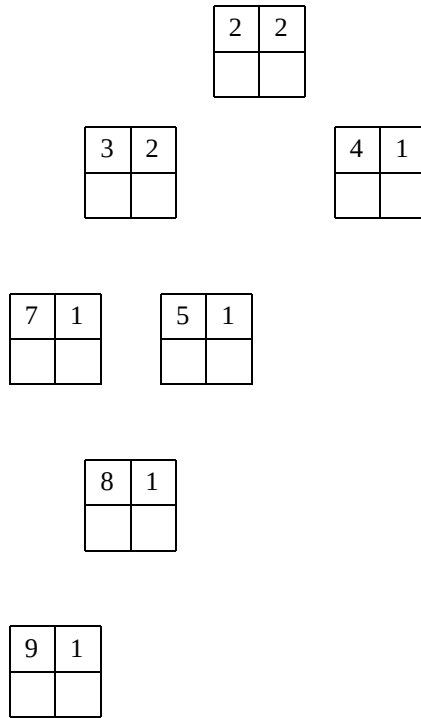


Abbildung 6.3

zur Wurzel von A eine Anzahl von Schritten erfordert, die proportional zur Höhe von A sein kann, weil der Pfad von p zur Wurzel im allgemeinen nicht der rechteste Pfad in A ist. Daher kann der Aufwand zur Ausführung der Operationen Entfernen eines beliebigen Knotens und Herabsetzen eines Schlüssels im schlechtesten Fall $\Omega(N)$ Schritte erfordern.

Dagegen können die Operationen des Einfügens und Entferns des Minimums in logarithmischer Schrittzahl ausgeführt werden, wenn es gelingt, das *Verschmelzen* (Merge) zweier Linksbäume entsprechend effizient durchzuführen. Wir sorgen dafür, daß der dazu erforderliche Aufwand von der Größenordnung der Summe der Längen der Pfade von den Wurzeln der beteiligten Bäume zum jeweils rechtesten Blatt ist. Weil der *kürzeste* Pfad in einem beliebigen Binärbaum mit N inneren Knoten natürlich höchstens die Länge $\lceil \log_2(N+1) \rceil$ haben kann, folgt dann sofort, daß die Operationen Einfügen, Entfernen des Minimums und Verschmelzen in logarithmischer Zeit ausführbar sind.

Die Operation *Merge* kann auf einfache Weise rekursiv erklärt werden. Betrachten wir das Problem, zwei Linksbäume A und B zu verschmelzen. Wir können ohne Einschränkung annehmen, daß der Schlüssel in der Wurzel von Baum A kleiner als der Schlüssel in der Wurzel von Baum B ist. Baum A und B werden zusammengefügt, indem zunächst der rechte Teilbaum von A mit Baum B zu einem neuen Linksb Baum C

verschmolzen wird. Dann wird C zum neuen rechten Teilbaum von A gemacht. Falls die Distanz der Wurzel des entstandenen Baumes im rechten Teilbaum, also die Distanz der Wurzel von C , größer ist als im linken, werden die beiden Teilbäume vertauscht. Die Distanz der Wurzel des neuen Baumes ist gleich der ihres rechten Sohnes plus 1. Das Zusammenfügen des rechten Teilbaums von A mit Baum B geschieht (rekursiv) nach derselben Vorschrift. Die Rekursion endet, wenn die Wurzel des Baumes mit dem kleineren Schlüssel keinen rechten Sohn mehr hat. Dann wird der andere Baum einfach als neuer rechter Teilbaum angehängt, und gegebenenfalls werden die beiden Teilbäume anschließend vertauscht.

Das Verfahren kann in Pascal wie folgt formuliert werden:

```
function Verschmelzen ( $A, B : \text{Linksbaum}$ ) :  $\text{Linksbaum}$ ;
begin
  if  $A = \text{nil}$  then Verschmelzen :=  $B$ 
  else if  $B = \text{nil}$  then Verschmelzen :=  $A$ 
  else begin
    if  $A \uparrow .\text{Schlüssel} > B \uparrow .\text{Schlüssel}$ 
      then Vertausche  $A$  mit  $B$ ;
    {jetzt gilt  $A \uparrow .\text{Schlüssel} < B \uparrow .\text{Schlüssel}$ }
     $A \uparrow .\text{rechts} := \text{Verschmelzen}(A \uparrow .\text{rechts}, B)$ ;
    if  $A \uparrow .\text{rechts} \uparrow .\text{Dist} > A \uparrow .\text{links} \uparrow .\text{Dist}$ 
      then vertausche  $A \uparrow .\text{rechts}$  mit  $A \uparrow .\text{links}$  in  $A$ ;
     $A \uparrow .\text{Dist} := A \uparrow .\text{rechts} \uparrow .\text{Dist} + 1$ ;
    Verschmelzen :=  $A$ 
  end
end
```

Man kann aus dieser rekursiven Formulierung unmittelbar ablesen, daß die Laufzeit des Verfahrens proportional zur Summe der Längen des rechtesten Pfades in A und in B ist. Linksbäume wurden von Crane 1972 erfunden, vgl. dazu 

6.1.4 Binomial Queues

Wir definieren für jedes $n \geq 0$ die Struktur eines *Binomialbaumes* B_n wie folgt:

- (i) B_0 ist ein aus genau einem Knoten bestehender Baum.
- (ii) B_{n+1} entsteht aus zwei Exemplaren von B_n , indem man die Wurzel eines Exemplars von B_n zum weiteren Sohn der Wurzel des anderen macht.

Graphisch kann man diese Definition auch kurz so mitteilen, wie es Abbildung 6.4 zeigt.

Die Abbildung 6.5 zeigt die Struktur der Binomialbäume $B_0, \dots, B_4$. Binomialbäume sind also keine Binärbäume. Wir haben in der Abbildung 6.5 alle Knoten, die denselben Abstand zur Wurzel haben, also alle Knoten gleicher Tiefe, nebeneinander gezeichnet. Aus der Definition kann man leicht die folgenden strukturellen Eigenschaften von Binomialbäumen ableiten:

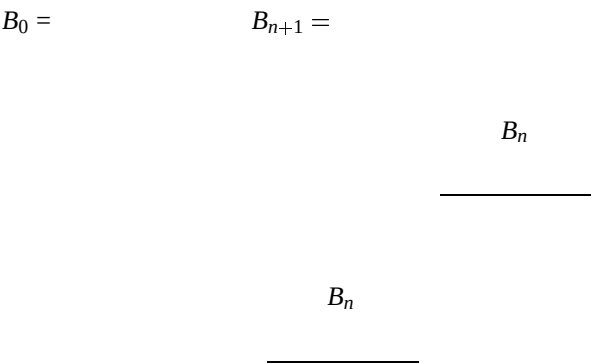


Abbildung 6.4

- (1) B_n besteht aus genau 2^n Knoten.
- (2) B_n hat die Höhe n .
- (3) Die Wurzel von B_n hat die Ordnung n , d.h. sie hat genau n Söhne.
- (4) Die n Teilbäume der Wurzel von B_n sind genau $B_{n-1}, B_{n-2}, \dots, B_1, B_0$.
- (5) B_n hat $\binom{n}{i}$ Knoten mit Tiefe i .

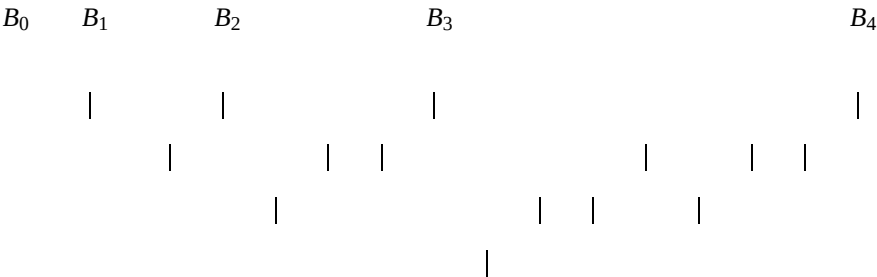


Abbildung 6.5

Wir wollen Binomialbäume zur Speicherung von Schlüsselmengen verwenden, so daß eine schwache Ordnungsbeziehung für die gespeicherten Schlüssel gilt, wie wir sie von Heaps kennen: Für jeden Knoten gilt, daß der in ihm gespeicherte Schlüssel kleiner ist als die Schlüssel seiner Söhne. Wir nennen einen Baum mit dieser Eigenschaft

heapgeordnet. Außerdem möchten wir nicht nur Mengen von N Schlüsseln speichern können, wenn $N = 2^n$, also eine Zweierpotenz ist. Dazu stellen wir N als Dualzahl dar: $N = (d_{n-1}d_{n-2} \dots d_0)_2$. Dann wählen wir für jedes j mit $d_j = 1$ einen Binomialbaum B_j ; die Schlüsselmenge wird nun durch den Wald dieser Binomialbäume repräsentiert. Jeder Binomialbaum für sich muß heapgeordnet sein.

Beispiel: Gegeben sei die folgende Menge von elf Schlüsseln $\{2, 4, 6, 8, 14, 15, 17, 19, 23, 43, 47\}$. Weil $11 = (1011)_2$ ist, können die Schlüssel in einem Wald t_{11} von drei Binomialbäumen B_3, B_1, B_0 mit jeweils acht, zwei und einem Knoten gespeichert werden. Eine zulässige Speicherung, bei der die Werte der Söhne stets größer sind als die in den Vätern gespeicherten Schlüssel, zeigt Abbildung 6.6.

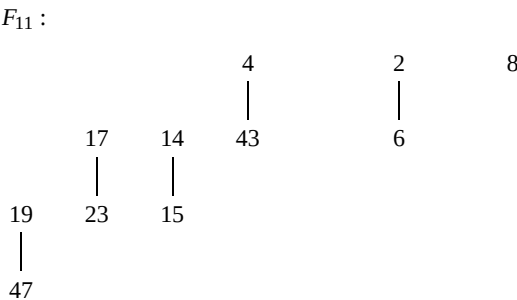


Abbildung 6.6

Man benötigt also zur Speicherung einer Menge von N Schlüsseln gerade so viele heapgeordnete Binomialbäume, wie Einsen in der Dualdarstellung von N auftreten. B_j wird genau dann benutzt, wenn an der j -ten Stelle in der Dualdarstellung von N die Ziffer 1 auftritt. Eine derartige Repräsentation einer Menge von N Schlüsseln nennen wir eine *Binomial Queue*. Denn wir werden jetzt zeigen, daß man alle für Priority Queues üblichen Operationen mit solchen Wäldern von Binomialbäumen durchführen kann.

Zunächst ist klar, wie man das Minimum einer in einer Binomial Queue F_N gespeicherten Menge von N Schlüsseln bestimmt. Man inspiziert die Wurzeln aller Binomialbäume des Waldes F_N , die die Queue bilden, und nimmt davon das Minimum. Da es natürlich höchstens $\lceil \log_2 N \rceil + 1$ Bäume in diesem Wald geben kann, ist klar, daß man das Minimum in $O(\log N)$ Schritten bestimmen kann.

Wir erklären jetzt, wie man zwei Binomial Queues zu einer neuen verschmelzen kann. (Dabei werden allerdings einige durchaus wesentliche Implementationsdetails zunächst offengelassen, die wir erst später angeben.) Das Verschmelzen zweier Binomialbäume B_n gleicher Größe mit jeweils genau 2^n Elementen ist ganz einfach. Die Struktur des durch Verschmelzen entstehenden Baumes ist ja bereits in der Definition festgelegt; wir müssen nur noch darauf achten, daß beim Zusammenfügen von zwei Exemplaren B_n zu B_{n+1} dasjenige Exemplar zur Wurzel von B_{n+1} wird, das den kleineren Schlüssel in der Wurzel hat.

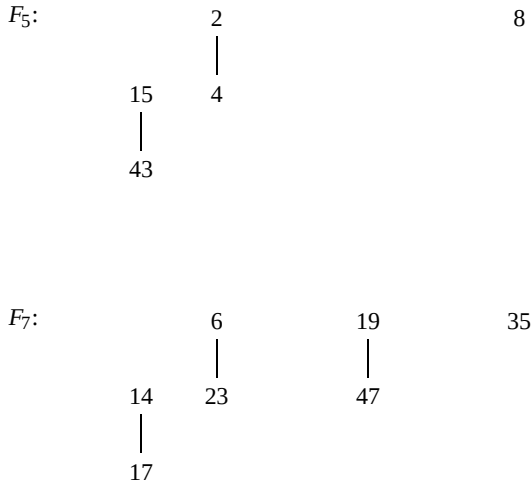


Abbildung 6.7

Das Zusammenfügen zweier Binomial Queues, die nicht genau aus zwei gleich-großen Binomialbäumen bestehen, orientiert sich am bekannten Schulverfahren zur Addition zweier Dualzahlen. Seien also zwei Binomial Queues F_{N_1} und F_{N_2} mit N_1 und N_2 Elementen gegeben; sie bestehen jeweils aus Wäldern von höchstens $\lceil \log_2 N_1 \rceil + 1$ und $\lceil \log_2 N_2 \rceil + 1$ Binomialbäumen. Das Verfahren zum Verschmelzen der zwei Binomial Queues betrachtet die Binomialbäume der Wälder F_{N_1} und F_{N_2} der Reihe nach in aufsteigender Größe. Wie bei der Addition von Dualzahlen betrachtet man in jedem Schritt zwei Binomialbäume der gegebenen Queues und eventuell einen als *Übertrag* erhaltenen Binomialbaum. Anfangs hat man keinen Übertrag. Im i -ten Schritt hat man als Operanden einen Binomialbaum B_i der ersten Queue, wenn in der Dualdarstellung von N_1 an der i -ten Stelle eine 1 auftritt, ferner einen Binomialbaum B_i der zweiten Queue, wenn in der Dualdarstellung von N_2 an der i -ten Stelle eine 1 auftritt, und eventuell einen Binomialbaum B_i als Übertrag.

Ist keiner der drei Operanden vorhanden, ist auch die i -te Komponente des Ergebnisses nicht vorhanden; tritt genau einer der drei genannten Operanden auf, bildet er die i -te Komponente des Ergebnisses, und es wird kein Übertrag für die nächsthöhere Stelle erzeugt. Treten genau zwei Operanden auf, werden sie zu einem Binomialbaum B_{i+1} wie oben angegeben zusammengefaßt und als Übertrag an die nächsthöhere Stelle weitergegeben; die i -te Komponente des Ergebnisses ist nicht vorhanden. Sind schließlich alle drei Operanden vorhanden, wird einer zur i -ten Komponente des Ergebnisses; die beiden anderen werden zu einem Binomialbaum B_{i+1} zusammengefaßt und als Übertrag an die nächsthöhere Stelle übertragen.

Wir erläutern das Verfahren an folgendem Beispiel. Gegeben seien die Binomial Queues F_5 und F_7 mit $N_1 = 5$ und $N_2 = 7$ Elementen, vgl. Abbildung 6.7. Addition von N_1 und N_2 im Dualsystem ergibt:

N_1		1	0	1
N_2		1	1	1
Übertrag	1	1	1	0
Ergebnis	1	1	0	0

Das Verschmelzen von F_5 und F_7 zu F_{12} zeigt Abbildung 6.8.

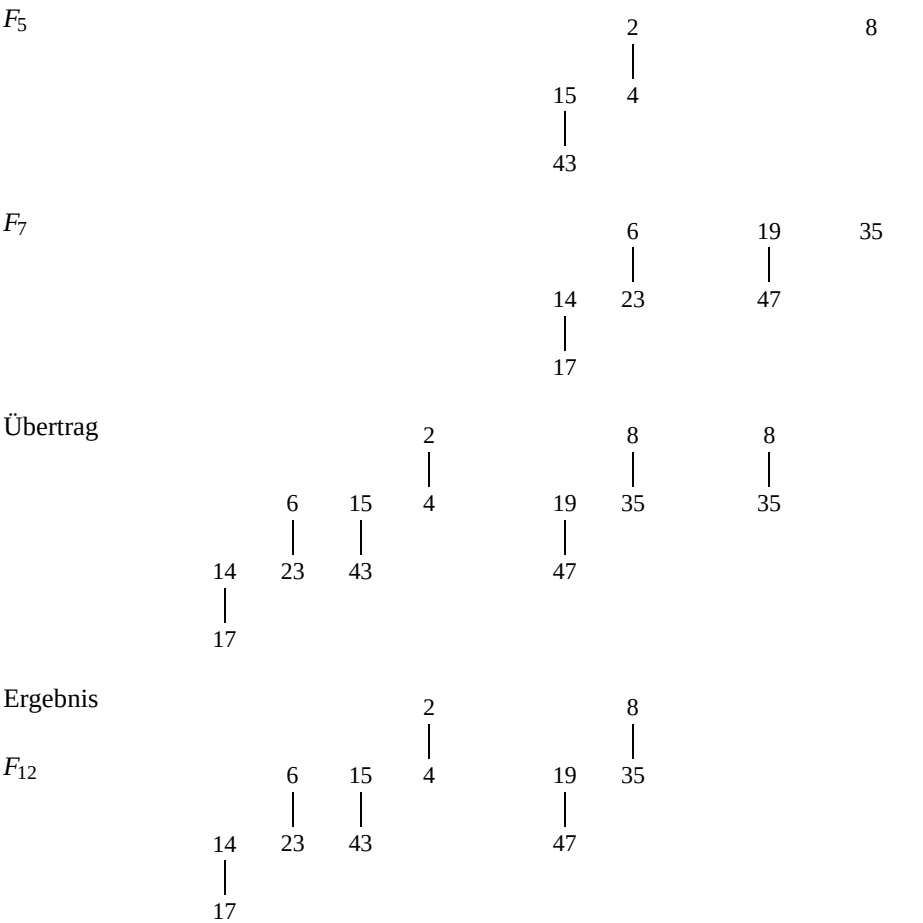


Abbildung 6.8

Es sollte klar sein, daß das Verschmelzen zweier Binomial Queues F_{N_1} und F_{N_2} mit N_1 und N_2 Elementen in $O(\log N_1 + \log N_2)$ Schritten ausführbar ist, wenn man voraussetzt, daß das Anhängen eines weiteren Sohnes an die Wurzel eines Binomialbaumes in konstanter Zeit möglich ist. Bevor wir auf diese Voraussetzung genauer eingehen, wollen wir uns zunächst überlegen, daß man die Operationen *Einfügen* eines neuen Elementes, *Entfernen des Minimums*, *Entfernen* eines beliebigen Elementes und *Herabsetzen* eines Schlüssels sämtlich auf das Verschmelzen von Binomial Queues zurückführen kann.

Für das Einfügen eines neuen Elementes ist dies offensichtlich.

Der minimale Schlüssel einer Binomial Queue F_N ist Schlüssel der Wurzel eines Binomialbaumes B_i im Wald von Binomialbäumen, die F_N bilden. Entfernt man diese Wurzel, zerfällt B_i in Teilbäume $B_{i-1}, B_{i-2}, \dots, B_0$; sie bilden einen Wald $F_{2^{i-1}}$. Läßt man B_i aus dem Wald F_N weg, bleibt ein Wald F_{N-2^i} übrig. Verschmelzen dieser beiden Wälder liefert das gewünschte Ergebnis.

Das Entfernen eines Schlüssels k , der nicht in der Wurzel eines Binomialbaumes B_i im die Binomial Queue bildenden Wald F_N auftritt, ist schwieriger. Wir können aber annehmen, daß k in B_i auftritt (allerdings nicht an der Wurzel), B_i Binomialbaum im Wald F_N . Wir entfernen B_i aus F_N und erhalten einen Wald F_{N_1} mit $N_1 = N - 2^i$.

B_i besteht aus zwei Exemplaren B_{i-1} , einem linken Teilbaum B_{i-1}^l und einem rechten Teilbaum B_{i-1}^r , vgl. Abbildung 6.9.

$B_i :$

B_{i-1}^r

B_{i-1}^l

Abbildung 6.9

Kommt k in B_{i-1}^l vor, bilden wir einen neuen Wald F_{N_2} , in den wir zunächst B_{i-1}^r aufnehmen; kommt k in B_{i-1}^r vor, nehmen wir in F_{N_2} zunächst B_{i-1}^l auf. Dann zerlegen wir B_{i-1} auf dieselbe Weise und nehmen immer wieder kleinere Binomialbäume zu F_{N_2} hinzu, bis wir bei einem Binomialbaum B_j angekommen sind, der k als Schlüssel der Wurzel hat. Dann entfernen wir diese Wurzel und nehmen die Teilbäume $B_{j-1}, \dots, B_0$ der Wurzel noch zu F_{N_2} hinzu. Insgesamt erhalten wir so zwei Wälder F_{N_1} und F_{N_2} , die nach dem oben angegebenen Verfahren verschmolzen werden können.

Entfernt man z.B. aus dem in Abbildung 6.6 gezeigten Wald F_{11} den Schlüssel 14, so zerfällt F_{11} zunächst in die in Abbildung 6.10 gezeigten Bäume F_3 und F_7 , die anschließend verschmolzen werden müssen.

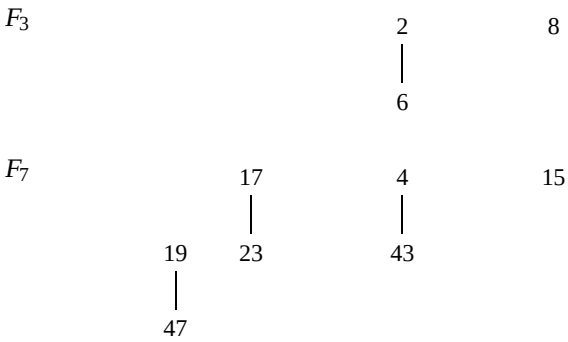


Abbildung 6.10

Das Herabsetzen eines Schlüssels kann man, wie bisher stets, auf das Entfernen des Schlüssels und das anschließende Wiedereinfügen des herabgesetzten Schlüssels zurückführen. Alternativ kann man auch den erniedrigten Schlüssel so oft mit seinem Vater vertauschen, bis die Heapordnung wiederhergestellt ist.

Eine *Implementation* dieser Verfahren verlangt es, Bäume mit unbeschränkter Ordnung programmtechnisch zu realisieren. Denn Binomialbäume B_n sind Bäume der Ordnung n , weil die Wurzel n Söhne hat. Man könnte natürlich einen maximalen Knotengrad als Obergrenze vorsehen und jedem Knoten erlauben, so viele Söhne zu haben, wie dieser Knotengrad angibt. Das hätte aber eine enorme Verschwendung von Speicherplatz zur Folge, die weder sinnvoll noch nötig ist.

Vuillemin [1] schlägt vor, Binomialbäume, und damit Binomial Queues, als Binärbäume wie folgt zu repräsentieren: Jeder Knoten eines Binomialbaumes enthält genau zwei Zeiger, einen Zeiger *l*link auf den linken Sohn und einen Zeiger *r*link auf seinen rechten Nachbarn. Hat ein Knoten keinen rechten Nachbarn, kann man den Zeiger *r*link auf den Vater des Knotens zurückweisen lassen. Nach diesem Prinzip kann man beliebige Vielwegbäume als Binärbäume repräsentieren, also nicht nur Binomialbäume.

Abbildung 6.11 zeigt als Beispiel eine Binärbaum-Repräsentation des Binomialbaumes B_3 aus dem Wald F_{11} von Abbildung 6.6.

Sollen zwei als Binärbäume repräsentierte Binomialbäume zu einem neuen verschmolzen werden, muß man den *l*link-Zeiger der Wurzel des einen Baumes auf die Wurzel des anderen umlegen und den *r*link-Zeiger der Wurzel des zweiten Baumes auf den linken Sohn der Wurzel des ersten Baumes zeigen lassen, falls dieser einen Sohn hatte; sonst läßt man den *r*link-Zeiger auf die Wurzel des neuen Baumes zurückweisen. Es ist klar, daß diese Operationen in konstanter Zeit ausführbar sind. Diese Ope-

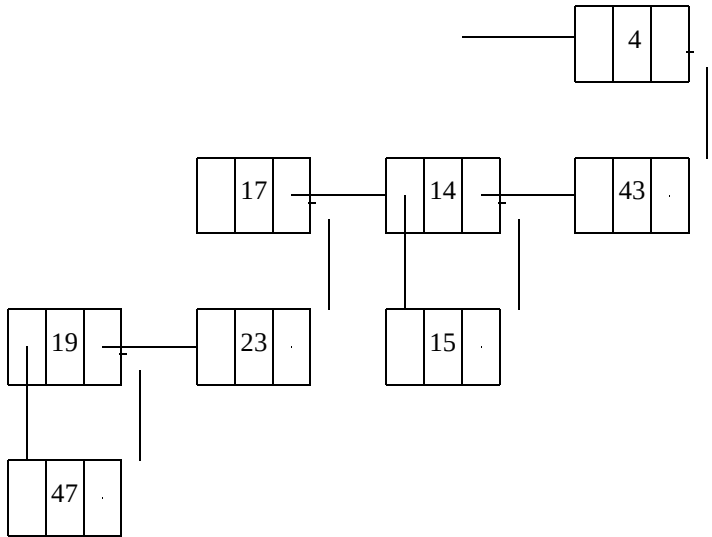


Abbildung 6.11

rationen bilden die Grundlage für eine Prozedur zum Verschmelzen zweier Binomial Queues. Für weitere Einzelheiten der programmtechnischen Realisierung der Algorithmen dieses Abschnitts konsultiere man [3]. Insgesamt ergibt sich, daß alle genannten Operationen Access Min, Einfügen, Mergen, Minimum Entfernen, Decrease Key, Delete in Zeit $O(\log N)$ ausführbar sind für eine Binomial Queue mit N Elementen.

6.1.5 Fibonacci-Heaps

Die Struktur von Binomialbäumen und Binomial Queues ist ebenso starr wie die von Heaps. Für eine gegebene Zahl N gibt es jeweils nur eine einzige Struktur mit N Knoten. Lediglich die Verteilung der Schlüssel ist nicht eindeutig bestimmt, weil nur verlangt wird, daß die Bäume heapgeordnet sein müssen. Fibonacci-Heaps sind wesentlich weniger starr. Ein *Fibonacci-Heap* (kurz: *F-Heap*) ist eine Kollektion heapgeordneter Bäume mit jeweils disjunkten Schlüsselmenge. Es wird keine weitere Forderung an die Struktur von F-Heaps gestellt. Dennoch haben F-Heaps eine implizit durch die für F-Heaps erklärten Operationen festgelegte Struktur. Die Klasse der F-Heaps ist die kleinste Klasse von heapgeordneten Bäumen, die gegen die später erklärten Operationen *Initialisieren* (des leeren F-Heaps), *Einfügen* eines Schlüssels, *Access Min*, *Delete Min*, *Decrease Key*, *Delete* und *Meld* abgeschlossen ist. Wir werden sehen, daß F-Heaps eng mit den im Abschnitt 6.1.4 behandelten Binomial Queues zusammenhängen.

Die genannten Operationen für F-Heaps verändern die Kollektion heapgeordneter Bäume. Es können neue heapgeordnete Bäume in die Kollektion aufgenommen werden oder zwei (oder mehrere) heapgeordnete Bäume zu einem neuen heapgeordneten Baum verschmolzen werden. Diese Operation des Verschmelzens von zwei heapgeordneten Bäumen ist genau die von Binomialbäumen bekannte Operation. Zwei heapgeordnete Bäume, deren Wurzeln denselben Rang r haben, können zu einem heapgeordneten Baum mit Rang $r + 1$ verschmolzen werden, indem man die Wurzel des Baumes mit dem größeren Schlüssel zum weiteren, $(r + 1)$ -ten Sohn der Wurzel des Baumes macht, der den kleineren Schlüssel in der Wurzel hat. Anders als bei Binomialbäumen und Binomial Queues kann es bei F-Heaps jedoch vorkommen, daß Bäume verschmolzen werden, die nicht dieselbe Knotenzahl haben. (Das gilt aber höchstens dann, wenn die Operationen Decrease Key und Delete in einer Operationsfolge für F-Heaps vorkommen.) Bevor wir jetzt der Reihe nach die oben genannten Operationen für F-Heaps erklären, wollen wir angeben, wie F-Heaps implementiert werden, damit wir die Zeit zur Ausführung der Operationen abschätzen können.

Ein F-Heap besteht aus einer Kollektion heapgeordneter Bäume; die Wurzeln dieser Bäume sind Elemente einer doppelt verketteten, zyklisch geschlossenen Liste. Diese Liste heißt die *Wurzelliste* des F-Heaps. Der F-Heap ist gegeben durch einen Zeiger auf das Element mit minimalem Schlüssel in der Liste. Dieses Element heißt das *Minimal-element* des F-Heaps. Jeder Knoten eines heapgeordneten Baumes hat einen Zeiger auf seinen Vater (wenn er einen Vater hat, und sonst einen **nil**-Zeiger) und einen Zeiger auf einen seiner Söhne. Ferner sind alle Söhne eines Knotens untereinander doppelt, zyklisch verkettet. Außerdem hat jeder Knoten ein Rangfeld, das die Anzahl seiner Söhne angibt, und ein Markierungsfeld, dessen Bedeutung später erklärt wird. Das Knotenformat eines in einem F-Heap auftretenden Baumes kann also durch folgende Typvereinbarung beschrieben werden:

```

type heap-ordered-tree =  $\uparrow$ Knoten;
      Knoten = record
        links, rechts :  $\uparrow$ Knoten;
        vater, sohn :  $\uparrow$ Knoten;
        key : integer;
        rank : integer;
        marker : boolean
      end

```

Natürlich kann man jede Binomial Queue auch als F-Heap auffassen und wie soeben angegeben implementieren. Abbildung 6.12 zeigt F_7 aus Abbildung 6.7 als F-Heap; wir haben allerdings die Rang- und Markierungsfelder weggelassen.

Wir erklären jetzt die Operationen für F-Heaps. Die Operationen Initialisieren, Einfügen, Access Min und Verschmelzen (Meld) ändern weder die Rang- noch die Markierungsfelder von bereits existierenden Knoten; sie sind wie folgt erklärt.

Initialisieren des leeren F-Heaps: Liefert einen **nil**-Zeiger.

Einfügen eines Schlüssels k in einen F-Heap h : Bilde einen F-Heap h' aus einem einzigen Knoten, der k speichert. (Dieser Knoten ist unmarkiert und hat Rang 0.) Verschmilz h und h' zu einem neuen F-Heap, vgl. unten.

Access Min: Das Minimum eines F-Heaps h ist im Minimalknoten von h gespeichert.

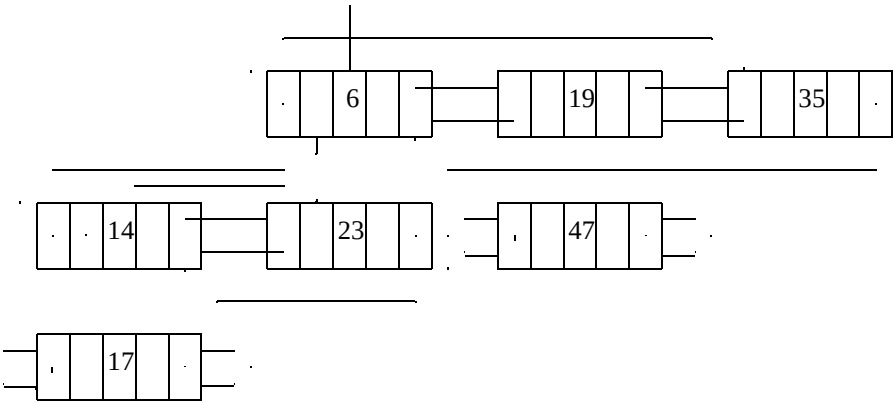


Abbildung 6.12

Das *Verschmelzen* (Meld) zweier F-Heaps h_1 und h_2 mit disjunkten Schlüsselmenge geschieht durch Aneinanderhängen der beiden Wurzellisten von h_1 und h_2 . Minimalelement des resultierenden F-Heaps ist das kleinere der beiden Minimalelemente von h_1 und h_2 ; als Ergebnis der Verschmelze-Operation wird ein Zeiger auf dieses Element abgeliefert.

Offenbar sind alle diese Operationen in Zeit $O(1)$ ausführbar, wenn man F-Heaps wie oben angegeben implementiert. Man beachte den Unterschied zwischen der Verschmelze-Operation (Meld-Operation) für F-Heaps und der entsprechenden Operation für Binomial Queues: Die Verschmelze-Operation für F-Heaps sammelt nur die den F-Heap bildenden heapgeordneten Bäume in der Wurzelliste, ohne diese Bäume zu größeren zu verschmelzen; die entsprechende Operation für Binomial Queues fügt die Bäume analog zur Addition zweier Dualzahlen zusammen. Dies einer Dualzahladdition entsprechende Zusammenfügen von heapgeordneten Bäumen erfolgt bei F-Heaps immer dann, wenn eine Delete-Min-Operation ausgeführt wird.

Das *Entfernen* des Minimalknotens (*Delete Min*) eines F-Heaps h geschieht folgendermaßen: Entferne den Minimalknoten aus der Wurzelliste von h und bilde eine neue Wurzelliste durch Einhängen der Liste der Söhne des Minimalknotens an Stelle des Minimalknotens in die Wurzelliste. (Das ist in konstanter Zeit möglich, wenn man die Vaterzeiger der in die Wurzelliste neu aufgenommenen Knoten erst beim anschließenden Durchlaufen der Wurzelliste adjustiert.) Anschließend werden so lange je zwei heapgeordnete Bäume, deren Wurzeln denselben Rang haben, zu einem neuen heapgeordneten Baum verschmolzen, bis eine Wurzelliste entstanden ist, deren sämtliche heapgeordneten Bäume verschiedenen Rang haben. Beim Verschmelzen zweier Bäume entsteht ein heapgeordneter Baum, dessen Wurzel einen um eins erhöhten Rang hat und dessen Markierungsfeld auf „unmarkiert“ gesetzt wird. Beim Durchlaufen der Wurzelliste und Verschmelzen von Bäumen merkt man sich zugleich die Wurzel des Baumes mit dem bislang minimalen Schlüssel. Am Ende wird dieser der Minimalknoten des resultierenden F-Heaps; man liefert als Ergebnis einen Zeiger auf diesen Knoten ab.

Die Operation Delete Min verlangt, daß man in einer Liste von Wurzeln von heapgeordneten Bäumen immer wieder Knoten vom selben Rang findet, die dann verschmolzen werden. Das kann man mit Hilfe eines Rang-Arrays erreichen, d.h. eines linearen Feldes, das mit den Rängen von 0 bis zum maximal möglichen Rang indiziert ist und Zeiger auf die Wurzeln heapgeordneter Bäume enthält. Zu jedem Rang enthält das Rang-Array höchstens einen Zeiger; anfangs ist das Rang-Array leer, d.h. es enthält noch keinen Zeiger. Dann durchläuft man die Wurzelliste, also die Liste der heapgeordneten Bäume, die verschmolzen werden sollen. Trifft man in dieser Liste auf einen Baum B mit Wurzel vom Rang r , versucht man, im Rang-Array einen Zeiger auf diesen Baum B an Position r einzutragen. Ist dort bereits ein Zeiger auf einen Baum B' (mit Wurzel vom gleichen Rang r) eingetragen, fügt man B und B' zu einem Baum mit Wurzel vom Rang $r + 1$ zusammen und versucht, einen Zeiger auf diesen Baum an Position $r + 1$ im Rang-Array einzutragen; der Eintrag an Position r im Rang-Array wird gelöscht. Jedes Element der Wurzelliste wird so genau einmal betrachtet, und am Ende enthält das Rang-Array für jeden Rang höchstens einen Zeiger auf eine Wurzel eines heapgeordneten Baumes. (Das Rang-Array kann dann wieder gelöscht werden.) Jetzt sollte auch der Zusammenhang mit den im Abschnitt 6.1.4 behandelten Binomial Queues klar sein. Man verschiebt einfach die der Addition von Dualzahlen entsprechenden Operationen an heapgeordneten Bäumen von der Verschmelze-Operation zur Delete-Min-Operation. Das hat den großen Vorteil, daß man zugleich mit der Ausführung der notwendigen Verschmelze-Operationen an heapgeordneten Bäumen auch das neue Minimalelement bestimmen kann.

Genauer gilt offenbar folgendes: Beginnt man mit einem anfangs leeren F-Heap und führt eine beliebige Folge von Einfüge-, Access-Min-, Meld- und Delete-Min-Operationen aus, so sind die Bäume in den Wurzellisten sämtlicher durch die Operationsfolge erzeugten F-Heaps stets Binomialbäume. Am Ende einer Delete-Min-Operation bilden die Bäume in der Wurzelliste des F-Heaps sogar eine Binomial Queue.

Bevor wir die Anzahl der zur Ausführung einer Delete-Min-Operation erforderlichen Schritte bestimmen, geben wir noch an, wie der Schlüssel eines Elementes *herabgesetzt* und wie ein Element aus einem F-Heap *entfernt* werden kann, das nicht das Minimalelement ist.

Um einen Schlüssel eines Knotens p eines F-Heaps h *herabzusetzen*, trennen wir p von seinem Vater $\wp p$ ab und nehmen p mit dem herabgesetzten Schlüssel in die Wurzelliste des F-Heaps auf. Natürlich müssen wir auch den Rang von $\wp p$ um 1 erniedrigen. Ist der herabgesetzte Schlüssel von p kleiner als der des Minimalelementes von h , machen wir p zum neuen Minimalelement.

Diese Veränderungen sind sämtlich in konstanter Zeit ausführbar. Im allgemeinen ist damit die Operation des Herabsetzens oder Entfernens eines Schlüssels aber noch nicht zu Ende. Wir wollen nämlich verhindern, daß ein Knoten mehr als zwei Söhne verliert, wenn auf diese Weise ein Knoten abgetrennt wird. (Denn dann könnte der heapgeordnete Baum zu „dünn“ werden.) Um das zu erreichen, benutzen wir die Markierung. Wir hatten einen Knoten als unmarkiert gekennzeichnet, wenn er Wurzel eines heapgeordneten Baumes geworden war, der durch Verschmelzen zweier Bäume mit Wurzeln vom gleichen Rang entstand. Wird nun im Verlauf einer Decrease-key- oder Delete-Operation p von seinem Vater $\wp p$ abgetrennt und ist $\wp p$ unmarkiert, so setzen wir $\wp p$ auf markiert. Ist aber $\wp p$ bereits markiert, so bedeutet das: $\wp p$ hat bereits einen seiner Söhne verloren. In diesem Fall trennen wir nicht nur p von $\wp p$ ab, sondern trennen auch

ϕp von dessen Vater $\phi\phi p$ ab, usw., bis wir auf einen unmarkierten Knoten stoßen, der dann markiert wird, falls er nicht in der Wurzelliste auftritt. Alle abgetrennten Knoten werden in die Wurzelliste des F-Heaps aufgenommen. Obwohl wir, um den Schlüssel eines Knotens p herabzusetzen oder p zu entfernen, eigentlich nur p von seinem Vater abtrennen wollten, weil an dieser Stelle ein Verstoß gegen die Heap-Ordnung vorliegen könnte, kann das Abtrennen von p von ϕp eine ganze Kaskade von weiteren Abtrennungen auslösen. Bevor wir uns überlegen, wieviele solcher indirekter Abtrennungen von Knoten (*cascading cuts*) vorkommen können, betrachten wir ein Beispiel. Nehmen wir an, daß in dem heapgeordneten Baum von Abbildung 6.13 der Schlüssel 31 auf 5 herabgesetzt werden soll und daß in dem Baum die Knoten 17, 13 und 7 (durch einen *) markiert sind, also bereits einen Sohn verloren haben. Dann führt das Abtrennen des Knotens 31 von seinem Vater dazu, daß auch 17, 13 und 7 abgetrennt werden, und man erhält die in Abbildung 6.14 gezeigte Liste von Bäumen.

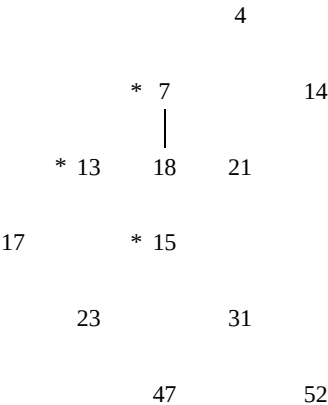


Abbildung 6.13

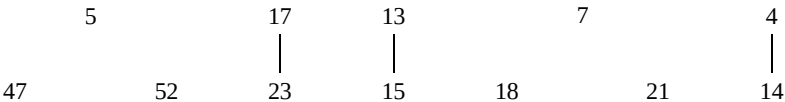


Abbildung 6.14

Das Entfernen eines Knotens p , der nicht das Minimalelement von h ist, kann wie folgt durchgeführt werden: Zunächst wird der Schlüssel von p auf einen Wert her-

abgesetzt, der kleiner als alle übrigen Schlüsselwerte in h ist. Anschließend wird die Operation Delete Min ausgeführt.

Daß die über die Markierung von Knoten gesteuerte Regel „Mache Knoten, die zwei Söhne verloren haben, zu Wurzeln“ wirklich verhindert, daß die in Wurzellisten von F-Heaps auftretenden Bäume zu „dünn“ werden, zeigen die folgenden Sätze.

Lemma 6.1 *Sei p ein Knoten eines F-Heaps h . Ordnet man die Söhne von p in der zeitlichen Reihenfolge, in der sie an p (durch Zusammenfügen) angehängt wurden, so gilt: Der i -te Sohn von p hat mindestens Rang $i - 2$.*

Zum Beweis nehmen wir an, p habe r Söhne. Es ist möglich, daß p schon mehr als r Söhne gehabt hat und davon einige wieder durch Abtrennen verloren hat. Ordnet man die noch vorhandenen r Söhne von p der zeitlichen Reihenfolge nach, in der sie an p angehängt wurden, so muß gelten: Als der i -te Sohn an p angehängt wurde (durch Verschmelzen zweier Wurzeln vom gleichen Rang), müssen sowohl p als auch sein i -ter Sohn wenigstens Rang $i - 1$ gehabt haben, und beide natürlich denselben Rang. Der i -te Sohn kann später höchstens einen Sohn verloren haben, denn andernfalls wäre er von p nach der oben angegebenen Regel abgetrennt worden. $\square$

Lemma 6.2 *Jeder Knoten p vom Rang k eines F-Heaps h ist Wurzel eines Teilbaumes mit wenigstens F_{k+2} Knoten.*

Zum Beweis definieren wir

S_k = Minimalzahl von Nachfolgern eines Knotens p vom Rang k
in einem F-Heap (einschließlich p).

Ein Knoten mit Rang 0 hat keinen Sohn, ein Knoten mit Rang 1 hat mindestens einen Sohn, also $S_0 = 1$, $S_1 = 2$. Betrachten wir jetzt also einen Knoten p vom Rang k . Wir können die k Söhne von p in der Reihenfolge ordnen, in der sie an p angehängt wurden. Der erste Sohn von p kann Rang 0 haben; für alle anderen gilt Lemma 6.1; zählt man noch p selbst hinzu, so folgt:

$$S_k \geq 2 + \sum_{i=0}^{k-2} S_i, \text{ für } k \geq 2. \quad (6.1)$$

Aus der Definition der Fibonacci-Zahlen ($F_0 = 0$, $F_1 = 1$, $F_{k+2} = F_{k+1} + F_k$) folgt sofort:

$$F_{k+2} = 2 + \sum_{i=2}^k F_i, \text{ für } k \geq 2. \quad (6.2)$$

Aus (6.1) und (6.2) leitet man durch vollständige Induktion über k her:

$$S_k \geq F_{k+2}, \text{ für } k \geq 0.$$

$\square$

Aufgrund von Lemma 6.2 haben Fredman und Tarjan  den Namen Fibonacci-Heap eingeführt. Wir wissen bereits, vgl. Abschnitt 3.2.3, daß die Fibonacci-Zahlen exponentiell (mit dem Faktor 1.618...) wachsen. Vergleichen wir nun F-Heaps und Binomial Queues: Binomial Queues bestehen aus Binomialbäumen; jeder Binomialbaum B_j mit Wurzel vom Rang j hat 2^j Knoten. Ein in der Wurzelliste eines F-Heaps auftretender Baum muß ebenfalls eine Anzahl von Knoten haben, die exponentiell mit dem Rang, d. h. mit der Anzahl der Söhne der Wurzel wächst. Genauer kann man aus Lemma 6.2 folgern, daß F-Heaps mit Wurzeln vom Rang 0, 1, 2, 3, 4 ... und minimaler Knotenzahl die in Abbildung 6.15 gezeigte Struktur haben müssen. (Der in Abbildung 6.13 gezeigte heapgeordnete Baum kann also in der Wurzelliste eines F-Heaps nicht auftreten!)

Wurzelsrang	0	1	2	3	4	...
Struktur von						...
F-Heaps mit						
minimaler						
Knotenzahl						
Knotenzahl	1	2	3	5	8	...

Abbildung 6.15

Umgekehrt folgt aus Lemma 6.2 natürlich auch, daß jeder Knoten eines F-Heaps mit insgesamt N Knoten einen Rang $k \leq 1.44 \dots \log_2 N$ hat. Das hat insbesondere zur Folge, daß durch Entfernen des Minimalknotens eines F-Heaps mit N Knoten die Wurzelliste höchstens um $O(\log N)$ Wurzeln heapgeordneter Bäume verlängert wird.

Wir wollen jetzt die Anzahl der Schritte (die Zeit oder die Kosten) nach oben hin abschätzen, die zur Ausführung der Operationen an F-Heaps erforderlich sind. Dabei interessieren wir uns für die Kosten pro Operation, gemittelt über eine beliebige Operationenfolge, beginnend mit einem anfangs leeren F-Heap. Schwierig ist allein die Abschätzung der Zahl der Verschmelze-Operationen nach Entfernen des Minimalknotens bei einer Delete-Min-Operation und der Zahl der indirekten Abtrennungen (cascading cuts) von Knoten nach einer Decrease-Key- oder Delete-Operation.

Es ist intuitiv klar, daß die Zahl der Verschmelze-Operationen mit der Zahl der Knoten in der Wurzelliste eines F-Heaps zusammenhängt. Jede Verschmelze-Operation verkürzt die Wurzelliste. Ebenso ist klar, daß die Zahl der markierten Knoten und damit die Zahl der indirekten Abtrennungen mit der Zahl der Decrease-Key- und Delete-Operationen zusammenhängen muß. Eine Markierung ist stets Folge einer solchen Operation.

Zur Abschätzung der wirklichen Gesamtkosten für eine Folge von Operationen an F-Heaps führen wir eine amortisierte Worst-case-Analyse durch und benutzen das Bankkonto-Paradigma aus Abschnitt 3.3. Wir ordnen jedem Bearbeitungszustand, der

nach Ausführung eines Anfangsstücks einer gegebenen Folge von Operationen erreicht wird, einen nichtnegativen *Kontostand* und der i -ten Operation der Folge eine *amortisierte Zeit* a_i zu: a_i ist die wirkliche Zeit t_i zur Ausführung der i -ten Operation zuzüglich dem Kontostand nach Ausführung der i -ten Operation minus dem Kontostand vor Ausführung der i -ten Operation. Die zur Durchführung einer Folge von Operationen erforderliche Gesamtzeit kann dann durch die gesamte amortisierte Zeit minus Nettozuwachs des Kontos abgeschätzt werden (vgl. dazu Abschnitt 3.3). Man kann den Kontostand als eine Menge von Zahlungseinheiten auffassen, mit denen man die zur Ausführung von Operationen anfallenden Kosten begleichen kann.

Wir ordnen einem aus dem anfangs leeren F-Heap durch eine Folge von Operationen erzeugten F-Heap h einen Kontostand $bal(h)$ wie folgt zu:

$$bal(h) = \begin{aligned} &\text{Anzahl Bäume in der Wurzelliste von } h \\ &+ 2 \cdot (\text{Anzahl markierter Knoten in } h, \text{ die} \\ &\quad \text{nicht in der Wurzelliste auftreten}) \end{aligned}$$

Die amortisierte Zeit zur Ausführung einer Einfüge-, Access-Min- und Meld-Operation ist $O(1)$. Denn die Einfüge-Operation erhöht lediglich die Zahl der Bäume in der Wurzelliste um 1; Access Min und Meld lassen die Gesamtzahl der Bäume und der markierten Knoten unverändert.

Um die amortisierten Kosten einer Delete-Min-Operation zu bestimmen, setzen wir zunächst voraus, daß jedes Verschmelzen zweier Bäume der Wurzelliste zu einem Baum genau eine Kosteneinheit verursacht, also durch das Verschwinden eines Baumes aus der Wurzelliste aufgewogen wird. Wir berücksichtigen daher bei der weiteren Analyse die Kosten des Verschmelzens nicht mehr. Die Anzahl der nicht in der Wurzelliste auftretenden markierten Knoten bleibt bei einer Delete-Min-Operation unverändert oder nimmt sogar ab, nämlich dann, wenn markierte Knoten in die Wurzelliste aufgenommen werden. Wir können uns daher bei der Untersuchung der Kontostandsänderung auf die Änderung der Anzahl Bäume in der Wurzelliste von h beschränken. Sei $w(h)$ diese Anzahl vor Entfernen des Minimums. Dann betragen die tatsächlichen Kosten der Delete-Min-Operation (ohne Berücksichtigung des Verschmelzens) gerade $O(\log N + w(h))$, da die — um maximal $O(\log N)$ Knoten vergrößerte — Wurzelliste von h einmal durchlaufen wird, um Bäume gleichen Ranges zu verschmelzen. Nach dem Verschmelzen enthält die Wurzelliste von h höchstens noch $O(\log N)$ Knoten. (Nach Ausführen einer Delete-Min-Operation ist h eine Binomial Queue; da h N Knoten enthält, besteht h aus höchstens $O(\log N)$ Bäumen.) Also sinkt der Kontostand von $O(w(h)) + 2 \cdot \text{Anzahl markierter Knoten}$ auf $O(\log N) + 2 \cdot \text{Anzahl markierter Knoten}$. Damit sind die amortisierten Kosten einer Delete-Min-Operation, also die tatsächlichen Kosten plus die Kontostandsänderung, gerade $O(\log N + w(h)) + O(\log N) - O(w(h)) = O(\log N)$.

Um die amortisierten Kosten einer Decrease-Key-Operation zu bestimmen, setzen wir voraus, daß jedes direkte und indirekte Abtrennen eines Knotens eine Kosteneinheit verursacht. Wird ein Knoten von seinem unmarkierten Vater abgetrennt, in die Wurzelliste aufgenommen und der Vater markiert, so verursacht dies eine Kosteneinheit. Zugleich nimmt der Kontostand um drei Einheiten zu. Die amortisierten Kosten dieser Operation sind also in $O(1)$. Nehmen wir nun an, ein Knoten p wird von einem markierten Vater ϕp abgetrennt; dann muß auch ϕp von dessen Vater $\phi\phi p$ abgetrennt werden usw., bis schließlich ein markierter Knoten von einem unmarkierten abgetrennt

wird. Jede Abtrennoperation, außer der letzten, verursacht eine Kosteneinheit, erhöht die Zahl der Bäume in der Wurzelliste um 1 und vermindert die Zahl der markierten Knoten, die zu $bal(h)$ beitragen, um 1; die amortisierten Kosten dafür sind also 0. Die letzte Abtrennoperation erhöht die Zahl der markierten Knoten um 1 und die Zahl der Bäume in der Wurzelliste um 1; sie verursacht ebenfalls eine Kosteneinheit. Insgesamt sind auch in diesem Fall die amortisierten Kosten in $O(1)$.

Weil eine Delete-Operation eine Decrease-Key-Operation mit anschließender Delete-Min-Operation ist, folgt sofort, daß auch die amortisierten Kosten einer Delete-Operation in $O(\log N)$ sind. Wir können unsere Überlegungen damit in folgendem Satz zusammenfassen.

Satz 6.1 *Führt man, beginnend mit dem anfangs leeren F-Heap, eine beliebige Folge von Operationen an Priority Queues aus, dann ist die dafür insgesamt benötigte Zeit beschränkt durch die gesamte amortisierte Zeit; die amortisierte Zeit einer einzelnen Delete-Min- und Delete-Operation ist in $O(\log N)$, die amortisierte Zeit aller anderen Operationen in $O(1)$.*

Wir können F-Heaps verwenden zur Implementation von Dijkstras Algorithmus zur Lösung des Single-source-shortest-paths-Problems für einen Graphen mit n Knoten und m Kanten. Der Algorithmus hat dann die Laufzeit $O(n \log n + m)$. Auch zur Implementation vieler anderer Algorithmen kann man F-Heaps verwenden.

Kürzlich wurden Relaxed Heaps in  als Alternative zu F-Heaps angegeben. Für sie gelten dieselben Schranken für die amortisierten Worst-case-Kosten zur Ausführung der Operationen an Priority Queues wie für F-Heaps. Für eine Variante von Relaxed Heaps erhält man aber dieselben Zeitschranken sogar für jeweils eine einzelne Operation im schlechtesten Fall. Die Struktur von Relaxed Heaps und die für sie erklärten Algorithmen zur Ausführung der Operationen an Priority Queues sind jedoch erheblich komplexer als für F-Heaps und übersteigen den Rahmen dieses Buches.

6.2 Union-Find-Strukturen

In einer ganzen Reihe von Algorithmen insbesondere aus dem Bereich der Algorithmen auf Graphen tritt als Teilaufgabe das Problem auf, für eine Menge von Objekten, z.B. für die Knoten oder Kanten eines Graphen, eine Einteilung in Äquivalenzklassen vorzunehmen. Man beginnt mit einer sehr feinen Einteilung, die sukzessive durch Vereinigen der Mengen vergrößert wird. Man kann diese Teilaufgabe als einen Spezialfall des Mengenmanipulationsproblems auffassen, der dadurch charakterisiert ist, daß auf einer Kollektion von Mengen die folgenden Operationen ausführbar sind.

Make-set(e, i) schafft eine neue Menge i mit e als einzigem Element; i ist also der Name der Menge; es wird vorausgesetzt, daß das Element e neu ist, also in keiner anderen Menge der Kollektion vorkommt.

Find(x) liefert den Namen der Menge, die das Element x enthält.

$Union(i, j, k)$ vereinigt die Mengen i und j zu einer neuen Menge mit Namen k . i und j werden aus der Kollektion von Mengen entfernt und k aufgenommen; es wird angenommen, daß i und j verschieden sind.

Wegen der bei der Operation *Make-set* gemachten Voraussetzung besteht die durch eine beliebige Folge dieser Operationen erzeugte Kollektion von Mengen stets aus paarweise disjunkten Mengen. Da es auf die Namen der Mengen nicht ankommt, kann man sie auch ganz unterdrücken und jeder Menge einen eindeutig bestimmten Repräsentanten, ein sogenanntes *kanonisches Element*, zuordnen. Das kanonische Element der durch $Make-set(e, i)$ geschaffenen Menge ist natürlich e . Die $Find(x)$ -Operation liefert das kanonische Element der Menge, in der x liegt. Der durch Vereinigung von zwei Mengen i und j entstehenden Menge kann man willkürlich ein neues kanonisches Element zuordnen, z.B. immer das kanonische Element von i . Wir verwenden daher in der Regel einfach die Operationen $Make-set(e)$, $Find(x)$, $Union(e, f)$ statt der oben angegebenen mit der offensichtlichen Bedeutung.

Das Problem, eine Datenstruktur zur Repräsentation einer Kollektion von paarweise disjunkten Mengen und Algorithmen zur Ausführung der Operationen *Make-set*, *Find* und *Union* auf dieser Kollektion zu finden, heißt das *Union-Find-Problem*.

Bevor wir mögliche Lösungen des Union-Find-Problems diskutieren, wollen wir ein einziges Beispiel für einen Algorithmus angeben, bei dessen Implementation man Lösungen des Union-Find-Problems verwenden kann.

6.2.1 Kruskals Verfahren zur Berechnung minimaler spannender Bäume

Wir lösen das Problem der Berechnung *minimaler spannender Bäume* für zusammenhängende, ungerichtete, gewichtete Graphen. Für eine ausführliche Behandlung dieses Problems verweisen wir auf das Kapitel 8.

Gegeben sei ein Graph G mit Knotenmenge V und Kantenmenge E . Jeder Kante $e \in E$ sei eine reelle Zahl $c(e)$ als Kosten (engl.: cost) zugeordnet. Der Graph sei ungerichtet und zusammenhängend, d.h. je zwei Knoten des Graphen seien durch mindestens einen (ungerichteten) Kantenzug miteinander verbunden. Wir verzichten wieder auf eine genaue, formale Definition. Man stelle sich den Graphen G einfach als Menge von Orten vor, die durch in beide Richtungen befahrbare Straßen miteinander verbunden sind. Die Kosten einer Kante $e = (v, w)$ ist dann die Länge der Straße e , die die Orte v und w miteinander verbindet.

Ein minimaler spannender Baum T (*minimum spanning tree*, kurz: *MST*) für G besteht aus allen Knoten V von G , enthält aber nur eine Teilmenge E' der Kantenmenge E von G , die alle Knoten des Graphen miteinander verbindet und die Eigenschaft hat, daß die Summe aller Kantengewichte den minimal möglichen Wert hat unter allen Teilmengen von E , die alle Knoten des Graphen G miteinander verbinden.

Im Bild der Orte und Straßen bedeutet die Konstruktion eines *MST* das Herausfinden eines Teilstraßennetzes kürzester Gesamtlänge, das noch alle Orte miteinander verbindet.

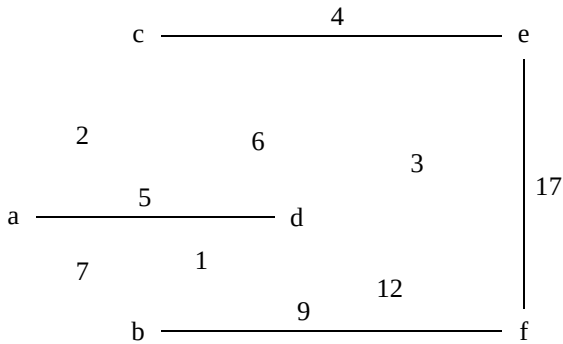


Abbildung 6.16

Als Beispiel betrachten wir den Graphen in Abbildung 6.16; das ist derselbe Graph wie in Abbildung 6.1, jedoch sind jetzt alle Kanten ungerichtet. Abbildung 6.17 zeigt einen *MST* für diesen Graphen.

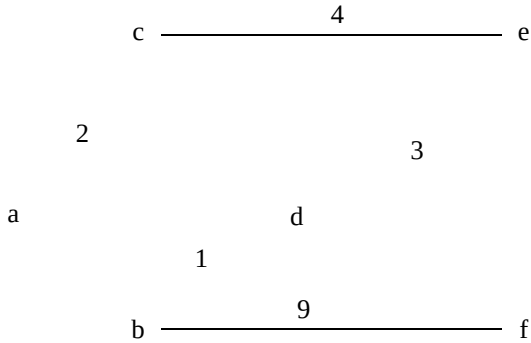


Abbildung 6.17

Es gibt zahlreiche Verfahren zur Konstruktion eines *MST*. Wir skizzieren ein Verfahren, das auf J. Kruskal zurückgeht. Die Idee des Verfahrens von Kruskal besteht darin, einen Wald von Teilbäumen des *MST* sukzessive zum *MST* zusammenwachsen zu lassen. Man beginnt mit Teilbäumen, die sämtlich nur aus je genau einem Knoten des gegebenen Graphen $G = (V, E)$ bestehen. Dann werden immer wieder je zwei verschiedene Teilbäume durch Hinzunahme einer Kante minimalen Gewichts zu einem verbunden, bis schließlich nur noch ein einziger Baum, eben der *MST*, übrigbleibt. Wir wollen hier wieder nicht die Frage der Korrektheit des Verfahrens diskutieren (siehe

dazu Abschnitt 8.6), sondern nur zeigen, wie Lösungen des Union-Find-Problems zur Implementation des Verfahrens verwendet werden können.

Das Verfahren von Kruskal geht aus von einer Kollektion K von einelementigen Knotenmengen. Die Knotenmengen werden sukzessive vergrößert, indem je zwei Mengen der Kollektion vereinigt werden, wenn sie durch eine Kante minimalen Gewichts miteinander verbunden werden können. Das Verfahren endet, wenn die Kollektion nur noch aus einer einzigen Menge (der Knotenmenge V des gegebenen Graphen) besteht. Etwas genauer kann das Verfahren wie folgt beschrieben werden:

```

procedure MST ( $((V,E) : \text{Graph})$ ;
  {berechnet zu einem zusammenhängenden, ungerichteten, gewichteten
   Graphen  $G = (V,E)$  einen minimalen spannenden Baum  $T = (V,E')$ }
begin
   $E' := \emptyset$ ;
   $K := \emptyset$ ;
  bilde Priority Queue  $Q$  aller Kanten in  $E$  mit den Kantengewichten
    als Prioritätsordnung;
  for all  $v \in V$  do Make-set ( $v$ );
  {jetzt besteht  $K$  aus allen Mengen  $\{v\}$ ,  $v \in V$ }
  while  $K$  enthält mehr als eine Menge do
    begin
       $(v,w) := \min(Q)$ ;  $\text{deletemin}(Q)$ ;
      if Find( $v$ )  $\neq$  Find( $w$ ) then
        begin
          Union( $v_0, w_0$ ), mit  $v_0 = \text{Find}(v)$ ,  $w_0 = \text{Find}(w)$ ;
           $E' := E' \cup \{(v,w)\}$ 
        end
      end
    end
  end

```

Wir verfolgen den Ablauf des Verfahrens am Beispiel des Graphen aus Abbildung 6.16. Anfangs besteht die Kollektion K aus den einelementigen Mengen $\{a\}$, $\{b\}$, $\{c\}$, $\{d\}$, $\{e\}$, $\{f\}$. Die Kante mit kleinstem Gewicht ist (b,d) . Also wird diese Kante zum Baum T hinzugenommen, und die zwei Mengen, die b und d enthalten, werden zu $\{b,d\}$ vereinigt. Dann wird die Kante (a,c) gewählt, $\{a\}$ und $\{c\}$ werden zu $\{a,c\}$ vereinigt und (a,c) wird zu T hinzugenommen. Als nächste wird die Kante (d,e) gewählt; weil d und e in verschiedenen Mengen der Kollektion K sind, werden die Mengen zu $\{b,d,e\}$ vereinigt und (d,e) in T aufgenommen. Dann wird die Kante (c,e) ausgewählt; wieder sind c und e in verschiedenen Mengen der Kollektion, so daß durch Vereinigung dieser Mengen $\{a,b,c,d,e\}$ entsteht und (c,e) in T aufgenommen wird. Die noch nicht betrachtete Kante mit kleinstem Gewicht ist (a,d) ; a und d liegen aber bereits in derselben Menge der Kollektion, so daß (a,d) nicht in T aufgenommen wird und keine Mengen von K vereinigt werden. Das entsprechende gilt für (c,d) und (a,b) . Die nächste betrachtete Kante ist (b,f) ; sie wird in T aufgenommen und die beiden Mengen, die b und f enthalten, zu einer Menge (der gesamten Knotenmenge) verschmolzen. Es müssen also keine weiteren Kanten mehr betrachtet werden. Tabelle 6.2 faßt alle Schritte nochmals zusammen.

Kollektion K	nächste betrachtete Kante	Hinzunahme zu T
$\{a\}, \{b\}, \{c\}, \{d\}, \{e\}, \{f\}$	(b, d)	<i>ja</i>
$\{a\}, \{b, d\}, \{c\}, \{e\}, \{f\}$	(a, c)	<i>ja</i>
$\{a, c\}, \{b, d\}, \{e\}, \{f\}$	(d, e)	<i>ja</i>
$\{a, c\}, \{b, d, e\}, \{f\}$	(c, e)	<i>ja</i>
$\{a, b, c, d, e\}, \{f\}$	(a, d)	<i>nein</i>
	(c, d)	<i>nein</i>
	(a, b)	<i>nein</i>
	(b, f)	<i>ja</i>
$\{a, b, c, d, e, f\}$		

Tabelle 6.2

6.2.2 Vereinigung nach Größe und Höhe

Die einfachste Möglichkeit zur Lösung des Union-Find-Problems besteht darin, jede Menge der Kollektion K durch einen (nichtsorti­erten) Baum beliebiger Ordnung zu repräsentieren; die Knoten des Baumes sind die Elemente der Menge. Es genügt, zu verlangen, daß die Wurzel des Baumes das kanonische Element der Menge enthält oder, falls man explizit mit Namen operiert, daß an der Wurzel der Name der Menge vermerkt ist. Jeder Knoten im Baum enthält einen Zeiger auf seinen Vater; die Wurzel zeigt auf sich selbst und enthält gegebenenfalls den Namen der Menge. Abbildung 6.18 zeigt ein Beispiel für eine Kollektion von zwei Mengen, die im Verlauf des Verfahrens von Kruskal auftritt.

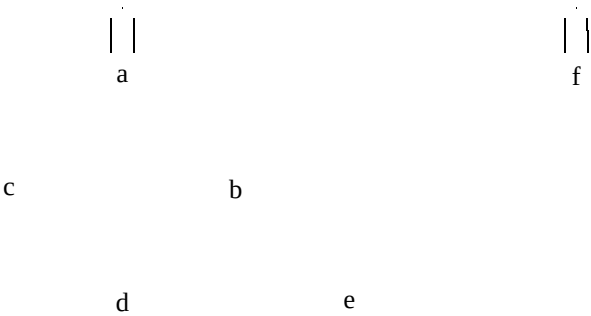


Abbildung 6.18

Wir nehmen an, daß man auf die Elemente der Mengen, also auf die Knoten in den die Menge repräsentierenden Bäumen, direkt zugreifen kann. Es liegt nahe, dazu einfach ein mit sämtlichen Elementen indiziertes Array zu verwenden, das zu jedem Element einen Verweis auf dessen Vater enthält. Diese Idee liefert eine sehr kompakte, zeigerlose Realisierung von Wäldern von Bäumen.

Im Falle des Beispiels aus Abbildung 6.18 nehmen wir also an, daß folgende Vereinbarungen gegeben sind:

```
type element = (a,b,c,d,e,f);
var p : array [element] of element
```

Die in Abbildung 6.18 gezeigte Situation wird durch folgende Belegung des Arrays p realisiert:

$$\begin{array}{rcccccc} x & : & a & b & c & d & e & f \\ p[x] & : & a & a & a & b & b & f \end{array}$$

Es ist klar, wie man die gewünschten Operationen ausführen kann:

$Make\text{-}set(x)$ liefert einen Baum mit einem einzigen Knoten x , dessen Vaterverweis auf sich selbst zurückweist.

Zur Ausführung von $Find(x)$ folgt man ausgehend vom Knoten x Vaterverweisen, bis man bei der Wurzel angelangt ist. Das merkt man daran, daß sich in der durchlaufenen Knotenfolge ein Knoten wiederholt. Sobald man bei der Wurzel angelangt ist, gibt man das Wurzelement als kanonisches Element der Menge aus, oder, falls man explizit mit Namen operiert, den bei der Wurzel gespeicherten Namen.

Zur Ausführung einer Vereinigungsoperation $Union(e, f)$ schaffen wir einen neuen Baum dadurch, daß wir (willkürlich) den Knoten f auf e zeigen lassen, also e zum kanonischen Element der durch Vereinigung neu entstehenden Menge machen.

Denken wir uns ein mit allen Elementen indiziertes Array p als global vereinbarte Variable gegeben, so kann man die Operationen wie folgt programmtechnisch realisieren.

```
var p: array [element] of element;

procedure Make-set ( $x$  : element);
begin p[x] := x end

procedure Union ( $e, f$  : element);
  begin p[f] := e end

function Find ( $x$  : element) : element;
var y : element;
begin y := x;
  while p[y]  $\neq$  y do y := p[y];
  Find := y
end
```

Make-set und *Union* sind in konstanter Zeit ausführbar; die Anzahl der Schritte zur Ausführung einer *Find*(x)-Operation ist proportional zur Anzahl der Knoten auf dem Pfad vom Knoten x zur Wurzel des Baumes. Weil wir keinerlei Bedingung an die Vereinigung zweier Bäume gestellt haben, kann der Aufwand für eine einzelne Find-Operation groß werden. Man betrachte dazu die folgende Operationsfolge:

$$\begin{aligned} & \text{Make-set}(i); & i = 1, \dots, N \\ & \text{Union}(i-1, i); & i = N, \dots, 2 \\ & \text{Find}(N) \end{aligned}$$

Offenbar wird ausgehend von N Bäumen mit je einem Knoten zunächst ein degenerierter Baum der Höhe N erzeugt, so daß die Find-Operation $\Omega(N)$ Schritte benötigt.

Es gibt zwei naheliegende Strategien, mit denen man verhindern kann, daß durch iteriertes Vereinigen von Bäumen zu linearen Listen degenerierte Bäume entstehen können: Vereinigung nach *Größe* und Vereinigung nach *Höhe*.

Wir haben nämlich beim oben angegebenen naiven Vereinigungsverfahren willkürlich festgesetzt, daß die durch eine Vereinigungsoperation $\text{Union}(e, f)$ entstehende Menge e als kanonisches Element haben soll. Natürlich hätten wir ebenso gut f als kanonisches Element wählen können und dazu den Knoten e auf f zeigen lassen. Man merkt sich nun jeweils an der Wurzel die Größe, d h. die gesamte Knotenzahl, bzw. die Höhe des Baumes und verfährt wie folgt.

Um zwei Bäume mit Wurzeln e und f zu vereinigen, macht man die Wurzel des Baumes mit kleinerer Größe (bzw. geringerer Höhe) zum direkten weiteren Sohn des Baumes mit der größeren Größe (bzw. Höhe). Falls die Größen von e und f (bzw. die Höhen) gleich sind, kann man e oder f zur Wurzel machen. Je nachdem, ob e oder f die Wurzel geworden ist, wird e oder f kanonisches Element der durch Vereinigung entstandenen Menge.

Es dürfte klar sein, wie man diese Strategien programmtechnisch realisieren kann. Die Funktion *Find* bleibt in jedem Fall unverändert. Wir geben die geänderten Prozeduren zur Ausführung einer *Make-set*- und *Union*-Operation für den Fall der Vereinigung nach Größe an. Dazu setzen wir voraus, daß ein weiteres Array *Größe* vereinbart ist, das zu jedem kanonischen Element eines Baumes die Anzahl der Elemente im Baum liefert.

procedure *Make-set* (x : *element*);

begin

$p[x] := x$;

$\text{Größe}[x] := 1$

end

procedure *Union* (e, f : *element*);

begin

if $\text{Größe}[e] < \text{Größe}[f]$ **then** *vertausche*(e, f);

 {jetzt ist e kanonisches Element der größeren Menge}

$p[f] := e$;

$\text{Größe}[e] := \text{Größe}[f] + \text{Größe}[e]$

end

Make-set und Union sind natürlich immer noch in konstanter Zeit ausführbar.

Lemma 6.3 *Das Verfahren Vereinigung nach Größe konserviert die folgende Eigenschaft von Bäumen: Ein Baum mit Höhe h hat wenigstens 2^h Knoten.*

Zum Beweis nehmen wir an, daß T_1 und T_2 Bäume mit den Größen $g(T_1)$ und $g(T_2)$ sind, die vereinigt werden sollen; h_1 und h_2 seien die Höhen von T_1 und T_2 . Der durch Vereinigung von T_1 und T_2 entstehende Baum $T_1 \cup T_2$ hat die in Abbildung 6.19 dargestellte Gestalt. D.h. wir nehmen ohne Einschränkung an, daß $g(T_1) \geq g(T_2)$ ist. Nach Voraussetzung hat T_i wenigstens 2^{h_i} , $i = 1, 2$, Knoten.

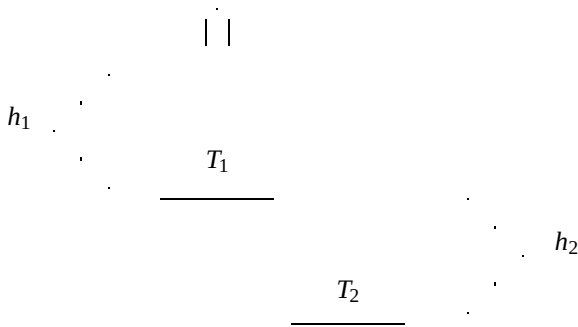


Abbildung 6.19

Fall 1: $\text{Höhe}(T_1 \cup T_2) = \max(\{h_1, h_2\})$.
Dann hat $T_1 \cup T_2$ trivialerweise wenigstens $2^{\text{Höhe}(T_1 \cup T_2)}$ Knoten.
Fall 2: Die Höhe des durch Vereinigung entstandenen Baumes ist gegenüber $\max(\{h_1, h_2\})$ um 1 gewachsen. Aufgrund der von uns getroffenen Annahmen ist das nur möglich, wenn $\text{Höhe}(T_1 \cup T_2) = h_2 + 1$ ist. Wir müssen die Größe $g(T_1 \cup T_2)$ des durch Vereinigung von T_1 und T_2 entstandenen Baumes abschätzen. Es gilt:

$$g(T_1) \geq g(T_2) \geq 2^{h_2}, \text{ also}$$
$$g(T_1 \cup T_2) = g(T_1) + g(T_2) \geq 2 \cdot 2^{h_2} = 2^{\text{Höhe}(T_1 \cup T_2)}$$

□

Als unmittelbare Folgerung aus Lemma 6.3 erhält man: Wird das Verfahren Vereinigung nach Größe iteriert angewandt, beginnend mit einer Folge von N Bäumen mit je genau einem Knoten, die N einelementige Mengen repräsentieren, so haben alle entstehenden Bäume eine Höhe $h \leq \log_2 N$.

Vereinigung nach Größe garantiert also, daß eine Find-Operation höchstens $O(\log N)$ Schritte kosten kann. Dasselbe gilt auch für die Strategie der Vereinigung nach Höhe. Denn auch für dieses Verfahren gilt die Aussage von Lemma 6.3 entsprechend, wie man leicht nachprüft. Das Verfahren Vereinigung nach Höhe hat gegenüber dem Verfahren Vereinigung nach Größe den (kleinen) Vorteil, daß die für die kanonischen Elemente mitzuführende Höheninformation nicht so stark wächst wie die Größe der Bäume; man kommt mit $\log \log N$ statt $\log N$ Bits Zusatzinformation für jeden Baum aus, um diese Strategie zu implementieren.

6.2.3 Methoden der Pfadverkürzung

Vereinigung nach Größe oder Höhe garantiert, daß die zur Ausführung einer Find-Operation zu durchlaufende Folge von Kanten (Vaterverweisen) nicht zu lang wird. Eine sehr drastische weitere Verkürzung dieser Pfade würde man dadurch erhalten, daß man alle Knoten des einen Baumes direkt auf die Wurzel des anderen zeigen läßt. Das ist natürlich nicht besonders effizient, weil dann die Vereinigungsoperation nicht mehr in konstanter Zeit ausführbar ist, sondern so viele Schritte benötigt, wie die (zweite) Menge Knoten hat.

Eine andere Möglichkeit zur Verkürzung von Pfaden, die bei Find-Operationen durchlaufen werden müssen, ist, die bei einer Find-Operation durchlaufenen Knoten unmittelbar oder zumindest näher an die Wurzel zu hängen. Das verteuert zwar die gerade durchgeführte Find-Operation, zahlt sich aber für künftige Find-Operationen aus, weil die dann noch zu durchlaufenden Pfade kürzer werden. Die naheliegendste Methode dieser Art ist die *Kompressionsmethode*: Sämtliche bei Ausführung einer Find-Operation durchlaufenen Knoten werden direkt an die Wurzel gehängt.

Diese Methode verlangt aber, daß man bei Ausführung von $Find(x)$ den von x zur Wurzel führenden Pfad zweimal durchläuft, weil man einen Knoten natürlich erst dann an die Wurzel anhängen kann, wenn man die Wurzel kennt. Die Kompressionsmethode kann wie folgt implementiert werden.

```
function Find( $x$  : element) : element;
var  $y, z, t$  : element;
begin
   $y := x$ ;
  while  $p[y] \neq y$  do  $y := p[y]$ ;
  {jetzt ist  $y$  die Wurzel; alle Knoten auf dem Pfad von  $x$  nach  $y$ 
   werden direkt an  $y$  angehängt}
   $z := x$ ;
  while  $p[z] \neq y$  do
    begin  $t := z$ ;  $z := p[z]$ ;  $p[t] := y$  end;
  Find :=  $y$ 
end
```


Ein Beispiel für die Wirkung der Kompressionsmethode zeigt Abbildung 6.20. Dort sind die vor Ausführung von $\text{Find}(x)$ vorhandenen Vaterverweise durchgezogen und die danach vorhandenen für die Knoten auf dem Pfad von x zur Wurzel gestrichelt gezeichnet.

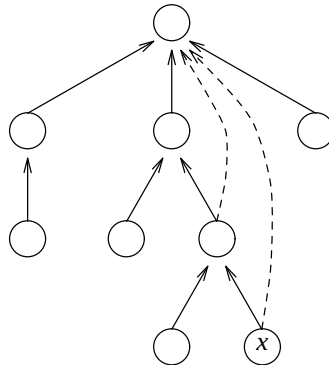


Abbildung 6.20

Die Analyse der Kompressionsmethode in Verbindung mit der Strategie Vereinigung nach Größe oder Vereinigung nach Höhe ist deshalb schwierig, weil die Kosten der Operationen von der Reihenfolge, in der sie ausgeführt werden, abhängen. Wir verweisen daher auf die Arbeit [10], in der die Kompressionsmethode und andere Methoden der Pfadverkürzung analysiert werden. Die Herleitung der kleinsten oberen Schranke für die amortisierten Worst-case-Kosten der Kompressionsmethode findet man auch in der Monographie von Tarjan [11].

Wir geben hier nur das Ergebnis der Analyse an. Sei m die Anzahl der Operationen und n die Anzahl der Elemente in allen Mengen. D.h. es werden n Make-set-Operationen und höchstens $n - 1$ Union-Operationen ausgeführt und es ist $m \geq n$. Die Aussage über die zur Ausführung der m Operationen benötigte Anzahl von Schritten macht Gebrauch von einer sehr schwach wachsenden Funktion, der Inversen der Ackermannfunktion. Die Ackermannfunktion $A(i, j)$ ist für $i, j \geq 1$ wie folgt definiert:

$$\begin{aligned} A(1, j) &= 2^j, \text{ für } j \geq 1, \\ A(i, 1) &= A(i - 1, 2), \text{ für } i \geq 2 \\ A(i, j) &= A(i - 1, A(i, j - 1)), \text{ für } i, j \geq 2 \end{aligned}$$

Die Inverse der Ackermannfunktion $\alpha(m, n)$ ist für $m \geq n \geq 1$ wie folgt definiert:

$$\alpha(m, n) = \min\{i \geq 1 \mid A(i, \lfloor m/n \rfloor) > \log n\}$$

Die bemerkenswerteste Eigenschaft der Ackermannfunktion ist ihr „explosives“ Wachstum. (Häufig wird in der ersten Definitionszeile der Ackermannfunktion $A(1, j) = j + 1$ gesetzt und nicht, wie oben angegeben $A(1, j) = 2^j$. Das explosive Wachstum tritt jedoch auch dann ein, nur etwas später.) Weil A sehr schnell wächst, folgt umgekehrt, daß α sehr langsam wächst. Es ist beispielsweise $A(3, 1) = A(2, 2) = A(1, A(2, 1)) = A(1, A(1, 2)) = A(1, 4) = 16$; also ist $\alpha(m, n) \leq 3$ für $n < 2^{16} = 65536$. $A(4, 1) = A(2, 16)$ ist bereits so riesig groß, daß $\alpha(m, n) \leq 4$ ist für alle praktisch auftretenden Werte von n und m .

Tarjan hat nun gezeigt: Benutzt man die Strategie Vereinigung nach Größe oder Vereinigung nach Höhe und benutzt man bei der Ausführung von Find-Operationen die Kompressionsmethode, so benötigt man zur Ausführung einer beliebigen Folge von $m \geq n$ Operationen $\Theta(m \cdot \alpha(m, n))$ Schritte. Die zur Ausführung einer einzelnen Operation in einer beliebigen Folge von Operationen erforderliche Schrittzahl ist also praktisch konstant.

Neben der Kompressionsmethode gibt es noch eine Reihe anderer Methoden zur Pfadverkürzung, die das Ziel verfolgen, bei Ausführung einer *Find*(x)-Operation den Pfad von x zur Wurzel nicht zweimal durchlaufen zu müssen. Wir geben zwei Methoden an, die asymptotisch dieselbe Laufzeit haben wie die Kompressionsmethode, vgl. .

Aufteilungsmethode (Splitting): Während der Ausführung einer Find-Operation teilt man den Suchpfad dadurch in zwei Pfade von etwa halber Länge auf, daß man jeden Knoten (mit Ausnahme des letzten und vorletzten) statt auf seinen Vater auf seinen Großvater zeigen läßt. Ein Beispiel zeigt Abbildung 6.21.

Die Funktion *Find* kann also wie folgt implementiert werden:

```
function Find( $x$  : element) : element;
var  $x, t$  : element;
begin
   $y := x$ ;
  while  $p[p[y]] \neq p[y]$  do
    begin
       $t := y$ ;  $y := p[y]$ ;  $p[t] := p[p[t]]$ 
    end
  end
```

Halbierungsmethode (Halving): Während der Ausführung einer Find-Operation läßt man jeden zweiten Knoten auf seinen Großvater zeigen (mit Ausnahme der eventuell letzten Knoten). Man ändert also die Verweise für den 1., 3., 5., ... Knoten, und läßt die Verweise für den 2., 4., 6., ... unverändert. Auf diese Weise wird die Länge der Suchpfade für nachfolgende Find-Operationen etwa halbiert. Ein Beispiel zeigt Abbildung 6.22.

Die Funktion *Find* kann jetzt wie folgt implementiert werden:

```
function Find( $x$  : element) : element;
var  $y, t$  : element;
begin
   $y := x$ ;
  while  $p[p[y]] \neq p[y]$  do
    begin
```

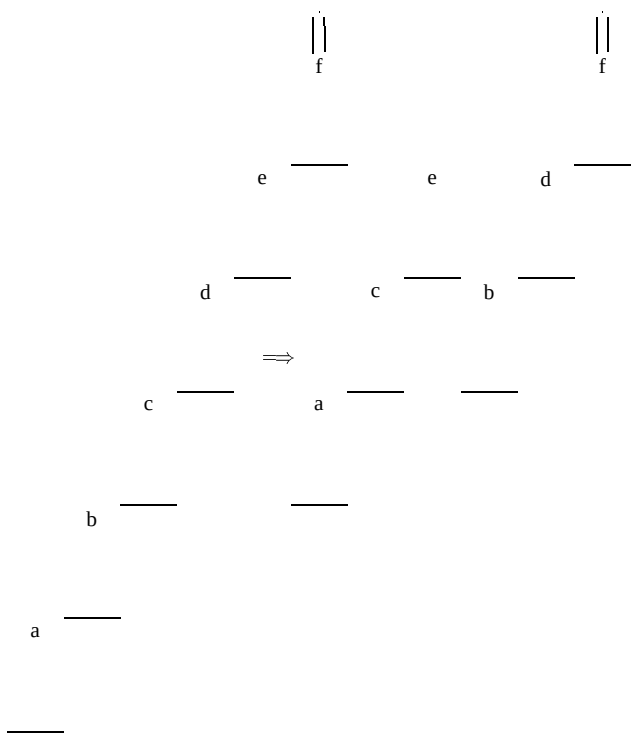


Abbildung 6.21

```
    t := p[p[y]]; p[y] := t; y := t
  end
end
```

Es ist klar, daß damit das Spektrum der möglichen Methoden zur Pfadverkürzung keineswegs erschöpft ist. Beispielsweise könnte man einen Suchpfad ebenso gut in drei, vier, usw. statt zwei etwa gleichlange Pfade aufteilen. In der Literatur sind eine Reihe weiterer Methoden vorgeschlagen und untersucht worden; man vergleiche dazu [].

6.3 Allgemeiner Rahmen

Wörterbücher (Dictionaries), Priority Queues und Union-Find-Strukturen kann man als Spezialfälle eines allgemeinen Mengenmanipulationsproblems auffassen, das wie folgt beschrieben werden kann. Gegeben ist eine Kollektion K von paarweise disjunkten

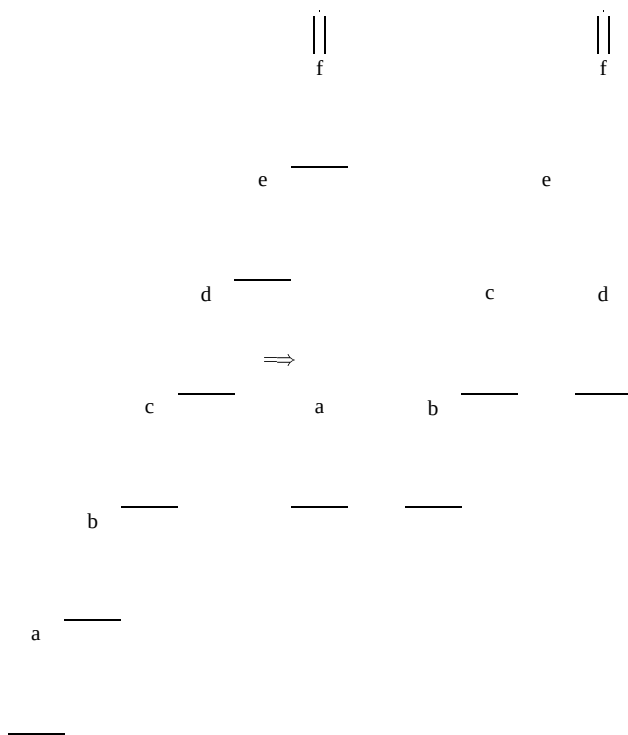


Abbildung 6.22

Mengen, deren Elemente zu einem Universum U gehören und deren Namen zu einer Menge N von Namen gehören.

$$\begin{aligned} K &= \{S_{n_1}, \dots, S_{n_t}\}, & S_{n_i} \cap S_{n_j} &= \emptyset, \text{ für } i \neq j. \\ U &\supseteq \bigcup K = \{x \in S \mid S \in K\} \\ N &\supseteq \{n_i \mid S_{n_i} \in K\} \end{aligned}$$

Das Universum sei eine geordnete Menge von Elementen. (Häufig nimmt man sogar an, daß das Universum U und die Namensmenge N die Menge der positiven ganzen Zahlen sind.)

Auf der Kollektion K soll eine beliebige Folge von Operationen, wie sie in Tabelle 6.3 angegeben sind, ausführbar sein. Diese Liste möglicher und sinnvoller Operationen für eine Kollektion K von Mengen ist keineswegs vollständig, sondern soll das breite Spektrum derartiger Operationen illustrieren.

Eine Lösung des Mengenmanipulationsproblems sollte natürlich berücksichtigen, welche Operationen mit welcher Häufigkeit, in welcher Reihenfolge ausgeführt werden. In vielen Fällen kann man jedoch eine Lösung wählen, deren Grobstruktur wie

<i>Make-set</i> (x, n):	Bilde eine Menge mit einzigem Element x und gebe ihr den Namen n . (Dabei wird vorausgesetzt, daß x und n neu sind.)
<i>Suche</i> (x, n):	Suche x in der Menge mit Namen n .
<i>Einfüge</i> (x, n):	Füge x in die Menge mit Namen n ein. (Dabei wird vorausgesetzt, daß x neu ist.)
<i>Entferne</i> (x, n):	Entferne x aus der Menge mit Namen n .
<i>Find</i> (x):	Bestimme den Namen der Menge, die x enthält.
<i>Union</i> (i, j, k):	Vereinige die Mengen mit Namen i und j zu einer Menge mit Namen k .
<i>Access-Min</i> (n):	Bestimme das Minimum in der Menge mit Namen n .
<i>Delete-Min</i> (n):	Entferne das Minimum in der Menge mit Namen n .
<i>Nachfolger</i> (x, n):	Bestimme das zu x nächstgrößere Element in der Menge mit Namen n .
<i>Vorgänger</i> (x, n):	Bestimme das zu x nächstkleinere Element in der Menge mit Namen n .
<i>(k)-tes Element</i> :	Bestimme das k -größte Element in $\bigcup K$

Tabelle 6.3

folgt beschrieben werden kann. Repräsentiere $\bigcup K \subseteq U$ durch einen balancierten, sortierten Binärbaum, den $\bigcup$ -Baum. Wenn man die Operation *k-tes Element* unterstützen möchte, ist es sinnvoll, an jedem Knoten p noch einen Zähler $z(p)$ mitzuführen, der die Anzahl der Schlüssel im Teilbaum mit Wurzel p angibt. Stelle jede Menge S_i der Kollektion K durch einen nichtsortierten *Mengenbaum* dar, den S_i -Baum. Der Knoten x im $\bigcup$ -Baum ist durch einen Zeiger mit dem Knoten x im S_i -Baum verbunden, wenn $x \in S_i$ ist. Die Menge aller Namen von Mengenbäumen ist in einem sortierten, balancierten N -Baum gespeichert. Die Wurzel eines jeden Mengenbaums ist durch je einen Verweis in beiden Richtungen mit seinem Namen im N -Baum verbunden.

Sollen Find-Operationen unterstützt werden, zeigt jeder Knoten eines Mengenbaumes auf seinen Vater. Sollen Access-Min- und Delete-Min-Operationen unterstützt werden, sind die Mengenbäume heapgeordnet. Falls die Union-Operation als Vereinigung nach Größe oder Höhe ausgeführt werden soll, muß man an den Wurzeln der Mengenbäume die Größe oder Höhe mitführen. Die in Abbildung 6.23 gezeigte Struktur der Lösung muß also auf den jeweils aktuell vorliegenden Fall zugeschnitten werden.

Wir geben an, wie einige der genannten Operationen ausgeführt werden können.

Einfüge(x, i): Füge x im $\bigcup$ -Baum ein; suche i im N -Baum, folge Zeiger zur Wurzel des S_i -Baumes, füge x in diesen Baum ein. (Ist beispielsweise S_i heapgeordnet, so beinhaltet das Einfügen von x in S_i auch die Wiederherstellung der Heapordnung.)

Entferne(x, i): Die Ausführung dieser Operation verlangt, x im Mengenbaum S_i zu finden. Da wir im allgemeinen nicht voraussetzen, daß diese Bäume Suchbäume sind, sucht man x zunächst im $\bigcup$ -Baum, folgt dem Zeiger von x zum Knoten gleichen Namens in einem der Mengenbäume, läuft dort zur Wurzel und stellt über den Verweis in

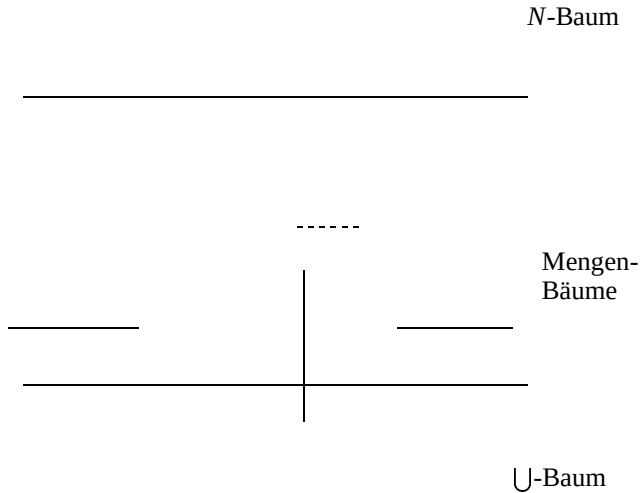


Abbildung 6.23

den N -Baum fest, ob x in S_i auftritt. Dann entfernt man gegebenenfalls x aus S_i und aus dem $\cup$ -Baum.

k-tes Element: Man beginnt bei der Wurzel p des $\cup$ -Baumes. Falls $z(p) < k$ ist, gibt es kein k -tes Element im $\cup$ -Baum. Sonst inspiziert man den linken Sohn λp und dessen Zähler $z(\lambda p)$. Falls $k \leq z(\lambda p)$ ist, setzt man die Suche nach dem k -ten Element rekursiv beim linken Sohn fort. Falls $k = z(\lambda p) + 1$ ist, ist das in p gespeicherte Element das gesuchte. Falls schließlich $k > z(\lambda p) + 1$ ist, setzt man die Suche rekursiv beim rechten Sohn von p fort, sucht dort aber nach dem $(k - z(\lambda p) - 1)$ -ten Element.

Es ist nicht schwer, sich in allen anderen Fällen zu überlegen, wie die Operationen ausgeführt werden können und welche zusätzlichen Voraussetzungen man gegebenenfalls über die Struktur der Mengenbäume usw. benötigt, um die gewünschten Operationen effizient ausführen zu können.

Die im vorigen Abschnitt 6.2 angegebenen Lösungen des Union-Find-Problems kann man folgendermaßen unter den hier angegebenen Rahmen subsummieren: Im Falle des Union-Find-Problems können $\cup$ -Baum und N -Baum jeweils zu Arrays vereinfacht werden. Falls man Namen unterdrücken möchte und mit kanonischen Elementen operiert, kann man auf den N -Baum (oder ein N -Array) sogar ganz verzichten.

6.4 Aufgaben

Aufgabe 6.1

Eine Vorrangwarteschlange für ganzzahlige Schlüssel soll als Bruder-Baum realisiert werden, wobei die Schlüssel in den Blättern gespeichert werden. Als Wegweiser soll an jedem binären inneren Knoten der kleinste Schlüsselwert seines Teilbaumes stehen.

- Geben Sie ein Einfüge-Verfahren für beliebige Schlüssel an und beschreiben Sie dieses, zusammen mit dem Knotenformat, in Pascal.
- Geben Sie je eine Umstrukturierungs-Invariante und eine Umstrukturierungs-Operation für den Fall des Einfügens eines beliebigen Schlüssels und des Entfernens des Minimums an. Beachten Sie, daß Schlüssel nicht unbedingt sortiert in symmetrischer Reihenfolge auftreten, und daß Wegweiser angepaßt werden müssen.
- Beschreiben Sie die beiden Umstrukturierungs-Operationen aus b) als Pascal-Prozeduren.
- Beschreiben Sie die Priority-Queue-Operationen Access Min und Delete Min ebenfalls in Pascal.

Aufgabe 6.2

Ein Linksbaum, der als Priority Queue für eine Menge ganzzahliger Schlüssel dient, kann konstruiert werden, indem man diese Schlüssel in einer beliebigen Reihenfolge in den anfangs leeren Linksbaum unter Zuhilfenahme der Funktion *Verschmelzen* einfügt.

- Beschreiben Sie eine Folge von N Schlüssel (für beliebiges, natürliches N), für die der durch fortgesetztes Einfügen entstehende Linksbaum zu einer linearen Liste degeneriert.
- Beschreiben Sie eine Folge von $2^k - 1$ Schlüssel ($k \geq 1$, beliebig), für die der durch fortgesetztes Einfügen entstehende Linksbaum ein vollständiger Binärbaum ist.
- Wieviele strukturell verschiedene Linksbäume für vier Schlüssel gibt es? Geben Sie für jeden dieser Bäume alle Reihenfolgen der Schlüssel 1, 2, 3, 4 an, durch die er bei fortgesetztem Einfügen in den anfangs leeren Linksbaum erzeugt werden kann.

Aufgabe 6.3

Eine Binomial Queue, also ein Wald von Binomialbäumen, soll durch fortgesetztes Einfügen ganzzahliger Schlüssel in die anfangs leere Queue erzeugt werden.

- Geben Sie eine Folge von vier Schlüssel an, für die die entstehende Binomial Queue strukturell gleich (gleich, wenn man keine Reihenfolge der Söhne unterstellt) ist mit dem entstehenden Linksbaum.

- b) Verfolgen Sie die Entwicklung einer anfangs leeren Binomial Queue beim Einfügen der Schlüssel 17, 9, 12, 8, 15, 6 und beim anschließenden Entfernen des Schlüssels 9.
- c) Definieren Sie das Knotenformat von Binomialbäumen für ganzzahlige Schlüssel in Pascal. Schreiben Sie in Pascal Prozeduren und Funktionen für die Operationen Access Min, Einfügen, Entfernen, Minimum Entfernen, Herabsetzen und Verschmelzen.

Aufgabe 6.4

Verfolgen Sie im einzelnen, wie sich der anfangs leere Fibonacci Heap verändert, wenn er als Priority Queue für das in Abschnitt 6.1.1 beschriebene Verfahren von Dijkstra zur Berechnung kürzester Pfade für den in Abbildung 6.1 gezeigten Graphen eingesetzt wird. Verfolgen Sie insbesondere für jede Operation die Änderung von Markierungen und des Kontostandes. Vergleichen Sie als alternative Implementierungen der Priority Queue für dieses Beispiel Binomial Queues und Linksbäume.

Aufgabe 6.5

Verfolgen Sie im einzelnen die Veränderungen einer Union-Find-Struktur zur Berechnung eines minimalen, spannenden Baumes nach Kruskal für das in Abbildung 6.16 angegebene Beispiel. Welche Pfadlängen ergeben sich für die einzelnen Find-Operationen, wenn man sich bei der Vereinigung nach der Höhe von Bäumen richtet? Welchen Effekt hat im Beispiel die Kompressionsmethode zur Pfadverkürzung?

Aufgabe 6.6

Bei der Kompressionsmethode zur Pfadverkürzung haben nach Erledigung einer Operation $Find(x)$ alle ursprünglich auf dem Pfad von x zur Wurzel des Baumes gelegenen Knoten die Entfernung 1 zur Wurzel. Entwerfen und implementieren Sie eine Pfadverkürzungsmethode, bei der diese Entfernung höchstens 2 beträgt, bei der man aber den Pfad von x zur Wurzel nur einmal durchläuft.

Literaturliste zu Kapitel 6: Manipulation von Mengen

Seite 377

- [3] A. V. Aho, J. E. Hopcroft und J. D. Ullman. The Design and Analysis of Computer Algorithms. Addison-Wesley, Reading, Massachusetts, 1974.
- [180] R. E. Tarjan. Data structures and network algorithms. In SIAM CBMS-NSF Regional Conference Series in Applied Mathematics 44, Philadelphia, 1983. SIAM.

Seite 378

- [89] D. E. Knuth. The Art of Computer Programming, Vol.3: Sorting and Searching. Addison-Wesley, Reading, Massachusetts, 1973.
- [3] A. V. Aho, J. E. Hopcroft und J. D. Ullman. The Design and Analysis of Computer Algorithms. Addison-Wesley, Reading, Massachusetts, 1974.
- [180] R. E. Tarjan. Data structures and network algorithms. In SIAM CBMS-NSF Regional Conference Series in Applied Mathematics 44, Philadelphia, 1983. SIAM.

Seite 379

- [180] R. E. Tarjan. Data structures and network algorithms. In SIAM CBMS-NSF Regional Conference Series in Applied Mathematics 44, Philadelphia, 1983. SIAM.
- [35] E. W. Dijkstra. A note on two problems in connexion with graphs. Numer. Math., 1:269-271, 1959.

Seite 387

- [89] D. E. Knuth. The Art of Computer Programming, Vol.3: Sorting and Searching. Addison-Wesley, Reading, Massachusetts, 1973.

Seiten 393, 394

- [190] J. Vuillemin. A data structure for manipulating priority queues. Comm. ACM, 21:309-315, 1978.

Seite 400

- [60] M. L. Fredman und R. E. Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. J. Assoc. Comput. Mach., 34:596-615, 1987.

Seite 402

- [39] J. R. Driscoll, H. N. Gabow, R. Shrairman und R. E. Tarjan. Relaxed heaps: An alternative to Fibonacci heaps with applications to parallel computation. Comm. ACM, 31:1343-1354, 1988.

Seite 404

- [96] J. B. Kruskal. On the shortest spanning subtree of a graph and the traveling salesman problem. In Proc. AMS 7, S. 48-50, 1956.

Seite 411

- [182] R. E. Tarjan und J. van Leeuwen. Worst case analysis of set union algorithms. J. Assoc. Comput. Mach., 31:245-281, 1984.
- [180] R. E. Tarjan. Data structures and network algorithms. In SIAM CBMS-NSF Regional Conference Series in Applied Mathematics 44, Philadelphia, 1983. SIAM.

Seiten 412, 413

- [182] R. E. Tarjan und J. van Leeuwen. Worst case analysis of set union algorithms. J. Assoc. Comput. Mach., 31:245-281, 1984.

Kapitel 7

Geometrische Algorithmen

7.1 Einleitung

Die Geometrie ist eines der ältesten Gebiete der Mathematik, dessen Wurzeln bis in die Antike zurückreichen. Algorithmische Aspekte und die Lösung geometrischer Probleme mit Hilfe von Computern haben aber erst in jüngster Zeit verstärktes Interesse gefunden. Der Grund dafür liegt sicherlich in gewandelten Anforderungen durch neue Anwendungen, die von der Bildverarbeitung, Computer-Graphik, Geographie, Kartographie usw. bis hin zum physischen Entwurf höchstintegrierter Schaltkreise reichen.

So ist in den letzten Jahren ein neues Forschungsgebiet entstanden, das unter dem Namen “Algorithmische Geometrie” (Computational Geometry) inzwischen einen festen Platz innerhalb des Gebiets “Algorithmen und Datenstrukturen” einnimmt. Der Name “Algorithmische Geometrie” geht zurück auf eine im Jahre 1978 erschienene Dissertation von M. Shamos, vgl. [1]. Im CAD-Bereich und in der Computer-Graphik wurde der Begriff allerdings schon früher mit etwas anderer Bedeutung verwendet, vgl. hierzu

[2]. Seit der Dissertation von Shamos ist eine wahre Flut von Arbeiten in diesem Gebiet erschienen. Es ist daher völlig unmöglich, die Hunderte von inzwischen untersuchten Problemen und erzielten Einzellösungen auch nur annähernd vollständig und systematisch darzustellen. Um eine bessere und vollständige Übersicht über das Gebiet zu erhalten, sollte der Leser die Bibliographie [3] mit über 600 Einträgen, die Übersichtsarbeit [4], die Monographie [5] und die Bücher [6], [7] und [8] konsultieren.

Wir werden uns in diesem Kapitel auf die Darstellung einiger weniger, aber durchaus grundlegender Probleme, Datenstrukturen und Algorithmen beschränken. Im Abschnitt 7.2 stellen wir das Scan-line-Prinzip vor, das sich als Mittel zur Lösung zahlreicher geometrischer Probleme inzwischen bewährt hat. Wie das Divide-and-conquer-Prinzip zur Lösung geometrischer Probleme eingesetzt werden kann, zeigt Abschnitt 7.3. Zur Speicherung und Manipulation von Daten mit einer räumlichen Komponente reichen die bekannten, zur Speicherung von Mengen ganzzahliger Schlüssel geeigneten Strukturen nicht aus. In Abschnitt 7.4 stellen wir einige Strukturen vor, die dafür in Frage kommen, und zwar Segment-, Intervall-, Bereichs- und Prioritäts-Suchbäume. In den Abschnitten 7.2, 7.3 und 7.4 haben wir es in der Regel mit Mengen

iso-orientierter Objekte in der Ebene zu tun, d. h. mit Mengen von Objekten, die zu rechtwinklig gewählten Koordinaten ausgerichtet sind. Beispiele sind Mengen horizontaler und vertikaler Liniensegmente in der Ebene oder Mengen von Rechtecken mit zueinander parallelen Seiten.

In Abschnitt 7.5 zeigen wir, wie Algorithmen, die für Mengen iso-orientierter Objekte entwickelt wurden, übertragen werden können auf Mengen beliebig orientierter Objekte. Das Verfahren ist auf solche Algorithmen anwendbar, die sich auf sogenannte "Skelettstrukturen" stützen, wie sie in Abschnitt 7.4 beschrieben werden. Die vielfältige Verwendbarkeit dieser Strukturen wird auch im Abschnitt 7.6 belegt. Dort werden ein Spezialfall eines Standardproblems aus der Computergraphik, das Hidden-line-Eliminationsproblem, und ein allgemeines Suchproblem behandelt. Eine insbesondere zur Lösung von geometrischen Nachbarschaftsanfragen nützliche Struktur, das sogenannte Voronoi-Diagramm, wird im Abschnitt 7.7 behandelt.

7.2 Das Scan-line-Prinzip

Geometrische Probleme treten in vielen Anwendungsbereichen auf. Wir wollen uns jedoch darauf beschränken, nur einen Anwendungsbereich exemplarisch etwas genauer zu betrachten und geometrische Probleme diskutieren, die beim Entwurf höchstintegrierter Schaltungen auftreten. Man kann den Entwurfsprozeß ganz grob in zwei Phasen einteilen, in die funktionelle und die physikalische Entwurfsphase. Ziel der zweiten Entwurfsphase ist schließlich die Herstellung der Fertigungsunterlagen (Masken) für die Chipproduktion. Vom Standpunkt der algorithmischen Geometrie aus betrachtet geht es hier darum, eine enorm große Anzahl von Rechtecken auf die verschiedenen Schichten (Diffusions-, Polysilikon-, Metall- usw. Schicht) so zu verteilen, daß die von ihnen repräsentierten Transistoren, Widerstände, Leitungen usw. die gewünschten Schaltfunktionen realisieren. Dabei sind zahlreiche Probleme zu lösen, die inhärent geometrischer Natur sind. Beispiele sind:

Das Überprüfen der geometrischen Entwurfsregeln (*design-rule checking*): Hier wird das Einhalten von durch die jeweilige Technologie vorgegebenen geometrischen Bedingungen, wie minimale Abstände, maximale Überlappungen usw., überprüft.

Schaltelement-Extraktion (*feature extraction*): Hier werden aus der geometrischen Erscheinungsform elektrische Schaltelemente und ihre Verbindungen untereinander extrahiert.

Plazierung und Verdrahtung: Die Schaltelemente müssen möglichst platzsparend und so angeordnet werden, daß die notwendigen (elektrischen) Verbindungen leicht herstellbar sind. Für diese beim VLSI-Design auftretenden geometrischen Probleme ist charakteristisch, daß die dabei vorkommenden geometrischen Objekte einfach sind, aber die Anzahl der zu verarbeitenden Daten sehr groß ist. Typisch ist eine Anzahl von 5 bis 10 Millionen iso-orientierter Rechtecke in der Ebene.

In der algorithmischen Geometrie hat man den iso-orientierten Fall besonders intensiv studiert, vgl. hierzu die Übersicht von Wood [1]. Das ist der Fall, in dem alle auftretenden Liniensegmente (also z.B. Rechteckseiten) und Linien parallel zu einer

der Koordinatenachsen verlaufen. Eine der leistungsfähigsten Techniken zur Lösung geometrischer Probleme, das sogenannte *Scan-line-Prinzip*, läßt sich in diesem Fall besonders einfach erklären und führt nicht selten zu optimalen Problemlösungen. Wir erklären dieses Prinzip jetzt genauer (vgl. auch [E]).

Gegeben sei eine Menge von achsenparallelen Objekten in der Ebene, z.B. eine Menge vertikaler und horizontaler Liniensegmente, eine Menge iso-orientierter Rechtecke oder eine Menge iso-orientierter, rechteckiger, einfacher Polygone. Die Idee ist nun, eine vertikale Linie (die sogenannte *Scan-line*) von links nach rechts (oder alternativ: eine horizontale Linie von oben nach unten) über die gegebene Menge zu schwenken, um ein die Menge betreffendes statisches, zweidimensionales geometrisches Problem in eine dynamische Folge eindimensionaler Probleme zu zerlegen. Die Scan-line teilt zu jedem Zeitpunkt die gegebene Menge von Objekten in drei disjunkte Teilmengen: die *toten* Objekte, die bereits vollständig von der Scan-line überstrichen wurden, die gerade *aktiven* Objekte, die gegenwärtig von der Scan-line geschnitten werden und die noch *inaktiven* (oder: *schlafenden*) Objekte, die erst künftig von der Scan-line geschnitten werden. Die Scan-line definiert eine lokale Ordnung auf der Menge der jeweils aktiven Objekte; sie muß gegebenenfalls den sich ändernden lokalen Verhältnissen angepaßt werden und kann für problemspezifische Aufgaben konsultiert werden. Während man die Scan-line über die Eingabeszene hinwegschwenkt, hält man eine dynamische, d.h. zeitlich veränderliche, problemspezifische *Vertikalstruktur L* aufrecht, in der man sich alle für das jeweils zu lösende Problem benötigten Daten merkt. Eine wichtige Beobachtung ist nun, daß man an Stelle eines kontinuierlichen Schwenks (englisch: sweep) die Scan-line in diskreten Schritten über die gegebene Szene führen kann. Sei Q die aufsteigend sortierte Menge aller x -Werte von Objekten. D.h.: Im Falle einer Menge von horizontalen Liniensegmenten ist Q die Menge der linken und rechten Endpunkte, im Falle einer Menge von Rechtecken ist Q die Menge aller linken und rechten Rechteckseiten, usw. Ganz allgemein wird Q gerade so gewählt, daß sich zwischen je zwei aufeinanderfolgenden Punkten in Q weder die Zusammensetzung der Menge der gerade aktiven Objekte noch deren relative Anordnung (längs der Scan-line) ändern. Dann genügt es, Q als Menge der *Haltepunkte* der Scan-line zu nehmen. Statt eines kontinuierlichen Schwenks "springt" man mit der Scan-line von Haltepunkt zu Haltepunkt in aufsteigender x -Reihenfolge.

Ein vom jeweils zu lösenden Problem unabhängiger algorithmischer Rahmen für das Scan-line-Prinzip sieht also wie folgt aus:

Algorithmus Scan-line-Prinzip

{liefert zu einer Menge iso-orientierter Objekte problemabhängige Antworten}

$Q :=$ objekt- und problemabhängige Folge von Haltepunkten in aufsteigender x -Reihenfolge;

$L := \emptyset$; {angeordnete Menge der jeweils aktiven Objekte}

while Q nicht leer **do**

begin

 wähle nächsten Haltepunkt aus Q und entferne ihn aus Q ;

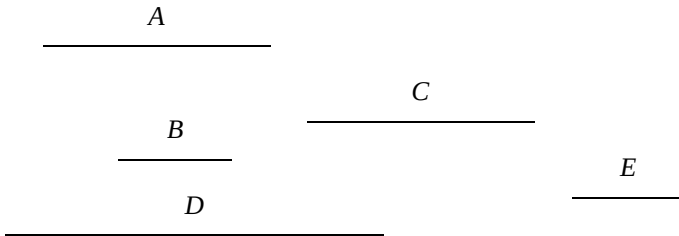
 update(L) und gib (problemabhängige) Teilantwort aus

end

Wir wollen das durch diesen Rahmen formulierte Scan-line-Prinzip jetzt auf drei konkrete Probleme anwenden.

7.2.1 Sichtbarkeitsproblem

Als einfachstes Beispiel für die Anwendung des Scan-line-Prinzips bringen wir die Lösung eines bei der Kompaktierung höchstintegrierter Schaltkreise auftretenden *Sichtbarkeitsproblems*. Zur Kompaktierung in y -Richtung müssen Abstandsbedingungen zwischen relevanten Paaren von (Schalt-) Elementen eingehalten werden. Dazu müssen die relevanten Paare zunächst einmal bestimmt werden. Hierzu genügt es, die beteiligten Elemente durch horizontale Liniensegmente darzustellen und die Menge aller Paare zu bestimmen, die sich sehen können (vgl. [1, 2]). Genauer: Zwei Liniensegmente s und s' in einer gegebenen Menge horizontaler Liniensegmente sind gegenseitig sichtbar, wenn es eine vertikale Gerade gibt, die s und s' , aber kein weiteres Liniensegment der Menge zwischen s und s' schneidet. Wir betrachten ein Beispiel mit fünf Liniensegmenten A, B, C, D, E :



Die Menge der gegenseitig sichtbaren Paare besteht genau aus den (ungeordneten) Paaren (A, B) , (A, D) , (B, D) , (C, D) .

Natürlich könnte man sämtliche gegenseitig sichtbaren Paare in einer Menge von N Liniensegmenten dadurch bestimmen, daß man alle $N(N - 1)/2$ Paare von Liniensegmenten betrachtet und für jedes Paar feststellt, ob es gegenseitig sichtbar ist oder nicht. Dieses *naive* Verfahren benötigt wenigstens $\Omega(N^2)$ Schritte. Es ist allerdings keineswegs offensichtlich, wie man für ein Paar von Segmenten schnell feststellt, ob es gegenseitig sichtbar ist oder nicht.

Andererseits kann es aber nur höchstens linear viele gegenseitig sichtbare Paare geben. Denn die Relation “ist gegenseitig sichtbar” läßt sich unmittelbar als ein planarer Graph auffassen: Die Knoten des Graphen sind die gegebenen Liniensegmente; zwei Liniensegmente sind durch eine Kante miteinander verbunden genau dann, wenn sie gegenseitig sichtbar sind. Da ein planarer Graph mit N Knoten aber höchstens $3N - 6$ Kanten haben kann, folgt, daß es auch nur höchstens ebensoviele Paare gegenseitig sichtbarer Liniensegmente gibt.

Die Anwendung des Scan-line-Prinzips auf das Sichtbarkeitsproblem liefert folgenden Algorithmus:

Algorithmus Sichtbarkeit

{liefert zu einer Menge $S = \{s_1, \dots, s_N\}$ horizontaler Liniensegmente in der Ebene die Menge aller Paare von gegenseitig sichtbaren Elementen in S }

$Q :=$ Folge der $2N$ Anfangs- und Endpunkte von Elementen in S in aufsteigender x -Reihenfolge;

$L := \emptyset$; *{Menge der jeweils aktiven Liniensegmente in aufsteigender y -Reihenfolge}*

while Q ist nicht leer **do**

begin

$p :=$ nächster (Halte)-Punkt von Q ;

if p ist linker Endpunkt eines Segments s

then

begin

 füge s in L ein;

 bestimme die Nachbarn s' und s'' von s in L und gib (s, s') und (s, s'') als Paare sichtbarer Elemente aus

end

else $\{p$ ist rechter Endpunkt eines Segments $s\}$

begin

 bestimme die Nachbarn s' und s'' von s in L ;

 entferne s aus L ;

 gib (s', s'') als Paar sichtbarer Elemente aus

end

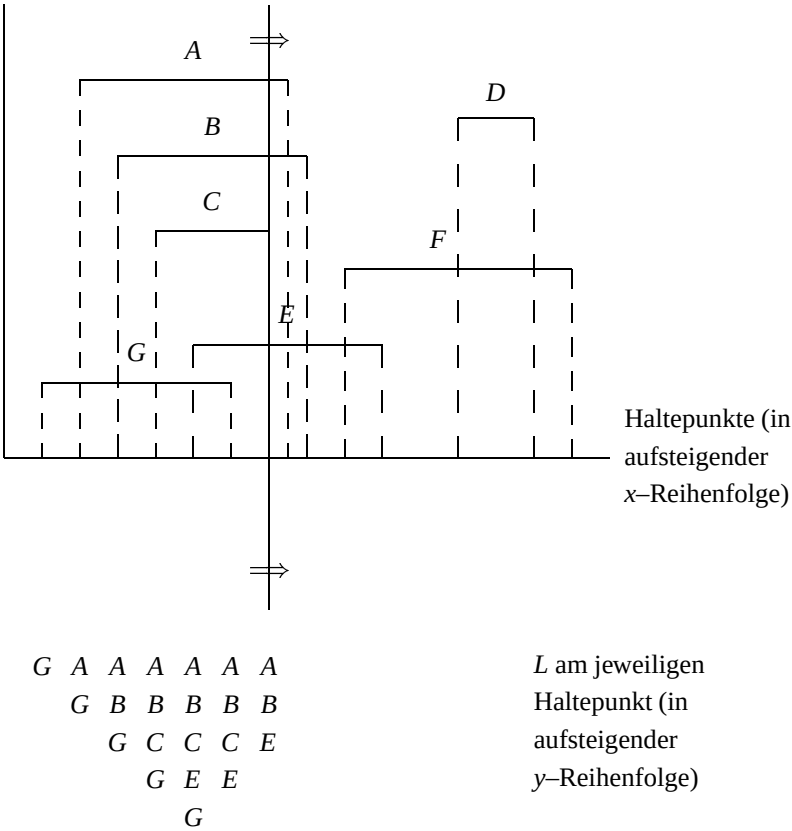
end $\{while\}$

Um die Formulierung des Algorithmus nicht unnötig zu komplizieren, haben wir stillschweigend einige Annahmen gemacht, die keine prinzipielle Bedeutung haben, d. h. insbesondere die asymptotische Effizienz des Verfahrens nicht beeinflussen. Wir nehmen an, daß sämtliche x -Werte von Anfangs- und Endpunkten sämtlicher Liniensegmente paarweise verschieden sind. Die Menge Q der Haltepunkte besteht also aus $2N$ verschiedenen Elementen. Wir setzen ferner voraus, daß die Bestimmung der Nachbarn eines Liniensegments in der nach aufsteigenden y -Werten geordneten Vertikalstruktur die Existenzprüfung einschließt. Wenn also beispielsweise ein Segment s keinen oberen, wohl aber einen unteren Nachbarn s'' hat, wird nach dem Einfügen von s in L nur das Paar (s, s'') ausgegeben. Bei der Implementation des Verfahrens für die Praxis muß man natürlich all diese Sonderfälle betrachten.

Abbildung 7.1 zeigt ein Beispiel für das Verfahren.

Die Menge L kann man als eine geordnete Menge von Punkten oder *Schlüsseln* auffassen, auf der die Operationen *Einfügen* eines Elementes, *Entfernen* eines Elementes und Bestimmen von Nachbarn, d. h. des *Vorgängers* und des *Nachfolgers* eines Elementes ausgeführt werden. Implementiert man L als balancierten Suchbaum, so kann man jede dieser Operationen in $O(\log n)$ Schritten ausführen, wenn n die maximale Anzahl der Elemente in L ist. Natürlich kann diese Anzahl niemals größer sein als die Gesamtzahl N der gegebenen horizontalen Liniensegmente. Für Entwurfsdaten (VLSI-Masken) als gegebener Menge von Liniensegmenten kann man erwarten, daß jeweils

höchstens $O(\sqrt{N})$ Objekte gerade aktiv sind. Dann benötigt man zur Speicherung von L nur $O(\sqrt{N})$ Platz. An jedem Haltepunkt müssen maximal vier der oben angegebenen Operationen ausgeführt werden; jede Operation benötigt höchstens Zeit $O(\log N)$. Insgesamt ergibt sich damit, daß man alle höchstens $3N - 6$ Paare gegenseitig sichtbarer Liniensegmente in einer Menge von N horizontalen Liniensegmenten in Zeit $O(N \log N)$ und Platz $O(N)$ bestimmen kann, wenn man das Scan-line-Prinzip benutzt. Das ist offensichtlich besser als das naive Verfahren.



Ausgabe: (A,G),(A,B), (B,G),(B,C), (C,G),(C,E), (E,G),(B,E)

Abbildung 7.1

Wir haben bei der Analyse des Scan-line-Verfahrens zur Lösung des Sichtbarkeitsproblems für eine Menge von N Liniensegmenten stillschweigend angenommen, daß die Menge der Anfangs- und Endpunkte der Liniensegmente bereits in aufsteigender Reihenfolge etwa als Elemente des Arrays Q vorliegt. Denn wir haben den Aufwand für das Sortieren nicht mitgezählt. Weil der für das Sortieren notwendige Aufwand von

der Größenordnung $\Theta(N \log N)$ ist, hätte die Berücksichtigung dieses Aufwands am Ergebnis natürlich nichts verändert. Allerdings legt die stillschweigende Annahme folgende Frage nahe: Gibt es ein Verfahren zur Bestimmung aller höchstens $3N - 6$ Paare von gegenseitig sichtbaren Liniensegmenten in einer Menge von N Liniensegmenten, das in Zeit $O(N)$ ausführbar ist, wenn man annimmt, daß die Anfangs- und Endpunkte der Liniensegmente bereits aufsteigend sortiert gegeben sind? Mit Ausnahme einiger Spezialfälle ist diese Frage bis heute offen, vgl. [5].

Als nächstes Beispiel für die Anwendung des Scan-line-Prinzips behandeln wir die geometrische Grundaufgabe der Bestimmung aller Paare von sich schneidenden Liniensegmenten in der Ebene. Zunächst behandeln wir den iso-orientierten Fall dieses Problems und dann den allgemeinen Fall.

7.2.2 Das Schnittproblem für iso-orientierte Liniensegmente

Gegeben sei eine Menge von insgesamt N vertikalen und horizontalen Liniensegmenten in der Ebene. Gesucht sind alle Paare von sich schneidenden Segmenten. Dieses Problem nennen wir das rechteckige Segment-Schnitt-Problem, kurz RSS-Problem.

Natürlich können wir das RSS-Problem mit der naiven “brute-force”-Methode in $O(N^2)$ Schritten lösen, indem wir sämtliche Paare von Liniensegmenten daraufhin überprüfen, ob sie einen Schnittpunkt haben. Es ist nicht schwer, Beispiele zu finden, für die es kein wesentlich besseres Verfahren gibt. Man betrachte etwa die Menge von $N/2$ horizontalen und $N/2$ vertikalen Liniensegmenten in Abbildung 7.2.

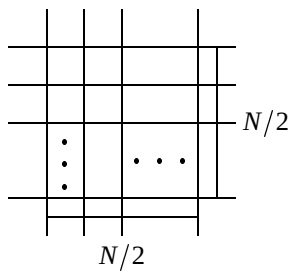


Abbildung 7.2

Hier gibt es $N^2/4$ Paare sich schneidender Segmente. Andererseits gibt es aber auch viele Fälle, in denen die Anzahl der Schnittpunkte klein ist und nicht quadratisch mit der Anzahl der gegebenen Segmente wächst. VLSI-Masken-Daten sind ein wichtiges Beispiel für diesen Fall. Deshalb ist man an Algorithmen interessiert, die in einem solchen Fall besser sind als das naive Verfahren. Wir zeigen jetzt, daß das Scan-line-Prinzip uns ein solches Verfahren liefert.

Zur Vereinfachung der Darstellung des Verfahrens nehmen wir an, daß alle Anfangs- und Endpunkte horizontaler Segmente und alle vertikalen Segmente paarweise verschiedene x -Koordinaten haben. Insbesondere können sich Segmente also nicht überlappen und Schnittpunkte kann es höchstens zwischen horizontalen und vertikalen Segmenten geben. Die Anwendbarkeit des Scan-line-Prinzips ergibt sich nun unmittelbar aus folgender Beobachtung: Merkt man sich beim Schwenken der Scan-line in der Vertikalstruktur L stets die gerade aktiven horizontalen Segmente und trifft man mit der Scan-line auf ein vertikales Segment s , so kann s höchstens Schnittpunkte mit den gerade aktiven horizontalen Segmenten haben. Damit erhalten wir:

Algorithmus zur Lösung des RSS-Problems

{liefert zu einer Menge $S = \{s_1, \dots, s_N\}$ von horizontalen und vertikalen Liniensegmenten in der Ebene die Menge aller Paare von sich schneidenden Segmenten in S }

$Q :=$ Menge der x -Koordinaten der Anfangs- und Endpunkte horizontaler Segmente und von vertikalen Segmenten in aufsteigender x -Reihenfolge;

$L := \emptyset$; *{Menge der jeweils aktiven horizontalen Segmente in aufsteigender y -Reihenfolge}*

while Q nicht leer **do**

begin

$p :=$ nächster (Halte)-Punkt von Q ;

if p ist linker Endpunkt eines horizontalen Segments s

then füge s in L ein

else

if p ist rechter Endpunkt eines horizontalen Segments s

then entferne s aus L

else

{ p ist x -Wert eines vertikalen Segments s mit unterem Endpunkt (p, y_u) und oberem Endpunkt (p, y_o) }

 bestimme alle horizontalen Segmente t aus L ,

 deren y -Koordinate $y(t)$ im Bereich

$y_u \leq y(t) \leq y_o$ liegt und gib (s, t) als Paar

 sich schneidender Segmente aus

end {while}

Abbildung 7.3 zeigt ein Beispiel für die Anwendung des Verfahrens.

Wir können annehmen, daß Q als sortiertes Array der Länge höchstens $2N$ vorliegt. (Gegebenenfalls müssen die x -Werte der gegebenen Segmente zuvor in Zeit $O(N \log N)$ und Platz $O(N)$ sortiert werden.)

Die Menge L kann man auffassen als eine geordnete Menge von Elementen. Sie besteht genau aus den y -Werten der horizontalen Liniensegmente. Auf dieser Menge werden folgende Operationen ausgeführt: Einfügen eines neuen Elementes, Entfernen eines Elementes und Bestimmen aller Elemente, die in einen gegebenen Bereich $[y_u, y_o]$ fallen. Die letzte Operation nennt man eine *Bereichsanfrage* (englisch: *range query*).

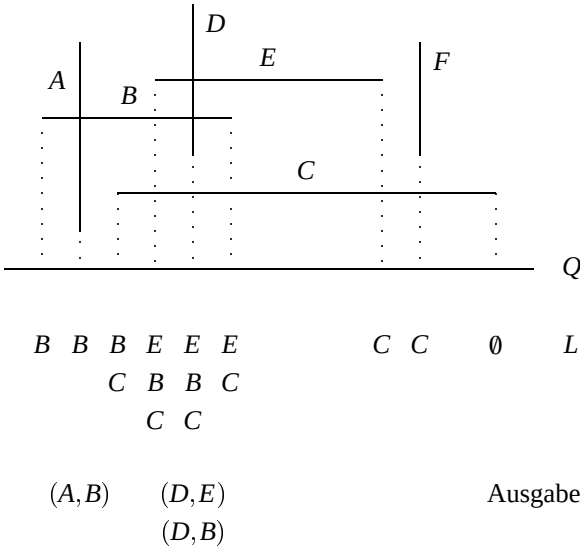


Abbildung 7.3

Eine naheliegende Möglichkeit zur Implementation von L besteht darin, die Elemente in aufsteigender Reihenfolge in den Blättern eines balancierten Blattsuchbaumes zu speichern. Verkettet man benachbarte Blätter zusätzlich doppelt, so kann man die Operationen Einfügen und Entfernen in $O(\log N)$ Schritten ausführen und Bereichsanfragen wie folgt beantworten: Um alle Elemente zu finden, die in einen gegebenen Bereich $[a, b]$ fallen, bestimmt man durch zwei aufeinanderfolgende Suchoperationen im Baum zunächst dasjenige Blatt mit kleinstem Wert größer gleich a und dasjenige Blatt mit größtem Wert kleiner gleich b . Dann läuft man der Kettung entlang und gibt die Elemente aus, die im Bereich $[a, b]$ liegen.

Abbildung 7.4 illustriert das Verfahren und die beschriebene Struktur.
Ist r die Anzahl der Elemente im Bereich $[a, b]$, so kann die Bereichsanfrage offenbar in Zeit $O(\log N + r)$ beantwortet werden.

Diese Implementation des Scan-line-Verfahrens zur Lösung des RSS-Problems erlaubt es also, sämtliche k Paare sich schneidender horizontaler und vertikaler Liniensegmente für eine gegebene Menge von N derartigen Segmenten in Zeit $O(N \log N + k)$ und Platz $O(N)$ zu berichten, wobei natürlich der Platz für die Antwort nicht mitgerechnet wird. Das Verfahren ist damit dem naiven Verfahren überlegen in allen Fällen, in denen k echt schwächer als quadratisch mit N wächst.

Man kann zeigen, vgl. [1], daß auch mindestens $\Omega(N \log N + k)$ Schritte erforderlich sind, um das RSS-Problem zu lösen.

Insgesamt folgt, daß das Scan-line-Verfahren zur Lösung des RSS-Problems zeit- und platzoptimal ist.

Wir überlegen uns nun, wie das Liniensegment-Schnittproblem gelöst werden kann, wenn die gegebene Menge nicht nur aus horizontalen und vertikalen Segmenten besteht.

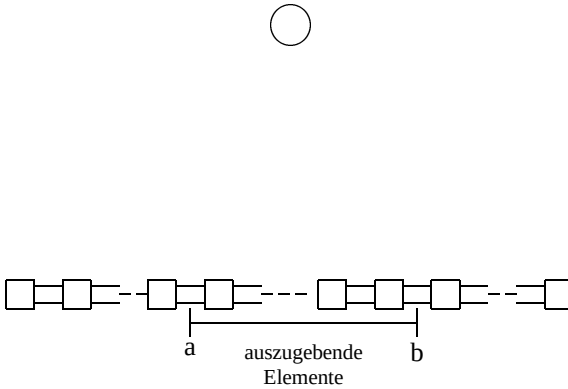


Abbildung 7.4

7.2.3 Das allgemeine Liniensegment-Schnittproblem

Für eine gegebene Menge von beliebig orientierten Liniensegmenten in der Ebene wollen wir die folgenden zwei Probleme lösen:

Schnittpunkttest: Stelle fest, ob es in der gegebenen Menge von N Segmenten wenigstens ein Paar sich schneidender Segmente gibt.

Schnittpunktaufzählung: Bestimme für eine gegebene Menge von N Liniensegmenten alle Paare sich schneidender Segmente.

Beide Probleme lassen sich natürlich auf die naive Weise in $O(N^2)$ Schritten lösen. Wir wollen zeigen, wie man beide Probleme mit Hilfe des Scan-line-Prinzips lösen kann. Um die Diskussion zahlreicher Sonderfälle vermeiden zu können, machen wir die Annahme, daß kein Liniensegment vertikal ist, daß sich in jedem Punkt höchstens zwei Liniensegmente schneiden und schließlich, daß alle Anfangs- und Endpunkte von Liniensegmenten paarweise verschiedene x -Koordinaten haben.

Anders als für eine Menge horizontaler Liniensegmente kann man für eine Menge beliebig orientierter Liniensegmente nur eine lokal gültige Ordnungsrelation “ist oberhalb von” wie folgt definieren. Seien A und B zwei Liniensegmente. Dann heißt A *x-oberhalb von* B , $A \uparrow_x B$, wenn die vertikale Gerade durch x sowohl A als auch B schneidet und der Schnittpunkt von x und A oberhalb des Schnittpunktes von x und B liegt. Im Beispiel von Abbildung 7.5 ist $C \uparrow_x B$, $A \uparrow_x C$ und $A \uparrow_x B$. Für jedes feste x ist $\uparrow_x$ offenbar eine Ordnungsrelation.

Zur Lösung des Schnittpunkttestproblems schwenken wir nun eine vertikale Scan-line von links nach rechts über die N gegebenen Liniensegmente. An jeder Stelle x sind die Liniensegmente, die von der Scan-line geschnitten werden, durch $\uparrow_x$ vollständig geordnet. Änderungen der Ordnung sind möglich, wenn die Scan-line auf den linken oder rechten Endpunkt eines Segments trifft, und ferner, wenn die Scan-line einen Schnittpunkt passiert.

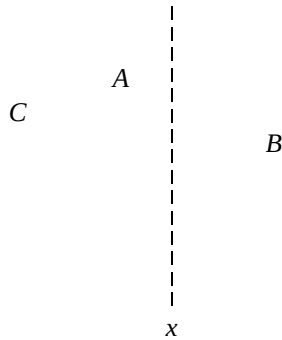


Abbildung 7.5

Für zwei beliebige Segmente A und B gilt: Wenn A und B sich schneiden, dann gibt es eine Stelle x links vom Schnittpunkt, so daß A und B in der Ordnung $\uparrow_x$ unmittelbar aufeinanderfolgen. (Hier machen wir von der Annahme Gebrauch, daß sich höchstens zwei Segmente in einem Punkt schneiden können!) Wenn wir also für je zwei Segmente A und B prüfen, ob sie sich schneiden, sobald sie an einer Stelle x bzgl. $\uparrow_x$ unmittelbar benachbart sind, können wir sicher sein, keinen Schnittpunkt zu übersehen, wenn es überhaupt einen gibt.

Diese Idee führt zu folgendem Algorithmus zur Lösung des Schnittpunkttestproblems:

Algorithmus zur Lösung des Schnittpunkttestproblems

{liefert zu einer Menge $S = \{s_1, \dots, s_N\}$ von Liniensegmenten in der Ebene "ja", falls es ein Paar sich schneidender Segmente in S gibt, und "nein" sonst}

$Q :=$ Folge der $2N$ Anfangs- und Endpunkte von Elementen in S in aufsteigender x -Reihenfolge;

$L := \emptyset$; {Menge der jeweils aktiven Liniensegmente in $\uparrow_x$ -Ordnung}
 gefunden := false;

while (Q ist nicht leer) **and not** gefunden **do**

begin

$p :=$ nächster Haltepunkt von Q ; { p habe x -Koordinate $p.x$ }

if p ist linker Endpunkt eines Segments s

then

begin

 füge s entsprechend der an der Stelle p

 gültigen Ordnung $\uparrow_{p.x}$ in L ein;

 bestimme den Nachfolger s' und den Vorgänger s''

 von s in L bzgl. $\uparrow_{p.x}$;

if $(s \cap s' \neq \emptyset)$ **or** $(s \cap s'' \neq \emptyset)$

then gefunden := true

end

```

else {p ist rechter Endpunkt eines Segments s}
  begin
    bestimme den Nachfolger s' und den Vorgänger s''
      von s bzgl. der an der Stelle p gültigen Ordnung  $\uparrow_{p,x}$ ;
    entferne s aus L;
    if  $s' \cap s'' \neq \emptyset$ 
      then gefunden := true
    end
  end; {while}
if gefunden
  then write('ja')
  else write('nein')

```

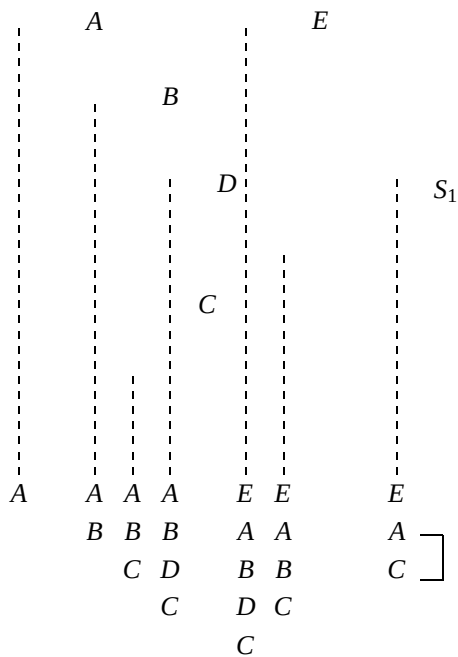
Wir haben hier wieder stillschweigend angenommen, daß die Bestimmung des Nachfolgers oder Vorgängers eines Elements die Existenzprüfung einschließt. Es ist leicht zu sehen, daß *L* an jeder Halteposition *x* der Scan-line die gerade aktiven Liniensegmente in korrekter $\uparrow_x$ -Anordnung enthält. Das Verfahren muß also einen Schnittpunkt finden, falls es überhaupt einen gibt. Das muß nicht notwendig der am weitesten links liegende Schnittpunkt zweier Segmente in *S* sein.

Wir verfolgen zwei Beispiele anhand der Abbildung 7.6.

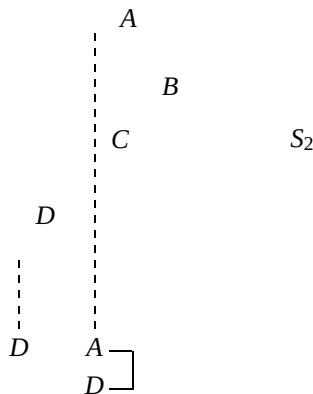
Im Fall (a) hält das Verfahren mit der Antwort "ja", sobald der Schnittpunkt *S*₁ von *A* und *C* gefunden wurde; im Fall (b) findet das Verfahren den Schnittpunkt *S*₂ von *A* und *D* bereits am zweiten Haltepunkt.

Die $2N$ Endpunkte der gegebenen Menge von Liniensegmenten können in $O(N \log N)$ Schritten nach aufsteigenden *x*-Werten sortiert werden. *L* kann man als balancierten Suchbaum implementieren. Dann kann jede der an einem Haltepunkt auszuführenden Operationen Einfügen, Entfernen, Bestimmen des Vorgängers und Nachfolgers eines Elementes in $O(\log N)$ Schritten ausgeführt werden. Damit folgt insgesamt, daß man mit Hilfe des Scan-line-Verfahrens in Zeit $O(N \log N)$ und Platz $O(N)$ feststellen kann, ob *N* Liniensegmente in der Ebene wenigstens einen Schnittpunkt haben oder nicht.

Was ist zu tun, um nicht nur festzustellen, ob in der gegebenen Menge von Liniensegmenten wenigstens ein Paar sich schneidender Segmente vorkommt, sondern um alle Paare sich schneidender Segmente aufzuzählen? Dann dürfen wir den oben angegebenen Algorithmus zur Lösung des Segmentschnittproblems nicht beenden, sobald der erste Schnittpunkt gefunden wurde. Vielmehr setzen wir das Verfahren fort und sorgen dafür, daß die lokale Ordnung der jeweils gerade aktiven Segmente repräsentierende Vertikalstruktur *L* auch dann korrekt bleibt, wenn die Scan-line einen Schnittpunkt passiert: Immer wenn die Scan-line den Schnittpunkt *s* zweier Segmente *A* und *B* passiert, wechseln *A* und *B* ihren Platz in der unmittelbar links und rechts vom Schnittpunkt gültigen lokalen "oberhalb-von"-Ordnung. Wir müssen also die Scan-line nicht nur an allen Anfangs- und Endpunkten von Liniensegmenten anhalten, sondern auch an allen während des Hinüberschwenkens gefundenen Schnittpunkten. Ein Schnittpunkt liegt stets rechts von der Position der Scan-line, an der er entdeckt wurde. Wir fügen also einfach jeden gefundenen Schnittpunkt in die nach aufsteigenden *x*-Werten geordnete Schlange der Haltepunkte ein, wenn er sich nicht schon dort befindet.



(a)



(b)

Abbildung 7.6

Algorithmus zur Lösung des Schnittpunktaufzählungsproblems
 {liefert zu einer Menge $S = \{s_1, \dots, s_N\}$ von Liniensegmenten in der Ebene alle Paare (s_i, s_j) mit: $s_i, s_j \in S$, $s_i \cap s_j \neq \emptyset$ und $i \neq j$ }

$Q :=$ nach aufsteigenden x -Werten angeordnete Prioritäts-Schlange der Haltepunkte; anfangs initialisiert als Folge der $2N$ Anfangs- und Endpunkte von Elementen in S in aufsteigender x -Reihenfolge;

$L := \emptyset$; {Menge der jeweils aktiven Segmente in $\uparrow_x$ -Ordnung}

while Q ist nicht leer **do**

begin

$p := \min(Q)$;

$\text{minentferne}(Q)$;

if p ist linker Endpunkt eines Segments s

then

begin

$\text{Einfügen}(s, L)$;

$s' := \text{Nachfolger}(s, L)$;

$s'' := \text{Vorgänger}(s, L)$;

if $s \cap s' \neq \emptyset$

then $\text{Einfügen}(s \cap s', Q)$;

if $s \cap s'' \neq \emptyset$

then $\text{Einfügen}(s \cap s'', Q)$

end

else

if p ist rechter Endpunkt eines Segments s

then

begin

$s' := \text{Nachfolger}(s, L)$;

$s'' := \text{Vorgänger}(s, L)$;

if $s' \cap s'' \neq \emptyset$

then $\text{Einfügen}(s' \cap s'', Q)$;

$\text{Entfernen}(s, L)$

end

else $\{p$ ist Schnittpunkt von s' und s'' , d.h.

$p = s' \cap s''$, und es sei s' oberhalb von s'' in $L\}$

begin

 gib das Paar (s', s'') mit Schnittpunkt p aus;

 vertausche s' und s'' in L ;

 {jetzt ist s'' oberhalb von s' }

$t'' := \text{Vorgänger}(s'', L)$;

if $s'' \cap t'' \neq \emptyset$

then $\text{Einfügen}(s'' \cap t'', Q)$;

$t' := \text{Nachfolger}(s', L)$;

if $s' \cap t' \neq \emptyset$

then $\text{Einfügen}(s' \cap t', Q)$

end

end {while}

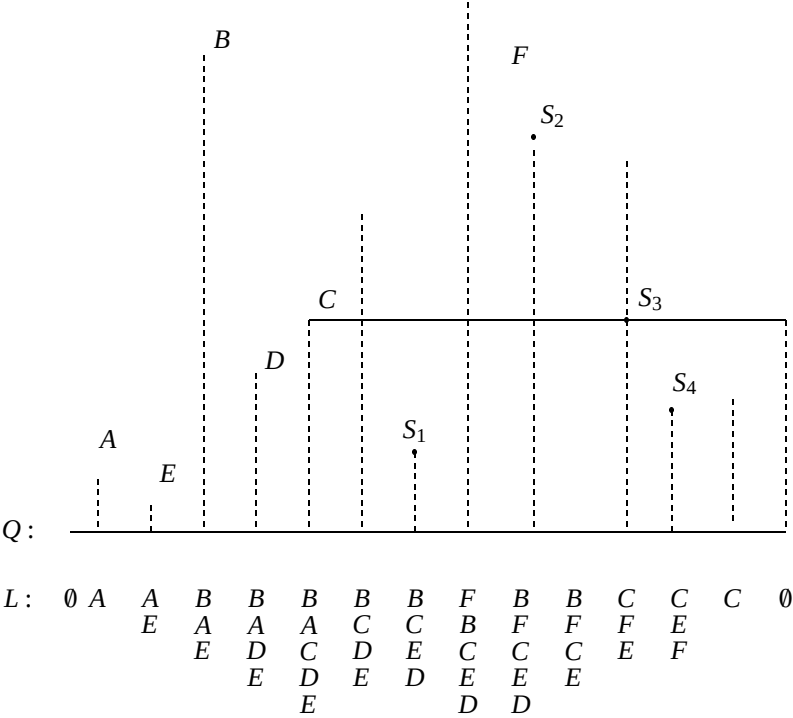


Abbildung 7.7

Um die Formulierung des Verfahrens nicht unnötig zu komplizieren, haben wir nicht nur angenommen, daß keine zwei Anfangs- und Endpunkte von Segmenten dieselbe x-Koordinate haben, sondern auch vorausgesetzt, daß kein Schnittpunkt dieselbe x-Koordinate wie ein Anfangs- oder Endpunkt eines Liniensegmentes hat. Unter dieser Annahme tritt an jeder Halteposition der Scan-line genau eines von drei möglichen Ereignissen ein: Ein Liniensegment beginnt, ein Liniensegment endet, oder es liegt ein Schnittpunkt zweier Liniensegmente vor. In der Realität ist diese Annahme natürlich selten erfüllt und auch nicht notwendig. Man kann beispielsweise vorschreiben, daß bei gleichzeitigem Vorliegen mehrerer Ereignisse am gleichen Haltepunkt $p \in Q$ die verschiedenen Ereignisse wie oben angegeben entsprechend ihren jeweiligen y-Koordinaten abgearbeitet werden.

Abbildung 7.7 zeigt ein Beispiel für das soeben beschriebene Verfahren. Beim beschriebenen Verfahren kann es vorkommen, daß ein- und derselbe Schnittpunkt mehrere Male gefunden wird (vgl. etwa Abbildung 7.6 (b)), bei der S_2 zweimal gefunden wird). Damit jeder Schnittpunkt aber nur einmal in Q vermerkt wird, lassen wir dem Einfügen eines Schnittpunkts S in Q eine Suche nach S in Q vorangehen; S wird dann nur bei erfolgloser Suche eingefügt. Neben der Suche nach einem beliebigen Element muß Q also das Einfügen eines beliebigen Elements, die Bestimmung eines

Elements mit kleinstem x -Wert und das Entfernen eines Elements mit kleinstem x -Wert unterstützen. Organisieren wir Q etwa als balancierten Binärbaum, z.B. als AVL-Baum, so kann die notwendige Initialisierung in $O(N \log N)$ Schritten durchgeführt werden. Die Größe des Baums ist stets beschränkt durch die Gesamtzahl der Anfangs-, End- und Schnittpunkte von Liniensegmenten. Das sind höchstens $O(N + N^2) = O(N^2)$. Daher kann man die erforderlichen Operationen stets in $O(\log(N^2)) = O(\log N)$ Schritten ausführen.

Auf der Vertikalstruktur L werden die Operationen Einfügen und Entfernen eines Elementes, Bestimmen des Vorgängers und Nachfolgers eines Elementes und das Vertauschen zweier Elemente ausgeführt. Ohne daß dies im Algorithmus explizit angegeben wird, sind alle diese Operationen abhängig von der am jeweiligen Punkt $p \in Q$ gültigen Ordnung $\uparrow_{p,x}$. Es ist klar, daß L als nach dieser Ordnung sortierter balancierter Suchbaum so implementiert werden kann, daß jede der genannten Operationen in $O(\log N)$ Schritten ausführbar ist, weil L höchstens N Elemente enthält. Nehmen wir nun an, daß es k Schnittpunkte gibt. Dann wird die **while**-Schleife genau $2N + k$ mal durchlaufen. Wir haben bereits gesehen, daß jede Operation auf Q innerhalb der **while**-Schleife in $O(\log(2N + k)) = O(\log N)$ und jede Operation auf L in $O(\log N)$ Schritten ausführbar ist. Bei $2N + k$ Durchläufen werden also insgesamt höchstens $O((N + k) \log N)$ Schritte benötigt.

Man kann also mit Hilfe des Scan-line-Verfahrens alle k Schnittpunkte von N gegebenen Liniensegmenten in der Ebene in $O((N + k) \log N)$ Schritten finden. Das ist besser als das naive Verfahren für nicht zu große k . Chazelle  hat zeigen können, daß man mit geschickter Anwendung der im nächsten Abschnitt 7.3 vorgestellten Divide-and-conquer-Technik zu Algorithmen kommt, die das Schnittpunktaufzählungsproblem in $O(N \log^2 N + k)$ bzw. $O(N \log^2 N / \log \log N + k)$ Schritten lösen. Schließlich konnten Chazelle und Edelsbrunner  zeigen, daß alle k Schnitte wie im iso-orientierten Fall in $O(N \log N + k)$ Schritten gelöst werden können.

Die von uns skizzierte Implementierung des Scan-line-Verfahrens zur Bestimmung aller k Schnittpunkte einer gegebenen Menge von N Liniensegmenten hat allerdings einen Speicherbedarf von $\Omega(N^2)$ im schlechtesten Fall. Denn Q kann bis zu $2N + k = \Omega(N^2)$ Elemente enthalten. Der Speicherbedarf für Q und damit der Gesamtspeicherbedarf läßt sich jedoch auf $O(N)$ drücken, wenn man wie folgt vorgeht: Man fügt nicht jeden an einer Halteposition $p \in Q$ gefundenen Schnittpunkt in Q ein. Vielmehr sichert man lediglich, daß Q auf jeden Fall den von der jeweils aktuellen Position p der Scan-line aus nächsten Schnittpunkt enthält. Dazu nimmt man für jedes aktive Liniensegment s höchstens einen Schnittpunkt in Q auf, nämlich unter allen Schnittpunkten, an denen s beteiligt ist und die man bis zu einer bestimmten Position entdeckt hat, den jeweils am weitesten links liegenden. Mit anderen Worten: Findet man im Verlauf des Verfahrens für ein Segment s einen weiteren Schnittpunkt, an dem s beteiligt ist, und liegt dieser links vom vorher gefundenen Schnittpunkt, so entfernt man den früher gefundenen Schnittpunkt und fügt den neuen in Q ein. Es ist nicht schwer zu sehen, daß man Q so implementieren kann, daß Q nur $O(N)$ Speicherplatz benötigt und alle auf Q auszuführenden Operationen in Zeit $O(\log N)$ ausführbar sind. (Ein balancierter Suchbaum leistet auch hier das Verlangte.) Um für jedes aktive Segment s leicht feststellen zu können, ob schon ein Schnittpunkt in Q ist, an dem s beteiligt ist, und welchen x -Wert dieser Schnittpunkt hat, kann man beispielsweise einen Zeiger von s auf diesen Schnittpunkt

punkt in Q verwenden. Diese Idee zur Reduktion des Speicherbedarfs geht zurück auf M. Brown



7.3 Geometrisches Divide-and-conquer

Eines der leistungsfähigsten Prinzipien zur algorithmischen Lösung von Problemen ist das Divide-and-conquer-Prinzip. Wir haben bereits im Abschnitt 1.2.2 eine problemunabhängige Formulierung dieses Prinzips angegeben. Wir folgen hier der Darstellung aus



Wenn wir versuchen, dieses Prinzip auf ein geometrisches Problem, wie das im vorigen Abschnitt behandelte Schnittproblem für iso-orientierte Liniensegmente, anzuwenden, stellt sich sofort die Frage: Wie soll man teilen? Eine Aufteilung ohne jede Beachtung der geometrischen Nachbarschaftsverhältnisse scheint wenig sinnvoll. Denn man möchte ja gerade besonderen Nutzen daraus ziehen, daß Schnitte im wesentlichen lokal, also zwischen räumlich nahen Segmenten auftreten. Versucht man aber eine Aufteilung etwa durch eine vertikale Gerade in eine linke und rechte Hälfte, so kann man im allgemeinen nicht verhindern, daß ausgedehnte geometrische Objekte, wie Liniensegmente, Rechtecke, Polygone usw., durchschnitten werden. Einen Ausweg aus dieser Schwierigkeit bietet das Prinzip der getrennten Repräsentation geometrischer Objekte. Wir erläutern dieses Prinzip im Abschnitt 7.3.1 für eine Menge horizontaler Liniensegmente bei Aufteilung durch eine vertikale Gerade und lösen das Schnittproblem für iso-orientierte Liniensegmente nach dem Divide-and-conquer-Prinzip. Im Abschnitt 7.3.2 zeigen wir, wie man Inklusions- und Schnittprobleme für Mengen iso-orientierter Rechtecke in der Ebene nach diesem Prinzip löst.

7.3.1 Segmentschnitt mittels Divide-and-conquer

Um eine gegebene Menge von N vertikalen und horizontalen Liniensegmenten in der Ebene leicht und eindeutig durch eine vertikale Gerade in eine linke und rechte Hälfte teilen zu können, benutzen wir eine getrennte Repräsentation horizontaler Segmente: Jedes horizontale Segment wird durch das Paar seiner Endpunkte repräsentiert. Anstatt mit einer Menge von vertikalen und horizontalen Segmenten operieren wir mit einer Menge von vertikalen Segmenten und Punkten. Beispielsweise repräsentieren wir die Menge von sieben Segmenten in Abbildung 7.8 durch die Menge von vier Segmenten und sechs Punkten in Abbildung 7.9.

Dabei bezeichnen wir für ein horizontales Segment h den linken Endpunkt von h mit $.h$ und den rechten Endpunkt von h mit $h..$. Wenn wir zur Vereinfachung der Präsentation die Annahme machen, daß keine zwei vertikalen Segmente und Anfangs- oder Endpunkte horizontaler Segmente dieselbe x -Koordinate haben, kann man das Divide-and-conquer-Verfahren zur Lösung des Schnittproblems für eine (getrennt repräsentierte) Menge von iso-orientierten Liniensegmenten in der Ebene wie folgt formulieren:

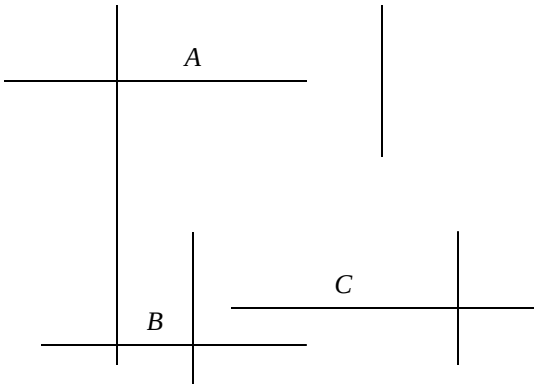


Abbildung 7.8

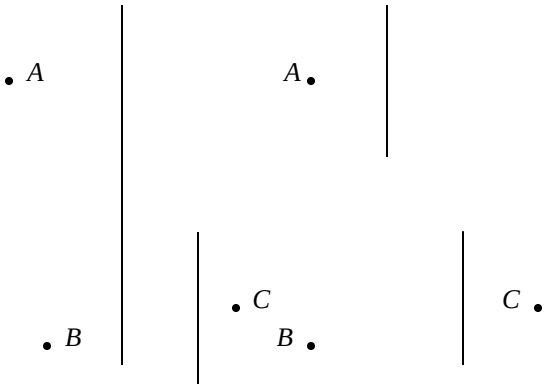
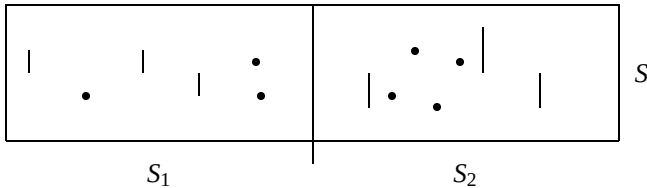


Abbildung 7.9

Algorithmus ReportCuts(S)

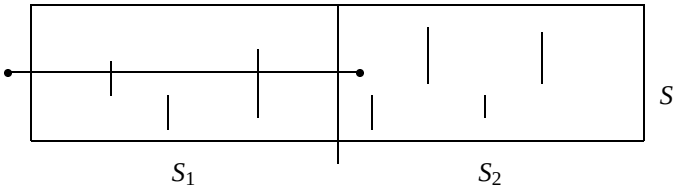
{liefert zu einer Menge S von vertikalen Liniensegmenten und linken und rechten Endpunkten horizontaler Liniensegmente in der Ebene in getrennter Repräsentation alle Paare von sich schneidenden vertikalen Segmenten in S und horizontalen Segmenten mit linkem oder rechtem Endpunkt in S }

1. Divide: Teile S (durch eine vertikale Gerade) in eine linke Hälfte S_1 und eine rechte Hälfte S_2 , falls S mehr als ein Element enthält; sonst enthält S kein sich schneidendes Paar:

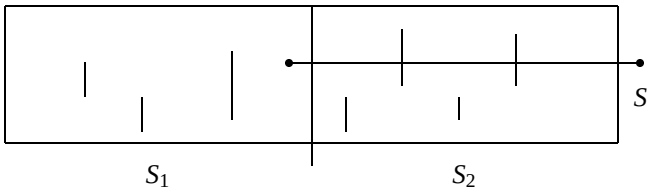


2. Conquer: ReportCuts(S_1); ReportCuts(S_2);
{alle Schnitte in S_1 oder S_2 zwischen Paaren von Segmenten, die wenigstens einmal repräsentiert sind, sind bereits berichtet}

3. Merge: Berichte alle Schnitte zwischen vertikalen Segmenten in S_1 und horizontalen Segmenten mit rechtem Endpunkt in S_2 , deren linker Endpunkt nicht in S_1 oder S_2 vorkommt:



Berichte alle Schnitte zwischen vertikalen Segmenten in S_2 und horizontalen Segmenten mit linkem Endpunkt in S_1 , deren rechter Endpunkt nicht in S_1 oder S_2 vorkommt:



Ende des Algorithmus ReportCuts

Ein Aufruf des Verfahrens *ReportCuts*(S) für eine gegebene Menge S bewirkt, daß das Verfahren wiederholt für immer kleinere, durch fortgesetzte Aufteilung entstehende Mengen aufgerufen wird, bis es schließlich für Mengen mit nur einem Element abbricht. Für die durch fortgesetzte Aufteilung entstehenden Mengen ist es möglich, daß nur der linke, nicht aber der rechte Endpunkt eines horizontalen Segments oder nur der rechte, nicht aber der linke Endpunkt auftritt. Das macht es erforderlich, sogleich das ganze Verfahren für Mengen dieser Art zu formulieren, so wie es oben geschehen ist. Wir zeigen nun die *Korrektheit* des Verfahrens und benutzen dazu die bereits als Kommentar zum Verfahren angegebene *Rekursionsinvariante*.

Ist S eine Menge von vertikalen Segmenten und linken oder rechten Endpunkten von horizontalen Segmenten, so sind nach Beendigung eines Aufrufs von *ReportCuts*(S) alle Schnitte zwischen vertikalen Segmenten in S und solchen horizontalen Segmenten berichtet, deren linker oder rechter Endpunkt (eventuell auch beide) in S vorkommt. Offenbar gilt diese Bedingung trivialerweise, wenn S nur aus einem einzigen Element besteht. In diesem Fall bricht das Verfahren *ReportCuts* ab; Schnitte werden nicht berichtet.

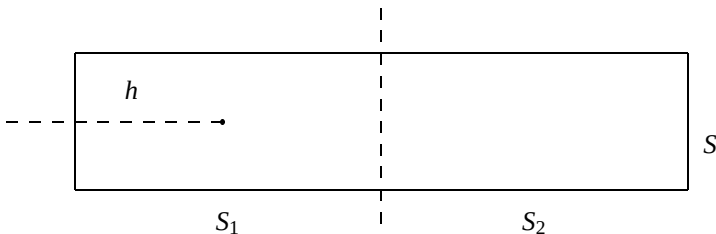
Wir zeigen jetzt: Wird S beim Aufruf von *ReportCuts*(S) aufgeteilt in eine linke Hälfte S_1 und eine rechte S_2 und gilt die Rekursionsinvariante bereits für S_1 und S_2 , so gilt sie auch für S .

Dazu betrachten wir ein beliebiges horizontales Segment h , dessen linker oder rechter Endpunkt in S vorkommt. Wir müssen zeigen, daß nach Beendigung des Aufrufs *ReportCuts*(S) alle Schnitte von h mit vertikalen Segmenten in S berichtet worden sind. Folgende Fälle sind möglich:

Fall 1: Beide Endpunkte von h liegen in S_1 . Da die Rekursionsinvariante nach Annahme für S_1 gilt, folgt, daß nach Beendigung des Aufrufs *ReportCuts*(S_1) im Conquer-Schritt alle Schnitte von h mit vertikalen Elementen in S_1 berichtet sind. h kann keine weiteren Schnitte mit vertikalen Segmenten in S haben.

Im *Fall 2*, daß beide Endpunkte von h in S_2 liegen, gilt das Analoge für Schnitte zwischen h und vertikalen Segmenten in S_2 .

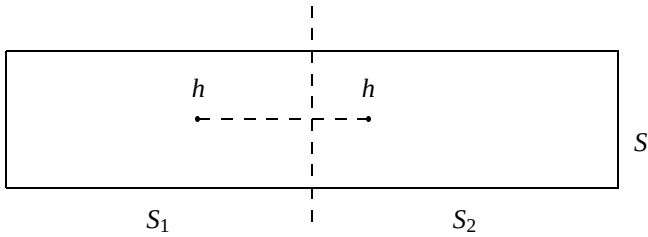
Fall 3: Nur der rechte Endpunkt von h ist in S_1 .



Von den vertikalen Segmenten in S kann h nur solche schneiden, die in S_1 vorkommen. Diese sind aber nach dem Aufruf von *ReportCuts*(S_1) bereits berichtet, da nach Annahme die Rekursionsinvariante für S_1 gilt.

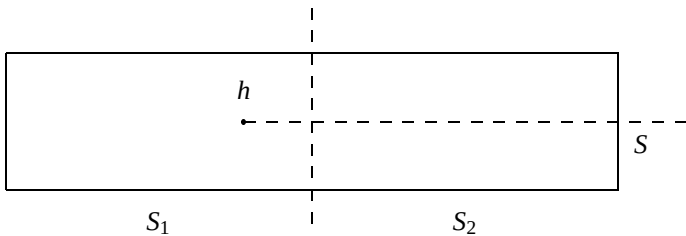
Im *Fall 4*, daß nur der linke Endpunkt von h in S_2 liegt, gilt das Analoge nach Beendigung des Aufrufs *ReportCuts*(S_2).

Fall 5: Der linke Endpunkt von h liegt in S_1 und der rechte Endpunkt von h in S_2 :



Da die Rekursionsinvariante für S_1 und S_2 gilt, folgt, daß nach Beendigung des Aufrufs $ReportCuts(S_1)$ und $ReportCuts(S_2)$ alle möglichen Schnitte von h mit vertikalen Segmenten in S bereits berichtet sind.

Fall 6: Der linke Endpunkt von h liegt in S_1 , aber der rechte Endpunkt von h liegt weder in S_1 noch in S_2 :



h kann Schnitte mit vertikalen Segmenten in S_1 und S_2 haben. Die Gültigkeit der Rekursionsinvariante für S_1 sichert, daß nach Beendigung des Aufrufs $ReportCuts(S_1)$ alle Schnitte von h mit vertikalen Segmenten in S_1 bereits berichtet sind. Es genügt also, im Merge-Schritt alle Schnitte zwischen h und vertikalen Segmenten in S_2 zu bestimmen, um alle Schnitte von h mit vertikalen Segmenten in S zu berichten.

Der *Fall 7*, daß der rechte Endpunkt von h in S_2 , aber der linke Endpunkt von h weder in S_1 noch in S_2 liegt, ist völlig symmetrisch zum Fall 6. Auch hier haben wir den Merge-Schritt gerade so eingerichtet, daß alle möglichen Schnitte zwischen h und vertikalen Segmenten in S berichtet werden.

Insgesamt ist die Gültigkeit der Rekursionsinvariante für S damit nachgewiesen. Für eine möglichst effiziente Implementation des Verfahrens kommt es darauf an, die Schnitte im Merge-Schritt schnell und möglichst mit einem zur Anzahl dieser Schnitte proportionalen Aufwand zu bestimmen. Dazu dienen drei Mengen $L(S)$, $R(S)$ und $V(S)$, die wir jeder Menge S zuordnen:

$$L(S) = \{y(h) \mid h \text{ ist horizontales Liniensegment mit:} \\ \quad .h \in S \text{ aber } h. \notin S\}$$

$$R(S) = \{y(h) \mid h \text{ ist horizontales Liniensegment mit:} \\ \quad .h \notin S \text{ aber } h. \in S\}$$

$$V(S) = \text{Menge der durch die vertikalen Segmente in } S \\ \text{definierten } y\text{-Intervalle} \\ = \{[y_u(v), y_o(v)] \mid v \text{ ist vertikales Liniensegment in } S\}$$

In diesen Definitionen haben wir mit $y(h)$ die y -Koordinate eines horizontalen Segmentes h bezeichnet und mit $y_u(v)$ bzw. $y_o(v)$ die untere bzw. obere y -Koordinate eines vertikalen Segmentes v .

Nehmen wir an, daß wir vor Beginn des Merge-Schrittes die Mengen

$$L(S_i), \quad R(S_i), \quad V(S_i), \quad i = 1, 2$$

bereits kennen. Dann kann man den Merge-Schritt auch so formulieren:
Bestimme alle Paare (h, v) mit (a) oder (b):

$$(a) \quad \begin{aligned} y(h) &\in R(S_2) \setminus L(S_1), \\ [y_u(v), y_o(v)] &\in V(S_1), \\ y_u(v) &\leq y(h) \leq y_o(v) \end{aligned}$$

$$(b) \quad \begin{aligned} y(h) &\in L(S_1) \setminus R(S_2), \\ [y_u(v), y_o(v)] &\in V(S_2), \\ y_u(v) &\leq y(h) \leq y_o(v) \end{aligned}$$

Aus $L(S_i)$, $R(S_i)$, $V(S_i)$, $i = 1, 2$, erhält man die $S = S_1 \cup S_2$ zugeordneten Mengen offenbar wie folgt:

$$\begin{aligned} L(S) &:= (L(S_1) \setminus R(S_2)) \cup L(S_2) \\ R(S) &:= (R(S_2) \setminus L(S_1)) \cup R(S_1) \\ V(S) &:= V(S_1) \cup V(S_2) \end{aligned}$$

Falls S nur aus einem einzigen Element besteht, können wir diese Mengen leicht wie folgt initialisieren:

Fall 1: $S = \{h\}$, d.h. S enthält nur den linken Endpunkt eines horizontalen Segmentes h .

$$L(S) := \{y(h)\}; \quad R(S) := \emptyset; \quad V(S) := \emptyset$$

Fall 2: $S = \{h\}$, d.h. S enthält nur den rechten Endpunkt eines horizontalen Segmentes h .

$$L(S) := \emptyset; \quad R(S) := \{y(h)\}; \quad V(S) := \emptyset$$

Fall 3: $S = \{v\}$, d.h. S enthält nur das vertikale Segment v .

$$L(S) := \emptyset; \quad R(S) := \emptyset; \quad V(S) := \{[y_u(v), y_o(v)]\}$$

Zur Implementation des Verfahrens speichern wir nun die gegebene Menge S von vertikalen Segmenten und linken und rechten Endpunkten horizontaler Segmente in einem nach aufsteigenden x -Werten sortierten Array. Dann kann das Teilen im Divide-Schritt in konstanter Zeit ausgeführt werden. Die einer Menge S zugeordneten Mengen $L(S)$ und $R(S)$ implementieren wir als nach aufsteigenden y -Werten sortierte, verkettete lineare Listen. $V(S)$ wird ebenfalls als nach unteren Endpunkten, also nach y_u -Werten sortierte, verkettete lineare Liste implementiert.

$L(S)$, $R(S)$ und $V(S)$ können dann aus den $L(S_i)$, $R(S_i)$, $V(S_i)$, $i = 1, 2$, zugeordneten Listen in $O(|S|)$ Schritten gebildet werden, indem man die bereits vorhandenen Listen ähnlich wie beim Sortieren durch Verschmelzen parallel durchläuft. Schließlich kann man im Merge-Schritt alle r Paare (h, v) , die die oben angegebenen Bedingungen (a) oder (b) erfüllen, mit Hilfe dieser Listen bestimmen in $O(|S| + r)$ Schritten.

Bezeichnen wir mit $T(N)$ die Anzahl der Schritte, die erforderlich ist, um das Verfahren *ReportCuts*(S) bei dieser Implementation für eine Menge S mit N Elementen auszuführen, wenn wir den Aufwand für das Sortieren von S und die Ausgabe nicht mitrechnen, so gilt folgende Rekursionsformel:

$$T(N) = \underbrace{O(1)}_{\text{Divide}} + \underbrace{2T\left(\frac{N}{2}\right)}_{\text{Conquer}} + \underbrace{O(N)}_{\text{Merge}}$$

und $T(1) = O(1)$.

Es ist wohlbekannt, daß diese Rekursionsformel die Lösung $O(N \log N)$ hat. Rechnen wir noch den Aufwand zur Ausgabe der insgesamt k Paare sich schneidender Segmente hinzu, so erhalten wir (inklusive Sortieraufwand):

Alle k Paare sich schneidender horizontaler und vertikaler Liniensegmente in einer gegebenen Menge von N derartigen Segmenten kann man mit Hilfe eines Divide-and-conquer-Verfahrens in Zeit $O(N \log N + k)$ und Platz $O(N)$ bestimmen.

Das ist dieselbe Zeit- und Platz-Komplexität, die auch das im vorigen Abschnitt besprochene Scan-line-Verfahren zur Lösung dieses Schnittproblems hat. Vergleicht man die Implementationen beider Verfahren, so fällt auf, daß das Divide-and-conquer-Verfahren mit einfachen Datenstrukturen auskommt: Verkettete, aufsteigend sortierte lineare Listen genügen. Im Falle des Scan-line-Verfahrens haben wir zu Bereichs-Suchbäumen modifizierte, balancierte Suchbäume benutzt.

7.3.2 Inklusions- und Schnittprobleme für Rechtecke

Das Divide-and-conquer-Prinzip läßt sich zur Lösung zahlreicher weiterer geometrischer Probleme benutzen, wenn man es zugleich mit dem Prinzip der getrennten Repräsentation der gegebenen geometrischen Objekte verbindet. Wir skizzieren kurz, wie man das Punkteinschluß- und das Rechteckschnittproblem in der Ebene auf diese Weise lösen kann.

Das Punkteinschluß-Problem für eine gegebene Menge von Rechtecken und Punkten in der Ebene ist das Problem, alle Paare (Punkt, Rechteck) zu bestimmen, für die das Rechteck den Punkt einschließt. Für das in Abbildung 7.10 angegebene Beispiel ist also die Antwort (p, A) , (q, A) , (r, A) , (q, B) , (r, B) , (s, B) , (s, C) .

Um eine gegebene Menge von Punkten und Rechtecken in der Ebene eindeutig in eine linke und eine rechte Hälfte zerlegen zu können, wählen wir zunächst eine getrennte Repräsentation für die Rechtecke: Jedes Rechteck wird durch seinen linken und seinen rechten Rand repräsentiert. Eine Menge von Rechtecken und Punkten wird also repräsentiert durch eine Menge von vertikalen Liniensegmenten und Punkten. Nun kann man einen Algorithmus *ReportInc* analog zum Algorithmus *ReportCuts* wie folgt entwerfen:

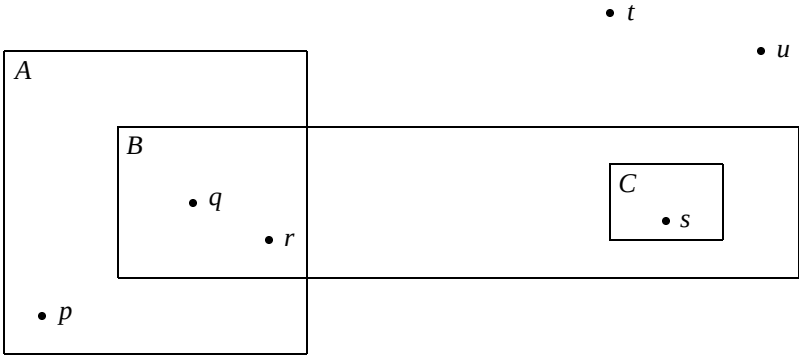
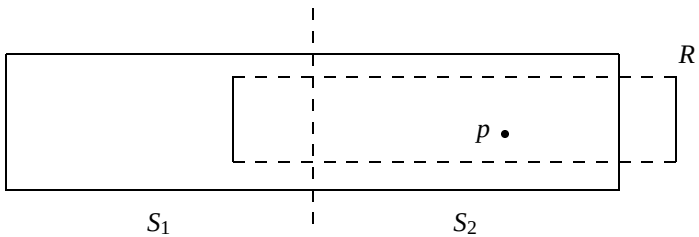


Abbildung 7.10

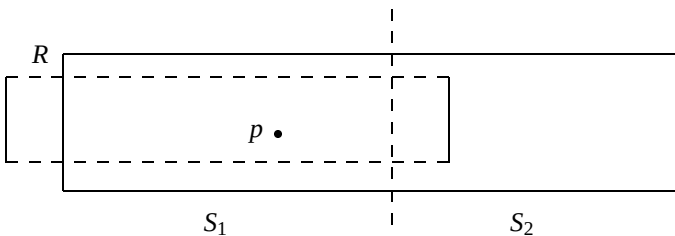
Algorithmus *ReportInc*(S)

{liefert zu einer Menge S von linken und rechten Rändern von Rechtecken (in getrennter Repräsentation) und Punkten in der Ebene alle Paare (p, R) von Punkten p und Rechtecken R mit Rand in S mit $p \in R$ }

1. *Divide*: Teile S (durch eine vertikale Gerade) in eine linke Hälfte S_1 und eine rechte Hälfte S_2 , falls S mehr als ein Element enthält; falls S nur aus einem Element besteht, ist nichts zu berichten;
2. *Conquer*: *ReportInc*(S_1); *ReportInc*(S_2);
3. *Merge*: Berichte alle Paare (p, R) mit: $p \in S_2$, der linke Rand von R ist in S_1 , aber der rechte Rand von R ist weder in S_1 noch in S_2 , und $p \in R$:



Berichte alle Paare (p, R) mit: $p \in S_1$, der rechte Rand von R ist in S_2 , aber der linke Rand von R ist weder in S_1 noch in S_2 , und $p \in R$:



Ende des Algorithmus *ReportInc*

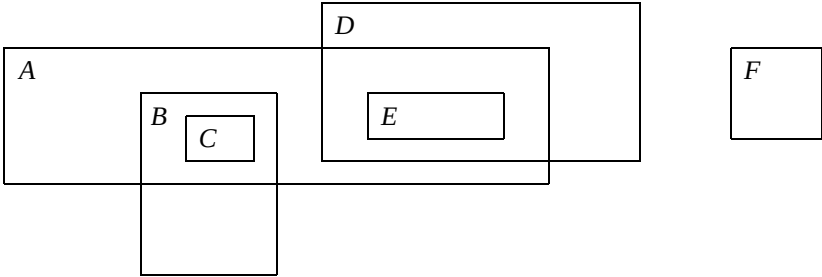


Abbildung 7.11

Der Nachweis der Korrektheit verläuft genauso wie im Falle des Algorithmus *ReportCuts* im vorigen Abschnitt:

Man zeigt, daß nach Ausführung eines Aufrufs *ReportInc(S)* für eine Menge von Punkten und (Rechtecke repräsentierenden) vertikalen Segmenten gilt: Alle Paare (p, R) von Inklusionen zwischen einem Punkt p und einem Rechteck R sind berichtet, für jeden Punkt p aus S und jedes Rechteck R , das in S wenigstens einmal (also: durch seinen linken oder rechten Rand oder durch beide) repräsentiert ist.

Für eine effiziente Implementation des Verfahrens kommt es offenbar darauf an, die im Merge-Schritt benötigten Mengen vertikaler Segmente effizient zu bestimmen, die eine linke (bzw. rechte) Rechteckseite in S_1 (bzw. S_2) repräsentieren, deren korrespondierende rechte (bzw. linke) Rechteckseite aber weder in S_1 noch in S_2 vorkommt. Das kann man ähnlich wie im Falle des Algorithmus *ReportCuts* im Abschnitt 7.3.1 machen und sichern, daß diese Mengen in konstanter Zeit initialisiert und in linearer Zeit im Merge-Schritt konstruiert werden können. Damit reduziert sich die im Merge-Schritt des Algorithmus *ReportInc* zu lösende Aufgabe auf das Problem, für eine nach unteren Endpunkten sortierte Menge von Intervallen und eine aufsteigend sortierte Menge von Punkten alle Paare (Punkt, Intervall) zu bestimmen, für die das Intervall den Punkt enthält. Es ist leicht zu sehen, daß das in einer Anzahl von Schritten möglich ist, die proportional zur Anzahl der Intervalle und Punkte und der Größe der Antwort ist. Insgesamt folgt:

Für eine Menge S von N Rechtecken und Punkten in der Ebene kann man alle k Paare (p, R) mit: p Punkt in S , R Rechteck in S und $p \in R$ mit Hilfe des Divide-and-conquer-Prinzips berichten in Zeit $O(N \log N + k)$ und Platz $O(N)$.

Die im Abschnitt 7.3.1 angegebene Lösung des rechteckigen Segmentschnittproblems und die hier skizzierte Lösung des Punkteinschlußproblems liefern zugleich auch eine Lösung des Rechteckschnittproblems für eine Menge iso-orientierter Rechtecke in der Ebene. Das ist das Problem, für eine gegebene Menge solcher Rechtecke alle Paare sich schneidender Rechtecke zu berichten. Dabei ist mit Rechteckschnitt sowohl Kantenschnitt als auch Inklusion gemeint.

Für das in Abbildung 7.11 angegebene Beispiel ist die gesuchte Antwort also die Menge:

$$\{(A, B), \quad (A, C), \quad (A, E), \quad (A, D), \quad (B, C), \quad (E, D)\}$$

Zur Lösung des Rechteckschnittproblems bestimmt man zunächst mit Hilfe des Verfahrens aus Abschnitt 7.3.1 alle Paare von Rechtecken, die sich an einer Kante schneiden. Dann wählt man für jedes der Rechtecke einen dieses Rechteck repräsentierenden Punkt, z.B. den Mittelpunkt, und bestimmt für die Menge aller Rechtecke und so erhaltenen Punkte alle Inklusionen von Punkten in Rechtecken. Das liefert alle Paare von Rechtecken, die sich vollständig einschließen (und außerdem manche, die sich schneiden). Insgesamt kann man auf diese Weise alle k Paare von sich schneidenden Rechtecken in einer Menge von N iso-orientierten Rechtecken in Zeit $O(N \log N + k)$ und Platz $O(N)$ bestimmen.

Wir bemerken abschließend, daß man das Rechteckschnittproblem auch direkt nach dem Divide-and-conquer-Prinzip lösen kann, ohne einen Umweg zu machen über das rechteckige Segmentschnitt- und das Punkteinschlußproblem. Weitere Beispiele für die Anwendung des Divide-and-conquer-Prinzips zur Lösung geometrischer Probleme findet man in  und .

7.4 Geometrische Datenstrukturen

Ganzzahlige Schlüssel kann man auffassen als Punkte auf der Zahlengeraden, also als nulldimensionale geometrische Objekte. Für sie ist charakteristisch, daß sie auf natürliche Weise geordnet sind. Eine große Vielfalt an Datenstrukturen zur Speicherung von Schlüsselmengen steht zur Auswahl. Je nachdem welche Operationen auf den Schlüsselmengen ausgeführt werden sollen, können wir Strukturen wählen, die die gewünschten Operationen besonders gut unterstützen. Zur Lösung der in den Abschnitten 7.2 und 7.3 behandelten geometrischen Probleme, des Sichtbarkeitsproblems und verschiedener Schnittprobleme für Liniensegmente in der Ebene, reichten die bekannten Strukturen aus. Es ist uns jedesmal gelungen, das geometrische Problem auf die Manipulation geeigneter Schlüsselmengen zu reduzieren.

Schon für Mengen von Punkten in der Ebene, erst recht für ausgedehnte geometrische Objekte, wie Liniensegmente, Rechtecke usw., reichen die bekannten Strukturen nicht mehr aus, wenn man typisch geometrische Operationen unterstützen möchte. Solche Operationen sind z.B.: Für eine gegebene Menge von Punkten in der Ebene und einen gegebenen, zweidimensionalen Bereich, berichte alle Punkte, die in den gegebenen Bereich fallen. Oder: Für eine gegebene Menge von Liniensegmenten in der Ebene und ein gegebenes Segment, berichte alle Segmente der Menge, die das gegebene Segment schneidet. Wir wollen in diesem Abschnitt einige neue, inhärent geometrische Datenstrukturen kennenlernen und zeigen, wie sie zur Lösung einer geometrischen Grundaufgabe benutzt werden können. Als Beispiel wählen wir das Rechteckschnittproblem. Das ist das Problem, für eine gegebene Menge von Rechtecken alle Paare sich schneidender Rechtecke zu finden. Im Abschnitt 7.4.1 zeigen wir zunächst, wie das Problem mit Hilfe des Scan-line-Prinzips gelöst werden kann und welche Anforderungen an für eine Lösung geeignete Datenstrukturen zu stellen sind. In den folgenden Abschnitten besprechen wir dann im einzelnen Segment-Bäume, Intervall-Bäume und Prioritäts-Suchbäume, die sämtlich zur Lösung des Rechteckschnittproblems geeignet sind. Die-

se Datenstrukturen müssen nicht nur typisch geometrische Operationen unterstützen, wie sie zur Lösung des Rechteckschnittproblems verwendet werden. Sie müssen auch das Einfügen und Entfernen geometrischer Objekte erlauben. Wir entwerfen alle drei Strukturen nach demselben Prinzip als halbdynamische, sogenannte Skelettstrukturen: Anstatt Strukturen zu benutzen, deren Größe sich der Menge der jeweils vorhandenen geometrischen Objekte voll dynamisch anpaßt, schaffen wir zunächst ein anfänglich leeres Skelett über einem diskreten Raster, das allen im Verlauf des Scan-line-Verfahrens benötigten Objekten Platz bietet. Dieses Vorgehen hat nicht nur den Vorzug größerer Einfachheit und Einheitlichkeit, es bietet auch die Basis für die Übertragung der in diesem Abschnitt für Mengen iso-orientierter Objekte entwickelten Verfahren auf nicht-iso-orientierte Objekte im Abschnitt 7.5.

7.4.1 Reduktion des Rechteckschnittproblems

Sei eine Menge von N iso-orientierten Rechtecken in der Ebene gegeben, d. h. alle linken und rechten und alle oberen und unteren Rechteckseiten sind zueinander parallel. Um nicht zahlreiche Sonderfälle diskutieren zu müssen, nehmen wir an, daß zwei Rechteckseiten höchstens einen Punkt gemeinsam haben können, und ferner, daß alle oberen und unteren Rechteckseiten paarweise verschiedene y -Koordinaten haben. Die Lösung des *Rechteckschnittproblems* verlangt, alle Paare sich schneidender Rechtecke zu berichten. “Rechteckschnitt” umfaßt dabei sowohl Kantenschnitt als auch Inklusion. Gerade das Entdecken aller Inklusionen erfordert zusätzlichen Aufwand. Denn um alle Paare von Rechtecken zu finden, die sich an einer Kante schneiden, können wir einfach das Scan-line-Verfahren zur Lösung des Schnittproblems für Mengen horizontaler und vertikaler Liniensegmente nehmen. Anstatt nun — wie im Falle der Anwendung des Divide-and-conquer-Prinzips, vgl. Abschnitt 7.3.2 — nur die Rechteckinklusionen mit Hilfe des Scan-line-Verfahrens zu bestimmen, wenden wir das Scan-line-Prinzip direkt auf das Rechteckschnittproblem an.

Wir schwenken eine horizontale Scan-line von oben nach unten über die gegebene Menge von Rechtecken. Dabei merken wir uns in einer Horizontalstruktur L stets die gerade aktiven Rechtecke, genauer die Schnitte der jeweils aktiven Rechtecke mit der Scan-line, also eine Menge von (horizontalen) Intervallen. Jedesmal, wenn wir auf einen oberen Rand eines Rechtecks treffen, bestimmen wir alle Intervalle in L , die sich mit dem oberen Rand überlappen. Das sind genau die Intervalle, die zu gerade aktiven Rechtecken gehören, die einen nichtleeren Durchschnitt mit R haben. Außerdem müssen wir in L ein neues Intervall einfügen, wenn wir auf den oberen Rand eines Rechtecks treffen, und aus R ein Intervall entfernen, wenn wir auf den unteren Rand eines Rechtecks treffen. Auf diese Weise reduzieren wir also das statische Schnittproblem für eine Menge von Rechtecken in der Ebene auf eine dynamische Folge von Überlappungsproblemen für horizontale Intervalle. Für ein Rechteck R bezeichnen wir die x -Koordinaten des linken und rechten Rands mit $x_l(R)$ und $x_r(R)$. $[x_l(R), x_r(R)]$ ist also ein R repräsentierendes Intervall. Wir nehmen stets an, daß $[x_l(R), x_r(R)]$ einen Verweis auf R enthält; mit anderen Worten: Man kann erkennen, welches Rechteck ein Intervall repräsentiert.

Jetzt formulieren wir das Scan-line-Verfahren zur Lösung des Rechteckschnittproblems:

Algorithmus Rechteckschnitt

{liefert zu einer Menge von N iso-orientierten Rechtecken in der Ebene die Menge aller k Paare von sich schneidenden Rechtecken}

$Q :=$ Folge der $2N$ oberen und unteren Rechteckseiten in abnehmender y -Reihenfolge;

$L := \emptyset$; {Menge der Schnitte der gerade aktiven Rechtecke mit der Scan-line}

while Q ist nicht leer **do**

begin

$q :=$ nächster Haltepunkt von Q ;

if q ist oberer Rand eines Rechtecks R , $q = [x_l(R), x_r(R)]$

then

begin

 bestimme alle Rechtecke R' derart, daß das

 Intervall $[x_l(R'), x_r(R')]$ in L ist und

$[x_l(R), x_r(R)] \cap [x_l(R'), x_r(R')] \neq \emptyset$

 und gebe (R, R') aus;

 füge $[x_l(R), x_r(R)]$ in L ein

end

else { q ist unterer Rand eines Rechtecks R }

 entferne $[x_l(R), x_r(R)]$ aus L

end

Abbildung 7.12 zeigt ein Beispiel für die Anwendung des Verfahrens. An der in diesem Beispiel gezeigten vierten Haltestelle der Scan-line enthält L die drei Intervalle $[.B, B.] = [x_l(B), x_r(B)]$, $[.C, C.] = [x_l(C), x_r(C)]$ und $[.D, D.] = [x_l(D), x_r(D)]$. L trifft auf den oberen Rand von A . Also müssen alle Intervalle in L bestimmt werden, die sich mit dem Intervall $[.A, A.] = [x_l(A), x_r(A)]$ überlappen. Das ist nur das Intervall $[.B, B.]$. Also wird nur das Paar (A, B) ausgegeben und anschließend $[.A, A.]$ in L eingefügt.

Man beachte, daß alle Intervalle, die jemals in L eingefügt werden, aus L entfernt werden oder für die Überlappungen festgestellt werden müssen, Intervalle über einer diskreten Menge von höchstens $2N$ Endpunkten sind: Das ist die Menge der x -Koordinaten der linken und rechten Rechteckseiten. Wir können uns die Menge der möglichen Intervallgrenzen als mit der Menge der Rechtecke gegeben denken. Da es offenbar nur auf die relative Anordnung der Intervallgrenzen ankommt, können wir der Einfachheit halber sogar annehmen, daß die Intervallgrenzen ganzzahlig und äquidistant sind.

Damit haben wir die Implementierung des Verfahrens reduziert auf das Problem, eine Datenstruktur zur Speicherung einer Menge L von Intervallen $[a, b]$ mit $a, b \in \{1, \dots, n\}$ zu finden, so daß folgende Operationen auf L ausführbar sind: Das Einfügen eines Intervalls in L , das Entfernen eines Intervalls aus L und das Ausführen von Überlappungsfragen, d.h. für ein gegebenes Intervall I : Bestimme alle Intervalle I' aus L , die sich mit I überlappen, d.h. für die $I \cap I' \neq \emptyset$ gilt.

Verschiedene Implementationen für L führen unmittelbar zu verschiedenen Lösungen des Rechteckschnittproblems. Wir besprechen zunächst zwei Möglichkeiten, die sich durch folgende weitere Reduktion der Überlappungsfrage ergeben.

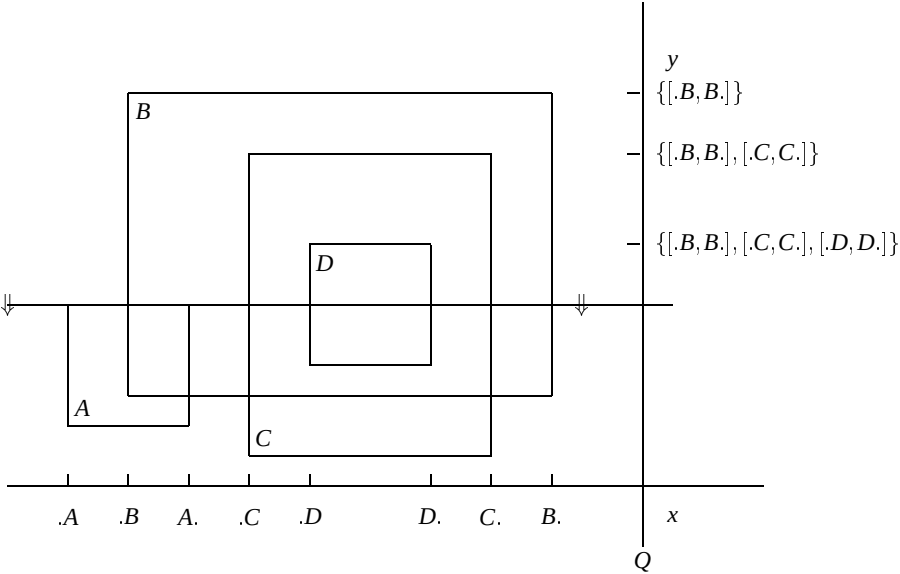


Abbildung 7.12

Nehmen wir an, es sollen alle Intervalle $[a', b']$ bestimmt werden, die sich mit einem gegebenen Intervall $[a, b]$ überlappen. Es gibt offenbar genau die folgenden vier Möglichkeiten für eine Überlappung:

- $$\begin{array}{c} a \text{ --- } b \\ a' \text{ --- } b' \end{array}$$

(1)
- $$\begin{array}{c} a \text{ --- } b \\ a' \text{ --- } b' \end{array}$$

(2)
- $$\begin{array}{c} a \text{ --- } b \\ a' \text{ --- } b' \end{array}$$

(3)
- $$\begin{array}{c} a \text{ --- } b \\ a' \text{ --- } b' \end{array}$$

(4)

D h., es ist $a' \in [a, b]$, wie im Fall (2) und (3), oder es ist $a \in [a', b']$, wie im Fall (1) und (4). Die Überlappungsfrage kann damit reduziert werden auf eine Bereichsanfrage (range query) und eine sogenannte inverse Bereichsanfrage oder *Aufspießfrage* (stabbing query). Denn es gilt:

$$\begin{aligned} & \{ [a', b'] \mid [a', b'] \cap [a, b] \neq \emptyset \} \\ &= \{ [a', b'] \mid a \text{ spießt } [a', b'] \text{ auf} \} \cup \{ [a', b'] \mid a' \text{ liegt im Bereich } [a, b] \} \end{aligned}$$

Dabei sagen wir: Ein Punkt spießt ein Intervall auf, wenn das Intervall den Punkt enthält.

Um also für ein gegebenes Intervall $[a, b]$ alle überlappenden Intervalle $[a', b']$ zu finden, genügt es offenbar:

1. alle Intervalle $[a', b']$ zu finden, die der linke Randpunkt a aufspießt, und
2. alle Intervalle $[a', b']$ zu finden, deren linker Randpunkt a' im Bereich $[a, b]$ liegt.

Die zweite Aufgabe ist mit bereits wohlbekannten Mitteln leicht lösbar: Man speichere alle linken Randpunkte in einem Bereichs-Suchbaum wie in Abschnitt 7.2.2 beschrieben.

Es genügt also, die erste Aufgabe zu lösen und eine Struktur zu entwerfen, die das Einfügen und Entfernen von Intervallen und das Beantworten von Aufspieß-Anfragen unterstützt. Wir bringen zwei Varianten einer derartigen Struktur, den Segment-Baum in Abschnitt 7.4.2 und den Intervall-Baum in Abschnitt 7.4.3.

7.4.2 Segment-Bäume

Segment-Bäume sind ein erstes Beispiel einer halb-dynamischen Skelettstruktur: Man baut zunächst ein leeres Skelett zur Aufnahme von Intervallen mit Endpunkten aus einer gegebenen Menge $\{1, \dots, n\}$. Man kann in dieses Skelett Intervalle einfügen oder daraus entfernen. Ferner kann man für einen gegebenen Punkt feststellen, welche aktuell vorhandenen Intervalle er aufspießt.

Jedes Intervall $[a, b]$ mit $a, b \in \{1, \dots, n\}$ kann man sich zusammengesetzt denken aus einer Folge von elementaren Segmenten $[i, i + 1]$, $1 \leq i < n$. Ein Segment-Baum wird nun wie folgt konstruiert: Man baut einen vollständigen Binärbaum, also einen Binärbaum, der auf jedem Niveau die maximale Knotenzahl hat. Die Blätter repräsentieren die elementaren Segmente. Jeder innere Knoten repräsentiert die Vereinigung (der Folge) der elementaren Segmente an den Blättern im Teilbaum dieses Knotens. Die Wurzel repräsentiert also das Intervall $[1, n]$. Das ist das leere Skelett eines Segment-Baumes. Das Skelett kann nun dynamisch mit Intervallen gefüllt werden, indem man den Namen eines einzufügenden Intervalls an genau diejenigen Knoten schreibt, die am nächsten bei der Wurzel liegen und ein Intervall repräsentieren, das vollständig in dem einzufügenden Intervall enthalten ist. Abbildung 7.13 zeigt das Beispiel eines Segment-Baumes, der die Intervalle $\{A, \dots, F\}$ mit Endpunkten in $\{1, \dots, 9\}$ enthält. An jedem Knoten sind das von ihm repräsentierte Intervall als durchgezogene Linie und die Liste der Namen von Intervallen angegeben, die diesem Knoten zugeordnet wurden (aus Gründen der Darstellung liegen Lücken zwischen Intervallen).

Bezeichnen wir mit $I(p)$ das durch den Knoten p des Segment-Baumes repräsentierte Intervall, so gilt: Der Name eines Intervalls I tritt in der Intervall-Liste des Knotens p auf genau dann, wenn $I(p) \subseteq I$ gilt und für keinen Knoten p' auf dem Pfad von der Wurzel zu p $I(p') \subseteq I$ gilt.

Daraus ergibt sich sofort folgendes Verfahren zum Einfügen eines Intervalls I :

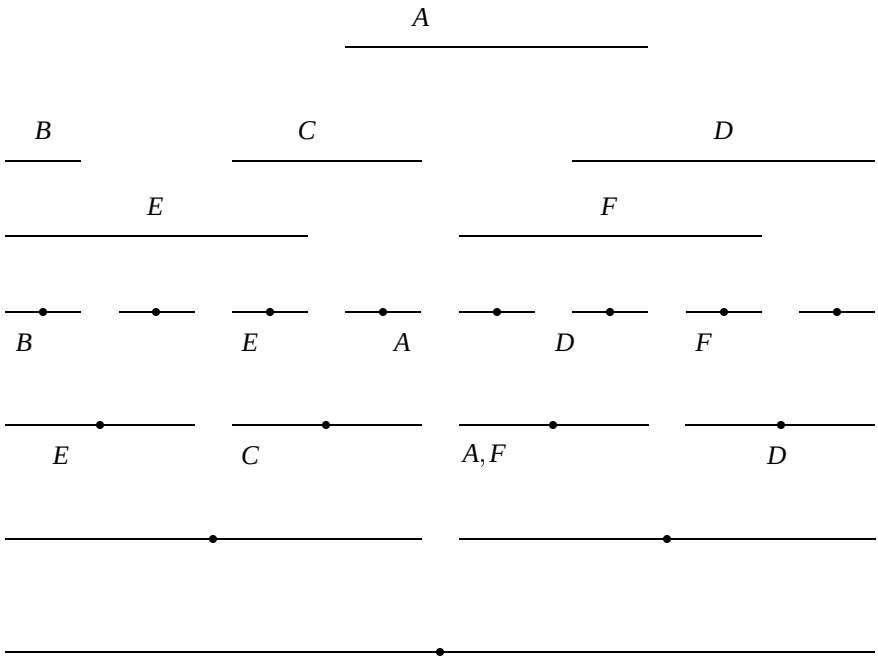


Abbildung 7.13

```

procedure Einfügen ( $I$  : Intervall;  $p$  : Knoten);
{anfangs ist  $p$  die Wurzel des Segment-Baumes}
  if  $I(p) \subseteq I$ 
  then
    füge  $I$  in die Intervall-Liste von  $p$  ein und fertig
  else
    begin
      if ( $p$  hat linken Sohn  $p_\lambda$ ) and  $(I(p_\lambda) \cap I \neq \emptyset)$ 
      then Einfügen( $I, p_\lambda$ );
      if ( $p$  hat rechten Sohn  $p_\rho$ ) and  $(I(p_\rho) \cap I \neq \emptyset)$ 
      then Einfügen( $I, p_\rho$ )
    end

```

Auf den ersten Blick könnte man den Verdacht haben, daß diese rekursiv formulierte Einfügeprozedur im schlimmsten Fall für sämtliche Knoten eines Segment-Baumes aufgerufen wird. Das ist jedoch keineswegs der Fall, wie folgende Überlegung zeigt: Wird die Einfügeprozedur nach einem Aufruf von $Einfügen(I, p)$ für beide Söhne p_λ und p_ρ eines Knotens p aufgerufen und bricht die Prozedur nicht bereits für einen dieser beiden Söhne ab, so kann die Einfügeprozedur für höchstens zwei der Enkel von p erneut aufgerufen werden. Das zeigt Abbildung 7.14. In dieser Abbildung ist durch ”*”

ein Aufruf der Einfügeprozedur und durch "†" angedeutet, daß das Einfüge-Verfahren hier abbricht, da diese Knoten ein ganz in I enthaltenes Intervall repräsentieren.

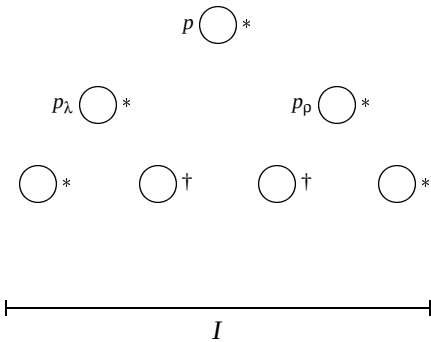


Abbildung 7.14

Die Folge der rekursiven Aufrufe der Einfügeprozedur kann man daher stets als einen sich höchstens einmal gabelnden Pfad darstellen, wie ihn Abbildung 7.15 zeigt.

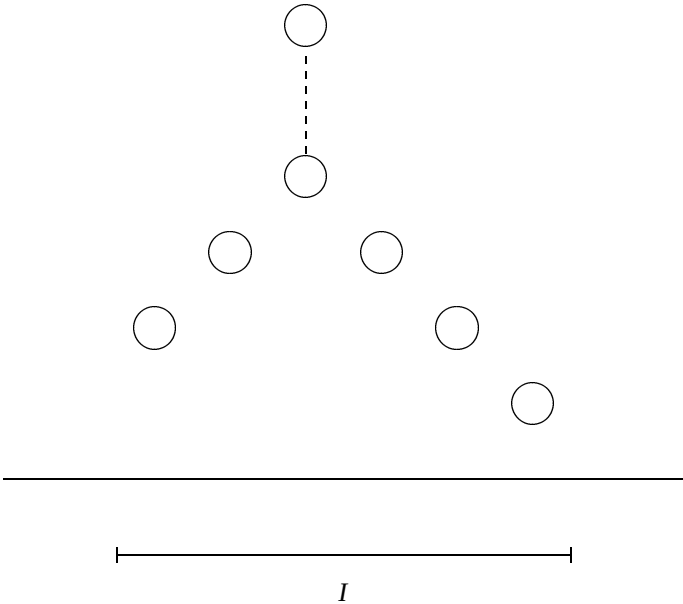


Abbildung 7.15

Aus dieser Überlegung kann man schließen:

1. Das Einfügen eines Intervalls ist in $O(\log N)$ Schritten ausführbar.
2. Jedes Intervall I kommt in höchstens $O(\log N)$ Intervall-Listen vor.

Denn der Segment-Baum mit $(N - 1)$ Segmenten hat die Höhe $\log N$.

Wir haben allerdings stillschweigend vorausgesetzt, daß das Einfügen eines Intervalls (bzw. eines Intervall-Namens) in die zu einem Knoten des Segment-Baumes gehörende Intervall-Liste in konstanter Schrittzahl möglich ist. Das ist leicht erreichbar, wenn wir die Intervall-Listen als verkettete Listen implementieren und neue Intervalle stets am Anfang oder Ende einfügen. Man beachte aber, daß wir dann unter Umständen Schwierigkeiten haben, ein Intervall in einer zu einem Knoten gehörenden Intervall-Liste zu finden und daraus gegebenenfalls zu entfernen. Man beachte schließlich noch, daß die Intervall-Listen auf einem beliebigen Pfad im Segment-Baum von der Wurzel zu einem Blatt paarweise disjunkt sein müssen. Denn sobald ein Intervall in die Liste eines Knotens p aufgenommen wurde, wird es in keine Liste eines Nachfolgers von p eingefügt.

Wie können Aufspieß-Fragen beantwortet werden? Um für einen gegebenen Punkt x alle im Segment-Baum gespeicherten Intervalle zu finden, die x aufspießt, benutzen wir den Segment-Baum als Suchbaum für x . D h. wir suchen nach dem Elementarsegment, das x enthält. Wir geben dann alle Intervalle in allen Listen aus, die zu Knoten auf dem Suchpfad gehören. Denn das sind genau sämtliche Intervalle, die x aufspießt. Genauer: Wir rufen die folgende Prozedur *report* für die Wurzel des Segment-Baumes und den Punkt x auf.

```

procedure report ( $p$  : Knoten;  $x$  : Punkt);
  {ohne Einschränkung ist  $x \in I(p)$ }
  gebe alle Intervalle der Liste von  $p$  aus;
  if  $p$  ist Blatt
    then fertig
  else
    begin
      if ( $p$  hat einen linken Sohn  $p_\lambda$ ) and ( $x \in I(p_\lambda)$ )
        then report( $p_\lambda, x$ );
      if ( $p$  hat einen rechten Sohn  $p_\rho$ ) and ( $x \in I(p_\rho)$ )
        then report( $p_\rho, x$ )
    end

```

Natürlich kann niemals zugleich $x \in I(p_\lambda)$ und $x \in I(p_\rho)$ sein. Daher werden in der Tat genau $\lceil \log_2 N \rceil$ Intervall-Listen betrachtet. Der Aufwand, die Intervalle auszugeben, ist damit proportional zu $\log N$ und zur Anzahl der Intervalle, die x enthalten. Insgesamt haben wir damit eine Struktur mit folgenden Charakteristika: Das Einfügen eines Intervalls ist in Zeit $O(\log N)$ möglich; die zum Beantworten einer Aufspieß-Frage erforderliche Zeit ist $O(\log N + k)$, wobei k die Größe der Antwort ist. Die Struktur hat den Speicherbedarf $O(N \log N)$.

Um ein Intervall aus dem Segment-Baum zu entfernen, können wir im Prinzip genauso vorgehen wie beim Einfügen: Wir bestimmen zunächst die $O(\log N)$ Knoten, in deren Intervall-Listen das zu entfernende Intervall vorkommt und entfernen es dann aus jeder dieser Listen. Da wir jedoch nicht wissen, wo das Intervall in diesen Listen vorkommt, bleibt uns nichts anderes übrig, als jede dieser Listen von vorn nach hinten zu durchsuchen. Das kann im schlimmsten Fall $O(N)$ Schritte für jede Liste kosten — ein nicht akzeptabler Aufwand. Wir wollen vielmehr erreichen, daß wir für jedes Intervall I alle Vorkommen von I in Intervall-Listen von Knoten des Segment-Baumes in einer Anzahl von Schritten bestimmen können, die proportional zu $\log N$ und zur Anzahl dieser Vorkommen ist.

Wir lösen das Problem folgendermaßen: Als Grundstruktur nehmen wir einen Segment-Baum, wie wir ihn bisher beschrieben haben. Darüberhinaus speichern wir alle im Segment-Baum vorkommenden Intervallnamen in einem alphabetisch sortierten Wörterbuch ab. D h., in einer Struktur, die das Suchen, Einfügen und Entfernen eines Intervallnamens in $O(\log N)$ Schritten erlaubt, wenn wir eine Implementation durch balancierte Bäume verwenden und N die insgesamt vorhandene Zahl von Intervallnamen ist. Jeder Intervallname I dieses Wörterbuches zeigt auf den Anfang einer verketteten Liste von Zeigern, die auf alle Vorkommen von I in der Grundstruktur weisen. Insgesamt erhalten wir damit eine Struktur, die grob wie in Abbildung 7.16 dargestellt werden kann.

Da wir den Segment-Baum im wesentlichen unverändert gelassen haben, können wir Aufspieß-Fragen wie bisher beantworten. Beim Einfügen eines neuen Intervalls müssen wir natürlich den Namen dieses Intervalls zusätzlich in das Wörterbuch einfügen und auch die verkettete Liste von Zeigern auf sämtliche Vorkommen des Intervalls im Segment-Baum aufbauen. Da jedes Intervall an höchstens $\log N$ Stellen im Segment-Baum vorkommen kann, ist der Gesamtaufwand für das auf diese Weise veränderte Einfügen eines Intervalls immer noch von der Größenordnung $O(\log N)$. Das Entfernen eines Intervalls kann jetzt genau umgekehrt zum Einfügen ebenfalls in $O(\log N)$ Schritten ausgeführt werden: Man sucht den Namen I des zu entfernenden Intervalls im Wörterbuch, findet dort die Verweise auf alle Vorkommen von I im Segment-Baum und kann I zunächst dort und anschließend auch im Wörterbuch löschen. Der gesamte Speicherbedarf dieser Struktur ist offenbar $O(N \log N)$.

Wir haben jetzt alles beisammen zur Lösung des eingangs gestellten Problems, alle k Paare von sich schneidenden Rechtecken in einer gegebenen Menge von N iso-orientierten Rechtecken mit Hilfe des Scan-line-Verfahrens zu bestimmen. Man verwendet als Horizontalstruktur ein Paar von zwei dynamischen, also Einfügungen und Streichungen erlaubenden Strukturen, einen Bereichs-Suchbaum zur Speicherung der linken Endpunkte der jeweils gerade aktiven Intervalle (= Schnitte der jeweils aktiven Rechtecke mit der Scan-line), und einen Segment-Baum für die jeweils aktiven Intervalle, um ein Wörterbuch für die Intervallnamen erweitert, wie eben beschrieben.

Die Strukturen liefern insgesamt eine Möglichkeit zur Implementation einer Menge L von N Intervallen derart, daß das Einfügen und Entfernen eines Intervalls stets in $O(\log N)$ Schritten möglich ist und alle r Intervalle aus L , die sich mit einem gegebenen Intervall überlappen, in Zeit $O(\log N + r)$ bestimmt werden können. Daher gilt:

Das Rechteckschnittproblem kann nach dem Scan-line-Verfahren mit Hilfe von Segment-Bäumen in Zeit $O(N \log N + k)$ und Platz $O(N \log N)$ gelöst werden. Dabei

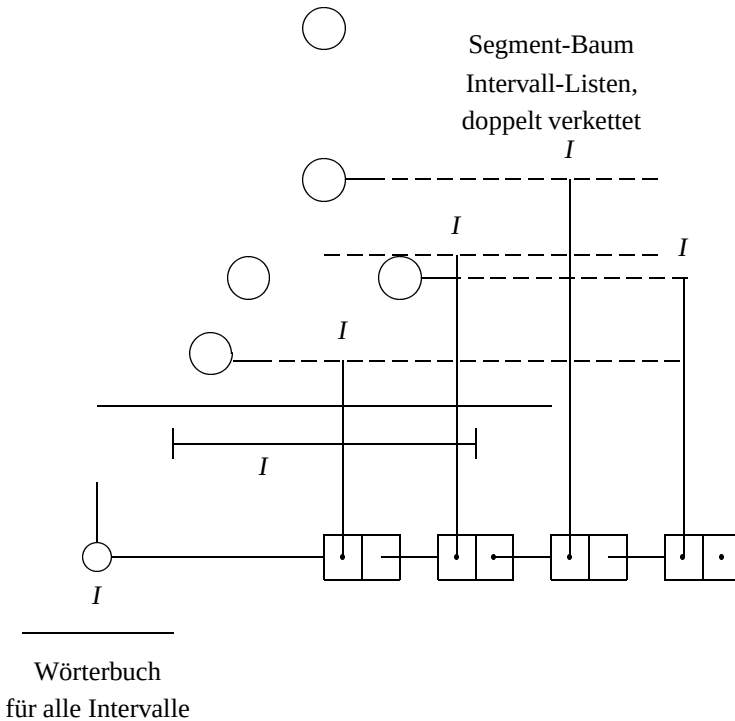


Abbildung 7.16

ist N die Anzahl der gegebenen Rechtecke und k die Anzahl der Paare sich schneidender Rechtecke.

Wir vergleichen dieses Ergebnis mit der in Abschnitt 7.3.2 erhaltenen Divide-and-conquer-Lösung desselben Problems: Die Laufzeit beider Verfahren ist dieselbe, aber der Speicherbedarf der Scan-line-Lösung ist nicht linear beschränkt. Wir werden im nächsten Abschnitt Intervall-Bäume als Alternative zu Segment-Bäumen vorstellen, die ebenfalls in einer dem Scan-line-Prinzip folgenden Lösung des Rechteckschnittproblems verwendet werden können, die aber nur linearen Speicherbedarf haben. Darauf kann man eine Scan-line-Lösung des Rechteckschnittproblems gründen, die Zeitbedarf $O(N \log N + k)$ und Platzbedarf $O(N)$ hat.

Segment-Bäume sind jedoch unabhängig von ihrer Verwendung in diesem Abschnitt von eigenem Interesse. Gerade die redundante Abspeicherung von Intervallen an vielen Knoten und die Freiheit, die Intervallnamen in den Knotenlisten beliebig, und damit auch nach neuen Kriterien anzuordnen, sind der Schlüssel zu weiteren Anwendungen (vgl. hierzu die Abschnitte 7.5 und 7.6).

Schließlich bemerken wir noch, daß man Segment-Bäume auch voll dynamisch machen kann in dem Sinne, daß ihre Größe nicht von der Größe des Skeletts, sondern nur von der Anzahl der jeweils gerade vorhandenen Intervalle abhängt. Einfüge- und

Entferne-Operationen sind aber noch komplizierter und damit einer Implementierung für die Praxis noch weniger zugänglich.

7.4.3 Intervall-Bäume

Wir wollen jetzt eine Datenstruktur zur Speicherung einer Menge von $O(N)$ Intervallen mit Endpunkten in einer diskreten Menge von $O(N)$ Endpunkten vorstellen, die nur linearen Speicherbedarf hat und die Operationen Einfügen eines Intervalls, Entfernen eines Intervalls und Aufspieß-Fragen in Zeit $O(\log N)$ bzw. $O(\log N + k)$ auszuführen erlaubt. Es dürfte unmittelbar klar sein, daß wir damit auch eine Verbesserung des Scanline-Verfahrens zur Lösung des Rechteckschnittproblems erhalten.

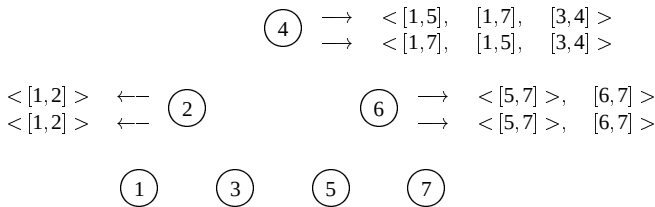
Da wir es stets nur mit einer endlichen Menge von Intervallen zu tun haben, können wir ohne Einschränkung annehmen, daß die Intervallgrenzen einer gegebenen Menge von höchstens N Intervallen auf die ganzen Zahlen $1, \dots, s$ fallen, wobei $s \leq 2N$ ist.

Ein Intervall-Baum zur Speicherung einer Menge von Intervallen mit Endpunkten in $\{1, \dots, s\}$ besteht aus einem Skelett und sortierten Intervalllisten, die mit den Knoten des Skeletts des Intervall-Baumes verbunden sind. Das Skelett des Intervall-Baumes ist ein vollständiger Suchbaum für die Schlüsselmenge $\{1, \dots, s\}$. Jeder innere Knoten dieses Suchbaumes ist mit zwei sortierten Intervall-Listen verbunden, einer u -Liste und einer o -Liste. Die u -Liste ist eine nach aufsteigenden unteren Endpunkten sortierte Liste von Intervallen und die o -Liste eine nach absteigenden oberen Endpunkten sortierte Liste von Intervallen. Ein Intervall $[l, r]$ mit $l, r \in \{1, \dots, s\}, l \leq r$, kommt in der u -Liste und o -Liste desjenigen Knotens im Skelett des Intervall-Baumes mit minimaler Tiefe vor, dessen Schlüssel im Intervall $[l, r]$ liegt.

Folgendes Beispiel zeigt einen Intervall-Baum für die Menge

$$\{[1, 2], [1, 5], [3, 4], [5, 7], [6, 7], [1, 7]\}$$

von Intervallen mit Endpunkten in $\{1, \dots, 7\}$.



In diesem Beispiel sind die u -Listen stets oben und die o -Listen unten an die jeweils zugehörigen Knoten geschrieben. Alle nicht explizit dargestellten u - und o -Listen sind leer. (Offenbar müssen die den Blättern zugeordneten Listen immer leer sein, wenn man nicht Intervalle $[i, i]$ mit $1 \leq i \leq s$ zuläßt!)

Bezeichnen wir für einen Knoten p eines Intervall-Baumes den Schlüssel von p mit $p.key$, den linken Sohn von p mit p_{λ} und den rechten mit p_{ρ} , so kann das Verfahren zum Einfügen eines Intervalls $I = [I, I.]$ in einen Intervall-Baum wie folgt beschrieben werden:

```

procedure Einfügen ( $I$  : Intervall;  $p$  : Knoten);
{anfangs ist  $p$  die Wurzel des Intervall-Baumes;  $I$  ist ein Intervall
 mit linkem Endpunkt  $.I$  und rechtem Endpunkt  $I$ .}
if  $p.key \in I$ 
then
    füge  $I$  entsprechend seinem unteren Endpunkt in die  $u$ -Liste
    von  $p$  und entsprechend seinem oberen Endpunkt in die
     $o$ -Liste von  $p$  ein und fertig!
else
    if  $p.key < .I$ 
    then Einfügen( $I, p_p$ )
    else  $\{p.key > I.\}$ 
        Einfügen( $I, p_\lambda$ )

```

Für jedes Intervall I und jeden Knoten p gilt, daß I entweder $p.key$ enthalten muß oder aber I liegt ganz rechts von $p.key$ (dann ist $p.key < .I$) oder I liegt ganz links von $p.key$ (dann ist $p.key > I$). Da wir angenommen hatten, daß alle möglichen Intervallgrenzen als Schlüssel von Knoten im Skelett des Segment-Baumes vorkommen, ist klar, daß das rekursiv formulierte Einfüge-Verfahren hält. Implementiert man die u -Liste und o -Liste eines jeden Knotens als balancierten Suchbaum, folgt, daß das Einfügen eines Intervalls in einer Anzahl von Schritten ausgeführt werden kann, die höchstens linear von der Höhe des Intervallbaum-Skeletts und logarithmisch von der Länge der einem Knoten zugeordneten u - und o -Listen abhängt. Mit der zu Beginn dieses Abschnitts gemachten Annahme sind das $O(\log N)$ Schritte.

Das Entfernen eines Intervalls I erfolgt natürlich genau umgekehrt zum Einfügen: Man bestimmt ausgehend von der Wurzel des Intervall-Baumes den Knoten p mit geringster Tiefe, für den $p.key \in I$ gilt. (Einen derartigen Knoten muß es stets geben!) Dann entfernt man I aus den sortierten u - und o -Listen von p . Offenbar kann man das ebenfalls in $O(\log N)$ Schritten ausführen.

Nun überlegen wir uns noch, wie Aufspieß-Fragen beantwortet werden können. Dabei nehmen wir an, daß der Punkt x , für den wir alle im Intervall-Baum gespeicherten Intervalle finden wollen, die x aufspießt, einer der Schlüssel des Skeletts ist. Das ist keine wesentliche Annahme, sondern soll lediglich sichern, daß eine Suche nach x im Skelett des Intervall-Baumes stets erfolgreich endet, und die Präsentation des Verfahrens vereinfacht wird.

Zur Bestimmung der Intervalle, die ein gegebener Punkt aufspießt, suchen wir im Skelettbaum nach x . Die Suche beginnt bei der Wurzel und endet beim Knoten mit Schlüssel x . Ist p ein beliebiger Knoten auf diesem Pfad und ist $p.key \neq x$, dann kann man nicht sämtliche Intervalle ausgeben, die in der u - bzw. o -Liste von p vorkommen, denn diese Listen enthalten Intervalle, die zwar $p.key$, aber möglicherweise x nicht aufspießt. Ist jedoch $p.key > x$, so könnte x die Intervalle eines Anfangsstücks der u -Liste von p durchaus ebenfalls aufspießen. Entsprechend kann x durchaus einige Intervalle eines Anfangsstücks der o -Liste aufspießen, wenn $p.key < x$ ist. Diese Intervalle müssen natürlich sämtlich ausgegeben werden. Abbildung 7.17 illustriert die beiden Fälle.

Wir haben angenommen, daß genau einer der drei Fälle $x = p.key$ oder $x < p.key$ oder $x > p.key$ möglich ist. Daher können wir das Verfahren zum Berichten aller Intervalle eines Intervall-Baumes, die x aufspießt, wie folgt formulieren:

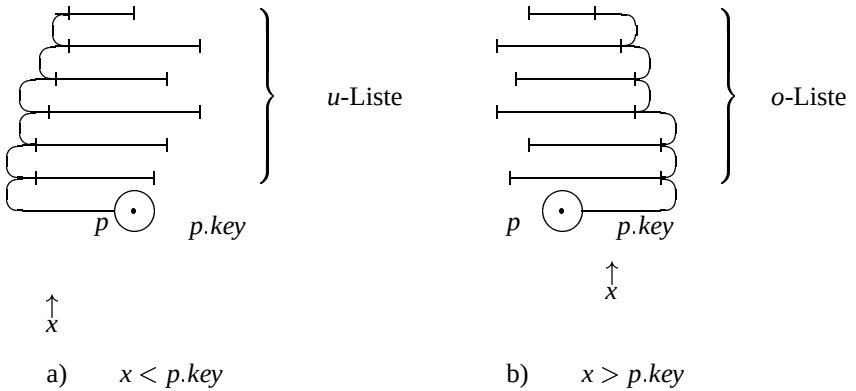


Abbildung 7.17

```

procedure report( $p$  : Knoten;  $x$  : Punkt);
if  $x = p.key$ 
  then
    gebe alle Intervalle der  $u$ -Liste (oder
      alle Intervalle der  $o$ -Liste) von  $p$  aus und fertig!
  else
    if  $x < p.key$ 
      then
        gebe alle Intervalle  $I$  der  $u$ -Liste von  $p$  mit
           $I \leq x$  aus {das ist ein Anfangsstück dieser Liste!}
        report ( $p_\lambda, x$ )
      else { $x > p.key$ }
        gebe alle Intervalle  $I$  der  $o$ -Liste von  $p$  mit
           $I \geq x$  aus {das ist ein Anfangsstück dieser Liste!}
        report ( $p_p, x$ )
  
```

Die Ausgabe eines Anfangsstücks einer sortierten Liste, die als balancierter Suchbaum implementiert ist, kann offensichtlich in einer Anzahl von Schritten erfolgen, die linear mit der Anzahl der ausgegebenen Elemente wächst.

Die u - und o -Listen eines Knotens des Skeletts eines Intervall-Baumes können maximal alle N Intervalle enthalten. Da jedoch jedes Intervall in der u - und o -Liste höchstens eines Knotens vorkommen kann, benötigt die Struktur insgesamt nur $O(N)$ Speicherplatz. Wir fassen unsere Überlegungen wie folgt zusammen:

Intervall-Bäume eignen sich zur Speicherung einer dynamisch veränderlichen Menge von höchstens N Intervallen mit Endpunkten im Bereich $\{1, \dots, s\}$, $s \leq 2N$. Sie haben Speicherbedarf $O(N)$ und erlauben das Einfügen eines Intervalls in Zeit $O(\log N)$, das Entfernen eines Intervalls in Zeit $O(\log N)$, und das Beantworten von Aufspieß-Fragen in Zeit $O(\log N + k)$, wobei k die Größe der Antwort ist.

Der Aufbau eines Intervall-Baumes kann durch Bildung des zunächst leeren Skeletts in Zeit $O(N)$ geschehen, das dann durch iteriertes Einfügen gefüllt wird.

Auf Grund der bereits zum Ende des vorigen Abschnitts 7.4.2 angestellten Überlegungen erhält man ferner:

Das Rechteckschnittproblem kann nach dem Scan-line-Verfahren mit Hilfe von Intervall-Bäumen in Zeit $O(N \log N + k)$ und Platz $O(N)$ gelöst werden. Dabei ist N die Zahl der gegebenen Rechtecke und k die Anzahl sich schneidender Paare von Rechtecken.

Intervall-Bäume haben gegenüber Segment-Bäumen den Vorzug, weniger Speicherplatz zu beanspruchen. Ihr Nachteil ist, daß sie weniger flexibel sind. Denn im Unterschied zu Segment-Bäumen kann man die Knotenlisten in Intervall-Bäumen nicht beliebig anordnen.

Intervall-Bäume wurden unabhängig voneinander von Edelsbrunner  und McCreight  erfunden. McCreight kommt jedoch auf ganz anderem Wege zu dieser Struktur  nennt sie Kachelbaum-Struktur (tile tree): Er benutzt die Darstellung von Intervallen durch Punkte in der Ebene, wie wir sie im nächsten Abschnitt kennenlernen werden. Für Intervall-Bäume gilt übrigens wie für Segment-Bäume, daß sie vollkommen dynamisch gemacht werden können; d.h. ihre Größe paßt sich der Anzahl der jeweils vorhandenen Intervalle dynamisch an. Wir haben dagegen eine halbdynamische Struktur: Ein anfangs leeres Skelett kann dynamisch gefüllt werden.

7.4.4 Prioritäts-Suchbäume

Wir haben bereits in Abschnitt 7.4.1 gezeigt, daß es zur Implementation des Scan-line-Verfahrens zur Lösung des Rechteckschnittproblems genügt, eine Implementation für eine Menge L von Intervallen zu finden, auf der folgende Operationen ausgeführt werden: Einfügen eines Intervalls, Entfernen eines Intervalls und für ein gegebenes Intervall I alle Intervalle I' aus L finden, die sich mit I überlappen, für die also $I \cap I' \neq \emptyset$ ist. Nachdem wir in den Abschnitten 7.4.2 und 7.4.3 zwei Möglichkeiten angegeben haben, die sich durch eine weitere Reduktion des Überlappungsproblems für Intervalle auf das Beantworten von Bereichs- und Aufspieß-Fragen ergaben, wollen wir jetzt das Überlappungsproblem direkt betrachten.

Jedes Intervall (mit Endpunkten aus einer festen, beschränkten Menge möglicher Endpunkte) kann man repräsentieren durch einen Punkt im zweidimensionalen Raum: Repräsentiere das Intervall $[l, r]$ mit $l \leq r$ durch den Punkt (r, l) . Dann bedeutet die Aufgabe, alle Intervalle $[x', y']$ zu bestimmen, die sich mit einem gegebenen Intervall $I = [x, y]$ überlappen, genau dasselbe wie die Aufgabe, alle Punkte (y', x') zu berichten, mit $x \leq y'$ und $x' \leq y$, d.h. alle Punkte, die rechts unterhalb des Frage-Punkts (x, y) liegen. Abbildung 7.18 erläutert dies genauer an einem Beispiel.

Es genügt also, eine Struktur zur Speicherung einer Menge von Punkten im zweidimensionalen Raum zu finden, derart, daß das Einfügen und Entfernen von Punkten möglichst effizient ausführbar ist und außerdem alle Punkte eines bestimmten Bereichs möglichst schnell berichtet werden können. Glücklicherweise sind die Bereiche, die wir zulassen müssen, von sehr spezieller Form. Ihre Grenzen sind parallel zu den gegebenen Koordinatenachsen, also iso-orientiert; mehr noch, sie sind stets *S-gegründet*

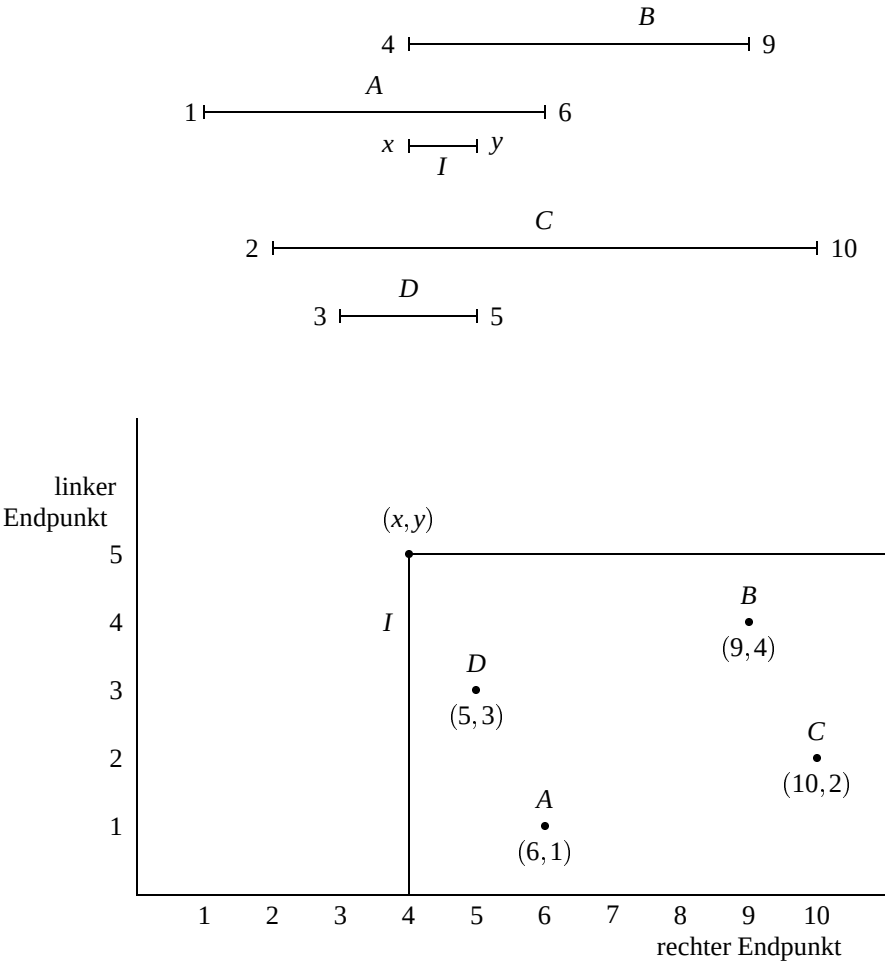


Abbildung 7.18

(*south-grounded*), d.h. die untere Bereichsgrenze fällt mit der x -Achse zusammen. Man kann einen solchen Bereich als *1.5-dimensional* ansehen. Denn er ist festgelegt durch einen eindimensionalen Bereich in x -Richtung (den linken und rechten Randwert) und durch eine Obergrenze in y -Richtung, vgl. Abbildung 7.19.

(Die zur Lösung des Überlappungsproblems für Intervalle benötigten Bereiche sind rechts offen, d.h. sie haben den maximal möglichen x -Wert als rechten Randwert.) Prioritäts-Suchbäume sind genau auf diese Situation zugeschnitten. Sie sind eine 1.5-dimensionale Struktur zur Speicherung von Punkten im zweidimensionalen Raum. Ein *Prioritäts-Suchbaum* ist ein Blattsuchbaum für die x -Werte und zugleich ein Heap für die y -Werte der Punkte. Genauer: Jeder Punkt (x,y) wird auf einem Suchpfad von der

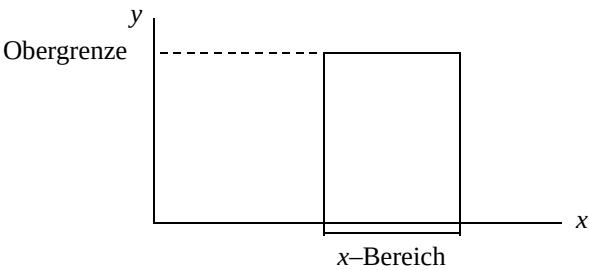


Abbildung 7.19

Wurzel zum Blatt x an einem inneren Knoten entsprechend seinem y -Wert abgelegt. D.h. die y -Werte nehmen auf jedem Suchpfad höchstens zu. Auch Prioritäts-Suchbäume kann man als volldynamische oder halbdynamische Skelettstrukturen über einem festen, beschränkten Universum entwickeln. Abbildung 7.20 zeigt einen Prioritätssuchbaum, der die Punkte A, B, C, D des ersten Beispiels über dem Universum $\{1, \dots, 10\}$ möglicher x -Koordinaten speichert.

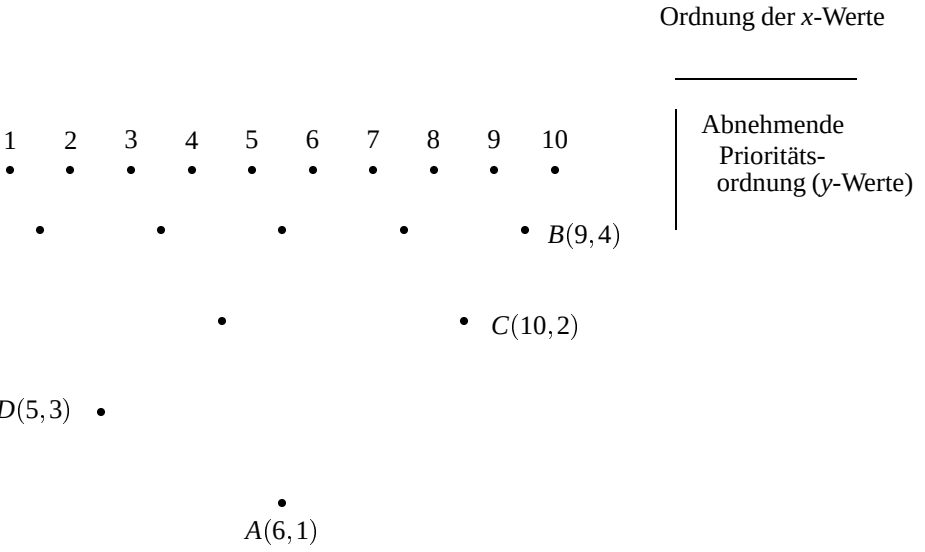


Abbildung 7.20

Wir haben die Punkte natürlich stets so nah wie möglich bei der Wurzel gespeichert. Wollen wir in diesen Prioritäts-Suchbaum als weiteren Punkt etwa den Punkt $E = (8, 1)$ einfügen, können wir so vorgehen: Wir folgen dem Suchpfad von der Wurzel zum Blatt mit Wert 8. Auf diesem Pfad muß der Punkt E abgelegt werden und zwar so, daß die y -Koordinaten aller unterwegs angetroffenen Punkte höchstens zunehmen. Daher legen wir den Punkt E an der Stelle ab, an der zuvor der Knoten C stand. Nun fahren wir mit $C = (10, 2)$ statt E fort und folgen dem Suchpfad zum Blatt 10. Dabei treffen wir auf den Knoten $B = (9, 4)$ und sehen, daß wir C dort ablegen müssen. Schließlich wird B beim Blatt 9 abgelegt.

Wir haben in den zur Veranschaulichung benutzten Figuren die Suchstruktur von Prioritätsbäumen nicht explizit deutlich gemacht, sondern vielmehr stillschweigend angenommen, daß an den inneren Knoten eines Prioritäts-Suchbaumes stets geeignete Wegweiser stehen, die eine Suche nach einem mit einem bestimmten x -Wert bezeichneten Blatt dirigieren. Eine Möglichkeit ist, das Maximum der Werte im linken Teilbaum zu nehmen. Wir wollen diesen Wert den die Suche dirigierenden *Splitwert* eines Knotens p nennen und mit $p.sv$ bezeichnen. Zur Vereinfachung nehmen wir ferner an, daß kein x -Wert eines Punktes doppelt auftritt. (Ist diese Voraussetzung für eine gegebene Menge von Punkten nicht erfüllt, so betrachte man statt einer Menge von Punkten (x, y) die Menge der Punkte $((x, y), y)$, wobei die erste Koordinate lexikographisch, also zuerst nach x , dann nach y geordnet ist.)

Jeder Knoten p eines Prioritäts-Suchbaumes kann höchstens einen Punkt speichern, den wir mit $p.Punkt$ bezeichnen. $p.Punkt$ kann undefiniert sein. Ist $p.Punkt$ definiert, sind $p.Punkt.x$ und $p.Punkt.y$ die Koordinaten des am Knoten p gespeicherten Punktes $p.Punkt$.

Wir beschreiben jetzt zunächst das leere Skelett eines Prioritäts-Suchbaumes zur Speicherung einer Menge von N Punkten $\{(x_1, y_1), \dots, (x_N, y_N)\}$: Es besteht aus einem vollständigen, binären Blattsuchbaum für die (nach Annahme paarweise verschiedenen) x -Werte $\{x_1, \dots, x_N\}$ der Punkte. Diese x -Werte sind die Splitwerte der Blätter in aufsteigender Reihenfolge von links nach rechts; die Punkt-Komponenten der Blätter sind undefiniert. Jeder innere Knoten des leeren Skeletts hat als Splitwert das Maximum der Splitwerte im linken Teilbaum; die Punktkomponenten sind ebenfalls undefiniert.

Das Verfahren zum (iterierten) Einfügen eines Punktes A aus der gegebenen Menge mit den Koordinaten $A.x$ und $A.y$ kann nun wie folgt formuliert werden:

```

procedure Einfügen ( $p$  : Knoten;  $A$  : Punkt);
  {anfangs ist  $p$  die Wurzel des Skeletts}
  if  $p.Punkt$  ist undefiniert
  then { $A$  ablegen}  $p.Punkt := A$ 
  else
    if  $p.Punkt.y \leq A.y$ 
    then {Suchpfad nach  $A.x$  folgen}
      begin
        if  $p.sv \geq A.x$ 
        then Einfügen( $p_l, A$ )
        else Einfügen( $p_r, A$ )
      end
    else { $p.Punkt.y > A.y$ }

```

```
begin
  {A ablegen und mit p.Punkt weitermachen}
  hilf := p.Punkt;
  p.Punkt := A;
  Einfügen(p, hilf)
end
```

Betrachten wir als Beispiel eine Menge *M* von acht Punkten:

$$M = \{ (1, 3), (2, 4), (3, 7), (4, 2), (5, 1), (6, 6), (7, 5), (8, 4) \}$$

Nach Einfügen der ersten drei Punkte (1, 3), (2, 4), (3, 7) in das anfänglich leere Skelett erhält man den Prioritäts-Suchbaum von Abbildung 7.21. Dabei sind die Splitwerte jeweils in der oberen und die Punkte in der unteren Hälfte der Knoten dargestellt.

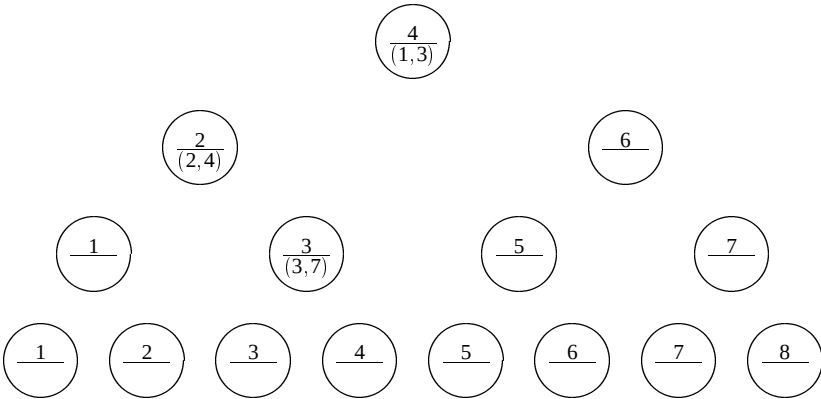


Abbildung 7.21

Einfügen des Punktes (4, 2) liefert den Baum von Abbildung 7.22.

Einfügen des Punktes (5, 1) liefert den Baum von Abbildung 7.23.

Einfügen der restlichen Punkte (6, 6), (7, 5), (8, 4) ergibt schließlich den Prioritäts-Suchbaum von Abbildung 7.24.

Wir hatten angenommen, daß nur Punkte aus der vorher bekannten Menge *M* mit paarweise verschiedenen *x*-Werten in das anfänglich leere Skelett eingefügt werden. Daher kann niemals der Fall eintreten, daß für einen dieser Punkte kein Platz auf dem Suchpfad von der Wurzel zu dem mit dem *x*-Wert des Punktes markierten Blatt ist: Spätestens in diesem Blatt findet der Punkt Platz.

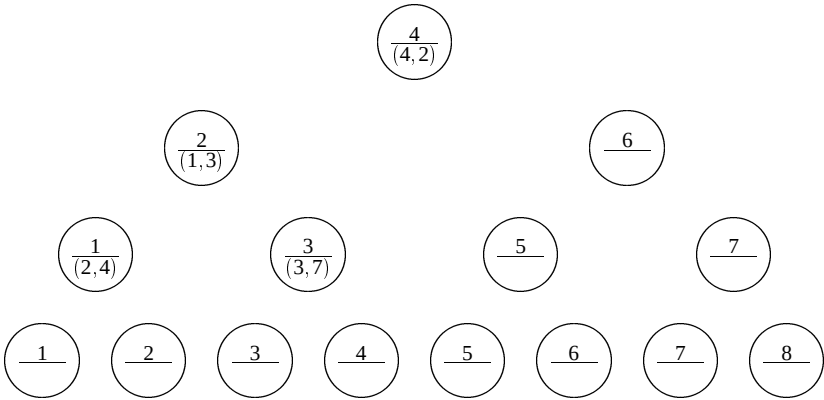


Abbildung 7.22

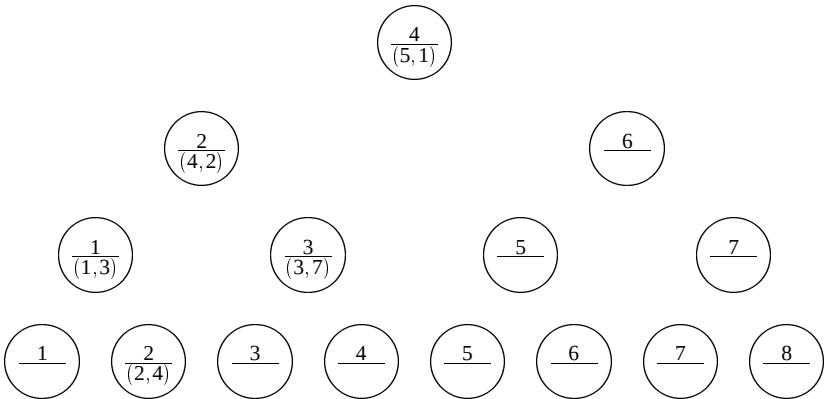


Abbildung 7.23

Um einen Punkt (x_0, y_0) aus dem Prioritäts-Suchbaum zu *entfernen*, sucht man zunächst den Knoten p mit $p.\text{Punkt} = (x_0, y_0)$. Die Suche wird allein durch den x -Wert des zu entfernenden Punktes und die Splitwerte der Knoten dirigiert. Hat höchstens einer der Söhne von p einen Punkt gespeichert, kann man diesen Punkt “hochziehen”, d. h. zur Punktkomponente von p machen und mit dem Sohn von p ebenso fortfahren, bis man bei den Blättern oder bei einem Knoten angelangt ist, der nur Söhne ohne Punktkomponenten hat. Haben beide Söhne von p einen Punkt gespeichert, ersetzt man $p.\text{Punkt}$ durch den Punkt mit dem kleineren y -Wert. Durch dieses Hochziehen entsteht dort eine Lücke, die auf dieselbe Weise geschlossen wird. Mit anderen Worten: Die durch das Entfernen eines Punktes im Innern des Prioritäts-Suchbaumes entstehende Lücke wird

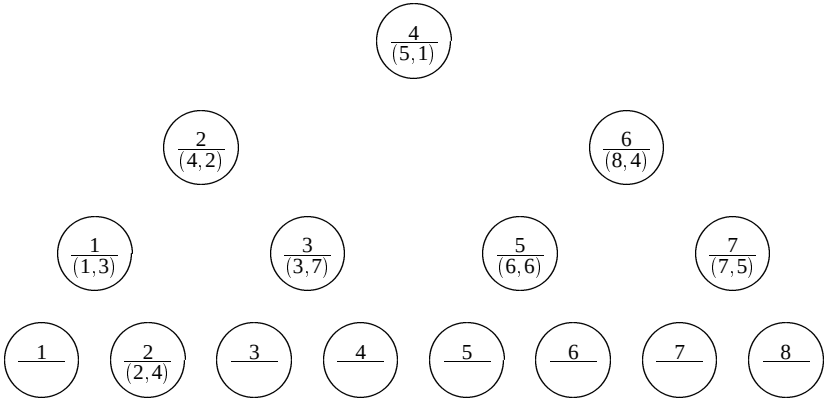


Abbildung 7.24

nach Art eines Ausscheidungskampfes unter den Punkten der Söhne geschlossen: Der Punkt mit dem jeweils kleineren y -Wert gewinnt und wird hochgezogen.

Das Verfahren zum Entfernen eines Punktes A kann damit wie folgt formuliert werden:

1. Schritt: {Suche nach einem Knoten p mit $p.Punkt = A$ }
 {anfangs ist p die Wurzel}

while ($p.Punkt$ ist definiert) **and** ($p.Punkt \neq A$) **do**

if $p.sv \geq A.x$

then $p := p_\lambda$

else $p := p_p$;

if $p.Punkt$ ist definiert

then { $p.Punkt = A$ } Schritt 2 ausführen

else A kommt nicht vor; {fertig}

2. Schritt: {Entfernen und nachfolgende Punkte hochziehen}

procedure Entfernen (p : Knoten);

 {anfangs ist $p.Punkt = A$ }

 entferne $p.Punkt$, d.h. setze $p.Punkt :=$ undefiniert;

 Fall 1: [$p_\lambda.Punkt$ ist definiert, und $p_p.Punkt$ ist definiert]

if $p_\lambda.Punkt.y < p_p.Punkt.y$

then

begin

$p.Punkt := p_\lambda.Punkt$;

 Entfernen(p_λ)

end

else

begin

$p.Punkt := p_p.Punkt$;

```

    Entfernen( $p_p$ )
  end;
Fall 2: [ $p_\lambda$ .Punkt ist definiert, aber  $p_p$ .Punkt nicht]
   $p$ .Punkt :=  $p_\lambda$ .Punkt;
  Entfernen( $p_\lambda$ );
Fall 3: [ $p_p$ .Punkt ist definiert, aber  $p_\lambda$ .Punkt nicht]
   $p$ .Punkt :=  $p_p$ .Punkt;
  Entfernen( $p_p$ );
Fall 4: [weder  $p_\lambda$ .Punkt noch  $p_p$ .Punkt ist definiert]
  {Hochziehen beendet} fertig!

```

Entfernt man beispielsweise aus dem letzten Baum im angegebenen Beispiel den Punkt (5, 1), müssen nacheinander die Punkte (4, 2), (1, 3) und (2, 4) hochgezogen werden. Man erhält den Baum von Abbildung 7.25.

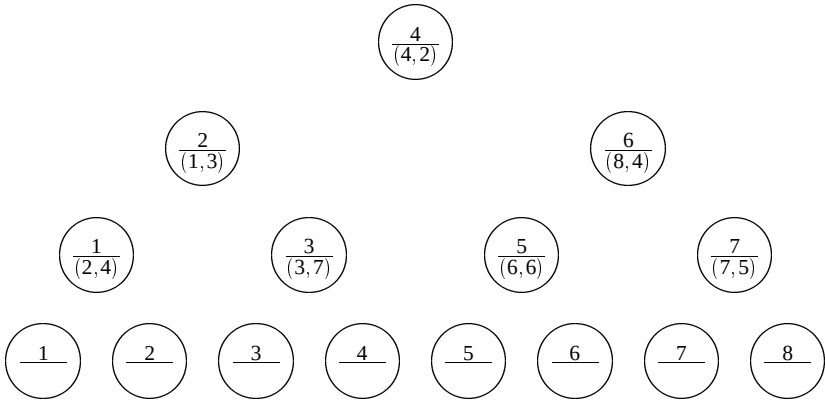


Abbildung 7.25

Es dürfte damit unmittelbar klar sein, daß das Einfügen und Entfernen von Punkten aus der ursprünglich gegebenen Menge von N Punkten stets in $O(\log N)$ Schritten möglich ist. Denn das Skelett des Prioritäts-Suchbaumes hat eine durch $\lceil \log_2 N \rceil$ beschränkte Höhe.

Wir überlegen uns nun, wie man alle in einem Prioritäts-Suchbaum gespeicherten Punkte (x, y) findet, deren x -Koordinaten in einem Bereich $[x_l, x_r]$ liegen und deren y -Koordinaten unterhalb eines Schwellenwertes y_0 bleiben.

Weil jeder Prioritäts-Suchbaum ein Suchbaum für die x -Werte ist, kann man den Bereich der Knoten mit zulässigen x -Werten von Punkten leicht eingrenzen. Unter diesen befinden sich die Knoten mit einem zulässigen y -Wert in einem Präfix des Baumes, d. h. sobald man auf einem Pfad von der Wurzel zu einem Blatt auf einen Punkt mit y -Wert $> y_0$ stößt, kann man die Ausgabe an dieser Stelle abbrechen. Grob vereinfacht

kann der Bereich der zulässigen Punkte wie in Abbildung 7.26 angegeben dargestellt werden.

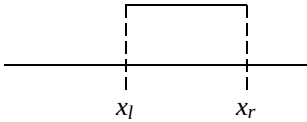


Abbildung 7.26

Jedem Knoten des Skeletts des Prioritäts-Suchbaumes kann man ein Intervall möglicher x -Werte eines an dem Knoten gespeicherten Punktes zuordnen. An der Wurzel ist das Intervall der gesamte zulässige x -Bereich, an den Blättern besteht er nur noch aus dem jeweiligen Splitwert. Um die Punkte zu bestimmen, deren x -Werte im vorgegebenen Bereich $[x_l, x_r]$ liegen, muß man höchstens die Knoten inspizieren, deren zugehörige Intervalle einen nichtleeren Durchschnitt mit dem Intervall $[x_l, x_r]$ haben. Das zeigt Abbildung 7.27.

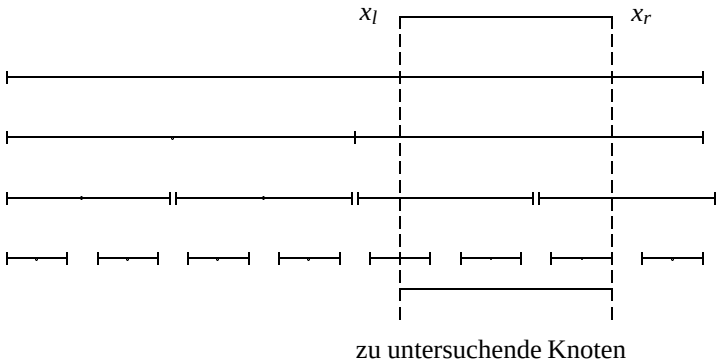


Abbildung 7.27

Den Bereich der höchstens in Frage kommenden Knoten kann man so abgrenzen: Man benutzt den Prioritäts-Suchbaum als Suchbaum für die Grenzen x_l und x_r des gegebenen Intervalls $[x_l, x_r]$. Alle Knoten auf den Suchpfaden von der Wurzel nach x_l bzw. x_r sowie sämtliche Knoten im Baum, die rechts vom Suchpfad nach x_l und links vom Suchpfad nach x_r liegen, können Punkte speichern, deren x -Wert in das gegebene Intervall fällt.

Unter diesen Knoten müssen diejenigen bestimmt werden, die einen Punkt mit y -Wert $\leq y_0$ gespeichert haben. Da die y -Werte von Punkten auf jedem Pfad von der Wurzel zu den Blättern zunehmen, kann man die gesuchten Punkte berichten in einer Anzahl von Schritten, die proportional zur Höhe des Skeletts und zur Anzahl k der berichteten Punkte ist, d.h. in $O(\log N + k)$ Schritten.

Kehren wir zurück zum Ausgangsproblem, die Menge aller Paare sich schneidender Rechtecke in einer Menge von N gegebenen Rechtecken in der Ebene zu bestimmen: Dieses Problem kann mit Hilfe des Scan-line-Verfahrens und Prioritäts-Suchbäumen zur Verwaltung der jeweils gerade aktiven Intervalle, d.h. der Schnitte der Scan-line mit den Rechtecken, in Zeit $O(N \log N + k)$ und Platz $O(N)$ gelöst werden. Dabei ist k die Anzahl der zu berichtenden Paare.

Für eine praktische Implementation mag es wünschenswert sein, anstelle einer Skelettstruktur der Größe $\Theta(N)$ während des Hinüberschwenkens der Scan-line über die Eingabe eine volldynamische Struktur zu verwenden, deren Größe sich der Anzahl der jeweils gerade aktiven Rechtecke anpaßt. Wie im Falle von Segment-Bäumen und Intervall-Bäumen kann man auch im Falle von Prioritätssuchbäumen eine auf einer geeigneten Variante von balancierten Bäumen gegründete, volldynamische Variante von Prioritätssuchbäumen entwerfen, die das Einfügen und Entfernen eines Intervalls in $O(\log n)$ Schritten erlaubt, wenn n die Anzahl der gerade gespeicherten Intervalle ist, und die es erlaubt, alle k Punkte in einem S-gegründeten Bereich in Zeit $O(\log n + k)$ zu berichten. Man vergleiche hierzu [1].

Wir skizzieren hier, wie man eine volldynamische Variante von Prioritäts-Suchbäumen erhält, die analog zu natürlichen Suchbäumen (random trees) zu einer gegebenen Folge von Punkten gebildet werden können und damit das Einfügen und Entfernen eines Punktes im Mittel in $O(\log n)$ Schritten erlauben. An Stelle des starren Skeletts verwenden wir als Suchstruktur einen natürlichen und damit von der Reihenfolge der Punkte abhängigen Blattsuchbaum. D.h. das Einfügen eines Punktes $A = (A.x, A.y)$ in den anfangs leeren Baum, der aus einem einzigen Knoten mit einem fiktiven Splitwert ∞ besteht, geschieht in zwei Phasen.

1. Phase: Suchbaum-Erweiterung

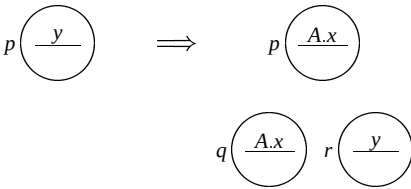
In dem bisher erzeugten Blattsuchbaum wird ein neues Blatt mit (Split-)Wert $A.x$ erzeugt und zwar so, daß im entstehenden Blattsuchbaum die x -Werte aller bisher eingefügten Punkte als Splitwerte der Blätter in aufsteigend sortierter Reihenfolge erscheinen und an jedem inneren Knoten als Splitwert stets das Maximum der Splitwerte im linken Teilbaum steht. Wir beginnen mit einer Suche im bisherigen Baum nach $A.x$:

```

p := Wurzel;
while (p ist kein Blatt) do
  if A.x ≤ p.sv
    then p := pλ
    else p := pρ

```

So findet man ein Blatt p mit Splitwert $p.sv$. Anfangs gilt trivialerweise $A.x \leq p.sv$. Wir sorgen dafür, daß diese Bedingung stets erhalten bleibt, indem wir p durch einen Knoten mit zwei Söhnen q und r ersetzen: Der linke Sohn q erhält als Splitwert $A.x$, der rechte als Splitwert $p.sv$ und p den neuen Splitwert $A.x$:



Ein eventuell bei p gespeicherter Punkt bleibt dort. Es gibt dann zu jedem Splitwert x genau ein Blatt mit Splitwert x und einen inneren Knoten auf dem Suchpfad zu diesem Blatt, der ebenfalls x als Splitwert hat.

2. Phase: Ablegen des Punktes A

Der Punkt A wird seinem y -Wert $A.y$ entsprechend auf dem Suchpfad von der Wurzel zum Blatt mit Splitwert $A.x$ so nah wie möglich an der Wurzel abgelegt. Diese Phase unterscheidet sich überhaupt nicht von dem für die Skelett-Variante von Prioritäts-Suchbäumen erklärten Einfügeverfahren.

Beispiel: Es sollen der Reihe nach die Punkte

$$(6, 4), (7, 3), (2, 2), (4, 6), (1, 5), (3, 9), (5, 1)$$

in den anfangs leeren Baum eingefügt werden. Einfügen des ersten Punktes $(6, 4)$ liefert den Baum von Abbildung 7.28.

Einfügen von $(7, 3)$ liefert den Baum von Abbildung 7.29. Zum Einfügen des nächsten Punktes $(2, 2)$ wird zunächst der unterliegende Suchbaum erweitert. Man erhält den Baum von Abbildung 7.30.

Ablegen des Punktes $(2, 2)$ verdrängt den Punkt $(7, 3)$ von der Wurzel und liefert den Baum von Abbildung 7.31.

Fügt man die restlichen Punkte auf dieselbe Weise ein, so erhält man schließlich den Prioritäts-Suchbaum von Abbildung 7.32.

Das Entfernen eines Punktes A verläuft umgekehrt zum Einfügen: Man sucht zunächst mit Hilfe des x -Wertes $A.x$ einen Knoten p , an dem A abgelegt wurde. Die durch das Entfernen dieses Punktes entstehende Lücke schließt man durch (iteriertes) Hochziehen von Punkten wie im Falle der Skelettstruktur. Man muß jetzt noch die unterliegende Suchbaumstruktur um ein Blatt mit Splitwert $A.x$ und einen inneren Knoten mit

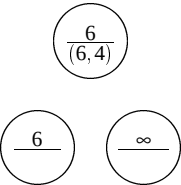


Abbildung 7.28

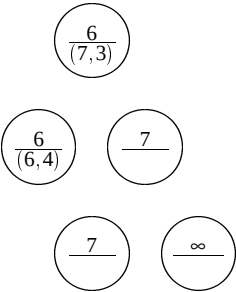


Abbildung 7.29

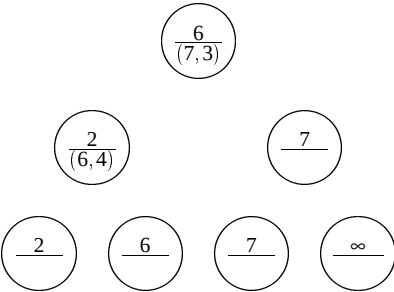


Abbildung 7.30

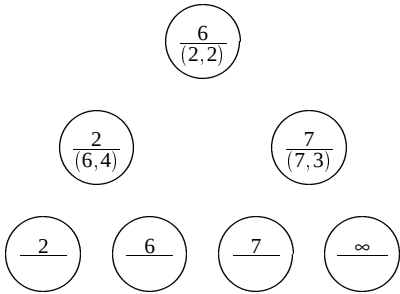


Abbildung 7.31

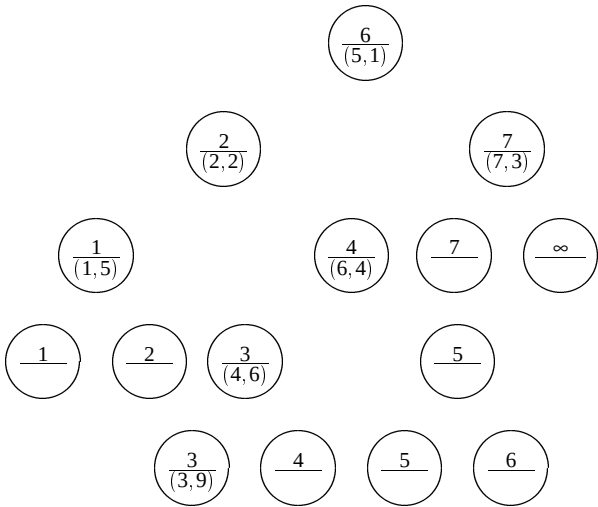


Abbildung 7.32

gleichem Splitwert verkleinern. Das geschieht wie folgt: Man sucht nach dem zu entfernenden Blatt p mit Splitwert $A.x$; unterwegs trifft man bei dieser Suche auch auf den inneren Knoten q mit Splitwert $A.x$. Zwei Fälle sind möglich:

Fall 1: [p ist rechter Sohn seines Vaters, vgl. Abbildung 7.33]
Dann muß der Splitwert des Vaters ϕp von p der symmetrische Vorgänger von $A.x$ sein. Man kann also ϕp durch den linken Teilbaum von ϕp ersetzen und den Splitwert $A.x$ von q durch den Splitwert y von ϕp ersetzen, ohne daß dadurch Suchpfade nach anderen x -Werten, die von $A.x$ verschieden sind, beeinflußt werden. Bei p kann höchstens der Punkt A abgelegt gewesen sein, den wir ja entfernt haben. Ein eventuell bei ϕp abgelegter Punkt B muß seinem y -Wert entsprechend in den linken Teilbaum von ϕp hinunterwandern. Dort ist Platz! Denn es gibt dort ein Blatt mit Splitwert $B.x$.

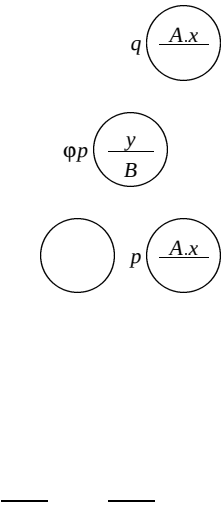


Abbildung 7.33

Fall 2: [p ist linker Sohn seines Vaters, vgl. Abbildung 7.34]

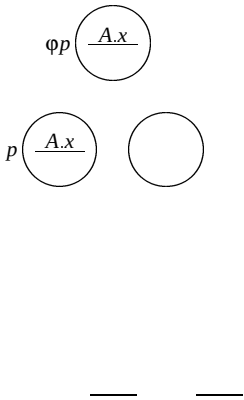


Abbildung 7.34

Dann muß der Vater ϕp von p ebenfalls $A.x$ als Splitwert haben; denn der Splitwert jedes inneren Knotens ist das jeweilige Maximum der Splitwerte im linken Teilbaum. Ersetzt man also (p und) ϕp durch den rechten Teilbaum von ϕp , wird jede Suche nach einem im rechten Teilbaum von ϕp stehenden x -Wert nach wie vor richtig gelenkt.

Einen eventuell bei ϕp abgelegten Punkt B muß man in den rechten Teilbaum von ϕp hinunter wandern lassen.

Verfolgen wir als Beispiel das Entfernen des Punktes $(5, 1)$ aus dem zuletzt erhaltenen Baum auf den vorhergehenden Seiten: Der Punkt ist an der Wurzel abgelegt. Die nach dem Entfernen entstehende Lücke wird zunächst durch Hochziehen der Punkte $(2, 2)$, $(6, 4)$, $(4, 6)$ und $(3, 9)$ geschlossen. Dann werden das Blatt und der innere Knoten mit Splitwert 5 entfernt und man erhält den Baum von Abbildung 7.35. Entfernen des Punktes $(6, 4)$ ergibt den Baum von Abbildung 7.36 (vgl. Fall 1).

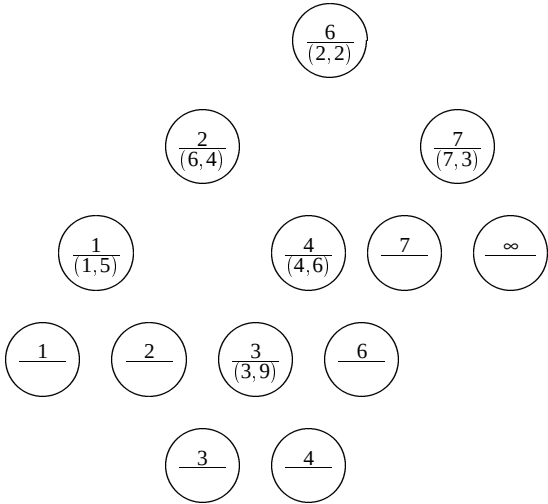
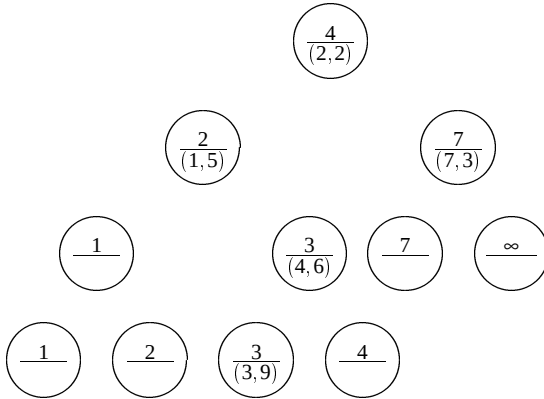


Abbildung 7.35

7.5 Das Zickzack-Paradigma

Wir haben bisher in erster Linie Probleme diskutiert, die Mengen iso-orientierter Objekte in der Ebene betrafen. In der Tat ist dieser Fall besonders gründlich untersucht worden mit dem Ergebnis, daß zahlreiche effiziente oder sogar optimale Algorithmen für diesen Fall gefunden wurden, vgl. [1]. In Wirklichkeit hat man es aber häufig mit Mengen beliebig orientierter Objekte in d -dimensionalen Raum, $d \geq 2$ zu tun. Beispiele sind Mengen beliebig orientierter Liniensegmente in der Ebene und Polygone oder polygonal begrenzte Flächen im dreidimensionalen Raum. Wir wollen in diesem Abschnitt der Frage nachgehen, ob und gegebenenfalls unter welchen Bedingungen sich

**Abbildung 7.36**

ein Verfahren zur Lösung eines algorithmischen Problems für Mengen iso-orientierter Objekte verallgemeinern läßt zu einem Verfahren zur Lösung des entsprechenden Problems für Mengen beliebig orientierter Objekte. Dabei möchte man natürlich möglichst wenig an Effizienz einbüßen. Wir werden zeigen, daß das für eine große Klasse von Verfahren möglich ist, genauer: für solche Verfahren, die dem Scan-line-Prinzip folgen und von halbdynamischen Skelettstrukturen Gebrauch machen, wie wir sie in Abschnitt 7.4 vorgestellt haben. Wir erläutern das Prinzip des Übertragens eines Verfahrens vom iso-orientierten auf den allgemeinen Fall am Beispiel des Schnittpunkts für Polygone. Das ist folgendes Problem: Gegeben sei eine Menge von p Polygonen mit insgesamt N Kanten in der Ebene. Gesucht sind alle Paare sich schneidender Polygone. Zwei Polygone schneiden sich, wenn sich entweder zwei Polygonkanten dieser Polygone schneiden oder das eine Polygon das andere vollständig einschließt. Wir lassen nur einfach geschlossene Polygone zu. Die Polygone können, müssen aber natürlich nicht konvex sein. Im Falle, daß alle Polygone konvex sind, kann die Lösung des Polygonschnittproblems vereinfacht werden. Die im folgenden skizzierte Lösung des Polygonschnittproblems kann leicht auf den Fall ausgedehnt werden, daß die gegebenen Polygone nicht sämtlich einfach geschlossene Polygone sind, sondern von allgemeinerer Art sind, d. h. z. B. Löcher enthalten. Jedes einfach geschlossene Polygon kann man sich gegeben denken als Folge seiner in Umlaufrichtung angeordneten Eckpunkte. Durchläuft man die Eckpunkte in dieser Reihenfolge, kehrt man zum Ausgangspunkt zurück; das Innere des Polygons soll dabei stets rechts liegen. Wir wollen das Polygonschnittproblem ähnlich wie das Rechteckschnittproblem lösen, indem wir dem Scan-line-Prinzip folgen und eine horizontale Scan-line von oben nach unten über die Menge der gegebenen Polygone hinwagschwenken, vgl. Abbildung 7.37.

Dabei merken wir uns wie im Fall des Rechteckschnittproblems die Schnitte der Polygone mit der Scan-line als eindimensionale Intervalle in einer dynamisch veränderlichen Datenstruktur. Was sind die Unterschiede zwischen dem iso-orientierten und diesem allgemeineren Fall?

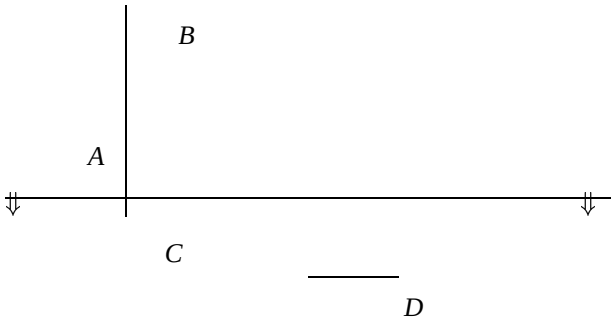


Abbildung 7.37

Zunächst bemerken wir, daß ein Polygon mehr Kantenschnitte mit der Scan-line haben kann, z.B. vier wie das Polygon *B* in Abbildung 7.37, und nicht nur zwei wie im Falle von Rechtecken. Die Anzahl der Schnitte kann die Größenordnung $\Omega(N)$ erreichen, wenn N die Gesamtzahl der Kanten ist. Allerdings kann die Scan-line höchstens zwei Kanten eines konvexen Polygons schneiden. In jedem Fall werden die Polygone durch wachsende und schrumpfende Intervalle auf der Scan-line repräsentiert. Das ist der zweite Unterschied zum iso-orientierten Fall. Trifft dort die Scan-line den oberen Rand eines Rechtecks, so wird das durch seinen linken und rechten Rand gegebene Intervall in die Menge der aktiven Intervalle aufgenommen und bleibt darin unverändert, bis die Scan-line den unteren Rechteckrand erreicht. Im Falle eines Polygons hingegen wachsen und schrumpfen diese Intervalle. Schließlich ist nicht zu sehen, wie man ein diskretes Raster finden könnte, das als Grundlage zum Bau einer halbdynamischen Skelettstruktur dienen könnte, das also Platz für alle beim Hinunterschwenken der Scan-line auftretenden Intervalle bietet. Dieser zuletzt genannte Unterschied ist der entscheidende. Denn das Wachsen und Schrumpfen von Intervallen, die die Schnitte der Scan-line mit den Polygonen bilden, kann man einfach ignorieren, solange man die jeweils korrekte, relative Anordnung der Intervallgrenzen aufrecht erhalten kann. Ferner kann man die Schnitte eines jeden Polygons mit der Scan-line in einem dem Polygon zugeordneten (balancierten) Suchbaum speichern (vgl. weiter unten). Es bleibt damit das Hauptproblem, einen Ersatz für das im iso-orientierten Fall offensichtlich vorhandene diskrete Raster zu finden, auf das man eine Skelettstruktur gründen kann. Im Falle des Rechteckschnittproblems wird nämlich das Raster von der Menge aller linken und rechten Rechteckseiten gebildet: Das ist eine angeordnete Menge von diskreten Punkten auf der x -Achse derart, daß jedes im Verlauf eines Scans von oben nach unten in der Vertikalstruktur abzuspeichernde Intervall ein Intervall über diesem Punktraster ist. Die Polygonkanten bilden dagegen eine Menge beliebig orientierter Liniensegmente

in der Ebene, die nicht in ähnlicher Weise eine Rasterung der x -Achse induzieren. Wie wir bereits bei der Lösung des allgemeinen Segmentschnittproblems im Abschnitt 7.2.3 gesehen haben, kann man auch nicht erwarten, daß die Polygonkanten in eine für den ganzen Scan feste Reihenfolge gebracht werden können, die für die jeweils von der Scan-line geschnittenen Kanten mit der Von-links-nach-rechts-Reihenfolge der Schnittpunkte längs der Scan-line übereinstimmt. Man wird höchstens eine lokal gültige Anordnung verlangen können, die an jedem Schnittpunkt zweier Kanten verändert werden muß.

Wir suchen also eine *Anfangsanordnung* der die Polygonkanten bildenden Menge von Liniensegmenten. Diese Anfangs-Anordnung liefert das zu Beginn des Scans *lokal gültige Raster*. Das lokal gültige Raster wird an jedem Schnittpunkt zweier Kanten dadurch verändert, daß die am Schnitt beteiligten Kanten ihre Plätze tauschen. Das lokal gültige Raster ist unser Ersatz für das im iso-orientierten Fall *global* gültige Raster der linken und rechten Rechteckseiten. Wir werden also jedes Polygon durch ein oder mehrere Intervalle über dem lokal gültigen Raster repräsentieren ebenso, wie wir im iso-orientierten Fall jedes Rechteck durch ein Intervall über dem globalen Raster der linken und rechten Rechteckseiten repräsentiert haben. Das einzige Problem besteht darin, eine geeignete Anfangsanordnung zu finden, mit der wir den Scan von oben nach unten beginnen können.

Wir können dieses Problem präziser formulieren, wenn wir den Begriff der für einen Scan von oben nach unten geeigneten Anfangsanordnung einer gegebenen Menge von Liniensegmenten in der Ebene wie folgt definieren:

Eine totale Ordnung " $<$ " einer Menge von Liniensegmenten in der Ebene heißt für einen Scan von oben nach unten *geeignete Anfangsanordnung*, wenn folgender Algorithmus *hsweep* ausführbar und korrekt ist, d.h. die als Kommentare vermerkten Zusicherungen gelten an den angegebenen Stellen.

Algorithmus hsweep

```

S := Folge der Liniensegmente in Anordnung " $<$ ", alle
    Segmente als "nicht vorhanden" markiert;
Q := Folge der oberen Endpunkte, unteren Endpunkte und
    Schnittpunkte von Liniensegmenten in absteigender
    y-Reihenfolge;
while Q ist nicht leer do
    begin
        p := nächster Punkt von Q;
        case Art von p of
            p oberer Endpunkt eines Segments s :
                markiere s als "vorhanden" in S;
            p unterer Endpunkt eines Segments s :
                markiere s als "nicht vorhanden" in S;
            p Schnittpunkt zweier Segmente s und t :
                {s und t sind als "vorhanden" markiert und in S sind
                 keine zwischen s und t stehenden Segmente
                 als "vorhanden" markiert}
                ersetze S durch die Anordnung, die durch
                    Vertauschen von s und t in S entsteht
    end

```

```

end {case}
  {die Anordnung der als "vorhanden" markierten Elemente
   in  $S$  stimmt überein mit der Links-nach-rechts-Anordnung
   dieser Segmente längs einer unmittelbar unterhalb von  $p$ 
   verlaufenden horizontalen Geraden}
end {while}
{alle Elemente von  $S$  sind als "nicht vorhanden" markiert}
end {Algorithmus hswEEP}

```

Die Anordnung der Segmente in S zwischen je zwei aufeinanderfolgenden Schnittpunkten (aus Q) ist das zwischen diesen Punkten lokal gültige Raster: Es bietet Platz für alle zwischen diesen Punkten beginnenden Segmente durch Eintrag in das "Skelett" S in der richtigen Anordnung von links nach rechts. Das Finden einer für einen Scan von oben nach unten geeigneten Anfangsanordnung ist trivial, wenn die gegebene Menge nur aus vertikalen Liniensegmenten besteht: Die gesuchte Anordnung ist dann einfach die Anordnung der Segmente in aufsteigender x -Reihenfolge. Das ist der iso-orientierte Fall, in dem keine Schnittpunkte vorkommen und die Anfangsanordnung nicht mehr verändert wird.

Für eine beliebige, gegebene Menge von Liniensegmenten in der Ebene ist das Finden der für einen Scan von oben nach unten geeigneten Anfangsanordnung nicht so leicht. Betrachten wir folgendes Beispiel einer Menge aus vier Segmenten A, B, C, D wie in Abbildung 7.38.

Um zu prüfen, ob $A < B < D < C$ eine für einen Scan von oben nach unten geeignete Anfangsanordnung ist, verfolgen wir, wie sich S an den ersten sieben "Haltepunkten" aus Q verändert; dabei stellen wir fest, daß die Anordnung der in S als "vorhanden" markierten Elemente nicht mit der Von-links-nach-rechts-Reihenfolge übereinstimmt, sobald der obere Endpunkt von D angetroffen wurde. Die Ordnung $A < B < D < C$ ist also nicht die gesuchte Anfangsanordnung.

Um zu einer gegebenen Menge von Liniensegmenten in der Ebene die für einen Scan von oben nach unten geeignete Anfangsanordnung zu finden, genügt es allerdings, gewissermaßen die richtige Brille aufzusetzen. Wir fassen die Menge von sich möglicherweise schneidenden Segmenten auf als eine Menge von sich nicht schneidenden Zickzacks, die sich höchstens berühren können. Zu jedem Liniensegment s assoziieren wir ein Zickzack z_s wie folgt: z_s beginnt am obersten Punkt von s . Dann folgt man s in absteigender y -Richtung bis zum ersten Schnittpunkt mit einem Segment s' . Jetzt folgt man s' statt s bis zum nächsten Schnittpunkt usw., bis schließlich der unterste Punkt eines Segments erreicht ist. Dort endet das Zickzack z_s .

Abbildung 7.39 zeigt noch einmal die Menge von vier Segmenten A, B, C, D und die zugehörige Menge von vier Zickzacks z_A, z_B, z_C, z_D .

Zickzacks schneiden sich nicht. Sie berühren sich nur an den Schnittpunkten der Segmente. Zickzacks sind monoton in y -Richtung. Man kann sie sich als elastische, geknickte Bänder vorstellen: Zieht man sie an den Enden in y -Richtung straff, berühren sie sich schließlich gar nicht mehr und werden vertikal. Das veranschaulicht intuitiv, daß Zickzacks die Rolle spielen, die vertikale Segmente im iso-orientierten Fall spielen.

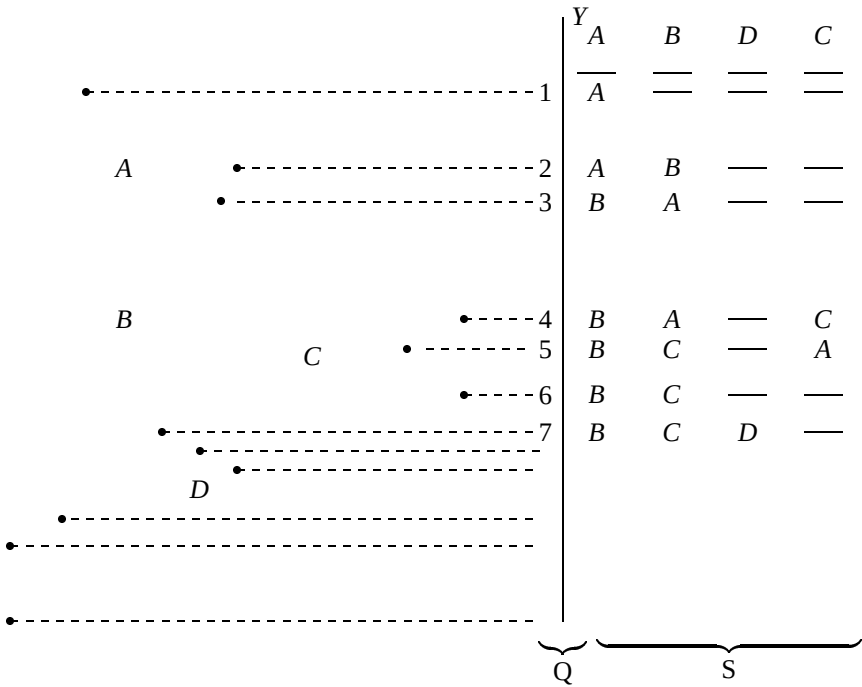


Abbildung 7.38

Zickzacks sind auf natürliche Art angeordnet durch eine vollständige Ordnung “ $\text{left}^+ | \text{above}$ ”. Seien z und z' zwei Zickzacks; dann gilt: $z \text{ left}^+ | \text{above } z'$ genau dann, wenn entweder eine horizontale Gerade l existiert, die z und z' schneidet und der Schnittpunkt $z \cap l$ links vom Schnittpunkt $z' \cap l$ liegt, oder z vollständig oberhalb von z' liegt, d. h. der untere Endpunkt von z oberhalb vom oberen Endpunkt von z' . Für das in Abbildung 7.39 angegebene Beispiel von vier Zickzacks ist die $\text{left}^+ | \text{above}$ -Ordnung z_A, z_D, z_B, z_C .

Die $\text{left}^+ | \text{above}$ -Ordnung der Zickzacks induziert eine Anordnung der ursprünglich gegebenen Menge von Liniensegmenten. Für das obige Beispiel ist es gerade die Anordnung A, D, B, C ; ganz allgemein ist die Zickzack-Ordnung zugleich die auf der Menge der Anfangsstücke induzierte Anordnung der Segmente. Es ist nun nicht überraschend, daß diese über die $\text{left}^+ | \text{above}$ -Ordnung von Zickzacks induzierte Anordnung von Liniensegmenten genau die gesuchte, für einen Scan von oben nach unten geeignete Anfangsanordnung von Liniensegmenten ist. Wir überlassen den Beweis dem Leser.

Man kann zeigen, daß für eine gegebene Menge von N Liniensegmenten in der Ebene mit k Schnittpunkten die $\text{left}^+ | \text{above}$ -Anordnung der zugehörigen Menge von Zickzacks und damit eine für einen Scan von oben nach unten geeignete Anfangsanordnung der Liniensegmente in Zeit $O((N + k) \log N)$ und Platz $O(N)$ berechnet werden kann (vgl. [1]).

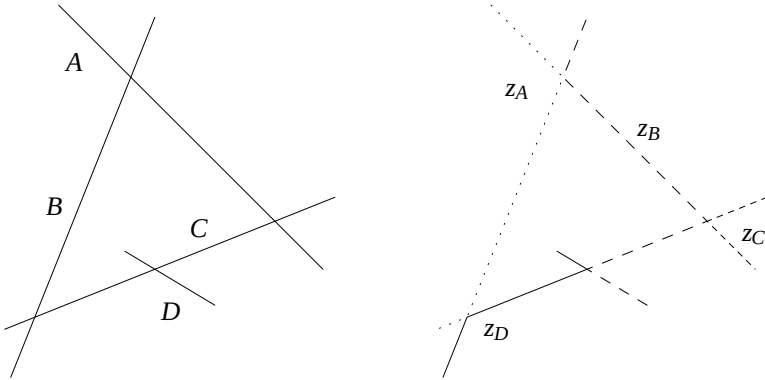


Abbildung 7.39

Wir kehren nun zum Ausgangsproblem zurück und zeigen, wie man das Polygon-schnittproblem nach dem Scan-line-Prinzip löst.

Das Verfahren kann ganz grob wie folgt beschrieben werden: Zunächst wird die gegebene Menge von Polygonen in eine (möglichst kleine) Menge von Zickzacks zerlegt. Für diese Zickzacks bestimmt man die $\text{left}^+|$ above-Ordnung; das liefert zugleich die für einen Scan von oben nach unten geeignete Anfangsanordnung der Polygonkanten. Man nimmt diese Anfangsanordnung als anfangs lokal gültiges Raster und baut darauf eine Skelettstruktur. In dieser Skelettstruktur speichert man die jeweils gerade aktiven Polygone als Intervalle über dem lokal gültigen Raster wie im iso-orientierten Fall. Im Unterschied zum iso-orientierten Fall muß man allerdings das lokal gültige Raster und damit die Skelettstruktur an allen Schittpunkten von Polygonkanten verändern.

Wir beschreiben nun die einzelnen Schritte genauer: Zuerst muß die gegebene Menge von Polygonen in Zickzacks zerlegt werden. Das könnte man natürlich so machen, daß man die Menge von Polygonen als Menge von Liniensegmenten auffaßt, die durch die Menge der Polygonkanten gegeben ist. Haben die Polygone insgesamt N Kanten, würde man also auch N Zickzacks bekommen. Man kommt jedoch im allgemeinen mit wesentlich weniger Zickzacks aus, da es nicht nötig ist, an einem “Knick”, d.h. an einem Punkt, an dem zwei Polygonkanten zusammenstoßen und dessen y -Wert nicht ein lokales Maximum ist, ein neues Zickzack zu beginnen. Es genügt, an jedem Punkt p mit lokal maximalem y -Wert ein neues Paar von Zickzacks zu beginnen. Wir wollen die an einem solchen Punkt zusammenstoßenden Kanten *Top-Segmente* des jeweiligen Polygons nennen. Abbildung 7.40 zeigt ein Beispiel.

Das Polygon aus Abbildung 7.40 kann in vier Zickzacks zerlegt werden: $a-f$, e , d , $b-c$. Je ein Paar beginnt an den Punkten q_1 und q_2 . Die von der $\text{left}^+|$ above-Ordnung der Zickzacks induzierte Ordnung der Top-Segmente ist a , e , d , b .

Da jedes konvexe Polygon genau eine Ecke mit maximalem y -Wert hat, zerfällt eine Menge von p konvexen Polygonen in genau $2p$ Zickzacks. Das Beispiel aus Abbildung 7.41 zeigt eine Menge von vier konvexen Polygonen A, B, C, D . In dieser Abbil-

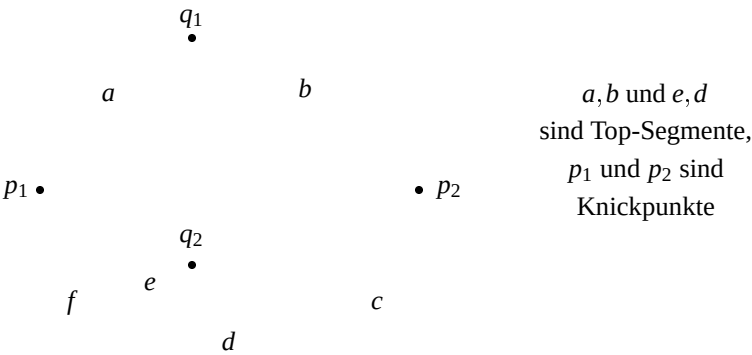


Abbildung 7.40

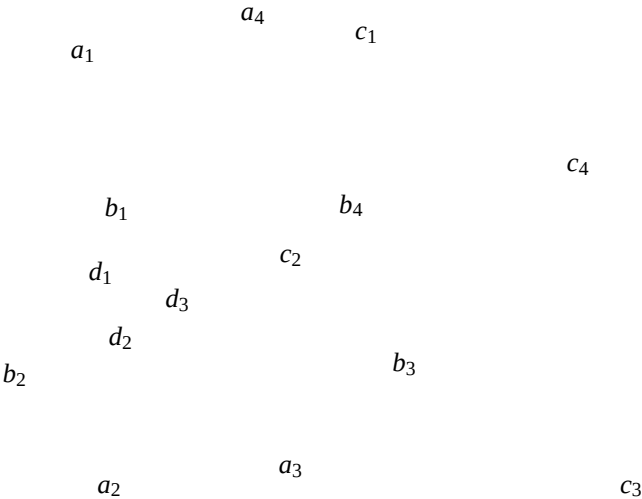


Abbildung 7.41

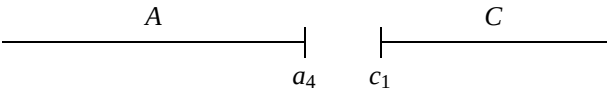
dung hat Polygon *A* die Kanten a_1, a_2, a_3, a_4 ; Polygon *B* hat die Kanten b_1, b_2, b_3, b_4 ; Polygon *C* hat die Kanten c_1, c_2, c_3, c_4 ; Polygon *D* hat die Kanten d_1, d_2, d_3 .

Die Menge dieser Polygone zerfällt in acht Zickzacks; die Anordnung dieser Zickzacks in left⁺ | above-Ordnung induziert die folgende, für einen Scan von oben nach unten geeignete Anfangsanordnung.

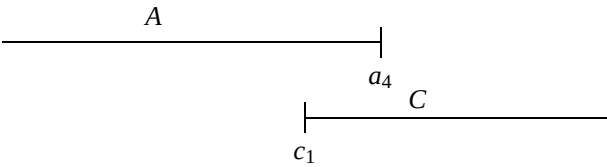
$$a_1, b_1, d_1, d_3, b_4, a_4, c_1, c_4.$$

Diese Anordnung liefert das zu Beginn des Scans von oben nach unten lokal gültige Raster: Die Polygone können als Intervalle über diesem Raster dargestellt werden.

Schwenkt man die Scan-line von oben nach unten über die Menge dieser Polygone, wird zuerst der oberste Punkt des Polygons *A* angetroffen. Das Polygon *A* wird als Intervall $[a_1, a_4]$ über dem Raster $a_1 \ b_1 \ d_1 \ d_3 \ b_4 \ a_4 \ c_1 \ c_4$ repräsentiert; d.h. das Intervall $[a_1, a_4]$ wird in die (anfangs leere) Menge der gerade aktiven Intervalle aufgenommen. Dann wird am zweiten Haltepunkt der Scan-line der oberste Punkt des Polygons *C* angetroffen; *C* kann als Intervall $[c_1, c_4]$ über demselben Raster repräsentiert werden. Der nächste Haltepunkt ist der oberste Punkt des Polygons *B*, das ebenfalls als Intervall $[b_1, b_4]$ über diesem Raster repräsentiert werden kann. Das Raster bleibt gültig, bis der erste Kantenschnittpunkt angetroffen wird. Das ist im obigen Beispiel der Schnittpunkt zwischen a_4 und c_1 . Die Situation unmittelbar oberhalb des Schnittpunktes



muß ersetzt werden durch die unmittelbar unterhalb des Schnittpunktes bis zum nächsten Haltepunkt gültige Situation:



Diese Änderung kann man so erreichen: Das das Polygon *A* repräsentierende Intervall mit rechtem Endpunkt a_4 und das das Polygon *C* repräsentierende Intervall mit linkem Endpunkt c_1 werden aus der Menge der aktiven Intervalle entfernt. Dann wird das lokal gültige Raster verändert, indem a_4 und c_1 ihre Plätze tauschen. Anschließend werden die *A* und *C* repräsentierenden Intervalle wieder in die Menge der aktiven Intervalle eingefügt. Für die ersten sieben Haltepunkte der Scan-line geben wir in Tabelle 7.1 das jeweils lokal gültige Raster und die jeweils aktive Menge von Intervallen zwischen den Haltepunkten an.

Jetzt besteht praktisch kein Unterschied mehr zwischen dem iso-orientierten Fall, also dem Rechteckschnittproblem, und dem allgemeinen Fall, also dem Polygonschnittproblem. Zu jedem Zeitpunkt wird die Menge der gerade aktiven, d.h. von der Scan-line geschnittenen Objekte (Rechtecke bzw. Polygone) repräsentiert durch eine dynamisch

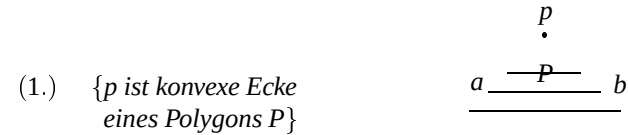
		a_1	b_1	d_1	d_3	b_4	a_4	c_1	c_4	Anfangsan- ordnung der Top-Segmente
(,1)	{									
(1,2)	{	a_1					a_4			
(2,3)	{	a_1					a_4	c_1	c_4	
(3,4)	{	a_1	b_1			b_4	a_4	c_1	c_4	
4		a_1	b_1	d_1	d_3	b_4	c_1	a_4	c_4	Kan- ten- schnitt
(4,5)	{	a_1	b_1			b_4	a_4	c_1	c_4	
5		a_1	b_1	d_1	d_3	c_1	b_4	a_4	c_4	Kan- ten- schnitt
(5,6)	{	a_1	b_1			b_4	a_4	c_1	c_4	
6		a_2	b_1	d_1	d_3	c_1	b_4	a_4	c_4	Knick
(6,7)	{	a_2	b_1			b_4	a_4	c_1	c_4	
7		a_2	b_1	d_1	d_3	c_1	b_4	a_3	c_4	Knick
$\vdots$										
Scan- line		aktive Intervalle								Ursache der Änderung des Rasters

Tabelle 7.1

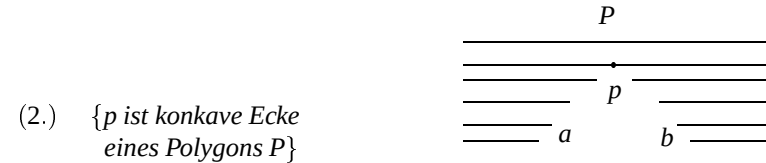
veränderliche Menge von Intervallen über einem jeweils lokal gültigen Raster. Die Überlappungsverhältnisse der jeweils aktiven Intervalle spiegeln genau die Schnittverhältnisse der von den Intervallen repräsentierten Polygone wieder. Die Intervalle kann man auch im nicht iso-orientierten Fall in ein anfangs leeres Skelett über dem jeweils gültigen Raster eintragen. Als Skelettstruktur kann man z.B. Segment- und Intervall-Bäume nehmen. Wir formulieren nun das Verfahren zur Bestimmung aller Schnitte in einer Menge von gegebenen Polygonen, indem wir den oben angegebenen Algorithmus *hsweep* um die für dieses Problem spezifischen Details ergänzen; wir lassen aber immer noch zahlreiche Implementationsdetails offen.

Wir nehmen an, daß die Kantenschnitte schon berechnet, aber noch nicht berichtet sind. Die Kantenschnitte werden nämlich ohnehin benötigt, um die Zickzack-Zerlegung zu bestimmen. Man zerlegt die Menge der Polygone in Zickzacks, bestimmt die Anfangsanordnung der Top-Segmente und baut eine anfangs leere Skelettstruktur *S* über diesem (lokalen) Raster.

```
Algorithmus Polygonschnitt
{berechnet zu einer Menge von Polygonen mit insgesamt N Kanten und
 k Kantenschnitten in der Ebene die Menge aller Paare von sich
 schneidenden Polygonen}
Q := Menge der oberen Endpunkte, unteren Endpunkte und Schittpunkte
      von Polygonkanten in abnehmender y-Reihenfolge;
while Q ist nicht leer do
  begin
    p := nächster Punkt von Q;
    case Art von p of
```

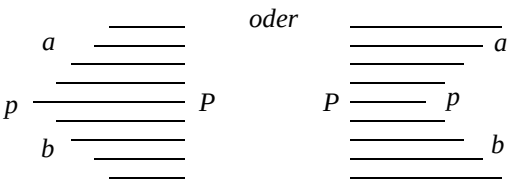


füge das *P* repräsentierende Intervall [*a*,*b*] in *S* ein;
bestimme jedes Intervall [*a'*,*b'*] aus *S*, das ein Polygon *Q* repräsentiert
und den Punkt *p* (bzw. eine der Kanten *a* oder *b*) enthält und
berichte das Paar (*P*,*Q*); {dies ist eine Aufspieß-Anfrage}



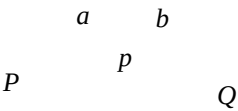
bestimme die *a* unmittelbar vorangehende Kante *a'* und die *b* unmittelbar
nachfolgende Kante *b'* von *P* in *x*-Richtung;
{oberhalb von *p* wird *P* durch das Intervall [*a'*,*b'*] repräsentiert}
entferne [*a'*,*b'*] aus *S* und füge [*a'*,*a*] und [*b*,*b'*] in *S* ein;

(3.) {p ist Knick}



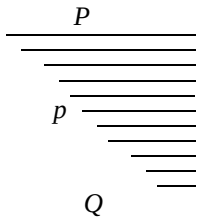
ersetze im lokalen Raster, also im Skelett von S , a durch b und ersetze alle Intervalle mit rechtem bzw. linkem Rand a durch solche mit rechtem bzw. linkem Rand b ;

(4.1) {p ist Schnittpunkt zweier Kanten a und b }

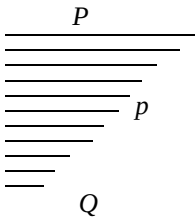


bestimme die a unmittelbar vorangehende Kante a' von P und die b unmittelbar nachfolgende Kante b' von Q in x -Richtung;
{oberhalb von p wird P durch $[a', a]$ und Q durch $[b, b']$ repräsentiert}
entferne $[a', a]$ und $[b, b']$ aus S ;
vertausche im lokalen Raster, also im Skelett von S , a und b ;
füge $[a', a]$ und $[b, b']$ wieder ein in S ;
berichte das Paar (P, Q) ;

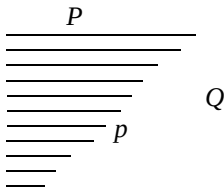
{Fälle (4.2) – (4.4) werden analog zu Fall (4.1) behandelt}



(4.2)



(4.3)



(4.4)

end {case}
end {while}
end {Algorithmus Polygonschnitt}

Wie kann man die an einer bestimmten Halteposition einer Kante eines Polygons P unmittelbar vorangehende bzw. unmittelbar nachfolgende Kante in x -Richtung bestimmen? Dazu merkt man sich zu jedem Polygon die jeweils gerade aktiven Kanten in Von-links-nach-rechts-Reihenfolge längs der Scan-line in einem P zugeordneten, balancierten Suchbaum. Da wir insgesamt nur N Kanten haben, können alle Bäume zusammen niemals mehr als $O(N)$ Platz beanspruchen. Das Einfügen und Entfernen von Kanten und das Bestimmen von unmittelbaren Vorgängern und Nachfolgern ist stets in $O(\log N)$ Schritten ausführbar.

Die Komplexität des Verfahrens zur Lösung des Polygonschnittproblems hängt jetzt davon ab, wie die Skelettstruktur S implementiert wird. Es ist offensichtlich, daß wir für S Analoga zu Segment- und Intervall-Bäumen bauen können. Der einzige Unterschied zu den entsprechenden Strukturen im iso-orientierten Fall besteht darin, daß wir von Zeit zu Zeit lokale Änderungen im Skelett vornehmen müssen. Die Größe bleibt dabei allerdings stets unverändert. Das Ersetzen eines Rasterpunktes durch einen neuen wie im Fall (3.) und das Vertauschen zweier Rasterpunkte, wie im Fall (4.), des oben angegebenen Algorithmus ist aber in jedem Fall in $O(\log N)$ Schritten möglich, da die Größe des Skeletts stets durch $O(N)$ beschränkt bleibt; im Falle konvexer Polygone sogar durch die Anzahl dieser Polygone. Die übrigen Operationen, nämlich das Einfügen und Entfernen von Intervallen und das Beantworten von Aufspieß-Anfragen, benötigen dieselbe Schrittzahl wie für gewöhnliche Segment- und Intervall-Bäume. Zählt man noch die Anzahl der Schritte hinzu, die für die Bestimmung der für den Scan von oben nach unten geeigneten Anfangsanordnung der Top-Segmente der gegebenen Polygone erforderlich ist, erhält man:

Für eine gegebene Menge von Polygonen mit insgesamt N Kanten und k Kantschnitten kann man alle r Paare sich schneidender Polygone berichten in Zeit $O((N + k + r) \log N)$. Der benötigte Speicherplatz ist von der Größenordnung $O(N \log N)$, falls Analoga zu Segment-Bäumen verwendet werden, und $O(N)$, falls Analoga zu Intervall-Bäumen verwendet werden. In beiden Fällen dürfen allerdings die k Schnittpunkte nicht explizit gespeichert werden, wie wir es bei der Formulierung des oben angegebenen Verfahrens angenommen haben; vielmehr muß man sie im Verlaufe des Verfahrens noch einmal mitberechnen. Will man das nicht, ist der Speicherbedarf $O(N \log N + k)$ bzw. $O(N + k)$.

Das Verfahren zur Lösung des Polygonschnittproblems läßt sich verhältnismäßig leicht ausbauen, um ein Grundproblem der graphischen Datenverarbeitung zu lösen, das sogenannte *Hidden-Line-Eliminationsproblem*. Nehmen wir an, eine Menge polygonal begrenzter, ebener Flächen im dreidimensionalen Raum sei gegeben. Wir möchten wissen, welche Kanten sichtbar sind, wenn man aus dem Unendlichen von oben auf diese Flächen blickt. Das ist eine anschauliche Formulierung des Problems, die verdeckten Kanten einer dreidimensionalen Szene bei orthographischer Parallelprojektion zu bestimmen.

Wir nehmen natürlich an, daß die polygonal begrenzten, ebenen Flächen sich nicht gegenseitig durchdringen können und nicht durchsichtig sind. Ist die Papierebene die Projektionsebene, könnte ein Betrachter beispielsweise die in Abbildung 7.42 dargestellte Szene sehen.

Zur Lösung dieses Problems kann man so vorgehen: Wir schwenken eine horizontale Scan-line über die in die Betrachtungsebene projizierte zweidimensionale Szene. D.h. wir haben eine Menge von Polygonen in der Ebene wie im Falle des Polygonschnittpro-

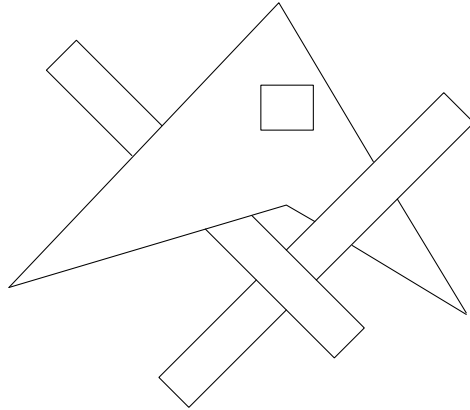


Abbildung 7.42

blems. Anders als beim Polygonschnittproblem merken wir uns jetzt aber zu jedem ein Polygon repräsentierenden, gerade aktiven Intervall dessen relative Distanz zum Betrachter. Für jede Position der Scan-line gilt: Eine Kante ist sichtbar genau dann, wenn sie ein Intervall begrenzt, das unter allen gerade aktiven Intervallen, die diese Kante enthalten, die geringste Distanz zum Betrachter hat. Anstelle von Aufspieß-Anfragen, die beim Polygonschnittproblem ausgeführt werden, um Inklusionen zu entdecken, müssen jetzt also Sichtbarkeitstests durchgeführt werden an allen Stellen, an denen sich die Sichtbarkeitsverhältnisse von Kanten ändern können. Das sind die Anfänge und Enden von Kanten und die Schnittpunkte zwischen je zwei Kanten. Verwendet man Segment-Bäume zur Speicherung der jeweils gerade aktiven Intervalle, kann man die an den Kanten des Skeletts stehenden Intervalle als nach Distanz zum Betrachter sortierte Listen organisieren. Dann ist das Einfügen und Entfernen von Intervallen in $O(\log^2 N)$ Schritten möglich. Ein Sichtbarkeitstest kann in $O(\log N)$ Schritten ausgeführt werden. Man durchläuft wie bei Aufspieß-Anfragen einen Suchpfad im Skelett des Segment-Baumes von der Wurzel zu dem Blatt, das der auf Sichtbarkeit zu prüfenden Kante entspricht, und inspiziert auf diesem Suchpfad jeweils nur ein Element der nach Distanz geordneten Intervall-Listen: Das Element mit der jeweils geringsten Distanz zum Betrachter. Am Ende weiß man dann, ob die auf Sichtbarkeit zu prüfende Kante ein Intervall begrenzt, das unter allen die Kante enthaltenden Intervallen, die gerade aktiv sind, die geringste Distanz zum Betrachter hat, also sichtbar ist, oder nicht.

Es ist zwar möglich, auch Intervall-Bäume zur Speicherung der jeweils gerade aktiven Intervalle zu nehmen; jedoch sind Sichtbarkeitstests dann nicht so einfach durchzuführen wie im Falle von Segment-Bäumen mit nach (relativer) Distanz zum Betrachter geordneten Intervall-Listen. Für weitere Einzelheiten verweisen wir auf [1].

7.6 Anwendungen geometrischer Datenstrukturen

Segment-Bäume und Intervall-Bäume sind Strukturen zur Speicherung von eindimensionalen Intervallen; Prioritäts-Suchbäume dienen zur Speicherung von Punkten in der Ebene. Wir haben diese Strukturen im Abschnitt 7.4 als *halbdynamische* Skelettstrukturen eingeführt: Man kann Objekte, d.h. Intervalle oder Punkte, eines festen Universums einfügen und entfernen und kann darüberhinaus bestimmte geometrische Anfragen effizient beantworten. Alle drei Strukturen lassen sich zur Lösung des Rechteckschnittproblems nach dem Scan-line-Prinzip benutzen. Wir wollen in diesem Abschnitt einige weitere Beispiele für die vielfältigen Anwendungsmöglichkeiten dieser Strukturen angeben. Im Abschnitt 7.6.1 lösen wir einen sehr einfachen Spezialfall des Hidden-Line-Eliminationsproblems (HLE). Dieser Spezialfall ist dadurch charakterisiert, daß alle Flächen in der gegebenen Szene iso-orientiert und parallel zur Projektionsebene sind. Man erhält für diesen Spezialfall des HLE-Problems eine Lösung, deren Komplexität von der Größe der Eingabe und der Größe der Ausgabe, d.h. der Anzahl der sichtbaren Kanten, nicht aber von der Anzahl der Kantenschnitte in der Projektion abhängt.

Im Abschnitt 7.6.2 diskutieren wir ein Suchproblem für Punktmengen in der Ebene mit Fenstern fester Größe. Als Fenster erlauben wir ein beliebiges Rechteck, das in der Ebene verschoben werden kann. Wir zeigen, daß Varianten von Prioritäts-Suchbäumen eine zur Speicherung der Punkte geeignete Struktur sind, die folgende Operationen unterstützt: Das Einfügen und Entfernen von Punkten und das Aufzählen aller Punkte, die in das Fenster bei einer gegebenen Lage fallen.

7.6.1 Ein Spezialfall des HLE-Problems

Eine dreidimensionale Szene kann man sich gegeben denken durch eine Menge undurchsichtiger, sich gegenseitig nicht durchdringender, polygonal begrenzter ebener Flächen im Raum. Wir wollen eine solche dreidimensionale Szene auf eine zweidimensionale Betrachtungsebene projizieren und die in der Projektion sichtbaren Kanten berechnen. Dazu setzen wir die orthographische Projektion voraus, d.h. wir setzen parallele, etwa senkrecht von oben kommende Projektionsstrahlen (Licht) voraus. Dies ist eine durchaus übliche Annahme. Wir machen jedoch eine weitere, sehr spezielle und in der Praxis wohl nur selten realisierte Annahme: Alle Flächen sollen rechteckig, iso-orientiert und parallel zur Projektionsebene sein. Ein aus $z = \infty$ auf die x - y -Ebene schauender Betrachter könnte also zum Beispiel das in Abbildung 7.43 gezeigte Bild sehen, wenn die x - y -Projektionsebene die Papierebene ist.

In diesem Fall kann man die sichtbaren Kanten der als undurchsichtig vorausgesetzten Flächen wie folgt bestimmen  Man baut die sichtbare Kontur der Flächen von vorn nach hinten auf: Begonnen wird mit der Fläche mit größtem z -Wert, da diese dem Betrachter am nächsten liegt. Von ihr sind alle Kanten sichtbar. Dann geht man die Flächen in der Reihenfolge wachsender Distanz zum Betrachter, also mit abnehmenden z -Werten, der Reihe nach durch. Jedesmal, wenn man dabei auf eine neue Fläche

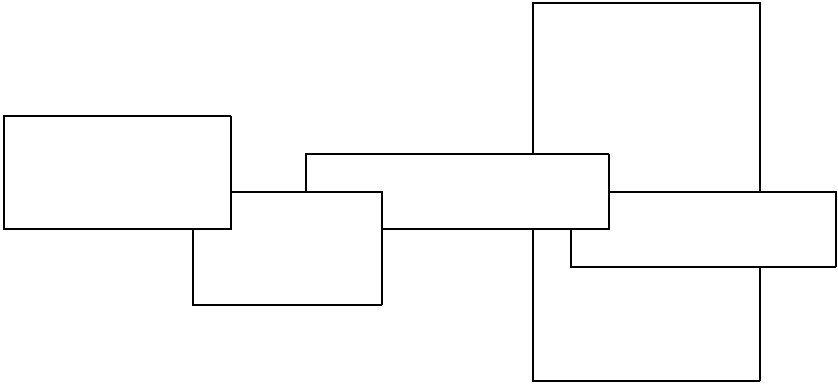


Abbildung 7.43

trifft, wird die Kontur des nunmehr sichtbaren Gebietes entsprechend aktualisiert, vgl. Abbildung 7.44.

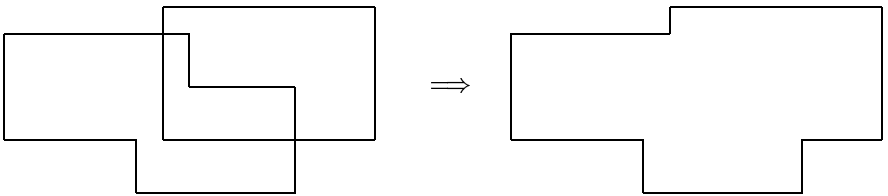


Abbildung 7.44

Es kommt also darauf an, die Menge der Rechtecke und ihre (sichtbare) Kontur so zu speichern, daß die oben angegebene Veränderung der Kontur effizient berechnet werden kann.

Wir verwenden dazu zwei Mengen E und F : E ist die Menge der Kanten der Kontur des bis zum jeweiligen z -Wert sichtbaren Gebietes; F ist eine Menge von Rechtecken, deren Vereinigung E als Kontur hat.

Man initialisiert E und F zunächst als leere Menge, sortiert die gegebene Menge R iso-orientierter und zur Projektionsebene paralleler Rechtecke nach abnehmenden z -Werten, also nach wachsender Distanz zum Betrachter, und geht dann wie folgt vor:

```
while noch nicht alle Rechtecke betrachtet do
  begin
    nimm nächstes Rechteck  $r \in R$ ;
```

- (1) *bestimme alle Schnitte zwischen Seiten von r und Kanten der Kontur;*
 - (1a) *für jede Kante $e \in E$, die von einer Seite von r geschnitten wird, berechne die außerhalb von r liegenden {sichtbaren!} Teile der neuen Kontur, füge sie in E ein und entferne e aus E ;*
 - (1b) *für jede Kante e' von r , die eine Kante der Kontur schneidet, berechne die außerhalb der Kontur liegenden Teile von e' , berichte diese Teile als sichtbar und füge sie in E ein;*
 - (2) *für jede Kante e' von r , die keine Kante der Kontur schneidet, stelle (mit Hilfe von F) fest, ob sie ganz innerhalb von E liegt (also unsichtbar ist) oder nicht;*
if e' ist nicht innerhalb E
then berichte e' als sichtbar und füge sie in E ein;
 - (3) *bestimme alle Kanten von E , die ganz innerhalb r liegen und entferne sie aus E ;*
 - (4) *füge r in F ein*
- end** {while}

Falls das nächste Rechteck r ganz innerhalb der aktuellen Kontur E liegt, bleibt E also unverändert, und es wird nichts berichtet. Falls das nächste Rechteck r das Gebiet mit Kontur E ganz einschließt, so wird r zur Kontur des neuen sichtbaren Gebietes. Die Kanten von r werden im Schritt (2) des Algorithmus als sichtbar berichtet und als Ergebnis von Schritt (2) und (3) wird die bisherige Kontur E durch die Kanten von r als neuer Kontur ersetzt.

Im allgemeinen wird das nächste Rechteck r einige Kanten der (alten) Kontur E schneiden, wie im in Abbildung 7.44 gezeigten Beispiel. In Abbildung 7.45 haben wir die Kanten mit den Nummern der Schritte markiert, in denen sie nach dem oben angegebenen Algorithmus betrachtet werden.

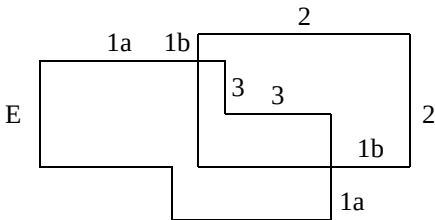


Abbildung 7.45

Die Frage, ob Kanten von E innerhalb von r liegen, wird gestellt, nachdem eventuelle Kantenschnitte zwischen E und r bereits behandelt wurden. Daher kann der Test, ob eine Kante von E innerhalb von r liegt, ersetzt werden durch einen Test, ob je ein Punkt einer Kante von E innerhalb r liegt. Aus demselben Grunde lässt sich auch die Frage,

ob eine Kante von r innerhalb oder außerhalb von E liegt, auf die entsprechende Frage für einen (eine Kante repräsentierenden) Punkt reduzieren.

Insgesamt folgt, daß es für eine Implementation des oben angegebenen Verfahrens genügt, folgende drei Teilprobleme zu lösen:

1. Ein *Segmentschnitt-Suchproblem*: Für eine gegebene Menge S horizontaler (vertikaler) Segmente und ein gegebenes vertikales (horizontales) Segment l , finde alle Segmente in S , die l schneidet.
2. Ein *zweidimensionales Aufspieß-Problem* (oder: eine zweidimensionale inverse Bereichsanfrage): Für eine gegebene Menge R von Rechtecken und einen gegebenen Punkt p , stelle fest, ob p in $\bigcup R$ liegt.
3. Eine *zweidimensionale Bereichsanfrage*: Für eine gegebene Menge P von Punkten und ein gegebenes Rechteck r , finde alle Punkte von P , die innerhalb r liegen.

Die Teilprobleme 2 und 3 treten im Schritt (2) und (3) des Algorithmus auf, nachdem das Teilproblem 1 im Schritt (1) behandelt wurde. In jedem Fall werden dynamische Lösungen für die drei Teilprobleme benötigt, weil im Verlaufe des Verfahrens Objekte in die jeweiligen Mengen eingefügt oder aus ihnen entfernt werden.

Wir skizzieren mögliche Lösungen für die drei Teilprobleme:

Zur Lösung des Segmentschnitt-Suchproblems für eine Menge horizontaler Segmente kann man *Segment-range-Bäume* verwenden. Das sind Segment-Bäume, deren Knotenlisten als Bereichs-Suchbäume organisiert sind und damit Bereichsanfragen unterstützen. Genauer: Man baut einen Segment-Baum als halbdynamische Skelettstruktur, die allen im Verlauf des Verfahrens angetroffenen horizontalen Segmenten Platz bietet. Die an den Knoten des Skeletts stehenden Listen von Intervallnamen werden als Bereichs-Suchbäume, d. h. z. B. als balancierte Blattsuchbäume mit doppelt verketteten Blättern organisiert, so daß Bereichsanfragen für vertikale Intervalle beantwortet werden können in einer Anzahl von Schritten, die proportional zum Logarithmus der Anzahl der Intervalle in der jeweiligen Liste und zur Anzahl der zu berichtenden Intervalle ist. In Abbildung 7.46 haben wir diese Struktur an Hand eines einfachen Beispiels veranschaulicht, indem wir die zweistufige, hierarchische Struktur in der Ebene ausgebreitet haben und an Stelle von Bereichs-Suchbäumen einfach vertikal angeordnete Intervall-Listen dargestellt haben.

Werden Segment-range-Bäume wie oben angegeben implementiert, so können die benötigten Operationen wie folgt ausgeführt werden:

Zum Einfügen eines neuen horizontalen Segments H bestimmt man die $\log N$ Knoten des Skeletts, in deren Knotenliste H eingefügt werden muß. Jede Knotenliste ist ein vertikal geordneter, balancierter Blattsuchbaum mit höchstens N Elementen. Daher kann H in eine einzelne Knotenliste in $\log N$ Schritten und insgesamt in $O(\log^2 N)$ Schritten in einen Segment-range-Baum eingefügt werden. Das Entfernen eines horizontalen Segments verläuft genau umgekehrt und kann ebenfalls in $O(\log^2 N)$ Schritten ausgeführt werden. Da ein Segment-Baum zur Speicherung von N horizontalen Segmenten in sämtlichen Knotenlisten höchstens insgesamt $N \log N$ Elemente hat, hat natürlich auch ein Segment-range-Baum einen Speicherbedarf der Größe $O(N \log N)$. Um für ein gegebenes vertikales Segment l alle horizontalen Segmente zu finden, die l schneiden, benutzt man den x -Wert von l als Suchschlüssel für eine Suche im Segment-Baum und



berichtet für jeden Knoten auf dem Suchpfad nach x alle im Intervall I liegenden Segmente durch eine Bereichsanfrage im jeweiligen Bereichs-Suchbaum. Offensichtlich können auf diese Weise alle k horizontalen Segmente, die I schneiden, in $O(\log^2 N + k)$ Schritten bestimmt werden.

Zur Lösung des zweidimensionalen Aufspieß-Problems benutzen wir *Segment-Segment-Bäume*: Das ist wiederum eine hierarchische Struktur, die aus einem Segment-Baum besteht, dessen Knotenlisten ebenfalls als Segment-Bäume organisiert sind. Genauer: Die horizontalen Projektionen der Rechtecke (auf die x -Achse) werden in einem Segment-Baum gespeichert. Enthält die Liste der Projektionen an einem Knoten dieses Segment-Baumes die (Namen der) Rechtecke $R_1, \dots, R_t$, so werden die vertikalen Projektionen dieser Rechtecke (auf die y -Achse) ebenfalls in einem Segment-Baum gespeichert, der diesem Knoten zugeordnet ist. Dann kann man durch eine Suche im Segment-Baum für die horizontalen Rechteckprojektionen nach dem x -Wert eines gegebenen Punktes $p \in (x_0, y_0)$ die höchstens $\log N$ Knoten mit daranhängenden Segment-Bäumen bestimmen, die die vertikalen Projektionen sämtlicher Rechtecke enthalten, deren horizontale Projektion von x_0 aufgespießt wird. Unter diesen findet man in $O(\log N + k_i)$ Schritten je Segment-Baum S_i alle in S_i enthaltenen Rechtecke, deren vertikale Projektion von y_0 aufgespießt wird.

Insgesamt lassen sich also alle k Rechtecke, die p aufspießt, in Zeit $O(\log^2 N + k)$ finden. Es ist nicht nötig und aus Speicherplatzgründen auch nicht sinnvoll, die Segment-Bäume zur Speicherung der vertikalen Projektionen über dem Raster aller möglichen y -Werte von Rechtecken zu bauen. Vielmehr genügt es, zu jedem Knoten im Segment-Baum für die horizontalen Projektionen vorab alle die Rechtecke zu bestimmen, die jemals in die Knotenliste dieses Knotens aufgenommen werden müssen; dann genügt es, den Segment-Baum für die vertikalen Projektionen, der an diesem Knoten hängt, über dem von den vorab bestimmten Rechtecken induzierten Raster zu bauen. Dann bleibt der gesamte Speicherbedarf des Segment-Segment-Baumes in der Größenordnung $O(N \log^2 N)$ und der Zeitbedarf zum Aufbau des leeren Skeletts bei $O(N \log N)$.

Das letzte Teilproblem, nämlich das Beantworten zweidimensionaler Bereichsanfragen für eine durch Einfüge- und Entferne-Operationen veränderliche Menge von Punkten, ist auf vielfältige Weise lösbar. Es gehört zu den am gründlichsten untersuchten zweidimensionalen Suchproblemen überhaupt. Entsprechend vielfältig ist das Spektrum der zur Speicherung der Punkte geeigneten Datenstrukturen. Wir skizzieren hier kurz eine mögliche Lösung mit Hilfe von *Range-range-Bäumen*: Ein Range-range-Baum für eine dynamisch veränderliche Menge von Punkten über einem festen Universum von N möglichen Punkten hat große Ähnlichkeit mit einem Segment-Segment-Baum: Man baut zunächst einen halbdynamischen Bereichs-Suchbaum, der eindimensionale Bereichsanfragen, etwa für x -Bereiche unterstützt. Das Skelett eines halbdynamischen Bereichs-Suchbaums unterscheidet sich nicht wesentlich vom Skelett eines Segment-Baumes. Das Universum der möglichen x -Werte wird in elementare Fragmente eingeteilt und über dieser Menge wird ein vollständiger Binärbaum gebaut. Jeder (innere) Knoten repräsentiert dann ein Intervall auf der x -Achse, das genau aus der Folge der elementaren Fragmente besteht, die durch die Blätter des Teilbaumes des Knotens repräsentiert werden. Jeder Knoten enthält eine Liste von Punkten: In die Liste des Knotens p kommen genau die Punkte, die in das von p repräsentierte Intervall fallen. Man sieht leicht, daß jeder Punkt in höchstens $\log N$ Knotenlisten vorkommen kann. Die Liste der Wurzel enthält alle aktuell vorhandenen Punkte und die Blätter enthalten

jeweils höchstens einen Punkt. Nehmen wir an, es sollen alle Punkte bestimmt werden, die in einen gegebenen Bereich fallen. Dabei nehmen wir ohne Einschränkung an, daß der Bereich aus einer zusammenhängenden Folge von Elementarfragmenten besteht. Dann kann man in $\log N$ Schritten die Knoten finden, die den gegebenen Bereich im Skelett repräsentieren, d h. die am nächsten bei der Wurzel liegen und ein Intervall repräsentieren, das ganz im gegebenen Bereich liegt. Die Punkte in den zu diesen Knoten gehörenden Punktlisten sind genau die gesuchten. Abbildung 7.47 zeigt ein Beispiel einer Menge von neun Punkten $\{A, \dots, I\}$ über einem Universum von 16 möglichen x -Werten.

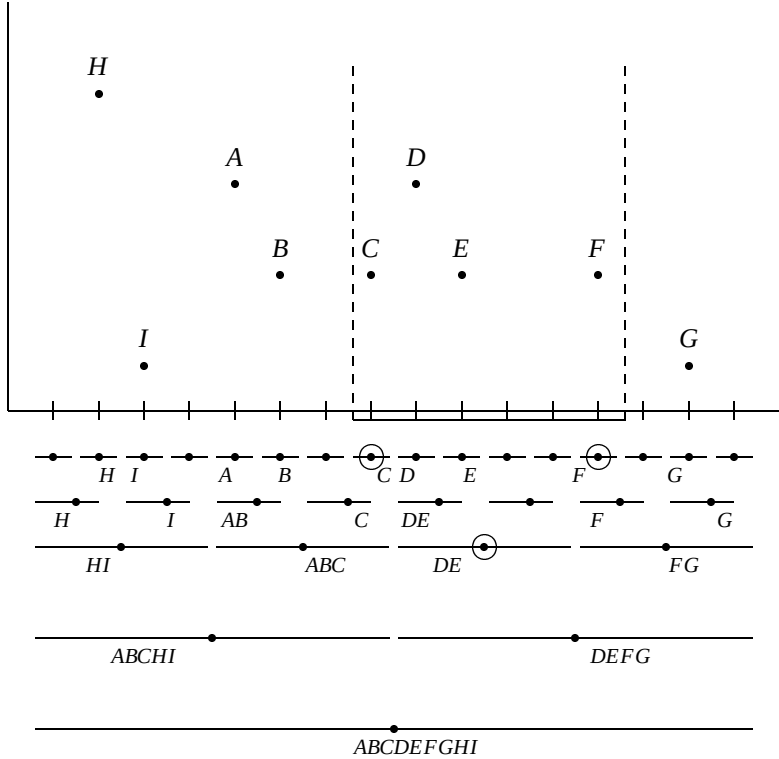


Abbildung 7.47

Der gegebene Bereich $[x_l, x_r]$ wird im Baum von Abbildung 7.47 durch die drei eingekreisten Knoten repräsentiert. Dort stehen genau die Punkte, deren x -Wert in den Bereich $[x_l, x_r]$ fällt. Zum Einfügen eines Punktes P sucht man im Baum nach P und fügt P in die Listen aller Knoten auf dem Suchpfad ein. Zum Entfernen eines Punktes P geht man umgekehrt vor, hat aber natürlich (wie bei Segment-Bäumen) das Problem, die Stellen innerhalb der Punktlisten zu finden, an denen P auftritt. Dieses Problem läßt

sich, wie bei Segment-Bäumen, mit Hilfe eines (globalen) Wörterbuches lösen. Insgesamt erhält man so eine Struktur mit folgenden Charakteristika:

Das Einfügen und Entfernen eines Punktes kann in $O(\log N)$ Schritten ausgeführt werden; für einen gegebenen eindimensionalen Bereich kann man alle k in den Bereich fallenden Punkte in Zeit $O(\log N + k)$ finden; der Platzbedarf ist von der Ordnung $O(N \log N)$.

Natürlich haben wir diese Struktur nicht entwickelt, nur um damit eindimensionale Bereichsanfragen beantworten zu können. (Dafür hätten wir auch balancierte Blattsuchbäume als volldynamische Struktur nehmen können.) Die soeben vorgestellten, analog zu Segment-Bäumen gebildeten halbdynamischen Bereichs-Suchbäume sind vielmehr geeignete Bausteine für hierarchisch aufgebaute Strukturen. Man kann auf ihrer Basis insbesondere Range-range-Bäume bauen, die zweidimensionale Bereichsanfragen unterstützen: Man organisiert die Punktlisten eines halbdynamischen Bereichs-Suchbaums, der Bereichsanfragen für x -Bereiche unterstützt, als halb- oder volldynamische Bereichs-Suchbäume, die Bereichsanfragen für y -Bereiche unterstützen.

In einer solchen Struktur lassen sich Punkte in $O(\log^2 N)$ Schritten einfügen und entfernen und alle k Punkte eines gegebenen zweidimensionalen Bereichs in $O(\log^2 N + k)$ Schritten aufzählen. Der Platzbedarf eines Range-range-Baums ist $O(N \log N)$, wenn die Bereichs-Suchbäume zur Unterstützung von Bereichsanfragen für y -Bereiche als volldynamische Bereichs-Suchbäume implementiert wurden.

Fassen wir noch einmal kurz zusammen, wie wir den zu Eingang dieses Abschnitts angegebenen Spezialfall des HLE-Problems lösen können: Wir gehen die Menge der gegebenen Rechtecke der Reihe nach mit wachsender Distanz vom Betrachter durch. Dabei merken wir uns die Kontur des jeweils sichtbaren Bereichs in einer Menge E horizontaler und vertikaler Liniensegmente, d. h. E wird als Paar von Segment-range-Bäumen, je ein Baum für die horizontalen und ein Baum für die vertikalen Kanten, repräsentiert. Weiter wird jede Kante von E durch einen Punkt repräsentiert und die Menge dieser Punkte in einem Range-range-Baum gespeichert. Schließlich wird eine Menge F von Rechtecken, deren Vereinigung die Kontur E hat, als Segment-Segment-Baum gespeichert. Wird dann ein neues Rechteck r angetroffen, so verändert man diese Strukturen wie im Algorithmus oben angegeben und gibt gegebenenfalls sichtbare Teile von Kanten von r aus. Es ist nicht schwer zu sehen, daß der insgesamt erforderliche Zeitaufwand von der Ordnung $O(N \log^2 N + q \cdot \log^2 N)$ ist, wenn q die Anzahl der sichtbaren Kanten und N die Anzahl der ursprünglich gegebenen Rechtecke ist.

7.6.2 Dynamische Bereichssuche mit einem festen Fenster

In diesem Abschnitt behandeln wir das Problem, für eine gegebene Menge von Punkten in der Ebene und einen gegebenen Bereich alle Punkte zu bestimmen, die in den Bereich fallen. Dieses Problem hat viele Varianten: Wir können annehmen, daß die Punktmenge fest, aber die Bereiche variabel sind. Die Bereiche können rechteckig, durch ein (konvexes) Polygon begrenzt oder kreisförmig sein. Man kann aber auch einen Bereich fester Größe und Gestalt annehmen, der wie ein Fenster über die Punktmenge verschoben werden kann. Man denke etwa an einen Bildschirm als Fenster, mit dem man auf eine Menge von Punkten blickt. Wir interessieren uns für diese Variante des Problems und

nehmen aber zusätzlich an, daß die Menge der Punkte nicht ein für allemal fest gegeben ist, sondern durch Einfügen und Entfernen von Punkten dynamisch verändert werden kann.

Wir setzen ein kartesisches x - y -Koordinatensystem in der Ebene voraus und bezeichnen die x - und y -Koordinaten eines Punktes a mit a_x und a_y , also $a = (a_x, a_y)$. Für zwei Punkte a und b sei $a + b = (a_x + b_x, a_y + b_y)$ und für eine Menge A von Punkten und einen Punkt q sei

$$A_q = A + q = \{(a_x + q_x, a_y + q_y) \mid a \in A\}.$$

Jetzt können wir das in diesem Abschnitt behandelte Problem präziser wie folgt formulieren: Sei P eine Menge von Punkten und sei W ein festes Fenster (z.B. ein Rechteck, Dreieck, konvexes Polygon, Kreis); für einen gegebenen Punkt q sollen folgende Operationen ausgeführt werden:

Einfügen(P, q): Fügt den Punkt q in die Menge P ein.

Entfernen(P, q): Entfernt den Punkt q aus der Menge P .

Window_W(P, q): Liefert alle Punkte in $P \cap W_q$.

Dann nennen wir eine Repräsentation von P zusammen mit Algorithmen zum Ausführen der Operationen *Einfügen*, *Entfernen*, *Window_W* eine Lösung des dynamischen Bereichssuchproblems mit Fenster W .

Wir behandeln den Fall, daß W ein Rechteck ist, das durch seinen linken, rechten, unteren und oberen Rand gegeben ist, also $W = (x_l, x_r, y_b, y_t)$. Das dynamische Bereichssuchproblem für ein rechteckiges Fenster W nennen wir auch kurz *DRW-Problem*. Wir zeigen, wie man das DRW-Problem mit (volldynamischen) Prioritäts-Suchbäumen löst.

Zur Lösung des DRW-Problems zerschneiden wir die euklidische Ebene in Gedanken in horizontale Streifen der Höhe $Y = \text{Höhe}(W) = y_t - y_b$. Wir nennen

$$s_i = \{p \mid iY \leq p_y < (i+1)Y\}$$

den i -ten Streifen. Wenn $p \in s_i$ ist, heißt i die Streifennummer von p ; sie wird mit $s(p)$ bezeichnet. Es ist klar, daß man für jeden Punkt p die Streifennummer $s(p)$ in konstanter Zeit berechnen kann.

Die Zerlegung der Ebene in Streifen der Höhe Y hat folgende wichtige Konsequenzen:

1. Für jede durch einen Punkt q gegebene Verschiebung W_q des Rechtecks W gilt: W_q schneidet höchstens zwei Streifen.
2. Für jeden Streifen s und jede durch einen Punkt q gegebene Verschiebung gilt entweder

(a) $W_q \cap s = \emptyset$ oder

(b) $W_q \cap s \neq \emptyset$ und $(W_q \cap s)$ ist S -gegründet in s oder $W_q \cap s$ ist N -gegründet in s .

Hier benutzen wir die bereits im Abschnitt 7.4.4 eingeführten Begriffe *S*-gegründet (für: Süd-gegründet) und *N*-gegründet (für: Nord-gegründet). Für einen Bereich R (ein Fenster) und einen Streifen s heißt R *S-gegründet* (bzw. *N-gegründet*) in s , wenn $R \cap s$ mit der orthogonalen Projektion von R auf die untere, also südliche (bzw. auf die obere, also nördliche) Begrenzung von s zusammenfällt. In dem in Abbildung 7.48 gezeigten Beispiel ist W_{q_2} *S-gegründet* in s_{i+1} und *N-gegründet* in s_i . Falls eine Verschiebung W_q von W sowohl *S*- als auch *N*-gegründet in einem Streifen s ist, muß offenbar $W_q \cap s = W_q$ sein. Abbildung 7.48 zeigt auch dafür ein Beispiel.

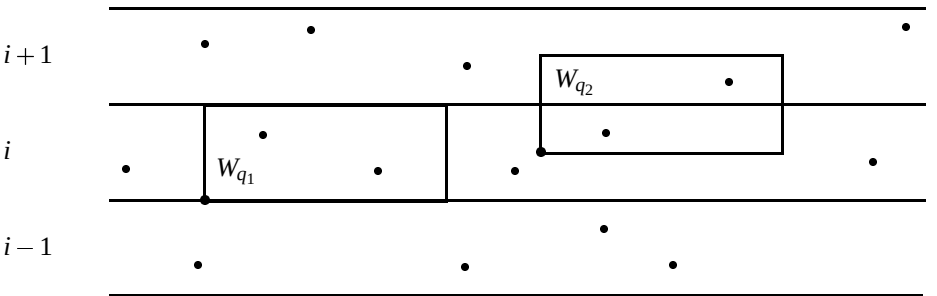


Abbildung 7.48

Die Idee zur Lösung des DRW-Problems ist nun, jedem Streifen s ein Paar von Prioritäts-Suchbäumen zuzuordnen, die die Punkte in s speichern. Ein Prioritäts-Suchbaum unterstützt *S*-gegründete Anfragen in s und der zweite Prioritäts-Suchbaum *N*-gegründete Anfragen in s . Natürlich können wir nicht annehmen, daß das Universum der in s fallenden Punkte im vorhinein bekannt und fest ist. Wir müssen also volldynamische Prioritäts-Suchbäume verwenden. Es kommen dafür nicht die in Abschnitt 7.4.4 als halbdynamische Skelettstruktur implementierten Prioritäts-Suchbäume, wohl aber die dort ebenfalls angegebene, analog zu natürlichen Suchbäumen entwickelte volldynamische Struktur in Frage. Sie erlaubt das Einfügen und Entfernen von Punkten im Mittel in logarithmischer Zeit und auch das Berichten aller k Punkte in einem *S*- (bzw. *N*-) gegründeten Bereich im Mittel in Zeit $O(\log N + k)$. Es ist bekannt, vgl. [1], daß Prioritäts-Suchbäume auch als balancierte Bäume gebaut werden können mit dem Ergebnis, daß die Operationen *Einfügen*, *Entfernen* und *Window_W* auch im schlechtesten Fall jeweils in $O(\log N)$ bzw. $O(\log N + k)$ Schritten ausführbar sind. Weil Prioritäts-Suchbäume (in jedem Fall) Blattsuchbäume für die x -Werte von Punkten in der Ebene sind, unterstützen sie Bereichsanfragen für x -Intervalle; weil Prioritäts-Suchbäume Heaps bzgl. der y -Werte von Punkten sind, erlauben sie es, alle Punkte zu berichten, deren y -Wert unterhalb eines gegebenen Schwellenwertes liegt. Beide Eigenschaften zusammen liefern gerade das, was wir brauchen: Um *S*-gegründete Anfragen beantworten zu können, speichern wir die Punkte eines Streifens s in einem s zugeordneten Prioritäts-Suchbaum derart, daß die Punkte mit kleinerem y -Wert näher bei der Wurzel stehen, d.h. die Prioritätsordnung ist die y -Ordnung. Um *N*-gegründete Anfragen beant-

worten zu können, speichern wir die Punkte mit größerem y -Wert näher bei der Wurzel, d. h. die Prioritätsordnung ist die negative y -Ordnung.

Ordnet man also jedem Streifen s ein Paar von volldynamischen Prioritäts-Suchbäumen zu, so kann man Punkte (im Streifen s) einfügen und entfernen, indem man diese Operationen in beiden s zugeordneten Prioritäts-Suchbäumen ausführt. Zur Beantwortung von S - bzw. N -gegründeten Bereichsanfragen konsultiert man jeweils nur einen Prioritäts-Suchbaum. Abbildung 7.49 veranschaulicht dies noch einmal.

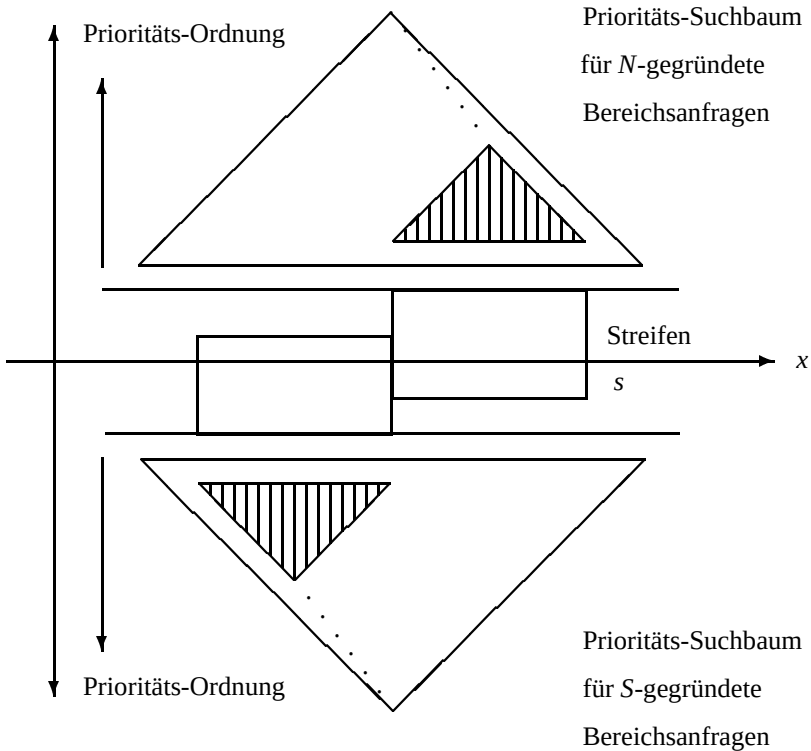


Abbildung 7.49

Um linearen Speicherplatz und einen Zeitbedarf von $O(\log N)$ bzw. $O(\log N + k)$ im Mittel bzw. im schlechtesten Fall in der Anzahl N der Punkte in P und der Anzahl k der bei einer $Window_W$ -Operation zu berichtenden Punkte zu erhalten, darf man allerdings leere Streifen nicht explizit repräsentieren. Deshalb speichert man die Streifennummern genau der nichtleeren Streifen in einem balancierten Suchbaum T_S : Jeder in T_S gespeicherten Streifennummer ordnen wir ein Paar von Prioritäts-Suchbäumen zu, die genau die Punkte, die im Streifen mit dieser Streifennummer liegen, enthalten. Damit kön-

nen die zur Lösung des DRW-Problems benötigten Operationen wie folgt ausgeführt werden.

Einfügen(P, q): Bestimme die Streifennummer $s(q)$ von q ; suche in T_S nach $s(q)$; wenn $s(q)$ in T_S bereits vorkommt, füge q in die beiden $s(q)$ zugeordneten Prioritäts-Suchbäume ein; andernfalls, d.h. wenn $s(q)$ nicht in T_S vorkommt, füge $s(q)$ in T_S ein, schaffe ein neues Paar von Prioritäts-Suchbäumen, die beide genau q speichern, und ordne dies Paar $s(q)$ zu.

Entfernen(P, q): Analog.

Window_W(P, q): Bestimme die Nummern der höchstens zwei nichtleeren Streifen, die W_q schneidet; suche in T_S nach diesen Nummern und benutze die den Nummern zugeordneten Prioritäts-Suchbäume, um die Punkte in den S - bzw. N -gegründeten Teilen von W_q zu berichten.

Mit demselben Zeitbedarf wie das DRW-Problem kann man auch das dynamische Bereichssuchproblem mit einem festen, dreieckigen Fenster lösen . Diese Lösung kann leicht auf den Fall ausgedehnt werden, daß das Fenster ein durch ein beliebiges, einfach geschlossenes Polygon begrenzter Bereich ist: Man zerlegt den Bereich in eine (feste, endliche) Anzahl von Dreiecken. Damit kann man die dynamische Bereichssuche mit polygonalem Fenster reduzieren auf eine feste Anzahl von dynamischen Bereichssuchen mit dreieckigem Fenster.

7.7 Distanzprobleme und ihre Lösung

Bei keinem der bisher betrachteten geometrischen Probleme hat die *Distanz* von Objekten eine Rolle gespielt. In diesem Abschnitt werden wir einige wichtige Probleme und Lösungen näher betrachten, bei denen die Distanz ein entscheidendes Kriterium ist. Wir beschränken uns auf die euklidische Ebene, also den $\mathbb{R}^2$ mit der (üblichen) Distanzfunktion $d(p_1, p_2)$, wobei $p_1 = (x_1, y_1)$ und $p_2 = (x_2, y_2)$ Punkte der Ebene sind:

$$d(p_1, p_2) := \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Anders ausgedrückt: Die Distanz zweier Punkte ist die Länge der geradlinigen Verbindung zwischen diesen Punkten. Man kann sich leicht davon überzeugen, daß dies tatsächlich eine Distanzfunktion ist, indem man die drei charakterisierenden Bedingungen überprüft:

- (1) Für alle $p_1, p_2 \in \mathbb{R}^2$ ist $d(p_1, p_2) = 0$ genau dann, wenn $p_1 = p_2$.
- (2) Für alle $p_1, p_2 \in \mathbb{R}^2$ ist $d(p_1, p_2) = d(p_2, p_1)$ (Symmetrie).
- (3) Für alle $p_1, p_2, p_3 \in \mathbb{R}^2$ ist $d(p_1, p_2) + d(p_2, p_3) \geq d(p_1, p_3)$ (Dreiecksungleichung).

Sehen wir uns nun einige Distanzprobleme näher an.

7.7.1 Distanzprobleme

Wir wollen im folgenden einige der bestuntersuchten Distanzprobleme betrachten; andere findet man bei [1], [2] und [3]. Für jedes Problem geben wir einen naiven Lösungsalgorithmus, die untere Schranke für die Zeitkomplexität der Lösung an.

Problem: *Dichtestes Punktepaar* (closest pair)

gegeben: Eine Menge P von N Punkten in der Ebene.

gesucht: Ein Paar p_1, p_2 von Punkten aus P mit minimaler Distanz.

Dieses Problem wird oft als eines der fundamentalen Probleme der algorithmischen Geometrie angesehen (vgl. [1]), weil es so einfach formuliert werden kann, zu einer effizienten Lösung aber bereits wichtige Prinzipien und Erkenntnisse erforderlich sind. Außerdem hat es auch vielerlei praktische Anwendungen. Sind etwa die Punkte (Projektionen der) Flugzeuge in der Nähe eines Flugplatzes, so sind die am nächsten benachbarten Punkte die Flugzeuge mit der größten Kollisionsgefahr (allerdings bewegen sich in diesem Fall die Punkte, vgl. [2]).

Ein naives Verfahren, das dichteste Punktepaar zu bestimmen, besteht offenbar darin, für jedes Punktepaar die Distanz zu berechnen und dann das Minimum der Distanzen ausfindig zu machen. Da es bei N Punkten $N(N-1)/2$ Punktepaare gibt, kostet dieses Verfahren $\Theta(N^2)$ Schritte.

Die entscheidende Frage ist jetzt, ob man die geometrische Information über die Lage der Punkte ausnutzen kann, um das Problem effizienter zu lösen. Im eindimensionalen Fall ist das ganz leicht. Für eine Menge eindimensionaler Punkte $p_1 = (x_1), p_2 = (x_2), \dots, p_N = (x_N)$ genügt es ja, die Punkte nach ihrem Koordinatenwert zu sortieren und sie dann in sortierter Reihenfolge zu betrachten. Die am dichtesten beieinanderliegenden Punkte sind offenbar Nachbarn in der Sortierreihenfolge. Damit ist das Problem im eindimensionalen Fall mit $O(N \log N)$ Rechenschritten lösbar.

Das ist gleichzeitig auch ein asymptotisch schnellstes Verfahren, wie folgende Überlegung zeigt. Das Problem, für eine gegebene Folge von Zahlen festzustellen, ob eine Zahl mehrmals in der Folge auftritt (element uniqueness), benötigt zur Lösung $\Omega(N \log N)$ Schritte für N Zahlen (vgl. [1] oder [2]). Dieses Problem läßt sich lösen, indem man nach dem dichtesten Zahlenpaar fragt. Ist die zugehörige Distanz 0, so gibt es eine Zahl, die mehr als einmal auftritt, sonst nicht. Folglich muß das Problem, das dichteste Zahlenpaar zu finden, ebenfalls mindestens $\Omega(N \log N)$ Schritte benötigen. Für Punkte in der Ebene (statt Zahlen, also eindimensionale Punkte) gilt diese untere Schranke erst recht.

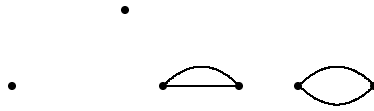
Betrachten wir zunächst noch einige Distanzprobleme, bevor wir der Frage nach einer bestmöglichen Lösung nachgehen.

Problem: *Alle nächsten Nachbarn* (all nearest neighbors)

gegeben: Eine Menge P von N Punkten in der Ebene.

gesucht: Für jeden Punkt $p_1 \in P$ ein nächster Nachbar $p_2 \in P$, d.h. ein Punkt $p_2 \neq p_1$ mit $d(p_1, p_2) = \min_{p \in P - \{p_1\}} \{d(p_1, p)\}$.

Die Antwort für dieses Problem besteht also aus N Punktepaaren. Man beachte, daß die Relation "nächster Nachbar" nicht symmetrisch ist: Wenn p_2 nächster Nachbar von p_1 ist, so muß noch nicht p_1 nächster Nachbar von p_2 sein. Das folgende Bild zeigt eine Menge von Punkten und die Relation "nächster Nachbar": " p_2 ist nächster Nachbar von p_1 " wird dargestellt durch einen Pfeil $p_1 \rightarrow p_2$.



Dieses Problem läßt sich auf naive Weise lösen, indem man für jeden Punkt $p_1 \in P$ die Distanz zu allen anderen Punkten in P berechnet und einen Punkt p_2 mit minimaler Distanz auswählt; p_2 ist ein nächster Nachbar von p_1 . Dieses Verfahren benötigt $\Theta(N^2)$ Schritte für eine Menge von N Punkten.

Man stellt leicht fest, daß das Problem für eindimensionale Punkte (wie auch schon das “closest pair”-Problem) wegen der Existenz einer totalen Ordnung auf den Punkten effizienter lösbar ist. So genügt es hier, die Punkte zu sortieren und anschließend für jeden Punkt seine beiden Nachbarn in der Sortierreihenfolge zu betrachten, denn nur sie kommen als nächste Nachbarn in Frage. Also kann auch dieses Problem für N eindimensionale Punkte mit $O(N \log N)$ Rechenschritten gelöst werden.

Das Problem, ein “dichtestes Punktepaar” zu finden, kann man lösen, indem man zuerst alle nächsten Nachbarn bestimmt, und dann ein Paar mit minimaler Distanz auswählt. Deshalb ist eine untere Schranke für die Laufzeit des “dichtestes Punktepaar”-Problems auch eine untere Schranke für das “alle nächsten Nachbarn”-Problem. Damit ist klar, daß dieses Problem mindestens $\Omega(N \log N)$ Schritte für eine Menge von N Punkten benötigt.

Betrachten wir nun das Problem, zu einer gegebenen Punktmenge ein kürzestes verbindendes Netzwerk zu finden. Die verschiedenen Versionen solcher Netzwerke, je nach Anforderungen an die Lösung, definieren kürzeste Verbindungen (etwa bei höchstintegrierten Schaltkreisen) oder einfach Ähnlichkeitsmaße für Punktmenge (etwa im Bereich der Mustererkennung). Wir interessieren uns dafür, einen minimalen spannenden Baum zu einer gegebenen Punktmenge zu finden.

Problem: *Minimaler spannender Baum* (minimum spanning tree)

gegeben: Eine Menge P von N Punkten in der Ebene.

gesucht: Ein minimaler spannender Baum für P , d.h. ein Baum, dessen Knoten gerade die Punkte aus P sind, dessen Kanten Verbindungen zwischen den Punkten sind, und der unter allen solchen Bäumen minimale Länge hat. Die Länge eines Baumes ist dabei die Summe der Längen seiner Kanten; die Länge einer Kante ist die (euklidische) Distanz der beiden Endpunkte.

Die Abbildung 7.50 zeigt eine Menge von sieben Punkten und einen minimalen spannenden Baum für diese Punktmenge.

Eine naive Lösung des Problems könnte darin bestehen, alle möglichen spannenden Bäume — das sind Bäume, deren Knoten gerade die Punkte aus P sind — zu berechnen, und einen mit minimaler Länge auszuwählen. Dazu müssen jedenfalls Kanten für alle Punktepaare betrachtet werden — das sind bereits $\Theta(N^2)$ Kanten. Jedes so operierende Lösungsverfahren muß also mindestens $\Theta(N^2)$ Schritte zur Lösung im schlimmsten Fall benötigen; womöglich benötigt es beträchtlich mehr.

Im eindimensionalen Fall läßt sich das Problem wieder ganz leicht durch Sortieren mit anschließendem Verbinden aller in der Sortierreihenfolge benachbarten Punkte lösen — also in $O(N \log N)$ Schritten für N Punkte.

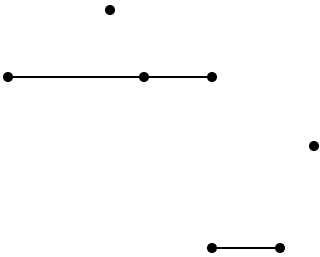


Abbildung 7.50

Es ist leicht einzusehen, daß jeder minimale spannende Baum für Punktmenge P eine verbindende Kante zwischen zwei dichtesten Punkten in P enthält. Betrachten wir zum Beweis einen spannenden Baum B , der für kein dichtestes Punktepaar eine verbindende Kante enthält. Nun verändern wir diesen Baum, indem wir eine solche Kante hinzufügen; das Resultat B' ist kein Baum mehr, weil es jetzt einen Zyklus gibt, wie die Abbildung 7.51 zeigt.

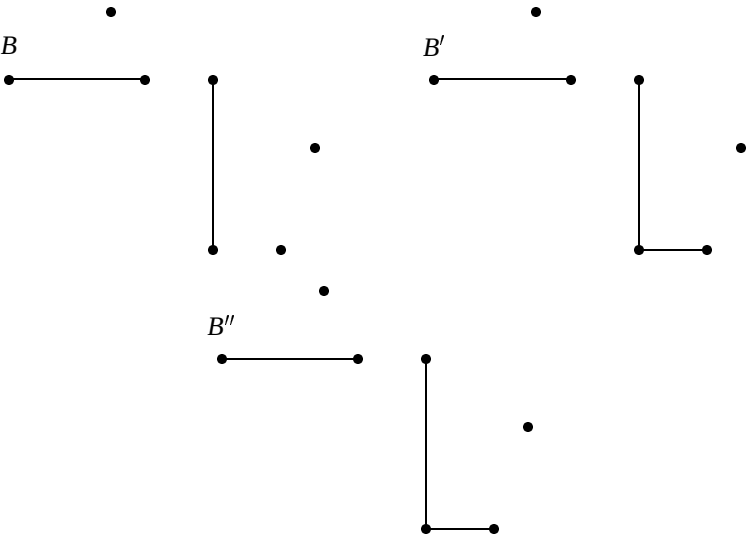


Abbildung 7.51

Aus B' machen wir durch Entfernen einer Kante des Zyklus (etwa der längsten Kante) wieder einen Baum B'' . Dann ist klar, daß die Länge von B'' geringer ist als die von B , und damit kann B kein minimaler spannender Baum sein.

Jetzt läßt sich das “dichteste Paar”-Problem lösen, indem man zunächst einen minimalen spannenden Baum berechnet, und dann nur noch unter allen $N - 1$ durch eine Kante verbundenen Punktepaares das dichteste ausfindig macht. Deshalb benötigt die Berechnung eines minimalen spannenden Baumes im schlimmsten Fall mindestens soviel Zeit wie das Finden eines dichtesten Paares, nämlich $\Omega(N \log N)$.

Wieder stellt sich die Frage, ob man die Lage der Punkte in der Ebene nutzen kann, um einen schnellen — vielleicht sogar optimalen — Algorithmus zur Berechnung eines minimalen spannenden Baumes zu finden.

Die bisher genannten sind Probleme, bei denen man für eine gegebene Punktemenge einmal eine Frage beantworten will. Im Gegensatz dazu geht es bei einer großen Klasse von Problemen darum, wiederholt auf einer Grundmenge von Elementen gewisse Operationen auszuführen, wie etwa beim Speichern und Wiederfinden von Informationen. In solchen Fällen ist es oft nützlich, die Grundmenge, unter Umständen mit einigem Rechenaufwand, so vorzubehandeln (preprocessing), daß nachfolgende Anfragen schnell ausgeführt werden können. Stellen wir uns etwa vor, ein Kunde im Reisebüro sucht nach einem Urlaubsangebot eines gewissen Typs (Badeurlaub), zu einer grob festgelegten Zeit. Er hat Idealvorstellungen in vielerlei Hinsicht (Ort, Dauer, Preis, Verpflegung, Lage des Hotels, etc.), die er insgesamt so gut es eben geht realisieren möchte. Das Reisebüro wird versuchen, aus der Grundmenge aller verfügbaren Urlaubsreisen eine möglichst passende herauszusuchen (best match). Faßt man die Attribute einer Urlaubsreise als Koordinaten in einem mehrdimensionalen Koordinatensystem auf und bringt man die Gewichtung der Attribute in einer Distanzfunktion zum Ausdruck, so sucht unser Urlaubswilliger in der Menge der angebotenen Urlaubsreisen vielleicht gerade nach einer Reise mit geringster Distanz zu seiner Idealvorstellung. Für den Fall von Punkten in der Ebene läßt sich das “best match”-Problem wie folgt formulieren.

Problem: *Suche nächsten Nachbarn* (nearest neighbor search, best match)

gegeben: Eine Menge P von N Punkten in der Ebene.

gesucht: Eine Datenstruktur und Algorithmen, die

1. P in der durch die Datenstruktur vorgeschriebenen Form speichern (preprocessing),
2. zu einem gegebenen, neuen Punkt q (Anfragepunkt, query point) einen Punkt aus P finden, der nächster Nachbar von q ist.

Ganz ohne Vorbehandlung läßt sich eine Anfrage nach einem nächsten Nachbarn von q in Zeit $\Theta(N)$ beantworten, indem die Distanz von q zu jedem Punkt in P berechnet und das Minimum ausgesucht wird.

Im eindimensionalen Fall kann man wieder durch Sortieren von P , also mit Vorbehandlungsaufwand $O(N \log N)$, eine schnelle Beantwortung dieser Anfrage erreichen, nämlich mittels binärer Suche. Endet die binäre Suche erfolgreich, so hat man genau den gesuchten Punkt gefunden; andernfalls ist ein nächster Nachbar einer der (höchstens zwei) Nachbarn der Stelle, an der die Suche endet. Wegen der Optimalität der binären Suche ist auch diese Nachbarschaftssuche optimal: man kann sie zur Suche nach einem Punkt verwenden. Damit ist $\Omega(\log N)$ eine untere Schranke für den Aufwand zur Suche eines nächsten Nachbarn im schlimmsten Fall, und zwar für jede Dimension.

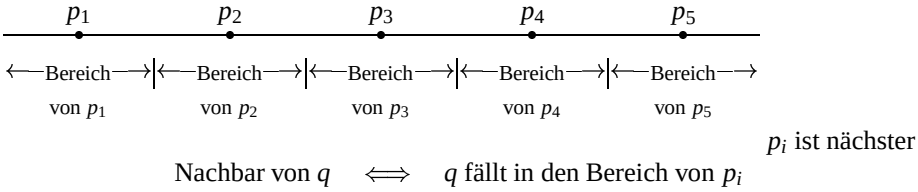


Abbildung 7.52

Erinnern wir uns: Alle gestellten Probleme können im eindimensionalen Fall leicht optimal gelöst werden, weil wir uns die Sortierung der Punktmenge zunutze machen können. Da es aber für zweidimensionale Punkte keine Sortierung gibt, läßt sich dieser Ansatz nicht auf höhere Dimensionen verallgemeinern. Bei näherem Hinsehen stellen wir aber fest, daß sich eine Eigenschaft der sortierten Punktmenge verallgemeinern läßt, die wir zur Lösung der Probleme genutzt haben: Wir haben (implizit) für jeden gegebenen Punkt p die Menge aller Punkte vorausberechnet, die näher bei p als bei irgendeinem anderen Punkt der Menge liegen. Bei der Suche nach dem nächsten Nachbarn beispielsweise haben wir dann nur noch feststellen müssen, zu welcher der vorausberechneten Punktmenge ein Anfragepunkt gehört; die Abbildung 7.52 illustriert diesen Sachverhalt.

Dasselbe Prinzip wollen wir nun auf zweidimensionale Punkte in der Ebene verallgemeinern, um eine schnellere Lösung für alle genannten Probleme zu erhalten.

7.7.2 Das Voronoi-Diagramm

Das Voronoi-Diagramm für eine Menge von Punkten in der Ebene teilt die Ebene ein in Gebiete gleicher nächster Nachbarn. Besteht die Menge lediglich aus zwei Punkten, so wird die Einteilung gerade durch die Mittelsenkrechte auf der Verbindungsstrecke der beiden Punkte realisiert (Abbildung 7.53).

Der geometrische Ort aller Punkte, die näher bei p_1 liegen als bei p_2 , ist die Halbebene $H(p_1|p_2)$; das entsprechende gilt für p_2 und $H(p_2|p_1)$. Allgemein nennen wir für eine gegebene Menge P von Punkten und einen Punkt $p \in P$ den geometrischen Ort aller Punkte der Ebene, die näher bei p liegen als bei irgendeinem anderen Punkt aus P , die *Voronoi-Region* $VR(p)$ von p . Sie ist stets der Durchschnitt aller Halbebenen von p , gebildet mit allen anderen Punkten aus P :

$$VR(p) = \bigcap_{p' \in P \setminus \{p\}} H(p|p')$$

Die Abbildung 7.54 zeigt eine Menge von sechs Punkten und die Voronoi-Region für einen der Punkte p_1 .

Das Studium dieser Regionen geht zurück auf den Mathematiker G. Voronoi (vgl. [10]). Man nennt sie manchmal auch Dirichlet-Gebiete oder Thiessen-Polygone (vgl. [10]). Die Menge aller Voronoi-Regionen für eine Menge von Punkten ist das *Voronoi-Diagramm*. Abbildung 7.55 zeigt ein Beispiel.

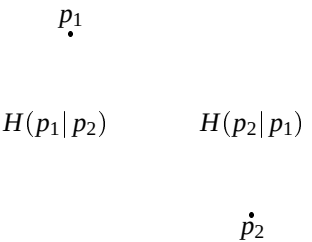


Abbildung 7.53

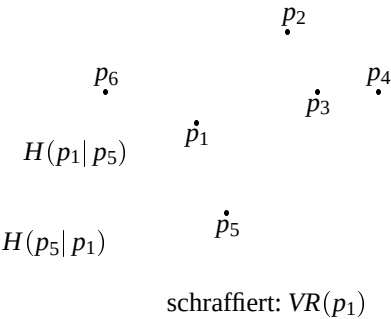


Abbildung 7.54

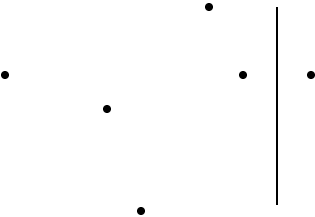


Abbildung 7.55

Wir betrachten das Voronoi-Diagramm als ebenes Netzwerk; die Knoten des Netzwerkes heißen *Voronoi-Knoten*, die Kanten *Voronoi-Kanten*.

Das Voronoi-Diagramm hat eine Reihe von Eigenschaften, die es erlauben, die eingangs gestellten Probleme effizient zu lösen. Für die Suche nach einem nächsten Nachbarn eines Anfragepunktes q genügt es, die Voronoi-Region $VR(p)$ des Punktes p zu bestimmen, in der q liegt; p ist dann ein nächster Nachbar von q . Bevor wir die Lösung der Probleme mit Hilfe des Voronoi-Diagramms beschreiben, wollen wir die Eigenschaften des Voronoi-Diagramms etwas genauer betrachten. Nehmen wir (zur Vermeidung einer umständlichen Sonderfallbetrachtung) an, daß keine vier Punkte der gegebenen Punktmenge auf einem gemeinsamen Kreis liegen.

Da jeder Punkt auf der Mittelsenkrechten der Verbindungsstrecke zwischen p und p' zu p und p' den gleichen Abstand hat, liegt auch jeder Voronoi-Knoten v gleich weit von allen Punkten aus P entfernt, deren Voronoi-Regionen an v grenzen:

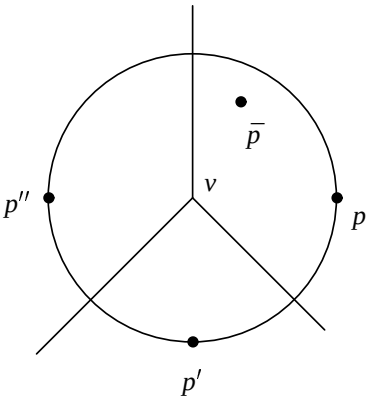
v
 $p_6 \bullet$

$\bullet p_2$

$d(p_1, v) = d(p_2, v) = d(p_6, v)$
 $\bullet p_1$

Weil keine vier Punkte aus P auf einem Kreis liegen, und weil zwei Punkte aus P keinen Voronoi-Knoten definieren, muß jeder Voronoi-Knoten genau drei Kanten begrenzen und auf dem Rand von genau drei Voronoi-Regionen liegen. Jeder Knoten des Voronoi-Diagramms hat also genau den Grad drei.

Ist p ein Punkt aus P einer an Voronoi-Knoten v angrenzenden Voronoi-Region, so liegen folglich gerade drei Punkte aus P , sagen wir p , p' und p'' , auf dem Kreis um v mit Radius $d(p, v)$. In diesem Kreis kann kein Punkt $\bar{p}$ aus P liegen. Dann wäre nämlich $d(\bar{p}, v) < d(p, v)$, und damit müßte $v \in VR(\bar{p})$ gelten, im Widerspruch zur Voraussetzung $v \in VR(p)$.



Man macht sich leicht klar, daß jeder nächste Nachbar eines Punktes $p \in P$ eine Kante der Voronoi-Region $VR(p)$ definiert; nächste Nachbarn haben also sich berührende Voronoi-Regionen.

Manche der Voronoi-Regionen sind beschränkt, andere sind unbeschränkt. Die unbeschränkten Regionen gehören genau zu denjenigen Punkten, die auf der konvexen Hülle von P liegen (Abbildung 7.56).

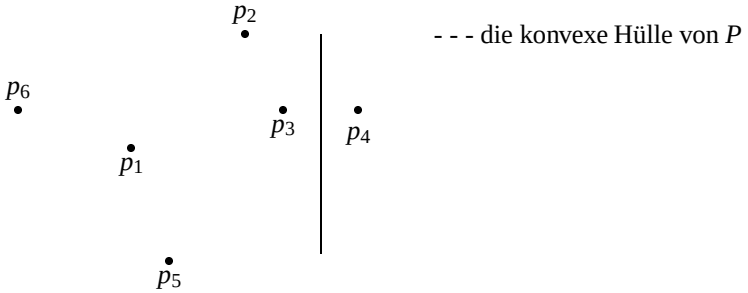


Abbildung 7.56

Diesen Sachverhalt kann man sich wie folgt klar machen. Betrachten wir zunächst eine beschränkte Voronoi-Region $VR(p)$ eines Punktes $p \in P$, und die reihum angrenzenden Voronoi-Regionen $VR(p'_1), VR(p'_2), \dots, VR(p'_k)$. In unserem Beispiel grenzen $VR(p_2), VR(p_3), VR(p_5)$ und $VR(p_6)$ an die beschränkte Region $VR(p_1)$. Dann muß p im Polygon mit den Eckpunkten $p'_1, p'_2, \dots, p'_k$ liegen, also nicht auf der konvexen Hülle. In unserem Beispiel liegt p_1 im Polygon mit Eckpunkten p_2, p_3, p_5 und p_6 .

Wir überlegen uns noch, daß der Schluß auch in der anderen Richtung gilt, d.h. daß für $p \in P$ nicht auf der konvexen Hülle $VR(p)$ beschränkt ist. Liegt p nicht auf der konvexen Hülle, so liegt p in Innern eines Dreiecks, dessen drei Eckpunkte p'_1, p'_2, p'_3 aus P stammen. In unserem Beispiel liegt p_1 im Innern des Dreiecks p_4, p_5, p_6 . Betrachten wir die drei Kreise, die durch p und jeweils zwei der Punkte p'_1, p'_2, p'_3 gehen (Abbildung 7.57). Jeder Punkt auf dem Rand der Vereinigung der drei Kreise K_{12}, K_{23} und K_{13} liegt näher an einem der Punkte p'_1, p'_2, p'_3 als an p . Dasselbe gilt ebenfalls für alle Punkte außerhalb $K_{12} \cup K_{23} \cup K_{13}$. Also muß $VR(p)$ ganz in der Vereinigung der drei Kreise enthalten sein; damit ist $VR(p)$ beschränkt.

Nun versuchen wir, die im Voronoi-Diagramm implizit repräsentierten Nachbarschaften explizit darzustellen. Dazu betrachten wir den dualen Graphen (das duale Netzwerk): Jeder (gegebene) Punkt $p \in P$ ist ein Knoten, und zwischen zwei Knoten p und p' gibt es genau dann eine (ungerichtete) Kante, wenn $VR(p)$ und $VR(p')$ sich berühren, also eine gemeinsame Voronoi-Kante haben. Die Kanten des dualen Graphen haben eine Länge, die gerade der Distanz der beiden Endknoten entspricht. Die Abbildung 7.58 zeigt den zum Voronoi-Diagramm dualen Graphen für unser Beispiel.

Der duale Graph trianguliert die Menge P , d.h., er definiert eine Zerlegung der konvexen Hülle von P in Dreiecke mit Punkten aus P als Eckpunkte. Dies kann man einsehen,

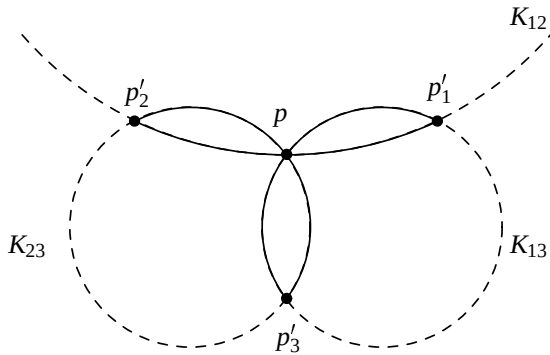


Abbildung 7.57

indem man jedem Voronoi-Knoten v das Dreieck des dualen Graphen mit Eckpunkten p'_1, p'_2, p'_3 zuordnet, wobei v auf dem Rand der Voronoi-Regionen $VR(p'_1), VR(p'_2), VR(p'_3)$ liegt. Dann zeigt man, daß sich diese Dreiecke nicht überlappen (sondern sich allenfalls berühren), und daß jeder Punkt der konvexen Hülle von P in einem solchen Dreieck liegt. Man beachte, daß v selbst nicht im zugehörigen Dreieck p'_1, p'_2, p'_3 liegen muß. Delaunay hat bereits 1934 gezeigt, daß der zum Voronoi-Diagramm duale Graph P trianguliert (); die so definierte Zerlegung heißt daher auch *Delaunay-Triangulierung*.

Damit ergibt sich direkt eine Aussage über die Anzahl der Voronoi-Knoten und Voronoi-Kanten eines Voronoi-Diagramms für eine Menge von N Punkten: ein solches Diagramm hat höchstens $2N - 4$ Knoten und höchstens $3N - 6$ Kanten. Weil die Delaunay-Triangulierung ein planarer Graph ist, besteht sie nach Euler aus höchstens $3N - 6$ Kanten und höchstens $2N - 4$ Dreiecken (Flächen). Jedem Dreieck entspricht ein Voronoi-Knoten, jeder Kante der Triangulierung entspricht eine gemeinsame Kante der beiden betreffenden Voronoi-Regionen.

Das Voronoi-Diagramm erlaubt also — sobald es erst einmal berechnet ist — eine sehr kompakte und trotzdem explizite Darstellung der Nachbarschaftsverhältnisse von Punkten. Überlegen wir uns zunächst genauer, wie das Voronoi-Diagramm gespeichert werden soll, und anschließend, wie wir es denn berechnen können.

7.7.3 Die Speicherung des Voronoi-Diagramms

Wir speichern das Voronoi-Diagramm als einen in die Ebene eingebetteten planaren Graphen. ( schlagen vor, eine doppelt verkettete Liste der Kanten (englisch: doubly connected edge list) abzuspeichern. Jede Kante wird durch ihre beiden Endpunkte (Knoten) angegeben; außerdem wird bei jeder Kante vermerkt, welche beiden Flächen sich auf beiden Seiten der Kante anschließen. Jeder Knoten v wird durch die beiden Koordinatenwerte (x_v, y_v) repräsentiert. Wir legen das übliche kartesische Koordinaten-

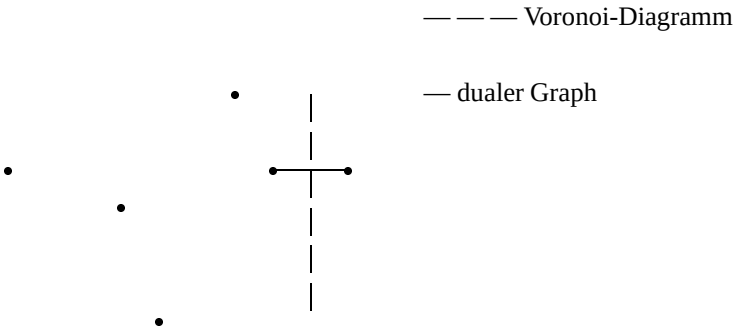


Abbildung 7.58

system zugrunde. Um die zu einer Fläche gehörenden Kanten nacheinander betrachten zu können, werden mit jeder Kante zwei Verweise auf die an den beiden Endknoten der Kante weiterführenden Kanten gespeichert. Genauer hat die doppelt verkettete Kantenliste die in Abbildung 7.59 gezeigte Gestalt.

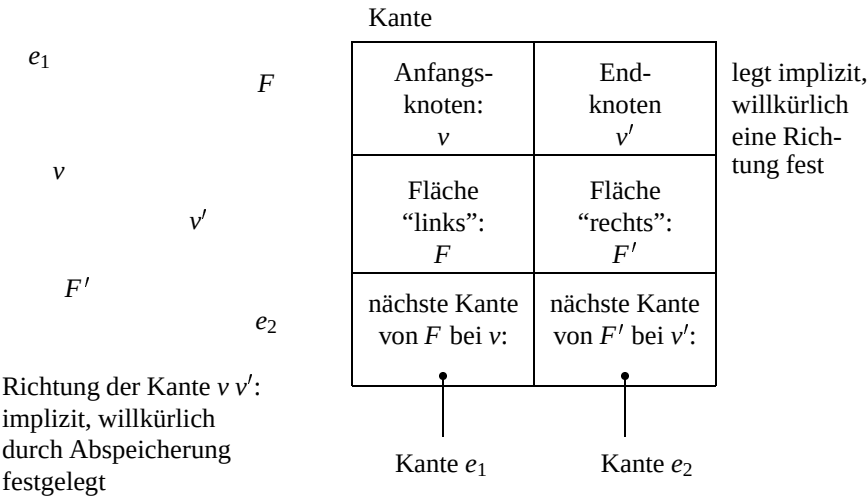


Abbildung 7.59

Verwenden wir jetzt die Definition

```

type
  kantenzeiger = ↑kante;
  kante = record
    anfangsknoten, endknoten: knoten;
    linkeflaeche, rechteflaeche: flaeche;
    anfangskante, endkante: kantenzeiger
  end

```

mit geeigneten Definitionen für *knoten* und *flaeche*, so können wir beispielsweise alle Kanten der Fläche *F* im Uhrzeigersinn direkt nacheinander besuchen, wenn wir schon eine Kante der Fläche *F* kennen:

```

var
  z1, z2 : kantenzeiger;
  .
  .
  .
  {sei z1 ein Zeiger auf eine zur Fläche F gehörende Kante;
   also entweder z1 ↑.linkeflaeche = F oder z1 ↑.rechteflaeche = F}
  z2 := z1; {starte das Umrunden der Fläche bei z1}
repeat
  {die aktuell betrachtete Kante ist z2 ↑}
  {fahre fort mit der nächsten zu F gehörenden Kante:}
  if z2 ↑.linkeflaeche = F
    then z2 := z2 ↑.anfangskante
    else z2 := z2 ↑.endkante
until z2 = z1 {Umrundung ist vollendet}

```

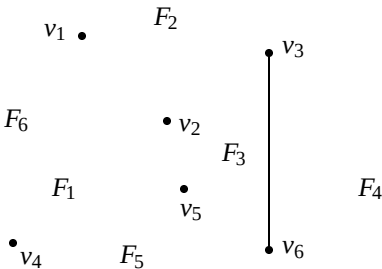
Entsprechend läßt sich leicht angeben, wie man alle mit einem Knoten inzidenten Kanten entgegen dem Uhrzeigersinn besuchen kann. Wichtig ist hier nur, daß die Laufzeit beider Operationen (Kanten einer Fläche besuchen; Kanten eines Knotens besuchen) lediglich proportional zur Anzahl der besuchten Kanten ist, wenn man bereits Zugriff auf eine der zu besuchenden Kanten hat.

Versehen wir die doppelt verkettete Kantenliste nun noch mit einem Anfangszeiger auf eine beliebige Kante, so läßt sich eine Startkante für eine Fläche oder einen Knoten in Zeit proportional zur Anzahl aller gespeicherten Kanten finden.

Für unser Voronoi-Diagramm-Beispiel ergibt sich beispielsweise die in Abbildung 7.60 gezeigte Situation.

7.7.4 Die Konstruktion des Voronoi-Diagramms

Preparata und Shamos [] weisen darauf hin, daß in einigen Anwendungsgebieten das Berechnen des Voronoi-Diagramms nicht ein Zwischenschritt beim Lösen eines Problems ist, sondern bereits die Lösung — Beispiele findet man in der Archäologie,



Voronoi-Diagramm

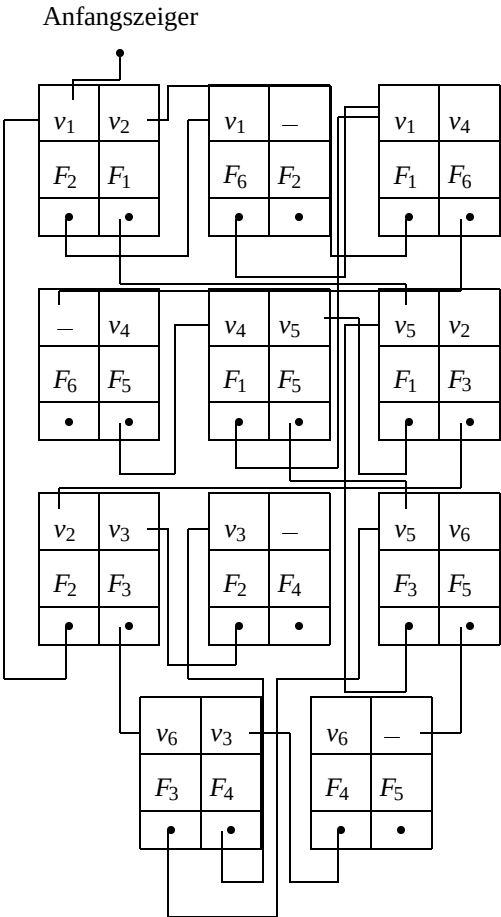


Abbildung 7.60

der Ökologie, der Chemie und der Physik. Wir wollen das Voronoi-Diagramm berechnen, um damit die eingangs beschriebenen Probleme effizienter zu lösen. Formulieren wir zunächst das Problem.

Problem: *Voronoi-Diagramm*

gegeben: Eine Menge P von N Punkten in der Ebene.

gesucht: Das Voronoi-Diagramm für P , als doppelt verkettete Kantenliste.

Ein naives Verfahren zur Berechnung des Voronoi-Diagramms könnte damit beginnen, daß für jeden Punkt $p \in P$ durch Betrachten aller anderen Punkte $p' \in P \setminus \{p\}$ die p betreffenden Halbebenen berechnet und ihr Durchschnitt gebildet werden. Damit erhält man die Voronoi-Region für jeden Punkt $p \in P$ in Zeit $\Omega(N)$ pro Punkt, also insgesamt in Zeit $\Omega(N^2)$. Solch ein Verfahren kann aber nicht als Grundlage für schnellere Algorithmen für unsere Ausgangsprobleme dienen. Fragen wir uns zunächst, wie lange denn die Berechnung des Voronoi-Diagramms mindestens dauern muß.

Im eindimensionalen Fall besteht das Voronoi-Diagramm gerade aus den “Trennstellen” für Gebiete gleicher nächster Nachbarn, wie am Ende des Abschnitts 7.7.1 angegeben. Die Voronoi-Region eines Punktes aus P ist also hier ein Intervall, das den Punkt enthält. Wenn man wieder fordert, daß aus einer Voronoi-“Kante” (das ist hier eine “Trennstelle”) in konstanter Zeit auf die angrenzenden Gebiete geschlossen werden kann und umgekehrt, so kann man das Voronoi-Diagramm für eine Menge von Zahlen zum Sortieren benutzen: Man beginnt bei der kleinsten Zahl und durchläuft alle Zahlen gemäß dem Voronoi-Diagramm. Da dieses Durchlaufen lediglich lineare Zeit, Sortieren aber $\Omega(N \log N)$ Zeit benötigt, muß das Berechnen des Voronoi-Diagramms $\Omega(N \log N)$ Zeit benötigen. Im eindimensionalen Fall ist das ja mittels Sortieren auch tatsächlich leicht erreichbar.

Wir werden jetzt zeigen, wie das Voronoi-Diagramm auch im zweidimensionalen Fall, also für Punkte in der Ebene, effizient berechnet werden kann. Dazu verwenden wir ein dem Divide-and-Conquer-Prinzip folgendes Verfahren:

1. Teile P in zwei etwa gleich große Teilmengen P_1 und P_2 .
2. Berechne die Voronoi-Diagramme für P_1 und P_2 rekursiv.
3. Verschmelze die beiden Voronoi-Diagramme für P_1 und P_2 zum Voronoi-Diagramm für P .

Das Ende der Rekursion ist erreicht, wenn das Voronoi-Diagramm für einen einzigen Punkt berechnet werden soll: das ist gerade die ganze Ebene. Wichtig ist, daß wir P so teilen, daß Schritt 3, das Verschmelzen der Teil-Diagramme, möglichst effizient ausführbar ist. Dabei hilft eine wichtige Beobachtung: Wenn P durch eine vertikale Linie in zwei Teile P_1 und P_2 geteilt wird, so bilden diejenigen Kanten des Voronoi-Diagramms, die sowohl an Voronoi-Regionen für Punkte aus P_1 als auch an Voronoi-Regionen für Punkte aus P_2 angrenzen, einen in vertikaler Richtung monotonen, zusammenhängenden Linienzug. Dieser Linienzug besteht am oberen und unteren Ende aus je einer Halbgeraden, mit Geradenstücken dazwischen. Die Abbildung 7.61 illustriert diese Aussage für unser Beispiel.

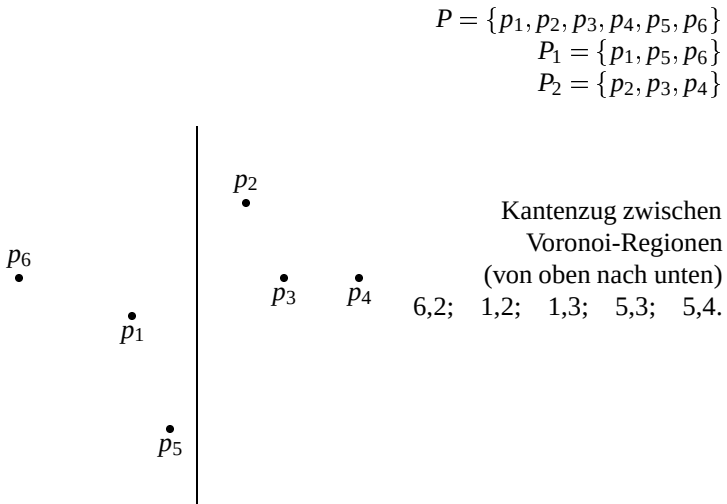


Abbildung 7.61

Das Voronoi-Diagramm für P setzt sich dann zusammen aus dem links dieses Kantenzugs liegenden Teil des Voronoi-Diagramms von P_1 , dem rechts des Kantenzugs liegenden Teil des Voronoi-Diagramms von P_2 und dem Kantenzug selbst (Abbildung 7.62).

Wir präzisieren jetzt das Verfahren zur Berechnung des Voronoi-Diagramms entsprechend.

Algorithmus Voronoi-Diagramm

{liefert zu einer Menge P von N Punkten in der Ebene das Voronoi-Diagramm $VD(P)$ in Form einer doppelt verketteten Kantenliste}

1. {Divide:}

Teile P durch eine vertikale Trennlinie T in zwei etwa gleich große Teilmengen P_1 (links von T) und P_2 (rechts von T), falls $|P| > 1$ ist; sonst ist $VD(P)$ die gesamte Ebene.
2. {Conquer:}

Berechne $VD(P_1)$ und $VD(P_2)$ rekursiv.
3. {Merge:}
 - (a) Berechne den P_1 und P_2 trennenden Kantenzug K , der Teil von $VD(P)$ ist.
 - (b) Schneide den rechts von K liegenden Teil von $VD(P_1)$ ab, und schneide den links von K liegenden Teil von $VD(P_2)$ ab.
 - (c) Vereinige $VD(P_1)$, $VD(P_2)$ und K ; das ist $VD(P)$.

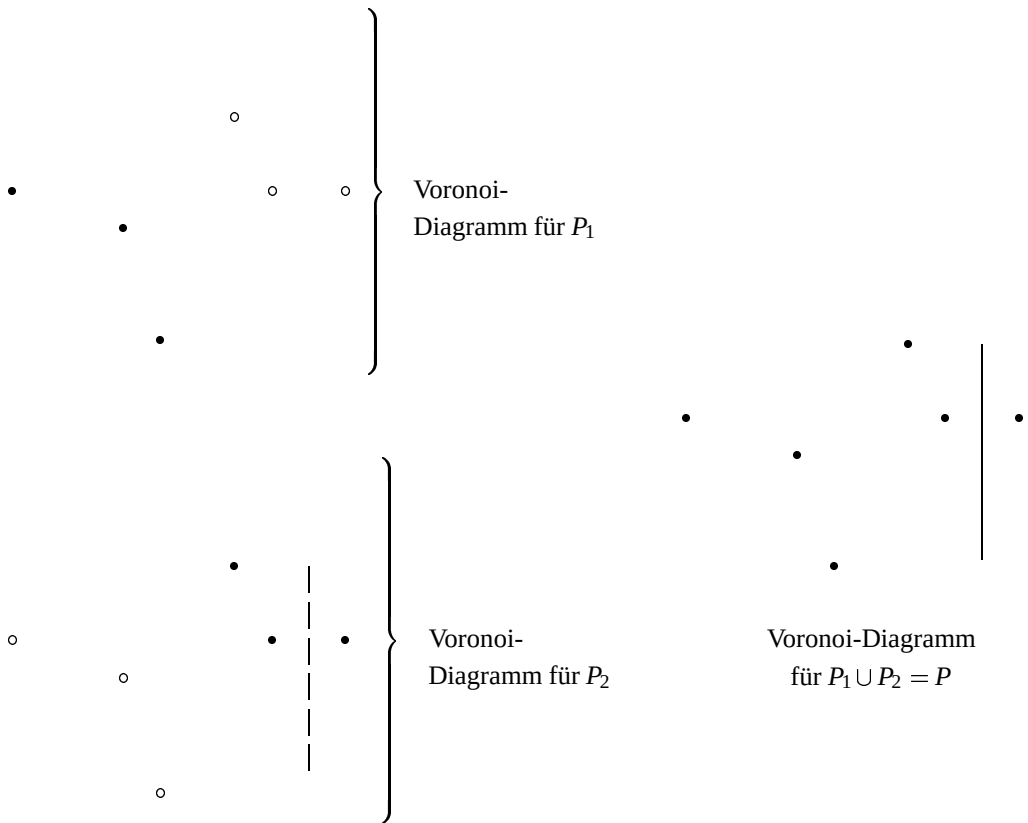


Abbildung 7.62

Schritt 1, das Aufteilen von P , ist gerade das Finden des Medians der x -Koordinaten aller Punkte, und das Verteilen der Punkte auf die beiden Teilmengen. Beides kann in linearer Zeit ausgeführt werden. Der kritische Schritt ist das Berechnen von K ; das anschließende Abschneiden der überstehenden Kanten von $VD(P_1)$ und $VD(P_2)$ kann durch das Durchlaufen der jeweiligen doppelt verketteten Kantenliste in linearer Zeit geschehen. Wir wollen uns jetzt überlegen, wie auch der trennende Kantenzug K in linearer Zeit berechnet werden kann. Dann ergibt sich für die Laufzeit $T(N)$ des Verfahrens zur Berechnung des Voronoi-Diagramms für N Punkte

$$\begin{aligned} T(N) &= 2 \cdot T(N/2) + O(N) \\ T(1) &= O(1) \end{aligned}$$

und damit

$$T(N) = O(N \log N);$$

das Verfahren ist also optimal.

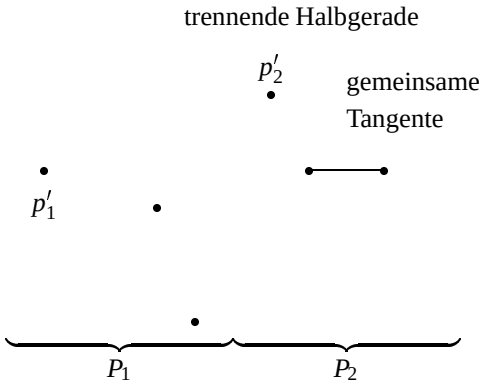


Abbildung 7.63

Wir berechnen den trennenden Kantenzug schrittweise, ein Geradenstück nach dem anderen (I $\Rightarrow$ II). Dabei beginnen wir mit der oberen Halbgeraden des Kantenzugs. Diese Halbgerade muß Teil der Mittelsenkrechten zweier Punkte sein, von denen einer zu P_1 und einer zu P_2 gehört. Da beide angrenzenden Voronoi-Regionen unbeschränkt sind, müssen beide Punkte auf der konvexen Hülle der Punktmenge P liegen. Wir können diese beiden Punkte also bestimmen, indem wir eine gemeinsame Tangente von "oben" an die beiden konvexen Hüllen der Punktmengen P_1 und P_2 legen, wie in Abbildung 7.63 gezeigt.

Erinnern wir uns: Die beiden Voronoi-Diagramme $VD(P_1)$ und $VD(P_2)$ für die Teilmengen P_1 und P_2 von P sind bereits (rekursiv) berechnet worden. Der $VD(P_1)$ und $VD(P_2)$ trennende Kantenzug K muß in $VD(P)$ so verlaufen, daß alle Punkte der Ebene links von K näher bei einem Punkt aus P_1 als bei einem Punkt aus P_2 liegen (das gilt symmetrisch für die Punkte rechts von K). Also sind die Geradenstücke, aus denen K besteht, Teile von Mittelsenkrechten mit einem Punkt aus P_1 und einem Punkt aus P_2 . Lassen wir nun einen Punkt k auf K von oben nach unten wandern, beginnend mit k auf der Mittelsenkrechten der beiden Tangentialpunkte $p'_1 \in P_1$ und $p'_2 \in P_2$. An der Stelle k_1 , an der k die Grenze einer der beiden Voronoi-Regionen $VR(p'_1)$ oder $VR(p'_2)$ erreicht, muß K von dieser Mittelsenkrechten abweichen, weil sich der nächstliegende Punkt in P_1 oder P_2 für K geändert hat. Nehmen wir ohne Beschränkung der Allgemeinheit an, daß K die Grenze von $VR(p'_1)$ erreicht, und daß $VR(p'_1)$ mit $VR(p'_1)$ diese Grenze bildet, wie in Abbildung 7.64 gezeigt.

Da K in vertikaler Richtung monoton fällt, wird nun p''_1 zum K nächstliegenden Punkt in P_1 . Also ergibt sich das nächste Geradenstück für K aus der Mittelsenkrechten der Verbindungsstrecke von p''_1 und p'_2 . Dieser Geraden folgt K solange, bis wieder die Grenze einer Voronoi-Region erreicht ist. Im Beispiel wird die Grenze von $VR(p'_2)$ erreicht; damit folgt K nunmehr der Mittelsenkrechten der Verbindungsstrecke von p''_1 und p'_2 . Dieser Prozeß wird solange fortgesetzt, bis K der Mittelsenkrechten der unter-

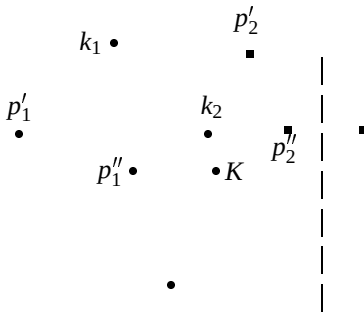


Abbildung 7.64

ren Tangentialstrecke an die beiden konvexen Hüllen von P_1 und P_2 folgt; dann ist K komplett beschrieben.

Die Berechnung des Kantenzugs K bei gegebenen Voronoi-Diagrammen $VD(P_1)$ und $VD(P_2)$ lässt sich also wie folgt beschreiben:

{Berechnung des trennenden Kantenzugs K bei gegebenen Voronoi-Diagrammen $VD(P_1)$, $VD(P_2)$; wird nur ausgeführt für $P_1 \neq \emptyset$, $P_2 \neq \emptyset$ }

1. *Ermittle die beiden oberen Tangentialpunkte $p'_1 \in P_1$ und $p'_2 \in P_2$ und die beiden unteren Tangentialpunkte $\underline{p}_1 \in P_1$ und $\underline{p}_2 \in P_2$. Bestimme die Mittelsenkrechte m der Verbindungsstrecke zwischen p'_1 und p'_2 . Wähle $k = (x_k, y_k)$ mit $y_k = \infty$ so, daß k auf m liegt. Setze $K := \emptyset$.*
2. **while** $(p'_1 \neq \underline{p}_1)$ **or** $(p'_2 \neq \underline{p}_2)$ **do**
begin *{Berechnung von K fortsetzen}*
ermittle Schnittpunkt s_1 von m mit $VR(p'_1)$ unterhalb k
und Schnittpunkt s_2 von m mit $VR(p'_2)$ unterhalb k ;
{nicht beide Schnittpunkte müssen existieren,
aber mindestens einer}
if $(s_1$ liegt oberhalb von $s_2)$ **or** $(s_2$ existiert nicht)
then $i := 1$
else $i := 2$;
füge Geradenstück m von k bis s_i zu K hinzu;
setze $k := s_i$;
sei p''_i der Punkt aus P_i , dessen Voronoi-Region $VR(p''_i)$


```

    in  $s_i$  an  $VR(p'_i)$  angrenzt;
    setze  $p'_i := p''_i$ 
end {while}

```

3. Füge m von k bis $k' = (x_{k'}, y_{k'})$ mit $y_{k'} = -\infty$ und k' auf m liegend zu K hinzu.

Die Zeit zur Berechnung von K darf $O(|P_1| + |P_2|)$ nicht übersteigen, wenn zur Berechnung des Voronoi-Diagramms für P nicht mehr als $O(N \log N)$ Zeit zur Verfügung steht, für $|P| = N$. Nehmen wir (induktiv) an, daß die konvexe Hülle für P_1 und P_2 bei der Berechnung von K bekannt ist, so können alle vier Tangentialpunkte in sublinearer Zeit berechnet werden. Mit Hilfe der berechneten gemeinsamen Tangenten läßt sich ebenso die konvexe Hülle von $P_1 \cup P_2$ in höchstens linearer Zeit angeben; die rekursive Konstruktion der konvexen Hülle ist also genügend effizient sichergestellt.

Alle Operationen im Innern der while-Schleife (Schritt 2) außer dem Ermitteln des nächsten Schnittpunktes benötigen lediglich konstante Schrittzahl, da mit Hilfe der doppelt verketteten Kantenlisten für $VD(P_1)$ und $VD(P_2)$ direkt auf benachbarte Voronoi-Regionen zugegriffen werden kann, wenn die gemeinsame Voronoi-Kante bekannt ist. Weil K aus $\Theta(|P_1| + |P_2|)$ Geradenstücken bestehen kann, benötigt dieser Teil der Operationen der Schleife also insgesamt höchstens $O(|P_1| + |P_2|)$ viele Schritte.

Das Finden aller nächsten Schnittpunkte von Mittelsenkrechten mit Voronoi-Regionen entlang K darf insgesamt ebenfalls höchstens $O(|P_1| + |P_2|)$ Schritte kosten. Daß diese Schrittzahl tatsächlich genügt, ist nicht so offensichtlich, wenn man bedenkt, daß K $\Theta(|P_1| + |P_2|)$ Voronoi-Regionen passieren kann und daß eine Voronoi-Region $\Theta(|P_1| + |P_2|)$ Kanten haben kann. Alle Voronoi-Regionen zusammen haben aber auch nur $O(|P_1| + |P_2|)$ Kanten. Da wir im Innern der **while**-Schleife aber jeweils zwei Schnittpunkte, s_1 und s_2 , berechnen, aber nur einen davon (den weiter oben liegenden) verwenden, müssen wir sicherstellen, daß die Kanten der Voronoi-Region des nicht verwendeten Schnittpunktes bei späteren Schnittpunktberechnungen nicht wieder inspiziert werden müssen. Es ist also nicht effizient genug, zur Schnittpunktberechnung für p'_1 (bzw. p'_2) alle Kanten von $VR(p'_1)$ (bzw. $VR(p'_2)$) zu besuchen, und für jede Kante eine Schnittpunktprüfung vorzunehmen.

Eine effiziente Realisierung der Schnittpunktberechnung findet man mit folgender Überlegung. Nehmen wir (ohne Beschränkung der Allgemeinheit) an, für p''_1 sei bereits eine Schnittpunktberechnung für die Mittelsenkrechte m der Verbindungsstrecke von p''_1 und p'_2 durchgeführt worden, aber der errechnete Schnittpunkt s_1 sei nicht gewählt worden. Dann wird p'_2 von p''_2 abgelöst, die neue Mittelsenkrechte sei m' . Diese Situation ist in Abbildung 7.65 gezeigt.

Für p''_1 muß erneut eine Schnittpunktberechnung von $VR(p''_1)$, diesmal mit m' , durchgeführt werden. Der Übergang von p'_2 zu p''_2 kann aber in K (von oben nach unten betrachtet) nur einen Knick nach rechts zur Folge haben. Also schneidet m' $VR(p''_1)$ im Uhrzeigersinn nach s_1 (das ist links von s_1). Das Entsprechende gilt auch, wenn K mehrere Male hintereinander Voronoi-Regionen von Punkten aus P_2 passiert, bevor K $VR(p''_1)$ verläßt. Daher genügt es bei der wiederholten Schnittpunktberechnung für $VR(p''_1)$, nur die im Uhrzeigersinn auf den zuletzt berechneten Schnittpunkt folgenden Kanten (inklusive dieser Kanten selbst) zu inspizieren. Sobald ein (nächster) Schnittpunkt gefunden ist, müssen wegen der Konvexität von $VR(p''_1)$ für die gegebene

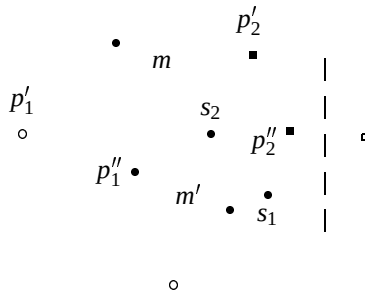


Abbildung 7.65

ne Mittelsenkrechte keine weiteren Kanten mehr betrachtet werden. Insgesamt werden so höchstens alle Kanten von $VR(p'_1)$ einmal betrachtet, zuzüglich der wiederholten Betrachtung je einer Kante für das Fortschreiten von K in P_2 .

Für wiederholtes Finden von Schnittpunkten für $VR(p'_2)$ gelten diese Betrachtungen entsprechend, wobei die Kanten von $VR(p'_2)$ entgegen dem Uhrzeigersinn besucht werden müssen.

Das Besuchen der Kanten einer Voronoi-Region im Gegenuhrzeigersinn ist (ebenso wie im Uhrzeigersinn, vgl. Abschnitt 7.7.3) in linearer Zeit möglich, weil alle Voronoi-Knoten nur mit drei Kanten inzidieren. Der Schnittpunkt einer Voronoi-Kante mit einer Geraden kann in konstanter Zeit berechnet werden; also können alle Schnittpunkte während der Konstruktion von K in linearer Zeit berechnet werden. Man kann sich leicht überlegen, wie man mit Hilfe der doppelt verketteten Kantenlisten der Voronoi-Diagramme für P_1 und P_2 und des Kantenzugs K eine doppelt verkettete Kantenliste des Voronoi-Diagramms für P erzeugt; wir überlassen es dem Leser, die Details auszufüllen.

Damit ist gezeigt, daß (rekursiv) aus $VD(P_1)$ und $VD(P_2)$ in linearer Zeit $VD(P)$ berechnet werden kann, daß also insgesamt das Voronoi-Diagramm $VD(P)$ für eine Menge P von N Punkten in $O(N \log N)$ Zeit bestimmt werden kann. Weil sich mit Hilfe des Voronoi-Diagramms sortieren läßt, ist diese Laufzeit optimal. Das Voronoi-Diagramm für N Punkte kann mit $O(N)$ Speicherplatzbedarf in Form einer doppelt verketteten Kantenliste abgespeichert werden.

7.7.5 Lösungen für Distanzprobleme

Wir wollen jetzt zeigen, wie das Voronoi-Diagramm zur Lösung der im Abschnitt 7.7.1 angegebenen Distanzprobleme eingesetzt werden kann.

Für das Problem, ein *dichtestes Punktepaar* (closest pair) in einer Menge P von N Punkten zu finden, ist eine optimale Lösung jetzt offensichtlich, da für jeden Punkt $p \in P$ jeder nächste Nachbar $p' \in P$ von p eine an $VR(p)$ angrenzende Voronoi-Region $VR(p')$ hat. Das Problem kann also wie folgt gelöst werden:

Algorithmus Dichtestes Punktepaar

{liefert zu einer Menge P von N Punkten in der Ebene ein Paar von Punkten mit minimaler Distanz unter allen Punktepaaren in P }

1. Konstruiere das Voronoi-Diagramm $VD(P)$ für P .
2. Durchlaufe die doppelt verkettete Kantenliste für $VD(P)$ und ermittle dabei das Minimum der Distanz benachbarter Punkte sowie ein Punktepaar, das diese Distanz realisiert.

Schritt 1 kann gemäß Abschnitt 7.7.4 in $O(N \log N)$ Zeit ausgeführt werden. Da die Anzahl der Knoten und Kanten des Voronoi-Diagramms mit $O(N)$ beschränkt ist und da zu jeder Voronoi-Kante ein Paar benachbarter Punkte gehört, ist Schritt 2 sogar in $O(N)$ Zeit ausführbar. Insgesamt ergibt sich also eine Laufzeit von $O(N \log N)$; diese Laufzeit ist optimal (vgl. Abschnitt 7.7.1).

Das Problem, *alle nächsten Nachbarn* (all nearest neighbors) zu finden, löst man völlig analog.

Algorithmus Alle nächsten Nachbarn

{liefert zu einer Menge P von N Punkten in der Ebene zu jedem Punkt in P einen nächsten Nachbarn in P , also eine Menge von N Punktepaaren}

1. Konstruiere das Voronoi-Diagramm $VD(P)$ für P .
2. Durchlaufe die doppelt verkettete Kantenliste für $VD(P)$ so, daß der Reihe nach für jeden Punkt p alle Voronoi-Kanten von $VR(p)$ betrachtet werden. Dabei wird für jeden Punkt ein nächster Nachbar unter allen Punkten mit benachbarter Voronoi-Region ermittelt.

Schritt 1 kann wiederum in $O(N \log N)$ Zeit ausgeführt werden und Schritt 2 benötigt sogar nur $O(N)$ Zeit, weil das zu einer Voronoi-Kante gehörige Punktepaar p, p' höchstens zweimal, nämlich bei der Bestimmung eines nächsten Nachbarn für p und für p' , betrachtet wird. Damit ist die gesamte Laufzeit $O(N \log N)$; das ist gemäß Abschnitt 7.7.1 optimal.

Das Problem, einen *minimalen spannenden Baum* (minimum spanning tree) für einen Graphen mit Kantenbewertungen zu finden, wird im Kapitel 8 ausführlich behandelt. Wir wollen hier ein Verfahren auf den Fall einer Menge von Punkten in der Ebene spezialisieren.

Algorithmus: *Minimaler spannender Baum*

{liefert zu einer Menge P von N Punkten in der Ebene einen minimalen spannenden Baum für P in Gestalt einer Menge von Kanten}

1. Beginne mit einer Menge von Bäumen, wobei jeder Baum ein Punkt der Menge ist.
2. Solange noch mehr als ein Baum vorhanden ist, führe aus:
 - 2.1. Wähle einen Baum T aus.
 - 2.2. Finde ein Punktepaar $p, p' \in P$ so, daß p zu T gehört, p' nicht zu T gehört und $d(p, p')$ minimal ist unter allen solchen Punktepaaren.
 - 2.3. Sei T' der Baum, zu dem p' gehört. Verbinde T und T' durch die Kante zwischen p und p' ; T und T' werden aus der Menge der Bäume gelöscht, und der neu entstandene Baum wird dort eingetragen.

Der entscheidende Schritt ist das Finden eines Paares dichtester Punkte, Schritt 2.2. Alle anderen Teile können effizient implementiert werden, wie in Kapitel 8 beschrieben. Es ist natürlich ineffizient, jedes Punktepaar in Schritt 2.2 zu überprüfen. Hier ist das Voronoi-Diagramm die entscheidende Hilfe, denn die Voronoi-Regionen eines in Schritt 2.2 gewählten Punktepaars müssen aneinander angrenzen.

Allgemein gilt für eine beliebige Aufteilung der Punktmenge P in disjunkte Teilmengen P_1 und P_2 , daß die kürzeste Verbindung zweier Punkte, von denen einer zu P_1 und einer zu P_2 gehört, zwischen Punkten mit angrenzenden Voronoi-Regionen realisiert wird. Um dies einzusehen, nehmen wir an, p'_1 und p'_2 seien zwei Punkte, die minimale Distanz zwischen P_1 und P_2 realisieren, mit $p'_1 \in P_1$ und $p'_2 \in P_2$. Wenn nun die Voronoi-Region $VR(p'_2)$ nicht an $VR(p'_1)$ angrenzt, so liegt der Mittelpunkt p_m der Verbindungsstrecke zwischen p'_1 und p'_2 außerhalb von $VR(p'_1)$. Damit schneidet der Rand von $VR(p'_1)$ die Verbindungsstrecke $p'_1 p'_2$ in einem Punkt p'_1 , der näher bei p'_1 liegt als p_m . Die Voronoi-Kante von $VR(p'_1)$ durch p'_1 trennt $VR(p'_1)$ und $VR(p')$, für einen Punkt $p' \in P$. Dieser Punkt p' liegt auf dem Kreis mit Radius $d(p'_1, p'_1)$ um den Punkt p'_1 , also jedenfalls innerhalb des Kreises mit Radius $d(p_m, p'_1)$ um Punkt p'_1 . Diese Situation ist in Abbildung 7.66 illustriert.

Damit ist $d(p', p'_1) < d(p'_2, p'_1)$ und auch $d(p', p'_2) < d(p'_2, p'_1)$. Ob nun p' zu P_1 oder P_2 gehört, stets ist die Folge, daß p'_1 und p'_2 kein Punktepaar mit minimaler Distanz zwischen P_1 und P_2 gewesen sein kann. Also grenzen die Voronoi-Regionen der Punkte p'_1 und p'_2 aneinander.

Damit genügt es, bei der Suche nach einem Punktepaar mit minimalem Abstand in Schritt 2.2 nur Punktepaare mit angrenzender Voronoi-Region zu betrachten. Der minimale spannende Baum ist also als Teil des zum Voronoi-Diagramm dualen Graphen mit geradlinigen Kanten, der Delaunay-Triangulierung, konstruierbar. Einen minimalen spannenden Baum für unser Beispiel zeigt die Abbildung 7.67.

Die Berechnung eines minimalen spannenden Baumes für einen Graphen mit N Knoten und Kanten kann in Zeit $O(N \log N)$ ausgeführt werden, wie wir in Kapitel 8 zeigen werden; für planare Graphen genügt sogar Zeit $O(N)$. Damit kann ein minimaler spannender Baum für eine Menge von Punkten in Zeit $O(N \log N)$ berechnet werden: Man berechnet das Voronoi-Diagramm in Zeit $O(N \log N)$, bildet den dualen Graphen, die Delaunay-Triangulierung, in Zeit $O(N)$ durch Durchlaufen der doppelt verketteten

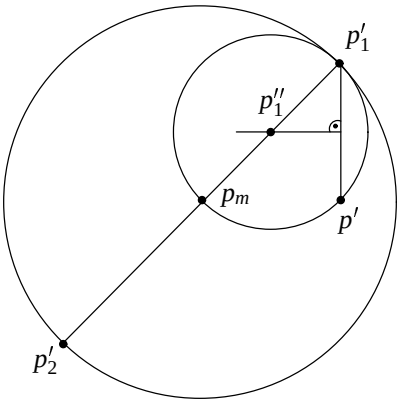


Abbildung 7.66

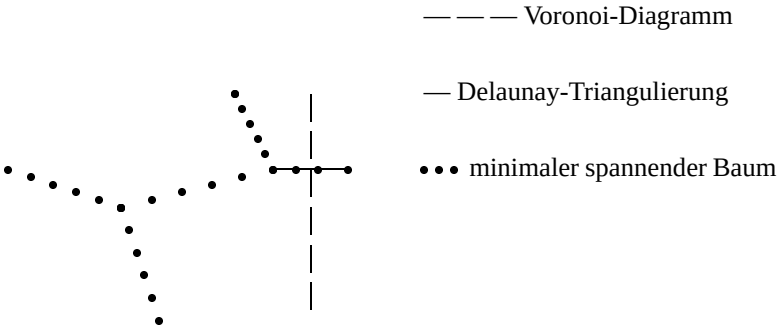


Abbildung 7.67

Kantenliste für das Voronoi-Diagramm und berechnet anschließend einen minimalen spannenden Baum der Delaunay-Triangulierung. Daß dies optimal ist, wurde bereits in Abschnitt 7.7.1 gezeigt.

Die bisher gelösten Probleme waren allesamt Probleme mit fest gegebener Objektmenge und vorgegebener Frage. Betrachten wir nun eine Lösung zum Problem der Anfrage nach einem nächsten Nachbarn bei gegebener, fester Punktmenge P für beliebige, zunächst unbekannte Anfragepunkte. Da viele dieser Anfragen beantwortet werden sollen, wollen wir mit einigem Vorbereitungsaufwand P so präparieren, daß Anfragen effizient beantwortet werden können. Die *Suche nach einem nächsten Nachbarn* für einen Anfragepunkt (nearest neighbor search, best match), wird dann in zwei Schritten erledigt:

Algorithmus 1 Vorbereitung für “Suche nächsten Nachbarn”

{liefert zu einer Menge P von N Punkten in der Ebene eine Datenstruktur für P mit einer effizienten Unterstützung der Suchanfrage}

Algorithmus 2 Suche nächsten Nachbarn

{liefert zu einem Anfragepunkt q der Ebene einen nächsten Nachbarn $p \in P$ }

Verwende die angebotene Suchanfrageoperation für P und q .

Wir müssen also lediglich noch den ersten Teil, den Vorbereitungsschritt, präzisieren. Dabei hilft wieder das Voronoi-Diagramm. Für einen Anfragepunkt q ist ein solcher Punkt $p \in P$ nächster Nachbar unter allen Punkten aus P , in dessen Voronoi-Region q fällt; $VR(p)$ war ja gerade entsprechend definiert (vgl. Abschnitt 7.7.2). Die zu unterstützende Operation für beliebiges q ist also das Finden der (einer) Voronoi-Region $VR(p)$, die q enthält (ein *point location problem*). Auch wenn das Voronoi-Diagramm als bekannt vorausgesetzt wird, ist diese Operation nicht ganz einfach effizient ausführbar. Unter den verschiedenen Methoden hierfür wollen wir eine näher betrachten, die Methode der *hierarchischen Triangulierung* 

Zunächst wird das zu betrachtende Gebiet trianguliert, also in Dreiecke zerlegt, deren Ecken aus der vorgegebenen Punktmenge stammen. In unserem Fall ist dies die Menge der Voronoi-Punkte, also *nicht* die Menge P . Da Voronoi-Regionen im allgemeinen mehr als drei Kanten besitzen, müssen sie in Dreiecke unterteilt werden; unbeschränkte Voronoi-Regionen werden hier zunächst ignoriert. Die entstehende Triangulierung der beschränkten Voronoi-Regionen umgeben wir mit einem Dreieck; die Differenzregion wird ebenfalls trianguliert. Die Abbildung 7.68 zeigt eine solche Triangulierung für unser Beispiel.

Die Anzahl der Dreiecke einer solchen Triangulierung ist proportional zur Anzahl der Voronoi-Knoten, also linear in der Anzahl N der Punkte in P . Die Triangulierung läßt sich in $O(N \log N)$ Schritten ermitteln, etwa mit Hilfe eines Scan-Line-Verfahrens.

In dieser Triangulierung des Voronoi-Diagramms von P suchen wir nun nach einem Dreieck, das q enthält. Ist das gefundene Dreieck Teil einer beschränkten Voronoi-Region $VR(p)$, so ist p nächster Nachbar von q ; andernfalls ist das gefundene Dreieck Teil einer unbeschränkten Voronoi-Region oder q liegt ganz außerhalb des umschließenden Dreiecks. Dann führen wir eine binäre Suche auf den zyklisch geordneten unbeschränkten Voronoi-Regionen (repräsentiert durch die trennenden Halbgeraden) aus, um einen Punkt zu finden, in dessen Voronoi-Region q liegt. Diese Suche kann in

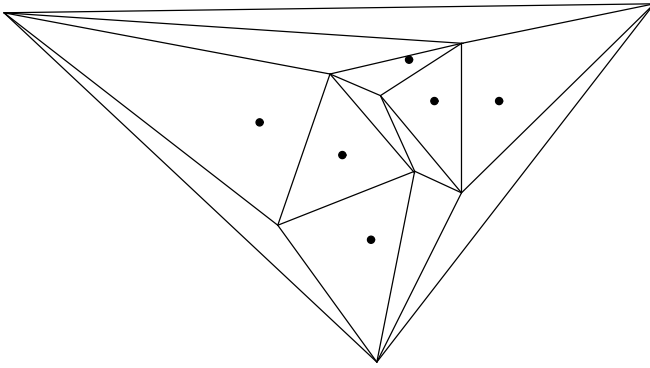


Abbildung 7.68

$O(\log N)$ Schritten beendet werden, wenn N die Anzahl der Punkte in P ist. Wesentlich für die Effizienz der Beantwortung der Suchanfrage ist jetzt noch, daß wir das Dreieck der Triangulierung, das q enthält, schnell finden.

Zu diesem Zweck vergrößern wir die bisher betrachtete Triangulierung in mehreren Schritten, bis wir schließlich nur noch ein Dreieck vorfinden. Ein Vergrößerungsschritt besteht darin, eine Menge von Punkten, die nicht durch Kanten verbunden sind (unabhängige Punkte) und nicht auf dem Rand der Triangulierung liegen, zusammen mit ihren inzidenten Kanten zu entfernen und die entstehenden polygonalen Gebiete neu zu triangulieren. Ein Vergrößerungsschritt macht also aus einer Triangulierung eine größere Triangulierung. Wir wenden nacheinander mehrere Vergrößerungsschritte an, bis die Triangulierung schließlich nur noch aus einem einzigen Dreieck besteht. Die Abbildungen 7.69 bis 7.73 zeigen eine Folge von fünf Triangulierungen für unser Beispiel. Mit $\odot$ markierte Punkte werden im nächsten Schritt entfernt; Kanten, die im letzten Schritt hinzugenommen wurden, sind gestrichelt gezeichnet. Die Dreiecke sind für spätere Bezugnahme mit Namen versehen.

Suchen wir nun mit einem Anfragepunkt q nach einem Dreieck der feinsten Triangulierung, das q enthält, so beginnen wir die Suche mit der größten Triangulierung. Für diese stellen wir fest, ob q überhaupt im Dreieck liegt. Dieser Test kann für einen gegebenen Punkt und ein gegebenes Dreieck in einer konstanten Anzahl von Schritten ausgeführt werden. Dann fahren wir mit der Suche in der nächstfeineren Triangulierung fort. Dort inspizieren wir alle Dreiecke, die mit dem soeben betrachteten einen nichtleeren Durchschnitt haben, denn nur in diesen Dreiecken kann q liegen. Eines der inspizierten Dreiecke muß q enthalten. Wir setzen das Verfahren mit diesem Dreieck und der nächstfeineren Triangulierung fort, bis wir schließlich in der feinsten Triangulierung dasjenige Dreieck bestimmt haben, das q enthält.

Um diesen Suchvorgang zu unterstützen, wird zunächst aus der Hierarchie der Triangulierungen eine spezielle verkettete Suchstruktur gebildet. Jeder Knoten der Suchstruktur repräsentiert ein Dreieck. Ein ausgezeichneter Knoten (die Wurzel) repräsentiert das Dreieck der größten Triangulierung. Die Blätter der Struktur repräsentieren die Dreiecke der feinsten Triangulierung. Für jedes im Verlauf der Vergrößerung der

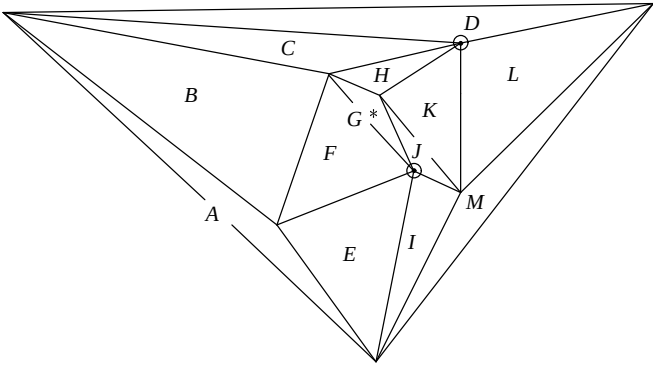


Abbildung 7.69

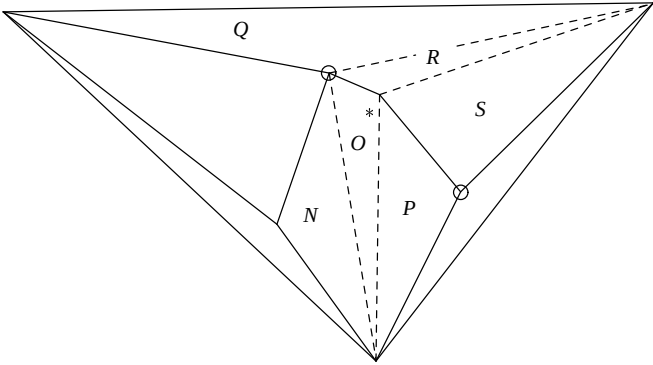


Abbildung 7.70

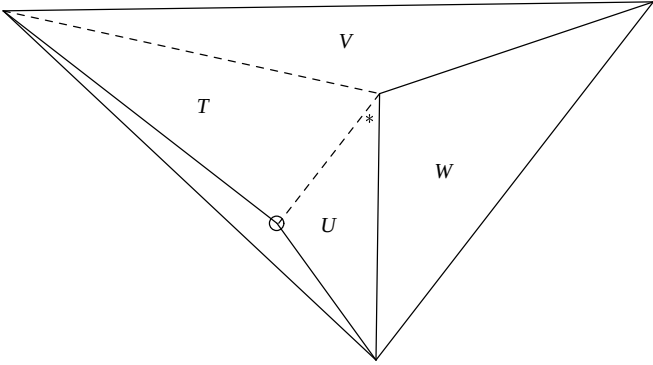


Abbildung 7.71

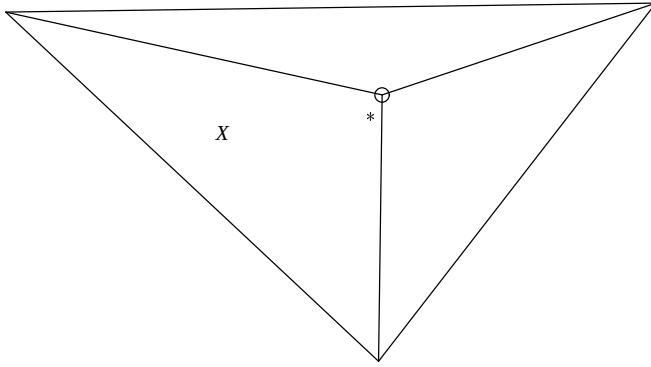


Abbildung 7.72

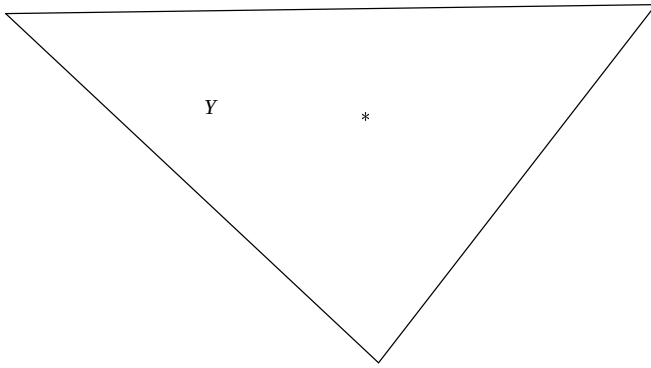


Abbildung 7.73

Triangulierung neugebildete Dreieck gibt es einen Knoten zwischen der Wurzel und den Blättern (inklusive der Wurzel selbst, die ja auch ein neugebildetes Dreieck repräsentiert). Die Verweise der Knoten untereinander sind wie folgt angelegt: Ein Knoten k , der Dreieck d repräsentiert, besitzt einen Zeiger auf Knoten k' mit Dreieck d' genau dann, wenn in einem Vergrößerungsschritt von Triangulierung T' zu Triangulierung T Dreieck d' entfernt wurde, Dreieck d neu entstand und d und d' sich überlappen.

Für unser Beispiel sieht die Struktur für die Hierarchie der Triangulierungen wie in Abbildung 7.74 gezeigt aus.

Verfolgen wir die Suche nach dem mit * in den Triangulierungen eingetragenen Punkt q^* . Zunächst stellen wir fest, ob q^* im Dreieck Y liegt. Da dies der Fall ist, prüfen wir für alle Nachfolger des Knotens Y , ob q^* im zugehörigen Dreieck liegt. Der Test mit V , W und X ergibt, daß q^* in X liegt. Jetzt ist X aktueller Knoten, und das Verfahren wird fortgesetzt. Unter den Nachfolgern T , U und A von X ist U das q^* enthaltende Dreieck. Von N und O enthält O q^* , und schließlich ist aus E , F und G das q^* enthaltende Dreieck

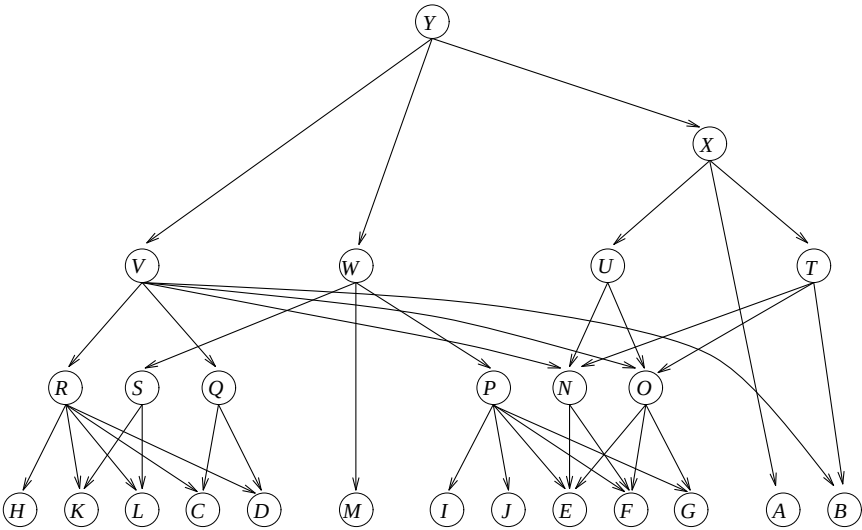


Abbildung 7.74

G. Da dies ein Blatt ist, sind wir bei der feinsten Triangulierung angelangt, und die zu Dreieck G gehörende Voronoi-Region $VR(p)$, die beispielsweise über einen weiteren Zeiger erreichbar ist, enthält q^* . Damit ist p_1 nächster Nachbar von q^* .

Die Laufzeit dieses Verfahrens hängt ab von der Länge des längsten Pfades in der Suchstruktur von der Wurzel zu einem Blatt und von der Anzahl der Nachfolger von Knoten. Das erstere ist gerade die Anzahl der Triangulierungen (Vergrößerungsschritte), das letztere die Anzahl der Dreiecke, die ein neugebildetes Dreieck in der nächstfeineren Triangulierung höchstens überlappen kann. Für beides ist offenbar die Wahl der zu entfernenden Punkte in einem Vergrößerungsschritt maßgebend.

Die Suche nach dem Elementardreieck (Dreieck der feinsten Triangulierung), das einen gegebenen Punkt enthält, kann nicht schneller als in $\Omega(\log N)$ Zeit für $\Theta(N)$ Elementardreiecke ausgeführt werden, weil dies schon eine untere Schranke für die Suche im eindimensionalen Fall ist. Ein Suchverfahren ist also sicher optimal, wenn es mit $O(\log N)$ Schritten auskommt. Das ist der Fall, wenn es höchstens $O(\log N)$ Triangulierungen gibt und wenn jedes neugebildete Dreieck höchstens eine konstante Anzahl von Dreiecken der nächstfeineren Triangulierung überlappt. Dann werden nämlich bei der Suche nur $O(\log N)$ Knoten insgesamt betrachtet. Weil die Anzahl der Elementardreiecke proportional ist zur Anzahl der Voronoi-Knoten und weil diese wiederum proportional ist zur Anzahl N der gegebenen Punkte, ergibt sich damit ein Suchverfahren, mit dem die Suche nach dem nächsten Nachbarn in Zeit $O(\log N)$ ausgeführt werden kann, das also optimal ist.

Auch der Speicherbedarf für eine solche Suchstruktur ist minimal: Da es insgesamt $\Theta(N)$ Knoten in dieser Struktur gibt, von denen jeder nur konstant viele Verweise speichert, genügt $\Theta(N)$ Speicherplatz.

Wir wollen nun die von Kirkpatrick vorgeschlagene Wahl für die zu entfernenden Punkte angeben, die beide gestellten Bedingungen erfüllt. Daß die Anzahl aller Trian-

gulierungen durch $O(\log N)$ beschränkt ist, zeigen wir, indem wir nachweisen, daß sich bei jedem Vergrößerungsschritt die Anzahl der Punkte einer Triangulierung mindestens um einen konstanten Faktor verringert. Die Regel für das Entfernen von Punkten ist dann die folgende: Entferne eine Menge unabhängiger Punkte, die jeweils einen Grad kleiner als g haben; g ist eine sorgfältig gewählte Konstante.

In einem Vergrößerungsschritt inspizieren wir also in beliebiger Reihenfolge alle Punkte der Triangulierung, die nicht auf dem Rand liegen, und entfernen jeden Punkt mit Grad kleiner als g , es sei denn, einer seiner Nachbarn ist bereits entfernt worden. Es ist offensichtlich, daß dann jedes neu gebildete Dreieck nur weniger als g alte Dreiecke überlappen kann.

Um zu zeigen, daß stets mindestens ein fester Anteil aller Punkte auf diese Weise entfernt werden kann, folgen wir dem Gang der vereinfachten Argumentation aus [1], die eine asymptotische Aussage abzuleiten gestattet. Nach Euler gibt es in einer Triangulierung mit $n = \Theta(N)$ Punkten genau $3n - 6$ Kanten, wenn der Rand der Triangulierung ein Dreieck ist. Summiert man die Grade aller Punkte, so ergibt sich ein Wert kleiner als $6n$, weil jede der $3n - 6$ Kanten zum Grad von genau 2 Punkten beiträgt. Also muß es mindestens $n/2$ Punkte mit Grad kleiner als 12 geben (sonst würde die Summe der Grade der $n/2$ Punkte mit höchsten Graden schon mindestens $6n$ betragen).

Wählen wir also für den das Entfernen bestimmenden Grad g den Wert 12. Wenn ein Punkt mit Grad kleiner als 12 entfernt wird, so können seine Nachbarn nicht mehr entfernt werden; die Anzahl dieser Nachbarn ist der Grad des Punktes, also weniger als 12. Von allen Punkten mit Grad kleiner als 12 können also gegebenenfalls die drei Eckpunkte auf dem Rand der Triangulierung nicht entfernt werden, und von den verbleibenden Punkten kann im schlechtesten Fall nur $\frac{1}{12}$ entfernt werden. Die Anzahl v der zu entfernenden Punkte ist also nach unten beschränkt durch

$$v \geq \lfloor \frac{1}{12}(\frac{n}{2} - 3) \rfloor$$

Der Anteil β der mindestens zu entfernenden Punkte unter n Punkten ist dann

$$\beta = \frac{v}{n} \geq \frac{1}{24} - \frac{1}{4n}$$

Für genügend großes n , etwa $n \geq 12$, ist dies

$$\beta \geq \frac{1}{24} - \frac{1}{48} = \frac{1}{48} > 0$$

Damit ist gezeigt, daß stets ein fester (wenn auch sehr kleiner) Bruchteil aller Punkte in einem Vergrößerungsschritt entfernt wird und folglich die Anzahl aller Triangulierungen mit $O(\log N)$ beschränkt ist. Also arbeitet der beschriebene Algorithmus zur Suche eines nächsten Nachbarn in einer Menge von N Punkten in optimaler Zeit, mit $O(\log N)$ Schritten.

Das Herstellen der hierarchischen Triangulierung beginnt mit der Berechnung des Voronoi-Diagramms in $O(N \log N)$ Schritten. Dann wird die feinste Triangulierung in $O(N \log N)$ Schritten berechnet. In jedem Vergrößerungsschritt werden alle Punkte und alle Kanten der jeweiligen Triangulierung inspiziert, um zu entscheiden, welche Punkte entfernt werden. Da jeweils ein fester Bruchteil aller Punkte (und damit auch aller Kanten) entfernt wird, inspiziert man somit insgesamt $O(N)$ Punkte und Kanten. Für jeden entfernten Punkt muß ein neu entstandenes Polygon trianguliert werden. Da dieses Polygon aber nur konstant viele Kanten besitzt (nämlich weniger als g), kann eine solche Triangulierung in konstanter Zeit gefunden werden. Für alle $O(N)$ Triangulierungen genügen also insgesamt $O(N)$ Schritte. Die Zeiger der Suchstruktur ergeben sich dabei asymptotisch ohne zusätzlichen Aufwand. Damit genügen $O(N \log N)$ Schritte für das Herstellen der Hierarchie der Triangulierungen.

Die Methode der hierarchischen Triangulierungen ist also ein effizientes Verfahren, um eine Suchstruktur über einer beliebig gegebenen Zerlegung eines Gebietes in Polygone zu konstruieren. Besteht die anfänglich gegebene Zerlegung aus insgesamt n Kanten, so kann die Suchstruktur der hierarchischen Triangulierungen in $O(n \log n)$ Zeit konstruiert werden; sie benötigt $O(n)$ Speicherplatz. Zu einem beliebigen Anfragepunkt kann dann das Polygon der ursprünglichen Zerlegung, in das der Anfragepunkt fällt, in Zeit $O(\log n)$ bestimmt werden.

7.8 Aufgaben

Aufgabe 7.1

Wir betrachten n Geraden in der Ebene, die in allgemeiner Position liegen sollen, d. h. keine drei Geraden schneiden sich in einem Punkt und keine Geraden sind parallel, vgl. das Beispiel in Abbildung 7.75.

- Zeigen Sie, daß sich die Geraden in $\binom{n}{2}$ Punkten schneiden.
- Zeigen Sie, daß die Geraden die Ebene in $\binom{n+1}{2} + 1$ Gebiete unterteilen. (Hinweis: Verwenden Sie eine imaginäre Scan-line, die von $x = -\infty$ bis $x = +\infty$ über die Geraden gleitet, als Zählhilfe. Betrachten Sie, wie sich die Anzahl der Gebiete bei Überquerung eines Schnittpunktes verändert. Sie können voraussetzen, daß es keine vertikalen Geraden gibt.)
- Berechnen Sie die Anzahl der Kanten, d. h. der Liniensegmente zwischen zwei Schnittpunkten und der Halbgeraden, auf denen sich kein Schnittpunkt befindet, die sich durch die n Geraden ergeben.

Aufgabe 7.2

Geben Sie an, in welcher Reihenfolge die Schnittpunkte in der folgenden Menge von Liniensegmenten in der Ebene berichtet werden, wenn Sie eine Scan-line von links nach rechts über die Ebene schwenken (Abbildung 7.76).

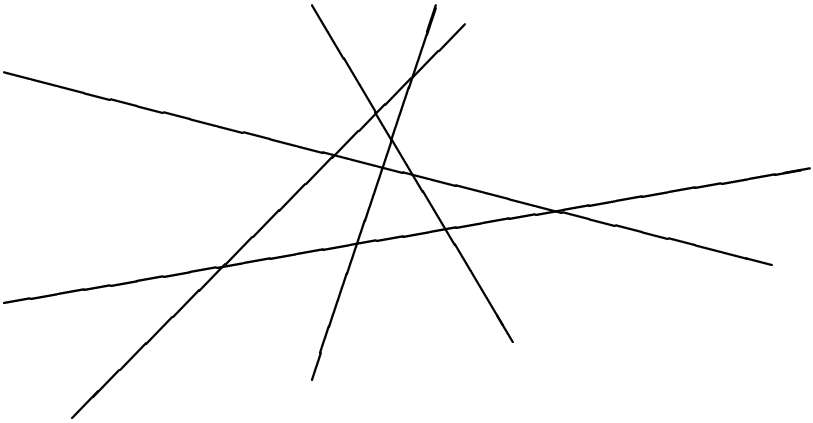


Abbildung 7.75: 5 Gerade zerteilen die Ebene in $\binom{6}{2} + 1 = 16$ Gebiete.

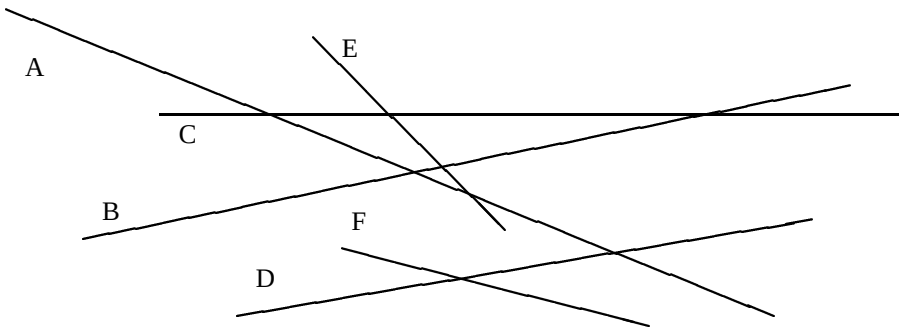


Abbildung 7.76

Aufgabe 7.3

Geben Sie ein Beispiel mit der kleinstmöglichen Anzahl von Liniensegmenten an, so daß der erste durch das Scan-line-Verfahren gefundene Schnittpunkt nicht der am weitesten links liegende ist.

Aufgabe 7.4

- a) Warum kann ein Scan-line-Verfahren für ein Problem der Größe n nie weniger als $cn \log n$ Schritte benötigen, für ein konstantes $c \in \mathbb{R}$?

- b) Betrachten Sie das folgende, sogenannte *Element-uniqueness Problem*: Zu einer Zahlenfolge von n reellen Zahlen ist festzustellen, ob in der Folge zwei gleiche Zahlen vorkommen. Man kann zeigen, daß zur Lösung dieses Problems mindestens $\Omega(n \log n)$ Schritte benötigt werden.

Zeigen Sie, daß es mindestens ebenso schwierig ist festzustellen, ob sich n horizontale und n vertikale Liniensegmente schneiden. (Hinweis: Nehmen Sie an, Sie haben ein Verfahren für das Schnittpunktproblem gegeben. Zeigen Sie, daß Sie durch eine geschickte Transformation das Element-uniqueness Problem lösen können. Sie können voraussetzen, daß auch einpunktige Liniensegmente zugelassen sind.)

Aufgabe 7.5

Wir betrachten n Punkte in der Ebene. Für zwei Punkte $x = (x_1, x_2)$ und $y = (y_1, y_2)$ sagen wir x *dominiert* y , falls $x_1 \geq y_1$ und $x_2 \geq y_2$. Ein Punkt ist *maximal*, wenn er von keinem anderen dominiert wird. Geben Sie ein möglichst effizientes Verfahren an, das alle maximalen Punkte berechnet.

Aufgabe 7.6

Wird eine Menge von Liniensegmenten, die wir uns als undurchsichtige Mauern vorstellen können, von einer punktförmigen Lichtquelle beschienen, so sind, im allgemeinen nur Teile der Segmente beleuchtet. Geben Sie ein möglichst effizientes Verfahren zur Berechnung der beleuchteten Segmentteile an und diskutieren Sie dessen Komplexität. (Hinweis: Verwenden Sie eine um die Lichtquelle rotierende Scan-line, wie in Abbildung 7.77 gezeigt.)

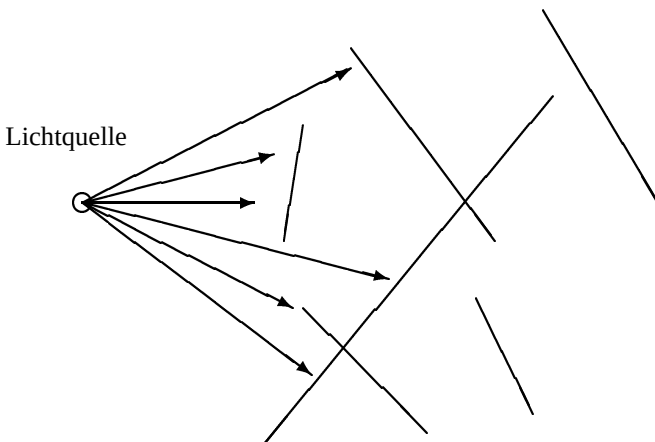


Abbildung 7.77: Ein Beispiel für eine Menge von Segmenten, die beleuchtet wird.

Aufgabe 7.7

Gegeben seien die horizontalen Segmente $A, \dots, H$, die durch ihre linken und rechten Endpunkte repräsentiert sind, sowie die vertikalen Segmente $a, \dots, g$. Durch (wiederholte) Aufteilung der Menge infolge rekursiver Aufrufe des Divide-and-conquer-Verfahrens zur Bestimmung aller Liniensegmentschnitte sind die in Abbildung 7.78 gezeigten Mengen S_1 und S_2 entstanden.

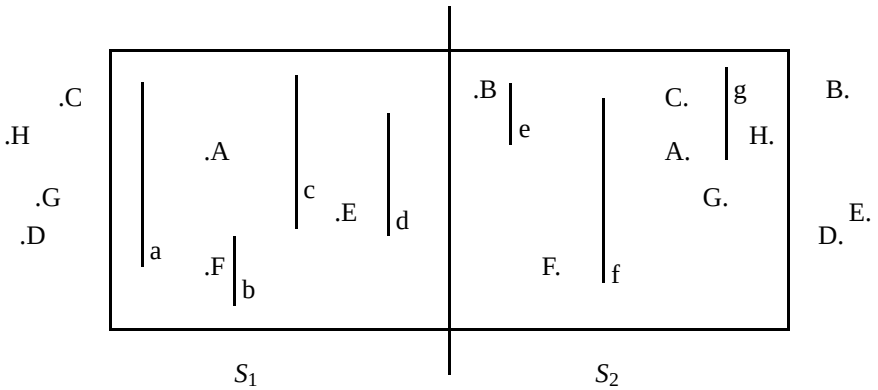


Abbildung 7.78

X bezeichnet den linken Endpunkt und $X.$ den rechten Endpunkt des Segments X . Geben Sie an, welche Segmentschnitte im Merge-Schritt (bei Vereinigung von S_1 und S_2) noch berichtet werden müssen.

Aufgabe 7.8

Wir betrachten eine Menge P von n Punkten in der Ebene. Die *konvexe Hülle* $\text{conv}(P)$ von P ist die kleinste konvexe Menge, die P enthält; $\text{conv}(P)$ ist offensichtlich ein konvexes Polygon. Geben Sie ein möglichst effizientes Verfahren zur Berechnung von $\text{conv}(P)$ an.

Aufgabe 7.9

Gegeben sei die Menge $\{A, B, C, D, E, F\}$ von Intervallen mit

$$A = [2, 3], B = [5, 9], C = [1, 4], D = [3, 7], E = [6, 8] \text{ und } F = [8, 10].$$

- Geben Sie einen Intervallbaum möglichst geringer Höhe zur Speicherung dieser Intervallmenge an.
- Führen Sie eine Aufspießanfrage für den Punkt $x = 3$ durch und geben Sie an, in welcher Reihenfolge die aufgespießten Intervalle entdeckt werden (ausgehend vom Intervallbaum aus a)).

Aufgabe 7.10

Bei der im Abschnitt 7.4.2 vorgestellten Version von Segment-Bäumen waren die Knotenlisten nicht-sortierte, doppelt verkettete Listen von Intervallnamen. Zusätzlich wurde (um das Entfernen von Intervallnamen zu unterstützen) ein separates Wörterbuch für alle Intervalle aufrechterhalten.

Überlegen Sie sich eine andere, möglichst effiziente Möglichkeit zur Organisation der Knotenlisten, die es erlaubt, auf das zusätzliche Wörterbuch zu verzichten. Geben Sie eine möglichst genaue Abschätzung der Worst-case-Laufzeit der Einfüge- und Entferne-Operation in Ihrer Datenstruktur an.

Aufgabe 7.11

Entwerfen Sie einen möglichst effizienten Algorithmus zur Lösung des folgenden Problems:

Gegeben sei eine Menge von n Rechtecken in der Ebene. Es ist der Umfang der von den Rechtecken gebildeten Polygone zu bestimmen. Unter Umfang sei die Länge des Randes einschließlich des Randes der entstehenden Löcher verstanden, vgl. das Beispiel in Abbildung 7.79.

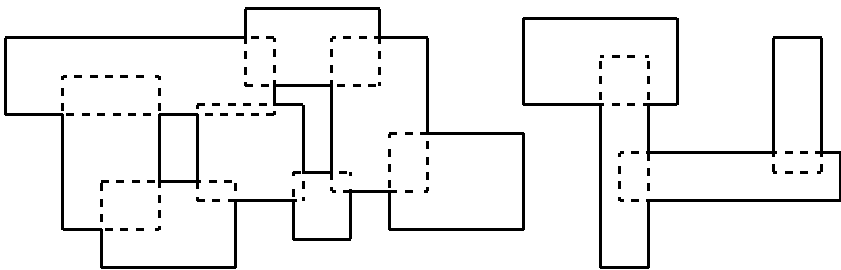


Abbildung 7.79: Die Länge der durchgezogenen Linie ist zu berechnen.

Formulieren Sie den Algorithmus in Pseudo-Pascal und geben Sie insbesondere Ihre Prozeduren zur Manipulation der verwendeten Datenstrukturen an. Analysieren Sie die Komplexität des von Ihnen verwendeten Verfahrens.

(Hinweis: Sie können davon ausgehen, daß die x -Koordinaten der vertikalen Seiten und die y -Koordinaten der horizontalen Seiten jeweils paarweise verschieden sind. Es ist ratsam, zwei Scan-line Durchläufe zu verwenden, womit man eine Laufzeit von $O(n \log n)$ erreichen kann.)

Aufgabe 7.12

Gegeben sie die folgende Menge von Punkten in der Ebene:

$(3,7)$ $(4,2)$ $(5,8)$ $(2,1)$ $(1,4)$ $(6,3)$ $(7,9)$ $(8,5)$

- a) Fügen Sie die Punkte der Reihe nach in das anfangs leere Skelett eines Prioritäts-Suchbaumes ein.

- b) Bestimmen Sie die Menge aller Punkte im Bereich $3 \leq x \leq 6$ und $y \leq 5$ durch eine Bereichsanfrage im Prioritäts-Suchbaum.
- c) Entfernen Sie die Punkte in der umgekehrten Reihenfolge aus dem Prioritäts-Suchbaum.

Aufgabe 7.13

Entwickeln Sie einen Einfügealgorithmus für einen balancierten Prioritäts-Suchbaum, der das Einfügen eines Punktes in logarithmischer Zeit ermöglicht. Verwenden Sie als zugrunde liegende Baumstruktur Rot-schwarz-Bäume.

Aufgabe 7.14

Ein Kantenzug C heißt monoton, falls jede horizontale Gerade C in höchstens einem Punkt schneidet. Ein Polygon P heißt monoton, falls der Rand von P in zwei monotone Kantenzüge zerlegt werden kann.

- a) Wieviele Kantenschnitte können zwei Polygone mit n_1 und n_2 Kanten maximal miteinander haben? Wieviele sind es, falls beide Polygone monoton sind?
- b) In wieviele Zickzacks zerfällt eine Menge von beliebigen Polygonen, monotonen Polygonen oder konvexen Polygonen, falls die Polygone insgesamt n Kanten haben?

Aufgabe 7.15

Zerlegen Sie die in Abbildung 7.80 dargestellte Menge von Polygonen in Zickzacks und bestimmen Sie die für einen Scan von oben nach unten geeignete Ordnung der Top-Segmente.

Aufgabe 7.16

Das *Slot-Assignment-Problem* für eine Menge horizontaler Liniensegmente in der Ebene ist folgendes Problem:

Finde die kleinste Zahl m (die minimale Slot-Anzahl) und eine Numerierung der Segmente mit "Slot-Nummern" aus $1, \dots, m$ derart, daß gilt: Für jede vertikale Gerade, die irgendwelche horizontalen Segmente schneidet, sind die Slot-Nummern der geschnittenen Segmente längs der Geraden absteigend (aber nicht notwendigerweise lückenlos) sortiert.

- a) Lösen Sie das Slot-Assignment-Problem für die Menge von Segmenten aus Abbildung 7.81.
- b) Geben Sie ein allgemeines Verfahren zur Lösung des Slot-Assignment-Problems an und analysieren Sie die Laufzeit Ihres Verfahrens.

(Hinweis: Es ist möglich, das Slot-Assignment-Problem für eine Menge von n Segmenten in Zeit $O(n \log n)$ und Platz $O(n)$ zu lösen!)

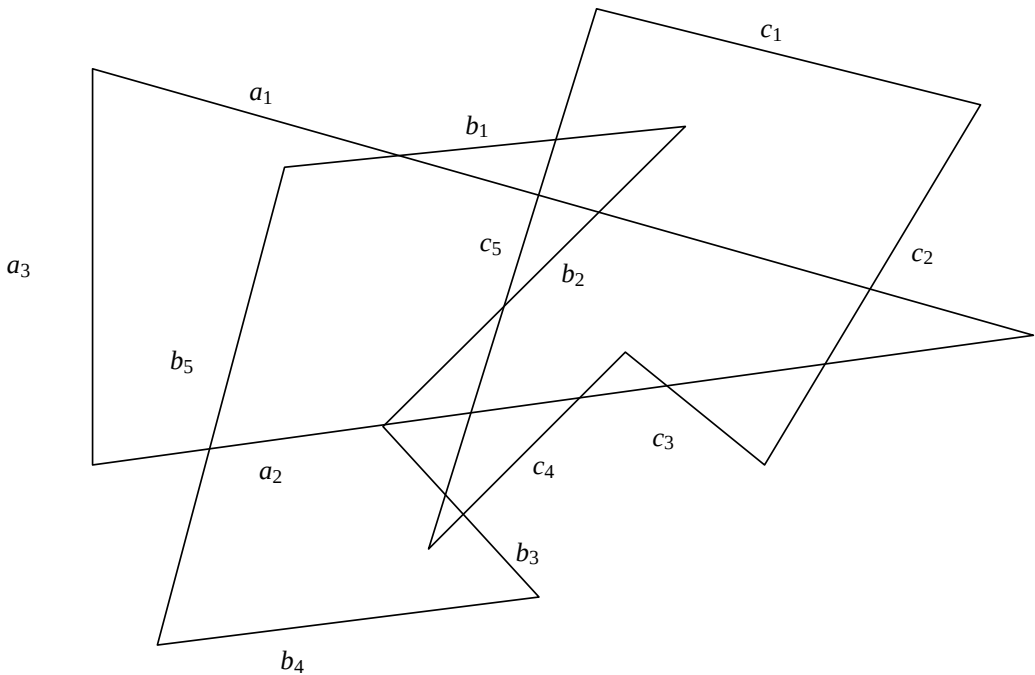


Abbildung 7.80

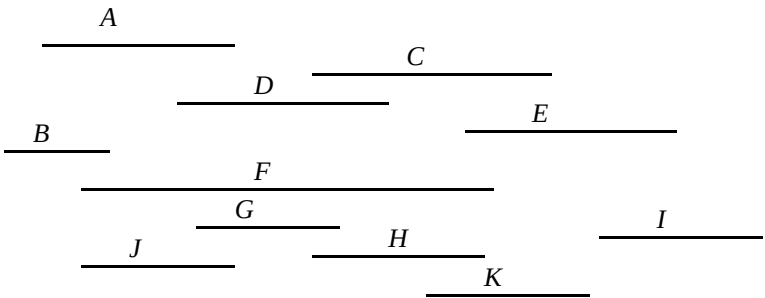


Abbildung 7.81

Aufgabe 7.17

Sei P eine Menge von n Punkten in der Ebene, von denen keine vier auf einem Kreis liegen, und $p_1, p_2, p_3 \in P$. Beweisen Sie, daß

- das Dreieck mit den Eckpunkten p_1, p_2 und p_3 genau dann ein Teil der Delaunay-Triangulierung ist, wenn der Kreis durch p_1, p_2 und p_3 keine weiteren Punkte aus P enthält;
- das Liniensegment von p_1 nach p_2 genau dann ein Teil der Delaunay-Triangulierung ist, wenn es einen Kreis K durch p_1 und p_2 gibt, der keine anderen Punkte von P enthält.

Aufgabe 7.18

Sei P eine Menge von n Punkten in der Ebene, von denen keine vier auf einem Kreis liegen. Der *Gabriel-Graph* $G(P)$ von P ist wie folgt definiert: Eine Kante $e = \langle p_1, p_2 \rangle$ mit $p_1, p_2 \in P$ gehört zu $G(P)$, falls für alle $p_3 \in P \setminus \{p_1, p_2\}$ gilt, daß

$$d^2(p_1, p_3) + d^2(p_2, p_3) > d^2(p_1, p_2)$$

ist, wobei d den euklidischen Abstand zwischen zwei Punkten bezeichnet.

- Zeigen Sie, daß der minimale spannende Baum von P ein Teilgraph des Gabriel-Graphen ist.
- Zeigen Sie, daß jede Kante des Gabriel-Graphen $G(P)$ auch eine Kante der Delaunay-Triangulierung $DT(P)$ ist (Hinweis: Beachten Sie Aufgabe 7.17).
- Zeigen Sie, daß $e \in DT(P)$ genau dann eine Kante von $G(P)$ ist, falls e die Kante e' des Voronoi-Diagramms schneidet — wobei e' die zu e duale Kante ist (vgl. Abbildung 7.82).

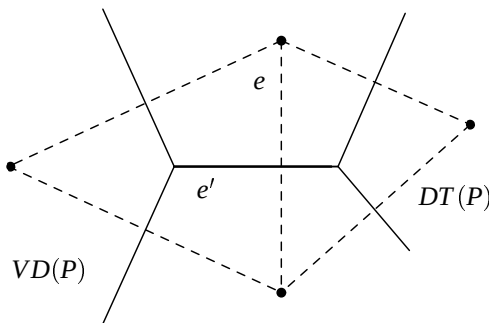


Abbildung 7.82: Eine Kante der Delaunay-Triangulierung schneidet die duale Voronoi-Kante.

- Geben Sie einen Algorithmus an, der in einer Zeit von $O(n)$ den Gabriel-Graphen berechnet, falls die Delaunay-Triangulierung gegeben ist.

Aufgabe 7.19

Geben Sie einen Algorithmus an, der zu einer gegebenen Menge von Punkten in linearer Zeit die konvexe Hülle berechnet, falls das Voronoi-Diagramm der Punkte schon vorliegt.

Aufgabe 7.20

Gegeben sei die Menge P von sieben Punkten in der Ebene, wie in Abbildung 7.83 dargestellt.

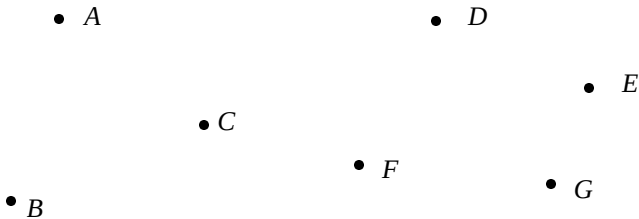


Abbildung 7.83

- Konstruieren Sie das Voronoi-Diagramm für diese Punktmenge.
- Geben Sie die Delaunay-Triangulierung von P an.
- Geben Sie den Gabriel-Graphen und einen minimalen spannenden Baum für P an.
- Geben Sie eine doubly connected edge list an, beschränkt auf alle Kanten der Voronoi-Regionen der Punkte A , B und C .
- Zeigen Sie graphisch, wie man aus den Voronoi-Diagrammen für die Mengen $\{A, B, C\}$ und $\{D, E, F, G\}$ das Voronoi-Diagramm für die gesamte Punktmenge konstruieren kann (Merge-Schritt des Divide-and-conquer-Algorithmus).

Aufgabe 7.21

Entwerfen Sie einen Algorithmus, der für einen gegebenen Punkt und ein konvexes Polygon testet, ob der Punkt innerhalb oder außerhalb dieses Polygons liegt. Nehmen Sie an, daß die Eckpunkte des Polygons als ein nach Winkeln sortiertes Array vorliegen. Bestimmen Sie den Aufwand ihres Verfahrens. (Hinweis: Das Verfahren sollte nicht mehr als $O(\log n)$ Schritte benötigen, falls n die Anzahl der Kanten des Polygons ist.)

Literaturliste zu Kapitel 7: Geometrische Algorithmen

Seite 419

- [166] M. I. Shamos. Computational Geometry. Dissertation, Dept. of Comput. Sci., Yale University, 1978.
- [59] A. R. Forrest. Guest editor's introduction to special issue on computational geometry. ACM Transactions on Graphics, 3(4):241-243, 1984.
- [42] H. Edelsbrunner. Algorithms in Combinatorial Geometry. Springer, Berlin, 1987.
- [149] F. P. Preparata und M. I. Shamos. Computational Geometry: An Introduction. Springer, 1985.
- [43] H. Edelsbrunner und J. van Leeuwen. Multidimensional data structures and algorithms, a bibliography. Technical Report 104, IIG, Technische Universität Graz, 1983.
- [183] G. Toussaint, Hrsg. Computational Geometry. Elsevier North-Holland, N. Y., 1985.
- [105] D. T. Lee und F. P. Preparata. Computational geometry - a survey. IEEE Transactions on Computers, C-33(12):1072-1102, 1984.
- [122] K. Mehlhorn. Data structures and algorithms, Vol. 3: Multidimensional searching and computational geometry. Springer, Berlin, 1984.

Seite 420

- [195] D. Wood. An isothetic view of computational geometry. Technical Report CS-84-01, Department of Computer Science, University of Waterloo, Jan. 1984.

Seite 421

- [130] J. Nievergelt und F. P. Preparata. Plane-sweep algorithms for intersecting geometric figures. Comm. ACM, 25:739-747, 1982.

Seite 422

- [163] M. Schlag, F. Luccio, P. Maestrini, D. T. Lee und C. K. Wong. A visibility problem in VLSI layout compaction. In F. P. Preparata, Hrsg., Advances in Computing Research, volume 2, S. 259-282. JAI Press, 1985.
- [108] T. Lengauer. Efficient algorithms for the constraint generation for integrated circuit layout compaction. In M. Nagl und J. Perl, Hrsg., Proc. WG'83, GraphTheoretic Concepts in Computer Science, Osnabrück, S. 219-230, Linz, 1983. Trauner.

Seite 425

- [163] M. Schlag, F. Luccio, P. Maestrini, D. T. Lee und C. K. Wong. A visibility problem in VLSI layout compaction. In F. P. Preparata, Hrsg., Advances in Computing Research, volume 2, S. 259-282. JAI Press, 1985.

Seite 427

- [149] F. P. Preparata und M. I. Shamos. Computational Geometry: An Introduction. Springer, 1985.

Seite 434

- [28] B. M. Chazelle. Reporting and counting arbitrary planar intersections. Technical Report CS-83-16, Dept. of Comp. Sci., Brown University, Providence, R.I., 1983.
- [29] B. M. Chazelle und H. Edelsbrunner. An optimal algorithm for intersecting line segments in the plane. In Proc. 29th Annual Symposium on Foundations of Computer Science, White Plains, S. 590-600, 1988.

Seite 435

- [24] K. Q. Brown. Comments on "Algorithms for reporting and counting geometric intersections". IEEE Transactions on Computers, C-29:147-148, 1980.
- [71] R. H. Güting. Optimal divide-and-conquer to compute measure and contour for a set of iso-oriented rectangles. Acta Informatica, 21:271-291, 1984.

Seite 444

[73] R. H. Güting und D. Wood. Finding rectangle intersections by divide-and conquer. IEEE Transactions on Computers, C-33:671-675, 1984.

[71] R. H. Güting. Optimal divide-and-conquer to compute measure and contour for a set of iso-oriented rectangles. Acta Informatica, 21:271-291, 1984.

Seite 457

[41] H. Edelsbrunner. Dynamic data structures for orthogonal intersection queries. Technical Report 59, IIG, Technische Universität Graz, 1980.

[119] E. M. McCreight. Efficient algorithms for enumerating intersecting intervals and rectangles. Technical Report PARC CSL-80-9, Xerox Palo Alto Res. Ctr., Palo Alto, CA, 1980.

Seite 466

[120] E.M. McCreight. Priority search trees. SIAM J. Comput., 14(2):257-276, 1985.

Seite 471

[195] D. Wood. An isothetic view of computational geometry. Technical Report CS-84-01, Department of Computer Science, University of Waterloo, Jan. 1984.

Seite 476

[139] Th. Ottmann und P. Widmayer. On the placement of line segments into a skeleton structure. Technical Report 114, Institut für Angewandte Informatik und Formale Beschreibungsverfahren Universität Karlsruhe, 1982.

Seite 484

[140] Th. Ottmann, P. Widmayer und D. Wood. A worst-case efficient algorithm for hidden line elimination. International Journal Comp. Math., 18:93-119, 1985.

[72] R.H. Güting und Th. Ottmann. New algorithms for special cases of the hidden line elimination problem. Computer Vision and Image Processing, 40:188-204, 1987.

Seite 494

[120] E.M. McCreight. Priority search trees. SIAM J. Comput., 14(2):257-276, 1985.

Seite 496

[88] R. Klein, O. Nurmi, Th. Ottmann und D. Wood. A dynamic fixed windowing problem. Algorithmica, 4:535-550, 1989.

Seite 497

[149] F. P. Preparata und M. I. Shamos. Computational Geometry: An Introduction. Springer, 1985.

[122] K. Mehlhorn. Data structures and algorithms, Vol. 3: Multidimensional searching and computational geometry. Springer, Berlin, 1984.

[105] D. T. Lee und F. P. Preparata. Computational geometry - a survey. IEEE Transactions on Computers, C-33(12):1072-1102, 1984.

[142] Th. Ottmann und D. Wood. Dynamical sets of points. Computer Vision, Graphics, and Image Processing, 27:157-166, 1984.

Seite 501

[149] F. P. Preparata und M. I. Shamos. Computational Geometry: An Introduction. Springer, 1985.

[189] G. Voronoi. Nouvelles applications des paramé'tres continus a` la the'orie des formes quadratiques. Deuxie`me Me'moire: Recherches sur les paralle'loe`dres primitifs. J. Reine angew.

Seite 505

- [34] B. Delaunay. Sur la sphère vide. Bull. Acad. Sci. USSR Sci. Mat. Nat., 7:793-800, 1934.
- [149] F.P. Preparata und M.I. Shamos. Computational Geometry: An Introduction. Springer, 1985.
- [126] D. E. Muller und F. P. Preparata. Finding the intersection of two convex polyhedra. Theoretical Computer Science, 7(2):217-236, 1978.

Seite 507

- [149] F. P. Preparata und M.I. Shamos. Computational Geometry: An Introduction. Springer, 1985.

Seite 512

- [167] M. I. Shamos und D. Hoey. Closest-point problems. In Proc. 16th Annual Symposium on Foundations of Computer Science, S. 151-162, 1975.

Seite 519

- [87] D. G. Kirkpatrick. Optimal search in planar subdivisions. SIAM J. Comput., 12(1):28-35, 1983.

Seite 524

- [149] F. P. Preparata und M. I. Shamos. Computational Geometry: An Introduction. Springer, 1985.

Kapitel 8

Graphenalgorithmen

Wie komme ich am schnellsten von Freiburg nach Königsberg, dem heutigen Kaliningrad? Wie komme ich am billigsten von Freiburg nach Königsberg? Wie transportiere ich ein Gut am billigsten von mehreren Anbietern zu mehreren Nachfragern? Wie ordne ich die Arbeitskräfte meiner Firma am besten denjenigen Tätigkeiten zu, für die sie geeignet sind? Wann kann ich frühestens mit meinem Hausbau fertig sein, wenn die einzelnen Arbeiten in der richtigen Reihenfolge ausgeführt werden? Wie besuche ich alle meine Kunden mit einer kürzestmöglichen Rundreise? Welche Wassermenge kann die Kanalisation in Freiburg höchstens verkraften? Wie muß ein Rundweg durch Königsberg aussehen, auf dem ich jede Brücke über den Pregel genau einmal überquere und am Schluß zum Ausgangspunkt zurückkomme? Diese und viele andere Probleme lassen sich als Probleme in Graphen formulieren und mit Hilfe von Graphenalgorithmen lösen. In einem Graphen wird dabei die wesentliche Struktur des Problems, befreit von unbedeutenden Nebenaspekten, repräsentiert.

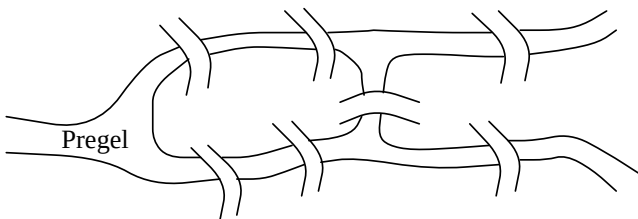


Abbildung 8.1

Abbildung 8.1 zeigt einen (verzerrten) Ausschnitt aus dem Stadtplan von Königsberg, Abbildung 8.2 zeigt den dazugehörigen Graphen. Das Wesentliche am Königsberger Brückenproblem ist die Verbindungsstruktur der einzelnen Stadtteile gemäß den sieben Brücken. Jeder Stadtteil ist im Graphen durch einen Punkt, genannt Knoten, wiedergegeben; eine Verbindung ist eine Linie von einem Knoten zu einem anderen Knoten,

genannt Kante. In unserem Beispiel entspricht eine Verbindung gerade einer Brücke. Bereits 1736 löste Euler  das Königsberger Brückenproblem: Er stellte fest, daß der gewünschte Rundweg nicht möglich ist.

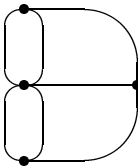


Abbildung 8.2

Im Laufe dieses Kapitels werden wir Beispiele für andere Graphenprobleme und entsprechende Lösungsalgorithmen kennenlernen. Insbesondere kann man sich vorstellen, daß Verbindungen — anders als beim Königsberger Brückenproblem — mit einer Richtung ausgezeichnet sind und in Gegenrichtung nicht benutzt werden dürfen, wie etwa Einbahnstraßen in einer Stadt. Ähnliches gilt bei der Kanalisation oder beim Hausbau (vgl. Abbildung 8.3, bei der ein Pfeil einem Vorgang entspricht). Betrachten wir zunächst solche Graphen.

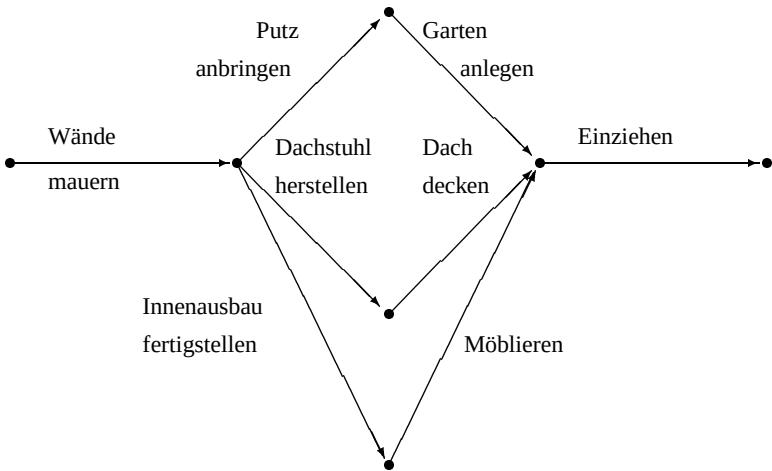


Abbildung 8.3

Ein *gerichteter Graph* $G = (V, E)$ (englisch: *digraph*) besteht aus einer Menge $V = \{1, 2, \dots, |V|\}$ von *Knoten* (englisch: *vertices*) und einer Menge $E \subseteq V \times V$ von *Pfeilen* (englisch: *edges, arcs*). Ein Paar $(v, v') \in E$ heißt *Pfeil von v nach v'*. Wir nennen v den

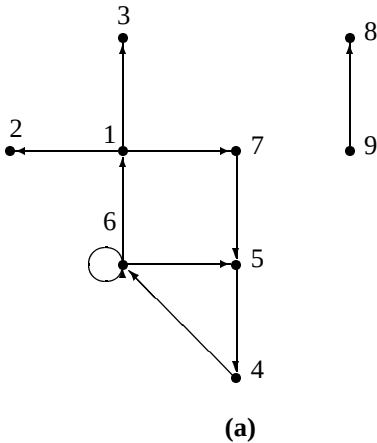
Anfangs- und v' den Endknoten des Pfeils (v, v') ; v und v' heißen auch *adjacent*; v (und ebenso v') heißt mit e *inzident*; ebenso nennen wir e inzident mit v und v' . Wir werden Knoten eines Graphen stets als Punkte, Pfeile als Verbindungslinien mit einer auf den Endknoten gerichteten Pfeilspitze darstellen. Wir beschränken uns auf endliche Mengen von Knoten und Pfeilen, also auf *endliche Graphen*; weil E eine Menge ist, kann in diesen Graphen jeder Pfeil höchstens einmal auftreten (wir erlauben keine *parallelen* Pfeile).

Für die Effizienz von Graphenalgorithmen, sowohl im Hinblick auf Speicherplatz als auch im Hinblick auf Laufzeit, ist es wichtig, Graphen geeignet zu speichern. Wir betrachten drei naheliegende Möglichkeiten der Speicherung eines Graphen $G = (V, E)$.

Speicherung in einer Adjazenzmatrix

Ein Graph $G = (V, E)$ wird in einer Boole'schen $|V| \times |V|$ -Matrix $A_G = (a_{ij})$, mit $1 \leq i \leq |V|$, $1 \leq j \leq |V|$ gespeichert, wobei

$$a_{ij} = \begin{cases} 0 & \text{falls } (i, j) \notin E; \\ 1 & \text{falls } (i, j) \in E. \end{cases}$$

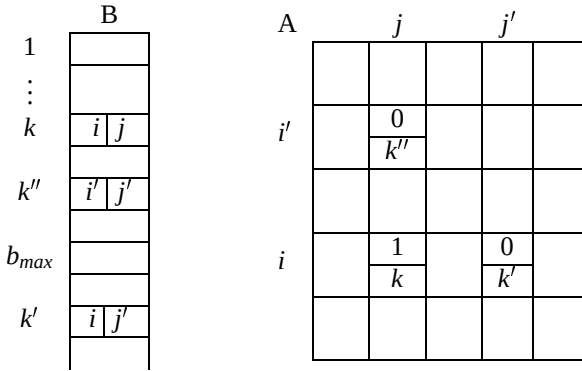


	1	2	3	4	5	6	7	8	9
1	0	1	1	0	0	0	1	0	0
2	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	1	0	0	0
5	0	0	0	1	0	0	0	0	0
6	1	0	0	0	1	1	0	0	0
7	0	0	0	0	1	0	0	0	0
8	0	0	0	0	0	0	0	0	0
9	0	0	0	0	0	0	0	1	0

(b)

Abbildung 8.4

Abbildung 8.4 (b) ist die Adjazenzmatrix zum Graphen aus Abbildung 8.4 (a).



$A[i, j]$ ist bedeutsam, $A[i, j']$ und $A[i', j]$ sind es nicht.

Abbildung 8.5

Bei der Speicherung eines Graphen mit Knotenmenge V in einer Adjazenzmatrix ergibt sich ein Speicherbedarf von $\Theta(|V|^2)$. Dieser Speicherbedarf ist nicht abhängig von der Anzahl der Pfeile im Graphen; enthält der Graph vergleichsweise wenige Pfeile, so ist der Speicherplatzbedarf vergleichsweise hoch. Verwendet man die Adjazenzmatrix *ohne Zusatzinformation*, so benötigen die meisten Algorithmen wegen der erforderlichen Initialisierung der Matrix oder der Berücksichtigung aller Einträge der Matrix $\Omega(|V|^2)$ Rechenschritte.

Dem läßt sich aber mit Zusatzinformationen abhelfen, die den Platzbedarf nicht über $O(|V|^2)$ hinaus erhöhen. Dies gelingt mit einem zusätzlichen Feld B , das für jeden in der Adjazenzmatrix benutzten Eintrag einen Feldeintrag enthält; für in der Adjazenzmatrix zwar vorhandene, aber nicht mit einer Bedeutung belegte Einträge gibt es im Feld keinen Eintrag (vgl. Abbildung 8.5).

Nun geht es darum, für gegebenen Zeilenindex i und Spaltenindex j der Matrix A festzustellen, ob $A[i, j]$ eine Bedeutung besitzt, also einen bereits benutzten Eintrag bezeichnet. Dazu speichern wir mit $A[i, j]$ neben dem gewünschten Bit für die Adjazenz von Knoten i mit Knoten j einen Index k des Feldes B . Im Feld B werden an Stelle k die Matrixindizes i und j gespeichert, wenn der Matrixeintrag Bedeutung besitzt. Im Feld B sind stets die Einträge mit Indizes 1 bis b_{max} bedeutsam. Setzen wir die Definitionen

```

const    knotenzahl = {Anzahl |V| der Knoten};
           pfeilzahl = {Anzahl |E| der Pfeile};
type     knotentyp = 1 .. knotenzahl;
           pfeiltyp = 1 .. pfeilzahl;
           bit = 0 .. 1;
           matrixeintrag = record
                           adjacent : bit;
                           index : pfeiltyp
                           end;
    
```

```

feldeintrag = record
               zeile, spalte : knotentyp
            end;
matrix = array [knotentyp, knotentyp] of matrixeintrag;
feld = array [pfeiltyp] of feldeintrag;
var   A : matrix;
       B : feld;
       i, j : knotentyp;
       bmax : pfeiltyp

```

voraus, so ist ein Eintrag $A[i, j]$ genau dann *bedeutsam* (echt, gültig), wenn $1 \leq A[i, j].index \leq bmax$, $B[A[i, j].index].zeile = i$ und $B[A[i, j].index].spalte = j$ gelten. Damit ist es gelungen, die Initialisierung der Matrix A durch die Initialisierung des Feldes B zu ersetzen:

```

{Initialisiere A;}
{Initialisiere B;}
bmax := 0

```

Die Laufzeit von Graphenalgorithmen bei Verwendung einer Adjazenzmatrix ist also nicht unbedingt durch $\Omega(|V|^2)$ nach unten beschränkt. Trotzdem bleiben typische Operationen, wie etwa das Inspizieren aller von einem gegebenen Knoten ausgehenden Pfeile, für Graphen mit wenigen Pfeilen ineffizient. Betrachten wir nun eine hierfür besser geeignete Speicherungsform.

Speicherung in Adjazenzlisten

Hier wird für jeden Knoten eine lineare, verkettete Liste der von diesem Knoten ausgehenden Pfeile gespeichert. Die Knoten werden als lineares Feld von $|V|$ Anfangszeigern auf je eine solche Liste verwaltet. Abbildung 8.6 zeigt Adjazenzlisten für den Graphen aus Abbildung 8.4 (a).

Die i -te Liste enthält ein Listenelement mit Eintrag j für jeden Endknoten eines Pfeils $(i, j) \in E$. In pascalähnlicher Notation läßt sich diese Struktur wie folgt definieren:

```

const   knotenzahl = {Anzahl |V| der Knoten};
type    knotentyp = 1 .. knotenzahl;
          pfeilzeiger =  $\uparrow$ pfeilelement;
          pfeilelement = record
                           endknoten : knotentyp;
                           next : pfeilzeiger
            end;
          feld = array [knotentyp] of pfeilzeiger;
var     adjazenzlisten : feld

```

Für einen Graphen $G = (V, E)$ benötigen Adjazenzlisten $\Theta(|V| + |E|)$ Speicherplätze. Adjazenzlisten unterstützen viele Operationen, z.B. das Verfolgen von Pfeilen in Graphen, sehr gut. Andere Operationen dagegen werden nur schlecht unterstützt, insbesondere das Hinzufügen und Entfernen von Knoten.

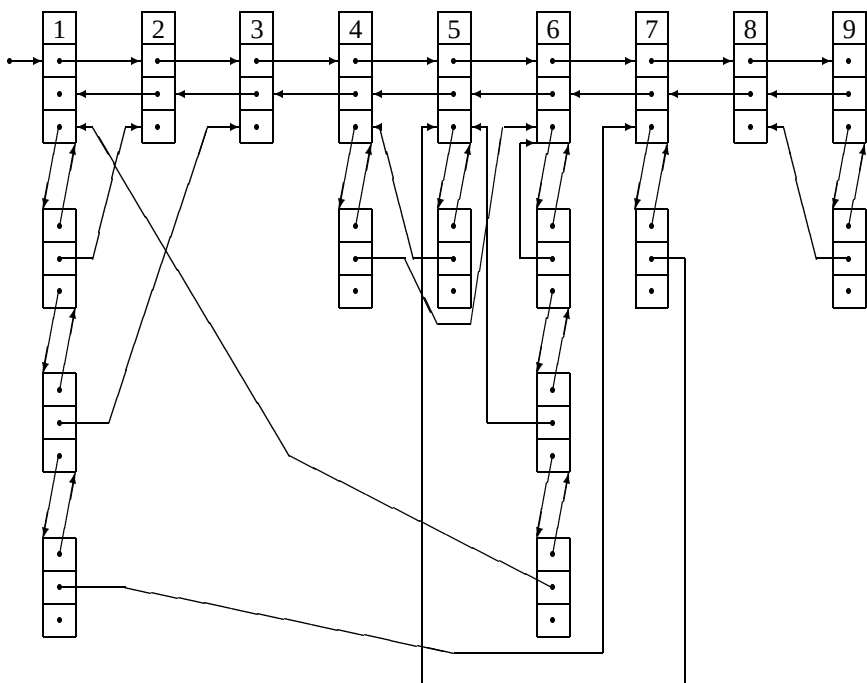


Abbildung 8.7

```

case pfeillistenanfang : boolean of
  true : (kno : knotenzeiger);
  false : (pre : pfeilzeiger)

```

```

end;

```

```

var    dcal : knotenzeiger;

```

Wegen der etwas einfacheren Struktur werden wir die Adjazenzlistenrepräsentation von Graphen überall dort der DCAL vorziehen, wo sich dies nicht negativ auf die Effizienz von Algorithmen auswirkt.

Bevor wir uns nun die algorithmische Lösung einiger Graphenprobleme genauer ansehen, wollen wir wichtige Grundbegriffe der Graphentheorie kurz rekapitulieren. Weitergehende Definitionen findet man in Standardlehrbüchern zur Graphentheorie und zu Graphenalgorithmen           und teilweise auch in Lehrbüchern über Algorithmen und Datenstrukturen.

Sei $G = (V, E)$ ein gerichteter Graph (englisch: *directed graph*; *Digraph*). Der *Eingangsgrad* (englisch: *indegree*) $\text{indeg}(v)$ eines Knotens v ist die Anzahl der in v einmündenden Pfeile, also $\text{indeg}(v) = |\{v' \mid (v', v) \in E\}|$. Im Digraphen des Beispiels der Abbildung 8.8 ist $\text{indeg}(0) = 1$, $\text{indeg}(2) = 2$. Der *Ausgangsgrad* (englisch: *outdegree*) $\text{outdeg}(v)$ ist die Anzahl der von v ausgehenden Pfeile, also $\text{outdeg}(v) = |\{v' \mid (v, v') \in E\}|$. Ein Digraph $G' = (V', E')$ ist ein *Teilgraph* von $G = (V, E)$, geschrieben als $G' \subseteq G$, falls

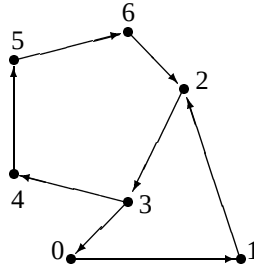


Abbildung 8.8

$V' \subseteq V$ und $E' \subseteq E$ ist. Für $V' \subseteq V$ induziert V' den Teilgraphen $(V', E \cap (V' \times V'))$, auch *Untergraph* genannt. Im durch V' induzierten Teilgraphen findet man also *alle* Pfeile aus E wieder, die lediglich mit Knoten aus V' inzidieren. Der durch $V - V'$ induzierte Teilgraph von G wird als $G - V'$ notiert; für einelementiges $V' = \{v'\}$ schreiben wir auch $G - v'$. Für den Digraphen der Abbildung 8.8 ist mit $V' = \{0, 3, 4, 5\}$ der Graph $(V', \{(3, 0), (4, 5)\})$ ein Teilgraph; der Graph $G' = (V', \{(3, 0), (3, 4), (4, 5)\})$ ist der durch V' induzierte Teilgraph.

Ein *Weg* (englisch: *path*) von v nach v' , wobei $v, v' \in V$, ist der durch eine Folge $(v_0, v_1, \dots, v_k)$ von Knoten mit $v_0 = v$, $v_k = v'$ und $(v_i, v_{i+1}) \in E$ für $0 \leq i < k$ beschriebene Teilgraph $G' = (V', E')$ von G , für den $V' = \{v_0, v_1, \dots, v_k\}$ und $E' = \{(v_i, v_{i+1}) \mid 0 \leq i < k\}$; k ist die *Länge* des Weges. Für jedes $v \in V$ gibt es also den trivialen Weg von v nach v mit Länge 0. In dem in Abbildung 8.8 gezeigten Digraphen ist beispielsweise die Knotenfolge $(2, 3, 4, 5, 6, 2, 3, 0)$ ein Weg von Knoten 2 nach Knoten 0.

Ein Weg heißt *einfach*, wenn kein Knoten mehrfach besucht wird, d.h., wenn für alle i, j mit $0 \leq i < j \leq k$ gilt, daß $v_i \neq v_j$ ist. Der im Beispiel genannte Weg im Digraph der Abbildung 8.8 ist also nicht einfach; Weg $(0, 1, 2, 3, 4)$ dagegen ist einfach.

Ein *Zyklus* ist ein Weg, der am Ausgangsknoten endet, also ein Weg von einem Knoten v nach v . Wir wollen im folgenden der Einfachheit halber triviale Wege und triviale Zyklen, also Wege und Zyklen, die nur aus einem Knoten und keinem Pfeil bestehen, aus unseren Betrachtungen ausschließen. Ein Digraph heißt *zyklenfrei* oder *azyklisch*, wenn er keinen Zyklus enthält. Der Digraph aus Abbildung 8.8 ist also nicht zyklenfrei: Er enthält die beiden (einfachen) Zyklen $(2, 3, 4, 5, 6, 2)$ und $(0, 1, 2, 3, 0)$.

Manchmal interessieren wir uns für Wege, die nur einen Teil aller Pfeile benutzen. Für $F \subseteq E$ schreiben wir $v \xrightarrow{*}_F v'$ genau dann, wenn es einen Weg von v nach v' gibt, der nur Pfeile aus F benutzt. Wenn $v \xrightarrow{*}_E v'$ gilt, so bezeichnen wir v' als von v aus *erreichbar*.

Wir haben Bäume und Ansammlungen von Bäumen bereits in anderen Kapiteln als Datenstrukturen kennengelernt. Auch als Graphen haben sie eine besondere Bedeutung. Ein Digraph $G = (V, E)$ heißt *gerichteter Wald*, wenn E zyklenfrei ist und $\text{indeg}(v) \leq 1$ für alle $v \in V$. Jeder Knoten v mit $\text{indeg}(v) = 0$ ist eine *Wurzel* des Waldes. Ein gerichteter Wald mit genau einer Wurzel ist ein *gerichteter Baum* (*Wurzelbaum*). Wie wir schon von der Datenstruktur Baum wissen, gibt es in einem gerichteten Baum von der Wurzel zu jedem Knoten genau einen Weg. Im Beispiel der Abbildung 8.8 ist der

oben beschriebene Teilgraph $(\{0, 3, 4, 5\}, \{(3, 0), (4, 5)\})$ ein gerichteter Wald mit Wurzeln 3 und 4; der von $\{0, 3, 4, 5\}$ induzierte Untergraph ist ein Baum mit Wurzel 3.

Für einen Knoten v eines gerichteten Baums ist der *Teilbaum mit Wurzel v* der von den *Nachfolgern* $\{v' \mid v \rightarrow_E^* v'\}$ von v induzierte Teilgraph. Für manche Berechnungen benötigen wir einen Wald, der alle Knoten eines gegebenen Digraphen enthält. Für einen Digraphen $G = (V, E)$ ist ein gerichteter Wald $W = (V, F)$ mit $F \subseteq E$ ein *spannender Wald* von G . Falls W ein Baum ist, heißt W *spannender Baum* von G .

In vielen Fällen kommt es uns auf die Richtung von Verbindungen zwischen Knoten nicht an. Dann vernachlässigen wir die Richtung von Pfeilen, beispielsweise indem wir erzwingen, daß zwischen zwei Knoten entweder kein Pfeil oder in jeder der beiden Richtungen ein Pfeil verläuft. Ein solcher Digraph $G = (V, E)$, für den $(v, v') \in E \iff (v', v) \in E$, heißt *ungerichteter Graph* oder einfach *Graph*. Ein Paar $((v, v'), (v', v))$ von Pfeilen heißt *Kante*. Abhängig vom modellierten Problem repräsentiert eine Kante eine in beiden Richtungen gleichzeitig benutzbare Verbindung, wie etwa eine Straße, oder eine wahlweise in jeder der beiden Richtungen — aber nicht gleichzeitig — benutzbare Verbindung, wie etwa ein Eisenbahngleis. Der Grad $\deg(v)$ eines Knotens v ist gerade gleich $\text{indeg}(v)$ (und ebenfalls $\text{outdeg}(v)$), also die Anzahl der mit v inzidenten Kanten. Ein ungerichteter Graph heißt *zyklenfrei* oder *azyklisch*, falls er keinen einfachen Zyklus mit wenigstens mit drei Pfeilen enthält (natürlich enthält jeder Graph mit einer Kante bereits einen Zyklus aus zwei Pfeilen). Die übrigen Definitionen im Zusammenhang mit gerichteten Graphen gelten entsprechend.

Wir werden eine Kante der Übersicht wegen stets als *eine* Verbindungslinie ohne Pfeilspitze zeichnen und als (v, v') notieren, wobei die Reihenfolge der Knoten ohne Bedeutung ist (manche Autoren verwenden auch $[v, v']$). Davon machen wir beispielsweise im Algorithmus zur Berechnung der zweifachen Zusammenhangskomponenten Gebrauch (siehe Abschnitt 8.4.1). Beide Knoten v und v' der Kante $(v, v') = (v', v)$ werden als Endknoten bezeichnet.

8.1 Topologische Sortierung

Ein Digraph kann stets als eine binäre Relation angesehen werden; ein zyklenfreier Digraph beschreibt also eine Halbordnung. Liest man etwa einen Pfeil als „ist teurer als“, so stößt man beim Betrachten des in Abbildung 8.4 (a) dargestellten Digraphen auf einen Widerspruch. Eine topologische Sortierung eines Digraphen ist nun eine vollständige Ordnung über den Knoten des Graphen, die mit der durch die Pfeile ausgedrückten partiellen Ordnung verträglich ist. Genauer: Eine topologische Sortierung eines Digraphen $G = (V, E)$ ist eine Abbildung $\text{ord}: V \rightarrow \{1, \dots, n\}$ mit $n = |V|$, so daß mit $(v, w) \in E$ auch $\text{ord}(v) < \text{ord}(w)$ gilt.

Nun ist G genau dann zyklenfrei, wenn es für G eine topologische Sortierung gibt. Dies überlegt man sich wie folgt. Es ist klar, daß aus der Existenz einer topologischen Sortierung die Zyklensfreiheit von G folgt. Daß es zu jedem zyklenfreien Digraphen $G = (V, E)$ auch eine topologische Sortierung gibt, kann man durch Induktion über die Knotenzahl zeigen. Falls $|V| = 1$, dann gibt es natürlich eine topologische Sortierung:

Man definiert einfach $\text{ord}(1) = 1$. Falls $|V| > 1$, so betrachtet man einen Knoten v mit $\text{indeg}(v) = 0$. Wegen der Zyklentreiheit von G muß es einen solchen Knoten geben. Durch Entfernen von v entsteht ein um einen Knoten verkleinerter Digraph. An dessen topologische Sortierung wird v vorne angefügt. Hieraus ergibt sich unmittelbar ein Algorithmus für die topologische Sortierung:

Algorithmus *Topologische Sortierung (Grobentwurf)*

{liefert zu einem Digraphen $G = (V, E)$ eine topologische Sortierung
 $\text{ord}[\text{knotentyp}]$ }

begin

$\text{lfd.Nr.} := 0$;

while G hat wenigstens einen Knoten v mit Eingangsgrad 0 **do**

begin

 erhöhe lfd.Nr. um 1;

$\text{ord}[v] := \text{lfd.Nr.}$;

$G := G - v$

end;

if $G = \emptyset$

then G ist zyklentrei

else G hat Zyklen

end {Topologische Sortierung}

Es ist noch zu klären, wie man einen Knoten mit Eingangsgrad 0 findet. Hier ist es naheliegend, an einem beliebigen Knoten zu beginnen und Pfeile rückwärts zu verfolgen. Da der Digraph G zyklentrei ist, trifft man nicht mehrmals auf einen Knoten. Also endet das Zurückverfolgen von Pfeilen spätestens, wenn alle Knoten besucht worden sind. Das Zurückverfolgen von Pfeilen kann aber nur in einem Knoten mit Eingangsgrad 0 enden. Damit hat man einen solchen Knoten gefunden.

Wenn man dazu jedoch stets den ganzen Digraphen durchläuft, so benötigt man pro Knoten wenigstens $\Omega(n)$ Schritte, insgesamt also wenigstens $\Omega(n^2)$ Schritte.

Es ist sicherlich effizienter, den jeweils aktuellen Eingangsgrad zu jedem Knoten zu speichern und auf dem neuesten Stand zu halten. Dann genügt es, statt einen Knoten aus G zu entfernen, die Eingangsgrade seiner direkten Nachfolger zu verringern. Um einen Knoten mit Eingangsgrad 0 schnell zu finden, verwalten wir die Menge aller Knoten mit aktuellem Eingangsgrad 0. Diese Menge ändert sich höchstens bei der Wahl eines Knotens für die topologische Sortierung und beim Verringern der Eingangsgrade direkter Nachfolger eines gewählten Knotens. Damit ergibt sich die folgende Präzisierung des Algorithmus für die topologische Sortierung eines Digraphen:

Algorithmus *Topologische Sortierung (Präzisierung)*

{liefert zu einem Digraphen $G = (V, E)$ eine topologische Sortierung
 $\text{ord}[\text{knotentyp}]$ }

var $\text{lfd.Nr.} : 0 \dots \text{knotenzahl}$;

$\text{Gradnull} : \text{stack of knotentyp}$;

$\text{Eingrad} : \text{array} [\text{knotentyp}] \text{ of } 0 \dots \text{knotenzahl} - 1$

begin

 1. setze $\text{Eingrad}[v]$ auf den Eingangsgrad von v in G ,

```

    für alle  $v \in V$ ;
2.  übernimm alle Knoten  $v \in V$  mit Eingangsgrad 0
    nach Gradnull;
3.   $lfd.Nr. := 0$ ;
4.  while Gradnull  $\neq \emptyset$  do
    begin
        wähle  $v \in$  Gradnull;
        entferne  $v$  aus Gradnull;
        erhöhe  $lfd.Nr.$  um 1;
         $ord[v] := lfd.Nr.$ ;
    { *1* } for all  $(v, w) \in E$  do
    { *2* }     begin
    { *3* }         erniedrige Eingrad[ $w$ ] um 1;
    { *4* }         if Eingrad[ $w$ ] = 0
    { *5* }             then füge  $w$  zu Gradnull hinzu
    { *6* }     end
    end;
5.  if  $lfd.Nr. =$  knotenzahl
    then  $G$  ist zyklensfrei
    else  $G$  hat Zyklus
end {Topologische Sortierung}

```

Die einzelnen Schritte des Algorithmus lassen sich leicht präzisieren, wenn wir die Speicherung des gegebenen Digraphen in Adjazenzlistenform annehmen, wie eingangs angegeben:

```

{1. setze Eingrad ... }
for  $v := 1$  to knotenzahl do Eingrad[ $v$ ] := 0;
for  $v := 1$  to knotenzahl do
    begin
         $p := adjazenzliste[v]$ ;
        while  $p \neq \text{nil}$  do
            begin
                erhöhe Eingrad[ $p \uparrow . \text{endknoten}$ ] um 1;
                 $p := p \uparrow . \text{next}$ 
            end
        end
    end

{2. übernimm ... }
Gradnull := leerer Stapel;
for  $v := 1$  to knotenzahl do
    if Eingrad[ $v$ ] = 0
    then füge  $v$  zu Gradnull hinzu;

{Die Zeilen { *1* } bis { *6* } in 4. while Gradnull ... }
 $p := adjazenzliste[v]$ ;
while  $p \neq \text{nil}$  do

```

```

begin
   $w := p \uparrow . \text{endknoten};$ 
   $\{ *3 * \};$ 
   $\{ *4 * \};$ 
   $\{ *5 * \};$ 
   $p := p \uparrow . \text{next}$ 
end

```

Damit benötigt Schritt 1 des Verfahrens eine Laufzeit von $O(|V| + |E|)$; Schritt 2 kommt wegen der konstanten Zeit für jede einzelne Stapeloperation mit einer Laufzeit von $O(|V|)$ aus, und Schritt 3 kann in konstanter Zeit ausgeführt werden. Die **while**-Schleife in Schritt 4 wird gerade $|V|$ -mal durchlaufen; in der inneren **while**-Schleife wird jeder Pfeil im Digraphen gerade einmal inspiziert. Damit benötigt Schritt 4 eine Laufzeit von $O(|V| + |E|)$. Mit der konstanten Laufzeit von Schritt 5 ergibt sich in der Summe eine Laufzeit von $O(|V| + |E|)$ für die Berechnung einer topologischen Sortierung für einen Digraphen $G = (V, E)$. Ebenfalls in Zeit $O(|V| + |E|)$ kann somit ein Digraph $G = (V, E)$ auf Zyklensfreiheit getestet werden.

8.2 Transitive Hülle

Beschäftigen wir uns nun mit der Erreichbarkeit von Knoten in einem Graphen, ausgehend von anderen Knoten. So kann man sich etwa fragen, welche Knoten von einem gegebenen Knoten aus erreichbar sind, oder ob es womöglich einen Knoten gibt, von dem aus jeder andere erreicht werden kann. In einem Zyklus beispielsweise kann jeder Knoten von jedem anderen aus erreicht werden. Um solche Fragen zu beantworten, kann es sinnvoll sein, von vornherein alle Erreichbarkeiten explizit zu berechnen. Sind die Knoten eines Digraphen beispielsweise Straßenkreuzungen und die Pfeile verbindende Einbahnstraßen, so ist Kreuzung Z von Kreuzung X aus gerade dann erreichbar, wenn es entweder einen Pfeil von X nach Z gibt oder eine Kreuzung Y , die von X aus erreichbar ist und von der aus Z erreichbar ist. Natürlich ist auch jede Kreuzung von sich selbst aus erreichbar. Dies führt zur Definition der reflexiven transitiven Hülle.

Ein Digraph $G^* = (V, E^*)$ ist die *reflexive, transitive Hülle* eines Digraphen $G = (V, E)$, wenn genau dann $(v, v') \in E^*$ ist, wenn es einen Weg von v nach v' in G gibt. Die reflexive, transitive Hülle (kurz: *Hülle*) des Digraphen aus Abbildung 8.8 enthält alle Pfeile zwischen Knoten, weil jeder Knoten von jedem aus erreicht werden kann. Für den speziellen Fall, daß der gegebene Digraph azyklisch ist, ist die Berechnung der transitiven Hülle einfacher als im allgemeinen Fall. Betrachten wir jedoch zunächst den allgemeinen Fall.

8.2.1 Transitive Hülle allgemein

Erinnern wir uns daran, daß wir die Existenz eines Weges von v nach v' in $G = (V, E)$ mit $v \rightarrow_E^* v'$ notieren. Wenn wir nun schon wissen, daß $v \rightarrow_E^* v'$ und $v' \rightarrow_E^* v''$ gelten, so können wir auf die Gültigkeit von $v \rightarrow_E^* v''$ schließen. Damit ergibt sich unmittelbar ein erster Ansatz eines Algorithmus zur Berechnung der transitiven Hülle. Beginnend mit der Adjazenzmatrix A für den gegebenen Digraphen suchen wir zu allen Pfeilen (i, j) alle Pfeile (j, k) und vermerken die daraus entstehenden Pfeile (i, k) in der Adjazenzmatrix:

Algorithmus *Berechnung von Pfeilen der reflexiven transitiven Hülle*

```
1.  for  $i := 1$  to knotenzahl do  $A[i, i] := 1$ ;  
2.  for  $i := 1$  to knotenzahl do  
    for  $j := 1$  to knotenzahl do  
        if  $A[i, j] = 1$  then  
            for  $k := 1$  to knotenzahl do  
                if  $A[j, k] = 1$  then  $A[i, k] := 1$   
end {Berechnung von Pfeilen}
```

Es ist klar, daß mit diesem Algorithmus tatsächlich einige Wege berechnet werden; man sieht aber auch leicht, daß nicht alle Wege gefunden werden. Abbildung 8.9 (a) zeigt ein Beispiel für einen Graphen, 8.9 (b) dessen Adjazenzmatrix, und 8.9 (c) das Resultat der Anwendung des Algorithmus zum Finden von Pfeilen der reflexiven transitiven Hülle. Man erkennt, daß alle aus bis zu zwei Pfeilen bestehenden Wege gefunden worden sind. Der aus drei Pfeilen bestehende Weg vom Knoten 1 zum Knoten 2 wurde aber nicht entdeckt. Wege größerer Länge werden gefunden, wenn man den Algorithmus wiederholt solange anwendet, bis sich keine neuen Pfeile ergeben. Dies ist aber nicht besonders effizient: Bereits die einfache Anwendung des Algorithmus benötigt wegen der drei im Schritt 2 geschachtelten **for**-Schleifen eine Laufzeit von $\Theta(|V|^3)$. Folgende Überlegung zeigt, daß es auch schneller geht.

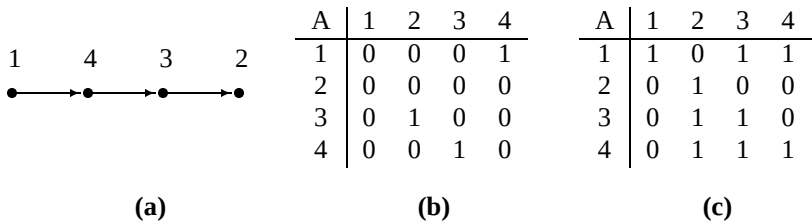


Abbildung 8.9

Zum Auffinden eines Weges vom Knoten i zum Knoten k betrachten wir nicht jede mögliche Zusammensetzung von Teilwegen, sondern nur eine spezielle. Ein Weg von einem Knoten i zu einem Knoten k ist entweder ein Pfeil von i nach k oder kann so in

einen Weg von i nach j und einen Weg von j nach k zerlegt werden, daß j die größte Nummer eines Knotens auf dem Weg zwischen i und k ist (ohne i und k selbst). Die Knotennummern sind dabei die mit dem Graphen willkürlich festgelegten, also nicht etwa die durch topologische Sortierung ermittelten. Wir ermitteln nun Wege in einer Reihenfolge, die sicherstellt, daß beim Zusammensetzen der beiden Wege von i nach j und von j nach k beide nur Zwischenknoten mit einer Nummer kleiner als j benutzen. Dies ist der Fall, wenn unser Algorithmus für aufsteigende Werte von j die folgende *Invariante* erfüllt: Für das aktuelle j sind alle Wege bereits bekannt, die nur Zwischenknoten mit Nummer kleiner als j benutzen. Es ist klar, daß die Invariante anfangs gilt. Beim Zusammenfügen bereits bekannter Wege benutzt jeder resultierende Weg nur Knoten, deren Nummer höchstens j ist, also nur Knoten mit Nummer kleiner als $j + 1$. Da alle beim Erhöhen von j neu gefundenen Wege den Knoten j benutzen müssen, wird auch jeder solche Weg tatsächlich gefunden. Damit ergibt sich der folgende Algorithmus für die Berechnung der reflexiven, transitiven Hülle eines Digraphen, der sich von dem zuvor angegebenen Algorithmus für das Finden von Pfeilen der Hülle nur durch das Vertauschen der beiden äußeren **for**-Schleifen in Schritt 2 unterscheidet []:

Algorithmus *Reflexive transitive Hülle*

```

1.  for  $i := 1$  to knotenzahl do  $A[i, i] := 1$ ;
2.  for  $j := 1$  to knotenzahl do
    for  $i := 1$  to knotenzahl do
        if  $A[i, j] = 1$  then
            for  $k := 1$  to knotenzahl do
                if  $A[j, k] = 1$  then  $A[i, k] := 1$ 
end {Reflexive transitive Hülle}
```

Die Laufzeit dieses Algorithmus ist offensichtlich beschränkt durch $O(|V|^3)$. Bei näherem Hinsehen zeigt sich, daß die innerste der drei **for**-Schleifen nur durchlaufen wird, wenn ein Pfeil von i nach j vorhanden ist. Dieser Pfeil kann aus dem gegebenen Digraphen G stammen; er kann aber auch im Verlauf der Berechnung der Hülle G^* ermittelt worden sein. Die innerste **for**-Schleife wird also nicht unbedingt $\Theta(|V|^2)$ -mal, sondern nur $O(|E^*|)$ -mal durchlaufen. Da jeder Durchlauf in $O(|V|)$ Schritten erledigt werden kann, ergibt sich die Gesamtlaufzeit zu $O(|V|^2 + |E^*| \cdot |V|)$.

8.2.2 Transitive Hülle für azyklische Digraphen

Betrachten wir nun das Problem der Berechnung der reflexiven, transitiven Hülle für azyklische Digraphen. Wir wollen uns die topologische Sortierung zunutze machen, indem wir die dort vergebenen Ordnungsnummern gerade als Knotennummern wählen. Wie man eine topologische Sortierung in linearer Zeit berechnen kann, wurde bereits in Abschnitt 8.1 erläutert. Wir nehmen an, daß der Digraph in Adjazenzlistenform, mit Knoten in topologischer Sortierung, gegeben ist.

Die Grundidee beim Berechnen der reflexiven, transitiven Hülle besteht darin, die Knoten in der Reihenfolge absteigender Nummern zu betrachten. Für einen betrachteten Knoten i mit Pfeil (i, j) kennen wir wegen der topologischen Sortierung bereits alle von j aus erreichbaren Knoten (vgl. Abbildung 8.10 (a)). Die Menge der von i aus erreichbaren Knoten besteht also aus i selbst und allen von j aus erreichbaren Knoten, vereinigt über alle Pfeile (i, j) .

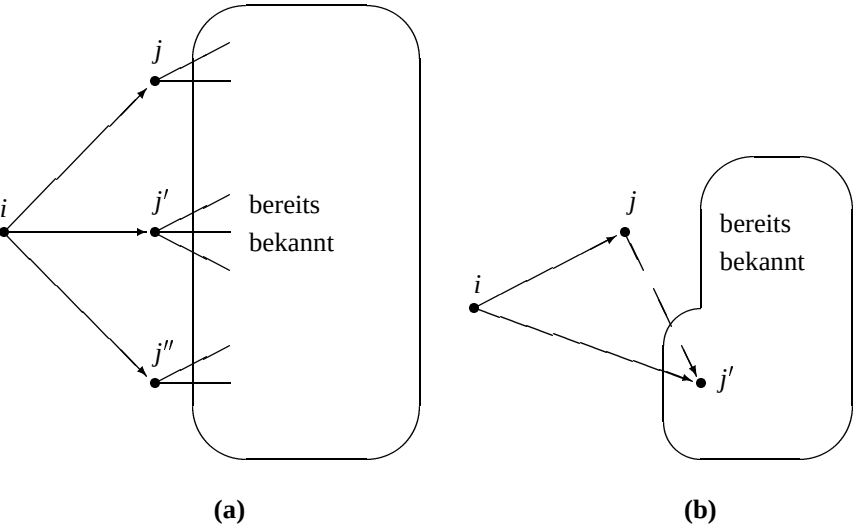


Abbildung 8.10

Für das aktuelle i betrachten wir die Endknoten j der Pfeile (i, j) aus Effizienzgründen in aufsteigender Reihenfolge ihrer Nummern. Falls nämlich bei Pfeilen (i, j) und (i, j') mit $j' > j$ Knoten j' bereits über Knoten j erreicht werden kann, so ist die Menge der über j' erreichbaren Knoten bereits in der Menge der über j erreichbaren Knoten enthalten, und j' muß zu diesem Zweck nicht weiter untersucht werden (siehe Abbildung 8.10 (b)).

Das skizzierte Verfahren läßt sich wie folgt präzisieren:

```
Algorithmus Reflexive transitive Hülle für azyklischen Digraphen
{liefert zu einem in Adjazenzlistenrepräsentation gegebenen,
 topologisch sortierten, azyklischen Digraphen  $G = (V, E)$  die reflexive,
 transitive Hülle von  $G$  im Feld erreichbar_ab[knotentyp]}
var  $i, j, k$  : knotentyp;
       $erreichbar$  : set of knotentyp;
       $erreichbar\_ab$  : array [knotentyp] of list of knotentyp;
begin
       $erreichbar := \emptyset$ ; {ab Knoten  $i$  als erreichbar bekannt}
      for  $i := knotenzahl$  downto 1 do
```

```

begin
    erreichbar_ab[i] := {i};
    erreichbar := {i};
    for all (i, j) ∈ E mit aufsteigendem j do
        if j ∉ erreichbar then
            for all k ∈ erreichbar_ab[j] do
                if k ∉ erreichbar then
                    begin
                        füge k zu erreichbar hinzu;
                        füge k zu erreichbar_ab[i] hinzu
                    end;
            {setze erreichbar := ∅ :}
        for all k ∈ erreichbar_ab[i] do
            entferne k aus erreichbar
    end
end {Reflexive, transitive Hülle für azyklischen Digraphen}

```

Daß dieser Algorithmus gerade G^* berechnet, zeigen folgende Überlegungen. Es sollte klar sein, daß der Algorithmus nur Pfeile aus E^* findet. Durch ein Widerspruchsargument kann man sich davon überzeugen, daß er alle Pfeile aus E^* auch tatsächlich findet. Nehmen wir dazu an, daß es einen Pfeil in der Hülle gibt, den der Algorithmus nicht findet. Wählen wir dann i als die größte Nummer, für die der Algorithmus den Pfeil (i, h) der Hülle nicht findet. Wenn (i, h) nicht gefunden wird, muß $(i, h) \notin E$ gelten. Betrachten wir jetzt den längsten Weg $i, j, \dots, h$ von i nach h . Weil i die größte solche Nummer ist, befindet sich h in der Liste $\text{erreichbar_ab}[j]$. Bei der Betrachtung des Pfeils (i, j) ist die Bedingung $j \notin \text{erreichbar}$ erfüllt, weil der Weg von i über j nach h der längste ist. Also wird h zur Liste $\text{erreichbar_ab}[i]$ hinzugefügt. Damit hat aber der Algorithmus den Pfeil (i, h) gefunden, ein Widerspruch zur Annahme.

Für jeden Knoten i sind die ab Knoten i erreichbaren Knoten einerseits als lineare Liste $\text{erreichbar_ab}[i]$ gespeichert. Alle Listenelemente können der Reihe nach besucht werden, in konstanter Laufzeit pro Listenelement. Außerdem kann jede Liste um weitere Elemente ergänzt werden, ebenfalls in konstanter Laufzeit pro Listenelement. Andererseits sind die ab dem aktuellen Knoten erreichbaren Knoten als Menge (Bitvektor) erreichbar gespeichert, damit das Enthaltensein eines Knotens in dieser Menge in konstanter Zeit geprüft werden kann; das Hinzufügen eines Elements zur Menge und das Entfernen eines Elements aus der Menge ist ebenfalls in konstanter Zeit möglich. Damit benötigt eine Abarbeitung der innersten der drei geschachtelten **for**-Schleifen eine Schrittzahl, die proportional ist zur Anzahl der ab j erreichbaren Knoten. Das für jeden weiteren Durchlauf der äußersten **for**-Schleife erforderliche Zurücksetzen der Menge der von i aus erreichbaren Knoten auf die leere Menge wird mit der entsprechenden **for**-Schleife in einer Schrittzahl erledigt, die proportional ist zur Anzahl der ab i erreichbaren Knoten. Die mittlere der drei geschachtelten **for**-Schleifen wird gerade einmal für jeden Pfeil ausgeführt. Wir müssen uns noch fragen, wie oft die innerste der drei geschachtelten **for**-Schleifen zur Ausführung kommt. Dazu betrachten wir für einen gegebenen azyklischen Digraphen $G = (V, E)$ den *reduzierten Graphen* $G_{\text{red}} = (V, E_{\text{red}})$, der durch $E_{\text{red}} = \{(i, j) \mid (i, j) \in E, \nexists k, i \neq k \neq j, \text{ mit } (i, k) \in E^*, (k, j) \in E^*\}$ definiert

ist. G_{red} ist also gerade G ohne transitive Pfeile. Die Definition des reduzierten Graphen ist so gewählt, daß $G^* = G_{red}^*$ gilt.

Daß die innerste der drei geschachtelten **for**-Schleifen im Algorithmus nur für Pfeile des reduzierten Graphen ausgeführt wird, sieht man wie folgt. Betrachten wir einen Pfeil (i, j) , der nicht zum reduzierten Graphen gehört. Dann gibt es im reduzierten Graphen Pfeile (i, k) und (k, j) , wobei wegen der topologischen Sortierung $k < j$ gilt; demnach wird Pfeil (i, k) vor (i, j) betrachtet. Weil j von k aus erreichbar ist, wird j bereits bei der Betrachtung des Pfeiles (i, k) zur Menge *erreichbar* hinzugefügt. Beim Betrachten des Pfeils (i, j) ist dann die für die Ausführung der innersten **for**-Schleife geforderte Bedingung nicht erfüllt.

Bringen wir nun unsere Überlegungen zur Laufzeit des Algorithmus zur Berechnung der Hülle zum Ende. Die letzte **for**-Schleife kostet einen Rechenschritt für jeden Pfeil der Hülle, also insgesamt Zeit $O(|E^*|)$. Die innerste der drei geschachtelten **for**-Schleifen wird für jeden Pfeil des reduzierten Graphen ausgeführt. Schlimmstenfalls sind jedes Mal größenordnungsmäßig alle Knoten erreichbar; dann ergibt sich hierfür insgesamt eine Laufzeit von $O(|E_{red}| \cdot |V|)$. Alle anderen Schritte zusammen können in Laufzeit $O(|V|)$ ausgeführt werden. Somit kann die reflexive, transitive Hülle eines azyklischen Digraphen $G = (V, E)$ in Zeit $O(|V| \cdot |E_{red}|) = O(|V| \cdot |E|) = O(|V|^3)$ ermittelt werden.

8.3 Durchlaufen von Graphen

Für manche Probleme ist es wichtig, alle Knoten eines Graphen zu betrachten. So kann man es etwa einer in einem Labyrinth eingeschlossenen Person nachfühlen, daß sie gerne sämtliche Kreuzungen von Gängen des Labyrinths in Augenschein nehmen will. Die Gänge des Labyrinths sind hier die Kanten des Graphen, und Kreuzungen von Gängen sind Knoten. Das Betrachten oder Inspizieren eines Knotens in einem Graphen nennt man auch oft *Besuchen* des Knotens. Manchmal ist es wichtig, die Knoten nach einer gewissen Systematik zu besuchen. So kann man sich leicht vorstellen, daß eine einzelne Person im Labyrinth einem Gang zunächst eine ganze Weile folgt, bevor sie vielleicht schließlich kehrt macht, also mit der Suche zunächst „in die Tiefe“ des Labyrinths geht; suchen dagegen mehrere Personen gleichzeitig, so werden sie eher vom Startpunkt aus ausschwärmen, also „in die Breite“ gehen.

Wir werden im folgenden die *Tiefensuche* und die *Breitensuche* als zwei Spezialfälle eines allgemeinen Knotenbesuchsalgorithmus kennenlernen. Es ist ganz erstaunlich, wieviel Information über die Struktur eines Graphen man alleine durch systematisches Besuchen der Knoten erhalten kann. Stellt etwa ein Graph ein Computernetz dar, wobei die Knoten des Graphen Computer und die Kanten des Graphen Verbindungsleitungen zwischen Computern sind, so kann man die Frage, ob nach dem Ausfall eines beliebigen Computers die anderen noch miteinander kommunizieren können, durch systematisches Besuchen aller Knoten lösen. Mittels spezialisierter Knotenbesuchsalgorithmen kann man aber nicht nur entscheiden, ob ein gegebener Graph *zweifach zusammenhängend* — wie für das Computernetz gefordert — ist, sondern man kann auch die größten

zweifach zusammenhängenden Teilgraphen (die zweifachen Zusammenhangskomponenten des Graphen) berechnen. Das Gerüst der Knotenbesuchsalgorithmen ist dabei stets dasselbe:

Algorithmus-Gerüst *Besuche Knoten*

{ besucht in einem gegebenen Graphen oder Digraphen $G = (V, E)$ der Reihe nach alle Knoten }

var B : **set of** knotentyp;
{ Menge der bereits besuchten Knoten }

begin

$B := \{b\}$, wobei b ein erster besuchter Knoten ist;

for all $e \in E$ **do**

markiere e als unbenutzt;

while *es gibt unbenutzte Kante/Pfeil $(v, v') \in E$ mit $v \in B$* **do**

begin

markiere (v, v') als benutzt;

$B := B \cup \{v'\}$

end

end *{ Besuche Knoten }*

Man überlegt sich leicht, daß B am Ende der Ausführung des Algorithmus *Besuche Knoten* die Menge aller von b aus erreichbaren Knoten enthält. Wir müssen noch präzisieren, wie die Menge B implementiert werden soll und welche unbenutzte Kante/Pfeil in der **while**-Schleife als jeweils nächste gewählt werden soll. Damit die die **while**-Schleife kontrollierende Bedingung schnell überprüft werden kann, speichern wir neben der Menge B noch eine weitere Knotenmenge $R \subseteq B$ derjenigen Knoten in B , von denen noch unbenutzte Kanten oder Pfeile ausgehen können — den *Rand* von B . Dann können wir den Knotenbesuchsalgorithmus wie folgt formulieren:

procedure *Durchlaufe* $G = (V, E)$ *ab Knoten* b ;

begin

$B := \{b\}$; $R := \{b\}$;

while $R \neq \emptyset$ **do**

begin

wähle Knoten $v \in R$;

if *es gibt keine unbenutzte Kante/Pfeil $(v, v') \in E$*

then *lösche v aus R ;*

else

begin

sei (v, v') die nächste unbenutzte Kante/Pfeil $\in E$;

if $v' \notin B$ **then**

begin

$B := B \cup \{v'\}$;

$R := R \cup \{v'\}$

end

end

end *{ while }*

end *{ Durchlaufe }*

Um zu entscheiden, welche Datenstrukturen für B und R am besten gewählt werden sollten, betrachten wir die mit B und R auszuführenden Operationen. Wir müssen B als

leere Menge initialisieren, ein Element zu B hinzufügen und prüfen können, ob ein gegebener Knoten in B enthalten ist. Für R müssen wir neben der Initialisierung als leere Menge ein Element hinzufügen können, prüfen können, ob R leer ist, ein beliebiges Element wählen können und ein gewähltes Element aus R entfernen können. Dabei ist das Initialisieren von B und R die einzige Operation, die beim Durchlaufen eines Graphen nur einmal ausgeführt wird; alle anderen Operationen werden wiederholt ausgeführt.

Wählen wir für B ein Boole'sches Array mit einem Element pro Knoten und für R eine Schlange oder einen Stapel, so benötigt jede Operation außer dem Initialisieren von B nur eine konstante Schrittzahl; das Initialisieren von B kann in $O(|V|)$ Schritten ausgeführt werden. Um für jeden Knoten $v \in V$ schnell entscheiden zu können, ob es noch eine unbenutzte Kante oder einen unbenutzten Pfeil $(v, v') \in E$ gibt, und um gegebenenfalls die nächste solche Kante zu wählen, speichern wir zusätzlich für jeden Knoten v einen Zeiger $p[v]$, der auf die nächste ungenutzte Kante in der Adjazenzliste des Knoten v zeigt. Mit den zusätzlichen Definitionen

```
var  $B$  : array [knotentyp] of boolean;
       $R$  : stack of knotentyp;
       $p$  : array [knotentyp] of pfeilzeiger
```

können wir die Prozedur für das Durchlaufen an zwei Stellen wie folgt präzisieren:

1. *es gibt keine unbenutzte Kante/Pfeil* $(v, v') \in E$:
 $p[v] = \mathbf{nil}$
2. *sei* (v, v') *die nächste unbenutzte Kante/Pfeil* $\in E$:
 $v' := p[v] \uparrow \mathit{endknoten}$;
 $p[v] := p[v] \uparrow \mathit{next}$

Die von der Prozedur *Durchlaufe* benötigte Zeit ist proportional zur Summe der Anzahlen der vom Startknoten b aus erreichbaren Knoten und Kanten/Pfeile, weil jeder Schleifendurchlauf nur konstant viele Schritte benötigt und einen Knoten oder eine Kante betrachtet, die danach nicht mehr betrachtet werden. Damit können alle Knoten eines Graphen in höchstens $O(|V| + |E|)$ Schritten besucht werden.

8.3.1 Einfache Zusammenhangskomponenten

Betrachten wir zunächst eine der einfachsten Anwendungen des linearen Knotenbesuchsalgorithmus. Hier geht es darum, zu einer gegebenen Menge V mit einer symmetrischen, binären Relation $E \subseteq V \times V$, deren reflexive, transitive Hülle eine Äquivalenzrelation ist, die Äquivalenzklassen zu bestimmen. Ist V die Menge der Knoten und E die Menge der Kanten eines ungerichteten Graphen, so sind dies gerade die größten zusammenhängenden Teilgraphen von $G = (V, E)$.

Genauer: Ein ungerichteter Graph G heißt genau dann *zusammenhängend*, wenn es für jedes Knotenpaar $(v, v') \in V$ einen Weg von v nach v' gibt. Eine *Zusammenhangskomponente* von G ist ein (bezüglich Mengeninklusion) maximaler zusammenhängender Untergraph von G . Ersetzen wir nun in der Prozedur *Durchlaufe* die Anweisung $B := \{b\}$ durch $B := B \cup \{b\}$, so berechnet der folgende Algorithmus gerade die Zusammenhangskomponenten eines ungerichteten Graphen $G = (V, E)$:

Algorithmus *Zusammenhangskomponenten*

```

for  $v := 1$  to knotenzahl do  $p[v] := \text{adjazenzliste}[v]$ ;
 $B := \emptyset$ ;
for  $v := 1$  to knotenzahl do
    if  $v \notin B$ 
        then Durchlaufe G ab Knoten v
end {Zusammenhangskomponenten}

```

Jeder Aufruf der Prozedur *Durchlaufe* im Algorithmus *Zusammenhangskomponenten* besucht die Knoten der Zusammenhangskomponente, die den Startknoten v enthält, und fügt diese zur Menge B hinzu. Die Laufzeit des Algorithmus *Zusammenhangskomponenten* ergibt sich damit zu $O(|V| + |E|)$.

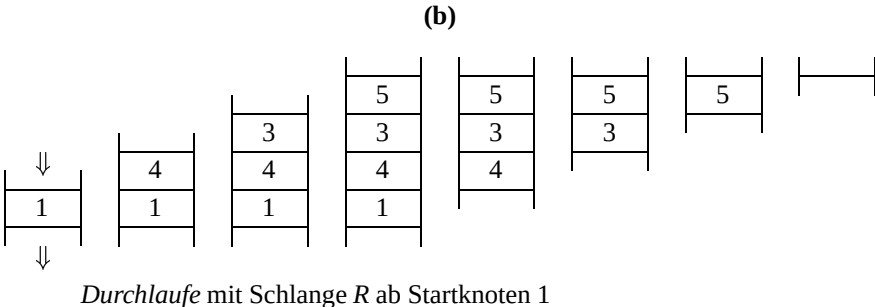
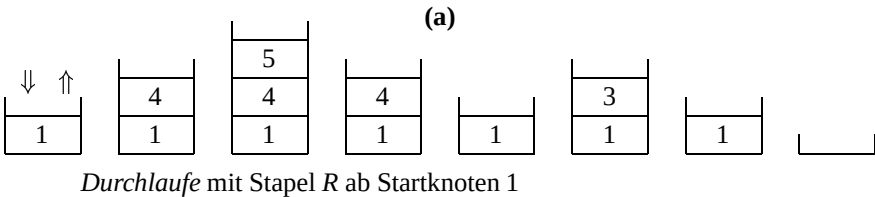
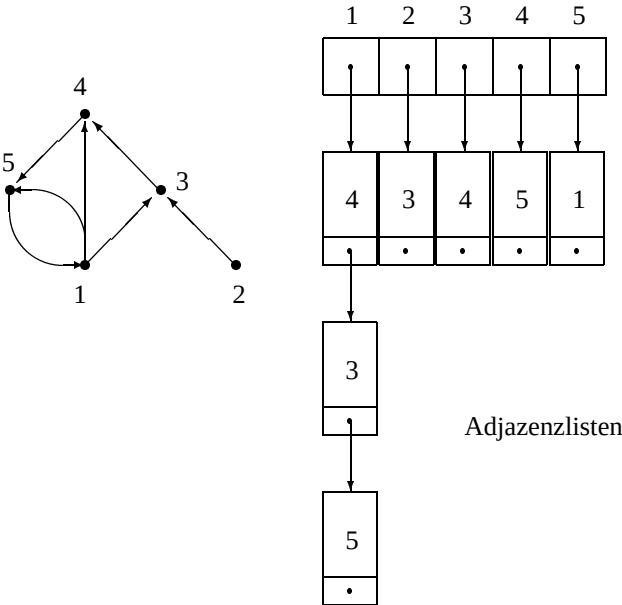
8.3.2 Strukturinformation durch Tiefensuche

Um beim systematischen Durchlaufen eines Graphen mehr über dessen Struktur zu erfahren, wollen wir dieses nun näher in Augenschein nehmen. Betrachten wir zunächst anhand eines Beispiels den Unterschied, der sich ergibt, wenn wir zum einen die noch unbenutzten Kanten als Stapel (*last in first out*), zum anderen als Schlange (*first in first out*) verwalten. Abbildung 8.11 (a) zeigt einen Digraphen und eine Adjazenzlistenrepräsentation; Abbildung 8.11 (b) und 8.11 (c) zeigen die Entwicklung von R als Stapel und als Schlange.

Ist R als Stapel realisiert, so trifft man die Knoten in der Reihenfolge 1, 4, 5, 3 erstmals an; Knoten 2 ist von Knoten 1 aus nicht erreichbar. Ist dagegen R als Schlange realisiert, so ergibt sich die Reihenfolge 1, 4, 3, 5.

Bei Verwendung eines Stapels für R reden wir von *Tiefensuche* (englisch: *depth first search*; *DFS*), bei einer Schlange von *Breitensuche* (englisch: *breadth first search*; *BFS*). Die Tiefensuche bietet sich oft für Zusammenhangsprobleme an, die Breiten-suche dagegen für Distanzprobleme, wie wir später noch sehen werden. Für manche Algorithmen, in denen es um Aussagen über die Struktur des gegebenen Graphen geht, ist die Tiefensuche von besonderer Bedeutung. Dabei betrachtet man nicht nur die Reihenfolge, in der man Knoten erstmals antrifft, sondern beispielsweise auch die Reihenfolge, in der man Knoten vom Stapel R wieder entfernt. In unserem Beispiel ist dies die Reihenfolge 5, 4, 3, 1. Die relative Position eines Knotens in der Reihenfolge, in der die Knoten auf den Stapel R abgelegt worden sind, nennen wir den *depth-first-begin-Index* (DFBI) eines Knotens. Im Beispiel der Abbildung 8.11 sind die DFBIIndizes der Knoten 1, 4, 5 und 3 gerade 1, 2, 3 und 4. Entsprechend bezeichnen wir als *depth-first-end-Index* (DFEI) eines Knotens seine relative Position in der Reihenfolge, in der die Knoten vom Stapel R entfernt werden. Im Beispiel der Abbildung 8.11 sind also die DFEIndizes der Knoten 5, 4, 3 und 1 gerade 1, 2, 3 und 4.

Formuliert man die Prozedur für das Durchlaufen eines Graphen ab einem Startknoten b rekursiv, anstatt explizit einen Stapel für R zu benutzen, so entspricht der DFB-Index gerade einer beim Prozeduraufruf vergebenen laufenden Nummer, der DFEIndex einer beim Beenden des Prozeduraufrufs vergebenen Nummer. Wenden wir die bei Bäumen übliche Terminologie (vgl. Kapitel 5) auf den Baum der rekursiven Aufrufe



(c)

Abbildung 8.11

an, so ist der DFBIndex gerade die Knotennummer in *Hauptreihenfolge* (*preorder*), der DFEIndex diejenige in *Nebenreihenfolge* (*postorder*).

Wir unterscheiden außerdem bei einem Digraphen die Pfeile nach der Rolle, die sie bei einer Tiefensuche spielen. Dazu teilen wir die Menge aller Pfeile in vier Klassen ein. Die Pfeile, denen die Tiefensuche folgt, die also als unbenutzte Pfeile gewählt werden, heißen *Baumpfeile*; die Menge *BP* der Baumpfeile bildet den *Tiefensuchbaum* (DFS-Baum) vom Startknoten der Tiefensuche aus. Im Beispiel der Abbildung 8.11 ist $BP = \{(1, 4), (4, 5), (1, 3)\}$; die Pfeile in *BP* können an den obersten beiden Elementen des Stapels *R* abgelesen werden, wenn ein neuer Knoten auf den Stapel abgelegt wird. Pfeile, die zu einem bereits erreichten Nachfolgerknoten im DFS-Baum führen, heißen *Vorwärtspfeile*. Jeder Pfeil in der Menge *VP* der Vorwärtspfeile gehört zur transitiven Hülle der Baumpfeile und kürzt einen Weg der Länge mindestens 2 im DFS-Baum ab. Im Beispiel der Abbildung 8.11 ist bei einer Tiefensuche ab Knoten 1 gerade der Pfeil (1, 5) ein Vorwärtspfeil. *Rückwärtspfeile* sind all diejenigen Pfeile, die von einem Knoten im DFS-Baum zu einem Vorgänger dieses Knotens im DFS-Baum weisen. Jeder Pfeil in der Menge *RP* der Rückwärtspfeile bildet also mit dem DFS-Baum einen Zyklus. Im Beispiel der Abbildung 8.11 ist der Pfeil (5, 1) der einzige Rückwärtspfeil für die Tiefensuche ab Knoten 1. Alle anderen Pfeile heißen *Seitwärtspfeile*; *SP* ist die Menge aller Seitwärtspfeile.

Die folgende rekursiv formulierte Prozedur für die Tiefensuche illustriert die Berechnung der Knotenindizes und die Klassifikation der Pfeile mit Hilfe eines kleinen Programmstücks:

```

procedure DFS für G ab Knoten v, kommend von w;
begin
  if  $v \notin B$ 
    then {v noch nicht besucht}
      begin
         $B := B \cup \{v\}$ ;
         $BP := BP \cup \{(w, v)\}$ ;
        erhöhe dfbi um 1; {aktueller DFBIndex}
         $DFBI[v] := dfbi$ ;
        for all  $(v, v') \in E$  do
          DFS für G ab  $v'$ , kommend von v;
          erhöhe dfei um 1; {aktueller DFEIndex}
           $DFEI[v] := dfei$ 
        end
      else {v bereits besucht : klassifiziere Pfeil}
        begin
          if  $w \rightarrow_{BP}^* v$ 
            then  $VP := VP \cup \{(w, v)\}$ 
          else if  $v \rightarrow_{BP}^* w$ 
            then  $RP := RP \cup \{(w, v)\}$ 
            else  $SP := SP \cup \{(w, v)\}$ 
          end
        end
      end {DFS}

```

begin $B := \emptyset;$ $dfbi := dfei := 0;$ $BP := VP := RP := SP := \emptyset;$ *DFS für G ab v , kommend von nirgends***end**

Wir haben noch nicht klargestellt, wie man denn die Bedingungen $w \rightarrow_{BP}^* v$ und $v \rightarrow_{BP}^* w$ für das Klassifizieren eines Pfeils als Vorwärtspfeil, Rückwärtspfeil oder Seitwärtspfeil effizient überprüfen kann. Hier helfen uns der *DFBIndex* und der *DFEIndex*. Von einem Knoten w kommt man im Tiefensuchbaum genau dann zu einem Knoten v , wenn der Aufruf der Prozedur *DFS* für w vor dem Aufruf von *DFS* für v liegt und *DFS* für v früher abgeschlossen ist als für w . Anders ausgedrückt heißt das, daß $w \rightarrow_{BP}^* v$ genau dann gilt, wenn $DFBI[w] \leq DFBI[v]$ und $DFEI[w] \geq DFEI[v]$ gelten. Ein Pfeil (w, v) ist genau dann ein Baumpfeil oder ein Vorwärtspfeil, wenn $DFBI[w] \leq DFBI[v]$ gilt. Andernfalls ist ein Pfeil ein Rückwärts- oder Seitwärtspfeil. Damit ergibt sich für die Tiefensuche eine Laufzeit von $O(|V| + |E|)$.

Bei ungerichteten Graphen sind die Verhältnisse einfacher. Zunächst kann es keine Seitwärtskanten geben, weil eine Tiefensuche einer solchen Kante folgen würde. Natürlich bilden die Baumkanten einen Baum der durch die Tiefensuche erreichten Knoten. Alle anderen Kanten werden durch die Tiefensuche zu Rückwärtskanten. Mit diesen Überlegungen genügt es also, bei der Tiefensuche für jeden Knoten bzw. Pfeil eine konstante Anzahl von Schritten aufzuwenden.

Wir wollen im folgenden Abschnitt ein Beispiel für die Anwendung der Tiefensuche betrachten; weitere Beispiele findet man etwa in .

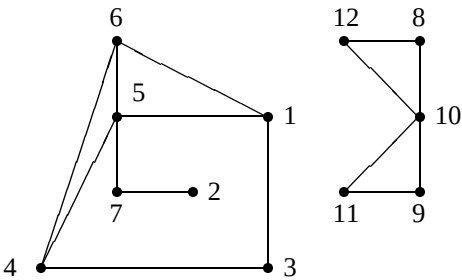
8.4 Zusammenhangskomponenten

Die Bestimmung einfacher Zusammenhangskomponenten ungerichteter Graphen haben wir im letzten Abschnitt, beim Durchlaufen von Graphen, bereits behandelt. Bei der Definition des Zusammenhangs in gerichteten Graphen ist es sinnvoll, die Richtung von Pfeilen zu berücksichtigen. So kann man sich etwa fragen, ob man in einem Netz von Einbahnstraßen einer Stadt überhaupt von jeder Kreuzung zu jeder anderen Kreuzung gelangen kann.

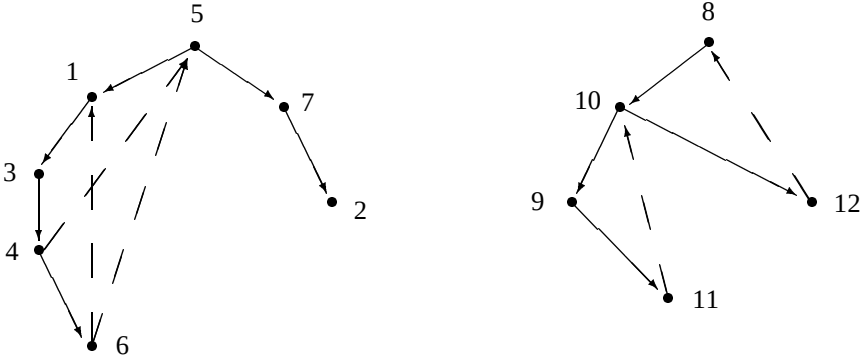
Wir bezeichnen einen Digraphen $G = (V, E)$ als *stark zusammenhängend*, wenn es einen Weg von jedem Knoten zu jedem anderen Knoten im Graphen gibt. Eine *starke Zusammenhangskomponente* (englisch: *strongly connected component*; *scc*) eines Digraphen G ist ein (bezüglich Mengeninklusion) maximaler, stark zusammenhängender Untergraph von G . Einen ungerichteten Graphen $G = (V, E)$ nennen wir *zweifach zusammenhängend* (englisch: *biconnected*), wenn nach dem Entfernen eines beliebigen Knotens v aus G der verbleibende Graph $G - v$ zusammenhängend ist. Eine *zweifache Zusammenhangskomponente* (englisch: *biconnected component*; *bcc*) eines ungerichteten Graphen ist ein (bezüglich Mengeninklusion) maximaler, zweifach zusammenhän-

gender Untergraph. In einem zweifach zusammenhängenden Graphen kann man einen beliebigen Knoten samt allen inzidenten Kanten entfernen, ohne daß der Graph zerfällt. Ein Knoten v ist ein *Schnittpunkt* (englisch: *cut point*, *articulation point*) eines Graphen G , wenn $G - v$ mehr Zusammenhangskomponenten hat als G . Durch Wegnahme eines Schnittpunkts zerfällt also eine Zusammenhangskomponente des Graphen.

Betrachten wir als Beispiel den in Abbildung 8.12 (a) gezeigten Graphen.



(a)



Knoten	1	2	3	4	5	6	7	8	9	10	11	12
DFBI	2	7	3	4	1	5	6	8	10	9	11	12
DFEI	4	5	3	2	7	1	6	12	9	11	8	10

(b)

Abbildung 8.12

Er besteht aus zwei einfachen Zusammenhangskomponenten; keine von beiden ist zweifach zusammenhängend. Die Schnittpunkte des Graphen sind die Knoten 5, 7 und

10. Die zweifachen Zusammenhangskomponenten sind die durch die Knotenmengen $\{1, 3, 4, 5, 6\}$, $\{2, 7\}$, $\{5, 7\}$, $\{8, 10, 12\}$ und $\{9, 10, 11\}$ induzierten Untergraphen.

8.4.1 Zweifache Zusammenhangskomponenten

Zur Berechnung der zweifachen Zusammenhangskomponenten ermitteln wir die Schnittpunkte eines Graphen mit folgenden Überlegungen. Ein Schnittpunkt ist die ausschließliche Verbindung von wenigstens zwei zweifachen Zusammenhangskomponenten. Wenn also ein Schnittpunkt v Wurzel eines Tiefensuchbaums ist, so hat v im Tiefensuchbaum mehr als einen Sohn, weil die Tiefensuche nicht anders als über v von der einen in die andere zweifache Zusammenhangskomponente gelangen kann. In Abbildung 8.12 (b) ist der mögliche Verlauf einer Tiefensuche, beginnend bei Knoten 5 und bei Knoten 8, mit dem sich ergebenden $DFBI$ und $DFEIndex$ gezeigt. Knoten 5 als Schnittpunkt und Wurzel eines Tiefensuchbaums hat einen Sohn für jede einfache Zusammenhangskomponente, die sich durch Entfernen des Knotens 5 ergibt.

Trifft man während der Tiefensuche auf einen Schnittpunkt v , d.h., ist v nicht Wurzel eines Tiefensuchbaums, so muß sich wenigstens eine zweifache Zusammenhangskomponente im Tiefensuchbaum in einem Teilbaum ab v befinden; aus einem solchen Teilbaum heraus darf also keine Kante zu einem Vorgänger von v führen. Anders ausgedrückt: Ist ein Schnittpunkt v nicht Wurzel eines Tiefensuchbaums, dann hat v einen Sohn v' , so daß kein Nachfolger von v' im Tiefensuchbaum, inklusive v' selbst, über eine Rückwärtskante mit einem Vorgänger von v verbunden ist. Im Beispiel der Abbildung 8.12 ist Knoten 10 ein solcher Schnittpunkt; von Knoten 9 und Knoten 11 führt keine Rückwärtskante über Knoten 10 hinaus. Das ist auch intuitiv plausibel, weil die Tiefensuche in der anderen der beiden zweifachen Zusammenhangskomponenten begonnen hat, die durch Knoten 10 verbunden sind.

Dies legt nahe, sich während der Tiefensuche für jeden Knoten zu merken, wie weit man über Rückwärtskanten höchstens im $DFBI$ zurückgelangen kann. Dies leistet ein für jeden Knoten v während der Tiefensuche zu berechnender Wert $P[v]$, der durch $P[v] := \min(\{DFBI[v]\} \cup \{DFBI[v'] \mid v' \text{ ist Vorgänger von } v \text{ im DFS-Baum und ist mit Rückwärtskante mit Nachfolger von } v \text{ verbunden}\})$ definiert ist. Wenn nun ein Schnittpunkt v nicht Wurzel eines DFS-Baumes ist, dann hat v einen Sohn v' mit $P[v'] \geq DFBI[v]$. Um die Berechnung von $P[v]$ in den rekursiv formulierten Tiefensuchalgorithmus einzubetten, formulieren wir $P[v]$ zunächst noch rekursiv, und zwar als $P[v] := \min(\{DFBI[v]\} \cup \{P[v'] \mid v' \text{ ist Sohn von } v\} \cup \{DFBI[v'] \mid (v, v') \text{ ist Rückwärtskante}\})$. Ein Programmstück, das nach diesem Verfahren die zweifachen Zusammenhangskomponenten zu einem Graphen mittels einer rekursiv formulierten Tiefensuche berechnet, ist dann das folgende:

procedure *DFSBC* für G ab Knoten v ;

begin

$B := B \cup \{v\}$;

erhöhe *dfbi* um 1;

$DFBI[v] := dfbi$;

$P[v] := dfbi$;

for all $(v, v') \in E$ **do**

{beachte, daß $(v, v') = (v', v)$ die Kante identifiziert, daß also in der Schleife jede Kante genau einmal bearbeitet wird}

```

begin
  lege  $(v, v')$  auf Stapel BCC;
  {Stapel BCC speichert begonnene bcc's}
  if  $v' \notin B$ 
    then  $\{(v, v') \text{ ist eine Baum-Kante}\}$ 
      begin
        Vater $[v'] := v$ ;
        DFSBCC für  $G$  ab Knoten  $v'$ ;
        if  $P[v'] \geq DFBI[v]$ 
          then  $\{v \text{ ist Schnittpunkt oder letzter Knoten dieser Komponente}\}$ 
            nimm jede Kante bis inkl.  $(v, v')$  vom Stapel BCC und
            berichte sie als bcc;
            {jetzt ist Sohn  $v'$  behandelt}
             $P[v] := \min(P[v], P[v'])$ 
          end
        else if  $v' \neq \text{Vater}[v]$ 
          then  $\{(v, v') \text{ ist Rückwärtskante}\}$ 
             $P[v] := \min(P[v], DFBI[v'])$ 
        end
      end  $\{\text{DFSBCC}\}$ 

begin
   $B := \emptyset$ ; {bereits besuchte Knoten}
   $dfbi := 0$ ;
  BCC := leerer Stapel;
  for all  $v \in V$  do
    if  $v \notin B$ 
      then DFSBCC für  $G$  ab Knoten  $v$ 
    end
  end

```

Abbildung 8.13 zeigt die Berechnung der zweifachen Zusammenhangskomponenten mit Hilfe von DFSBCC für den in 8.12 (a) gezeigten Graphen ab Knoten 5, wenn die Tiefensuche verläuft wie in 8.12 (b) skizziert. Momentaufnahmen des Stapels BCC sind unmittelbar vor und nach jeder Entnahme der Kanten einer zweifachen Zusammenhangskomponente wiedergegeben.

Aus der Effizienz der Tiefensuche und den zusätzlich erforderlichen Operationen mit Stapel BCC, auf dem jede Kante des Graphen gerade einmal abgelegt wird, ergibt sich als Laufzeit für die Berechnung der zweifachen Zusammenhangskomponenten eines ungerichteten Graphen $G = (V, E)$ unmittelbar $O(|V| + |E|)$.

Knoten	1	2	3	4	5	6	7	8	9	10	11	12
<i>P</i>	2	7	3	4	1	5	6	8	10	9	11	12
	1		1	1		2			9	8	9	8
						1						

Stapel *BCC*:

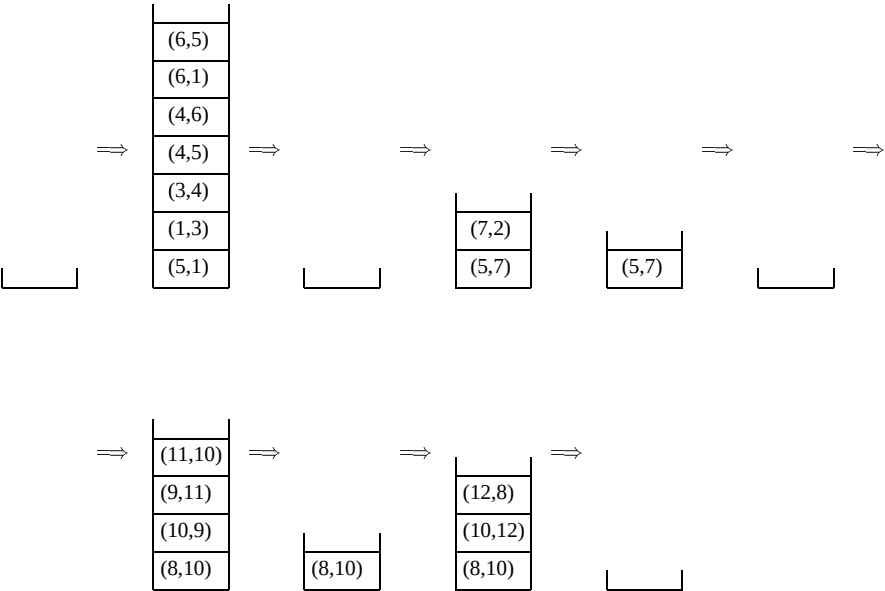
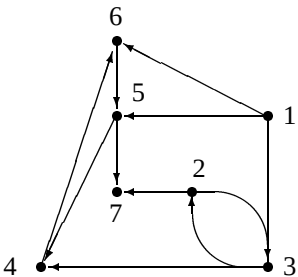


Abbildung 8.13

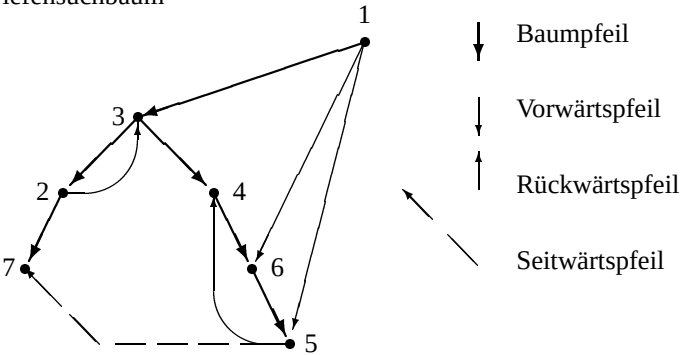
8.4.2 Starke Zusammenhangskomponenten

Betrachten wir nun das Problem, zu einem gegebenen Digraphen die starken Zusammenhangskomponenten zu berechnen. Im Beispiel der Abbildung 8.14 (a) sind dies die durch die vier Knotenmengen {1}, {2,3}, {4,5,6} und {7} induzierten Untergraphen. Abbildung 8.14 (b) zeigt den Verlauf und das Resultat einer beim Knoten 1 beginnenden Tiefensuche. Wir wollen uns nun überlegen, in welcher Reihenfolge die Tiefensuche die Knoten starker Zusammenhangskomponenten komplett besucht hat, also wieder verläßt. Im Beispiel der Abbildung 8.14 ist die erste komplett besuchte starke Zusammenhangskomponente diejenige mit Knotenmenge {7}; kein Pfeil verläßt diese Komponente, und der größte DFEIndex eines Knotens dieser Komponente ist 1. Die nächste durch die Tiefensuche komplett besuchte starke Zusammenhangskomponente ist diejenige mit Knotenmenge {4,5,6}. Der einzige Pfeil, der diese Komponente verläßt, führt zu einem Knoten einer bereits berechneten starken Zusammenhangskomponente (Pfeil (5,7) führt zu Knoten 7).



(a)

Tiefensuchbaum



Tiefensuche

Knoten	1	2	3	4	5	6	7
DFBI	1	3	2	5	7	6	4
DFEI	7	2	6	5	3	4	1

(b)

Abbildung 8.14

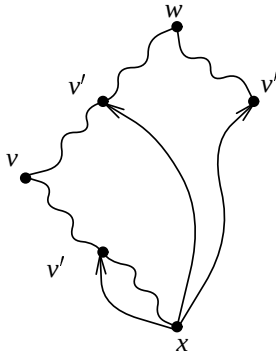


Abbildung 8.15

Natürlich kann eine starke Zusammenhangskomponente bei der Tiefensuche nicht in mehrere Tiefensuchbäume zerfallen, weil ja jeder Knoten der starken Zusammenhangskomponente von jedem anderen aus erreichbar ist. Diejenigen Pfeile einer starken Zusammenhangskomponente, die in einem DFS-Wald Baumpfeile sind, bilden zusammen *einen* DFS-Baum. Die Wurzel des DFS-Baums für eine starke Zusammenhangskomponente nennen wir *Wurzel der Zusammenhangskomponente*. Im Beispiel der Abbildung 8.14 sind die Knoten 7, 4, 3 und 1 Wurzeln von starken Zusammenhangskomponenten. Wir wollen starke Zusammenhangskomponenten berechnen, indem wir ihre Wurzeln in einem Tiefensuchwald bestimmen. Weil der DFEIndex der Wurzel eines Teilbaums der größte DFEIndex der Knoten dieses Teilbaums ist, betrachten wir die Wurzeln von starken Zusammenhangskomponenten in der Reihenfolge aufsteigender DFEIndizes. Seien dies die Wurzeln $w_1, w_2, \dots, w_k$. Haben wir eine Wurzel w_i einer starken Zusammenhangskomponente in einem Tiefensuchbaum gefunden, so gehören zu dieser Komponente all diejenigen Knoten, die im Teilbaum des Tiefensuchbaums mit Wurzel w_i stehen, aber nicht auch in bereits identifizierten Teilbäumen mit Wurzeln $w_1, \dots, w_{i-1}$. Im Beispiel der Abbildung 8.14 sind dies etwa die Knoten des Teilbaums mit Wurzel 3, die nicht auch im Teilbaum mit Wurzel 4 oder im Teilbaum mit Wurzel 7 liegen, also die Knoten 2 und 3.

Während der Tiefensuche berechnen wir für jeden Knoten v einen Wert $Q[v]$, der uns darüber Auskunft gibt, ob v Wurzel einer starken Zusammenhangskomponente ist. Dazu definieren wir $Q[v] := \min(\{DFBI[v]\} \cup \{DFBI[v']\})$ für einen Nachfolger x von v ist $(x, v') \in RP \cup SP$, und die Wurzel w der starken Zusammenhangskomponente von v' ist Vorgänger von v). Die Begriffe Nachfolger und Vorgänger beziehen sich dabei auf den betrachteten Tiefensuchbaum. Dann ist die Wurzel einer starken Zusammenhangskomponente der Knoten v mit $Q[v] = DFBI[v]$. Abbildung 8.15 illustriert, auf welche Arten ein Zyklus von Knoten v über die Knoten x, v' und w zum Knoten v möglich ist. Man beachte, daß dabei ein Knoten v' , der Nachfolger von v ist, wegen $DFBI[v'] > DFBI[v]$ nichts zu $Q[v]$ beiträgt. Zur Einbettung der Berechnung von Q in die rekursiv formulierte Tiefensuche läßt sich Q auch rekursiv formulieren:

$Q[v] := \min(\{DFBI[v]\} \cup \{Q[v'] \mid v' \text{ ist Sohn von } v\} \cup \{DFBI[v'] \mid (v, v') \in RP \cup SP, \text{ und die Wurzel } w \text{ der starken Zusammenhangskomponente von } v' \text{ ist Vorgänger von } v\})$.

Das folgende Programmstück berechnet zu einem gegebenen Digraphen die starken Zusammenhangskomponenten nach diesem Verfahren, wobei die Vereinbarung eines Feldes

var gestapelt: array [knotentyp] **of** boolean

vorausgesetzt wird:

procedure DFSSCC für G ab Knoten v ;

begin

$B := B \cup \{v\}$; {Menge bereits besuchter Knoten}

erhöhe $dfbi$ um 1;

$DFBI[v] := dfbi$;

$Q[v] := dfbi$;

lege v auf Stapel SCC;

{Stapel SCC speichert Knoten, die noch keiner scc zugeordnet sind}

$gestapelt[v] := true$;

for all $(v, v') \in E$ **do**

if $v' \notin B$

then { v' noch nicht besucht}

begin

DFSSCC für G ab Knoten v' ;

$Q[v] := \min(Q[v], Q[v'])$

end

else if $DFBI[v'] < DFBI[v]$ **and** $gestapelt[v']$

then $Q[v] := \min(Q[v], DFBI[v'])$;

if $Q[v] = DFBI[v]$ {Wurzel einer scc}

then nimm jeden Knoten u bis incl. v vom Stapel SCC und berichte scc, und setze jeweils $gestapelt[u] := false$

end; {DFSSCC}

begin

$B := \emptyset$; {anfangs noch kein Knoten besucht}

$dfbi := 0$;

SCC := leerer Stapel;

for all $v \in V$ **do** $gestapelt[v] := false$;

for all $v \in V$ **do**

if $v \notin B$

then DFSSCC für G ab Knoten v

end

Abbildung 8.16 zeigt die Berechnung der starken Zusammenhangskomponenten mit Hilfe von DFSSCC für den in Abbildung 8.14 (a) gezeigten Graphen ab Knoten 1, wenn die Tiefensuche verläuft wie in Abbildung 8.14 (b) skizziert. Momentaufnahmen des Stapels SCC sind unmittelbar vor und nach jeder Entnahme der Pfeile einer starken Zusammenhangskomponente wiedergegeben.

Knoten	1	2	3	4	5	6	7
Q	1	2 2	2	5	7 5	6 5	4

Stapel SCC

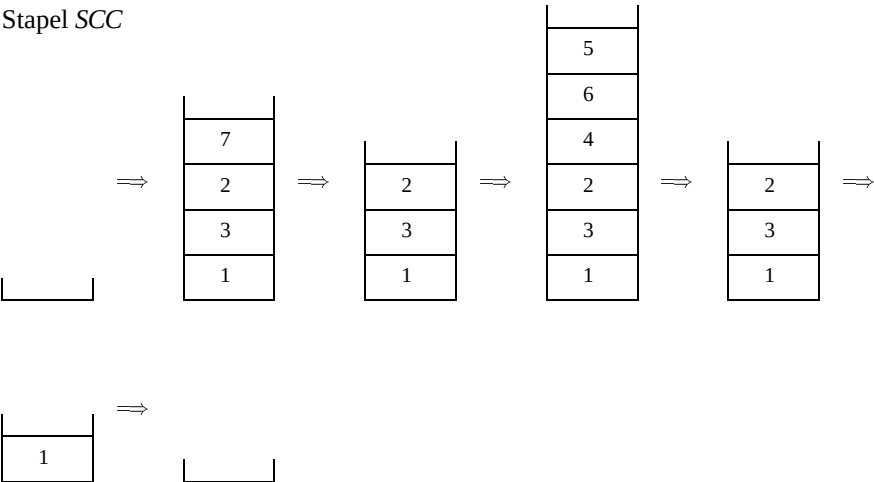


Abbildung 8.16

Aus der Effizienz der Tiefensuche und der zusätzlich erforderlichen Operationen mit Stapel SCC, auf dem jeder Knoten des Graphen gerade einmal abgelegt wird, sowie der Überprüfung, ob ein Knoten gestapelt ist, die mit Hilfe des Feldes *gestapelt* in konstanter Zeit stattfindet, ergibt sich für die Berechnung der starken Zusammenhangskomponenten eines Digraphen $G = (V, E)$ als Laufzeit unmittelbar $O(|V| + |E|)$.

Interpretiert man die Menge der Pfeile als eine Relation über der Menge der Knoten, so definieren die starken Zusammenhangskomponenten gerade die Äquivalenzklassen der Relation. Wenn man den gegebenen Digraphen verdichtet, indem man jede starke Zusammenhangskomponente durch einen Knoten ersetzt und Pfeile zwischen Knoten derselben Zusammenhangskomponente wegläßt, so stellt der entstehende zyklenfreie, verdichtete Digraph gerade die partielle Ordnung über den Äquivalenzklassen der Relation dar. Für den in Abbildung 8.14 angegebenen Beispielgraphen zeigt Abbildung 8.17 den verdichteten Graphen.

Genauer: Für einen gegebenen Digraphen $G = (V, E)$ mit Knotenmengen $V_1, \dots, V_k$ für k starke Zusammenhangskomponenten heißt der Digraph $G' = (V', E')$ mit $V' = \{1, \dots, k\}$ und $E' = \{(i, j) \mid \exists v \in V_i, v' \in V_j, (v, v') \in E\}$ *verdichteter Digraph*. G' ist azyklisch. Für die Graphen mit wenigen starken Zusammenhangskomponenten führt der Umweg über den verdichteten Digraphen zu einem schnelleren Algorithmus zur Berechnung der reflexiven, transitiven Hülle, gemäß folgender Beobachtung. Ein Pfeil (i, j) in der reflexiven, transitiven Hülle des verdichteten Digraphen impliziert Pfeile von allen Knoten der starken Zusammenhangskomponente V_i zu allen Knoten der starken Zusammenhangskomponente V_j in der reflexiven, transitiven Hülle des gegebenen

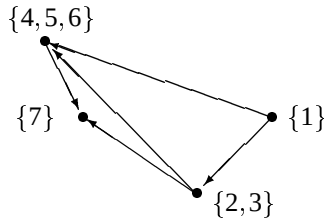


Abbildung 8.17

Graphen. Außerdem gibt es in der reflexiven, transitiven Hülle des gegebenen Graphen G Pfeile zwischen allen Knoten innerhalb jeder starken Zusammenhangskomponente. Damit läßt sich die reflexive, transitive Hülle eines gegebenen Digraphen $G = (V, E)$ wie folgt berechnen:

1. Berechne die starken Zusammenhangskomponenten $V_1, \dots, V_k$.
2. Berechne den verdichteten Digraphen $G' = (V', E')$.
3. Berechne die reflexive, transitive Hülle $G'^* = (V', E'^*)$ von G' .
4. Berechne die reflexive, transitive Hülle $G^* = (V, E^*)$ von G .

Die ersten beiden Teile dieses Algorithmus benötigen jeweils $O(|V| + |E|)$ Schritte; Teil 3 kann gemäß Abschnitt 8.2 schlimmstenfalls in $O(k^3)$ Schritten gelöst werden, und Teil 4 benötigt offenbar höchstens $O(|E^*|)$ Schritte. Damit kann für einen gegebenen Digraphen $G = (V, E)$ mit k starken Zusammenhangskomponenten die reflexive, transitive Hülle in Zeit $O(|V| + |E^*| + k^3)$ berechnet werden.

8.5 Kürzeste Wege

Bei der Modellierung realer Probleme durch Graphen ist es oft wichtig, nicht nur das Vorhandensein oder Fehlen von Knoten und Kanten zu unterscheiden. Vielmehr müssen Knoten und Kanten Eigenschaften zugeordnet werden, die für die Lösung des Problems wesentlich sind. Beispielsweise haben Kanalisationsrohre eine gewisse maximale Transportkapazität, Arbeiten an einem Haus eine minimale, eine maximale und eine erwartete Dauer und Bahnstrecken eine Länge und (je nach Tarif) einen Preis. In diesem Abschnitt interessieren wir uns für kostengünstigste Wege in Graphen, wenn jeder Kante/jedem Pfeil ein Kostenwert zugeordnet ist. Meist redet man dabei, stellvertretend für allerlei Interpretationen der Kosten, von der *Länge* von Kanten/Pfeilen; ein kostengünstigster Weg ist dann ein *kürzester* Weg. Wir wollen auch zulassen, daß Kanten/Pfeile eine negative Länge haben. Dann können wir Gewinne und Verluste modellieren, aber auch längste Wege durch kürzeste Wege ausdrücken, nämlich mit negativ gemachten Längen der einzelnen Kanten/Pfeile.

Ein ungerichteter Graph $G = (V, E)$ mit einer reellwertigen Bewertungsfunktion $c : E \rightarrow \mathbb{R}$ (englisch: *cost*) heißt *bewerteter Graph*. Für eine Kante $e \in E$ heißt $c(e)$ *Bewertung (Länge, Gewicht, Kosten)* der Kante e . Die Länge $c(G)$ des Graphen G ist die Summe der Längen aller Kanten, also $c(G) = \sum_{e \in E} c(e)$. Damit ist für einen Weg $p = (v_0, v_1, \dots, v_k)$ die Länge dieses Wegs gerade $c(p) = \sum_{i=0}^{k-1} c((v_i, v_{i+1}))$. Für Graphen ohne Bewertung, die wir bisher betrachtet haben, haben wir die Länge von Wegen so definiert, als sei $c \equiv 1$. Die *Entfernung* d (Distanz; englisch: *distance*) von einem Knoten v zu einem Knoten v' ist definiert als $d(v, v') = \min\{c(p) \mid p \text{ ist Weg von } v \text{ nach } v'\}$, falls es überhaupt einen Weg von v nach v' gibt; sonst ist $d(v, v') = \infty$. Ein Weg p zwischen v und v' mit $c(p) = d(v, v')$ heißt *kürzester Weg* (englisch: *shortest path*) zwischen v und v' ; wir bezeichnen ihn mit $sp(v, v')$.

Ganz entsprechend heißt ein Digraph $G = (V, E)$ mit Bewertungsfunktion $c : E \rightarrow \mathbb{R}$ *bewerteter Digraph*; wenn er keine Knoten ohne inzidente Pfeile hat, heißt er *Netzwerk*. Die übrigen Begriffe sind entsprechend definiert.

Ist die Länge jeder Kante nicht negativ, also $c : E \rightarrow \mathbb{R}_0^+$, so heißt $G = (V, E)$ mit c *Distanzgraph* (in Abschnitt 6.1.1 haben wir Distanzgraphen betrachtet und die Kosten einer Kante entsprechend als Länge bezeichnet). Die Berechnung kürzester Wege in Distanzgraphen ist einfacher und kann schneller ausgeführt werden als in beliebigen bewerteten Graphen, weil sich in Distanzgraphen Wege durch Hinzunahme weiterer Kanten nicht verkürzen können. Algorithmen für das Finden kürzester Wege zwischen gegebenen Knoten in ungerichteten Distanzgraphen operieren nach dem Grundmuster der Breitensuche. Als Folge davon werden beim Berechnen eines kürzesten Weges von einem gegebenen *Anfangsknoten* zu einem gegebenen *Endknoten* auch kürzeste Wege vom Anfangsknoten zu vielen anderen Knoten des Graphen ermittelt. Das Verfahren zur Berechnung eines kürzesten Weges zwischen Anfangs- und Endknoten (*one-to-one shortest path*, *single pair shortest path*) unterscheidet sich vom Verfahren zur Berechnung der kürzesten Wege von einem Anfangsknoten zu allen anderen Knoten des Graphen (*one-to-all shortest paths*, *single source shortest paths*) nur durch das Abbruchkriterium. Im schlimmsten Fall haben beide Verfahren dieselbe Laufzeit; wir werden daher im folgenden das Problem kürzester Wege von einem zu allen anderen Knoten zunächst für Distanzgraphen und dann für beliebige bewertete Graphen betrachten. Dem Problem, zu jedem Paar von Knoten einen kürzesten Weg zu finden, werden wir uns am Schluß dieses Abschnitts zuwenden.

8.5.1 Kürzeste Wege in Distanzgraphen

Wir betrachten das Problem, zu einem gegebenen Distanzgraphen $G = (V, E)$ mit $c : E \rightarrow \mathbb{R}_0^+$ je einen kürzesten Weg von einem gegebenen Anfangsknoten s (englisch: *source*) zu jedem anderen Knoten des Graphen zu finden. Abbildung 8.18 zeigt ein Beispiel für einen ungerichteten Distanzgraphen; neben jeder Kante ist deren Länge vermerkt. Man sieht leicht, daß ein kürzester Weg, beispielsweise von Knoten 1 zu Knoten 8, gefunden werden kann, indem man eine Art äquidistanter Welle um den Knoten 1 solange wachsen läßt, bis sie den Knoten 8 erreicht.

Wichtig für das dieser Idee zugrunde liegende Verlängern eines Wegs durch Hinzunahme einer weiteren Kante ist das *Optimalitätsprinzip*: Für jeden kürzesten Weg

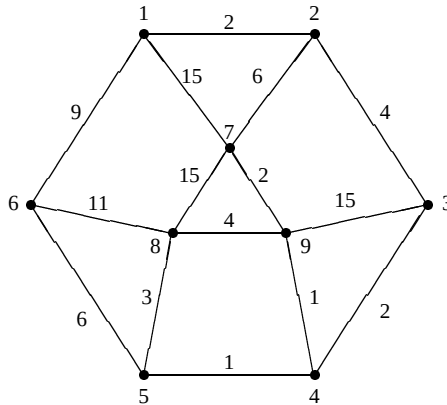


Abbildung 8.18

$p = (v_0, v_1, \dots, v_k)$ von v_0 nach v_k ist jeder Teilweg $p' = (v_i, \dots, v_j)$, $0 \leq i < j \leq k$, ein kürzester Weg von v_i nach v_j . Wäre dies nicht so, gäbe es also einen kürzeren Weg p'' von v_i nach v_j , so könnte auch in p der Teilweg p' durch p'' ersetzt werden, und der entstehende Weg von v_0 nach v_k wäre kürzer als p ; dies ist aber ein Widerspruch zu der Annahme, daß p ein kürzester Weg von v_0 nach v_k ist. Damit können wir länger werdende kürzeste Wege durch Hinzunahme einzelner Kanten zu bereits bekannten kürzesten Wegen mit folgender *Invariante* berechnen:

1. Für alle kürzesten Wege $sp(s, v)$ und Kanten (v, v') gilt:
 $c(sp(s, v)) + c((v, v')) \geq c(sp(s, v'))$.
2. Für wenigstens einen kürzesten Weg $sp(s, v)$ und eine Kante (v, v') gilt:
 $c(sp(s, v)) + c((v, v')) = c(sp(s, v'))$.

Abbildung 8.19 zeigt, wie die entsprechende Berechnung kürzester Wege realisiert werden kann. Jeder Knoten gehört zu einer von drei Klassen: Er ist entweder *gewählter Knoten*, *Randknoten* oder *unerreichter Knoten*. Zu jedem gewählten Knoten ist ein kürzester Weg vom Anfangsknoten s bereits bekannt; zu jedem Randknoten kennt man einen Weg von s , und für jeden unerreichten Knoten kennt man noch keinen solchen Weg.

Wir merken uns für jeden Knoten v die bisher berechnete, vorläufige *Entfernung* zum Anfangsknoten s , den *Vorgänger* von v auf dem bisher berechneten, vorläufig kürzesten Weg von s nach v und eine Markierung, die darüber Auskunft gibt, ob der Knoten bereits *gewählt* ist oder nicht. Außerdem speichern wir die Menge R der Randknoten. Dann realisiert der folgende, von Dijkstra  bereits 1959 vorgeschlagene Algorithmus die Berechnung kürzester Wege von einem Knoten zu allen anderen in der skizzierten Weise:

Algorithmus kürzeste Wege in $G = (V, E)$ mit $c : E \rightarrow \mathbb{R}_0^+$ von einem Knoten $s \in V$ zu allen anderen

1. *{Initialisierung:}*
 - 1.1 *{anfangs sind alle Knoten außer s unerreicht:}*

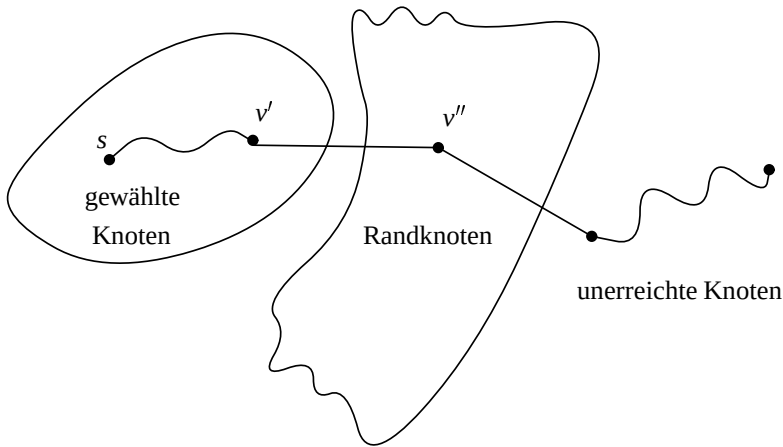


Abbildung 8.19

```

for all  $v \in V - \{s\}$  do
  begin
     $v.Vorgänger := \text{undefiniert};$ 
     $v.Entfernung := \infty;$ 
     $v.gewählt := \text{false}$ 
  end;
1.2 { $s$  ist gewählt;}
   $s.Vorgänger := s;$ 
   $s.Entfernung := 0;$ 
   $s.gewählt := \text{true};$ 
1.3 {alle zu  $s$  adjazenten Knoten gehören zum Rand  $R$ ;}
   $R := \emptyset;$ 
  ergänze  $R$  bei  $s$ ;
2. {berechne Wege ab  $s$ ;}
  while  $R \neq \emptyset$  do
    begin
      {wähle nächstgelegenen Randknoten;}
2.1 wähle  $v \in R$  mit  $v.Entfernung$  minimal, und entferne  $v$  aus  $R$ ;
2.2  $v.gewählt := \text{true};$ 
2.3 ergänze  $R$  bei  $v$ 
    end
  end {kürzeste Wege}

```

Das Ergänzen des Randes R bei einem gewählten Knoten v besteht in der Hinzunahme aller unerreichten Knoten zum Rand R und im Anpassen der möglicherweise kürzer gewordenen Entfernungen zu Randknoten:

ergänze Rand R bei v :

for all $(v, v') \in E$ **do**

if not $v'.\text{gewählt}$ **and** $(v.\text{Entfernung} + c((v, v')) < v'.\text{Entfernung})$

then $\{v' \text{ ist (kürzer) über } v \text{ erreichbar}\}$

begin

$v'.\text{Vorgänger} := v$;

$v'.\text{Entfernung} := v.\text{Entfernung} + c((v, v'))$;

 vermerke v' in R

end

Abbildung 8.20 zeigt, wie für den in Abbildung 8.18 gezeigten Graphen eine Suche nach allen kürzesten Wegen vom Knoten 1 aus nach diesem Algorithmus verläuft. Der jeweils gewählte Knoten und die aktuelle Menge R der Randknoten sind im Zeitablauf angegeben. Man sieht, wie sich vorläufige Distanzen von Randknoten ändern können, etwa am Beispiel des Knotens 8. Wenn die von Knoten 1 ausgesandte äquidistante Welle mit aktueller Distanz 8 den Knoten 7 erreicht hat, wird Knoten 8 als mit 7 adjazenter Knoten zu einem Randknoten; seine vorläufige Distanz zu Knoten 1, die er über den Vorgängerknoten 7 realisiert, beträgt 23. Nach der Wahl des Knotens 6 verringert sich diese Distanz auf 20, nach Wahl von Knoten 9 auf 13 und nach Wahl von Knoten 5 schließlich auf 12. Anhand der Liste der gewählten Knoten und der dazugehörigen Vorgängerinformation läßt sich ein kürzester Weg von Knoten 1 zu jedem anderen Knoten rekonstruieren. Für Knoten 8 beispielsweise findet man den Vorgänger 5, für 5 den Vorgänger 4, für 4 den Knoten 3, für 3 den Knoten 2 und für 2 schließlich den Knoten 1 als Vorgänger.

Wir haben in der Illustration des Verlaufs der Berechnung kürzester Wege keine bestimmte Implementierung für die Menge der Randknoten unterstellt; schon dieses Beispiel macht aber klar, daß die geeignete Verwaltung der Randknotenmenge für die Effizienz des Verfahrens wesentlich ist. Rekapitulieren wir vor der Diskussion der verschiedenen Möglichkeiten hierfür die auf dem Rand auszuführenden Operationen:

- Rand R als leer initialisieren;
- prüfen, ob Rand R leer ist;
- wählen und entfernen des Knotens mit minimaler Entfernung aus dem Rand R ;
- neuen oder geänderten Eintrag im Rand R vermerken.

Wir betrachten die folgenden drei Implementierungsvorschläge, mit denen die angegebenen Operationen unterschiedlich gut unterstützt werden.

Keine explizite Speicherung des Randes

Der Rand wird nicht explizit gespeichert, sondern genauso behandelt wie die unerreichten Knoten. Für jeden Knoten ist also nur an der *gewählt*-Markierung erkennbar, ob er gewählt ist oder nicht. Mit der angegebenen Initialisierung der Entfernungswerte aller Knoten führt dies zum richtigen Ergebnis; diese Tatsache haben wir bereits beim Ergänzen des Randes ausgenutzt. Die angegebenen Operationen können dann wie folgt realisiert werden:

- diese Operation ist implizit, kann also entfallen;
- für alle Knoten v wird **not** $v.\text{gewählt}$ überprüft;

Knoten $\hat{=}$ (Nr., Entfernung, Vorgänger)	
gewählt	Randknoten
(1,0,1)	(2,2,1), (6,9,1), (7,15,1)
(2,2,1)	(6,9,1), (7,8,2), (3,6,2)
(3,6,2)	(6,9,1), (7,8,2), (4,8,3), (9,21,3)
(7,8,2)	(6,9,1), (4,8,3), (9,10,7), (8,23,7)
(4,8,3)	(6,9,1), (8,23,7), (9,9,4), (5,9,4)
(6,9,1)	(9,9,4), (5,9,4), (8,20,6)
(9,9,4)	(5,9,4), (8,13,9)
(5,9,4)	(8,12,5)
(8,12,5)	$\emptyset$

Abbildung 8.20

- (c) unter allen Knoten v mit **not** $v.gewählt$ wird das Minimum von $v.Entfernung$ berechnet; das Entfernen des Minimums aus dem Rand ist implizit, kann also entfallen;
- (d) diese Operation ist implizit, kann also entfallen.

Damit benötigt Schritt 1 des Algorithmus eine Laufzeit von $O(|V|)$, und in Schritt 2 werden $\Theta(|V|)$ Schleifendurchläufe mit Laufzeit jeweils $O(|V|)$ ausgeführt. Die Gesamtlaufzeit ist also $O(|V|^2)$. Diese von Dijkstra vorgeschlagene Implementierung ist sehr effizient für Graphen mit vielen Kanten. Bei $\Omega(|V|^2)$ Kanten ist die Laufzeit linear in der Größe der Eingabe, also größenordnungsmäßig optimal. Für Graphen mit weniger Kanten (*dünnere* Graphen) lohnt es sich, über andere Implementierungen des Randes nachzudenken.

Verwaltung der Randknoten in einem Heap

Da die für den Rand R benötigten Operationen (a) bis (c) gerade Heap-Operationen sind, können diese in konstanter Zeit für Operationen (a) und (b) und in logarithmischer Zeit für Operation (c) ausgeführt werden. Ist der in Operation (d) im Rand zu vermerkende Eintrag neu, so kann er gerade als Einfügeoperation im Heap in logarithmischer Zeit realisiert werden. Wenn der Heap — wie üblich — die Suche nach einem beliebigen Eintrag und das Löschen dieses Eintrags nicht unterstützt, so kann ein Knoten mit geänderter Entfernung einfach zusätzlich in den Heap eingefügt werden. Dann ist ein und derselbe Knoten unter Umständen mit mehreren verschiedenen Entfernungen im Rand gespeichert. Der Algorithmus arbeitet trotzdem korrekt, wenn man für jeden Knoten nur die erste Entnahme dieses Knotens aus dem Heap beachtet und alle weiteren ignoriert.

Da bei dieser Implementierung für jede Kante gerade ein Eintrag in den Heap vorgenommen wird, enthält dieser nie mehr als $O(|E|)$ Knoten. Weil mit $|E| \leq |V|^2$ auch $\log |E| \leq 2 \cdot \log |V|$ und damit $O(\log |E|) = O(\log |V|)$ gilt, kostet sowohl das Eintragen aller Knoten in den Heap als auch das Entfernen aller Knoten aus dem Heap jeweils $O(|E| \log |V|)$ Rechenschritte. Das ist sehr effizient für dünne Graphen, aber schlechter als Dijkstras einfache Implementierung für sehr dichte Graphen, also insbesondere

wenn $|E| = \Omega(|V|^2)$. Eine bessere Laufzeit erhält man mit einer anderen Heapstruktur, die sich für verschiedene Graphenprobleme sehr gut eignet.

Verwaltung der Randknoten in einem Fibonacci-Heap

Fibonacci-Heaps  (vgl. Kapitel 6) unterstützen die Operationen (a), (b) und (d) in konstanter amortisierter Laufzeit; lediglich Operation (c) benötigt logarithmische Zeit. Operation (d) wird für unerreichte Knoten als Einfügeoperation im Fibonacci-Heap realisiert und für Randknoten, deren Entfernung sich vermindert, als Decrease-Key-Operation. Die maximale Größe des Fibonacci-Heaps ist somit $O(|V|)$. Die $|E|$ Neueinträge und Änderungen von Knoten im Fibonacci-Heap können in Zeit $O(|E|)$ ausgeführt werden. Mit der $(|V| - 1)$ -maligen Ausführung der Operation (c), die jeweils in Zeit $O(\log|V|)$ erledigt werden kann, ergibt sich eine Gesamtlaufzeit von $O(|E| + |V|\log|V|)$ für das Finden der kürzesten Wege von einem zu allen anderen Knoten in einem Distanzgraphen, für ungerichtete ebenso wie für gerichtete Graphen

 Wählt man diese Implementierung, so kann man den Algorithmus *kürzeste Wege* spezieller als Prozedur *shortestpath* wie in Abschnitt 6.1.1 formulieren. Dort ist $v.Entfernung$ mit $d(v)$ bezeichnet, gewählte Knoten sind diejenigen in S , und auf die Berechnung der Wege (also von Vorgängerknoten auf kürzesten Wegen) wurde verzichtet.

8.5.2 Kürzeste Wege in beliebig bewerteten Graphen

Die Berechnung kürzester Wege ändert sich erheblich, wenn wir auch negative Kantenbewertungen zulassen, also eine Längenfunktion $c : E \rightarrow \mathbb{R}$ voraussetzen. Ändern wir beispielsweise in dem in Abbildung 8.18 gezeigten Graphen die Bewertung der Kante $(2, 7)$ auf -6 und die der Kante $(2, 3)$ auf -4 , so sind nicht nur die zuvor gefundenen kürzesten Wege nun keine kürzesten mehr, sondern es gibt plötzlich gar keinen kürzesten Weg mehr im Graphen. Der Grund dafür ist die Existenz eines Zyklus negativer Länge, nämlich des Zyklus $(2, 3, 4, 9, 7, 2)$ mit Länge -5 . Zu jedem denkbaren Weg zwischen zwei Knoten kann man nun einen kürzeren Weg finden, indem man einen Abstecher zu diesem *negativen Zyklus* macht und ihn — unter Umständen mehrfach — durchläuft. In einem bewerteten, ungerichteten oder gerichteten Graphen, der einen Weg von einem Knoten s zu einem Knoten t enthält, gibt es einen kürzesten Weg von s nach t genau dann, wenn kein Weg von s nach t einen Zyklus negativer Länge enthält. Wenn es einen kürzesten Weg von s nach t gibt, dann gibt es natürlich auch einen einfachen kürzesten Weg von s nach t .

Selbst im Falle negativer Kantenbewertungen lassen sich alle kürzesten Wege von einem Anfangsknoten s mit Hilfe einer Breitensuche bestimmen: Man berechnet die Länge von Wegen für zunehmende Kantenzahl. Man kann aber nicht, wie im vorangehenden Abschnitt, die Länge eines Weges zu einem gewählten Knoten als endgültig kürzest ansehen, weil das Hinzunehmen von Kanten die Länge eines Weges verkürzen kann. Die Länge eines Weges vom Anfangsknoten s aus lässt sich genau dann verkürzen, wenn der folgende *Auswahlschritt* von Ford  angewandt werden kann:

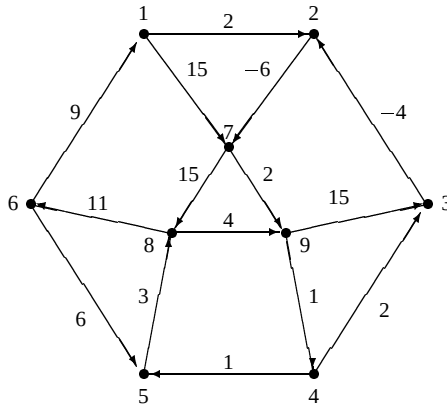


Abbildung 8.21

Auswahlschritt von Ford:

wähle eine Kante $(v, v') \in E$ mit
 $v.\text{Entfernung} + c((v, v')) < v'.\text{Entfernung}$;
 $v'.\text{Vorgänger} := v$;
 $v'.\text{Entfernung} := v.\text{Entfernung} + c((v, v'))$;

Wählen wir als vorläufige Entfernung $v.\text{Entfernung}$ anfangs 0 für $v = s$ und ∞ für $v \neq s$, dann bewahrt Fords Auswahlschritt die folgende *Invariante*: Wenn $v.\text{Entfernung}$ einen endlichen Wert hat, dann gibt es einen Weg von s nach v mit Länge $v.\text{Entfernung}$.

Ein *Auswahlverfahren nach Ford* sei nun jedes Verfahren, das den Auswahlschritt von Ford wiederholt solange anwendet, bis dies nicht mehr möglich ist. Wenn ein Auswahlverfahren nach Ford anhält, dann ist $v.\text{Entfernung}$ für jeden von s aus erreichbaren Knoten v die Länge eines kürzesten Wegs von s nach v und für alle anderen Knoten ∞ . Ein Auswahlverfahren nach Ford hält nicht an, wenn es einen von s aus erreichbaren negativen Zyklus im Graphen gibt.

Eine Implementierung eines Auswahlverfahrens nach Ford muß noch spezifizieren, wie denn eine Kante (v, v') im Auswahlschritt gewählt werden soll. Hierfür eignet sich eine Breitensuche, ähnlich wie bei Distanzgraphen, wobei aber lediglich Randknoten von Bedeutung sind. Zwischen gewählten und unerreichten Knoten wird nicht unterschieden. Durch Abänderung des Algorithmus für kürzeste Wege in Distanzgraphen ergibt sich damit das folgende Auswahlverfahren nach Ford:

Algorithmus kürzeste Wege in $G = (V, E)$ mit $c : E \rightarrow \mathbb{R}$ von einem Knoten $s \in V$ zu allen anderen

1. {Initialisierung;}
 - 1.1 {anfangs kennt man für alle Knoten außer s keinen Weg;}

for all $v \in V - \{s\}$ **do**
begin

```

    v.Vorgänger := undefiniert;
    v.Entfernung :=  $\infty$ 
  end;
1.2 {für s ist ein Weg bekannt:}
    s.Vorgänger := s;
    s.Entfernung := 0;
1.3 {alle zu s adjazenten Knoten gehören zum Rand R:}
    R :=  $\emptyset$ ;
    verschiebe R bei s;
2. {berechne Wege ab s:}
    while R  $\neq \emptyset$  do
      begin
        wähle  $v \in R$  und entferne v aus R;
        verschiebe R bei v
      end
    end {kürzeste Wege}

```

Beim Verschieben des Randes bei einem Knoten v werden alle mit v inzidenten Kanten auf ihre Eignung für den Auswahlsschritt von Ford überprüft, und für die geeigneten Kanten wird der Auswahlsschritt durchgeführt:

```

verschiebe R bei v :
  for all  $(v, v') \in E$  do
    if  $v.\text{Entfernung} + c((v, v')) < v'.\text{Entfernung}$ 
      then { $v'$  ist (kürzer) über v erreichbar}
        begin
           $v'.\text{Vorgänger} := v$ ;
           $v'.\text{Entfernung} := v.\text{Entfernung} + c((v, v'))$ ;
          vermerke  $v'$  in R, falls  $v'$  dort nicht bereits vermerkt ist
        end
      end

```

Die Prüfung, ob ein Knoten v' bereits im Rand vermerkt ist, kann mit Hilfe eines Bits pro Knoten leicht in konstanter Zeit erfolgen. Da es unerheblich ist, welcher Knoten aus dem Rand gewählt wird, kann als an der Breitensuche orientierte Datenstruktur für den Rand beispielsweise eine Schlange gewählt werden.

Man sieht, daß dieser Algorithmus demjenigen für die Berechnung kürzester Wege mit positiven Kantenbewertungen stark ähnelt; es ist instruktiv, sich die Unterschiede durch vergleichende Betrachtung beider Algorithmen deutlich zu machen.

Für den in Abbildung 8.21 gezeigten Digraphen haben wir in Abbildung 8.22 den Verlauf des Algorithmus bis zu den ersten zehn Randverschiebeoperationen angegeben. Die verschiedenen Inhalte der Schlange der Randknoten sind in zeitlicher Abfolge waagerecht nebeneinander dargestellt. Bei jedem Randknoten sind die aktuelle Entfernung zu Knoten 1 und der zugehörige Vorgänger mit angegeben, obwohl sie natürlich nicht in der Schlange verwaltet werden (sonst müßte man beispielsweise die Position eines Knotens in der Schlange kennen, um seinen Entfernungswert zu ändern). Man sieht beispielsweise, wie zunächst für Knoten 8 ein Weg der Länge 11 von Knoten 1 über Knoten 2 und Knoten 7 gefunden wird; erst später findet man den kürzeren Weg mit Länge 3 von Knoten 1 über Knoten 2, 7, 9, 4 und 5.

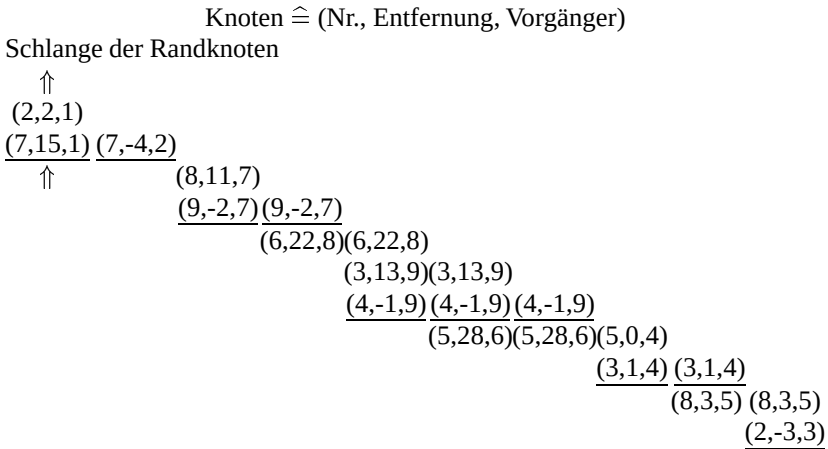


Abbildung 8.22

Der waagerechte Querstrich ist ein spezieller Markierungseintrag in der Schlange, der einfach ans Schlangenende angehängt wird, sobald er am Schlangenkopf angekommen ist. Er dient der Illustration der *Phasen* des Algorithmus. In Phase 1 werden alle Knoten erreicht, die mit *einem* Pfeil vom Anfangsknoten aus erreichbar sind; im Beispiel sind dies die Knoten 2 und 7. In Phase $j + 1$ werden alle Knoten erreicht, die mit *einem* Pfeil von den in Phase j erreichten Knoten aus erreichbar sind. Natürlich müssen die in Phase j erreichten Knoten dort nicht unbedingt erstmals erreicht worden sein. In der Schlange der Randknoten befinden sich zwischen Schlangenkopf und Phasen-Ende-Markierung und zwischen Phasen-Ende-Markierung und Schlangenende jeweils höchstens die Knoten der entsprechenden Phase, unter Umständen auch weniger. So werden etwa im gezeigten Beispiel in Phase 2 die Knoten 7, 8 und 9 erreicht, aber Knoten 7 befindet sich aus Phase 1 zu dem Zeitpunkt noch in der Schlange, zu dem er über Knoten 2 in Phase 2 erreicht wird.

Die Laufzeit des Algorithmus läßt sich abschätzen, wenn man die einzelnen Phasen betrachtet. In jeder Phase wird jeder Knoten höchstens einmal betrachtet, zusammen mit seinen inzidenten Kanten. Dies ergibt wegen der in konstanter Zeit ausführbaren einzelnen Schlangenoperationen eine Laufzeit von $O(|E|)$ für jede Phase. Weil es stets einen einfachen, also zyklensfreien, kürzesten Weg gibt — es sei denn, ein Weg über einen negativen Zyklus ist möglich —, genügt es, Wege mit höchstens $|V|$ Knoten zu betrachten. Damit kann die Berechnung kürzester Wege nach höchstens $|V|$ Phasen abgebrochen werden. Man kann also ein Auswahlverfahren nach Ford für einen bewerteten Digraphen $G = (V, E)$ so implementieren, daß es in $O(|V| \cdot |E|)$ Schritten kürzeste Wege von einem zu allen anderen Knoten berechnet, falls diese existieren. Andernfalls hält dieses Verfahren nicht an. Zählt man allerdings die Anzahl der Phasen mit, dann kann man in jedem Fall nach dem Ende der $|V|$ -ten Phase anhalten. Wenn nämlich nach dem Ende der $|V|$ -ten Phase der Rand R nicht leer ist, gibt es einen vom Anfangsknoten s aus erreichbaren, negativen Zyklus in G .

Natürlich ist man bei der Reihenfolge, in der man im Schritt 2 des Algorithmus Randknoten auswählt, nicht auf die durch die Implementierung des Randes als Schlange festgelegte Reihenfolge angewiesen. Entscheidet man sich bei einem azyklischen Graphen etwa für die Reihenfolge einer topologischen Sortierung, so ist für jeden aus dem Rand gewählten Knoten die Berechnung der Entfernung endgültig. Die Laufzeit des Algorithmus verkürzt sich damit zu $O(|E|)$. Dies ist ein für die Netzplantechnik wichtiges Ergebnis, weil man damit auch längste Wege in azyklischen Graphen schnell berechnen kann: Man multipliziert einfach die Längen der Pfeile mit -1 und berechnet danach kürzeste Wege.

8.5.3 Alle kürzesten Wege

Wir betrachten nun das Problem, für jedes Paar v und v' von Knoten einen kürzesten Weg von v nach v' zu berechnen. Dieses Problem läßt sich einfach dadurch lösen, daß wir einen Algorithmus zum Finden kürzester Wege von einem zu allen anderen Knoten für jeden Knoten anwenden. Für einen Distanzgraphen ergibt sich bei dieser Vorgehensweise eine Laufzeit von $O(|V| \cdot (|E| + |V| \log |V|))$, für einen beliebigen, bewerteten Graphen ohne negative Zyklen eine Laufzeit von $O(|E| \cdot |V|^2)$. Daß es auch schneller geht, wollen wir uns für beliebige, bewertete Graphen ohne negative Zyklen überlegen. Das Verfahren, das wir hierfür verwenden wollen, hat folgende Grobstruktur:

Algorithmus alle kürzesten Wege in $G = (V, E)$ mit $c : E \rightarrow \mathbb{R}$

1. Transformiere G in einen Distanzgraphen G' so, daß kürzeste Wege erhalten bleiben;
 2. wende Algorithmus kürzeste Wege für jeden Knoten in G' an
- end** {alle kürzesten Wege}

Dabei kann der kritische Schritt, die Transformation von G in einen Distanzgraphen G' , wie folgt realisiert werden  Zunächst nimmt man einen neuen Knoten s zum Graphen hinzu und verbindet s mit je einem Pfeil mit jedem anderen Knoten des Graphen (siehe Abbildung 8.23). Wir wählen der Einfachheit halber Pfeillänge 0 für jeden dieser Pfeile, obgleich man interessanterweise jeden einzelnen Pfeil beliebig bewerten könnte.

Damit ist die Länge eines kürzesten Weges von s zu einem beliebigen anderen Knoten des Graphen stets höchstens 0. Betrachten wir nun einen Pfeil (v, v') aus G . Einer der Wege von s nach v' führt über v . Weil ein kürzester Weg $sp(s, v')$ von s nach v' nicht länger sein kann als der Umweg über v , gilt offenbar $c(sp(s, v')) \leq c(sp(s, v)) + c((v, v'))$. Damit gilt für die durch $c'((v, v')) := c((v, v')) + c(sp(s, v)) - c(sp(s, v'))$ definierte Länge c' im transformierten Graphen unmittelbar $c'((v, v')) \geq 0$. Der transformierte Graph ist also ein Distanzgraph. In dem in Abbildung 8.23 gezeigten Beispiel ergibt die Transformation für den Pfeil (v, v') eine Länge von 4 und für den Pfeil (v'', v') eine Länge von 0. Beim Aufsummieren der transformierten Längen entlang eines Weges von einem Knoten v zu einem Knoten w neutralisieren sich die Längen kürzester Wege von s zu Zwischenknoten auf dem Weg von v nach w ; lediglich die Längen kürzester Wege von s nach v und nach w bleiben übrig. Für *jeden* Weg p von einem Knoten v zu einem Knoten w gilt also $c'(p) = c(p) + c(sp(s, v)) - c(sp(s, w))$. Damit bleibt die relative

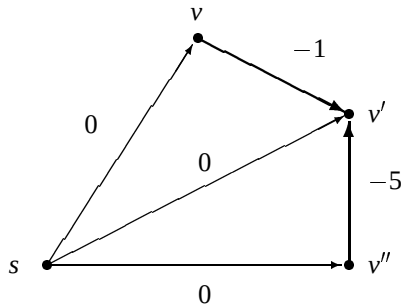


Abbildung 8.23

Ordnung der Längen aller Wege von v nach w bei der Transformation erhalten. Insbesondere bleibt also ein kürzester Weg in G auch ein kürzester Weg in G' . Algorithmisch kann die Transformation wie folgt realisiert werden:

Algorithmus transformiere $G = (V, E)$ mit $c : E \rightarrow \mathbb{R}$ in $G' = (V', E')$ mit $c' : E' \rightarrow \mathbb{R}_0^+$:

1. $V' := V \cup \{s\}$;
 $E' := E \cup \{(s, v) \mid v \in V\}$;
for all $v \in V$ **do**
 $c((s, v)) := 0$;
2. berechne kürzeste Wege in G' von s zu allen anderen Knoten $v \in V$ und vermerke die Länge jeweils in $v.$ Entfernung;
3. **for all** $(v, v') \in E$ **do**
 $c'((v, v')) := c((v, v')) + v.$ Entfernung $- v'.$ Entfernung
end {transformiere}

Schritt 1 der Transformation kann in Laufzeit $O(|V|)$ bewältigt werden; für Schritt 2 genügt eine Laufzeit von $O(|V| \cdot |E|)$, wie im vorangehenden Abschnitt gezeigt wurde. Schritt 3 kann in Zeit $O(|E|)$ erledigt werden, so daß die gesamte Transformation in Zeit $O(|V| \cdot |E|)$ durchgeführt werden kann. Die $|V|$ -malige Anwendung des Algorithmus für kürzeste Wege in einem Distanzgraphen mit einer Laufzeit von jeweils $O(|E| + |V| \log |V|)$ führt zu einer Gesamtlaufzeit des Verfahrens von $O(|V| \cdot (|E| + |V| \log |V|))$. Damit können alle kürzesten Pfade in einem beliebigen, bewerteten Graphen ebenso schnell berechnet werden wie in einem Distanzgraphen.

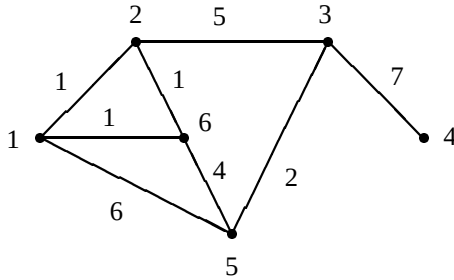


Abbildung 8.24

8.6 Minimale spannende Bäume

Ein *minimaler spannender Baum* (englisch: *minimum spanning tree*; *MST*) eines Graphen G ist ein spannender Baum von G von minimaler Gesamtlänge unter allen spannenden Bäumen von G . Minimale spannende Bäume sind oft dann von Interesse, wenn es darum geht, aus einer Vielzahl möglicher Kanten diejenigen auszuwählen, die alle Knoten mit kürzester Gesamtlänge verbinden. So kann man sich etwa vorstellen, daß in dem in Abbildung 8.24 gezeigten Graphen die Knoten hausinterne Telefonanschlüsse einer großen Firma repräsentieren und die Kantenlängen Kosten für das Legen einer entsprechenden Direktleitung sind. Telefongespräche von einer Sprechstelle zur anderen sollen auch über Zwischenstationen, also indirekt, geschaltet werden können.

In der Tat hat die amerikanische Telefonfirma AT&T die Gebühren für hausinterne Netze von Firmenkunden nach der Länge eines minimalen spannenden Baumes aller denkbaren Direktleitungen — und nicht nach der Länge der tatsächlich verlegten Leitungen — berechnet. Bei diesem Berechnungsverfahren kann es natürlich vorkommen, daß durch das Hinzunehmen weiterer Telefonanschlüsse die Gesamtkosten gesenkt werden. Dies ist leicht am Beispiel der Abbildung 8.24 einzusehen: Würde man in dem von Knotenmenge $\{2,3,4,5\}$ induzierten Untergraphen Knoten 3 entfernen und dafür Knoten 2 und 4 mit einer Kante der Länge 12, Knoten 2 und 5 mit einer Kante der Länge 7 und Knoten 4 und 5 mit einer Kante der Länge 9 direkt verbinden, so würde die Länge eines minimalen spannenden Baumes von 14 auf 16 wachsen. Das Problem, einen kürzesten Baum in einem Graphen von Telefondirektleitungen zu finden, der neben den in der Firma wirklich benötigten Telefonsprechstellen auch *optionale* Sprechstellen enthält, die nur in das Telefonnetz einbezogen werden sollen, wenn dadurch dessen Gesamtlänge verkürzt wird, ist ungleich aufwendiger zu lösen als das Problem des Findens eines minimalen spannenden Baumes; wir werden es in diesem Buch nicht weiter betrachten.

Zur Berechnung eines minimalen spannenden Baumes in einem zusammenhängenden, ungerichteten Graphen wollen wir ein *gieriges* (englisch: *greedy*) Verfahren verwenden. Bei gierigen Verfahren werden Entscheidungen, die den Rechenprozeß der

Lösung näher bringen, auf der Basis der vom Rechenprozeß bis dahin gesammelten Informationen gefällt und nicht mehr revidiert. Im Unterschied zu Verfahren, die Lösungsschritte ausprobieren und gegebenenfalls revidieren müssen, sind gierige Verfahren stets vergleichsweise effizient. Wir wählen das folgende Verfahren:

Algorithmus-Gerüst Minimaler spannender Baum

{liefert zu einem zusammenhängenden, ungerichteten, bewerteten Graphen $G = (V, E)$ mit $c : E \rightarrow \mathbb{R}$ einen minimalen spannenden Baum $T' = (V, E')$ von G }

begin

$E' := \emptyset;$

while noch nicht fertig **do**

begin

wähle geeignete Kante $e \in E;$

$E' := E' \cup \{e\}$

end

end {Minimaler spannender Baum}

Es bleibt hier im wesentlichen offen, welches geeignete Kanten sind und wie man sie wählt. Wir präzisieren das Verfahren als *Auswahlprozeß* für Kanten von G . Dabei hat eine Kante stets einen von drei Zuständen: Sie ist entweder *gewählt*, *verworfen* oder *unentschieden*. Anfangs ist jede Kante unentschieden. Es soll stets die *Auswahlinvariante* gelten, daß es einen minimalen spannenden Baum von G gibt, der alle gewählten und keine verworfenen Kanten enthält. Zu Beginn ist dies natürlich erfüllt, da alle Kanten unentschieden sind. Am Ende des Verfahrens sollen alle Kanten gewählt oder verworfen sein. Dann gilt mit der Invariante offenbar, daß gerade die gewählten Kanten einen minimalen spannenden Baum bilden.

Im Laufe der Jahre sind verschiedene effiziente Algorithmen vorgeschlagen worden, die nach diesem Verfahren operieren und für Kanten gemäß einer von zwei Regeln entscheiden, ob sie gewählt oder verworfen werden. Eine dieser Regeln betrachtet Schnitte im Graphen. Ein *Schnitt* (englisch: *cut*) in einem Graphen $G = (V, E)$ ist eine Zerlegung von V in S und $\bar{S} = V - S$. Eine Kante *kreuzt* den Schnitt, wenn sie mit einem Knoten aus S und einem aus $\bar{S}$ inzident ist. Im Beispiel der Abbildung 8.24 ist $S = \{2, 4, 5\}$ und $\bar{S} = \{1, 3, 6\}$ ein Schnitt, den alle Kanten außer $(1, 6)$ kreuzen. Die folgenden beiden Regeln dienen der Entscheidung darüber, ob eine unentschiedene Kante gewählt oder verworfen wird:

Regel 1: Wähle eine Kante

Wähle einen Schnitt, den keine gewählte Kante kreuzt.

Wähle eine kürzeste unter den unentschiedenen Kanten, die den Schnitt kreuzen.

Regel 2: Verwirf eine Kante

Wähle einen einfachen Zyklus, der keine verworfene Kante enthält.

Verwirf eine längste unter den unentschiedenen Kanten im Zyklus.

Verschiedene effiziente Algorithmen für das Berechnen minimaler spannender Bäume unterscheiden sich nun zum einen in der Reihenfolge, in der diese beiden Regeln angewandt werden, und zum anderen in der Art, wie ein Schnitt oder ein Zyklus gewählt wird. Allen gemeinsam sind die folgenden beiden Präzisierungen des Algorithmusgerüsts *Minimaler spannender Baum*:

Wähle geeignete Kante $e \in E$:

repeat

wende eine anwendbare Auswahlregel an und

until Kante $e \in E$ mit Regel 1 gewählt oder es gibt keine unentschiedene Kante mehr

noch nicht fertig:

es gibt noch unentschiedene Kanten

Jedes so operierende Verfahren ist ein korrektes Verfahren zum Berechnen eines minimalen spannenden Baumes. Weil das Algorithmusgerüst *Minimaler spannender Baum* mit den beiden angegebenen Präzisierungen Grundlage aller von uns behandelten Verfahren zum Berechnen minimaler spannender Bäume ist, wollen wir Überlegungen zu seiner Korrektheit etwas ausführlicher anstellen.

Satz 8.1 *Jedes nach dem Algorithmusgerüst Minimaler spannender Baum mit den beiden angegebenen Präzisierungen operierende Verfahren wählt oder verwirft jede Kante eines zusammenhängenden, ungerichteten, bewerteten Graphen und bewahrt die Auswahlinvariante.*

Beweis: Wir zeigen zunächst, daß die Auswahlinvariante bewahrt wird. Wir wissen bereits, daß die Invariante anfangs erfüllt ist, denn jeder zusammenhängende, ungerichtete, bewertete Graph besitzt einen minimalen spannenden Baum, und jeder minimale spannende Baum erfüllt die Invariante. Wir betrachten jetzt den Effekt der Anwendung jeder der beiden Regeln auf die Invariante.

Betrachten wir zunächst *Regel 1: Wähle eine Kante*. Sei e die mit Regel 1 gewählte Kante und sei T ein minimaler spannender Baum, der die Invariante erfüllt, bevor e gewählt wird. T enthält also alle vor der Wahl von e gewählten und keine der vor der Wahl von e verworfenen Kanten. Gehört nun e zu den Kanten von T , so wird offensichtlich die Invariante bewahrt. Gehört andererseits e nicht zu den Kanten von T , so betrachten wir den in Regel 1 gewählten Schnitt $S, \bar{S}$ (vgl. Abbildung 8.25). Wenigstens eine Kante des Wegs in T , der die beiden Endknoten von e verbindet, kreuzt diesen Schnitt; nennen wir eine solche Kante e' . Weil T die Invariante erfüllt, kann e' nicht verworfen sein. Weil Regel 1 auf den Schnitt angewandt wurde, kann e' nicht gewählt sein. Also ist e' unentschieden und wegen Regel 1 nicht kürzer als e . Dann erhalten wir aus T durch Entfernen von e' und Hinzufügen von e einen Baum $T' = (T - \{e'\}) \cup \{e\}$, der die Invariante nach Anwendung von Regel 1 erfüllt und ein minimaler spannender Baum ist.

Betrachten wir nun *Regel 2: Verwirf eine Kante*. Sei e die durch Regel 2 verworfene Kante und T ein minimaler spannender Baum, der die Invariante vor der Anwendung von Regel 2 erfüllt. Falls e nicht zu T gehört, so wird die Invariante bewahrt. Falls aber e zu T gehört, so wird T durch das Entfernen von e in zwei Teile geteilt, die einen Schnitt für G bilden; e kreuzt diesen Schnitt. Weil Regel 2 angewandt werden konnte, liegt e in einem einfachen Zyklus, der keine verworfene Kante enthält; dieser Zyklus enthält wenigstens eine andere Kante, wir nennen sie e' , die den Schnitt kreuzt (siehe Abbildung 8.26). Weil e' nicht zu T gehört, ist e' unentschieden; weil mit Regel 2 Kante e verworfen wird, ist e' nicht länger als e . Dann erhalten wir aus T durch Entfernen von e

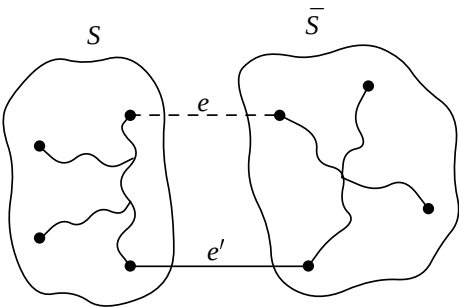


Abbildung 8.25

und Hinzunehmen von e' einen minimalen spannenden Baum $T' = (T - \{e\}) \cup \{e'\}$, der die Invariante nach Anwendung von Regel 2 erfüllt. Also wird die Auswahlinvariante im Algorithmus bewahrt.

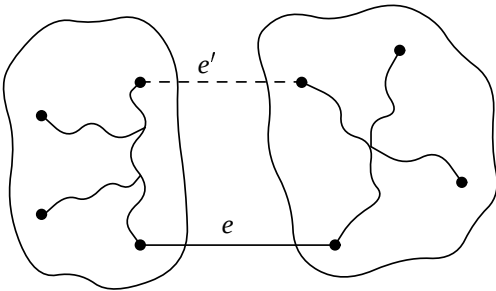


Abbildung 8.26

Wir zeigen, daß keine Kante unentschieden bleibt, indem wir aus der gegenteiligen Annahme einen Widerspruch herleiten. Nehmen wir also an, e sei eine Kante, die unentschieden bleibt. Zu jedem Zeitpunkt im Verlauf der Rechnung bilden die bereits gewählten Kanten eine Menge *gewählter Bäume*. Falls beide Endknoten von e im selben gewählten Baum liegen, ist Regel 2 anwendbar. Es kann also eine Kante verworfen werden (nicht unbedingt e). Falls beide Endknoten von e in verschiedenen gewählten Bäumen liegen, ist Regel 1 anwendbar. Es kann also eine Kante gewählt werden (nicht unbedingt e). Damit sichert die Existenz einer unentschiedenen Kante die Anwendbarkeit einer Auswahlregel; mit der Anwendung einer Auswahlregel verringert sich aber die Anzahl unentschiedener Kanten um 1. Damit kann keine Kante unentschieden bleiben. □

Betrachten wir nun im Einzelnen einige Algorithmen zur Berechnung eines minimalen spannenden Baumes. Wir werden zur Beschreibung der Algorithmen stets nur angeben, auf welche Weise Kanten gewählt oder verworfen werden.

Der Algorithmus von Boruvka

Dies ist der historisch erste Algorithmus zur Berechnung minimaler spannender Bäume; wir wollen ihn hier nur kurz skizzieren; eine parallelisierte Version hiervon, Sollins Algorithmus, wird in Kapitel 9 genauer behandelt. Für einen Graphen $G = (V, E)$ ist am Anfang jeder einzelne Knoten ein gewählter Baum. In einem *Auswahlschritt* wird für jeden gewählten Baum eine kürzeste Kante zu einem anderen Baum gewählt. Gibt es für einen Baum mehr als eine kürzeste Kante zu einem anderen Baum, so wird diejenige gewählt, die mit einem Knoten kleinster Nummer inzidiert. Auf diese Weise wird vermieden, daß in einem Auswahlschritt durch ungeschickte Entscheidung für eine von mehreren kürzesten Kanten ein Zyklus entsteht. Die wiederholte Anwendung des Auswahlschritts liefert einen minimalen spannenden Baum. Im Beispiel der Abbildung 8.24 werden im ersten Auswahlschritt die Kanten $(1, 2)$, $(2, 1)$, $(3, 5)$, $(4, 3)$, $(5, 3)$, $(6, 1)$ gewählt, wobei wir die Kanten $(1, 2)$ und $(3, 5)$ jeweils zweimal aufgeführt haben, weil sie einmal von Baum 1 bzw. Baum 3 aus und einmal von Baum 2 bzw. Baum 5 aus gewählt werden. Im zweiten Auswahlschritt wird der minimale spannende Baum durch Auswahl von Kanten $(5, 6)$, $(6, 5)$ vervollständigt.

Der Algorithmus von Kruskal

Anfangs ist jeder einzelne Knoten des Graphen ein gewählter Baum. Dann wird auf jede Kante e in aufsteigender Reihenfolge der Kantenlängen folgender *Auswahlschritt* angewandt: Falls e beide Endknoten im selben gewählten Baum hat, verwirft e , sonst wähle e . Abbildung 8.27 zeigt die gewählten Bäume und die gewählten Kanten für das Beispiel in Abbildung 8.24.

Bei einer effizienten Implementierung des Verfahrens von Kruskal muß man außer der Sortierung von Kanten nach ihrer Länge die bereits gewählten Bäume so verwalten, daß zwei gewählte Bäume zu einem gewählten Baum verbunden werden können, und daß geprüft werden kann, in welchem Baum der Endknoten einer Kante liegt. Dies gelingt gerade mit Hilfe einer Union-find-Struktur, wie sie in Kapitel 6 beschrieben ist. Eine solche Struktur bietet die folgenden Operationen an:

- *Find*(v) ist der Name des gewählten Baumes, zu dem Knoten v gehört;
- *Union*(v, w) vereinigt Bäume mit Namen v und w zu einem Baum mit Namen v ;
- *Make-set*(v) kreiert den Baum, dessen einziger Knoten v ist.

Damit kann Kruskals Verfahren im Algorithmusgerüst *Minimaler spannender Baum* wie folgt präzisiert werden:

```

begin {Kruskal}
   $E' := \emptyset$ ;
  sortiere  $E$  nach aufsteigender Länge;
  for all  $v \in V$  do
    Make-set( $v$ );
  for all  $(v, w) \in E$ , aufsteigend, do
    if Find( $v$ )  $\neq$  Find( $w$ ) then {wähle Kante  $(v, w)$ }
      begin
        Union(Find( $v$ ), Find( $w$ ));
         $E' := E' \cup \{(v, w)\}$ 
      end
  end {Kruskal}

```

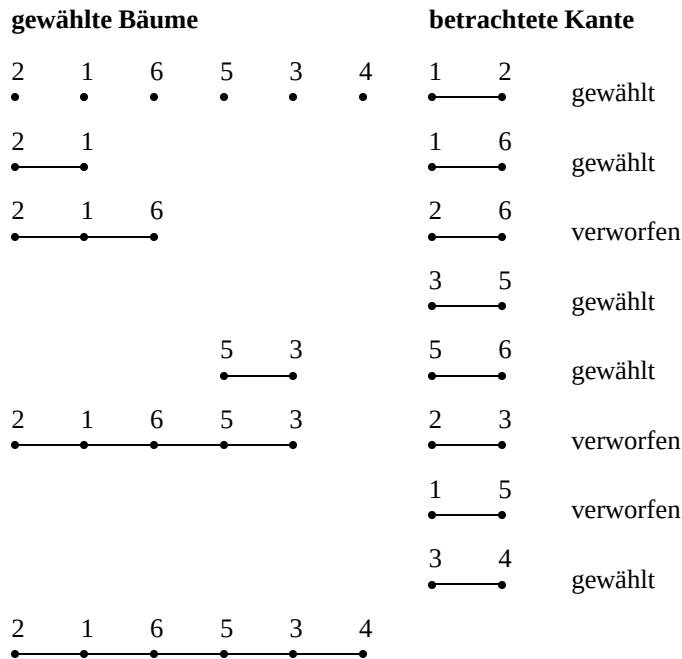


Abbildung 8.27

Das Verfahren von Kruskal ist auch schon in Abschnitt 8.6 beschrieben. Dort ist die Kollektion von Mengen explizit angesprochen, auf die hier nur über die Operationen der Union-Find-Struktur zugegriffen wird. Überdies ist in Abschnitt 6.2.1 eine Alternative zum Sortieren angegeben, das Verwalten der Kanten nach ihrer Länge in einer Prioritätswarteschlange. Beide Varianten sind asymptotisch gleich effizient. Das Sortieren der Kanten des Graphen kann in Zeit $O(|E| \log |E|) = O(|E| \log |V|)$ ausgeführt werden; für $O(|V|)$ *Make-set*-, $O(|E|)$ *Find*- und $O(|V|)$ *Union*-Operationen benötigt man nicht mehr als $O(|E| \alpha(|E|, |V|)) = O(|E| \log |V|)$ Schritte. Damit ergibt sich die gesamte Laufzeit des Verfahrens für einen Graphen $G = (V, E)$ zu $O(|E| \log |V|)$. Aber es geht noch schneller.

Der Algorithmus von Jarník, Prim, Dijkstra

Dieses Verfahren ähnelt Dijkstras Verfahren zur Berechnung kürzester Wege. Zu jedem Zeitpunkt bilden die gewählten Kanten *einen* gewählten Baum. Wir beginnen mit einem beliebigen Anfangsknoten s des Graphen und führen den folgenden *Auswahlschritt* $(|V| - 1)$ -mal aus: Wähle eine Kante mit minimaler Länge, für die genau ein Endknoten zum gewählten Baum gehört. Zu Beginn besteht der gewählte Baum aus dem Anfangsknoten s ; später bilden alle gewählten Kanten und deren inzidente Knoten den gewählten Baum. Abbildung 8.28 zeigt den Verlauf des Algorithmus, angewandt auf den Graphen der Abbildung 8.24, beginnend mit Anfangsknoten 1.

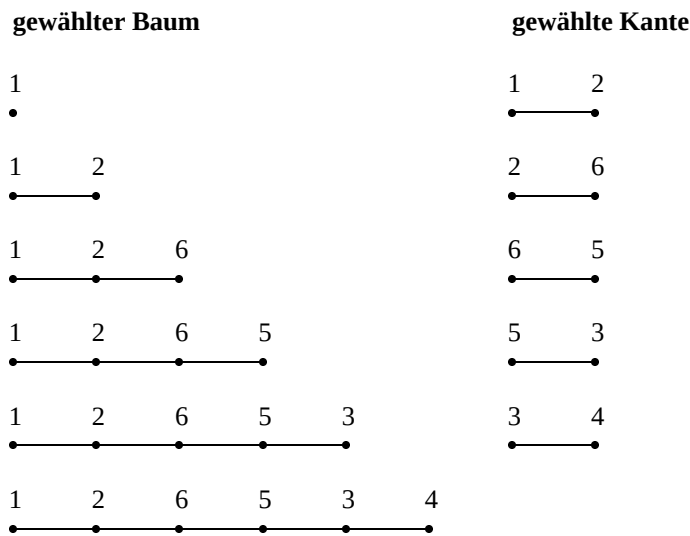


Abbildung 8.28

Da hierbei nur *ein* Baum wächst, benötigen wir im Unterschied zu Kruskals Algorithmus keine Union-find-Struktur; statt dessen genügt eine Priority Queue. Wie bei Dijkstras Algorithmus für kürzeste Wege hängt die Effizienz der Implementierung des Verfahrens ab von der Wahl einer Datenstruktur für die Priority Queue. Die beste Wahl ist hier der Fibonacci-Heap [1]. Dann unterscheidet sich der Algorithmus für minimale spannende Bäume von dem für kürzeste Wege nur dadurch, daß anstelle der Entfernung zum Anfangsknoten für die kürzesten Wege nunmehr die Entfernung zum nächsten Knoten im gewählten Baum verwaltet werden muß. Dies kann aber auf die gleiche Weise geschehen wie beim Algorithmus zum Finden kürzester Wege. Damit läßt sich dieser Algorithmus zum Finden eines minimalen spannenden Baumes für einen zusammenhängenden, ungerichteten, bewerteten Graphen $G = (V, E)$ so implementieren, daß er mit einer Laufzeit von $O(|E| + |V| \log |V|)$ auskommt. In [2] ist ein noch schnellerer Algorithmus beschrieben, bei dem mehrere Bäume ein wenig wachsen und dann zu Superknoten kollabieren; dasselbe Verfahren wird auf den so kondensierten Graphen angewandt, bis schließlich ein minimaler spannender Baum erreicht ist.

8.7 Flüsse in Netzwerken

Welchen Verkehrsfluß (in Fahrzeugen pro Minute) kann ich höchstens durch eine Stadt leiten, deren Straßennetz gegeben ist? Welche Wassermenge kann ich durch die Kanalisation höchstens abtransportieren? Solche und andere Flußprobleme in Netzwer-

ken sind in vielen Varianten und Verkleidungen ausgiebig untersucht worden. Obgleich schon 1962 ein inzwischen klassisches Buch zu diesem Thema  erschien, werden auch heute noch immer wieder neue und bessere Algorithmen für Flußprobleme gefunden. Wir betrachten hier das Problem, maximale Flüsse in Netzwerken zu finden, bei denen Pfeile Verbindungen repräsentieren, durch die Güter fließen können. Dabei hat jeder Pfeil nur eine beschränkte Kapazität; beispielsweise verträgt ein Straßenstück nur einen Durchsatz von 10 Fahrzeugen je Minute, oder ein Kanalisationsrohr verkraftet nicht mehr als 20 Liter pro Sekunde.

Sei im folgenden $G = (V, E)$ ein gerichteter Graph mit einer Kapazitätsfunktion $c : E \rightarrow \mathbb{R}^+$ (englisch: capacity) und zwei ausgezeichneten Knoten, einer *Quelle* q und einer *Senke* s . Unser Ziel ist es, einen maximalen Fluß von q nach s zu ermitteln. Ein Fluß durch einen Pfeil muß die Kapazitätsbeschränkung dieses Pfeils einhalten; an jedem Knoten muß der Fluß erhalten bleiben, also gleichviel hinein- wie herausfließen (außer an der Quelle und an der Senke). Wir definieren daher einen *Fluß* als eine Funktion $f : E \rightarrow \mathbb{R}_0^+$, wobei gilt:

- *Kapazitätsbeschränkung*: für alle $e \in E$ ist $f(e) \leq c(e)$;
- *Flußerhaltung*: für alle $v \in V - \{q, s\}$ ist $\sum_{(v', v) \in E} f((v', v)) - \sum_{(v, v'') \in E} f((v, v'')) = 0$.

Der Einfachheit halber wird oft angenommen, daß kein Pfeil in q mündet und kein Pfeil s verläßt; wir wollen hier im allgemeinen auf diese Annahme verzichten, aber unsere Beispiele manchmal so beschränken.

Betrachten wir das in Abbildung 8.29 gezeigte Beispiel. An jedem Pfeil e ist dort

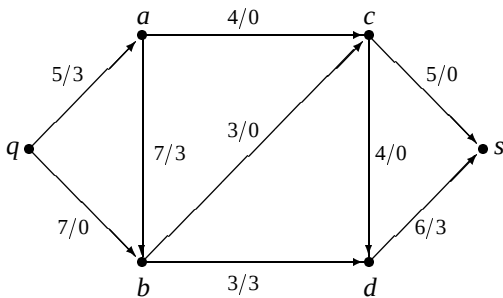


Abbildung 8.29

$c(e)/f(e)$ angegeben. Es fließt also gerade ein Fluß von Knoten q über Knoten a, b, d zu Knoten s . Der Wert $w(f)$ eines Flusses f ist die Summe der Flußwerte aller q verlassenden Pfeile, also $w(f) = \sum_{(q, v) \in E} f((q, v)) - \sum_{(v', q) \in E} f((v', q))$. In unserem Beispiel ist $w(f) = 3$. Ein *maximaler Fluß* in G ist ein Fluß f in G mit maximalem Wert $w(f)$ unter allen Flüssen in G . Für das Problem, einen maximalen Fluß in einem gegebenen

Digraphen zu ermitteln, sind im Laufe der Zeit zahlreiche, verschiedene Algorithmen vorgeschlagen worden. Wir werden im folgenden einige der wichtigsten vorstellen.

Überlegen wir uns aber zunächst, wie groß ein maximaler Fluß überhaupt sein kann. Es ist intuitiv plausibel, daß nicht mehr im Netzwerk fließen kann, als aus der Quelle herausfließt oder in die Senke hineinfließt. In unserem Beispiel verlassen höchstens 12 Einheiten die Quelle, und höchstens 11 fließen in die Senke. Aber nicht nur Quelle und Senke begrenzen den Wert eines maximalen Flusses, sondern jeder Schnitt durch den Graphen, der q von s trennt. Ein (q von s trennender) *Schnitt* ist eine Zerlegung der Knotenmenge V in zwei Teilmengen Q und S , so daß q zu Q und s zu S gehört. Die *Kapazität* $c(Q, S)$ eines Schnittes Q, S ist die Summe der Kapazitäten von Pfeilen, die von Q nach S führen, also $c(Q, S) = \sum_{v \in Q, v' \in S, (v, v') \in E} c((v, v'))$. Ein Schnitt mit kleinster Kapazität unter allen möglichen Schnitten heißt *minimaler Schnitt*. In dem in Abbildung 8.29 gezeigten Beispiel ist etwa $Q = \{q, b\}, S = \{a, c, d, s\}$ ein Schnitt; die Kapazität $c(Q, S)$ dieses Schnitts ist $c((q, a)) + c((b, c)) + c((b, d)) = 11$. Für einen Fluß f und einen Schnitt Q, S ist der (*Netto-*) *Fluß über den Schnitt* $f(Q, S) = \sum_{v \in Q, v' \in S, (v, v') \in E} f((v, v')) - \sum_{v \in Q, v' \in S, (v', v) \in E} f((v', v))$. In unserem Beispiel ist also der Fluß $f(\{q, b\}, \{a, c, d, s\}) = f((q, a)) + f((b, c)) + f((b, d)) - f((a, b)) = 3 + 0 + 3 - 3 = 3$. Daß dies gerade dem Wert des Flusses $w(f)$ entspricht, ist kein Zufall.

Ganz allgemein gilt für jeden Fluß f und jeden Schnitt Q, S , daß der Fluß $f(Q, S) = w(f)$ ist. Dies sieht man wie folgt ein. Nach Definition ist

$$f(Q, S) = \sum_{\substack{v \in Q, \\ v' \in S, \\ (v, v') \in E}} f((v, v')) - \sum_{\substack{v \in Q, \\ v' \in S, \\ (v', v) \in E}} f((v', v)).$$

Addieren wir zur rechten Seite dieser Gleichung

$$\sum_{\substack{v \in Q, \\ v' \in Q, \\ (v, v') \in E}} f((v, v')) - \sum_{\substack{v \in Q, \\ v' \in Q, \\ (v', v) \in E}} f((v', v)) + \sum_{\substack{v \in Q, \\ v' \in Q, \\ (v', v) \in E}} f((v', v)) - \sum_{\substack{v \in Q, \\ v' \in Q, \\ (v, v') \in E}} f((v, v')),$$

so können wir die Summanden neu zusammenfassen zu

$$f(Q, S) = \sum_{\substack{v \in Q, \\ v' \in V, \\ (v, v') \in E}} f((v, v')) - \sum_{\substack{v \in Q, \\ v' \in V, \\ (v', v) \in E}} f((v', v)) + \sum_{\substack{v \in Q, \\ v' \in Q, \\ (v', v) \in E}} f((v', v)) - \sum_{\substack{v \in Q, \\ v' \in Q, \\ (v, v') \in E}} f((v, v')).$$

Wegen der Flußerhaltung ergeben die ersten beiden Summanden zusammen gerade $w(f)$; die letzten beiden Summanden ergeben 0, und somit ist die Behauptung nachgewiesen. Für das in Abbildung 8.29 angegebene Beispiel kann man leicht überprüfen, daß der Fluß für jeden Schnitt 3 beträgt.

Wegen der Kapazitätsbeschränkung kann man sofort schließen, daß der Fluß über einen beliebigen Schnitt dessen Kapazität nicht übersteigen kann. Damit ist der Wert eines maximalen Flusses sicher nicht größer als die Kapazität eines minimalen Schnittes; wir werden noch sehen, daß in der Tat beide Werte gleich sind.

Maximaler Fluß durch zunehmende Wege

Für das in Abbildung 8.29 gezeigte Beispiel hat der Fluß seinen maximalen Wert offenbar noch nicht erreicht. Zwar können wir den Fluß entlang des Weges q, a, b, d, s nicht mehr erhöhen, weil Pfeil (b, d) bereits die maximal mögliche Menge transportiert. Aber es gibt noch andere Wege, bei denen die Kapazitäten nicht voll ausgenutzt sind. So lassen sich zum Beispiel entlang des Weges q, a, c, s zwei weitere Einheiten transportieren. Erhöhen wir außerdem den Fluß auf dem Weg q, b, c, s um 3 Einheiten, so erhalten wir die in Abbildung 8.30 gezeigte Situation.

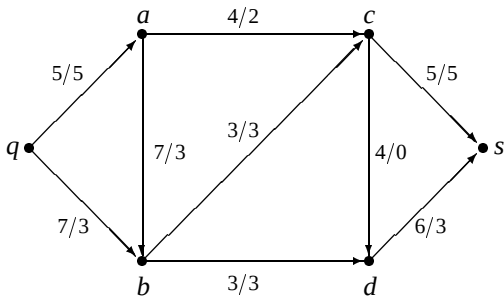


Abbildung 8.30

Jetzt ist auf jedem Weg von q nach s wenigstens ein Pfeil *gesättigt*, d.h., der Fluß auf diesem Pfeil entspricht gerade der Kapazität des Pfeils. Trotzdem ist der Wert des Flusses nur 8, obgleich die Kapazität des minimalen Schnitts $\{q, a, b\}, \{c, d, s\}$ 10 beträgt. Der Fluß ist also nicht maximal. Dies haben wir einer unglücklichen Entscheidung im Knoten a zu verdanken: Dort werden drei Flußeinheiten über Knoten b weitergeleitet, wodurch auf dem Pfeil (a, c) nur noch zwei Einheiten transportiert werden müssen. Im Knoten b ergibt sich aber ein Engpaß, weil von ihm aus nur sechs Einheiten weitergeleitet werden können. Es wäre also besser gewesen, zwei Einheiten vom Knoten a über den Knoten c weiterzuleiten und damit Platz zu schaffen für zwei Einheiten, die vom Knoten q über den Knoten b geleitet werden könnten. Von Knoten c aus könnten die zwei Einheiten über Knoten d zum Knoten s gelangen.

Wir können dieses Abändern von Flüssen in Wegen von q nach s ausdrücken, wenn wir nicht nur das Erhöhen eines Flusses entlang eines Pfeiles mit noch freier *Restkapazität* $rest(e) = c(e) - f(e)$ in Betracht ziehen, sondern auch das Verringern eines Flusses entlang eines Pfeiles, also gewissermaßen das Erhöhen eines Flusses entgegen der Pfeilrichtung. Einen Fluß $f(e)$ kann man natürlich höchstens um $f(e)$ Einheiten verringern; dann ergibt sich für $f(e)$ der Wert 0. In unserem Beispiel bedeutet dies gerade, daß wir den Weg q, b, a, c, d, s betrachten und feststellen, daß wir den Fluß durch Pfeil (q, b) um 4 erhöhen, durch (a, b) um 3 senken, durch (a, c) um 2 erhöhen, durch (c, d) um 4 erhöhen und durch (d, s) um 3 Einheiten erhöhen können. Also läßt sich der

Fluß um das Minimum dieser Werte, nämlich 2 erhöhen. Ein solcher Weg ohne Rücksicht auf die Pfeilrichtungen (ein *ungerichteter* Weg) von q nach s , auf dem man den Fluß erhöhen kann, wird *zunehmender Weg* genannt. Für jeden Pfeil e auf einem zunehmenden Weg, der in Pfeilrichtung durchlaufen wird (ein *Vorwärtspfeil*), ist $f(e) < c(e)$, also $rest(e) > 0$; für jeden Pfeil, der in Gegenrichtung durchlaufen wird (ein *Rückwärtspfeil*), gilt $f(e) > 0$. Der *Restgraph* zu einem Fluß f beschreibt gerade alle Flußvergrößerungsmöglichkeiten: Er enthält einen Pfeil e , wenn $rest(e) > 0$ gilt; er enthält den zu e entgegengesetzten Pfeil, wenn $f(e) > 0$ gilt.

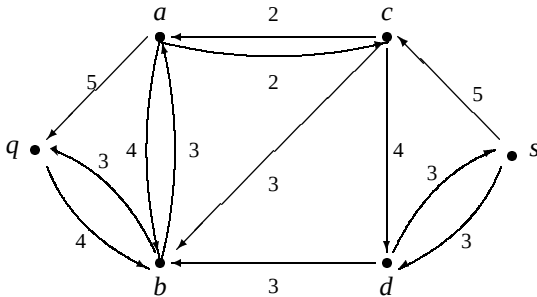


Abbildung 8.31

Abbildung 8.31 zeigt den Restgraphen zu dem in Abbildung 8.30 gezeigten Fluß. Jeder Weg im Restgraphen von q nach s ist ein zunehmender Weg für den gegebenen Fluß. In unserem Beispiel ist der einzige zunehmende Weg der einzige einfache Weg von q nach s im Restgraphen, also der Weg q, b, a, c, d, s . Nach der Flußvergrößerung um 2 Einheiten auf diesem Weg ergibt sich der in Abbildung 8.32 gezeigte Fluß; im zugehörigen Restgraphen führt kein Weg mehr von q nach s . Der Fluß hat den Wert 10, ist also maximal. Wir haben im Restgraphen nur solche Pfeile e eingezeichnet, für die $rest(e) > 0$ gilt, wobei $rest(e)$ genauer wie folgt definiert ist:

$$rest((v, v')) = \begin{cases} c((v, v')) - f((v, v')), & \text{falls } (v, v') \in E \\ f((v', v)) & \text{falls } (v', v) \in E. \end{cases}$$

Bereits 1956 wurde gezeigt , daß ein Fluß f genau dann maximal ist, wenn es für f keinen zunehmenden Weg gibt, und daß genau dann der Wert des Flusses f der Kapazität eines minimalen Schnitts entspricht. Dies sieht man wie folgt ein. Wenn es einen zunehmenden Weg für einen Fluß f gibt, dann können wir den Fluß entlang dieses Wegs vergrößern. Damit ist klar, daß es für einen maximalen Fluß f keinen zunehmenden Weg geben kann. Nehmen wir jetzt also an, daß es für f keinen zunehmenden Weg gibt. Sei X die Menge aller im Restgraphen von q aus erreichbaren Knoten, und sei $\bar{X} = V - X$. Weil es für f keinen zunehmenden Weg gibt, gehört q zu X und s zu $\bar{X}$. Also ist $X, \bar{X}$ ein Schnitt. Nach Definition gibt es im Restgraphen keinen Pfeil von

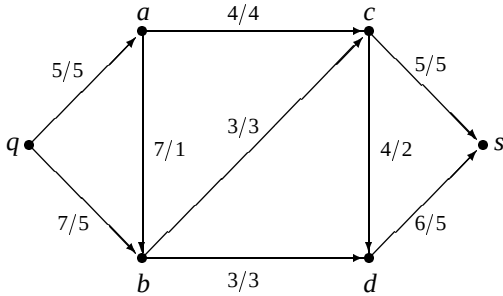


Abbildung 8.32

einem Knoten in X zu einem Knoten in $\overline{X}$. Also gilt $f(e) = c(e)$ für jeden Pfeil e im gegebenen Graphen G , der von einem Knoten in X zu einem Knoten in $\overline{X}$ führt. Damit ist $w(f) = c(X, \overline{X})$; der Wert des Flusses f entspricht also der Kapazität eines Schnitts. Der Wert eines jeden Flusses, also auch $w(f)$, ist durch die Kapazität c_{min} eines minimalen Schnitts beschränkt. Wegen $w(f) \leq c_{min}$ und $c_{min} \leq c(X, \overline{X})$ folgt mit $w(f) = c(X, \overline{X})$ auch $w(f) = c_{min} = c(X, \overline{X})$, d.h., $X, \overline{X}$ muß ein minimaler Schnitt und f ein maximaler Fluß sein.

Beliebige zunehmende Wege

Hieraus ergibt sich unmittelbar die in [1] vorgestellte Methode zur Konstruktion eines maximalen Flusses durch wiederholtes Einbeziehen zunehmender Wege:

```
Algorithmus Maximaler Fluß durch zunehmende Wege [1]
{berechnet zu einem Digraphen  $G = (V, E)$  mit Kapazität  $c: E \rightarrow \mathbb{R}^+$ 
  einen maximalen Fluß  $f: E \rightarrow \mathbb{R}_0^+$  für  $G$ }
begin
  1. {Initialisiere mit Nullfluß;}
     for all  $e \in E$  do
        $f(e) := 0$ ;
  2. {iterierte Flußvergrößerung;}
     while es gibt einen zunehmenden Weg  $p$  do
       begin
          $r := \min\{rest(e) \mid e \text{ liegt auf Weg } p \text{ im Restgraphen}\}$ ;
         erhöhe  $f$  entlang  $p$  um  $r$ 
       end
     end {Maximaler Fluß}
```

Hierbei ist es sinnvoll, neben der Kapazität c für jede Kante auch einen aktuellen Flußwert f zu speichern. Das Erhöhen des Flusses f entlang eines Weges p um einen Betrag r wird für Vorwärtspfeile durch Erhöhen von f um r und für Rückwärtspfeile durch Erniedrigen von f um r realisiert.

Genau genommen arbeitet der vorgestellte Algorithmus aber noch nicht einmal korrekt: Man kann sich überlegen, daß er für irrationale Kapazitäten nicht unbedingt terminieren muß und daß aufeinanderfolgende Flußwerte zwar konvergieren, aber nicht unbedingt zum Wert des maximalen Flusses. Beschränken wir jedoch die Kapazitäten auf ganze (oder rationale) Zahlen, so ist der vorgeschlagene Algorithmus korrekt. Bei ganzzahligen Kapazitäten ist auch ein maximaler Fluß ganzzahlig, und bei jedem Durchlauf der **while**-Schleife wird der gefundene Fluß wenigstens um 1 erhöht. Ein maximaler Fluß $f_{\max}$ wird also mit höchstens $w(f_{\max})$ Durchläufen der **while**-Schleife gefunden. Damit hängt aber die Laufzeit des Algorithmus nicht nur von der Anzahl der Knoten und Kanten des gegebenen Graphen ab. Abbildung 8.33 zeigt einen Beispielgraphen mit vier Knoten und fünf Kanten, bei dem der Algorithmus $2 \cdot c_1$ Durchläufe der **while**-Schleife benötigt, wenn abwechselnd die Wege q, a, b, s und q, b, a, s als zunehmende Wege gewählt werden.

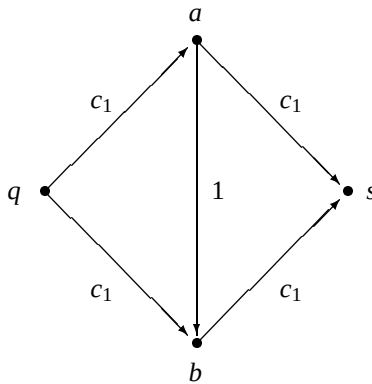


Abbildung 8.33

Kürzeste zunehmende Wege

Eine Laufzeitschranke, die lediglich von der Größe des Graphen abhängt, erhält man, wenn man als zunehmenden Weg immer einen mit möglichst wenigen Pfeilen wählt. Bestimmt man solche kürzesten zunehmenden Wege für die einzelnen Flußvergrößerungsschritte, so vergrößert sich die Anzahl der Pfeile auf einem kürzesten Weg von q nach s nach höchstens $|E|$. Schleifendurchläufen wenigstens um 1. Damit ist die Anzahl der erforderlichen Iterationen beschränkt durch $(|V| - 1) \cdot |E|$. Weil man einen einzelnen zunehmenden Weg mittels Breitensuche in $O(|E|)$ Schritten finden kann, ergibt sich eine Laufzeit von insgesamt $O(|V| \cdot |E|^2)$ Schritten für das Berechnen eines maximalen Flusses. Es geht aber noch schneller.

Alle kürzesten zunehmenden Wege

Wir betrachten wiederholt Flüsse, die sich nicht entlang eines Weges im gegebenen Graphen vergrößern lassen. Für den in Abbildung 8.29 gezeigten Graphen ist dies bei dem in Abbildung 8.30 gezeigten Fluß der Fall. Ein solcher Fluß enthält auf jedem Weg von q nach s einen gesättigten Pfeil; wir bezeichnen ihn als *blockierenden Fluß*. Abbildung 8.34 zeigt einen Fluß für den Graphen aus Abbildung 8.29; Abbildung 8.35 zeigt den dazugehörigen Restgraphen. Zur Bestimmung eines kürzesten zunehmenden Weges von q nach s sind nicht alle Pfeile im Restgraphen von Interesse. Vielmehr genügt es, für jeden von q aus erreichbaren Knoten v im Restgraphen einen kürzesten Weg von q nach v zu kennen.

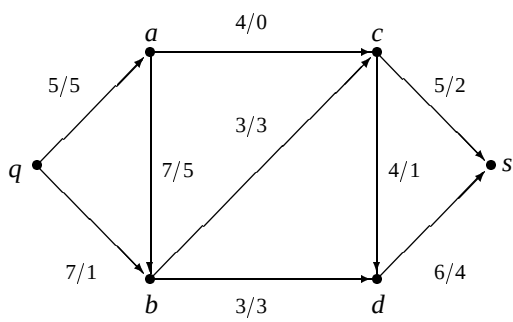


Abbildung 8.34

Für Knoten v bezeichnen wir die Länge (das ist die Anzahl der Pfeile) eines kürzesten Weges von q nach v im Restgraphen als *Niveau* von v .

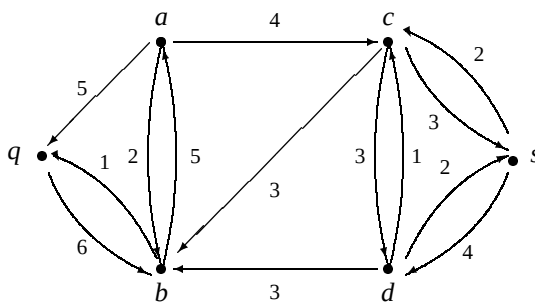


Abbildung 8.35

Für einen Fluß f ist der *Niveaugraph* derjenige Teilgraph des Restgraphen, der nur die von q aus erreichbaren Knoten enthält und nur solche Pfeile, die auf einem kürzesten Weg liegen. Ein Pfeil (v, v') des Restgraphen gehört also genau dann zum Niveaugraphen, wenn $\text{Niveau}(v') = \text{Niveau}(v) + 1$ gilt. Abbildung 8.36 zeigt den Niveaugraphen zu dem in Abbildung 8.35 gezeigten Fluß.

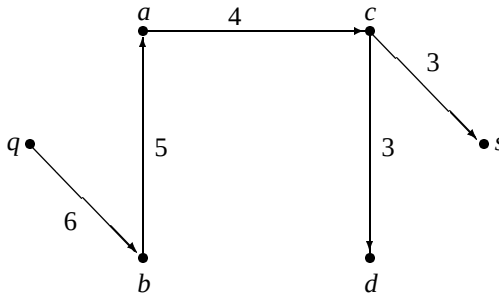


Abbildung 8.36

Der Niveaugraph enthält jeden kürzesten vergrößernden Weg, aber nicht unbedingt jeden vergrößernden Weg. Mit einer Breitensuche kann der Niveaugraph in Zeit $O(|E|)$ konstruiert werden. Damit ergibt sich die folgende Variante des Schritts 2 zur iterierten Flußvergrößerung im Algorithmus *Maximaler Fluß durch zunehmende Wege*:

2. $\{\text{iterierte Flußvergrößerung nach Dinic}\}$
while s gehört zum Niveaugraphen für f **do**
 begin
 $f_b :=$ ein blockierender Fluß im Niveaugraphen für f ;
 $f := f \oplus f_b$
 end

Dabei bezeichnet $\oplus$ das bereits erläuterte Addieren zweier Flüsse unter Berücksichtigung der Pfeilrichtung. Für den in Abbildung 8.36 gezeigten Niveaugraphen ist ein blockierender Fluß der Fluß der Stärke 3 entlang des Weges q, b, a, c, s . Die Addition dieses Flusses zu dem in Abbildung 8.34 gezeigten ergibt den in Abbildung 8.37 gezeigten Fluß. Abbildungen 8.38 bis 8.40 setzen das Beispiel bis zu einem maximalen Fluß und einem Niveaugraphen fort, der s nicht enthält.

Die Anzahl der im Verlauf der Berechnung erforderlichen iterierten Flußvergrößerungen ist vergleichsweise gering. Weil bei jeder Flußvergrößerung ein blockierender Fluß im Niveaugraphen zum aktuellen Fluß hinzugefügt wird, wächst das Niveau der Senke s bei jeder Iteration wenigstens um 1, sofern s von q aus im Niveaugraphen überhaupt erreichbar bleibt. Bei jeder Iteration werden also gleichzeitig alle kürzesten Wege zu einer Flußvergrößerung herangezogen. Damit berechnet der Algorithmus mit

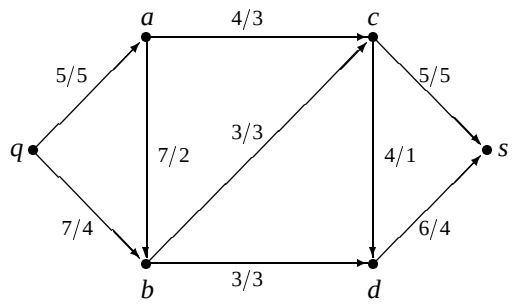


Abbildung 8.37

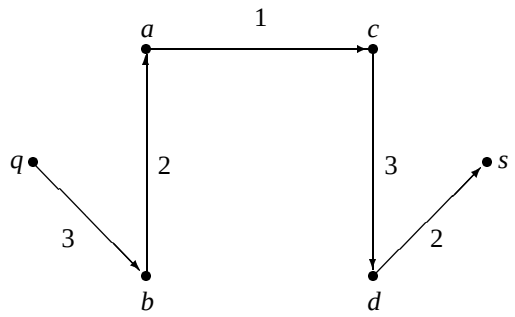


Abbildung 8.38

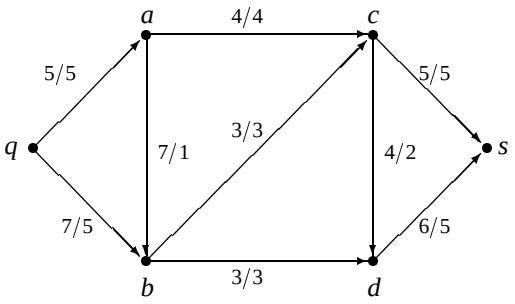


Abbildung 8.39

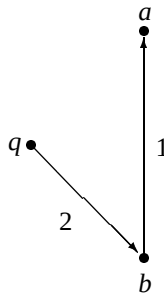


Abbildung 8.40

iterierter Flußvergrößerung nach  einen maximalen Fluß mit höchstens $|V| - 1$ Iterationen.

In speziellen Fällen kommt dieser Algorithmus sogar mit weniger Iterationen aus. Man kann sich überlegen , daß für ein Netzwerk mit ganzzahligen Kapazitäten, in dem außer der Quelle und der Senke jeder Knoten genau einen einmündenden Pfeil mit Kapazität 1 (und beliebig viele ausgehende Pfeile) oder einen ausgehenden Pfeil mit Kapazität 1 (und beliebig viele einmündende Pfeile) hat, $2 \left\lceil \sqrt{|V| - 2} \right\rceil$ Iterationen genügen. Wir werden dieses spezielle Ergebnis im nächsten Abschnitt zu einer Laufzeitabschätzung einsetzen.

Wir müssen uns jetzt noch überlegen, wie man einen blockierenden Fluß schnell findet. Beim einfachsten Verfahren  wählt man einen Weg von q nach s und erhöht auf diesem Weg den Fluß so, daß einer der Pfeile gesättigt wird. Dann entfernt man alle gesättigten Pfeile. Dies wird solange wiederholt, bis s nicht mehr von q aus erreichbar ist. Sobald dies der Fall ist, ist auf jedem Weg von q nach s ein Pfeil gesättigt, also ein blockierender Fluß erreicht.

Das Finden eines Weges von q nach s kann man als Tiefensuche organisieren. Inspizierte Pfeile, die schließlich nicht zu einem Weg zu s gehören, werden gelöscht. Wenn man einen Pfeil betrachtet hat, so gehört dieser also entweder zu einem Weg von q nach s oder er wird gelöscht. Für einen gefundenen Weg, der höchstens aus $|V| - 1$ Pfeilen bestehen kann, wird der Wert der Flußvergrößerung als kleinste Restkapazität von Pfeilen auf diesem Weg ermittelt. Beim Durchführen der Flußvergrößerung müssen alle Restkapazitäten von Pfeilen auf dem gefundenen Weg angepaßt und wenigstens ein Pfeil entfernt werden. Weil bei jeder Flußvergrößerung wenigstens ein Pfeil aus dem verbleibenden Graphen entfernt wird, entsteht nach höchstens $|E|$ Flußvergrößerungen ein blockierender Fluß. Da insgesamt höchstens jede Kante einmal gelöscht wird und jede Flußvergrößerung in $O(|V|)$ Schritten durchgeführt werden kann, findet der Algorithmus von Dinic  einen blockierenden Fluß in höchstens $O(|V| \cdot |E|)$ Schritten und damit einen maximalen Fluß in höchstens $O(|V|^2 \cdot |E|)$ Schritten. Im oben erwähnten

Spezialfall [1] findet der Algorithmus von Dinic [2] einen blockierenden Fluß in Zeit $O(|E|)$ und einen maximalen Fluß in Zeit $O(\sqrt{|V|} \cdot |E|)$.

In letzter Zeit sind einige weitere Methoden vorgeschlagen worden, einen blockierenden Fluß zu berechnen. Ein Verfahren, bei dem man einen Knoten nach dem anderen sättigt — und nicht, wie bei Dinic, einen Pfeil nach dem anderen — ist in [3] erstmals vorgestellt worden. Später wurde diese Methode in [4] vereinfacht. Man kann hierbei einen Knoten in Zeit $O(|V|)$ sättigen; die Konstruktion eines blockierenden Flusses kostet also nur noch $O(|V|^2)$ Schritte, und ein maximaler Fluß kann in $O(|V|^3)$ Schritten ermittelt werden.

Eine andere Realisierung der Grundidee, Knoten zu sättigen, ist in [5] vorgeschlagen worden. Hier merkt man sich für jeden Knoten v den maximal zusätzlich noch möglichen *Durchsatz* durch Knoten v . So kann man etwa im Beispiel der Abbildung 8.34 den Durchsatz nur für die Knoten c und d erhöhen, weil bei Knoten a alle einmündenden Pfeile und bei Knoten b alle ausgehenden Pfeile gesättigt sind. Der Durchsatz bei Knoten c kann um 4 Einheiten erhöht werden, weil sowohl vier zusätzliche Einheiten von a nach c als auch von c weg, nach d und s , fließen können, wenn man den Rest des Netzwerks außer Betracht läßt. Einen blockierenden Fluß findet man dann, indem man wiederholt über einen Knoten mit kleinstem maximal möglichem zusätzlichem Durchsatz gerade so viele Einheiten von der Quelle q zur Senke s schickt, wie dieser Durchsatz angibt. Bei geeigneter Implementierung kommt dieses Verfahren ebenfalls mit $O(|V|^3)$ Schritten aus.

In anderen Verfahren [6, 7] wurde versucht, einen Pfeil nach dem anderen zu sättigen und die Laufzeit des Verfahrens durch Verwendung einer geeigneten Datenstruktur zu reduzieren. Der schnellste dieser Philosophie folgende Algorithmus [8] verwendet eine Datenstruktur für dynamische Bäume. Jeder Baumknoten speichert eine reelle Zahl, die *Kosten* des Knotens. Die vorgeschlagene Datenstruktur bietet für eine Menge knotendisjunkter Bäume die folgenden Operationen an:

- *maketree*(v) : stellt einen neuen Baum her, dessen einziger Knoten v mit Kosten 0 ist.
- *findroot*(v) : liefert die Wurzel des Baumes, der Knoten v enthält.
- *findcost*(v) : liefert den Knoten v' und seine Kosten c , wobei c das Minimum der Kosten aller Knoten auf dem Pfad von v zur Wurzel *findroot*(v) ist und v' auf diesem Pfad der am nächsten bei der Wurzel liegende Knoten mit Kosten c ist.
- *addcost*(v, c) : addiere c zu den Kosten jedes Knotens auf dem Pfad von v zur Wurzel *findroot*(v).
- *link*(v, v') : verbinde die beiden Bäume mit Knoten v und v' durch einen Pfeil (v', v). Hier wird angenommen, daß v die Wurzel des einen Baumes ist, und daß v und v' nicht im selben Baum liegen.
- *cut*(v) : teile den Baum, der Knoten v enthält, durch Entfernen der Kante, die v mit dem Vater von v verbindet, in zwei Bäume. Hier wird angenommen, daß v keine Wurzel ist.

Um einen blockierenden Fluß zu finden, speichert man für jeden Knoten einen inzidenten Pfeil, auf dem man möglicherweise den Fluß vergrößern kann. Diese Pfeile zusammen ergeben im Graphen eine Menge von Bäumen. Für $|V|$ insgesamt verwaltete Knoten kann jede der sechs angebotenen dynamischen Baumoperationen in einer amortisierten Laufzeit von $O(\log |V|)$ ausgeführt werden, wobei sich die Folge der aus-

zuführenden Operationen durch eine Umformulierung von Dinics Algorithmus ergibt. Mit $O(|E|)$ Baumoperationen kostet das Berechnen eines blockierenden Flusses dann $O(|E| \log |V|)$ Schritte; ein maximaler Fluß kann also in Zeit $O(|V| \cdot |E| \log |V|)$ berechnet werden.

8.8 Zuordnungsprobleme

Zuordnungsprobleme, bei denen es um eine insgesamt bestmögliche Bildung von Paaren von Elementen über einer Grundmenge geht, lassen sich oft günstig durch Graphen repräsentieren. Die Elemente der Grundmenge sind die Knoten des Graphen und die Kanten beschreiben alle möglichen Paarbildungen. Repräsentiert beispielsweise jeder Knoten einen Teilnehmer an einer Gruppenreise und jede Kante die Bereitschaft der beiden Teilnehmer, in einem gemeinsamen Doppelzimmer zu übernachten, so kann man sich fragen, wieviele Zimmer unter dieser Voraussetzung mindestens benötigt werden. Weil jeder Teilnehmer nur in *einem* Doppelzimmer übernachten soll, ist dies im Graphen die Frage nach einer größtmöglichen Teilmenge der Kanten, bei der jeder Knoten des Graphen mit höchstens einer Kante inzidiert. In dem in Abbildung 8.41 gezeigten Fall sieht man, daß für die sechs Reiset Teilnehmer drei Doppelzimmer genügen.

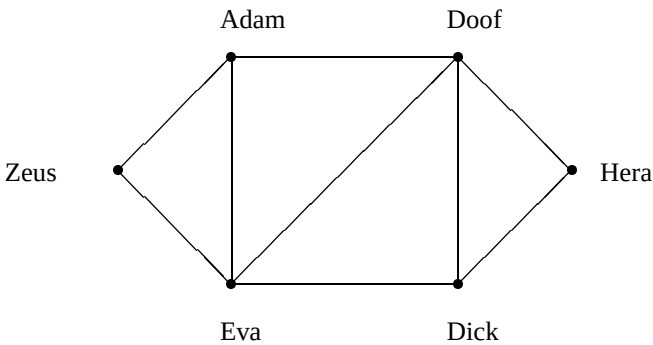


Abbildung 8.41

Für einen ungerichteten Graphen $G = (V, E)$ ist eine *Zuordnung* Z (englisch: *matching*) eine Teilmenge der Kanten von G , so daß keine zwei Kanten in Z denselben Endknoten haben. Die Anzahl $|Z|$ der Kanten in Z heißt *Größe* der Zuordnung. Ein Knoten ist bezüglich einer Zuordnung Z *alleine* (englisch: *unmatched*), wenn er nicht Endknoten einer Kante in Z ist. Z ist eine *perfekte Zuordnung* (englisch: *perfect matching*), wenn mit Z kein Knoten alleine bleibt. In dem in Abbildung 8.41 gezeigten Beispiel gibt es gleich mehrere perfekte Zuordnungen, darunter beispielsweise $\{(Zeus,$

Eva), (Adam, Doof), (Dick, Hera)). Da es eine perfekte Zuordnung für einen gegebenen Graphen nicht unbedingt geben muß, interessiert man sich für bestmögliche Zuordnungen. Eine Zuordnung Z für einen Graphen $G = (V, E)$ ist *nicht erweiterbar* (englisch: *maximal*), wenn es keine Kante $e \in E$ gibt, die man noch zu Z hinzunehmen könnte, für die also $Z \cup \{e\}$ eine Zuordnung für G bleibt. In unserem Beispiel ist etwa die Zuordnung $\{(Adam, Eva), (Dick, Doof)\}$ nicht erweiterbar; trotzdem gibt es im Graphen eine Zuordnung, die mehr Kanten enthält. Eine Zuordnung Z mit maximaler Größe $|Z|$ ist eine *maximale Zuordnung* (englisch: *maximum matching*).

Beim Versuch, die Realität etwas genauer zu modellieren, wird man im Beispiel der Abbildung 8.41 vielleicht feststellen, daß Adam zwar bereit ist, ein Doppelzimmer mit Zeus, Eva oder Doof zu teilen, daß ihm aber nicht jede dieser Möglichkeiten gleich lieb ist. Ordnet man nun jeder Kante im Graphen eine Maßzahl für die Zufriedenheit der beiden Reiseteilnehmer bei einer gemeinsamen Übernachtung zu, so ergibt sich etwa die in Abbildung 8.42 gezeigte Situation. Hier können wir nach einer Zuordnung fragen, die die Summe der Zufriedenheiten maximiert. Das ist offenbar die Zuordnung $\{(Adam, Eva), (Dick, Doof)\}$, auch wenn dabei Zeus und Hera alleine bleiben.

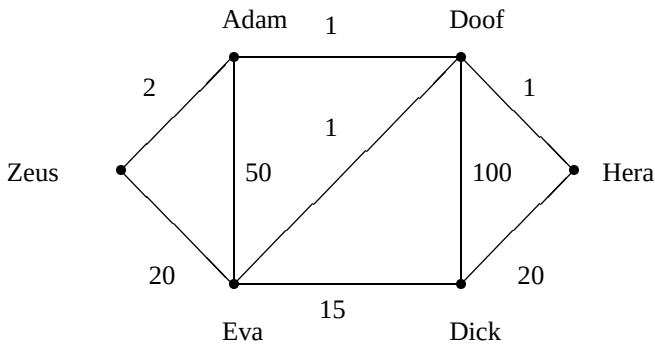


Abbildung 8.42

Für einen ungerichteten, bewerteten Graphen $G = (V, E)$ mit Kantenbewertung $w : E \rightarrow \mathbb{R}$ ist das *Gewicht* (englisch: *weight*) einer Zuordnung Z die Summe der Gewichte der Kanten in Z . Wir interessieren uns hier für eine *maximale gewichtete Zuordnung* (englisch: *maximum weight matching*), also eine Zuordnung mit maximalem m Gewicht. Wenn beispielsweise in einer Firma mit k Mitarbeitern $m_1, \dots, m_k$ die k Tätigkeiten $t_1, \dots, t_k$ auszuführen sind und eine Maßzahl $w(m_i, t_j)$ für die Eignung des Mitarbeiters m_i für Tätigkeit t_j bekannt ist, sofern Mitarbeiter m_i Tätigkeit t_j überhaupt ausführen kann, so kann eine maximale gewichtete Zuordnung von Mitarbeitern und Tätigkeiten erwünscht sein. Abbildung 8.43 zeigt eine Situation, in der die Zuordnung $\{(m_1, t_1), (m_2, t_3), (m_3, t_2), (m_4, t_5), (m_5, t_4), (m_6, t_6)\}$ maximales Gewicht hat.

Wie in diesem Beispiel lassen sich auch in vielen anderen Fällen die Knoten des Graphen so in zwei Gruppen teilen, daß es nur Kanten zwischen Knoten verschiedener Gruppen gibt. In unserem Beispiel ist es etwa unsinnig, von der Eignung eines

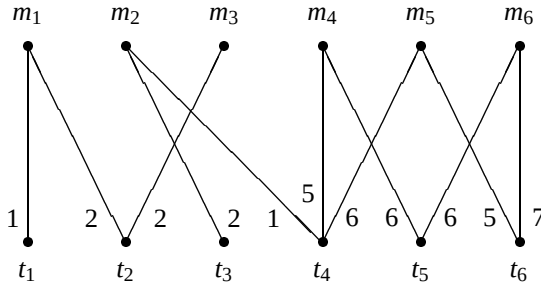


Abbildung 8.43

Mitarbeiters für einen anderen Mitarbeiter oder einer Tätigkeit für eine andere Tätigkeit zu reden. Das Entsprechende gilt beispielsweise, wenn es um die Zuordnung von Studienanfängern zu Studienplätzen oder von Männern zu Frauen bei einem Eheanbahnungsinstitut geht. Weil man in solchen Situationen eine maximale Zuordnung oder eine maximale gewichtete Zuordnung schneller und einfacher finden kann, wollen wir diese separat betrachten. Wir nennen einen Graphen $G = (V, E)$ *bipartit* (englisch: *bipartite*), wenn sich die Knotenmenge V so in zwei Teilmengen X und Y zerlegen läßt (also $V = X \cup Y$ und $X \cap Y = \emptyset$ gilt), daß $E \subseteq X \times Y$, also keine Kante zwei Knoten in X oder zwei Knoten in Y verbindet.

8.8.1 Maximale Zuordnungen in bipartiten Graphen

Betrachten wir zunächst bipartite Graphen ohne Gewichtungsfunktion. Abbildung 8.44 zeigt einen solchen Graphen $G = (X \cup Y, E)$ mit $X = \{x_1, \dots, x_6\}$ und $Y = \{y_1, \dots, y_6\}$ sowie eine Zuordnung, ausgedrückt durch dicker gezeichnete Kanten.

Man sieht leicht, daß diese Zuordnung nicht maximal ist: Eine Zuordnung mit mehr Kanten erhält man beispielsweise, indem man die Paare (x_1, y_1) und (x_3, y_2) anstatt (x_1, y_2) in die Zuordnung aufnimmt. Um aus einer gegebenen Zuordnung eine maximale Zuordnung zu ermitteln, kann es also nötig sein, eine für die Zuordnung bereits gewählte Kante wieder aus der Zuordnung zu entfernen. Das Entfernen von Kanten aus einer bereits gefundenen Zuordnung läßt sich aber nicht auf einzelne Kanten beschränken. So kann man die in Abbildung 8.44 dargestellte Zuordnung nicht vergrößern, indem man eine einzelne der Kanten (x_2, y_4) oder (x_4, y_5) entfernt und danach möglichst viele Kanten zur Zuordnung hinzunimmt, aber man kann die Zuordnung vergrößern, wenn man diese beiden Kanten aus der Zuordnung entfernt und statt dessen die Kanten (x_2, y_3) , (x_4, y_4) und (x_6, y_5) in die Zuordnung aufnimmt. Dies erinnert an das Konzept der *zunehmenden Wege* bei den im Abschnitt 8.7 vorgestellten Algorithmen zum Finden maximaler Flüsse.

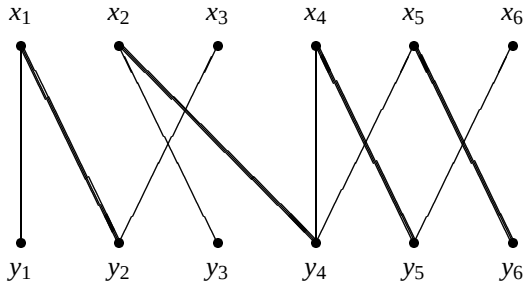


Abbildung 8.44

In der Tat kann man das Zuordnungsproblem für bipartite Graphen als Flußproblem formulieren. Dazu statten wir die Knotenmenge mit zwei zusätzlichen Knoten aus, einer Quelle q und einer Senke s . Jede Kante (x_i, y_j) des Graphen $G = (X \cup Y, E)$ wird im Graphen $G' = (X \cup Y \cup \{q, s\}, E')$ zu einem Pfeil von x_i nach y_j . Außerdem gibt es von q einen Pfeil zu jedem Knoten $x_i \in X$ und von jedem Knoten $y_j \in Y$ einen Pfeil nach s . Es ist also $E' = E \cup \{(q, x) \mid x \in X\} \cup \{(y, s) \mid y \in Y\}$, wobei die Kanten aus E wie beschrieben zu Pfeilen werden.

Abbildung 8.45 zeigt den zum Graphen G in Abbildung 8.44 gehörenden Flußgraphen G' und den Fluß für die dort gezeigte Zuordnung. Als Kapazitätsfunktion wählen wir hierbei $c : E' \rightarrow \{1\}$. Man sieht in diesem Beispiel sofort, daß der Ablösung von (x_1, y_2) in der dargestellten Zuordnung durch (x_1, y_1) und (x_3, y_2) ein zunehmender Weg von q nach s entspricht, nämlich der Weg q, x_3, y_2, x_1, y_1, s . Jedem Fluß f in G' entspricht eine Zuordnung $Z = \{(x_i, y_j) \mid f((x_i, y_j)) = 1\}$ in G , wobei $|Z| = w(f)$ gilt. Ebenso entspricht jede Zuordnung Z in G durch Hinzunahme der Pfeile (q, x) und (y, s) für alle $(x, y) \in Z$ einem Fluß f in G' , für den $|Z| = w(f)$ gilt. Eine maximale Zuordnung in G entspricht also einem maximalen Fluß in G' . Somit können wir im bipartiten Graphen G eine maximale Zuordnung berechnen, indem wir in G' einen maximalen Fluß bestimmen. Dies ist, wie wir in Abschnitt 8.7 bereits gesehen haben, für Graphen der speziellen Art von G' in Zeit $O(\sqrt{|V|} \cdot |E|)$ möglich.

Wir können das Konzept zunehmender Wege in G' in ein entsprechendes Konzept für G übertragen. Dazu genügt die Feststellung, daß auf einem zunehmenden Weg in G' jeder Vorwärtspfeil e den aktuellen Fluß $f(e) = 0$ und jeder Rückwärtspfeil e' den aktuellen Fluß $f(e') = 1$ transportiert. Einem zunehmenden Weg $q, x_i, \dots, y_j, s$ in G' entspricht in G ein Weg $x_i, \dots, y_j$. Weil dieser Weg in G' mit einem Vorwärtspfeil beginnt und mit einem Vorwärtspfeil endet und sich Vorwärtspfeile und Rückwärtspfeile stets abwechseln, ist die Anzahl der Vorwärtspfeile auf diesem Weg um 1 größer als die Anzahl der Rückwärtspfeile. So enthält im Beispiel der Abbildung 8.45 der Weg $x_6, y_5, x_4, y_4, x_2, y_3$ die Vorwärtspfeile (x_6, y_5) , (x_4, y_4) und (x_2, y_3) und die Rückwärtspfeile (x_4, y_5) und (x_2, y_4) . Einem solchen Weg in G' entspricht in G ein Weg, der abwechselnd aus Kanten besteht, die zur Zuordnung gehören bzw. nicht zur Zuordnung

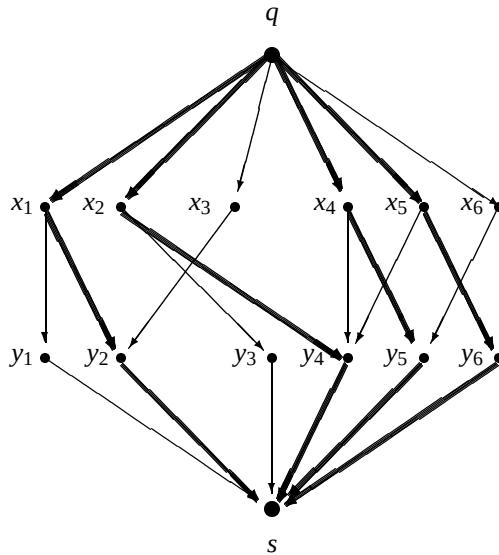


Abbildung 8.45

gehören. Solche Wege spielen bei Zuordnungen die Rolle, die zunehmende Wege bei Flüssen spielen.

Für eine gegebene Zuordnung Z nennen wir jede für die Zuordnung verwendete Kante $e \in Z$ *gebunden*; jede Kante $e' \in E - Z$ ist *frei*. Jeder Knoten, der mit einer gebundenen Kante inzidiert, ist ein *gebundener Knoten*, jeder andere Knoten ist *frei*. Ein Weg in G , dessen Kanten abwechselnd gebunden und frei sind, heißt *alternierender Weg*. Die *Länge* eines alternierenden Wegs ist die Anzahl der Kanten auf diesem Weg. Natürlich kann nicht jeder alternierende Weg zur Vergrößerung einer Zuordnung benützt werden. Dies geht nur dann, wenn die beiden Knoten an den beiden Enden des Wegs frei sind. Ein alternierender Weg mit zwei freien Knoten an den beiden Enden heißt deshalb *vergrößernd*. So sind im Beispiel der Abbildung 8.44 die Wege x_6, y_5 und x_2, y_4, x_5, y_6 zwar alternierend, aber nicht vergrößernd; der alternierende Weg $y_3, x_2, y_4, x_4, y_5, x_6$ ist dagegen vergrößernd. Aus einer Zuordnung, die einen vergrößernden Weg besitzt, kann man offensichtlich eine größere Zuordnung gewinnen, indem man entlang des vergrößernden Weges jede freie Kante zu einer gebundenen und jede gebundene zu einer freien Kante macht. Im Beispiel der Abbildung 8.44 kann man also die Kanten (x_2, y_3) , (x_4, y_4) und (x_6, y_5) zu gebundenen Kanten und die Kanten (x_2, y_4) und (x_4, y_5) zu freien Kanten machen und somit die Größe der gezeigten Zuordnung um 1 erhöhen. Das Konzept vergrößernder Wege kann man auch in allgemeinen, also nicht bipartiten, Graphen einsetzen.

8.8.2 Maximale Zuordnungen im allgemeinen Fall

Besitzt eine Zuordnung Z in einem Graphen $G = (V,E)$ einen vergrößerten Weg, so ist Z nicht von maximaler Größe. In dem in Abbildung 8.41 gezeigten Beispiel besitzt die Zuordnung $\{(Adam, Eva), (Dick, Doof)\}$ gleich mehrere vergrößerte Wege, darunter den Weg Zeus, Eva, Adam, Doof, Dick, Hera. Macht man auf diesem Weg alle gebundenen Kanten zu freien Kanten und alle freien Kanten zu gebundenen Kanten, so erhält man die vergrößerte Zuordnung $\{(Zeus, Eva), (Adam, Doof), (Dick, Hera)\}$. Im gezeigten Beispiel ist dies sogar eine maximale Zuordnung.

Daß man mit vergrößerten Wegen schließlich auch wirklich eine maximale Zuordnung erreicht, zeigt folgende Überlegung. Sei Z eine beliebige Zuordnung und Z_{max} eine größte Zuordnung für einen gegebenen Graphen; sei $k = |Z_{max}| - |Z|$ der Unterschied in der Größe beider Zuordnungen. Für den in Abbildung 8.46 gezeigten Graphen ist beispielsweise $Z_{max} = \{(1, 2), (3, 4), (5, 8), (6, 7), (9, 12), (10, 11)\}$.

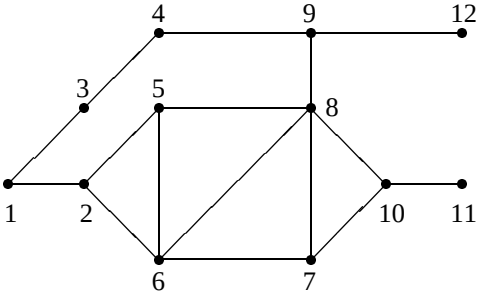


Abbildung 8.46

Für $Z = \{(4,9), (5,6), (7,8), (10,11)\}$ ergibt sich $k = |Z_{max}| - |Z| = 6 - 4 = 2$. Betrachten wir nun die symmetrische Differenz Z_{sym} von Z_{max} und Z , also $Z_{sym} = (Z_{max} - Z) \cup (Z - Z_{max})$. Im gezeigten Beispiel ergibt sich $Z_{sym} = \{(1, 2), (3, 4), (4, 9), (5, 6), (5, 8), (6, 7), (7, 8), (9, 12)\}$. Jeder Knoten des Graphen inzidiert mit höchstens zwei Kanten in Z_{sym} , nämlich höchstens einer von Z und einer von Z_{max} . In unserem Beispiel inzidieren gerade die Knoten 4, 5, 6, 7, 8 und 9 mit jeweils zwei Kanten. Der durch Z_{sym} induzierte Teilgraph von G kann keinen Zyklus ungerader Länge enthalten, weil jede Kante des Zyklus aus Z_{max} oder aus Z kommen muß und sich im Zyklus Kanten von Z_{max} mit Kanten von Z abwechseln müssen. Der durch Z_{sym} induzierte Teilgraph kann also nur Zyklen gerader Länge und natürlich Wege beliebiger Länge enthalten. Auf jedem Weg und in jedem Zyklus müssen die Kanten bezüglich Z alternieren, d.h. abwechselnd gebunden und frei sein; dasselbe gilt natürlich für Z_{max} .

Weil Z_{max} k Kanten mehr enthält als Z , und in Z_{sym} alle Kanten von $Z_{max} \cup Z$ außer den gemeinsamen Kanten enthalten sind, ist in Z_{sym} die Anzahl der aus Z_{max} stammenden Kanten um k höher als die Anzahl der aus Z stammenden Kanten. In unserem Beispiel stammen fünf der acht Kanten in Z_{sym} aus Z_{max} , das sind $k = 2$ Kanten mehr als aus Z .

Da jeder Zyklus in Z_{sym} genauso viele Kanten aus Z_{max} wie aus Z enthält, müssen auf Wegen (ohne Zyklen) in Z_{sym} k Kanten mehr aus Z_{max} stammen als aus Z . Daher muß es in Z_{sym} wenigstens k Wege geben, die mit einer Kante aus Z_{max} beginnen und mit einer solchen enden, und auf denen Kanten aus Z_{max} und aus Z alternieren. In unserem Beispiel gibt es zwei solche Wege, nämlich den Weg 1, 2 und den Weg 3, 4, 9, 12. Weil Z_{max} eine Zuordnung ist, können solche Wege keine gemeinsamen Knoten haben. All diese alternierenden Wege sind also knotendisjunkt und vergrößernd für Z , weil beide Endknoten bezüglich Z frei sind. Weil Z_{max} und Z Zuordnungen sind, ist die Summe der Längen aller solchen Wege durch die Anzahl der Knoten des Graphen beschränkt. Bei wenigstens k knotendisjunkten Wegen hat also wenigstens ein solcher Weg höchstens die Länge $|V|/k - 1$.

Wir können also jetzt eine beliebige, aber noch nicht maximale Zuordnung vergrößern, indem wir vergrößernde Wege finden und die Zuordnung entsprechend anpassen. Bei bipartiten Graphen kann man für eine gegebene Zuordnung Z einen vergrößernden Weg finden, indem man mit der Suche bei einem freien Knoten beginnt und entlang eines bezüglich Z alternierenden Weges fortschreitet. Sobald man wieder bei einem freien Knoten angekommen ist, ist ein vergrößernder Weg gefunden. Zu einem freien Startknoten kann man einen entsprechenden alternierenden Baum mit Hilfe einer Breitensuche ermitteln. Abbildung 8.47 zeigt einen alternierenden Breitensuchbaum für die in Abbildung 8.44 gezeigte Zuordnung und den Startknoten y_3 der Breitensuche.

In allgemeinen Graphen kann man mit einer solch einfachen Breitensuche vergrößernde Wege nicht unbedingt finden. Betrachten wir als Beispiel den in Abbildung 8.46 gezeigten Graphen und die Zuordnung $Z = \{(6, 7), (8, 10)\}$ und versuchen wir nun, vom freien Knoten 2 aus mit Hilfe eines alternierenden Baums einen vergrößernden Weg zu finden. Wenn wir den alternierenden Baum auf einen Teilgraphen beschränken, so hat er beispielsweise die in Abbildung 8.48 gezeigte Gestalt. Die Breitensuche sorgt dafür, daß Knoten 10 besucht wird, bevor die Nachfolger von Knoten 8 im alternierenden Baum in Betracht gezogen werden. Wenn jeder Knoten, wie bei der Breitensuche üblich, nur einmal besucht werden darf, so verhindert das Finden des alternierenden Weges 2, 6, 7, 10, der nicht mit einem freien Knoten endet, daß der alternierende Weg 2, 6, 7, 8, 10, 11 gefunden wird, obwohl dieser mit einem freien Knoten enden würde. Die reine Breitensuche ist also hier nicht in der Lage, vergrößernde Wege auch wirklich zu finden.

Die Ursache des Problems liegt darin, daß ein und derselbe Knoten auf mehreren verschiedenen alternierenden Wegen in gerader und in ungerader Entfernung vom Startknoten auftreten kann. So tritt in unserem Beispiel Knoten 10 auf dem alternierenden Weg 2, 6, 7, 10 in ungerader Entfernung vom Startknoten 2 auf, während er auf dem alternierenden Weg 2, 6, 7, 8, 10 in gerader Entfernung vom Startknoten auftritt. Man kann aber nicht einfach in einer Abänderung der reinen Breitensuche das zweimalige Besuchen eines jeden Knotens erlauben, nämlich je einmal für die gerade und einmal für die ungerade Entfernung vom Startknoten, denn dann können auch Knotenfolgen gefunden werden, die keinen vergrößernden Weg beschreiben. Eine entsprechend modifizierte Breitensuche kann für den in Abbildung 8.46 gezeigten Graphen und die Zuordnung $Z = \{(6, 7), (8, 10)\}$ für Startknoten 2 die Knotenfolge 2, 6, 7, 8, 10, 7, 6, 5 liefern, obwohl diese Knotenfolge keinen vergrößernden Weg beschreibt.

Man kann sich überlegen, daß das Finden eines vergrößernden Weges von einem freien Knoten v aus nur dann schwierig ist, wenn es einen alternierenden Weg p von v

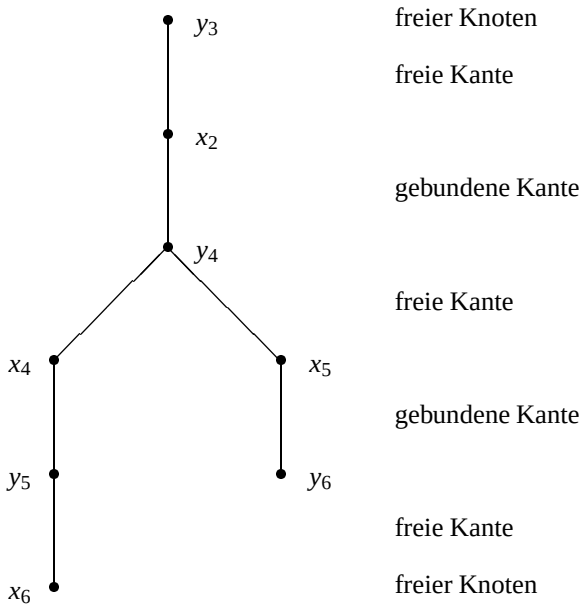


Abbildung 8.47

zu einem Knoten v' in gerader Entfernung von v gibt, und wenn eine Kante v' mit einem anderen Knoten v'' verbindet, der auf dem Weg p ebenfalls in gerader Entfernung von v liegt (vgl. Abbildung 8.49).

Der Teil des Weges p von v'' nach v' heit zusammen mit der Kante (v', v'') *Blte*; eine Blte ist also ein Zyklus ungerader Lnge. Knoten v'' heit *Basis* der Blte. Der Teil des Weges p von v nach v'' heit *Stiel* der Blte.

In dem in Abbildung 8.49 gezeigten Beispiel gibt es sowohl einen alternierenden Weg von v nach i als auch einen alternierenden Weg von v nach j . Den ersteren erhlt man, wenn man im Zyklus ungerader Lnge im Uhrzeigersinn fortschreitet, den letzteren erhlt man durch Besuchen einiger Knoten des Zyklus entgegen dem Uhrzeigersinn. Diese beiden Wege kann man finden, wenn man die Blte auf einen Knoten schrumpfen lt, also den Zyklus ungerader Lnge in einen Knoten kollabiert. Jede Kante, die vor dem Schrumpfen mit einem Knoten des Zyklus inzident war, ist nach dem Schrumpfen mit dem die Blte reprsentierenden Knoten inzident. Abbildung 8.50 zeigt den Effekt des Schrumpfens der Blte fr die in Abbildung 8.49 gezeigte Situation.

Wenn ein Graph G' aus einem Graphen G durch Schrumpfen einer Blte entsteht, so gibt es in G' genau dann einen vergroernden Weg, wenn es einen solchen in G gibt. Davon kann man sich wie folgt berzeugen. Schlieen wir zunchst aus der Existenz eines vergroernden Weges in G' auf die Existenz eines solchen Weges in G . Dies ist offensichtlich, wenn ein in G' betrachteter vergroernder Weg die Blte nicht

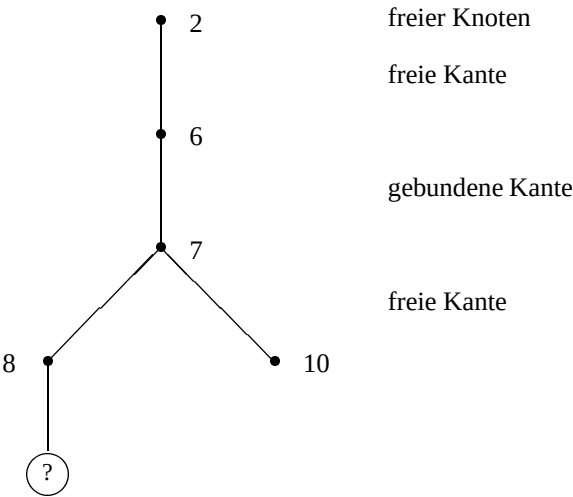


Abbildung 8.48

enthält. Enthält dagegen der betrachtete Weg in G' den Knoten b , der die geschrumpfte Blüte repräsentiert, so expandieren wir b zur vollen Blüte. Falls der betrachtete Weg nur einen Knoten der Blüte passiert, bleibt er erhalten, wenn b durch diesen Knoten ersetzt wird (vgl. Abbildung 8.51 (a)). Falls der betrachtete Weg jedoch mehr als einen Knoten der Blüte passiert, so eignet sich genau einer der beiden möglichen Wege durch einen Teil der Blüte als Verbindung zwischen den beiden Teilen des zerfallenen Weges (vgl. Abbildung 8.51 (b)).

Der Schluß auf die Existenz eines vergrößernden Weges in G' aus der Existenz eines solchen Weges in G ist schwieriger. Wir führen den Nachweis indirekt, indem wir einen

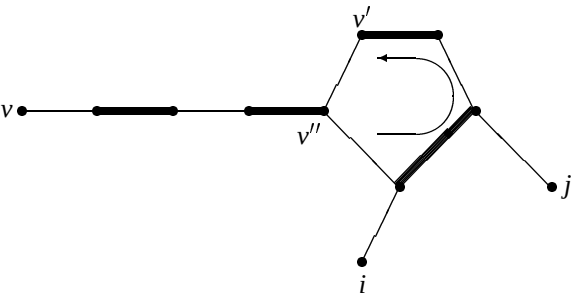


Abbildung 8.49

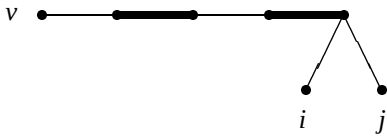


Abbildung 8.50

Algorithmus angeben, der einen vergrößernden Weg mit Hilfe des Schrumpfens von Blüten findet.

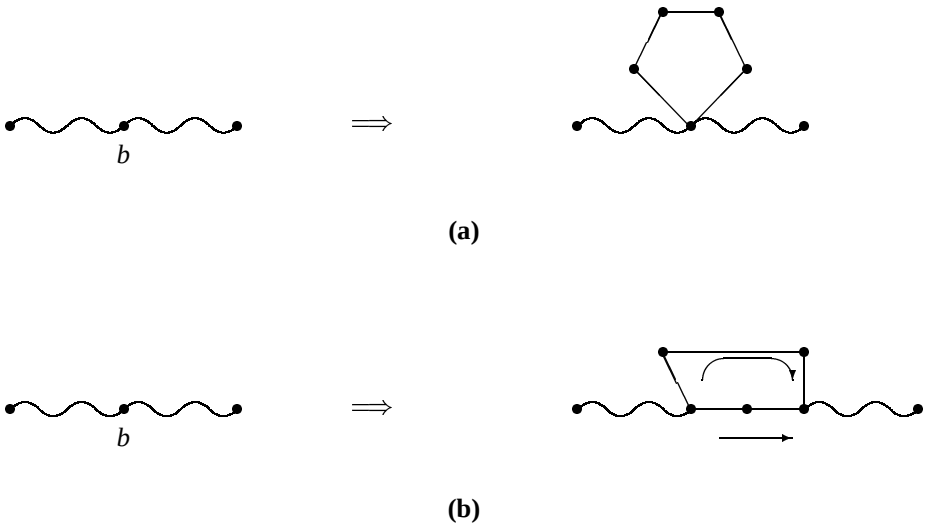


Abbildung 8.51

Der von Edmonds vorgeschlagene Algorithmus beginnt das Durchlaufen eines Graphen bei einem freien Knoten und konstruiert dabei einen Wald von Bäumen mit alternierenden Wegen. Sowie eine Blüte entdeckt ist, wird sie zu einem Knoten geschrumpft. Zum Zwecke des Durchlaufens des Graphen ersetzen wir jede Kante (v, v') durch die beiden Pfeile (v, v') und (v', v) . Jeder Knoten hat stets einen von drei Zuständen: Er ist entweder *unerreicht*, *gerade* oder *ungerade*. Zu jedem Knoten v merken wir uns dessen Vorgänger $p(v)$ beim Durchlaufen des Graphen. Für einen gebundenen Knoten v bezeichnet $Partner(v)$ denjenigen Knoten, der mit derselben gebundenen Kante inzidiert wie v . Dann findet der folgende Algorithmus einen vergrößernden Weg in G' , wenn es einen solchen in G gibt:

Algorithmus Vergrößernder Weg
{ liefert zu einem Digraphen $G = (V, E)$ und einer Zuordnung $Z \subseteq E$ einen vergrößernden Weg in G bezüglich Z , falls es einen solchen

```

gibt}
begin
  1. {Initialisiere:}
    for all  $v \in V$ ,  $v$  frei bezüglich  $Z$ , do
       $v.Zustand := gerade$ ;
    for all  $v \in V$ ,  $v$  gebunden bezüglich  $Z$ , do
       $v.Zustand := unerreicht$ ;
  2. {Suche vergrößernden Weg:}
    repeat {prüfe einen Pfeil:}
      wähle einen noch nicht untersuchten Pfeil  $(v, v')$ ,
        für den  $v.Zustand = gerade$  ist;
      case  $v'.Zustand$  of
        ungerade : {Fall 1} tue nichts;
        unerreicht : begin {Fall 2}
           $v'.Zustand := ungerade$ ;
           $Partner(v').Zustand := gerade$ ;
           $p(v') := v$ ;
           $p(Partner(v')) := v'$ ;
        end;
      gerade : if  $v$  und  $v'$  sind im selben Baum then
        begin {Fall 3}
           $v'' :=$  nächster gemeinsamer Vorfahr
            von  $v$  und  $v'$  im Baum;
          schrumpfe die Blüte  $v, v', \dots, v'', \dots, v$  in
            den Knoten  $v''$  und passe dabei  $p$  an
        end
      else {Fall 4}
        verbinde  $v$  und  $v'$ 
        {dies ergibt vergrößernden Weg zwischen den
          Wurzeln der Bäume, die  $v$  und  $v'$  enthalten}
      until  $v'.Zustand = gerade$  und  $v$  und  $v'$  sind nicht
        im selben Baum {Fall 4 ist aufgetreten}
      or kein Pfeil  $(v, v')$  mit  $v.Zustand = gerade$  ist
        noch nicht untersucht
    end {Vergrößernder Weg}

```

Die entscheidenden Aktionen im Algorithmus finden in den mit Fall 2, Fall 3 und Fall 4 markierten Situationen statt; Fall 1 ist unkritisch (er tritt beispielsweise bei Zyklen gerader Länge auf). Im Fall 2 wird ein bisher gefundener alternierender Weg um eine freie und eine gebundene Kante verlängert. Im Fall 3 wird eine Blüte geschrumpft. Im Fall 4 müssen zwei bereits gefundene alternierende Wege mit jeweils gerader Kantenzahl durch Hinzunahme einer freien Kante verbunden werden; damit erhält man einen vergrößernden Weg, und die Ausführung des Algorithmus ist beendet.

Betrachten wir als Beispiel den in Abbildung 8.46 dargestellten Graphen mit der Zuordnung $Z = \{(6, 7), (8, 10)\}$. Abbildung 8.52 zeigt einen Ausschnitt dieses Graphen, wobei der Zustand *gerade* für einen Knoten durch ein Pluszeichen angegeben ist; den Zustand *ungerade* werden wir durch ein Minuszeichen angeben. Wählen wir als ersten

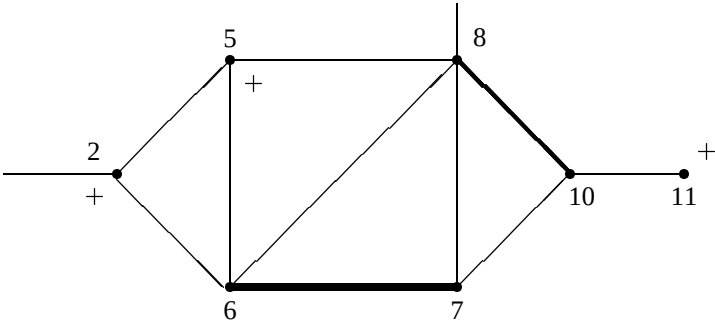


Abbildung 8.52

Pfeil den Pfeil (2,6), so liegt Fall 2 vor, weil Knoten 6 bislang unerreicht ist. Wir setzen also den Zustand von Knoten 6 auf *ungerade*, den Zustand von Knoten 7, das ist der Zuordnungspartner von Knoten 6, auf *gerade*, und merken uns Knoten 2 als Vorgänger von Knoten 6 und Knoten 6 als Vorgänger von Knoten 7. Die entstehende Situation ist in Abbildung 8.53 gezeigt.

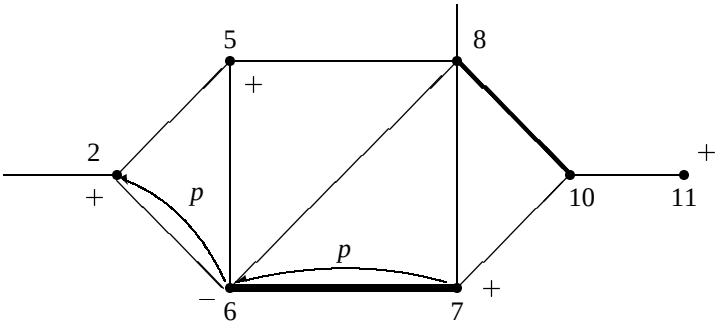


Abbildung 8.53

Im Effekt ist also aus einem Weg der Länge 0, nämlich Knoten 2 alleine, durch Hinzunahme der freien Kante (2,6) und der gebundenen Kante (6,7) ein alternierender Weg der Länge 2 konstruiert worden. Wählen wir beim nächsten Durchlauf der **repeat**-Schleife des Algorithmus *Vergrößernder Weg* den Pfeil (7,10), so liegt wieder Fall 2 vor, und der Weg 2, 6, 7 wird über Knoten 7 hinaus zu Knoten 10 und Knoten 8 weitergeführt. Abbildung 8.54 zeigt die entstehende Situation.

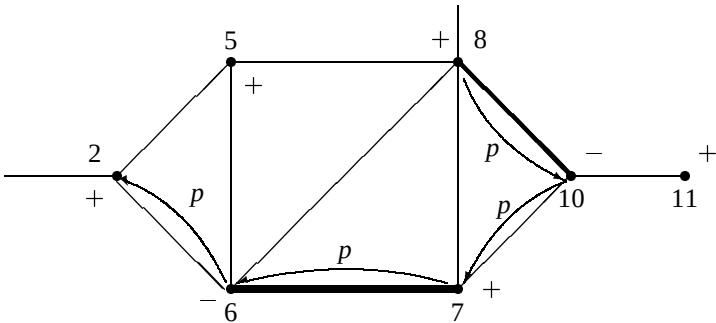


Abbildung 8.54

Wählen wir bei der nächsten Iteration den Pfeil (11,10), so liegt Fall 1 vor. Pfeil (11,10) ist damit untersucht, ohne daß sich an einem Weg etwas geändert hat. Wählen wir bei der nächsten Iteration Pfeil (8,7), so tritt erstmals Fall 3 ein. Knoten 8 und Knoten 7 befinden sich im Baum mit Wurzel 2. Der nächste gemeinsame Vorfahr von Knoten 8 und Knoten 7 ist Knoten 7. Wir schrumpfen also die Blüte 8, 7, 10 in den Knoten 7 und bezeichnen diesen Knoten jetzt als 7'. Abbildung 8.55 zeigt die entstandene Situation.

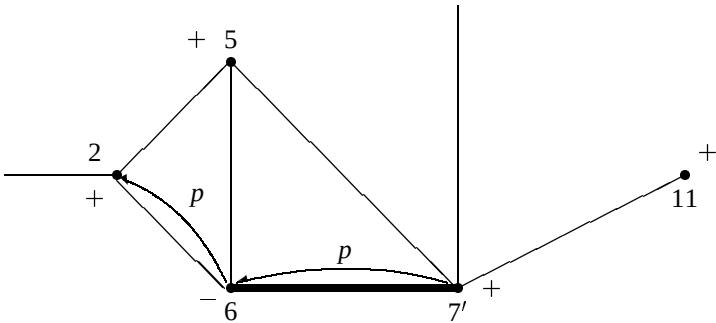


Abbildung 8.55

Knoten 7' ist im Zustand *gerade*, weil Knoten 7 vor dem Schrumpfen der Blüte im Zustand *gerade* war. Betrachten wir bei der nächsten Iteration den Pfeil (11,7'), so liegt Fall 4 vor. Knoten 7' befindet sich im Baum mit Wurzel 2, und Knoten 11 bildet einen eigenen Baum. Jetzt werden die beiden Bäume mit der Kante (11,7') verbunden; es ent-

steht ein vergrößernder Weg zwischen den beiden Wurzeln der Bäume, also zwischen Knoten 2 und Knoten 11.

Um einen vergrößernden Weg im ursprünglich gegebenen Graphen zu finden, werden alle betroffenen Blüten wieder expandiert. Für den expandierten Weg von Knoten 2 nach Knoten 11 gibt es nun zwei Möglichkeiten, die Blüte 8, 7, 10 zu durchlaufen. Die eine Möglichkeit, nämlich die Kante (7,10) in der Blüte zu wählen, scheidet aus, weil der entstehende Weg 2, 6, 7, 10, 11 nicht alternierend ist. Ein alternierender Weg von Knoten 2 nach Knoten 11 ergibt sich, wenn man innerhalb der Blüte den Weg 7, 8, 10 einschlägt. Der entstehende, alternierende Weg 2, 6, 7, 8, 10, 11 ist ein vergrößernder Weg, weil beide Endknoten frei sind.

In der Literatur sind verschiedene effiziente Implementierungen dieses Algorithmus von Edmonds vorgeschlagen worden. In [1] ist eine spezielle Struktur zur Verwaltung disjunkter Mengen für eingeschränkte Fälle vorgestellt worden, die sich zum Verwalten von Blüten und Teilbäumen im Graphen eignet. Mit dieser Struktur gelingt es, eine maximale Zuordnung für einen Graphen $G = (V, E)$ in Zeit $O(|V| \cdot |E|)$ zu finden.

Später [2] wurde ein Algorithmus gefunden, der im wesentlichen den Algorithmus für eine bipartite Zuordnung um das Schrumpfen von Blüten ergänzt und mit einer Laufzeit von $O(\sqrt{|V|} \cdot |E|)$ auskommt. Somit ist das Berechnen einer maximalen Zuordnung für einen beliebigen Graphen größenordnungsmäßig nicht teurer als für einen bipartiten Graphen, ein beachtliches Ergebnis.

8.8.3 Maximale gewichtete Zuordnungen

Das Berechnen maximaler gewichteter Zuordnungen ähnelt dem Berechnen maximaler Zuordnungen ohne Kantengewichte sehr stark. Außer alternierenden Wegen betrachten wir hier aber auch alternierende Zyklen, weil auch diese das Gewicht einer Zuordnung vergrößern können. Das Gewicht $w(p)$ eines alternierenden Weges oder Zyklus p ist das Gesamtgewicht der freien Kanten in p abzüglich dem Gesamtgewicht der gebundenen Kanten in p , also $w(p) = \sum_{e \in p, e \notin Z} w(e) - \sum_{e \in p, e \in Z} w(e)$. Wir vergrößern eine Zuordnung Z , indem wir die Anzahl der Kanten in Z um 1 erhöhen. Daß dies stets durch Berücksichtigung eines Z vergrößernden Wegs mit maximalem Gewicht geschehen kann, zeigt folgende Überlegung. Sei Z eine Zuordnung maximalen Gewichts unter allen Zuordnungen der Größe $|Z|$, und sei p ein vergrößernder Weg für Z mit maximalem Gewicht. Dann ist das Resultat der Vergrößerung von Z durch p eine Zuordnung mit maximalem Gewicht unter allen Zuordnungen der Größe $|Z| + 1$. Um dies einzusehen, betrachten wir eine Zuordnung $Z_{\max}$ mit maximalem Gewicht unter allen Zuordnungen der Größe $|Z| + 1$. Betrachten wir jetzt die symmetrische Differenz Z_{sym} zwischen Z und $Z_{\max}$, also $Z_{\text{sym}} = (Z - Z_{\max}) \cup (Z_{\max} - Z)$. Definieren wir nun das Gewicht eines Wegs oder Zyklus in Z_{sym} mit Bezug auf Z , also als Differenz der Gewichte freier Kanten bezüglich Z und gebundener Kanten bezüglich Z , so hat jeder Zyklus oder Weg gerader Länge das Gewicht 0. Dies muß gelten, weil sich Kanten aus $Z_{\max}$ mit Kanten aus Z abwechseln und sowohl $Z_{\max}$ als auch Z maximales Gewicht haben muß, denn hätte Z ein von $Z_{\max}$ verschiedenes Gewicht, so könnte man die Zuordnung mit dem geringeren Gewicht durch diejenige mit dem größeren Gewicht ersetzen. Weil durch einen vergrößernden Weg zu Z genau eine Kante hinzukommt, stammt in Z_{sym} genau eine Kante

mehr aus $Z_{\max}$ als aus Z . Die Wege in Z_{sym} können so zu Paaren zusammengefaßt werden, daß für jedes Paar gleich viele Kanten aus Z und aus $Z_{\max}$ kommen, und daß das Gewicht jedes Paares von Wegen 0 ist, mit Ausnahme eines einzigen Wegs ungerader Länge. Dieser Weg kann zur Vergrößerung für Z verwendet werden; er führt zu einer Zuordnung der Größe $|Z| + 1$ mit demselben Gewicht wie $Z_{\max}$.

Die dieser Überlegung entsprechende iterierte Vergrößerung des Gewichts einer Zuordnung verläuft mit abnehmender Zunahme des Gewichts in jedem Iterationsschritt. Zum Berechnen einer Zuordnung maximalen Gewichts genügt es also, die Iteration anzuhalten, wenn die Gewichtszunahme negativ würde. Die Überlegungen zu Blüten gelten wie für maximale Zuordnungen. Wegen der zusätzlichen Berücksichtigung von Kantengewichten sind die bekannten Algorithmen für maximale gewichtete Zuordnungen aber nicht ganz so effizient. Die schnellsten Implementierungen   bzw.  erreichen eine Laufzeit von $O(|V|^3)$ bzw. $O(|V| \cdot |E| \log |V|)$.

8.9 Aufgaben

Aufgabe 8.1

- Geben Sie an, wie der in Abbildung 8.8 dargestellte Graph in einer Adjazenzmatrix, in Adjazenzlisten und in einer doppelt verketteten Pfeilliste gespeichert wird.
- Ignorieren Sie die Pfeilrichtungen dieses Graphen und deuten Sie ihn als ungerichteten Graphen, so daß also jeder Pfeil als Kante interpretiert wird. Geben Sie an, wie der so definierte ungerichtete Graph in einer Adjazenzmatrix, in Adjazenzlisten und in einer doppelt verketteten Pfeilliste gespeichert wird.
- Welche Besonderheiten ergeben sich im allgemeinen beim Speichern ungerichteter Graphen gegenüber gerichteten Graphen für die drei Speicherungsformen?

Aufgabe 8.2

- Schreiben Sie ein Pascal-Programm, das es gestattet, eine der drei Speicherungsformen zu wählen und einen Graphen einzugeben. Die Eingabe soll interaktiv durch Angabe von Knoten und Pfeilen bzw. Kanten in beliebiger Reihenfolge erfolgen können. Außerdem soll es möglich sein, einen auf einer externen Datei gespeicherten Graphen einzulesen und einen Graphen auf einer externen Datei zu speichern.
- Ergänzen Sie das Programm aus Teilaufgabe a) um einige Prozeduren zum Editieren eines Graphen. Es soll mindestens möglich sein, einzelne Knoten und Kanten bzw. Pfeile hinzuzufügen und zu löschen. Löscht man einen Knoten, so sollen auch alle inzidenten Kanten bzw. Pfeile gelöscht werden.

- c) Ergänzen Sie das Programm aus Teilaufgabe b) um eine graphische Ausgabemöglichkeit von Graphen. Weil ein automatisches, schönes Zeichnen von Graphen sehr schwierig ist, sollen Positionen von Knoten (z.B. Koordinaten) mit dem Graphen abgespeichert und ebenfalls editiert werden können. Kanten bzw. Pfeile solle als geradlinige Verbindungen der entsprechenden Knoten gezeichnet werden.
- d) Ergänzen Sie das Programm aus Teilaufgabe c) um eine interaktive, graphische Benutzerschnittstelle. Es soll also nicht nur die Ausgabe graphisch möglich sein, sondern auch die Eingabe durch den Benutzer. Man sollte wenigstens Knoten und Kanten bzw. Pfeile graphisch selektieren können (Anklicken), beispielsweise um sie zu löschen oder um Knoten zu verschieben. Folgeeffekte, wie das Löschen der mit einem gelöschten Knoten inzidenten Kanten oder das Verziehen von Kanten sollen automatisch graphisch berücksichtigt werden.

Aufgabe 8.3

Geben Sie für jede der drei Speicherungsformen (möglichst sinnvolle) Operationen an, die bei dieser Speicherungsform zumindest in gewissen Fällen

- a) effizienter als bei den beiden anderen
- b) weniger effizient als bei den beiden anderen

ausgeführt werden können.

Aufgabe 8.4

- a) Berechnen Sie nach dem in Abschnitt 8.1 vorgestellten Algorithmus eine topologische Sortierung des in Abbildung 8.3 dargestellten Digraphen. Wieviele verschiedene topologische Sortierungen gibt es in diesem Beispiel?
- b) Modifizieren Sie den Algorithmus zur topologischen Sortierung so, daß er diese für einen in einer Adjazenzmatrix mit Zusatzinformation über bedeutsame Einträge gespeicherten Digraphen berechnet. Welche Laufzeit hat der modifizierte Algorithmus?

Aufgabe 8.5

- a) In Mehrbenutzer-Betriebssystemen konkurrieren verzahnt ablaufende Prozesse um Betriebsmittel. Hat beispielsweise ein Prozeß p den Farbdrucker f gerade belegt, und benötigt ein anderer Prozeß p' ebenfalls f , so muß p' warten, bis p wieder f freigibt. Dies definiert eine binäre Relation: p' wartet auf p . Wenn in dieser Relation ein Zyklus auftritt (p' wartet wegen des Farbdruckers auf p ; p wartet wegen des Lochstreifenlesers auf p'), so ist das Fortsetzen der Prozesse auf Dauer behindert, die Prozesse sind verklemmt. Es ist dann wünschenswert, gewisse Prozesse abubrechen, die gebundenen Betriebsmittel freizugeben und diese Prozesse später erneut zu starten.

Entwerfen Sie einen möglichst effizienten Algorithmus, der in einer Menge von Prozessen und Warte-Beziehungen der Prozesse untereinander feststellt, wie durch Abbrechen einer möglichst kleinen Anzahl von Prozessen alle bestehenden Verklemmungen aufgelöst werden können. Welche Laufzeit hat Ihr Algorithmus?

- b) Entwerfen Sie einen möglichst effizienten Algorithmus, der aus einem gegebenen Digraphen durch Entfernen einer möglichst kleinen Anzahl von Pfeilen einen zyklenfreien Digraphen herstellt. Welche Laufzeit hat Ihr Algorithmus?

Aufgabe 8.6

- a) Entwerfen Sie einen möglichst effizienten Algorithmus, der zu einem gegebenen, zyklenfreien Digraphen die Anzahl der verschiedenen topologischen Sortierungen berechnet. Welche Laufzeit hat Ihr Algorithmus?
- b) Entwerfen Sie einen möglichst effizienten Algorithmus, der zu einem gegebenen Digraphen die Anzahl der verschiedenen einfachen Zyklen berechnet (für den Digraphen in Abbildung 8.4 ist diese Anzahl 3). Welche Laufzeit hat Ihr Algorithmus?

Aufgabe 8.7

Entwerfen Sie einen möglichst effizienten Algorithmus zur Berechnung der reflexiven, transitiven Hülle zu einem beliebigen, in Adjazenzlistenrepräsentation gegebenen Digraphen. Welche Laufzeit hat dieser Algorithmus, insbesondere für Graphen mit wenigen Kanten?

Aufgabe 8.8

Bei einer Meinungsumfrage hat ein Befragter aus einer vorgelegten Liste von Tätigkeitspaare von Tätigkeiten gebildet, wobei er die erste Tätigkeit der zweiten vorzieht; über manche Tätigkeitspaare hat er keine Aussage gemacht. So äußert er beispielsweise, Bier trinken oder fernsehen sei schöner als Holz hacken, Holz hacken schöner als Schach spielen, und Bier trinken sei schöner als Schach spielen. Auf die zuletzt genannte Präferenz hätte der Interviewer allerdings auch (durch Transitivität) selbst schließen können. Nehmen Sie an, daß der Befragte konsistent geantwortet hat, also keine Tätigkeit schöner findet als diese selbst (über Transitivität).

- a) Entwerfen Sie einen möglichst effizienten Algorithmus, der aus der Menge aller Tätigkeitspaare, die ein Befragter angegeben hat, diejenigen entfernt, auf die man durch die verbleibenden schließen kann. Geben Sie die Laufzeit Ihres Algorithmus in Abhängigkeit von der Anzahl der angegebenen und der übrigen Tätigkeitspaare an.
- b) Entwerfen Sie einen möglichst effizienten Algorithmus, der zur Menge aller durch einen Befragten angegebenen Tätigkeitspaare all diejenigen Tätigkeitspaare ermittelt, auf die man nicht über Transitivität schließen kann. Geben Sie die Laufzeit Ihres Algorithmus in Abhängigkeit von der Anzahl der angegebenen und der Anzahl der zu ermittelnden Tätigkeitspaare an.
- c) Entwerfen Sie einen möglichst effizienten Algorithmus, der die Menge der Tätigkeiten so in kleinstmögliche Teilmengen zerlegt, daß über Tätigkeiten aus verschiedenen Teilmengen nie eine Präferenzaussage vorliegt. Geben Sie die Laufzeit Ihres Algorithmus in Abhängigkeit von der Anzahl der Tätigkeiten und der Anzahl der angegebenen Tätigkeitspaare an.

Aufgabe 8.9

Berechnen Sie den DFBIndex und den DFEIndex eines jeden Knotens sowie die Klassifikation aller Pfeile in Baum-, Vorwärts-, Rückwärts- und Seitwärtspfeile für den Graphen in Abbildung 8.11 und jeden der Startknoten 2, 3, 4 und 5 einer Tiefensuche. In welchen dieser Fälle kann man die Knoten nach dem allgemeinen Knotenbesuchsalgorithmus auch in einer anderen Reihenfolge besuchen?

Aufgabe 8.10

Ein Graph ist dreifach zusammenhängend, wenn er nach dem Entfernen zweier beliebiger Knoten samt aller inzidenten Kanten noch zusammenhängend ist.

- Entwerfen Sie einen möglichst effizienten Algorithmus, der prüft, ob ein gegebener Graph dreifach zusammenhängend ist. Welche Laufzeit hat Ihr Algorithmus?
- Entwerfen Sie einen möglichst effizienten Algorithmus, der alle dreifachen Zusammenhangskomponenten eines gegebenen Graphen berechnet.
- Wenden Sie Ihren Algorithmus auf das in Abbildung 8.12 (a) dargestellte Beispiel an.

Aufgabe 8.11

Eine Kante in einem zusammenhängenden, ungerichteten Graphen heißt *Brücke*, wenn das Entfernen dieser Kante den Graphen in zwei Teile zerfallen läßt.

Entwerfen Sie einen möglichst effizienten Algorithmus, der zu einem gegebenen Graphen alle Brücken ermittelt. Wie schnell arbeitet Ihr Algorithmus?

Aufgabe 8.12

In einer Stadt verspricht man sich eine Beschleunigung des Verkehrsflusses, wenn man aus den bisher in beiden Fahrtrichtungen benutzbaren, oftmals engen Straßen Einbahnstraßen macht. Danach soll es natürlich noch möglich sein, von jedem Ort an jeden anderen zu gelangen.

Entwerfen Sie einen möglichst effizienten Algorithmus, der zu einem gegebenen Netz von Straßen, die in beiden Richtungen befahrbar sind, ein solches Einbahnstraßennetz findet, wann immer dies möglich ist. Zeigen Sie, daß dies genau dann möglich ist, wenn das gegebene Netz zusammenhängend ist und keine Brücken enthält. (Mehr über dieses und ähnliche Probleme findet man in )

Aufgabe 8.13

Verfolgen Sie anhand des Beispiels von Abbildung 8.18 Dijkstras Algorithmus zum Finden aller kürzesten Wege von Knoten 4 aus, wenn die Randknoten in einem Fibonacci-Heap verwaltet werden.

Aufgabe 8.14

Um eine wichtige, geheime Botschaft von A nach B zu befördern, werden aus Sicherheitsgründen zwei Kuriere losgeschickt, die völlig verschiedene Wege von A nach B in einem Netz von Wegen wählen müssen. Diese Wege sollen so gewählt werden, daß der längere der beiden möglichst kurz ist. Entwerfen Sie einen Algorithmus, der zwei solche Wege wählt, wenn

- a) Wege im Netz in beiden Richtungen benutzbar sind;
- b) Wege nur in einer Richtung benutzbar sind.

Aufgabe 8.15

Das Finden eines kürzesten Weges in einem bewerteten, ungerichteten Graphen mit beliebiger Kantenbewertung scheitert im allgemeinen an der möglichen Existenz negativer Zyklen; in diesem Fall existiert kein kürzester Weg, weil der negative Zyklus mehrmals durchlaufen werden kann. Dieses Problem verschwindet, wenn wir nur einfache Wege suchen, also solche Wege, die jeden Knoten höchstens einmal betreten.

Entwerfen Sie für diesen Fall einen möglichst effizienten Algorithmus zum Finden eines kürzesten Weges zwischen zwei gegebenen Knoten. Welche Laufzeit hat Ihr Algorithmus?

Aufgabe 8.16

Versehen Sie die Pfeile in Abbildung 8.3 mit Werten für die Dauern der entsprechenden Vorgänge und berechnen Sie mit einem Auswahlverfahren nach Ford die Mindestdauer des Gesamtprojekts. Wählen Sie dabei Randknoten gemäß einer topologischen Sortierung.

Aufgabe 8.17

In einem Distanzgraphen kann man hoffen, einen kürzesten Weg zwischen zwei gegebenen Knoten schnell zu finden, wenn man eine Breitensuche wie bei Dijkstras Algorithmus nicht nur bei einem der beiden Knoten startet, sondern gleichzeitig bei beiden. Präzisieren Sie diese Idee und entwerfen Sie einen entsprechenden Algorithmus. Implementieren Sie Ihren Algorithmus und Dijkstras Algorithmus und experimentieren Sie.

Aufgabe 8.18

Entwerfen Sie einen Algorithmus zur Berechnung eines kürzesten Weges zwischen zwei gegebenen Knoten eines Distanzgraphen, der in Matrixform gespeichert ist. Der Algorithmus soll auf der Matrix operieren. Welche Laufzeit hat Ihr Algorithmus? Welchen Effekt hat die Matrixspeicherung auf die Berechnung aller kürzesten Wege im Graphen?

Aufgabe 8.19

Geben Sie an, wie man Dijkstras Algorithmus zur Berechnung kürzester Wege so modifizieren kann, daß er neben der Länge auch die Anzahl der kürzesten Wege von einem gegebenen Startknoten zu einem anderen Knoten berechnet.

Aufgabe 8.20

Entwerfen Sie einen möglichst effizienten Algorithmus, der in einem bewerteten, ungerichteten Graphen einen Weg zwischen zwei gegebenen Knoten findet, bei dem

- a) die Länge der längsten Kante möglichst klein ist;
- b) die Länge der kürzesten Kante möglichst groß ist.

Aufgabe 8.21

Verfolgen Sie die Berechnung eines minimalen, spannenden Baums für den in Abbildung 8.18 dargestellten Graphen nach jedem der in Abschnitt 8.6 vorgestellten Verfahren.

Aufgabe 8.22

Entwerfen Sie einen Algorithmus zur Berechnung eines spannenden Baums für einen gegebenen Distanzgraphen, bei dem

- a) die Länge der längsten Kante möglichst klein ist;
- b) die Länge des längsten Weges zwischen zwei Knoten — das ist der Durchmesser des Baumes — möglichst klein ist;
- c) der größte Knotengrad möglichst klein ist.

Aufgabe 8.23

Entwerfen sie einen möglichst effizienten Algorithmus, der in einem gegebenen Distanzgraphen einen Knoten — das Zentrum — findet, dessen größte Entfernung zu irgendeinem anderen Knoten des Graphen minimal ist. Welche Laufzeit hat Ihr Algorithmus?

Aufgabe 8.24

Legen Sie für jede Kante des in Abbildung 8.46 gezeigten Graphen eine Kapazität und eine Orientierung fest, so daß sich ein Kapazitätsdigraph mit Quelle 11 und Senke 12 ergibt. Berechnen Sie einen maximalen Fluß von der Quelle zur Senke nach dem Algorithmus

- a) Flußvergrößerung durch einzelne kürzeste zunehmende Wege;
- b) Flußvergrößerung durch kürzesten Weg im Niveaugraphen.

Aufgabe 8.25

Ändern Sie die zur Berechnung eines maximalen Flusses vorgestellten Algorithmen so, daß auch Mindestkapazitäten von Pfeilen berücksichtigt werden. Dabei soll der Fluß entlang eines Pfeiles für jeden Pfeil

- a) zwischen der Mindest- und der Maximalkapazität für diesen Pfeil liegen;
- b) entweder 0 sein oder zwischen der Mindest- und der Maximalkapazität für diesen Pfeil liegen.

Aufgabe 8.26

Entwerfen Sie einen möglichst effizienten Algorithmus, der zu einem gegebenen Kapazitätsdigraphen einen Fluß

- a) mit möglichst vielen gesättigten Pfeilen;
- b) mit mindestens einer Flußeinheit für jeden Pfeil;

- c) mit einem möglichst niedrigen Durchfluß durch den Knoten mit größtem Durchfluß bei einem maximalen Fluß

berechnet.

Aufgabe 8.27

Bestimmen Sie für den Graphen in Abbildung 8.18 eine maximale Zuordnung nach der Methode der vergrößernden Wege; ignorieren Sie die Bewertungen der Kanten.

Aufgabe 8.28

Bestimmen Sie für den Graphen in Abbildung 8.18 eine maximale, gewichtete Zuordnung.

Aufgabe 8.29

Entwerfen Sie einen möglichst effizienten Algorithmus, der für einen gegebenen, ungerichteten Graphen die Anzahl der maximalen Zuordnungen ermittelt.

Aufgabe 8.30

Bestimmen Sie eine möglichst scharfe obere Schranke für die Anzahl der freien Knoten bezüglich einer maximalen Zuordnung in einem beliebigen, ungerichteten Graphen.

Aufgabe 8.31

Entwerfen Sie einen möglichst effizienten Algorithmus, der eine gegebene Zuordnung in einem ungerichteten Graphen maximal erweitert. Gesucht ist dabei eine Zuordnung, in der alle als gebunden gegebenen Kanten gebunden sind, und die maximal ist unter allen solchen Zuordnungen.

Aufgabe 8.32

- Wir verallgemeinern den Begriff der Zuordnung so, daß ein gebundener Knoten zu mehr als einer gebundenen Kante gehören darf. Entwerfen Sie einen möglichst effizienten Algorithmus, der für einen gegebenen, ungerichteten Graphen eine möglichst kleine Menge gebundener Kanten berechnet, so daß alle Knoten gebunden sind.
- Entwerfen Sie einen möglichst effizienten Algorithmus, der eine kleinstmögliche Menge von Knoten eines gegebenen, ungerichteten Graphen wählt, so daß jede Kante mit wenigstens einem Knoten inzidiert.
- Entwerfen Sie einen möglichst effizienten Algorithmus, der eine größtmögliche Menge von Knoten eines gegebenen Graphen wählt, so daß jede Kante mit höchstens einem Knoten inzidiert.

Wie vereinfachen sich diese Probleme, wenn wir nur bipartite Graphen als Eingabe zulassen?

Literaturliste zu Kapitel 8: Graphenalgorithmen

Seite 536

[48] L. Euler. Solutio problematis ad geometriam situs pertinentis. Comment. Acad. Sci. Imper. Petropol., 8:128-140, 1736.

Seite 541

- [75] F. Harary. Graph Theory. Addison-Wesley, Reading, Massachusetts, 1969.
[65] A. Gibbons. Algorithmic graph theory. Cambridge University Press, Cambridge, 1985.
[66] M. C. Golumbic. Algorithmic graph theory and perfect graphs. Academic Press, New York, 1980.
[30] N. Christofides. Graph theory: An algorithmic approach. Academic Press, New York, 1975.
[18] C. Berge. Graphs and Hypergraphs. North-Holland, Amsterdam, 1973.
[104] E. L. Lawler. Combinatorial optimization: Networks and matroids. Holt, Rinehart, and Winston, New York, 1976.
[121] K. Mehlhorn. Data structures and algorithms, Vol. 2: Graph algorithms and NP-completeness. Springer, Berlin, 1984.
[45] J. Edmonds und R. M. Karp. Theoretical improvements in algorithmic efficiency for network flow problems. J. Assoc. Comput. Mach., 19:248-264, 1972.
[83] D. Jungnickel. Graphen, Netzwerke und Algorithmen. BI-Wissenschaftsverlag, Mannheim, Wien, Zürich, 1987.
[144] C. H. Papadimitriou und K. Steiglitz. Combinatorial optimization: Networks and complexity. Prentice-Hall, Englewood Cliffs, New Jersey, 1982.

Seite 548

[191] S. Warshall. A theorem on Boolean matrices. J. Assoc. Comp. Mach., 9:11-12, 1962.

Seite 557

[121] K. Mehlhorn. Data structures and algorithms, Vol. 2: Graph algorithms and NP-completeness. Springer, Berlin, 1984.

Seiten 568, 571

[35] E. W. Dijkstra. A note on two problems in connexion with graphs. Numer. Math., 1:269-271, 1959.

Seite 572

- [60] M. L. Fredman und R. E. Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. J. Assoc. Comput. Mach., 34:596-615, 1987.
[56] L. R. Ford Jr. Network flow theory. Paper P-923, RAND Corp., Santa Monica, CA, 1956.

Seite 576

[45] J. Edmonds und R. M. Karp. Theoretical improvements in algorithmic efficiency for network flow problems. J. Assoc. Comput. Mach., 19:248-264, 1972.

Seite 582

- [21] O. Boruvka. O jiste'm proble'mu minima'lni'm. Pra'ca Moravske' Pjri'rodovjedecke' Spolejcnosti, 3:37-58, 1926.
[96] J. B. Kruskal. On the shortest spanning subtree of a graph and the traveling salesman problem. In Proc. AMS 7, S. 48-50, 1956.

Seite 583

- [35] E. W. Dijkstra. A note on two problems in connexion with graphs. Numer. Math., 1:269-271, 1959.
[82] V. Jarni'k. O jiste'm proble'mu minima'lni'm. Pra'ca Moravske' Pri'rodovjedecke' Spolejcnosti, 6:57-63, 1930.
[150] R. C. Prim. Shortest connection networks and some generalizations. Bell System Techn. J., 36:1389-1401, 1957.

Seite 584

[60] M. L. Fredman und R. E. Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. J. Assoc. Comput. Mach., 34:596-615, 1987.

Seite 585

[58] L. R. Ford Jr. und D. R. Fulkerson. Flows in networks. Princeton University Press, Princeton, N.J., 1962.

Seite 588

[57] L. R. Ford Jr. und D. R. Fulkerson. Maximal flow through a network. Canad. J. Math., 8:399-404, 1956.

[46] P. Elias, A. Feinstein und C. E. Shannon. Note on maximum flow through a network. IRE Trans. Inform. Theory, IT-2:117-119, 1956.

Seite 589

[57] L. R. Ford Jr. und D. R. Fulkerson. Maximal flow through a network. Canad. J. Math., 8:399-404, 1956.

Seite 590

[45] J. Edmonds und R. M. Karp. Theoretical improvements in algorithmic efficiency for network flow problems. J. Assoc. Comput. Mach., 19:248-264, 1972

Seite 592

[37] E. A. Dinic. Algorithm for solution of a problem of maximal flow in a network with power estimation. Soviet Math. Dokl., 11:1277-1280, 1970.

Seite 594

[37] E. A. Dinic. Algorithm for solution of a problem of maximal flow in a network with power estimation. Soviet Math. Dokl., 11:1277-1280, 1970.

[50] S. Even und R. E. Tarjan. Network flow and testing graph connectivity. SIAM J. Comput., 4:507-518, 1975.

Seite 595

[37] E. A. Dinic. Algorithm for solution of a problem of maximal flow in a network with power estimation. Soviet Math. Dokl., 11:1277-1280, 1970.

[50] S. Even und R. E. Tarjan. Network flow and testing graph connectivity. SIAM J. Comput., 4:507-518, 1975.

[85] A. V. Karzanov. Determining the maximal flow in a network by the method of preflows. Soviet Math. Dokl., 15:434-437, 1974.

[180] R. E. Tarjan. Data structures and network algorithms. In SIAM CBMS-NSF Regional Conference Series in Applied Mathematics 44, Philadelphia, 1983. SIAM.

[115] V. M. Malhotra, M. P. Kumar und S. N. Maheshwari. An $O(|V|^3)$ algorithm for finding maximum flows in networks. Information Processing Letters, 7:277- 278, 1978.

[169] Y. Shiloach. An $O(n * l * \log^2(l))$ maximum-flow algorithm. Tech. Report STAN CS-78-802, Computer Science Department, Stanford University, CA, 1978.

[64] Z. Galil und A. Naamad. An $O(E * V * \log(2)V)$ algorithm for the maximum flow problem. J. Comput. System Sci., 21:203-217, 1980.

[171] D. D. Sleator und R. E. Tarjan. A data structure for dynamic trees. J. Computer and System Sciences, 26:362-391, 1983.

Seite 599

[37] E. A. Dinic. Algorithm for solution of a problem of maximal flow in a network with power estimation. Soviet Math. Dokl., 11:1277-1280, 1970.

Seiten 603, 605

[44] J. Edmonds. Paths, trees, and flowers. *Canad. J. Math.*, 17:449-467, 1965.

Seite 609

[62] H. N. Gabow und R. E. Tarjan. A linear-time algorithm for a special case of disjoint set union. In *Proc. 15th Annual ACM Symposium on Theory of Computing*, S. 246-251, 1983.

[125] S. Micali und V. V. Vazirani. An $O((\sqrt{|V|}) * |E|)$ algorithm for finding maximum matching in general graphs. In *Proc. 21st Annual Symposium on Foundations of Computer Science*, S. 17-27, 1980.

Seite 610

[61] H. N. Gabow. Implementation of algorithms for maximum matching on nonbipartite graphs. Dissertation, Dept. Electrical Engineering, Stanford Univ., Stanford, CA, 1973.

[104] E. L. Lawler. *Combinatorial optimization: Networks and matroids*. Holt, Rinehart, and Winston, New York, 1976.

[63] Z. Galil, S. Micali und H. Gabow. Maximal weighted matching on general graphs. In *Proc. 23rd Annual Symposium on Foundations of Computer Science*, S. 255-261, 1982.

Seite 613

[162] F. E. Roberts. Graph theory and its applications to problems of society. In *SIAM CBMS-NSF Regional Conference Series in Applied Mathematics 29*, Philadelphia, 1978. SIAM.

Kapitel 9

Ausgewählte Themen

9.1 Suchen in Texten

In zahlreichen Anwendungen von Computern spielen Texte eine dominierende Rolle. Man denke etwa an Texteditoren, Literaturdatenbanken, Bibliothekssysteme und Systeme zur Symbolmanipulation. Der Begriff *Text* wird hier meistens in einem sehr allgemeinen Sinne benutzt. Texte sind nicht weiter strukturierte Folgen beliebiger Länge von Zeichen aus einem endlichen Alphabet. Das Alphabet kann Buchstaben, Ziffern und zahlreiche Sonderzeichen enthalten.

Der diesen Anwendungen zugrundeliegende Datentyp ist der Typ *string* (*Zeichenkette*). Wir lassen offen, wie dieser Datentyp programmtechnisch realisiert wird. Als Möglichkeiten kommen z.B. die Bereitstellung des Datentyps *string* als einer der Grundtypen der Sprache in Frage oder die Realisierung als *File_of_characters*, als *Array_of_characters* oder als verkettete Liste von Zeichen. Unabhängig von der programmtechnischen Realisierung soll jeder Zeichenkette eine nichtnegative, ganzzahlige Länge zugeordnet werden können und der Zugriff auf das i -te Zeichen einer Zeichenkette für jedes $i \geq 1$ möglich sein. Algorithmen zur Verarbeitung von Zeichenketten (*string processing*) umfassen ein weites Spektrum. Dazu gehören das Suchen in Texten und allgemeiner das Erkennen bestimmter Muster (*pattern matching*), das Verschlüsseln und Komprimieren von Texten, das Analysieren (*parsing*) und Übersetzen von Texten und viele andere Algorithmen.

Wir wollen in diesem Abschnitt nur das Suchen in Texten behandeln und einige klassische Algorithmen zur Lösung dieses Problems angeben. Das Suchproblem kann genauer wie folgt formuliert werden:

Gegeben sind eine Zeichenkette (*Text*) $a_1 \dots a_N$ von Zeichen aus einem endlichen Alphabet Σ und eine Zeichenkette, das *Muster* (*pattern*), $b_1 \dots b_M$, mit $b_i \in \Sigma$, $1 \leq i \leq M$. *Gesucht* sind ein oder alle Vorkommen von $b_1 \dots b_M$ in $a_1 \dots a_N$, d.h. Indizes i mit $1 \leq i \leq (N - M + 1)$ und $a_i = b_1$, $a_{i+1} = b_2$, ..., $a_{i+M-1} = b_M$.

In der Regel ist die Länge N des Textes sehr viel größer als die Länge M des Musters. Als Beispiel verweisen wir auf das Oxford English Dictionary (OED): Die zweite, im Jahre 1989 publizierte Ausgabe des OED umfaßt etwa 616500 definierte Stichworte

und beansprucht 540 Mb Speicherplatz bzw. 20 Bände mit insgesamt 21728 Seiten in der gedruckten Version.

Das OED ist ein Beispiel für *statischen* Text; Änderungen sind verhältnismäßig selten und im Verhältnis zum Gesamtumfang geringfügig. Demgegenüber ist der durch Texteditoren manipulierte Text *dynamisch*; Änderungen sind häufig und erheblich. Will man das Suchproblem für statischen Text lösen, so kann es sich lohnen, den Text durch Hinzufügen von geeigneter Information (einem *Index*) so aufzubereiten, daß die Suche für verschiedene Muster gut unterstützt und insbesondere nicht das Durchsuchen des gesamten Textes erforderlich wird. Bei dynamischem Text lohnt sich eine aufwendige Vorverarbeitung in der Regel nicht. Es kann sich in diesem Fall aber auszahlen, Suchalgorithmen von der Struktur des Musters und vom zugrundeliegenden Alphabet abhängig zu machen.

Wir geben im folgenden eine Reihe von Algorithmen zur Lösung in dynamischen Texten an und verweisen für den anderen Fall (statische Texte) und neueste Ergebnisse über Algorithmen zur Textsuche auf .

9.1.1 Das naive Verfahren zur Textsuche

Am einfachsten läßt sich das Problem, ein Vorkommen des Musters $b_1 \dots b_M$ im Text $a_1 \dots a_N$ zu finden, wie folgt lösen: Man legt das Muster, beginnend beim ersten Zeichen des Textes, der Reihe nach an jeden Teilstring des Textes mit Länge M an und vergleicht zeichenweise von links nach rechts, ob eine Übereinstimmung zwischen Muster und Text vorliegt oder nicht (ein *Mismatch*), solange, bis man ein Vorkommen des Musters im Text gefunden oder das Ende des Textes erreicht hat. In Pascal-ähnlicher Notation kann das Verfahren so beschrieben werden:

```

for  $i := 1$  to  $N - M + 1$  do
  begin
     $found := true$ ;
    for  $j := 1$  to  $M$  do
      if  $a_{i+j-1} \neq b_j$  then  $found := false$ ;
    if  $found$  then  $write('B \text{ kommt vor von Position'}$ ,  $i$ ,
       $'bis Position'$ ,  $i + M - 1$ );
  end

```

Bei diesem Verfahren muß das Muster B offensichtlich $(N - M + 1)$ -mal an den Text A angelegt und dann jeweils ganz durchlaufen werden. Das bedeutet, daß stets $(N - M + 1) \cdot M$ Vergleiche ausgeführt werden. Die Laufzeit des Verfahrens ist also von der Größenordnung $\Theta(N \cdot M)$. Eine Verbesserung ist möglich, wenn man das Muster jeweils nur bis zum ersten Mismatch durchläuft:

```

function bruteforce ( $a, b : string$ ;  $M, N : integer$ ) : integer;
  {liefert den Beginn des Musters  $b[1..M]$  im Text  $a[1..N]$  oder
   einen Wert  $> N$ , falls  $b$  in  $a$  nicht vorkommt}
  var  $i, j : integer$ ;
  begin

```

```

i := 1;
j := 1;
repeat
  if  $a_i = b_j$ 
  then
    begin
       $i := i + 1;$ 
       $j := j + 1$ 
    end
  else
    begin
       $i := i - j + 2;$ 
       $j := 1$ 
    end
  until  $(j > M) \text{ or } (i > N);$ 
  if  $j > M$ 
  then  $bruteforce := i - M$ 
  else  $bruteforce := i$ 
end

```

Jetzt werden in vielen praktischen Fällen nur noch $O(M + N)$ Vergleiche zwischen Zeichen im Text und Zeichen im Muster durchgeführt. Einen solchen Fall zeigt das Beispiel in Abbildung 9.1; hier wird in einem Text mit Länge 50 (einschließlich Leerzeichen und Komma) nach einem Muster mit Länge 4 gesucht. Nach insgesamt 51 Vergleichen wird ein Vorkommen des Musters im Text entdeckt. Der Grund dafür ist, daß in den meisten Fällen ein Mismatch bereits beim ersten Buchstaben auftritt und daher das Muster sofort an die nächste Textposition verschoben werden kann. Andererseits ist es natürlich nicht schwer, Beispiele zu finden, in denen das naive Verfahren mindestens (NM) Schritte benötigt, um ein Vorkommen des Musters im Text zu finden: Man wähle als Text eine Zeichenfolge bestehend aus $N - 1$ Nullen und einer 1 als letztem Zeichen. Das Muster sei ähnlich aufgebaut, d.h. auf $M - 1$ Nullen folge eine 1. Dann wird stets erst beim Vergleich des letzten Zeichens im Muster mit einem Zeichen im Text ein Mismatch entdeckt. Bis man das Vorkommen des Musters im Text gefunden hat, werden also $(N - M) \cdot M + M = \Omega(MN)$ Zeichen verglichen.

Das naive Verfahren ist gedächtnislos in folgendem Sinne: Dieselbe Textstelle wird unter Umständen mehrfach inspiziert; das Verfahren merkt sich nicht, welche Zeichen im Text bereits mit einem Anfangsstück des Musters übereingestimmt haben, bis ein Mismatch auftrat. Das im folgenden Abschnitt dargestellte Verfahren von Knuth-Morris-Pratt nutzt diese Information. Es kann erreicht werden, daß der Zeiger i auf die nächste Textstelle, anders als beim naiven Algorithmus, niemals zurückgesetzt werden muß.

9.1.2 Das Verfahren von Knuth-Morris-Pratt

Dem Verfahren liegt folgende Idee zugrunde: Tritt beim Vergleich des Musters mit dem Text an der j -ten Stelle des Musters ein Mismatch auf, so haben die vorangehenden

er sprach abrakadabra, es bewegte sich aber nichts
aber
aber
aber ...
aber
aber ...
aber
aber

Abbildung 9.1

$j - 1$ Zeichen im Muster und Text übereingestimmt. Wir nutzen jetzt diese Information, um das Muster nach dem Mismatch nicht stets um eine Position, wie beim naiven Verfahren, sondern so weit wie möglich nach rechts zu verschieben. Betrachten wir nun ein Beispiel für ein binäres Alphabet:

	i
Text:	... 010110101 ...
Muster:	0101 0 1

Beim Vergleich des fünften Zeichens im Muster mit dem darüberstehenden i -ten Zeichen im Text tritt ein Mismatch auf. Die vorangehenden vier Zeichen 0101 des Musters haben also mit den darüberstehenden Zeichen im Text übereingestimmt. Wird das Muster um nur eine Position nach rechts verschoben, so tritt mit Sicherheit wieder ein Mismatch auf, und zwar schon an der ersten Stelle. Wie weit kann man das Muster nach rechts verschieben, ohne ein Vorkommen im Text zu übersehen? Offenbar kann man das Muster gleich um zwei Positionen nach rechts verschieben und erneut das i -te Zeichen im Text mit dem darunterstehenden Zeichen im Muster vergleichen. Im vorliegenden Beispiel weiß man, daß keine Übereinstimmung vorliegen kann, da die 0 an der fünften Stelle im Muster nicht mit dem darüberstehenden Zeichen im Text übereingestimmt hat. Das den Mismatch verursachende Zeichen im Text muß also eine 1 gewesen sein. Sie führt abermals zu einem Mismatch beim Vergleich mit dem dritten Zeichen im Muster. Im allgemeinen, d.h. für Texte über beliebigen Alphabeten, kann man aber so nicht argumentieren. Wir bestimmen dann die maximal mögliche Verschiebung des Musters nach rechts allein unter Ausnutzung der Kenntnis der Zeichen im Muster, die mit den darüberstehenden Zeichen im Text übereingestimmt haben, bis ein Mismatch auftrat.

Die allgemeine Situation ist in Abbildung 9.2 dargestellt und kann folgendermaßen beschrieben werden. Nehmen wir an, beim Vergleich des j -ten Zeichens im Muster mit dem i -ten Zeichen im Text tritt ein Mismatch auf, d.h.:

- 1. Die letzten $j - 1$ gelesenen Zeichen im Text stimmen mit den ersten $j - 1$ Zeichen des Musters überein.
- 2. Das gerade gelesene i -te Zeichen im Text ist verschieden vom j -ten Zeichen im Muster.

Mit welchem Zeichen im Muster kann man das i -te Textzeichen als nächstes vergleichen, so daß man kein Vorkommen des Musters im Text übersieht?

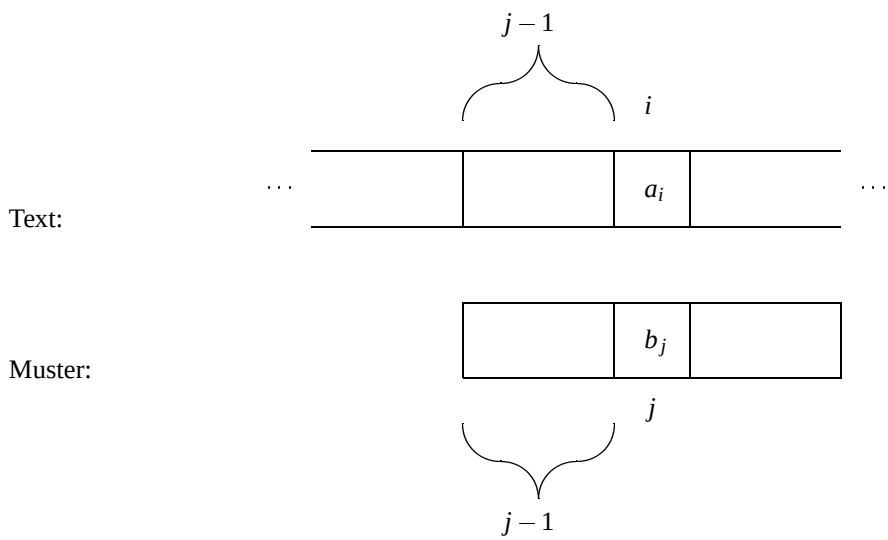


Abbildung 9.2

Dazu muß man offenbar von dem Anfangsstück des Musters mit Länge $j - 1$ ein Endstück maximaler Länge l bestimmen, das ebenfalls Anfangsstück des Musters ist. Dann ist die Position $l + 1$ im Muster, die wir $next[j]$ nennen wollen, die von rechts her nächste Stelle im Muster, die man mit dem i -ten Zeichen im Text mit der Chance auf Übereinstimmung vergleichen muß.

Falls im Vergleich des i -ten Zeichens im Text mit dem Zeichen an Position $next[j]$ im Muster kein Mismatch mehr auftritt, verschiebt man den Zeiger im Text und im Muster um eine Position nach rechts und vergleicht die Zeichen an den Positionen $i + 1$ und $next[j] + 1$ in Text und Muster.

Falls im Vergleich des i -ten Zeichens im Text mit dem Zeichen an Position $next[j]$ im Muster jedoch erneut ein Mismatch auftritt, gehen wir entsprechend vor: Wir bestimmen die Länge l' des längsten echten Endstücks des Anfangsstücks mit Länge $next[j] - 1$, das zugleich Anfangsstück des Musters ist, und vergleichen das i -te Zeichen im Text mit dem Zeichen an Position $l' + 1 = next[next[j]]$. Falls immer noch ein Mismatch auftritt, muß man wie beschrieben fortfahren, d.h.

$$next[next[\dots next[j] \dots]]$$

bestimmen. Es müssen also immer wieder für Anfangsstücke des Musters Endstücke bestimmt werden, die selbst Anfangsstücke maximaler Länge des Musters sind. Das i -te Zeichen im Text muß der Reihe nach mit den Zeichen an den Positionen j , $next[j]$, $next[next[j]] \dots$ im Muster verglichen werden. Das geschieht solange, bis erstmals kein Mismatch mehr auftritt oder man an der Position 1 im Muster angekommen ist. Im letzten Fall kann man offenbar den Textzeiger i um eine Position nach rechts verschieben und das Zeichen an Position $i + 1$ mit dem ersten Zeichen im Muster vergleichen.

Es gilt also für jedes j mit $2 \leq j \leq M$, M Länge des Musters:

$next[j] = 1 + \text{Länge des längsten echten Endstücks der ersten } j - 1 \text{ Zeichen,}$
das zugleich Anfangsstück des Musters ist.

Wir setzen noch $next[1] = 0$. Nehmen wir nun an, daß das $next$ -Array bekannt ist, so kann das Verfahren von Knuth-Morris-Pratt wie folgt beschrieben werden:

```

function kmp_search ( $a, b : \text{string}; M, N : \text{integer}$ ) :  $\text{integer}$ ;
var  $i, j : \text{integer}$ ;
begin
     $i := 1$ ;
     $j := 1$ ;
    repeat
        if  $a_i = b_j$  or  $j = 0$ 
        then
            begin
                 $i := i + 1$ ;
                 $j := j + 1$ 
            end
        else
             $j := next[j]$ 
        until ( $j > M$ ) or ( $i > N$ );
        if  $j > M$ 
        then  $kmp\_search := i - M$ 
        else  $kmp\_search := i$ 
    end

```

Man kann aus dieser Formulierung des Verfahrens unmittelbar ablesen, daß der Zeiger i , der auf die jeweils nächste zu inspizierende Stelle im Text weist, nie zurückgesetzt wird. Der Zeiger j kann natürlich zurückgesetzt werden. Mit jeder Zuweisung $j := next[j]$ verringert sich der Wert von j um wenigstens 1; für $j = 1$ wird $next[j] = 0$ und $j = 0$ und damit beim nächsten Durchlauf der **repeat**-Schleife sowohl i als auch j um 1 erhöht. j kann natürlich insgesamt nur so oft herabgesetzt werden, wie es erhöht wurde. Da jedoch i und j innerhalb der **repeat**-Schleife stets gemeinsam erhöht werden, kann j insgesamt nur so oft herabgesetzt werden, wie i heraufgesetzt wurde. Weil die **repeat**-Schleife für $i > N$ abbricht, folgt, daß die Anweisung $j := next[j]$ insgesamt höchstens N -mal ausgeführt wird. Nimmt man also an, daß das $next$ -Array bekannt ist, so benötigt das Verfahren $O(N)$ Schritte.

Wir müssen jetzt noch angeben, wie man die Belegung des *next*-Arrays berechnet. Das geschieht durch ein Programm, das eine ganz ähnliche Struktur hat wie das bereits angegebene Verfahren *kmp_search*. Darin kommt zum Ausdruck, daß wir das Muster mit sich selbst vergleichen.

```

procedure initnext;
  var i, j : integer;
  begin
    i := 1;
    j := 0;
    next[i] := 0;
    repeat
      if  $b_i = b_j$  or  $j = 0$ 
      then
        begin
          i := i + 1;
          j := j + 1;
          next[i] := j
        end
      else
        j := next[j]
      until  $i > M$ 
    end

```

Das *next*-Array muß für alle i , $2 \leq i \leq M$, M Länge des Musters $B = b_1 \dots b_M$, so belegt werden, daß gilt: Ist $next[i] = j$, so ist $j - 1$ die Länge des längsten echten Endstücks des Anfangsstücks mit Länge $i - 1$, das zugleich Anfangsstück des Musters ist. Zunächst wird $next[1] = 0$ gesetzt, wie wir das im Verfahren *kmp_search* verlangt haben. Nehmen wir jetzt an, daß $next[1], \dots, next[i]$ im angegebenen Sinne bereits richtig belegt wurden. Nach Ausführung der Zuweisung $next[i] := j$ ist also $j - 1$ die Länge des längsten echten Endstücks des Anfangsstücks mit Länge $i - 1$, das zugleich Anfangsstück des Musters ist. Wir vergleichen nun b_i und b_j .

Fall 1: $b_i = b_j$

Dann kennen wir das längste echte Endstück des Musters im Anfangsstück mit Länge i , das zugleich Anfangsstück des Musters ist. Es enthält das nächste Zeichen b_i bzw. b_j und hat damit die Länge j . Nach Definition ist folglich $next[i + 1] = j + 1$.

Fall 2: $b_i \neq b_j$

Zur Bestimmung des längsten echten Endstücks des Anfangsstücks mit Länge i des Musters, das zugleich Anfangsstück des Musters ist, müssen wir genauso vorgehen, wie wir das beim Vergleich des Musters mit dem Text getan haben (i ist dabei Textzeiger, j Musterzeiger). Wir vergleichen der Reihe nach b_i mit den Zeichen an den Positionen $next[j]$, $next[next[j]] \dots$, bis erstmals eine Übereinstimmung mit b_i erreicht wurde oder der Zeiger j bei 0 angekommen ist. Im letzten Fall wissen wir, daß das leere Wort das längste echte Endstück des Anfangsstücks des Musters mit Länge i ist, das zugleich Anfangsstück des Musters ist, und wir können $next[i + 1] = 1$ setzen. Sonst sei j' die Position, für die erstmals eine Übereinstimmung bei b_i festgestellt wurde. Dann müssen wir setzen: $next[i + 1] = j' + 1$.

Wieviele Schritte benötigt das oben angegebene Verfahren zur Bestimmung des *next*-Arrays? i und j durchlaufen Positionen im Muster, dabei wird i nur erhöht, während j entweder erhöht oder wieder herabgesetzt wird. j kann natürlich insgesamt nur so oft herabgesetzt werden, wie es erhöht wurde. Da j jedoch stets nur gemeinsam mit i erhöht wird, die Schleife aber bei $i > M$ abbricht, gilt: Die Gesamtzahl aller Ausführungen der Anweisung $j := \text{next}[j]$ in allen Schleifendurchläufen der **repeat**-Schleife ist höchstens M . Damit folgt, daß das *next*-Array in $O(M)$ Schritten bestimmt werden kann. Das Verfahren von Knuth-Morris-Pratt benötigt also insgesamt höchstens $O(M + N)$ Schritte, um ein Muster mit Länge M in einem Text mit Länge N zu finden.

Die Existenz eines Verfahrens zur Textsuche mit Zeitkomplexität $O(M + N)$, statt $\Theta(M \cdot N)$ wie beim naiven Verfahren, folgt aus einem allgemeinen Satz von S. Cook über die Simulierbarkeit von gewissen Automaten. Die wichtigste Referenz für die von uns angegebene Version des Verfahrens ist . Man kann das Verfahren auch in einem automatentheoretischen Gewand präsentieren: Zu einem gegebenen Muster wird ein endlicher Automat konstruiert, der den gegebenen Text liest und genau dann in einen ausgezeichneten Endzustand übergeht, wenn ein Vorkommen des Musters im Text gefunden wurde. Die Zustände des Automaten repräsentieren, welches Anfangsstück des Musters bereits entdeckt wurde. Diese Darstellung des Verfahrens wurde z.B. in gewählt. Eine Erweiterung auf die gleichzeitige Suche nach mehreren Mustern findet man in .

9.1.3 Das Verfahren von Boyer-Moore

Bei dem Verfahren von Boyer und Moore werden die Zeichen im Muster nicht von links nach rechts, sondern von rechts nach links mit den Zeichen im Text verglichen. Man legt das Muster zwar der Reihe nach an von links nach rechts wachsende Textpositionen an, beginnt aber einen Vergleich zwischen Zeichen im Text und Zeichen im Muster immer beim letzten Zeichen im Muster. Tritt dabei kein Mismatch auf, hat man ein Vorkommen des Musters im Text gefunden. Tritt jedoch ein Mismatch auf, so wird eine Verschiebung des Musters berechnet, d.h. eine Anzahl von Positionen, um die man das Muster nach rechts verschieben kann, bevor ein erneuter Vergleich zwischen Muster und Text, wieder beginnend mit dem letzten Zeichen im Muster, durchgeführt wird. In vielen Fällen ist es möglich, das Muster um große Distanzen nach rechts zu verschieben und so nur einen Bruchteil der Textzeichen zu inspizieren. Betrachten wir als Beispiel noch einmal den Text aus Abbildung 9.1. Wird das Muster an die erste Textposition angelegt, so wird zuerst das Textzeichen *s* mit dem letzten Zeichen des Musters verglichen. Es tritt ein Mismatch auf. Da das Textzeichen *s* im Muster überhaupt nicht vorkommt, kann man das Muster gleich um die Musterlänge, also um vier Positionen nach rechts verschieben. Falls das den Mismatch verursachende Zeichen doch im Muster auftritt, wie z.B. in folgender Situation

... abrakadabra ...
 aber

Für die meisten Zeichen c des Alphabets ist $\text{delta-1}(c) = M$. Falls c im Muster $B = b_1 \dots b_M$ vorkommt, ist $\text{delta-1}(c)$ der Abstand des rechtensten Vorkommens von c in B vom Musterende.

Natürlich ist man eigentlich nicht an der Verschiebung des Musters, sondern an der möglichen Verschiebung des Textzeigers nach rechts interessiert. Ferner möchte man nach Auftreten eines Mismatches den Textzeiger auf jeden Fall über die Position hinaus nach rechts verschieben, an der man zuletzt begonnen hat, Zeichen in Muster und Text von rechts nach links zu vergleichen. In jedem Fall kann man die Verschiebung des Textzeigers aus dem delta-1 -Wert berechnen. Sei c das den Mismatch verursachende Zeichen, und seien i und j die aktuellen Positionen im Text und im Muster.

Fall 1: $[M - j + 1 > \text{delta-1}(c)$, siehe Abbildung 9.4]

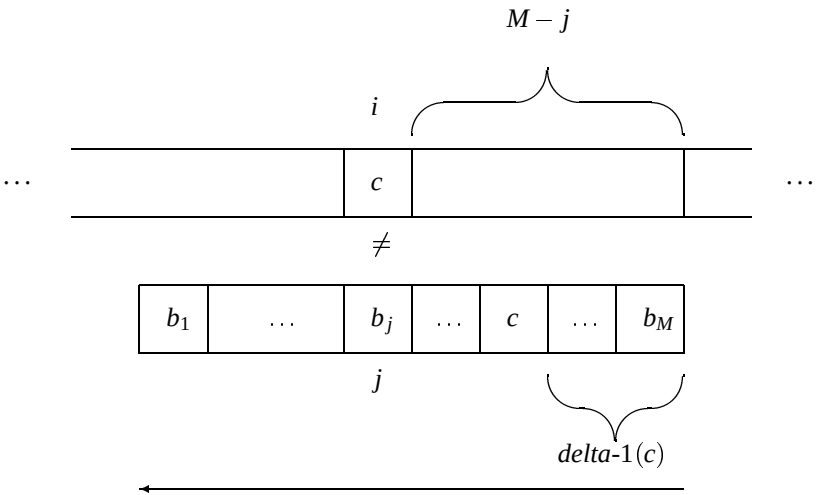


Abbildung 9.4

Wir setzen $i := i + M - j + 1$; $j := M$. Dies ist eine besonders einfache Version des Verfahrens von Boyer-Moore. Denn durch die Ersetzung $i := i + M - j + 1$ wird das Muster gegenüber seiner vorherigen Position ja nur um eine Position nach rechts verschoben erneut an den Text angelegt. Offensichtlich könnte man sich noch zusätzlich die Information zunutze machen, daß das den Mismatch verursachende Zeichen c rechts von dem im Muster auftretenden c an Position $\text{delta-1}(c)$ von rechts nicht vorkommt. Daher könnte man i sogar auf den größeren Wert $i + M - j + \text{delta-1}(c)$ setzen.

Fall 2: $[M - j + 1 \leq \text{delta-1}(c)$, siehe Abbildung 9.5]

Setze $i := i + \text{delta-1}(c)$; $j := M$.

Denken wir uns die delta-1 -Tabelle gegeben, so kann eine dieser Vorkommens-Heuristik folgende, vereinfachte Version des Verfahrens von Boyer-Moore wie folgt beschrieben werden:

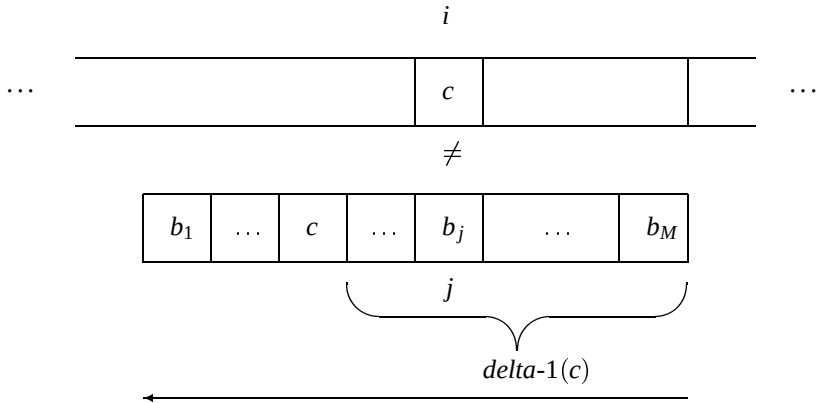


Abbildung 9.5

```
function bmeinfach (a, b : string; M, N : integer) : integer;
var i, j : integer;
begin
  i := M;
  j := M;
  repeat
    if ai = bj
      then
        begin
          i := i - 1;
          j := j - 1;
        end
      else {Mismatch verursacht durch ai; Textzeiger entsprechend Fall 1
            oder Fall 2 heraufsetzen; Musterzeiger an das Ende
            des Musters}
        begin
          if M - j + 1 > delta-1(ai)
            then i := i + M - j + 1
            else i := i + delta-1(ai);
          j := M;
        end
      until (j < 1) or (i > N);
    bmeinfach := i + 1
  end
```

Es ist leicht zu sehen, daß diese vereinfachte Version des Verfahrens von Boyer-Moore im schlechtesten Fall nicht besser ist als das naive Verfahren zur Textsuche, also $\Omega(NM)$ Schritte benötigt. (Man betrachte ein Muster 10...0 mit Länge M und durchsuche einen aus lauter Nullen bestehenden Text nach diesem Verfahren.)

Von Boyer und Moore wurde daher in  eine zweite Heuristik zur Berechnung der möglichen Verschiebung des Musters benutzt, die sogenannte *Match-Heuristik*. Ähnlich wie beim Verfahren von Knuth-Morris-Pratt nutzt diese Heuristik die Information über den bis zum Auftreten des Mismatch bereits inspizierten, mit einem Endstück des Musters übereinstimmenden Text. Betrachten wir dazu folgendes Beispiel:

Text: orange ananas banana ...
Muster: banana

Nehmen wir also an, daß die letzten m Zeichen im Muster mit den darüberstehenden m Zeichen im Text übereinstimmen und an der Position j der von rechts her erste Mismatch auftritt. Wir wollen die letzten m Zeichen das *Submuster* des Musters (für dieses m und j) nennen. Wir suchen dann von rechts her im Muster nach einem weiteren Vorkommen des Submusters. Haben wir ein solches Vorkommen gefunden, so können wir das Muster so weit nach rechts verschieben, daß das weitere Vorkommen des Submusters im Muster dem Vorkommen des Submusters im Text gegenübersteht. Diesem zweiten Vorkommen des Submusters im Muster darf natürlich nicht das gleiche Zeichen vorangehen wie dem ersten, denn sonst würde dieses Zeichen sicher wieder einen Mismatch verursachen.

Im oben angegebenen Beispiel kommt das Submuster **ana** im Muster **banana** noch einmal vor, und das dem zweiten Vorkommen vorangehende Zeichen **b** ist verschieden von dem Zeichen **n**, das dem ersten Vorkommen vorangeht:

banana
 banana

Wir können das Muster also um zwei Positionen nach rechts verschieben und fortfahren, von rechts her Zeichen im Muster mit Zeichen im Text zu vergleichen, beginnend mit dem letzten Zeichen im Muster.

Es bezeichne $wrw(j)$ die Position, an der das von rechts her nächste Vorkommen des Submusters beginnt. Dabei ist j die Position, an der der erste Mismatch auftrat, also $b_{j+1} \dots b_M$ das Submuster. Es wird angenommen, daß das dem zweiten Vorkommen des Submusters vorangehende Zeichen, also das Zeichen an Position $wrw(j) - 1$, vom Zeichen b_j verschieden ist.

Im obigen Beispiel ist $j = 3$, denn das dritte Zeichen **n** im Muster **banana** hat den Mismatch verursacht. $wrw(3) = 2$, denn das von rechts her nächste Vorkommen des Submusters **ana** beginnt an Position 2 im Muster.

Eine Funktion $\text{delta-2}(j)$ gibt an, um wieviele Positionen der Zeiger i auf das aktuelle Zeichen im Text nach rechts verschoben werden kann, wenn der erste Mismatch im Muster an Position j auftrat. Der Vergleich der Zeichen in Muster und Text beginnt nach jedem Mismatch jeweils neu mit dem letzten Zeichen des Musters. Es muß daher jeweils nur berechnet werden, um welche Distanz der Zeiger i im Text bewegt werden kann. Durch Verschieben nach rechts um $m = M - j$ Positionen wird der Zeiger i an das dem letzten Zeichen im Muster gegenüberliegende Zeichen im Text bewegt; das Muster kann jetzt um $j + 1 - wrw(j)$ Positionen nach rechts bewegt werden. Der Textzeiger muß noch um denselben Betrag erhöht werden. Insgesamt ergibt sich also, daß nach Auftreten eines Mismatches an Position j der Textzeiger um $\text{delta-2}(j)$ Positionen nach rechts bewegt werden kann, mit

$$\text{delta-2}(j) = M + 1 - wrw(j).$$

Wir berechnen die Werte von $wrw(j)$ und $\text{delta-2}(j)$ für $j = 5, 4, 3, 2, 1$ und das oben angegebene Beispiel des Musters **banana**. Sei zunächst $j = 5$; das an Position $j + 1 = 6$ beginnende Submuster **a** tritt im Muster noch zwei weitere Male auf. Dem von rechts her nächsten **a** geht aber das gleiche Zeichen voran wie dem **a** an Position 6. Es ist daher $wrw(5) = 2$ und $\text{delta-2}(5) = 5$.

Sei nun $j = 4$; das an Position 5 beginnende Submuster **na** kommt noch einmal vor; beiden Vorkommen geht aber dasselbe Zeichen **a** voran. Innerhalb des Musters gibt es also überhaupt kein weiteres Vorkommen des Submusters mit der verlangten Eigenschaft. Denkt man sich aber das Muster nach links um „don't care“-Symbole fortgesetzt und setzt $wrw(4) = -1$, so ergibt sich für $\text{delta-2}(4)$ der Wert 8, also genau der Wert, um den man den Textzeiger nach rechts verschieben muß, wenn man das Muster um M Positionen nach rechts verschieben kann und an Position $j = 4$ ein Mismatch auftrat. Den Wert $wrw(3) = 2$ haben wir schon begründet. Damit ergibt sich $\text{delta-2}(3) = 5$. Sei schließlich $j = 2$. Das an Position 3 beginnende Submuster kommt im Muster nicht noch einmal vor. Es ist $wrw(j) = -3$:

Position j :		-4	-3	-2	-1	0	1	2	3	4	5	6
Muster:	...	*	*	*	*	*	b	a	n	a	n	a
Submuster:			n	a	n	a						

Damit ergibt sich $\text{delta-2}(2) = 10$. Auf ähnliche Weise erhält man $wrw(1) = -4$ und $\text{delta-2}(1) = 11$.

Offenbar hängt die delta-2 -Tabelle nur vom Muster ab und kann ganz ähnlich berechnet werden wie im Verfahren von Knuth-Morris-Pratt, indem man das Muster gewissermaßen über sich selbst hinwegschiebt. Für jedes j , $1 \leq j < M$, enthält $\text{delta-2}(j)$ als Wert die Distanz, um die man den Textzeiger i nach rechts schieben muß, wenn beim Vergleich des Zeichens an Position i im Text ein Mismatch mit dem Zeichen an Position j im Muster aufgetreten ist.

Das Verfahren von Boyer und Moore in der ursprünglich angegebenen Version benutzt beide Heuristiken zur Berechnung der Verschiebung des Musters und folgt jeweils der, die den größeren Wert liefert. Denken wir uns also die delta-1 -Tabelle und die delta-2 -Tabelle gegeben, so kann man das Verfahren wie folgt formulieren:

```

function boyermoore ( $a, b : \text{string}; M, N : \text{integer}$ ) :  $\text{integer}$ ;
var  $i, j : \text{integer}$ ;
begin
     $i := M$ ;
     $j := M$ ;
    repeat
        if  $a_i = b_j$ 
        then
            begin
                 $i := i - 1$ ;
                 $j := j - 1$ 
            end
        else {Mismatch; Muster verschieben}

```

```

begin
     $i := i + \max\{\text{delta-1}(a_i) + 1, \text{delta-2}(j)\};$ 
     $j := M$ 
end
until  $(j < 1)$  or  $(i > N);$ 
     $\text{boyermoore} := i + 1$ 
end

```

Man kann sich leicht überlegen, daß nach Auftreten eines Mismatches das Muster stets um wenigstens eine Position nach rechts verschoben wird, also der Textzeiger i um wenigstens $M - j + 1$ Positionen, wenn ein Mismatch an Position j im Muster auftrat. Die bei der vereinfachten Version des Verfahrens von Boyer-Moore gemachte Fallunterscheidung ist also jetzt entbehrlich.

Die verwendeten Tabellen delta-1 und delta-2 hängen nur vom Alphabet und vom gegebenen Muster ab. Wie in  gezeigt wurde, trägt die delta-2 -Tabelle zur Schnelligkeit des Algorithmus in der Praxis kaum etwas bei. Der einzige Zweck dieser Tabelle ist es, Muster mit mehrfach auftretenden Submustern optimal zu nutzen und eine Laufzeit des Verfahrens von $\Theta(M \cdot N)$ im schlechtesten Fall zu verhindern. Weil Muster mit wiederholt auftretenden Submustern aber relativ selten vorkommen, insbesondere, wenn die Muster kurz sind, kann man auf die delta-2 -Tabelle auch ganz verzichten.

Wir haben das Verfahren von Boyer-Moore so formuliert, daß der Algorithmus hält, wenn das erste Vorkommen des Musters im Text gefunden wurde. Die Laufzeit dieses Verfahrens beträgt $O(M + N)$. Natürlich ist es einfach, das Verfahren so zu verändern, daß es alle r Vorkommen des Musters im Text findet. Die Laufzeit beträgt dann $O(N + rM)$.

In der Praxis hat sich die vereinfachte Version des Verfahrens von Boyer-Moore ausgezeichnet bewährt. Man kann erwarten, daß das Verfahren für genügend kurze Muster und hinreichend große Alphabete etwa $O(N/M)$ Schritte durchführt, d.h. das Verfahren inspiziert nur jedes M -te Textzeichen und das Muster kann nahezu immer um die gesamte Musterlänge nach rechts verschoben werden.

9.1.4 Signaturen

Die von uns angegebenen Verfahren zur Suche in Texten benutzen als einzige Grundoperation den Vergleich von Zeichen im Muster und Zeichen im Text. Man kann Zeichen und Zeichenketten aber auch Zahlen zuordnen und Algorithmen entwerfen, die diese Zuordnung nutzen und arithmetische Operationen verwenden. Eine sehr einfache Möglichkeit besteht darin, *jedem* Teilstring des Textes mit Länge M durch eine Hashfunktion h eine Zahl zuzuordnen. Ist dann h so beschaffen, daß Adreßkollisionen sehr unwahrscheinlich sind, so hat man ein Vorkommen des Musters gefunden, wenn der Wert der Hashfunktion h für einen Teilstring mit Länge M gleich dem Wert $h(b_1 \dots b_M)$ des Musters ist. Man berechnet also zu jedem Teilstring mit Länge M ein h -Bild als *Signatur* des Textes. Weil man nur einen einzigen h -Wert sucht, muß man die h -Werte, also die Hashtafel, natürlich nicht speichern. Attraktiv wird ein Verfahren zur Textsuche über die Berechnung von Signaturen natürlich erst dann, wenn die Berechnung der

Signatur einfach und zwar inkrementell möglich ist. D.h. der h -Wert von zwei aufeinanderfolgenden Teilstrings mit Länge M sollte sich wie folgt berechnen lassen:

$$\dots \overline{a_{i+1}a_{i+2} \dots a_{i+M}a_{i+M+1}} \dots$$

$h(a_{i+2} \dots a_{i+M+1})$ ist eine einfache Funktion von $h(a_{i+1} \dots a_{i+M})$. Ein Verfahren dieser Art wurde erstmals von Karp und Rabin angegeben. Sie fassen eine Zeichenkette mit Länge M als d -adische Zahl auf, wobei d die Alphabetgröße ist, und benutzen als Hashfunktion die Funktion $h(k) = k \bmod p$ für eine geeignet gewählte, große Primzahl p . Man kann dann zeigen, daß das Verfahren von Karp und Rabin mit hoher Wahrscheinlichkeit nur $O(M + N)$ Schritte benötigt.

Gonnet und Baeza-Yates haben Verfahren zur Textsuche angegeben, bei denen die Berechnung der Signatur nur noch vom gegebenen Muster abhängt. Ihre Verfahren lassen sich leicht über die reine Textsuche (exact match) hinaus ausdehnen auf den Fall, daß auch „don't care“-Symbole, Komplementärsymbole (wie z.B. $\bar{c}$ zur Bezeichnung aller Zeichen, die von c verschieden sind) und mehrfache Muster in Suchanfragen vorkommen.

9.1.5 Approximative Zeichenkettensuche

Das Problem, in einem gegebenen Text alle Vorkommen eines gegebenen Musters zu finden, kann auf naheliegende Weise zum k -Mismatch-Problem verallgemeinert werden: Gegeben sind ein Text $a_1 \dots a_N$, ein Muster $b_1 \dots b_M$ und eine Zahl k , $0 \leq k < M$. Gesucht sind alle Vorkommen von Mustern $b'_1 \dots b'_M$ der Länge M im Text derart, daß sich $b_1 \dots b_M$ und $b'_1 \dots b'_M$ an höchstens k Positionen unterscheiden.

Für $k = 0$ ist dies das uns bereits bekannte Textsuchproblem, das wir mit Hilfe verschiedener, in den vorangehenden Abschnitten vorgestellter Algorithmen lösen können. Als Beispiel für den Fall $k = 2$ betrachten wir verschiedene Textstücke mit acht Buchstaben, die mit dem Muster mismatch verglichen werden. Ein Vergleich des jeweiligen Textstücks mit dem Muster führt zu einem positiven Ergebnis, wenn das Muster und das jeweilige Textstück an höchstens zwei Stellen verschiedene Buchstaben haben.

Muster:	mismatch	
Text 1:	miscatch	<u>ja</u>
Text 2:	dispatch	<u>ja</u>
Text 3:	respatch	<u>nein</u>

Das naive Verfahren zur Textsuche kann leicht auf diesen allgemeineren Fall ausgedehnt werden: Man legt das Muster der Reihe nach an jeder Position des Textes beginnend an, vergleicht zeichenweise von links nach rechts, ob eine Übereinstimmung zwischen Muster und Text vorliegt, und zählt die Anzahl der aufgetretenen Nichtübereinstimmungen (Mismatches). In Pascal-ähnlicher Notation kann das Verfahren so beschrieben werden:

```

procedure mismatch ( $a, b : \text{string}; N, M, k : \text{integer}$ );
{liefert alle Positionen im Text  $a[1 \dots N]$ , an denen ein Vorkommen des
Musters  $b[1 \dots M]$  mit höchstens  $k$  Mismatches beginnt}
var  $i, j, m : \text{integer}$ ;
begin
  for  $i := 1$  to  $N - M + 1$  do
    begin
       $m := 0$ ;
      for  $j := 1$  to  $M$  do
        if  $a_{i+j-1} \neq b_j$  then  $m := m + 1$ ;
        if  $m \leq k$ 
          then write('höchstens',  $m$ , 'Mismatches an Position',  $i$ )
      end
    end
  end

```

Es ist offensichtlich, daß das Verfahren Zeit $\Theta(M \cdot N)$ benötigt.

Wie im Falle der exakten Zeichenkettensuche, also wie für den Spezialfall des 0-Mismatch-Problems, kann man auch das k -Mismatch-Problem für $k > 0$ dadurch effizienter zu lösen versuchen, daß man etwa die Verfahren von Knuth-Morris-Pratt oder Boyer-Moore geeignet verallgemeinert. Überlegungen dazu findet man beispielsweise in .

In Anwendungen bei Texteditoren oder bei der „Dekodierung“ von DNA-Sequenzen in der Biologie viel wichtiger ist aber eine andere Verallgemeinerung des Textsuchproblems: Statt einfach die Anzahl der Buchstaben zu zählen, die verschieden sind, prüft man, wieviele Buchstaben eingefügt, gelöscht oder geändert werden müssen, um eine Übereinstimmung zwischen Text und Muster herzustellen. Das führt zum Begriff der *Editier-* (oder: *Evolutions-*)distanz und zu Algorithmen für die approximative Zeichenkettensuche, die auf dem algorithmischen Prinzip des *dynamischen Programmierens* beruhen. Das ist die Methode, immer größere optimale Teillösungen eines Problems iterativ „von unten nach oben“ zu berechnen, d.h. angefangen bei optimalen Lösungen von „trivialen“ Anfangsproblemen bis zur optimalen Gesamtlösung. Wir haben diese Methode bereits für die Konstruktion optimaler Suchbäume im Abschnitt 5.7 benutzt.

Editierdistanz

Wir wollen die folgenden *Editier-Operationen* zur Veränderung von Zeichenreihen zulassen: *Löschen*, *Einfügen* und *Ändern* eines einzelnen Symbols an einer bestimmten Stelle. Wir können diese Operationen als „Ersetzungsregeln“ in der Form $\alpha \rightarrow \beta$ mitteilen, wobei α und β Buchstaben des zugrunde liegenden Alphabets Σ oder aber das Zeichen ε für das leere Wort sind. Die Veränderung einer Zeichenkette A durch eine Editier-Operation $\alpha \rightarrow \beta$ bedeutet dann, daß ein Vorkommen von α in A durch β ersetzt wird. Da das leere Wort ε „überall“ in A vorkommt, heißt das insbesondere, daß eine Einfüge-Operation $\varepsilon \rightarrow a$ das Einfügen eines Zeichens a an jeder Position von A erlaubt.

Jeder Editier-Operation $\alpha \rightarrow \beta$ werden nichtnegative Kosten $c(\alpha \rightarrow \beta)$ zugeordnet. Man interessiert sich insbesondere für den Fall, daß die Kosten jeder Editier-Operation einheitlich gleich 1 gewählt werden (Einheitskosten-Modell). Im Einheitskosten-

Modell gilt also für zwei beliebige Zeichen $a, b \in \Sigma$, $a \neq b$: $c(a \rightarrow b) = c(\varepsilon \rightarrow b) = c(a \rightarrow \varepsilon) = 1$ und natürlich $c(a \rightarrow a) = 0$.

Sind nun zwei Zeichenketten $A = a_1 \dots a_m$ und $B = b_1 \dots b_n$ gegeben, so definieren wir als *Editierdistanz* $D(A, B)$ die minimalen Kosten, die eine Folge von Editier-Operationen hat, die A in B überführt.

Beispiel für eine Folge von Editier-Operationen, die **auto** in **rad** überführt:

auto	Operation $u \rightarrow \varepsilon$ an Position 2 liefert
ato	Operation $t \rightarrow d$ an Position 2 liefert
ado	Operation $o \rightarrow \varepsilon$ an Position 3 liefert
ad	Operation $\varepsilon \rightarrow r$ an Position 0 liefert
rad	

Im Einheitskosten-Modell hat diese Folge von Editier-Operationen die Kosten 4. Es ist nicht schwer zu sehen, daß es keine Folge von Editier-Operationen mit geringeren Kosten gibt, die **auto** in **rad** überführt. Daher ist $D(\text{auto}, \text{rad}) = 4$.

Es ist üblich anzunehmen, daß ein durch eine Editier-Operation einmal eingefügtes, gelöscht oder geändertes Zeichen nicht nochmals verändert, also gelöscht, eingefügt oder geändert wird. Diese Annahme gilt für die Folge der Editier-Operationen mit minimal möglichen Kosten sicher dann, wenn die Kostenfunktion für die Editier-Operationen eine „Dreiecksungleichung“ erfüllt, d.h. wenn gilt

$$c(\alpha \rightarrow \gamma) \leq c(\alpha \rightarrow \beta) + c(\beta \rightarrow \gamma),$$

falls $\alpha \neq \beta \neq \gamma$, und $c(\alpha \rightarrow \beta) > 0$, falls $\alpha \neq \beta$. Das ist insbesondere im Einheitskosten-Modell erfüllt.

Zwei Probleme sind im Zusammenhang mit Editierdistanzen von besonderem Interesse:

Problem 1: (Berechnung der Editierdistanz)

Berechne für zwei gegebene Zeichenketten A und B möglichst effizient die Editierdistanz $D(A, B)$ und eine kostenminimale Folge von Editier-Operationen, die A in B überführt.

Problem 2: (Approximative Zeichenkettensuche)

Gegeben seien ein Text A und ein Muster B sowie eine Zahl $k \geq 0$. Gesucht sind alle Vorkommen von Zeichenreihen B' in A , so daß $D(B, B') \leq k$ ist.

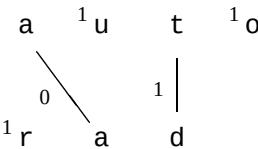
Für $k = 0$ ist Problem 2 natürlich wieder das gewöhnliche Zeichenketten-Suchproblem. Das k -Mismatch-Problem kann als Spezialfall von Problem 2 aufgefaßt werden, wenn man nur Änderungen von Zeichen, also weder Einfügen noch Löschen von Zeichen zuläßt.

Wir behandeln zunächst Verfahren zur Lösung von Problem 1 und werden dann sehen, daß dabei verwendete Methoden auch zur Lösung von Problem 2 benutzt werden können. Dabei setzen wir zur Vereinfachung stets das Einheitskosten-Modell voraus und überlassen es dem Leser, sich zu überlegen, wie Verfahren auf den Fall unterschiedlicher Kosten ausgedehnt werden können.

Berechnung der Editierdistanz

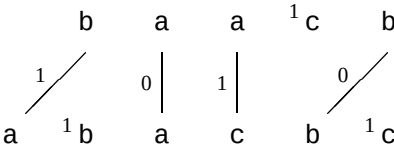
Eine Folge von Editier-Operationen mit minimalen Kosten, die eine Zeichenreihe A in eine andere Zeichenreihe B überführt, ändert jedes von einer Operation betroffene Zeichen höchstens einmal. Wir können uns daher auch vorstellen, daß die Operationen nicht nacheinander, sondern alle gleichzeitig ausgeführt werden. Das führt zum Begriff der *Spur* (englisch: *trace*), die A in B transformiert. Wir verzichten auf eine formal exakte Definition dieses Begriffs und verweisen dazu auf [1, 2]. Stattdessen teilen wir Spuren in folgender Weise graphisch mit: Wird auf ein Zeichen a in A eine Änderungs-operation $a \rightarrow b$ ausgeführt, so verbinden wir das Zeichen a in A (an der Position, an der diese Operation ausgeführt wird) mit dem entsprechenden Zeichen b in B durch eine Kante; die Kante wird mit 1 beschriftet, wenn $a \neq b$ ist, und mit 0 sonst. Ein Zeichen a in A , auf das eine Lösch-Operation $a \rightarrow \epsilon$ angewandt wird, erhält einen linken oberen Index 1; ein Zeichen b in B , das durch eine Einfüge-Operation $\epsilon \rightarrow b$ entstanden ist, erhält einen linken oberen Index 1. Die Summe der Indizes und Kantenbeschriftungen sind die Kosten der Spur.

Beispiel: Die oben angegebene Folge von vier Editier-Operationen, die $A = \text{auto}$ in $B = \text{rad}$ transformiert, kann zur folgenden Spur zusammengefaßt werden:

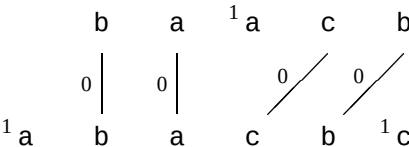


Aus der Annahme, daß zur Transformation von A in B jedes Zeichen höchstens einmal geändert werden darf, folgt, daß eine Spur keine sich kreuzenden Kanten enthalten kann. Statt alle Folgen von Editier-Operationen zu betrachten genügt es also, alle Spuren ohne sich kreuzende Kanten zu betrachten. Die Editierdistanz $D(A, B)$ ist gleich den Kosten einer *optimalen* Spur, also einer Spur mit minimalen Kosten, die A in B transformiert. Aus einer Spur kann man leicht eine Folge von Editier-Operationen ablesen, die A in B transformiert und die genau die Kosten der Spur hat.

Beispiel: Seien $A = \text{baacb}$ und $B = \text{abacbc}$.
Dann ist

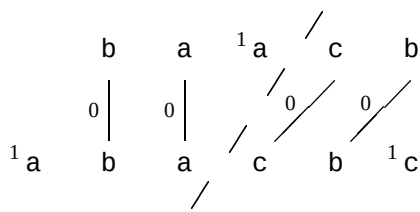


eine Spur mit Kosten 5. Das ist keine optimale Spur. Eine optimale Spur mit den Kosten 3 ist:



Aus dieser Spur kann man ablesen, daß Löschen von **a** an der Position 3 in *A*, dann Einfügen von **a** am Anfang und Einfügen von **c** am Ende *A* in *B* transformiert. Offenbar kann man aus jeder Spur, die *A* in *B* transformiert, auch sofort eine Spur erhalten, die umgekehrt *B* in *A* transformiert und dieselben Kosten hat. Man muß dazu nur alle Operationen, die *A* in *B* transformieren, umkehren. Daher ist klar, daß (im Einheitskosten-Modell) $D(A,B) = D(B,A)$ gilt.

Offenbar kann man Spuren in der Regel auf vielfältige Art teilen, so daß die Teile selbst wieder Spuren zur Transformation kürzerer Zeichenreihen sind. Beispielsweise kann man die (optimale) Spur



entlang der gestrichelten Linie teilen und erhält zwei Spuren, die **baa** in **aba** und **cb** in **cbc** transformieren.

Der Schlüssel zur Berechnung einer optimalen Spur nach der Methode der dynamischen Programmierung besteht nun in der Beobachtung, daß jede durch Teilung einer optimalen Spur entstandene Spur selbst wieder optimal sein muß. Denn wäre das nicht der Fall, dann könnte man durch Ersetzen eines nicht optimalen Teils einer optimalen Spur durch einen Teil mit geringeren Kosten die Gesamtkosten verringern, so daß die ursprünglich gegebene Spur nicht optimal gewesen sein kann. Daher kann man immer „längere“ optimale Spuren nach der Methode des dynamischen Programmierens aus „kürzeren“ berechnen. Genauer besteht das Verfahren zur Berechnung der Editierdistanz $D(A,B)$, also der Kosten einer optimalen Spur zur Transformation einer Zeichenreihe $A = a_1 \dots a_m$ in eine Zeichenreihe $B = b_1 \dots b_n$, darin, für jedes Paar (i,j) mit $0 \leq i \leq m$ und $0 \leq j \leq n$ die Kosten $D_{i,j}$ einer optimalen Spur zu berechnen, die $a_1 \dots a_i$ in $b_1 \dots b_j$ transformiert. Wir berechnen also für alle i, j mit $0 \leq i \leq m$ und $0 \leq j \leq n$

$$D_{i,j} = D(a_1 \dots a_i, b_1 \dots b_j).$$

Dabei ist das erste Argument von D das leere Wort, falls $i = 0$, und das zweite Argument von D das leere Wort, falls $j = 0$. Dann ist offenbar die gesuchte Editierdistanz $D(A,B) = D_{m,n}$.

Zunächst gilt offensichtlich

$$\begin{aligned} D_{0,0} &= D(\epsilon, \epsilon) = 0, \\ D_{0,j} &= D(\epsilon, b_1 \dots b_j) = j, \text{ für } 1 \leq j \leq n, \end{aligned}$$

da genau j Einfüge-Operationen in die (anfangs) leere Zeichenreihe erforderlich sind, um $b_1 \dots b_j$ zu erzeugen. Ferner ist

$$D_{i,0} = D(a_1 \dots a_i, \epsilon) = i, \text{ für } 1 \leq i \leq m,$$

da genau i Lösch-Operationen nötig sind, um aus $a_1 \dots a_i$ das leere Wort zu erzeugen.

Nun überlegen wir uns, wie wir für $i \geq 1$ und $j \geq 1$ den Wert $D_{i,j}$ aus $D_{i-1,j}$, $D_{i,j-1}$ und $D_{i-1,j-1}$ berechnen können. Dazu betrachten wir eine optimale Spur, die $a_1 \dots a_i$ in $b_1 \dots b_j$ transformiert. Am rechten Ende dieser Spur liegt dann einer der folgenden drei Fälle vor.

Fall 1: [Ändern: a_i und b_j sind durch eine Kante miteinander verbunden, die mit 1 beschriftet ist, falls $a_i \neq b_j$ ist, und mit 0, falls $a_i = b_j$ ist]

Läßt man diese Kante weg, so erhält man eine Spur mit minimalen Kosten $D_{i-1,j-1}$, die $a_1 \dots a_{i-1}$ in $b_1 \dots b_{j-1}$ transformiert:

$$\begin{array}{ccc} \cdots & a_{i-1} & | & a_i \\ & & | & | \\ \cdots & b_{j-1} & | & b_j \\ \underbrace{\hspace{1.5cm}} & & & \end{array}$$

Spur mit Kosten $D_{i-1,j-1}$

Für die Kosten $D_{i,j}$ gilt in diesem Fall

$$D_{i,j} = D_{i-1,j-1} + \begin{cases} 1; & \text{falls } a_i \neq b_j \\ 0; & \text{falls } a_i = b_j \end{cases}$$

Fall 2: [Einfügen: b_j ist nicht durch eine Kante mit einem Zeichen aus A verbunden]
Läßt man b_j weg, so erhält man eine Spur mit minimalen Kosten $D_{i,j-1}$, die $a_1 \dots a_i$ in $b_1 \dots b_{j-1}$ transformiert:

$$\begin{array}{ccc} \cdots & a_{i-1} & a_i & | \\ & & & | \\ \cdots & b_{j-1} & & | & {}^1b_j \\ \underbrace{\hspace{1.5cm}} & & & & \end{array}$$

Spur mit Kosten $D_{i,j-1}$

Für die Kosten $D_{i,j}$ gilt in diesem Fall

$$D_{i,j} = D_{i,j-1} + 1.$$

Fall 3: [Löschen: a_i ist nicht durch eine Kante mit einem Zeichen aus B verbunden]
Läßt man a_i weg, so erhält man eine Spur mit minimalen Kosten $D_{i-1,j}$, die $a_1 \dots a_{i-1}$ in $b_1 \dots b_j$ transformiert:

$$\begin{array}{ccc} \cdots & a_{i-1} & | & {}^1a_i \\ & & | & | \\ \cdots & b_{j-1} & b_j & | \\ \underbrace{\hspace{1.5cm}} & & & \end{array}$$

Spur mit Kosten $D_{i-1,j}$

Für die Kosten $D_{i,j}$ gilt in diesem Fall

$$D_{i,j} = D_{i-1,j} + 1.$$

Wir überlegen uns noch, daß dies alle zu betrachtenden Fälle sind. Weil eine Spurenkreuzungsfrei ist, kann es nicht vorkommen, daß a_i und b_j auf zwei verschiedenen Kanten liegen. Schließlich liegt wegen der Optimalität der Spur, die $a_1 \dots a_i$ in $b_1 \dots b_j$ transformiert, wenigstens eines der beiden Zeichen a_i und b_j auf einer Kante (andernfalls wäre ein Kante von a_i nach b_j billiger). Damit ist unsere Fallunterscheidung vollständig und eindeutig.

Wir erhalten insgesamt also die folgende Rekursionsformel für die gesuchten Werte $D_{i,j}$, $0 \leq i \leq m$, $0 \leq j \leq n$:

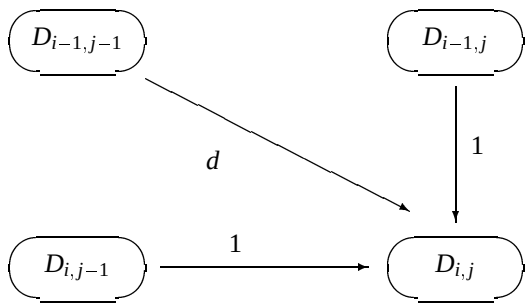
$$\begin{aligned} D_{0,0} &= 0 \\ D_{0,j} &= j, \quad \text{für } 1 \leq j \leq m, \\ D_{i,0} &= i, \quad \text{für } 1 \leq i \leq n, \end{aligned}$$

und für $0 < i \leq m$ und $0 < j \leq n$:

$$D_{i,j} = \min \left\{ D_{i-1,j-1} + \begin{cases} 1; & \text{falls } a_i \neq b_j \\ 0; & \text{falls } a_i = b_j \end{cases}, D_{i,j-1} + 1, D_{i-1,j} + 1 \right\}$$

Diese Darstellung zeigt, daß man die Werte $D_{i,j}$ z.B. zeilenweise oder spaltenweise und daher in Zeit $O(m \cdot n)$ und Platz $O(m)$ oder $O(n)$ berechnen kann. Die Editierdistanz $D(A, B) = D(a_1 \dots a_m, b_1 \dots b_n) = D_{m,n}$ kann man dann in der rechten unteren Ecke der Matrix $(D_{i,j})$ ablesen.

Man kann sich eine vollständige Übersicht über alle möglichen Spuren, die A in B transformieren, und über alle möglichen Wege zur Berechnung der $(m+1) \cdot (n+1)$ Werte $D_{i,j}$ mit Hilfe der angegebenen Rekursionsformel verschaffen. Dazu ordnet man jedem Paar (i, j) mit $0 \leq i \leq m$ und $0 \leq j \leq n$ einen (mit dem Wert $D_{i,j}$ beschrifteten) Knoten eines (planaren) Graphen zu; die Knoten werden in Form einer Matrix mit $m+1$ Zeilen und $n+1$ Spalten angeordnet. Um nicht zu viele Bezeichnungen einführen zu müssen, bezeichnen wir auch den Knoten an Position (i, j) mit $D_{i,j}$. Es ist aus dem Kontext stets eindeutig zu entnehmen, ob $D_{i,j}$ den Knoten an der Position (i, j) oder dessen Wert bezeichnet. Der Knoten in der linken oberen Ecke erhält den Wert 0. Die Knoten in der 0-ten Zeile sind jeweils durch eine waagerechte, mit 1 beschriftete Kante miteinander verbunden. Jede Kante repräsentiert eine Einfüge-Operation, die ausgehend vom anfangs leeren Wort das jeweils nächste Zeichen von B liefert. Daher erhalten die Knoten der 0-ten Zeile auch der Reihe nach die Werte $1, 2, \dots, n$. Entsprechend sind die Knoten der 0-ten Spalte jeweils durch eine senkrechte, mit 1 beschriftete Kante miteinander verbunden. Jede Kante repräsentiert eine Löschoption, die das jeweils nächste Zeichen von A löscht. Daher erhalten die Knoten der 0-ten Spalte der Reihe nach die Werte $1, 2, \dots, m$. Alle anderen Knoten werden nach folgendem Schema durch mit 0 oder 1 beschriftete Kanten miteinander verbunden:



Die Diagonalkante ist mit $d = 1$ beschriftet, falls $a_i \neq b_j$, und mit $d = 0$, falls $a_i = b_j$ ist. Sie repräsentiert also eine Änderungsoperation $a_i \rightarrow b_j$. Entsprechend repräsentiert die horizontale Kante eine Einfüge-Operation $\epsilon \rightarrow b_j$ und die senkrechte Kante eine Lösch-Operation $a_i \rightarrow \epsilon$. Abbildung 9.6 zeigt als Beispiel einen Graphen für $A = baac$ und $B = abac$.

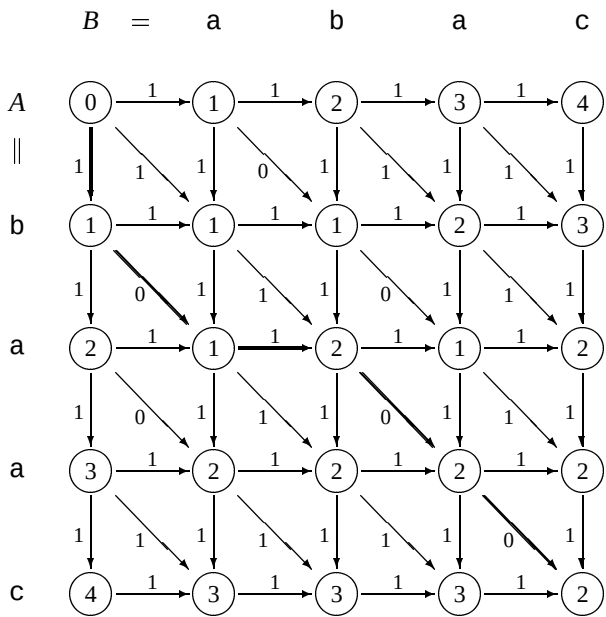


Abbildung 9.6

Man beachte, daß für $0 < i \leq m$ und $0 < j \leq n$ $D_{i,j}$ das Minimum der Werte $D_{i,j-1} + 1$, $D_{i-1,j} + 1$ und $D_{i-1,j-1} + d$ ist. Jedem Weg von der linken oberen zur rechten unteren Ecke entspricht eine Spur, die A in B transformiert. Umgekehrt entspricht auch jeder Spur ein solcher Weg. Wir nennen den resultierenden Graphen daher auch *Spurgraphen*.

Beispielsweise entspricht dem fett gezeichneten Weg des Spurgraphen in Abbildung 9.6 die folgende Spur:

$A =$

¹ baac

0

0

a¹ bac

Falls es sich, wie in diesem Beispiel, um eine optimale Spur handelt, sind die Werte der Knoten längs eines solchen Weges jeweils genau die Summen der Kantenbeschriftungen. Genau die Wege mit dieser Eigenschaft repräsentieren daher die optimalen Spuren und sämtliche Möglichkeiten zur Berechnung von $D_{m,n} = D(A,B)$.

In jedem Fall sind die Kosten einer Spur die Summe der Kantenbeschriftungen des die Spur repräsentierenden Weges im Spurgraphen.

Betrachten wir jetzt das Problem, für eine gegebene Zahl s festzustellen, ob $D_{m,n} \leq s$ ist. Natürlich kann man dieses Problem lösen, indem man alle $(m+1) \cdot (n+1)$ Werte $D_{i,j}$ im Spurgraphen berechnet und nachsieht, ob $D_{m,n} \leq s$ ist. Das ist aber nicht immer nötig. Denn jede horizontale und jede vertikale Kante eines eine Spur repräsentierenden Weges im Spurgraphen liefert den Beitrag 1 zu den Kosten der Spur. Die Gesamtkosten können also höchstens dann unterhalb der vorgegebenen Schranke s bleiben, wenn der Weg höchstens s horizontale und vertikale Kanten insgesamt enthält. Weil die Zahl der horizontalen und vertikalen Kanten, die man mindestens durchlaufen muß, um vom Knoten $D_{0,0}$ im Spurgraphen zum Knoten $D_{i,j}$ zu gelangen, gleich $|i-j|$ ist, folgt: Sobald $|i-j| > s$ ist, kann der Knoten $D_{i,j}$ auf keinem Weg von $D_{0,0}$ zu $D_{m,n}$ liegen, dessen Kosten $\leq s$ bleiben. Zur Prüfung, ob $D_{m,n} \leq s$ ist, genügt es also, alle $D_{i,j}$ zu berechnen, für die $|i-j| \leq s$ bleibt. Sie liegen auf einem Streifen links und rechts von der Diagonalen durch $D_{0,0}$, vgl. Abbildung 9.7.

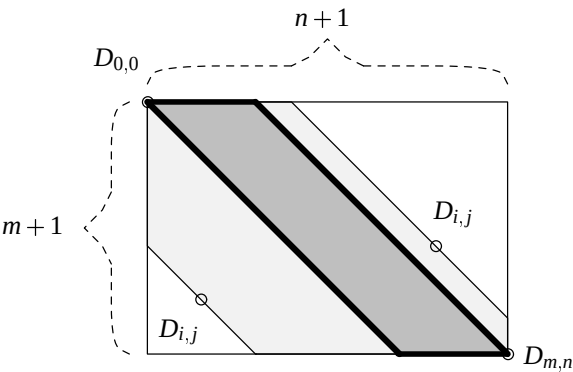


Abbildung 9.7

Insbesondere folgt natürlich, daß $D_{m,n} \leq s$ nur möglich ist, wenn $|m-n| \leq s$ ist. Wie groß kann der Wert $D_{m,n}$ höchstens sein? Offenbar nicht länger als die Länge eines Weges von $D_{0,0}$ nach $D_{m,n}$ mit minimaler Kantenzahl. Nehmen wir (wie

in Abbildung 9.7 geschehen) ohne Einschränkung an, daß $n \geq m$ ist, so haben alle Wege mit $n - m$ horizontalen und m Diagonalkanten die minimal mögliche Kantenzahl. Sie verlaufen im dunkel schraffierten Bereich von Abbildung 9.7. Es ist also $D_{m,n} = D(A, B) \leq m + (n - m) = n$. Diese Beobachtung entspricht natürlich beispielsweise der Möglichkeit, A in B dadurch zu transformieren, daß man die ersten m Buchstaben von B durch Ändern der m Buchstaben von A erzeugt und anschließend die noch fehlenden $n - m$ Buchstaben von B durch Einfüge-Operationen erzeugt.

Falls $s < n - m$ ist, gibt es sicher keinen Weg im Spurgraphen, der $D_{0,0}$ mit $D_{m,n}$ verbindet und Kosten $\leq s$ hat, weil man auf jeden Fall wenigstens $n - m$ horizontale Kanten durchlaufen muß, um von $D_{0,0}$ nach $D_{m,n}$ zu gelangen. Sonst genügt es, die $n - m + 1$ Diagonalen der Länge m im stark schraffierten Bereich von Abbildung 9.7 auszuwerten und je $1/2(s - (n - m))$ kürzere Diagonalen unterhalb der Diagonalen durch $D_{0,0}$ und oberhalb der Diagonalen durch $D_{m,n}$. Denn nur Wege in diesem Diagonalband können Spuren entsprechen mit Gesamtkosten, die s nicht übersteigen. Der Aufwand zur Berechnung der Werte $D_{i,j}$ in diesem Bereich kann daher nach oben abgeschätzt werden durch

$$(n - m + 1) \cdot m + (s - (n - m))(m - 1) \\ = sm - s + n \leq sm - (n - m) + n = O(s \cdot m)$$

In  ist gezeigt, daß man mit Platz $O(\min(s, m, n))$ auskommt, um diese Rechnung durchzuführen.

Man erhält so insgesamt:

Satz 9.1 Für zwei Zeichenreihen $A = a_1 \dots a_m$ und $B = b_1 \dots b_n$ mit $m \leq n$ und eine gegebene Zahl s kann man in Zeit $O(s \cdot m)$ und Platz $O(\min(s, m))$ feststellen, ob $D(A, B) \leq s$ ist.

Die besonders regelmäßige Struktur des Spurgraphen läßt noch weitere Verbesserungen, d.h. eine weitere Reduzierung des Zeit- und Platzbedarfs zur Berechnung der Editierdistanz zu. Dazu vergleiche man z.B.  .

Approximative Zeichenkettsuche

Um in einem gegebenen Text $A = a_1 \dots a_n$ für ein gegebenes $k \geq 0$ und ein Muster $B = b_1 \dots b_m$ alle Vorkommen von Zeichenreihen B' in A zu finden, für die $D(B, B') \leq k$ ist, kann man natürlich wie folgt vorgehen: Man betrachtet für jedes Paar (j, j') mit $1 \leq j \leq j' \leq n$ das Teilstück $a_j a_{j+1} \dots a_{j'}$ von A und berechnet die Editierdistanz $D(a_j a_{j+1} \dots a_{j'}, B)$. Falls sie kleiner oder gleich k ist, hat man ein approximatives Vorkommen von B in A gefunden.

Wieviele Schritte benötigt dies *naive Verfahren* zur approximativen Zeichenkettsuche? Da es $n(n - 1)/2$ Teilstücke $a_j a_{j+1} \dots a_{j'}$ von A gibt, die betrachtet werden, und die Prüfung, ob für die Editierdistanz $D(a_j a_{j+1} \dots a_{j'}, B) \leq k$ gilt, nach Satz 9.1 in Zeit $O(k \cdot \min(j' - j, m))$ durchgeführt werden kann, folgt: Das naive Verfahren findet alle approximativen Vorkommen von B in A in Zeit $O(n(n - 1)/2 \cdot k \cdot \min(j' - j, m)) = O(n^2 \cdot k \cdot m)$. Das ist wenig praktikabel, weil im allgemeinen n sehr groß im Vergleich zu m und k ist.

Um zu effizienteren Verfahren für die approximative Zeichenkettensuche zu kommen, ist es zunächst vernünftig, die Problemstellung leicht zu verändern. Anstatt alle Paare (j, j') von Indizes mit $1 \leq j \leq j' \leq n$ zu finden, für die $D(a_j a_{j+1} \dots a_{j'}, B) \leq k$ ist, bestimmt man für jede Stelle j im Text A ein *ähnlichstes*, bei j endendes Teilstück von A . Das ist ein Teilstück von A , das an der Position j endet und die minimal mögliche Editierdistanz zum Muster B hat. Wir lösen also das folgende Problem 2' anstelle des oben formulierten Problems 2:

Problem 2': (Bestimmung ähnlichster Teile)

Gegeben sind ein Text $A = a_1 \dots a_n$ und ein Muster $B = b_1 \dots b_m$. Gesucht ist für jedes j , $1 \leq j \leq n$, ein j' mit $1 \leq j' \leq j$, so daß für jedes j'' mit $1 \leq j'' \leq j$ gilt: $D(a_{j'} \dots a_j, B) \leq D(a_{j''} \dots a_j, B)$. (Das Teilstück $a_{j'} \dots a_j$ von A ist also ein zu B ähnlichstes Teilstück, das an Position j endet.)

Wir werden Problem 2' so lösen, daß wir zu jeder Textstelle nicht nur ein dort endendes, dem Muster möglichst ähnliches Teilstück finden, sondern auch die Editierdistanz zwischen diesem Teilstück und dem Muster B bestimmen. Daher können wir eine Lösung von Problem 2' auch als eine Lösung von Problem 2 auffassen: Für jede Textstelle j können wir feststellen, ob es überhaupt ein an der Stelle j endendes Teilstück gibt, das eine Editierdistanz von höchstens k zum Muster B hat; und wenn das der Fall ist, kennen wir ein dort endendes, zu B ähnlichstes Stück von A . Die übrigen Teilstücke von A mit Editierdistanz kleiner oder gleich k zu B lassen sich daraus durch Verlängern oder Verkürzen gewinnen.

Bestimmung ähnlichster Teile

Wir werden uns jetzt überlegen, daß das Problem 2' auf ganz ähnliche Weise gelöst werden kann wie das Problem 1, nämlich durch sukzessive Berechnung aller Werte einer $(m+1) \cdot (n+1)$ -Matrix wie folgt:

$$\begin{aligned} D_{0,j} &= 0, & \text{für } 0 \leq j \leq n, \\ D_{i,0} &= i, & \text{für } 0 \leq i \leq m, \end{aligned}$$

und für $0 < i \leq m, 0 < j \leq m$

$$D_{i,j} = \min \left\{ D_{i-1,j-1} + \begin{cases} 1; & \text{falls } a_i \neq b_j \\ 0; & \text{falls } a_i = b_j \end{cases}, D_{i,j-1} + 1, D_{i-1,j} + 1 \right\}.$$

Diese Matrix unterscheidet sich also von der Matrix zur Berechnung der Editierdistanz zwischen A und B nur durch die Initialisierung der 0-ten Zeile: Dort treten ausschließlich Nullen auf.

In Analogie zum Spurgraphen können wir alle Werte $D_{i,j}$ in einem *Abhängigkeitsgraphen* veranschaulichen. Das ist ein Graph mit $(m+1) \cdot (n+1)$ Knoten. Darin ist der Knoten $D_{i,j}$ mit $D_{i-1,j}$, $D_{i,j-1}$ oder $D_{i-1,j-1}$ durch eine Kante verbunden, wenn der Wert $D_{i,j}$ unter Rückgriff auf diese Werte erhalten werden kann. Genauer gilt für $i > 0$ und $j > 0$: Es gibt eine Kante zwischen $D_{i-1,j}$ und $D_{i,j}$, wenn $D_{i,j} = D_{i-1,j} + 1$ ist. Ferner gibt es eine Kante zwischen $D_{i,j-1}$ und $D_{i,j}$, wenn $D_{i,j} = D_{i,j-1} + 1$ ist; und schließlich gibt es eine Kante zwischen $D_{i-1,j-1}$ und $D_{i,j}$, wenn $D_{i,j} = D_{i-1,j-1}$ und $a_i = b_j$ ist oder wenn $D_{i,j} = D_{i-1,j-1} + 1$ und $a_i \neq b_j$ ist.

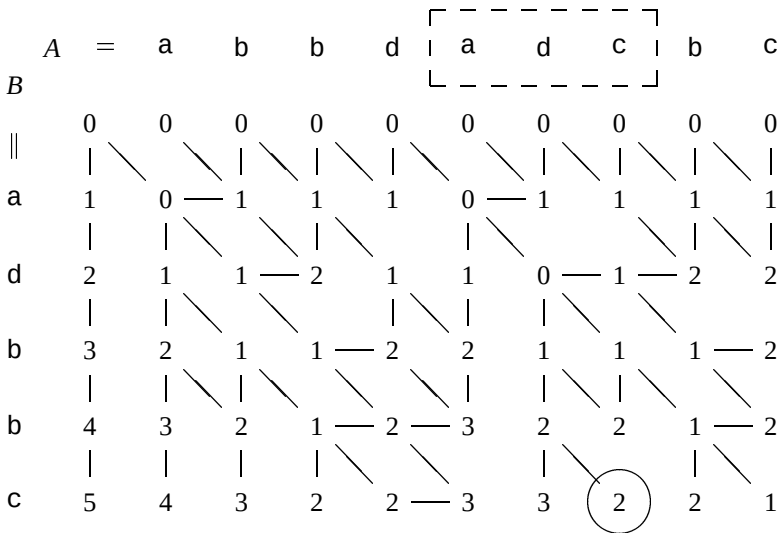


Abbildung 9.8

Abbildung 9.8 zeigt als Beispiel den Abhängigkeitsgraphen für den Text $A = abbdadcbc$ und das Muster $B = adbbc$.

Ähnlich wie für die optimalen Wege im Spurgraphen gilt für *jeden* Weg im Abhängigkeitsgraphen, daß die Werte längs eines jeden Weges von links oben nach rechts unten nur zunehmen. Die Wege im Abhängigkeitsgraphen entsprechen optimalen Spuren in folgendem Sinne: Gibt es im Abhängigkeitsgraphen einen Weg von $D_{0,j'-1}$ nach $D_{i,j}$, so ist $a_{j'} \dots a_j$ ein zu $b_1 \dots b_i$ ähnlichstes, bei j endendes Teilstück von A mit $D(b_1 \dots b_i, a_{j'} \dots a_j) = D_{i,j}$. Das kann man leicht durch Induktion beweisen, weil jeder Weg zum Knoten $D_{i,j}$ im Abhängigkeitsgraphen über einen der Knoten $D_{i-1,j}$, $D_{i,j-1}$ oder $D_{i-1,j-1}$ führen muß. Man findet also ein zu $B = b_1 \dots b_m$ ähnlichstes Teilstück von A , das bei Position j endet, wenn man einen Weg von $D_{m,j}$ zur Zeile 0 zurückverfolgt: Ist $D_{0,j'-1}$ durch einen (nach rechts und unten gerichteten) Weg mit $D_{m,j}$ verbunden, so ist $a_{j'} \dots a_j$ ein gesuchtes Teilstück, vgl. Abbildung 9.9.

Der Wert $D_{m,j}$ gibt die Editierdistanz des bei j endenden, zu B ähnlichsten Teils von A an. So entnimmt man beispielsweise der Abbildung 9.8, daß adc ein an Position 7 endendes, zum Muster B ähnlichstes Teilstück von A ist, das die Editierdistanz 2 zu B hat.

Alle Stellen j in der letzten Zeile, an denen Werte $D_{m,j} \leq k$ auftreten, sind also Stellen, an denen Teile von A mit Editierdistanz höchstens k zum Muster B enden können. Insgesamt erhalten wir damit:

Satz 9.2 Für einen Text $A = a_1 \dots a_n$ und ein Muster $B = b_1 \dots b_m$ kann man in Zeit und Platz $O(m \cdot n)$ zu jeder Stelle j , $1 \leq j \leq n$, im Text ein zu B ähnlichstes, bei j endendes Teilstück von A finden.

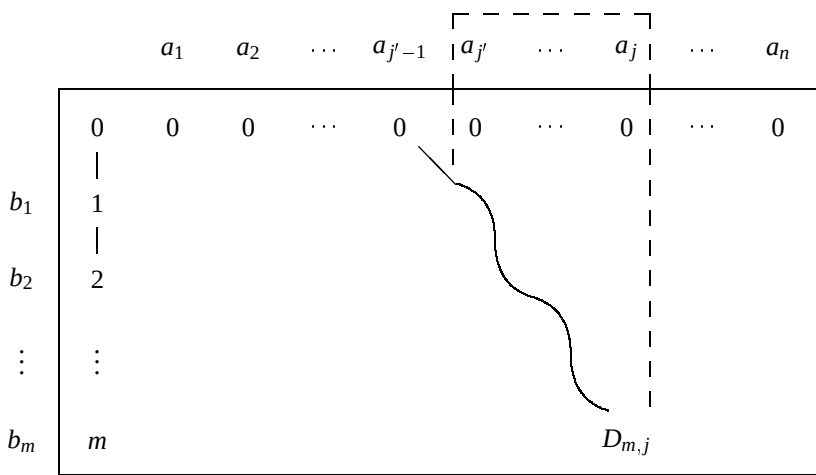


Abbildung 9.9

Dieses Ergebnis wurde von Sellers in [1] bewiesen. Es wurde später in vielfältiger Weise verbessert. Ein Ziel ist dabei, Algorithmen zu entwickeln, die in zwei Phasen arbeiten, einer ersten, nur von m und evtl. der Alphabetgröße abhängigen Aufbereitungsphase für das Muster und einer zweiten, dann nur noch von n abhängigen Textinspektionsphase. Daß das prinzipiell möglich ist, zeigt folgende Überlegung: Man kann die Spalten des Abhängigkeitsgraphen als Zustände eines (allerdings sehr großen) endlichen Automaten auffassen. Man berechnet dann zunächst alle möglichen Zustandsübergänge voraus, d.h. zu jedem möglichen Zustand und jedem Zeichen des zugrundeliegenden Alphabets berechnet man den Folgezustand. Dann inspiziert man den Text mit diesem endlichen Automaten. Diese und andere Verbesserungen des oben angegebenen Verfahrens von Sellers findet man in [2] und in der Übersicht in [3]

9.2 Parallele Algorithmen

Wir sind bisher stets davon ausgegangen, daß die Instruktionen von Programmen durch einen einzigen Prozessor sequentiell nacheinander ausgeführt werden. Als Modell eines solchen, nach dem Von-Neumann-Prinzip aufgebauten Rechners haben wir im Abschnitt 1.1 die Random-Access-Maschine (RAM) eingeführt. Eine Beschleunigung von Algorithmen für Rechner dieses Typs kann nur dadurch erfolgen, daß man die Arbeitsgeschwindigkeiten der einzelnen Systemkomponenten (Prozessor, Speicher, Datenübertragungswege) erhöht. Hier ist man inzwischen fast an der Grenze des physikalisch Möglichen angelangt. Eine weitere Steigerung der Rechengeschwindigkeit ist jedoch erreichbar, wenn man die Von-Neumann-Architektur verläßt und sogenannte

Parallelrechner mit vielen Prozessoren benutzt, die es erlauben, mehrere Verarbeitungsschritte gleichzeitig auszuführen.

Parallelität bedeutet aus algorithmischer Sicht, daß man Probleme daraufhin untersucht, ob sich mehrere zur Lösung erforderliche Teilaufgaben unabhängig voneinander und damit parallel erledigen lassen. Solche Verfahren können dann unter Umständen auf geeigneten Parallelrechnern implementiert werden.

Inzwischen wurde eine große Zahl verschiedener Parallelrechner vorgeschlagen und teilweise auch realisiert. Analog zur RAM hat man auch idealisierte Modelle von Parallelrechnern vorgeschlagen und studiert. Das wichtigste Modell ist die Parallel-Random-Access-Maschine (PRAM). Sie besteht aus p Prozessoren $P_1, \dots, P_p$, die sämtlich auf einen gemeinsamen Speicher zugreifen können. Außer diesem gemeinsamen Speicher verfügt jeder Prozessor noch über einen privaten Arbeitsspeicher. Die p Prozessoren sind synchronisiert, d.h. sie führen Rechenschritte gleichzeitig, taktweise durch. Ein Rechenschritt eines Prozessors besteht aus drei Phasen. Zuerst kann ein Prozessor den Inhalt einer Zelle des gemeinsamen Speichers lesen, dann eine Rechnung unter Benutzung seines privaten Arbeitsspeichers ausführen und schließlich das Ergebnis der Rechnung in eine Zelle des gemeinsamen Speichers übertragen. Die Kommunikation der Prozessoren untereinander erfolgt über den gemeinsamen Speicher. Jeder Prozessor kann mit jedem anderen Daten in zwei Rechenschritten austauschen.

Man unterscheidet PRAM-Modelle häufig weiter danach, ob mehrere Prozessoren gleichzeitig Daten aus derselben Zelle des gemeinsamen Speichers lesen oder dorthin schreiben dürfen. Das führt zu den EREW (exclusive read exclusive write), CREW (concurrent read exclusive write) und CRCW (concurrent read concurrent write) PRAM-Modellen. Wir zeigen im Abschnitt 9.2.1 an einigen einfachen Beispielen, welche Auswirkung auf die Laufzeit von Algorithmen der Wechsel des Maschinenmodells von der RAM zur PRAM hat.

Natürlich ist die Annahme, daß unbeschränkt viele Prozessoren auf dieselbe Speicherzelle zugreifen können und so miteinander verbunden sind, nicht sehr realistisch. Man kann stattdessen auch Parallelrechner betrachten, bei denen mehrere Prozessoren über ein sogenanntes Verbindungsnetz miteinander kommunizieren. In diesem Fall sind identische Prozessoren an den Knoten eines Graphen plazierte. Die Prozessoren kommunizieren untereinander längs der Kanten des Graphen. Eine ganze Reihe unterschiedlicher Verbindungsnetze sind studiert und zum Teil realisiert worden. Die Struktur des Verbindungsnetzes bestimmt weitgehend, für welche Aufgaben der parallele Rechner besonders geeignet ist. Insbesondere ist die Frage interessant, welche Verbindungsnetze als Basis von universellen Parallelrechnern in Frage kommen. Ein prominenter Vertreter eines Verbindungsnetzes ist der Shuffle-exchange-Graph. Wir werden im Abschnitt 9.2.2 zeigen, wie das Sortieren von Zahlen in einem Netz von Prozessoren durchgeführt werden kann, die an den Knoten eines Shuffle-exchange-Graphen plazierte sind. Eine spezielle Form der Parallelverarbeitung auf in der Regel für bestimmte Aufgaben spezialisierten Rechnern sind systolische Arrays und Algorithmen. Wir bringen einige einfache Beispiele im Abschnitt 9.2.3.

Ziel dieses Abschnittes kann es nicht sein, einen auch nur einigermaßen vollständigen Überblick über das umfangreiche Gebiet der parallelen Algorithmen und Parallelrechner zu geben. Es soll vielmehr an einigen Beispielen illustriert werden, daß ein Wechsel des Rechnermodells erhebliche Auswirkungen auf die Form und Effizienz von Lösungen für Probleme hat, die wir bisher auf Rechnern des Von-Neumann-Typs gelöst ha-

ben. Als Einstieg in die umfangreiche Literatur zum Thema Parallelität verweisen wir auf das Lehrbuch von Leighton [Le], auf die zusammenfassende Übersicht [Le] und auf die Bücher von Quinn [Qu], Parberry [Pa] und Petkov [Pe] über systolische Algorithmen.

9.2.1 Einfache Beispiele paralleler Algorithmen

Für manche sequentiellen Algorithmen liegt die Parallelisierbarkeit auf der Hand. Betrachten wir als erstes Beispiel die Aufgabe, das Minimum in einer gegebenen Menge von N Schlüsseln zu finden. Jeder sequentielle Algorithmus zur Bestimmung des Minimums muß wenigstens $N - 1$ Schlüsselvergleiche durchführen, vgl. Abschnitt 2.1.1. Natürlich kann man das Minimum von N Schlüsseln $k_1, \dots, k_N$ auch tatsächlich mit $N - 1$ Schlüsselvergleichen auf folgende Weise finden:

```

min := k1;
for i := 2 to N do
  if ki < min then min := ki;

```

Offensichtlich kann man jedoch auch anders vorgehen. Man bestimmt zunächst in einem ersten Durchgang $k'_1 = \min(k_1, k_2)$, $k'_2 = \min(k_3, k_4)$, $k'_3 = \min(k_5, k_6), \dots, k'_{N/2} = \min(k_{N-1}, k_N)$. Dann bestimmt man in einem zweiten Durchgang $k''_1 = \min(k'_1, k'_2)$, $k''_2 = \min(k'_3, k'_4)$ usw. Nach $\lceil \log_2 N \rceil$ Durchgängen hat man dann das Minimum gefunden. Offenbar können sämtliche Minimumbestimmungen eines Durchgangs parallel aufgeführt werden. Nehmen wir nun an, daß wir $\lceil N/2 \rceil$ Prozessoren zur Verfügung haben und die N Schlüssel anfangs in Speicherzellen $m[1], \dots, m[N]$ des gemeinsamen Speichers der Prozessoren stehen. In einem ersten Durchgang lesen die $\lceil N/2 \rceil$ Prozessoren gleichzeitig jeweils den Inhalt zweier aufeinanderfolgender Speicherzellen, der letzte Prozessor eventuell zweimal denselben Schlüssel, berechnen das Minimum der jeweils gelesenen Werte und schreiben es in die ersten $\lceil N/2 \rceil$ Speicherzellen zurück. Jeder Prozessor P_i , $1 \leq i \leq \lceil N/2 \rceil$, liest also $m[2i - 1]$ und $m[2i]$, berechnet $\min = \min(m[2i - 1], m[2i])$ und speichert $\min$ in Zelle $m[i]$. In einem zweiten Durchgang lesen $\lceil N/4 \rceil$ Prozessoren wiederum gleichzeitig jeweils den Inhalt zweier aufeinanderfolgender Speicherzellen, berechnen das Minimum der jeweils gelesenen Werte und schreiben es in die ersten $\lceil N/4 \rceil$ Speicherzellen zurück usw. Nach $r = \lceil \log_2 N \rceil$ Durchgängen steht dann das Minimum in der Speicherzelle $m[1]$. Folgende Tabelle 9.1 zeigt die Belegung des gemeinsamen Speichers nach jedem Durchgang für ein kleines Beispiel.

Der Inhalt der mit „—“ markierten Zellen hat sich nicht verändert. Lesekonflikte treten nicht auf. Es werden Werte des gemeinsamen Speichers überschrieben, aber Schreibkonflikte treten dabei ebenfalls nicht auf. Man kann also das Minimum von N Schlüsseln mit einer EREW-PRAM mit $\lceil N/2 \rceil$ Prozessoren in $O(\log N)$ Zeit berechnen.

Offenbar kann man diese als *binäre Fan-in-Technik* bekannte Methode des Akkumulierens von Werten in $\lceil \log N \rceil$ Schritten auf eine ganze Reihe weiterer Probleme anwenden. Wir geben einige Beispiele.

$\sum_{i=1}^N a_i$ kann mit Hilfe von $\lceil N/2 \rceil$ Prozessoren in Zeit $O(\log N)$ berechnet werden.

m:	1	2	3	4	5	6	7	
	15	2	43	17	4	8	47	Anfangsbelegung
	2	17	4	47	—	—	—	nach 1. Durchgang
	2	4	—	—	—	—	—	nach 2. Durchgang
	2	—	—	—	—	—	—	nach 3. Durchgang

Tabelle 9.1

Denn nehmen wir ohne Einschränkung an, daß $N = 2^r$ ist. Wir benutzen im ersten Durchgang die $N/2$ Prozessoren, um $a_{2i-1} + a_{2i}$ für $1 \leq i \leq N/2$, also die Partialsummen aus je zwei Summanden, zu berechnen. Im zweiten Durchgang werden $N/4$ Prozessoren benutzt, um die Partialsummen aus je vier Summanden zu berechnen, usw. Schließlich berechnet ein Prozessor aus den Partialsummen $a_1 + \dots + a_{N/2}$ und $a_{N/2+1} + \dots + a_N$ das Ergebnis. Natürlich funktioniert dasselbe Verfahren auch für die Berechnung von $\prod_{i=1}^N a_i$.

Das Produkt zweier $N \times N$ Matrizen kann mit N^3 Prozessoren in Zeit $O(\log N)$ berechnet werden.

Zur Berechnung von $C = A \cdot B$, mit $C = (c_{ij})$ und $c_{ij} = \sum_{k=1}^N a_{ik} \cdot b_{kj}$, verwendet man für jedes Element der Produktmatrix N Prozessoren. Mit Hilfe dieser N Prozessoren berechnet man zunächst die N Produkte $a_{i1} \cdot b_{1j}, a_{i2} \cdot b_{2j}, \dots, a_{iN} \cdot b_{Nj}$ und daraus wie oben angegeben in $O(\log N)$ Zeit das Element c_{ij} durch wiederholte Verdopplung der Anzahl der Summanden der Partialsummen. Die insgesamt N^3 Prozessoren müssen zwar dieselben Elemente der Ausgangsmatrizen A und B gleichzeitig lesen können, Schreibkonflikte sind aber vermeidbar, so daß sich das Verfahren auf einer CREW-PRAM implementieren läßt.

Falls mehrere Prozessoren gleichzeitig in dieselbe Speicherzelle des gemeinsamen Speichers schreiben dürfen, ist das Ergebnis von Schreiboperationen zunächst nur dann wohldefiniert, wenn alle Prozessoren denselben Wert in eine Zelle schreiben. Mögliche Schreibkonflikte, also der Versuch, verschiedene Werte in dieselbe Zelle zu schreiben, können nach unterschiedlichen Strategien aufgelöst werden, die uns hier nicht weiter interessieren. Wir wollen jedoch zeigen, daß das Minimum von N Schlüsseln auf einer CRCW-PRAM in konstanter Zeit berechnet werden kann, ohne daß Schreibkonflikte auftreten. Dazu nehmen wir an, daß die Schlüssel in Zellen $a[1], \dots, a[N]$ gespeichert sind und zusätzlich N Speicherzellen $b[1], \dots, b[N]$ des gemeinsamen Speichers genutzt werden können. Für alle i und j mit $1 \leq i, j \leq N$ führen die Prozessoren P_{ij} gleichzeitig die folgenden vier Schritte aus, die wir an einem Beispiel mit sieben Schlüsseln erläutern.

	1	2	3	4	5	6	7
a:	15	2	43	2	4	8	47

- 1. Schritt: P_{i1} schreibt 0 nach $b[i]$.
- 2. Schritt: P_{ij} liest $a[i]$ und $a[j]$ und schreibt eine 1 nach b_j genau dann, wenn $a[i] < a[j]$. Mit Ausnahme jeder Position j , an der ein minimales Element steht, wird also in b die 0 überall durch eine 1 überschrieben. Für das Beispiel ergibt sich folgende Belegung von b :

b:

1	2	3	4	5	6	7
1	0	1	0	1	1	1

- 3. Schritt: P_{ij} liest $b[i]$ und schreibt eine 1 nach $b[j]$ genau dann, wenn $i < j$ und $b[i] = 0$. Dadurch bleibt nur für das kleinste i mit $b[i] = 0$ der Wert 0 erhalten; alle anderen Werte werden durch eine 1 überschrieben. In unserem Beispiel erhalten wir:

b:

1	2	3	4	5	6	7
1	0	1	1	1	1	1

- 4. Schritt: P_{i1} liest $b[i]$ und schreibt $a[i]$ nach $b[1]$ genau dann, wenn $b[i] = 0$ ist. Jetzt steht das Minimum in $b[1]$.

Als letztes Beispiel wollen wir zeigen, wie ein *Verfahren zur Berechnung eines minimalen spannenden Baumes (MST)* eines Graphen parallelisiert werden kann. Das in Abschnitt 8.6 beschriebene Verfahren von Boruvka zur Berechnung des MST besteht darin, einen Wald von Teilbäumen des MST sukzessiv zum MST zusammenwachsen zu lassen. Man beginnt mit Teilbäumen, die sämtlich nur aus je genau einem Knoten des gegebenen Graphen bestehen. Dann werden immer wieder je zwei verschiedene Teilbäume durch Hinzunahme einer Kante minimalen Gewichtes zu einem Baum verbunden, bis ein einziger Baum, der MST, entstanden ist. Man kann versuchen, eine parallele Version dieses Verfahrens dadurch zu erhalten, daß man gleichzeitig Kanten minimalen Gewichtes wählt, die verschiedene Teilbäume miteinander verbinden. Wie man leicht sieht, kann eine nicht weiter eingeschränkte Wahl aber zu Zyklen führen, wenn im Graphen Kanten gleichen Gewichtes auftreten.

Nehmen wir an, daß die Knoten des gegebenen Graphen mit den natürlichen Zahlen $1, \dots, N$ bezeichnet werden. Dann kann man auf den Kanten eine lexikographische Anordnung ungeordneter Paare, die sogenannte *Min-max-Ordnung* „ $\prec$ “, wie folgt einführen.

Es gilt für die ungerichteten Kanten (u, v) und (u', v')
 $(u, v) \prec (u', v')$ genau dann, wenn $\min\{u, v\} < \min\{u', v'\}$ oder
 $(\min\{u, v\} = \min\{u', v'\} \text{ und } \max\{u, v\} < \max\{u', v'\})$.

Wählen wir nun für jeden Knoten i eine Kante (i, j) mit minimalem Gewicht so, daß (i, j) die bezüglich der Min-max-Ordnung erste Kante dieser Art ist, so werden Zyklen vermieden. Denn nehmen wir beispielsweise an, es gäbe einen Dreierzyklus. Für drei Knoten i, j und k mit $i < j < k$ seien die Kanten (i, j) , (j, k) und (k, i) gewählt worden. Weil zum Knoten i die Kante (i, j) und nicht die Kante (i, k) gewählt wurde, muß für die Gewichte $g(i, j)$ und $g(i, k)$ dieser Kanten gelten:

$$g(i, j) \leq g(i, k) = g(k, i).$$

Aus analogen Gründen muß auch

$$g(j, k) \leq g(j, i) = g(i, j),$$

$$g(k, i) \leq g(k, j) = g(j, k)$$

sein. Daraus erhält man $g(i, j) = g(j, k) = g(k, i)$. Dann kann aber für j nicht die Kante (j, k) gewählt worden sein, weil (j, i) eine Kante mit gleichem Gewicht ist, aber $(j, i) \prec (j, k)$ gilt. Eine ähnliche Argumentation zeigt die Unmöglichkeit von Zyklen beliebiger Länge.

Der folgende Algorithmus zur Berechnung eines minimalen spannenden Baumes stammt von Sollin. Er setzt voraus, daß der Graph G die Knotenmenge $\{1, \dots, N\}$ besitzt und die Kanten implizit durch die Gewichtsfunktion g gegeben sind mit $g(i, j) = \infty$, falls i und j in G nicht miteinander verbunden sind.

```

procedure Sollin ( $G : \text{Graph}$ ; var  $F : \text{Wald}$ );
  { $F$  ist Wald von Teilbäumen des MST für  $G$ , am Ende ist
    $F = \{T\}$ ,  $T$  MST für  $G$ }
  var  $i : \text{integer}$ ; {Laufindex}
  begin
    {initialisiere  $F$  als Menge von  $N$  Teilbäumen mit genau einem Knoten
     und keiner Kante}
    for  $i := 1$  to  $N$  do  $T_i = \{i\}$ ;
     $F := \{T_1, \dots, T_N\}$ ;
    while  $|F| > 1$  do
      begin
        for each  $T \in F$  do {parallel}
          begin
            finde bezüglich „ $\prec$ “ erstes Paar von Knoten  $(u, v)$ 
              mit  $u \in T, v \in T' \in F \setminus \{T\}, g(u, v)$  minimal
          end;
            berechne neuen Wald  $F$  durch Verschmelzen von Bäumen,
              die durch zuvor gewählte Kanten miteinander verbunden sind
          end {while}
      end {Sollin}
  
```

Wir geben ein Beispiel für Sollins Algorithmus an. Dabei folgen wir der Konvention, beim Verschmelzen von zwei Bäumen T_i und T_j dem neuen Baum den Namen $T_{\min\{i, j\}}$ zu geben. Gegeben sei der Graph aus Abbildung 9.10.

Der Initialisierungsschritt liefert den Wald $F = \{T_1, \dots, T_8\}$ mit $T_i = \{i\}$, $1 \leq i \leq 8$. Nach einmaliger Ausführung der Anweisungen in der **while**-Schleife erhält man den Wald von Abbildung 9.11 mit den Teilbäumen $T_1 = \{1, 3, 8\}$, $T_2 = \{2, 4, 5\}$, $T_6 = \{6, 7\}$ und den in der Abbildung 9.11 gezeigten Kanten.

Im nächsten Schritt wird nun für T_1 die Kante $(8, 2)$ mit Gewicht 3, für T_2 dieselbe Kante und für T_6 die Kante $(6, 5)$ gewählt. Durch Verschmelzen der durch Kanten verbundenen Bäume entsteht ein einziger Baum, der MST aus Abbildung 9.12.

Offenbar wird die Anzahl der Bäume im Wald F bei einmaliger Ausführung der Anweisungen der **while**-Schleife wenigstens um die Hälfte reduziert. Daher kann die **while**-Schleife höchstens $\log_2 N$ -mal durchlaufen werden.

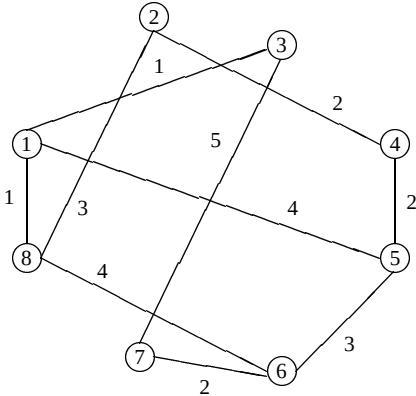


Abbildung 9.10

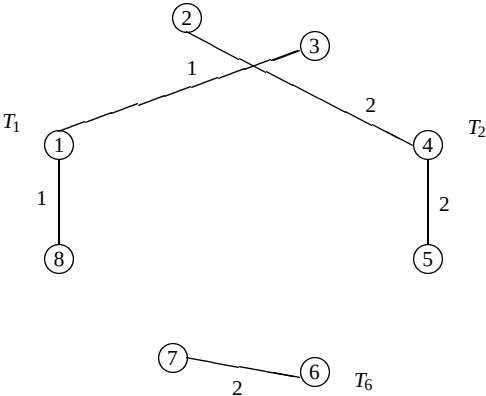


Abbildung 9.11

Nehmen wir jetzt an, wir hätten zur Ausführung des Algorithmus von Sollin N Prozessoren $P_1, \dots, P_N$ zur Verfügung. Für jedes $i, 1 \leq i \leq N$, wird Prozessor P_i dem Knoten i zugeordnet. Wir können annehmen, daß die den Graphen G vollständig charakterisierende Gewichtsfunktion g als Adjazenzmatrix im gemeinsamen Speicher der N Prozessoren abgelegt ist. In einem Bereich $t[1 \dots N]$ des gemeinsamen Speichers merkt man sich für jedes $i, 1 \leq i \leq N$, den Index j des Baumes T_j , in dem der Knoten i jeweils liegt. Der Initialisierungsschritt besteht also darin, daß für jedes $i, 1 \leq i \leq N$, P_i den Wert i nach $t[i]$ schreibt. Das ist parallel in konstanter Zeit ausführbar. Jeder Durchlauf der **while**-Schleife kann jetzt in drei Schritten erledigt werden.

Im ersten Schritt bestimmt jeder Prozessor P_i den nächsten, mit i verbundenen Knoten $j = nn(i)$, der nicht in dem Baum liegt, der i enthält. Diese Suche nach $nn(i)$, d.h. nach dem kleinsten j mit $g(i, j)$ minimal und $j \notin T_{t[i]}$ kann P_i offenbar in Zeit $O(N)$ erledigen. Im zweiten Schritt wird jetzt für jeden Baum eine bezüglich der Min-max-Ordnung

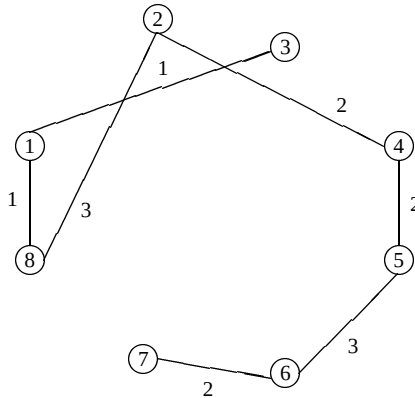


Abbildung 9.12

kleinste Kante minimalen Gewichts bestimmt, die ihn mit einem anderen Baum verbindet. Dazu inspiziert jeder Prozessor P_i noch einmal alle mit i verbundenen Knoten. Trifft P_i dabei auf einen Knoten $k \neq i$ mit $t[i] = t[k]$ und $g(i, nn(i)) = g(i, k)$, weiß P_i , daß es zwei Kanten minimalen Gewichts gibt, die den Baum, in dem der Knoten i liegt, mit einem anderen Baum verbinden können, nämlich die Kanten $(i, nn(i))$ und (i, k) . P_i merkt sich dann, daß die Kante $(i, nn(i))$ nicht in Frage kommt (d.h.: P_i scheidet aus), genau dann, wenn $(i, k) \prec (i, nn(i))$ ist. Dieser zweite Schritt ist offenbar ebenfalls parallel in Zeit $O(N)$ ausführbar. Die im zweiten Schritt nicht ausgeschiedenen Prozessoren enthalten jetzt genau die Kanten, die zum Verschmelzen von Bäumen des aktuellen Waldes herangezogen werden müssen. Das geschieht im dritten Schritt. In diesem Schritt werden die Einträge im Array t wie folgt verändert: Der Reihe nach teilt jeder noch aktive Prozessor, der eine Verbindungskante (i, j) gespeichert hat, allen anderen Prozessoren mit, daß der Name $\max(t[i], t[j])$ durch $\min(t[i], t[j])$ ersetzt werden muß. Jeder Prozessor prüft für sich, ob der Knoten, den er repräsentiert, in einem Baum liegt, der von dieser Namensänderung betroffen ist; die Namensänderung wird dann gleichzeitig in konstanter Zeit ausgeführt. Damit kann das Verschmelzen von Bäumen im dritten Schritt insgesamt in Zeit $O(N)$ ausgeführt werden.

Mit Hilfe von N Prozessoren kann man also jeden Durchlauf der **while**-Schleife in Zeit $O(N)$ ausführen, wobei jedesmal $O(N^2)$ Einzeloperationen durchgeführt werden. Wir fassen unsere Überlegungen in einem Satz zusammen.

Satz 9.3 Für einen gewichteten Graphen mit N Knoten kann man mit Hilfe von N Prozessoren einen minimalen spannenden Baum in Zeit $O(N \log N)$ berechnen. Dabei werden von den N Prozessoren insgesamt $O(N^2 \log N)$ Operationen ausgeführt.

Ein wesentlicher Grund für den Zeitbedarf des Sollin'schen Algorithmus bei Verwendung von N Prozessoren liegt darin, daß bei jedem Durchlauf durch die **while**-Schleife alle N Prozessoren Minima bestimmen müssen. Das kostet jeweils $\Theta(N)$ Schritte und führt damit zur Gesamtlaufzeit $O(N \log N)$. Unter Benutzung von N^2 Prozessoren kann

man die Laufzeit des Verfahrens drücken, weil man die Bestimmung des Minimums mit je N^2 Prozessoren in konstanter Zeit erledigen kann.

Schließlich kann man das Verfahren von Sollin ohne Effizienzverlust auch noch auf Parallelrechnern mit stark eingeschränkten Kommunikationsmöglichkeiten implementieren, vgl. [1]. Eine ausführliche Übersicht über parallele Graphenalgorithmen und ihre Implementierung auf verschiedenen Parallelrechnern enthält die Arbeit [2].

9.2.2 Paralleles Mischen und Sortieren

Wir untersuchen jetzt die Frage, ob durch den Einsatz von mehreren Prozessoren die zum Sortieren von N Schlüsseln erforderliche Zeit verkürzt werden kann. Es liegt nahe, zunächst die seriellen, also für Rechner des Von-Neumann-Typs mit nur einem Prozessor, entwickelten Sortierv Verfahren auf ihre Parallelisierbarkeit hin zu untersuchen. Ein typischer Schritt in einem seriellen Sortierv Verfahren ist, daß der Prozessor zwei Schlüssel miteinander vergleicht. Die restlichen Schlüssel stehen „ungenutzt“ im Speicher. Es ist daher naheliegend, jedem Paar von Schlüsseln einen Prozessor zuzuordnen, der eine solche Vergleichsoperation ausführen kann. Wir stellen uns also vor, daß der zum Sortieren benutzte Parallelrechner eine große, von der Zahl N der zu sortierenden Schlüssel abhängige Zahl von sogenannten *Compare-exchange-Moduln* hat, vgl. Abbildung 9.13.

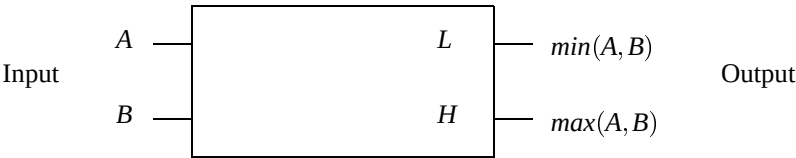


Abbildung 9.13

Ein Compare-exchange-Modul (oder: Vergleichsmodul) kann zwei Werte gleichzeitig lesen, sie miteinander vergleichen und geordnet wieder ausgeben. Der kleinere Schlüssel verläßt den Vergleichsmodul über den mit L (für: Low) und der größere über den mit H (für: High) gekennzeichneten Ausgang. Die N Schlüssel müssen auf die Compare-exchange-Moduln verteilt werden, d.h. es ist die Frage zu beantworten, welche Schlüssel zu welchem Zeitpunkt in welchem Vergleichsmodul zusammentreffen. Wir wollen in diesem Abschnitt nicht voraussetzen, daß die Vergleichsmoduln über einen gemeinsamen Speicher kommunizieren. Wir suchen vielmehr ein festes Verbindungsnetz für die Vergleichsmoduln.

Ein auf einem einzigen Prozessor seriell ablaufendes Sortierprogramm kann seinen Ablauf von Ereignissen abhängig machen, die erst während der Programmausführung auftreten. Ein in Hardware realisiertes Verbindungsschema ist jedoch unveränderlich. Betrachten wir als Beispiel das folgende, zum Sortieren von drei Schlüsseln geeignete, serielle Programmstück.

```

if  $A > B$  then vertausche( $A, B$ );
if  $B > C$  then begin
    vertausche( $B, C$ );
    if  $A > B$  then vertausche( $A, B$ )
end

```

Man überprüft leicht, daß für beliebige Anfangswerte von A , B und C die Werte dieser Variablen nach Ausführung des Programmstücks aufsteigend sortiert sind. Es werden aber z.B. für die Eingabe $A = 2$, $B = 1$, $C = 3$ Teile des Programms nicht ausgeführt. Der letzte Vergleich ist in diesem Fall unnötig und unterbleibt. Ein aus Vergleichsmoduln aufgebautes Verbindungsnetz kann seine Struktur jedoch nicht von den Eingangsdaten abhängig machen. Dennoch ist Sortieren möglich, wie das in Abbildung 9.14 gezeigte Netz aus drei Vergleichsmoduln zeigt.

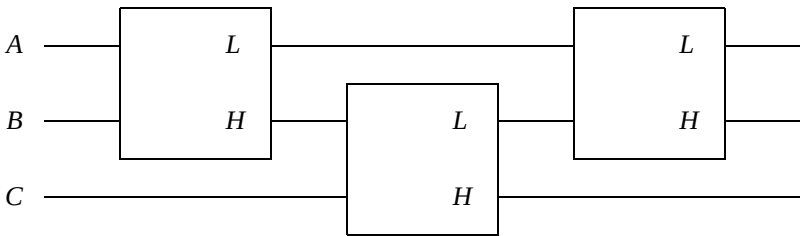


Abbildung 9.14

Den Variablen A , B , C entsprechen die Eingänge des Verbindungsnetzes. Es ist leicht zu überprüfen, daß die bei A , B , C eingegebenen Schlüssel das Netz in aufsteigend sortierter Reihenfolge über die drei rechten Ausgänge verlassen. Wenn wir annehmen, daß ein Paar von Schlüsseln in einer Zeiteinheit verarbeitet werden kann, folgt sofort, daß die am linken Ende des Sortiernetzes eingegebene Folge nach drei Zeiteinheiten am rechten Ende, also am Ausgang des Netzes, in sortierter Reihenfolge vorliegt.

In seriellen Sortierv Verfahren spielen Merge-Strategien eine wichtige Rolle. Man zerlegt die zu sortierende Folge, sortiert die entstandenen Teilfolgen und verschmilzt die sortierten Teilfolgen zur sortierten Gesamtfolge. Soll diese Technik auch für paralleles Sortieren eingesetzt werden, so benötigt man Verschmelzungsverfahren, die es erlauben, zwei sortierte Schlüsselfolgen mit immer der gleichen Operationsfolge zu einer sortierten Folge zu verschmelzen. Wir erläutern jetzt zwei solcher Verfahren, die unter dem Namen *Odd-even-merge* und *Bitonic-merge* bekannt sind, vgl. 

Wir erläutern zunächst das *Odd-even-merge-Verfahren*. Gegeben seien zwei Folgen $a_1, \dots, a_n$ und $b_1, \dots, b_n$ von jeweils aufsteigend sortierten Zahlen gleicher Länge, d.h. es gilt für alle i , $1 \leq i < n$, $a_i \leq a_{i+1}$ und $b_i \leq b_{i+1}$. Wir wollen diese zwei Folgen zu einer einzigen, aufsteigend sortierten Folge der Länge $2n$ verschmelzen. Wir lösen diese Aufgabe rekursiv und nehmen der Einfachheit halber an, daß $n = 2^k$ für ein $k \geq 0$ ist. Ist $n = 1$, werden a_1 und b_1 miteinander verglichen und in die richtige Reihenfolge gebracht. Ist $n > 1$, so betrachten wir zunächst die Folgen halber Länge mit *ungeradzahligem* Index $a_1, a_3, \dots, a_{n-1}$ und $b_1, b_3, \dots, b_{n-1}$ und verschmelzen sie auf dieselbe Weise zu einer aufsteigend sortierten Folge $c_1, \dots, c_n$. Dann betrachten wir die Folgen halber Länge mit *geradzahligem* Index $a_2, a_4, \dots, a_n$ und $b_2, b_4, \dots, b_n$ und verschmelzen sie zu einer aufsteigend sortierten Folge $d_1, \dots, d_n$. Nun kann man zeigen, daß für jedes i , $1 \leq i < n$, das Element c_{i+1} unmittelbar vor oder unmittelbar nach dem Element d_i der Größe nach eingeordnet werden muß. (Einen Beweis findet man in [1] oder in [2].) Wir können aus $c_1, \dots, c_n$ und $d_1, \dots, d_n$ also eine sortierte Folge $e_1, \dots, e_{2n}$ herstellen, indem wir setzen:

$$\begin{aligned} e_1 &= c_1 \\ e_{2i} &= \min(c_{i+1}, d_i), \text{ für } 1 \leq i < n \\ e_{2i+1} &= \max(c_{i+1}, d_i), \text{ für } 1 \leq i < n \\ e_{2n} &= d_n. \end{aligned}$$

Beispiel: Gegeben seien die aufsteigend sortierten Folgen

$$\begin{array}{cccc} a: & 2 & 15 & 19 & 43 \\ b: & 4 & 8 & 17 & 47 \end{array}$$

Verschmelzen der Teilfolgen mit geradzahligem bzw. ungeradzahligem Index ergibt

$$\begin{array}{cccc} c: & 2 & 4 & 17 & 19 \\ d: & 8 & 15 & 43 & 47 \end{array}$$

Vergleichen und gegebenenfalls Vertauschen der Paare (c_{i+1}, d_i) , also $(4,8)$, $(17,15)$, $(19,43)$, ergibt die sortierte Folge

$$e: \quad 2 \quad 4 \quad 8 \quad 15 \quad 17 \quad 19 \quad 43 \quad 47.$$

Es ist offensichtlich, daß das Odd-even-merge-Verfahren als Netzwerk von Vergleichsmoduln realisiert werden kann. Für $n = 1$ besteht das Netzwerk genau aus einem Vergleichsmodul. Für $n > 1$, wobei der Einfachheit halber $n = 2^k$ für ein $k > 0$ gelte, hat das Netzwerk genau $2n$ Eingabeleitungen, die linear angeordnet sind, und zwar für die Folgen der Eingabewerte $a_1, b_1, a_3, b_3, \dots, a_{n-1}, b_{n-1}$ und $a_2, b_2, a_4, b_4, \dots, a_n, b_n$, in dieser Reihenfolge, und $2n$ Ausgabeleitungen $e_1, e_2, \dots, e_{2n}$. Nehmen wir an, wir hätten bereits ein Netzwerk zum Verschmelzen zweier Folgen der Länge $n/2$, so erhält man ein Netzwerk zum Verschmelzen von zwei Folgen mit Länge n , wenn man es aus gegebenen Netzen und Vergleichsmoduln wie in Abbildung 9.15 gezeigt zusammensetzt. Dabei gehört der links gezeigte Teil sich kreuzender Leitungen nicht zum Netzwerk; er sorgt lediglich dafür, daß beim Zusammensetzen von Netzen die zu verschmelzenden Eingabefolgen korrekt verzahnt an die Teil-Netzwerke weitergeleitet werden.

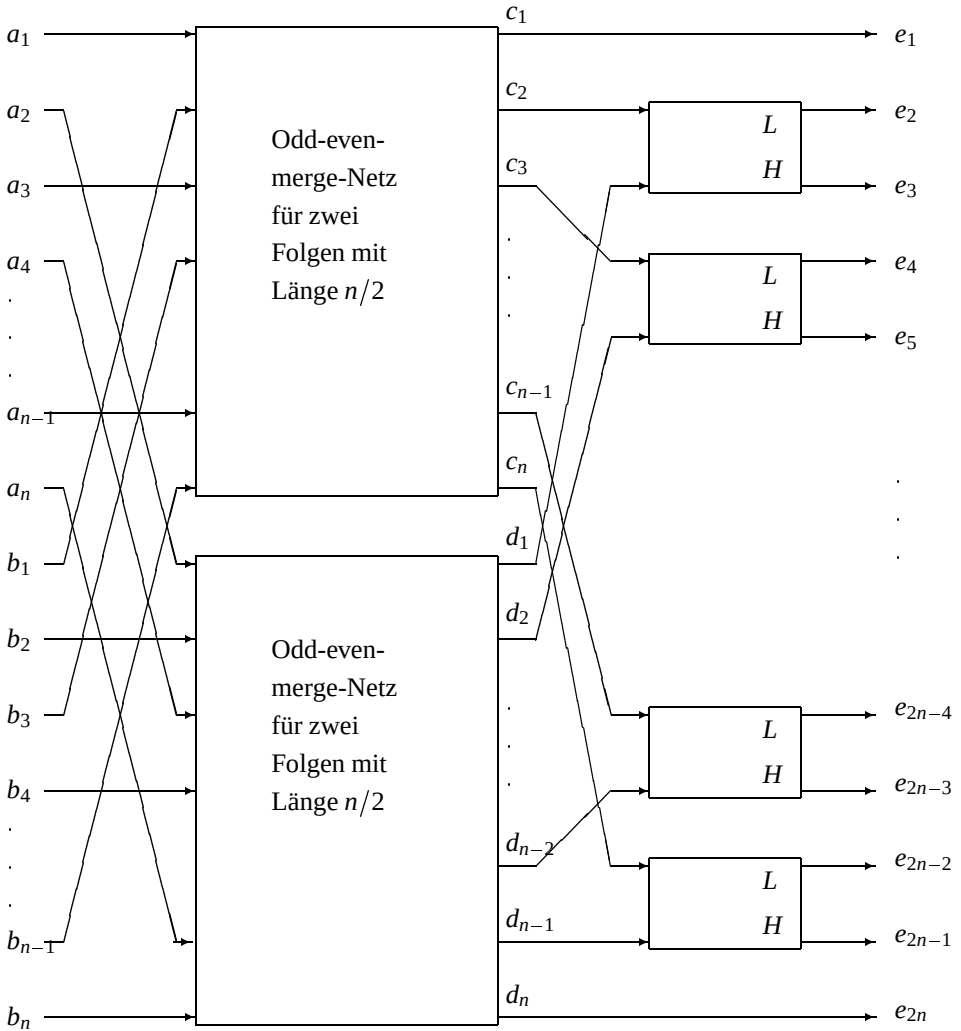


Abbildung 9.15

Wir nennen ein Netzwerk zum Verschmelzen von zwei sortierten Folgen mit Längen $n/2$ nach dem Odd-even-merge-Verfahren ein *OEM-Netz der Größe n* . Das in Abbildung 9.15 gezeigte Verfahren zur Konstruktion von OEM-Netzen der Größe n zeigt unmittelbar, daß eine in ein OEM-Netz der Größe $n = 2^k$ eingegebene Zahl höchstens k Vergleichsmoduln durchläuft bis sie das Netz verläßt.

Analog zum reinen 2-Wege-Mergesort, vgl. Abschnitt 2.4.2, kann man jetzt $n = 2^k$ Zahlen wie folgt sortieren: Man beginnt mit n Folgen der Länge 1 und verschmilzt sie gleichzeitig mit 2^{k-1} OEM-Netzen der Größe 2^1 zu $n/2$ Folgen der Länge 2. Dann verschmilzt man $n/2$ Folgen der Länge 2 mit 2^{k-2} OEM-Netzen der Größe 2^2 zu Folgen

der Länge 2^2 usw. Daraus kann man unmittelbar ein Konstruktionsprinzip für ein Sortiernetz zum parallelen Sortieren von $n = 2^k$ Zahlen ablesen: Ein Sortiernetz für zwei Zahlen ist ein Vergleichsmodul. Ein Sortiernetz für $n > 2$ Zahlen erhält man aus zwei Sortiernetzen für $n/2$ Zahlen und einem OEM-Netz der Größe n wie in Abbildung 9.16 dargestellt. Wir nennen ein nach diesem Prinzip aufgebautes Sortiernetz ein OES-Netz der Größe n .

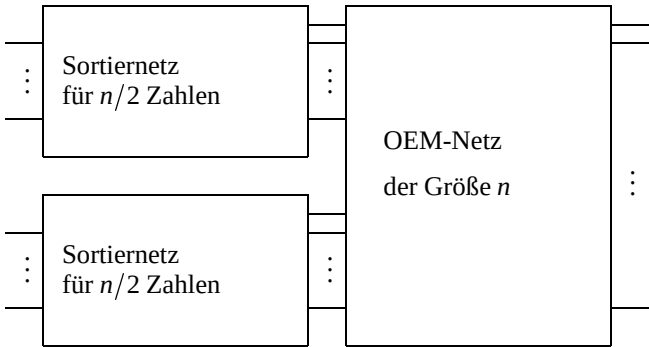


Abbildung 9.16

Abbildung 9.17 zeigt explizit ein OES-Netz der Größe 8. Offenbar können alle in einer Spalte untereinander stehenden Vergleichsmoduln des Netzes parallel arbeiten.

Man kann aus dem Verfahren zur Konstruktion von OES-Netzen der Größe $n = 2^k$ unmittelbar ablesen, daß ein in das Netz eingegebener Schlüssel höchstens $1 + 2 + \dots + k = k(k + 1)/2$ Vergleichsmoduln durchläuft, bevor er das Netz (an der richtigen Stelle) wieder verläßt. Ferner enthält ein OES-Netz der Größe n offenbar höchstens $(1 + 2 + \dots + k) \cdot n/2$ Vergleichsmoduln insgesamt, für größere n sogar weit weniger. Wegen $k = \log_2 n$ folgt damit sofort:

Satz 9.4 n Zahlen können in Zeit $O(\log^2 n)$ mit Hilfe eines aus $O(n \log^2 n)$ Vergleichsmoduln bestehenden Netzes sortiert werden.

Nicht alle in ein OES-Netz eingegebenen Schlüssel durchlaufen dieselbe Anzahl von Vergleichsmoduln, bevor sie das Netz verlassen. Wir geben jetzt ein Verfahren zum Verschmelzen zweier sogenannter *bitonischer* Folgen an, das schließlich zu einem sehr regelmäßig aufgebauten Sortiernetz führt. Eine Zahlenfolge heißt *bitonisch*, wenn sie durch Aneinanderhängen einer absteigend an eine aufsteigend sortierte Zahlenfolge oder durch zyklische Vertauschung aus einer solchen Zahlenfolge entsteht. Hier sind einige Beispiele bitonischer Folgen, die wir auf naheliegende Weise zugleich graphisch veranschaulicht haben.

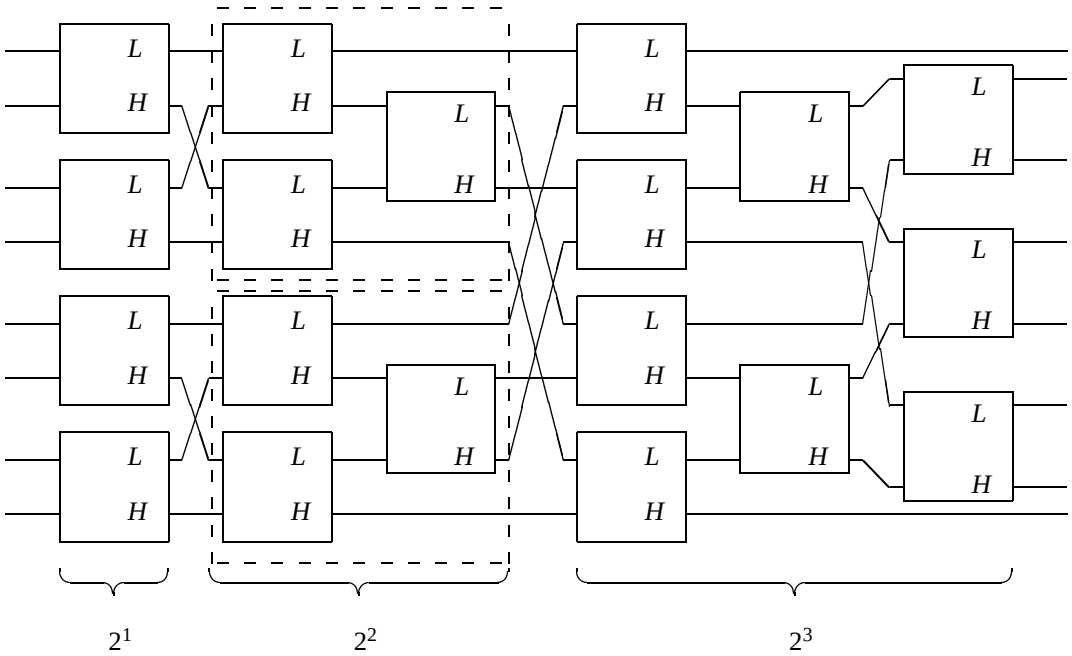
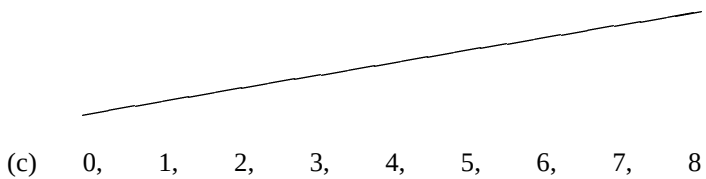
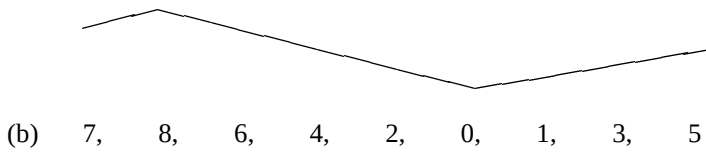
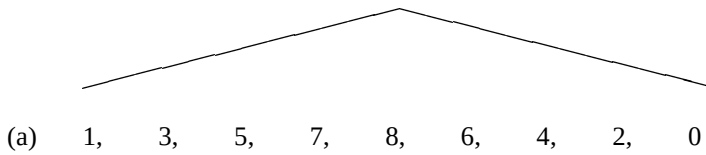


Abbildung 9.17



Das *Bitonic-merge-Verfahren* überführt zwei bitonische Zahlenfolgen in sortierte Folgen. Es basiert auf der Beobachtung, daß eine bitonische Folge in zwei bitonische Folgen zerlegt werden kann, indem man je zwei $n/2$ Positionen voneinander entfernte Elemente miteinander vergleicht und gegebenenfalls vertauscht, wobei n die Länge der bitonischen Folge ist. Genauer gilt:

Lemma 9.1 Sei $a = a_0, \dots, a_{n-1}$ eine bitonische Folge. Sei $b_i = \min(a_i, a_{i+n/2})$ und $c_i = \max(a_i, a_{i+n/2})$ für $0 \leq i < n/2$. Dann sind die Folgen $b = b_0, \dots, b_{n/2-1}$ und $c = c_0, c_1, \dots, c_{n/2-1}$ ebenfalls bitonisch. Darüberhinaus gilt $b_i \leq c_j$ für alle i und j .

Zum Beweis nehmen wir zunächst an, daß die gegebene Folge aus zwei gleichlangen Teilfolgen besteht, von denen die erste $a_0, \dots, a_{n/2-1}$ aufsteigend und die zweite $a_{n/2}, \dots, a_{n-1}$ absteigend sortiert ist. Die Bildung der Folgen b und c aus a kann durch Abbildung 9.18 veranschaulicht werden.

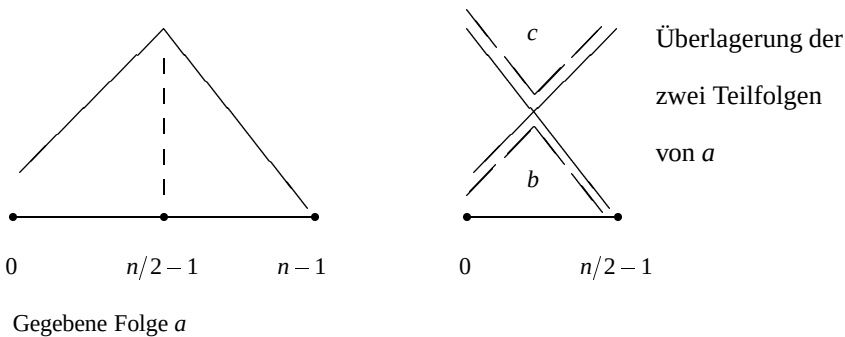


Abbildung 9.18

Es ist klar, daß die so gebildeten Folgen b und c bitonisch sind und alle Elemente von c größer als alle Elemente von b sein müssen. Man sieht leicht, daß die Behauptung auch dann noch gilt, wenn die beiden Teilfolgen von a unterschiedliche Länge haben oder a durch zyklische Vertauschung aus einer zunächst auf- und dann absteigend sortierten Folge entsteht. Abbildung 9.19 zeigt ein weiteres Beispiel für die Bildung der Folgen b und c . □

Aus dem Lemma kann man ein rekursives Konstruktionsprinzip zur Konstruktion von Netzen zum Sortieren von bitonischen Folgen ablesen. Wir nennen ein Netzwerk zum Sortieren einer bitonischen Folge mit Länge n nach dem Bitonic-merge-Verfahren ein *BM-Netz der Größe n* . Ein Vergleichsmodul ist ein BM-Netz der Größe 2. Nehmen wir an, wir haben bereits zwei BM-Netze der Größe $n/2$. Dann ist das in Abbildung 9.20 gezeigte Netz ein BM-Netz der Größe n .

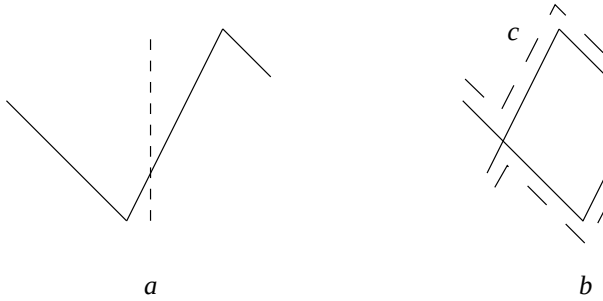


Abbildung 9.19

Für die spätere Realisierung eines Sortiernetzes weisen wir bereits hier auf eine wichtige Eigenschaft von BM-Netzen hin. Nehmen wir an, daß $n = 2^k$ ist und die Folgenindizes der in das BM-Netz der Größe n eingegebenen Schlüssel als Dualzahlen der Länge k dargestellt werden. Dann kann man aus Abbildung 9.20 sofort ablesen, daß die Schlüssel, die in einem Vergleichsmodul in der ersten Spalte des Netzes miteinander verglichen werden, Indizes haben, deren Dualdarstellung sich genau an der höchstwertigen, also k -ten Position von rechts unterscheidet.

Beispiel: Ist $n = 8$, so werden in der ersten Spalte von Vergleichsmoduln die Schlüsselpaare mit folgenden Indizes in Dualdarstellung mit Länge 3 miteinander verglichen:

$$(000, 100) \quad (001, 101) \quad (010, 110) \quad (011, 111)$$

Wegen des rekursiven Aufbaus von BM-Netzen gilt eine entsprechende Aussage natürlich auch für die in BM-Netzen mit Größe $n/2$ in Abbildung 9.20 auftretenden Vergleichsmoduln.

Eine in ein BM-Netz der Größe $n = 2^k$ eingegebene bitonische Zahlenfolge verläßt das Netz aufsteigend sortiert, nachdem jede Zahl genau k Vergleichsmoduln durchlaufen hat. Natürlich kann man auf dieselbe Weise ein Netz konstruieren, das eine bitonische Folge absteigend sortiert. Dazu genügt es, die Ausgänge L und H der Vergleichsmoduln in Abbildung 9.20 zu vertauschen und anzunehmen, daß die zwei in der rekursiven Konstruktion eines BM-Netzes der Größe n benutzten BM-Netze der Größe $n/2$ jeweils eine von oben nach unten absteigend sortierte Folge liefern. Wir kennzeichnen ein BM-Netz, das eine aufsteigend bzw. absteigend sortierte Folge liefert durch ein „+“ bzw. „-“. Mit Hilfe solcher Netze kann man jetzt rekursiv Netze zum Sortieren von Folgen der Länge n konstruieren. Wir nennen ein Netz dieser Art ein *BS-Netz der Größe n* und nehmen der Einfachheit halber wieder an, daß $n = 2^k$ für ein $k \geq 0$ ist. Falls $n = 2$ ist, definieren wir als auf- bzw. absteigend sortierendes BS-Netz der Größe 2 die aus je einem Vergleichsmodul bestehenden Netze, vgl. Abbildung 9.21.

Nehmen wir an, wir haben bereits zwei BS-Netze der Größe $n/2$, die zwei Folgen der Länge $n/2$ auf- bzw. absteigend sortieren. Dann erhalten wir ein BS-Netz der Größe n , das aufsteigend sortiert, indem wir es wie in Abbildung 9.22 gezeigt mit einem BM-Netz der Größe n verbinden. Ein BS-Netz der Größe n , das absteigend sortiert, erhält man analog.

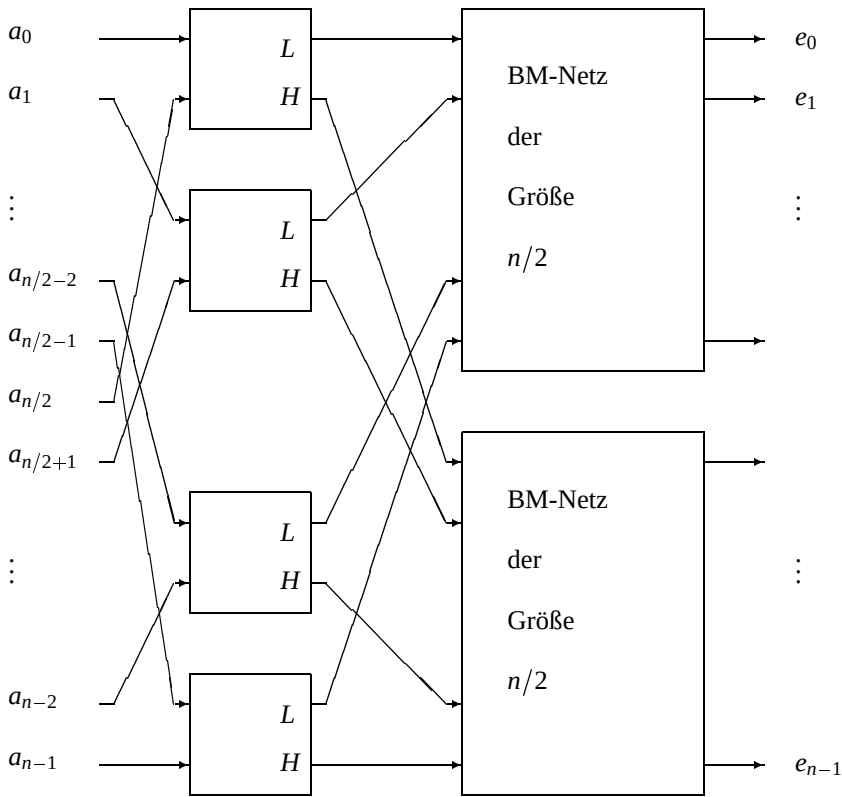


Abbildung 9.20

Abbildung 9.23 zeigt explizit ein nach diesem Prinzip konstruiertes BS-Netz für Zahlenfolgen der Länge 8. Die von links her erste Spalte von Vergleichsmoduln sortiert vier Paare von Zahlen zu auf- bzw. absteigenden Folgen der Länge 2; nach der ersten Spalte hat man also zwei bitonische Folgen mit Länge 4. Die nächsten zwei Spalten von Vergleichsmoduln stellen daraus eine bitonische Folge mit Länge 8 her und die letzten drei Spalten von Vergleichsmoduln stellen daraus schließlich eine aufsteigend sortierte Folge her.

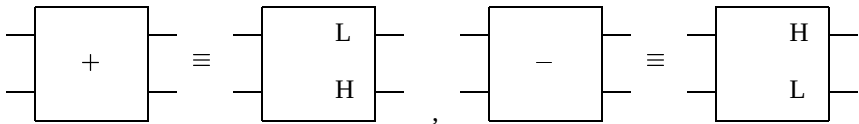


Abbildung 9.21

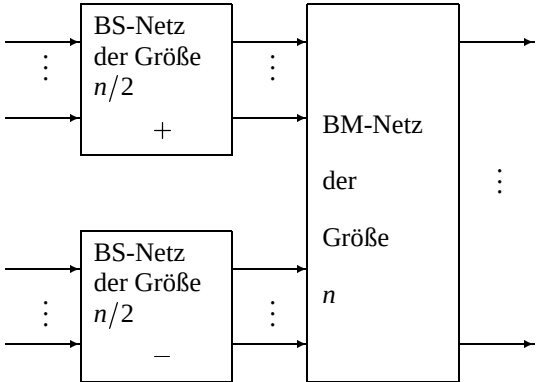


Abbildung 9.22

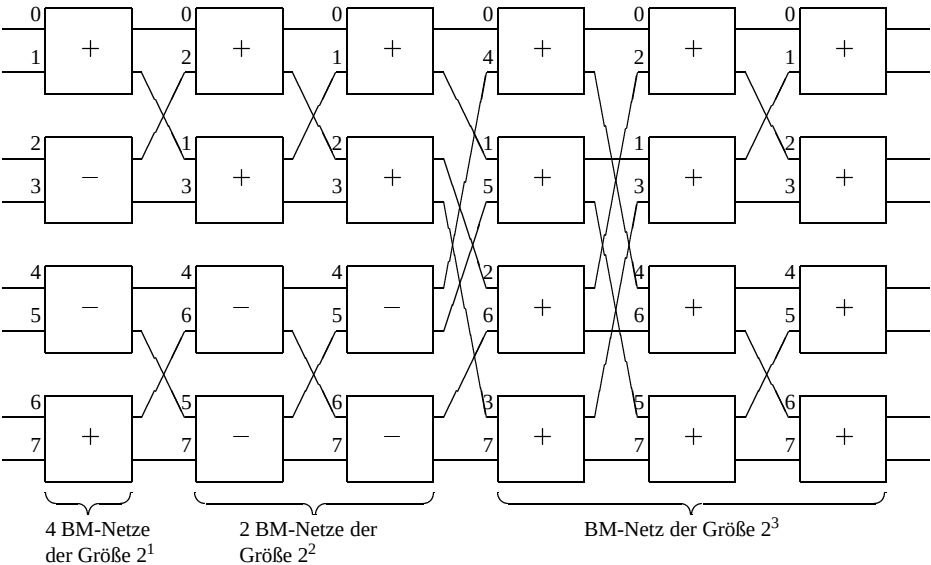


Abbildung 9.23

Wie im Falle des Odd-even-mergesort folgt auch hier, daß $n = 2^k$ Zahlen in $k(k+1)/2$ Schritten mit Hilfe eines BS-Netzes der Größe n sortiert werden können. Dabei besteht ein BS-Netz der Größe n aus $n/2$ BS-Netzen der Größe 2, $n/4$ BS-Netzen der Größe 4 usw. Jedes BS-Netz der Größe 2^j besteht wiederum aus j Spalten von Vergleichsmoduln, die nach dem in Abbildung 9.20 angegebenen Prinzip miteinander verbunden sind. Ein BS-Netz der Größe n besteht also aus $O(n \log^2 n)$ Vergleichsmoduln. Damit gilt der oben für OES-Netze formulierte Satz auch für BS-Netze.

Von H.S. Stone [1] wurde gezeigt, daß man mit nur $n/2$ Vergleichsmoduln insgesamt auskommen kann. Die Vergleichsmoduln werden allerdings mehrfach benutzt und die Eingänge zuvor geeignet permutiert. Betrachten wir ein BS-Netz für $n = 2^k$ Zahlen, also z.B. das Netz aus Abbildung 9.23 für acht Zahlen. Es ist aus $1 + 2 + 3 + \dots + k$ Spalten von je $n/2$ Vergleichsmoduln aufgebaut. Stellt man die Indizes aller n Schlüssel als Dualzahlen gleicher Länge k dar, so werden in der ersten Spalte Schlüssel in einen Vergleichsmodul zusammengeführt, deren Index sich genau an der 0-ten Position unterscheidet. Die Indizes von Schlüssel, die in Vergleichsmoduln der nächsten zwei Spalten zusammentreffen, unterscheiden sich durch die Bits an den Positionen 1 (in Spalte 2) und 0 (in Spalte 3) usw. Wir zählen dabei Bitpositionen wie üblich von rechts nach links, beginnend mit Position 0. D.h. die Bitpositionen, an denen sich die Indizes von miteinander verglichenen Schlüssel unterscheiden, sind der Reihe nach die folgenden Positionen:

$$0; 1, 0; 2, 1, 0; \dots; k-1, \dots, 1, 0.$$

Ein *Shuffle-exchange-Netz* der Größe $n = 2^k$ ist ein Netz, das die Eingänge so vertauscht, daß sich die Indizes je zweier aufeinanderfolgender Ausgänge genau im höchstwertigen Bit unterscheiden, also im Bit an Position $k-1$. Wird dasselbe Netz zweimal hintereinander durchlaufen, unterscheiden sich die Indizes der Eingänge von je zwei aufeinanderfolgenden Ausgängen an der zweithöchsten Bitposition, also an Bitposition $k-2$ usw. Abbildung 9.24 zeigt ein Shuffle-exchange-Netz der Größe 8.

Damit liegt es nahe, ein Sortiernetz aus einer einzigen Spalte von Vergleichsmoduln zu konstruieren und die Eingänge mit Hilfe eines Shuffle-exchange-Netzes zunächst so lange zu permutieren, bis die Schlüssel mit den richtigen Indizes in Vergleichsmoduln zusammentreffen. Abbildung 9.25 zeigt ein solches Netz für $n = 8$.

Bevor die $n = 2^k$ Schlüssel miteinander verglichen werden, deren Indizes sich an den Bitpositionen $j-1, \dots, 0$ unterscheiden, muß man die Vergleichsmoduln zunächst „abschalten“ und die Eingänge $k-j$ -mal das Shuffle-exchange-Netz durchlaufen lassen, für j von 1 bis k . Ein nach dem in Abbildung 9.25 angegebenen Prinzip aus $n/2$ (abschaltbaren) Vergleichsmoduln aufgebautes Sortiernetz muß also $k^2 = \log^2 n$ -mal durchlaufen werden, um n Schlüssel zu sortieren. Man erhält also:

Satz 9.5 Mit Hilfe eines aus $n/2$ Vergleichsmoduln aufgebauten, nach dem Shuffle-exchange-Prinzip verbundenen Netzes können n Schlüssel in Zeit $O(\log^2 n)$ sortiert werden.

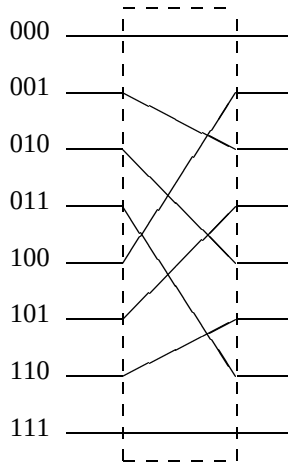


Abbildung 9.24

Weil das Sortieren von n Zahlen mit Hilfe eines einzigen Prozessors $\Omega(n \log n)$ Vergleichsoperationen von Schlüsseln erfordert, wird man nicht erwarten können, daß das Produkt der Zahl der Vergleichsmoduln eines Sortiernetzes und der zum parallelen Sortieren erforderlichen Zeit unter $\Omega(n \log n)$ liegt. Das schließt aber nicht aus, daß es Sortiernetze geben kann, die n Zahlen in logarithmischer Zeit mit $O(n)$ Prozessoren sortieren können. Ein wichtiges, neues Ergebnis in dieser Richtung stammt von Ajtai, Komlós und Szemerédi. Sie zeigen, daß ein aus $O(n \log n)$ Vergleichsmoduln bestehendes Netz n Zahlen in Zeit $O(\log n)$ sortieren kann.

9.2.3 Systolische Algorithmen

Der Begriff *systolische Algorithmen* stammt von Kung und Leiserson. Damit sollen Algorithmen mit folgenden Eigenschaften charakterisiert werden: Sie können mit Hilfe weniger Typen einfacher Prozessoren implementiert werden. Der Daten- und Kontrollfluß ist einfach und regulär. D.h. die einzelnen Prozessoren lassen sich in einem regelmäßigen Netz mit nur lokalen Verbindungen anordnen. Es wird extensiv Parallelverarbeitung und das Fließbandprinzip (Pipelining) zur Verarbeitung der Daten benutzt. Typischerweise bewegen sich mehrere Datenströme mit konstanter Geschwindigkeit über vorgegebene Wege im Netz und werden an Stellen, an denen sie sich treffen, parallel verarbeitet. Man stellt sich vor, daß die Rechnung nach einem globalen Takt abläuft. Alle beteiligten Prozessoren arbeiten schrittweise simultan. Zu jedem Zeitpunkt, d.h. in jedem Takt kann ein Prozessor nur mit seinen durch die vorgegebene Geometrie verbundenen Nachbarn kommunizieren. Die beteiligten Prozessoren verarbeiten also einen oder mehrere Datenströme, indem sie rhythmisch pulsierend operieren und Daten aufnehmen, verarbeiten und weiterleiten ähnlich wie das Blut durch die Arterien gepumpt

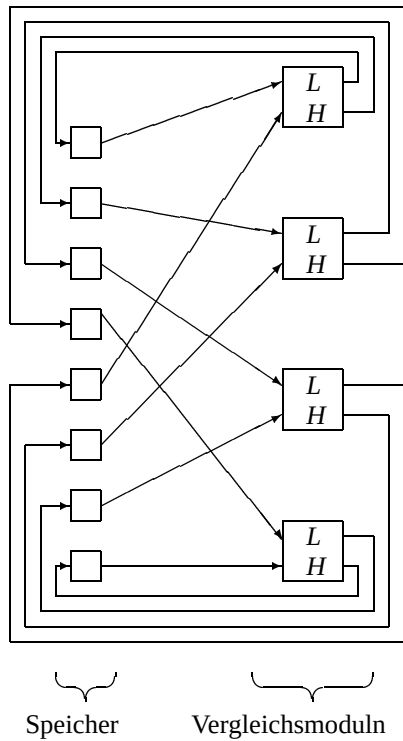


Abbildung 9.25

wird. Diese Analogie hat den Algorithmen und Arrays von Prozessoren den Namen systolisch eingebracht.

Kung und Leiserson zeigen unter anderem, wie man zwei Bandmatrizen mit Bandweite q in einem hexagonalen Array von q^2 Prozessoren miteinander multiplizieren kann. Dabei werden die Datenströme zur Berechnung der Produktmatrix $C = A \cdot B$ so aufeinander abgestimmt, daß die Ergebnismatrix C parallel zur Eingabe von A und B berechnet werden kann.

Wir beschränken uns auf einfachere Geometrien systolischer Netze und zeigen als repräsentatives Beispiel für diese Klasse von Algorithmen, wie eine Matrix-Vektor-Multiplikation auf einem linearen systolischen Array durchgeführt werden kann. Gegeben seien eine Matrix $A = (a_{ij})$ und ein Vektor $x = (x_1, \dots, x_n)^T$. Die Elemente des Produkts

$$(y_1, \dots, y_n)^T = A \cdot (x_1, \dots, x_n)^T$$

lassen sich wie folgt berechnen:

$$\begin{aligned} y_i^{(1)} &= 0 \\ y_i^{(k+1)} &= y_i^{(k)} + a_{ik}x_k \end{aligned}$$

$$y_i = y_i^{(n+1)}$$

Denn durch Induktion über k zeigt man leicht, daß $y_i^{(k)} = \sum_{j=1}^{k-1} a_{ij} \cdot x_j$, für alle k mit $2 \leq k \leq n+1$, ist.

Häufig ist A eine $n \times n$ Band-Matrix mit Bandweite $w = p + q - 1$ und x ein Vektor mit Länge n wie in folgendem Beispiel für $p = 2$ und $q = 3$ (vgl. Abbildung 9.26).

$$q = 3 \left\{ \begin{array}{cccccc} & \overbrace{a_{11} & a_{12}}^{p=2} & & & \\ a_{21} & a_{22} & a_{23} & & 0 & \\ a_{31} & a_{32} & a_{33} & a_{34} & & \\ & \underline{a_{42} \quad a_{43}} \quad \underline{a_{44} \quad a_{45}} & & & & \\ & & a_{53} & \dots & & \\ & & & \ddots & & \\ & 0 & & & & \end{array} \right\} \cdot \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ \vdots \end{bmatrix}$$

Abbildung 9.26

In diesem Beispiel ist

$$y_i = a_{i(i-2)}x_{i-2} + a_{i(i-1)}x_{i-1} + a_{ii}x_i + a_{i(i+1)}x_{i+1}.$$

Das Matrix-Vektor-Produkt kann nun dadurch berechnet werden, daß man die Elemente von A und x durch ein systolisches Array hindurchschiebt, das aus w linear miteinander verbundenen Prozessoren besteht, die jeweils einen Schritt zur Berechnung des Produkts $A \cdot x$ ausführen. Genauer läßt sich die Rechnung wie folgt beschreiben. Die y_i sind anfangs Null und wandern von rechts nach links, die x_i wandern von links nach rechts und die a_{ij} von oben nach unten wie in Abbildung 9.27.

In jedem geraden Takt wird das nächste y_i von rechts und in jedem ungeraden Takt das nächste x_j von links eingegeben. Die a_{ij} werden abwechselnd auf die geraden und ungeraden Prozessoren eingegeben. Die Datenströme während der ersten vier Takte veranschaulicht die folgende Tabelle 9.2.

Darin sind die von den Prozessoren durchgeführten Rechnungen nicht angegeben. Jedes y_i summiert auf seinem Weg durch das Array von Prozessoren der Reihe nach alle seine Produktterme

$$a_{i(i-2)}x_{i-2}, a_{i(i-1)}x_{i-1}, a_{ii}x_i, a_{i(i+1)}x_{i+1}$$

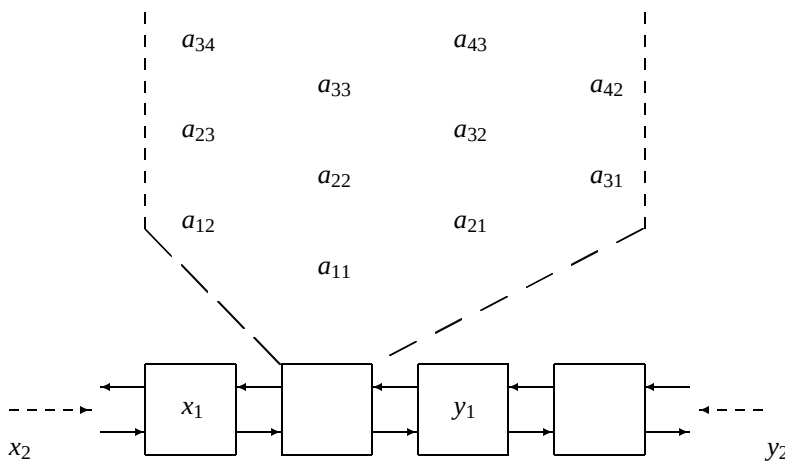


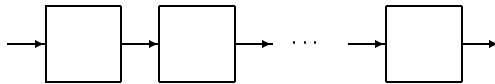
Abbildung 9.27

Zeit/ Takt				
1	x_1	—	y_1	—
2	—	x_1, y_1 a_{11}	—	y_2
3	x_2, y_1 a_{12}	—	x_1, y_2 a_{21}	—
4	—	x_2, y_2 a_{22}	—	x_1, y_3 a_{32}

Tabelle 9.2

auf, bevor es das Array am linken Ende verläßt. Beispielsweise verläßt y_1 das Array im vierten Takt mit Wert $y_1 = a_{11}x_1 + a_{12}x_2$, nachdem der Produktterm $a_{11}x_1$ im zweiten und $a_{12}x_2$ im dritten Takt berechnet wurde. Benachbarte Prozessoren sind jeweils abwechselnd aktiv. Ist $w = p + q - 1$ die Bandweite von A (und ohne Einschränkung w gerade), so werden nach w Takten die Komponenten des Produkts $y = Ax$ am linken Endprozessor ausgegeben, und zwar bei jedem zweiten Takt die nächste Komponente von y . Damit berechnet dieses systolische Array alle n Komponenten des Produkts $y = Ax$ in Zeit $2n + w$.

Die in diesem Beispiel benutzten Prozessoren sind „gedächtnislos“. Denn die jeweils nach links oder rechts weitergegebenen Daten hängen nur von den Eingaben, aber nicht von lokal zwischengespeicherten Werten ab. Im allgemeinen läßt man zu, daß die Prozessoren ein (beschränktes) Speichervermögen haben. Hat man beispielsweise eine lineare Folge von N Prozessoren und hat jeder Prozessor einen lokalen Speicher, der



zwei Schlüssel aufnehmen kann, so kann man mit Hilfe eines solchen N -Prozessor-Vektors $2N$ Schlüssel in Zeit $O(N)$ sortieren. Die $2N$ Schlüssel werden am linken Ende der Reihe nach eingegeben. Ein Prozessor wartet stets, bis er (erstmalig) zwei Schlüssel erhalten hat. Im nächsten Takt werden dann gleichzeitig und parallel ausgeführt: Weitergeben des Minimums der gespeicherten zwei Schlüssel an den rechten Nachbarn und Aufnahme des nächsten Schlüssels von links. Schließt man die zu sortierende Folge von Schlüsseln dadurch ab, daß man schließlich nur noch den „fiktiven“ Schlüssel ∞ von links her eingibt, so hat nach insgesamt $4N$ Schritten eine sortierte Schlüsselfolge den N -Prozessor-Vektor verlassen.

9.3 Aufgaben

Aufgabe 9.1

Verändern Sie die Funktion `kmp_search` aus Abschnitt 9.1.2 so, daß sie nicht nur die Position des ersten Vorkommens eines Musters von links in einem gegebenen Text, sondern alle Positionen, an denen das Muster im Text auftritt, liefert.

Aufgabe 9.2

Gegeben sei das Muster `abrakadabra` mit Länge 11. Berechnen Sie für dieses Mu-

ster die Werte $next[j]$ für alle j mit $1 \leq j \leq 11$. Geben Sie ferner die Anzahl der Vergleichsoperationen zwischen den Zeichen (des deutschen Alphabets einschließlich des Leerzeichens und der Satzzeichen) an, die das Verfahren von Knuth-Morris-Pratt ausführt, bis das Muster im Text

er sprach abrakadabra, aber ...

erstmal gefunden wird.

Aufgabe 9.3

Die Linearität des Verfahrens von Knuth-Morris-Pratt kann man sich anschaulich folgendermaßen klarmachen (vgl. *kmp_search*): Jeder Schritt (jeder Durchgang durch die **until**-Schleife) bewegt entweder den Textzeiger nach rechts oder das Muster. Beides kann jedoch höchstens N -mal geschehen, d.h. die Laufzeit ist linear in N . Um dieses Argument mathematisch umzusetzen, definieren wir eine *Potentialfunktion* $p(i, j) := 2i - j$, die sich aus dem Textzeiger und der Position des Musters ergibt. Zeigen Sie, daß jeder Durchlauf durch die **until**-Schleife das Potential erhöht, und folgern Sie daraus, daß das Verfahren von Knuth-Morris-Pratt lineare Laufzeit hat.

Aufgabe 9.4

Die in diesem Text dargestellte Variante des Verfahrens von Knuth-Morris-Pratt beruht auf einem Array *next*, das folgendermaßen definiert wurde:

$$next[j] := \begin{cases} 1 + \max\{0 \leq k \leq j-1 \mid b_1 \dots b_k = b_{j-k} \dots b_{j-1}\} & \text{falls } j > 1 \\ 0 & \text{falls } j = 1 \end{cases}$$

Dabei wird im Falle eines Mismatches an Stelle j die Information, welches Zeichen an Stelle j gelesen wurde, nicht ausgenutzt. Die folgende Definition des Arrays *next1* stellt dagegen sicher, daß das nach einem Mismatch mit b_j verglichene Zeichen von diesem verschieden ist. Die Verschiebungen des Musters sind also im allgemeinen größer als bei *next*. Der lineare Platzbedarf bleibt jedoch erhalten.

$$next1[j] := \begin{cases} 1 + \max\{0 \leq k \leq j-1 \mid b_1 \dots b_k = b_{j-k} \dots b_{j-1} \text{ und } b_{k+1} \neq b_j\} & \text{falls } j > 1 \text{ und ein solches } k \text{ existiert} \\ 0 & \text{sonst} \end{cases}$$

Lösen Sie die folgenden Aufgaben:

- Berechnen Sie *next1* für das Wort abrakadabra.
- Zeigen Sie, daß sich *next1* in Zeit $O(M)$ berechnen läßt (Hinweis: Benutzen Sie *next*).

Aufgabe 9.5

Berechnen Sie die möglichen Verschiebungen $\text{delta}-1(a)$ für jedes Zeichen a des deutschen Alphabets einschließlich des Leerzeichens und der Satzzeichen und $\text{delta}-2(j)$ nach der Vorkommens- und Match-Heuristik für das Muster **abrakadabra** und alle j mit $1 \leq j \leq 10$. Geben Sie ferner die genaue Zahl der Vergleichsoperationen zwischen Zeichen an, die das Verfahren von Boyer-Moore benötigt, um das Muster in dem in Aufgabe 9.2 genannten Text zu finden. Ändern Sie anschließend das Verfahren von Boyer-Moore so ab, daß alle Vorkommen eines Musters im Text gefunden werden.

Aufgabe 9.6

Unter einer *shared-memory* Prozessorarchitektur mit CRCW (Concurrent Read Concurrent Write) versteht man eine parallele Anordnung von Prozessoren $P_1, \dots, P_n$, die sich einen gemeinsamen Speicher teilen und bei der eine beliebige Anzahl von Prozessoren *gleichzeitig* von einer Speicherzelle lesen oder in eine Speicherzelle schreiben können. Ein Algorithmus für diese Architektur ist zulässig, falls zu jedem Zeitpunkt sichergestellt ist, daß

- niemals gleichzeitig ein Prozessor eine Speicherzelle lesen und ein anderer in sie schreiben möchte und,
 - falls zwei Prozessoren gleichzeitig in eine Speicherzelle schreiben, so schreiben sie denselben Wert.
- a) Entwerfen Sie zunächst einen sequentiellen Algorithmus, der in linearer Zeit für einen gegebenen Punkt p und ein Polygon P mit den Kanten $e_1, \dots, e_n$ feststellt, ob p innerhalb oder außerhalb von P liegt. (Hinweis: Betrachten Sie die Anzahl der Schnittpunkte eines (horizontalen) Strahls, der in p beginnt, mit den Kanten von P . Sie können davon ausgehen, daß alle Ecken von P eine von p verschiedene y -Koordinate haben.)
- b) Entwerfen Sie einen parallelen Algorithmus für das obige Problem, wobei ihnen eine CRCW-Architektur zur Verfügung stehe. Für diesen und den folgenden Aufgabenteil gelte, daß die Anzahl der Prozessoren gleich der Anzahl der Kanten von P sei. Ihr Algorithmus sollte nicht mehr als $O(\log n)$ Schritte benötigen. (Sie können davon ausgehen, daß eine Speicherzelle in der Lage ist, die Beschreibung einer Kante oder eine beliebige ganze Zahl aufzunehmen.)
- c) Entwerfen Sie einen parallelen Algorithmus für das obige Problem in einer CRCW-Umgebung, falls P konvex ist. Können Sie eine Laufzeit von $O(1)$ erreichen? Wie lange benötigt man, wenn es nicht erlaubt ist, gleichzeitig in eine Speicherzelle zu schreiben?

Aufgabe 9.7

Entwerfen Sie ein Netzwerk aus n Vergleichsmoduln, das für beliebige Zahlenfolgen der Länge n das Maximum der Zahlen in einer Zeit von $O(\log n)$ bestimmt. Sie können davon ausgehen, daß die Zahlen über n Eingabeleitungen simultan an dem Netz anliegen.

Aufgabe 9.8

Gegeben seien zwei aufsteigend sortierte Folgen $a_1, \dots, a_n$ und $b_1, \dots, b_n$, d.h. es gilt für alle $1 \leq i < n$, daß $a_i \leq a_{i+1}$ und $b_i \leq b_{i+1}$. Sei $c_1, \dots, c_n$ die Folge von Zahlen, die sich durch Verschmelzen der Folgen $a_1, a_3, a_5, \dots$ und $b_1, b_3, b_5, \dots$ ergibt, und $d_1, \dots, d_n$ die resultierende Folge bei Verschmelzung von $a_2, a_4, a_6, \dots$ und $b_2, b_4, b_6, \dots$. Zeigen Sie, daß für

$$\begin{aligned} e_1 &:= c_1 \\ e_{2i} &:= \min\{c_{i+1}, d_i\} && \text{für } 1 \leq i \leq n-1 \text{ und} \\ e_{2i+1} &:= \max\{c_{i+1}, d_i\} && \text{für } 1 \leq i \leq n-1 \text{ und} \\ e_{2n} &:= d_n \end{aligned}$$

gilt: $e_i \leq e_{i+1}$ für $1 \leq i \leq 2n-1$.

Literaturliste zu Kapitel 9: Ausgewählte Themen

Seite 618

[11] R. A. Baeza-Yates. Efficient Text Searching. PhD Dissertation, University of Waterloo, Research Report CS-89-17, Department of Computer Science, University of Waterloo, Ontario, Canada, 1989.

Seite 624

[91] D. E. Knuth, J. Morris und V. Pratt. Fast pattern matching in strings. SIAM Journal on Computing, 6:323-350, 1977.

[31] S. A. Cook. Linear time simulation of deterministic two-way pushdown automata. In Proc. IFIP Congress 71, TA-2, S. 172-179, Amsterdam, 1971. North Holland.

[6] J. Albert und Th. Ottmann. Automaten, Sprachen und Maschinen für Anwender. BI-Wissenschaftsverlag, Mannheim, 1983.

[2] A. V. Aho und M. Corasick. Efficient string matching: An aid to bibliographic search. Comm. ACM, 18:333-340, 1975.

[22] R. S. Boyer und J. S. Moore. A fast string searching algorithm. Comm. ACM, 20(10):762-772, 1977.

Seite 628

[22] R. S. Boyer und J. S. Moore. A fast string searching algorithm. Comm. ACM, 20(10):762-772, 1977.

Seite 630

[81] R. N. Hoorspool. Practical fast searching in strings. Software-Practice and Experience, 10:501-506, 1980.

Seite 631

[84] R. Karp und M. Rabin. Efficient randomized pattern-matching algorithms. IBM Journal of Research and Development, 31:249-260, 1987.

[11] R. A. Baeza-Yates. Efficient Text Searching. PhD Dissertation, University of Waterloo, Research Report CS-89-17, Department of Computer Science, University of Waterloo, Ontario, Canada, 1989.

Seite 632

[11] R. A. Baeza-Yates. Efficient Text Searching. PhD Dissertation, University of Waterloo, Research Report CS-89-17, Department of Computer Science, University of Waterloo, Ontario, Canada, 1989.

Seiten 634, 640

[187] E. Ukkonen. Finding approximate patterns in strings. J. of Algorithms, 6:132-137, 1985.

[186] E. Ukkonen. Algorithms for approximate string matching. Information and Control, 64:100-188, 1985.

Seite 643

[165] P. H. Sellers. The theory and computation of evolutionary distances: Pattern recognition. Journal of Algorithms, 1:359-373, 1980.

[68] G. H. Gonnet und R. Baeza-Yates. Handbook of Algorithms and Data Structures, 2. Auflage. Addison-Wesley, 1991.

[186] E. Ukkonen. Algorithms for approximate string matching. Information and Control, 64:100-188, 1985.

Seite 645

[5] S. G. Akl. Parallel Sorting Algorithms. Academic Press, 1985.

[145] I. Parberry. Parallel Complexity Theory. Pitman, London, 1987.

[147] N. Petkov. Systolische Algorithmen und Arrays. Akademie Verlag, Berlin, 1989.

- [159] C. C. Ribeiro. Parallel computer models and combinatorial algorithms. In Annals of Discrete Mathematics, volume 31, S. 325-364, 1987.
- [107] F. T. Leighton. Introduction to Parallel Algorithms and Architectures: Arrays, Trees, Hypercubes. Morgan Kaufmann Publishers, 1992.
- [153] M. J. Quinn. Designing Efficient Algorithms for Parallel Computers. McGraw Hill, New York, 1987.

Seite 651

- [154] M. J. Quinn und N. Deo. Parallel graph algorithms. ACM Computing Surveys, 16(3):319-348, 1984.
- [15] J. Bentley und Th. Ottmann. The power of a onedimensional vector of processors. In H. Noltemeier, Hrsg., Proc. WG'80, Graph-theoretic Concepts in Computer Science, S. 80-89. Lecture Notes in Computer Science 100, Springer, 1980.

Seite 652

- [19] D. Bitton, D. J. de Witt, D. K. Hsiao und J. Menon. A taxonomy of parallel sorting. ACM Computing Surveys, 16(3):287-318, September 1984.

Seite 653

- [5] S. G. Akl. Parallel Sorting Algorithms. Academic Press, 1985.
- [89] D. E. Knuth. The Art of Computer Programming, Vol.3: Sorting and Searching. Addison-Wesley, Reading, Massachusetts, 1973.
- [176] H. S. Stone. Parallel processing with the perfect shuffle. IEEE Transactions on Computers, C-20(2):153-161, 1971.

Seite 662

- [4] M. Ajtai, J. Komlo's und E. Szemere'di. An $O(n \log n)$ sorting network. In Proc. 15th Annual ACM Symposium on Theory of Computing, S. 1-9, 1983.
- [97] H. T. Kung und C. E. Leiserson. Algorithms for VLSI processor arrays. In L. Conway, Hrsg., Introduction to VLSI Systems. Addison Wesley, Reading, MA, 1980.

Literaturverzeichnis

- [1] G. M. Adelson-Velskii und Y. M. Landis. An algorithm for the organization of information. *Doklady Akademii Nauk SSSR*, 146:263–266, 1962. English Translation: *Soviet Math.* 3, 1259–1263.
- [2] A.V. Aho und M. Corasick. Efficient string matching: An aid to bibliographic search. *Comm. ACM*, 18:333–340, 1975.
- [3] A.V. Aho, J.E. Hopcroft und J.D. Ullman. *The Design and Analysis of Computer Algorithms*. Addison-Wesley, Reading, Massachusetts, 1974.
- [4] M. Ajtai, J. Komlós und E. Szemerédi. An $O(n \log n)$ sorting network. In *Proc. 15th Annual ACM Symposium on Theory of Computing*, S. 1–9, 1983.
- [5] S.G. Akl. *Parallel Sorting Algorithms*. Academic Press, 1985.
- [6] J. Albert und Th. Ottmann. *Automaten, Sprachen und Maschinen für Anwender*. BI-Wissenschaftsverlag, Mannheim, 1983.
- [7] B. Allen und J.I. Munro. Selforganizing search trees. *J. Assoc. Comput. Mach.*, 25(4):526–535, 1978.
- [8] O. Amble und D.E. Knuth. Ordered hash tables. *Computer Journal*, 17:135–142, 1974.
- [9] A. Andersson und Th. Ottmann. New tight bounds on uniquely represented dictionaries. In *SIAM Journal of Computing*, volume 24, S. 1091–1103, October 1995.
- [10] C.R. Aragon und R.G. Seidel. Randomized search trees. In *Proc. 30th IEEE Symposium on Foundations of Computer Science*, S. 540–545, 1989.
- [11] R.A. Baeza-Yates. *Efficient Text Searching*. PhD Dissertation, University of Waterloo, Research Report CS-89-17, Department of Computer Science, University of Waterloo, Ontario, Canada, 1989.
- [12] R. Bayer. Symmetric binary B-trees: Data structures and maintenance algorithms. *Acta Informatica*, 1:290–306, 1972.

- [13] J.R. Bell und C.H. Kaman. The linear quotient hash code. *Comm. ACM*, 13:675–677, 1970.
- [14] M. BenOr. Lower bounds for algebraic computation trees. In *Proc. 15th ACM Annual Symposium on Theory of Computing*, S. 80–86, 1983.
- [15] J. Bentley und Th. Ottmann. The power of a onedimensional vector of processors. In H. Noltemeier, Hrsg., *Proc. WG'80, Graph-theoretic Concepts in Computer Science*, S. 80–89. Lecture Notes in Computer Science 100, Springer, 1980.
- [16] J.L. Bentley. Programming pearls. *Comm. ACM*, 27:865–871, 1984.
- [17] J.L. Bentley und C. McGeoch. Amortized analyses of self-organizing sequential search heuristics. *Comm. ACM*, 28:404–411, 1985.
- [18] C. Berge. *Graphs and Hypergraphs*. North-Holland, Amsterdam, 1973.
- [19] D. Bitton, D.J. de Witt, D.K. Hsiao und J. Menon. A taxonomy of parallel sorting. *ACM Computing Surveys*, 16(3):287–318, September 1984.
- [20] M. Blum, R.W. Floyd, V.R. Pratt, R.L. Rivest und R.E. Tarjan. Time bounds for selection. *J. Computer and System Sciences*, 7:488–461, 1972.
- [21] O. Boruvka. O jistém problému minimálním. *Práce Moravské Přírodovědecké Společnosti*, 3:37–58, 1926.
- [22] R.S. Boyer und J.S. Moore. A fast string searching algorithm. *Comm. ACM*, 20(10):762–772, 1977.
- [23] R.P. Brent. Reducing the retrieval time of scatter storage techniques. *Comm. ACM*, 16:105–109, 1973.
- [24] K.Q. Brown. Comments on “Algorithms for reporting and counting geometric intersections”. *IEEE Transactions on Computers*, C-29:147–148, 1980.
- [25] J.L. Carter und M.N. Wegman. Universal classes of hash functions. *Journal of Computer and System Sciences*, 18:143–154, 1979.
- [26] P. Celis. *Robin Hood Hashing*. Ph.D. dissertation, Technical Report CS-86-14, Waterloo, Ontario, Canada, 1986.
- [27] P. Celis, P.-A. Larson und J.I. Munro. Robin Hood hashing. In *Proc. 26th Annual Symposium on Foundations of Computer Science*, S. 281–288. Computer Society Press of the IEEE, 1985.
- [28] B.M. Chazelle. Reporting and counting arbitrary planar intersections. Technical Report CS-83-16, Dept. of Comp. Sci., Brown University, Providence, R.I., 1983.
- [29] B.M. Chazelle und H. Edelsbrunner. An optimal algorithm for intersecting line segments in the plane. In *Proc. 29th Annual Symposium on Foundations of Computer Science, White Plains*, S. 590–600, 1988.

- [30] N. Christofides. *Graph theory: An algorithmic approach*. Academic Press, New York, 1975.
- [31] S.A. Cook. Linear time simulation of deterministic two-way pushdown automata. In *Proc. IFIP Congress 71, TA-2*, S. 172–179, Amsterdam, 1971. North Holland.
- [32] J. Culberson. The effect of updates in binary search trees. In *Proc. 17th ACM Annual Symposium on Theory of Computing, Providence, Rhode Island*, S. 205–212, 1985.
- [33] K. Culik, Th. Ottmann und D. Wood. Dense multiway trees. *ACM Trans. Database Systems*, 6:486–512, 1981.
- [34] B. Delaunay. Sur la sphère vide. *Bull. Acad. Sci. USSR Sci. Mat. Nat.*, 7:793–800, 1934.
- [35] E.W. Dijkstra. A note on two problems in connexion with graphs. *Numer. Math.*, 1:269–271, 1959.
- [36] E.W. Dijkstra. Smoothsort, an alternative for sorting in situ. *Science of Computer Programming*, 1:223–233, 1982. Vgl. auch: Errata, *Science of Computer Programming* 2:85, 1985.
- [37] E.A. Dinic. Algorithm for solution of a problem of maximal flow in a network with power estimation. *Soviet Math. Dokl.*, 11:1277–1280, 1970.
- [38] W. Dobosiewicz. Sorting by distributive partitioning. *Information Processing Letters*, 7(1):1–6, 1978.
- [39] J.R. Driscoll, H.N. Gabow, R. Shrairman und R.E. Tarjan. Relaxed heaps: An alternative to Fibonacci heaps with applications to parallel computation. *Comm. ACM*, 31:1343–1354, 1988.
- [40] B. Durian. Quicksort without a stack. In J. Gruska, B. Rován und J. Wiederman, Hrsg., *Proc. Math. Foundations of Computer Science, Prag*, S. 283–289. Lecture Notes in Computer Science 233, Springer, 1986.
- [41] H. Edelsbrunner. Dynamic data structures for orthogonal intersection queries. Technical Report 59, IIG, Technische Universität Graz, 1980.
- [42] H. Edelsbrunner. *Algorithms in Combinatorial Geometry*. Springer, Berlin, 1987.
- [43] H. Edelsbrunner und J. van Leeuwen. Multidimensional data structures and algorithms, a bibliography. Technical Report 104, IIG, Technische Universität Graz, 1983.
- [44] J. Edmonds. Paths, trees, and flowers. *Canad. J. Math.*, 17:449–467, 1965.
- [45] J. Edmonds und R.M. Karp. Theoretical improvements in algorithmic efficiency for network flow problems. *J. Assoc. Comput. Mach.*, 19:248–264, 1972.

- [46] P. Elias, A. Feinstein und C.E. Shannon. Note on maximum flow through a network. *IRE Trans. Inform. Theory*, IT-2:117–119, 1956.
- [47] R.J. Enbody und H.C. Du. Dynamic hashing schemes. *ACM Computing Surveys*, 20(2):85–113, 1988.
- [48] L. Euler. Solutio problematis ad geometriam situs pertinentis. *Comment. Acad. Sci. Imper. Petropol.*, 8:128–140, 1736.
- [49] S. Even. *Graph algorithms*. Computer Science Press, Potomac, Maryland, 1979.
- [50] S. Even und R.E. Tarjan. Network flow and testing graph connectivity. *SIAM J. Comput.*, 4:507–518, 1975.
- [51] R. Fagin, J. Nievergelt, N. Pippenger und H.R. Strong. Extendible hashing — a fast access method for dynamic files. *ACM Trans. Database Systems*, 4(3):315–344, 1979.
- [52] W. Feller. *An Introduction to Probability Theory and its Applications, Volume I*. John Wiley & Sons, New York, 1968.
- [53] P. Flajolet. On the performance evaluation of extendible hashing and trie searching. *Acta Informatica*, 20:345–369, 1983.
- [54] P. Flajolet und C. Puech. Partial match retrieval of multidimensional data. *J. Assoc. Comput. Mach.*, 33(2):371–407, 1986.
- [55] R.W. Floyd. Algorithm 245, treesort 3. *Comm. ACM*, 7:701, 1964.
- [56] L.R. Ford Jr. *Network flow theory*. Paper P-923, RAND Corp., Santa Monica, CA, 1956.
- [57] L.R. Ford Jr. und D.R. Fulkerson. Maximal flow through a network. *Canad. J. Math.*, 8:399–404, 1956.
- [58] L.R. Ford Jr. und D.R. Fulkerson. *Flows in networks*. Princeton University Press, Princeton, N.J., 1962.
- [59] A.R. Forrest. Guest editor's introduction to special issue on computational geometry. *ACM Transactions on Graphics*, 3(4):241–243, 1984.
- [60] M.L. Fredman und R.E. Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. *J. Assoc. Comput. Mach.*, 34:596–615, 1987.
- [61] H.N. Gabow. *Implementation of algorithms for maximum matching on nonbipartite graphs*. Dissertation, Dept. Electrical Engineering, Stanford Univ., Stanford, CA, 1973.
- [62] H.N. Gabow und R.E. Tarjan. A linear-time algorithm for a special case of disjoint set union. In *Proc. 15th Annual ACM Symposium on Theory of Computing*, S. 246–251, 1983.

- [63] Z. Galil, S. Micali und H. Gabow. Maximal weighted matching on general graphs. In *Proc. 23rd Annual Symposium on Foundations of Computer Science*, S. 255–261, 1982.
- [64] Z. Galil und A. Naamad. An $O(E \cdot V \cdot \log^2 V)$ algorithm for the maximum flow problem. *J. Comput. System Sci.*, 21:203–217, 1980.
- [65] A. Gibbons. *Algorithmic graph theory*. Cambridge University Press, Cambridge, 1985.
- [66] M.C. Golumbic. *Algorithmic graph theory and perfect graphs*. Academic Press, New York, 1980.
- [67] G.H. Gonnet. *Handbook of Algorithms and Data Structures*. Addison-Wesley, 1984.
- [68] G.H. Gonnet und R. Baeza-Yates. *Handbook of Algorithms and Data Structures, 2. Auflage*. Addison-Wesley, 1991.
- [69] G.H. Gonnet und I. Munro. Efficient ordering of hash tables. *SIAM J. Comput.*, 8(3):463–478, 1979.
- [70] L.J. Guibas und R. Sedgewick. A dichromatic framework for balanced trees. In *Proc. 19th Annual Symposium on Foundations of Computer Science, Ann Arbor, Michigan*, S. 8–21, 1978.
- [71] R.H. Güting. Optimal divide-and-conquer to compute measure and contour for a set of iso-oriented rectangles. *Acta Informatica*, 21:271–291, 1984.
- [72] R.H. Güting und Th. Ottmann. New algorithms for special cases of the hidden line elimination problem. *Computer Vision and Image Processing*, 40:188–204, 1987.
- [73] R.H. Güting und D. Wood. Finding rectangle intersections by divide-and-conquer. *IEEE Transactions on Computers*, C-33:671–675, 1984.
- [74] S. Hanke, Th. Ottmann und E. Soisalon-Soininen. *Relaxed Balancing Made Simple*. Technical report, Institut für Informatik, Universität Freiburg, Germany and Laboratory of Information Processing Science, Helsinki University, Finland, 1996. (anonymous ftp from [ftp.informatik.uni-freiburg.de](ftp://ftp.informatik.uni-freiburg.de) in directory /documents/reports/report71/) (<http://hyperg.informatik.uni-freiburg.de/Report71>).
- [75] F. Harary. *Graph Theory*. Addison-Wesley, Reading, Massachusetts, 1969.
- [76] J.H. Hester und D.S. Hirschberg. Self-organizing linear search. *ACM Computing Surveys*, 17:295–311, 1985.
- [77] P. Heyderhoff, Hrsg. *Bundeswettbewerb Informatik: Aufgaben und Lösungen, Band 1*. Ernst Klett Schulbuchverlag, 1989.
- [78] K. Hinrichs. *The Grid File System: Implementation and case studies of applications*. Ph.D. dissertation, Institut für Informatik, ETH Zürich, Schweiz, 1985.

- [79] D.S. Hirschberg. An insertion technique for one-sided height-balanced trees. *Comm. ACM*, 19:471–473, 1976.
- [80] C.A.R. Hoare. Quicksort. *Computer Journal*, 5:10–15, 1962.
- [81] R.N. Hoorspool. Practical fast searching in strings. *Software-Practice and Experience*, 10:501–506, 1980.
- [82] V. Jarník. O jistém problému minimálním. *Práce Moravské Přírodovědecké Společnosti*, 6:57–63, 1930.
- [83] D. Jungnickel. *Graphen, Netzwerke und Algorithmen*. BI-Wissenschaftsverlag, Mannheim, Wien, Zürich, 1987.
- [84] R. Karp und M. Rabin. Efficient randomized pattern-matching algorithms. *IBM Journal of Research and Development*, 31:249–260, 1987.
- [85] A.V. Karzanov. Determining the maximal flow in a network by the method of preflows. *Soviet Math. Dokl.*, 15:434–437, 1974.
- [86] J.L.W. Kessels. On-the-fly optimization of data structures. In *Comm. ACM*, 26, S. 895–901, 1983.
- [87] D.G. Kirkpatrick. Optimal search in planar subdivisions. *SIAM J. Comput.*, 12(1):28–35, 1983.
- [88] R. Klein, O. Nurmi, Th. Ottmann und D. Wood. A dynamic fixed windowing problem. *Algorithmica*, 4:535–550, 1989.
- [89] D.E. Knuth. *The Art of Computer Programming, Vol.3: Sorting and Searching*. Addison-Wesley, Reading, Massachusetts, 1973.
- [90] D.E. Knuth. Big omicron and big omega and big theta. *SIGACT News*, 8(2):18–24, 1976.
- [91] D.E. Knuth, J. Morris und V. Pratt. Fast pattern matching in strings. *SIAM Journal on Computing*, 6:323–350, 1977.
- [92] D. König. Graphok és matrixok. *Matematikai és Fizikai Lapok*, 38:116–119, 1931.
- [93] D.C. Kozen. *The Design and Analysis of Algorithms*. Springer, New York u.a., 1991. Texts and Monographs in Computer Science.
- [94] R. Krishnamurthy und K.-Y. Whang. *Multilevel Grid Files*. IBM Research Report, Yorktown Heights, 1985.
- [95] M.A. Kronrod. An optimal ordering algorithm without a field of operation. *Dokladi Akademii Nauk SSSR*, 186:1256–1258, 1969.
- [96] J.B. Kruskal. On the shortest spanning subtree of a graph and the traveling salesman problem. In *Proc. AMS* 7, S. 48–50, 1956.

- [97] H.T. Kung und C.E. Leiserson. Algorithms for VLSI processor arrays. In L. Conway, Hrsg., *Introduction to VLSI Systems*. Addison Wesley, Reading, MA, 1980.
- [98] K. Larsen. AVL trees with relaxed balance. In *Proc. 8th International Parallel Processing Symposium, IEEE Computer Society Press*, S. 888–893, 1994.
- [99] K. Larsen und R. Fagerberg. B-trees with relaxed balance. In *Proc. 9th International Parallel Processing Symposium, IEEE Computer Society Press*, S. 196–202, 1995.
- [100] P.A. Larson. Dynamic hashing. *BIT*, 18:184–201, 1978.
- [101] P.A. Larson. Linear hashing with partial expansions. In *Proc. 6th Conference on Very Large Data Bases*, S. 224–232, Montreal, 1980.
- [102] P.A. Larson. Dynamische Hashverfahren. *Informatik-Spektrum*, 6(1):7–19, 1983.
- [103] P.A. Larson. Dynamic Hash Tables. *Comm. ACM*, 31(4):446–457, 1988.
- [104] E.L. Lawler. *Combinatorial optimization: Networks and matroids*. Holt, Rinehart, and Winston, New York, 1976.
- [105] D.T. Lee und F.P. Preparata. Computational geometry — a survey. *IEEE Transactions on Computers*, C-33(12):1072–1102, 1984.
- [106] J. van Leeuwen und H.M. Overmars. Stratified balanced search trees. *Acta Informatica*, 18:345–359, 1983.
- [107] F.T. Leighton. *Introduction to Parallel Algorithms and Architectures: Arrays, Trees, Hypercubes*. Morgan Kaufmann Publishers, 1992.
- [108] T. Lengauer. Efficient algorithms for the constraint generation for integrated circuit layout compaction. In M. Nagl und J. Perl, Hrsg., *Proc. WG'83, Graph-Theoretic Concepts in Computer Science, Osnabrück*, S. 219–230, Linz, 1983. Trauner.
- [109] E.E. Lindstrom, J.S. Vitter und C.K. Wong, Hrsg. *IEEE Transactions on Computers, Special Issue on Sorting*, C-34. 1985.
- [110] W. Litwin. Virtual hashing: a dynamically changing hashing. In *Proc. 4th Conference on Very Large Data Bases*, S. 517–523, 1978.
- [111] W. Litwin. *Hachage Virtuel: une nouvelle technique d'adressage de memoires*. Ph.D. thesis, Univ. Paris VI, 1979. Thèse de Doctorat d'Etat.
- [112] W. Litwin. Linear hashing: A new tool for file and table addressing. In *Proc. 6th Conference on Very Large Data Bases*, S. 212–223, Montreal, 1980.
- [113] V.Y. Lum, P.S.T. Yuen und M. Dodd. Key-to-address transform techniques: a fundamental performance study on large existing formatted files. *Comm. ACM*, 14:228–235, 1971.

- [114] G.E. Lyon. Packed scatter tables. *Comm. ACM*, 21(10):857–865, 1978.
- [115] V.M. Malhotra, M.P. Kumar und S.N. Maheshwari. An $O(|V|^3)$ algorithm for finding maximum flows in networks. *Information Processing Letters*, 7:277–278, 1978.
- [116] E.G. Mallach. Scatter storage techniques: A unifying viewpoint and a method for reducing retrieval times. *The Computer Journal*, 20(2):137–140, 1977.
- [117] H. Mannila. Measures of presortedness and optimal sorting algorithms. *IEEE Transactions on Computers*, C-34:318–325, 1985.
- [118] H.A. Maurer, Th. Ottmann und H.-W. Six. Implementing dictionaries using binary trees of very small height. *Information Processing Letters*, 5(1):11–14, 1976.
- [119] E.M. McCreight. Efficient algorithms for enumerating intersecting intervals and rectangles. Technical Report PARC CSL–80–9, Xerox Palo Alto Res. Ctr., Palo Alto, CA, 1980.
- [120] E.M. McCreight. Priority search trees. *SIAM J. Comput.*, 14(2):257–276, 1985.
- [121] K. Mehlhorn. *Data structures and algorithms, Vol. 2: Graph algorithms and NP-completeness*. Springer, Berlin, 1984.
- [122] K. Mehlhorn. *Data structures and algorithms, Vol. 3: Multidimensional searching and computational geometry*. Springer, Berlin, 1984.
- [123] K. Mehlhorn. *Datenstrukturen und effiziente Algorithmen, Band 1, Sortieren und Suchen*. Teubner, Stuttgart, 1986.
- [124] H. Mendelson. Analysis of extendible hashing. *IEEE Trans. Softw. Eng.*, SE-8(6):611–619, 1982.
- [125] S. Micali und V.V. Vazirani. An $O(\sqrt{|V|} \cdot |E|)$ algorithm for finding maximum matching in general graphs. In *Proc. 21st Annual Symposium on Foundations of Computer Science*, S. 17–27, 1980.
- [126] D.E. Muller und F.P. Preparata. Finding the intersection of two convex polyhedra. *Theoretical Computer Science*, 7(2):217–236, 1978.
- [127] J.I. Munro und X. Papadakis. Deterministic skip lists. In *Proc. 3rd Annual Symposium On Discrete Algorithms (SODA)*, S. 367–375, 1992.
- [128] I. Nievergelt und C.K. Wong. On binary search trees. In *Proc. IFIP Congress 71 North-Holland Publishing Co., Amsterdam*, S. 91–98, 1972.
- [129] J. Nievergelt, H. Hinterberger und K.C. Sevcik. The grid file: An adaptable, symmetric multikey file structure. *ACM Trans. Database Systems*, 9(1):38–71, 1984.
- [130] J. Nievergelt und F.P. Preparata. Plane-sweep algorithms for intersecting geometric figures. *Comm. ACM*, 25:739–747, 1982.

- [131] J. Nievergelt und E.M. Reingold. Binary search trees of bounded balance. *SIAM Journal on Computing*, 2:33–43, 1973.
- [132] O. Nurmi und E. Soisalon Soininen. Uncoupling updating and rebalancing in chromatic binary trees. In *Proc. 10th ACM Symposium on Principles of Database Systems*, S. 192–198, 1991.
- [133] O. Nurmi, E. Soisalon Soininen und D. Wood. Concurrency control in database structures with relaxed balance. In *Proc. 6th ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, San Diego, California*, S. 170–176, 1987.
- [134] H. Olivié. *A Study of Balanced Binary Trees and Balanced One-Two Trees*. PhD thesis, University of Antwerpen, 1980.
- [135] H. Olivié. A new class of balanced search trees: Half-balanced binary search trees. *RAIRO Informatique Théorique*, 16:51–71, 1982.
- [136] J.A. Orenstein. A dynamic hash file for random and sequential accessing. In *Proc. 9th Conference on Very Large Data Bases*, S. 132–141, Florenz, 1983.
- [137] Th. Ottmann, H.-W. Six und D. Wood. Right brother trees. *Comm. ACM*, 21:769–776, 1978.
- [138] Th. Ottmann, H.-W. Six und D. Wood. On the correspondence between AVL trees and brother trees. *Computing*, 23:43–54, 1979.
- [139] Th. Ottmann und P. Widmayer. On the placement of line segments into a skeleton structure. Technical Report 114, Institut für Angewandte Informatik und Formale Beschreibungsverfahren Universität Karlsruhe, 1982.
- [140] Th. Ottmann, P. Widmayer und D. Wood. A worst-case efficient algorithm for hidden line elimination. *International Journal Comp. Math.*, 18:93–119, 1985.
- [141] Th. Ottmann und D. Wood. A comparison of iterative and defined classes of search trees. *International Journal of Computer and Information Sciences*, 11:155–178, 1982.
- [142] Th. Ottmann und D. Wood. Dynamical sets of points. *Computer Vision, Graphics, and Image Processing*, 27:157–166, 1984.
- [143] T. Papadakis, J.I. Munro und P.V. Poblete. Analysis of the expected search cost in skip lists. In *Proc. 2nd Scandinavian Workshop on Algorithm Theory*, S. 160–172. Lecture Notes in Computer Science 447, Springer, 1990.
- [144] C.H. Papadimitriou und K. Steiglitz. *Combinatorial optimization: Networks and complexity*. Prentice-Hall, Englewood Cliffs, New Jersey, 1982.
- [145] I. Parberry. *Parallel Complexity Theory*. Pitman, London, 1987.
- [146] W.W. Peterson. Addressing for random-access storage. *IBM J. Research and Development*, 1:130–146, 1957.

- [147] N. Petkov. *Systolische Algorithmen und Arrays*. Akademie Verlag, Berlin, 1989.
- [148] G. Poonan. *Optimal Placement of Entries in Hash Tables*. ACM Computer Science Conference, 25, 1976.
- [149] F.P. Preparata und M.I. Shamos. *Computational Geometry: An Introduction*. Springer, 1985.
- [150] R.C. Prim. Shortest connection networks and some generalizations. *Bell System Techn. J.*, 36:1389–1401, 1957.
- [151] W. Pugh. Skip lists: A probabilistic alternative to balanced trees. In *Proc. Workshop of Algorithms and Data Structures*, S. 437–449, 1989. Lecture Notes in Computer Science 382.
- [152] W. Pugh. Skip lists: A probabilistic alternative to balanced trees. *Comm. ACM*, 33(6):668–676, 1990. (Erste Fassung in [151]).
- [153] M.J. Quinn. *Designing Efficient Algorithms for Parallel Computers*. McGraw-Hill, New York, 1987.
- [154] M.J. Quinn und N. Deo. Parallel graph algorithms. *ACM Computing Surveys*, 16(3):319–348, 1984.
- [155] C.E. Radtke. The use of quadratic residue search. *Comm. ACM*, 13:103–105, 1970.
- [156] K.R. Räihä und S.H. Zweben. An optimal insertion algorithm for one-sided height-balanced binary search trees. *Comm. ACM*, 22:508–512, 1979.
- [157] K. Ramamohanarao und R. Sacks-Davis. Recursive linear hashing. *ACM Trans. Database Systems*, 9(3):369–391, 1984.
- [158] M. Regnier. Analysis of grid file algorithms. *BIT*, 25(2):335–357, 1985.
- [159] C.C. Ribeiro. Parallel computer models and combinatorial algorithms. In *Annals of Discrete Mathematics*, volume 31, S. 325–364, 1987.
- [160] R.L. Rivest. Partial-match retrieval algorithms. *SIAM J. Comput.*, 5(1):19–50, 1976.
- [161] R.L. Rivest. Optimal arrangement of keys in a hash table. *J. Assoc. Comput. Mach.*, 25(2):200–209, 1978.
- [162] F.E. Roberts. Graph theory and its applications to problems of society. In *SIAM CBMS-NSF Regional Conference Series in Applied Mathematics* 29, Philadelphia, 1978. SIAM.
- [163] M. Schlag, F. Luccio, P. Maestrini, D.T. Lee und C.K. Wong. A visibility problem in VLSI layout compaction. In F.P. Preparata, Hrsg., *Advances in Computing Research*, volume 2, S. 259–282. JAI Press, 1985.

- [164] A. Schmitt. On the number of relational operators necessary to compute certain functions of real variables. *Acta Informatica*, 19:297–304, 1983.
- [165] P.H. Sellers. The theory and computation of evolutionary distances: Pattern recognition. *Journal of Algorithms*, 1:359–373, 1980.
- [166] M.I. Shamos. *Computational Geometry*. Dissertation, Dept. of Comput. Sci., Yale University, 1978.
- [167] M.I. Shamos und D. Hoey. Closest-point problems. In *Proc. 16th Annual Symposium on Foundations of Computer Science*, S. 151–162, 1975.
- [168] D.L. Shell. A high-speed sorting procedure. *Comm. ACM*, 2:30–32, 1959.
- [169] Y. Shiloach. *An $O(n \cdot I \log^2 I)$ maximum-flow algorithm*. Tech. Report STAN-CS-78-802, Computer Science Department, Stanford University, CA, 1978.
- [170] H.-W. Six und L. Wegner. EXQUISIT: Applying quicksort to external files. In *Proc. 19th Annual Allerton Conference on Communication, Control and Computing*, S. 348–354, 1981.
- [171] D.D. Sleator und R.E. Tarjan. A data structure for dynamic trees. *J. Computer and System Sciences*, 26:362–391, 1983.
- [172] D.D. Sleator und R.E. Tarjan. Amortized efficiency of list update and paging rules. *Comm. ACM*, 28:202–208, 1985.
- [173] D.D. Sleator und R.E. Tarjan. Self-adjusting binary search trees. *Journal of the ACM*, 32:652–686, 1985.
- [174] L. Snyder. On uniquely represented data structures. In *Proc. 18th Annual Symposium on Foundations of Computer Science, Providence, Rhode Island*, S. 142–147, 1977.
- [175] T.A. Standish. *Data Structure Techniques*. Addison-Wesley, Reading, Massachusetts, 1980.
- [176] H.S. Stone. Parallel processing with the perfect shuffle. *IEEE Transactions on Computers*, C-20(2):153–161, 1971.
- [177] V. Strassen. Gaussian elimination is not optimal. *Numer. Math.*, 13:354–356, 1969.
- [178] M. Tamminen. Order preserving extendible hashing and bucket tries. *BIT*, 21(4):419–435, 1981.
- [179] M. Tamminen. The extendible cell method for closest point problems. *BIT*, 22:27–41, 1982.
- [180] R.E. Tarjan. Data structures and network algorithms. In *SIAM CBMS-NSF Regional Conference Series in Applied Mathematics 44*, Philadelphia, 1983. SIAM.

- [181] R.E. Tarjan. Updating a balanced search tree in $O(1)$ rotations. *Information Processing Letters*, 16:253–257, 1983.
- [182] R.E. Tarjan und J. van Leeuwen. Worst case analysis of set union algorithms. *J. Assoc. Comput. Mach.*, 31:245–281, 1984.
- [183] G. Toussaint, Hrsg. *Computational Geometry*. Elsevier North-Holland, N. Y., 1985.
- [184] L. Trabb Pardo. Stable sorting and merging with optimal space and time bounds. *SIAM J. Comput.*, 6:351–372, 1977.
- [185] V. Turan Sós. On the theory of diophantine approximations. *Acta Math. Acad. Sci. Hung.*, 8:461–472, 1957.
- [186] E. Ukkonen. Algorithms for approximate string matching. *Information and Control*, 64:100–188, 1985.
- [187] E. Ukkonen. Finding approximate patterns in strings. *J. of Algorithms*, 6:132–137, 1985.
- [188] J.D. Ullman. A note on the efficiency of hash functions. *J. Assoc. Comput. Mach.*, 19(3):569–575, 1972.
- [189] G. Voronoi. Nouvelles applications des paramètres continus à la théorie des formes quadratiques. Deuxième Mémoire: Recherches sur les paralléloèdres primitifs. *J. Reine angew. Math.*, 134:198–287, 1908.
- [190] J. Vuillemin. A data structure for manipulating priority queues. *Comm. ACM*, 21:309–315, 1978.
- [191] S. Warshall. A theorem on Boolean matrices. *J. Assoc. Comp. Mach.*, 9:11–12, 1962.
- [192] L. Wegner. Quicksort for equal keys. *IEEE Transactions on Computers*, C-34:362–366, 1985.
- [193] F.A. Williams. Handling identifiers as internal symbols in language processors. *Comm. ACM*, 2(6):21–24, 1959.
- [194] J.W.J. Williams. Algorithm 232. *Comm. ACM*, 7:347–348, 1964.
- [195] D. Wood. An isothetic view of computational geometry. Technical Report CS-84-01, Department of Computer Science, University of Waterloo, Jan. 1984.
- [196] A.C. Yao. On random 2-3 trees. *Acta Informatica*, 9:159–170, 1978.
- [197] A.C. Yao. A note on the analysis of extendible hashing. *Information Processing Letters*, 11:84–86, 1980.
- [198] A.C. Yao. Uniform hashing is optimal. *J. Assoc. Comput. Mach.*, 32(3):687–693, 1985.

- [199] A.C. Yao und F.F. Yao. The complexity of searching an ordered random table. In *Proc. 17th Annual Symposium on Foundations of Computer Science*, S. 173–177, 1976.
- [200] S.H. Zweben und M.A. McDonald. An optimal method for deletion in one-sided height-balanced trees. *Comm. ACM*, 21:441–445, 1978.

Index

O -Notation, 4

$\Omega(g)$, 4

Ω -Notation, 4

„don't care“-Symbole, 629

2-3-4-Bäume, 329

2-Ebenen-Sprungliste, 352

2d-Bäume, 367

3-Median-Strategie, 85

A-sort, 116

abstrakte Datentypen, 17, 20

Operationen für, 17

Access Min, 378, 383, 395, 415

addcost, 595

adjazent, 537

Adjazenzliste, 539

Adjazenzmatrix, 537

Adreßkollision, 169

Adreßtabellenverdoppelung, 215

ADT, 17

Implementierung eines, 20

Äquivalenzklassen, 553, 565

Äquivalenzrelation, 553

Algorithmen, 1

-begriff, 1

geometrische, 419

Korrektheit von, 2

parallele, *siehe* parallele Algorithmen

systolische, *siehe* systolische Algorithmen

Algorithmische Geometrie, 419

alleine bzgl. einer Zuordnung, 596

amortisierte Kosten, 163, 310, 400

Analyse

amortisierte Worst-case-, 163, 310, 400

des statischen Falls, 283

Fringe-, 285

Gestalts-, 252, 256

Random-tree-, 252

Anfangsanordnung, 474

arithmetischer Ausdruck, 59

Auswertung eines, 37

articulation point, 558

Aufteilungsmethode, 412

Aufspalten, 238

Aufspießproblem, 447, 451, 455

zweidimensionales, 488

Aufteilung in situ, 106

Aufteilungsphase, 128

Ausgangsgrad, 541

Auswahl, 148, 149

Auswahlbaum, 132

Auswahlproblem, 148

Auswahlschritt, 582, 583

Auswahlsort, 67

Auswahl und Ersetzen, 134

Automat

endlicher, *siehe* endlicher Automat

average-case-effizient, 273

average case, 3, 66

AVL-ausgeglichen, 260

AVL-Bäume, 260

azyklisch, 542

B-Bäume, 317

der Ordnung m , 319

- symmetrische binäre, 336
- Balancefaktor, 264
- Balanceinformation, 328
- balancierte Binärbäume, *siehe* Binärbäume, balancierte
- Bankkonto-Paradigma, 163, 310, 400
- Baum, 235
 - 2-3-4-, 329
 - 2d-, 367
 - AVL, *siehe* AVL-Bäume
 - dichter, 327
 - gefädelter, 250
 - geordneter, 235
 - gerichteter, 542
 - gewichtsbalancierter, 289
 - halb-balanciert, 336
 - Höhe eines, 237
 - Konstruieren eines, 238
 - leerer, 236
 - minimal spannender, *siehe* minimaler spannender Baum
 - natürlicher, 239, 245
 - Ordnung eines, 235
 - Quadranten-, 365
 - Rechts-Bruder-, 328
 - Rot-schwarz, *siehe* Rot-Schwarz-Bäume
 - Schichten eines, 336
 - spannender, *siehe* spannender Baum
 - Straßen eines, 336
 - stratifizierter, 336
 - Vielweg-, 236, 322
 - vollständiger, 237
- Baumpfeile, 556
- BB[α]-Baum, 289
- bcc, 557
- Belegungsfaktor, 170
- Bereichsanfrage, 219, 220, 223, 365, 426
 - partielle, 220, 365
 - zweidimensionale, 488
- best case, 3, 66
- best match, 500
- best match query, 207
- Besuchskosten, 253
- Bewegungen, 66
- Bewertung, 567
- biconnected, 557
- biconnected component, 557
- Binärbäume, 235
 - balancierte, 260
 - Durchlaufordnungen in, 248
 - linksseitig höhenbalancierte, 328
- Binärbaum-Sondieren, 193
- binary tree hashing, 193
- Binomialbäume, 387
- Binomialkoeffizient, 37
 - verallgemeinerter, 59
- Binomial Queues, 387, 389
- binsearch, 154, 155
- Binsort, 105
- bipartit, 598
- Birthday Paradox, 171
- Bitonic-merge-Verfahren, 652, 657
- bitonische Folge, 655
- Bittabelle, 212
- Blätter, 235
- Blattsuchbäume, 239, 240, 274, 334, 335
- Blockadresse, 318
- Blockregion, 220
- Blockzugriff, 205
- Blöcke, 318
- Blüte, 603
 - Basis der, 603
 - Schrumpfen der, 603
 - Stiel der, 603
- BM-Netz, 657
- bmeinfach, 627
- Boruvka
 - Algorithmus von, 582
- bottom, 34
- Boyer-Moore
 - Verfahren von, 624
- boyermoore, 629
- Breitensuche, 551, 567, 602
- BrentEinfügen, 193
- Brents Algorithmus, 192
- Bruder-Bäume, 273, 383
 - 1-2-, 274
 - zufällige 1-2-, 288
- Bruderstrategie, 227
- Brüder, 218
- bruteforce, 618

- BS-Netz, 658
- Bubblesort, 66, 73, 74
- Bucketsort, 105
- buddy merge, 227
- c-Ebenen-Sprungliste, 352, 353
- capacity, 585
- cascading cuts, 398
- closest pair, 497
- clustering
 - primary, 185
 - secondary, 186
- Coalesced Hashing, 200, 202
- Code-Baum
 - binärer, 364
- Compare-exchange-Modul, 651
- comparisons, 66
- Computational Geometry, 419
- concurrent, 333
- concurrent read concurrent write, 644
- concurrent read exclusive write, 644
- cut, 595
- cut point, 558
- Dateilevel, 206
- Dateiverdoppelung, 211, 215
- Datenblock, 204
 - virtueller, 215
- Datenblocksplit, 224
- Datensatz, 318
- Datenstrukturen, 1, 16, 20
 - für dynamische Bäume, 595
 - geometrische, 444
 - halbdynamische, 445
 - randomisierte, 42
- Datentypen, 20
- deadlock, 227
- Decrease Key, 378
- Delaunay-Triangulierung, 505, 517
- delete, 281
- Delete Min, 378, 396, 415
- $\delta\text{-}2(j)$, 628
- $\delta\text{-}1$ -Tabelle, 625
- depth-first-begin-Index, 554
- depth-first-end-Index, 554
- dequeue, 34
- design-rule checking, 420
- DFBI, 554
- DFEI, 554
- DFS, 556
- DFSBCC, 559
- DFSSCC, 564
- Dichte Bäume, 327
- dichtestes Punktepaar, 497, 516
- Dictionary, 40, 238
- Digraph, 536
 - bewerteter, 567
 - verdichteter, 565
- Dijkstra
 - Algorithmus von Jarník, Prim und, 583
- Directory, 221
- direkte Verkettung der Überläufer, 177
- Dirichlet-Gebiete, 501
- distance, 567
- Distanz
 - euklidische, 16
 - von Objekten, 496
- Distanzgraph, 567
- Distanz zweier nacheinander einzufügender Elemente, 121
- Divide-and-conquer
 - geometrisches, 435
 - Segmentschnitt mittels, 435
- Divide-and-conquer-Strategie, 9, 11, 12, 154, 435, 441
- Divide-and-Conquer-Strategie, 509
- Divisions-Rest-Methode, 171
- Dominanzzahl, 52
- Doppelrotation, 264, 265, 268, 294
- Double Hashing, 190
- doubly connected arc list, 540
- doubly connected edge list, 505
- DRW-Problem, 493
- Dummy-Elemente, 27
- Dummy-Knoten, 243
- Durchgang, 128
- Durchlaufen
 - eines Baumes, 238
 - von Graphen, 551
- Durchlaufen eines Graphen, 552, 553
- Durchlaufordnungen in Binärbäumen, 248
- Durchsatz, 595

dynamische Bereichssuche mit festem Fenster, 492

Effizienz, 2

Einfügen, 21, 23, 29, 41, 46, 176, 178, 182, 183, 196, 199, 202, 210, 214, 220, 224, 238, 244, 263, 264, 276, 292, 298, 323, 378, 383, 385, 392, 395, 415, 446, 449, 455, 460, 493, 496

Einfügesort, 70

Eingangsgrad, 541

Element

(*k*)-tes, 415

i-kleinstes, 67

k-tes, 416

kanonisches, 403

element uniqueness, 497

empty, 110

endlicher Automat, 624

enqueue, 34

Entfernen, 21, 24, 30, 41, 47, 176, 178, 182, 183, 196, 202, 220, 238, 246, 263, 270, 276, 281, 292, 298, 324, 383, 392, 396, 398, 415, 446, 452, 455, 462, 467, 493, 496

beliebiger Elemente, 378

des Minimums, 385, 392

eines beliebigen Elementes, 392

eines beliebigen inneren Knotens, 385

Entfernung, 567

Entscheidungsbaum, 114, 138

algebraischer, 143

rationaler, 140

erreichbar, 542

Erreichbarkeit, 546

Erweiterbares Hashing, 215

exclusive read exclusive write, 644

Expansion, 211

partielle, 211

Externspeicher, 126, 204

Externzugriff, 127

F-Heap, 394

Fädelungszeiger, 251

Faktor

konstanter, 5

Fan-in-Technik

binäre, 645

Farbwechsel, 330

feature extraction, 420

Fibonacci-Heap, 394, 572, 584

Fibonacci-Suche, *siehe* Suche, Fibonacci

Fibonacci-Zahlen, 137, 156, 262, 274, 399

höherer Ordnung, 137

fibsearch, 158

FIFO-Prinzip (first in first out), 34

Find, 41, 402, 407, 410, 412, 415

findcost, 595

findroot, 595

Finger, 118

beweglicher, 121

Fließbandprinzip, 662

Fluß

maximaler, 585

Fluß, 585

-erhaltung, 585

blockierender, 591, 595

in Netzwerken, 584

maximaler durch zunehmende Wege, 589, 592

über den Schnitt, 586

Folge

bitonische, 655

Ford

Auswahlschritt von, 573

Auswahlverfahren nach, 573

Frequency Count, 161

Fringe-Analyse, 285, 326

Funktionen

erzeugende, 256

Gabriel-Graph, 532

Geometrische Algorithmen, 419

Geometrische Datenstrukturen, 444

Geometrisches Divide-and-conquer, 435

gerichteter Graph, 536

Gestalts-Analyse, 252

- Gestaltsanalyse, 256
- Gewicht, 289, 293, 312, 356, 567
- Gewichtsbalancierte Bäume, 289
- Gitterzelle, 220
- goldener Schnitt, 172
- Grad
 - Ausgangs-, 541
 - Eingangs-, 541
- Graph, 543
 - bewerteter, 567
 - Distanz-, *siehe* Distanzgraph
 - Durchlaufen eines, *siehe* Durchlaufen von Graphen
 - gerichteter, 536
 - Niveau-, *siehe* Niveaugraph
 - reduzierter, 550
 - Rest-, *siehe* Restgraph
 - Teil-, 541
 - ungerichteter, 543
 - Unter-, 542
- Graphenalgorithmen, 535
- greedy, 578
- Gridfile, 219, 221
 - Mehr-Ebenen-, 223
- Größenordnung, 4
- Häufung
 - primär, 185
 - sekundäre, 186
- Halbebene, 501
- Halbierungsmethode, 412
- Halbordnung, 543
- Halde, 89, 378
- Haltepunkte, 421
- Hashadresse, 169
- Hashfunktion, 169, 171
 - perfekte, 173
- Hashfunktionen
 - universelle Klasse von, 173
- Hashing
 - Coalesced, 200, 202
 - Double, 190
 - Erweiterbares, 215
 - Lineares, 206
 - Ordered, 194, 196
 - Robin-Hood-, 199
 - Virtuelles, 211
- Hashtabelle, 169
- Hashverfahren, 169
 - dynamische, 170, 204
 - offene, 181, 182
- Hauptreihenfolge, 248, 556
- Heap, 89, 132, 148, 378, 571
 - Aufbauen eines, 95
- Heap-Bedingung, 90
- heapgeordnet, 389
- Heapsort, 89, 95
- Herabsetzen
 - eines Schlüssels, 397
- Herabsetzen eines Schlüssels, 378, 385, 392, 393
- Hidden-Line-Eliminationsproblem, 483, 485
- Hintergrundspeicher, 317
- höchstintegrierte Schaltungen
 - Entwurf von, 420
 - Kompaktierung von, 422
- Höhe eines Baumes, 237
- höhenbalanciert, 260
- Höhenbedingung, 261
- Horizontalstruktur, 445
- hsweep, 474
- Hülle
 - konvexe, 504, 528
 - reflexive transitive, *siehe* reflexive transitive Hülle
 - reflexive transitive, *siehe* reflexive transitive Hülle
 - transitive, *siehe* Transitive Hülle
- indegree, 541
- Indextabelle, 318
- Infixnotation, 60
- init, 110
- Initialisieren, 28, 34, 378, 395
- initnext, 623
- Inorder, 248
- Insert, 378
- Intervall-Bäume, 454
- Intervall-Liste, 448, 454
- Inversion, 71
- Inversionszahl, 71, 113, 116
- inzident, 537

Jarník

Algorithmus von Prim, Dijkstra
und, 583

Kachelbaum-Struktur, 457

Kanten, 536, 543

Auswahlprozeß für, 579

gebundene, 600

Länge von, 566

Kantenliste

doppelt verkettete, 505

Kantenzug

trennender, 511, 513

Kapazität, 586

-sbeschränkung, 585

-sfunktion, 585

Rest-, *siehe* Restkapazität

Keller, 203

Klammerausdruck

wohlgeformter, 35

kmp search, 622

Knoten, 235, 535, 536

Anfangs-, 537, 567

Besuchen eines, 551, 552

End-, 537, 567

gebundene, 600

innere, 235

Tiefe eines, 237

unäre, 273

Knuth-Morris-Pratt

Verfahren von, 619

Kollisionsauflösung, 169

Kompaktierung, 422

Komprehensionsschema, 40

Kompressionsmethode, 410

Kontur, 485

konvexe Hülle, 504, 528

Kopfzeiger, 27

Korrektheitsnachweis, 2, 5

Kosten, 567

Kostenmaß

Einheits-, 3

logarithmisches, 3

Kruskal, 582

Algorithmus von, 582

kürzeste Wege, *siehe* Wege, kürzeste

Länge, 567

eines Weges, 542

Länge von Kanten/Pfeilen, 566

Laufzeit, 3

Laufzeitanalyse, 5

leer, 33

Level, 94, 214

LIFO-Prinzip (last in first out), 34

Lineare Listen, 21

sequentiell gespeicherte, 22

verkettete Speicherung, 25

verkettet gespeicherte, 22, 32

Lineares Hashing, 206

Rekursionsebenen von, 211

Lineares Sondieren, 184

linear probing, 185

Liniensegment-Schnittproblem

allgemeines, 428

link, 595

Links bäume, 384

Listen

Selbstanordnung von, 161

verkettet gespeicherte, nicht sor-
tierte, 382

verkettet gespeicherte, sortierte, 383

Listenhöhe, 44

Löschen, 225

Löschmarke, 341

loop, 78

Make set, 402, 407, 415

maketree, 595

Match-Heuristik, 628

matching, 596

Matrix-Vektor-Produkt, 664

Matrizen

Multiplikation zweier, 11

Produkt zweier, 646

Maximum-Subarray-Problem, 12

maximum matching, 597

maximum weight matching, 597

Median, 148

Median-of-median-Strategie, 150

Mehr-Ebenen-Gridfile, 223

Mehrbenutzerumgebungen, 333

Meld, 378

Mengen, 40

- Kollektionen paarweise disjunkter, 41
- Mengenbaum, 415
- Mengenmanipulationsproblem, 40, 377, 402, 413
 - allgemeines, 42
- Merge, 99, 226, 378, 386
- Mergesort, 96–98
 - 2-Wege-, 96
 - ausgeglichenes 2-Wege-, 128
 - ausgeglichenes Mehr-Wege-, 132
 - balanced 2-way-, 128, 130
 - cascade, 137
 - kaskadierendes, 137
 - Mehrphasen-, 135
 - Natürliches 2-Wege-, 102
 - natural-, 103
 - oscillating, 137
 - oszillierendes, 137
 - polyphase, 135
 - Reines 2-Wege-, 100
 - straight-, 101
 - straight 2-way, 100
- Methode
 - axiomatische, 18
 - konstruktive, 18
- Minimalelement, 395
- minimaler spannender Baum, 403, 498, 517, 578, 579, 647
- Minimum Entfernen, 378
- minimum spanning tree, 498, 578
- Minimum Suchen, 378
- Minimum von Schlüsseln, 646
- Mismatch, 618
- Move-to-front, 161, 305
- Move-to-root, 305
- movements, 66
- Multiplikation
 - langer ganzer Zahlen, 11
 - zweier Matrizen, 11
- Multiplikationsverfahren, 5
- multiplikative Methode, 172
- N-gegründet, 494
- Nachbarn
 - Gebiete gleicher nächster, 16
- Nachbarschaftsanfrage, 16
- Nachbarstrategie, 226
- Nachfolger, 415, 543
 - symmetrischer, 246
- nächste Nachbarn
 - alle, 497, 516
 - Gebiete gleicher, 501
 - Suche nach, 500, 519
- Natürliche Bäume, 239
- Nearest-neighbor-query, 16
- nearest neighbors
 - all, 497
- nearest neighbor search, 500
- nearest neighbour query, 207
- Nebenreihenfolge, 248, 556
- Netzplantechnik, 576
- Netzwerk, 567
- Niveau, 94, 237
- Niveaugraph, 592
- Nord-gegründet, 494
- Odd-even-merge, 652
- OEM-Netz, 654
- OES-Netz, 655
- Offene Hashverfahren, 181
- one-to-all shortest paths, 567
- one-to-one shortest path, 567
- Optimalitätsprinzip, 567
- orderedEinfügen, 196
- Ordered Hashing, 194, 196
- orderedSuchen, 196
- Ordnung, 320
- ordnungserhaltend, 207
- Ordnungsrelation, 63
- overflow bucket sharing, 211
- Parallel-Random-Access-Maschine, 644
- parallele Algorithmen, 643
- Paralleles Mischen und Sortieren, 651
- Parallelrechner, 644
- partial match query, 220, 365
- partial range query, 220
- pass, 128
- path, 542
- pattern matching, 617
- Pattern Matching, *siehe* Zeichenketten-suche
- perfect matching, 596

- Pfad, 235
- Pfadlänge
 - gesamte interne, 258
 - gewichtete, 356
 - interne, 252
 - normierte gewichtete, 357
- Pfadverkürzung, 410
- Pfeile, 536
 - gesättigte, 587
 - Länge von, 566
 - parallele, 537
 - Rückwärts-, *siehe* Rückwärtspfeile
 - Seitwärts-, *siehe* Seitwärtspfeile
 - Vorwärts-, *siehe* Vorwärtspfeile
- Pfeilliste
 - doppelt verkettete, 540
- Phase, 135
- Pipelining, 662
- Pivotelement, 77, 149
- Plazierung und Verdrahtung, 420
- Polygonschnittproblem, 472, 481
- Polynomprodukt, 8
- pop, 34
- pophead, 33, 110
- poptail, 34
- Post-office-Problem, 16
- Postfixnotation, 60
- Postorder, 248
- Potentialfunktion, 667
- Preorder, 248
- Prim
 - Algorithmus von Jarník, Dijkstra und, 583
- Primärblock, 207
- primäre Häufung, 185
- primary clustering, 185
- Priorität, 35, 296, 304
- Prioritäts-Suchbaum, 297, 457, 458, 494
- Prioritätsordnung, 378
- Priority Queues, 35, 378, 389
- probing
 - linear, 185
 - random, 187
 - uniform, 187
- Problemstapel, 38
- Produkt zweier Matrizen, 646
- Pseudoschlüssel, 207
- Pull-down-Marke, 341
- Punkteinschluß-Problem, 441
- Punktepaar
 - dichtestes, 497, 516
- push, 34
- Push-up-Marke, 337
- pushhead, 33
- pushtail, 33, 110
- Quicksort
 - median of three, 86
 - randomisiertes, 86
- Quadranten-Bäume, 365
- Quadratisches Sondieren, 186
- Qualle, 351
- Quelle, 585
- Quicksort, 76–78
 - mit konstantem zusätzlichem Speicherplatz, 84
 - mit logarithmisch beschränkter Rekursionstiefe, 83
- Radix-exchange-sort, 105, 106
- Radixsort, 105, 109, 110
- Rand, 285
- Randknoten, 568
- Random-Access-Maschine, 2
- Random-tree-Analyse, 252
- Randomisierung, 173
- random probing, 187
- Rang, 141, 236, 312
- Range-range-Bäume, 490
- range query, 207, 365, 426
- Raster, 474
- read, 127
- rear, 34
- Rechenzeit, 2
- Rechteckschnittproblem, 441, 446, 457
 - Reduktion des, 445
- Rechts-Bruder-Bäume, 328
- reflexive transitive Hülle, 546
- reflexive transitive Hülle, 548
 - für azyklischen Digraphen, 549
- Reihenfolge
 - Haupt-, *siehe* Hauptreihenfolge

- Neben-, *siehe* Nebenreihenfolge
- symmetrische, *siehe* symmetrische Reihenfolge
- Rekursionselimination
 - Schema zur, 39
- Rekursionsformel, 11, 14, 54, 81, 254, 441
- Rekursionsgleichung, 150, 288
- Rekursionsinvariante, 438
- Relation, 543
- Relaxed Heaps, 402
- relaxiertes Balancieren, 335
- rem, 114
- replacement selection, 134
- report, 451, 456
- ReportCuts, 437
- ReportInc, 442
- reset, 127
- Restgraph, 588
- Restkapazität, 587
- rewrite, 127
- Robin-Hood-Hashing, 199
- Rot-schwarz-Bäume, 329, 336
- Rotation, 264, 265, 267, 294, 298, 305, 330
- Rückwärtspfeile, 556, 588
- Run-Zahl, 113
- Runs, 102
- S-gegründet, 457, 494
- Sammelphase, 108
- Satzschlüssel, 318
- Scale, 221
- Scan-line-Prinzip, 14, 420, 421
- Schichtenmodell, 329, 333
- Schlange, 33, 34
- Schleife
 - Terminierung einer, 7
- Schleifeninvariante, 7
- Schlüssel, 63, 147, 377
 - i*-kleinster, 149
 - arithmetische Eigenschaften der, 105
 - Herabsetzen eines, 393
 - mehrdimensionale, 219
 - Minimum von, 646
- Schlüsselvergleiche, 66
- Schnitt, 586
 - minimaler, 586
- Schnittproblem
 - für iso-orientierte Liniensegmente, 425
 - Polygon-, 472, 481
 - Rechteck-, 441
- Schnittpunkt, 558
- Schnittpunktaufzählungsproblem, 428, 432
- Schnittpunkttestproblem, 428, 429
- Schreibkonflikte, 646
- Schwanzzeiger, 27
- secondary clustering, 186
- Segment-Bäume, 448
- Segment-range-Bäume, 488
- Segment-Schnitt-Problem
 - rechteckiges, 425
- Segment-Segment-Bäume, 490
- Segmentschnitt
 - mittels Divide-and-conquer, 435
- Segmentschnitt-Suchproblem, 488
- Segmentteile
 - Berechnung der beleuchteten, 527
- Seiten, 319
- Seitwärtspfeile, 556
- Sektoren, 318
- Sekundärblock, 207
- sekundäre Häufung, 186
- Sekundärspeicher, 126
- Selbstanordnung, 304
- Selbstanordnung von Listen, 161
- selection, 148
- selection tree, 132
- Senke, 585
- separate Verkettung der Überläufer, 176
- Shakersort, 76
- Shellsort, 66, 71, 72
- Shuffle-exchange-Graph, 644
- Shuffle-exchange-Netz, 661
- Sichtbarkeitsproblem, 422, 423
- Sichtbarkeitstest, 484
- sift down, 91
- Signatures, 630
- single pair shortest path, 567
- single source shortest paths, 567
- Skelett, 454

- Skelettstruktur, 445, 448, 459, 477
- Skip-Liste, 42
 - perfekte, 44
 - randomisierte, 46
- Slot-Assignment-Problem, 530
- smart searching, 200
- Smoothsort, 96
- Sohn, 235
 - linker, 235
 - rechter, 235
- Sollin, 648
- Sondieren
 - Binärbaum-, 193
 - lineares, 184
 - quadratisches, 186
 - uniformes, 187
 - zufälliges, 187
- Sondierungsfolge, 181
- Sortieren, 63
 - durch Auswahl, 66, 89
 - durch Einfügen, 69
 - durch Fachverteilung, 107
 - durch iteriertes Einfügen, 117
 - durch lokales Einfügen, 123
 - durch natürliches Verschmelzen, 125
 - durch rekursives Teilen, 77
 - durch Verschmelzen, 96
 - Externes, 126
 - mit abnehmenden Inkrementen, 72
 - vorsortierter Daten, 111
- Sortierindexfunktion, 141
- Sortiernetz, 655, 658
- Sortierproblem, 63
- Sortierung
 - topologische, *siehe* Topologische Sortierung
- Sortiervorgehen
 - allgemeine, 89
 - allgemeines, 138
 - externe, 64
 - In-situ-, 76, 96
 - interne, 64
 - m-optimales, 116
 - Rahmen für, 65
 - stabiles, 96
- south-grounded, 458
- spannender Baum, 543
- Speicherbedarfsanalyse, 5
- Speicherplatz, 2, 3
- Speicherplatzausnutzung, 326
- Speicherstrukturen, 20
- Speicherung
 - doppelt verkettete, 31
 - einfach verkettete, 31
- Sperrstrategien, 334
- Splay-Baum, 305, 310
- Splay-Operation, 305
- Split, 41, 224
- Splitentscheidung, 225
- Splitwert, 460
- Spuren, 318
- stabbing query, 447
- stabil, 145
- Stapel, 33, 34
- Stopper, 23, 28, 70, 80, 153, 243
- string processing, 617
- strongly connected component, 557
- Stufe, 94
- Submuster, 628
- Suchbäume, 239
 - alphabetische, 362
 - fast optimale, 362
 - Konstruktion optimaler, 357
 - mehrdimensionale, 362
 - optimale, 317, 356, 357
 - Prioritäts-, 297
 - randomisierte, 296, 300
 - von beschränkter Balance, 289
 - zufällige, 300
- Suchbaum
 - zufälliger, 252
- Suche
 - binäre, 154
 - erfolglose, 32, 147, 170, 199
 - erfolgreiche, 147, 170
 - exakte, 223
 - exponentielle, 159
 - Fibonacci-, 156
 - Interpolations-, 160
 - partielle, 220, 223
 - sequentielle, 153
- suchen, 41

- Suchen, 21, 26, 28, 41, 44, 147, 176, 177, 182, 183, 196, 202, 220, 238, 240, 242, 263, 276, 292, 298, 322, 415
- Suchhäufigkeiten, 238
- Suchkosten, 49
- Suchpfadlänge
 - durchschnittliche, 252, 253
- Süd-gegründet, 494
- sweep, 421
- symmetrische Reihenfolge, 248
- symmetrischer Nachfolger, 246
- symmetrischer Vorgänger, 248
- symtraverse, 249
- Synonyme, 169
- systolische Algorithmen, 662
- systolisches Array, 663
- Teilbaum, 236, 543
- Teilen eines überlaufenden Knotens, 323
- Teilfolgen
 - längste aufsteigende, 113
 - längstmögliche sortierte, 102
- Teilgraph, 541
 - induzierter, 542
- Text, 617
- Textsuche, 618
- Thiessen-Polygone, 501
- Tiefe
 - eines Blattes, 138
 - mittlere, 139
 - globale, 218
 - lokale, 218
- Tiefe eines Knotens, 237
- Tiefensuchbaum, 556
- Tiefensuche, 551, 554
- tile tree, 457
- top, 33
- Top-down-Update, 335
- Top-Segmente, 477
- Topologische Sortierung, 543, 544
- Transitive Hülle
 - für azyklische Digraphen, 548
- Transpositionsregel, 161, 305
- Treap, 296
- Triangulierung, 519
- hierarchische, 519, 525
- Tries, 363
 - binäre, 364
- Überläufer, 176
 - direkte Verkettung der, 177
 - separate Verkettung der, 176
 - Verkettung der, 176
- Überlappungsproblem, 446
 - für Intervalle, 458
- Überlaufkette, 176
- Umfang, 529
- Umstrukturierung als Hintergrundprozeß, 335
- ungerichteter Graph, 543
- uniformes Sondieren, 187
- uniform probing, 187
- Union, 41, 403, 407, 415
- Union-Find-Problem, 42, 403
- Union-Find-Struktur, 42, 377, 402
- unmatched, 596
- untere Schranken, 138
 - für die maximale und mittlere Zahl von Vergleichsoperationen, 140
- Untergraph, 542
- Vater, 235
- Verbindungsnetz, 644, 652
- Vereinigung
 - nach Größe, 408, 409
 - nach Höhe, 408
- Vergleichsmodul, 651
- Vergleichsoperationen, 138
- Verhalten
 - im besten Fall, 3
 - im Mittel, 3
 - im schlechtesten Fall, 3
- Verkettung, 30
- Verkettung der Überläufer, 176
- Verklemmung, 227
- Verschmelzen, 226, 378, 386, 387, 389, 396
 - Schranke für das Durchführen des, 226
 - Schranke für die Überprüfung des, 226
- Verschmelzen in situ, 104

- Verschmelzen zweier Teilfolgen, 98
- Verschmelzestrategie, 226
- Verschmelzungsphase, 128
- Versickern eines Schlüssels, 91–93
- Vertauschung
 - kostenfreie, 163
 - zahlungspflichtige, 163
- Verteilungsphase, 108
- Verteilungszahlen, 109
- Vertikalstruktur, 421
- Vielwegbäume, 236, 322
- Virtuelles Hashing, 211
- Vorgänger, 235, 415
 - symmetrischer, 248
- Vorkommens-Heuristik, 625
- Voronoi-Diagramm, 16, 501, 509, 510, 515
- Voronoi-Kanten, 503
- Voronoi-Knoten, 503
- Voronoi-Region, 16, 501
- Vorrangwarteschlange, 35, 377, 378
- Vorsortierung, 102
 - Maße für, 112
- Vorwärtspfeile, 556, 588
- Wachstum, 5
- Wachstumsordnungen von Funktionen, 4
- Wald
 - gerichteter, 542
 - spannender, 543
- Warteschlange, 35
- Wege, 542
 - alle kürzesten, 576
 - alle kürzesten zunehmenden, 591
 - alternierende, 600
 - einfache, 542
 - Gewicht alternierender, 609
 - kürzeste, 566–568, 573
 - kürzeste in Distanzgraphen, 567
 - kürzeste zunehmende, 590
 - Länge von, 542
 - vergrößernde, 600, 605
 - zunehmende, 587, 588, 598
- Wegweiser, 240
- weight, 289
- Window, 493, 496
- Wörterbuch, 40, 238, 273, 283, 295
 - operationen, 238, 319
 - problem, 41
 - größen-eindeutig, 350
 - mengen-eindeutig, 350
 - ordnungs-eindeutig, 350
- Wörterbuchproblem, 377
- Worst-case-Analyse, 4
 - amortisierte, 163, 310, 400
- worst-case-effizient, 273
- worst case, 3, 66
- write, 127
- Wurzel, 235, 351, 542
- Wurzel-Directory, 223
- Wurzelbalance, 289, 293
- Wurzelbaum, 542
- Wurzelliste, 395
- Zeichenketten, 617
 - Verarbeitung von, 617
- Zeichenkettensuche
 - approximative, 631
 - exakte, 617
- Zickzack, 475
- Zickzack-Ordnung, 476
- Zickzack-Paradigma, 471
- zig-Operation, 306
- zig-zag-Operation, 306
- zig-zig-Operation, 306
- zufälliges Sondieren, 187
- Zufalls-Strategie, 86
- Zugriff, 21, 41
 - direkter, 318
 - sequentieller, 318
- Zugriffs-Lemma, 312
- Zugriffshäufigkeiten, 356, 361
 - für Elemente linearer Listen, 160
- Zuordnung, 596
 - alleine bzgl. einer, 596
 - Gewicht einer, 597
 - Größe der, 596
 - maximale, 597
 - maximale gewichtete, 597, 609
 - maximale in bipartiten Graphen, 598
 - nicht erweiterbare, 597
 - perfekte, 596

- Zuordnungsprobleme, 596
- Zurückhängen mit Vorausschauen, 30
- Zusammenfügen, 238, 378, 384
- zusammenhängend, 553
 - stark, 557
 - zweifach, 551, 557
- Zusammenhangskomponenten, 553, 554
 - einfache, 553
 - starke, 557, 561, 564
 - Wurzeln der, 563
 - zweifache, 557, 559
- Zwei-Zugriffs-Prinzip, 218, 223
- zweifach zusammenhängend, 551
- Zyklen, 542
 - negative, 572
- zyklenfrei, 542
- Zyklenfreiheit, 544